

BPF Performance Tools

Linux System and Application
Observability

Brendan Gregg

◆◆ Addison-Wesley

ВРФ

профессиональная оценка
производительности

Брендан Грегг



Санкт-Петербург • Москва • Минск

2024

ББК 32.973.2-018.2
УДК 004.451.9
Г79

Грегг Брендан

- Г79 BPF: профессиональная оценка производительности. — СПб.: Питер, 2024. — 880 с.: ил. — (Серия «Для профессионалов»).
- ISBN 978-5-4461-1689-8

Инструменты оценки производительности на основе BPF дают беспрецедентную возможность анализа систем и приложений. Вы сможете улучшить производительность, устранить проблемы в коде, повысить безопасность и сократить расходы. Книга «BPF: профессиональная оценка производительности» — ваш незаменимый гайд по применению этих инструментов.

Брендан Грегг — эксперт и пионер проекта BPF — представляет более 150 готовых инструментов анализа и отладки, рекомендации по их применению, а также пошаговые инструкции по разработке ваших собственных инструментов. Вы узнаете, как анализировать процессоры, память, дисковый ввод/вывод, файловую систему, сети, языки программирования, приложения, контейнеры, гипервизоры, безопасность и ядро. Вы сможете выработать глубокое понимание того, как улучшить буквально любую Linux-систему или приложение.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018.2
УДК 004.451.9

Права на издание получены по соглашению с Pearson Education Inc. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-0136554820 англ.
ISBN 978-5-4461-1689-8

© 2020 Pearson Education, Inc.
© Перевод на русский язык ООО «Прогресс книга», 2023
© Издание на русском языке, оформление ООО «Прогресс книга», 2023
© Серия «Для профессионалов», 2023

КРАТКОЕ СОДЕРЖАНИЕ

https://t.me/it_boooks/2

Предисловие	29
Вступление	30
Глава 1. Введение	44
Глава 2. Основы технологии	60
Глава 3. Анализ производительности	117
Глава 4. BCC	137
Глава 5. bpftrace	182
Глава 6. Процессоры	236
Глава 7. Память	301
Глава 8. Файловые системы	337
Глава 9. Дисковый ввод/вывод	406
Глава 10. Сети	455
Глава 11. Безопасность	556
Глава 12. Языки	584
Глава 13. Приложения	658
Глава 14. Ядро	702

Глава 15. Контейнеры.....	738
Глава 16. Гипервизоры.....	756
Глава 17. Другие инструменты BPF для анализа производительности	775
Глава 18. Советы, рекомендации и типичные проблемы	795
Приложение А. Однострочные сценарии для bpftrace	811
Приложение В. Шпаргалка по bpftrace	816
Приложение С. Разработка инструментов ВСС	819
Приложение D. С BPF.....	833
Приложение Е. Инструкции BPF.....	852
Глоссарий	858
Библиография	868

ОГЛАВЛЕНИЕ

Предисловие	29
Вступление	30
Где могут пригодиться инструменты оценки производительности BPF?	31
Об этой книге	31
Новые инструменты	32
О графическом пользовательском интерфейсе	32
О версиях Linux	33
О чем здесь не рассказывается	34
Структура	34
Для кого эта книга	35
Авторские права на исходный код	36
Дополнительные материалы и ссылки	37
Условные обозначения	37
Благодарности	38
Об авторе	42
От издательства	43
Глава 1. Введение	44
1.1. Что такое BPF и eBPF?	44
1.2. Что такое трассировка, прослушивание, выборка, профилирование и наблюдаемость?	45
1.3. Что такое BCC, bpftrace и IO Visor?	46
1.4. Первый взгляд на BCC: быстрый анализ	48
1.5. Область видимости механизма трассировки BPF	50
1.6. Динамическая инструментация: kprobes и uprobes	52
1.7. Статическая инструментация: точки трассировки и USDT	54

1.8. Первый взгляд на bpftrace: трассировка open()	55
1.9. Назад к ВСС: трассировка open()	57
1.10. Итоги	59
Глава 2. Основы технологии	60
2.1. BPF в иллюстрациях	61
2.2. BPF	61
2.3. Расширенный BPF (eBPF)	63
2.3.1. Зачем инструментам оценки производительности нужен BPF	65
2.3.2. BPF и модули ядра	68
2.3.3. Разработка программ для BPF	68
2.3.4. Обзор инструкций BPF: bpftool	69
2.3.5. Обзор инструкций BPF: bpftrace	76
2.3.6. BPF API	77
2.3.7. Управление конкурентностью в BPF	82
2.3.8. Интерфейс sysfs для BPF	83
2.3.9. BPF Type Format (BTF)	83
2.3.10. BPF CO-RE	84
2.3.11. Ограничения BPF	85
2.3.12. Дополнительные источники о BPF	86
2.4. Обход трассировки стека	86
2.4.1. Стеки на основе указателя на список фреймов	86
2.4.2. Использование отладочной информации	88
2.4.3. Last Branch Record (LBR)	88
2.4.4. ORC	88
2.4.5. Символы	89
2.4.6. Для дополнительного чтения	89
2.5. Флейм-графики	89
2.5.1. Трассировка стека	90
2.5.2. Профилирование трассировки стека	90
2.5.3. Флейм-график	91
2.5.4. Особенности флейм-графика	92

2.5.5. Разновидности	94
2.6. Источники событий	94
2.7. kprobes	94
2.7.1. Как работает kprobes.....	95
2.7.2. Интерфейсы kprobes.....	97
2.7.3. BPF и kprobes.....	98
2.7.4. Дополнительные источники информации о kprobes	99
2.8. uprobes	99
2.8.1. Как работает uprobes.....	100
2.8.2. Интерфейсы uprobes.....	101
2.8.3. BPF и uprobes.....	101
2.8.4. Оверхед uprobes и будущие улучшения	102
2.8.5. Дополнительные источники информации о uprobes	103
2.9. Точки трассировки.....	103
2.9.1. Инструментация точек трассировки	104
2.9.2. Как работают точки трассировки.....	105
2.9.3. Интерфейсы точек трассировки.....	106
2.9.4. BPF и точки трассировки	106
2.9.5. Неструктурированные точки трассировки в BPF	107
2.9.6. Дополнительные источники информации	108
2.10. USDT.....	108
2.10.1. Добавление поддержки USDT.....	109
2.10.2. Как работает USDT	110
2.10.3. BPF и USDT.....	111
2.10.4. Дополнительные источники информации о USDT	112
2.11. Динамический USDT	112
2.12. PMC	113
2.12.1. Режимы PMC.....	114
2.12.2. PEBS	115
2.12.3. Облачные вычисления	115
2.13. perf_events	115
2.14. Итоги.....	116

Глава 3. Анализ производительности	117
3.1. Обзор	117
3.1.1. Цели.....	118
3.1.2. Действия.....	119
3.1.3. Многочисленные проблемы производительности.....	119
3.2. Методологии оценки производительности.....	120
3.2.1. Определение характера рабочей нагрузки.....	120
3.2.2. Анализ с последовательным увеличением детализации	121
3.2.3. Метод USE	122
3.2.4. Чек-листы.....	123
3.3. Чек-лист инструментов Linux для анализа за 60 секунд	124
3.3.1. uptime	124
3.3.2. dmesg tail	125
3.3.3. vmstat 1	125
3.3.4. mpstat -P ALL 1.....	126
3.3.5. pidstat 1.....	127
3.3.6. iostat -xz 1.....	127
3.3.7. free -m.....	128
3.3.8. sar -n DEV 1	129
3.3.9. sar -n TCP,ETCP 1.....	129
3.3.10. top.....	130
3.4. Чек-лист инструментов BCC.....	130
3.4.1. execsnoop.....	131
3.4.2. opensnoop	131
3.4.3. ext4slower.....	132
3.4.4. biolatency.....	132
3.4.5. biosnoop	133
3.4.6. cachestat.....	133
3.4.7. tcpconnect	134
3.4.8. tcpaccept	134
3.4.9. tcpretrans.....	134
3.4.10. runqlat.....	135

3.4.11. profile.....	135
3.5. Итоги	136
Глава 4. ВСС.....	137
4.1. Компоненты ВСС.....	138
4.2. Возможности ВСС.....	139
4.2.1. Возможности в пространстве ядра	139
4.2.2. Возможности в пространстве пользователя.....	140
4.3. Установка ВСС.....	140
4.3.1. Требования к конфигурации ядра	140
4.3.2. Ubuntu	141
4.3.3. RHEL.....	141
4.3.4. Другие дистрибутивы	142
4.4. Инструменты ВСС	142
4.4.1. Инструменты, рассматриваемые в книге	142
4.4.2. Характеристики инструментов.....	143
4.4.3. Специализированные инструменты	144
4.4.4. Многоцелевые инструменты	146
4.5. funccount.....	147
4.5.1. Примеры funccount	147
4.5.2. Синтаксис funccount	149
4.5.3. Однострочные сценарии funccount	150
4.5.4. Порядок использования funccount.....	150
4.6. stackcount	151
4.6.1. Пример stackcount.....	152
4.6.2. Создание флейм-графиков с помощью stackcount	153
4.6.3. Искаженные трассировки.....	154
4.6.4. Синтаксис stackcount	154
4.6.5. Однострочные сценарии stackcount.....	155
4.6.6. Порядок использования stackcount	155
4.7. trace	156
4.7.1. Пример trace	157
4.7.2. Синтаксис trace	157

4.7.3. Однострочные сценарии trace.....	159
4.7.4. trace и структуры	159
4.7.5. Использование trace для отладки утечек дескрипторов файлов.....	160
4.7.6. Порядок использования trace	162
4.8. argdist.....	163
4.8.1. Синтаксис argdist.....	164
4.8.2. Однострочные сценарии argdist	165
4.8.3. Порядок использования argdist.....	166
4.9. Документация инструментов.....	167
4.9.1. Страница справочного руководства: opensnoop	167
4.9.2. Файл с примерами: opensnoop	170
4.10. Разработка инструментов BCC.....	172
4.11. Внутреннее устройство BCC	172
4.12. Отладка BCC.....	174
4.12.1. Отладка с помощью printf().....	174
4.12.2. Отладочный вывод BCC.....	177
4.12.3. Флаги отладки BCC	178
4.12.4. bpfflist.....	179
4.12.5. bpftool	179
4.12.6. dmesg.....	179
4.12.7. Сброс событий.....	180
4.13. Итоги	181
Глава 5. bpftrace	182
5.1. Компоненты bpftrace	183
5.2. Возможности bpftrace	184
5.2.1. Источники событий bpftrace	184
5.2.2. Действия bpftrace.....	184
5.2.3. Общие возможности bpftrace	185
5.2.4. Сравнение bpftrace с другими инструментами мониторинга	185
5.3. Установка bpftrace.....	187
5.3.1. Требования к конфигурации ядра	187
5.3.2. Ubuntu	187

5.3.3. Fedora.....	188
5.3.4. Действия после сборки.....	188
5.3.5. Другие дистрибутивы	188
5.4. Инструменты bpftrace.....	188
5.4.1. Инструменты, рассматриваемые в книге	188
5.4.2. Характеристики инструментов.....	190
5.4.3. Использование инструментов.....	190
5.5. Однострочные сценарии bpftrace	191
5.6. Документация bpftrace	192
5.7. Программирование на bpftrace.....	192
5.7.1. Порядок использования	193
5.7.2. Структура программы.....	193
5.7.3. Комментарии.....	194
5.7.4. Формат определения зондов	194
5.7.5. Подстановочные символы в определениях зондов.....	195
5.7.6. Фильтры.....	196
5.7.7. Действия	196
5.7.8. Hello, World!	196
5.7.9. Функции	197
5.7.10. Переменные.....	197
5.7.11. Функции карт	198
5.7.12. Определение продолжительности выполнения <code>vfs_read()</code>	199
5.8. Порядок использования bpftrace	201
5.9. Типы зондов в bpftrace	202
5.9.1. <code>tracepoint</code>	203
5.9.2. <code>usdt</code>	204
5.9.3. <code>kprobe</code> и <code>kretprobe</code>	205
5.9.4. <code>uprobe</code> и <code>uretprobe</code>	206
5.9.5. <code>software</code> и <code>hardware</code>	206
5.9.6. <code>profile</code> и <code>interval</code>	208
5.10. Управление потоком выполнения в bpftrace.....	209
5.10.1. Фильтры	209
5.10.2. Тернарные операторы	209

5.10.3. Инструкция if.....	210
5.10.4. Развернутые циклы	210
5.11. Операторы bpftrace.....	210
5.12. Переменные bpftrace.....	211
5.12.1. Встроенные переменные.....	211
5.12.2. Встроенные переменные pid, comm и uid.....	212
5.12.3. Встроенные переменные kstack и ustack.....	212
5.12.4. Встроенные переменные: позиционные параметры.....	214
5.12.5. Временные переменные	215
5.12.6. Карты	215
5.13. Функции bpftrace.....	216
5.13.1. printf()	217
5.13.2. join()	218
5.13.3. str().....	219
5.13.4. kstack() и ustack()	219
5.13.5. ksym() и usym().....	220
5.13.6. kaddr() и uaddr().....	221
5.13.7. system().....	221
5.13.8. exit()	222
5.14. Функции-карты в bpftrace	222
5.14.1. count()	223
5.14.2. sum(), avg(), min() и max().....	224
5.14.3. hist()	224
5.14.4. lhist().....	225
5.14.5. delete().....	226
5.14.6. clear() и zero().....	226
5.14.7. print().....	227
5.15. Направления развития bpftrace в будущем	228
5.15.1. Режимы явной адресации.....	228
5.15.2. Другие расширения	229
5.15.3. ply.....	230
5.16. Внутреннее устройство bpftrace	230

5.17. Отладка bpftrace	231
5.17.1. Отладка с помощью printf().....	232
5.17.2. Режим отладки	232
5.17.3. Режим подробного вывода	234
5.18. Итоги	235
Глава 6. Процессоры	236
6.1. Основы	237
6.1.1. Основы работы процессоров	237
6.1.2. Возможности BPF	240
6.1.3. Стратегия.....	242
6.2. Традиционные инструменты	243
6.2.1. Статистика ядра	244
6.2.2. Статистика оборудования	247
6.2.3. Выборка характеристик работы оборудования.....	249
6.2.4. Выборка по времени	250
6.2.5. Получение статистик и трассировка событий.....	254
6.3. Инструменты BPF	257
6.3.1. execsnoop.....	258
6.3.2. exitsnoop	261
6.3.3. runqlat	262
6.3.4. runqlen.....	266
6.3.5. runqslower	269
6.3.6. cpudist	270
6.3.7. cpufreq	272
6.3.8. profile.....	274
6.3.9. offcputime.....	278
6.3.10. syscount	283
6.3.11. argdist и trace.....	285
6.3.12. funccount.....	288
6.3.13. softirqs.....	290
6.3.14. hardirqs	291
6.3.15. smpcalls.....	292

6.3.16. llcstat	296
6.3.17. Другие инструменты	296
6.4. Однострочные сценарии для BPF	297
6.4.1. BCC	297
6.4.2. bpftrace	298
6.5. Дополнительные упражнения	299
6.6. Итоги	300
Глава 7. Память	301
7.1. Основы	302
7.1.1. Основы управления памятью	302
7.1.2. Возможности BPF	307
7.1.3. Стратегия	309
7.2. Традиционные инструменты	310
7.2.1. Журнал ядра	311
7.2.2. Статистики ядра	312
7.2.3. Аппаратные статистики и выборки	315
7.3. Инструменты BPF	317
7.3.1. oomkill	318
7.3.2. memleak	319
7.3.3. mmapnoop	321
7.3.4. brkstack	323
7.3.5. shmsnoop	324
7.3.6. faults	325
7.3.7. ffaults	327
7.3.8. vmscan	328
7.3.9. drsnoop	331
7.3.10. swapin	332
7.3.11. hfaults	333
7.3.12. Другие инструменты	334
7.4. Однострочные сценарии для BPF	334
7.4.1. BCC	334
7.4.2. bpftrace	335

7.5. Дополнительные упражнения	335
7.6. Итоги	336
Глава 8. Файловые системы	337
8.1. Основы	338
8.1.1. Основы файловых систем	338
8.1.2. Возможности BPF	341
8.1.3. Стратегия	342
8.2. Традиционные инструменты	344
8.2.1. df	344
8.2.2. mount	344
8.2.3. strace	345
8.2.4. perf	346
8.2.5. fatrace	348
8.3. Инструменты BPF	349
8.3.1. opensnoop	351
8.3.2. statsnoop	353
8.3.3. syncsnoop	355
8.3.4. mmapfiles	357
8.3.5. scread	358
8.3.6. fmapfault	360
8.3.7. filelife	361
8.3.8. vfsstat	362
8.3.9. vfscount	364
8.3.10. vfssize	365
8.3.11. fsrwstat	367
8.3.12. fileslower	369
8.3.13. filetop	371
8.3.14. writesync	374
8.3.15. filetype	375
8.3.16. cachestat	378
8.3.17. writeback	380
8.3.18. dcstat	382

8.3.19. dcsnoop	384
8.3.20. mountsnoop	386
8.3.21. xfsslower	386
8.3.22. xfsdist	388
8.3.23. ext4dist.....	390
8.3.24. icstat.....	393
8.3.25. bufgrow	395
8.3.26. readahead.....	396
8.3.27. Другие инструменты	397
8.4. Однострочные сценарии для BPF.....	398
8.4.1. BCC.....	398
8.4.2. bpftrace.....	399
8.4.3. Примеры использования однострочных сценариев BPF	401
8.5. Дополнительные упражнения	404
8.6. Итоги	405
Глава 9. Дисковый ввод/вывод.....	406
9.1. Основы.....	407
9.1.1. Основы дисков.....	407
9.1.2. Возможности BPF	410
9.1.3. Стратегия.....	412
9.2. Традиционные инструменты.....	412
9.2.1. iostat.....	413
9.2.2. perf.....	415
9.2.3. blktrace.....	416
9.2.4. Логирование SCSI.....	417
9.3. Инструменты BPF	418
9.3.1. biolatency.....	419
9.3.2. biosnoop	425
9.3.3. biotop	429
9.3.4. bitesize.....	430
9.3.5. seeksize	432
9.3.6. biopattern	434

9.3.7. biostacks.....	435
9.3.8. bioerr.....	438
9.3.9. mdflush.....	441
9.3.10. iosched.....	442
9.3.11. scsilatency	444
9.3.12. scsiresult	446
9.3.13. nvmlatency.....	448
9.4. Однострочные сценарии для BPF.....	450
9.4.1. BCC.....	451
9.4.2. bpftrace.....	451
9.4.3. Примеры использования однострочных сценариев BPF	452
9.5. Дополнительные упражнения	453
9.6. Итоги	454
Глава 10. Сети.....	455
10.1. Основы.....	456
10.1.1. Основы организации сетей	456
10.1.2. Возможности BPF	464
10.1.3. Стратегия	466
10.1.4. Типичные ошибки трассировки	467
10.2. Традиционные инструменты.....	468
10.2.1. ss	469
10.2.2. ip	470
10.2.3. nstat	471
10.2.4. netstat.....	472
10.2.5. sar	474
10.2.6. nicstat	475
10.2.7. ethtool	475
10.2.8. tcpdump	477
10.2.9. /proc	478
10.3. Инструменты BPF.....	479
10.3.1. sockstat	481
10.3.2. sofamily	483

10.3.3. soprocol.....	486
10.3.4. soconnect	488
10.3.5. soaccept.....	490
10.3.6. socketio	493
10.3.7. socksize.....	495
10.3.8. sormem	497
10.3.9. soconnlat.....	500
10.3.10. so1stbyte.....	503
10.3.11. tcpconnect.....	505
10.3.12. tcpaccept	507
10.3.13. tcplife.....	511
10.3.14. tcptop	515
10.3.15. tcpsnoop	516
10.3.16. tcpretrans	517
10.3.17. tcpsynbl	520
10.3.18. tcpwin.....	522
10.3.19. tcpnagle.....	524
10.3.20. udpconnect	526
10.3.21. gethostlatency.....	527
10.3.22. ipecn	529
10.3.23. superping.....	530
10.3.24. qdisc-fq.....	533
10.3.25. qdisc-cbq, qdisc-cbs, qdisc-codel, qdisc-fq_codel, qdisc-red и qdisc-tbf	535
10.3.26. netsize.....	536
10.3.27. nettxlat.....	539
10.3.28. skbdrop	541
10.3.29. skblife	543
10.3.30. ieee80211scan.....	545
10.3.31. Другие инструменты.....	547
10.4. Однострочные сценарии для BPF	547
10.4.1. BCC	548

10.4.2. bpftrace	549
10.4.3. Примеры использования однострочных сценариев BPF	551
10.5. Дополнительные упражнения	554
10.6. Итоги	555
Глава 11. Безопасность	556
11.1. Основы	556
11.1.1. Возможности BPF	557
11.1.2. Непривилегированные пользователи BPF	561
11.1.3. Настройка безопасности BPF	562
11.1.4. Стратегия	563
11.2. Инструменты BPF	563
11.2.1. execsnoop	565
11.2.2. elfsnoop	565
11.2.3. modsnoop	567
11.2.4. bashreadline	568
11.2.5. shellsnoop	569
11.2.6. ttysnoop	570
11.2.7. opensnoop	572
11.2.8. eperm	573
11.2.9. tcpconnect и tcpaccept	574
11.2.10. tcpreset	575
11.2.11. capable	576
11.2.12. setuids	579
11.3. Однострочные сценарии для BPF	581
11.3.1. BCC	581
11.3.2. bpftrace	582
11.3.3. Примеры использования однострочных сценариев BPF	582
11.4. Итоги	583
Глава 12. Языки	584
12.1. Основы	584
12.1.1. Компилируемые языки	585

12.1.2. Языки с динамической компиляцией	586
12.1.3. Интерпретируемые языки.....	588
12.1.4. Возможности BPF.....	589
12.1.5. Стратегия	590
12.1.6. Инструменты BPF	591
12.2. C	591
12.2.1. Символы функций на C	592
12.2.2. Трассировка стека программного кода на C.....	596
12.2.3. Трассировка функций на C	597
12.2.4. Трассировка смещений в функциях на C.....	598
12.2.5. Трассировка программного кода на C с помощью зондов USDT.....	598
12.2.6. Трассировка программного кода на C с помощью однострочных сценариев.....	599
12.3. Java	601
12.3.1. Трассировка libjvm.....	602
12.3.2. jnistacks	603
12.3.3. Имена потоков выполнения в Java.....	605
12.3.4. Символы методов Java.....	607
12.3.5. Приемы трассировки стека Java.....	609
12.3.6. Зонды USDT в Java	613
12.3.7. profile.....	619
12.3.8. offcputime.....	623
12.3.9. stackcount	629
12.3.10. javastat.....	632
12.3.11. javathreads.....	633
12.3.12. javacalls.....	635
12.3.13. javaflow	636
12.3.14. javagc.....	637
12.3.15. javaobjnew	638
12.3.16. Однострочные сценарии для трассировки кода на Java.....	639
12.4. Командная оболочка bash	640
12.4.1. Подсчет вызовов функций	642

12.4.2. Трассировка аргументов функции (bashfunc.bt).....	643
12.4.3. Задержки в функциях (bashfunclat.bt)	645
12.4.4. /bin/bash.....	647
12.4.5. Зонды USDT в /bin/bash.....	650
12.4.6. Однострочные сценарии для трассировки bash	651
12.5. Другие языки.....	651
12.5.1. JavaScript (Node.js)	652
12.5.2. C++	654
12.5.3. Golang.....	654
12.6. Итоги	657
Глава 13. Приложения.....	658
13.1. Основы.....	659
13.1.1. Основы приложений.....	659
13.1.2. Пример приложения: сервер MySQL.....	660
13.1.3. Возможности BPF.....	661
13.1.4. Стратегия	662
13.2. Инструменты BPF.....	663
13.2.1. execsnoop.....	665
13.2.2. threadsnoop	666
13.2.3. profile.....	668
13.2.4. threaded	671
13.2.5. offcputime.....	672
13.2.6. offcpuhist	676
13.2.7. syscount	679
13.2.8. ioprofile	681
13.2.9. Указатели фреймов в libc.....	682
13.2.10. mysqld_qlower.....	683
13.2.11. mysqld_clat	686
13.2.12. signals.....	689
13.2.13. killsnoop	691
13.2.14. pmlock и pmheld	692

13.2.15. napttime	697
13.2.16. Другие инструменты	698
13.3. Однострочные сценарии для BPF	698
13.3.1. BCC	698
13.3.2. bpftrace	699
13.4. Примеры использования однострочных сценариев BPF	700
13.4.1. Подсчет количества вызовов в секунду функций из библиотеки libpthread для доступа к условным переменным	700
13.5. Итоги	701
Глава 14. Ядро	702
14.1. Основы	703
14.1.1. Основы ядра	703
14.1.2. Возможности BPF	705
14.2. Стратегия	707
14.3. Традиционные инструменты	708
14.3.1. Ftrace	708
14.3.2. perf sched	711
14.3.3. slabtop	712
14.3.4. Другие инструменты	712
14.4. Инструменты BPF	713
14.4.1. loads	714
14.4.2. offcputime	715
14.4.3. wakeuptime	717
14.4.4. offwaketime	719
14.4.5. mlock и mheld	720
14.4.6. Спин-блокировки	724
14.4.7. kmem	725
14.4.8. kpages	726
14.4.9. memleak	727
14.4.10. slabratetop	728
14.4.11. numamove	729
14.4.12. workq	730

14.4.13. Тасклеты	732
14.4.14. Другие инструменты	732
14.5. Однострочные сценарии для BPF	733
14.5.1. BCC	733
14.5.2. bpftrace	734
14.6. Примеры использования однострочных сценариев BPF	735
14.6.1. Подсчет обращений к системным вызовам по именам функций системных вызовов	735
14.6.2. Подсчет запусков hrtimer функциями ядра	736
14.7. Сложности	736
14.8. Итоги	737
Глава 15. Контейнеры	738
15.1. Основы	738
15.1.1. Возможности BPF	740
15.1.2. Сложности	741
15.1.3. Стратегия	743
15.2. Традиционные инструменты	744
15.2.1. Анализ на уровне хоста	744
15.2.2. Анализ на уровне контейнера	745
15.2.3. systemd-cgtop	745
15.2.4. kubectl top	746
15.2.5. docker stats	746
15.2.6. /sys/fs/cgroups	747
15.2.7. perf	747
15.3. Инструменты BPF	748
15.3.1. runqlat	748
15.3.2. pidnss	749
15.3.3. blkthrot	751
15.3.4. overlayfs	752
15.4. Однострочные сценарии для BPF	754
15.5. Дополнительные упражнения	755
15.6. Итоги	755

Глава 16. Гипервизоры	756
16.1. Основы	756
16.1.1. Возможности BPF	758
16.1.2. Возможные стратегии	759
16.2. Традиционные инструменты	760
16.3. Инструменты BPF для анализа на уровне гостевой ОС	760
16.3.1. Гипервызовы Xen	761
16.3.2. xenhyper	765
16.3.3. Обратные вызовы Xen	766
16.3.4. cpusholen	768
16.3.5. Трассировка выходов в HVM	769
16.4. Инструменты BPF анализа на уровне хоста	769
16.4.1. kvmexits	770
16.4.2. Возможные направления развития в будущем	773
16.5. Итоги	774
Глава 17. Другие инструменты BPF для анализа производительности	775
17.1. Vector и Performance Co-Pilot (PCP)	776
17.1.1. Визуализация	777
17.1.2. Визуализация: тепловые карты	777
17.1.3. Визуализация: табличное представление данных	779
17.1.4. Метрики BCC	780
17.1.5. Внутреннее устройство	781
17.1.6. Установка PCP и Vector	782
17.1.7. Подключение и просмотр данных	782
17.1.8. Настройка BCC PMDA	784
17.1.9. Возможные направления развития в будущем	785
17.1.10. Для дополнительного чтения	785
17.2. Grafana и Performance Co-Pilot (PCP)	785
17.2.1. Установка и настройка	786
17.2.2. Подключение и просмотр данных	786
17.2.3. Возможные направления развития в будущем	788
17.2.4. Для дополнительного чтения	788

17.3. Экспортер Cloudflare eBPF Prometheus (с Grafana).....	788
17.3.1. Сборка и запуск экспортера ebpf	789
17.3.2. Настройка Prometheus для мониторинга экземпляра ebpf_exporter.....	789
17.3.3. Создание запроса в Grafana	790
17.3.4. Для дополнительного чтения.....	790
17.4. kubectl-trace	790
17.4.1. Трассировка узлов.....	791
17.4.2. Трассировка подов и контейнеров.....	791
17.4.3. Для дополнительного чтения.....	793
17.5. Другие инструменты.....	793
17.6. Итоги	794
Глава 18. Советы, рекомендации и типичные проблемы.....	795
18.1. Типичная частота событий и оверхед.....	795
18.1.1. Частота	796
18.1.2. Выполняемые действия.....	798
18.1.3. Проверяй себя.....	800
18.2. Выборка с частотой 49 или 99 Гц.....	800
18.3. Желтые свиньи и серые крысы.....	801
18.4. Пишите целевое ПО	802
18.5. Изучайте системные вызовы	803
18.6. Не усложняйте	804
18.7. Отсутствие событий.....	805
18.8. Отсутствие трассировок стека	806
18.8.1. Как исправить проблему отсутствия трассировок стека.....	807
18.9. Отсутствие символов (имен функций) в выводе	808
18.9.1. Как исправить проблему отсутствия символов: среда выполнения с JIT (Java, Node.js, ...).....	808
18.9.2. Как исправить проблему отсутствия символов: двоичные файлы ELF (C, C++, ...)	809
18.10. Отсутствие функций в трассировке.....	809
18.11. Циклы обратной связи.....	810
18.12. Сброс событий.....	810

Приложение А. Однострочные сценарии для bpftrace	811
Приложение В. Шпаргалка по bpftrace	816
Приложение С. Разработка инструментов ВСС	819
Ресурсы.....	819
Пять советов	819
Примеры инструментов.....	820
Инструмент 1: hello_world.py	820
Инструмент 2: sleepsnoop.py	821
Инструмент 3: bitehist.py	823
Инструмент 4: biolatency.....	827
Дополнительная информация.....	832
Приложение D. С BPF.....	833
Почему на С?	833
Пять советов	835
Программы на С.....	836
ВНИМАНИЕ: изменения в API	837
Компиляция	837
Инструмент 1: Hello, World!.....	838
Инструмент 2: bigreads.....	841
Инструмент 3: bitehist.....	846
perf C.....	849
Инструмент 1: bigreads.....	849
Дополнительная информация.....	851
Приложение Е. Инструкции BPF	852
Вспомогательные макросы	852
Инструкции	855
Кодирование	856
Ссылки.....	857
Глоссарий	858
Библиография	868

ПРЕДИСЛОВИЕ

Иногда программисты говорят, что они «стряпают патч» («cook a patch»), а не «реализуют» (implement). Я начал увлекаться программированием еще в школе. Чтобы получить хороший код, программист должен выбрать лучшие «ингредиенты». Разные языки программирования предлагают множество разных строительных блоков — «ингредиентов», но когда дело доходит до программирования ядра Linux, то кроме самого ядра у вас ничего нет.

В 2012 году я занимался добавлением новых возможностей в ядро, но нужных мне «ингредиентов» тогда не оказалось. Я мог бы начать писать строительные блоки внутри ядра, и они были бы готовы к использованию годы спустя. Но я решил создать «универсальный ингредиент», который в руках опытного программиста мог бы стать сетевым мостом уровня 2 или сетевым маршрутизатором уровня 3 внутри ядра.

У меня было несколько важных требований: «универсальный ингредиент» должен быть безопасным в использовании независимо от уровня подготовки программиста, использующего его. Он не должен давать возможности злонамеренному или неопытному разработчику приготовить из него вирус — «универсальный ингредиент» не должен допускать этого.

В ядре Linux уже был похожий механизм, известный как BPF (Berkeley Packet Filter — пакетный фильтр Беркли), поддерживавший минимальный набор команд, которые можно использовать для фильтрации пакетов перед передачей их приложениям, таким как tcpdump. Я заимствовал это имя и назвал свой «ингредиент» eBPF, где «е» означает «extended» — расширенный.

Через несколько лет различия между eBPF и классическим BPF стерлись. Мой «универсальный ингредиент» повсеместно стали называть BPF. Известные корпорации создали на его основе большие системы для обслуживания миллиардов людей, в том числе меня и вас. Заложенный в него принцип «быть безопасным при любых условиях» позволил многим «поварам» стать всемирно известными «шефами».

Первым шеф-поваром BPF был Брендан Грегг. Он увидел, что BPF можно использовать не только для обеспечения работы сети и ее безопасности, но также для анализа производительности, самодиагностики и наблюдения. Однако для создания таких инструментов и интерпретации их результатов измерений нужны большой опыт и знания.

Надеюсь, что эта книга станет вашей настольной «кулинарной книгой», в которой известный шеф-повар учит, как использовать BPF на Linux-кухне.

*Алексей Старовойтов (Alexei Starovoitov)
Сизтл, штат Вашингтон
Август, 2019*

ВСТУПЛЕНИЕ

...Расширенные сценарии использования BPF: ...безумные...

Алексей Старовойтов, создатель нового BPF, февраль 2015 [1]

В июле 2014 года Алексей Старовойтов посетил офисы Netflix в калифорнийском Лос-Гатосе, чтобы обсудить новую захватывающую технологию, которую разрабатывал: расширенный пакетный фильтр Беркли (сокращенно eBPF, или просто BPF). В то время BPF был малопонятной технологией, ускоряющей фильтрацию пакетов, и у Алексея была идея, как расширить область ее применения за рамки сетевых пакетов. Алексей работал в паре с другим сетевым инженером, Дэниелом Боркманом (Daniel Borkmann), поставив себе цель превратить BPF в универсальную виртуальную машину, способную выполнять продвинутые сетевые и другие программы. Это была невероятная идея. Но меня особенно заинтересовала возможность использования BPF в качестве инструмента анализа производительности, и я увидел, как BPF может дать мне необходимые программные возможности. Мы договорились с Алексеем, что если он сумеет реализовать свою идею, то я разработаю инструменты анализа производительности, использующие BPF.

Сейчас BPF может подключаться к самым разным источникам событий. Он превратился в новую популярную технологию системной инженерии, разработкой которой занимается множество активных специалистов. Я разработал и опубликовал уже более 70 инструментов анализа производительности на основе BPF, которые используются во всем мире, в том числе на серверах Netflix, Facebook и других компаний. Специально для этой книги я написал еще несколько разработок, а также включил в нее инструменты, созданные другими авторами. И теперь рад поделиться плодами своего труда в книге «BPF: профессиональная оценка производительности», чтобы вы могли воспользоваться ими на практике для анализа производительности, устранения неполадок и многого другого.

Как перформанс-инженер, я одержим использованием инструментов производительности в стремлении достичь совершенства. Слепыми пятнами в системах называют узкие места в производительности и ошибки в программном обеспечении. В своей предыдущей работе я использовал технологию DTrace и посвятил ей книгу «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X, and FreeBSD», изданную Prentice Hall в 2011 году. В ней я поделился инструментами, использующими DTrace и разработанными для этих операционных систем. Теперь мне выпала интересная возможность поделиться похожими инструментами для Linux — инструментами, способными видеть и делать намного больше.

ГДЕ МОГУТ ПРИГОДИТЬСЯ ИНСТРУМЕНТЫ ОЦЕНКИ ПРОИЗВОДИТЕЛЬНОСТИ BPF?

Инструменты оценки производительности на основе BPF помогут вам максимально эффективно использовать свои системы и приложения, повысить их производительность, сократить оверхеды и устранить проблемы, имеющиеся в ПО. Они позволят вам проанализировать намного более широкий круг аспектов, чем традиционные инструменты, и найти ответы на самые разные вопросы непосредственно в среде эксплуатации.

ОБ ЭТОЙ КНИГЕ

Эта книга рассказывает об инструментах на основе BPF, которые применяются в основном для анализа и оценки производительности, но их можно использовать и в других областях: для поиска и устранения неполадок в ПО, для анализа безопасности и многого другого. Самое сложное в изучении BPF — не как писать код: любой интерфейс вы можете изучить примерно за день. Намного сложнее понять, что именно делать: какие из многих тысяч событий анализировать? Эта книга поможет найти ответ на этот вопрос, предоставив всю необходимую информацию об анализе производительности на множестве примеров анализа различных программных и аппаратных целей с помощью инструментов оценки производительности BPF на промышленных серверах Netflix.

Технология BPF обладает уникальными особенностями, но только потому, что расширяет круг доступных возможностей, а не дублирует их. Для эффективного использования BPF важно понимать, в каких случаях лучше применять традиционные инструменты анализа производительности, включая `iostat(1)` и `perf(1)`, а в каких — инструменты BPF. Традиционные инструменты (они также описаны здесь) могут напрямую решать проблемы с производительностью, а если нет, позволяют получить полезный контекст и подсказки для дальнейшего анализа с помощью BPF.

Во многих главах этой книги указаны цели обучения, чтобы вы могли выбрать наиболее важные для себя разделы. Представленный здесь материал используется в учебных курсах по анализу BPF внутри Netflix, а некоторые главы включают дополнительные упражнения¹.

Многие описанные в книге инструменты на основе BPF взяты из репозитория BCC и `bpftrace`, являющихся частью проекта Linux Foundation IO Visor. У них открытый исходный код, они доступны бесплатно не только на сайтах репозитория, но и в различных дистрибутивах Linux. Я также написал много новых инструментов `bpftrace` для этой книги и включил в нее их исходный код.

¹ В число упражнений включены также некоторые сложные и «нерешенные» проблемы, для которых мне еще только предстоит найти рабочее решение. Возможно, что некоторые из этих проблем нельзя решить без изменения ядра или приложения.

Эти инструменты были созданы не ради демонстрации различных возможностей BPF, а для решения практических задач. Они нужны мне для производственных задач, которые невозможно решить с помощью текущего набора инструментов анализа.

Исходный код инструментов, реализованных на bpftrace, я также включил в книгу. Если вы хотите модифицировать или разработать новые инструменты bpftrace, то можете изучить язык bpftrace из главы 5, а также учиться на примерах из многочисленных листингов кода, приведенных ниже. Этот код помогает понять, что делает каждый инструмент и какие события он использует: это все равно что включить в книгу псевдокод, который можно запустить.

Программные интерфейсы BCC и bpftrace достигли зрелости, но может случиться и так, что изменения в них в будущем нарушат работоспособность каких-то инструментов, код которых включен в книгу, и их потребуется обновить. Если это произойдет с инструментом, использующим BCC или bpftrace, загляните в их репозитории и проверьте наличие обновленных версий. Если инструмент написан для этой книги, зайдите на сайт книги: <http://www.brendangregg.com/bpf-performance-tools-book.html>. Но самое важное не работоспособность инструмента, а ваше *знание* того, как он устроен, и желание, чтобы он работал. Самое сложное в практике использования BPF — знать и понимать, что с ним делать. Даже неработающие инструменты могут служить источником полезных идей.

НОВЫЕ ИНСТРУМЕНТЫ

Для этой книги я разработал более 80 новых инструментов анализа, чтобы предоставить вам исчерпывающий набор инструментов. Одновременно они служат как примеры кода. Многие из них показаны на рис. В.1. Названия уже существовавших ранее инструментов показаны черным, а новые, созданные для этой книги, — серым шрифтом. В книге будут рассмотрены и существующие, и новые инструменты, но на многих последующих диаграммах черно-серая дифференциация не используется.

О ГРАФИЧЕСКОМ ПОЛЬЗОВАТЕЛЬСКОМ ИНТЕРФЕЙСЕ

Некоторые из инструментов BCC уже применяются как источники информации для графических интерфейсов — предоставляют временные ряды данных для отображения линейных графиков, трассировку стека для построения флейм-графиков или посекундные гистограммы для тепловых карт. Я предполагаю, что все больше людей будет использовать инструменты BPF не напрямую, а через графические интерфейсы. Но независимо от способа использования, они могут предложить огромный объем информации. Я расскажу об этих данных — как их интерпретировать и как создавать новые инструменты самостоятельно.

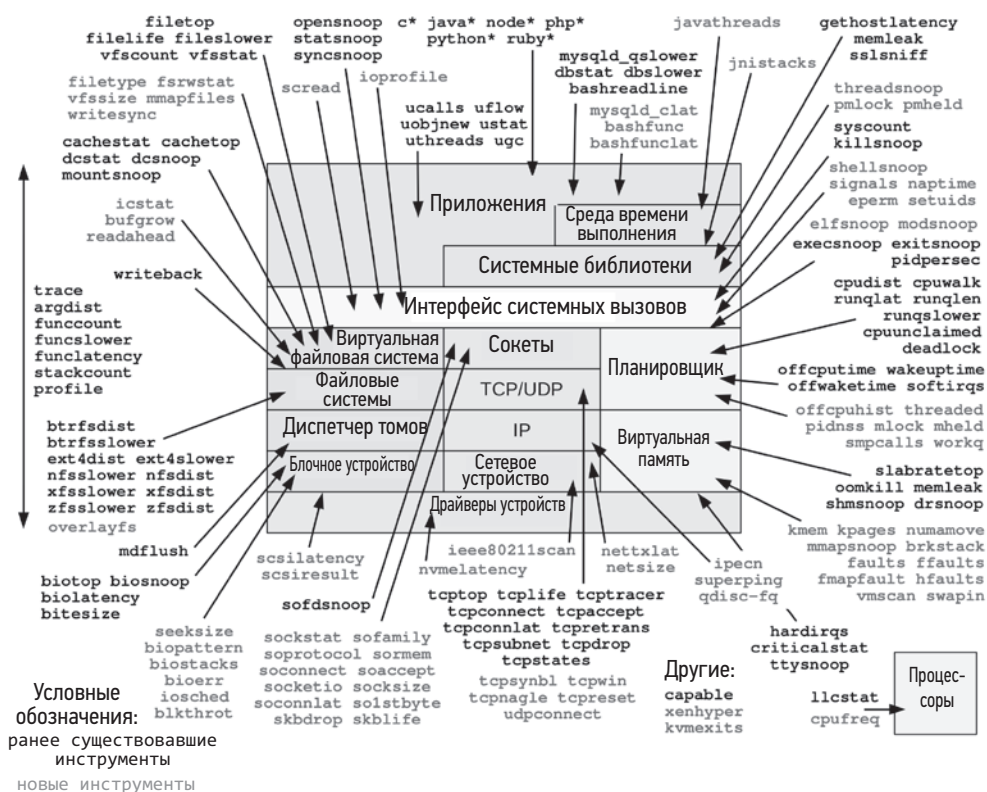


Рис. В.1. Инструменты анализа производительности на основе BPF: ранее существовавшие и новые инструменты, написанные для этой книги

О ВЕРСИЯХ LINUX

В этой книге представлено много технологий Linux, часто с номером версии ядра и годом появления. Кое-где я также называю разработчиков, чтобы вы могли найти вспомогательные материалы, написанные авторами этих технологий.

Расширенный BPF добавлялся в Linux по частям. Первая часть была добавлена в ядро 3.18 в 2014 году, и следующие продолжали добавляться в ядра 4.x и 5.x. Для опробования инструментов BPF, представленных здесь, рекомендуется использовать Linux 4.9 или выше. Примеры для книги взяты из ядер с версиями от 4.9 до 5.3.

Начата работа по внедрению расширенного BPF в другие ядра, и в будущем издании этой книги вполне может рассматриваться не только Linux.

О ЧЕМ ЗДЕСЬ НЕ РАССКАЗЫВАЕТСЯ

BPF — обширная тема, и кроме создания инструментов оценки производительности этот механизм можно использовать в самых разных ситуациях, многие из которых не рассмотрены здесь, в том числе для создания программно-конфигурируемых сетей и брандмауэров, защиты контейнеров и разработки драйверов устройств.

Эта книга посвящена использованию инструментов bpftrace и BCC, а также разработке новых инструментов bpftrace, но в ней не затронуты вопросы разработки новых инструментов для BCC. Листинги кода инструментов BCC, как правило, слишком длинные, чтобы включить их в книгу, но я все же представлю некоторые примеры в приложении С. В приложении D вы найдете примеры разработки инструментов на языке C, а в приложении E — список инструкций BPF с описаниями, который поможет желающим глубже понять, как работают инструменты BPF.

Эта книга не специализируется на использовании одного языка или приложения, как многие другие, в которых описаны средства анализа и отладки. Скорее всего, вы будете использовать некоторые из этих инструментов наряду с инструментами BPF для решения проблем и обнаружите, что разные наборы инструментов могут дополнять друг друга и дают различные подсказки. Здесь приведены основные инструменты анализа, которые есть в Linux. Владение ими поможет найти простые решения, прежде чем перейти к инструментам BPF и заглянуть еще дальше и глубже.

Эта книга содержит краткое описание основ и стратегии для каждого вида анализа. Более подробную информацию ищите в моей предыдущей книге «Systems Performance: Enterprise and the Cloud»¹ [Gregg 13b].

СТРУКТУРА

Книга делится на три части. Первая часть, главы с 1-й по 5-ю, охватывает основы, знание которых необходимо для использования BPF: анализ производительности, технологии трассировки ядра и два основных интерфейса трассировки BPF: BCC и bpftrace.

Вторая часть включает главы с 6-й по 16-ю и описывает цели трассировки с использованием BPF: процессоры, память, файловые системы, дисковый ввод/вывод, сетевые операции, безопасность, языки, приложения, ядро, контейнеры и гипервизоры. Можно читать по порядку, а можно произвольно переходить к любой главе, представляющей для вас интерес. Все эти главы написаны по общему шаблону: предварительное обсуждение, предложения по стратегии анализа, а затем описание конкретных инструментов BPF. В текст также включены функциональные схемы,

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

помогающие освоить сложные темы и получить более полное представление об используемых инструментах.

Последняя часть, включающая главы 17 и 18, охватывает некоторые дополнительные темы: другие инструменты BPF, а также советы, приемы и типичные проблемы.

В приложениях приводятся однострочные примеры использования `bpftrace` и краткие инструкции по его применению, введение в разработку инструментов для BCC, а также инструментов на C для BPF, в том числе с использованием `perf(1)` (инструмент Linux), и, наконец, список инструкций BPF с краткими описаниями.

В книге вы встретите много терминов и сокращений. Где это возможно, они объясняются. Полное описание приводится в глоссарии.

В конце вступления есть раздел «Дополнительные материалы и ссылки», где перечислены дополнительные источники информации. В конце книги вы найдете список использованной литературы.

ДЛЯ КОГО ЭТА КНИГА

Книга адресована широкому кругу читателей. Для применения инструментов BPF, о которых я рассказываю, вам не потребуется писать программы: вы можете использовать книгу как сборник рецептов с уже готовыми инструментами. Если вы решите создавать свои инструменты, то приведенные примеры исходного кода и глава 5 помогут быстро научиться этому.

Наличие опыта в сфере анализа производительности также не требуется; каждая глава кратко излагает все необходимое.

В частности, книга адресована:

- **Системным администраторам, SRE-инженерам, администраторам баз данных, перформанс-инженерам и сотрудникам служб поддержки**, отвечающим за эксплуатацию систем. Они могут использовать ее как справочник по диагностике проблем производительности, анализу использования ресурсов и устранению неполадок.
- **Разработчикам приложений**, которые могут использовать описанные здесь инструменты для анализа собственного кода и его исследования в комплексе с системными событиями. Например, события дискового ввода/вывода можно исследовать в совокупности с кодом приложения, вызвавшим их. Это позволит получить более полное представление о поведении, чем это делается с помощью типичных прикладных инструментов, не имеющих прямого доступа к событиям ядра.
- **Инженерам по безопасности**, которые могут узнать, как отслеживать все события, обнаруживать подозрительное поведение и создавать белые списки типичной активности (см. главу 11).

- **Разработчикам средств мониторинга производительности**, которые могут почерпнуть идеи по добавлению новых возможностей в свои продукты.
- **Разработчикам ядра**, которые могут научиться писать однострочные инструменты для bpftrace с целью отладки собственного кода.
- **Студентам**, изучающим операционные системы и приложения, которые могут использовать инструменты BPF для исследования действующих систем новыми и нестандартными способами. Вместо изучения абстрактных технологий ядра на бумаге студенты могут воочию увидеть, как они работают.

Проще говоря, эта книга фокусируется на использовании инструментов BPF и предполагает минимальный уровень знаний, включая знание сетей (например, что такое адрес IPv4) и владение командной строкой.

АВТОРСКИЕ ПРАВА НА ИСХОДНЫЙ КОД

Здесь содержится исходный код многих инструментов BPF. В код каждого инструмента включен комментарий с описанием источника: взят из ВСС, bpftrace или написан специально для этой книги. Полный текст примечания об авторских правах для любого инструмента из ВСС или bpftrace ищите в соответствующем репозитории.

Ниже приводится примечание об авторских правах на новые инструменты, разработанные мной для этой книги. Этот комментарий включен во все листинги инструментов, хранящиеся в репозитории книги, вы должны оставить его при передаче исходного кода другим лицам или адаптации под свои условия:

```
/*
 * Copyright 2019 Brendan Gregg.
 * Licensed under the Apache License, Version 2.0 (the "License").
 * This was originally created for the BPF Performance Tools book
 * published by Addison Wesley. ISBN-13: 9780136554820
 * When copying or porting, include this comment.
 */
```

Допускаю, что какие-то из этих инструментов будут включены в коммерческие продукты для целей мониторинга, как это было с моими более ранними разработками. Если вы возьмете на вооружение инструмент, созданный для этой книги, добавьте в документацию ссылку на эту книгу, технологию BPF и на меня.

Авторские права на рисунки:

Рисунки с 17.2 по 17.9: скриншоты инструмента Vector, © 2016 Netflix, Inc.

Рисунок 17.10: скриншот grafana-pcp-live, Copyright 2019 © Grafana Labs

Рисунки с 17.11 по 17.14: скриншоты Grafana, Copyright 2019 © Grafana Labs

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ И ССЫЛКИ

Сайт этой книги:

<http://www.brendangregg.com/bpf-performance-tools-book.html>

Здесь можно найти исходный код всех инструментов, содержащихся в книге, а также список опечаток и отзывы читателей.

Многие из представленных здесь инструментов доступны и в репозиториях проектов, где они поддерживаются и совершенствуются. Там можно найти самые последние версии этих инструментов:

<https://github.com/iovisor/bcc>

<https://github.com/iovisor/bpfttrace>

В репозиториях есть и подробные справочные и учебные руководства, которые написал я, а сообщество BPF продолжает поддерживать и обновлять.

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

В книге рассматриваются различные виды технологий, а способ подачи материала дает дополнительный контекст.

В листингах, показывающих вывод инструментов, жирный шрифт указывает на выполняемую команду или на что-то интересное. Приглашение к вводу в виде хеша (#) означает, что команда или инструмент запущены от имени суперпользователя root (администратора). Например:

```
# id
uid=0(root) gid=0(root) groups=0(root)
```

Приглашение к вводу в виде доллара (\$) означает, что команда или инструмент запущены от имени обычного, непривилегированного пользователя:

```
$ id
uid=1000(bgregg) gid=1000(bgregg) groups=1000(bgregg),4(adm),27(sudo)
```

Некоторые приглашения к вводу включают префикс с именем каталога, чтобы показать рабочий каталог:

```
bpfttrace/tools$ ./biolatency.bt
```

Для выделения новых терминов и иногда для отображения текста, который требуется заменить, используется курсив.

Большинству инструментов, представленных здесь, для запуска необходимы привилегии суперпользователя root, о чем говорит приглашение к вводу в виде хеша. Чтобы инструмент мог запустить не суперпользователь, в начало строки нужно

добавить команду `sudo(8)` (расшифровывается как `super-user do` — выполнить с привилегиями суперпользователя).

Для некоторых команд требуются одиночные кавычки, чтобы командная оболочка случайно не выполнила ненужную (хотя и маловероятную) операцию подстановки. Это хорошая привычка, рекомендую взять ее на вооружение. Например:

```
# funccount 'vfs_*
```

За именами команд и системных вызовов Linux следует номер раздела справочного руководства `man`, заключенный в круглые скобки, например команда `ls(1)`, системный вызов `read(2)` и команда системного администрирования `funccount(8)`. Пустые скобки обозначают вызовы функций на языке программирования, например функция ядра `vfs_read()`. При включении команды с аргументами в текст абзацев они выделяются моноширинным шрифтом.

Длинный вывод команды, не уместающийся по ширине страницы, усекается и в конец добавляется многоточие в квадратных скобках (`[...]`). Строки, содержащие только пару символов `^C`, указывают, что была нажата комбинация `Ctrl-C` для завершения программы.

Библиографические ссылки оформляются в виде чисел в квадратных скобках, например `[123]`.

БЛАГОДАРНОСТИ

Над созданием компонентов, нужных для инструментов BPF, работали многие люди. Их вклад может быть малозаметным: решение непонятных проблем в инфраструктуре трассировки ядра, инструментах компиляции, верификаторе инструкций или в других сложных компонентах. Такая работа часто недооценивается и остается незамеченной. Тем не менее именно благодаря их усилиям вы можете использовать инструменты BPF. Многие из этих инструментов были написаны мной. Может сложиться ложное представление, будто я написал их в одиночку, — на самом деле я опирался на множество различных технологий и плоды труда других людей. Я хотел бы поблагодарить за работу их, а также всех тех, кто участвовал в создании этой книги.

Технологии, затронутые в книге, и их авторы:

- **eBPF**: спасибо Алексею Старовойтову (Facebook; ранее PLUMgrid) и Дэниелу Боркману (Isovalent; ранее Cisco, Red Hat) за создание технологии, управление разработкой, поддержку кода BPF в ядре и реализацию их идеи eBPF. Спасибо всем другим участникам eBPF и особенно Дэвиду С. Миллеру (David S. Miller, Red Hat) за поддержку и совершенствование технологии. На момент написания этой книги в BPF-сообществе насчитывалось 249 человек, внесших свой вклад в код BPF, а общее число коммитов, отправленных с 2014 года, составило 3224. Основными разработчиками BPF после Дэниеля и Алексея, если судить по

числу коммитов, являются: Якуб Кичински (Jakub Kicinski, Netronome), Йонгхонг Сонг (Yonghong Song, Facebook), Мартин Ка Фай Лау (Martin KaFai Lau, Facebook), Джон Фастабенд (John Fastabend, Isovalent; ранее Intel), Квентин Моне (Quentin Monnet, Netronome), Джеспер Дангаард Брауэр (Jesper Dangaard Brouer, Red Hat), Андрей Игнатов (Andrey Ignatov, Facebook) и Станислав Фомичев (Stanislav Fomichev, Google).

- **ВСС:** спасибо Брэндану Бланко (Brenden Blanco, VMware; ранее PLUMgrid) за создание и развитие ВСС. К числу основных участников этого проекта относятся: Саша Гольдштейн (Sasha Goldshtein, Google; ранее SELA), Йонгхонг Сонг (Yonghong Song, Facebook; ранее PLUMgrid), Тен Цинь (Teng Qin, Facebook), Поль Шиньон (Paul Chaignon, Orange), Висент Марти (Vicent Martí, github), Марк Дрейтон (Mark Drayton, Facebook), Алан Макаливи (Allan McAleavy, Sky) и Гари Чинг-Пан Лин (Gary Ching-Pang Lin, SUSE).
- **bpfftrace:** спасибо Аластеру Робертсону (Alastair Robertson, Yellowbrick Data; ранее G-Research, Cisco) за создание bpfftrace и за высокие требования к качеству кода и наличию всеобъемлющих тестов. Спасибо всем остальным авторам bpfftrace, особенно Матеусу Марчини (Matheus Marchini, Netflix; ранее Shtima), Виллиану Гасперу (William Gasper, Shtima), Дейлу Хэмелю (Dale Hamel, Shopify), Аугусто Меккингу Каринги (Augusto Mecking Caringi, Red Hat) и Дэну Сюю (Dan Xu, Facebook).
- **ply:** спасибо Тобиасу Вальдекранцу (Tobias Waldekranz) за разработку первого высокоуровневого инструмента трассировки на основе BPF.
- **LLVM:** спасибо Алексею Старовойтову, Чандлеру Карруту (Chandler Carruth, Google), Йонгхонг Сонгу и другим за поддержку BPF для LLVM, на которой основаны ВСС и bpfftrace.
- **kprobes:** спасибо всем, кто проектировал, разрабатывал и работал над поддержкой динамического анализа ядра Linux, которая широко используется в этой книге. Среди них Ричард Мур (Richard Moore, IBM), Супарна Бхаттачарья (Suparna Bhattacharya, IBM), Вамси Кришна Сангаварапу (Vamsi Krishna Sangavarapu, IBM), Прасанна С. Панчамухи (Prasanna S. Panchamukhi, IBM), Анант Н. Мавинакаянахалли (Ananth N. Mavinakayanahalli, IBM), Джеймс Кенистон (James Keniston, IBM), Навин Н Рао (Naveen N Rao, IBM), Хьен Нгуен (Hien Nguyen, IBM), Масами Хирамацу (Masami Hiramatsu, Linaro; ранее Hitachi), Расти Линч (Rusty Lynch, Intel), Анил Кешавамурти (Anil Keshavamurthy, Intel), Расти Рассел (Rusty Russell), Уилл Коэн (Will Cohen, Red Hat) и Дэвид С. Миллер (David S. Miller, Red Hat).
- **uprobes:** спасибо Шрикару Дронамражу (Srikar Dronamraju, IBM), Джиму Кенистону (Jim Keniston) и Олегу Нестерову (Oleg Nesterov, Red Hat) за разработку инструментов уровня пользователя для динамического анализа ядра Linux и Петеру Зийльстре (Peter Zijlstra) за рецензирование этой книги.
- **точки трассировки:** спасибо Матье Деснойерсу (Mathieu Desnoyers, EfficiOS) за его вклад в технологию трассировки для Linux. В частности, Матье разработал

и предложил статические точки трассировки для включения в ядро, что позволило создавать стабильные инструменты трассировки и приложения.

- **perf**: спасибо Арнальдо Карвальо де Мело (Arnaldo Carvalho de Melo, Red Hat) за его работу над утилитой `perf(1)`, добавившей в ядро новые возможности, которые используются инструментами BPF.
- **Ftrace**: спасибо Стивену Постедту (Steven Rostedt, VMware; ранее Red Hat) за трассировщик Ftrace и вообще за его вклад в технологию трассировки. Ftrace помог в разработке инструментов трассировки на основе BPF, так как я по возможности перепроверял вывод своих инструментов, сопоставляя его с выводом эквивалентных инструментов в Ftrace. Спасибо также Тому Занусси (Tom Zanussi, Intel), недавно внесшему свой вклад в историю Ftrace.
- **(Классический) BPF**: спасибо Ван Якобсону (Van Jacobson) и Стиву Маккану (Steve McCanne).
- **Динамическая инструментация**: спасибо профессору Бартону Миллеру (Barton Miller, Университет штата Висконсин в городе Мэдисон) и его студенту Джефффри Холлингсворту (Jeffrey Hollingsworth) за создание области динамической инструментации в 1992 году [Hollingsworth 94], которая во многом обусловила появление DTrace, SystemTap, BCC, bpftrace и других динамических трассировщиков. Большинство инструментов в этой книге основаны на динамической инструментации (точнее, те из них, что используют kprobes и uprobes).
- **LTT**: спасибо Кариму Ягмуре (Karim Yaghmour) и Мишелю Р. Дагенаису (Michel R. Dagenais) за разработку LTT — первого трассировщика для Linux — в 1999 году. Также спасибо Кариму за его неустанные усилия по продвижению технологий трассировки в сообществе Linux, а также за создание и поддержку более поздних трассировщиков.
- **Dprobes**: Спасибо Ричарду Дж. Муру (Richard J. Moore) и его команде в IBM за разработку DProbes в 2000 году — первой технологии динамической инструментации для Linux, которая привела к появлению современной технологии kprobes.
- **SystemTap**: несмотря на то что SystemTap не используется в этой книге, работа Фрэнка Ч. Иглера (Frank Ch. Eigler, Red Hat) и других пользователей SystemTap очень способствовала совершенствованию технологий трассировки в Linux. Часто они первыми внедряли трассировку в новые области и обнаруживали ошибки в технологиях трассировки ядра.
- **ktap**: спасибо Джови Чжанвэю (Jovi Zhangwei) за ktap — высокоуровневый трассировщик, который помог в создании поддержки трассировщиков для Linux на основе виртуальных машин.
- Также спасибо инженерам Sun Microsystems Брайану Кантриллу (Bryan Cantrill), Майку Шапиро (Mike Shapiro) и Адаму Левенталю (Adam Leventhal) за их выдающуюся работу по разработке DTrace — первой технологии динамической инструментации, появившейся в 2005 году и получившей широкое распространение. Спасибо специалистам по маркетингу и продажам в Sun,

популяризаторам и многим другим, как внутри, так и за пределами Sun, что помогли сделать DTrace широко известной в мире и увеличить спрос на трассировщики в Linux.

Спасибо всем остальным, не перечисленным здесь, кто также вносил свой вклад в эти технологии на протяжении многих лет.

Кроме создания перечисленных технологий, многие из этих людей помогли в работе над моей книгой: Дэниел Боркман сделал весьма ценные технические замечания и предложения для нескольких глав. Алексей Старовойтов дал критические отзывы и советы к тексту с описанием ядра eBPF (а также написал предисловие для книги). Аластер Робертсон поделился информацией для главы о bpftrace, а Йонгхонг Сонг давал консультации по BTF во время разработки BTF.

Многие, кто сыграл активную роль в разработке технологий BPF, помогали в работе над этой книгой, предоставляя материалы и оставляя замечания.

Спасибо Матеусу Марчини (Matheus Marchini, Netflix), Полу Шеньону (Paul Chaignon, Orange), Дейлу Хэмелу (Dale Hamel, Shopify), Амеру Азеру (Amer Ather, Netflix), Мартину Спайеру (Martin Spier, Netflix), Брайану В. Кернигану (Brian W. Kernighan, Google), Джоэлу Фернандесу (Joel Fernandes, Google), Джесперу Брауэру (Jesper Brouer, Red Hat), Грегу Данну (Greg Dunn, AWS), Джулии Эванс (Julia Evans, Stripe), Токе Хойланд-Йоргенсену (Toke Høiland-Jørgensen, Red Hat), Станиславу Козине (Stanislav Kozina, Red Hat), Иржи Ольсу (Jiri Olsa, Red Hat), Дженсу Аксбо (Jens Axboe, Facebook), Джону Хасламу (Jon Haslam, Facebook), Андрию Накрийко (Andrii Nakryiko, Facebook), Сагуну Диллону (Sargun Dhillon, Netflix), Алексу Маэстретти (Alex Maestretti, Netflix), Джозефу Линчу (Joseph Lynch, Netflix), Ричарду Эллингу (Richard Elling, Viking Enterprise Solutions), Брюсу Кертису (Bruce Curtis, Netflix) и Хавьеру Гондувилле Кото (Javier Honduvilla Coto, Facebook). Благодаря им многие разделы были переписаны, дополнены и улучшены. С некоторыми разделами мне помогли Матье Деснойер (Mathieu Desnoyers, EfficiOS) и Масами Хираматсу (Masami Hiramatsu, Linaro). Клэр Блэк (Claire Black) выполнила окончательную проверку глав и дала свои замечания.

Мой коллега Джейсон Кох (Jason Koch) написал большую часть главы «Другие инструменты» и прокомментировал почти все главы (он писал замечания вручную в печатной копии толщиной около двух дюймов).

Ядро Linux — сложный и постоянно меняющийся программный продукт, и я высоко ценю труд Джонатана Корбета (Jonathan Corbet) и Джейка Эджа (Jake Edge) из lwn.net по обобщению большого числа сложнейших тем. Многие из их статей упоминаются в библиографии в конце книги.

Для завершения этой книги также потребовалось добавить множество новых возможностей в интерфейсы BCC и bpftrace и устранить ряд проблемы. Я и мои коллеги написали тысячи строк кода, чтобы обеспечить возможность создания инструментов, представленных в этой книге. В связи с этим я хочу выразить особую благодарность Матеусу Марчини (Matheus Marchini), Виллиану Гасперу (Willian Gasper), Дейлу

Хэмелу (Dale Hamel), Дэну Сюю (Dan Xu) и Аугусто Каринги (Augusto Caringi) за своевременные исправления.

Спасибо моим нынешнему и бывшему руководителям в Netflix Эду Хантеру (Ed Hunter) и Кобурну Вотсону (Coburn Watson) за поддержку моей работы над BPF. Также спасибо моим коллегам Скотту Эммонсу (Scott Emmons), Брайану Майлзу (Brian Moyses) и Габриелю Муносу (Gabrielle Munoz) за помощь в установке BCC и bpftrace на производственных серверах в Netflix, благодаря которым мне удалось получить множество примеров скриншотов.

Спасибо моей жене Дейдрэ Страуган (Deirdré Straughan, AWS) за ее научную редактуру и предложения, а также за поддержку еще одной книги. Мои навыки писателя заметно усовершенствовались благодаря ее многолетней помощи. И спасибо моему сыну Митчеллу за поддержку и терпение, пока я был занят написанием.

Написать эту книгу меня вдохновила книга о DTrace, написанная мной и Джимом Мауро (Jim Mauro). Упорный труд Джима, направленный на достижение максимального успеха книги о DTrace, и наши бесконечные дискуссии о структуре и описаниях инструментов во многом помогли повысить качество этой книги. Джим, спасибо за все.

Отдельное спасибо старшему редактору издательства Pearson Грегу Доенчу (Greg Doench) за помощь и энтузиазм, проявленный при работе над этим проектом.

Эта работа дала мне шанс показать возможности BPF. Из 156 инструментов, представленных здесь, 135 написаны мной, в том числе 89 новых инструментов, созданных специально для этой книги (вообще их больше ста, если считать все разновидности, хотя я не рассчитывал достигнуть этого рубежа). Для создания новых инструментов потребовалось заняться дополнительными исследованиями, выполнить настройки сред серверных и клиентских приложений, провести эксперименты и произвести тестирование. Порой это было очень утомительно, но в итоге я испытал приятное чувство глубокого удовлетворения, зная, что эти инструменты для многих будут иметь немалую ценность.

Брендан Грегг (Brendan Gregg)

Сан-Хосе, Калифорния (а перед этим Сидней, Австралия),
ноябрь 2019

ОБ АВТОРЕ

Брендан Грегг — старший перформанс-инженер в Netflix, основной контрибьютор проекта BPF (eBPF), помогавший разрабатывать и поддерживать оба интерфейса BPF. Впервые применил BPF для анализа производительности и создал десятки инструментов на основе BPF. Автор бестселлера «Systems Performance: Enterprise and the Cloud»¹.

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

ОТ ИЗДАТЕЛЬСТВА

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Глава 1

ВВЕДЕНИЕ

https://t.me/it_boooks/2

Эта глава знакомит с основными терминами и технологиями и показывает применение некоторых инструментов анализа производительности на основе BPF. Представленные здесь технологии будут подробно описаны в последующих главах.

1.1. ЧТО ТАКОЕ BPF И EBPF?

BPF расшифровывается как Berkeley Packet Filter (пакетный фильтр Беркли). Эта малоизвестная когда-то технология, разработанная в 1992 году, создавалась для увеличения производительности инструментов захвата пакетов [McCanne 92]. В 2013 году Алексей Старовойтов предложил существенно переделанную версию BPF [2], развитие которой продолжил с Дэниелом Боркманом. В 2014 она была включена в ядро Linux [3]. Это превратило BPF в механизм выполнения общего назначения, который можно использовать для самых разных целей, включая создание продвинутых инструментов анализа производительности.

Из-за широкой области применения BPF трудно описать. Она позволяет запускать мини-программы для обработки самых разных событий, происходящих в ядре и приложениях. Те, кто знаком с JavaScript, наверняка заметят некоторые сходства: с помощью JavaScript можно запускать на сайте мини-программы для обработки событий в браузере, например щелчков мыши, что дает возможность создавать веб-приложения для самых разных целей. Механизм BPF позволяет ядру запускать мини-программы в ответ на события в системе и в приложениях, таких как дисковый ввод/вывод, что открывает дорогу для новых системных технологий. Он делает ядро полностью программируемым и дает пользователям (включая прикладных программистов) возможность настраивать и контролировать свои системы для решения насущных проблем.

BPF — это гибкая и эффективная технология, состоящая из набора инструкций, хранимых объектов и вспомогательных функций. Она определяет набор виртуальных инструкций, поэтому ее можно считать виртуальной машиной. Эти инструкции исполняются средой выполнения BPF в ядре Linux, которая включает интерпретатор и JIT-компилятор, преобразующие инструкции BPF в машинные инструкции. Инструкции BPF сначала попадают в верификатор,

который проверяет их безопасность и гарантирует, что программа BPF не приведет к сбою или повреждению ядра (что не мешает конечному пользователю писать нелогичные программы, которые могут выполняться, но не иметь смысла). Компоненты BPF подробно описаны в главе 2.

До сих пор технология BPF в основном использовалась для организации сетевых взаимодействий, наблюдаемости и обеспечения безопасности. В этой книге основное внимание уделяется наблюдаемости (трассировке).

Расширенную версию BPF часто обозначают аббревиатурой eBPF (extended BPF — расширенная BPF), но официальной считается прежнее обозначение — BPF, без «е», поэтому в этой книге я буду использовать аббревиатуру «BPF». Ядро содержит только один механизм выполнения BPF (расширенный BPF), который выполняет инструкции как расширенной технологии BPF, так и «классической» BPF¹.

1.2. ЧТО ТАКОЕ ТРАССИРОВКА, ПРОСЛУШИВАНИЕ, ВЫБОРКА, ПРОФИЛИРОВАНИЕ И НАБЛЮДАЕМОСТЬ?

Эти термины используются для классификации методов и инструментов анализа.

Трассировка (tracing) — технология фиксации (записи) происходящих событий, основанная на использовании соответствующих инструментов BPF. Возможно, вам уже приходилось иметь дело с некоторыми специализированными инструментами трассировки. Инструмент `strace(1)`, например, фиксирует и выводит события обращения к системным вызовам. Есть и инструменты, которые не трассируют, а измеряют события, используя фиксированные статистические счетчики, а затем выводят их, как, например, `top(1)`. Отличительная черта трассировщика — это способность фиксировать исходные события и их метаданные. Такая информация может иметь очень большой объем, требующий последующей обработки и обобщения. Программные трассировщики, использующие BPF, могут запускать небольшие программы для обработки событий и формировать статистические метрики «на лету» или выполнять другие действия, чтобы избежать последующей дорогостоящей обработки.

Не у всех трассировщиков в названии есть слово «trace», как `strace(1)`. Например, `tcpdump(8)` — это еще один специализированный инструмент трассировки сетевых пакетов. (Возможно, его следовало назвать `tcptrace`?) В ОС Solaris была своя версия `tcpdump` с именем `snoop(1M)`², названная так потому, что использовалась для прослушивания (snooping) сетевых пакетов. Я был первым, кто разработал

¹ Программы на основе классической версии BPF [McCanne 92] автоматически выполняются под управлением расширенного механизма BPF. Кроме того, развитие классической технологии BPF было прекращено.

² Раздел 2 в справочном руководстве Solaris предназначен для команд администрирования и обслуживания (в Linux ему соответствует раздел 8).

и опубликовал множество инструментов трассировки для Solaris, в названиях которых (может и неправильно) использовал «snoop». Поэтому у нас теперь есть ехесnoop(8), opensnoop(8), biosnoop(8) и т. д. Прослушивание, вывод событий и трассировка обычно обозначают одно и то же. Эти инструменты будут описаны в следующих главах.

Термин «трассировка» (tracing) встречается не только в названиях инструментов, но также в описании механизма BPF, когда тот используется для наблюдаемости.

Инструменты **выборки (sampling)** выполняют некоторые измерения, чтобы составить общую картину цели. Их также называют инструментами создания профиля, или профилирования. Есть BPF-инструмент под названием profile(8), который берет выполняемый код по таймеру. Например, он может производить выборку каждые 10 миллисекунд, или, иначе говоря, 100 раз в секунду (на каждом процессоре). Преимущество инструментов выборки состоит в том, что у них обычно более низкий уровень оверхеда, чем у трассировщиков, потому что они оценивают только одно из большого числа событий. С другой стороны, выборка дает только приблизительную картину и может пропускать некоторые важные события.

Под **наблюдаемостью (observability)** понимается исследование системы через наблюдение с использованием специальных инструментов. К инструментам наблюдаемости относятся инструменты трассировки и выборки, а также инструменты, основанные на фиксированных счетчиках. К ним не относятся инструменты бенчмаркинга, которые изменяют состояние системы, экспериментируя с рабочей нагрузкой. Инструменты BPF, представленные в этой книге, все относятся к категории инструментов наблюдаемости и используют BPF для трассировки программ.

1.3. ЧТО ТАКОЕ BCC, BPFTRACE И IO VISOR?

Программировать, используя непосредственно инструкции BPF, довольно сложно, поэтому для языков высокого уровня были разработаны интерфейсы. Для трассировки в основном используются интерфейсы BCC и bpftrace.

Первой инфраструктурой трассировки на основе BPF стала **BCC** (BPF Compiler Collection — коллекция компиляторов для BPF). Она предоставляет среду программирования на C для использования BPF и интерфейсы для других языков: Python, Lua и C++. Она также дала начало библиотекам libbcc и текущей версии libbpf¹ с функциями для обработки событий с помощью программ BPF. Репозиторий BCC содержит более 70 инструментов на основе BPF для анализа производительности и устранения неполадок. Вы можете установить BCC в своей системе и использовать готовые инструменты, не написав ни строчки кода для BCC. В этой книге вы познакомитесь со многими такими инструментами.

¹ Первая версия libbpf была разработана Ван Нанем (Wang Nan) для использования с perf [4]. Сейчас libbpf — это часть исходного кода ядра.

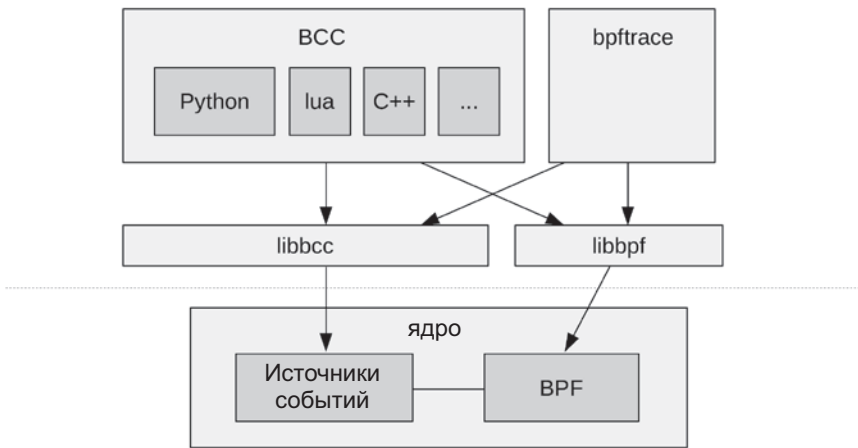


Рис. 1.1. BCC, bpftrace и BPF

bpftrace — более новый интерфейс, предоставляющий специализированный язык высокого уровня для разработки инструментов BPF. Язык bpftrace настолько лаконичен, что мне удалось вставить в эту книгу исходный код инструментов, чтобы показать, какие механизмы они используют и как именно. bpftrace построен на основе библиотек libbcc и libbpf.

Связь между BCC и bpftrace можно видеть на рис. 1.1. Они прекрасно дополняют друг друга: bpftrace идеально подходит для создания однострочных и нестандартных коротких сценариев, а BCC лучше подходит для сложных сценариев и демонов и позволяет использовать другие библиотеки. Например, многие BCC-инструменты на Python используют библиотеку `argparse` для сложного и точного управления аргументами командной строки.

Сейчас разрабатывается еще один интерфейс BPF — `ply` [5]. Предполагается, что он должен получиться максимально легковесным и с минимумом зависимостей, чтобы его можно было использовать во встраиваемых системах Linux. Если для вашей среды `ply` подходит лучше, чем bpftrace, эта книга все равно будет вам полезна в качестве руководства по анализу с использованием BPF. Десятки инструментов на bpftrace, представленные здесь, с таким же успехом могут выполняться с `ply`, если их переписать с помощью синтаксиса `ply`. (В будущих версиях `ply` может появиться прямая поддержка синтаксиса bpftrace.) Основное внимание в этой книге уделяется интерфейсу bpftrace как более развитому и обладающему всеми возможностями, необходимыми для анализа всех целей.

BCC и bpftrace не входят в код ядра и размещены в проекте Linux Foundation на Github с названием **IO Visor**:

<https://github.com/iovisor/bcc>

<https://github.com/iovisor/bpftrace>

В этой книге под словами *трассировка с использованием BPF* я буду подразумевать инструменты, использующие любой из интерфейсов — BCC и bpftrace.

1.4. ПЕРВЫЙ ВЗГЛЯД НА ВСС: БЫСТРЫЙ АНАЛИЗ

Посмотрим на некоторые результаты, возвращаемые разными инструментами. Следующий инструмент следит за новыми процессами и выводит сводную информацию о каждом сразу после его запуска. Этот ВСС-инструмент `execsnoop(8)` трассирует системный вызов `execve(2)`, который является вариантом `exec(2)` (отсюда и имя). Установка инструментов ВСС описана в главе 4, и в следующих главах эти инструменты будут представлены подробнее.

```
# execsnoop
PCOMM      PID    PPID    RET    ARGS
run         12983  4469    0      ./run
bash        12983  4469    0      /bin/bash
svstat      12985  12984   0      /command/svstat /service/httpd
perl        12986  12984   0      /usr/bin/perl -e $1=<>;$1=~/(d+) sec;print
$1||0
ps          12988  12987   0      /bin/ps --ppid 1 -o pid,cmd,args
grep        12989  12987   0      /bin/grep org.apache.catalina
sed         12990  12987   0      /bin/sed s/^ *//;
cut         12991  12987   0      /usr/bin/cut -d -f 1
xargs       12992  12987   0      /usr/bin/xargs
echo        12993  12992   0      /bin/echo
mkdir       12994  12983   0      /bin/mkdir -v -p /data/tomcat
mkdir       12995  12983   0      /bin/mkdir -v -p /apps/tomcat/webapps
^C
#
```

В полученном выводе видно, какие процессы запускались, пока происходила трассировка: в основном это настолько кратковременные процессы, что они невидимы для других инструментов. Здесь можно видеть строки, соответствующие запуску стандартных утилит Unix: `ps(1)`, `grep(1)`, `sed(1)`, `cut(1)` и т. д. Но рассматривая этот вывод на книжной странице, нельзя сказать, насколько быстро выводились показанные строки. Чтобы исправить этот недостаток, добавим параметр командной строки `-t`, который заставит `execsnoop(8)` выводить время, прошедшее с начала трассировки:

```
# execsnoop -t
TIME(s) PCOMM      PID    PPID    RET    ARGS
0.437   run         15524  4469    0      ./run
0.438   bash        15524  4469    0      /bin/bash
0.440   svstat      15526  15525   0      /command/svstat /service/httpd
0.440   perl        15527  15525   0      /usr/bin/perl -e $1=<>;$1=~/(d+)
sec;prin...
0.442   ps          15529  15528   0      /bin/ps --ppid 1 -o pid,cmd,args
[...]
0.487   catalina.sh 15524  4469    0      /apps/tomcat/bin/catalina.sh start
```

```

0.488  dirname      15549 15524  0  /usr/bin/dirname /apps/tomcat/bin/
                                catalina.sh
1.459  run           15550 4469   0  ./run
1.459  bash           15550 4469   0  /bin/bash
1.462  svstat         15552 15551  0  /command/svstat /service/nflx-httpd
1.462  perl           15553 15551  0  /usr/bin/perl -e $1=<>;$1=~/(\\d+)
sec;/prin...
[...]
```

Я сократил вывод (о чем свидетельствует [...]), но новый столбец с отметкой времени помогает заметить закономерность: новые процессы запускаются с интервалом в 1 секунду. Просматривая вывод, я могу сказать, что каждую секунду запускается 30 новых процессов, после чего следует пауза в 1 секунду.

В этом выводе показана реальная проблема, которая была в Netflix и которую я анализировал с помощью `execsnoop(8)`. Это происходило на сервере для микробенчмаркинга, но результаты бенчмарков слишком сильно отличались, чтобы им доверять. Я запустил `execsnoop(8)`, когда система должна была простаивать, и обнаружил проблему! Каждую секунду эти процессы запускались и нарушали работу бенчмарков. Причина крылась в неправильно настроенном сервисе, который пытался запуститься каждую секунду, терпел неудачу и запускался снова. После того как сервис был деактивирован, эти процессы перестали появляться (что было подтверждено с помощью `execsnoop(8)`) и бенчмарки стали стабильными.

Вывод `execsnoop(8)` помогает в анализе производительности с использованием методологии под названием *характеристика рабочей нагрузки*, которая поддерживается многими другими инструментами BPF, описанными здесь. Эта методология решает простую задачу: определить величину рабочей нагрузки. Понимания, как распределяется рабочая нагрузка, часто достаточно для решения проблем, без нужды углубляться в исследование задержек или в детальный анализ. Здесь к системе была применена процедура определения рабочей нагрузки. Более подробно с этой и другими методологиями вы познакомитесь в главе 3.

Попробуйте запустить инструмент `execsnoop(8)` в своих системах и оставьте его поработать в течение часа. Что необычного вы заметили?

`execsnoop(8)` выводит информацию о каждом событии, однако другие инструменты используют BPF для получения сводной информации. Еще один инструмент, который можно использовать для быстрого анализа, — `biolatency(8)`, который обобщает сведения об операциях ввода/вывода для блочного устройства (дисковый ввод/вывод) в виде гистограммы задержки.

Ниже показан вывод инструмента `biolatency(8)` на промышленной базе данных, которая чувствительна к высокой задержке, так как имеет соглашение об уровне обслуживания по доставке запросов в течение определенного количества миллисекунд.

```

# biolatency -m
Tracing block device I/O... Hit Ctrl-C to end.
```

```

^C
msecs      : count      distribution
  0 -> 1    : 16335      *****|
  2 -> 3    : 2272       *****|
  4 -> 7    : 3603       *****|
  8 -> 15   : 4328       *****|
 16 -> 31   : 3379       *****|
 32 -> 63   : 5815       *****|
 64 -> 127  : 0          |
128 -> 255  : 0          |
256 -> 511  : 0          |
512 -> 1023 : 11         |

```

После запуска инструмент `biolateness(8)` включает регистрацию событий блочного ввода/вывода, а их задержки вычисляются и обобщаются механизмом BPF. Когда инструмент завершается (пользователь нажимает `Ctrl-C`), выводится сводная информация. Я добавил параметр `-m`, чтобы обеспечить вывод информации в миллисекундах.

В этом выводе есть интересная деталь — бимодальный характер распределения задержек, а также наличие выбросов. Наиболее типичный режим (как видно на ASCII-гистограмме) приходится на диапазон от 0 до 1 миллисекунды, которому соответствует 16 355 операций ввода/вывода, зарегистрированных во время трассировки. Скорее всего, к этим операциям относятся попадания в дисковый кэш, а также операции с устройством флеш-памяти. Второй наиболее типичный режим охватывает диапазон от 32 до 63 миллисекунд и включает намного более медленные операции, чем ожидалось, и вероятно, это замедление обусловлено постановкой запросов в очередь. Для более подробного исследования этого режима можно использовать другие инструменты BPF. Наконец, 11 операций ввода/вывода попали в диапазон от 512 до 1023 миллисекунд. Эти очень медленные операции называют *выбросами*. Теперь, когда мы знаем, что они есть, их можно более детально изучить с помощью других инструментов BPF. Для команды базы данных это приоритетная задача: если БД будет блокироваться на этих операциях ввода/вывода, произойдет превышение целевого уровня задержки.

1.5. ОБЛАСТЬ ВИДИМОСТИ МЕХАНИЗМА ТРАССИРОВКИ BPF

Механизму трассировки BPF доступен для наблюдения весь программный стек, что позволяет создавать новые инструменты и производить инструментацию по мере необходимости. Механизм трассировки BPF можно использовать на промышленном сервере без всякой подготовки, то есть без перезагрузки системы или перезапуска приложений в каком-то специальном режиме. Его можно сравнить с рентгеновским зрением: если нужно исследовать какой-то компонент ядра, устройство или

прикладную библиотеку, вы сможете увидеть все это в таком свете, в каком никто и никогда не видел их — вживую на продакшене.

Для иллюстрации на рис. 1.2 показан обобщенный программный стек системы, который я аннотировал названиями инструментов трассировки на основе BPF, предназначенными для наблюдения за различными компонентами. Все эти инструменты созданы в проектах BCC и bpftrace, а также специально для этой книги. Многие из них будут описаны в следующих главах.

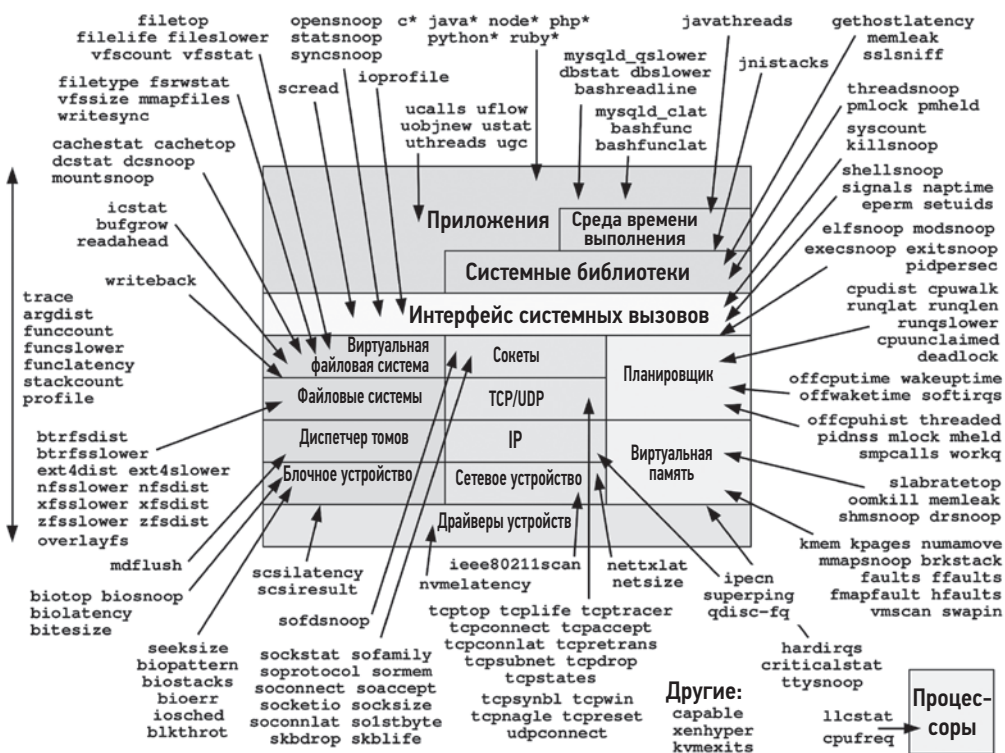


Рис. 1.2. Инструменты оценки производительности на основе BPF и их области видимости

Посмотрите на различные инструменты, которые можно использовать для исследования таких компонентов, как планировщик процессов в ядре, виртуальная память, файловые системы и т. д. Из диаграммы на рис. 1.2 можно заметить, что у механизма BPF нет слепых зон.

В табл. 1.1 перечислены инструменты, традиционно используемые для исследования этих компонентов, а также указана способность механизма ВРФ наблюдать за этими компонентами.

Таблица 1.1. Традиционные инструменты анализа

Компоненты	Традиционные инструменты анализа	Механизм трассировки BPF
Приложения со средами выполнения своих языков: Java, Node.js, Ruby, PHP	Отладчики среды времени выполнения	Да, с поддержкой среды времени выполнения
Приложения на компилируемых языках: C, C++, Golang	Системные отладчики	Да
Системные библиотеки: /lib/*	ltrace(1)	Да
Интерфейс системных вызовов	strace(1), perf(1)	Да
Ядро: планировщик, файловые системы, сетевые протоколы (TCP, IP и др.)	Ftrace, perf(1) для выборки	Да, и более подробно
Аппаратное обеспечение: процессоры, устройства	perf, sag, счетчики в /proc	Да, прямо или косвенно ¹

Традиционные инструменты могут дать полезную начальную информацию о компоненте, исследование которого можно продолжить на более детальном уровне с помощью инструментов трассировки BPF. В главе 3 кратко описывается базовый анализ производительности с помощью системных инструментов, помогающий получить начальную информацию.

1.6. ДИНАМИЧЕСКАЯ ИНСТРУМЕНТАЦИЯ: KPROBES И UPROBES

Механизм трассировки BPF поддерживает несколько источников событий для обеспечения видимости всего программного стека. Особого упоминания заслуживает поддержка динамической инструментации (иногда ее называют динамической трассировкой) — возможность вставлять контрольные точки в действующее ПО в процессе его выполнения. Динамическая инструментация не порождает оверхед, когда не используется, так как в этом случае ПО выполняется в своем первоначальном виде. Она часто применяется инструментами BPF для определения начала и конца выполнения функций в ядре и в приложении (для любой из десятков тысяч функций, которые обычно имеются в программном стеке). Такое глубокое и всеобъемлющее видение напоминает суперсилу!

¹ BPF может не иметь возможности непосредственно инструментировать прошивку на устройстве, но он может косвенно определять поведение на основе отслеживания событий драйвера ядра или РМС.

Впервые идея динамической инструментации была предложена в 1990-х годах [Hollingsworth 94]. Она основывалась на технологии, используемой отладчиками для вставки точек останова по произвольным адресам команд. Встретив такую точку, динамически инструментированное ПО записывает нужную информацию и автоматически продолжает выполнение, не передавая управление интерактивному отладчику. Тогда же были разработаны первые инструменты динамической трассировки (например, kerninst [Tamches 99]), включавшие языки трассировки, но эти инструменты оставались малоизвестными и редко используемыми. Отчасти это было связано с тем, что их применение сопряжено с большим риском: динамическая трассировка требует изменения инструкций в адресном пространстве, и любая ошибка может привести к немедленному повреждению кода и аварийному завершению процесса или ядра.

Первая поддержка динамической инструментации в Linux была реализована в 2000 году в IBM, она получила название DProbes, но набор исправлений был отклонен¹. Динамическая инструментация для функций ядра (kprobes), добавленная в Linux в 2004 году и корнями уходящая в DProbes, все еще оставалась малоизвестной и сложной в использовании.

Все изменилось в 2005 году, когда Sun Microsystems выпустила свою версию динамической трассировки DTrace с простым в использовании языком D и включила ее в ОС Solaris 10. В ту пору Solaris была известной операционной системой, славившейся стабильностью работы, поэтому включение в нее пакета DTrace помогло доказать, что динамическая трассировка может быть безопасной для применения в промышленных системах. Это стало поворотным моментом для технологии. Я опубликовал много статей, где показаны реальные случаи использования DTrace, а также разработал и опубликовал множество инструментов DTrace. Кроме того, отдел маркетинга в Sun продвигал не только продажи, но и технологии Sun. Считалось, что это дает дополнительные конкурентные преимущества. Sun Educational Services включили DTrace в стандартные курсы изучения Solaris и разработали специальные курсы по DTrace. Все это способствовало превращению динамической инструментации из малопонятной технологии в широко известную и востребованную.

В 2012 году в Linux была добавлена поддержка динамической инструментации для функций пользовательского уровня в виде uprobes. Механизм трассировки BPF использует и kprobes, и uprobes для динамической инструментации всего программного стека.

Чтобы показать, как можно применять динамическую трассировку, в табл. 1.2 приведены примеры зондов bpftrace, которые используют kprobes и uprobes (bpftrace подробно рассматривается в главе 5).

¹ Причины отказа принять DProbes в ядро Linux обсуждаются в первом примере в статье Энди Клина (Andi Kleen) «On submitting kernel patches», где дается ссылка на источник в Documentation/process/submitting-patches.rst [6].

Таблица 1.2. Примеры использования `kprobe` и `uprobe` в `bpfttrace`

Зонд	Описание
<code>kprobe:vfs_read</code>	Инструментирует начало выполнения функции ядра <code>vfs_read()</code>
<code>kretprobe:vfs_read</code>	Инструментирует возврат ¹ из функции ядра <code>vfs_read()</code>
<code>uprobe:/bin/bash:readline</code>	Инструментирует начало выполнения функции <code>readline()</code> в <code>/bin/bash</code>
<code>uretprobe:/bin/bash:readline</code>	Инструментирует возврат из функции <code>readline()</code> в <code>/bin/bash</code>

1.7. СТАТИЧЕСКАЯ ИНСТРУМЕНТАЦИЯ: ТОЧКИ ТРАССИРОВКИ И USDT

Динамическая инструментация имеет и обратную сторону: инструменты могут переименовываться или удаляться в новой версии ПО. Это называется *проблемой стабильности интерфейса*. После обновления ядра или прикладного ПО иногда оказывается, что инструмент BPF работает не так, как должен. Он может вызывать ошибку, сообщая о невозможности найти функцию для инструментации, или вообще ничего не выводить. Другая проблема в том, что компиляторы могут преобразовывать функции во встраиваемые фрагменты кода с целью оптимизации, делая их недоступными для инструментации через `kprobes` или `uprobes`².

Одно из решений проблем стабильности интерфейса и встраивания функций — это статическая инструментация, когда стабильные имена событий внедряются в ПО и поддерживаются разработчиками. Механизм трассировки BPF поддерживает точки трассировки для статической инструментации ядра и статически определяемые точки трассировки на уровне пользователя (User-level Statically Defined Tracing, USDT) для статической инструментации на уровне пользователя. Недостаток статической инструментации в том, что поддержка таких точек инструментации становится бременем для разработчиков, поэтому если они существуют, их количество обычно ограничено.

Эти детали важны, только если вы собираетесь разрабатывать свои инструменты BPF. В таком случае рекомендуется сначала попытаться использовать статическую трассировку (с использованием точек трассировки и USDT), а к динамической трассировке (с помощью `kprobes` и `uprobes`) переходить только тогда, когда статическая недоступна.

¹ Функция имеет одну точку входа, но может иметь несколько точек выхода: она может вызывать `return` в нескольких местах. Зонд, настроенный на возврат из функции, реагирует на все точки выхода. (См. главу 2, где объясняется, как это возможно.)

² Одно из возможных решений — трассировка по смещению в функции, но этот прием еще менее стабилен, чем трассировка входа в функцию.

В табл. 1.3 приводятся примеры зондов bpftrace для статической инструментации с использованием точек трассировки и USDT. Об упомянутой в этой таблице точке трассировки `open(2)` рассказано в разделе 1.8.

Таблица 1.3. Примеры точек трассировки и USDT в bpftrace

Зонд	Описание
<code>tracepoint:syscalls:sys_enter_open</code>	Инструментирует системный вызов <code>open(2)</code>
<code>usdt:/usr/sbin/mysqld:mysql:query__start</code>	Инструментирует вызов <code>query__start</code> из <code>/usr/sbin/mysqld</code>

1.8. ПЕРВЫЙ ВЗГЛЯД НА BPFTRACE: ТРАССИРОВКА OPEN()

Начнем знакомство с bpftrace с попытки выполнить трассировку системного вызова `open(2)`. Для этого есть статическая точка трассировки (`syscalls:sys_enter_open`¹). Я покажу дальше короткую программу на bpftrace в командной строке: однострочный сценарий.

От вас пока не требуется понимать код этого однострочного сценария; язык bpftrace и инструкции по установке описаны ниже, в главе 5. Но вы наверняка догадаетесь, что делает эта программа, даже не зная языка, потому что он достаточно прост и понятен (понятность языка — признак хорошего дизайна). А пока просто обратите внимание на вывод инструмента.

```
# bpftrace -e 'tracepoint:syscalls:sys_enter_open { printf("%s %s\n", comm,
    str(args->filename)); }'
Attaching 1 probe...
slack /run/user/1000/gdm/Xauthority
slack /run/user/1000/gdm/Xauthority
slack /run/user/1000/gdm/Xauthority
slack /run/user/1000/gdm/Xauthority
^C
#
```

Вывод показывает имя процесса и имя файла, переданное системному вызову `open(2)`: bpftrace трассирует всю систему, поэтому в выводе будет видно любое приложение, вызывающее `open(2)`. Каждая строка вывода соответствует одному системному вызову, и этот сценарий является примером инструмента, который выводит информацию о каждом событии. Механизм трассировки BPF можно использовать не только для анализа промышленных серверов. Например, я запускал его на своем ноутбуке, когда писал эту книгу, и он показывал файлы, которые открывало приложение чата Slack.

¹ Для доступа к этим точкам трассировки системных вызовов ядро Linux должно быть собрано с включенным параметром `CONFIG_FTRACE_SYSCALLS`.

Программа для BPF определена в одинарных кавычках. Она была скомпилирована и запущена сразу, как только я нажал **Enter** для запуска команды `bpfftrace`. `bpfftrace` также активировала точку трассировки `open(2)`. Когда я нажал **Ctrl-C**, чтобы остановить команду, точка трассировки `open(2)` была деактивирована и моя маленькая программа для BPF была остановлена и удалена. Вот как работает инструментация в механизме трассировки BPF: активация точки трассировки и выполнение производятся, только пока выполняется команда, и могут длиться всего несколько секунд.

Этот сценарий генерировал вывод медленнее, чем я ожидал: думаю, что я пропустил какие-то события системного вызова `open(2)`. Ядро поддерживает несколько вариантов `open`, а я трассировал только один из них. С помощью `bpfftrace` можно вывести список всех точек трассировки `open`, используя параметр `-l` и подстановочный знак:

```
# bpfftrace -l 'tracepoint:syscalls:sys_enter_open*'
tracepoint:syscalls:sys_enter_open_by_handle_at
tracepoint:syscalls:sys_enter_open
tracepoint:syscalls:sys_enter_openat
```

Как мне кажется, вариант `openat(2)` используется чаще. Мое предположение подтвердил другой однострочный сценарий для `bpfftrace`:

```
# bpfftrace -e 'tracepoint:syscalls:sys_enter_open* { @[probe] = count(); }'
Attaching 3 probes...
^C
```

```
@[tracepoint:syscalls:sys_enter_open]: 5
@[tracepoint:syscalls:sys_enter_openat]: 308
```

Повторюсь: детали кода этого однострочного сценария я объясню в главе 5. А пока вам важно понимать только вывод. Теперь сценарий выводит количество задействованных точек трассировки, а не события. Результат подтверждает, что системный вызов `openat(2)` вызывается чаще — в данном случае 308 раз против пяти вызовов `open(2)`. Эта информация вычисляется в ядре программой BPF.

Я могу добавить вторую точку трассировки в свой сценарий и трассировать сразу два системных вызова, `open(2)` и `openat(2)`. Но этот новый сценарий получится слишком громоздким для командной строки, поэтому лучше сохранить его в выполняемом файле, чтобы его было легче править с помощью текстового редактора. Это уже было сделано: `bpfftrace` поставляется со сценарием `opensnoop.bt`, который трассирует начало и конец каждого системного вызова и выводит данные в виде столбцов:

```
# opensnoop.bt
Attaching 3 probes...
Tracing open syscalls... Hit Ctrl-C to end.
PID   COMM          FD ERR PATH
2440   snmp-pass      4   0  /proc/cpuinfo
2440   snmp-pass      4   0  /proc/stat
25706  ls              3   0  /etc/ld.so.cache
25706  ls              3   0  /lib/x86_64-linux-gnu/libselinux.so.1
```

```

25706 ls          3  0 /lib/x86_64-linux-gnu/libc.so.6
25706 ls          3  0 /lib/x86_64-linux-gnu/libpcre.so.3
25706 ls          3  0 /lib/x86_64-linux-gnu/libdl.so.2
25706 ls          3  0 /lib/x86_64-linux-gnu/libpthread.so.0
25706 ls          3  0 /proc/filesystems
25706 ls          3  0 /usr/lib/locale/locale-archive
25706 ls          3  0 .
1744  snmpd         8  0 /proc/net/dev
1744  snmpd        -1  2 /sys/class/net/lo/device/vendor
2440  snmp-pass     4  0 /proc/cpuinfo
^C
#

```

В столбцах выводится следующая информация: идентификатор процесса (PID), имя команды процесса (COMM), дескриптор файла (FD), код ошибки (ERR) и путь к файлу, который системный вызов пытался открыть (PATH). Инструмент `opensnoop.bt` можно использовать для устранения неполадок в ПО, которое будет пытаться открыть файлы, используя неправильный путь, а также для определения местоположения конфигурационных файлов и журналов по событиям обращения к ним. Он также может выявить некоторые проблемы с производительностью, например слишком быстрое открытие файлов или слишком частую проверку неправильных местоположений. У этого инструмента множество применений.

`bpfttrace` поставляется с более чем 20 подобными готовыми инструментами, а ВСС — с более чем 70. Помимо помощи в непосредственном решении проблем, эти инструменты показывают код, изучив который можно понять, как трассировать разные цели. Иногда могут наблюдаться некоторые странности, как мы видели на примере трассировки системного вызова `open(2)`, и код инструментов способен подсказать их причины.

1.9. НАЗАД К ВСС: ТРАССИРОВКА OPEN()

Теперь рассмотрим ВСС-версию `opensnoop(8)`:

```

# opensnoop
PID  COMM          FD ERR PATH
2262 DNS Res~er #657  22  0 /etc/hosts
2262 DNS Res~er #654 178  0 /etc/hosts
29588 device poll    4  0 /dev/bus/usb
29588 device poll    6  0 /dev/bus/usb/004
29588 device poll    7  0 /dev/bus/usb/004/001
29588 device poll    6  0 /dev/bus/usb/003
^C
#

```

Вывод этого инструмента выглядит очень похожим на вывод более раннего однострочного сценария, по крайней мере он имеет те же столбцы. Но в выводе этого инструмента `opensnoop(8)` есть то, чего нет в `bpfttrace`-версии: его можно вызвать с разными параметрами командной строки:

opensnoop -h

порядок использования: opensnoop [-h] [-T] [-x] [-p PID] [-t TID]
 [-d DURATION] [-n NAME]
 [-e] [-f FLAG_FILTER]

Трассирует системные вызовы open()

необязательные аргументы:

-h, --help вывести эту справку и выйти
 -T, --timestamp включить отметку времени в вывод
 -x, --failed показать только неудачные вызовы open
 -p PID, --pid PID трассировать только этот PID
 -t TID, --tid TID трассировать только этот TID
 -d DURATION, --duration DURATION общая продолжительность трассировки в секундах
 -n NAME, --name NAME выводить только имена процессов, содержащие это имя
 -e, --extended_fields показать дополнительные поля
 -f FLAG_FILTER, --flag_filter FLAG_FILTER фильтровать по аргументу с флагами (например, O_WRONLY)

примеры:

```
./opensnoop                # трассировать все системные вызовы open()
./opensnoop -T            # включить отметки времени
./opensnoop -x            # показать только неудачные вызовы open
./opensnoop -p 181        # трассировать только PID 181
./opensnoop -t 123        # трассировать только TID 123
./opensnoop -d 10        # трассировать только 10 секунд
./opensnoop -n main       # выводить только имена процессов, содержащие "main"
./opensnoop -e            # показать дополнительные поля
./opensnoop -f O_WRONLY -f O_RDWR # выводить только вызовы для записи
```

Инструменты bpftrace, как правило, проще и выполняют одну конкретную задачу. Инструменты BCC, напротив, обычно сложнее и поддерживают несколько режимов работы. Конечно, можно изменить инструмент для bpftrace, чтобы он отображал только неудачные вызовы open, но уже есть BCC-версия, поддерживающая такую возможность с параметром -x:

opensnoop -x

PID	COMM	FD	ERR	PATH
991	irqbalance	-1	2	/proc/irq/133/smp_affinity
991	irqbalance	-1	2	/proc/irq/141/smp_affinity
991	irqbalance	-1	2	/proc/irq/131/smp_affinity
991	irqbalance	-1	2	/proc/irq/138/smp_affinity
991	irqbalance	-1	2	/proc/irq/18/smp_affinity
20543	systemd-resolve	-1	2	/run/systemd/netif/links/5
20543	systemd-resolve	-1	2	/run/systemd/netif/links/5
20543	systemd-resolve	-1	2	/run/systemd/netif/links/5
[...]				

Этот вывод показывает повторяющиеся сбои. Такие закономерности могут указывать на неэффективность или неправильную конфигурацию, которые можно исправить.

Инструменты ВСС часто имеют несколько параметров для изменения поведения, что делает их более универсальными, чем инструменты `bpfttrace`. Они могут послужить хорошей отправной точкой: с их помощью вы сумеете решить проблему, не написав ни строчки кода для BPF. Но если им не хватит глубины видимости, можно переключиться на `bpfttrace`, имеющий более простой язык для разработки, и создать свои инструменты.

`bpfttrace` впоследствии можно преобразовать в более сложный инструмент ВСС, поддерживающий разнообразные параметры, подобно инструменту `opensnoop(8)`, который был показан выше. Инструменты ВСС также могут поддерживать различные события: использовать точки трассировки, если они доступны, а в противном случае переключаться на `kprobes`. Но имейте в виду, что программировать инструменты ВСС намного сложнее, и эта тема выходит за рамки книги, где основное внимание уделяется программированию на `bpfttrace`. В приложении С вы найдете ускоренный курс по разработке инструментов ВСС.

1.10. ИТОГИ

Инструменты трассировки BPF могут использоваться для анализа производительности и устранения неполадок, и есть два основных проекта, которые предоставляют их: ВСС и `bpfttrace`. В этой главе мы познакомились с расширенным механизмом BPF, ВСС, `bpfttrace`, а также с динамической и статической инструментацией.

Следующая глава более подробно расскажет об этих технологиях. Если вы спешите приступить к решению проблем, можете пропустить главу 2 и перейти к главе 3 или другой, затрагивающей интересующую вас тему. В этих последующих главах широко используются термины, многие из которых объясняются в главе 2, но их описание также можно найти в глоссарии.

Глава 2

ОСНОВЫ ТЕХНОЛОГИИ

https://t.me/it_boooks/2

В главе 1 были представлены различные технологии, используемые инструментами BPF. В главе 2 они рассматриваются более подробно: история их развития, интерфейсы, внутреннее устройство и использование в комплексе с BPF.

Это самая техническая глава в книге, и для краткости изложения я предполагаю, что у вас есть некоторое представление о внутренних компонентах ядра и программировании на уровне инструкций¹.

Цель не в том, чтобы запомнить каждую страницу в этой главе, а в том, чтобы:

- познакомиться с историей происхождения BPF и современной ролью расширенного BPF;
- узнать, как выполнять обход фреймов стека и другие приемы;
- узнать, как читать флейм-графики;
- узнать, как использовать kprobes и uprobes, и познакомиться с причинами их нестабильности;
- разобраться с назначением точек трассировки, зондов USDT и динамического USDT;
- познакомиться со счетчиками мониторинга производительности (PMC) и приемами их использования с инструментами трассировки BPF;
- познакомиться с перспективными разработками: BTF и другими средствами обхода стека BPF.

Эта информация поможет освоить последующие главы, но если хотите, можете просто бегло просмотреть главу 2 и вернуться к ней позднее, когда потребуется более глубокое понимание. Глава 3 даст возможность начать практическое применение инструментов BPF для поиска и устранения узких мест производительности.

¹ Для знакомства с внутренними компонентами ядра обращайтесь к руководствам, охватывающим системные вызовы, пространства выполнения ядра и пользователя, задачи/потоки выполнения, виртуальную память и VFS, например [Gregg 13b].

2.1. BPF В ИЛЛЮСТРАЦИЯХ

На рис. 2.1 показаны многие из технологий этой главы и то, как они связаны.

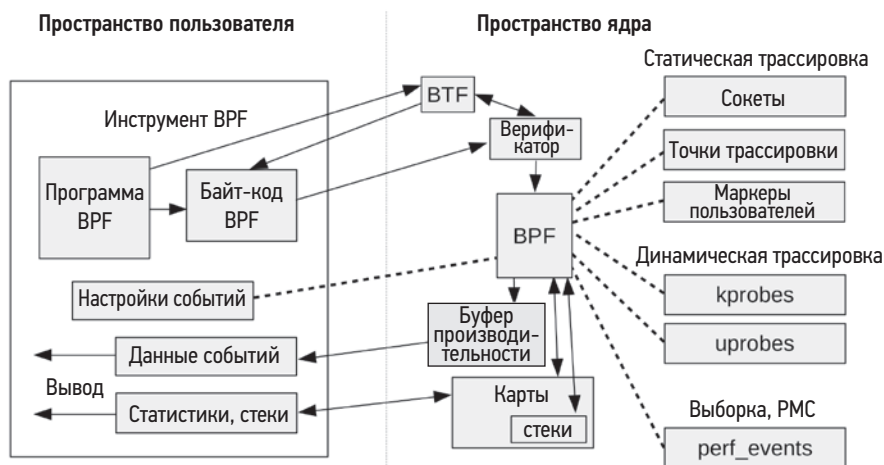


Рис. 2.1. Технологии трассировки BPF

2.2. BPF

Первоначально механизм BPF был разработан для ОС BSD и описан в 1992 году в статье «The BSD Packet Filter: A New Architecture for User-level Packet Capture» [McCanne 92]. Она была представлена на зимней конференции USENIX 1993 года в Сан-Диего вместе со статьей «Measurement, Analysis, and Improvement of UDP/IP Throughput for the DECstation 5000» [7]. Рабочие станции DEC давно ушли в прошлое, но технология BPF сохранилась как стандартное решение для фильтрации пакетов.

В BPF используется интересный принцип работы: выражение фильтра определяется конечным пользователем с помощью набора инструкций для виртуальной машины BPF (иногда их называют байт-кодом BPF) и передается ядру для выполнения интерпретатором. Это позволяет выполнять фильтрацию на уровне ядра без дорогостоящего копирования каждого пакета на уровне пользовательского процесса, что обеспечивает высокую производительность фильтрации пакетов, как, например, в `tcpdump(8)`. Это решение также обеспечивает дополнительную безопасность, так как фильтры из пространства пользователя можно проверить на безопасность перед выполнением. Учитывая, что фильтрация пакетов происходит в пространстве ядра, требования к безопасности были очень жесткими. На рис. 2.2 показано, как это работает.

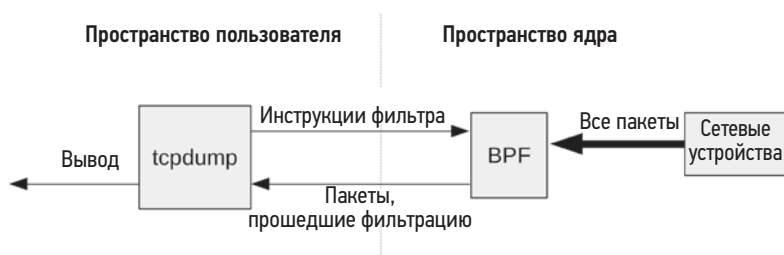


Рис. 2.2. tcpdump и BPF

При желании можно запустить команду `tcpdump(8)` с параметром `-d`, чтобы получить инструкции BPF, выражающие фильтр. Например:

```
# tcpdump -d host 127.0.0.1 and port 80
(000) ldh      [12]
(001) jeq      #0x800          jt 2      jf 18
(002) ld       [26]
(003) jeq      #0x7f000001     jt 6      jf 4
(004) ld       [30]
(005) jeq      #0x7f000001     jt 6      jf 18
(006) ldb      [23]
(007) jeq      #0x84          jt 10     jf 8
(008) jeq      #0x6           jt 10     jf 9
(009) jeq      #0x11          jt 10     jf 18
(010) ldh      [20]
(011) jset     #0x1fff        jt 18     jf 12
(012) ldx      4*([14]&0xf)
(013) ldh      [x + 14]
(014) jeq      #0x50          jt 17     jf 15
(015) ldh      [x + 16]
(016) jeq      #0x50          jt 17     jf 18
(017) ret      #262144
(018) ret      #0
```

Оригинальный механизм BPF, который сейчас называют «классическим BPF», представлял собой ограниченную виртуальную машину. Она имела два регистра, внутреннее хранилище с 16 ячейками памяти и счетчик программных инструкций. Все они работали с 32-битными регистрами¹. В Linux классический BPF появился в 1997 году, в ядре 2.1.75 [8].

После включения BPF в ядро Linux были реализованы некоторые важные улучшения. Эрик Думазет (Eric Dumazet) добавил в ядро Linux 3.0, вышедшее в июле 2011 года [9], динамический (JIT) компилятор BPF, имеющий более высокую производительность по сравнению с интерпретатором. В 2012 году Уилл Дрюри

¹ В 64-битном ядре классический BPF использует 64-битные адреса, но 32-битные данные, а нагрузки скрыты за некоторыми внешними вспомогательными функциями ядра.

(Will Drewry) добавил фильтры BPF для политик безопасности системных вызовов `seccomp` [10]. Это было первое использование BPF за рамками сетевого стека, показавшее потенциал применения BPF в роли универсального механизма выполнения.

2.3. РАСШИРЕННЫЙ BPF (EBPF)

Расширенная версия BPF был спроектирована Алексеем Старовойтовым, тогда работавшим в компании PLUMgrid и изучавшим новые способы создания программно-определяемых сетевых решений. Она могла бы стать первым серьезным усовершенствованием BPF за последние 20 лет и подняла бы BPF до уровня виртуальной машины общего назначения¹. Когда расширенный BPF находился еще на стадии предложения, Дэниел Боркман, специалист по ядру из Red Hat, помог переработать его для включения в ядро в качестве замены имеющегося BPF². Эта расширенная версия BPF была успешно включена в ядро, и с тех пор в ее развитии участвовали многие другие разработчики (см. раздел «Благодарности»).

В расширенный BPF было добавлено больше регистров, вместо 32-разрядных слов стали использоваться 64-разрядные, создано гибкое хранилище «карт» и разрешены вызовы некоторых ограниченных функций ядра³. Он также включает JIT-компиляцию с отображением «один к одному» в машинные инструкции и регистры, что позволяет повторно использовать методы оптимизации команд, реализованные ранее для BPF. Верификатор BPF тоже обновился и теперь поддерживает обработку этих расширений и отклоняет любой небезопасный код.

Основные различия между классическим и расширенным BPF перечислены в табл. 2.1.

¹ BPF часто называют виртуальной машиной, но такое название лишь отражение спецификации. Реализация этого механизма в Linux (среда выполнения) включает интерпретатор и JIT-компилятор в машинный код. Термин «виртуальная машина» предполагает наличие дополнительного слоя выполнения поверх процессора, но на самом деле его нет. Инструкции, генерируемые JIT-компилятором, выполняются непосредственно на процессоре, как любой другой код ядра. Обратите внимание, что после обнаружения уязвимости Spectre некоторые дистрибутивы безоговорочно включают JIT-компиляцию для архитектуры x86, что полностью исключает интерпретатор из работы (по мере компиляции).

² Алексей и Дэниел с тех пор сменили место работы. Сейчас они по-прежнему мейнтейнеры реализации BPF в ядре: направляют развитие, проверяют патчи и принимают решения о включении тех или иных возможностей.

³ Без необходимости перегрузки инструкций, как это было в классическом BPF, где этот подход отличался излишней сложностью, выражающейся в необходимости изменять JIT-компилятор для поддержки каждой из них.

Таблица 2.1. Основные различия между классическим и расширенным BPF

Фактор	Классический BPF	Расширенный BPF
Количество регистров	2: A, X	10: R0–R9 и R10, как указатель на список фреймов стека, доступный только для чтения
Разрядность регистров	32 бита	64 бита
Хранилище	16 ячеек памяти: M[0–15]	512 байт в пространстве стека плюс неограниченное хранилище «карт»
Ограниченные вызовы ядра	Очень ограниченный круг, определяемый JIT-компилятором	Да, через инструкцию <code>bpf_call</code>
Целевые события	Пакеты, <code>seccomp-BPF</code>	Пакеты, функции в пространстве ядра, функции в пространстве пользователя, точки трассировки, пользовательские маркеры, счетчики производительности (PMC)

Первоначальное предложение Алексея, отправленное в сентябре 2013 года, было набором патчей «extended BPF» [2]. А в декабре 2013 года Алексей уже предложил использовать его для трассировки фильтров [11]. После обсуждения и разработки совместно с Дэниелем в марте 2014 года [3] [12] патчи начали включаться в ядро Linux¹. Компоненты JIT-компилятора были включены в Linux 3.15 в июне 2014 года, а системный вызов `bpf(2)` для управления механизмом BPF был добавлен в Linux 3.18 в декабре 2014 года [13]. Позднее в ветку Linux 4.x была добавлена поддержка BPF для `kprobes`, `uprobes`, точек трассировки и `perf_events`.

В самых ранних наборах патчей эта технология сокращенно обозначалась как eBPF, но позже Алексей перешел к использованию более простого названия BPF². Все сообщество разработчиков BPF теперь называют ее в списке рассылки `net-dev` [14] просто BPF.

На рис. 2.3 показана архитектура среды выполнения BPF в Linux, где видно, как инструкции BPF проходят проверку в верификаторе BPF перед выполнением

¹ До предоставления доступа через системный вызов `bpf(2)` его также называли «внутренним BPF». Поскольку первоначально BPF относился к категории сетевых технологий, патчи были отправлены и приняты мейнтейнером сетевой подсистемы Дэвидом С. Миллером (David S. Miller). Сейчас BPF превратился в самостоятельный компонент ядра со своим сообществом, и все патчи, связанные с ним, добавляются в собственные деревья ядра `bpftool` и `bpftool-next`. Но по традиции все пулл-реквесты в дерево BPF все еще принимаются Дэвидом С. Миллером.

² Я предложил Алексею придумать другое, более удачное название. Но это оказалось не просто, и мы — инженеры — застряли на: «eBPF, а на самом деле просто BPF, что означает Berkeley Packet Filter, хотя сейчас эта технология не имеет ничего общего с Berkeley, пакетами или фильтрацией». Теперь название «BPF» следует рассматривать как название технологии, а не как аббревиатуру.

виртуальной машины BPF. Реализация виртуальной машины BPF включает как интерпретатор, так и JIT-компилятор: JIT-компилятор генерирует машинные инструкции для выполнения непосредственно на процессоре. Верификатор отклоняет небезопасные операции, включая неограниченные циклы: программы BPF должны завершаться в ограниченное время.

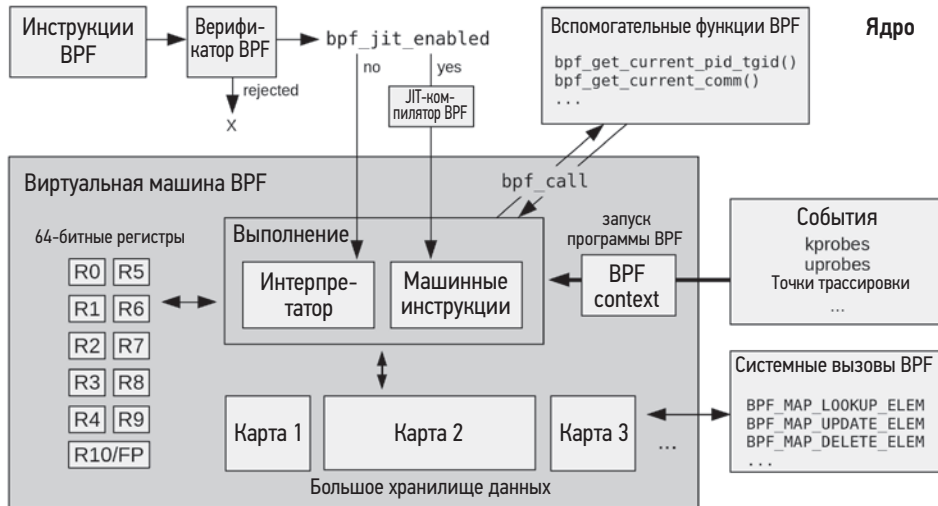


Рис. 2.3. Архитектура среды выполнения BPF

BPF может использовать вспомогательные функции для определения состояния ядра и карт BPF для хранения. Программа BPF запускается по событиям, включая события kprobes, uprobes и точки трассировки.

В разделах ниже обсуждается, зачем инструментам оценки производительности нужен механизм BPF и как писать программы для расширенного BPF, а также дается обзор инструкций BPF, BPF API, ограничений BPF и BTF. Эти разделы закладывают основы понимания особенностей работы BPF при использовании bpftrace и BCC. В приложении D рассказано о программировании BPF на языке C, а в приложении E описаны инструкции BPF.

2.3.1. Зачем инструментам оценки производительности нужен BPF

Инструменты оценки производительности используют возможность программирования расширенного BPF. Программы для BPF могут вычислять задержки и статистические характеристики. Эти возможности сами по себе интересны, хотя есть множество других инструментов трассировки с такими же возможностями. Отличительная черта BPF — это эффективность и безопасность для промышленного

использования, а также его включение в ядро Linux. С помощью BPF эти инструменты можно запускать в промышленной среде без добавления любых новых компонентов ядра.

Рассмотрим некоторые результаты и диаграмму, чтобы понять, как инструменты оценки производительности используют BPF. Пример ниже получен с помощью одного из ранних инструментов BPF, который я опубликовал под названием `bitehist`. Он сообщает объем дискового ввода/вывода в виде гистограммы [15]:

```
# bitehist
```

```
Tracing block device I/O... Interval 5 secs. Ctrl-C to end.
```

kbytes	:	count	distribution
0 -> 1	:	3	
2 -> 3	:	0	
4 -> 7	:	3395	*****
8 -> 15	:	1	
16 -> 31	:	2	
32 -> 63	:	738	*****
64 -> 127	:	3	
128 -> 255	:	1	

На рис. 2.4 показано, как BPF улучшает эффективность этого инструмента.

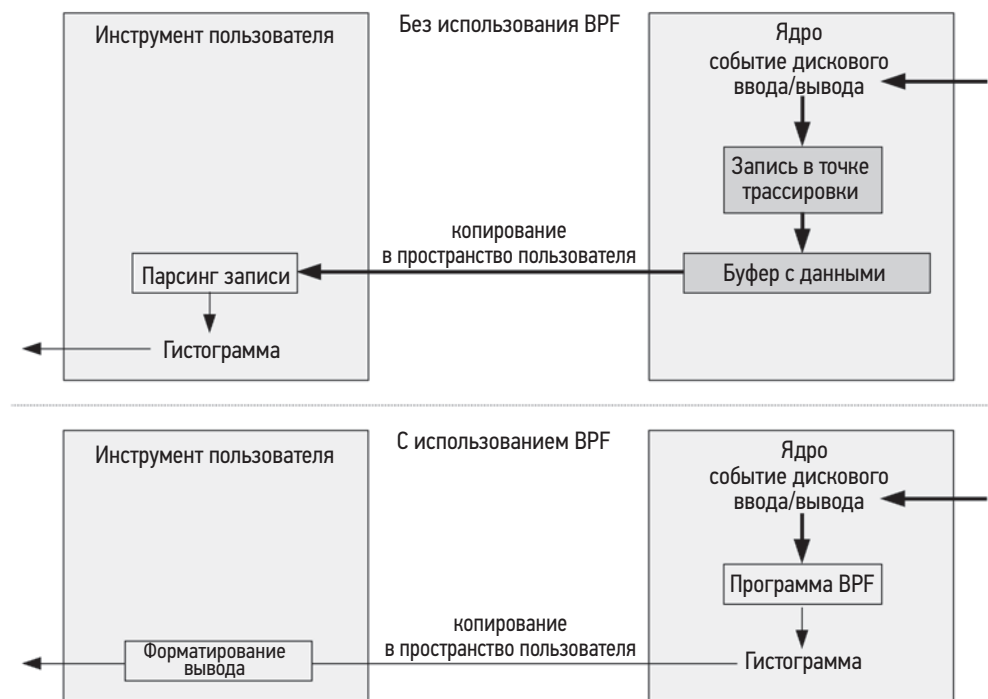


Рис. 2.4. Создание гистограммы до и после появления BPF

Главное отличие — при использовании BPF гистограмма генерируется в контексте ядра, что значительно уменьшает объем данных, копируемых в пространство пользователя. Этот выигрыш в эффективности настолько значителен, что инструменты можно использовать в промышленной среде (обычно это дорогостоящее мероприятие). Рассмотрим это более подробно.

Без использования BPF для получения гистограммы требуется¹:

1. В ядре: включить перехват событий дискового ввода/вывода.
2. В ядре для каждого события: сохранить в буфере данных запись с информацией о событии. Если используются точки трассировки (что желательно), запись содержит несколько полей метаданных о дисковом вводе/выводе.
3. В пространстве пользователя: периодически копировать буфер со всеми событиями в пространство пользователя.
4. В пространстве пользователя: выполнить обход всех событий в буфере, извлечь из метаданных события значение поля с числом байтов. Остальные поля игнорировать.
5. В пространстве пользователя: сгенерировать гистограмму, опираясь на значения в поле с числом байтов.

Шаги со 2-го по 4-й имеют высокий оверхед в системах с большим объемом ввода/вывода. Представьте, что каждую секунду в пространство пользователя передаются для анализа и обобщения 10 000 записей с данными трассировки дискового ввода/вывода.

С использованием BPF программа `bitesize` должна:

1. В ядре: включить перехват событий дискового ввода/вывода и установить программу для BPF, определяемую программой `bitesize`.
2. В ядре для каждого события: запустить программу для BPF, которая извлекает только поле с числом байтов и сохраняет его в карте BPF для гистограммы.
3. В пространстве пользователя: прочитать карту BPF для гистограммы один раз и вывести ее.

Этот метод избавляет от оверхедов на копирование событий в пространство пользователя и их повторную обработку. Он также позволяет избежать копирования неиспользуемых полей метаданных. Единственные данные из вывода инструмента,

¹ Это оптимальная последовательность шагов, но не единственно возможная. Можно установить трассировщик из библиотеки `out-of-tree`, например `SystemTap`, но в зависимости от версии ядра и дистрибутива это может оказаться непростой задачей. Также можно изменить код ядра или разработать свой модуль `kprobe`, но оба этих метода сложны и несут свои риски. Я разработал собственный обходной путь, который назвал «хактограмма» (`hacktogram`), и использовал для создания нескольких счетчиков `perf(1)` с фильтрами диапазонов для каждого столбика в гистограмме [16]. Это было ужасно.

которые копируются в пространство пользователя, — это столбец «count», являющийся массивом чисел.

2.3.2. BPF и модули ядра

Другой способ оценить преимущества механизма BPF — сравнить его с модулями ядра. Точки трассировки и kprobes существуют уже довольно давно, и их можно использовать непосредственно из загружаемых модулей ядра. Вот какие преимущества дает использование BPF перед модулями ядра:

- Программы для BPF проверяются с помощью верификатора: модули ядра могут содержать ошибки (способные вызвать крах ядра) или уязвимости безопасности.
- BPF поддерживает богатый набор структур посредством карт.
- Программы для BPF можно скомпилировать один раз и выполнять где угодно, потому что набор команд BPF, карты, вспомогательные функции и инфраструктура имеют стабильный ABI. (Но это не относится к отдельным программам трассировки для BPF, которые используют нестабильные компоненты, например kprobes, для инструментации структур ядра. Решение для этого случая приводится в разделе 2.3.10.)
- Программы для BPF не требуют компиляции модулей ядра.
- Программирование для BPF осваивается легче, чем программирование модулей ядра, что делает его доступным для большего числа людей.

Обратите внимание, что использование BPF для оценки работы сетевых сервисов дает дополнительные преимущества, в том числе возможность атомарной замены программ для BPF. Модуль ядра сначала нужно полностью выгрузить, а затем загрузить новую версию, что может вызвать сбой в работе сервиса.

Преимущество модулей ядра в том, что они могут задействовать другие функции и возможности ядра, не ограничиваясь узким кругом вспомогательных функций BPF. Однако это создает дополнительный риск появления ошибок при неправильном использовании функций ядра.

2.3.3. Разработка программ для BPF

BPF предлагает множество интерфейсов, доступных для программирования. Вот основные языки для трассировки, от самого низкого до самого высокого уровня:

- LLVM
- BCC
- bpftrace

Компилятор LLVM поддерживает BPF как одну из целей компиляции. Программы для BPF можно писать на языках высокого уровня, поддерживаемых компилятором

LLVM, например на C (через Clang) или LLVM Intermediate Presentation (IR), а затем компилировать в инструкции BPF. LLVM включает оптимизатор, улучшающий эффективность и размер генерируемых инструкций BPF.

Разработка для BPF на LLVM IR — несомненное улучшение, но переход на BCC или bpftrace — еще лучше. BCC позволяет писать программы для BPF на языке C, а bpftrace предлагает свой язык высокого уровня. Внутренне они используют LLVM IR и библиотеку LLVM для компиляции в инструкции BPF.

Инструменты оценки производительности, представленные в этой книге, запрограммированы на BCC и bpftrace. Программирование непосредственно в инструкциях BPF или LLVM IR больше подходит для разработчиков, занимающихся созданием внутренних компонентов BCC и bpftrace, и обсуждение этой темы выходит за рамки книги. Это умение не требуется тем из нас, кто использует и разрабатывает инструменты оценки производительности для BPF¹. Если вы хотите освоить программирование инструкций BPF или просто удовлетворить свое любопытство, вот несколько ресурсов для дополнительного чтения:

- В приложении E приводится краткое описание инструкций и макросов BPF.
- Описание инструкций BPF можно найти в дереве исходного кода ядра Linux, в файле `Documentation/networking/filter.txt` [17].
- Описание LLVM IR есть в онлайн-документации LLVM. Начните с описания класса `llvm::IRBuilderBase` [18].
- Справочное руководство по BPF и XDP в документации к подсистеме Cilium [19].

Большинство из нас никогда не будет программировать с помощью инструкций BPF, но многие будут время от времени их встречать, например, столкнувшись с проблемами в инструментах. В следующих двух разделах я приведу примеры использования `bpftool(8)` и `bpftrace`.

2.3.4. Обзор инструкций BPF: `bpftool`

Интерфейс `bpftool(8)` был добавлен в Linux 4.15 для просмотра и управления объектами BPF, включая программы и карты. Его реализация находится в исходном коде Linux в `tools/bpf/bpftool`. В этом разделе я расскажу, как использовать `bpftool(8)` для поиска загруженных BPF-программ и вывода их содержимого.

bpftool

По умолчанию `bpftool(8)` выводит типы объектов, с которыми работает. Для Linux 5.2:

¹ Имея 15-летний опыт использования DTrace, не могу вспомнить, чтобы когда-нибудь кому-нибудь потребовалось напрямую писать программы с использованием промежуточного формата D (D Intermediate Format, DIF) (эквивалент BPF-инструкций в DTrace).

```
# bpftool
Usage: bpftool [OPTIONS] OBJECT { COMMAND | help }
       bpftool batch file FILE
       bpftool version

OBJECT := { prog | map | cgroupp | perf | net | feature | btf }
OPTIONS := { {-j|--json} [{-p|--pretty}] | {-f|--bpfds} |
            {-m|--mapcompat} | {-n|--nomount} }
```

Для каждого объекта есть своя справочная страница. Например, для программ:

```
# bpftool prog help
Usage: bpftool prog { show | list } [PROG]
       bpftool prog dump xlated PROG [{ file FILE | opcodes | visual | linum }]
       bpftool prog dump jited PROG [{ file FILE | opcodes | linum }]
       bpftool prog pin PROG FILE
       bpftool prog { load | loadall } OBJ PATH \
                   [type TYPE] [dev NAME] \
                   [map { idx IDX | name NAME } MAP] \
                   [pinmaps MAP_DIR]
       bpftool prog attach PROG ATTACH_TYPE [MAP]
       bpftool prog detach PROG ATTACH_TYPE [MAP]
       bpftool prog tracelog
       bpftool prog help
MAP := { id MAP_ID | pinned FILE }
PROG := { id PROG_ID | pinned FILE | tag PROG_TAG }
TYPE := { socket | kprobe | kretprobe | classifier | action | q
[...]
```

Подкоманды **perf** и **prog** можно использовать для поиска и вывода программ трассировки. К возможностям **bpftool(8)**, которые здесь не рассматриваются, относятся: добавление программ, чтение и запись карт, работа с **sgroups** и получение списка возможностей **BPF**.

bpftool perf

Подкоманда **perf** выводит список программ **BPF**, подключенных через **perf_event_open()**, и это обычное дело для программ **BCC** и **bpfttrace** в версиях **Linux 4.17** и выше. Например:

```
# bpftool perf
pid 1765 fd 6: prog_id 26 kprobe func blk_account_io_start offset 0
pid 1765 fd 8: prog_id 27 kprobe func blk_account_io_done offset 0
pid 1765 fd 11: prog_id 28 kprobe func sched_fork offset 0
pid 1765 fd 15: prog_id 29 kprobe func ttwu_do_wakeup offset 0
pid 1765 fd 17: prog_id 30 kprobe func wake_up_new_task offset 0
pid 1765 fd 19: prog_id 31 kprobe func finish_task_switch offset 0
pid 1765 fd 26: prog_id 33 tracepoint inet_sock_set_state
pid 21993 fd 6: prog_id 232 uprobe filename /proc/self/exe offset 1781927
pid 21993 fd 8: prog_id 233 uprobe filename /proc/self/exe offset 1781920
pid 21993 fd 15: prog_id 234 kprobe func blk_account_io_done offset 0
pid 21993 fd 17: prog_id 235 kprobe func blk_account_io_start offset 0
pid 25440 fd 8: prog_id 262 kprobe func blk_mq_start_request offset 0
pid 25440 fd 10: prog_id 263 kprobe func blk_account_io_done offset 0
```

В этом списке видно три разных PID с разными программами BPF:

- PID 1765 — идентификатор агента Vector BPF PMDA для анализа экземпляра (подробности см. в главе 17).
- PID 21993 — версия biolateness(8), реализованная на основе bpftrace. Она выполняет две проверки uprobes: начало и конец программы bpftrace, а также две проверки kprobes: начала и конца блочного ввода/вывода (исходный код этой программы приводится в главе 9).
- PID 25440 — версия biolateness(8), реализованная на основе BCC(8), которая сейчас использует другую проверку начала блочного ввода/вывода.

Поле `offset` показывает смещение точки инструментации от начала объекта. В bpftrace смещение 1781920 соответствует функции `BEGIN_trigger` в двоичном файле bpftrace, а смещение 1781927 соответствует функции `END_trigger` (в чем можно убедиться, выполнив команду `readelf -s bpftrace`).

В поле `prog_id` выводятся числовые идентификаторы программ BPF, которые можно вывести с помощью следующих подкоманд.

bpftrace prog show

Подкоманда `prog show` выводит список всех программ (не только тех, что основаны на `perf_event_open()`):

```
# bpftrace prog show
[...]
232: kprobe name END tag b7cc714c79700b37 gpl
      loaded_at 2019-06-18T21:29:26+0000 uid 0
      xlated 168B jited 138B memlock 4096B map_ids 130
233: kprobe name BEGIN tag 7de8b38ee40a4762 gpl
      loaded_at 2019-06-18T21:29:26+0000 uid 0
      xlated 120B jited 112B memlock 4096B map_ids 130
234: kprobe name blk_account_io_ tag d89dcf82fc3e48d8 gpl
      loaded_at 2019-06-18T21:29:26+0000 uid 0
      xlated 848B jited 540B memlock 4096B map_ids 128,129
235: kprobe name blk_account_io_ tag 499ff93d9cfff0eb2 gpl
      loaded_at 2019-06-18T21:29:26+0000 uid 0
      xlated 176B jited 139B memlock 4096B map_ids 128
[...]
258: cgroup_skb tag 7be49e3934a125ba gpl
      loaded_at 2019-06-18T21:31:27+0000 uid 0
      xlated 296B jited 229B memlock 4096B map_ids 153,154
259: cgroup_skb tag 2a142ef67aad174 gpl
      loaded_at 2019-06-18T21:31:27+0000 uid 0
      xlated 296B jited 229B memlock 4096B map_ids 153,154
262: kprobe name trace_req_start tag 1dfc28ba8b3dd597 gpl
      loaded_at 2019-06-18T21:37:51+0000 uid 0
      xlated 112B jited 109B memlock 4096B map_ids 158
      btf_id 5
263: kprobe name trace_req_done tag d9bc05b87ea5498c gpl
```

```
loaded_at 2019-06-18T21:37:51+0000 uid 0
xlated 912B jited 567B memlock 4096B map_ids 158,157
btf_id 5
```

В этом списке есть числовые идентификаторы программ bpftrace (с 232 по 235) и программ BCC (262 и 263), а также другие загруженные программы BPF. Обратите внимание, что для программ BCC kprobe выводится информация о формате BPF (BPF Type Format, BTF) в поле `btf_id` в этих выходных данных. Более подробно о BTF рассказывается в разделе 2.3.9, а пока просто имейте в виду, что BTF — это аналог отладочной информации для BPF.

bpf tool prog dump xlated

По числовому идентификатору можно получить исходный код любой программы BPF. В режиме `xlate` выводятся ассемблерные инструкции BPF. Вот код программы с идентификатором 234, это программа bpftrace, отслеживающая завершение блочного ввода/вывода¹:

```
# bpf tool prog dump xlated id 234
0: (bf) r6 = r1
1: (07) r6 += 112
2: (bf) r1 = r10
3: (07) r1 += -8
4: (b7) r2 = 8
5: (bf) r3 = r6
6: (85) call bpf_probe_read#-51584
7: (79) r1 = *(u64 *) (r10 -8)
8: (7b) *(u64 *) (r10 -16) = r1
9: (18) r1 = map[id:128]
11: (bf) r2 = r10
12: (07) r2 += -16
13: (85) call __htab_map_lookup_elem#93808
14: (15) if r0 == 0x0 goto pc+1
15: (07) r0 += 56
16: (55) if r0 != 0x0 goto pc+2
[...]
```

Вывод показывает вызов одной из ограниченных вспомогательных функций ядра, доступных в BPF: `bpf_probe_read()`. (Другие вспомогательные функции перечислены в табл. 2.2.)

Теперь сравните этот код с кодом программы BCC с идентификатором 263, отслеживающей завершение блочного ввода/вывода, которая была скомпилирована с BTF²:

```
# bpf tool prog dump xlated id 263
int trace_req_done(struct pt_regs * ctx):
; struct request *req = ctx->di;
```

¹ Этот код может не совпадать с тем, который пользователь загрузил в ядро, так как верификатор BPF может подменять некоторые инструкции для оптимизации (например, встраивая код поиска по карте) или из соображений безопасности (например, Spectre).

² Чтобы получить его, нужна версия LLVM 9.0, включающая поддержку BTF по умолчанию.

```

    0: (79) r1 = *(u64 *)(r1 +112)
; struct request *req = ctx->di;
    1: (7b) *(u64 *)(r10 -8) = r1
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
    2: (18) r1 = map[id:158]
    4: (bf) r2 = r10
;
    5: (07) r2 += -8
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
    6: (85) call __htab_map_lookup_elem#93808
    7: (15) if r0 == 0x0 goto pc+1
    8: (07) r0 += 56
    9: (bf) r6 = r0
; if (tsp == 0) {
    10: (15) if r6 == 0x0 goto pc+101
; delta = bpf_ktime_get_ns() - *tsp;
    11: (85) call bpf_ktime_get_ns#88176
; delta = bpf_ktime_get_ns() - *tsp;
    12: (79) r1 = *(u64 *)(r6 +0)
[...]
```

Этот вывод включает информацию об исходном коде (выделена жирным) из BTF. Обратите внимание, что это другая программа (содержит другие инструкции и вызовы).

Модификатор `linum` добавляет информацию об исходном файле и номера строк, в том числе из BTF, если она доступна (выделена жирным):

```

# bpftool prog dump xlated id 263 linum
int trace_req_done(struct pt_regs * ctx):
; struct request *req = ctx->di; [file:/virtual/main.c line_num:42 line_col:29]
    0: (79) r1 = *(u64 *)(r1 +112)
; struct request *req = ctx->di; [file:/virtual/main.c line_num:42 line_col:18]
    1: (7b) *(u64 *)(r10 -8) = r1
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
[file:/virtual/main.c line_num:46 line_col:39]
    2: (18) r1 = map[id:158]
    4: (bf) r2 = r10
[...]
```

Здесь информация о номерах строк относится к виртуальным файлам, которые BCC создает при запуске программ.

Модификатор `opcodes` добавляет коды инструкций BPF (выделены жирным):

```

# bpftool prog dump xlated id 263 opcodes
int trace_req_done(struct pt_regs * ctx):
; struct request *req = ctx->di;
    0: (79) r1 = *(u64 *)(r1 +112)
      79 11 70 00 00 00 00 00
; struct request *req = ctx->di;
    1: (7b) *(u64 *)(r10 -8) = r1
      7b 1a f8 ff 00 00 00 00
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
    2: (18) r1 = map[id:158]
```

```

18 11 00 00 9e 00 00 00 00 00 00 00 00 00 00
4: (bf) r2 = r10
bf a2 00 00 00 00 00 00
[...]
```

Коды инструкций BPF описаны в приложении Е.

Есть и модификатор `visual`, который выводит граф потока управления в формате DOT для отображения с помощью внешнего ПО, например GraphViz и его инструмента отображения ориентированных графов `dot(1)` [20]:

```

# bpf2tool prog dump xlated id 263 visual > biolateness_done.dot
$ dot -Tpng -Elen=2.5 biolateness_done.dot -o biolateness_done.png
```

После этого можно заглянуть в файл PNG, чтобы посмотреть порядок выполнения инструкций. GraphViz предоставляет различные инструменты: для отображения данных в формате DOT я обычно использую `dot(1)`, `neato(1)`, `fdp(1)` и `sfdp(1)`. Эти инструменты поддерживают различные настройки (например, параметр `-Elen`, определяющий длину ребра). На рис. 2.5 показан результат использования `osage(1)` из GraphViz для визуализации потока выполнения этой программы BPF.

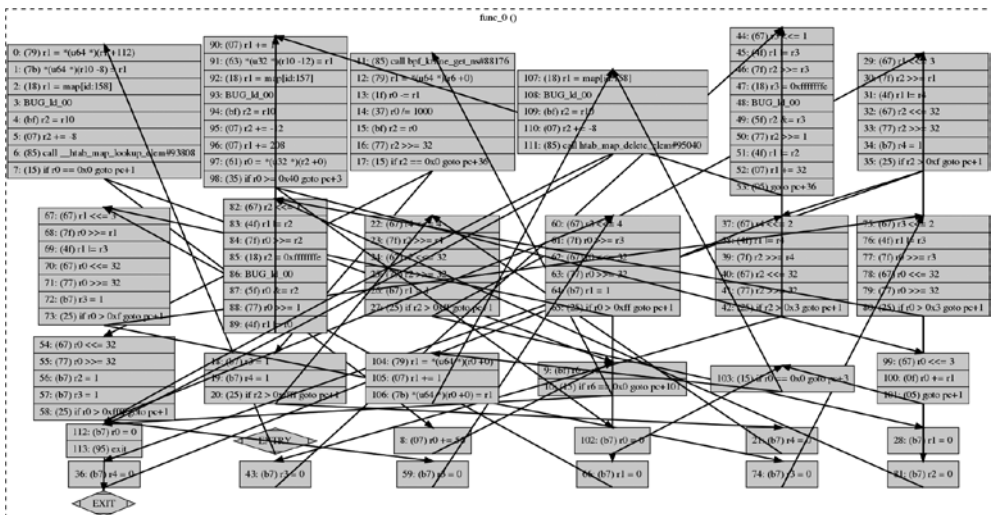


Рис. 2.5. Диаграмма потока выполнения инструкций BPF, полученная с помощью GraphViz `osage(1)`

Как видите, программа довольно сложная. Другие инструменты GraphViz размещают блоки кода так, чтобы не допустить объединения стрелок в пучки, но создают файлы гораздо большего размера. Если вам понадобится исследовать инструкции подобной программы BPF, поэкспериментируйте с разными инструментами и выберите тот, который подойдет лучше всего.

bpftool prog dump jited

Подкоманда `prog dump jited` выводит машинный код, выполняемый процессором. В этом разделе показан код для архитектуры `x86_64`, но в BPF есть JIT-компиляторы для всех основных архитектур, поддерживаемых ядром Linux. Для программы BCC, отслеживающей завершение блочного ввода/вывода:

```
# bpftool prog dump jited id 263
int trace_req_done(struct pt_regs * ctx):
0xfffffffffc082dc6f:
; struct request *req = ctx->di;
    0: push    %rbp
    1: mov     %rsp,%rbp
    4: sub     $0x38,%rsp
    b: sub     $0x28,%rbp
    f: mov     %rbx,0x0(%rbp)
   13: mov     %r13,0x8(%rbp)
   17: mov     %r14,0x10(%rbp)
   1b: mov     %r15,0x18(%rbp)
   1f: xor     %eax,%eax
   21: mov     %rax,0x20(%rbp)
   25: mov     0x70(%rdi),%rdi
; struct request *req = ctx->di;
   29: mov     %rdi,-0x8(%rbp)
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
   2d: movabs  $0xfffff96e680ab0000,%rdi
   37: mov     %rbp,%rsi
   3a: add     $0xfffffffffffffffff8,%rsi
; tsp = bpf_map_lookup_elem((void *)bpf_pseudo_fd(1, -1), &req);
   3e: callq   0xfffffffffc39a49c1
[...]
```

Как отмечалось выше, наличие BTF для этой программы позволяет команде `bpftool(8)` включить строки исходного кода — иначе бы их не было.

bpftool btf

`bpftool(8)` также может выводить числовые идентификаторы BTF. Например, вот BTF ID 5 для программы BCC, отслеживающей завершение блочного ввода/вывода:

```
# bpftool btf dump id 5
[1] PTR '(anon)' type_id=0
[2] TYPEDEF 'u64' type_id=3
[3] TYPEDEF '__u64' type_id=4
[4] INT 'long long unsigned int' size=8 bits_offset=0 nr_bits=64 encoding=(none)
[5] FUNC_PROTO '(anon)' ret_type_id=2 vlen=4
    'pkt' type_id=1
    'off' type_id=2
    'bofs' type_id=2
    'bsz' type_id=2
[6] FUNC 'bpf_dext_pkt' type_id=5
[7] FUNC_PROTO '(anon)' ret_type_id=0 vlen=5
    'pkt' type_id=1
```

```

        'off' type_id=2
        'bofs' type_id=2
        'bsz' type_id=2
        'val' type_id=2
[8] FUNC 'bpf_dins_pkt' type_id=7
[9] TYPEDEF 'uintptr_t' type_id=10
[10] INT 'long unsigned int' size=8 bits_offset=0 nr_bits=64 encoding=(none)
[...]
[347] STRUCT 'task_struct' size=9152 vlen=204
        'thread_info' type_id=348 bits_offset=0
        'state' type_id=349 bits_offset=128
        'stack' type_id=1 bits_offset=192
        'usage' type_id=350 bits_offset=256
        'flags' type_id=28 bits_offset=288
[...]

```

Этот пример показывает, что BTF включает информацию о типах и структурах.

2.3.5. Обзор инструкций BPF: bpftrace

В отличие от команды `tcpdump(8)`, которая выводит инструкции BPF при вызове с ключом `-d`, `bpftrace` выводит инструкции, если вызывается с ключом `-v`¹:

```

# bpftrace -v biolatency.bt
Attaching 4 probes...
Program ID: 677
Bytecode:
0: (bf) r6 = r1
1: (b7) r1 = 29810
2: (6b) *(u16 *) (r10 -4) = r1
3: (b7) r1 = 1635021632
4: (63) *(u32 *) (r10 -8) = r1
5: (b7) r1 = 20002
6: (7b) *(u64 *) (r10 -16) = r1
7: (b7) r1 = 0
8: (73) *(u8 *) (r10 -2) = r1
9: (18) r7 = 0xfffff96e697298800
11: (85) call bpf_get_smp_processor_id#8
12: (bf) r4 = r10
13: (07) r4 += -16
14: (bf) r1 = r6
15: (bf) r2 = r7
16: (bf) r3 = r0
17: (b7) r5 = 15
18: (85) call bpf_perf_event_output#25
19: (b7) r0 = 0
20: (95) exit
[...]

```

¹ Я заметил только сейчас, что для единообразия должен был бы использовать для этого ключ `-d`.

Такой вывод можно получить, даже если программа столкнется с ошибкой внутри bpftrace. Занимаясь разработкой внутренних компонентов bpftrace, легко столкнуться с ситуацией, когда программа отвергается верификатором BPF и не запускается. В таком случае можно распечатать инструкции и изучить их, чтобы определить и устранить проблему.

Большинство людей никогда не столкнется с внутренними ошибками bpftrace или ВСС и не будет изучать инструкции BPF. Если у вас возникла такая проблема, отправьте тикет в проект bpftrace или ВСС или, если сумеете, исправьте ее самостоятельно.

2.3.6. BPF API

Для лучшего понимания возможностей BPF в следующих разделах описаны отдельные части расширенного BPF API из файла `include/uapi/linux/bpf.h` в Linux 4.20.

Вспомогательные функции BPF

Программы BPF не могут вызывать произвольные функции ядра. Для решения определенных задач предусмотрен ограниченный круг «вспомогательных» функций, которые BPF может вызвать. Некоторые из этих функций перечислены в табл. 2.2.

Таблица 2.2. Некоторые вспомогательные функции BPF

Вспомогательная функция BPF	Описание
<code>bpf_map_lookup_elem(map, key)</code>	Выполняет поиск ключа <code>key</code> в карте <code>map</code> и возвращает его значение (указатель)
<code>bpf_map_update_elem(map, key, value, flags)</code>	Записывает значение <code>value</code> в элемент с ключом <code>key</code>
<code>bpf_map_delete_elem(map, key)</code>	Удаляет элемент с ключом <code>key</code> из карты <code>map</code>
<code>bpf_probe_read(dst, size, src)</code>	Безопасно читает <code>size</code> байт из адреса <code>src</code> и копирует их в <code>dst</code>
<code>bpf_ktime_get_ns()</code>	Возвращает время в наносекундах, прошедших с момента загрузки
<code>bpf_trace_printk(fmt, fmt_size, ...)</code>	Вспомогательная функция для отладки. Выводит в <code>TraceFS trace{ _pipe }</code>
<code>bpf_get_current_pid_tgid()</code>	Возвращает значение — 64-битное целое без знака, содержащее текущий TGID (в пространстве пользователя называется PID) в старшем 32-битном слове и PID (в пространстве пользователя называется идентификатором потока ядра) в младшем слове
<code>bpf_get_current_comm(buf, buf_size)</code>	Копирует имя задачи в буфер <code>buf</code>
<code>bpf_perf_event_output(ctx, map, data, size)</code>	Записывает данные <code>data</code> в кольцевой буфер <code>perf_event</code> ; используется для регистрации отдельных событий

Таблица 2.2 (окончание)

Вспомогательная функция BPF	Описание
<code>bpf_get_stackid(ctx, map, flags)</code>	Извлекает трассировку стека в пространстве пользователя или ядра и возвращает идентификатор
<code>bpf_get_current_task()</code>	Возвращает структуру текущей задачи, которая содержит множество подробностей о запущенном процессе и ссылки на другие структуры, описывающие состояние системы. Обратите внимание, что эта функция относится к нестабильному API
<code>bpf_probe_read_str(dst, size, ptr)</code>	Копирует строку, завершающуюся символом NULL, из небезопасного указателя <code>ptr</code> в буфер с адресом <code>dst</code> , но не более <code>size</code> байт (включая байт NULL)
<code>bpf_perf_event_read_value(map, flags, buf, size)</code>	Читает счетчик <code>perf_event</code> и сохраняет его в буфере <code>buf</code> . С помощью этой функции программы BPF читают счетчики мониторинга производительности (PMC)
<code>bpf_get_current_cgroup_id()</code>	Возвращает числовой идентификатор текущей группы <code>cgroup</code>
<code>bpf_spin_lock(lock),</code> <code>bpf_spin_unlock(lock)</code>	Механизм управления конкурентностью (concurrency) в сетевых программах

Некоторые из этих вспомогательных функций показаны в выводах, полученных выше с помощью `bpftool(8) xlated` и `bpftool -v`.

Понятие *текущий* (`current`), встречающееся в описаниях функций, относится к текущему потоку, который в данный момент выполняется процессором.

Обратите внимание, что файл `include/uapi/linux/bpf.h` содержит подробное описание многих из этих вспомогательных функций. Вот выдержка из описания функции `bpf_get_stackid()`¹:

```
* int bpf_get_stackid(struct pt_reg *ctx, struct bpf_map *map, u64 flags)
*     Описание
*
*     Отыскивает стек в пространстве пользователя или ядра и возвращает
*     его идентификатор. Для этого функции следует передать *ctx*,
*     указатель на контекст, в котором выполняется трассируемая
*     программа, и указатель на *map* типа BPF_MAP_TYPE_STACK_TRACE.
*
*     В последнем аргументе *flags* передается число пропускаемых
*     фреймов стека (от 0 до 255) с маской BPF_F_SKIP_FIELD_MASK.
*     Это число можно комбинировать со следующими флагами:
*
*     BPF_F_USER_STACK
```

¹ В файле `bpf.h` функции описываются на английском языке, здесь приведен перевод. — *Примеч. пер.*

```

*           Выбрать стек из пространства пользователя,
*           а не из пространства ядра.
* **BPF_F_FAST_STACK_CMP**
*           Сравнить стеки только по значению хеша.
* **BPF_F_REUSE_STACKID**
*           Если два разных стека хешируются в один и тот же *stackid*,
*           отбросить более старый.
*
*           Возвращает 32-битный дескриптор стека, который в дальнейшем можно
*           комбинировать с другими данными (включая идентификаторы других
*           стеков) и использовать как ключи в картах. Это может пригодиться
*           для создания разных графиков (флейм-графики или графики
*           ожидания вызова (off-cpu)).
[...]
```

Эти файлы можно просматривать онлайн на любом сайте, где размещены исходные тексты ядра Linux, например: <https://github.com/torvalds/linux/blob/master/include/uapi/linux/bpf.h>.

На самом деле вспомогательных функций гораздо больше, но большинство из них предназначено для реализации программно-определяемых сетей. Текущая версия Linux (5.2) включает 98 вспомогательных функций.

bpf_probe_read()

`bpf_probe_read()` — одна из особенно важных вспомогательных функций. Доступ к памяти в BPF ограничен регистрами BPF и стеком (а также картами BPF через вспомогательные функции). Функция `bpf_probe_read()` позволяет читать данные из произвольной памяти (например, из другой памяти ядра вне BPF), попутно выполняя проверки безопасности и отключая ошибки доступа к страницам, чтобы гарантировать, что попытка чтения не вызовет сбой в контексте проверки (что, в свою очередь, может вызвать проблемы с ядром).

Помимо чтения памяти ядра эта функция также часто используется для копирования данных из пространства пользователя в пространство ядра. Порядок работы зависит от архитектуры: в архитектуре x86_64 адресные пространства пользователя и ядра не перекрываются, поэтому режим можно определить по адресу. Это не относится к другим архитектурам, таким как SPARC [21], для поддержки которых в BPF используются дополнительные вспомогательные функции: `bpf_probe_read_kernel()` и `bpf_probe_read_user()`¹.

Команды системного вызова bpf

В табл. 2.3 перечислены некоторые операции BPF, которые можно выполнять из пространства пользователя.

¹ Эта потребность была отмечена Дэвидом С. Миллером на саммите LSFMM 2019.

Таблица 2.3. Некоторые команды системного вызова `bpf`

bpf_cmd	Описание
<code>BPF_MAP_CREATE</code>	Создает карту BPF: гибкое хранилище объектов, которое можно использовать как хеш-таблицу (ассоциативный массив), хранящую пары ключ — значение
<code>BPF_MAP_LOOKUP_ELEM</code>	Ищет элемент с заданным ключом
<code>BPF_MAP_UPDATE_ELEM</code>	Изменяет элемент с заданным ключом
<code>BPF_MAP_DELETE_ELEM</code>	Удаляет элемент с заданным ключом
<code>BPF_MAP_GET_NEXT_KEY</code>	Выполняет итерации по всем ключам в карте
<code>BPF_PROG_LOAD</code>	Проверяет и загружает программу BPF
<code>BPF_PROG_ATTACH</code>	Подключает программу к событию
<code>BPF_PROG_DETACH</code>	Отключает программу от события
<code>BPF_OBJ_PIN</code>	Создает экземпляр объекта BPF в <code>/sys/fs/bpf</code>

Эти команды передаются в первом аргументе системному вызову `bpf(2)`. Увидеть их в действии можно с помощью `strace(1)`. Например, исследуем системные вызовы `bpf(2)`, выполняемые при запуске инструмента BCC `execsnoop(8)`:

```
# strace -ebpf execsnoop
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_PERF_EVENT_ARRAY, key_size=4,
value_size=4, max_entries=8, map_flags=0, inner_map_fd=0, ...}, 72) = 3
bpf(BPF_PROG_LOAD, {prog_type=BPF_PROG_TYPE_KPROBE, insn_cnt=513,
insns=0x7f31c0a89000, license="GPL", log_level=0, log_size=0, log_buf=0,
kern_version=266002, prog_flags=0, ...}, 72) = 4
bpf(BPF_PROG_LOAD, {prog_type=BPF_PROG_TYPE_KPROBE, insn_cnt=60,
insns=0x7f31c0a8b7d0, license="GPL", log_level=0, log_size=0, log_buf=0,
kern_version=266002, prog_flags=0, ...}, 72) = 6
PCOMM      PID  PPID  RET  ARGS
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=3, key=0x7f31ba81e880, value=0x7f31ba81e910,
flags=BPF_ANY}, 72) = 0
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=3, key=0x7f31ba81e910, value=0x7f31ba81e880,
flags=BPF_ANY}, 72) = 0
[...]
```

Команды в этом выводе выделены жирным. Обычно я стараюсь избегать использования `strace(1)`, поскольку его текущая реализация `ptrace()` может значительно замедлить целевой процесс — более чем в 100 раз [22]. Здесь я использовал эту команду, потому что она уже поддерживает системный вызов `bpf(2)` и преобразует числа в читаемые строки (например, «`BPF_PROG_LOAD`»).

Типы программ BPF

Тип программы BPF определяет тип событий, к которым ее можно подключать, и аргументы для событий. Основные типы программ трассировки на основе BPF перечислены в табл. 2.4.

Таблица 2.4. Типы программ трассировки на основе BPF

bpf_prog_type	Описание
BPF_PROG_TYPE_KPROBE	Для kprobes и uprobes
BPF_PROG_TYPE_TRACEPOINT	Для точек трассировки
BPF_PROG_TYPE_PERF_EVENT	Для perf_events, включая счетчики мониторинга производительности (PMC)
BPF_PROG_TYPE_RAW_TRACEPOINT	Для точек трассировки, без обработки аргументов

Предыдущий вывод команды `strace(1)` включал два вызова `BPF_PROG_LOAD` с типом `BPF_PROG_TYPE_KPROBE`, так как эта версия `execsnoop(8)` использует `kprobe` и `kretprobe` для инструментации начала и окончания выполнения `exesve()`.

В `bpf.h` есть дополнительные типы программ для анализа сетевых взаимодействий и для других целей, включая те, что перечислены в табл. 2.5.

Таблица 2.5. Некоторые другие типы программ BPF

bpf_prog_type	Описание
BPF_PROG_TYPE_SOCKET_FILTER	Для подключения к сокетам — традиционная сфера использования BPF
BPF_PROG_TYPE_SCHED_CLS	Для классификации управляющего трафика
BPF_PROG_TYPE_XDP	Для программ eXpress Data Path
BPF_PROG_TYPE_CGROUP_SKB	Для фильтров пакетов в cgroup (skb)

Типы карт BPF

Типы карт BPF, включая те, что перечислены в табл. 2.6, определяют разные типы карт.

Таблица 2.6. Некоторые типы карт BPF

bpf_map_type	Описание
BPF_MAP_TYPE_HASH	Хеш-таблица: хранилище пар ключ — значение
BPF_MAP_TYPE_ARRAY	Массив элементов
BPF_MAP_TYPE_PERF_EVENT_ARRAY	Интерфейс циклических буферов <code>perf_event</code> для отправки записей трассировки в пространство пользователя
BPF_MAP_TYPE_PERCPU_HASH	Быстрая хеш-таблица, поддерживаемая для каждого процессора
BPF_MAP_TYPE_PERCPU_ARRAY	Быстрый массив, поддерживаемый для каждого процессора

Таблица 2.6 (окончание)

bpf_map_type	Описание
BPF_MAP_TYPE_STACK_TRACE	Хранилище для трассировок стека, индексируемое идентификаторами стеков
BPF_MAP_TYPE_STACK	Хранилище для трассировок стека

Предыдущий вывод команды `strace(1)` включает команду `BPF_MAP_CREATE` создания карты с типом `BPF_MAP_TYPE_PERF_EVENT_ARRAY`, которая используется в `hexesnoop(8)` для передачи событий в пространство пользователя для печати.

В `bpf.h` определено еще много типов карт для специальных целей.

2.3.7. Управление конкурентностью в BPF

Первоначально в BPF отсутствовали средства управления конкурентностью, и только в Linux 5.1 были добавлены вспомогательные циклические блокировки (`spin lock`). (Но пока они недоступны для использования в программах трассировки.) В ходе трассировки параллельно выполняющиеся потоки могут одновременно искать и изменять поля карты BPF, что может приводить к повреждениям, когда один поток затирает данные, записанные другим потоком. Эта проблема известна как проблема «потерянных изменений», когда конкурентные операции чтения и записи перекрываются, что приводит к потере изменений. Интерфейсы трассировки, `BCC` и `bpftrace`, по возможности, используют хеш-таблицы и массивы, отдельные для каждого процессора, чтобы избежать таких повреждений. Они создают экземпляры для каждого логического процессора, чтобы параллельные потоки не изменяли общие данные. Например, карту, ведущую счет событиям, можно изменять для каждого процессора в отдельности, а затем объединять отдельные значения, когда потребуется получить суммарное количество.

В качестве примера ниже приводится однострочная программа `bpftrace`, использующая отдельный хеш для каждого процессора с целью подсчета:

```
# strace -febpf bpftrace -e 'k:vfs_read { @ = count(); }'
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_PERCPU_HASH, key_size=8, value_size=8,
max_entries=128, map_flags=0, inner_map_fd=0}, 72) = 3
[...]
```

А вот однострочная программа `bpftrace`, использующая обычный хеш:

```
# strace -febpf bpftrace -e 'k:vfs_read { @++; }'
bpf(BPF_MAP_CREATE, {map_type=BPF_MAP_TYPE_HASH, key_size=8, value_size=8,
max_entries=128, map_flags=0, inner_map_fd=0}, 72) = 3
[...]
```

При одновременном их использовании в восьмипроцессорной системе для трассировки часто вызываемой функции, которая может работать параллельно, получаются следующие результаты:

```
# bpftrace -e 'k:vfs_read { @cpuhash = count(); @hash++; }'
Attaching 1 probe...
^C
```

```
@cpuhash: 1061370
@hash: 1061269
```

Сравнение счетчиков показывает, что количество событий в нормальном хеше занижено на 0.01%.

Помимо карт, поддерживаемых для каждого процессора в отдельности, есть и другие механизмы управления конкурентностью, в том числе инструкция исключающего сложения (BPF_XADD), карта в карте, которая может обновлять целые карты атомарно, и циклические блокировки BPF. Изменение обычного хеша и карты LRU с помощью `bpf_map_update_elem()` выполняется атомарно и не подвержено состоянию гонки при одновременном выполнении нескольких операций записи. Циклические блокировки, которые были добавлены в Linux 5.1, управляются с помощью вспомогательных функций `bpf_spin_lock()` и `bpf_spin_unlock()` [23].

2.3.8. Интерфейс sysfs для BPF

В Linux 4.4 появились команды для организации доступа к программам и картам BPF через виртуальную файловую систему, обычно монтируемую в `/sys/fs/bpf`. Эта возможность, получившая название «закрепление» (pinning), имеет множество применений. Она позволяет создавать программы BPF, действующие постоянно (подобно демонам) и продолжающие работать после завершения процесса, который их загрузил. Это предоставляет программам пользовательского уровня еще один способ взаимодействия с работающей программой BPF: они могут читать и писать данные в карты BPF.

Механизм закрепления не используется инструментами трассировки на основе BPF, представленными в этой книге, которые запускаются и останавливаются подобно стандартным утилитам Unix. Тем не менее любой из этих инструментов можно превратить в закрепленный, если потребуется. Обычно этот прием используется в сетевых программах (например, в Cilium [24]).

В качестве примера можно привести ОС Android, где механизм закрепления используется для автоматической загрузки и закрепления BPF-программ, находящихся в `/system/etc/bpf` [25]. Для взаимодействия с этими закрепленными программами в библиотеке Android предусмотрены специальные функции.

2.3.9. BPF Type Format (BTF)

Одна из повторяющихся проблем, описанная в этой книге, — это отсутствие информации об инструментируемом коде, что затрудняет разработку инструментов BPF. И идеальным решением этих проблем будет представленный здесь формат BTF.

ВТФ (BPF Type Format) — это формат метаданных, которые определяют отладочную информацию для описания программ BPF, карт BPF и многого другого. Название «ВТФ» выбрано потому, что первоначально этот формат использовался для описания типов данных, однако позднее был расширен для включения информации об определенных подпрограммах, об именах файлов с исходным кодом и номерах строк в этих файлах, а также о глобальных переменных.

Отладочная информация ВТФ может встраиваться в двоичный файл `vmlinux` или генерироваться вместе с программами BPF компилятором Clang или LLVM JIT, что упрощает проверку программ BPF с помощью загрузчиков (например, `libbpf`) и инструментов (например, `bpftool(8)`). Инструменты проверки и трассировки, включая `bpftool(8)` и `perf(1)`, могут извлекать эту информацию и, опираясь на нее, выводить аннотированные исходники программ BPF или пары ключ — значение, хранящиеся в картах с учетом их структуры на языке C вместо «сырого» шестнадцатеричного дампа. Это показывают примеры использования `bpftool(8)` для вывода кода программ ВСС, скомпилированных с использованием LLVM-9.

Помимо описания программ BPF, формат ВТФ можно использовать как универсальный формат описания всех структур данных в ядре. В некотором смысле его можно считать легковесной альтернативой отладочной информации в ядре для использования BPF и более полной и надежной альтернативой заголовкам ядра.

Инструменты трассировки BPF часто требуют установки заголовков ядра (обычно распространяются в виде пакета `linux-headers`), чтобы можно было перемещаться по различным структурам на языке C. Но эти заголовки содержат определения не всех структур, имеющихся в ядре, что затрудняет разработку некоторых инструментов BPF: решить эту проблему можно, определив отсутствующие структуры в исходном коде инструментов BPF. Есть и проблемы со сложными заголовками, которые обрабатываются неправильно. В таких случаях `bpftool` предпочитает прервать выполнение вместо использования потенциально неправильных смещений полей внутри структур. ВТФ позволяет решить и эту проблему, предоставляя надежные определения для всех структур. (Результат выполнения `bpftool bt` показывает, как можно включить `task_struct`.) В будущем двоичный файл ядра Linux `vmlinux`, содержащий ВТФ, будет самоописательным.

На момент написания этой книги формат ВТФ все еще находился в разработке. Чтобы поддержать инициативу «скомпилировать один раз, выполнять везде» (`compile-once-run-everywhere`), нужно добавить в ВТФ больше информации. Новую информацию о ВТФ ищите в `Documentation/bpf/btf.rst` в исходных текстах ядра [26].

2.3.10. BPF CO-RE

Цель проекта BPF Compile Once — Run Everywhere (CO-RE) — один раз скомпилировать программы BPF в байт-код BPF, сохранить, а затем распространять и выполнять их в других системах. Это позволит не устанавливать компиляторы BPF (LLVM и Clang) везде и всюду, что может быть непросто во встраиваемых

системах Linux, имеющих ограниченный объем памяти, а также избежать затрат вычислительных ресурсов и памяти на запуск компилятора при каждом запуске инструмента BPF.

В проекте CO-RE (программист Андрий Накрийко (Andrii Nakryiko)) ведется работа над такими проблемами, как разные смещения в структурах ядра в разных системах, что требует переопределять смещения полей в байт-коде BPF при необходимости. Другая проблема — отсутствие членов структур, из-за этого приходится применять условный доступ к полям в зависимости от версии ядра, его конфигурации и/или предоставленных пользователем флагов времени выполнения. Проект CO-RE будет использовать информацию BTF, и на момент написания этой книги он все еще находился в стадии разработки.

2.3.11. Ограничения BPF

Программы BPF не могут вызывать произвольные функции ядра, им доступны только вспомогательные функции BPF, перечисленные в определении API. Но их список может быть расширен в будущих версиях ядра. Программы BPF также накладывают ограничения на использование циклов: было бы небезопасно разрешать программам BPF использовать бесконечные циклы в произвольных потоках kprobes, потому что они могут содержать критические блокировки, блокирующие работу остальной части системы. К обходным решениям этой проблемы можно отнести развертывание циклов и добавление вспомогательных функций, позволяющих решать задачи, которые требуют применения циклов. В Linux 5.3 появилась поддержка ограниченных циклов в BPF, для которых определяется проверяемый верхний предел времени выполнения¹.

Размер стека BPF ограничен значением `MAX_BPF_STACK`, равным 512. Этот предел иногда превращается в серьезное ограничение в инструментах трассировки BPF, особенно при сохранении в стеке нескольких строковых буферов: один буфер, объявленный как `char[256]`, занимает половину этого стека. Планов по увеличению этого ограничения нет. Для решения проблемы нехватки стека предлагается использовать хранилище карт BPF, которое фактически бесконечно. Началась работа по переносу строк `bpfttrace` в хранилище карт вместо стека.

Первоначально число инструкций в программе BPF было ограничено 4096. Иногда длинные программы BPF сталкивались с этим пределом (и сталкивались бы с ним гораздо чаще без оптимизаций компилятора LLVM, которые сокращают количество команд). В Linux 5.2 этот предел существенно увеличен и не должен

¹ Возможно, вам интересно узнать, станет ли BPF полной по Тьюрингу. Сам набор команд BPF позволяет создать автомат, полный по Тьюрингу, но учитывая ограничения безопасности, которые устанавливает верификатор, программы BPF больше не являются полными по Тьюрингу (например, из-за проблемы прерывания выполнения).

вызывать проблем¹. Цель верификатора BPF — принять любую безопасную программу, и ограничения не должны мешать этому.

2.3.12. Дополнительные источники о BPF

Вот список дополнительных источников о расширенном BPF:

- Documentation/networking/filter.txt в исходных текстах ядра [17];
- Documentation/bpf/bpf_design_QA.txt в исходных текстах ядра [29];
- страница справочного руководства bpf(2) [30];
- страница справочного руководства bpf-helpers(7) [31];
- статья Джонатана Корбета (Jonathan Corbet) «BPF: the universal in-kernel virtual machine» [32];
- статья Сучакры Шармы (Suchakra Sharma) «BPF Internals — II» [33];
- справочное руководство по BPF и XDP в документации к Cilium [19].

Дополнительные примеры программ BPF приводятся в главе 4 и в приложениях C, D и E.

2.4. ОБХОД ТРАССИРОВКИ СТЕКА

Трассировка стека — бесценный инструмент для понимания пути кода, который привел к событию, а также профилирования кода ядра и уровня пользователя для наблюдения за тем, на что тратится время выполнения. Для записи трассировок стека BPF предоставляет специальные типы карт и может извлекать их, выполняя обход стека с использованием указателей на список фреймов или ORC. В будущем в BPF могут также появиться другие методы обхода стека.

2.4.1. Стеки на основе указателя на список фреймов

Метод с использованием указателя на список фреймов (frame pointer) основан на соглашении, в соответствии с которым заголовок связанного списка фреймов стека всегда находится в регистре процессора (в x86_64 это регистр RBP) и адрес возврата хранится с известным смещением (+8) относительно RBP [Hubicka 13]. Это означает, что любой отладчик или трассировщик, приостанавливающий программу, может прочитать RBP и извлечь трассировку стека, выполняя обход связанного списка RBP и выбирая адреса с известным смещением, как показано на рис. 2.6.

¹ Предел был увеличен до миллиона инструкций (BPF_COMPLEXITY_LIMIT_INSNS) [27]. Предел 4096 (BPF_MAXINSNS) продолжает действовать для непривилегированных программ BPF [28].

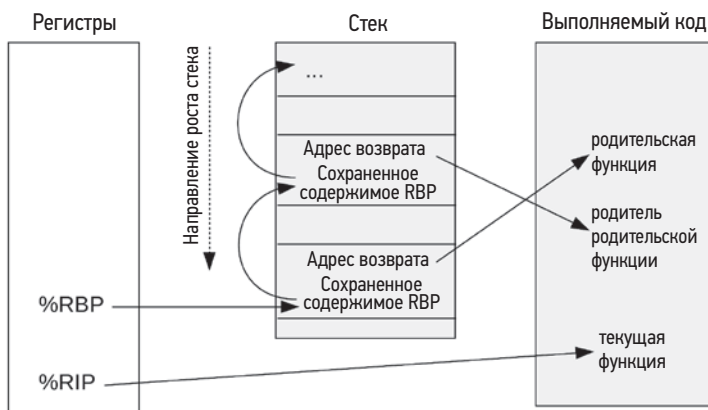


Рис. 2.6. Обход стека с использованием указателя на список фреймов (x86_64)

В описании AMD64 ABI отмечается, что использование регистра RBP в качестве указателя на список фреймов стека — это всего лишь соглашение, и ему можно не следовать, что позволит сэкономить на инструкциях оформления пролога и эпилога функций и использовать RBP как регистр общего назначения.

Компилятор gcc по умолчанию опускает формирование указателя на фрейм и использует RBP в роли регистра общего назначения, что препятствует методу обхода стека на основе указателя на список фреймов. Это поведение по умолчанию можно отменить, использовав параметр `-fno-omit-frame-pointer`. Вот три замечания из патча, в котором реализован отказ от поддержки указателя фрейма по умолчанию [34]:

- Патч был предложен для архитектуры i386, где есть четыре регистра общего назначения. Освобождение регистра RBP увеличивает число полезных регистров с четырех до пяти, что позволяет получить существенный выигрыш в производительности. Но в архитектуре x86_64 уже имеется 16 регистров, пригодных для использования, что делает этот патч менее целесообразным. [35].
- Предполагалось, что проблема обхода стека решена навсегда благодаря поддержке других методов в gdb(1). При этом не учитывается, что обход стека выполняется в ограниченном контексте с отключенными прерываниями.
- Необходимость выглядеть более выигрышно в бенчмарках по сравнению с компилятором icc от Intel.

На сегодняшний день большинство ПО для архитектуры x86_64 скомпилировано с настройками gcc по умолчанию, что мешает трассировке стека с использованием указателя на список фреймов. В последний раз, когда я изучал прирост производительности от пропуска указателя фрейма в нашей производственной среде, он обычно составлял менее 1 % и часто был настолько близок к нулю, что его трудно было измерить. Многие микросервисы в Netflix работают с включенной поддержкой

указателей фреймов, потому что выигрыш в производительности механизмов профилирования перевешивает незначительную потерю производительности.

Использование указателей на фреймы не единственный способ обхода стека. В числе других методов можно указать использование отладочной информации, LBR и ORC.

2.4.2. Использование отладочной информации

Дополнительная отладочная информация часто доступна в пакетах, содержащих ELF-файлы с отладочной информацией в формате DWARF. Она включает разделы, которые отладчики, такие как `gdb(1)`, могут использовать для обхода трассировки стека, даже когда регистр указателя на список фреймов не используется. К ELF-разделам относятся: `.eh_frame` и `.debug_frame`.

Файлы с отладочной информацией также включают разделы с информацией об именах файлов с исходным кодом и номерах строк, в результате чего такие файлы значительно превышают в размерах оригинальные двоичные файлы. Например, в главе 12 упоминается файл `libjvm.so` с размером 17 Мбайт, версия которого с отладочной информацией имеет размер 222 Мбайт. В некоторых средах файлы с отладочной информацией не устанавливаются из-за слишком большого размера.

Сейчас BPF не поддерживает этот метод обхода стека: он связан со значительной нагрузкой на процессор и требует чтения разделов ELF. Это усложняет реализацию в ограниченном контексте BPF с отключенными прерываниями.

Обратите внимание, что интерфейсы BPF — BCC и `bpfttrace` — могут использовать отладочную информацию в файлах для разрешения символов.

2.4.3. Last Branch Record (LBR)

Запись последней ветви (Last Branch Record, LBR) — это особенность процессоров Intel, обеспечивающая сохранение информации о выполнении ветвей в аппаратном буфере, в том числе ветвей, созданных вызовами функций. Этот метод не требует дополнительных затрат и может использоваться для восстановления трассировки стека. Но он ограничен по глубине в зависимости от поколения процессора и поддерживает запись от 4 до 32 ветвей. Трассировка стека для промышленного ПО, особенно на Java, может превышать 32 фрейма.

В настоящее время LBR не поддерживается механизмом BPF, но такая поддержка может появиться в будущем. Ограниченная трассировка стека лучше, чем ее отсутствие.

2.4.4. ORC

Новый формат представления отладочной информации, разработанный для трассировки стека, Ooops Rewind Capability (ORC), требует меньше ресурсов процессора, чем DWARF [36]. Метод на основе ORC использует секции ELF

`.orc_unwind` и `.orc_unwind_ip`. Сейчас этот метод реализован только для ядра Linux. На архитектурах с ограниченным числом регистров может быть желательно скомпилировать ядро без указателя на список фреймов и для трассировки стека использовать ORC.

Раскручивание стека методом ORC доступно в ядре в виде функции `perf_callchain_kernel()`, которую использует BPF. Это означает, что BPF тоже поддерживает трассировку стека методом ORC. Стеки ORC еще не разработаны для пространства пользователя.

2.4.5. Символы

Трассировки стека записываются в ядре в виде массива адресов, которые потом преобразуются в символы (например, в имена функций) программой в пространстве пользователя. Может случиться так, что между сбором трассировки и преобразованием символы изменятся, что приведет к получению неправильных имен или их отсутствию. Эта проблема обсуждается в разделе 12.3.4. Возможно, в будущем поддержка преобразования символов будет добавлена в ядро, чтобы ядро могло собирать и преобразовывать трассировку стека немедленно.

2.4.6. Для дополнительного чтения

Трассировки стека и указатели на списки фреймов для C и Java рассмотрены в главе 12, а в главе 18 подводятся общие итоги.

2.5. ФЛЕЙМ-ГРАФИКИ

В последующих главах я буду подробно рассматривать флейм-графики (flame graphs), поэтому здесь лишь кратко расскажу, как их читать и использовать.

Флейм-графики — это способ визуализации трассировки стека, который я изобрел, работая над проблемой производительности MySQL и пытаясь сравнить два профиля использования процессора, состоящие из тысяч страниц текста [Gregg 16]¹. Их можно использовать для визуализации не только профилей использования процессора, но также для трассировок стека, полученных любым профилировщиком или трассировщиком. В следующих разделах я покажу, как их применить для трассировки с помощью BPF событий ожидания (off-CPU), сбоев страниц и многого другого. В этом разделе расскажу только о визуализации.

¹ Общая идея вывода SVG и поддержка интерактивности на JavaScript заимствованы из визуализации стека вызовов в `function_call_graph.rb`, созданной Нилакантом Надгиром (Neelakanth Nadgir), которая сама была создана по образу и подобию `CallStackAnalyzer` Роша Бурбоне (Roch Bourbonnais) и `vftrace` Яна Бурхоута (Jan Boerhout).

2.5.1. Трассировка стека

Трассировка стека, которую иногда называют обратной трассировкой стека или трассировкой вызова, — это последовательность функций, отражающая поток выполнения кода. Например, если `func_a()` вызвала `func_b()`, а та вызвала `func_c()`, трассировку стека в этой точке можно записать так:

```
func_c
func_b
func_a
```

Дно стека (`func_a`) является началом, а строки над ним показывают, как протекало выполнение. Другими словами, вершина стека (`func_c`) представляет текущую функцию, а перемещение вниз позволяет увидеть ее родословную: родительскую функцию, затем родителя родительской функции, и т. д.

2.5.2. Профилирование трассировки стека

Временная выборка трассировок стека может включать тысячи стеков, каждый из которых состоит из десятков или сотен строк. Чтобы изучить такой объем данных, профилировщик `perf(1)` в Linux представляет выборки в виде дерева вызовов и показывает проценты для каждого пути. Инструмент BCC `profile(8)` представляет трассировки стека иначе, показывая количество каждой уникальной трассировки. Фактические примеры использования `perf(1)` и `profile(8)` будут показаны в главе 6. Оба инструмента позволяют быстро выявить патологические проблемы в ситуациях, когда один и тот же стек занимает большую часть процессорного времени. Но при решении многих других проблем производительности, в том числе небольших, для поиска виновника может потребоваться изучить сотни страниц вывода профилировщика. Флейм-графики помогают решить эту проблему.

Чтобы понять, как пользоваться флейм-графиками, рассмотрим искусственный пример вывода профилировщика процессора, показывающий частоту каждой трассировки стека:

```
func_e
func_d
func_b
func_a
1
```

```
func_b
func_a
2
```

```
func_c
func_b
func_a
7
```

В выводе показаны трассировки стека с их счетчиками. Всего есть 10 выборок. Например, путь `func_a() -> func_b() -> func_c()` встречается семь раз. Этот путь показывает, что в момент выборки процессором выполнялась функция `func_c()`. Путь `func_a() -> func_b()` встречается два раза и показывает, что в момент выборки процессором выполнялась функция `func_b()`. Наконец, путь, который заканчивается функцией `func_e()`, встречается один раз.

2.5.3. Флейм-график

На рис. 2.7 показан флейм-график, соответствующий предыдущим результатам профилирования.

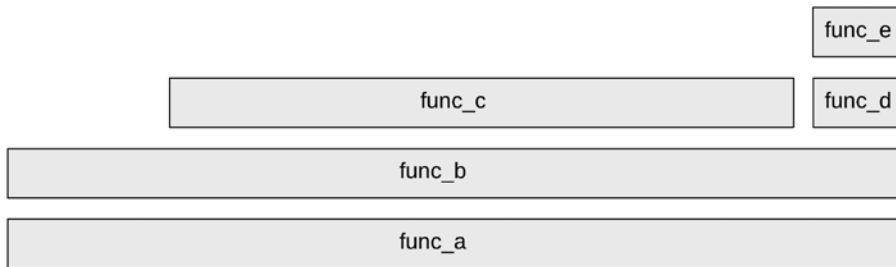


Рис. 2.7. Флейм-график

У этого флейм-графика следующие свойства:

- Каждый прямоугольник представляет функцию в стеке («фрейм стека»).
- Ось Y отражает глубину стека (количество фреймов в стеке), простирающегося от корня до листа вверх. Глядя снизу вверх, можно понять поток выполнения кода. Глядя сверху вниз — определить родословную функции.
- Ось X охватывает совокупность выборки. Важно отметить, что она *не* показывает ход времени слева направо, как на большинстве графиков. Фреймы располагаются в направлении слева направо в порядке уменьшения их количества в выборках. С учетом упорядочения вдоль оси Y это означает, что начало находится внизу слева (как в большинстве графиков) и представляет точку 0, а. Длина по оси X имеет значение: ширина прямоугольника отражает его присутствие в профиле. Функции, представленные широкими прямоугольниками, имеют большее присутствие в профиле, чем функции, представленные узкими прямоугольниками.

По сути, флейм-график — это диаграмма смежности с перевернутым расположением элементов [Bostock 10], он применяется для визуализации иерархии набора трассировок стека.

На рис. 2.7 показан наиболее часто встречающийся в профиле путь выполнения, который представлен самой широкой «башней» в середине, от `func_a()` до `func_c()`. Поскольку флейм-график показывает, какие функции выполнялись процессором

в момент зондирования, можно сказать, что его верхний край определяет функции, выполнявшиеся процессором (on-CPU), как показано на рис. 2.8.

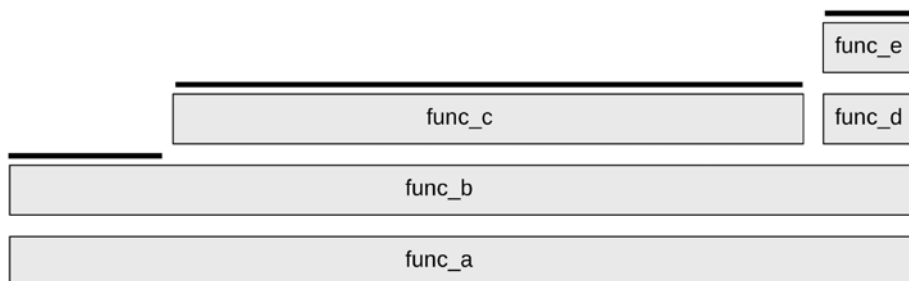


Рис. 2.8. Функции, выполнявшиеся процессором, на флейм-графике

На рис. 2.8 видно, что `func_c()` занимала 70% процессорного времени, `func_b()` — 20%, а `func_e()` — 10%. Другие функции — `func_a()` и `func_d()` — ни разу не были обнаружены непосредственно выполняющимися на процессоре.

Читая флейм-график, сначала ищите самые широкие башни и попробуйте понять их.

В больших профилях, включающих тысячи выборок, пути выполнения кода могут встречаться лишь по несколько раз и составлять настолько узкие башни, что в них не остается места для включения имен функций. Как оказывается, это преимущество: все свое внимание вы, естественно, уделите более широким башням, имеющим разборчивые имена функций, и их изучение поможет понять основную часть профиля.

2.5.4. Особенности флейм-графика

В следующих разделах описаны особенности моей оригинальной реализации флейм-графика [37].

Цветовые палитры

Фреймы можно окрашивать с использованием разных схем. По умолчанию для каждого фрейма используется случайно выбранный теплый цвет, что помогает визуально различать соседние башни. Но со временем я добавил несколько цветовых схем. Я обнаружил, что для конечных пользователей флейм-графиков намного полезнее, когда:

- **Оттенок** определяет тип кода¹. Например, красный может обозначать собственный код в пространстве пользователя, оранжевый — собственный код

¹ Это решение было предложено моим коллегой Амером Азером (Amer Ather). Моя первая реализация этого решения была основана на использовании регулярного выражения, написанного на скорую руку.

в пространстве ядра, желтый — для C++, зеленый — для функций на интерпретируемом языке, голубой — для встроенных функций, и так далее в зависимости от используемого языка. Пурпурный можно использовать для выделения совпадений, найденных в процессе поиска. Некоторые разработчики настраивают флейм-графики так, чтобы их собственный код всегда выделялся определенным оттенком.

- **Насыщенность** некоторым образом соответствует имени функции. Это обеспечивает некоторую цветовую градацию, помогающую различать соседние башни, сохраняя при этом одинаковые цвета для одинаковых имен функций, что упрощает сравнение нескольких флейм-графиков.
- **Цвет фона** служит визуальным напоминанием о типе флейм-графика. Например, желтый фон можно использовать для флейм-графиков распределения процессорного времени, синий — для флейм-графиков ожидания или ввода/вывода и зеленый — для флейм-графиков использования памяти.

Другая полезная цветовая схема — схема, используемая для представления на графиках количества инструкций на такт (Instructions Per Cycle, IPC), где дополнительное измерение, IPC, визуализируется как градиентная окраска каждого фрейма — от синего цвета к белому или красному.

Реакция на движение курсора

Оригинальное ПО для построения флейм-графиков создает файлы SVG со встроенным кодом на JavaScript, которые можно загружать в браузер и исследовать в интерактивном режиме. Одна из особенностей этого решения заключается в том, что при наведении курсора на фреймы отображается всплывающая подсказка, показывающая процент присутствия этого фрейма в профиле.

Масштабирование

Щелкнув на фрейме, можно увеличить его горизонтальный масштаб¹. Это позволяет исследовать узкие фреймы и увеличивать их масштаб, чтобы читать имена соответствующих функций.

Поиск

Щелкнув на кнопке поиска или нажав Ctrl-F, можно ввести слово и выполнить его поиск — фреймы, соответствующие этому поисковому запросу, будут выделены пурпурным цветом. Также будет показан совокупный процент, определяющий, как часто встречается трассировка стека, содержащая искомое слово. Это позволяет быстро выяснить, какая доля профиля приходится на определенные области кода.

¹ Функцию горизонтального масштабирования для флейм-графиков разработал Адриан Майе (Adrien Mahieux).

Например, можно выполнить поиск по строке «tcp_» и узнать, какая доля приходится на код TCP в ядре.

2.5.5. Разновидности

С помощью d3 [38] в Netflix разрабатывается версия флейм-графика, обладающая еще более богатыми интерактивными возможностями¹. Она имеет открытый исходный код и используется в ПО Netflix FlameScope [39].

Некоторые реализации флейм-графиков по умолчанию переворачивают график по оси Y, создавая «диаграмму в форме сосулек» с корнем вверх. Эта инверсия гарантирует, что корень и его непосредственные функции будут видны на графиках, высота которых превышает высоту экрана. Моя оригинальная реализация флейм-графика поддерживает эту инверсию, предлагая флаг `--inverted`. Я предпочитаю использовать этот способ отображения для объединения листьев с корнями. Еще один вариант флейм-графика объединяет первыми листья, а потом корни. Таким способом можно сначала объединить трассировки по общей функции на процессоре, а затем рассмотреть их родословные, например циклические блокировки.

Флейм-диаграммы выглядят как флейм-графики, они были созданы по их образу и подобию [Tikhonovsky 13], но ось X в них отражает течение времени, а не долю присутствия. Флейм-графики широко используются в веб-инструментах анализа для исследования кода на JavaScript, так как прекрасно подходят для исследования временных закономерностей в однопоточных приложениях. Некоторые инструменты профилирования поддерживают и флейм-графики, и флейм-диаграммы.

Дифференциальные флейм-графики отображают различия между двумя профилями.

2.6. ИСТОЧНИКИ СОБЫТИЙ

На рис. 2.9 показаны различные источники и примеры событий, доступные для инструментации.

Эти источники событий описаны в следующих разделах.

2.7. KPROBES

Механизм kprobes предлагает возможность динамической инструментации ядра. Его разработала команда IBM на основе их трассировщика DProbes в 2000 году, но трассировщик DProbes не был включен в ядро Linux в отличие от kprobes. Поддержка kprobes появилась в версии Linux 2.6.9, вышедшей в 2004 году.

¹ Флейм-графики на основе d3 созданы моим коллегой Мартином Спиром (Martin Spier).

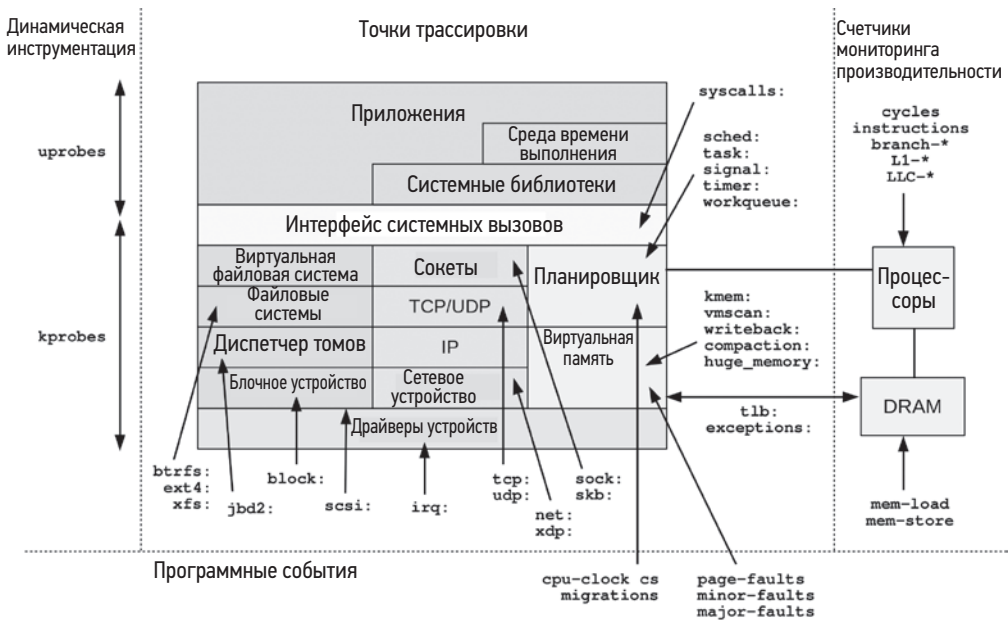


Рис. 2.9. Поддержка событий в BPF

kprobes может генерировать инструментируемые события для любой функции ядра и инструментировать отдельные инструкции внутри этих функций. Он может делать это динамически, в промышленных средах, не требуя перезагружать систему или запускать ядро в каком-то специальном режиме. Это удивительный механизм: с его помощью можно использовать любую из десятков тысяч функций в ядре Linux для создания новых нестандартных метрик.

Технология kprobes имеет также интерфейс, называемый kretprobes, предназначенный для наблюдения за возвратом из функций и исследования возвращаемых значений. Применяя kprobes и kretprobes к одной и той же функции, можно получить временные метки для вычисления продолжительности выполнения функции, которая может пригодиться для анализа производительности.

2.7.1. Как работает kprobes

Ниже описана последовательность инструментации инструкций в ядре с помощью kprobes [40]:

А. Если инструментация выполняет kprobe:

1. Байты из целевого адреса копируются и сохраняются механизмом kprobes (ровно столько, сколько необходимо, чтобы заменить их инструкцией точки останова).

2. В целевой адрес записывается инструкция точки останова: `int3` в `x86_64`. (Если возможна оптимизация `kprobe`, записывается инструкция `jmp`.)
 3. Когда поток выполнения достигнет этой точки останова, обработчик точки останова проверит, была ли она установлена механизмом `kprobes`, и если это так, вызовет обработчик `kprobe`.
 4. Затем будут выполнены оригинальные инструкции, которые были заменены инструкцией точки останова, и работа потока возобновится.
 5. Когда трассировка больше не нужна, оригинальные байты будут скопированы обратно в целевой адрес и код вернется в исходное состояние.
- В. Если инструментация выполняет `kprobe` и целевой адрес уже используется трассировщиком `Ftrace` (как правило, это точки входа в функции), возможна оптимизация `kprobe` на основе `Ftrace`, где [Hiramatsu 14]:
1. Обработчик `Ftrace kprobe` регистрируется как операция `Ftrace` для трассируемой функции.
 2. Функция выполняет свой встроенный вызов в прологе функции (`__fentry__` в `gcc 4.6+` и `x86`), который вызывает `Ftrace`. `Ftrace` вызывает обработчик `kprobe`, после чего управление возвращается в функцию.
 3. Когда `kprobe` больше не нужен, обработчик `Ftrace kprobe` удаляется из `Ftrace`.
- С. Если инструментация выполняет `kretprobe`:
1. Создается точка `kprobe` для входа в функцию.
 2. Когда поток выполнения достигает точки входа в функцию, `kprobe` сохраняет адрес возврата и замещает его возвратом в функцию `kretprobe_trampoline()`.
 3. Когда функция наконец выполняет возврат (например, инструкцию `ret`), процессор передает управление в функцию `kretprobe_trampoline()`, которая вызывает обработчик `kretprobe`.
 4. Завершая работу, обработчик `kretprobe` выполняет возврат по сохраненному адресу.
 5. Когда `kretprobe` больше не нужен, точка `kprobe` удаляется.

В зависимости от архитектуры и других факторов, обработчики `kprobe` могут работать с отключенным вытеснением или запрещенными прерываниями.

Модификация инструкций в коде ядра в масштабе реального времени может показаться невероятно рискованным предприятием, тем не менее она вполне безопасна. Имеется «черный список» функций, которые `kprobe` не будет инструментировать, включая функции самого механизма `kprobes`, чтобы избежать состояния рекурсивного перехвата¹. Кроме того, в `kprobes` используются безопасные приемы вставки

¹ Предотвратить возможность трассировки функции ядра можно, перечислив их с помощью макроса `NOKPROBE_SYMBOL()`.

точек останова: чтобы другие ядра процессора не выполнили инструкции в момент их изменения, в архитектуре x86 подставляется инструкция `int3`, а если для замены используется инструкция `jmp`, то вызывается `stop_machine()`. Наибольший риск на практике представляет инструментация часто используемых функций ядра: в этом случае небольшой оверхед, который добавляется к каждому вызову, может накапливаться и замедлять работу системы.

kprobes не работает в некоторых 64-битных системах ARM, где изменения в разделе кода ядра запрещены по соображениям безопасности.

2.7.2. Интерфейсы kprobes

Оригинальная технология kprobes использовалась в разработке модулей ядра, которые определяли пред- и постобработчики, написанные на языке C, и регистрировали их с помощью вызова API `kprobe: register_kprobe()`. Затем вы могли загрузить свой модуль ядра и выдать пользовательскую информацию через системные сообщения вызовами `printk()`. После завершения работы нужно было вызвать `unregister_kprobe()`.

Я не видел, чтобы кто-то использовал этот интерфейс напрямую, и единственное его упоминание встретилось мне в 2010 году в статье «Kernel instrumentation using kprobes» из электронного журнала по проблемам безопасности Phrack, написанной исследователем с ником ElfMaster¹ [41]. Это не является неудачным решением в механизме kprobes, потому что изначально он создавался для использования с Dprobes. Сейчас есть три интерфейса kprobes:

- API `kprobe: register_kprobe()` и др.;
- на основе Ftrace, через файл `/sys/kernel/debug/tracing/kprobe_events`, позволяющий включать и отключать kprobes путем записи в него конфигурационной строки;
- `perf_event_open()`: используется инструментом `perf(1)`, а в последнее время механизмом трассировки в BPF, после добавления поддержки в ядро Linux 4.17 (`perf_kprobe pmu`).

Наиболее часто kprobes используется через интерфейсные трассировщики, включая `perf(1)`, SystemTap и трассировщики BCC и bpftrace в BPF.

Исходная реализация kprobes также включала вариант под названием jprobes — интерфейс, предназначенный для трассировки входа в функции ядра. Со временем мы поняли, что kprobes обладает достаточными возможностями, чтобы удовлетворить все требования, и в 2018 году Масами Хирамацу (Masami Hiramatsu) — администратор kprobe — удалил интерфейс jprobes из Linux.

¹ По случайному совпадению, через три дня после того как я написал это предложение, я встретился с ElfMaster, и он научил меня многим тонкостям анализа ELF. В том числе он рассказал, как разбирать таблицы ELF, о чем вы узнаете в главе 4.

2.7.3. BPF и kprobes

Механизм kprobes поддерживает динамическую инструментацию ядра для BCC и bpftrace и используется многими инструментами. Его интерфейсы:

- **BCC:** `attach_kprobe()` и `attach_kretprobe()`;
- **bpftrace:** `kprobe` и `kretprobe`.

Интерфейс `kprobe` в BCC поддерживает инструментацию точек входа в функции и инструкции с заданным смещением, а `bpftrace` поддерживает только инструментацию точек входа. Интерфейс `kretprobe` для обоих трассировщиков инструментирует выход из функции.

В качестве примера из BCC инструмент `vfsstat(8)` инструментирует ключевые вызовы интерфейса виртуальной файловой системы (Virtual File System, VFS) и выводит посекундные сводки:

```
# vfsstat
TIME      READ/s    WRITE/s   CREATE/s   OPEN/s    FSYNC/s
07:48:16:    736     4209        0         24        0
07:48:17:    386     3141        0         14        0
07:48:18:    308     3394        0         34        0
07:48:19:    196     3293        0         13        0
07:48:20:   1030     4314        0         17        0
07:48:21:    316     3317        0         98        0
[...]
```

Отслеживаемые зонды можно увидеть в исходном коде `vfsstat`:

```
# grep attach_ vfsstat.py
b.attach_kprobe(event="vfs_read", fn_name="do_read")
b.attach_kprobe(event="vfs_write", fn_name="do_write")
b.attach_kprobe(event="vfs_fsync", fn_name="do_fsync")
b.attach_kprobe(event="vfs_open", fn_name="do_open")
b.attach_kprobe(event="vfs_create", fn_name="do_create")
```

Это вызовы функций `attach_kprobe()`. Инструментируемые функции ядра можно увидеть в операторе присваивания «`event =`».

Для примера из `bpftrace` приведу однострочный сценарий, подсчитывающий вызовы всех функций VFS, с именами, начинающимися с «`vfs_`»:

```
# bpftrace -e 'kprobe:vfs_* { @[probe] = count() }'
Attaching 54 probes...
^C

@[kprobe:vfs_unlink]: 2
@[kprobe:vfs_rename]: 2
@[kprobe:vfs_readlink]: 2
@[kprobe:vfs_statx]: 88
@[kprobe:vfs_statx_fd]: 91
@[kprobe:vfs_getattr_nosec]: 247
```

```
@[kprobe:vfs_getattr]: 248
@[kprobe:vfs_open]: 320
@[kprobe:vfs_writes]: 441
@[kprobe:vfs_write]: 4977
@[kprobe:vfs_read]: 5581
```

Этот вывод показывает, что в процессе трассировки функция `vfs_unlink()` была вызвана два раза, а функция `vfs_read()` — 5581 раз.

Возможность подсчитывать число вызовов любой функции ядра — это полезная возможность, она может использоваться для определения характера рабочей нагрузки на подсистемы ядра¹.

2.7.4. Дополнительные источники информации о kprobes

Вот список дополнительных источников информации о kprobes:

- Documentation/kprobes.txt в исходных текстах ядра Linux [42];
- статья Судханшу Госвами (Sudhanshu Goswami) «An Introduction to kprobes» [40];
- статья Прасанна Панчамухи (Prasanna Panchamukhi) «Kernel Debugging with kprobes» [43].

2.8. UPROBES

Механизм uprobes предлагает возможность динамической инструментации в пространстве пользователя. Работа над ним началась много лет назад с создания интерфейса utrace, напоминающего интерфейс kprobes. В итоге технология uprobes вошла в состав ядра Linux 3.5, вышедшего в июле 2012 года [44].

Механизм uprobes похож на kprobes, но предназначен для трассировки процессов, выполняющихся в пространстве пользователя. uprobes может инструментировать точки входа в функции, выполняющиеся в пространстве пользователя, а также инструкции со смещением от начала функций и выходы из функций.

Механизм uprobes также основан на файлах: когда производится трассировка функции в выполняемом файле, инструментуются все процессы, использующие этот файл, включая те, которые только будут запущены. Это позволяет отслеживать библиотечные вызовы в масштабе всей системы.

¹ На момент написания этой книги я все еще использовал Ftrace, потому что этот механизм быстрее инициализирует и удаляет инструментированные точки. См. мой инструмент `funcscout(8)` из моего репозитория Ftrace `perf-tools`. В настоящее время ведется работа по повышению скорости работы BPF kprobe с применением пакетных операций. Я надеюсь, что она завершится, когда вы будете читать эти строки.

2.8.1. Как работает uprobes

uprobes имеет много общего с kprobes: на место целевой инструкции вставляется инструкция останова, которая передает выполнение обработчику uprobe. Когда uprobe больше не нужен, целевые инструкции возвращаются в исходное состояние. Интерфейс uretprobes, как и его аналог kretprobes, инструментирует точку входа в функцию, а адрес возврата подменяется адресом служебной функции.

Вы можете увидеть это в действии, используя отладчик. Например, дизассемблирование функции `readline()` из оболочки `bash(1)`:

```
# gdb -p 31817
[...]
(gdb) disas readline
Dump of assembler code for function readline:
    0x000055f7fa995610 <+0>: cmpl $0xffffffff,0x2656f9(%rip) # 0x55f7fabfad10
<rl_pending_input>
    0x000055f7fa995617 <+7>: push %rbx
    0x000055f7fa995618 <+8>: je 0x55f7fa99568f <readline+127>
    0x000055f7fa99561a <+10>: callq 0x55f7fa994350 <rl_set_prompt>
    0x000055f7fa99561f <+15>: callq 0x55f7fa995300 <rl_initialize>
    0x000055f7fa995624 <+20>: mov 0x261c8d(%rip),%rax # 0x55f7fabf72b8
<rl_prep_term_function>
    0x000055f7fa99562b <+27>: test %rax,%rax
[...]
```

А вот как выглядит та же функция после инструментации с помощью uprobes (или uretprobes):

```
# gdb -p 31817
[...]
(gdb) disas readline
Dump of assembler code for function readline:
    0x000055f7fa995610 <+0>: int3
    0x000055f7fa995611 <+1>: cmp $0x2656f9,%eax
    0x000055f7fa995616 <+6>: callq *0x74(%rbx)
    0x000055f7fa995619 <+9>: jne 0x55f7fa995603 <rl_initialize+771>
    0x000055f7fa99561b <+11>: xor %ebp,%ebp
    0x000055f7fa99561d <+13>: (bad)
    0x000055f7fa99561e <+14>: (bad)
    0x000055f7fa99561f <+15>: callq 0x55f7fa995300 <rl_initialize>
    0x000055f7fa995624 <+20>: mov 0x261c8d(%rip),%rax # 0x55f7fabf72b8
<rl_prep_term_function>
[...]
```

Обратите внимание, что первая инструкция превратилась в точку останова `int3` (архитектура `x86_64`).

Для инструментации функции `readline()` я использовал однострочный сценарий `bpfttrace`:


```
# bpftrace -e 'uprobe:/bin/bash:readline { @ = count() }'
Attaching 1 probe...
^C
```

```
@: 4
```

Он подсчитывает число вызовов `readline()` во всех оболочках `bash`, текущих и будущих, выводит счетчик и завершает работу при нажатии `Ctrl-C`. Когда `bpftrace` завершается, `uprobe` удаляет точку останова и восстанавливает оригинальные инструкции.

2.8.2. Интерфейсы uprobes

Механизм `uprobes` имеет два интерфейса:

- на основе `Ftrace`, через файл `/sys/kernel/debug/tracing/uprobe_events`, позволяющий включать и отключать `uprobes` путем записи в него конфигурационной строки;
- `perf_event_open()`: используется инструментом `perf(1)`, а в последнее время механизмом трассировки в `BPF`, после добавления поддержки в ядро `Linux 4.17` (`perf_uprobe_pmu`).

В ядре есть и функция `register_uprobe_event()`, напоминающая `register_kprobe()`, но она не является частью общедоступного API.

2.8.3. BPF и uprobes

Механизм `uprobes` поддерживает динамическую инструментацию кода в пространстве пользователя для `BCC` и `bpftrace` и используется многими инструментами. Его интерфейсы:

- **BCC**: `attach_uprobe()` и `attach_uretprobe()`;
- **bpftrace**: `uprobe` и `uretprobe`.

Интерфейс `uprobe` в `BCC` поддерживает инструментацию точек входа в функции и инструкций с заданным смещением, а `bpftrace` в настоящее время поддерживает только инструментацию точек входа. Интерфейс `uretprobe` для обоих трассировщиков инструментирует выход из функции.

В качестве примера из `BCC`: инструмент `gethostlatency(8)` инструментирует попытки разрешения сетевых имен (DNS) — вызовы функций `getaddrinfo(3)`, `gethostbyname(3)` и т. д.:

```
# gethostlatency
TIME      PID      COMM          LATms HOST
01:42:15  19488    curl          15.90 www.brendangregg.com
01:42:37  19476    curl          17.40 www.netflix.com
01:42:40  19481    curl          19.38 www.netflix.com
01:42:46  10111    DNS Res~er #659 28.70 www.google.com
```

Отслеживаемые зонды можно увидеть в исходном коде `gethostlatency`:

```
# grep attach_ gethostlatency.py
b.attach_uprobe(name="c", sym="getaddrinfo", fn_name="do_entry", pid=args.pid)
b.attach_uprobe(name="c", sym="gethostbyname", fn_name="do_entry",
b.attach_uprobe(name="c", sym="gethostbyname2", fn_name="do_entry",
b.attach_uretprobe(name="c", sym="getaddrinfo", fn_name="do_return",
b.attach_uretprobe(name="c", sym="gethostbyname", fn_name="do_return",
b.attach_uretprobe(name="c", sym="gethostbyname2", fn_name="do_return",
```

Это вызовы функций `attach_uprobe()` и `attach_uretprobe()`. Инструментируемые функции можно увидеть в операторе присваивания «`sym=`».

В качестве примера из `bpfftrace`: вот однострочный сценарий, подсчитывающий вызовы всех функций `gethost` из системной библиотеки `libc`:

```
# bpfftrace -l 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethost*'
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname2
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostname
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostid
[...]
# bpfftrace -e 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethost* { @[probe] =
count(); }'
Attaching 10 probes...
^C
```

```
@[uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostname]: 2
```

Этот вывод показывает, что в процессе трассировки функция `gethostname()` была вызвана два раза.

2.8.4. Оверхед `uprobes` и будущие улучшения

`uprobes` может прикрепляться к событиям, которые происходят миллионы раз в секунду, например к процедурам выделения на уровне пользователя: `malloc()` и `free()`. Несмотря на то что BPF оптимизирован для достижения наивысшей производительности, любой, даже очень небольшой оверхед, помноженный на миллионы раз в секунду, может оказаться весьма внушительным. В некоторых случаях трассировка `malloc()` и `free()` с использованием BPF может замедлить целевое приложение в десятки раз. По этой причине желательно избегать трассировки подобных функций. Такие замедления допустимы только при устранении неполадок в тестовой среде или в уже не работающей промышленной среде. В главе 18 вы найдете раздел о частоте операций, который поможет обойти это ограничение. По возможности старайтесь определить, какие события происходят часто, чтобы избежать их трассировки, и ищите более редкие события, трассировка которых поможет в решении той же проблемы.

В будущем трассировка пространства пользователя может быть значительно улучшена, возможно, даже к тому времени, когда вы будете читать эти строки. Вместо того чтобы продолжать использовать текущий подход `uprobes`, который

упирается в ядро, обсуждается решение с общей библиотекой, которое обеспечит BPF-трассировку пользовательского пространства без переключения режима ядра.

Этот подход много лет использовался в LTTng-UST, при этом он имел производительность в десятки, а то и в сотни раз выше [45].

2.8.5. Дополнительные источники информации о `uprobes`

Дополнительную информацию ищите в файле `Documentation/trace/uprobetracer.txt` в исходных текстах ядра Linux [46].

2.9. ТОЧКИ ТРАССИРОВКИ

Точки трассировки (tracepoints) используются для статической инструментации ядра. К их числу относятся трассировочные вызовы, добавленные разработчиками в подходящих местах. Они компилируются в двоичный файл ядра. Разработанные Матье Деснойерсом в 2007 году, точки трассировки первоначально назывались маркерами ядра (Kernel Markers) и были включены в ядро Linux 2.6.32 в 2009 году. В табл. 2.7 приводится сравнение `kprobes` и точек трассировки.

Таблица 2.7. Сравнение `kprobes` и точек трассировки

Характеристика	<code>kprobes</code>	Точки трассировки
Тип	Динамический	Статический
Примерное число событий	50 000+	100+
Поддержка разработчиками ядра	Нет	Требуется
Оверхед при неиспользовании	Нет	Небольшие (пустые инструкции и метаданные)
Стабильность API	Нет	Да

Поддержка точек трассировки целиком и полностью возлагается на разработчиков ядра, к тому же область, которую можно исследовать с помощью точек трассировки, намного уже. С другой стороны, точки трассировки обеспечивают стабильный API¹: инструменты, использующие точки трассировки, должны продолжать работать в новых версиях ядра, тогда как инструменты, использующие `kprobes`, не дают такой гарантии, если имя трассируемой функции изменится.

Всегда старайтесь сначала использовать точки трассировки, если они доступны, и обращаться к `kprobes`, только если точек трассировки оказалось недостаточно.

Имена точек трассировки имеют формат: *подсистема:имя-события* (например, `kmem:kmalloc`) [47]. В описаниях трассировщиков первый компонент называют по-разному: система, подсистема, класс или провайдер.

¹ Я бы назвал его «условно стабильным». Мне доводилось видеть изменения в точках трассировки.

2.9.1. Инструментация точек трассировки

В этом разделе я расскажу, как была добавлена в ядро точка трассировки `sched:sched_process_exec`.

Для точек трассировки имеются заголовочные файлы в `include/trace/events`. Вот выдержка из `sched.h`:

```
#define TRACE_SYSTEM sched
[...]
/*
 * Точка трассировки для exec:
 */
TRACE_EVENT(sched_process_exec,

    TP_PROTO(struct task_struct *p, pid_t old_pid,
             struct linux_binprm *bprm),

    TP_ARGS(p, old_pid, bprm),

    TP_STRUCT__entry(
        __string( filename, bprm->filename)
        __field(   pid_t,      pid )
        __field(   pid_t,      old_pid )
    ),

    TP_fast_assign(
        __assign_str(filename, bprm->filename);
        __entry->pid      = p->pid;
        __entry->old_pid   = old_pid;
    ),

    TP_printk("filename=%s pid=%d old_pid=%d", __get_str(filename),
              __entry->pid, __entry->old_pid)
);
```

Этот код определяет систему для трассировки как `sched`, а имя точки трассировки — как `sched_process_exec`. Следующие строки определяют метаданные, включая «строку формата» в вызове `TP_printk()`, который выводит некоторую полезную информацию при ее использовании с помощью инструмента `perf(1)`.

Предыдущая информация также доступна во время выполнения через инфраструктуру `Ftrace` в `/sys`, в файлах `format`, которые создаются для каждой точки трассировки. Например:

```
# cat /sys/kernel/debug/tracing/events/sched/sched_process_exec/format
name: sched_process_exec
ID: 298
format:
    field:unsigned short common_type; offset:0; size:2; signed:0;
    field:unsigned char common_flags; offset:2; size:1; signed:0;
    field:unsigned char common_preempt_count; offset:3; size:1; signed:0;
    field:int common_pid; offset:4; size:4; signed:1;

    field:__data_loc char[] filename; offset:8; size:4; signed:1;
```

```

        field:pid_t pid; offset:12; size:4; signed:1;
        field:pid_t old_pid; offset:16; size:4; signed:1;
print fmt: "filename=%s pid=%d old_pid=%d", __get_str(filename), REC->pid,
REC->old_pid

```

Эти файлы format помогают трассировщикам определить, какие метаданные связаны с той или иной точкой трассировки.

Следующая точка трассировки вызывается из файла fs/exec.c через `trace_sched_process_exec()`:

```

static int exec_binprm(struct linux_binprm *bprm)
{
    pid_t old_pid, old_vpid;
    int ret;

    /* Нужно извлечь pid перед тем, как load_binary изменит его */
    old_pid = current->pid;
    rcu_read_lock();
    old_vpid = task_pid_nr_ns(current, task_active_pid_ns(current->parent));
    rcu_read_unlock();

    ret = search_binary_handler(bprm);
    if (ret >= 0) {
        audit_bprm(bprm);
        trace_sched_process_exec(current, old_pid, bprm);
        ptrace_event(PTRACE_EVENT_EXEC, old_vpid);
        proc_exec_connector(current);
    }
    [...]
}

```

Вызов функции `trace_sched_process_exec()` отмечает местоположение точки трассировки.

2.9.2. Как работают точки трассировки

Важно, чтобы неактивные точки трассировки имели как можно более низкий оверхед, чтобы не платить производительностью за то, что не используется. Матье Деснойерс (Mathieu Desnoyers) предложил решение, которое основано на приеме, получившем название «исправление статического перехода» (static jump patching)¹. Он работает следующим образом, при условии, что доступна необходимая функция компилятора (`asm goto`):

1. Во время компиляции ядра в местоположение точки трассировки добавляется инструкция, которая ничего не делает. Фактически используемая инструкция зависит от архитектуры: для x86_64 это 5-байтовая инструкция пустой операции (`nop`). Такой ее размер выбран для того, чтобы потом эту инструкцию можно было заменить 5-байтовой инструкцией перехода (`jmp`).

¹ В более ранних версиях загружались непосредственные инструкции, операнды которых можно было изменять между 0 и 1 и управлять так потоком к точке трассировки [Desnoyers 09a], [Desnoyers 09b]. Но потом предпочтение было отдано приему исправления перехода.

2. Обработчик точки трассировки также добавляется в конец функции, которая перебирает массив зарегистрированных обратных вызовов. Это немного увеличивает размер сегмента кода (так как обработчик — это небольшая подпрограмма, которая тут же возвращает управление после перехода в нее), что может оказать небольшое влияние на кэш команд.
3. Во время выполнения, когда трассировщик включает точку трассировки (при этом она может уже использоваться другими действующими трассировщиками):
 - а) в массив обратных вызовов точки трассировки, синхронизированный через RCU, добавляется новый обратный вызов для этого трассировщика;
 - б) если до этого точка трассировки была неактивна, на место инструкции пустой операции записывается переход в обработчик трассировки.
4. Когда трассировщик отключает точку трассировки:
 - а) из массива обратных вызовов точки трассировки, синхронизированного через RCU, удаляется обратный вызов для данного трассировщика;
 - б) если был удален последний обратный вызов, на место статического перехода записывается инструкция пустой операции.

Это уменьшает оверхед на неактивную точку трассировки до минимального уровня.

Если компилятор не поддерживает `asm goto`, используется резервный метод: вместо замены инструкции пустой операции пор инструкцией перехода `jmp` используется условное ветвление, основанное на чтении переменной из памяти.

2.9.3. Интерфейсы точек трассировки

Точки трассировки поддерживают два интерфейса:

- на основе `Ftrace`, через каталог `/sys/kernel/debug/tracing/events`, в котором есть подкаталоги для каждой системы и файлы для каждой точки трассировки (точки трассировки можно включать и отключать, выполняя запись в эти файлы);
- `perf_event_open()`: используется инструментом `perf(1)`, а в последнее время механизмом трассировки в BPF (через `perf_tracepoint pmu`).

2.9.4. BPF и точки трассировки

Точки трассировки делают возможной статическую инструментацию ядра инструментами `BCC` и `bpfttrace`. Их интерфейсы:

- **BCC:** `TRACEPOINT_PROBE()`;
- **bpfttrace:** точки трассировки.

Поддержка точек трассировки в BPF появилась в Linux 4.7. До ее появления я разработал множество инструментов `BCC` и использовал `kprobes`. Это означает, что

в БСС меньше примеров точек трассировки, чем мне хотелось бы, просто из-за порядка, в котором разрабатывалась поддержка.

Интересным примером БСС и точек трассировки может служить инструмент `tcplife(8)`. Он выводит различные детали о сеансах TCP (более подробно рассматривается в главе 10):

```
# tcplife
PID  COMM      LADDR      LPORT RADDR      RPORT TX_KB RX_KB MS
22597 recordProg 127.0.0.1  46644 127.0.0.1  28527 0     0 0.23
3277  redis-serv 127.0.0.1  28527 127.0.0.1  46644 0     0 0.28
22598 curl      100.66.3.172 61620 52.205.89.26 80     0     1 91.79
22604 curl      100.66.3.172 44400 52.204.43.121 80     0     1 121.38
22624 recordProg 127.0.0.1  46648 127.0.0.1  28527 0     0 0.22
[...]
```

Я написал этот инструмент до того, как в ядре Linux появилась подходящая точка трассировки, поэтому использовал `kprobe` в функции ядра `tcp_set_state()`. Подходящая точка трассировки — `sock:inet_sock_set_state` — была добавлена в Linux 4.16. После этого я изменил инструмент, чтобы он поддерживал оба механизма и мог работать со старыми и новыми ядрами. Инструмент определяет две программы — одну для `tracpoints` и одну для `kprobes`, а затем выбирает, какую из них запустить с помощью следующего теста:

```
if (BPF.tracepoint_exists("sock", "inet_sock_set_state")):
    bpf_text += bpf_text_tracepoint
else:
    bpf_text += bpf_text_kprobe
```

В качестве примера из `bpftrace` я приведу однострочный сценарий, который использует точку трассировки `sched:sched_process_exec`, показанную ранее:

```
# bpftrace -e 'tracepoint:sched:sched_process_exec { printf("exec by %s\n", comm); }'
Attaching 1 probe...
exec by ls
exec by date
exec by sleep
^C
```

Этот сценарий для `bpftrace` выводит имена процессов, вызвавших `exec()`.

2.9.5. Неструктурированные точки трассировки в BPF

Алексей Старовойтов разработал новый интерфейс для точек трассировки под названием `BPF_RAW_TRACEPOINT`, который был добавлен в Linux 4.17 в 2018 году. Он позволяет избежать затрат на создание аргументов стабильной точки трассировки, которые могут не потребоваться, и передает в точку трассировки необработанные аргументы. Это напоминает доступ к точкам трассировки, как если бы они были зондами `kprobes`: вы получаете нестабильный API, зато открывается доступ

к большому количеству полей и не приходится платить значительной потерей производительности за трассировку. Также BPF_RAW_TRACEPOINT немного стабильнее, чем kprobes, потому что стабильны имена точек трассировки, нестабильны только аргументы.

Приведя результаты стресс-теста [48], Алексей показал, что производительность с использованием BPF_RAW_TRACEPOINT выше, чем с kprobes и стандартными точками трассировки:

`samples/bpf/test_overhead performance on 1 cpu:`

tracepoint	base	kprobe+bpf	tracepoint+bpf	raw_tracepoint+bpf
task_rename	1.1M	769K	947K	1.0M
urandom_read	789K	697K	750K	755K

Этот интерфейс может быть особенно интересен для технологий, использующих точки трассировки в режиме 24×7 , так как его использование поможет уменьшить overhead на обработку включенных точек трассировки.

2.9.6. Дополнительные источники информации

Дополнительную информацию ищите в файле `Documentation/trace/tracepoints.rst` в исходных текстах ядра Linux, написанном Матье Деснойерсом [47].

2.10. USDT

Статически заданные точки трассировки на уровне пользователя (User-level Statically Defined Tracing, USDT) — это версия точек трассировки для пространства пользователя. Поддержка USDT для BCC была реализована Сашей Гольдштейном, а для bpftrace — мною и Матеусом Марчини (Matheus Marchini).

Есть множество технологий трассировки или журналирования для ПО в пространстве пользователя, и многие приложения реализуют свои средства журналирования событий, которые можно включать при необходимости. Отличие USDT заключается в том, что для его активации требуется внешний системный трассировщик. Точки USDT в приложении не могут быть использованы и ничего не делают без внешнего трассировщика.

Свою популярность технология USDT получила благодаря утилите DTrace от Sun Microsystems, и теперь она доступна во многих приложениях¹. Для Linux был разработан способ использования USDT, основанный на трассировщике SystemTap. Инструменты трассировки BCC и bpftrace используют этот способ и могут использовать события USDT.

¹ В этом есть и моя заслуга: я продвигал технологию USDT, добавляя зонды USDT в Firefox для проверки JavaScript и в другие приложения, и поддерживал применение технологии USDT от других провайдеров.

Во многих приложениях еще можно заметить явные следы, оставшиеся от DTrace: они не компилируют зонды USDT по умолчанию, требуя указать параметр конфигурации, такой как `--enable-dtrace-probes` или `--with-dtrace`.

2.10.1. Добавление поддержки USDT

Зонды USDT можно добавить в приложение с помощью заголовков и инструментов из пакета `systemtap-sdt-dev` или с помощью своих заголовков. Зонды определяются как макросы, которые можно расставить в нужных местах в коде, чтобы создать точки инструментации USDT. В проекте BCC есть пример `examples/usdt_sample`, показывающий реализацию поддержки USDT. Его можно скомпилировать с использованием заголовков из пакета `systemtap-sdt-dev` или из библиотеки Facebook Folly¹ на C++ [11]. В следующем разделе я покажу пример использования Folly.

Folly

Рассмотрим шаги для добавления поддержки USDT с использованием библиотеки Folly:

1. Подключить заголовочный файл в целевом исходном коде:

```
#include "folly/tracing/StaticTracepoint.h"
```

2. Расставить зонды USDT в нужных местах в виде:

```
FOLLY_SDT(provider, name, arg1, arg2, ...)
```

Аргумент «provider» определяет группу зондов, «name» — имя зонда, а «arg1», «arg2» и т. д. — необязательные аргументы. Пример реализации поддержки USDT в BCC содержит такой код:

```
FOLLY_SDT(usdt_sample_lib1, operation_start, operationId,  
request.input().c_str());
```

Он определяет зонд как `usdt_sample_lib1:operation_start` с двумя аргументами. В этом же примере есть зонд `operation_end`.

3. Сборка ПО. Проверить зонд USDT можно с помощью `readelf(1)`:

```
$ readelf -n usdt_sample_lib1/libusdt_sample_lib1.so  
[...]  
Displaying notes found in: .note.stapsdt  
Owner          Data size  Description  
stapsdt        0x00000047 NT_STAPSDT (SystemTap probe descriptors)  
  Provider: usdt_sample_lib1  
  Name: operation_end  
  Location: 0x000000000000fdd2, Base: 0x0000000000000000, Semaphore:  
0x0000000000000000  
  Arguments: -8@%rbx -8@%rax
```

¹ Название Folly расшифровывается как «Facebook Open Source Library».

```

stapsdt                0x0000004f NT_STAPSDT (SystemTap probe descriptors)
  Provider: usdt_sample_lib1
  Name: operation_start
  Location: 0x000000000000febe, Base: 0x0000000000000000, Semaphore:
0x0000000000000000
  Arguments: -8@-104(%rbp) -8@%rax

```

При вызове с параметром `-n` команда `readelf(1)` выводит раздел заметок, в котором должна отображаться информация о скомпилированных зондах USDT.

4. Необязательный шаг. Аргументы, которые хотелось бы добавить в зонд, не всегда есть в готовом к использованию виде в точке его местоположения. Для их получения нужно выполнить дорогостоящие вызовы функций. Чтобы не выполнять эти вызовы, когда зонд не используется, добавьте семафор в исходный код:

```
FOLLY_SDT_DEFINE_SEMAPHORE(provider, name)
```

В этом случае точка размещения зонда приобретает такой вид:

```

if (FOLLY_SDT_IS_ENABLED(provider, name)) {
    ... дорогостоящая обработка аргументов ...
    FOLLY_SDT_WITH_SEMAPHORE(provider, name, arg1, arg2, ...);
}

```

Теперь дорогостоящая обработка аргументов будет производиться, только когда зонд используется (включен). Адрес семафора можно увидеть в выводе `readelf(1)`, и инструменты трассировки могут установить его перед началом использования зонда.

Наличие семафоров, защищающих зонды, немного усложняет работу с инструментами трассировки: им обычно приходится передавать PID процесса, чтобы они установили семафор для этого PID.

2.10.2. Как работает USDT

На этапе компиляции приложения по адресу зонда USDT помещается пустая инструкция (пор). Затем в процессе инструментации ядро динамически заменяет эту инструкцию инструкцией точки останова, используя механизм `uprobes`.

По аналогии с `uprobes`, я могу проиллюстрировать работу USDT, правда, для этого придется приложить чуть больше усилий. В этом случае зонд находится по адресу `0xb2`. Это смещение от начала двоичного сегмента, поэтому чтобы определить полный адрес, нужно также узнать, где начинается сегмент. Адрес начала сегмента может меняться из-за использования формата, независимого от расположения выполняемого кода (Position Independent Executable, PIE), в котором эффективно используется прием рандомизации размещения адресного пространства (Address Space Layout Randomization, ASLR):

```

# gdb -p 4777
[... ]
(gdb) info proc mappings

```

process 4777
Mapped address spaces:

Start Addr	End Addr	Size	Offset	objfile
0x55a75372a000	0x55a75372b000	0x1000	0x0	/home/bgregg/Lang/c/tick
0x55a75392a000	0x55a75392b000	0x1000	0x0	/home/bgregg/Lang/c/tick
0x55a75392b000	0x55a75392c000	0x1000	0x1000	/home/bgregg/Lang/c/tick

[...]

Итак, сегмент начинается с адреса 0x55a75372a000. Вот вывод инструкции, находящийся по этому адресу, плюс смещение зонда 0x6a2:

```
(gdb) disas 0x55a75372a000 + 0x6a2
[...]
0x000055a75372a695 <+11>: mov    %rsi, -0x20(%rbp)
0x000055a75372a699 <+15>: movl   $0x0, -0x4(%rbp)
0x000055a75372a6a0 <+22>: jmp    0x55a75372a6c7 <main+61>
0x000055a75372a6a2 <+24>: nop
0x000055a75372a6a3 <+25>: mov    -0x4(%rbp), %eax
0x000055a75372a6a6 <+28>: mov    %eax, %esi
0x000055a75372a6a8 <+30>: lea    0xb5(%rip), %rdi    # 0x55a75372a764
[...]
```

А вот как выглядит тот же адрес после включения зонда USDT:

```
(gdb) disas 0x55a75372a000 + 0x6a2
[...]
0x000055a75372a695 <+11>: mov    %rsi, -0x20(%rbp)
0x000055a75372a699 <+15>: movl   $0x0, -0x4(%rbp)
0x000055a75372a6a0 <+22>: jmp    0x55a75372a6c7 <main+61>
0x000055a75372a6a2 <+24>: int3
0x000055a75372a6a3 <+25>: mov    -0x4(%rbp), %eax
0x000055a75372a6a6 <+28>: mov    %eax, %esi
0x000055a75372a6a8 <+30>: lea    0xb5(%rip), %rdi    # 0x55a75372a764
[...]
```

На месте инструкции `por` появилась инструкция `int3` (точка останова в архитектуре x86_64). Достигнув ее, ядро выполнит подключенную программу BPF с аргументами для зонда USDT. После отключения зонда инструкция `por` будет восстановлена.

2.10.3. BPF и USDT

Механизм USDT предлагает инструментам BCC и `bpftrace` возможность статической инструментации кода в пространстве пользователя. Его интерфейсы:

- **BCC:** `USDT().enable_probe();`
- **bpftrace:** `usdt.`

Например, вот как инструментировать зонд `loop` из предыдущего примера:

```
# bpftrace -e 'usdt:/tmp/tick:loop { printf("got: %d\n", arg0); }'
Attaching 1 probe...
```

```
got: 0
got: 1
got: 2
got: 3
got: 4
^C
```

Этот однострочный сценарий bpftrace также выводит целочисленный аргумент, переданный в зонд.

2.10.4. Дополнительные источники информации о USDT

Вот список дополнительных источников информации о USDT:

- статья Брендана Грегга «Hacking Linux USDT with Ftrace» [49];
- статья Саши Гольдштейна «USDT Probe Support in BPF/BCC» [50];
- статья Дейла Хамела «USDT Tracing Report» [51].

2.11. ДИНАМИЧЕСКИЙ USDT

Описанные выше зонды USDT добавляются в исходный код и компилируются в исполняемый двоичный файл в виде инструкций пор в точках инструментации и в виде метаданных в разделе примечаний ELF. Но некоторые языки программирования, например Java (с виртуальной машиной JVM), интерпретируют исходный код или компилируют его прямо в процессе выполнения. Для добавления точек инструментации в такой код был разработан динамический механизм USDT.

Обратите внимание, что JVM уже содержит множество зондов USDT в своем коде на C++ — для событий сборки мусора (GC), загрузки классов и других высокоуровневых действий. Эти зонды USDT позволяют инструментировать функции JVM. Но зонды USDT нельзя добавить в код на Java, который компилируется «на лету». USDT предполагает использование предварительно скомпилированного ELF-файла с разделом примечаний, содержащим описания зондов, которого просто нет для кода, создаваемого JIT-компилятором Java.

Динамический механизм USDT решает эту проблему так:

- Предварительно компилируется разделяемая библиотека с требуемыми зондами USDT, встроенными в функции. Эта библиотека может быть написана на C или C++ и в ней должен быть ELF-раздел примечаний с описаниями зондов USDT. Эти зонды можно инструментировать так же, как и любой другой зонд USDT.
- При необходимости библиотека загружается как обычно с помощью `dlopen(3)`.
- В приложение на целевом языке добавляются вызовы функций из этой библиотеки. Эти вызовы могут быть реализованы с помощью API, подходящего для этого языка и скрывающего фактический вызов функции из библиотеки.

Это решение было реализовано для Node.js и Python Матеусом Марчини в библиотеке libstapsdt¹, которая позволяет определять и вызывать зонды USDT из программного кода на этих языках. Поддержку других языков легко добавить, создав обертки для этой библиотеки, как это сделал Дейл Хамел (Dale Hamel) для Ruby, используя поддержку C-расширений для Ruby [54].

Например, в Node.js JavaScript:

```
const USDT = require("usdt");
const provider = new USDT.USDTProvider("nodeProvider");
const probe1 = provider.addProbe("requestStart", "char *");
provider.enable();

[...]
```

```
probe1.fire(function() { return [currentRequestString]; });
```

```
[...]
```

Вызов `probe1.fire()` выполняет свою анонимную функцию, только если зонд был инструментирован извне. При необходимости внутри этой функции можно обрабатывать аргументы перед передачей в зонд, не беспокоясь об оверхеде на вычисления, потому что обработка не выполняется, если зонд не используется.

Библиотека libstapsdt автоматически создает разделяемую библиотеку, содержащую зонды USDT и раздел примечаний, во время выполнения и отображает этот раздел в адресное пространство выполняемой программы.

2.12. PMC

Счетчики мониторинга производительности (Performance Monitoring Counter, PMC), или инструментируемые счетчики производительности (Performance Instrumentation Counter, PIC), счетчики производительности процессора (CPU Performance Counter, CPC) и события модуля мониторинга производительности (Performance Monitoring Unit events, PMU events). Все эти термины обозначают одно и то же: программируемые аппаратные счетчики в процессоре.

Несмотря на большое количество PMC, Intel выбрала семь PMC в качестве «архитектурного набора», обеспечивающего общий обзор некоторых основных функций [Intel 16]. Наличие такого архитектурного набора счетчиков PMC можно проверить с помощью инструкции CPUID. Этот набор полезных счетчиков PMC представлен в табл. 2.8.

Счетчики мониторинга производительности PMC — важный ресурс для анализа производительности. Только с помощью счетчиков PMC можно измерить эффективность инструкций процессора, долю попаданий в кэш процессора, использование

¹ Для libstapsdt см. [52], [53]. С этой целью начата разработка новой библиотеки libusdt, которая может изменить следующий пример кода. Обязательно ознакомьтесь с возможностями более новой версии libusdt.

памяти и шины устройств, число тактов простоя и т. д. Использование этих метрик при анализе производительности поможет в поиске разных небольших оптимизаций производительности.

Таблица 2.8. Архитектурный набор счетчиков PMC от Intel

Имя события	Umask	Селектор события	Пример мнемоники маски события
Непрерывных тактов ядра (UnHalted Core Cycles)	00H	3CH	CPU_CLK_UNHALTED.THREAD_P
Инструкций выбрано (Instruction Retired)	00H	C0H	INST_RETIRED.ANY_P
Опорных тактов (UnHalted Reference Cycles)	01H	3CH	CPU_CLK_THREAD_UNHALTED.REF_XCLK
Обращений к кэшу последнего уровня (LLC References)	4FH	2EH	LONGEST_LAT_CACHE.REFERENCE
Пропусков кэша последнего уровня (LLC Misses)	41H	2EH	LONGEST_LAT_CACHE.MISS
Выбрано инструкций ветвей (Branch Instruction Retired)	00H	C4H	BR_INST_RETIRED.ALL_BRANCHES
Пропусков выборки ветвей (Branch Misses Retired)	00H	C5H	BR_MISP_RETIRED.ALL_BRANCHES

Счетчики PMC — странный ресурс. Несмотря на доступность сотен счетчиков PMC, в процессоре есть лишь фиксированное количество регистров (возможно, всего шесть), которого может оказаться недостаточно для одновременного измерения большого числа счетчиков. Вам придется выбрать, какие счетчики измерять в этих шести регистрах, или циклически переключаться между разными наборами счетчиков PMC. (Инструмент `perf(1)` в Linux автоматически поддерживает такое циклическое переключение.) Другие программные счетчики не страдают этими ограничениями.

2.12.1. Режимы PMC

Счетчики PMC можно использовать в одном из двух режимов:

- **Счет:** в этом режиме счетчики PMC отслеживают частоту событий. Ядро может читать счетчики в любое время, например раз в секунду. Оверхед этого режима практически равен нулю.
- **Выборка при переполнении:** в этом режиме счетчики PMC могут посылать ядру прерывания при появлении трассируемых событий, чтобы ядро могло получить дополнительную информацию. Однако трассируемые события могут происходить миллионы или миллиарды раз в секунду, и отправка прерывания для каждого из них приведет к практически полной остановке системы. Решение

состоит в том, чтобы посылать прерывание, когда количество событий превысит число, заданное программно (например, один раз на каждые 10 000 промахов кэша последнего уровня или на каждый миллион тактов простоя).

Режим выборки при переполнении представляет наибольший интерес для трассировки с помощью BPF, потому что он генерирует события, которые можно инструментировать с использованием своих программ BPF. Оба интерфейса, BCC и bpftrace, поддерживают события PMC.

2.12.2. PEBS

Метод выборки при переполнении может давать неверный адрес инструкции, вызвавшей событие, из-за задержки прерывания (иногда эффект задержки называют «заносом» — «skid») или выполнения инструкции вне очереди. Для профилирования тактов процессора такой занос не является проблемой, и некоторые профилировщики даже намеренно вносят сдвиг, чтобы избежать попадания в одну и ту же точку (или используют частоту выборки со смещением, например, 99 Гц). Но для измерения других событий, таких как промахи кэша последнего уровня, адрес инструкции, ответственной за событие, должен быть точным.

В Intel было разработано решение под названием *точная выборка на основе событий* (*Precise Event-Based Sampling, PEBS*). В решении PEBS используются аппаратные буферы для записи значения регистра указателя инструкций, имевшегося в момент события PMC. Инфраструктура perf_events в Linux поддерживает использование PEBS.

2.12.3. Облачные вычисления

Многие облачные среды пока не предоставляют доступа к PMC своим гостям. Но технически это вполне возможно. Например, гипервизор Xen имеет параметр командной строки `vrmi`, позволяющий открыть гостевой доступ к различным наборам PMC [55]¹. Например, Amazon включила множество счетчиков PMC для своих пользователей гипервизора Nitro.

2.13. PERF_EVENTS

Механизм perf_events используется командой perf(1) для зондирования и трассировки, он был добавлен в Linux 2.6.31 в 2009 году. Важно отметить, что perf (1) и perf_events были объектами пристального внимания и непрерывно развивались долгие годы. Сейчас трассировщики BPF могут обращаться к механизму perf_events

¹ Я написал для Xen код поддержки разных режимов PMC: `iprc` — только для счетчиков PMC подсчета инструкций и `arch` — для архитектурного набора Intel. Мой код просто служит брандмауэром к существующей поддержке `vrmi` в Xen.

и использовать его возможности. Первое время BCC и bpftrace использовали `perf_events` для своего циклического буфера, затем для инструментации счетчиков РМС, а теперь и для всех событий, доступных через `perf_event_open()`.

Инструменты трассировки BPF используют внутренние компоненты `perf(1)`, но для `perf(1)` был разработан и добавлен в BPF свой интерфейс, что превратило `perf(1)` в еще один трассировщик BPF. В отличие от BCC и bpftrace, исходный код `perf(1)` находится в дереве исходных текстов Linux, поэтому `perf(1)` — единственный трассировщик BPF, встроенный в Linux.

Поддержка `perf(1)` в BPF все еще находится в стадии разработки, и ею пока трудно пользоваться. Кроме того, знакомство с ней выходит за рамки этой книги, которая посвящена инструментам BCC и bpftrace. Тем не менее в приложении D вы найдете пример `perf BPF`.

2.14. ИТОГИ

В инструментах оценки производительности BPF используется множество разных технологий, включая расширенный BPF, средства динамической инструментации кода в пространстве ядра и пользователя (`kprobes` и `uprobes`), средства статической трассировки кода в пространстве ядра и пользователя (точки трассировки и маркеры пользователя) и `perf_events`. Механизм BPF способен также извлекать трассировки стека, используя прием обхода списка фреймов или ORC, которые можно визуализировать в виде флэйм-графиков. Эти технологии мы кратко рассмотрели в этой главе; я также привел ссылки на источники дополнительной информации.

Глава 3

АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ

https://t.me/it_boooks/2

Инструменты из этой книги можно использовать для анализа производительности, устранения неполадок, оценки безопасности и многого другого. Чтобы вы поняли, как все это делается, в этой главе я дам краткое введение в анализ производительности.

Цели обучения:

- основные цели анализа производительности и приемы, используемые для их достижения;
- определение характера рабочей нагрузки;
- метод USE;
- анализ с последовательным увеличением детализации (drill-down);
- методика чек-листа;
- быстрый поиск узких мест с использованием традиционных инструментов и чек-листа инструментов Linux для анализа за 60 секунд;
- быстрый поиск узких мест с использованием чек-листа инструментов BCC/BPF.

Эта глава начинается с описания целей и приемов анализа производительности, а затем перечисляет методики и традиционные (не BPF) инструменты, которые можно применить в первую очередь. Традиционные инструменты помогут быстро найти узкие места или получить подсказки для последующего анализа с использованием BPF. В конце главы приводится чек-лист инструментов BPF. Другие инструменты BPF описываются в последующих главах.

3.1. ОБЗОР

Прежде чем углубиться в анализ производительности, перечислим преследуемые цели, а также приемы, которые помогут в их достижении.

3.1.1. Цели

Основная цель — увеличение производительности конечного пользователя и уменьшение эксплуатационных расходов. Из этого следует, что должна быть некоторая метрика, которая поможет определить факт достижения цели или количественно оценить нехватку чего-то. К таким метрикам можно отнести:

- **величину задержки:** время выполнения запроса или операции, обычно измеряется в миллисекундах;
- **частоту:** частоту операций или запросов в секунду;
- **пропускную способность:** объем данных, перемещаемых в единицу времени, обычно битов или байтов в секунду;
- **коэффициент использования:** занятость ресурса в процентах с течением времени;
- **стоимость:** соотношение цена/производительность.

Производительность конечного пользователя можно определить как время, которое требуется приложению для ответа на запросы пользователя. Цель состоит в том, чтобы уменьшить это время. Время ожидания ответа часто называют *задержкой* (latency). Его можно улучшить, проанализировав, из чего складывается время обработки запроса, и разбив его на компоненты: время выполнения кода на процессоре, время ожидания ресурсов (диски, сети и блокировки), время ожидания, пока планировщик выделит процессор, и т. д. Можно написать инструмент BPF, непосредственно определяющий время выполнения запроса приложением, плюс задержки, генерируемые множеством разных компонентов. Но такой инструмент будет узкоспециализированным для этого приложения и повлечет значительные оверхеды при одновременном отслеживании множества различных событий. На практике чаще используются небольшие и более конкретные инструменты, позволяющие определить время и задержки, порождаемые конкретными компонентами. В этой книге описано множество таких небольших и специализированных инструментов.

Сокращение эксплуатационных расходов включает наблюдение за использованием программных и аппаратных ресурсов и поиск приемов оптимизации, позволяющих сократить расходы вашей компании на пользование облачными услугами или услугами центров обработки данных. Для этого может потребоваться провести анализ другого типа, например регистрацию и анализ особенностей использования компонентов, а не время (задержку) обработки запросов. Многие инструменты в этой книге поддерживают и эту цель.

В попытках провести анализ производительности с использованием инструментов BPF легко сгенерировать гигантский массив чисел, а затем часами пытаться определить природу некоторой метрики, чтобы в итоге понять, что она совершенно неважна. Разработчики, обеспокоенные явно плохими показателями, нередко присылали мне скриншоты вывода инструментов. Первый вопрос, который я задавал: «У вас есть явные проблемы с производительностью?» Многие из них отвечали: «Нет, мы просто подумали, что этот результат выглядит... необычно». Да, результат

действительно может выглядеть необычно, но сначала нужно определить цель: должны ли мы уменьшить задержку в обработке запросов или эксплуатационные расходы? Цель определяет контекст дальнейшего анализа.

3.1.2. Действия

Инструменты BPF оценки производительности можно использовать не только для анализа конкретной проблемы. Ниже приведен список действий, направленных на улучшение производительности [Gregg 13b], и указано, как инструменты BPF могут пригодиться в каждом из этих случаев:

	Действие, связанное с оценкой производительности	Инструменты BPF
1	Определение характеристик ПО или оборудования	Создание гистограмм задержек при различных нагрузках
2	Анализ производительности разрабатываемого кода перед интеграцией	Для определения узких мест и поиска оптимизаций, увеличивающих общую производительность
3	Выполнение нерегрессионного тестирования сборок ПО до и после релиза	Для определения особенностей использования кода и различных источников задержек, помогающих быстро обнаружить причины регрессии
4	Бенчмаркинг и бенчмаркетинг для релизов ПО	Для изучения производительности с целью найти возможности улучшения показателей
5	Проверка концепции (PoC) в целевой среде	Для создания гистограмм задержек с целью определить соответствие производительности соглашениям об уровне обслуживания
6	Мониторинг ПО в промышленной среде	Для создания инструментов, которые могут работать круглосуточно и определять метрики, недоступные иначе
7	Анализ производительности при наличии явных проблем	Для решения конкретной проблемы производительности с возможностью инструментации пользовательского кода, если потребуется

Очевидно, что многие инструменты, представленные в этой книге, подходят для изучения перечисленных проблем, но они могут использоваться и для мониторинга, и для нерегрессионного тестирования, и в других случаях.

3.1.3. Многочисленные проблемы производительности

Используя инструменты, описанные в этой книге, будьте готовы обнаружить многочисленные проблемы с производительностью. В такой ситуации важно определить, какая проблема имеет наибольшее значение, то есть вносит наибольший вклад в задержку или стоимость запроса. Если вы не предполагаете обнаружить множество проблем, зайдите в баг-трекер базы данных, файловой системы или программного компонента и выполните поиск по слову «performance» («производительность»). ПО часто имеет многочисленные проблемы с производительностью, часть из

которых может даже не упоминаться в трекере. Самое важное — найти наиболее существенную проблему.

У любой проблемы может быть несколько причин. Часто, когда вы устраняете одну, наружу вылезают другие, или же после устранения одной причины обнаруживается проблема в другом компоненте.

3.2. МЕТОДОЛОГИИ ОЦЕНКИ ПРОИЗВОДИТЕЛЬНОСТИ

Имея так много инструментов оценки производительности (например, kprobes, uprobes, точки трассировки, USDT, PMC, как рассказывалось в главе 2), порой трудно понять, что делать с разнообразными данными, которые они дают. В течение многих лет я изучал, создавал и документировал методологии оценки производительности. *Методология* — это процесс, посредством которого можно определить направление дальнейших исследований, выполняемые шаги и конечную точку. Моя предыдущая книга «Systems Performance»¹ описывает десятки методологий [Gregg 13b]. В этом разделе я перечислю те из них, которые можно использовать вместе с инструментами BPF.

3.2.1. Определение характера рабочей нагрузки

Цель определения характеристик рабочей нагрузки — понять, какая нагрузка применяется. При этом не требуется оценивать показатели производительности, такие как задержки. По моему личному опыту, наибольший выигрыш дает «устранение ненужной работы». Такую ненужную работу можно найти, изучив, из чего состоит рабочая нагрузка.

Шаги для определения характера рабочей нагрузки:

1. Определить источник нагрузки (например, PID, имя процесса, UID, IP-адрес).
2. Определить, как формируется нагрузка (путь выполнения кода, трассировка стека, флейм-график).
3. Определить вид нагрузки (число операций ввода/вывода в секунду, пропускная способность, тип).
4. Определить характер изменения нагрузки с течением времени (поинтервальные метрики).

Многие из инструментов, представленных в этой книге, помогут ответить на эти вопросы. Один из таких инструментов — `vfsstat(8)`:

```
# vfsstat
TIME      READ/s    WRITE/s   CREATE/s   OPEN/s    FSYNC/s
18:35:32:    231        12         4         98        0
18:35:33:    274        13         4        106        0
```

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

```
18:35:34:      586      86      4      251      0
18:35:35:      241      15      4      99      0
18:35:36:      232     10      4      98      0
[...]
```

Он выводит подробную информацию о рабочей нагрузке на уровне виртуальной файловой системы (Virtual File System, VFS) и решает задачу шага 3, подсказывая типы операций и скорость их выполнения, а также задачу шага 4, сообщая характеристики в разные интервалы времени.

Чтобы привести простой пример реализации шага 1, покажу однострочный сценарий для bpftrace (здесь приводится неполный вывод):

```
# bpftrace -e 'kprobe:vfs_read { @[comm] = count(); }'
Attaching 1 probe...
^C
```

```
@[rtkit-daemon]: 1
[...]
@[gnome-shell]: 207
@[Chrome_IOThread]: 222
@[chrome]: 225
@[InputThread]: 302
@[gdbus]: 819
@[Web Content]: 1725
```

Этот вывод показывает, что процесс «Web Content» выполнил 1725 вызовов `vfs_read()`, пока я проводил трассировку.

В этой книге будут приводиться дополнительные примеры инструментов для выполнения этих шагов, включая создание флейм-графиков для шага 2.

Если для целей вашего анализа нет доступных инструментов, создайте свои собственные средства определения характера рабочей нагрузки.

3.2.2. Анализ с последовательным увеличением детализации

Анализ с последовательным увеличением детализации (drill-down) предполагает исследование некоторой метрики и поиск способов ее декомпозиции, а затем декомпозиция самого большого компонента и так далее, пока не будет найдена основная причина или причины.

Попробуем провести аналогию. Представьте, что вы обнаружили, что с вашей банковской карты списали большую сумму. Чтобы понять, куда ушли деньги, вы входите в личный кабинет на сайте банка и выводите список транзакций. Там вы обнаруживаете один большой платеж в книжном интернет-магазине. Вы переходите на сайт книжного магазина, чтобы увидеть, на какие книги была потрачена эта сумма, и обнаруживаете, что случайно приобрели 1000 экземпляров моей книги (спасибо вам за это!). Это пример анализа с последовательным увеличением детализации:

поиск подсказки, углубление, обнаружение дальнейших подсказок и так далее, пока проблема не будет решена.

Основные шаги при приведении такого анализа:

1. Начать исследования с самого верхнего уровня.
2. Исследовать проблему на следующем уровне детализации.
3. Исследовать наиболее интересный компонент или подсказку.
4. Если проблема не решена, вернуться к шагу 2.

Для подобного анализа могут привлекаться пользовательские инструменты, лучше подходящие для bpftrace, чем для ВСС.

Один из видов drill-down-анализа включает разложение задержки на компоненты. Представьте такую последовательность анализа:

1. Задержка выполнения запроса составляет 100 мс (миллисекунд).
2. Из них 10 мс запрос обрабатывается на процессоре, а 90 мс тратится на ожидание.
3. Из всего времени ожидания 89 мс приходится на ожидание файловой системы.
4. Файловая система тратит 3 мс на ожидание освобождения блокировки и 86 мс на ожидание готовности устройства хранения.

Проанализировав эти результаты, можно прийти к выводу, что проблема кроется в устройстве хранения, но это будет лишь один из ответов. Drill-down-анализ также можно использовать для уточнения контекста. Рассмотрим альтернативную последовательность:

1. Приложение тратит 89 мс на ожидание файловой системы.
2. Файловая система тратит 78 мс на запись файла и 11 мс — на чтение.
3. Из времени на запись файловая система тратит 77 мс на ожидание доступа к отметкам времени для обновления.

Теперь можно прийти к выводу, что проблема кроется в доступе к отметкам времени в файловой системе и их можно отключить (это параметр монтирования файловой системы). Это более грамотное заключение, чем вывод о необходимости более быстрых дисков.

3.2.3. Метод USE

Для анализа ресурсов я разработал методологию USE [Gregg 13c].

По этой методологии для каждого ресурса проверяются:

- 1) коэффициент использования (Utilization);
- 2) насыщение (Saturation);
- 3) наличие ошибок (Errors).

Первая задача — найти или нарисовать схему программных и аппаратных ресурсов, а затем последовательно перебрать их и определить эти три метрики. На рис. 3.1 показан пример аппаратного обеспечения типичной системы, включая компоненты и шины, которые можно проверить.

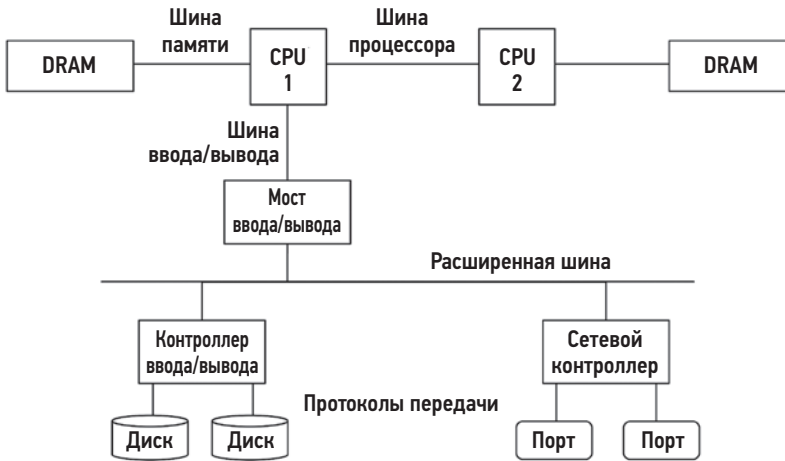


Рис. 3.1. Аппаратные компоненты для анализа методом USE

Проверьте имеющиеся у вас инструменты мониторинга и их способность определять коэффициент использования (utilization), насыщенность (saturation) и наличие ошибок (errors) для каждого элемента на рис. 3.1. Какие компоненты вы не можете оценить?

Преимущество этой методологии в том, что она начинается с вопросов, имеющих значение, а не с ответов в форме метрик и попытки пойти в обратном направлении, чтобы выяснить, почему они важны. Она также выявляет «слепые пятна»: сначала определяет вопросы, на которые хотелось бы получить ответы, независимо от наличия удобного инструмента для измерения.

3.2.4. Чек-листы

Чек-лист оценки производительности содержит список используемых инструментов и измеряемых параметров. Основу чек-листа составляют с десяток распространенных проблем и инструкции по их выявлению. Они хорошо подходят для выполнения самыми разными сотрудниками вашей компании и позволяют масштабировать свои навыки.

В разделах ниже приведены примеры двух чек-листов: один основан на использовании традиционных (не BPF) инструментов и может применяться для быстрого анализа (в первые 60 секунд), а другой содержит инструменты ВСС, которые желательно применить на ранних этапах.

3.3. ЧЕК-ЛИСТ ИНСТРУМЕНТОВ LINUX ДЛЯ АНАЛИЗА ЗА 60 СЕКУНД

Этот чек-лист можно использовать для анализа любых проблем с производительностью. Он отражает мои действия, которые я обычно выполняю в первые 60 секунд после входа в систему Linux. Чек-лист был опубликован мной и командой разработчиков Netflix [56]:

Инструменты:

1. `uptime`
2. `dmesg | tail`
3. `vmstat 1`
4. `mpstat -P ALL 1`
5. `pidstat 1`
6. `iostat -xz 1`
7. `free -m`
8. `sar -n DEV 1`
9. `sar -n TCP,ETCP 1`
10. `top`

В разделах ниже рассказано, как использовать каждый из этих инструментов. Может показаться неуместным в книге о BPF обсуждать инструменты, не относящиеся к BPF. Но если этого не сделать, мы рискуем упустить из виду важный доступный ресурс. Эти команды помогут решить некоторые проблемы с производительностью или хотя бы навести на мысль, где они таятся, направляя вас к использованию последующих инструментов BPF, помогающих найти корни проблемы.

3.3.1. uptime

```
$ uptime
03:16:59 up 17 days,  4:18,  1 user,  load average: 2.74, 2.54, 2.58
```

Эта команда позволяет быстро оценить величину средней нагрузки и узнать число задач (процессов), ожидающих в очереди на выполнение. В Linux в это число входят процессы, ожидающие выделения им процессорного времени, а также процессы, заблокированные в непрерываемых операциях ввода/вывода (обычно дискового ввода/вывода). Эта информация позволяет получить общее представление о нагрузке на ресурсы (или спросе), которую затем можно изучить с помощью других инструментов.

Команда выводит три числа — экспоненциально сглаженные скользящие средние с временным окном 1, 5 и 15 минут. Эти числа дают представление о том, как

меняется нагрузка со временем. В примере выше числа показывают небольшое увеличение средней нагрузки в последнюю минуту.

Средние значения нагрузки желательно проверить при первом же проявлении проблемы, чтобы выяснить, сохраняется ли она. В отказоустойчивых средах сервер, испытывающий проблемы с производительностью, может быть автоматически удален из службы к тому времени, когда вы зайдете в систему для ее анализа. Высокая средняя нагрузка за 15 минут в сочетании с низкой средней нагрузкой в 1 минуту может быть признаком того, что вы вошли в систему слишком поздно, чтобы заметить проблему.

3.3.2. dmesg | tail

```
$ dmesg | tail
[1880957.563150] perl invoked oom-killer: gfp_mask=0x280da, order=0, oom_score_adj=0
[...]
[1880957.563400] Out of memory: Kill process 18694 (perl) score 246 or sacrifice child
[1880957.563408] Killed process 18694 (perl) total-vm:1972392kB, anon-rss:1953348kB,
file-rss:0kB
[2320864.954447] TCP: Possible SYN flooding on port 7001. Dropping request. Check
SNMP counters.
```

Эта команда выведет 10 последних системных сообщений, если они есть. Поищите в этих строках ошибки, которые могут вызвать проблемы с производительностью. Например, в этом выводе видна принудительная остановка приложения из-за нехватки памяти и удаление запроса TCP. Сообщение TCP даже подсказывает направление для дальнейшего анализа: счетчики SNMP.

3.3.3. vmstat 1

```
$ vmstat 1
procs -----memory----- ---swap-- -----io----- -system-- -----cpu-----
 r  b swpd  free  buff  cache  si  so    bi    bo    in  cs us sy id wa st
34  0    0 200889792 73708 591828  0  0    0    0    5   6 10 96  1  3  0  0
32  0    0 200889920 73708 591860  0  0    0    0   592 13284 4282 98  1  1  0  0
32  0    0 200890112 73708 591860  0  0    0    0   9501 2154 99  1  0  0  0
[...]
```

Этот инструмент выводит статистику виртуальной памяти и первоначально был создан в BSD. Он также выводит другие системные показатели. При вызове с аргументом 1 он выводит 1-секундные сводки. Имейте в виду, что первая строка чисел — это сводка за период, *прошедший с момента загрузки* (кроме счетчиков памяти).

Столбцы для проверки:

- **r**: количество запущенных процессов, ожидающих очереди для выполнения на процессоре. Этот параметр дает более полное представление о насыщенности

процессора, чем средние значения нагрузки, так как не включает процессы, ожидающие завершения ввода/вывода. Если значение «г» больше числа процессоров, это указывает на высокую насыщенность.

- **free:** объем свободной памяти в килобайтах. Если в этом поле отображается слишком много цифр, это может означать, что в системе достаточно свободной памяти. Команда `free -m`, которая описана в разделе 3.3.7, лучше объясняет состояние свободной памяти.
- **si и so:** объем памяти, загруженной (swap-in) из устройства подкачки и выгруженной (swap-out) в него. Если эти столбцы содержат ненулевые значения, значит, в системе недостаточно памяти. Используются, только если настроены устройства подкачки.
- **us, sy, id, wa и st:** разбивка процессорного времени в среднем по всем имеющимся процессорам. Это время выполнения в пространстве пользователя, системное время (в пространстве ядра), время бездействия, ожидания ввода/вывода и утраченное время (отданное другим гостевым системам или, через гипервизор Xen, драйверам в изолированном домене).

Этот пример показывает, что основное процессорное время расходуется на выполнение кода в пространстве пользователя. Это наталкивает на мысль, что следует перейти к анализу действующего кода с использованием профилировщиков.

3.3.4. mpstat -P ALL 1

```
$ mpstat -P ALL 1
[...]
```

	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
03:16:41 AM	all	14.27	0.00	0.75	0.44	0.00	0.00	0.06	0.00	0.00	84.48
03:16:42 AM	0	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
03:16:42 AM	1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
03:16:42 AM	2	8.08	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	91.92
03:16:42 AM	3	10.00	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.00	88.00
03:16:42 AM	4	1.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	98.99
03:16:42 AM	5	5.10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	94.90
03:16:42 AM	6	11.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	89.00
03:16:42 AM	7	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	90.00

```
[...]
```

Эта команда выводит разбивку процессорного времени для каждого процессора по состояниям. Здесь можно наблюдать проблему: на процессоре 0 достигнут уровень нагрузки 100% в пространстве пользователя, что говорит об узком месте в одном потоке.

Обратите внимание и на высокое время `%iowait`, которое можно отдельно исследовать с помощью инструментов анализа дискового ввода/вывода, и на высокое время `%sys`, которое можно исследовать с помощью системных вызовов и трассировки ядра, а также инструментов профилирования процессора.

3.3.5. pidstat 1

```
$ pidstat 1
Linux 4.13.0-19-generic (...)      08/04/2018      _x86_64_      (16 CPU)
03:20:47 AM  UID      PID      %usr  %system  %guest    %CPU  CPU  Command
03:20:48 AM  0        1307     0.00   0.98    0.00    0.98   8  irqbalance
03:20:48 AM  33       12178   4.90   0.00    0.00    4.90   4  java
03:20:48 AM  33       12569  476.47 24.51   0.00   500.98  0  java
03:20:48 AM  0       130249   0.98   0.98    0.00    1.96   1  pidstat

03:20:48 AM  UID      PID      %usr  %system  %guest    %CPU  CPU  Command
03:20:49 AM  33       12178   4.00   0.00    0.00    4.00   4  java
03:20:49 AM  33       12569  331.00 21.00   0.00   352.00  0  java
03:20:49 AM  0       129906   1.00   0.00    0.00    1.00   8  sshd
03:20:49 AM  0       130249   1.00   1.00    0.00    2.00   1  pidstat

03:20:49 AM  UID      PID      %usr  %system  %guest    %CPU  CPU  Command
03:20:50 AM  33       12178   4.00   0.00    0.00    4.00   4  java
03:20:50 AM  113      12356   1.00   0.00    0.00    1.00  11  snmp-pass
03:20:50 AM  33       12569  210.00 13.00   0.00   223.00  0  java
03:20:50 AM  0       130249   1.00   0.00    0.00    1.00   1  pidstat
[...]
```

pidstat(1) выводит информацию о потреблении процессорного времени каждым процессом. Для этой цели чаще используется другой популярный инструмент — top(1), но в отличие от него pidstat(1) по умолчанию выводит информацию с прокруткой, что позволяет увидеть изменения с течением времени. Этот вывод показывает, что процесс Java каждую секунду потребляет разное количество процессорного времени. Проценты суммируются для всех процессоров¹, поэтому 500% эквивалентны 100% загрузке пяти процессоров.

3.3.6. iostat -xz 1

```
$ iostat -xz 1
Linux 4.13.0-19-generic (...)      08/04/2018      _x86_64_      (16 CPU)
[...]
```

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle			
	22.90	0.00	0.82	0.63	0.06	75.59			

Device:	rrqm/s	wrqm/s	r/s	w/s	rkB/s	wkB/s	avgrq-sz	avgqu-sz
await r_await w_await svctm %util								
nvme0n1	0.00	1167.00	0.00	1220.00	0.00	151293.00	248.02	2.10
1.72 0.00 1.72 0.21 26.00								
nvme1n1	0.00	1164.00	0.00	1219.00	0.00	151384.00	248.37	0.90
0.74 0.00 0.74 0.19 23.60								

¹ Обратите внимание, что недавно в pidstat(1) было внесено изменение, ограничивающее сумму 100% [36]. Это привело к тому, что для многопоточных приложений, потребляющих больше 100%, результат получался неправильным. В конечном итоге изменение было отменено, но имейте в виду, что если вы увидите такое ограничение, это будет означать, что вы используете измененную версию pidstat(1).

```
md0      0.00      0.00      0.00 4770.00      0.00 303113.00 127.09      .00
0.00      0.00      0.00      0.00      0.00
[...]
```

Этот инструмент выводит метрики ввода/вывода устройства хранения. Строки в выводе не уместились по ширине страницы, поэтому их пришлось перенести, что затрудняет чтение.

Столбцы для проверки:

- **r/s, w/s, kB/s и kB/s:** число выполненных операций чтения и записи, объем прочитанных и записанных данных в килобайтах в секунду для устройства. Характеризуют рабочую нагрузку. Проблема производительности может быть всего лишь следствием чрезмерной нагрузки.
- **await:** средняя продолжительность ввода/вывода в миллисекундах. Это время, которое затрачивает приложение, состоящее из времени ожидания в очереди и времени обслуживания. Среднее время ожидания, превышающее предполагаемое, говорит о насыщении устройства или о проблемах с ним.
- **avgqu-sz:** среднее количество запросов, отправленных устройству. Значения, превышающие единицу, могут говорить о насыщении (хотя устройства, особенно виртуальные устройства, расположенные на нескольких внутренних дисках, обычно работают по запросам параллельно).
- **%util:** коэффициент использования устройства. В действительности этот процент загруженности показывает время, в течение которого устройство было занято работой каждую секунду. Это не коэффициент использования в смысле планирования емкости, поскольку устройства могут выполнять запросы параллельно¹. Обычно значения, превышающие 60%, вызывают снижение производительности (что можно видеть в столбце await), хотя многое зависит от самого устройства. Значения, близкие к 100%, обычно указывают на насыщенность.

Пример выше показывает рабочую нагрузку записи ~300 Мбайт/с на виртуальное устройство md0, которое, похоже, основывается на двух физических устройствах nvme.

3.3.7. free -m

```
$ free -m
          total        used        free      shared  buff/cache   available
Mem:      122872       39158       3107        1166        80607        81214
Swap:            0            0            0
```

¹ Это вызывает путаницу, когда устройство со 100%-ной загрузкой по данным `iostat(1)` может принять более высокую рабочую нагрузку. Эта метрика просто сообщает, что устройство было занято 100% времени, а не использовалось на 100%: оно могло бы справиться с большим объемом работы. Столбец `%util` в выводе `iostat(1)` может особенно сильно вводить в заблуждение в отношении томов, поддерживаемых пулом из нескольких дисков, имеющих повышенную способность выполнять работу параллельно.

Эта команда выводит информацию о доступной памяти в мегабайтах. Значение в столбце `available` (доступно) не должно быть близко к нулю. Оно показывает, какой объем реальной памяти доступен системе, включая буферы и кэши страниц¹. Наличие некоторой памяти, отведенной для кэша, улучшает производительность файловой системы.

3.3.8. `sar -n DEV 1`

```
$ sar -n DEV 1
Linux 4.13.0-19-generic (...)      08/04/2018      _x86_64_      (16 CPU)

03:38:28 AM      IFACE      rxpck/s      txpck/s      rxkB/s      txkB/s      rxcmp/s      txcmp/s
rxmcsst/s %ifutil
03:38:29 AM      eth0      7770.00      4444.00      10720.12      5574.74      0.00      0.00
0.00      0.00
03:38:29 AM      lo      24.00      24.00      19.63      19.63      0.00      0.00
0.00      0.00
03:38:29 AM      IFACE      rxpck/s      txpck/s      rxkB/s      txkB/s      rxcmp/s      txcmp/s
rxmcsst/s %ifutil
03:38:30 AM      eth0      5579.00      2175.00      7829.20      2626.93      0.00      0.00
0.00      0.00
03:38:30 AM      lo      33.00      33.00      1.79      1.79      0.00      0.00
0.00      0.00
[...]
```

Утилита `sar(1)` поддерживает множество режимов для вывода разных групп метрик. Здесь я использовал ее для получения информации о сетевых интерфейсах. По значениям в столбцах `rxkB/s` и `txkB/s` можно судить, был ли достигнут соответствующий предел пропускной способности сетевого интерфейса.

3.3.9. `sar -n TCP,ETCP 1`

```
# sar -n TCP,ETCP 1
Linux 4.13.0-19-generic (...)      08/04/2019      _x86_64_      (16 CPU)

03:41:01 AM      active/s      passive/s      iseg/s      oseg/s
03:41:02 AM      1.00      1.00      348.00      1626.00

03:41:01 AM      atmptf/s      estres/s      retrans/s      isegerr/s      orsts/s
03:41:02 AM      0.00      0.00      1.00      0.00      0.00

03:41:02 AM      active/s      passive/s      iseg/s      oseg/s
03:41:03 AM      0.00      0.00      521.00      2660.00
```

¹ Вывод команды `free(1)` недавно изменился. Раньше буферы и кэш отображались в отдельных столбцах, а объем доступной памяти (столбец `available`) не выводился и должен был вычисляться конечным пользователем. Мне больше нравится новая версия. Отдельные столбцы для буферов и кэшей можно вывести, добавив ключ `-w`, включающий режим широкого (wide) вывода.

```
03:41:02 AM atmpth/s  estres/s  retrans/s  isegerr/s  orsts/s
03:41:03 AM      0.00      0.00      0.00      0.00      0.00
[...]
```

На этот раз я использовал утилиту `sar(1)` для получения характеристик и ошибок TCP.

Столбцы для проверки:

- **active/s**: количество TCP-соединений, инициированных локально в течение последней секунды (например, вызовом `connect()`);
- **passive/s**: количество TCP-соединений, инициированных удаленной стороной в течение последней секунды (например, вызовом `accept()`);
- **retrans/s**: число повторных попыток передачи пакетов TCP в течение последней секунды.

Счетчики активных (`active/s`) и пассивных (`passive/s`) соединений можно использовать для определения характера рабочей нагрузки.

3.3.10. top

```
top - 03:44:14 up 17 days,  4:46,  1 user,  load average: 2.32, 2.20, 2.21
Tasks: 474 total,   1 running, 473 sleeping,   0 stopped,   0 zombie
%Cpu(s): 29.7 us,  0.4 sy,  0.0 ni, 69.7 id,  0.1 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 12582137+total,  3159704 free, 40109716 used, 82551960 buff/cache
KiB Swap:   0 total,      0 free,      0 used. 83151728 avail Mem

   PID USER      PR  NI   VIRT   RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 12569 www      20   0  2.495t 0.051t 0.018t S 484.7 43.3 13276.02 java
 12178 www      20   0 12.214g 3.107g 16540 S   4.9  2.6  553:41 java
125312 root      20   0      0      0      0 S   1.0  0.0   0:13.20 kworker/u256:0

128697 root      20   0      0      0      0 S   0.3  0.0   0:02.10 kworker/10:2
[...]
```

Многие из этих метрик вы уже видели в примерах использования других инструментов, но иногда полезно выполнить повторную проверку, завершив анализ вызовом утилиты `top(1)` и взглянув на текущие параметры работы системы и процессов.

Если повезет, этот 60-секундный анализ поможет подметить некоторые особенности функционирования системы. Вы можете использовать эти подсказки для перехода к соответствующим инструментам BPF для дальнейшего анализа.

3.4. ЧЕК-ЛИСТ ИНСТРУМЕНТОВ ВСС

Этот чек-лист написан мной [30] и находится в репозитории ВСС, в файле `docs/tutorial.md`. Это универсальный чек-лист инструментов ВСС для продолжения анализа системы:

1. `execsnoop`
2. `opensnoop`
3. `ext4slower` (or `btrfs*`, `xfs*`, `zfs*`)
4. `biolatency`
5. `biosnoop`
6. `cachestat`
7. `tcpconnect`
8. `tcpaccept`
9. `tcpretrans`
10. `runqlat`
11. `profile`

Эти инструменты дают дополнительную информацию о новых процессах, открытых файлах, задержках в файловой системе, задержках в дисковом вводе/выводе, производительности кэша файловой системы, TCP-соединениях и повторных попытках передачи пакетов, задержках планировщика, а также нагрузке на процессор. Подробно о них мы поговорим в последующих главах.

3.4.1. `execsnoop`

```
# execsnoop
PCOMM      PID    RET  ARGS
supervise   9660    0    ./run
supervise   9661    0    ./run
mkdir       9662    0    /bin/mkdir -p ./main
run         9663    0    ./run
[...]
```

Команда `execsnoop(8)` сообщает о запуске новых процессов, выводя одну строку всякий раз, когда выполняется системный вызов `execve(2)`. Особое внимание уделяйте короткоживущим процессам, так как они могут потреблять значительные вычислительные ресурсы, но не отображаться большинством инструментов мониторинга, которые получают списки выполняющихся процессов через равные интервалы времени. Более подробно `execsnoop(8)` рассматривается в главе 6.

3.4.2. `opensnoop`

```
# opensnoop
PID  COMM      FD  ERR  PATH
1565  redis-server  5   0    /proc/1565/stat
1603  snmpd       9   0    /proc/net/dev
1603  snmpd      11   0    /proc/net/if_inet6
1603  snmpd     -1   2    /sys/class/net/eth0/device/vendor
1603  snmpd      11   0    /proc/sys/net/ipv4/neigh/eth0/retrans_time_ms
```

```

1603  snmpd          11    0 /proc/sys/net/ipv6/neighbor/eth0/retrans_time_ms
1603  snmpd          11    0 /proc/sys/net/ipv6/conf/eth0/forwarding
[...]
```

Всякий раз, когда выполняется системный вызов `open(2)` (или его варианты), `opensnoop(8)` выводит одну строку, включающую подробную информацию о том, какой путь был открыт и был ли он успешным (столбец ошибок ERR). Информация об открываемых файлах многое говорит о работе приложений: какие файлы данных, конфигураций и журналов они используют. Иногда приложения показывают низкую производительность и действуют неправильно, постоянно пытаются прочитать несуществующие файлы. Более подробно `opensnoop(8)` рассматривается в главе 8.

3.4.3. ext4slower

```

# ext4slower
Tracing ext4 operations slower than 10 ms
TIME      COMM          PID    T BYTES  OFF_KB   LAT(ms)  FILENAME
06:35:01  cron             16464  R 1249   0        16.05    common-auth
06:35:01  cron             16463  R 1249   0        16.04    common-auth
06:35:01  cron             16465  R 1249   0        16.03    common-auth
06:35:01  cron             16465  R 4096   0        10.62    login.defs
06:35:01  cron             16464  R 4096   0        10.61    login.defs
[...]
```

`ext4slower(8)` отслеживает выполнение обычных операций в файловой системе `ext4` (чтение, запись, открытие и синхронизация) и сообщает о тех, у которых продолжительность выполнения превышает определенный порог. С помощью этого инструмента можно выявить проблему одного типа: когда приложение слишком долго ждет завершения операции ввода/вывода в файловой системе на отдельном медленном диске. Есть варианты `ext4slower(8)` для других файловых систем, включая `btrfs slower(8)`, `xfs slower(8)` и `zfs slower(8)`. Более подробную информацию ищите в главе 8.

3.4.4. biolatency

```

# biolatency -m
Tracing block device I/O... Hit Ctrl-C to end.
^C
```

msecs	: count	distribution
0 -> 1	: 16335	*****
2 -> 3	: 2272	*****
4 -> 7	: 3603	*****
8 -> 15	: 4328	*****
16 -> 31	: 3379	*****
32 -> 63	: 5815	*****
64 -> 127	: 0	
128 -> 255	: 0	
256 -> 511	: 0	
512 -> 1023	: 1	

`biolatency(8)` отслеживает задержки дискового ввода/вывода (то есть время от момента обращения к устройству до завершения операции) и выводит их в виде гистограммы. Этот инструмент помогает получить более полное представление о дисковом вводе/выводе, чем средние значения, которые выводит `iostat(1)`. Иногда можно видеть несколько модальных значений — значений, встречающихся чаще других в распределении, и в этом примере как раз показано такое мультимодальное распределение. Одно модальное значение приходится на диапазон от 0 до 1 миллисекунды, а другое — на диапазон от 8 до 15 миллисекунд¹. В выводе также хорошо видны выбросы: в этом примере есть один выброс в диапазоне от 512 до 1023 миллисекунд. Более подробно `biolatency(8)` рассматривается в главе 9.

3.4.5. biosnoop

```
# biosnoop
TIME(s)      COMM      PID    DISK    T  SECTOR  BYTES  LAT(ms)
0.000004001  supervise  1950   xvda1  W  13092560 4096    0.74
0.000178002  supervise  1950   xvda1  W  13092432 4096    0.61
0.001469001  supervise  1956   xvda1  W  13092440 4096    1.24
0.001588002  supervise  1956   xvda1  W  13115128 4096    1.09
1.022346001  supervise  1950   xvda1  W  13115272 4096    0.98
[...]
```

`biosnoop(8)` выводит для каждой операции дискового ввода/вывода строку с подробностями о ней, включая величину задержки. Эта информация позволяет более детально изучать дисковый ввод/вывод, и наибольший интерес в этом выводе представляют закономерности, упорядоченные во времени (например, следование операций чтения за операциями записи). Более подробно `biosnoop(8)` рассматривается в главе 9.

3.4.6. cachestat

```
# cachestat
HITS    MISSES  DIRTIES  HITRATIO  BUFFERS_MB  CACHED_MB
53401   2755    20953    95.09%    14           90223
49599   4098    21460    92.37%    14           90230
16601   2689    61329    86.06%    14           90381
15197   2477    58028    85.99%    14           90522
[...]
```

¹ Гистограмма выглядит несколько искаженной из-за логарифмического распределения: каждый следующий сегмент охватывает все больший диапазон. Если бы мне потребовалось оценить работу диска с более высокой точностью, я мог бы изменить инструмент `biolatency(8)`, чтобы он выводил линейную гистограмму с более высоким разрешением. Либо использовать `biosnoop(8)` для журналирования информации о дисковом вводе/выводе, а затем импортировать ее в электронную таблицу и построить гистограмму там.

cachestat(8) каждую секунду (или через каждый заданный интервал) выводит строку со сводной информацией о работе кэша файловой системы. На основе этой информации можно определить низкую частоту попадания в кэш и высокую частоту ошибок. Это поможет в настройке производительности. Более подробно cachestat(8) рассматривается в главе 8.

3.4.7. tcpconnect

```
# tcpconnect
PID  COMM      IP  SADDR      DADDR      DPORT
1479  telnet    4  127.0.0.1   127.0.0.1  23
1469  curl      4  10.201.219.236  54.245.105.25  80
1469  curl      4  10.201.219.236  54.67.101.145  80
1991  telnet    6  ::1         ::1        23
2015  ssh       6  fe80::2000:bff:fe82:3ac fe80::2000:bff:fe82:3ac 22
[...]
```

Для каждого TCP-соединения, устанавливаемого локальными приложениями (например, вызовом connect()), tcpconnect(8) выводит строку с подробностями, включая начальный и конечный адреса. Наибольший интерес в этом выводе представляют соединения, которые могут указывать на неэффективную конфигурацию или действия злоумышленника. Более подробно tcpconnect(8) рассматривается в главе 10.

3.4.8. tcpsaccept

```
# tcpsaccept
PID  COMM  IP  RADDR      LADDR      LPORT
907  sshd   4  192.168.56.1  192.168.56.102  22
907  sshd   4  127.0.0.1    127.0.0.1    22
5389 perl   6  1234:ab12:2040:5020:2299:0:5:0 1234:ab12:2040:5020:2299:0:5:0 7001
[...]
```

tcpsaccept(8) часто используется в паре с tcpconnect(8). Для каждого TCP-соединения, устанавливаемого извне (например, вызовом accept()), этот инструмент выводит строку с подробностями, включая начальный и конечный адреса. Более подробно tcpsaccept(8) рассматривается в главе 10.

3.4.9. tcpretrans

```
# tcpretrans
TIME  PID  IP  LADDR:LPORT  T>  RADDR:RPORT  STATE
01:55:05 0    4  10.153.223.157:22  R>  69.53.245.40:34619 ESTABLISHED
01:55:05 0    4  10.153.223.157:22  R>  69.53.245.40:34619 ESTABLISHED
01:55:17 0    4  10.153.223.157:22  R>  69.53.245.40:22957 ESTABLISHED
[...]
```

Для каждой попытки повторно передать TCP-пакет tcpretrans(8) выводит одну строку с подробностями, включая начальный и конечный адреса, а также состояние

TCP-соединения. Попытки повторной передачи TCP-пакетов вызывают задержки и уменьшают пропускную способность. Повторные передачи, когда сеанс TCP находится в состоянии ESTABLISHED, могут говорить о проблемах с внешними сетями. Для состояния SYN_SENT это может указывать на насыщение процессора и потерю пакетов в пространстве ядра. Более подробно tcpretrans (8) рассматривается в главе 10.

3.4.10. runqlat

```
# runqlat
```

```
Tracing run queue latency... Hit Ctrl-C to end.
```

```
^C
```

usecs	: count	distribution
0 -> 1	: 233	*****
2 -> 3	: 742	*****
4 -> 7	: 203	*****
8 -> 15	: 173	*****
16 -> 31	: 24	*
32 -> 63	: 0	
64 -> 127	: 30	*
128 -> 255	: 6	
256 -> 511	: 3	
512 -> 1023	: 5	
1024 -> 2047	: 27	*
2048 -> 4095	: 30	*
4096 -> 8191	: 20	
8192 -> 16383	: 29	*
16384 -> 32767	: 809	*****
32768 -> 65535	: 64	***

runqlat(8) определяет, как долго потоки ждут своей очереди выполнения на процессоре, и выводит полученные результаты в виде гистограммы. Вы можете выявить необычно длительные задержки в ожидании доступа к процессору из-за его насыщения, неправильной конфигурации или проблем в планировщике. Более подробно runqlat(8) рассматривается в главе 6.

3.4.11. profile

```
# profile
```

```
Sampling at 49 Hertz of all threads by user + kernel stack... Hit Ctrl-C to end.
```

```
^C
```

```
[...]
```

```
copy_user_enhanced_fast_string
copy_user_enhanced_fast_string
_copy_from_iter_full
tcp_sendmsg_locked
tcp_sendmsg
inet_sendmsg
sock_sendmsg
```

```
sock_write_iter
new_sync_write
__vfs_write
vfs_write
SyS_write
do_syscall_64
entry_SYSCALL_64_after_hwframe
[unknown]
[unknown]
-                iperf (24092)
58
```

profile(8) — это профилировщик процессора, с помощью которого можно определить, какой код потребляет больше всего вычислительных ресурсов. Инструмент отбирает трассировки стека через определенные интервалы времени и выводит уникальные трассировки и сколько раз они встретились. В усеченном примере выше показана только одна трассировка стека с числом вхождений 58. Более подробно profile(8) рассматривается в главе 6.

3.5. ИТОГИ

Цель анализа производительности — увеличение производительности конечного пользователя и уменьшение эксплуатационных расходов. Есть много инструментов и метрик, которые помогут проанализировать производительность. Их так много, что выбор наиболее подходящих для конкретной ситуации может стать непростой задачей. Определиться с выбором вам помогут методологии оценки производительности, которые определяют, с чего начать, какие шаги выполнить и где закончить.

В этой главе был дан краткий обзор таких методологий оценки производительности, как определение характера рабочей нагрузки, анализ задержек, метод USE и чек-листы. Затем мы остановились на чек-листе инструментов Linux для анализа производительности за 60 секунд, который можно использовать как отправную точку для решения любой проблемы с производительностью. Он может решить проблемы непосредственно или хотя бы подсказать, в каком направлении двигаться уже с применением инструментов BPF. Также в эту главу включен чек-лист инструментов BCC, о которых более подробно речь пойдет в последующих главах.

Глава 4

ВСС

Коллекция компиляторов BPF (BPF Compiler Collection, ВСС; иногда пишется строчными буквами «bcc» после названий проекта и пакета) — это проект с открытым исходным кодом, включающий фреймворк компиляторов и библиотек для сборки ПО BPF. Это основной интерфейс для BPF, поддерживаемый разработчиками BPF. Обычно в нем раньше всех начинают использоваться самые свежие дополнения в подсистеме трассировки BPF в ядре. Проект ВСС также включает более 70 готовых инструментов на основе BPF для анализа и устранения неполадок, многие из которых описаны в этой книге.

Проект ВСС был создан Бренденом Бланко (Brenden Blanco) в апреле 2015 года. По приглашению Алексея Старовойтова я присоединился к проекту в 2015 году и активно участвовал в разработке инструментов, написании документации и тестировании. Сейчас проект ВСС насчитывает большое число участников и по умолчанию устанавливается на серверы в Netflix и Facebook.

Цели обучения:

- познакомиться с возможностями и компонентами ВСС, включая инструменты и документацию;
- получить представление о преимуществах специализированных и универсальных инструментов;
- изучить приемы использования многофункционального инструмента `funccount(8)` для подсчета событий;
- изучить приемы использования многофункционального инструмента `stackcount(8)` для определения путей выполнения кода;
- изучить приемы использования многофункционального инструмента `trace(8)` для вывода информации о событиях;
- изучить приемы использования многофункционального инструмента `argdist(8)` для получения информации о распределении;
- (дополнительно) познакомиться с некоторыми внутренними деталями ВСС;
- познакомиться с приемами отладки в ВСС.

Эта глава знакомит с возможностями ВСС, показывает, как установить этот проект, дает краткий обзор инструментов и их типов, а также документации и заканчивается туром по внутренним компонентам ВСС и приемам отладки. Если хотите создавать свои инструменты для ВСС, обязательно изучите и эту главу, и главу 5 (о bpfttrace), после этого вы сможете выбрать интерфейс, который лучше соответствует вашим потребностям. В приложении С вы познакомитесь с разработкой инструментов ВСС на практических примерах.

4.1. КОМПОНЕНТЫ ВСС

На рис. 4.1 показана структура каталогов репозитория проекта ВСС.

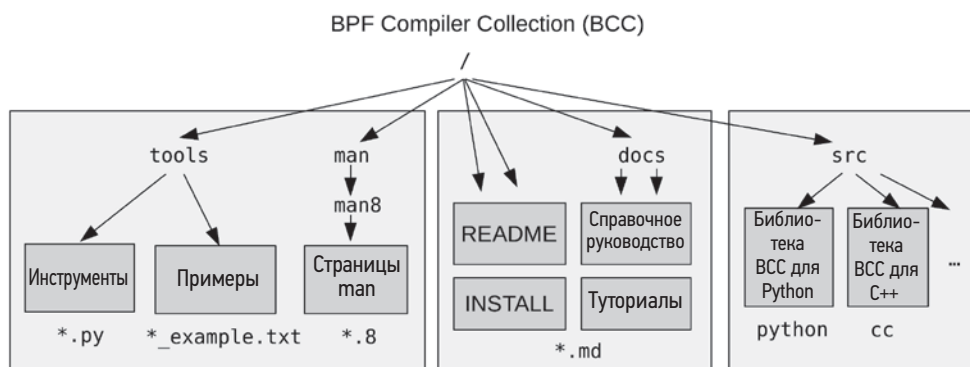


Рис. 4.1. Структура каталогов ВСС

В репозитории ВСС есть документация с описанием инструментов, страницы справочного руководства (man), файлы с примерами, справочники и руководства по использованию и разработке инструментов ВСС. Есть библиотеки поддержки для разработки инструментов ВСС на Python, C++ и Lua (на рис. 4.1 не показаны). Планируются библиотеки поддержки для других языков.

Репозиторий находится по адресу <https://github.com/iovisor/bcc>.

Инструменты на Python в репозитории ВСС хранятся в файлах .py, но при установке ВСС с помощью диспетчера пакетов расширение обычно не указывается. Окончательное местоположение инструментов ВСС и страниц справочного руководства зависит от используемого пакета — в разных дистрибутивах Linux пакеты упаковываются по-разному. Инструменты могут устанавливаться в /usr/share/bcc/tools, /sbin или /snap/bin, а в имена самих инструментов, в начало или в конец, может добавляться аббревиатура «bcc», чтобы было видно, что они принадлежат коллекции ВСС. Эти различия подробнее будут описаны в разделе 4.3.

4.2. ВОЗМОЖНОСТИ ВСС

ВСС — это проект с открытым исходным кодом, созданный и поддерживаемый инженерами из различных компаний. Это некоммерческий продукт, иначе отдел маркетинга постарался бы разрекламировать его многочисленные возможности.

Списки возможностей, или фич-листы (feature lists), — если они достаточно точны — помогают в изучении новой технологии. Разрабатывая BPF и ВСС, я написал списки желаемых возможностей [57]. Поскольку эти возможности уже реализованы, они превратились в фич-листы как для пространства ядра, так и для пространства пользователя. Они обсуждаются в следующих разделах.

4.2.1. Возможности в пространстве ядра

ВСС может использовать некоторые механизмы ядра — BPF, kprobes, uprobes и т. д. В списке ниже в скобках указаны некоторые детали реализации:

- динамическая инструментация в пространстве ядра (поддержка kprobes в BPF);
- динамическая инструментация в пространстве пользователя (поддержка uprobes в BPF);
- статическая трассировка в пространстве ядра (поддержка точек трассировки в BPF);
- выборка событий по времени (BPF с `perf_event_open()`);
- события РМС (BPF с `perf_event_open()`);
- фильтрация (с использованием программ BPF);
- вывод отладочной информации (`bpf_trace_printk()`);
- вывод информации о каждом событии (`bpf_perf_event_output()`);
- простые переменные (глобальные переменные и переменные для потоков выполнения, с использованием карт BPF);
- ассоциативные массивы (с использованием карт BPF);
- подсчет частоты (с использованием карт BPF);
- гистограммы (степенные, линейные и настраиваемые, с использованием карт BPF);
- отметки времени и временные интервалы (`bpf_ktime_get_ns()` и программы BPF);
- трассировки стека в пространстве ядра (карта стека в BPF);
- трассировки стека в пространстве пользователя (карта стека в BPF);
- запись в кольцевые буферы (`perf_event_attr.write_backward`);
- инструментация с низким оверхедом (BPF JIT, карты сводок в BPF);
- безопасность в промышленной среде (верификатор BPF).

Подробнее об этих возможностях в пространстве ядра рассказывается в главе 2.

4.2.2. Возможности в пространстве пользователя

Интерфейс BCC в пространстве пользователя и проект BCC предлагают следующие возможности в пространстве пользователя:

- статическую трассировку в пространстве пользователя (точки трассировки USDT в стиле SystemTap с использованием uprobes);
- вывод отладочной информации (в Python с `BPF.trace_pipe()` и `BPF.trace_fields()`);
- вывод информации о каждом событии (макрос `BPF_PERF_OUTPUT` и `BPF.open_perf_buffer()`);
- поинтервальный вывод (`BPF.get_table()` и `table.clear()`);
- вывод гистограмм (`table.print_log2_hist()`);
- навигацию по структурам на C в пространстве ядра (отображается в вызовы `bpf_probe_read()`);
- разрешение символов в пространстве ядра (`ksym()` и `ksymaddr()`);
- разрешение символов в пространстве пользователя (`usymaddr()`);
- поддержку разрешения отладочных символов;
- поддержку точек трассировки BPF (`TRACEPOINT_PROBE`);
- поддержку трассировки стека BPF (`BPF_STACK_TRACE`);
- другие вспомогательные макросы и функции;
- примеры (в каталоге `/examples`);
- множество инструментов (в каталоге `/tools`);
- tutorиалы (`/docs/tutorial*.md`);
- справочное руководство (`/docs/reference_guide.md`).

4.3. УСТАНОВКА BCC

Пакеты BCC доступны для многих дистрибутивов Linux, включая Ubuntu, RHEL, Fedora и Amazon Linux, что делает установку простым делом. При желании BCC можно собрать из исходных кодов. Самые свежие инструкции по установке и сборке вы найдете в файле `INSTALL.md` в репозитории BCC [58].

4.3.1. Требования к конфигурации ядра

Основные компоненты BPF, используемые инструментами BCC, были добавлены в ядро Linux между релизами 4.1 и 4.9. Но и в более поздних версиях добавлялись дополнительные улучшения, поэтому чем новее ваше ядро, тем лучше. Рекомендую использовать ядро Linux 4.9 (выпущено в декабре 2016 года) или более новое.

Также должны быть включены некоторые параметры конфигурации ядра: `CONFIG_BPF=y`, `CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_EVENTS=y`,

CONFIG_BPF_JIT=y и CONFIG_HAVE_EBPF_JIT=y. Во многих современных дистрибутивах Linux эти параметры включены по умолчанию, поэтому менять их вряд ли придется.

4.3.2. Ubuntu

Пакет BCC для Ubuntu с именем bpfcc-tools находится в репозитории Multiverse. Установите его:

```
sudo apt-get install bpfcc-tools linux-headers-$(uname -r)
```

Эта команда поместит инструменты в каталог /sbin, добавив к именам файлов окончание «-bpfcc»:

```
# ls /sbin/*-bpfcc
/usr/sbin/argdist-bpfcc
/usr/sbin/bashreadline-bpfcc
/usr/sbin/biolatency-bpfcc
/usr/sbin/biosnoop-bpfcc
/usr/sbin/biotop-bpfcc
/usr/sbin/bitesize-bpfcc
[...]
# opensnoop-bpfcc
PID      COMM          FD ERR PATH
29588    device poll      4   0  /dev/bus/usb
[...]
```

Последние стабильные и подписанные пакеты можно также получить из репозитория iovisor:

```
sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys 4052245BD4284CDD
echo "deb https://repo.iovisor.org/apt/$(lsb_release -cs) $(lsb_release -cs) main"| \
sudo tee /etc/apt/sources.list.d/iovisor.list
sudo apt-get update
sudo apt-get install bcc-tools libbcc-examples linux-headers-$(uname -r)
```

В этом случае инструменты будут установлены в каталог /usr/share/bcc/tools.

Наконец, BCC для Ubuntu доступен также в системе управления пакетами snap:

```
sudo snap install bcc
```

В этом случае инструменты будут установлены в каталог /snap/bcc/bin (он может быть включен в переменную среды \$PATH) и доступны под именами, начинающимися с «bcc.» (например, bcc.opensnoop).

4.3.3. RHEL

Пакет с BCC включен в официальный репозиторий Red Hat Enterprise Linux 7.6 и устанавливается командой:

```
sudo yum install bcc-tools
```

Инструменты устанавливаются в каталог `/usr/share/bcc/tools`.

4.3.4. Другие дистрибутивы

В файле `INSTALL.md` приводятся инструкции по установке для Fedora, Arch, Gentoo и openSUSE, а также инструкции по установке из исходных кодов.

4.4. ИНСТРУМЕНТЫ ВСС

На рис. 4.2 показаны основные компоненты систем и инструменты ВСС для наблюдения за ними¹.

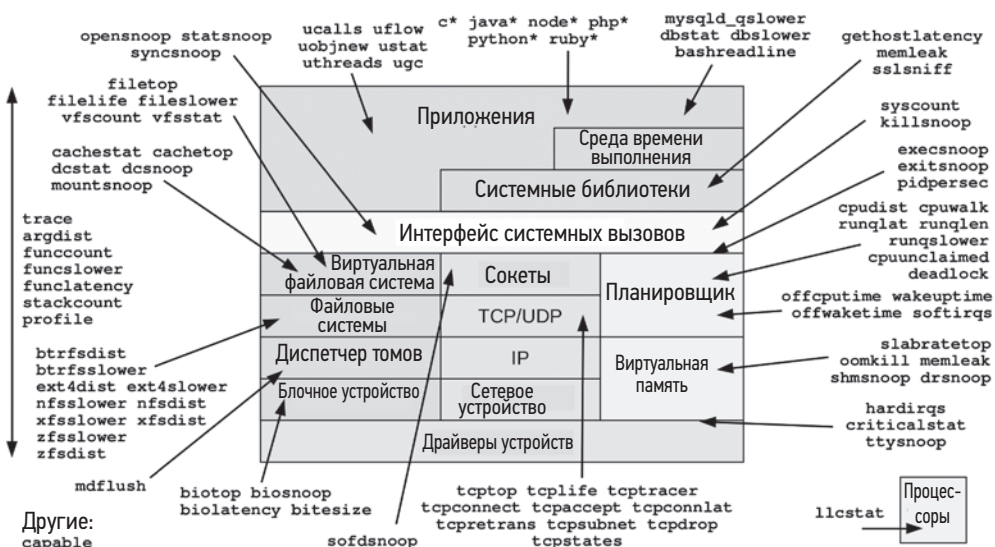


Рис. 4.2. Инструменты ВСС для оценки производительности

4.4.1. Инструменты, рассматриваемые в книге

В табл. 4.1 перечислены инструменты, сгруппированные по области применения. Они подробно описаны в следующих главах.

¹ Я создал этот рисунок для репозитория ВСС, где можно найти последнюю его версию [60]. Собираюсь обновить его после публикации книги и переноса наиболее важных новых инструментов `bpftrace` из этой книги в ВСС.

Таблица 4.1. Инструменты ВСС, сгруппированные по области применения и главам

Область применения	Инструменты	Глава
Отладка/многоцелевые	trace, argdist, funccount, stackcount, opensnoop trace, argdist, funccount, stackcount, opensnoop	4
Процессоры	execsnoop, runqlat, runqlen, cpudist, profile, offcputime, syscount, softirq, hardirq	6
Память	memleak	7
Файловые системы	opensnoop, filelife, vfsstatt, fileslower, cachestat, writeback, dcstat, xfsslower, xfsdist, ext4dist	8
Дисковый ввод/вывод	biolatency, biosnoop, biotop, bitesize	9
Сети	tcpconnect, tcpaccept, tcplife, tcptrans	10
Безопасность	capable	11
Языки	javastat, javacalls, javathreads, javaflow, javagc	12
Приложения	mysqld_qlower, signals, killsnoop	13
Ядро	wakeuptime, offwaketime	14

Отмечу, что в этих главах рассматриваются и дополнительные инструменты bpftrace, не перечисленные в табл. 4.1.

Прочитав эту главу и главу 5, далее вы можете свободно переходить к любой интересующей вас главе, используя эту книгу как справочное руководство.

4.4.2. Характеристики инструментов

Инструменты ВСС имеют следующие характеристики:

- решают реальные проблемы наблюдаемости, настраиваются по мере необходимости;
- предназначены для выполнения в продакшене с привилегиями пользователя root;
- для каждого инструмента есть страница в справочном руководстве (в каталоге man/man8);
- для каждого инструмента есть файл с примерами вывода с пояснениями (в каталоге tools/*_example.txt);
- многие инструменты принимают параметры и аргументы, большинство из них выводит сообщение о порядке использования, если вызвать их с параметром -h;
- исходный код каждого инструмента начинается с вводного блочного комментария;
- исходный код каждого инструмента соответствует единому стилю (проверяется с помощью pep8).

Для поддержки согласованности новые дополнения к инструментам проверяются мейнтейнерами ВСС. Авторам дополнений предлагается следовать руководству CONTRIBUTING_SCRIPTS.md [59].

Инструменты ВСС специально разрабатывались в едином стиле с системными инструментами, например `vmstat(1)` и `iostat(1)`. Как и в случае с `vmstat(1)` и `top(1)`, полезно иметь некоторое представление о том, как работают инструменты ВСС, особенно с точки зрения оверхеда инструмента. Эта книга подробно рассказывает, как работают эти инструменты, и почти всегда описывает ожидаемый оверхед. Внутренние компоненты ВСС и используемые ими технологии ядра описываются в этой главе и в главе 2.

ВСС поддерживает возможность использования из разных языков. При этом основными языками, на которых пишутся инструменты ВСС, являются Python (для взаимодействия с компонентами в пространстве пользователя) и C (для взаимодействия с компонентами BPF в пространстве ядра). Инструментам на Python/C уделяются наибольшее внимание и поддержка со стороны разработчиков ВСС, поэтому они подробно рассматриваются в этой книге.

Одно из предложений в руководстве для контрибьюторов гласит: «Пишите инструмент для решения проблемы — и не более». Это поощряет разработку специализированных инструментов, а не многоцелевых, где это возможно.

4.4.3. Специализированные инструменты

Философия Unix заключается в том, чтобы каждый инструмент делал что-то одно, и делал это хорошо. Одним из проявлений этой философии стало создание небольших высококачественных инструментов, которые можно сочетать друг с другом с помощью конвейеров для выполнения более сложных задач. В результате появилось множество небольших специализированных инструментов, которые используются и по сей день, например `grep(1)`, `cut(1)` и `sed(1)`.

В ВСС есть много похожих специализированных инструментов, включая `opensnoop(8)`, `execsnoop(8)` и `biolatency(8)`. `opensnoop(8)` — их яркий представитель. Посмотрим, как настроить параметры и формат вывода для трассировки системных вызовов `open(2)`:

opensnoop -h

порядок использования: `opensnoop [-h] [-T] [-U] [-x] [-p PID] [-t TID] [-u UID] [-d DURATION] [-n NAME] [-e] [-f FLAG_FILTER]`

Трассирует системные вызовы `open()`

необязательные аргументы:

<code>-h, --help</code>	вывести эту справку и выйти
<code>-T, --timestamp</code>	включить отметку времени в вывод
<code>-U, --print-uid</code>	вывести столбец UID
<code>-x, --failed</code>	показать только неудачные вызовы <code>open</code>
<code>-p PID, --pid PID</code>	трассировать только этот PID
<code>-t TID, --tid TID</code>	трассировать только этот TID

```

-u UID, --uid UID      трассировать только этот UID
-d DURATION, --duration DURATION
                        общая продолжительность трассировки в секундах
-n NAME, --name NAME   выводить только имена процессов, содержащие это имя
-e, --extended_fields   показать дополнительные поля
-f FLAG_FILTER, --flag_filter FLAG_FILTER
                        фильтровать по аргументу с флагами (например, O_WRONLY)

```

примеры:

```

./opensnoop           # трассировать все системные вызовы open()
./opensnoop -T        # включить отметки времени
./opensnoop -U        # добавить в вывод столбец UID
./opensnoop -x        # показать только неудачные вызовы open
./opensnoop -p 181     # трассировать только PID 181
./opensnoop -t 123     # трассировать только TID 123
./opensnoop -u 1000    # трассировать только UID 1000
./opensnoop -d 10      # трассировать только 10 секунд
./opensnoop -n main    # выводить только имена процессов, содержащие "main"
./opensnoop -e        # показать дополнительные поля
./opensnoop -f O_WRONLY -f O_RDWR # выводить только вызовы для записи

```

```

# opensnoop
PID  COMM          FD ERR PATH
29588 device poll    4  0 /dev/bus/usb
29588 device poll    6  0 /dev/bus/usb/004
[...]
```

Основные преимущества инструментов BPF, написанных в этом стиле:

- **Легко осваиваются начинающими:** вывода по умолчанию обычно вполне достаточно. Это означает, что начинающие могут немедленно приступить к использованию инструмента, не заботясь об особенностях использования командной строки и инструментации событий. Например, чтобы с помощью `opensnoop(8)` получить полезную информацию, выполните простую команду `opensnoop` — не нужно знать, как инструментировать системный вызов `open(2)` с помощью `kprobes` или точек трассировки.
- **Простота сопровождения:** чем меньше кода содержит инструмент, тем проще разработчику сопровождать его. Универсальные инструменты могут поддерживать разные рабочие нагрузки разными способами, из-за чего даже небольшое изменение в коде потребует нескольких часов тестирования с разными рабочими нагрузками, чтобы подтвердить их работоспособность. Для конечного пользователя это означает, что такие инструменты с большей вероятностью будут работать тогда, когда нужно.
- **Примеры кода:** каждый небольшой инструмент снабжен кратким и практичным примером кода. Многие изучающие разработку инструментов BCC начинают свой путь с таких специализированных инструментов и пытаются настраивать и расширять их по мере необходимости.
- **Настраиваемые аргументы и формат вывода:** аргументы инструмента, позиционные параметры и выходные данные не должны учитывать возможность

решения других задач и могут настраиваться для одной-единственной цели. Это увеличивает удобство использования и удобочитаемости вывода.

Если вы только начинаете осваивать ВСС, разработка специализированных инструментов — хороший способ накопить опыт перед переходом к более сложным многоцелевым инструментам.

4.4.4. Многоцелевые инструменты

В ВСС есть множество многоцелевых инструментов, которые способны решать различные задачи. Их сложнее освоить, чем специализированные, зато у них более широкие возможности. Тем, кто применяет многоцелевые инструменты лишь изредка, необязательно глубоко изучать их — достаточно собрать коллекцию однострочных сценариев и использовать по мере необходимости.

Преимущества многоцелевых инструментов:

- **Более широкая область видимости:** вместо решения одной задачи или исследования одной цели можно просматривать сразу несколько разных компонентов.
- **Меньше повторяющегося кода:** нет необходимости писать один и тот же код в нескольких инструментах.

В числе наиболее мощных многоцелевых инструментов ВСС можно назвать: `funccount(8)`, `stackcount(8)`, `trace(8)` и `argdist(8)`, о которых пойдет речь дальше. Эти многоцелевые инструменты часто позволяют выбирать события для трассировки. Но чтобы воспользоваться этой гибкостью, нужно знать, какие точки `kprobes`, `uprobes` и другие события использовать и как их использовать. В главах, описывающих конкретные темы, мы вернемся к специализированным инструментам.

В табл. 4.2 перечислены многоцелевые инструменты, о которых пойдет речь в этой главе.

Таблица 4.2. Многоцелевые инструменты из этой главы

Инструмент	Где находится	Цель	Описание
<code>funccount</code>	ВСС	Программное обеспечение	Подсчитывает события, включая вызовы функций
<code>stackcount</code>	ВСС	Программное обеспечение	Подсчитывает трассировки стека, приводящие к событиям
<code>trace</code>	ВСС	Программное обеспечение	Выводит информацию о событиях
<code>argdist</code>	ВСС	Программное обеспечение	Оценивает распределение событий

Полный список инструментов с описанием их возможностей ищите в репозитории ВСС. Далее мы рассмотрим наиболее важные из них.

4.5. FUNCCOUNT

`funccount(8)`¹ подсчитывает события, в частности вызовы функций, и помогает ответить на вопросы:

- Вызывается ли эта функция (в ядре или в пространстве пользователя)?
- Как часто вызывается эта функция?

Для большей эффективности `funccount(8)` хранит счетчик событий в пространстве ядра, используя карту ВРФ, и передает в пространство пользователя только общее количество. Это значительно снижает издержки `funccount(8)` по сравнению с инструментами, передающими данные при каждом изменении и выполняющими постобработку, тем не менее при наблюдении за событиями, следующими с большой частотой, оверхед может быть существенным. Например, операции управления динамической памятью (`malloc()`, `free()`) могут выполняться миллионы раз в секунду, и использование `funccount(8)` для их трассировки увеличивает загрузку процессора на 30%. Подробнее о типичной частоте следования разных событий и оверхеде я расскажу в главе 18.

Ниже показаны приемы использования `funccount(8)`, а также объясняется синтаксис и возможности этого инструмента.

4.5.1. Примеры funccount

1. Вызывается ли функция `tcp_drop()`?

```
# funccount tcp_drop
Tracing 1 functions for "tcp_drop"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
tcp_drop                            3
Detaching...
```

Ответ: да. Инструмент просто подсчитывает число вызовов функции ядра `tcp_drop()`, пока не будет нажата комбинация Ctrl-C. Здесь за время трассировки функция была вызвана три раза.

2. Какая функция из подсистемы VFS в ядре вызывается чаще всего?

```
# funccount 'vfs_*'
Tracing 55 functions for "vfs_*"... Hit Ctrl-C to end.
^C
```

¹ Немного истории: первую версию я разработал 12 июля 2014 года. Для подсчета вызовов функций ядра в ней использовался `Ftrace`. 9 сентября 2015 года я выпустил версию на основе BCC. 18 ноября 2016 года Саша Гольдштейн добавил в версию на основе BCC поддержку других типов событий: вызовы пользовательских функций (`uprobes`), точки трассировки и `USDT`.

FUNC	COUNT
vfs_rename	1
vfs_readlink	2
vfs_lock_file	2
vfs_statfs	3
vfs_fsync_range	3
vfs_unlink	5
vfs_statx	189
vfs_statx_fd	229
vfs_open	345
vfs_getattr_nosec	353
vfs_getattr	353
vfs_writev	1776
vfs_read	5533
vfs_write	6938
Detaching...	

Для трассировки вызовов всех функций в ядре, имена которых начинаются с «vfs_», в этой команде использует подстановочный знак в стиле командной оболочки. Судя по результатам, чаще всего вызывается функция `vfs_write()`, за период трассировки она была вызвана 6938 раз.

3. Сколько раз в секунду вызывается функция `pthread_mutex_lock()` в пространстве пользователя?

```
# funccount -i 1 c:pthread_mutex_lock
Tracing 1 functions for "c:pthread_mutex_lock"... Hit Ctrl-C to end.
```

FUNC	COUNT
pthread_mutex_lock	1849

FUNC	COUNT
pthread_mutex_lock	1761

FUNC	COUNT
pthread_mutex_lock	2057

FUNC	COUNT
pthread_mutex_lock	2261
[...]	

Частота немного изменяется, но остается близкой к 2000 вызовов в секунду. Эта команда инструментирует функцию из библиотеки `libc` и подсчитывает частоту вызовов для всей системы в целом: результаты показывают общее число вызовов для всех процессов.

4. Какая из строковых функций в `libc` вызывается чаще всего в системе в целом?

```
# funccount 'c:str*'
Tracing 59 functions for "c:str*"... Hit Ctrl-C to end.
```

FUNC	COUNT
strndup	3
strerror_r	5

strerror	5
strtof32x_1	350
strtoul	587
strtoll	724
strtok_r	2839
strdup	5788
Detaching...	

В течение периода трассировки чаще других вызывалась функция `strdup()` — 5788 раз.

5. Какой системный вызов вызывается чаще всего?

```
# funccount 't:syscalls:sys_enter_*'
Tracing 316 functions for "t:syscalls:sys_enter_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
syscalls:sys_enter_creat            1
[...]
syscalls:sys_enter_read             6582
syscalls:sys_enter_write            7442
syscalls:sys_enter_mprotect         7460
syscalls:sys_enter_gettid           7589
syscalls:sys_enter_ioctl            10984
syscalls:sys_enter_poll             14980
syscalls:sys_enter_recvmsg          27113
syscalls:sys_enter_futex            42929
Detaching...
```

Ответ на этот вопрос можно получить из разных источников событий. В этом случае я использовал точки трассировки, находящиеся на входах в системные вызовы («`sys_enter_*`»). Судя по результатам, в период трассировки чаще других вызывался системный вызов `futex()` — 42 929 раз.

4.5.2. Синтаксис funccount

С помощью параметров и аргументов можно изменять поведение `funccount(8)` и задавать событие для наблюдения:

```
funccount [options] eventname
```

Синтаксис аргумента `eventname`:

- **name** или **p:name**: инструментировать функцию ядра с именем `name()`;
- **Lib:name** или **p:Lib:name**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в библиотеке `lib`;
- **path:name**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в файле `path`;
- **t:system:name**: инструментировать точку трассировки с именем `system:name`;
- **u:Lib:name**: инструментировать зонд USDT в библиотеке `lib` с именем `name`;

- *: подстановочный символ, соответствующий любой строке. Вместо него можно использовать параметр `-r` с регулярным выражением.

Этот синтаксис заимствован из Ftrace. Для инструментации функций в пространстве ядра или в пространстве пользователя `funccount(8)` использует `kprobes` и `uprobes`.

4.5.3. Однострочные сценарии `funccount`

Подсчет вызовов функций виртуальной файловой системы в ядре:

```
funccount 'vfs_*
```

Подсчет вызовов функций TCP в ядре:

```
funccount 'tcp_*
```

Определение частоты вызовов в секунду функций TCP:

```
funccount -i 1 'tcp_send*'
```

Определение частоты операций блочного ввода/вывода в секунду:

```
funccount -i 1 't:block:*
```

Определение частоты запуска новых процессов в секунду:

```
funccount -i 1 t:sched:sched_process_fork
```

Определение частоты вызовов в секунду функции `getaddrinfo()` (разрешение имен) из библиотеки `libc`:

```
funccount -i 1 c:getaddrinfo
```

Подсчет вызовов всех функций «os.*» в библиотеке `libgo`:

```
funccount 'go:os.*'
```

4.5.4. Порядок использования `funccount`

Выше я показал далеко не все возможности `funccount(8)`, дополнительную информацию можно найти в справочном сообщении¹:

`funccount -h`

порядок использования: `funccount [-h] [-p PID] [-i INTERVAL] [-d DURATION] [-T] [-r] [-D] pattern`

Подсчитывает вызовы функций, точек трассировки и зондов USDT

¹ Справка выводится на английском языке, здесь приведен перевод. Аналогично — в следующих примерах с выводом справочной информации. — *Примеч. пер.*

позиционные аргументы:

pattern выражение, определяющее события

необязательные аргументы:

-h, --help вывести эту справку и выйти
 -p PID, --pid PID трассировать только этот PID
 -i INTERVAL, --interval INTERVAL
 интервал вывода в секундах
 -d DURATION, --duration DURATION
 общая продолжительность трассировки в секундах
 -T, --timestamp включить отметку времени в вывод
 -r, --regex использовать регулярное выражение. По умолчанию
 допускается только символ подстановки "*"
 -D, --debug вывести программу BPF перед запуском (для отладки)

примеры:

```
./funccount 'vfs_*'        # подсчитать вызовы функций ядра с именами на "vfs_"
./funccount -r '^vfs.*'    # то же, но с применением регулярного выражения
./funccount -Ti 5 'vfs_*'   # выводить каждые 5 секунд, с отметками времени
./funccount -d 10 'vfs_*'   # проводить трассировку только 10 секунд
./funccount -p 185 'vfs_*' # подсчитать вызовы vfs только для PID 181
```

[...]

Параметр, определяющий интервал (-i), позволяет превратить однострочный сценарий с funccount в мини-инструмент, показывающий частоту следования событий в секунду. Можно определять и свои метрики на основе тысяч доступных событий и при необходимости фильтровать по идентификатору целевого процесса с помощью -p.

4.6. STACKCOUNT

stackcount(8)¹ подсчитывает трассировки стека, которые привели к событию. Как и в funccount(8), событием может быть функция в пространстве ядра или пользователя, точка трассировки или зонд USDT. stackcount(8) позволяет ответить на вопросы:

- Почему произошло это событие? Каков путь кода?
- Какие пути ведут к этому событию и как часто код следует этими путями?

Для большей эффективности подсчет трассировок выполняется полностью в контексте ядра, с использованием специальной карты BPF для трассировок стека.

¹ Немного истории: первая версия, разработанная мной 12 января 2016-го, поддерживала только kprobes, а 9 июля 2016-го Саша Гольдштейн добавил другие типы событий: uprobes и точки трассировки. Раньше я часто использовал kprobe -s из репозитория Ftrace perf-tools, чтобы вывести трассировку стека для каждого события, но вывод получался подробным. Я задумался о возможности подсчета трассировок в ядре, что привело к появлению stackcount(8). Я также попросил Тома Занусси реализовать подсчет с использованием триггеров hist в Ftrace, что он и сделал.

Код в пространстве пользователя получает идентификаторы трассировок, подсчитывает их, а затем извлекает сами трассировки из карты BPF для разрешения символов и вывода. По аналогии с `funccount(8)`, величина оверхеда зависит от скорости следования инструментируемого события, но в целом заметно выше, потому что для каждого события `stackcount(8)` выполняет обход стека и запись результатов.

4.6.1. Пример `stackcount`

Однажды, экспериментируя с `funccount(8)`, я заметил, что в простаивающей системе наблюдается необычно высокая частота вызовов функций ядра `ktime_get()` — больше 8000 раз в секунду. Эта функция вызывается, чтобы определить время, но зачем простаивающей системе так часто определять время?

Вот пример использования `stackcount(8)` для определения путей выполнения кода, ведущих к вызову `ktime_get()`:

```
# stackcount ktime_get
Tracing 1 functions for "ktime_get"... Hit Ctrl-C to end.
^C
[...]

    ktime_get
    nvme_queue_rq
    __blk_mq_try_issue_directly
    blk_mq_try_issue_directly
    blk_mq_make_request
    generic_make_request
    dmccrypt_write
    kthread
    ret_from_fork
    52
[...]

    ktime_get
    tick_nohz_idle_enter
    do_idle
    cpu_startup_entry
    start_secondary
    secondary_startup_64
    1077

Detaching...
```

Вывод растянулся на сотни страниц и содержал более 1000 трассировок стека. Здесь я показал только две из них. Каждая трассировка содержит вызовы функций, по одной в строке, и завершается числом вхождений. Например, первая трассировка стека показывает путь кода через `dmccrypt_write()`, `blk_mq_make_request()` и `nvme_queue_rq()`. Я бы предположил (не читая код), что он сохраняет время начала операции ввода/вывода для последующего использования. То время, пока работал

сценарий, этот путь к `ktime_get()` встретился 52 раза. Чаще всего путь к `ktime_get()` начинался из кода, обслуживающего бездействие системы.

Параметр `-P` позволяет добавить имена и идентификаторы PID процессов:

```
# stackcount -P ktime_get
[...]

ktime_get
tick_nohz_idle_enter
do_idle
cpu_startup_entry
start_secondary
secondary_startup_64
  swapper/2 [0]
  207
```

Судя по результатам, вызов `ktime_get()` через `do_idle()` производит процесс «swapper/2» с PID 0. Это еще раз подтверждает, что значительный вклад в частоту вызовов вносит поток, выполняющийся при простое системы. Параметр `-P` генерирует больше выходных данных, потому что ранее сгруппированные трассировки стека выводятся отдельно для каждого PID.

4.6.2. Создание флейм-графиков с помощью stackcount

Иногда для некоторых событий `stackcount(8)` выводит небольшое число трассировок стека, которые легко просмотреть. А в случаях, как в примере с `ktime_get()`, когда вывод имеет длину в сотни страниц, для визуализации выходных данных можно использовать флейм-графики. (Подробности о флейм-графиках см. в главе 2.) Оригинальное ПО для создания флейм-графиков [37] принимает информацию с трассировками стека в свернутом формате, в котором каждая трассировка располагается в одной строке, фреймы (имена функций) разделены точками с запятой, а в конце строки находится пробел и счетчик. `stackcount(8)` генерирует вывод в этом формате, если использовать параметр `-f`.

Пример ниже трассирует вызовы `ktime_get()` в течение 10 секунд (`-D 10`), разделяет трассировки по процессам (`-P`) и генерирует вывод, пригодный для создания флейм-графика:

```
# stackcount -f -P -D 10 ktime_get > out.stackcount01.txt
$ wc out.stackcount01.txt
 1586   3425 387661 out.stackcount01.txt
$ git clone http://github.com/brendangregg/FlameGraph
$ cd FlameGraph
$ ./flamegraph.pl --hash --bgcolors=grey < ../out.stackcount01.txt \
  > out.stackcount01.svg
```

Утилита `wc(1)` используется, чтобы показать, что вывод с результатами насчитывает 1586 строк — число уникальных комбинаций из трассировок стека и названий процессов. На рис. 4.3 приводится скриншот с получившимся файлом SVG.

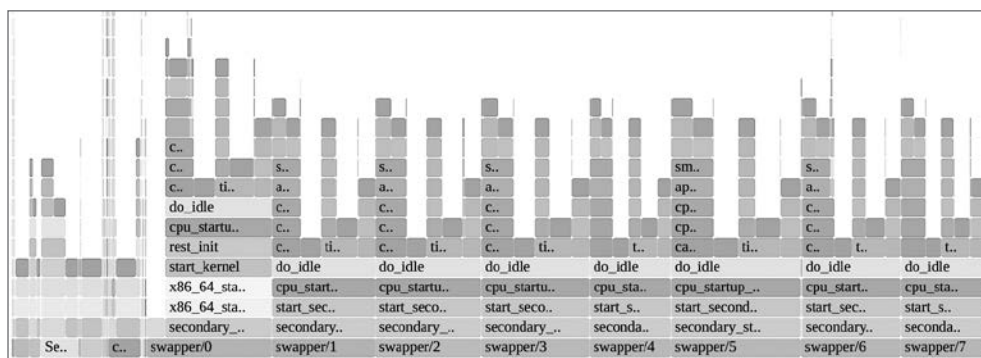


Рис. 4.3. Флейм-график, полученный в результате трассировки вызова `ktime_get()` с помощью `stackcount(8)`

Флейм-график показывает, что большинство вызовов `ktime_get()` было произведено из восьми потоков, выполняющихся во время бездействия системы, — по одному для каждого процессора в этой системе, о чем говорят схожие башни. Другие источники видны на графике слева, в виде узких башен.

4.6.3. Искривленные трассировки

Трассировки стеков и основные проблемы, связанные с их получением на практике, обсуждаются в главах 2, 12 и 18. Искривленные трассировки и пропущенные символы — обычное явление.

Например, в первой трассировке в этом разделе есть ссылка на функцию `tick_nohz_idle_enter()`, через которую пролегает путь к `ktime_get()`. Но этой функции нет в исходном коде ядра. То, что вы видите, — это вызов функции `tick_nohz_start_idle()`, которая определена в файле `kernel/time/tick-sched.c`:

```
static void tick_nohz_start_idle(struct tick_sched *ts)
{
    ts->idle_entrytime = ktime_get();
    ts->idle_active = 1;
    sched_clock_idle_sleep_event();
}
```

Это одна из тех коротких функций, которые компиляторы любят встраивать прямо в точку вызова, из-за чего фактический вызов `ktime_get()` происходит из родительской функции. Отсутствие символа `tick_nohz_start_idle` в `/proc/kallsyms` (в этой системе) лишь подтверждает, что эта функция была преобразована во встроенную.

4.6.4. Синтаксис `stackcount`

При вызове `stackcount(8)` можно передать аргумент с событием для наблюдения:

```
stackcount [options] eventname
```

Аргумент `eventname` имеет тот же синтаксис, что и `funcname(8)`:

- **`name`** или **`p:name`**: инструментировать функцию ядра с именем `name()`;
- **`lib:name`** или **`p:lib:name`**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в библиотеке `lib`;
- **`path:name`**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в файле `path`;
- **`t:system:name`**: инструментировать точку трассировки с именем `system:name`;
- **`u:lib:name`**: инструментировать зонд USDT в библиотеке `lib` с именем `name`;
- **`*`**: подстановочный символ, соответствующий любой строке, вместо него можно использовать параметр `-r` с регулярным выражением.

4.6.5. Однострочные сценарии stackcount

Подсчет трассировок стека, приводящих к операции блочного ввода/вывода:

```
stackcount t:block:block_rq_insert
```

Подсчет трассировок стека, приводящих к отправке IP-пакетов:

```
stackcount ip_output
```

Подсчет трассировок стека, приводящих к отправке IP-пакетов, с разделением по PID:

```
stackcount -P ip_output
```

Подсчет трассировок стека, приводящих к блокировке потока и переходу в режим ожидания:

```
stackcount t:sched:sched_switch
```

Подсчет трассировок стека, приводящих к системному вызову `read()`:

```
stackcount t:syscalls:sys_enter_read
```

4.6.6. Порядок использования stackcount

Выше я показал далеко не все возможности `stackcount(8)`, дополнительную информацию можно найти в справочном сообщении:

```
# stackcount -h
```

```
порядок использования: stackcount [-h] [-p PID] [-i INTERVAL] [-D DURATION] [-T]
                             [-r] [-s] [-P] [-K] [-U] [-v] [-d] [-f] [--debug]
                             pattern
```

Подсчитывает события и соответствующие им трассировки стека

позиционные аргументы:

pattern выражение, определяющее события

необязательные аргументы:

- h, --help вывести эту справку и выйти
- p PID, --pid PID трассировать только этот PID
- i INTERVAL, --interval INTERVAL
 интервал вывода в секундах
- d DURATION, --duration DURATION
 общая продолжительность трассировки в секундах
- T, --timestamp включить отметку времени в вывод
- r, --regex использовать регулярное выражение. По умолчанию
 допускается только символ подстановки "*"
 - s, --offset выводить смещения адресов
 - P, --perpid выводить трассировки отдельно для каждого процесса
 - K, --kernel-stacks-only
 только стек в пространстве ядра
 - U, --user-stacks-only
 только стек в пространстве пользователя
 - v, --verbose показывать простые адреса
 - d, --delimited вставлять разделитель между стеками в пространствах
 ядра/пользователя
 - f, --folded вывести результаты в свернутом формате
 - debug вывести программу BPF перед запуском (для отладки)

примеры:

```
./stackcount submit_bio      # подсчитать трассировки для submit_bio в ядре
./stackcount -d ip_output    # добавить разделитель между стеками
                             # ядра/пользователя
./stackcount -s ip_output    # показать смещения символов
./stackcount -sv ip_output   # показать смещения и простые адреса
./stackcount 'tcp_send*'     # подсчитать трассировки для функций с именами,
                             # начинающимися с tcp_send*
./stackcount -r '^tcp_send.*' # то же, но с применением регулярного выражения
./stackcount -Ti 5 ip_output  # выводить каждые 5 секунд, с отметками времени
./stackcount -p 185 ip_output # подсчитать трассировки ip_output только
                             # для PID 185
```

[...]

В ближайшее время планируется добавить параметр для ограничения глубины записываемого стека.

4.7. TRACE

trace(8)¹ — это многофункциональный инструмент BCC для трассировки отдельных событий из разных источников: kprobes, uprobes, tracepoints и USDT.

¹ Немного истории: этот инструмент был разработан Сашей Гольдштейном и включен в состав BCC 22 февраля 2016 года.

Он отвечает на вопросы:

- С какими аргументами вызывались функции из пространства ядра или пространства пользователя?
- Какое значение вернула функция? Возникла ли ошибка при ее выполнении?
- Как эта функция была вызвана? Какова трассировка стека в пространстве пользователя и в пространстве ядра?

Поскольку каждое событие выводится в отдельной строке, `trace(8)` подходит для трассировки редких событий. Очень частые события, например сетевые пакеты, переключение контекста и распределение памяти, могут происходить миллионы раз в секунду, и `trace(8)` будет сообщать о каждом из них, что повлечет значительный оверхед. Один из способов уменьшить оверхед — фильтровать и выводить только интересующие события. Но часто возникающие события лучше анализировать с помощью других инструментов, которые производят обобщение информации в пространстве ядра, — `funccount(8)`, `stackcount(8)` и `argdist(8)`. `argdist(8)` рассматривается в следующем разделе.

4.7.1. Пример trace

В примере ниже — перехват событий открытия файлов путем трассировки функции ядра `do_sys_open()` и `trace(8)`-версия `opensnoop(8)`:

```
# trace 'do_sys_open "%s", arg2'
PID    TID    COMM   FUNC           -
29588  29591  device poll   do_sys_open    /dev/bus/usb
29588  29591  device poll   do_sys_open    /dev/bus/usb/004
[...]
```

`arg2` — это второй аргумент функции `do_sys_open()` — имя открываемого файла — и имеет тип `char *`. Последний столбец («—») — это значение, определяемое строкой формата, переданной в аргументе команде `trace(8)`.

4.7.2. Синтаксис trace

С помощью параметров и аргументов можно изменять поведение `trace(8)` и задавать точки трассировки для наблюдения (одну или несколько):

```
trace [options] probe [probe ...]
```

Синтаксис аргумента `probe`:

```
probe (signature) (boolean filter) "format string", arguments
```

Сигнатуру `signature` зонда (точки трассировки) можно опустить, она нужна лишь в некоторых случаях (см. раздел 4.7.4). Фильтр `filter` тоже необязателен, и в нем

можно использовать логические операторы: `==`, `<`, `>` и `!=`. Строка формата с аргументами также необязательна. В ее отсутствие `trace(8)` выводит строку с метаданными для каждого события, но не выводит пользовательское поле.

Синтаксис `probe` напоминает синтаксис `eventname` команды `funccount(8)`, но дополнительно поддерживает возможность инструментации возврата из трассируемых функций:

- **`name`** или **`p:name`**: инструментировать функцию ядра с именем `name()`;
- **`r:name`**: инструментировать возврат из функции ядра с именем `name()`;
- **`lib:name`** или **`p:lib:name`**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в библиотеке `lib`;
- **`r:lib:name`**: инструментировать возврат из функции в пространстве пользователя с именем `name()`, находящуюся в библиотеке `lib`;
- **`path:name`**: инструментировать функцию в пространстве пользователя с именем `name()`, находящуюся в файле `path`;
- **`r:path:name`**: инструментировать возврат из функции в пространстве пользователя с именем `name()`, находящуюся в файле `path`;
- **`t:system:name`**: инструментировать точку трассировки с именем `system:name`;
- **`u:lib:name`**: инструментировать зонд USDT в библиотеке `lib` с именем `name`;
- **`*`**: подстановочный символ, соответствующий любой строке; вместо него можно использовать параметр `-r` с регулярным выражением.

В качестве строк формата поддерживаются спецификаторы `printf()`:

- **`%u`**: целое без знака;
- **`%d`**: целое со знаком;
- **`%lu`**: длинное целое без знака;
- **`%ld`**: длинное целое со знаком;
- **`%llu`**: длинное длинное целое без знака (`unsigned long long`);
- **`%lld`**: длинное длинное целое со знаком (`long long`);
- **`%hu`**: короткое целое без знака;
- **`%hd`**: короткое целое со знаком;
- **`%x`**: целое без знака в шестнадцатеричном формате;
- **`%lx`**: длинное целое без знака в шестнадцатеричном формате;
- **`%llx`**: длинное длинное целое (`unsigned long long`) без знака в шестнадцатеричном формате;
- **`%c`**: символ;
- **`%K`**: символ (имя) в пространстве ядра;
- **`%U`**: символ (имя) в пространстве пользователя;
- **`%s`**: строка.

В целом синтаксис строк формата в `trace(8)` напоминает синтаксис строк формата в других языках. Рассмотрим однострочный сценарий с `trace(8)`:

```
trace 'c:open (arg2 == 42) "%s %d", arg1, arg2'
```

Эту команду можно выразить на C-подобном языке так (дано только для примера, `trace(8)` не выполнит ее):

```
trace 'c:open { if (arg2 == 42) { printf("%s %d\n", arg1, arg2); } }'
```

Возможность настраивать аргументы события для вывода часто используется на практике, и в этом отношении `trace(8)` — незаменимый инструмент.

4.7.3. Однострочные сценарии trace

В справочном сообщении приводится множество однострочных сценариев. Здесь я покажу дополнительные сценарии.

Трассировка вызовов функции `do_sys_open()` с выводом имен открываемых файлов:

```
trace 'do_sys_open "%s", arg2'
```

Трассировка возврата из функции ядра `do_sys_open()` с выводом возвращаемого значения:

```
trace 'r::do_sys_open "ret: %d", retval'
```

Трассировка функции `do_nanosleep()` с выводом аргумента режима и трассировкой стека в пространстве пользователя:

```
trace -U 'do_nanosleep "mode: %d", arg2'
```

Трассировка запросов в библиотеку `pam` на аутентификацию:

```
trace 'pam:pam_start "%s: %s", arg1, arg2'
```

4.7.4. trace и структуры

Для вывода информации о некоторых структурах ВСС использует системные заголовочные файлы, а также заголовочные файлы ядра. Например, вот сценарий, трассирующий вызовы `do_nanosleep()` и отображающий адрес задачи:

```
trace 'do_nanosleep(struct hrtimer_sleeper *t) "task: %x", t->task'
```

К счастью, структура `hrtimer_sleeper` находится в пакете с заголовочными файлами ядра (`include/linux/hrtimer.h`), поэтому ВСС автоматически читает ее определение.

Заголовочные файлы с определениями структур, которых нет в пакете с заголовочными файлами ядра, можно подключить вручную. Например, этот однострочный

сценарий трассирует вызовы `udpv6_sendmsg()`, только когда указан порт приемника 53 (DNS; здесь число 13568 — это 32-битный номер порта 53, записанный в прямом порядке следования байтов (big-endian)):

```
trace -I 'net/sock.h' 'udpv6_sendmsg(struct sock *sk) (sk->sk_dport == 13568)'
```

Для правильной интерпретации структуры `struct sock` нужен файл `net/sock.h`, поэтому здесь он подключается с помощью параметра `-I`. Этот прием работает, только когда в системе доступен полный исходный код ядра.

Разрабатываемая новая технология представления информации о типах — BPF Type Format (BTF) — должна избавить от необходимости устанавливать исходный код ядра. Она предусматривает встраивание информации о структурах в скомпилированные двоичные файлы (см. главу 2).

4.7.5. Использование `trace` для отладки утечек дескрипторов файлов

Вот гораздо более сложный пример. Я разработал его, отлаживая проблему утечки дескрипторов файлов на промышленном сервере Netflix. Цель состояла в том, чтобы получить больше информации о дескрипторах сокетов, которые не были освобождены. Трассировка стека функции распределения нового сокета `sock_alloc()` позволяет получить такую информацию. Но мне требовался способ различать ресурсы, которые были освобождены (вызовом `sock_release()`) и которые не были освобождены. Проблема показана на рис. 4.4.

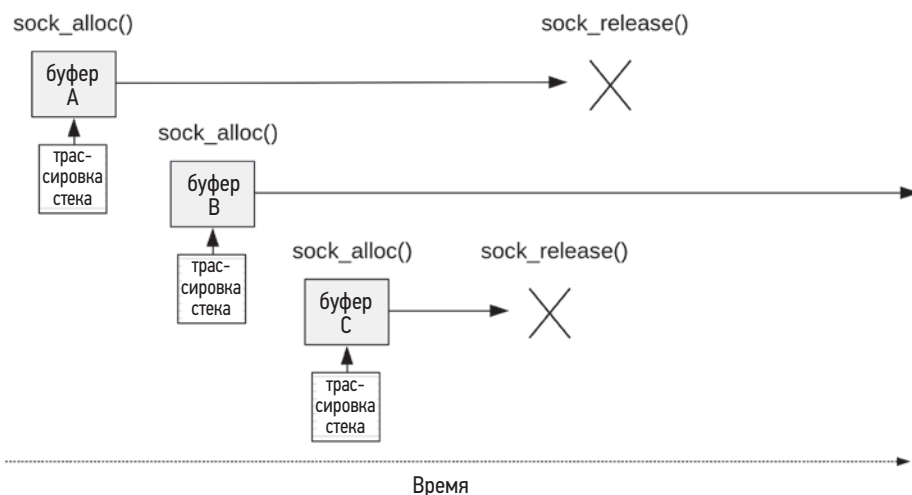


Рис. 4.4. Утечка дескрипторов сокетов

Проще всего отследить `sock_alloc()` и вывести трассировку стека, но это приведет к трассировке стека для буферов А, В и С. В данном случае интерес представляет только буфер В — тот, который не был освобожден (во время трассировки).

Мне удалось решить эту проблему с помощью однострочного сценария, хотя для этого потребовалась постобработка. Вот этот сценарий и некоторые выходные данные:

```
# trace -tKU 'r::sock_alloc "open %llx", retval' '__sock_release "close %llx", arg1'
TIME    PID    TID    COMM          FUNC          -
1.093199 4182    7101    nf.dependency.M sock_alloc      open ffff9c76526dac00
    kretprobe_trampoline+0x0 [kernel]
    sys_socket+0x55 [kernel]
    do_syscall_64+0x73 [kernel]
    entry_SYSCALL_64_after_hwframe+0x3d [kernel]
    __socket+0x7 [libc-2.27.so]
    Ljava/net/PlainSocketImpl::socketCreate+0xc7 [perf-4182.map]
    Ljava/net/Socket::setSoTimeout+0x2dc [perf-4182.map]
    Lorg/apache/http/impl/conn/DefaultClientConnectionOperator::openConnectio...
    Lorg/apache/http/impl/client/DefaultRequestDirector::tryConnect+0x60c [pe...
    Lorg/apache/http/impl/client/DefaultRequestDirector::execute+0x1674 [perf...

[...]

[...]
6.010530 4182    6797    nf.dependency.M __sock_release  close ffff9c76526dac00
    __sock_release+0x1 [kernel]
    __fput+0xea [kernel]
    ____fput+0xe [kernel]
    task_work_run+0x9d [kernel]
    exit_to_usermode_loop+0xc0 [kernel]
    do_syscall_64+0x121 [kernel]
    entry_SYSCALL_64_after_hwframe+0x3d [kernel]
    dup2+0x7 [libc-2.27.so]
    Ljava/net/PlainSocketImpl::socketClose+0xc7 [perf-4182.map]
    Ljava/net/Socket::close+0x308 [perf-4182.map]
    Lorg/apache/http/impl/conn/DefaultClientConnection::close+0x2d4 [perf-418...
```

Он инструментирует возврат из функции ядра `sock_alloc()` и выводит возвращаемое значение, адрес сокета и трассировку стека (за счет применения параметров `-K` и `-U`). Он также следит за вызовом функции ядра `__sock_release()` и за ее вторым аргументом: здесь показаны адреса закрытых сокетов. Параметр `-t` выводит отметки времени событий.

Я сократил этот вывод (вывод и стеки Java были очень длинными), чтобы показать только одну пару вызовов создания и освобождения сокета с адресом `0xffff9c76526dac00`. Я обработал полученные результаты, чтобы найти открытые, но не закрытые файловые дескрипторы (то есть дескрипторы, для которых нет соответствующего события закрытия), а затем использовал трассировку стека функции выделения дескриптора, чтобы определить пути кода, ответственные за утечку файлового дескриптора (здесь не показано).

Эту проблему можно решить и с помощью специального инструмента BCC, напоминающего memleak(8), который описан в главе 7. Он сохраняет трассировки стека в карте BPF, а затем удаляет их при появлении событий освобождения ресурса. Потом эту карту можно вывести, чтобы увидеть ресурсы, долгое время остававшиеся неосвобожденными, и соответствующие им трассировки стека.

4.7.6. Порядок использования trace

Выше я показал далеко не все возможности trace(8), дополнительную информацию можно найти в справочном сообщении:

trace -h

```
порядок использования: trace.py [-h] [-b BUFFER_PAGES] [-p PID] [-L TID] [-v]
                             [-Z STRING_SIZE] [-S] [-M MAX_EVENTS] [-t]
                             [-T] [-C] [-B] [-K] [-U] [-a] [-I header]
                             probe [probe ...]
```

Подключается к функции и выводит трассировочные сообщения.

позиционные аргументы:

probe описатель точки трассировки (см. примеры)

необязательные аргументы:

```
-h, --help                    вывести эту справку и выйти
-b BUFFER_PAGES, --buffer-pages BUFFER_PAGES
                             количество страниц для кольцевого буфера perf_events
                             (по умолчанию: 64)
-p PID, --pid PID            идентификатор процесса для трассировки (необязательно)
-L TID, --tid TID            идентификатор потока для трассировки (необязательно)
-v, --verbose                вывести получившуюся программу BPF перед выполнением
-Z STRING_SIZE, --string-size STRING_SIZE
                             максимальный размер для чтения из строк
-S, --include-self           не фильтровать собственный PID из трассировки
-M MAX_EVENTS, --max-events MAX_EVENTS
                             число событий, после вывода которых завершить
                             трассировку
-t, --timestamp              вывести столбец с отметкой времени (время,
                             прошедшее от начала трассировки)
-T, --time                   вывести столбец с текущим временем
-C, --print_cpu              вывести идентификатор процессора
-B, --bin_cmp                разрешить использовать STRCMP для двоичных значений
-K, --kernel-stack          вывести трассировку стека из пространства ядра
-U, --user-stack            вывести трассировку стека из пространства пользователя
-a, --address                вывести виртуальный адрес в стеках
-I header, --include header
                             подключить дополнительные заголовочные файлы
                             к программе BPF
```

ПРИМЕРЫ:

```
trace do_sys_open
```

```

    Трассировать системный вызов open и выводить информацию по умолчанию
trace 'do_sys_open "%s", arg2'
    Трассировать системный вызов open и выводить имена открываемых файлов
trace 'sys_read (arg3 > 20000) "read %d bytes", arg3'
    Трассировать системный вызов read и выводить сообщение для каждой
    попытки прочитать >20000 байт
trace 'r::do_sys_open "%llx", retval'
    Трассировать возврат из системного вызова open и выводить
    возвращаемое значение
trace 'c:open (arg2 == 42) "%s %d", arg1, arg2'
    Трассировать вызов open() из libc, только если аргумент
    с флагами (arg2) содержит число 42
[...]
```

Поскольку этот инструмент, поддерживающий целый мини-язык программирования, используется на практике довольно редко, примеры в конце справочного сообщения очень важны.

Утилита trace(8) — ценный инструмент, но это не полноценный язык. Описание другого, более полного языка — языка bpftrace — приводится в главе 5.

4.8. ARGDIST

argdist(8)¹ — это многофункциональный инструмент, который суммирует аргументы. Вот еще один реальный пример от Netflix: сервер Nadoor страдал от проблем с производительностью TCP, и мы отследили их до сообщений окон нулевого размера. Я использовал однострочный сценарий argdist(8), чтобы суммировать размер окна в рабочей среде. Вот часть вывода этого исследования:

```
# argdist -H 'r::__tcp_select_window():int:$retval'
[21:50:03]
```

\$retval	: count	distribution
0 -> 1	: 6100	*****
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 0	
32 -> 63	: 0	
64 -> 127	: 0	
128 -> 255	: 0	
256 -> 511	: 0	
512 -> 1023	: 0	
1024 -> 2047	: 0	
2048 -> 4095	: 0	
4096 -> 8191	: 0	
8192 -> 16383	: 24	

¹ Немного истории: этот инструмент был разработан Сашей Гольдштейном и включен в БСС 12 февраля 2016 года.

```

16384 -> 32767      : 3535      | *****
32768 -> 65535      : 1752      | *****
65536 -> 131071     : 2774      | *****
131072 -> 262143     : 1001      | *****
262144 -> 524287     : 464        | ***
524288 -> 1048575    : 3           |
1048576 -> 2097151   : 9           |
2097152 -> 4194303   : 10          |
4194304 -> 8388607   : 2           |
[21:50:04]
[...]
```

Эта команда инструментует возврат из функции ядра `__tcp_select_window()` и на основе возвращаемых значений строит гистограмму с шагом, равным степени двойки (-N). По умолчанию `argdist(8)` выводит информацию один раз в секунду. Гистограмма показывает наличие проблемы с окном нулевого размера в строке «0->1»: сообщение окна встретилось 6100 раз. Используя этот инструмент, мы проверяли, сохранилась ли проблема, пока вносили исправления в систему для ее устранения.

4.8.1. Синтаксис `argdist`

С помощью аргументов `argdist(8)` можно задать тип сводной информации, инструментуемые события и то, на основе каких данных должна строиться сводная информация:

```
argdist {-C|-N} [options] probe
```

`argdist(8)` требует передачи аргумента -C или -N:

- **-C**: подсчитать число событий;
- **-N**: вывести гистограмму с шагом между столбцами, равным степеням.

Синтаксис аргумента `probe`:

```
probe (signature)[:type[,type...]:expr[,expr...][:filter]][#label]
```

probe и **signature** имеют практически тот же синтаксис, что и в команде `trace(8)`, он отличается только отсутствием поддержки сокращенной формы записи имен функций ядра. То есть функция ядра `vfs_read()` должна указываться как «`r::vfs_read`» — сокращенная форма записи «`vfs_read`» не поддерживается. Параметр `signature` обычно обязательный. Этот аргумент можно опустить, но тогда вместо него нужно передать пустые круглые скобки ().

Аргумент **type** подсказывает тип значения для обобщения: `u32` — для 32-битных целых чисел без знака, `u64` — для 64-битных целых чисел без знака и т. д. Поддерживаются и многие другие типы, в том числе `char *` для строк.

Аргумент **expr** — это выражение для обобщения. Это может быть аргумент функции или аргумент точки трассировки. Поддерживаются также специальные

переменные, которые можно использовать только при инструментации возвратов из функций:

- **\$retval**: возвращаемое значение функции;
- **\$latency**: время от входа в функцию до возврата из нее в наносекундах;
- **\$entry(*param*)**: значение *param* при входе в зонд (точку трассировки).

Аргумент **filter** необязательный и служит для выбора событий, добавляемых в сводку. Поддерживает логические операторы: `==`, `<`, `>` и `!=`.

Аргумент **label** необязательный и служит для добавления произвольного описательного текста в вывод.

4.8.2. Однострочные сценарии argdist

В справочном сообщении приводится множество однострочных сценариев. Здесь я покажу дополнительные.

Вывести гистограмму результатов (размеров), возвращаемых функцией ядра `vfs_read()`:

```
argdist.py -H 'r::vfs_read()'
```

Вывести гистограмму результатов (размеров), возвращаемых функцией `read()` из библиотеки `libc` в пространстве пользователя для PID 1005:

```
argdist -p 1005 -H 'r:c:read()'
```

Подсчитать число обращений к системным вызовам по их идентификаторам с использованием точки трассировки `raw_syscalls:sys_enter`:

```
argdist.py -C 't:raw_syscalls:sys_enter():int:args->id'
```

Подсчитать значения аргумента `size` для `tcp_sendmsg()`:

```
argdist -C 'p::tcp_sendmsg(struct sock *sk, struct msghdr *msg, size_t size):u32:size'
```

Вывести гистограмму распределения значений аргумента `size` в вызовах `tcp_sendmsg()`:

```
argdist -H 'p::tcp_sendmsg(struct sock *sk, struct msghdr *msg, size_t size):u32:size'
```

Подсчитать количество вызовов функции `write()` из библиотеки `libc` для PID 181 по дескрипторам файлов:

```
argdist -p 181 -C 'p:c:write(int fd):int:fd'
```

Подсчитать операции чтения по процессам, для которых величина задержки была > 0.1 мс:

```
argdist -C 'r::__vfs_read():u32:$PID:$latency > 100000
```

4.8.3. Порядок использования argdist

Выше я показал далеко не все возможности argdist(8), дополнительную информацию можно найти в справочном сообщении:

argdist.py -h

```
порядок использования: argdist.py [-h] [-p PID] [-z STRING_SIZE] [-i INTERVAL]
                             [-d DURATION] [-n COUNT] [-v] [-c] [-T TOP]
                             [-H specifier] [-C specifier] [-I header]
```

Трассирует функцию и выводит сводную информацию о значениях ее параметров.

необязательные аргументы:

```
-h, --help                вывести эту справку и выйти
-p PID, --pid PID         идентификатор процесса для трассировки (необязательно)
-z STRING_SIZE, --string-size STRING_SIZE
                           максимальный размер для чтения из строковых аргументов
                           char*
-i INTERVAL, --interval INTERVAL
                           интервал вывода в секундах (по умолчанию 1 секунда)
-d DURATION, --duration DURATION
                           общая продолжительность трассировки в секундах
-n COUNT, --number COUNT
                           сколько раз вывести результаты
-v, --verbose             вывести получившуюся программу BPF перед выполнением
-c, --cumulative           не очищать гистограммы и счетчики в начале каждого
                           интервала
-T TOP, --top TOP         число выводимых ТОП-результатов (не имеет смысла
                           для гистограмм)
-H specifier, --histogram specifier
                           определение зонда для создания гистограммы (см.
                           примеры ниже)
-C specifier, --count specifier
                           определение зонда для подсчета (см. примеры ниже)
-I header, --include header
                           подключить дополнительные заголовочные файлы
                           к программе BPF, можно указать полный путь или
                           относительный -- от текущего рабочего каталога
                           или от начала пути поиска по умолчанию заголовочных
                           файлов ядра
```

Синтаксис определения зонда:

```
{p,r,t,u}:{[library],category}:function(signature)
[:type[,type...]:expr[,expr...][:filter]][#label]
```

Где:

```
p,r,t,u    -- вход в функцию, выход из функции, точка трассировки
              ядра или зонд USDT
```

в определения зондов, соответствующих выходу из функций, можно использовать специальные переменные `$retval`, `$entry(param)`, `$latency`

<code>library</code>	-- библиотека, содержащая функцию (для функций ядра не указывается)
<code>category</code>	-- категория точки трассировки ядра (например, <code>net</code> , <code>sched</code>)
<code>function</code>	-- имя функции для трассировки (или имя точки трассировки)
<code>signature</code>	-- параметры функции, как определено в заголовочном файле на C
<code>type</code>	-- тип выражения отбора (поддерживается несколько типов)
<code>expr</code>	-- выражение отбора (поддерживается несколько выражений)
<code>filter</code>	-- фильтр для выбора отбираемых значений
<code>label</code>	-- метка этого зонда для отображения в выводе

ПРИМЕРЫ:

```
argdist -H 'p::__kmalloc(u64 size):u64:size'
Вывести гистограмму объемов памяти, выделяемой обращением к kmalloc
argdist -p 1005 -C 'p:c:malloc(size_t size):size_t:size:size==16'
Подсчитать, как часто процесс с PID 1005 вызывает malloc,
чтобы выделить блок памяти с размером 16 байт
argdist -C 'r:c:gets():char*:(char*)$retval#snooped strings'
Получить все строки, возвращаемые функцией gets()
argdist -H 'r::__kmalloc(size_t size):u64:$latency/$entry(size)#ns per byte'
Вывести гистограмму распределения времени в наносекундах на байт,
затраченного функцией kmalloc
argdist -C 'p::__kmalloc(size_t sz, gfp_t flags):size_t:sz:flags&GFP_ATOMIC'
Подсчитать, как часто вызывается kmalloc для выделения блоков памяти
с размером GFP_ATOMIC
[...]
```

`argdist(8)` позволяет создавать весьма мощные однострочные сценарии. В главе 5 я покажу дополнительные возможности получения сводной информации о распределении, которые не поддерживаются `argdist(8)`.

4.9. ДОКУМЕНТАЦИЯ ИНСТРУМЕНТОВ

Для каждого инструмента ВСС есть своя страница справочного руководства (`man`) и файл с примерами. В каталоге ВСС/`examples` есть несколько примеров кода, которые действуют как инструменты, но не имеют документации с описанием за пределами их кода. Для инструментов из каталога `/tools` или установленных в другое место в вашей системе из дистрибутива должна быть документация.

В разделах ниже обсуждается документация инструментов на примере `opensnoop(8)`.

4.9.1. Страница справочного руководства: `opensnoop`

Установив инструменты с помощью диспетчера пакетов, можно обнаружить, что команда `man opensnoop` работает. При исследовании содержимого репозитория для форматирования страниц справочного руководства (в формате ROFF) воспользуйтесь командой `proff(1)`.

Структура man-страниц инструментов повторяет структуру аналогичных страниц с описанием утилит Linux. За прошедшие годы я совершенствовал подход к наполнению страницы справочного руководства, уделяя особое внимание деталям¹. Эта справочная страница, например, содержит мои пояснения и советы:

```
bcc$ nroff -man man/man8/opensnoop.8
```

```
opensnoop(8)                      System Manager's Manual                      opensnoop(8)
```

ИМЯ

opensnoop - Трассирует системные вызовы `open()`. Использует eBPF/bcc в ядре Linux.

ОБЗОР

```
opensnoop.py [-h] [-T] [-U] [-x] [-p PID] [-t TID] [-u UID]
              [-d DURATION] [-n NAME] [-e] [-f FLAG_FILTER]
```

ОПИСАНИЕ

opensnoop трассирует системный вызов `open()`, показывает, какие процессы и какие файлы открывают. Это пригодится для определения местоположения файлов журналов и конфигураций или для поиска причин аварийного завершения приложений, особенно в момент запуска.

С этой целью трассируется функция ядра `sys_open()` с использованием механизма динамической трассировки, который должен обновляться, чтобы соответствовать любым изменениям в этой функции.

Использует функцию `bpf_perf_event_output()`, появившуюся в Linux 4.5; для более старых ядер 4.5 см. раздел `tools/old` (инструменты/старые), в котором перечисляются инструменты, использующие старый механизм.

Так как этот инструмент использует BPF, применять его может только пользователь `root`.

[...]

Эта страница находится в разделе 8, потому что opensnoop — это команда системного администрирования, требующая привилегий суперпользователя, о чем отдельно говорится в конце раздела ОПИСАНИЕ (DESCRIPTION). В будущем расширенный BPF может стать доступным для пользователей, не обладающих привилегиями `root`, как и команда `perf(1)`. Если это случится, эти страницы будут перемещены в раздел 1.

Раздел ИМЯ (NAME) включает краткое описание инструмента. В нем говорится, что он предназначен для Linux и использует eBPF/BCC (я разработал несколько версий этих инструментов для разных ОС и трассировщиков).

В разделе ОБЗОР (SYNOPSIS) перечисляются параметры командной строки.

¹ Я написал и опубликовал более 200 страниц справочного руководства для разработанных мной инструментов оценки производительности.

В разделе ОПИСАНИЕ рассказывается, что делает инструмент и чем он полезен. Очень важно описать пользу инструмента как можно более простыми словами, и не менее важно рассказать, какие реальные проблемы он решает (что очевидно не для всех). Эта информация помогает разработчику лишний раз убедиться в полезности инструмента. Иногда, пытаясь написать этот раздел, я осознавал, что у описываемого инструмента слишком узкий круг применения, чтобы выпускать его в обращение.

В разделе ОПИСАНИЕ указывайте основные предостережения. Лучше предупредить пользователей о проблеме, чем позволить им столкнуться с ней в самый неожиданный момент. К ним относится, например, стандартное предупреждение о стабильности динамической трассировки и требуемых версиях ядра.

Продолжим:

ТРЕБОВАНИЯ

CONFIG_BPF и bcc.

ПАРАМЕТРЫ:

-h Вывести справочное сообщение.
-T Добавить столбец с отметкой времени.

[...]

В разделе ТРЕБОВАНИЯ (REQUIREMENTS) перечисляются все особенности, а в разделе ПАРАМЕТРЫ (OPTIONS) — все параметры командной строки:

ПРИМЕРЫ

Трассировать все системные вызовы open():
opensnoop

Трассировать все системные вызовы open() в течение 10 секунд:
opensnoop -d 10

[...]

В разделе ПРИМЕРЫ (EXAMPLES) описываются различные возможности инструмента и приводятся приемы использования. Пожалуй, это самый полезный раздел.

ПОЛЯ

TIME(s) Время вызова в секундах.

UID Идентификатор пользователя

PID Идентификатор процесса

TID Идентификатор потока

COMM Имя процесса

FD Дескриптор файла (в случае успеха) или -1 (в случае ошибки)

ERR Код ошибки (см. errno.h)

[...]

В разделе ПОЛЯ (FIELDS) описываются все поля, которые могут быть в выводе инструмента. Если поля имеют единицы измерения, они должны быть указаны. В этом примере говорится, что значение в поле «TIME(s)» измеряется в секундах.

ОВЕРХЕД

Трассирует функцию ядра `open` и выводит строку для каждого события. Обычно частота вызовов невелика ($< 1000/c$), поэтому оверхед тоже относительно невелик. Если ваше приложение очень часто вызывает `open()`, протестируйте инструмент с ним, определите величину оверхеда и только потом используйте.

Раздел ОВЕРХЕД (OVERHEAD) описывает величину ожидаемого оверхеда. Если пользователь будет знать о высоком оверхеде, то сможет подготовиться и успешно использовать инструмент. В этом примере утверждается, что оверхед будет низким.

ИСТОЧНИК

Этот инструмент входит в набор `bcc`.

<https://github.com/iovisor/bcc>

Также загляните в файл `_examples.txt`, распространяемый вместе с `bcc`, в нем вы найдете примеры использования этого инструмента, вывода и дополнительные комментарии.

ОС

Linux

СТАБИЛЬНОСТЬ

Нестабилен - находится в разработке.

АВТОР

Brendan Gregg

СМ. ТАКЖЕ

`funccount(1)`

В последних разделах отмечается, что этот инструмент входит в набор инструментов БСС, и перечисляются дополнительные источники информации: файл с примерами и родственные инструменты в разделе СМ. ТАКЖЕ (SEE ALSO).

Если есть версии инструмента для других трассировщиков или основанные на других механизмах, лучше явно сообщить об этом в справке. Для многих инструментов БСС были созданы версии, использующие `bpfttrace`, и соответствующие им справочные страницы в `bpfttrace` говорят об этом в разделах ИСТОЧНИК (SOURCE).

4.9.2. Файл с примерами: `opensnoop`

Просмотр примеров вывода — один из лучших способов объяснения инструментов, поскольку их вывод бывает интуитивно понятным, а это признак хорошего дизайна

инструмента. У каждого инструмента в ВСС есть специальный текстовый файл с примерами.

В первой строке в этом файле сообщаются название и версия инструмента. Затем идут примеры вывода — от простого к сложному:

```
bcc$ more tools/opensnoop_example.txt
```

Примеры использования opensnoop, eBPF/bcc-версии для Linux.

opensnoop трассирует все обращения к системному вызову `open()` в системе и выводит различные детали этих обращений. Пример вывода:

```
# ./opensnoop
PID    COMM      FD ERR PATH
17326  <...>      7  0  /sys/kernel/debug/tracing/trace_pipe
1576   snmpd      9  0  /proc/net/dev
1576   snmpd     11  0  /proc/net/if_inet6
1576   snmpd     11  0  /proc/sys/net/ipv4/neigh/eth0/retrans_time_ms
[...]
```

Пока выполнялась трассировка, процесс `snmpd` открывал различные файлы в каталоге `/proc` (для чтения метрик), а процесс `"run"` читал различные библиотеки и файлы конфигурации (похоже, что это запуск нового процесса).

С помощью `opensnoop` можно определить, какие конфигурационные файлы и файлы журналов использовало приложение в момент запуска.

Параметр `-p` позволяет фильтровать события по PID, сама фильтрация происходит в ядре. В этом примере я использовал параметр `-T` для вывода отметок времени:

```
./opensnoop -Tp 1956
TIME(s)  PID    COMM      FD ERR PATH
0.000000000 1956  supervise  9  0  supervise/status.new
0.000289999 1956  supervise  9  0  supervise/status.new
1.023068000 1956  supervise  9  0  supervise/status.new
1.023381997 1956  supervise  9  0  supervise/status.new
2.046030000 1956  supervise  9  0  supervise/status.new
2.046363000 1956  supervise  9  0  supervise/status.new
3.068203997 1956  supervise  9  0  supervise/status.new
3.068544999 1956  supervise  9  0  supervise/status.new
```

Как показывает этот вывод, каждую секунду процесс `supervise` дважды открывал файл `status.new`.
[...]

В файле примеров подробно объясняется, что выводит инструмент, особенно это относится к первому примеру.

В конце файла примеров приведена копия сообщения о порядке использования. Это может показаться излишним, но бывает полезно при просмотре онлайн. Обычно в файлах примеров показаны не все параметры, поэтому наличие сообщения о порядке использования в конце файла поможет узнать, что еще умеет инструмент.

4.10. РАЗРАБОТКА ИНСТРУМЕНТОВ ВСС

Поскольку большинство читателей предпочтут программировать на более высокоуровневом языке `bpfttrace`, основное внимание в этой книге уделяется разработке инструментов на основе `bpfttrace`, а ВСС используется как источник ранее написанных инструментов. Разработка инструментов для ВСС описана в приложении С.

Зачем разрабатывать инструменты для ВСС, если есть более удобный `bpfttrace`? ВСС подходит для создания сложных инструментов с различными аргументами и параметрами командной строки и предоставляет полную свободу выбора в настройке вывода и выполняемых действий. Например, инструмент ВСС может использовать сетевые библиотеки для отправки данных о событиях на сервер сообщений или в базу данных. Для сравнения, `bpfttrace` хорошо подходит для однострочных сценариев или коротких инструментов, которые не принимают никаких аргументов или принимают один аргумент и выводят только текст.

ВСС также предоставляет низкоуровневый интерфейс для BPF-программ, написанных на С, а также для компонентов, действующих в пространстве пользователя и написанных на Python или других поддерживаемых языках. Но такая гибкость дается не бесплатно: на разработку инструментов ВСС может уйти в десять раз больше времени, чем на разработку инструментов `bpfttrace`, и может потребоваться написать в десять раз больше кода.

Независимо от выбора основы — ВСС или `bpfttrace`, базовую функциональность инструмента часто можно без особого труда перенести с одной на другую, когда вы окончательно определитесь с выбором. Также `bpfttrace` можно использовать для разработки прототипа и проверки концепций, а потом полностью переписать инструмент для ВСС.

Ссылки на ресурсы, советы по разработке инструментов ВСС и примеры кода с пояснениями приводятся в приложении С.

В следующих разделах рассмотрены внутреннее устройство ВСС и приемы отладки. Даже если вы только используете, а не разрабатываете инструменты ВСС, все равно могут возникнуть ситуации, когда придется отладить инструмент. Понимание внутренних механизмов ВСС поможет в этом.

4.11. ВНУТРЕННЕЕ УСТРОЙСТВО ВСС

В состав ВСС входят:

- программный интерфейс для С++ для конструирования BPF-программ в пространстве ядра, в том числе:
 - препроцессор для преобразования ссылок на память в вызовы функции `bpf_probe_read()` (и варианты этой функции, которые могут появиться в будущих ядрах);

- драйверы на C++ для:
 - компиляции BPF-программы с использованием Clang/LLVM;
 - загрузки BPF-программы в ядро;
 - подключения BPF-программы к событиям;
 - чтения/записи карт BPF;
- API для создания инструментов BPF на Python, C++ и Lua.

Все это показано на рис. 4.5.

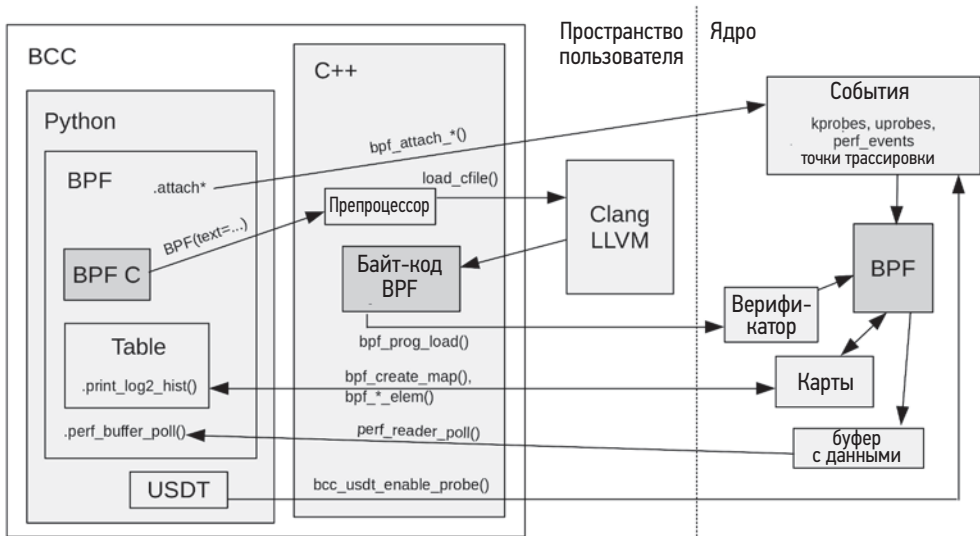


Рис. 4.5. Внутреннее устройство BCC

Объекты BPF, Table и USDT на Python, изображенные на рис. 4.5, — это обертки для соответствующих реализаций в `libbcc` и `libbcc_bpf`.

Объект Table взаимодействует с картами BPF. Эти таблицы доступны как элементы объекта BPF (через «магические методы» в Python: `__getitem__`), поэтому следующие две строки эквивалентны:

```
counts = b.get_table("counts")
counts = b["counts"]
```

USDT — это отдельный объект в Python, потому что его поведение отличается от `kprobes`, `uprobes` и точек трассировки. На этапе инициализации он должен подключаться к идентификатору процесса или пути в файловой системе, потому что, в отличие от других типов событий, некоторые зонды USDT требуют установки семафоров в образе процесса для их активации. С помощью этих семафоров приложение может определить, используется ли зонд USDT сейчас и нужно ли

подготовить его аргументы или эту операцию можно пропустить для увеличения производительности.

Компоненты C++ скомпилированы в библиотеки `libbss_bpf` и `libbss`, которые используются также другим ПО (например, `bpftrace`). `libbss_bpf` компилируется из исходного кода ядра Linux в `tools/lib/bpf` (возникшего из ВСС).

Ниже перечислены шаги, которые выполняет ВСС, чтобы загрузить BPF-программу и инструментировать события:

1. Создается объект BPF на Python, и ему передается BPF-программа на C.
2. Препроцессор ВСС обрабатывает BPF-программу на C и заменяет обращения к памяти вызовами `bpf_probe_read()`.
3. Clang компилирует BPF-программу на C в промежуточный код LLVM IR.
4. Генератор кода ВСС добавляет дополнительные инструкции LLVM IR, если необходимо.
5. LLVM компилирует IR-код в байт-код BPF.
6. При необходимости создаются карты.
7. Байт-код передается в ядро и проверяется верификатором BPF.
8. Включаются события, и BPF-программа подключается к ним.
9. Программа ВСС читает полученные данные из карт или из кольцевого буфера `perf_event`.

Следующий раздел проливает больше света на внутреннее устройство.

4.12. ОТЛАДКА ВСС

Есть разные способы отладки проблем в инструментах ВСС, кроме вставки вызовов `printf()`. В этом разделе рассмотрим инструкции вывода, режимы отладки ВСС, `bpflist` и приемы сброса событий. Если вы читаете этот раздел в поисках помощи в устранении неполадок, обратитесь к главе 18, где описаны распространенные проблемы — пропуск событий, стеков и символов.

На рис. 4.6 изображен процесс компиляции программы и различные инструменты отладки, которые можно использовать для проверки.

Более подробно эти инструменты рассматриваются в следующих разделах.

4.12.1. Отладка с помощью `printf()`

Отладка с помощью `printf()` может показаться грубым приемом по сравнению с использованием более сложных инструментов отладки, но при этом она быстрая и эффективная. Инструкции `printf()` можно добавлять не только в код на

Python, но также в код BPF. Для этого есть специальная вспомогательная функция `bpf_trace_printk()`. Она отправляет вывод в специальный буфер Ftrace, который можно прочитать с помощью `cat(1)` из файлов `/sys/kernel/debug/tracing/trace_pipe`.

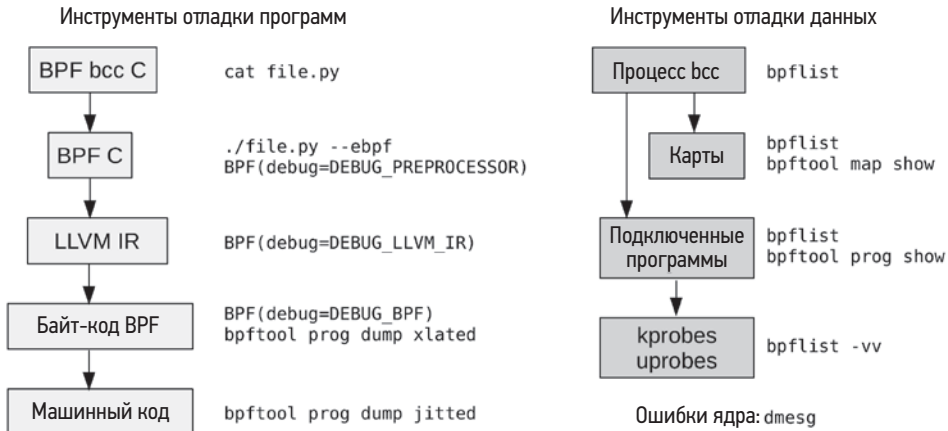


Рис. 4.6. Отладка BCC

Представьте, что вы столкнулись с проблемой в `biol latency(8)`: он компилируется и работает, но вывод выглядит неправильно. Вы можете вставить инструкцию `printf()`, чтобы убедиться, что зонды срабатывают и используемые переменные имеют ожидаемые значения. Вот пример добавления инструкции вывода в `biol latency.py` (выделена жирным):

```
[...]
// время блокировки ввода/вывода
int trace_req_start(struct pt_regs *ctx, struct request *req)
{
    u64 ts = bpf_ktime_get_ns();
    start.update(&req, &ts);
    bpf_trace_printk("BDG req=%llx ts=%lld\\n", req, ts);
    return 0;
}
[...]
```

Здесь BDG — это мои инициалы. Я добавил их, чтобы различать вывод, произведенный в моем сеансе отладки.

Теперь запустим инструмент:

```
# ./biol latency.py
Tracing block device I/O... Hit Ctrl-C to end.
```

и с помощью `cat(1)` прочитаем содержимое Ftrace-файла `trace_pipe` в другом терминале:

```
# cat /sys/kernel/debug/tracing/trace_pipe
[...]
```

kworker/4:1H-409	[004]		2542952.834645:	0x00000001: BDG
req=ffff8934c90a1a00 ts=2543018287130107				
dmcrypt_write-354	[004]		2542952.836083:	0x00000001: BDG
req=ffff8934c7df3600 ts=2543018288564980				
dmcrypt_write-354	[004]		2542952.836093:	0x00000001: BDG
req=ffff8934c7df3800 ts=2543018288578569				
kworker/4:1H-409	[004]		2542952.836260:	0x00000001: BDG
req=ffff8934c90a1a00 ts=2543018288744416				
kworker/4:1H-409	[004]		2542952.837447:	0x00000001: BDG
req=ffff8934c7df3800 ts=2543018289932052				
dmcrypt_write-354	[004]		2542953.611762:	0x00000001: BDG
req=ffff8934c7df3800 ts=2543019064251153				
kworker/u16:4-5415	[005]	d...	2542954.163671:	0x00000001: BDG
req=ffff8931622fa000 ts=2543019616168785				

Вывод включает различные поля, которые по умолчанию добавляет Ftrace, а за ними следует наше сообщение, переданное в вызов `bpf_trace_printk()`.

Если команде `cat(1)` передать файл `trace` вместо `trace_pipe`, она выведет заголовки:

```
# cat /sys/kernel/debug/tracing/trace
# tracer: nop
#
#
#          _-----> irqs-off
#          / _-----> need-resched
#          | / _-----> hardirq/softirq
#          || / _-----> preempt-depth
#          ||| / _-----> delay
#
# TASK-PID   CPU#  ||||   TIMESTAMP   FUNCTION
#   |   |   |   |   |   |   |   |
# kworker/u16:1-31496 [000] d... 2543476.300415: 0x00000001: BDG
req=ffff89345af53c00 ts=2543541760130509
# kworker/u16:4-5415 [000] d... 2543478.316378: 0x00000001: BDG
req=ffff89345af54c00 ts=2543543776117611
[...]
```

Вот основные различия между этими файлами:

- **trace**: выводит заголовки, не блокирует выполнение;
- **trace_pipe**: блокирует выполнение до появления следующего сообщения и очищает сообщения после чтения.

Буфер Ftrace (доступный через файлы `trace` и `trace_pipe`) используется также другими инструментами Ftrace, поэтому ваши отладочные сообщения могут перемешиваться с другими сообщениями. Этот прием хорош для отладки — вы можете отфильтровать сообщения, чтобы увидеть только те, которые вам интересны (так, для этого примера можно использовать: `grep BDG /sys/.../trace`).

С помощью инструмента `bpftool(8)`, описанного в главе 2, можно вывести содержимое буфера Ftrace, выполнив команду `bpftool prog tracelog`.

4.12.2. Отладочный вывод BCC

Некоторые инструменты, например `funccount(8)` `-D`, уже имеют параметры для получения отладочного вывода. Прочитайте справочное сообщение (вызвав команду с параметром `-h` или `--help`), возможно, инструмент уже имеет такой параметр. Многие инструменты поддерживают недокументированный параметр `--ebpf`, который обеспечивает вывод программы BPF, сгенерированной инструментом¹. Например:

```
# opensnoop --ebpf

#include <uapi/linux/ptrace.h>
#include <uapi/linux/limits.h>
#include <linux/sched.h>

struct val_t {
    u64 id;
    char comm[TASK_COMM_LEN];
    const char *fname;
};

struct data_t {
    u64 id;
    u64 ts;
    u32 uid;
    int ret;
    char comm[TASK_COMM_LEN];
    char fname[NAME_MAX];
};

BPF_HASH(infotmp, u64, struct val_t);
BPF_PERF_OUTPUT(events);

int trace_entry(struct pt_regs *ctx, int dfd, const char __user *filename, int
flags)
{
    struct val_t val = {};
    u64 id = bpf_get_current_pid_tgid();
    u32 pid = id >> 32; // PID хранится в старшей половине слова
    u32 tid = id;       // Привести к типу u32 и получить младшую половину
    u32 uid = bpf_get_current_uid_gid();
    [...]
```

Эту возможность удобно использовать, когда ядро отказывается выполнять программу BPF: можно вывести ее и попробовать определить причину.

¹ Параметр `--ebpf` был добавлен для поддержки BCC PCP PMDA (см. главу 17), и поскольку этот параметр не предназначен для конечного пользователя, он не упоминается в справочном сообщении, чтобы исключить его хаотичное использование.

4.12.3. Флаги отладки BCC

BCC поддерживает возможность отладки, доступную для всех инструментов: добавление флагов отладки в инициализатор объекта BPF. Например, в `opensnoop` есть строка:

```
b = BPF(text=bpf_text)
```

Ее можно изменить и добавить параметр `debug`:

```
b = BPF(text=bpf_text, debug=0x2)
```

В этом случае при работе программы BPF выведутся выполняемые ею инструкции:

```
# opensnoop
0: (79) r7 = *(u64 *)(r1 +104)
1: (b7) r1 = 0
2: (7b) *(u64 *)(r10 -8) = r1
3: (7b) *(u64 *)(r10 -16) = r1
4: (7b) *(u64 *)(r10 -24) = r1
5: (7b) *(u64 *)(r10 -32) = r1
6: (85) call bpf_get_current_pid_tgid#14
7: (bf) r6 = r0
8: (7b) *(u64 *)(r10 -40) = r6
9: (85) call bpf_get_current_uid_gid#15
10: (bf) r1 = r10
11: (07) r1 += -24
12: (b7) r2 = 16
13: (85) call bpf_get_current_comm#16
14: (67) r0 <= 32
[...]
```

Параметр `debug` — это набор однобитовых флагов, которые можно комбинировать. Они перечислены в `src/cc/bpf_module.h` и показаны здесь:

Бит	Имя	Описание
0x1	DEBUG_LLVM_IR	Выводит промежуточный код, полученный компилятором LLVM
0x2	DEBUG_BPF	Выводит байт-код BPF и состояние регистров
0x4	DEBUG_PREPROCESSOR	Выводит результат обработки препроцессором (напоминает действие параметра <code>--ebpf</code>)
0x8	DEBUG_SOURCE	Выводит инструкции на языке Ассемблера со встроенным исходным кодом
0x10	DEBUG_BPF_REGISTER_STATE	Выводит состояние регистров для всех инструкций
0x20	DEBUG_BTF	Выводит отладочную информацию BTF (в иных случаях ошибки BTF игнорируются)

Если передать параметр `debug=0x1f`, будет выведена полная отладочная информация, которая может занять десятки страниц.

4.12.4. bpflist

Инструмент `bpflist(8)` выводит список инструментов, выполняющих программы BPF, и некоторые подробности. Например:

```
# bpflist
PID      COMM          TYPE  COUNT
30231    opensnoop      prog   2
30231    opensnoop      map    2
```

Как показывает вывод, в данном случае выполняется инструмент `opensnoop(8)` с PID 30231, использующий две программы BPF и две карты. Это нормальная ситуация: `opensnoop(8)` инструментитрует два события отдельными программами BPF и использует одну карту для обмена информацией с зондами и другую карту — для передачи данных в пользовательское пространство.

В режиме `-v` (`verbose` — подробный вывод) инструмент подсчитывает `kprobes` и `uprobes`, а в режиме `-vv` (`very verbose` — очень подробный вывод) — подсчитывает и выводит списки `kprobes` и `uprobes`. Например:

```
# bpflist -vv
open kprobes:
p:kprobes/p_do_sys_open_bcc_31364 do_sys_open
r:kprobes/r_do_sys_open_bcc_31364 do_sys_open

open uprobes:

PID      COMM          TYPE  COUNT
1        systemd      prog   6
1        systemd      map    6
31364    opensnoop    map    2
31364    opensnoop    kprobe  2
31364    opensnoop    prog    2
```

Как показывает вывод, здесь запущено две программы BPF: `systemd` (PID 1) и `opensnoop` (PID 31364). В режиме `-vv` дополнительно выводится список открытых `kprobes` и `uprobes`. Обратите внимание, что PID потребителя — 31364 — включен в имена `kprobes`.

4.12.5. bpftool

Инструмент `bpftool` находится в дереве исходных текстов Linux и может выводить списки запущенных программ и выполняемых инструкций BPF, взаимодействовать с картами и многое другое. Я рассказывал о нем в главе 2.

4.12.6. dmesg

Иногда сообщения об ошибках, возникающих в BPF или в источниках событий, выводятся в системный журнал. Их можно просмотреть с помощью `dmesg(1)`. Например:

```
# dmesg
[...]
[8470906.869945] trace_kprobe: Could not insert probe at vfs_read+0: -2
```

Эта ошибка возникла при попытке создать kprobe в функции ядра `vfs_read()`. Это явная опечатка, потому что функции `vfs_read()` нет.

4.12.7. Сброс событий

Разработка ПО обычно включает цикл написания нового кода и последующего исправления ошибок. Появление ошибки в инструментах или библиотеках ВСС может вызвать сбой в работе ВСС после включения трассировки, из-за чего источники событий в ядре останутся включенными, но при этом не будет ни одного процесса, использующего их события, что повлечет ненужный оверхед.

Такая проблема была у старых интерфейсов на основе Ftrace в `/sys`, которые первоначально использовались в ВСС для управления всеми источниками событий, кроме `perf_events` — счетчиков производительности (PMC). В `perf_events` применялась функция `perf_event_open()`, использующая дескрипторы файлов. Преимущество `perf_event_open()` в том, что в случае сбоя процесс запускает процедуру освобождения файловых дескрипторов в ядре, которая, в свою очередь, выполняет отключение соответствующих источников событий. В Linux 4.17 и выше в ВСС начал использоваться интерфейс `perf_event_open()` для всех источников событий, поэтому проблема источников, оставшихся включенными после сбоя, ушла в прошлое.

Пользователи более старых ядер могут прибегнуть к помощи инструмента ВСС с именем `reset-trace.sh`, который сбрасывает состояние Ftrace в ядре, удаляя все включенные события трассировки. Используйте этот инструмент, только если знаете, что в системе нет других потребителей событий (не только ВСС, но и любых других трассировщиков), потому что он прервет работу всех включенных источников событий.

Вот вывод, полученный в результате запуска инструмента на одном из моих серверов разработки ВСС:

```
# reset-trace.sh -v
Reseting tracing state...

Checking /sys/kernel/debug/tracing/kprobe_events
Needed to reset /sys/kernel/debug/tracing/kprobe_events
kprobe_events, before (line enumerated):
  1 r:kprobes/r_d_lookup_1_bcc_22344 d_lookup
  2 p:kprobes/p_d_lookup_1_bcc_22344 d_lookup
  3 p:kprobes/p_lookup_fast_1_bcc_22344 lookup_fast
  4 p:kprobes/p_sys_execve_1_bcc_12659 sys_execve
[...]
kprobe_events, after (line enumerated):

Checking /sys/kernel/debug/tracing/uprobe_events
Needed to reset /sys/kernel/debug/tracing/uprobe_events
```



```

uprobe_events, before (line enumerated):
  1 p:uprobes/p__proc_self_exe_174476_1_bcc_22344 /proc/self/exe:0x0000000000174476
  2 p:uprobes/p__bin_bash_ad610_1_bcc_12827 /bin/bash:0x0000000000ad610
  3 r:uprobes/r__bin_bash_ad610_1_bcc_12833 /bin/bash:0x0000000000ad610
  4 p:uprobes/p__bin_bash_8b860_1_bcc_23181 /bin/bash:0x00000000008b860
[...]
uprobe_events, after (line enumerated):

Checking /sys/kernel/debug/tracing/trace
Checking /sys/kernel/debug/tracing/current_tracer
Checking /sys/kernel/debug/tracing/set_ftrace_filter
Checking /sys/kernel/debug/tracing/set_graph_function
Checking /sys/kernel/debug/tracing/set_ftrace_pid
Checking /sys/kernel/debug/tracing/events/enable
Checking /sys/kernel/debug/tracing/tracing_thresh
Checking /sys/kernel/debug/tracing/tracing_on

Done.

```

В режиме подробного вывода (-v) инструмент reset-trace.sh сообщает обо всех выполняемых шагах. Пустые строки после сброса kprobe_events и uprobe_events говорят об успехе.

4.13. ИТОГИ

Проект ВСС включает более 70 инструментов BPF, многие из которых поддерживают возможность настройки поведения с помощью параметров командной строки. Все они снабжены документацией: страницы в справочном руководстве и файлы примеров. Большинство из них специализированные и предназначены для наблюдения за одним видом деятельности. Некоторые инструменты многоцелевые. В этой главе я рассмотрел четыре таких инструмента: funccount(8) — для подсчета событий, stackcount(8) — для подсчета путей выполнения, приведших к определенным событиям, trace(8) — для получения информации о каждом из заданных событий и argdist(8) — для получения сведений об аргументах событий в виде счетчиков или гистограмм. В этой главе вы также познакомились с инструментами отладки ВСС. В приложении С приведены примеры разработки новых инструментов ВСС.

Глава 5

BPFTRACE

bpftrace — это трассировщик с открытым исходным кодом, основанный на BPF и BCC. Проект bpftrace, так же как BCC, включает множество инструментов оценки производительности с сопроводительной документацией. Он также предоставляет язык программирования высокого уровня, позволяющий писать мощные инструменты.

Вот как с помощью bpftrace можно обобщить информацию, возвращаемую системным вызовом `vfs_read()` (байты или коды ошибок), и представить ее в виде гистограммы:

```
# bpftrace -e 'kretprobe:vfs_read { @bytes = hist(retval); }'
```

```
Attaching 1 probe...
```

```
^C
```

```
@bytes:
```

(..., 0)	223	@@@@@@@@@@@@@@	
[0]	110	@@@@@@	
[1]	581	@@@	
[2, 4)	23	@	
[4, 8)	9		
[8, 16)	844	@@@	
[16, 32)	44	@@	
[32, 64)	67	@@@@	
[64, 128)	50	@@@	
[128, 256)	24	@	
[256, 512)	1		

Проект bpftrace был создан Аластером Робертсоном в декабре 2016 года. Поскольку bpftrace выглядел хорошо продуманным и прекрасно вписывался в набор инструментов BCC/LLVM/BPF, я присоединился к проекту и участвовал в разработке кода, инструментов оценки производительности и документации. Постепенно к нам присоединились другие разработчики, и к концу 2018 года мы закончили реализацию набора основных возможностей.

Эта глава познакомит вас с проектом bpftrace и его возможностями, некоторыми из его инструментов и документацией, а также с языком программирования bpftrace. В заключение я представлю обзор приемов отладки и внутреннего устройства bpftrace.

Цели обучения:

- получить представление о возможностях bpftrace и сравнить их с возможностями других инструментов;
- узнать, где искать инструменты и документацию и как использовать эти инструменты;
- научиться читать исходный код на языке bpftrace, который часто будет встречаться в следующих главах;
- разработать новые однострочные сценарии и инструменты на языке bpftrace;
- (дополнительно) познакомиться с внутренним устройством bpftrace.

Если вы хотите сразу же приступить к изучению программирования на языке bpftrace, переходите к разделу 5.7, а затем возвращайтесь сюда, чтобы продолжить знакомство с bpftrace.

bpftrace идеально подходит для создания специализированных инструментов и коротких сценариев, тогда как ВСС больше подходит для разработки сложных инструментов и демонов.

5.1. КОМПОНЕНТЫ BPFTRACE

На рис. 5.1 показана общая структура каталогов проекта bpftrace.

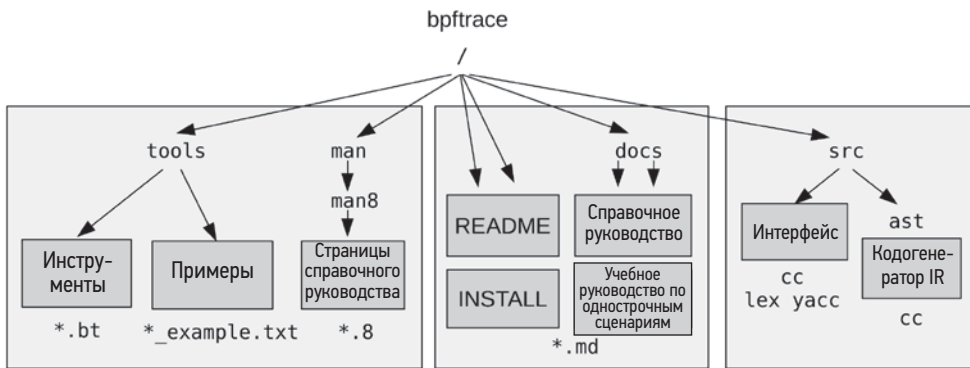


Рис. 5.1. Общая структура каталогов проекта bpftrace

В репозитории bpftrace есть документация с описанием инструментов, страницы справочного руководства (man), файлы с примерами, руководство по программированию на bpftrace (однострочных сценариев) и справочное руководство по языку программирования. Файлы с кодом инструментов на языке bpftrace имеют расширение .bt.

Для парсинга исходного кода на языке bpftrace используются lex и yacc, а для парсинга структур — Clang. Программы на bpftrace сначала компилируются в промежуточное представление с помощью LLVM, а затем в байт-код BPF с применением библиотек LLVM. Подробнее об этом в разделе 5.16.

5.2. ВОЗМОЖНОСТИ BPFTRACE

Фич-листы помогут вам в изучении этой новой технологии. В свое время я создал списки желаемых возможностей bpftrace, чтобы ориентироваться на них при разработке. Теперь они превратились в фич-листы и будут представлены в этом разделе. В главе 4 я разделил возможности ВСС на доступные в пространстве ядра и в пространстве пользователя, поскольку это разные интерфейсы. В bpftrace есть только один интерфейс: интерфейс программирования на bpftrace. Поэтому я сгруппировал возможности bpftrace, разделив их на источники событий, действия и общие возможности.

5.2.1. Источники событий bpftrace

В источниках событий используются технологии из пространства ядра, которые были представлены в главе 2. Интерфейс bpftrace (тип события указан в скобках) поддерживает:

- динамическую инструментацию в пространстве ядра (`kprobe`);
- динамическую инструментацию в пространстве пользователя (`uprobe`);
- статическую трассировку в пространстве ядра (`tracepoint, software`);
- статическую трассировку в пространстве пользователя (`usdt, через libbcc`);
- выборку событий по времени (`profile`);
- интервальные события (`interval`);
- события РМС (`hardware`);
- искусственные события (`BEGIN, END`).

Эти типы событий более подробно описываются в разделе 5.9. В будущем планируется расширить круг источников событий. Возможно, когда вы будете читать эти строки, уже появятся новые источники, в том числе события сокетов и `skb`, неструктурированные точки трассировки, точки останова в памяти и пользовательские счетчики производительности.

5.2.2. Действия bpftrace

В ответ на события можно выполнять определенные действия. Ниже приводится список основных действий. Полный список ищите в справочном руководстве к bpftrace:

- фильтрация (предикаты);
- вывод событий (`printf()`);
- использование основных переменных (`global`, `$local` и `per[tid]`);
- использование встроенных переменных (`pid`, `tid`, `comm`, `nsecs`, ...);
- использование ассоциативных массивов (`key[value]`);
- подсчет частоты (`count()` или `++`);
- вычисление статистик (`min()`, `max()`, `sum()`, `avg()`, `stats()`);
- построение гистограмм (`hist()`, `lhist()`);
- определение времени и разности между двумя отметками времени (`nsecs` и хранилище хешей);
- получение трассировок стека в пространстве ядра (`kstack`);
- получение трассировок стека в пространстве пользователя (`ustack`);
- разрешение символов в пространстве ядра (`ksym()`, `kaddr()`);
- разрешение символов в пространстве пользователя (`usym()`, `uaddr()`);
- навигация по структурам C (`->`);
- доступ к массивам (`[]`);
- выполнение команд оболочки (`system()`);
- вывод файлов (`cat()`);
- использование позиционных параметров (`$1`, `$2`, ...).

Более подробно действия описаны в разделе 5.7. При острой необходимости можно добавлять дополнительные действия, но желательно, чтобы язык оставался как можно более компактным и простым для изучения.

5.2.3. Общие возможности bpftrace

Ниже приводится список общих возможностей bpftrace и компонентов в репозитории:

- средства инструментации с низким оверхедом (BPF JIT и карты);
- безопасность использования в промышленной среде (верификатор BPF);
- большое количество инструментов (в каталоге `/tools`);
- учебное руководство (`/docs/tutorial_one_liners.md`);
- справочное руководство (`/docs/reference_guide.md`).

5.2.4. Сравнение bpftrace с другими инструментами мониторинга

Сравним bpftrace с другими трассировщиками, которые умеют инструментировать все типы событий:

- **perf(1):** в сравнении с компактным и высокоуровневым языком bpftrace язык сценариев perf(1) выглядит слишком многословным. perf(1) поддерживает эффективный способ передачи событий в двоичном формате через perf record и режимы формирования сводной информации в памяти — perf top. bpftrace реализует эффективные методы обобщения информации в ядре, например построение гистограмм, тогда как встроенные в ядро сводки perf(1) ограничены счетчиками (perf stat). Возможности perf(1) расширяются с помощью программ BPF, но пишутся на другом языке, не таком высокоуровневой, как bpftrace. Пример программы BPF для perf(1) ищите в приложении D.
- **Ftrace:** bpftrace предлагает высокоуровневый язык программирования, похожий на C и awk, тогда как средства инструментации в Ftrace, включая триггеры hist, имеют свой, ни на что не похожий синтаксис. Ftrace имеет меньше зависимостей, поэтому он лучше подходит для использования в небольших встраиваемых системах Linux. Также Ftrace поддерживает такие режимы инструментации, как подсчет вызовов функций, которые действуют более оптимально, чем источники событий в bpftrace. (Моя версия funcckcount(8) на основе Ftrace запускается и останавливается быстрее и имеет меньший оверхед, чем эквивалентная версия на основе bpftrace.)
- **SystemTap:** оба трассировщика, bpftrace и SystemTap, предлагают высокоуровневые языки программирования. Но bpftrace основан на технологиях, встроенных в ядро Linux, а SystemTap добавляет свои модули ядра, которые оказались ненадежными в системах, отличных от RHEL. Но в SystemTap уже идут работы по реализации поддержки BPF по аналогии с bpftrace, что должно повысить надежность трассировщика в этих других системах. Сейчас у SystemTap больше вспомогательных функций в своих библиотеках для инструментации различных событий.
- **LTtng:** LTtng оптимизирует вывод дампов событий и предоставляет инструменты для их анализа. Он использует иной подход к анализу производительности, чем bpftrace, ориентированный на выполнение анализа в масштабе реального времени.
- **Прикладные инструменты:** область применения прикладных инструментов и инструментов среды выполнения ограничены пространством пользователя. bpftrace, напротив, способен также инструментировать и обрабатывать события в пространстве ядра, позволяя идентифицировать источники проблем, недоступные прикладным инструментам. Однако прикладные инструменты имеют свои преимущества — обычно они адаптированы для использования с конкретным приложением или средой выполнения. Профилировщик базы данных MySQL уже знает, как инструментировать запросы, а профилировщик JVM — сборку мусора. В bpftrace все это приходится реализовывать вручную.

Вы не обязаны использовать только bpftrace. Ваша цель — решать проблемы, и иногда решение удастся найти намного быстрее, если использовать комбинацию разных инструментов.

5.3. УСТАНОВКА BPFTRACE

Устанавливать bpftrace лучше с помощью диспетчера пакетов в вашем дистрибутиве Linux, но на момент написания этой книги такие пакеты только начали появляться. Первыми появились пакеты от Canonical¹ и для дистрибутива Debian², которые также будут доступны для Ubuntu 19.04. Если пакета для вашего дистрибутива нет, вы сможете собрать bpftrace из исходного кода. Загляните в файл `INSTALL.md` в репозитории bpftrace, где приводятся сведения о последней версии пакета и инструкции по сборке [63].

5.3.1. Требования к конфигурации ядра

Рекомендуется использовать ядро Linux 4.9 (релиз в декабре 2016 года) или более новое. Основные компоненты BPF, которые используются в bpftrace, были добавлены между релизами 4.1 и 4.9. В последующих версиях продолжали добавляться улучшения, поэтому чем новее ядро, тем лучше. Документация BCC включает список возможностей BPF в разных версиях ядра Linux, который поможет понять, почему более поздние версии ядра лучше (см. [64]).

Некоторые параметры конфигурации ядра должны быть включены. Они включены по умолчанию во многих дистрибутивах, поэтому, как правило, ничего менять не надо. Вот эти параметры: `CONFIG_BPF=y`, `CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`, `CONFIG_HAVE_EBPF_JIT=y`, `CONFIG_BPF_EVENTS=y`.

5.3.2. Ubuntu

Как только пакет bpftrace станет доступен в дистрибутиве Ubuntu, его можно будет установить парой команд:

```
sudo apt-get update
sudo apt-get install bpftrace
```

Также bpftrace можно собрать и установить из исходных кодов:

```
sudo apt-get update
sudo apt-get install bison cmake flex g++ git libelf-dev zlib1g-dev libfl-dev \
    systemtap-sdt-dev llvm-7-dev llvm-7-runtime libclang-7-dev clang-7
git clone https://github.com/iovisor/bpftrace
mkdir bpftrace/build; cd bpftrace/build
cmake -DCMAKE_BUILD_TYPE=Release ..
make
make install
```

¹ Спасибо Колину Яну Кингу (Colin Ian King) [61].

² Спасибо Винсенту Бернату (Vincent Bernat) [62].

5.3.3. Fedora

Как только пакет `bpftrace` станет доступен в Fedora, его можно будет установить командой:

```
sudo dnf install -y bpftrace
```

Также `bpftrace` можно собрать и установить из исходных кодов:

```
sudo dnf install -y bison flex cmake make git gcc-c++ elfutils-libelf-devel \
    zlib-devel llvm-devel clang-devel bcc-devel
git clone https://github.com/iovisor/bpftrace
cd bpftrace
mkdir build; cd build; cmake -DCMAKE_BUILD_TYPE=DEBUG ..
make
```

5.3.4. Действия после сборки

Чтобы убедиться, что сборка прошла успешно, запустите набор тестов и какой-нибудь однострочник:

```
sudo ./tests/bpftrace_test
sudo ./src/bpftrace -e 'kprobe:do_nanosleep { printf("sleep by %s\n", comm); }'
```

Запустите `sudo make install`, чтобы установить двоичный файл `bpftrace` в `/usr/local/bin/bpftrace` и инструменты в `/usr/local/share/bpftrace/tools`. Вы можете изменить каталог установки с помощью параметра команды `cmake(1)`, например `-DCMAKE_INSTALL_PREFIX=/usr/local`, где `/usr/local` — это значение по умолчанию.

5.3.5. Другие дистрибутивы

Сначала проверьте доступность пакета `bpftrace`, и если он отсутствует, прочитайте инструкции по установке `bpftrace` в файле `INSTALL.md`.

5.4. ИНСТРУМЕНТЫ BPFTRACE

На рис. 5.2 показаны основные компоненты системы, а также инструменты из репозитория `bpftrace`, которые описываются в этой книге.

Названия уже существовавших ранее инструментов показаны черным, а новых, созданных для этой книги, — серым шрифтом. Некоторые варианты инструментов здесь не показаны (например, варианты `qdisc`, которые обсуждаются в главе 10).

5.4.1. Инструменты, рассматриваемые в книге

В табл. 5.1 перечислены инструменты, сгруппированные по области применения. Они подробно описаны в следующих главах.

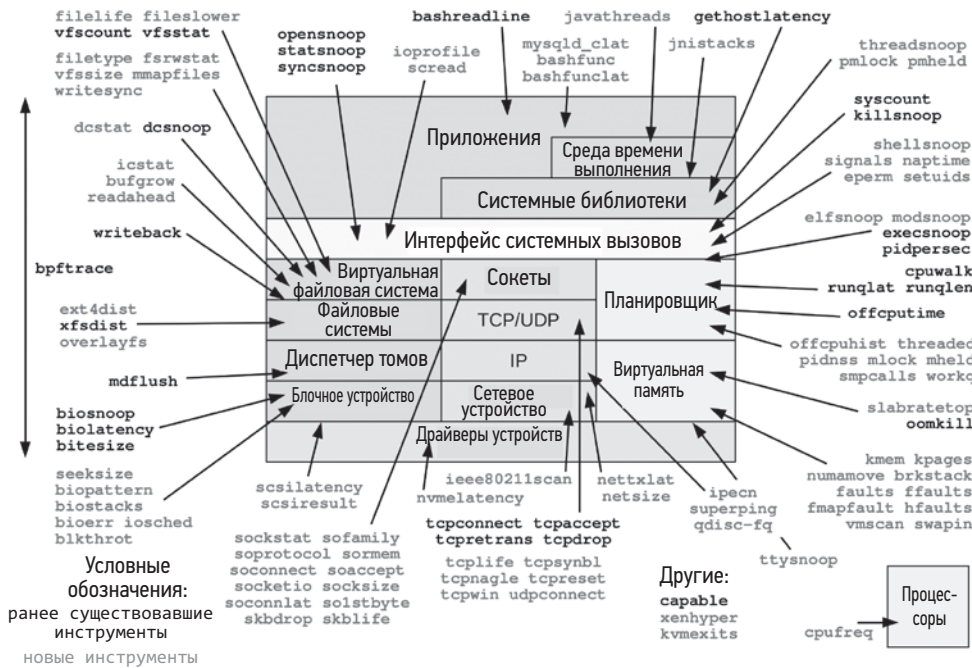


Рис. 5.2. Инструменты bpftrace для оценки производительности

Таблица 5.1. Отдельные инструменты bpftrace, сгруппированные по области применения и главам

Область применения	Инструменты	Глава
Процессоры	execsnoop.bt, runqlat.bt, runqlen.bt, cpuwalk.bt, offcputime.bt	6
Память	oomkill.bt, failts.bt, vmscan.bt, swapin.bt	7
Файловые системы	vfsstat.bt, filelife.bt, xfsdist.bt	8
Дисковый ввод/вывод	biosnoop.bt, biolateness.bt, bitesize.bt, biostacks.bt, scsilateness.bt, nvmlateness.bt	9
Сети	tcpaccept.bt, tcpconnect.bt, tcpdrop.bt, tcptrans.bt, gethostlatency.bt	10
Безопасность	ttysnoop.bt, elfsnoop.bt, setuids.bt	11
Языки	jnistacks.bt, javacalls.bt	12
Приложения	threadsnoop.bt, pmheld.bt, napttime.bt, mysqlq_d_qlower.bt	13
Ядро	mlock.bt, mhheld.bt, kmem.bt, kpages.bt, workq.bt	14
Контейнеры	pidnss.bt, blkthrot.bt	15

Таблица 5.1 (окончание)

Область применения	Инструменты	Глава
Гипервизоры	xenhyper.bt, cpustolen.bt, kvmexits.bt	16
Отладка/многоцелевые	execsnoop.bt, threadsnoop.bt, opensnoop.bt, killsnoop.bt, signals.bt	6, 8, 13

Отмечу, что в этих главах рассматриваются также инструменты ВСС, не перечисленные в табл. 5.1.

Прочитав эту главу, вы можете свободно переходить к любой интересующей вас главе и использовать эту книгу как справочное руководство.

5.4.2. Характеристики инструментов

Инструменты bpftrace имеют следующие характеристики:

- решают реальные проблемы наблюдаемости;
- предназначены для выполнения в промышленных средах с привилегиями пользователя root;
- для каждого инструмента есть страница в справочном руководстве (в каталоге man/man8);
- у каждого инструмента есть файл с примерами вывода с пояснениями (в каталоге tools/*_examples.txt);
- исходный код каждого инструмента начинается с вводного блочного комментария;
- инструменты создавались максимально простыми и короткими (более сложные инструменты оставлены для реализации на основе ВСС).

5.4.3. Использование инструментов

Инструменты, собранные в виде выполняемых файлов, можно запускать из командной строки с привилегиями пользователя root:

```
bpftrace/tools$ ls -lh opensnoop.bt
-rwxr-xr-x 1 bgregg bgregg 1.1K Nov 13 10:56 opensnoop.bt*

bpftrace/tools$ ./opensnoop.bt
ERROR: bpftrace currently only supports running as the root user.

bpftrace/tools$ sudo ./opensnoop.bt
Attaching 5 probes...
Tracing open syscalls... Hit Ctrl-C to end.
PID   COMM          FD ERR PATH
25612  bpftrace       23   0  /dev/null
1458   Xorg           118  0  /proc/18416/cmdline
[...]
```

Эти инструменты можно поместить в каталог с другими инструментами системного администрирования, например в `/usr/local/sbin`.

5.5. ОДНОСТРОЧНЫЕ СЦЕНАРИИ BPFTRACE

В этом разделе приведена подборка однострочных сценариев, которые полезны как сами по себе, так и для показа различных возможностей bpftrace. В следующем разделе описывается язык программирования, а в последующих главах приведены дополнительные однострочные сценарии для конкретных целей. Многие из этих сценариев обобщают данные в памяти (ядра) и ничего не выводят, пока вы не нажмете Ctrl-C.

Показывает, кто и что выполняет:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%s -> %s\n", comm, str(args->filename)); }'
```

Показывает новые процессы с аргументами:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Показывает, какие файлы открывает каждый процесс вызовом `openat()`:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename)); }'
```

Подсчитывает число системных вызовов, выполненных каждой программой:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
```

Подсчитывает число системных вызовов по их именам:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Подсчитывает число системных вызовов, выполненных каждым процессом:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Показывает общее число байтов, прочитанных каждым процессом:

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read /args->ret/ { @[comm] = sum(args->ret); }'
```

Показывает распределение размеров блоков, прочитанных каждым процессом:

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read { @[comm] = hist(args->ret); }'
```

Показывает объемы дискового ввода/вывода для каждого процесса:

```
bpftrace -e 'tracepoint:block:block_rq_issue { printf("%d %s %d\n", pid, comm, args->bytes); }'
```

Подсчитывает число страниц, загруженных каждым процессом:

```
bpftrace -e 'software:major-faults:1 { @[comm] = count(); }'
```

Подсчитывает число отказов страниц для каждого процесса:

```
bpftrace -e 'software:faults:1 { @[comm] = count(); }'
```

Профилирует стек в пространстве пользователя для PID 189 с частотой 49 Гц:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

5.6. ДОКУМЕНТАЦИЯ BPFTRACE

Для каждого инструмента bpftrace есть своя страница справочного руководства (man) и файл с примерами, в точности как в проекте BCC. Формат и назначение этих файлов описывается в главе 4.

Для тех, кто хочет научиться писать свои однострочные сценарии и инструменты, я подготовил tutorial для bpftrace [65] и справочное руководство по bpftrace [66]. Их можно найти в репозитории, в каталоге /docs.

5.7. ПРОГРАММИРОВАНИЕ НА BPFTRACE

Этот раздел содержит краткое руководство по использованию bpftrace и программированию на языке bpftrace. Структура этого раздела повторяет оригинальную статью для awk [Aho 78], где дано описание этого языка на шести страницах. Сам язык bpftrace имеет схожие черты с awk и C, а также с трассировщиками, такими как DTrace и SystemTap.

Ниже приводится пример программы на bpftrace: она измеряет время, затраченное на выполнение функции ядра `vfs_read()`, и выводит результаты в виде гистограммы. В этом коротком разделе описываются основные компоненты этого инструмента.

```
#!/usr/local/bin/bpftrace

// эта программа измеряет продолжительность выполнения vfs_read()

kprobe:vfs_read
{
    @start[tid] = nsecs;
}

kretprobe:vfs_read
/@start[tid]/
{
    $duration_us = (nsecs - @start[tid]) / 1000;
    @us = hist($duration_us);
    delete(@start[tid]);
}
```

В следующих пяти разделах подробно исследуем программирование на bpftrace и познакомимся с событиями, видами проверок, операторами, переменными, функциями и типами карт.

5.7.1. Порядок использования

Команда

```
bpftrace -e program
```

запустит программу `program` и инструментирует все события, которые в ней определены. Программа будет выполняться до нажатия Ctrl-C либо пока явно не вызовет функцию `exit()`. Код программы передается команде bpftrace как аргумент параметра `-e`, и вся команда целиком называется *однострочным сценарием*. Также программу можно сохранить в файл и запустить командой:

```
bpftrace file.bt
```

Расширение `.bt` можно не использовать, но оно помогает определить тип файла. В начало файла можно поместить ссылку на интерпретатор¹:

```
#!/usr/local/bin/bpftrace
```

После этого файл с программой можно сделать выполняемым (`chmod a+x file.bt`) и запускать как любую другую программу:

```
./file.bt
```

Имейте в виду, что утилита bpftrace должна запускаться с привилегиями пользователя `root`². В некоторых системах можно использовать командную оболочку с привилегиями `root` и запускать программы из нее напрямую, в других требуется использовать команду повышения привилегий `sudo(1)`:

```
sudo ./file.bt
```

5.7.2. Структура программы

Программа bpftrace определяет серию зондов и связанных с ними действий:

```
probes { actions }
probes { actions }
...
```

¹ Некоторые предпочитают использовать `#!/usr/bin/env bpftrace`, что позволяет найти bpftrace в любом каталоге, включенном в `$PATH`. Но команда `env(1)` страдает различными проблемами, поэтому не используется в БСС. Сейчас команда `env(1)` используется в проекте bpftrace, но в будущем ситуация может измениться по тем же причинам.

² bpftrace проверяет UID 0; в будущем могут проверяться определенные привилегии.

При срабатывании зонда выполняются соответствующие ему действия. Перед списком действий можно добавить необязательное выражение фильтра:

```
probes /filter/ { actions }
```

В этом случае действия выполнятся, только если фильтр вернет true. В чем-то это напоминает структуру программ для awk(1):

```
/pattern/ { actions }
```

Программирование для awk(1) напоминает программирование для bpftrace: в блоке указываем несколько действий — и они могут выполняться в любом порядке, в зависимости от результата, возвращаемого pattern или probe + filter.

5.7.3. Комментарии

В файлы с программами на bpftrace можно добавлять однострочные комментарии с помощью «//»:

```
// это комментарий
```

Комментарии не выполняются. Поддерживаются и многострочные (блочные) комментарии, которые оформляются так же, как в C:

```
/*  
 * это многострочный  
 * комментарий  
 */
```

Этот же синтаксис можно использовать, чтобы закомментировать часть строки (например, /* комментарий */).

5.7.4. Формат определения зондов

Определение зонда начинается с имени типа зонда, за которым следует иерархия идентификаторов, разделенных двоеточиями:

```
type:identifier1[:identifier2[...]]
```

Иерархия зависит от типа зонда. Например:

```
kprobe:vfs_read  
uprobe:/bin/bash:readline
```

Зонды типа kprobe инструментируют вызовы функций ядра и требуют указать только один идентификатор: имя функции. Зонды типа uprobe инструментируют вызовы функций в пространстве пользователя и требуют указывать путь к двоичному файлу и имя функции.

Для выполнения одних и тех же действий перечислите несколько зондов через запятую. Например:

```
probe1,probe2,... { actions }
```

Есть два специальных типа зондов, для которых не нужно указывать дополнительные идентификаторы BEGIN и END. Они срабатывают в начале и в конце программы bpftrace (в точности как в awk(1)).

Более подробно о типах зондов и особенностях их использования идет речь в разделе 5.9.

5.7.5. Подстановочные символы в определениях зондов

Некоторые типы зондов допускают использование подстановочных символов. Например, определение

```
kprobe:vfs_*
```

инструментирует все зонды kprobes (функции ядра), имена которых начинаются с «vfs_».

Инструментация слишком большого количества зондов может стать причиной огромного оверхеда. Чтобы избежать этого, у bpftrace есть параметр настройки, ограничивающий максимальное количество зондов, которые можно активировать. Этот параметр устанавливается с помощью переменной среды BPFTRACE_MAX_PROBES (сейчас его значение по умолчанию 512¹).

Перед использованием подстановочных символов в программе можно проверить, сколько зондов будет инструментировано. Для этого выполните команду `bpftrace -l`:

```
# bpftrace -l 'kprobe:vfs_*'
kprobe:vfs_fallocate
kprobe:vfs_truncate
kprobe:vfs_open
kprobe:vfs_setpos
kprobe:vfs_llseek
[...]
bpftrace -l 'kprobe:vfs_*' | wc -l
56
```

Здесь групповому имени «kprobe:vfs_» соответствует 56 зондов. Имя зонда здесь указано в кавычках, чтобы избежать нежелательной интерпретации подстановочных символов командной оболочкой.

¹ Использование более 512 зондов сильно замедляет запуск и остановку bpftrace, так как они обрабатываются последовательно. В планах — реализация пакетной инструментации зондов. После этого предел можно будет значительно увеличить или вообще убрать.

5.7.6. Фильтры

Фильтры — это логические выражения, которые определяют необходимость выполнения действий. Фильтр

```
/pid == 123/
```

разрешит выполнить соответствующие действия, только если встроенная переменная `pid` (идентификатор процесса, PID) равна 123.

Если проверка не указана:

```
/pid/
```

фильтр проверит, что переменная `pid` не равна 0 (то есть `/pid/` — это то же самое, что `/pid != 0/`). Фильтры можно объединять с использованием логических операторов, таких как логическое И (`&&`). Например:

```
/pid > 100 && pid < 1000/
```

Этот фильтр вернет `true`, только если оба выражения вернут `true`.

5.7.7. Действия

Действие может быть выражено одной или несколькими инструкциями, разделенными точкой с запятой:

```
{ action one; action two; action three }
```

После последней инструкции тоже можно добавить точку с запятой. Инструкции записываются на языке `bpftrace`, который похож на C, и могут манипулировать переменными или вызывать функции `bpftrace`. Например, это действие:

```
{ $x = 42; printf("$x is %d", $x); }
```

присвоит переменной `$x` число 42, а затем напечатает его, вызвав функцию `printf()`. В разделах 5.7.9 и 5.7.11 перечисляются другие доступные функции.

5.7.8. Hello, World!

Теперь вы без труда поймете эту простую программу: она выводит строку «Hello, World!» в момент запуска `bpftrace`:

```
# bpftrace -e 'BEGIN { printf("Hello, World!\n"); }'  
Attaching 1 probe...  
Hello, World!  
^C
```


Программу можно сохранить в файле:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Hello, World!\n");
}
```

Оформлять отступы в блоках кода не требуется, но они улучшают читаемость.

5.7.9. Функции

Кроме `printf()` — функции форматированного вывода — есть еще встроенные функции, в том числе:

- **`exit()`**: производит выход из программы `bpftrace`;
- **`str(char *)`**: возвращает строку по указателю;
- **`system(format[, arguments ...])`**: выполняет команду в командной оболочке.

Следующее действие:

```
printf("got: %llx %s\n", $x, str($x)); exit();
```

выведет значение переменной `$x` в шестнадцатеричном формате, а затем попытается интерпретировать его как указатель на массив символов, завершающийся пустым символом `NULL (char *)`, выведет его как строку и завершит выполнение программы.

5.7.10. Переменные

Поддерживается три типа переменных: встроенные переменные, временные и карты.

Встроенные переменные (built-in variables) предопределены и предоставляются `bpftrace`. Обычно они доступны только для чтения. К ним относятся `pid` (идентификатор процесса), `comm` (имя процесса), `nsecs` (отметка времени в наносекундах) и `curtask` (адрес `task_struct` текущего потока).

Временные переменные (scratch variables) можно использовать для временного хранения результатов вычислений. Их имена начинаются с префикса «\$». Сама переменная и ее тип определяются первой операцией присваивания. Эти инструкции:

```
$x = 1;
$y = "hello";
$z = (struct task_struct *)curtask;
```

объявляют целочисленную переменную `$x`, строковую переменную `$y` и переменную `$z` как указатель на структуру `task_struct`. Их можно использовать только в блоке, в котором определено действие. При попытке сослаться на переменную

до присваивания ей значения `bpftrace` сообщит об ошибке (это поможет найти опечатки).

Переменные-карты (`map variables`) хранятся в хранилище карт BPF, и их имена должны начинаться с префикса «@». В этих переменных можно сохранять данные для их передачи между действиями. Программа

```
probe1 { @a = 1; }
probe2 { $x = @a; }
```

присвоит число 1 переменной `@a`, когда возникнет событие `probe1`, а затем, когда возникнет событие `probe2`, присвоит значение `@a` переменной `$x`. Если сначала возникнет событие `probe1`, а затем `probe2`, переменная `$x` получит значение 1, в противном случае — значение 0 (значение по умолчанию для неинициализированной переменной).

После имени переменной-карты можно указать ключ и использовать такие переменные на манер хеш-таблицы (ассоциативного массива). Инструкция

```
@start[tid] = nsecs;
```

часто используется в практике: в данном случае она сохранит значение встроенной переменной `nsecs` в карте с именем `@start` и с ключом `tid` (идентификатором текущего потока). Такой прием позволяет потокам хранить отметки времени, которые не будут затерты другими потоками.

```
@path[pid, $fd] = str(arg0);
```

Это пример карты с множественными ключами, здесь роль ключей играют значение встроенной переменной `pid` и значение временной переменной `$fd`.

5.7.11. Функции карт

Картам можно присваивать специальные функции. Эти функции хранят и выводят данные. Например, инструкция

```
@x = count();
```

подсчитывает события и при выводе вернет значение счетчика. Она использует карту, хранящую данные для каждого процессора отдельно, соответственно, `@x` превращается в специальный объект с типом счетчика. Эта инструкция тоже подсчитывает события:

```
@x++;
```

Но здесь используется карта с одним общим целочисленным счетчиком, а не со счетчиками для каждого процессора в отдельности. Такие глобальные целочисленные переменные пригодятся в программах, где нужно целое число, а не счетчик. Но имейте в виду, что из-за одновременных изменений такой переменной

в нескольких потоках выполнения может возникнуть погрешность подсчета (см. раздел 2.3.7 в главе 2).

Инструкция

```
@y = sum($x);
```

суммирует значения переменной `$x` и при выводе вернет общий результат. Инструкция

```
@z = hist($x);
```

сохранит значение `$x` в гистограмму и при выводе вернет счетчики корзин и гистограмму в формате ASCII.

Некоторые функции карт манипулируют картами напрямую. Например,

```
print(@x);
```

выведет карту `@x`. Но она используется не часто, потому что после завершения bpftrace все карты выводятся автоматически.

Некоторые функции карт оперируют ключами в картах. Например, инструкция

```
delete(@start[tid]);
```

удалит из карты `@start` пару ключ — значение с ключом `tid`.

5.7.12. Определение продолжительности выполнения `vfs_read()`

Познакомившись с новыми синтаксическими конструкциями, вы сможете понять более сложный пример. Вот программа `vfsread.bt`, которая измеряет продолжительность выполнения функции ядра `vfs_read` и выводит гистограмму распределения продолжительностей в микросекундах:

```
#!/usr/local/bin/bpftrace

// эта программа измеряет продолжительность выполнения vfs_read()

kprobe:vfs_read
{
    @start[tid] = nsecs;
}

kretprobe:vfs_read
/@start[tid]/
{
    $duration_us = (nsecs - @start[tid]) / 1000;
    @us = hist($duration_us);
    delete(@start[tid]);
}
```

Код определяет время выполнения функции ядра `vfs_read()`, проверяя ее начало с помощью `kprobe` и сохраняя временную метку в хеше `@start` с ключом ID потока, а затем проверяя конец с помощью `kretprobe` и вычисляя разницу в этот момент между текущим временем и временем запуска. Фильтр проверяет наличие отметки времени, когда функция была вызвана. В противном случае вычисление разности возвращало бы ошибочный результат.

Пример вывода:

```
# bpftrace vfsread.bt
```

Attaching 2 probes...

 $\wedge C$

@us:

[0]	23	@
[1]	138	@@@@@@@@
[2, 4)	538	@@
[4, 8)	744	@@
[8, 16)	641	@@
[16, 32)	122	@@@@@@@@
[32, 64)	13	
[64, 128)	17	@
[128, 256)	2	
[256, 512)	0	
[512, 1K)	1	

Программа продолжает выполняться до нажатия Ctrl-C, затем выводит эти строки и завершается. Этой карте гистограммы было дано имя «us», чтобы включить в вывод единицы измерения, потому что имя карты выводится вместе с данными. Давая картам осмысленные имена, такие как «bytes» и «latency_ns», можно аннотировать вывод и сделать его более понятным.

Программу можно немного модифицировать. Взгляните на изменения в строке, где присваивается функция `hist()`:

```
@us[pid, comm] = hist($duration_us);
```

Теперь для каждой пары имя/идентификатор процесса будет создана своя гистограмма. В результате вывод изменится так:

```
# bpftrace vfsread.bt
```

Attaching 2 probes...

 $\wedge C$

```
@us[1847, gdbus]:
```

[illegible]

```
@us[1630, ibus-daemon]:
```

[2, 4) 9 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

```
[4, 8)          17 |aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa|
@us[29588, device poll]:
[1]            13 |aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa|
[2, 4)         15 |aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa|
[4, 8)          4 |aaaaaaaaaaaaaaaa|
[8, 16)         4 |aaaaaaaaaaaaaaaa|
[...]
```

Это пример одной из самых полезных возможностей bpftrace. Традиционные системные инструменты — iostat(1) и vmstat(1) — имеют фиксированный вывод, который очень непросто настроить. Но с помощью bpftrace наблюдаемые метрики можно продолжать разбивать на части и дополнять метриками из других событий, пока не будут найдены нужные ответы.

5.8. ПОРЯДОК ИСПОЛЬЗОВАНИЯ BPFTRACE

Без аргументов (или с параметром -h) команда bpftrace выводит справочное сообщение с основными параметрами и переменными среды, а также примеры однострочных сценариев:

bpftrace

ПОРЯДОК ИСПОЛЬЗОВАНИЯ:

```
bpftrace [options] filename
bpftrace [options] -e 'program'
```

ПАРАМЕТРЫ:

-B MODE	режим буферизации вывода ('line', 'full' или 'none')
-d	выполнить пробный прогон и вывести отладочную информацию
-o file	перенаправить вывод программы в файл
-dd	выполнить пробный прогон и вывести подробную отладочную информацию
-e 'program'	выполнить эту программу
-h, --help	вывести эту справку
-I DIR	добавить каталог DIR в путь поиска подключаемых заголовков
--include FILE	добавить #include file перед передачей препроцессору
-l [search]	вывести список событий
-p PID	включить события USDT для PID
-c 'CMD'	выполнить CMD и включить события USDT для этого процесса
--unsafe	разрешить небезопасные встроенные функции
-v	выводить подробные сообщения
-V, --version	вывести версию bpftrace

СРЕДА:

BPFTRACE_STRLLEN	[по умолчанию: 64] байт в стеке BPF на str()
BPFTRACE_NO_CPP_DEMANGLE	[по умолчанию: 0] отключить интерпретацию символов C++
BPFTRACE_MAP_KEYS_MAX	[по умолчанию: 4096] максимальное число ключей в карте
BPFTRACE_CAT_BYTES_MAX	[по умолчанию: 10k] максимальное число байтов для чтения встроенной функцией cat
BPFTRACE_MAX_PROBES	[по умолчанию: 512] максимальное число инструментируемых событий

ПРИМЕРЫ:

```
bpftrace -l '*sleep*'
    вывести все события, содержащие слово "sleep"
bpftrace -e 'kprobe:do_nanosleep { printf("PID %d sleeping...\n", pid); }'
    трассировать процессы, вызывающие sleep
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
    подсчитать число системных вызовов для каждого имени процесса
```

Это сообщение выводит bpftrace версии v0.9-232-g60e6, выпущенной 15 июня 2019 года. По мере добавления новых возможностей сообщение может становиться громоздким, поэтому в будущем появятся его короткая и длинная версии. Проверьте вывод своей текущей версии, чтобы убедиться, что это произошло.

5.9. ТИПЫ ЗОНДОВ В BPFTRACE

В табл. 5.2 перечислены поддерживаемые типы зондов. Для многих из них есть псевдонимы — сокращенные команды, помогающие писать однострочники.

Таблица 5.2. Типы зондов в bpftrace

Тип	Псевдоним	Описание
tracepoint	t	Инструментируют статические точки трассировки в ядре
usdt	U	Инструментируют статические точки трассировки в пространстве пользователя
kprobe	k	Инструментируют динамические точки вызовов функций ядра
kretprobe	kr	Инструментируют динамические точки возврата из функций ядра
uprobe	u	Инструментируют динамические точки вызовов функций в пространстве пользователя
uretprobe	ur	Инструментируют динамические точки возврата из функций в пространстве пользователя
software	s	Программные события в пространстве ядра
hardware	h	Инструментируют аппаратные счетчики
profile	p	Производят выборку по времени для всех процессоров
interval	i	Производят выборку в течение интервала (для одного процессора)
BEGIN		Запуск bpftrace
END		Завершение bpftrace

Эти типы зондов — интерфейсы к существующим технологиям в ядре. Описание технологий kprobe, uprobe, tracepoint, USDT и PMC (используемые аппаратными событиями) ищите в главе 2.

Некоторые события, например события планировщика, события выделения памяти или события получения сетевых пакетов, могут возникать очень часто. Чтобы уменьшить оверхед на их трассировку, попробуйте использовать как можно более редкие события. Подробнее о приемах минимизации оверхеда, применимых как в BCC, так и в bpftrace, рассказывается в главе 18.

В следующих разделах я расскажу об особенностях использования типов зондов в bpftrace.

5.9.1. tracepoint

Зонды типа tracepoint инструментируют статические точки трассировки в ядре. Формат определения:

```
tracepoint:tracepoint_name
```

tracepoint_name — это полное имя точки трассировки, включая двоеточие, разделяющее имена классов в иерархии и имя события. Например, точку трассировки net:netif_rx можно инструментировать в bpftrace с помощью зонда tracepoint:net:netif_rx.

Обычно точки трассировки поддерживают аргументы — поля, доступ к которым из bpftrace можно получить через встроенные аргументы. Например, net:netif_rx имеет поле len, содержащее длину пакета, доступ к которому можно получить с помощью args->len.

Для тех, кто только начинает осваивать bpftrace и трассировку в целом, точки трассировки системных вызовов — это отличные цели для инструментации. Они позволяют получить представление об использовании ресурсов ядра и имеют хорошо документированный API: страницы в справочном руководстве по системным вызовам. Например, точки трассировки

```
syscalls:sys_enter_read
syscalls:sys_exit_read
```

инструментируют начало и конец системного вызова read(2). Страница справочного руководства так определяет его сигнатуру:

```
ssize_t read(int fd, void *buf, size_t count);
```

В точке трассировки sys_enter_read эти аргументы должны быть доступны как args->fd, args->buf и args->count. Убедиться в этом можно, выполнив команду bpftrace с параметрами -l (list — список) и -v (verbose — подробно):

```
# bpftrace -lv tracepoint:syscalls:sys_enter_read
tracepoint:syscalls:sys_enter_read
    int __syscall_nr;
    unsigned int fd;
    char * buf;
    size_t count;
```

Страница справочного руководства описывает эти аргументы и возвращаемое значение системного вызова `read(2)`, которое можно инструментировать с помощью точки трассировки `sys_exit_read`. Эта точка трассировки имеет дополнительный аргумент, не описываемый там: `__syscall_nr` — номер системного вызова.

Интересный пример — трассировка точек входа и выхода из системного вызова `clone(2)`, который создает новые процессы (подобно `fork(2)`). При появлении каждого события я выведу текущее имя процесса и PID, используя встроенные переменные `bpftrace`. Для события выхода я также выведу возвращаемое значение, используя аргумент точки трассировки:

```
# bpftrace -e 'tracepoint:syscalls:sys_enter_clone {  
    printf("-> clone() by %s PID %d\n", comm, pid); }  
    tracepoint:syscalls:sys_exit_clone {  
        printf("<- clone() return %d, %s PID %d\n", args->ret, comm, pid); }'  
Attaching 2 probes...  
-> clone() by bash PID 2582  
<- clone() return 27804, bash PID 2582  
<- clone() return 0, bash PID 27804
```

Этот системный вызов необычен тем, что имеет один вход и два выхода! Запустив трассировку, я выполнил команду `ls(1)` в терминале с командной оболочкой `bash(1)`. В выводе выше видно, что родительский процесс (PID 2582) входит в `clone(2)`, а затем из этого системного вызова выполняется два возврата: первый возврат делается в родительском процессе, которому возвращается PID (27804) дочернего процесса, и второй — в дочернем процессе, которому возвращает ноль (признак успеха). Когда происходит запуск дочернего процесса, он все еще имеет имя «`bash`», потому что к этому моменту еще не был выполнен системный вызов `exec(2)`, запускающий команду «`ls`». Этот вызов тоже можно отследить:

```
# bpftrace -e 't:syscalls:sys_*_execve { printf("%s %s PID %d\n", probe, comm,  
    pid); }'  
Attaching 2 probes...  
tracepoint:syscalls:sys_enter_execve bash PID 28181  
tracepoint:syscalls:sys_exit_execve ls PID 28181
```

Мы видим, что процесс PID 28181 входит в системный вызов `execve(2)` как «`bash`» и выходит как «`ls`».

5.9.2. usdt

Зонды этого типа инструментуют статические точки трассировки в пространстве пользователя. Формат определения:

```
usdt:binary_path:probe_name  
usdt:library_path:probe_name  
usdt:binary_path:probe_namespace:probe_name  
usdt:library_path:probe_namespace:probe_name
```

Зонды типа `usdt` могут инструментировать двоичный код выполняемых файлов или динамических библиотек, для которых должен указываться полный путь.

`probe_name` — это имя зонда USDT в двоичном файле. Например, зонд `query__start` в сервере MySQL доступен (в зависимости от каталога установки) как `usdt:/usr/local/sbin/mysqld:query__start`.

Если пространство имен зонда не указано, по умолчанию используется имя выполняемого файла или библиотеки. Для многих зондов пространство имен и имя файла отличаются, поэтому для них обязательно нужно указывать пространство имен. Пример этого — зонды из пространства имен «hotspot» в `libjvm` (библиотека JVM). Например (полный путь к библиотеке урезан):

```
usdt:/.../libjvm.so:hotspot:method__entry
```

Все аргументы зондов USDT доступны в виде атрибутов встроенной переменной `args`.

Получить список зондов, доступных в файле, можно с помощью параметра `-l`, например:

```
# bpftrace -l 'usdt:/usr/local/cpython/python'
usdt:/usr/local/cpython/python:line
usdt:/usr/local/cpython/python:function__entry
usdt:/usr/local/cpython/python:function__return
usdt:/usr/local/cpython/python:import__find__load__start
usdt:/usr/local/cpython/python:import__find__load__done
usdt:/usr/local/cpython/python:gc__start
usdt:/usr/local/cpython/python:gc__done
```

Можно получить и список зондов USDT в выполняющемся процессе, в этом случае вместо имени файла следует использовать параметр `-p PID`.

5.9.3. `kprobe` и `kretprobe`

Зонды этого типа используются для динамической инструментации ядра. Формат определения:

```
kprobe: function_name
kretprobe: function_name
```

Зонды `kprobe` инструментируют точку входа в функцию, а `kretprobe` — точку возврата из функции. `function_name` — это имя функции в ядре. Например, функцию `vfs_read()` в ядре можно инструментировать как `kprobe:vfs_read` и `kretprobe:vfs_read`.

Зонды `kprobe` имеют аргументы `arg0`, `arg1`, ..., `argN` — входные аргументы функции, как целые 64-битные целые без знака. Если какой-то из них является указателем на структуру языка C, его можно привести к типу этой структуры¹. Будущая

¹ Под «приведением типа» в языке C понимается изменение типа объекта в программе. Пример этого можно найти в исходном коде инструмента `gunklen(8)` для `bpftrace`, который приводится в главе 6.

технология BPF Type Format (BTF), возможно, станет делать такое приведение автоматически (см. главу 2).

Единственный аргумент `kretprobe` — встроенная переменная `retval` — содержит возвращаемое значение функции. `retval` всегда имеет тип `uint64`. Если фактическое возвращаемое значение не соответствует этому типу, его необходимо привести к нему.

5.9.4. `uprobe` и `uretprobe`

Зонды этого типа используются для динамической инструментации кода в пространстве пользователя. Формат определения:

```
uprobe:binary_path:function_name
uprobe:library_path:function_name
uretprobe:binary_path:function_name
uretprobe:library_path:function_name
```

Зонды `uprobe` инструментуют точку входа в функцию, а `uretprobe` — точку возврата из функции. `function_name` — это имя функции. Например, функцию `readline()` в `/bin/bash` можно инструментировать как `uprobe:/bin/bash:readline` и `uretprobe:/bin/bash:readline`.

Зонды `uprobe` имеют аргументы `arg0`, `arg1`, ..., `argN` — входные аргументы функции, как целые 64-битные целые без знака. Их можно приводить к типам структур на языке C¹. Единственный аргумент `uretprobe` — встроенная переменная `retval` — содержит возвращаемое значение функции. `retval` всегда имеет тип `uint64`, и при необходимости его нужно привести к фактическому типу возвращаемого значения.

5.9.5. `software` и `hardware`

Зонды этого типа инструментуют предопределенные программные (`software`) и аппаратные (`hardware`) события. Формат определения:

```
software:event_name:count
software:event_name:
hardware:event_name:count
hardware:event_name:
```

Программные события похожи на точки трассировки `tracepoint`, но определены для метрик, основанных на подсчете и выборках. Аппаратные события — это счетчики РМС для анализа на уровне процессора.

События этих двух типов могут возникать настолько часто, что инструментация любого из них повлечет значительные издержки, снижающие производительность системы. Этого можно избежать, используя выборку и поле `count`. Если поле `count` определено, зонд будет срабатывать один раз на каждые `[count]` событий. Если поля

¹ Возможно, что в будущем технология BTF станет доступна в пространстве пользователя и двоичные файлы смогут включать описания типов своих структур, как и в ядре.

count нет, используется значение по умолчанию. Например, зонд software:page-faults:100 будет срабатывать один раз на каждые 100 отказов страниц.

Доступные программные (software) события перечислены в табл. 5.3 (имейте в виду: этот список зависит от версии ядра).

Таблица 5.3. Программные события

Имя события	Псевдоним	Счетчик выборки по умолчанию	Описание
cpu-clock	cpu	1000000	Фактическое время процессора
task-clock		1000000	Время использования процессора задачами (увеличивается, только когда задача выполняется на процессоре)
page-faults	faults	100	Отказы страниц
context-switches	cs	1000	Переключение контекста
cpu-migrations		1	Миграция потоков процессора
minor-faults		100	Незначительные отказы страниц: устранены за счет памяти
major-faults		1	Значительные отказы страниц: устранены за счет ввода/вывода из хранилища
alignment-faults		1	Ошибки выравнивания
emulation-faults		1	Ошибки эмуляции
dummy		1	Искусственное событие для тестирования
bpf-output		1	Канал вывода BPF

Аппаратные события, которые зависят от версии ядра и типа процессора, перечислены в табл. 5.4.

Таблица 5.4. Аппаратные события

Имя события	Псевдоним	Счетчик выборки по умолчанию	Описание
cpu-cycles	cycles	1000000	Такты процессора
instructions		1000000	Инструкции процессора
cache-references		1000000	Обращения к кэшу процессора последнего уровня
cache-misses		1000000	Промахи кэша процессора последнего уровня
branch-instructions	branches	100000	Инструкции ветвления

Таблица 5.4 (окончание)

Имя события	Псевдоним	Счетчик выборки по умолчанию	Описание
bus-cycles		100000	Такты шины
frontend-stalls		1000000	Пропуски циклов внешнего интерфейса процессора (например, на время выборки инструкций)
backend-stalls		1000000	Пропуски внутренних циклов процессора (например, на время загрузки/сохранения данных)
ref-cycles		1000000	Циклы обращения к процессору (не масштабируется в турборежиме)

Аппаратные события возникают гораздо чаще, поэтому для них используются большие значения по умолчанию для счетчиков выборки.

5.9.6. profile и interval

Зонды этого типа инструментируют события, имеющие отношение к времени. Формат определения:

```
profile:hz:rate
profile:s:rate
profile:ms:rate
profile:us:rate
interval:s:rate
interval:ms:rate
```

Зонды `profile` срабатывают для всех процессоров и используются для определения параметров загрузки процессора. Зонды `interval` срабатывают только для одного процессора и используются для вывода интервальных метрик.

Второе поле в определении зонда определяет единицы измерения, а последнее — частоту. Во втором поле можно указать:

- **hz**: герцы (событий в секунду);
- **s**: секунды;
- **ms**: миллисекунды;
- **us**: микросекунды.

Например, зонд `profile:hz:99` срабатывает 99 раз в секунду для всех процессоров. Частота 99 используется чаще, чем 100, чтобы избежать проблем, связанных с попаданием в одну и ту же точку. Зонд `interval:s:1` будет срабатывать один раз в секунду и может использоваться для вывода метрик раз в секунду.

5.10. УПРАВЛЕНИЕ ПОТОКОМ ВЫПОЛНЕНИЯ В BPFTRACE

bpftrace поддерживает три типа проверок: фильтры, тернарные (трехместные) операторы и инструкции `if`. Эти проверки изменяют поток выполнения программы, опираясь на логические выражения, которые поддерживают операторы:

- `==`: равно;
- `!=`: не равно;
- `>`: больше, чем;
- `<`: меньше, чем;
- `>=`: больше или равно;
- `<=`: меньше или равно;
- `&&`: И;
- `||`: ИЛИ.

Выражения можно группировать с помощью круглых скобок.

Поддержка циклов ограничена, потому что для большей безопасности верификатор BPF отклоняет любой код, который может вызвать бесконечный цикл. bpftrace поддерживает развернутые циклы, а в будущих версиях должна появиться поддержка ограниченных циклов.

5.10.1. Фильтры

О фильтрах шла речь выше, они определяют, будет ли выполнено действие. Формат определения:

```
probe /filter/ { action }
```

В фильтрах можно использовать логические операторы. Если задан фильтр `/pid == 123/`, действие будет выполнено, только если встроенная переменная `pid` содержит значение 123.

5.10.2. Тернарные операторы

Тернарный оператор состоит из трех элементов — проверки и двух возможных исходов. Формат определения:

```
test ? true_statement : false_statement
```

Например, вот как можно использовать тернарный оператор, чтобы определить абсолютное значение `$x`:

```
$abs = $x >= 0 ? $x : - $x;
```

5.10.3. Инструкция if

Синтаксис инструкции if:

```
if (test) { true_statements }  
if (test) { true_statements } else { false_statements }
```

Эта инструкция часто используется в программах, выполняющих разные действия для IPv4 и IPv6. Например:

```
if ($inet_family == $AF_INET) {  
    // IPv4  
    ...  
} else {  
    // IPv6  
    ...  
}
```

Инструкция «else if» в настоящее время не поддерживается.

5.10.4. Развернутые циклы

BPF работает в ограниченной среде, где должна быть возможность проверить, что программа завершается и не застревает в бесконечном цикле. Для организации циклов в программах bpftrace поддерживает циклы в форме функции unroll().

Ее синтаксис:

```
unroll (count) { statements }
```

Аргумент count — это целочисленный литерал (константа) с максимально возможным значением 20. Передача переменной в аргументе count не поддерживается, потому что число итераций цикла должно быть известно на этапе компиляции.

В ядре Linux 5.3 появилась поддержка ограниченных циклов в BPF, которая также должна появиться в будущих версиях bpftrace, например, для организации циклов while.

5.11. ОПЕРАТОРЫ BPFTRACE

В предыдущем разделе я перечислил логические операторы для использования в проверках. Но bpftrace поддерживает также следующие операторы:

- **=**: присваивание;
- **+**, **-**, *****, **/**: сложение, вычитание, умножение, деление;
- **++**, **--**: автоинкремент и автодекремент;
- **&**, **|**, **^**: поразрядное И, поразрядное ИЛИ, поразрядное ИСКЛЮЧАЮЩЕЕ ИЛИ;
- **!**: логическое отрицание (НЕ);

- <<, >>: сдвиг влево, сдвиг вправо;
- +=, -=, *=, /=, %=, &=, ^=, <<=, >>=: составные операторы.

Эти операторы действуют так же, как и в C.

5.12. ПЕРЕМЕННЫЕ BPFTRACE

Как рассказывалось в разделе 5.7.10, bpftrace поддерживает три вида переменных: встроенные, временные и переменные-карты.

5.12.1. Встроенные переменные

Встроенные переменные предопределены в bpftrace и обычно доступны только для чтения. Наиболее важные из них перечислены в табл. 5.5.

Таблица 5.5. Наиболее важные встроенные переменные в bpftrace

Переменная	Тип	Описание
pid	целое число	Идентификатор процесса (tgid в ядре)
tid	целое число	Идентификатор потока (pid в ядре)
uid	целое число	Идентификатор пользователя
username	строка	Имя пользователя
nsecs	целое число	Отметка времени в наносекундах
elapsed	целое число	Время в наносекундах, прошедшее с начала инициализации bpftrace
cpu	целое число	Идентификатор процессора
comm	строка	Имя процесса
kstack	строка	Трассировка стека в пространстве ядра
ustack	строка	Трассировка стека в пространстве пользователя
arg0, ..., argN	целое число	Аргументы зондов некоторых типов (см. раздел 5.9)
args	структура	Аргументы зондов некоторых типов (см. раздел 5.9)
retval	целое число	Возвращаемые значения для зондов некоторых типов (см. раздел 5.9)
func	строка	Имя трассируемой функции
probe	строка	Полное имя текущего зонда
curtask	целое число	task_struct в ядре как 64-битное целое без знака (допускается приведение типа)
cgroup	целое число	Идентификатор cgroup
\$1, ..., \$N	int, char *	Позиционные параметры программы bpftrace

Все целочисленные переменные имеют тип `uint64`. Они связаны с текущим потоком выполнения, зондом, функцией и процессором. Актуальный список встроенных переменных ищите в онлайн-руководстве «bpftrace Reference Guide» [66].

5.12.2. Встроенные переменные `pid`, `comm` и `uid`

Многие встроенные переменные просты в использовании. Вот, например, как с помощью переменных `pid`, `comm` и `uid` можно узнать, кто осуществил системный вызов `setuid()`:

```
# bpftrace -e 't:syscalls:sys_enter_setuid {
    printf("setuid by PID %d (%s), UID %d\n", pid, comm, uid); }'
Attaching 1 probe...
setuid by PID 3907 (sudo), UID 1000
setuid by PID 14593 (evil), UID 33
^C
```

Факт обращения к этому системному вызову еще не означает, что тот был успешным. Чтобы определить успех/неудачу, можно проверить возвращаемое значение, используя другую точку трассировки:

```
# bpftrace -e 'tracepoint:syscalls:sys_exit_setuid {
    printf("setuid by %s returned %d\n", comm, args->ret); }'
Attaching 1 probe...
setuid by sudo returned 0
setuid by evil returned -1
^C
```

Здесь используется еще одна встроенная переменная `args`. В точках трассировки `tracepoint` переменная `args` имеет тип структуры с полями.

5.12.3. Встроенные переменные `kstack` и `ustack`

Переменные `kstack` и `ustack` возвращают трассировку стека в пространстве ядра и пользователя соответственно в виде многострочного текста. Они могут возвращать до 127 фреймов стека. Функции `kstack()` и `ustack()`, о которых идет речь ниже, позволяют выбирать число фреймов стека.

Вот пример использования `kstack` для вывода трассировки стека ядра операции блочного ввода/вывода:

```
# bpftrace -e 't:block:block_rq_insert { printf("Block I/O by %s\n", kstack); }'
Attaching 1 probe...

Block I/O by
    blk_mq_insert_requests+203
    blk_mq_sched_insert_requests+111
    blk_mq_flush_plug_list+446
    blk_flush_plug_list+234
    blk_finish_plug+44
```



```

dmccrypt_write+593
kthread+289
ret_from_fork+53

```

Block I/O by

```

blk_mq_insert_requests+203
blk_mq_sched_insert_requests+111
blk_mq_flush_plug_list+446
blk_flush_plug_list+234
blk_finish_plug+44
__do_page_cache_readahead+474
ondemand_readahead+282
page_cache_sync_readahead+46
generic_file_read_iter+2043
ext4_file_read_iter+86
new_sync_read+228
__vfs_read+41
vfs_read+142
kernel_read+49
prepare_binprm+239
do_execveat_common.isra.34+1428
sys_execve+49
do_syscall_64+115
entry_SYSCALL_64_after_hwframe+61

```

[...]

Трассировки стека выводятся в порядке от дочерних вызовов к родительским, каждый фрейм описывается именем функции и ее смещением.

Встроенные переменные `kstack` и `ustack` можно также использовать как ключи в картах, что позволяет подсчитывать их количество. Вот подсчет трассировок стека в пространстве ядра, приведших к блочной операции ввода/вывода:

```
# bpftrace -e 't:block:block_rq_insert { @[kstack] = count(); }'
```

```
Attaching 1 probe...
```

```
^C
```

```
[...]
```

```

@[
  blk_mq_insert_requests+203
  blk_mq_sched_insert_requests+111
  blk_mq_flush_plug_list+446
  blk_flush_plug_list+234
  blk_finish_plug+44
  dmccrypt_write+593
  kthread+289
  ret_from_fork+53
]: 39

```

```

@[
  blk_mq_insert_requests+203
  blk_mq_sched_insert_requests+111
  blk_mq_flush_plug_list+446
  blk_flush_plug_list+234
  blk_finish_plug+44

```

```
__do_page_cache_readahead+474
ondemand_readahead+282
page_cache_sync_readahead+46
generic_file_read_iter+2043
ext4_file_read_iter+86
new_sync_read+228
__vfs_read+41
vfs_read+142
sys_read+85
do_syscall_64+115
entry_SYSCALL_64_after_hwframe+61
]: 52
```

Здесь показаны только две последние трассировки, встретившиеся 39 и 52 раза. Подсчет выполняется намного эффективнее, чем вывод каждой трассировки, так как подсчет ведется в пространстве ядра¹.

5.12.4. Встроенные переменные: позиционные параметры

Позиционные параметры передаются в программу через командную строку и получают имена по аналогии с позиционными параметрами в командной оболочке. Имя \$1 соответствует первому аргументу, \$2 — второму, и т. д.

Вот пример простой программы watchconn.bt:

```
BEGIN
{
    printf("Watching connect() calls by PID %d\n", $1);
}

tracepoint:syscalls:sys_enter_connect
/pid == $1/
{
    printf("PID %d called connect()\n", $1);
}
```

Она ждет, пока процесс с заданным PID вызовет функцию connect():

```
# ./watchconn.bt 181
Attaching 2 probes...
Watching connect() calls by PID 181
PID 181 called connect()
[...]
```

Позиционные параметры также работают с этими типами вызовов:

```
bpftrace ./watchconn.bt 181
bpftrace -e 'program' 181
```

¹ BPF превращает каждую трассировку в уникальный идентификатор, а затем подсчитывает частоту этих идентификаторов. bpftrace читает эти счетчики и затем извлекает трассировки, соответствующие идентификаторам.

По умолчанию они интерпретируются как целые числа. Если в таком параметре передается строковый аргумент, его значение следует извлекать с помощью вызова `str()`. Например:

```
# bpftrace -e 'BEGIN { printf("Hello, %s!\n", str($1)); }' Reader
Attaching 1 probe...
Hello, Reader!
^C
```

Если параметр, используемый в программе, не был передан в командной строке, он получит нулевое значение в целочисленном контексте или «» — в строковом, при использовании `str()`.

5.12.5. Временные переменные

Формат определения:

```
$name
```

Эти переменные можно использовать для временного хранения результатов вычислений в пределах блока, определяющего действие. Типы переменных определяются первой операцией присваивания и могут хранить целые числа, строки, указатели на структуры и структуры.

5.12.6. Карты

Формат определения:

```
@name
@name[key]
@name[key1, key2[, ...]]
```

Для хранения этих переменных в BPF используются объекты-карты — хеш-таблицы (ассоциативные массивы), способные хранить значения разных типов. Значения могут храниться с одним или несколькими ключами. Ключи и значения в картах должны иметь непротиворечивые типы.

Как и в случае с временными переменными, типы определяются при первом присваивании, в том числе при присваивании специальных функций. Тип карты определяется типом ключей, если они есть, и типом значений. Рассмотрим инструкции присваивания:

```
@start = nsecs;
@last[tid] = nsecs;
@bytes = hist(retval);
@who[pid, comm] = count();
```

Карты `@start` и `@last` получают целочисленный тип, потому что им обеим присваивается целое число: значение встроенной целочисленной переменной с отметкой

времени в наносекундах (nsecs). Карта @last также получит ключ целочисленного типа, потому что он используется в операции присваивания: идентификатор потока (tid). Карта @bytes получит особый тип — тип гистограммы и будет хранить и возвращать гистограммы. Наконец, карта @who получит составные ключи, включающие целое число (pid) и строку (comm), и значения — функцию-карту count(). Эти функции описываются в разделе 5.14.

5.13. ФУНКЦИИ BPFTRACE

bpftrace предлагает множество встроенных функций для решения самых разных задач. Наиболее важные из них перечислены в табл. 5.6.

Таблица 5.6. Наиболее важные функции bpftrace

Функция	Описание
printf(char *fmt [, ...])	Форматированный вывод
time(char *fmt)	Форматированный вывод времени
join(char *arr[])	Выводит массив строк, объединяя их через пробел
str(char *s [, int len])	Возвращает строку, на которую ссылается указатель s, с необязательным ограничителем длины len
kstack(int limit)	Возвращает трассировку стека ядра с глубиной до <i>limit</i>
ustack(int limit)	Возвращает трассировку стека в пространстве пользователя с глубиной до <i>limit</i>
kSYM(void *p)	Определяет символ по адресу в пространстве ядра и возвращает строку с ним
uSYM(void *p)	Определяет символ по адресу в пространстве пользователя и возвращает строку с ним
kaddr(char *name)	Определяет адрес символа в пространстве ядра
uaddr(char *name)	Определяет адрес символа в пространстве пользователя
reg(char *name)	Возвращает значение, хранящееся в указанном регистре
ntop([int af,] int addr)	Возвращает строковое представление IP-адреса
system(char *fmt [, ...])	Выполняет команду в командной оболочке
cat(char *filename)	Выводит содержимое указанного файла
exit()	Завершает выполнение bpftrace

Некоторые из этих функций действуют асинхронно: ядро ставит событие в очередь, и спустя короткое время событие обрабатывается в пространстве пользователя. Асинхронными являются функции printf(), time(), cat(), join() и system(). Функции

kstack(), ustack(), ksym() и usym() записывают адреса синхронно, но разрешение символов выполняется асинхронно.

Актуальный список функций ищите в онлайн-руководстве «bpftrace Reference Guide» [66]. В следующих разделах приведено описание некоторых из функций.

5.13.1. printf()

Функция printf() служит для форматированного вывода и действует так же, как в С. Синтаксис:

```
printf(format [, arguments ...])
```

Строка format может содержать любое текстовое сообщение, а также экранированные последовательности, начинающиеся с символа «\», и описатели полей, начинающиеся с символа «%». Если аргументов нет, описатели полей не нужны.

Основные экранированные последовательности:

- \n: перевод строки;
- \": двойная кавычка;
- \\: обратный слеш.

Другие экранированные последовательности можно найти на странице справочного руководства с описанием функции printf(1).

Описатели полей начинаются с символа «%» и имеют формат:

```
% [-] width type
```

Дефис «-» означает вывод с выравниванием по левому краю. По умолчанию выравнивание производится по правому краю.

width — это ширина поля в символах.

Элемент type может быть одним из следующих:

- %u, %d: целое без знака, целое со знаком;
- %lu, %ld: длинное целое без знака, длинное целое со знаком;
- %llu, %lld: длинное длинное целое без знака, длинное длинное целое со знаком;
- %hu, %hd: короткое целое без знака, короткое целое со знаком;
- %x, %lx, %llx: в шестнадцатеричном формате: целое без знака, длинное целое без знака, длинное длинное целое без знака;
- %c: символ;
- %s: строка.

Следующий вызов printf():

```
printf("%16s %-6d\n", comm, pid)
```

выведет значение встроенной переменной `comm` как строку в поле шириной 16 символов и с выравниванием по правому краю, значение встроенной переменной `pid` как целое число в поле с шириной 6 символов и с выравниванием по левому краю, и в конце — символ перевода строки.

5.13.2. `join()`

`join()` — специальная функция для объединения и вывода элементов массива строк через пробел. Синтаксис:

```
join(char *arr[])
```

Следующий однострочный сценарий выводит информацию о попытках выполнения команд с их аргументами:

```
# bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
Attaching 1 probe...
ls -l
df -h
date
ls -l bashreadline.bt biolatency.bt biosnoop.bt bitesize.bt
```

Он выводит массив `argv` системного вызова `execve()`. Обратите внимание, что эта информация именно о попытках выполнения: успешность выполнения системного вызова можно определить по значению `args->ret` точки трассировки `syscalls:sys_exit_execve`.

Функция `join()` может оказаться очень удобной в некоторых случаях, но она имеет ограничения по числу объединяемых аргументов и их размерам¹. Если вывод выглядит усеченным, возможно, вы достигли этих пределов и нужно использовать другой подход.

Была проделана работа по изменению поведения `join()`, чтобы она возвращала строку, а не выводила ее. Когда такое изменение вступит в силу, предыдущий однострочник `bpftrace` будет выглядеть так:

```
# bpftrace -e 'tracepoint:syscalls:sys_enter_execve {
    printf("%s\n", join(args->argv)); }'
```

Кроме того, это изменение исключит `join()` из категории асинхронных функций².

¹ Текущие ограничения — 16 аргументов и не более 1 Кбайт на каждый. Она выводит все аргументы, пока не достигнет аргумента со значением `NULL` или не превысит предел в 16 аргументов.

² См. проблему 26 в `bpftrace`, чтобы узнать о статусе этого изменения [67]. Оно не относится к числу первоочередных, потому что до сих пор у `join()` был один вариант использования: объединение `args->argv` для точки трассировки системного вызова `execve`.

5.13.3. str()

str() возвращает строку, на которую ссылается указатель (char *). Синтаксис:

```
str(char *s [, int length])
```

Функция readline() в командной оболочке bash(1) возвращает строку, и ее можно вывести, как показано ниже¹:

```
# bpftrace -e 'ur:/bin/bash:readline { printf("%s\n", str(retval)); }'
Attaching 1 probe...
ls -lh
date
echo hello BPF
^C
```

Этот однострочный сценарий может отображать все интерактивные команды bash, выполняемые в системе.

По умолчанию размер строки ограничен 64 байтами, но этот предел можно настроить с помощью переменной среды BPFTRACE_STRLLEN. Строки с размерами более 200 байт недопустимы. Это известное ограничение, и однажды оно может быть значительно увеличено².

5.13.4. kstack() и ustack()

kstack() и ustack() похожи на встроенные переменные kstack и ustack, но принимают обязательный аргумент limit и необязательный аргумент mode. Синтаксис:

```
kstack(limit)
kstack(mode[, limit])
ustack(limit)
ustack(mode[, limit])
```

Следующий сценарий выводит три верхних фрейма в трассировках стека ядра, ведущих к операции блочного ввода/вывода, используя для этого точку трассировки block:block_rq_insert:

```
# bpftrace -e 't:block:block_rq_insert { @[kstack(3), comm] = count(); }'
Attaching 1 probe...
^C
```

¹ Этот сценарий предполагает наличие функции readline() в двоичном файле bash(1); некоторые сборки bash(1) могут вызывать эту функцию из libreadline, и тогда этот сценарий нужно изменить соответственно. См. раздел 12.2.3 в главе 12.

² Подробности см. в проблеме bpftrace с номером 305 [68]. Дело заключается в том, что для хранения строк сейчас используется стек BPF, который ограничен 512 байтами и, следовательно, ограничивает размер строк 200 байтами. В дальнейшем предполагается для хранения строк использовать карты BPF, способные хранить очень большие строки (в несколько мегабайт).

```
@[
    __elv_add_request+231
    blk_execute_rq_nowait+160
    blk_execute_rq+80
, kworker/u16:3]: 2
@[
    blk_mq_insert_requests+203
    blk_mq_sched_insert_requests+111
    blk_mq_flush_plug_list+446
, mysqld]: 2
@[
    blk_mq_insert_requests+203
    blk_mq_sched_insert_requests+111
    blk_mq_flush_plug_list+446
, dmccrypt_write]: 961
```

Сейчас максимальный размер трассировки стека ограничен 1024 фреймами.

Аргумент `mode` позволяет по-разному форматировать вывод. Поддерживаются только два режима: «bpftrace», используемый по умолчанию, и «perf», который выводит трассировку в формате, подобном формату утилиты Linux `perf(1)`. Например:

```
# bpftrace -e 'k:do_nanosleep { printf("%s", ustack(perf)); }'
Attaching 1 probe...
[...]
7f220f1f2c60 nanosleep+64 (/lib/x86_64-linux-gnu/libpthread-2.27.so)
7f220f653fdd g_timeout_add_full+77 (/usr/lib/x86_64-linux-gnu/libglib-
2.0.so.0.5600.3)
7f220f64fbc0 0x7f220f64fbc0 ([unknown])
841f0f 0x841f0f ([unknown])
```

В будущем будут добавлены другие режимы вывода.

5.13.5. `ksym()` и `usym()`

Функции `ksym()` и `usym()` преобразуют адреса в соответствующие имена символов (строки). `ksym()` преобразует адреса в пространстве ядра, а `usym()` — в пространстве пользователя. Синтаксис:

```
ksym(addr)
usym(addr)
```

Например, точка трассировки `timer:hrtimer_start` имеет аргумент — указатель на функцию. Вот как с ее помощью реализовать подсчет частоты вызовов:

```
# bpftrace -e 'tracepoint:timer:hrtimer_start { @[args->function] = count(); }'
Attaching 1 probe...
^C

@[-1169374160]: 3
@[-1168782560]: 8
@[-1167295376]: 9
@[-1067171840]: 145
```



```
@[-1169062880]: 200
@[-1169114960]: 2517
@[-1169048384]: 8237
```

Здесь выводятся простые адреса в памяти. С помощью `ksym()` их можно преобразовать в имена функций:

```
# bpftrace -e 'tracepoint:timer:hrtimer_start { @[ksym(args->function)] = count(); }'
Attaching 1 probe...
^C
```

```
@[sched_rt_period_timer]: 4
@[watchdog_timer_fn]: 8
@[timerfd_tmrproc]: 15
@[intel_uncore_fw_release_timer]: 1111
@[it_real_fn]: 2269
@[hrtimer_wakeup]: 7714
@[tick_sched_timer]: 27092
```

Функция `usym()` в своей работе опирается на таблицы символов, входящие в состав двоичных файлов.

5.13.6. `kaddr()` и `uaddr()`

Функции `kaddr()` и `uaddr()` принимают имя символа и возвращают его адрес. `kaddr()` преобразует символы в пространстве ядра, а `uaddr()` — в пространстве пользователя. Синтаксис:

```
kaddr(char *name)
uaddr(char *name)
```

Вот как можно получить адрес символа «`ps1_prompt`» в пространстве пользователя при вызове функции командной оболочки `bash(1)` и затем получить и вывести строку, хранящуюся по этому адресу:

```
# bpftrace -e 'uprobe:/bin/bash:readline {
    printf("PS1: %s\n", str(*uaddr("ps1_prompt"))); }'
Attaching 1 probe...
PS1: \[\e[34;1m\]\u@\h:\w>\[\e[0m\]
PS1: \[\e[34;1m\]\u@\h:\w>\[\e[0m\]
^C
```

Эта команда выведет содержимое символа — в данном случае строку приглашения PS1 в `bash(1)`.

5.13.7. `system()`

Функция `system()` выполняет указанную команду в командной оболочке. Синтаксис:

```
system(char *fmt [, arguments ...])
```

Поскольку эта функция позволяет выполнить любую команду, она считается небезопасной, и для ее вызова команде bpftrace нужно явно передать параметр `--unsafe`.

Вот как можно вызвать команду `ps(1)`, чтобы вывести дополнительную информацию о процессе, вызвавшем `nanosleep()`:

```
# bpftrace --unsafe -e 't:syscalls:sys_enter_nanosleep { system("ps -p %d\n",
    pid); }'
Attaching 1 probe...
  PID TTY          TIME CMD
29893 tty2      05:34:22 mysqld
  PID TTY          TIME CMD
29893 tty2      05:34:22 mysqld
  PID TTY          TIME CMD
29893 tty2      05:34:22 mysqld
[...]
```

Если трассируемое событие возникает очень часто, функция `system()` породит шквал новых процессов, потребляющих вычислительные ресурсы. Используйте `system()` только при необходимости.

5.13.8. `exit()`

Эта функция завершает выполнение программы bpftrace. Синтаксис:

```
exit()
```

Функцию можно использовать для ограничения времени выполнения зондов. Например:

```
# bpftrace -e 't:syscalls:sys_enter_read { @reads = count(); }
    interval:s:5 { exit(); }'
Attaching 2 probes...
@reads: 735
```

Как показано в этом примере, за 5 секунд произошло 735 обращений к системному вызову `read()`. Все карты выводятся после завершения bpftrace.

5.14. ФУНКЦИИ-КАРТЫ В BPFTRACE

Карты — это специальные хеш-таблицы, хранящиеся в BPF, которые можно использовать для разных целей, например для хранения пар ключ — значение или статистических сводок. bpftrace предоставляет встроенные функции для привязки карт и операций с ними — в основном для карт со статистическими сводками. Наиболее важные функции-карты перечислены в табл. 5.7.

Некоторые из этих функций действуют асинхронно: ядро ставит событие в очередь, а спустя короткое время событие обрабатывается в пространстве пользователя. К асинхронным функциям относятся: `print()`, `clear()` и `zero()`. Не забывайте о своих собственных им задержках, когда будете писать свои программы.

Таблица 5.7. Наиболее важные функции-карты в bpftrace

Функция	Описание
count()	Подсчитывает число вхождений
sum(int n)	Подсчитывает сумму значений
avg(int n)	Вычисляет среднее значение
min(int n)	Запоминает минимальное значение
max(int n)	Запоминает максимальное значение
stats(int n)	Возвращает общее количество, среднее и сумму
hist(int n)	Выводит гистограмму значений с шагом, равным степени двойки
lhist(int n, int min, int max, int step)	Выводит линейную гистограмму значений
delete(@m[key])	Удаляет пару ключ — значение из карты
print(@m[, top[, div]])	Выводит содержимое карты с необязательными ограничениями на вывод определенного числа наибольших значений и делителем
clear(@m)	Удаляет все пары ключ — значение из карты
zero(@m)	Сбрасывает все значения в карте в ноль

Актуальный список функций можно найти в онлайн-руководстве «bpftrace Reference Guide» [66]. В следующих разделах приведено описание некоторых из функций.

5.14.1. count()

Функция count() подсчитывает число вхождений. Синтаксис:

```
@m = count();
```

Эту функцию можно использовать с определениями зондов, включающими подстановочные символы, и со встроенными счетчиками событий:

```
# bpftrace -e 'tracepoint:block:* { @[probe] = count(); }'
Attaching 18 probes...
^C
```

```
@[tracepoint:block:block_rq_issue]: 1
@[tracepoint:block:block_rq_insert]: 1
@[tracepoint:block:block_dirty_buffer]: 24
@[tracepoint:block:block_touch_buffer]: 29
@[tracepoint:block:block_rq_complete]: 52
@[tracepoint:block:block_getrq]: 91
@[tracepoint:block:block_bio_complete]: 102
@[tracepoint:block:block_bio_remap]: 180
@[tracepoint:block:block_bio_queue]: 270
```

Используя интервальные зонды (`interval`), можно организовать вывод частоты вхождений в течение каждого интервала, например:

```
# bpftrace -e 'tracepoint:block:block_rq_i* { @[probe] = count(); }
    interval:s:1 { print(@); clear(@); }'
Attaching 3 probes...
@[tracepoint:block:block_rq_issue]: 1
@[tracepoint:block:block_rq_insert]: 1

@[tracepoint:block:block_rq_insert]: 6
@[tracepoint:block:block_rq_issue]: 8

@[tracepoint:block:block_rq_issue]: 1
@[tracepoint:block:block_rq_insert]: 1
[...]
```

Того же эффекта можно добиться с помощью `perf(1)` и `perf stat`, а также `Ftrace`. Но `bpftrace` предлагает более широкие возможности: зонд `BEGIN` может содержать вызов `printf()` для описания выходных данных, а интервальный зонд (`interval`) может включать вызов `time()` вывода дополнительных отметок времени.

5.14.2. `sum()`, `avg()`, `min()` и `max()`

Эти функции вычисляют простые статистические данные — сумму, среднее, минимальное и максимальное значение — и хранят их в карте. Синтаксис:

```
sum(int n)
avg(int n)
min(int n)
max(int n)
```

Вот пример использования `sum()` для подсчета общего числа байтов, прочитанных системным вызовом `read(2)`:

```
# bpftrace -e 'tracepoint:syscalls:sys_exit_read /args->ret > 0/ {
    @bytes = sum(args->ret); }'
Attaching 1 probe...
^C

@bytes: 461603
```

Карте было присвоено имя «`bytes`», чтобы в выводе была дополнительная подсказка. Обратите внимание, что в этом примере используется фильтр для отсеивания отрицательных значений `args->ret`. Положительное значение, возвращаемое системным вызовом `read(2)`, определяет число прочитанных байтов, а отрицательное является кодом ошибки. Это описано в справочной странице для `read(2)`.

5.14.3. `hist()`

`hist()` сохраняет значение в гистограмме с шагом, равным степени двойки. Синтаксис:

```
hist(int n)
```



```
@ret:
(..., 0)          101 |@@@
[0, 100)         1569 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[100, 200)        5  |
[200, 300)        0  |
[300, 400)        3  |
[400, 500)        0  |
[500, 600)        0  |
[600, 700)        3  |
[700, 800)        0  |
[800, 900)        0  |
[900, 1000)       0  |
[1000, ...)       5  |
```

Судя по результатам, в большинстве случаев данные читались блоками от нуля до 100 и менее байт. Диапазоны обозначаются с использованием той же нотации, что и в `hist()`. Строка «(..., 0)» показывает количество ошибок: в период трассировки системный вызов `read(2)` столкнулся с ошибкой 101 раз. Обратите внимание, что подсчет ошибок лучше вести иначе, например, с разделением по кодам ошибок:

```
# bpftrace -e 'tracepoint:syscalls:sys_exit_read /args->ret < 0/ {
    @[- args->ret] = count(); }'
Attaching 1 probe...
^C
```

```
@[11]: 57
```

Код 11 соответствует ошибке `EAGAIN` (попробуй еще раз). Столкнувшись с этой ошибкой, `read(2)` возвращает код -11.

5.14.5. delete()

`delete()` удаляет пару ключ — значение из карты. Синтаксис:

```
delete(@map[key])
```

В зависимости от типа карты требуется указать несколько ключей.

5.14.6. clear() и zero()

`clear()` удаляет все пары ключ — значение из карты, а `zero()` сбрасывает все значения в ноль. Синтаксис:

```
clear(@map)
zero(@map)
```

Когда программа завершается, `bpftrace` по умолчанию выводит все карты. Некоторые карты, например, используемые для вычисления разницы между отметками времени, не предназначены для вывода в составе результатов. Их можно очистить с помощью зонда `END` и предотвратить автоматический вывод:

```
[...]
END
{
    clear(@start);
}
```

5.14.7. print()

print() выводит содержимое карт. Синтаксис:

```
print(@m [, top [, div]])
```

Функция принимает два необязательных аргумента: целое число top определяет, сколько наибольших значений из карты требуется вывести, а целое число div определяет делитель для выводимых значений.

В программе ниже показано применение аргумента top. Она выводит пять самых часто вызываемых функций ядра, имена которых начинаются с «vfs_»:

```
# bpftrace -e 'kprobe:vfs_* { @[probe] = count(); } END { print(@, 5); clear(@); }'
Attaching 55 probes...
^C
@[kprobe:vfs_getattr_nosec]: 510
@[kprobe:vfs_getattr]: 511
@[kprobe:vfs_writev]: 1595
@[kprobe:vfs_write]: 2086
@[kprobe:vfs_read]: 2921
```

Как видите, в период, когда выполнялась трассировка, чаще других вызывалась функция vfs_read(), она была вызвана 2921 раз.

В программе ниже показано применение аргумента div. Она выводит время в миллисекундах, проведенное процессами в vfs_read():

```
# bpftrace -e 'kprobe:vfs_read { @start[tid] = nsecs; }
               kretprobe:vfs_read /@start[tid]/ {
                   @ms[comm] = sum(nsecs - @start[tid]); delete(@start[tid]); }
               END { print(@ms, 0, 1000000); clear(@ms); clear(@start); }'
Attaching 3 probes...
[...]
@ms[Xorg]: 3
@ms[InputThread]: 3
@ms[chrome]: 4
@ms[Web Content]: 5
```

Обязательно ли использовать аргумент делителя? Казалось бы, того же эффекта можно добиться иначе, например:

```
@ms[comm] = sum((nsecs - @start[tid]) / 1000000);
```

Проблема в том, что sum() работает с целыми числами, просто отбрасывая дробную часть. В результате продолжительность любой операции, длящейся меньше 1 миллисекунды, будет оценена как имеющая нулевую длительность. Поэтому из-за ошибок

округления будет получено ошибочное значение. Эту проблему можно решить, если передать в `sum()` длительность в наносекундах, а затем разделить итоговую сумму на делитель, как это делает `print()` с аргументом `div`.

В будущем функция `print()` в `bpftrace` может быть расширена для вывода переменных любых типов, а не только карт.

5.15. НАПРАВЛЕНИЯ РАЗВИТИЯ BPFTRACE В БУДУЩЕМ

В ближайшем будущем планируется добавить в `bpftrace` несколько расширений. Более подробную информацию об этих расширениях ищите в примечаниях к релизу и в документации в репозитории проекта `bpftrace`: <https://github.com/iovisor/bpftrace>.

В исходном коде `bpftrace`, включенном в эту книгу, нет запланированных изменений. Если вам нужны эти изменения, проверьте наличие обновлений на сайте книги: <http://www.brendangregg.com/bpf-performance-tools-book.html>.

5.15.1. Режимы явной адресации

Самым крупным расширением в `bpftrace` будет явный доступ к адресному пространству для поддержки будущего разделения `bpf_probe_read()` на `bpf_probe_read_kernel()` и `bpf_probe_read_user()` [69]. Это разделение нужно для поддержки некоторых архитектур процессоров¹. Оно не должно повлиять ни на один из инструментов в этой книге. Оно также должно привести к добавлению в `bpftrace` функций `kptr()` и `uptr()` для определения режима адресации. Предполагается, что они будут иметь весьма ограниченный круг применения: при возможности `bpftrace` будет определять контекст адресного пространства по типу зонда или используемой функции. Ниже показано, как должен работать контекст зонда:

`kprobe/kretprobe` (контекст ядра):

- **arg0...argN, retval**: при разыменовании получают адреса в пространстве ядра;
- ***addr**: разыменование адреса в пространстве ядра;
- **str(addr)**: извлекает строку из пространства ядра, завершающуюся пустым символом (NULL);
- ***uptr(addr)**: разыменование адреса в пространстве пользователя;
- **str(uptr(addr))**: извлекает строку из пространства пользователя, завершающуюся пустым символом (NULL).

`uprobe/uretprobe` (контекст пространства пользователя):

- **arg0...argN, retval**: при разыменовании получают адреса в пространстве пользователя;

¹ «Они редки, но есть. По крайней мере `sparc32` и старая архитектура `x86` с адресуемым пространством `4G:4G`». — Линус Торвальдс [70].

- ***addr**: разыменование адреса в пространстве пользователя;
- **str(addr)**: извлекает строку из пространства пользователя, завершающуюся пустым символом (NULL);
- ***kptr(addr)**: разыменование адреса в пространстве ядра;
- **str(kptr(addr))**: извлекает строку из пространства ядра, завершающуюся пустым символом (NULL).

То есть *addr и str() продолжат работать, но будут ссылаться на адресное пространство в контексте зонда: память ядра для kprobes и память пользователя для uprobes. Для перекрестной ссылки между адресными пространствами придется использовать функции kptr() и uptr(). Некоторые функции, например curtask(), всегда будут возвращать указатель на память ядра, независимо от контекста (как и следовало ожидать).

Некоторые типы зондов по умолчанию соответствуют контексту ядра, но среди них будут исключения, описанные в справочном руководстве «bpftrace Reference Guide» [66]. Одним из таких исключений станут точки трассировки системных вызовов, которые ссылаются на память в адресном пространстве пользователя, поэтому их контекстом будет считаться пространство пользователя.

5.15.2. Другие расширения

Среди других планируемых расширений можно назвать:

- дополнительные типы зондов для поддержки точек наблюдения за памятью¹, сокетов и программ skb, а также неструктурированных точек трассировки;
- установка зондов uprobe и kprobe со смещением;
- циклы for и while, использующие поддержку ограниченных циклов в BPF, появившуюся в Linux 5.3;
- неструктурированные счетчики РМС (с поддержкой маски umask и селекторов событий);
- добавление в uprobes поддержки относительных имен (например, определение uprobe:/lib/x86_64-linux-gnu/libc.so.6:... и uprobe:libc:... должны работать одинаково);
- функция signal() для отправки сигналов (включая SIGKILL) процессам;
- функция return() или override() для изменения возвращаемых событий (с использованием bpf_override_return());
- функция ehist() для получения экспоненциальных гистограмм. Любой инструмент, использующий степенную гистограмму (hist), можно будет переключить на использование ehist() для увеличения разрешающей способности;

¹ Дэн Сюй (Dan Xu) уже разработал доказательство реализуемости идеи точек наблюдения за памятью, которая будет добавлена в bpftrace [71].

- `rcomm` для получения имени процесса, `comm` возвращает имя потока, которое часто совпадает с именем процесса, но не всегда. Например, приложения на Java могут устанавливать `comm` и определять отдельные имена для каждого потока, в этом случае `rcomm` будет по-прежнему возвращать «Java»;
- вспомогательная функция для преобразования файловых указателей в полные пути.

Как только эти дополнения станут доступны, вы сможете использовать их в некоторых инструментах из этой книги, например заменить `hist()` на `ehist()`, чтобы получить более высокое разрешение, а в некоторых инструментах `uprobe` использовать относительные имена библиотек вместо полных путей.

5.15.3. ply

Интерфейс `ply` для BPF, созданный Тобиасом Вальдекранцем (Tobias Waldekranz), поддерживает высокоуровневый язык программирования, похожий на язык `bpftrace`, и имеет минимум зависимостей (без LLVM и Clang). Благодаря этому он отлично подходит для сред с ограниченными ресурсами. Его единственный существенный недостаток — отсутствие навигации по структурам и возможности подключения заголовочных файлов (которая нужна многим инструментам в этой книге).

Вот пример инструментации точки трассировки `open(2)`:

```
# ply 'tracepoint:syscalls/sys_enter_open {  
    printf("PID: %d (%s) opening: %s\n", pid, comm, str(data->filename)); }'  
ply: active  
PID: 22737 (Chrome_IOThread) opening: /dev/shm/.org.chromium.Chromium.dh4msB  
PID: 22737 (Chrome_IOThread) opening: /dev/shm/.org.chromium.Chromium.dh4msB  
PID: 22737 (Chrome_IOThread) opening: /dev/shm/.org.chromium.Chromium.2m1lx4  
[...]
```

Этот однострочный сценарий практически идентичен эквивалентному сценарию для `bpftrace`. В будущих версиях `ply` может появиться прямая поддержка языка `bpftrace`, что позволит запускать однострочные сценарии для `bpftrace`. Такие сценарии обычно не используют навигацию по структурам, кроме аргументов точек трассировки (как показано в примере выше), которые `ply` уже поддерживает. В отдаленном будущем, с развитием технологии BTF, `ply` сможет использовать BTF для извлечения информации о структурах и запускать более сложные инструменты `bpftrace`.

5.16. ВНУТРЕННЕЕ УСТРОЙСТВО BPFTRACE

На рис. 5.3 показано внутреннее устройство `bpftrace`.

Для подключения к зондам, загрузки программ и использования USDT трассировщик `bpftrace` обращается к `libbcc` и `libbpf`. Он также использует LLVM для компиляции программ в байт-код BPF.

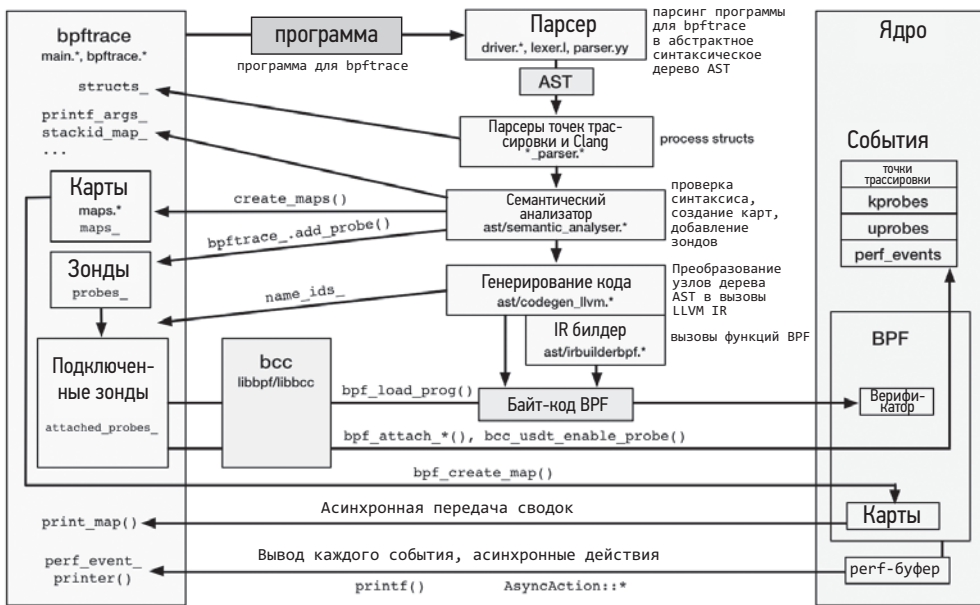


Рис. 5.3. Внутреннее устройство bpftrace

Язык bpftrace определяется в файлах `lex` и `yacc`, которые обрабатываются с помощью `flex` и `bison`. Результатом является абстрактное синтаксическое дерево (Abstract Syntax Tree, AST), представляющее программу. Затем парсеры точек трассировки (TP) и Clang обрабатывают структуры. Семантический анализатор проверяет языковые элементы и генерирует ошибки, обнаружив неправильное использование. Следующий шаг — генерация кода, то есть преобразование узлов абстрактного синтаксического дерева (AST) в инструкции на промежуточном языке LLVM IR, которые затем LLVM компилирует в байт-код BPF.

В следующем разделе описываются режимы отладки bpftrace, которые помогают увидеть все эти шаги в действии: `-d` выводит AST и LLVM IR, а `-v` — байт-код BPF.

5.17. ОТЛАДКА BPFTRACE

Есть разные способы отладки программ для bpftrace. В этом разделе поговорим об инструкции `printf()` и режимах отладки bpftrace. Если вы читаете этот раздел в надежде получить помощь в устранении проблемы, загляните в главу 18, где описываются наиболее типичные проблемы, в том числе пропуск событий, пропуск стеков и пропуск символов.

bpftrace — мощный язык, который в действительности имеет набор жестких ограничений для безопасной совместной работы и предотвращения неправильного использования. Для сравнения, интерфейс BCC, поддерживающий возможность

создания программ на C и Python, предлагает гораздо более богатые возможности, предназначенные не только для трассировки и совместной работы. В результате программы bpftrace «падают», генерируя информативные сообщения, что намного упрощает дальнейшую отладку. А вот программы ВСС могут падать совершенно неожиданно, и для их отладки требуются специальные режимы.

5.17.1. Отладка с помощью printf()

Добавляя инструкции printf(), можно убедиться, что зонды действительно срабатывают, и проверить соответствие значений переменных ожиданиям. Следующая программа выводит гистограмму продолжительности выполнения vfs_read(). Но если ее запустить, можно обнаружить выбросы с аномально большой продолжительностью. Найдете ошибку сами?

```
kprobe:vfs_read
{
    @start[tid] = nsecs;
}

kretprobe:vfs_read
{
    $duration_ms = (nsecs - @start[tid]) / 1000000;
    @ms = hist($duration_ms);
    delete(@start[tid]);
}
```

Если программа bpftrace будет запущена, когда vfs_read() уже выполняется, то сработает только зонд kretprobe и вычислит продолжительность, как «nsecs - 0», потому что значение @start[tid] не инициализировано. Исправить ошибку можно, добавив фильтр в kretprobe, проверяющий, что @start[tid] не равен нулю. Эту проблему легко обнаружить, применив инструкцию printf() для проверки входных данных:

```
printf("$duration_ms = (%d - %d) / 1000000\n", nsecs, @start[tid]);
```

bpftrace поддерживает также специальные режимы отладки (описываются далее), но ошибки вроде этой проще обнаружить с помощью инструкции printf(), добавленной в нужное место.

5.17.2. Режим отладки

Параметр -d запускает bpftrace в режиме отладки. В этом режиме трассировщик не выполняет программу, а показывает, в какой код LLVM IR она была скомпилирована. Имейте в виду, что наибольший интерес этот режим представляет только для разработчиков самого bpftrace и он описан здесь лишь для общего знакомства.

В этом режиме сначала выводится абстрактное синтаксическое дерево (Abstract Syntax Tree, AST), представляющее программу:

```
# bpftrace -d -e 'k:vfs_read { @[pid] = count(); }'
```

```
Program
```

```
k:vfs_read
=
  map: @
    builtin: pid
  call: count
```

затем следует промежуточный код LLVM IR программы:

```
; ModuleID = 'bpftrace'
source_filename = "bpftrace"
target datalayout = "e-m:e-p:64:64-i64:64-n32:64-S128"
target triple = "bpf-pc-linux"

; Function Attrs: nounwind
declare i64 @llvm.bpf.pseudo(i64, i64) #0

; Function Attrs: argmemonly nounwind
declare void @llvm.lifetime.start.p0i8(i64, i8* nocapture) #1

define i64 @"kprobe:vfs_read"(i8* nocapture readnone) local_unnamed_addr section
"s_kprobe:vfs_read_1" {
entry:
  %"@_val" = alloca i64, align 8
  %"@_key" = alloca [8 x i8], align 8
  %1 = getelementptr inbounds [8 x i8], [8 x i8]* %"@_key", i64 0, i64 0
  call void @llvm.lifetime.start.p0i8(i64 -1, i8* nonnull %1)
  %get_pid_tgid = tail call i64 @inttoptr (i64 14 to i64 ())*()
  %2 = lshr i64 %get_pid_tgid, 32
  store i64 %2, i8* %1, align 8
  %pseudo = tail call i64 @llvm.bpf.pseudo(i64 1, i64 1)
  %lookup_elem = call i8* @inttoptr (i64 1 to i8* (i8*, i8*)) (i64 %pseudo, [8 x i8]*
nonnull %"@_key")
  %map_lookup_cond = icmp eq i8* %lookup_elem, null
  br i1 %map_lookup_cond, label %lookup_merge, label %lookup_success

lookup_success:                                     ; preds = %entry
  %3 = load i64, i8* %lookup_elem, align 8
  %phitmp = add i64 %3, 1
  br label %lookup_merge

lookup_merge:                                       ; preds = %entry, %lookup_success
  %lookup_elem_val.0 = phi i64 [ %phitmp, %lookup_success ], [ 1, %entry ]
  %4 = bitcast i64* %"@_val" to i8*
  call void @llvm.lifetime.start.p0i8(i64 -1, i8* nonnull %4)
  store i64 %lookup_elem_val.0, i64* %"@_val", align 8
  %pseudo1 = call i64 @llvm.bpf.pseudo(i64 1, i64 1)
  %update_elem = call i64 @inttoptr (i64 2 to i64 (i8*, i8*, i8*, i64*)) (i64 %pseudo1,
[8 x i8]* nonnull %"@_key", i64* nonnull %"@_val", i64 0)
  call void @llvm.lifetime.end.p0i8(i64 -1, i8* nonnull %1)
  call void @llvm.lifetime.end.p0i8(i64 -1, i8* nonnull %4)
  ret i64 0
}

; Function Attrs: argmemonly nounwind
```

```
declare void @llvm.lifetime.end.p0i8(i64, i8* nocapture) #1

attributes #0 = { nounwind }
attributes #1 = { argmemonly nounwind }
```

Есть также режим `-dd` — режим вывода подробной отладочной информации, — в котором `bpftrace` дополнительно выводит код LLVM IR до и после оптимизации.

5.17.3. Режим подробного вывода

Параметр `-v` переводит `bpftrace` в режим подробного вывода. В этом режиме трассировщик выводит дополнительную информацию в процессе работы программы. Например:

```
# bpftrace -v -e 'k:vfs_read { @[pid] = count(); }'
Attaching 1 probe...

Program ID: 5994

Bytecode:
0: (85) call bpf_get_current_pid_tgid#14
1: (77) r0 >>= 32
2: (7b) *(u64 *) (r10 -16) = r0
3: (18) r1 = 0xffff892f8c92be00
5: (bf) r2 = r10
6: (07) r2 += -16
7: (85) call bpf_map_lookup_elem#1
8: (b7) r1 = 1
9: (15) if r0 == 0x0 goto pc+2
   R0=map_value(id=0,off=0,ks=8,vs=8,imm=0) R1=inv1 R10=fp0
10: (79) r1 = *(u64 *) (r0 +0)
   R0=map_value(id=0,off=0,ks=8,vs=8,imm=0) R1=inv1 R10=fp0
11: (07) r1 += 1
12: (7b) *(u64 *) (r10 -8) = r1
13: (18) r1 = 0xffff892f8c92be00
15: (bf) r2 = r10
16: (07) r2 += -16
17: (bf) r3 = r10
18: (07) r3 += -8
19: (b7) r4 = 0
20: (85) call bpf_map_update_elem#2
21: (b7) r0 = 0
22: (95) exit

from 9 to 12: safe
processed 22 insns, stack depth 16

Attaching kprobe:vfs_read
Running...
^C

@[6169]: 1
@[28178]: 1
[...]
```

Значение `Program ID` можно использовать с `bpftool` для получения информации о состоянии ядра BPF, как было показано в главе 2. Далее следует байт-код BPF и имя подключенного зонда.

Как и в случае с параметром `-d`, подробный вывод полезен только разработчикам внутренних компонентов `bpfttrace`. Пользователям `bpfttrace` не нужно читать байт-код BPF.

5.18. ИТОГИ

`bpfttrace` — мощный трассировщик, предлагающий лаконичный и высокоуровневый язык программирования. Эта глава описывает его возможности, инструменты и примеры однострочных сценариев. Охватывает приемы программирования и содержит разделы с кратким описанием зондов, средств управления потоком выполнения, переменных и функций. Заканчивается она знакомством с внутренним устройством `bpfttrace` и возможностями отладки.

Следующие главы охватывают конкретные цели анализа и описывают инструменты ВСС и `bpfttrace`. Преимущество инструментов `bpfttrace` заключается в лаконичности их исходного кода, что позволило включить его в эту книгу.

Глава 6

ПРОЦЕССОРЫ

Процессоры (CPU) выполняют все программное обеспечение; именно они обычно являются отправной точкой для анализа производительности. Обнаружив, что производительность рабочей нагрузки ограничена возможностями CPU, вы сможете продолжить исследования, применив инструменты анализа нагрузки на процессор. Есть множество профилировщиков и метрик, которые покажут, как используется CPU. Тем не менее (что удивительно) есть ряд областей, где трассировка BPF еще больше поможет в анализе CPU.

Цели обучения:

- познакомиться с режимами работы CPU, особенностями поведения планировщика и кэшами процессора;
- понять, как с помощью BPF анализировать работу планировщика и нагрузку на CPU;
- изучить стратегию успешного анализа производительности CPU;
- решать проблемы, связанные с короткоживущими процессами, потребляющими значительную долю вычислительных ресурсов;
- обнаруживать и оценивать проблемы, связанные с задержкой в очереди выполнения;
- определять потребление CPU методом профилирования трассировок стека и подсчетом количества вызовов функций;
- определять причины захвата и освобождения CPU потоками;
- оценивать потребление CPU путем трассировки системных вызовов;
- исследовать потребление CPU обработчиками программных и аппаратных прерываний;
- применять однострочные сценарии `bpfttrace`, чтобы исследовать потребление CPU нестандартными методами.

В начале этой главы я познакомлю вас с основами, которые необходимы для анализа использования CPU, поведения планировщика и кэшей CPU. Я расскажу, на какие вопросы сможет ответить BPF, и дам общую стратегию исследований.

Чтобы не изобретать велосипед и направить повествование в правильное русло, сначала я покажу традиционные инструменты анализа нагрузки на CPU, а затем инструменты BPF, включая однострочники BPF. В конце главы дам несколько дополнительных упражнений.

6.1. ОСНОВЫ

В этом разделе рассматриваются основы работы CPU, возможности BPF и стратегия успешного анализа производительности CPU.

6.1.1. Основы работы процессоров

Режимы работы процессоров

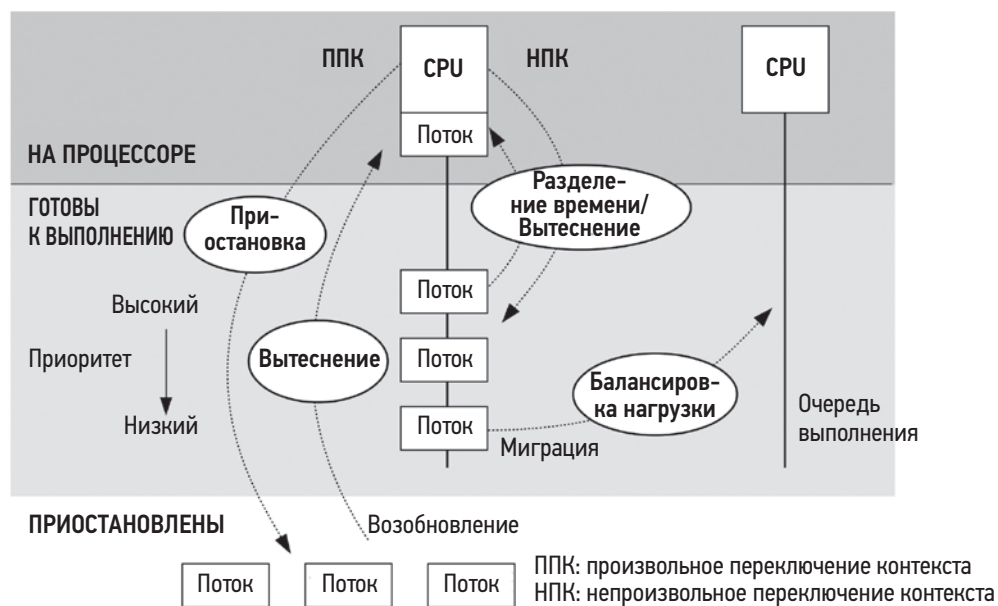
Процессоры и другие ресурсы управляются ядром, которое работает в специальном привилегированном состоянии, называемом *системным режимом*. Пользовательские приложения выполняются в пользовательском режиме и могут обращаться к ресурсам, только посылая запросы ядру. Эти запросы могут быть явными, например системные вызовы, или неявными — отказы страниц, вызванные высоким потреблением памяти. Ядро отслеживает количество времени, когда CPU простаивал, а также то, сколько времени потратил CPU на выполнение в пользовательском и системном режимах. Есть множество инструментов, позволяющих узнать время выполнения в пользовательском и системном режимах.

Обычно ядро работает только по требованию, когда происходит системный вызов или прерывание. Но есть исключения, например служебные потоки, которые действуют в фоновом режиме, потребляя ресурсы CPU. Пример — процедура балансировки страниц памяти в системах с неоднородным доступом к памяти (Non-Uniform Memory Access, NUMA), которые могут потреблять значительные вычислительные ресурсы без явного обращения со стороны пользовательских приложений. (Ее можно настроить или отключить.) Некоторые файловые системы также имеют фоновые процедуры, например, для периодической проверки контрольных сумм на целостность данных.

Планировщик

Ядро также отвечает за распределение вычислительных ресурсов между потребителями. Эту функцию в ядре выполняет планировщик. Основными потребителями являются потоки выполнения (их также называют задачами), которые принадлежат процессам или процедурам ядра. Еще одна категория потребителей CPU включает обработчики прерываний: прерывания могут быть программными (вызываются ПО) или аппаратными.

На рис. 6.1 изображен планировщик: показаны потоки выполнения, ожидающие доступа к CPU в очереди выполнения, и смена состояний этих потоков.



Здесь изображены потоки в трех состояниях: НА ПРОЦЕССОРЕ — потоки, выполняющиеся на CPU, ГОТОВЫ К ВЫПОЛНЕНИЮ — потоки, которые готовы к работе и ожидают своей очереди, и ПРИОСТАНОВЛЕННЫ — потоки, заблокированные в ожидании некоторого события. Сюда же относятся потоки, находящиеся в состоянии непрерываемого ожидания. Потоки, ожидающие в очереди выполнения, сортируются по значению приоритета, которое может устанавливаться ядром или пользовательскими процессами для увеличения производительности более важных задач. (Очереди выполнения — это то, как первоначально было реализовано планирование, этот термин и ментальная модель все еще используются для описания ожидающих задач. При этом планировщик CFS (Completely Fair Scheduler) в Linux в реальности использует красно-черное дерево будущего выполнения задач.)

В этой книге используется терминология, основанная на следующих состояниях потока: термин «на CPU» (on-CPU) обозначает состояние НА ПРОЦЕССОРЕ, а «вне CPU» (off-CPU) — все другие состояния, то есть когда поток не выполняется на CPU.

Потоки покидают CPU одним из двух способов: (1) намеренно, если блокируются при вводе/выводе, блокировке или спящем режиме; (2) принудительно, если превысили свое запланированное выделение процессорного времени и отменяются, чтобы другие потоки могли работать, либо если они вытеснены потоком с более высоким приоритетом. Когда процессор переключается с одного процесса или

потока на другой, он переключает адресные пространства и другие метаданные. Это называется переключением контекста¹.

На рис. 6.1 показана миграция потоков. Если поток находится в состоянии готовности к выполнению и ожидает в очереди, когда другой CPU простаивает, планировщик может перенести поток в очередь на выполнение неактивного CPU, чтобы запустить его пораньше. Но для оптимизации производительности планировщик старается избегать миграций, если стоимость миграции может превысить ее выгоды, предпочитая оставлять потоки на одном и том же CPU, кэш которого может все еще оставаться «горячим».

Кэши процессоров

На рис. 6.1 показано, как выглядят CPU (планировщик) с точки зрения ПО. На рис. 6.2, напротив, показано, как выглядят кэши CPU с точки зрения аппаратуры.

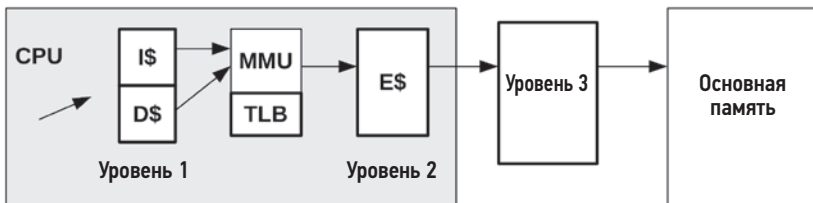


Рис. 6.2. Аппаратные кэши

В зависимости от модели и типа процессоры, как правило, имеют несколько уровней кэш-памяти. Объем этой памяти и задержка при работе с ней увеличиваются от уровня к уровню. Самый первый кэш — это кэш уровня 1. Он делится на отдельные кэши команд (IS) и данных (DS), имеющие небольшой объем (в несколько килобайт) и высокую скорость доступа (наносекунды). Конечный кэш (E\$) — кэш последнего уровня (Last-Level Cache, LLC) — имеет больший объем (несколько мегабайт) и действует намного медленнее. На процессоре с тремя уровнями кэшем последнего уровня является кэш уровня 3. Кэши уровней 1 и 2 обычно есть отдельно для каждого ядра CPU, а кэш уровня 3 — общий для всего кристалла. Блок управления памятью (Memory Management Unit, MMU), отвечающий за преобразование виртуальных адресов в физические, имеет свой кэш — буфер ассоциативной трансляции (Translation Lookaside Buffer, TLB).

Процессоры улучшались десятилетиями. Увеличивалась тактовая частота, добавлялись ядра и аппаратные потоки. Также увеличивалась пропускная способность памяти и уменьшались задержки доступа к ней, чему в немалой степени

¹ Кроме переключения контекста может также выполняться переключение режима: неблокирующие системные вызовы Linux могут (в зависимости от процессора) переключаться только между режимами пользователя и ядра.

способствовало добавление и увеличение размеров кэшей CPU. Но темп увеличения производительности памяти уступает темпу увеличения производительности процессоров. Сейчас производительность рабочих нагрузок в большей степени ограничивается производительностью памяти, а не ядер CPU.

Для дополнительного чтения

Цель этого краткого обзора — знакомство с основами, которые нужно знать, чтобы пользоваться инструментами. Более подробное обсуждение CPU с аппаратной и программной точек зрения вы найдете в главе 6 книги «Systems Performance»¹ [Gregg 13b].

6.1.2. Возможности BPF

Традиционные инструменты для анализа производительности предоставляют разные сведения об использовании процессора. Например, вы можете определить загрузку CPU по процессам, частоту переключения контекста и длины очередей выполнения. Эти традиционные инструменты описываются в общих чертах в следующем разделе.

Инструменты трассировки BPF дают массу дополнительной информации и отвечают на вопросы:

- Какие новые процессы были запущены? Каков их жизненный цикл?
- Почему время работы в режиме системы такое большое? Виноваты ли системные вызовы? Что они делают?
- Как долго потоки выполняются на процессоре после каждого возобновления?
- Как долго потоки ждут доступа к процессору в очереди на выполнение?
- Какой длины достигают очереди на выполнение?
- Насколько сбалансированы очереди на выполнение для разных процессоров?
- Почему потоки намеренно оставляют CPU? Как долго они пребывают в приостановленном состоянии после этого?
- Какие программные и аппаратные прерывания расходуют наибольший объем вычислительных ресурсов?
- Как часто процессоры бездействуют, когда в других очередях на выполнение есть ожидающие потоки?
- Каков процент попаданий в кэш последнего уровня в приложениях?

На эти вопросы можно ответить с помощью BPF, инструментируя точки трассировки планировщика и системных вызовов, зонды kprobes во внутренних функциях планировщика и uprobes в функциях приложения, а также счетчики производительности

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

РМС для определения периодов высокой и низкой нагрузки на CPU. Эти источники событий также можно смешивать: программа BPF может использовать uprobes для извлечения контекста приложения, а затем связывать его с инструментированными событиями РМС. Такая программа может также определять процент попаданий в кэш последнего уровня, например, по запросу приложения.

Метрики, доступные в BPF, могут извлекаться для каждого события в отдельности или поставляться в виде сводной статистики, а их распределения — в виде гистограмм. Также можно извлекать трассировки стека, чтобы по ним устанавливать причины событий. Все эти действия оптимизированы с использованием карт BPF в ядре и выходных буферов.

Источники событий

В табл. 6.1 перечислены источники событий для анализа использования CPU.

Таблица 6.1. Источники событий для получения информации о потреблении CPU

Типы событий	Источники событий
Функции ядра	kprobes, kretprobes
Пользовательские функции	uprobes, uretprobes
Системные вызовы	точки трассировки системных вызовов
Программные прерывания	точки трассировки irq:softirq*
Аппаратные прерывания	точки трассировки irq:irq_handler*
События очередей заданий	точки трассировки очередей заданий (см. главу 14)
Выборка по времени	счетчики производительности РМС или выборка метрик по времени
События питания CPU	точки трассировки системы питания
Циклы CPU	счетчики производительности РМС

Оверхед

При трассировке событий планировщика эффективность обретает особую важность, так как некоторые события планировщика, например переключение контекста, могут происходить миллионы раз в секунду. Даже при том, что программы BPF могут быть очень короткими и быстрыми (выполняться за микросекунды), их запуск для каждого переключения контекста может привести к накоплению существенных оверхедов. В некоторых случаях трассировка планировщика увеличивает нагрузку на систему более чем на 10%. Если бы не оптимизированная реализация BPF, оверхед мог бы оказаться непомерно высоким.

Трассировка планировщика с помощью BPF применяется для краткосрочного и специализированного анализа, при этом нужно понимать, что она сопряжена с оверхедом. Оверхед можно количественно оценить посредством тестирования

и экспериментов, например, проверяя стабильность загрузки CPU, когда инструмент BPF используется и когда не используется.

Инструменты трассировки CPU могут не иметь существенного оверхеда, если не будут учитывать частые события планировщика. Инструментация редких событий, например запуска процессов и миграции потоков (происходящих не чаще чем тысячу раз в секунду), порождает незначительный оверхед. Профилирование (выборка по времени) тоже позволяет ограничить оверхед уменьшением частоты выборки до незначительных пропорций.

6.1.3. Стратегия

Новичкам в сфере анализа производительности CPU часто трудно понять, с чего начать — с какой цели и с какого инструмента. Специально для них я приведу общую стратегию:

1. Прежде чем тратить время на инструменты анализа, убедитесь, что рабочая нагрузка на CPU запущена. Проверьте загрузку CPU системы (например, с помощью `mpstat(1)`) и убедитесь, что все CPU включены (иногда по какой-то причине некоторые могут быть отключены).
2. Убедитесь, что рабочая нагрузка имеет вычислительный характер:
 - а) определите наибольшую загрузку CPU в системе или на одном процессоре (например, с помощью `mpstat(1)`);
 - б) определите наибольшую задержку в очереди на выполнение (например, с помощью `runqlat(1)` из ВСС). Программные ограничения, например, в контейнерах, могут искусственно ограничивать доступ процессов к CPU, поэтому производительность вычислительного приложения может быть низкой, даже когда система практически простаивает. Этот нелогичный сценарий можно опознать, изучив задержку в очереди на выполнение.
3. Оцените загрузку CPU в процентах для всей системы с разбивкой по процессам, режимам CPU и отдельным процессорам. Это можно сделать с помощью традиционных инструментов (например, `mpstat(1)`, `top(1)`). Ищите процессы, режимы или процессоры с самой высокой нагрузкой:
 - а) если время выполнения в режиме системы велико, подсчитайте число системных вызовов по их типам и процессам, а также проанализируйте аргументы (например, с помощью `perf(1)`, однострочников `bpftrace` и `sysstat(8)` из ВСС), которые могут стать причиной неэффективного выполнения;
4. Используйте профилировщик для выборки трассировок стека, которые затем можно визуализировать в виде флейм-графика. Эти графики помогут увидеть многие проблемы с процессором.
5. Выявив с помощью профилировщика основных потребителей CPU, подумайте, можно ли создать собственные инструменты для получения подробной

информации. Профилировщики позволяют выявить функции, оказывающие наибольшую нагрузку, но не дают информации об их аргументах и объектах, с которыми работают. А она может оказаться очень ценной для понимания причин значительного потребления процессора. Например:

- а) в пространстве ядра: если файловая система потребляет большой объем вычислительных ресурсов, вызывая `stat()` для файлов, желательно определить имена файлов. (Это можно выяснить, например, с помощью `statsnoop(8)` из BCC или с помощью точек трассировки или `kprobes` и инструментов BPF.);
 - б) в пространстве пользователя: если приложение тратит много времени на обработку запросов, нужно определить, что это за запросы. (Если инструмент для конкретного приложения недоступен, его можно разработать с использованием зондов `USDT` или `uprobes` и инструментов BPF).
6. Измерьте время на обработку аппаратных прерываний, которое может не отображаться профилировщиками, осуществляющими выборку по времени (например, с помощью `hardirqs(1)` из BCC).
 7. Просмотрите перечень инструментов BPF в разделе 6.3 (далее в этой главе) и попробуйте их применить.
 8. Измерьте количество инструкций CPU, выполняемых за один такт (IPC), используя счетчики РМС, чтобы получить представление о том, сколько процессоров простаивает (например, с помощью `perf(1)`). Для этого используйте разные счетчики производительности РМС, позволяющие определить низкий коэффициент попадания в кэш (например, `llcstat` из BCC), пропуски тактов из-за перегрева и т. д.

В следующих разделах я подробно расскажу об инструментах, вовлеченных в этот процесс.

6.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Традиционные инструменты (см. табл. 6.2) позволяют получить метрики загрузки CPU для каждого процесса (потока) и для каждого процессора, частоту принудительного и намеренного переключения контекста, среднюю длину очереди на выполнение и общее время ожидания в этих очередях. Профилировщики помогут увидеть и количественно оценить параметры работающего ПО, а инструменты на основе счетчиков производительности РМС могут показать, насколько хорошо работают процессоры на уровне тактов.

Помимо решения проблем, традиционные инструменты также помогают определить необходимость последующего использования инструментов BPF. Здесь эти инструменты классифицированы по источникам событий и типам измерений: статистика ядра, статистика оборудования и трассировка событий.

Таблица 6.2. Традиционные инструменты

Инструмент	Тип	Описание
uptime	Статистика ядра	Показывает среднюю загрузку и общее время работы системы
top	Статистика ядра	Показывает потребление CPU каждым процессом и время выполнения в разных режимах
mpstat	Статистика ядра	Показывает время работы каждого CPU в каждом режиме
perf	Статистика ядра, статистика оборудования, трассировка событий	Профилирует (выполняя выборки через заданные интервалы времени) трассировки стека, собирает статистику по событиям и осуществляет трассировку счетчиков производительности РМС, точек трассировки, зондов USDT, kprobes и uprobes
Ftrace	Статистика ядра, трассировка событий	Собирает статистику вызовов функций ядра и осуществляет трассировку зондов kprobes и uprobes

В следующих разделах я в общих чертах опишу основные функции этих инструментов. Подробную информацию ищите на соответствующих страницах справочного руководства и других ресурсах, в том числе в книге «Systems Performance»¹ [Gregg 13b].

6.2.1. Статистика ядра

Инструменты, относящиеся к этой категории, получают информацию из источников в ядре, в том числе и из файловой системы */proc*. Их главное преимущество в том, что метрики уже подсчитываются ядром, поэтому использование сопряжено лишь с незначительным оверхедом. Кроме того, для доступа к ним обычно не требуются привилегии root.

Средняя нагрузка

`uptime(1)` — одна из немногих команд, которая выводит информацию о средней загрузке системы:

```
$ uptime
00:34:10 up 6:29, 1 user, load average: 20.29, 18.90, 18.70
```

Три числа в конце описывают среднюю нагрузку за последние 1, 5 и 15 минут. Сравнивая эти числа, можно определить, увеличивалась или уменьшалась нагрузка за последние 15 минут или оставалась постоянной. Результаты в примере выше были получены в производственном облаке с 48 процессорами и показывают, что средняя нагрузка за последнюю минуту (20.29) немного увеличилась по сравнению со средней нагрузкой за 15 минут (18.70).

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

Средние значения нагрузки — это не просто средние, а экспоненциально затухающие скользящие суммы, отражающие нагрузку за последние 1, 5 и 15 минут. Отражаемые ими метрики обобщают потребность системы в вычислительных ресурсах: число задач в состоянии готовности к выполнению, а также число задач, находящихся в состоянии непрерываемого ожидания [72]. Чтобы узнать среднюю загрузку процессоров, эти числа нужно разделить на количество процессоров. Получившиеся результаты покажут, работает ли система в состоянии насыщения процессоров, которое характеризуется превышением полученного результата числа 1.0. Но включение в подсчет средних значений загрузки задач, находящихся в состоянии непрерываемого ожидания (то есть заблокированных в операциях дискового ввода/вывода и в блокировках), ставит под сомнение эту интерпретацию, поэтому они полезны только для изучения тенденций во времени. Чтобы определить истинную загрузку CPU, без учета ожидающих задач, используйте другие инструменты, например `offcputime(8)`, основанные на BPF. Дополнительную информацию об инструменте `offcputime(8)` ищите в разделе 6.3.9 и в главе 14.

top

Инструмент `top(1)` выводит информацию о потреблении CPU процессами в виде таблицы вместе с информацией о системе в целом:

```
$ top
top - 00:35:49 up 6:31, 1 user, load average: 21.35, 19.96, 19.12
Tasks: 514 total, 1 running, 288 sleeping, 0 stopped, 0 zombie
%Cpu(s): 33.2 us, 1.4 sy, 0.0 ni, 64.9 id, 0.0 wa, 0.0 hi, 0.4 si, 0.0 st
KiB Mem : 19382528+total, 1099228 free, 18422233+used, 8503712 buff/cache
KiB Swap: 0 total, 0 free, 0 used. 7984072 avail Mem

  PID USER      PR  NI   VIRT    RES    SHR  S  %CPU  %MEM     TIME+ COMMAND
 3606 www       20   0 0.197t 0.170t 38776 S 1681 94.2   7186:36 java
 5737 snmp      20   0 22712 6676 4256 S  0.7  0.0    0:57.96 snmp-pass
 403 root       20   0      0      0      0 I  0.3  0.0    0:00.17 kworker/41:1
 983 root       20   0   9916   128      0 S  0.3  0.0    1:29.95 rngd
29535 bgregg     20   0 41020 4224 3072 R  0.3  0.0    0:00.11 top
   1 root       20   0 225308 8988 6656 S  0.0  0.0    0:03.09 systemd
   2 root       20   0      0      0      0 S  0.0  0.0    0:00.01 kthreadd
[...]
```

Эти данные были получены на экземпляре в продакшен и показывают, что основным потребителем CPU является только один процесс: процесс `java`, который потребляет в общей сложности 1681% CPU, в сумме по всем CPU. То есть в этой системе с 48 процессорами процесс `java` потребляет 35% общей вычислительной мощности. Это согласуется с общесистемной средней загрузкой, равной 34.6% (сумма 33.2% загрузки в пространстве пользователя и 1.4% в пространстве ядра).

`top(1)` позволяет быстро выявить проблемы с загрузкой процессоров, обусловленной неожиданным поведением некоторого процесса. В разработке ПО часто встречается ошибка, когда поток выполнения застревает в бесконечном цикле, и такие ошибки легко обнаруживаются с помощью команды `top(1)`, в выводе которой

зациклившийся процесс отображается как потребляющий 100% CPU. Последующий анализ с применением профилировщиков и инструментов BPF поможет подтвердить, действительно ли процесс зациклился, а не занят сложными вычислениями.

По умолчанию `top(1)` периодически обновляет вывод, чтобы экран мог работать как дашборд в режиме реального времени. Это представляет определенные сложности: проблемы могут появляться и исчезать, прежде чем вы успеете сделать скриншот. Иногда наличие вывода инструментов и скриншотов помогает в коллективном обсуждении и поиске проблем. В подобных случаях можно применить `pidstat(1)`, отображающий текущее потребление CPU процессами. Также информацию о потреблении CPU процессами можно фиксировать с помощью систем мониторинга, если они используются.

Есть и другие варианты утилиты `top(1)`, такие как `htop(1)`, которые имеют больше параметров настройки. К сожалению, многие варианты `top(1)` больше внимания уделяют улучшению визуального представления информации, а не показателям производительности, в результате они выглядят красивее, но не помогают пролить больше света на проблемы, чем исходная утилита `top(1)`. В числе исключений можно назвать `tiptop(1)`, черпающую информацию из счетчиков производительности RMC; `atop(1)`, использующую события процессов для отображения информации о короткоживущих процессах; а также `biotop(8)` и `tcptop(8)`, использующие BPF (и созданные мной).

mpstat(1)

`mpstat(1)` можно применять для исследования метрик отдельных процессоров:

```
$ mpstat -P ALL 1
Linux 4.15.0-1027-aws (api-...)    01/19/2019    _x86_64_    (48 CPU)
```

			%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
12:47:47	AM	CPU										
12:47:48	AM	all	35.25	0.00	1.47	0.00	0.00	0.46	0.00	0.00	0.00	62.82
12:47:48	AM	0	44.55	0.00	1.98	0.00	0.00	0.99	0.00	0.00	0.00	52.48
12:47:48	AM	1	33.66	0.00	1.98	0.00	0.00	0.00	0.00	0.00	0.00	64.36
12:47:48	AM	2	30.21	0.00	2.08	0.00	0.00	0.00	0.00	0.00	0.00	67.71
12:47:48	AM	3	31.63	0.00	1.02	0.00	0.00	0.00	0.00	0.00	0.00	67.35
12:47:48	AM	4	26.21	0.00	0.00	0.00	0.00	0.97	0.00	0.00	0.00	72.82
12:47:48	AM	5	68.93	0.00	1.94	0.00	0.00	3.88	0.00	0.00	0.00	25.24
12:47:48	AM	6	26.26	0.00	3.03	0.00	0.00	0.00	0.00	0.00	0.00	70.71
12:47:48	AM	7	32.67	0.00	1.98	0.00	0.00	1.98	0.00	0.00	0.00	63.37
[...]												

Здесь приводится неполный вывод, потому что в этой системе с 48 процессорами инструмент выводит 48 строк каждую секунду: по одной строке на каждый процессор. Эти данные можно использовать для выявления проблем с балансировкой, когда одни процессоры имеют высокую загрузку, а другие простаивают. Дисбаланс в загрузке процессоров может возникнуть по многим причинам, таким как неправильно настроенный размер пула потоков в приложении, например, слишком маленький, чтобы задействовать все процессоры; наличие программных ограничений, из-за

которых процесс или контейнер не может использовать все процессоры; а также программные ошибки.

Время работы каждого процессора разбивается на множество режимов, включая время, израсходованное на обработку аппаратных (%irq) и программных (%soft) прерываний. Эти метрики можно дополнительно исследовать с помощью инструментов BPF — `hardirqs(8)` и `softirqs(8)`.

6.2.2. Статистика оборудования

Аппаратное обеспечение тоже служит источником полезной информации, особенно счетчики контроля производительности (Performance Monitoring Counters, PMC), доступные на процессорах. Некоторые из этих счетчиков были представлены в главе 2.

perf(1)

`perf(1)` — это многоцелевой инструмент, способный получать информацию из множества разных источников. Впервые он появился в Linux в 2.6.31 (2009) и с тех пор считается стандартным профилировщиком Linux. Его исходный код находится в каталоге `tools/perf` в дереве с исходными текстами Linux. В свое время я опубликовал подробное руководство по использованию `perf` [73]. Среди его многочисленных мощных возможностей — использование PMC в режиме подсчета:

```
$ perf stat -d gzip file1
```

```
Performance counter stats for 'gzip file1':
```

3952.239208	task-clock (msec)	#	0.999 CPUs utilized	
6	context-switches	#	0.002 K/sec	
0	cpu-migrations	#	0.000 K/sec	
127	page-faults	#	0.032 K/sec	
14,863,135,172	cycles	#	3.761 GHz (62.35%)	
18,320,918,801	instructions	#	1.23 insn per cycle	(74.90%)
3,876,390,410	branches	#	980.809 M/sec	(74.90%)
135,062,519	branch-misses	#	3.48% of all branches	(74.97%)
3,725,936,639	L1-dcache-loads	#	942.741 M/sec	(75.09%)
657,864,906	L1-dcache-load-misses	#	17.66% of all L1-dcache hits	(75.16%)
50,906,146	LLC-loads	#	12.880 M/sec	(50.01%)
1,411,636	LLC-load-misses	#	2.77% of all LL-cache hits	(49.87%)

Команда `perf stat` подсчитывает события, указанные в параметре `-e`. Если события не указаны, по умолчанию используется базовый набор PMC, а если команда вызвана с параметром `-d`, как здесь, то расширенный набор. Вывод и порядок использования могут немного отличаться, в зависимости от версии Linux и типа процессора. Этот пример получен в Linux 4.15.

С помощью `perf` можно получить полный список счетчиков PMC, доступных в текущей системе для данного типа процессора и версии `perf`:

```
$ perf list
[...]
  mem_load_retired.l3_hit
    [Retired load instructions with L3 cache hits as data sources Supports address
when precise (Precise event)]
  mem_load_retired.l3_miss
    [Retired load instructions missed L3 cache as data sources Supports address
when precise (Precise event)]
[...]
```

В этом списке отображаются имена событий, которые можно передать как аргументы в параметре `-e`. Например, подсчитать эти события на всех процессорах (здесь для этого передается параметр `-a`, который лишь недавно стал подразумеваться по умолчанию) и запустить вывод с интервалом 1000 миллисекунд (`-I 1000`) можно так:

```
# perf stat -e mem_load_retired.l3_hit -e mem_load_retired.l3_miss -a -I 1000
#           time              counts unit events
   1.001228842             675,693  mem_load_retired.l3_hit
   1.001228842             868,728  mem_load_retired.l3_miss
   2.002185329             746,869  mem_load_retired.l3_hit
   2.002185329             965,421  mem_load_retired.l3_miss
   3.002952548           1,723,796  mem_load_retired.l3_hit
[...]
```

Здесь выводится количество указанных событий, возникших в течение секунды во всей системе в целом.

Сейчас доступны сотни счетчиков РМС, задокументированных в руководствах производителей процессоров [Intel 16], [AMD 10]. С помощью счетчиков РМС и регистров, зависящих от модели (Model-Specific Registers, MSR), можно выяснить, как работают внутренние компоненты CPU, текущую тактовую частоту, температуру и энергопотребление, пропускную способность внутренних связей в CPU и шины памяти и многое другое.

tlbstat

Чтобы показать, как использовать РМС, я создал инструмент `tlbstat`, читающий счетчики РМС, связанные с буфером ассоциативной трансляции (Translation Lookaside Buffer, TLB). Цель состояла в том, чтобы проанализировать влияние на производительность исправлений в механизме изоляции таблицы страниц ядра (Kernel Page Table Isolation, KPTI), которые устраняют уязвимость Meltdown [74], [75]:

```
# tlbstat -C0 1
K_CYCLES  K_INSTR   IPC  DTLB_WALKS  ITLB_WALKS  K_DTLBCYC  K_ITLBCYC  DTLB%  ITLB%
2875793   276051    0.10 89709496   65862302   787913    650834    27.40  22.63
2860557   273767    0.10 88829158   65213248   780301    644292    27.28  22.52
2885138   276533    0.10 89683045   65813992   787391    650494    27.29  22.55
2532843   243104    0.10 79055465   58023221   693910    573168    27.40  22.63
[...]
```

tlbstat выводит следующие столбцы:

- **K_CYCLES**: тысячи тактов CPU;
- **K_INSTR**: тысячи инструкций CPU;
- **IPC**: инструкций на такт;
- **DTLB_WALKS**: число обходов TLB данных;
- **ITLB_WALKS**: число обходов TLB инструкций;
- **K_DTLBCYC**: тысячи тактов, когда хотя бы один обработчик отсутствия страницы (Page-Miss Handler, PMH) был активен во время обхода TLB данных;
- **K_ITLBCYC**: тысячи тактов, когда хотя бы один обработчик PMH был активен во время обхода TLB инструкций;
- **DTLB%**: доля активных тактов TLB данных в общем числе тактов;
- **ITLB%**: доля активных тактов TLB инструкций в общем числе тактов.

Результаты, показанные выше, были получены во время нагрузочного тестирования, когда издержки KPTI были наибольшими: они показывают 27% тактов CPU в DTLB и 22% в ITLB. То есть практически половина общесистемных вычислительных ресурсов была использована блоком управления памятью, обслуживающим преобразование виртуальных адресов в физические. Если tlbstat покажет вам аналогичные значения при обычных производственных рабочих нагрузках, сосредоточьтесь на настройке TLB.

6.2.3. Выборка характеристик работы оборудования

perf(1) может использовать счетчики РМС в другом режиме и посылать прерывание ядру при каждом срабатывании счетчика, чтобы оно могло сохранить состояние события. Например, команда ниже записывает трассировку стека (-g) для каждого события промаха кэша L3 (-e ...) на всех процессорах (-a) в течение 10 секунд (sleep 10, фиктивная команда, используемая для установки длительности):

```
# perf record -e mem_load_retired.l3_miss -c 50000 -a -g -- sleep 10
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 3.355 MB perf.data (342 samples) ]
```

Выборки можно обобщить с помощью perf report или вывести с помощью perf list:

```
# perf list
kworker/u17:4 11563 [007] 2707575.286552: mem_load_retired.l3_miss:
    7ffffba5d8c52 move_freepages_block ([kernel.kallsyms])
    7ffffba5d8e02 steal_suitable_fallback ([kernel.kallsyms])
    7ffffba5da4a8 get_page_from_freelist ([kernel.kallsyms])
    7ffffba5dc3fb __alloc_pages_nodemask ([kernel.kallsyms])
    7ffffba63a8ea alloc_pages_current ([kernel.kallsyms])
    7ffffc01faa5b crypt_page_alloc ([kernel.kallsyms])
    7ffffba5d3781 mempool_alloc ([kernel.kallsyms])
    7ffffc01fd870 kcryptd_crypt ([kernel.kallsyms])
```

```

7ffffba4a983e process_one_work ([kernel.kallsyms])
7ffffba4a9aa2 worker_thread ([kernel.kallsyms])
7ffffba4b0661 kthread ([kernel.kallsyms])
7ffffbae02205 ret_from_fork ([kernel.kallsyms])

```

[...]

Здесь показана единственная выборка трассировки стека. Функции перечислены сверху вниз — от дочерних к родительским. В этом примере — последовательность вызовов функций ядра, приведшая к событию промаха кэша L3.

Обратите внимание, что для минимизации проблем с обработкой прерываний предпочтительнее использовать счетчики РМС, поддерживающие точную выборку по событиям (Precise Event-Based Sampling, PEBS).

В процессе выборки аппаратных счетчиков РМС также можно запускать программы BPF. Например, вместо копирования всех выбранных трассировок стека в пространство пользователя через буфер `perf buffer` можно более эффективно подсчитать их частоту в пространстве ядра с помощью BPF.

6.2.4. Выборка по времени

Многие профилировщики поддерживают выборку по таймеру (выборка указателя инструкций или трассировок стека через определенный интервал времени). Такие профилировщики обеспечивают приблизительный и недорогой способ сбора информации о том, какое ПО потребляет ресурсы процессора. Есть разные типы профилировщиков, одни работают только в пространстве пользователя, а другие способны действовать в пространстве ядра. Профилировщики, действующие в пространстве ядра, обычно предпочтительнее, потому что могут захватывать трассировки стеков как в ядре, так и в пользовательском приложении и обеспечивать более полную картину.

perf

`perf(1)` — это профилировщик ядра, поддерживающий выборку по времени с помощью программных событий или счетчиков РМС: по умолчанию используется наиболее точный метод. В следующем примере в течение 30 секунд выполняется захват стеков всех процессоров со скоростью 99 Гц (выборка в секунду на процессор):

```

# perf record -F 99 -a -g -- sleep 30
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.661 MB perf.data (2890 samples) ]

```

Частота 99 Гц (вместо 100) выбрана специально, чтобы избежать совпадения с выборкой в других программах, что могло бы исказить результаты. (Более подробно эта проблема объясняется в главе 18.) Кроме того, частота, близкая к 100 (а не к 10 или 10 000) выбрана с целью соблюсти баланс между детальностью и оверхедом: при слишком низкой частоте вы не получите достаточного количества выборок, чтобы

увидеть полную картину выполнения, включающую короткие и длинные пути. При слишком высокой частоте оверхед на выборку данных будет искажать метрики производительности и получаемые результаты.

При выполнении эта команда `perf(1)` записывает выборки в файл `perf.data`: для оптимизации она использует буфер в пространстве ядра и выбирает наиболее подходящее число операций записи в файловую систему. Как сообщает вывод, для записи этих данных команде было достаточно возобновить выполнение только один раз.

Полученные результаты можно обобщить с помощью `perf report`, а с помощью `perf script` вывести каждую выборку. Например:

```
# perf report -n --stdio
[...]
```

#	Children	Self	Samples	Command	Shared Object	Symbol
#						
#						
	99.41%	0.08%	2	iperf	libpthread-2.27.so	[.] __libc_write
	--99.33%	--				__libc_write
	--98.51%	--				entry_SYSCALL_64_after_hwframe
	--98.38%	--				do_syscall_64
	--98.29%	--				sys_write
	--97.78%	--				vfs_write

```
[...]
```

В отчете `perf report` выводит дерево вызовов функций от родительских к дочерним (порядок можно изменить на обратный, который по умолчанию использовался в более ранних версиях). К сожалению, в данном случае мало что можно сказать об этой выборке. Более того, полный вывод состоит из 6000 строк. Полный вывод команды `perf script` со всеми событиями занимает больше 60 000 строк. В высоконагруженных системах объем этих результатов легко может увеличиться в 10 раз и более. В таких ситуациях полученные трассировки стека лучше визуализировать в виде флейм-графика.

Флейм-графики использования процессора

Флейм-графики, с которыми мы познакомились в главе 2, позволяют представить трассировки стека в более наглядной форме. Они прекрасно подходят для профилирования CPU и широко используются для этой цели.

На рис. 6.3 показан флейм-график, который отображает результаты профилирования, полученные в предыдущем разделе.



Рис. 6.3. Флейм-график использования CPU

Представив полученные данные в виде флейм-графика, мы увидим, что основным потребителем CPU является процесс `iperf`, а также в каких функциях он проводит основное время. На графике четко выделяются две «горячие» функции: `copy_user_enhanced_fast_string()` и `move_freepages_block()`, которые образуют два плато и вызываются через `sock_sendmsg()`. Справа находится башня, соответствующая пути получения ответа TCP, — это `iperf` тестирует петлевой (локальный) интерфейс.

Ниже приведены шаги для создания флейм-графика с использованием `perf(1)` для выборки трассировок стеков с частотой 49 Гц в течение 30 секунд, а также моя реализация построения флейм-графиков:

```
# git clone https://github.com/brendangregg/FlameGraph
# cd FlameGraph
# perf record -F 49 -ag -- sleep 30
# perf script --header | ./stackcollapse-perf.pl | ./flamegraph.pl > flame1.svg
```

Программа `stackcollapse-perf.pl` преобразует вывод команды `perf script` в стандартный формат и передает результат программе `flamegraph.pl`. В репозитории `FlameGraph` есть несколько инструментов преобразования для других профилировщиков. Программа `flamegraph.pl` создает флейм-график в виде файла SVG со встроенным сценарием на JavaScript для поддержки интерактивности при отображении в браузере. Программа `flamegraph.pl` поддерживает множество параметров настройки. Чтобы получить подробную информацию о них, выполните команду `flamegraph.pl --help`.

Советую сохранить вывод `perf script --header` для последующего анализа. В Netflix была создана более совершенная реализация флейм-графика с использованием `d3`, а также дополнительный инструмент `FlameScope`, который умеет принимать вывод `perf script` и визуализировать профили в виде тепловых карт с субсекундным смещением, где можно выбирать временные диапазоны для просмотра флейм-графика [76], [77].

Внутреннее устройство

Когда `perf(1)` производит выборку по времени, он пытается использовать аппаратные события переполнения тактов CPU на основе РМС, которые вызывают немаскируемое прерывание (Non-Maskable Interrupt, NMI). Обработчик этого прерывания, собственно, и производит выборку. Но в облачной среде счетчики РМС часто отключаются во многих типах экземпляров. Это можно заметить в выводе `dmesg(1)`:

```
# dmesg | grep PMU
[ 2.827349] Performance Events: unsupported p6 CPU model 85 no PMU driver,
software events only.
```

В таких системах `perf(1)` возвращается к программному прерыванию на основе таймера высокого разрешения `hrtimer`. Увидеть это можно, запустив `perf` с параметром `-v`:

```
# perf record -F 99 -a -v
Warning:
The cycles event is not supported, trying to fall back to cpu-clock-ticks
[...]
```

Обычно этого программного прерывания вполне достаточно, но имейте в виду, что в коде ядра есть такие пути выполнения, которые не могут быть прерваны этим

прерыванием: пути с отключенными IRQ (включая некоторые пути в планировщике и обработчиках аппаратных событий). Образцов этих путей кода не будет в полученном профиле.

Дополнительная информация о работе счетчиков РМС приводится в разделе 2.12 в главе 2.

6.2.5. Получение статистик и трассировка событий

Инструменты трассировки событий также можно использовать для анализа потребления CPU. В Linux для этой цели традиционно используются `perf(1)` и `Ftrace`. Эти инструменты могут не только отслеживать события и сохранять информацию о каждом из них, но и подсчитывать события в контексте ядра.

perf

`perf(1)` может инструментировать точки трассировки, `kprobes`, `uprobes` и (с недавнего времени) зонды `USDT`. Иногда трассировка этих событий помогает получить дополнительный логический контекст причин потребления CPU.

Например, представьте, что система страдает от проблемы высокой загрузки CPU, но с помощью `top(1)` не удастся выявить процессы, ответственные за это. Проблема может быть обусловлена выполнением короткоживущих процессов. Для проверки этой гипотезы подсчитаем число вызовов в точке трассировки `sched_process_exec` в масштабе всей системы с помощью `perf script`, чтобы увидеть, как часто происходят обращения к системному вызову `exec()`:

```
# perf stat -e sched:sched_process_exec -I 1000
#           time                counts unit events
    1.000258841                169    sched:sched_process_exec
    2.000550707                168    sched:sched_process_exec
    3.000676643                167    sched:sched_process_exec
    4.000880905                167    sched:sched_process_exec
[...]
```

Как показывает этот вывод, `exec` вызывается больше 160 раз в секунду. Теперь воспользуемся `perf record`, чтобы зафиксировать каждое событие, а затем выведем их с помощью `perf script`¹:

```
# perf record -e sched:sched_process_exec -a
^C[ perf record: Woken up 1 times to write data ]
```

¹ Для тех, кому интересно, почему я не использовал `strace(1)`: текущая реализация `strace(1)` использует точки останова, которые могут значительно замедлить целевую систему (более чем в 100 раз), что опасно для производственной среды. Сейчас разрабатывается несколько альтернатив, включая подкоманду `perf trace`, в том числе основанных на использовании BPF. Кроме того, в этом примере мы трассируем системный вызов `exec()` в масштабе всей системы, что не под силу нынешней реализации `strace(1)`.

```
[ perf record: Captured and wrote 3.464 MB perf.data (95 samples) ]
# perf script
  make 28767 [007] 712132.535241: sched:sched_process_exec: filename=/usr/bin/make
pid=28767 old_pid=28767
  sh 28768 [004] 712132.537036: sched:sched_process_exec: filename=/bin/sh
pid=28768 old_pid=28768
  cmake 28769 [007] 712132.538138: sched:sched_process_exec: filename=/usr/bin/cmake
pid=28769 old_pid=28769
  make 28770 [001] 712132.548034: sched:sched_process_exec: filename=/usr/bin/make
pid=28770 old_pid=28770
  sh 28771 [004] 712132.550399: sched:sched_process_exec: filename=/bin/sh
pid=28771 old_pid=28771
[...]
```

Вывод показывает, что у выполняемых процессов были имена, включая `make`, `sh` и `cmake`, это заставляет меня подозревать, что виновником стала сборка ПО. Короткоживущие процессы — настолько распространенная проблема, что для их выявления был создан специальный инструмент BPF: `execsnoop(8)`. Он выводит такую информацию, как имя процесса, PID, номер процессора, отметка времени (в секундах), имя события и его аргументы.

`perf(1)` имеет специальную подкоманду `perf sched` для анализа работы планировщика CPU. В ней для анализа поведения планировщика используется подход «сначала получить полный объем информации, а потом обработать» и реализованы разнообразные отчеты, отображающие время ожидания, среднюю и максимальную задержку планировщика, а также ASCII-график выполнения потоков на CPU и миграции. Например:

```
# perf sched record -- sleep 1
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 1.886 MB perf.data (13502 samples) ]
# perf sched timehist
Samples do not have callchains.
      time      cpu  task name          wait time sch delay  run time
              [tid/pid]      (msec)      (msec)      (msec)
-----
[...]
```

991963.885740	[0001]	:17008[17008]	25.613	0.000	0.057
991963.886009	[0001]	sleep[16999]	1000.104	0.006	0.269
991963.886018	[0005]	cc1[17083]	19.908	0.000	9.948

```
[...]
```

Команда выводит описание каждого события переключения контекста планировщика в виде строки с временем ожидания (`wait time`), задержкой планировщика (`sch delay`) и временем выполнения на CPU (`run time`), все времена выводятся в миллисекундах. В этом примере видно команду `sleep(1)`, которая оставалась в состоянии ожидания в течение 1 секунды, и процесс `cc1`, который выполнялся 9,9 миллисекунды и ожидал своей очереди 19,9 миллисекунды.

Подкоманда `perf sched` помогает в решении самых разных проблем с планировщиком, включая проблемы с его реализацией в ядре (планировщик ядра — это

сложный код, учитывающий в своей работе множество требований). Но стиль «дамп и постобработка» обходится дорого: в этом примере запись событий планировщика производилась в течение 1 секунды в системе с восемью процессорами, и в результате получился файл `perf.data` размером 1,9 Мбайт. В более крупной и нагруженной системе может получиться и файл размером в несколько сотен мегабайт, что вызовет проблемы из-за необходимости расходовать процессорное время на создание этого файла и пропускную способность ввода/вывода для его записи на диск.

Поскольку количество событий планировщика весьма велико, для их анализа вывод `perf(1)` часто преобразуется в визуальное представление. Для этой цели в `perf(1)` есть своя подкоманда `timechart`.

Везде, где это возможно, я советую вместо `perf sched` использовать инфраструктуру BPF, которая может формировать сводки в пространстве ядра, отвечающие на аналогичные вопросы, и выводить результаты (например, с помощью инструментов `runqlat(8)` и `runqlen(8)`, описанных в разделах 6.3.3 и 6.3.4).

Ftrace

Ftrace — это набор различных средств трассировки, разработанный Стивеном Ростедтом и включенный в ядро Linux в версии 2.6.27 (2008). Как и `perf(1)`, этот набор инструментов можно использовать для изучения контекста потребления CPU через точки трассировки и другие события.

Например, моя коллекция `perf-tools` [78] широко использует Ftrace для инструментации и включает такие инструменты, как `funccount(8)` для подсчета числа вызовов функций. В примере ниже подсчитываются обращения к файловой системе `ext4` путем сопоставления с именами функций, начинающимися с «ext»:

```
# perf-tools/bin/funccount 'ext*'
Tracing "ext*"... Ctrl-C to end.
^C
FUNC                                COUNT
[...]
```

<code>ext4_do_update_inode</code>	523
<code>ext4_inode_csum.isra.56</code>	523
<code>ext4_inode_csum_set</code>	523
<code>ext4_mark_iloc_dirty</code>	523
<code>ext4_reserve_inode_write</code>	523
<code>ext4_inode_table</code>	551
<code>ext4_get_group_desc</code>	564
<code>ext4_nonda_switch</code>	586
<code>ext4_bio_write_page</code>	604
<code>ext4_journal_check_start</code>	1001
<code>ext4_es_can_be_merged</code>	1111
<code>ext4_file_getattr</code>	7159
<code>ext4_getattr</code>	7285

Здесь оставлены только наиболее часто используемые функции. Чаще других вызывалась функция `ext4_getattr()` — 7285 раз в течение трассировки.

Вызовы функций потребляют процессорное время, а их имена часто помогают понять характер рабочей нагрузки. В случаях, когда имя функции неоднозначно, можно найти исходный код функции в интернете и по нему выяснить, что она делает. Это особенно верно для функций ядра Linux, которое распространяется с открытым исходным кодом.

Ftrace имеет много полезных инструментов, а не так давно в эту коллекцию была добавлена поддержка гистограмм и большого количества счетчиков («хронологических триггеров»). В отличие от BPF, Ftrace не поддерживает возможность программирования, поэтому ее нельзя использовать для получения и представления данных нестандартными способами.

6.3. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа работы процессора и устранения неполадок. Они показаны на рис. 6.4 и перечислены в табл. 6.3.

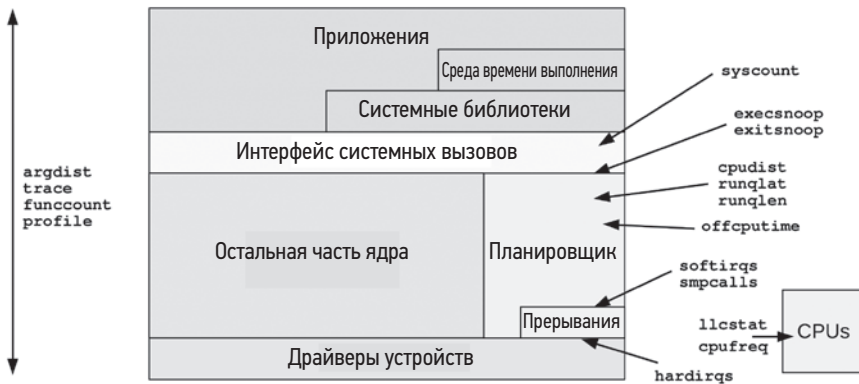


Рис. 6.4. Инструменты BPF для анализа работы CPU

Все эти инструменты либо находятся в репозиториях BCC и `bpfttrace`, описанных в главах 4 и 5, либо были написаны специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях, BCC и `bpfttrace`. В табл. 6.3 также указаны источники этих инструментов (BT — сокращение от «`bpfttrace`»).

Актуальные списки параметров инструментов BCC и `bpfttrace` и описание их возможностей ищите в соответствующих репозиториях. Далее я расскажу только о наиболее важных возможностях.

Таблица 6.3. Инструменты для анализа использования процессоров

Инструмент	Источник	Цель	Описание
execsnoop	ВСС/ВТ	Планировщик	Выводит список вновь запущенных процессов
exitsnoop	ВСС	Планировщик	Выводит продолжительность выполнения процесса и причину завершения
runqlat	ВСС/ВТ	Планировщик	Выводит сводную информацию о задержках в очередях на выполнение
runqlen	ВСС/ВТ	Планировщик	Сообщает длины очередей на выполнение
runqslower	ВСС	Планировщик	Выводит информацию о времени ожидания в очереди на выполнение, превышающий определенный порог
cpudist	ВСС	Планировщик	Выводит сводную информацию о времени выполнения на процессоре
cpufreq	книга	Процессоры	Выбирает значения тактовой частоты CPU по процессам
profile	ВСС	Процессоры	Выбирает трассировки стека по процессорам
offcputime	ВСС/книга	Планировщик	Трассировки стека и времени ожидания доступности процессора
syscount	ВСС/ВТ	Системные вызовы	Подсчет системных вызовов по типам и процессам
argdist	ВСС	Системные вызовы	Может использоваться для анализа системных вызовов
trace	ВСС	Системные вызовы	Может использоваться для анализа системных вызовов
funccount	ВСС	Программное обеспечение	Подсчитывает вызовы функций
softirqs	ВСС	Прерывания	Обобщает время обработки программных прерываний
hardirqs	ВСС	Прерывания	Обобщает время обработки аппаратных прерываний
smpcalls	книга	Ядро	Количество SMP-вызовов удаленных процессоров
llcstat	ВСС	РМС	Выводит сводную информацию о попаданиях в кэш последнего уровня по процессам

6.3.1. execsnoop

execsnoop(8)¹ — это инструмент для ВСС и bpftrace, трассирующий запуск новых процессов в масштабе системы. Он помогает обнаруживать проблемы, связанные

¹ Немного истории: первая версия execsnoop на основе DTrace была выпущена 24 марта 2004 года. Я создал ее для решения распространенной проблемы производительности в Solaris, связанной с короткоживущими процессами. Ранее анализ этой проблемы

с короткоживущими процессами, потребляющими вычислительные ресурсы, а также может использоваться для отладки выполнения ПО, включая сценарии запуска приложений.

Вот пример вывода версии для BCC:

```
# execsnoop
PCOMM      PID   PPID  RET  ARGS
sshd       33096 2366   0    /usr/sbin/sshd -D -R
bash       33118 33096   0    /bin/bash
groups     33121 33119   0    /usr/bin/groups
ls         33123 33122   0    /bin/ls /etc/bash_completion.d
lesspipe   33125 33124   0    /usr/bin/lesspipe
basename   33126 33125   0    /usr/bin/basename /usr/bin/lesspipe
dirname    33129 33128   0    /usr/bin/dirmame /usr/bin/lesspipe
tput       33130 33118   0    /usr/bin/tput setaf 1
dircolors  33132 33131   0    /usr/bin/dircolors -b
ls         33134 33133   0    /bin/ls /etc/bash_completion.d
mesg       33135 33118   0    /usr/bin/mesg n
sleep      33136 2015   0    /bin/sleep 30
sh         33143 33139   0    /bin/sh -c command -v debian-sa1 > /dev/null &&...
debian-sa1 33144 33143   0    /usr/lib/sysstat/debian-sa1 1 1
sa1        33144 33143   0    /usr/lib/sysstat/sa1 1 1
sadc       33144 33143   0    /usr/lib/sysstat/sadc -F -L -S DISK 1 1 /var/lo...
sleep      33148 2015   0    /bin/sleep 30
[...]
```

Инструмент зафиксировал момент, когда пользователь вошел в систему, используя SSH, и запустил процессы, в том числе `sshd(8)`, `groups(1)` и `mesg(1)`. Здесь также видны процессы системного регистратора активности, `sar` (system activity recorder), записывающего метрики в свои журналы, включая `sa1(8)` и `sadc(8)`.

Инструмент `execsnoop(8)` удобно применять для поиска короткоживущих процессов, потребляющих значительные вычислительные ресурсы. Часто их трудно обнаружить из-за короткого периода работы — они могут успеть завершиться до того, как `top(1)` и подобные ему инструменты или агенты мониторинга заметят их. В главе 1 был показан пример, когда сценарий запуска многократно проваливался, пытаясь запустить приложение и ухудшая производительность системы. Мы легко смогли обнаружить его с помощью `execsnoop(8)`. Мы использовали `execsnoop(8)` для отладки многих проблем: возмущений от фоновых заданий, медленного или неудачного запуска приложений, медленного или неудачного запуска контейнера и т. д.

основывался на включении учета процессов или аудита BSM и выборке событий `exes` из журналов, но у обоих вариантов был большой недостаток: механизм учета процессов урезал имя и аргументы процесса до восьми символов. Мой инструмент `execsnoop` можно запустить немедленно, без включения специальных режимов аудита, и он может показать гораздо более длинный фрагмент командной строки. `execsnoop` устанавливается по умолчанию в OS X и в некоторых версиях Solaris и BSD. 7 февраля 2016 года я выпустил версию для BCC и 15 ноября 2017 года версию для `bpftrace`. В последнем случае пришлось добавить в `bpftrace` встроенную функцию `join()`.

execsnoop(8) трассирует системный вызов `execve(2)` (наиболее часто используемый вариант `exec(2)`) и сообщает подробную информацию об аргументах и возвращаемом значении `execve(2)`. Таким способом он перехватывает запуск новых процессов, следующих процедуре `fork(2)/clone(2) -> exec(2)`, а также повторный запуск самих себя, когда процесс повторно вызывает `exec(2)`. Некоторые приложения создают новые процессы без вызова `exec(2)`, например, формируя пул рабочих процессов только с помощью `fork(2)` или `clone(2)`. `execsnoop(8)` не включает их в свой вывод, потому что они не вызывают `execve(2)`. Это не обычная ситуация: приложения должны создавать пулы рабочих потоков, а не процессов.

Поскольку ожидается относительно небольшая частота запуска новых процессов (<1000/с), этот инструмент должен иметь незначительный оверхед.

BCC

Версия для BCC поддерживает несколько параметров, в том числе:

- **-x**: включить в вывод неудачные вызовы `exec()`;
- **-n pattern**: вывести только команды, содержащие `pattern`;
- **-l pattern**: вывести только команды, аргументы которых содержат `pattern`;
- **--max-args args**: максимальное число аргументов для вывода (значение по умолчанию: 20).

bpfttrace

Ниже приводится реализация `execsnoop(8)` для `bpfttrace`, в которой обобщены основные функциональные возможности инструмента. Эта версия выводит основные столбцы и не поддерживает параметры:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("%-10s %-5s %s\n", "TIME(ms)", "PID", "ARGS");
}

tracepoint:syscalls:sys_enter_execve
{
    printf("%-10u %-5d ", elapsed / 1000000, pid);
    join(args->argv);
}
```

BEGIN выводит заголовок. Для перехвата событий вызова `exec()` используется точка трассировки `syscalls:sys_enter_execve`. Обработчик этого события выводит время, прошедшее с момента запуска программы, идентификатор процесса, а также имя команды и ее аргументы. Он применяет функцию `join()` к полю `args->argv` точки трассировки, чтобы вывести имя команды и аргументы в одну строку.

В будущих версиях bpftrace функция `join()` может измениться — предполагается, что вместо вывода строки она будет просто возвращать ее¹, что позволит реализовать инструмент иначе:

```
tracepoint:syscalls:sys_enter_execve
{
    printf("%-10u %-5d %s\n", elapsed / 1000000, pid, join(args->argv));
}
```

Версия для ВСС инструментирует и вход, и выход из системного вызова `execve()`, что позволяет вывести возвращаемое значение. Аналогичную возможность легко добавить в программу bpftrace².

В главе 13 вы найдете аналогичный инструмент, `threadsnoop(8)`, который трассирует запуск потоков выполнения вместо процессов.

6.3.2. exitsnoop

`exitsnoop(8)`³ — это инструмент для ВСС, который трассирует события завершения процессов и сообщает их возраст и причину завершения. Возраст — это время от момента запуска процесса до завершения; в него включается время выполнения процесса на CPU и проведенное в очереди ожидания. Как и `execsnoop(8)`, инструмент `exitsnoop(8)` помогает в отладке проблем с короткоживущими процессами, давая различную информацию, которая помогает определить тип рабочей нагрузки. Например:

```
# exitsnoop
PCOMM      PID    PPID    TID    AGE(s)  EXIT_CODE
cmake      8994    8993    8994    0.01    0
sh         8993    8951    8993    0.01    0
sleep      8946    7866    8946    1.00    0
cmake      8997    8996    8997    0.01    0
sh         8996    8995    8996    0.01    0
make       8995    8951    8995    0.02    0
cmake      9000    8999    9000    0.02    0
sh         8999    8998    8999    0.02    0
git        9003    9002    9003    0.00    0
DOM Worker 5111    4183    8301    221.25  0
sleep      8967    26663   8967    7.31    signal 9 (KILL)
git        9004    9002    9004    0.00    0
[...]
```

¹ См. описание проблемы #26 в bpftrace [67].

² Эту и последующие программы bpftrace легко расширить и реализовать в них вывод большего числа деталей. Для краткости и простоты я не стал этого делать.

³ Немного истории: этот инструмент было создан Артуро Мартин-де-Николасом (Arturo Martin-de-Nicolas) 4 мая 2019 года.

Здесь видно множество короткоживущих процессов, завершивших выполнение, — `stake(1)`, `sh(1)` и `make(1)`. По всей видимости, при трассировке выполнялась сборка ПО. Один процесс `sleep(1)` успешно завершился (код выхода 0) через 1.00 секунду, а другой процесс `sleep(1)` завершился через 7.31 секунды, получив сигнал `KILL`. Также в вывод попало событие завершения потока «DOM Worker», выполнявшегося на протяжении 221.25 секунды.

`exitsnoop(8)` инструментирует точку трассировки `sched:sched_process_exit` и ее аргументы и использует функцию `bpf_get_current_task()`, чтобы получить время запуска из структуры задачи (нестабильная деталь интерфейса). Поскольку, как предполагается, эта точка трассировки срабатывает нечасто, `exitsnoop(8)` будет иметь незначительный оверхед.

Порядок использования:

```
exitsnoop [options]
```

Параметры `options`:

- **-p PID**: трассировать только этот процесс;
- **-t**: включать в вывод отметки времени;
- **-x**: трассировать только события завершения с кодом ошибки (с ненулевым кодом выхода).

Сейчас версия `exitsnoop(8)` для `bpftool` не реализована, но в этом нет ничего сложного и такая реализация может послужить отличным упражнением для изучающих программирование на `bpftool`¹.

6.3.3. runqlat

`runqlat(8)`² — это инструмент для BCC и `bpftool`, предназначенный для анализа задержек планировщика CPU, которые часто называют задержками очереди на выполнение (даже при том, что современные реализации планировщиков не используют очереди). Его можно использовать для выявления и количественной оценки проблем с насыщением CPU, когда спрос на вычислительные ресурсы превышает возможности системы. `runqlat(8)` измеряет время, которое каждый поток (задача) тратит на ожидание своей очереди выполнения на CPU.

¹ Если вы напишете эту версию и решите опубликовать ее, не забудьте указать автора оригинальной версии для BCC: Артуро Мартин-де-Николас.

² Немного истории: первая версия — `disqlat.d` — была создана мной на основе `DTrace` и выпущена 13 августа 2012 года. Я использовал специализированные зонды `DTrace` и руководствовался примерами из руководства «Dynamic Tracing Guide», изданного в январе 2005 года [Sun 05]. Имя `dispq` — это сокращение от «dispatcher queue» (очередь диспетчера), еще одного названия очереди на выполнение. Версию для BCC — `runqlat` — я выпустил 7 февраля 2016 года, а версию для `bpftool` — 17 сентября 2018 года.

Ниже показан результат запуска `runqlat(8)` для ВСС на производственном экземпляре с 48 процессорами и загрузкой, равной 42%. Аргументы «10 1» для `runqlat(8)` определяют 10-секундный интервал и однократный вывод:

```
# runqlat 10 1
```

```
Tracing run queue latency... Hit Ctrl-C to end.
```

usecs	: count	distribution
0 -> 1	: 3149	
2 -> 3	: 304613	*****
4 -> 7	: 274541	*****
8 -> 15	: 58576	*****
16 -> 31	: 15485	**
32 -> 63	: 24877	***
64 -> 127	: 6727	
128 -> 255	: 1214	
256 -> 511	: 606	
512 -> 1023	: 489	
1024 -> 2047	: 315	
2048 -> 4095	: 122	
4096 -> 8191	: 24	
8192 -> 16383	: 2	

В большинстве случаев потоки ждали менее 15 микросекунд (столбики гистограммы, соответствующие периодам от 2 до 15 микросекунд). Это относительно немного для нормально функционирующей системы и вполне ожидаемо для системы, работающей с 42%-ной загрузкой CPU. Время от времени задержка в очереди на выполнение достигала в этом примере 8–16 миллисекунд, но это единичные выбросы.

`runqlat(8)` инструментрует события возобновления после ожидания и переключения контекста, чтобы определить время от возобновления до запуска. В высоконагруженных производственных системах эти события могут следовать очень часто и достигать миллиона событий в секунду и даже больше. Несмотря на все оптимизации, используемые в BPF, при таких частотах добавление даже одной микросекунды на событие может вызвать заметный оверхед¹. Используйте этот инструмент с осторожностью.

Неправильная настройка процедуры сборки

Вот еще один пример для сравнения. На этот раз трассировка выполняется на сервере с 36 процессорами в период сборки ПО, при этом для инструмента сборки по ошибке было установлено число параллельных заданий, равное 72, что вызвало перегрузку процессоров:

¹ Вот простой пример: если в системе с 10 процессорами скорость переключения контекста составляет 1 М/с, добавление 1 микросекунды на обработку каждого переключения контекста потребует 10% ресурсов процессора ($100\% \times (1 \times 1\,000\,000 / 10 \times 1\,000\,000)$). В главе 18 вы найдете некоторые примеры измерения оверхеда BPF, в реальной жизни они обычно намного меньше 1 микросекунды на событие.

```
# runqlat 10 1
```

```
Tracing run queue latency... Hit Ctrl-C to end.
```

usecs	count	distribution
0 -> 1	: 1906	***
2 -> 3	: 22087	*****
4 -> 7	: 21245	*****
8 -> 15	: 7333	*****
16 -> 31	: 4902	*****
32 -> 63	: 6002	*****
64 -> 127	: 7370	*****
128 -> 255	: 13001	*****
256 -> 511	: 4823	*****
512 -> 1023	: 1519	**
1024 -> 2047	: 3682	*****
2048 -> 4095	: 3170	*****
4096 -> 8191	: 5759	*****
8192 -> 16383	: 14549	*****
16384 -> 32767	: 5589	*****
32768 -> 65535	: 372	
65536 -> 131071	: 10	

В этом распределении наблюдается три модальных значения, из которых самый длительный период ожидания соответствует интервалу с центром от 8 до 16 миллисекунд. Это уже существенная задержка в выполнении потоков.

Эту конкретную проблему легко идентифицировать с помощью других инструментов и метрик. Например, `sar(1)` может показывать загрузку процессора (`-u`) и метрики очереди выполнения (`-q`):

```
# sar -uq 1
```

```
Linux 4.18.0-virtual (...) 01/21/2019    _x86_64_    (36 CPU)
```

	CPU	%user	%nice	%system	%iowait	%steal	%idle
11:06:25 PM	all	88.06	0.00	11.94	0.00	0.00	0.00

	runq-sz	plist-sz	ldavg-1	ldavg-5	ldavg-15	blocked
11:06:26 PM	72	1030	65.90	41.52	34.75	0

```
[...]
```

В этом выводе `sar(1)` видно, что доля времени простоя CPU составила 0% и средний размер очереди выполнения равен 72 (включая выполняющиеся и готовые к выполнению задачи), что намного больше 36 — числа доступных CPU.

В главе 15 мы рассмотрим пример, где с помощью `runqlat(8)` определяется задержка для каждого контейнера.

BCC

Порядок использования версии для BCC:

```
runqlat [options] [interval [count]]
```

Параметры `options` версии для BCC:

- **-m**: выводить время в миллисекундах;
- **-P**: выводить гистограммы для каждого процесса (PID);
- **--pidnss**: выводить гистограммы для каждого пространства имен PID;
- **-p PID**: трассировать только этот процесс;
- **-T**: включать в вывод отметки времени.

Параметр **-T** можно использовать для формирования вывода через определенные интервалы. Например, `runqlat -T 1` будет выводить данные с интервалом в 1 секунду.

bpfftrace

Ниже приводится реализация `runqlat(8)` для `bpfftrace`, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры.

```
#!/usr/local/bin/bpfftrace

#include <linux/sched.h>

BEGIN
{
    printf("Tracing CPU scheduler... Hit Ctrl-C to end.\n");
}

tracepoint:sched:sched_wakeup,
tracepoint:sched:sched_wakeup_new
{
    @qtime[args->pid] = nsecs;
}

tracepoint:sched:sched_switch
{
    if (args->prev_state == TASK_RUNNING) {
        @qtime[args->prev_pid] = nsecs;
    }

    $ns = @qtime[args->next_pid];
    if ($ns) {
        @usecs = hist((nsecs - $ns) / 1000);
    }
    delete(@qtime[args->next_pid]);
}

END
{
    clear(@qtime);
}
```

Программа записывает отметку времени в точках трассировки `sched_wakeup` и `sched_wakeup_new` для каждого ключа `args->pid`, представляющего идентификатор потока ядра.

Обработчик события `sched_switch` сохраняет отметку времени, используя ключ `args->prev_pid`, если задача находится в состоянии готовности к выполнению (`TASK_RUNNING`). Он обрабатывает принудительное переключение контекста, когда поток оставляет CPU и возвращается в очередь на выполнение. Этот обработчик также проверяет, была ли отметка времени сохранена для следующего запускаемого процесса, и если это так, вычисляет разность времени и сохраняет ее в гистограмме `@usecs`.

Поскольку здесь используется макрос `TASK_RUNNING`, к программе подключается файл заголовка `linux/sched.h` (`#include`), содержащий его определение.

Версия для BCC может группировать вывод по PID. Эту возможность легко реализовать в версии для `bpfttrace`, добавив ключ `pid` в карту `@usecs`. Другое усовершенствование в версии для BCC — пропуск записи задержки в очереди на выполнение для PID 0, чтобы исключить трассировку задержек планирования потока ядра, выполняющегося в периоды бездействия¹. И снова эту особенность легко можно реализовать в версии для `bpfttrace`.

6.3.4. runqlen

`runqlen(8)`² — это инструмент для BCC и `bpfttrace`, предназначенный для выборки длин очередей на выполнение, подсчета количества задач, ожидающих своей очереди, и представления полученной информации в виде линейной гистограммы. Этот инструмент можно использовать для дальнейшего исследования проблем с задержками в очереди на выполнение или для приближенной оценки.

Ниже приведен пример использования `runqlen(8)` для BCC на производственном экземпляре с 48 процессорами и загрузкой, равной 42% (этот же экземпляр, который был показан в примере использования `runqlat(8)`). Аргументы «10 1» для `runqlen(8)` определяют 10-секундный интервал и однократный вывод:

```
# runqlen 10 1
Sampling run queue length... Hit Ctrl-C to end.
```

runqlen	: count	distribution
0	: 47284	*****
1	: 211	
2	: 28	
3	: 6	
4	: 4	
5	: 1	
6	: 1	

¹ Спасибо Ивану Бабру (Ivan Babrou) за ее добавление.

² Немного истории: первую версию — `dispsqlen.d` — я написал 27 июня 2005 года, желая узнать, как меняются длины очередей на выполнение. Версию для BCC я выпустил 12 декабря 2016 года, а версию для `bpfttrace` — 7 октября 2018 года.

Согласно полученным результатам, большую часть времени очередь на выполнение была нулевой длины, то есть потокам не требовалось ждать своей очереди.

Длину очереди на выполнение я считаю второстепенной метрикой производительности, а задержку в очереди — первостепенной. В отличие от длины, задержка прямо влияет на производительность. Представьте себе очередь к кассе в продуктовом магазине. Что для вас важнее: длина очереди или время ожидания? `runqlat(8)` более важный инструмент. Но зачем тогда вообще использовать `runqlen(8)`?

Во-первых, `runqlen(8)` можно использовать для продолжения анализа проблем, обнаруженных с помощью `runqlat(8)`, и выявления причин, почему задержки оказываются слишком большими. Во-вторых, `runqlen(8)` выбирает данные по времени, с частотой 99 Гц, тогда как `runqlat(8)` трассирует события планировщика. Выборка по времени имеет ничтожный оверхед по сравнению с трассировкой планировщика инструментом `runqlat(8)`. Для мониторинга в режиме 24×7 сначала лучше использовать `runqlen(8)`, чтобы подтвердить существование проблем (так как этот метод обходится дешевле), а затем применить `runqlat(8)` для количественной оценки задержки.

Четыре потока, один процессор

В этом примере рабочая нагрузка, состоящая из четырех занятых потоков, была привязана к CPU 0. Чтобы получить гистограмму для каждого процессора, использовался инструмент `runqlen(8)` с параметром `-C`:

```
# runqlen -C
```

```
Sampling run queue length... Hit Ctrl-C to end.
```

```
^C
```

```
cpu = 0
  runqlen      : count      distribution
    0          : 0          |
    1          : 0          |
    2          : 0          |
    3          : 551        |*****|

cpu = 1
  runqlen      : count      distribution
    0          : 41         |*****|

cpu = 2
  runqlen      : count      distribution
    0          : 126        |*****|

[...]
```

Очередь на выполнение на CPU 0 имеет длину, равную трем: один поток выполняется на процессоре и три потока ожидают своей очереди. Вывод результатов для каждого процессора в отдельности помогает проверить правильную балансировку нагрузки на процессоры планировщиком.

BCC

Порядок использования версии для BCC:

```
runqlen [options] [interval [count]]
```

Параметры options:

- **-C**: выводить отдельную гистограмму для каждого CPU;
- **-O**: выводить заполнение очереди на выполнение;
- **-T**: включать в вывод отметки времени.

Заполнение очереди на выполнение — это отдельная метрика, показывающая долю времени, проведенную потоком в ожидании. Иногда ее можно использовать, когда для мониторинга, оповещения и построения графиков требуется единственная метрика.

bpfttrace

Ниже приводится реализация runqlen(8) для bpfttrace, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры.

```
#!/usr/local/bin/bpfttrace

#include <linux/sched.h>

struct cfs_rq_partial {
    struct load_weight load;
    unsigned long runnable_weight;
    unsigned int nr_running;
};

BEGIN
{
    printf("Sampling run queue length at 99 Hertz... Hit Ctrl-C to end.\n");
}

profile:hz:99
{
    $task = (struct task_struct *)curtask;
    $my_q = (struct cfs_rq_partial *)$task->se.cfs_rq;
    $len = $my_q->nr_running;
    $len = $len > 0 ? $len - 1 : 0; // учесть текущую выполняемую задачу
    @runqlen = lhist($len, 0, 100, 1);
}
```

Программа ссылается на член nr_running структуры cfs_rq, но эта структура недоступна в стандартных заголовках ядра. По этой причине программа начинается с определения структуры cfs_rq_partial, достаточного для извлечения необходимого

члена. Этот обходной путь не понадобится, когда будет внедрен механизм BTF (см. главу 2).

Основной зонд — `profile:hz:99`, определяющий длину очереди на всех процессорах с частотой 99 Гц. Длина определяется путем перехода от структуры текущей задачи к очереди на выполнение, в которой та находится, и чтения длины этой очереди. При изменении исходного кода ядра может потребоваться скорректировать определение структуры и имена ее членов.

Эту версию `bpfttrace` можно усовершенствовать, добавив разбивку по процессорам с помощью ключа `cpi` в `@runqlen`.

6.3.5. runqslower

`runqslower(8)`¹ — это инструмент для BCC, который выявляет задержки в очереди на выполнение, превышающие заданное пороговое значение, а также показывает процессы, подвергшиеся задержкам, и продолжительность задержки. В примере ниже показан вывод, полученный на производственном экземпляре с 48 процессорами, который в период трассировки работал с загрузкой, равной 45%:

```
# runqslower
Tracing run queue latency higher than 10000 us
TIME      COMM                PID      LAT(us)
17:42:49  python3                 4590      16345
17:42:50  pool-25-thread-        4683      50001
17:42:53  ForkJoinPool.co        5898      11935
17:42:56  python3                 4590      10191
17:42:56  ForkJoinPool.co        5912      13738
17:42:56  ForkJoinPool.co        5908      11434
17:42:57  ForkJoinPool.co        5890      11436
17:43:00  ForkJoinPool.co        5477      10502
17:43:01  grpc-default-wo        5794      11637
17:43:02  tomcat-exec-296        6373      12083
[...]
```

Согласно этим результатам, в течение 13 секунд было 10 случаев задержки в очереди на выполнение выше порогового значения по умолчанию, равного 10 000 микросекунд (10 миллисекунд). Это может показаться удивительным для сервера с 55% свободных вычислительных ресурсов, но в данном случае речь идет о высоконагруженном многопоточном приложении. Кроме того, возможен некоторый дисбаланс очередей, пока планировщик не сможет переместить потоки на свободные процессоры. Этот инструмент помогает выявить подобные приложения.

Сейчас этот инструмент использует `kprobes` для функций ядра `ttwu_do_wakeup()`, `wake_up_new_task()` и `finish_task_switch()`. В будущей версии, скорее всего, будут использоваться точки трассировки планировщика, по аналогии с предыдущей

¹ Немного истории: был создан Иваном Бабру 2 мая 2018 года.

версий `runqlat(8)` для `bpfttrace`. Оверхед аналогичен оверхеду `runqlat(8)`; использование этого инструмента может вызвать значительный оверхед в высоконагруженных системах из-за дороговизны `kprobes`, даже если `runqslower(8)` не будет выводить никаких данных.

Порядок использования:

```
runqslower [options] [min_us]
```

Параметры `options`:

- **-p PID**: трассировать только этот процесс.

По умолчанию используется порог 10 000 микросекунд.

6.3.6. `cpudist`

`cpudist(8)`¹ — это инструмент для ВСС, предназначенный для анализа распределения времени выполнения потоков на CPU после возобновления. Эту информацию можно использовать для определения характеристик рабочих нагрузок и обоснования последующих настроек и проектных решений. Вот пример вывода, полученный на производственном экземпляре с 48 процессорами:

```
# cpudist 10 1
```

```
Tracing on-CPU time... Hit Ctrl-C to end.
```

usecs	count	distribution
0 -> 1	103865	*****
2 -> 3	91142	*****
4 -> 7	134188	*****
8 -> 15	149862	*****
16 -> 31	122285	*****
32 -> 63	71912	*****
64 -> 127	27103	*****
128 -> 255	4835	*
256 -> 511	692	
512 -> 1023	320	
1024 -> 2047	328	
2048 -> 4095	412	
4096 -> 8191	356	
8192 -> 16383	69	
16384 -> 32767	42	
32768 -> 65535	30	
65536 -> 131071	22	
131072 -> 262143	20	
262144 -> 524287	4	

¹ Немного истории: я создал `cpudists` 27 апреля 2005 года, чтобы выяснить распределение времени выполнения на CPU процессов, ядра и потока, выполняющегося в периоды бездействия. Саша Гольдштейн разработал версию `cpudist` для ВСС(8) 29 июня 2016 года и добавил возможность получения распределения для каждого процесса.

Как показывают эти результаты, приложение в производственной среде обычно выполняется на процессоре в течение очень короткого времени: от 0 до 127 микросекунд.

Вот пример другой рабочей нагрузки, выполняющей интенсивные вычисления, с числом потоков, превышающим количество доступных процессоров (время в гистограмме измеряется в миллисекундах (-m)):

```
# cpudist -m
Tracing on-CPU time... Hit Ctrl-C to end.
^C
```

msecs	: count	distribution
0 -> 1	: 521	*****
2 -> 3	: 60	****
4 -> 7	: 272	*****
8 -> 15	: 308	*****
16 -> 31	: 66	*****
32 -> 63	: 14	*

В этой гистограмме четко видна мода, соответствующая длительности выполнения на процессоре от 4 до 15 миллисекунд: вероятно, потоки исчерпывают выделенные им кванты времени, после чего планировщик принудительно переключает контекст.

Этот инструмент помог понять влияние изменений в производственной среде в Netflix, ускоривших работу приложения машинного обучения в три раза. Команда `perf(1)` показала, что скорость переключения контекста упала, а `cpudist(8)` помог понять, к чему это привело: после изменений приложение стало выполняться на процессоре в течение 2–4 миллисекунд между переключениями контекста, тогда как раньше выполнялось не более 3 микросекунд.

В своей работе `cpudist(8)` использует события переключения контекста, которые в высоконагруженных системах могут следовать очень часто (до миллиона событий в секунду и даже больше). Как и `runqlat(8)`, этот инструмент может давать существенный оверхед, поэтому используйте его с осторожностью.

Порядок использования:

```
cpudist [options] [interval [count]]
```

Параметры `options`:

- **-m**: выводить время в миллисекундах (по умолчанию время измеряется в микросекундах);
- **-O**: вместо времени выполнения на процессоре выводить время ожидания в очереди на выполнение;
- **-P**: выводить гистограммы для каждого процесса отдельно;
- **-p PID**: трассировать только этот процесс.

Сейчас `cpudist(8)` не имеет версии для `bpfftrace`. Я не стал заниматься ее реализацией и в конце главы предложу вам реализовать ее самостоятельно.

6.3.7. cpufreq

cpufreq(8)¹ выполняет выборку значений частоты процессора и отображает ее в виде гистограммы для системы в целом и отдельных гистограмм для процессов. Работает только при настройке регулятора частоты CPU на определенный режим, например энергосбережения (powersave), и может использоваться для определения тактовой частоты, с которой выполняются приложения. Например:

```
# cpufreq.bt
```

Sampling CPU freq system-wide & by process. Ctrl-C to end.

 $\wedge C$

[...]

```
@process_mhz[snmpd]:
```

[1200, 1400) 1 |
|

```
@process mhz[python3]:
```

[illegible]

```
@process_mhz[java]:
```

[illegible]

```
@system mhz:
```

[1200, 1400)	22041	@@
[1400, 1600)	903	@@
[1600, 1800)	474	@
[1800, 2000)	368	
[2000, 2200)	30	
[2200, 2400)	3	
[2400, 2600)	21	

¹ Немного истории: я создал этот инструмент специально для книги 24 апреля 2019 года и использовал в качестве прообраза BPF-инструмент `time_in_state` из Android, созданный Коннором О'Брайеном (Connor O'Brien) на основе первоначальных набросков Джоэля Фернандеса (Joel Fernandes). Для более точного отслеживания частоты он использует точки трассировки.

[2600, 2800)	33		
[2800, 3000)	15		
[3000, 3200)	270		
[...]			

Как показывают эти результаты, при трассировке частота для системы в целом находилась в диапазоне от 1200 до 1400 МГц, то есть это практически бездействующая система. Похожее распределение частот наблюдалось в процессе `java`, и лишь в 18 замерах частота достигла диапазона 3.0–3.2 ГГц. Это приложение в основном выполняло операции дискового ввода/вывода, во время которых процессоры переходили в состояние энергосбережения. Процессы `python3`, напротив, работали в основном на максимальной скорости.

В своей работе этот инструмент использует точки трассировки, срабатывающие при изменении частоты, и производит замеры с частотой 100 Гц. Как предполагается, инструмент должен иметь низкий или незначительный оверхед. Предыдущий вывод получен в системе, где регулятор частоты настроен на режим энергосбережения (`powersave`), как указано в `/sys/devices/system/cpu/cpufreq/.../scaling_governor`. Когда система настроена на работу с максимальной производительностью, этот инструмент ничего не показывает, так как нет колебаний тактовой частоты: процессоры действуют на самой высокой частоте.

Вот выдержка из результатов исследования рабочей нагрузки, которую я только что обнаружил:

```
@process_mhz[nginx]:
[1200, 1400) 35 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1400, 1600) 17 | @@@@@@@@@@@@@@@@@@@@@@
[1600, 1800) 16 | @@@@@@@@@@@@@@@@@@
[1800, 2000) 17 | @@@@@@@@@@@@@@@@@@
[2000, 2200) 0  |
[2200, 2400) 0  |
[2400, 2600) 0  |
[2600, 2800) 0  |
[2800, 3000) 0  |
[3000, 3200) 0  |
[3200, 3400) 0  |
[3400, 3600) 0  |
[3600, 3800) 50 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

Приложение `nginx`, действующее в производственной среде, большую часть времени работало на низких тактовых частотах процессора. Регулятор `scaling_governor` в этой системе был настроен на работу в режиме энергосбережения по умолчанию.

Исходный код `cpufreq(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Sampling CPU freq system-wide & by process. Ctrl-C to end.\n");
}
```

```

tracepoint:power:cpu_frequency
{
    @curfreq[cpu] = args->state;
}

profile:hz:100
/@curfreq[cpu]/
{
    @system_mhz = lhist(@curfreq[cpu] / 1000, 0, 5000, 200);
    if (pid) {
        @process_mhz[comm] = lhist(@curfreq[cpu] / 1000, 0, 5000, 200);
    }
}

END
{
    clear(@curfreq);
}

```

Значения частоты определяются с помощью точки трассировки `power:cpu_frequency` и сохраняются в BPF-карте `@curfreq` отдельно для каждого процессора. Гистограммы трассируют частоты от 0 до 5000 МГц с шагом 200 МГц. Эти параметры можно изменить при желании.

6.3.8. profile

`profile(8)`¹ — это инструмент для ВСС, который выбирает трассировки стека с заданным интервалом и сообщает частоту каждой из них. Это самый полезный инструмент в ВСС для понимания особенностей потребления CPU, так как он обобщает практически все пути кода, характеризующиеся высоким потреблением вычислительных ресурсов. (См. описание инструмента `hardirqs(8)` в разделе 6.3.14, где рассказывается о дополнительных потребителях CPU.) У него относительно невысокий оверхед, потому что частота событий привязана к частоте выборки, которую можно настраивать.

По умолчанию этот инструмент выбирает трассировки стека для всех процессоров в обоих пространствах, пользователя и ядра, с частотой 49 Гц. Эти аспекты можно настроить с помощью параметров, при этом выбранные параметры выводятся перед результатами. Например:

¹ Немного истории: в прошлом было создано много профилировщиков, в том числе `gprof` в 1982 году [Graham 82] (переписан в 1988 году Джейм Фенласоном (Jay Fenlason) для проекта GNU). Я разработал версию для ВСС 15 июля 2016 года, взяв за основу код Саши Гольдштейна, Эндрю Бирчалла (Andrew Birchall), Евгения Верещагина (Evgeny Vereshchagin) и Тена Циня (Teng Qin). Моя первая версия не имела поддержки в ядре и использовала грубый прием: я добавил точку трассировки для выборки замеров, которая использовалась вместе с `perf_event_open()`. Питер Зижистра (Peter Zijistra) отверг мой код, добавляющий эту точку трассировки в ядро Linux, отдав предпочтение поддержке профилирования с помощью BPF, которую добавил Алексей Старовойтов.

profile

Sampling at 49 Hertz of all threads by user + kernel stack... Hit Ctrl-C to end.

^C

```

sk_stream_alloc_skb
sk_stream_alloc_skb
tcp_sendmsg_locked
tcp_sendmsg
sock_sendmsg
sock_write_iter
__vfs_write
vfs_write
ksys_write
do_syscall_64
entry_SYSCALL_64_after_hwframe
__GI__write
[unknown]
-                               iperf (29136)

```

1

[...]

```

__free_pages_ok
__free_pages_ok
skb_release_data
__kfree_skb
tcp_ack
tcp_rcv_established
tcp_v4_do_rcv
__release_sock
release_sock
tcp_sendmsg
sock_sendmsg
sock_write_iter
__vfs_write
vfs_write
ksys_write
do_syscall_64
entry_SYSCALL_64_after_hwframe
__GI__write
[unknown]
-                               iperf (29136)

```

1889

```

get_page_from_freelist
get_page_from_freelist
__alloc_pages_nodemask
skb_page_frag_refill
sk_page_frag_refill
tcp_sendmsg_locked
tcp_sendmsg
sock_sendmsg
sock_write_iter
__vfs_write
vfs_write
ksys_write
do_syscall_64

```

```

entry_SYSCALL_64_after_hwframe
__GI__write
[unknown]
-                iperf (29136)
2673

```

Как видите, трассировки стека выводятся в виде списков функций, за которыми следуют тире («-»), имя процесса, его PID в скобках и в конце число — сколько раз встретилась эта трассировка стека. Трассировки стека выводятся в порядке увеличения частоты встречаемости.

Полный вывод в этом примере состоит из 17 254 строк, но здесь показаны только первая и две последние трассировки стека. Наиболее частая трассировка стека, соответствующая пути от `vfs_write()` до `get_page_from_freelist()`, наблюдалась 2673 раза.

Флейм-графики использования процессора

Флейм-графики позволяют представить трассировки стека в более наглядной форме и помогают быстро проанализировать вывод `profile(8)`. В этой книге они рассмотрены в главе 2.

Для поддержки флейм-графиков `profile(8)` предлагает параметр `-f`, обеспечивающий вывод в свернутом формате: трассировки стека выводятся в одну строку с функциями, разделенными точкой с запятой. Например, вот как можно организовать запись 30-секундного профиля в файл `out.stacks01` с включенными аннотациями ядра (`-a`):

```

# profile -af 30 > out.stacks01
# tail -3 out.stacks01
iperf;
[unknown];__GI__write;entry_SYSCALL_64_after_hwframe[k];do_syscall_64[k];ksys_write[k];vfs_write[k];__vfs_write[k];sock_write_iter[k];sock_sendmsg[k];tcp_sendmsg[k];tcp_sendmsg_locked[k];__copy_from_iter_full[k];copyin[k];copy_user_enhanced_fast_string[k];copy_user_enhanced_fast_string[k] 5844
iperf;
[unknown];__GI__write;entry_SYSCALL_64_after_hwframe[k];do_syscall_64[k];ksys_write[k];vfs_write[k];__vfs_write[k];sock_write_iter[k];sock_sendmsg[k];tcp_sendmsg[k];release_sock[k];__release_sock[k];tcp_v4_do_rcv[k];tcp_rcv_established[k];tcp_ack[k];__kfree_skb[k];skb_release_data[k];__free_pages_ok[k];__free_pages_ok[k] 10713
iperf;
[unknown];__GI__write;entry_SYSCALL_64_after_hwframe[k];do_syscall_64[k];ksys_write[k];vfs_write[k];__vfs_write[k];sock_write_iter[k];sock_sendmsg[k];tcp_sendmsg[k];tcp_sendmsg_locked[k];sk_page_frag_refill[k];skb_page_frag_refill[k];__alloc_pages_nodemask[k];get_page_from_freelist[k];get_page_from_freelist[k] 15088

```

Здесь показаны только последние три строки. Этот вывод можно передать моей реализации построения флейм-графиков и получить результат в формате SVG:

```

$ git clone https://github.com/brendangregg/FlameGraph
$ cd FlameGraph
$ ./flamegraph.pl --color=java < ../out.stacks01 > out.svg

```


Flamegraph.pl поддерживает разные цветовые палитры. Здесь используется палитра java, в которой насыщенность цвета зависит от аннотаций ядра («_ [k]»). Получившееся изображение в формате SVG показано на рис. 6.5.

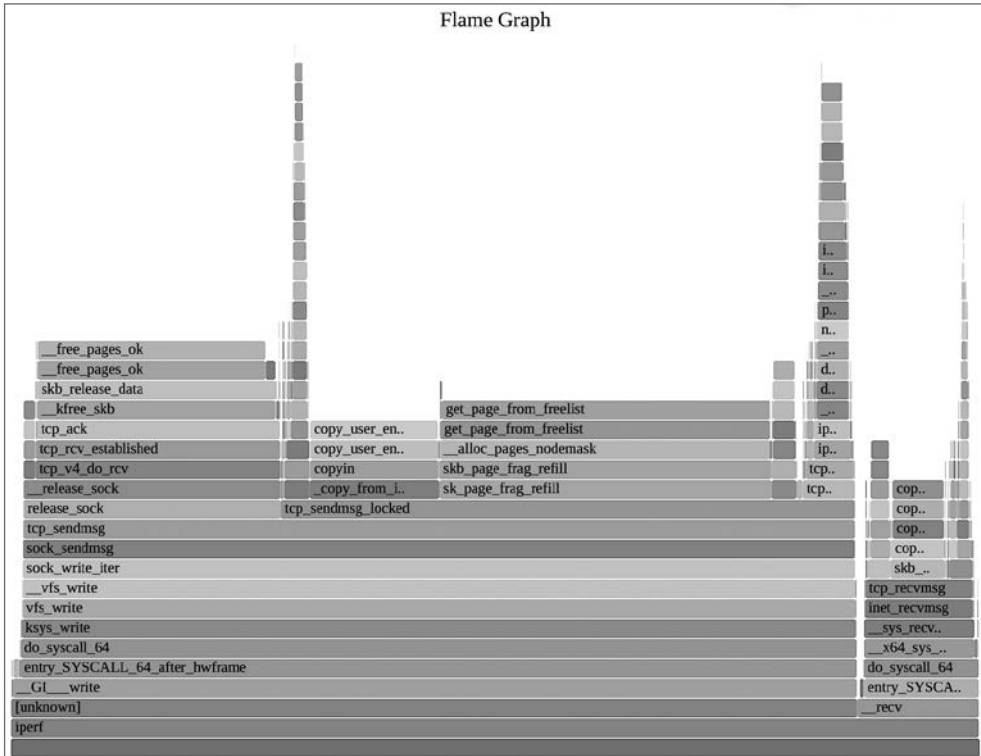


Рис. 6.5. Флейм-график использования CPU, полученный выборкой трассировок стека с помощью BPF

Этот флейм-график показывает, что самые горячие пути в коде заканчиваются в `get_page_from_freelist()` и `__free_pages_ok()` — это самые широкие башни, ширина которых пропорциональна частоте в профиле. В браузере изображение SVG масштабируется щелчком мыши, что позволяет расширить узкие башни и прочитать названия соответствующих им функций.

От других профилировщиков `profile(8)` отличается тем, что вычисляет частоту в пространстве ядра. Другие профилировщики, основанные на ядре, например `perf(1)`, отправляют каждую выбранную трассировку стека в пространство пользователя, где она подвергается последующей обработке и обобщению. Это может потребовать значительных вычислительных ресурсов в зависимости от особенностей вызова для записи выборок. Кроме того, для записи выборок может быть задействована файловая система и дисковый ввод/вывод. `profile(8)` предотвращает такой оверхед.

Порядок использования:

```
profile [options] [-F frequency]
```

Параметры options:

- **-U**: выбирать трассировки стека только в пространстве пользователя;
- **-K**: выбирать трассировки стека только в пространстве ядра;
- **-a**: добавить аннотации во фреймы стека (например, «`_[k]`» во фреймы ядра);
- **-d**: добавить разделители между стеками в пространстве ядра/пользователя;
- **-f**: вывести результаты в свернутом формате;
- **-p PID**: трассировать только этот процесс.

bpftrace

Основные возможности `profile(8)` легко реализовать в виде однострочника для `bpftrace`:

```
bpftrace -e 'profile:hz:49 /pid/ { @samples[ustack, kstack, comm] = count(); }'
```

Он подсчитывает частоту трассировок стека, используя в качестве ключа принадлежность к пространству ядра и пространству пользователя, а также имя процесса. Фильтр `pid` исключает из выборки процесс с нулевым идентификатором, то есть поток, выполняющийся в периоды бездействия. Вы можете настроить этот сценарий так, как пожелаете.

6.3.9. offcputime

`offcputime(8)`¹ — это инструмент для ВСС и `bpftrace`, подсчитывающий время, потраченное потоками в ожидании на блокировках и в очереди на выполнение, и отображающий соответствующие трассировки стека. Этот инструмент помогает понять, почему потоки не выполнялись на процессоре. `offcputime(8)` является аналогом `profile(8)`. Вместе они показывают все время, потраченное потоками в системе:

¹ Немного истории: я разработал методологию анализа времени вне CPU и реализовал ее в виде однострочников для DTrace еще в 2005 году, после изучения провайдера `sched` в DTrace и его зонда `sched::off-cpu`. Когда я впервые объяснил свою идею инженеру из Sun в Аделаиде, он сказал, что мне не следует использовать название «off-CPU», так как процессор не выключается (здесь игра слов: выражение «off-CPU» можно перевести как «вне процессора» и как «выключение процессора». — *Примеч. пер.*)! Моиими первыми инструментами анализа времени вне процессора стали `uoffcpu.d` и `koffcpu.d`, созданные в 2010 году для моей книги о DTrace [Gregg 11]. Первые инструменты для Linux я опубликовал 26 февраля 2015 года. Они использовали `perf(1)` и имели чрезвычайно высокий оверхед. Наконец, 13 января 2016 года я разработал эффективную версию `offcputime` для ВСС и 16 февраля 2019 года версию на основе `bpftrace` специально для этой книги.

`profile(8)` показывает время выполнения на процессоре, а `offcputime(8)` — время ожидания вне процессора.

В примере ниже — результаты трассировки в течение 5 секунд с использованием `offcputime(8)` для BCC:

offcputime 5

Tracing off-CPU time (us) of all threads by user + kernel stack for 5 secs.

[...]

```
finish_task_switch
schedule
schedule_timeout
wait_woken
sk_stream_wait_memory
tcp_sendmsg_locked
tcp_sendmsg
inet_sendmsg
sock_sendmsg
sock_write_iter
new_sync_write
__vfs_write
vfs_write
Sys_write
do_syscall_64
entry_SYSCALL_64_after_hwframe
__write
[unknown]
-                iperf (14657)
5625
```

[...]

```
finish_task_switch
schedule
schedule_timeout
wait_woken
sk_wait_data
tcp_recvmsg
inet_recvmsg
sock_recvmsg
SYSC_recvfrom
sys_recvfrom
do_syscall_64
entry_SYSCALL_64_after_hwframe
recv
-                iperf (14659)
1021497
```

[...]

```
finish_task_switch
schedule
schedule_hrttimeout_range_clock
schedule_hrttimeout_range
poll_schedule_timeout
```

```

do_select
core_sys_select
sys_select
do_syscall_64
entry_SYSCALL_64_after_hwframe
__libc_select
[unknown]
-                               offcputime (14667)
    5004039

```

Здесь показаны только три трассировки из нескольких сотен. В каждой перечислены фреймы в пространстве ядра (если они есть), фреймы в пространстве пользователя, имя процесса и PID и, наконец, общее время в микросекундах, когда эта комбинация наблюдалась. Первая трассировка показывает, что `iperf(1)` был заблокирован в `sk_stream_wait_memory()` в течение 5 миллисекунд. Вторая показывает, что `iperf(1)` ждал появления данных в сокете в вызове `sk_wait_data()`, потратив в общей сложности 1.02 секунды. И последняя показывает сам инструмент `offcputime(8)`, ожидавший в системном вызове `select(2)` в течение 5.00 секунды. Как нетрудно догадаться, это пятисекундный тайм-аут, указанный в командной строке.

Обратите внимание, что во всех трех случаях трассировки стеков в пространстве пользователя неполные. Они заканчиваются в библиотеке `libc`, текущая версия которой не поддерживает указатели на фреймы. В `offcputime(8)` это более очевидно, чем в `profile(8)`, потому что заблокированные стеки часто проходят через системные библиотеки, такие как `libc` или `libpthread`. См. обсуждение проблемы неполных трассировок стеков и ее решения в главах 2, 12, 13 и 18, а также в разделе 13.2.9.

На практике `offcputime(8)` часто использовался для поиска различных проблем, включая выявление задержек при попытке получить блокировку и соответствующих им трассировок стека.

В своей работе `offcputime(8)` использует события переключения контекста и фиксирует время от момента, когда поток оставляет CPU, до момента его возврата вместе с трассировкой стека. Для большей эффективности суммирование времен и подсчет трассировок стека производятся в контексте ядра. Как вы помните, события переключения контекста могут следовать очень часто, поэтому этот инструмент может давать значительный оверхед (иногда больше 10%) в высоконагруженных системах. По этой причине его лучше использовать в течение коротких периодов времени, чтобы снизить влияние на производственную среду.

Флейм-графики простоев

По аналогии с `profile(8)`, вывод `offcputime(8)` может получиться настолько объемным, что для его анализа предпочтительнее будет использовать флейм-графики, хотя и иного типа, чем показанные в главе 2. Вместо флейм-графика использования

процессора результаты работы `offcputime(8)` можно представить в виде графика времени простоя¹.

Пример создания флейм-графика простоев на основе трассировки стеков в пространстве ядра в течение 5 секунд:

```
# offcputime -fKu 5 > out.offcputime01.txt
$ flamegraph.pl --hash --bgcolors=blue --title="Off-CPU Time Flame Graph" \
  < out.offcputime01.txt > out.offcputime01.svg
```

Использував параметр `--bgcolors`, я изменил цвет фона на синий, чтобы придать флейм-графику простоев визуальное отличие от флейм-графиков использования процессора. При желании вы можете изменить цвет фреймов с помощью параметра `--colors`. Я опубликовал множество флейм-графиков простоев, используя синюю палитру для оформления фреймов².

Команды в примере выше создали флейм-график, показанный на рис. 6.6.

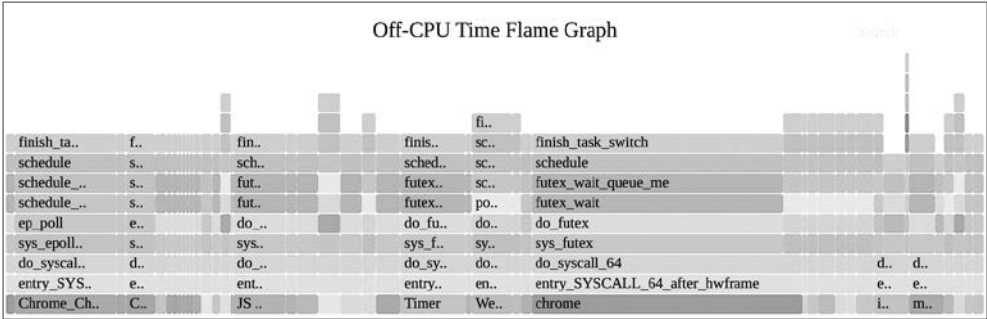


Рис. 6.6. Флейм-график простоев

На этом флейм-графике преобладают ожидающие потоки. Интересующие приложения можно изучить, щелкая на их именах для увеличения масштаба. Дополнительные сведения о флейм-графиках простоев, включая примеры с полными трассировками стеков в пространстве пользователя, см. в главах 12, 13 и 14.

BCC

Порядок использования:

```
offcputime [options] [duration]
```

¹ Впервые эта идея была предложена Ичунь Чжаном (Yichun Zhang) [80].
² Сейчас я предпочитаю менять только цвет фона на синий и оформлять фреймы в той же палитре, что и во флейм-графиках использования процессора.

Параметры `options`:

- **-f**: вывести результаты в свернутом формате;
- **-p PID**: трассировать только этот процесс;
- **-u**: трассировать только потоки в пространстве пользователя;
- **-k**: трассировать только потоки в пространстве ядра;
- **-U**: выбирать трассировки стека только в пространстве пользователя;
- **-K**: выбирать трассировки стека только в пространстве ядра.

Некоторые из этих параметров помогут уменьшить оверхед за счет фильтрации и записи данных только для одного PID или одного типа стека.

bpfttrace

Ниже приводится реализация версии `offcputime(8)` для `bpfttrace`, в которой обобщены основные функциональные возможности инструмента. Эта версия поддерживает необязательный аргумент PID для выбора цели трассировки:

```
#!/usr/local/bin/bpfttrace

#include <linux/sched.h>

BEGIN
{
    printf("Tracing nanosecond time in off-CPU stacks. Ctrl-C to end.\n");
}

kprobe:finish_task_switch
{
    // записать время простоя предыдущего потока
    $prev = (struct task_struct *)arg0;
    if ($1 == 0 || $prev->tgid == $1) {
        @start[$prev->pid] = nsecs;
    }

    // получить время запуска текущего потока
    $last = @start[tid];
    if ($last != 0) {
        @[kstack, ustack, comm] = sum(nsecs - $last);
        delete(@start[tid]);
    }
}

END
{
    clear(@start);
}
```

Эта программа записывает отметку времени для потока, который оставляет CPU, а также суммирует время простоя для потока, который запускается, и использует единственный зонд `kprobe` в `finish_task_switch()`.

6.3.10. syscount

`syscount(8)`¹ — это инструмент для BCC и `bpfftrace`, подсчитывающий количество системных вызовов во всей системе в целом. Он включен в эту главу, потому что может стать отправной точкой для анализа случаев высокой загрузки процессоров.

В следующем примере показано применение версии `syscount(8)` для BCC с целью получить ежесекундную частоту системных вызовов (`-i 1`) в производственном экземпляре:

```
# syscount -i 1
Tracing syscalls, printing top 10... Ctrl+C to quit.
[00:04:18]
SYSCALL                      COUNT
futex                        152923
read                         29973
epoll_wait                   27865
write                        21707
epoll_ctl                    4696
poll                         2625
writev                       2460
recvfrom                     1594
close                        1385
sendto                       1343
[...]
```

Здесь показаны 10 наиболее часто используемых системных вызовов с отметкой времени. Чаще других вызывался `futex(2)` — в течение 1 секунды было сделано более 150 000 вызовов. Для дальнейшего исследования каждого системного вызова загляните на соответствующие им страницы справочного руководства `man` и используйте другие инструменты BPF для трассировки и проверки их аргументов (например, `trace(8)` для BCC или однострочники для `bpfftrace`). В некоторых ситуациях можно использовать `strace(1)`, чтобы быстро понять, как используется этот системный вызов. Но имейте в виду, что текущая реализация `strace(1)` на основе `ptrace` может замедлить производительность целевого приложения в сто и более раз, что, в свою очередь, может вызвать серьезные проблемы в производственных средах (например, превышение задержек, регламентируемых уровнем обслуживания, или запуск аварийного переключения). `strace(1)` следует рассматривать как последнее средство после попыток использовать инструменты BPF.

Для подсчета системных вызовов по идентификаторам процессов можно использовать параметр `-P`:

```
# syscount -Pi 1
Tracing syscalls, printing top 10... Ctrl+C to quit.
[00:04:25]
PID      COMM                      COUNT
```

¹ Немного истории: первую версию, основанную на `Ftrace` и `perf(1)`, я написал для коллекции `perf-tools` 7 июля 2014 года, а 15 февраля 2017 года Саша Гольдштейн выпустил версию для BCC.

3622	java	294783
990	snmpd	124
2392	redis-server	64
4790	snmp-pass	32
27035	python	31
26970	sshd	24
2380	svscan	11
2441	atlas-system-ag	5
2453	apache2	2
4786	snmp-pass	1
[...]		

Процесс java выполнил за 1 секунду почти 300 000 системных вызовов. Другие инструменты помогли выяснить, что на это было потрачено всего 1.6% системного времени в системе с 48 процессорами.

В своей работе этот инструмент использует точку трассировки `raw_syscalls:sys_enter` вместо привычных `syscalls:sys_enter_*`. Использование этой единой точки трассировки, которая может видеть все системные вызовы, ускоряет начальную инструментацию. Но она имеет свой недостаток: ей доступны только идентификаторы системных вызовов, которые нужно дополнительно преобразовать в имена. Для этого BCC предоставляет библиотечный вызов `syscall_name()`.

Оверхед этого инструмента может оказаться существенным при очень высокой частоте системных вызовов. Ради интереса я выполнил нагрузочное тестирование в однопроцессорной системе, обеспечив частоту системных вызовов в 3.2 миллиона в секунду. Во время работы `syscount(8)` производительность рабочей нагрузки уменьшилась на 30%. Эти цифры помогают оценить оверхед в производственной среде: экземпляр с 48 процессорами и с частотой системных вызовов 300 000 в секунду выполняет примерно 6000 системных вызовов в секунду на каждом процессоре, поэтому ожидаемое падение производительности составит 0.06% ($30\% \times 6250 / 3\,200\,000$). Я пытался измерить это падение непосредственно в производственной среде, но его величина оказалась слишком мала и неразличима на фоне меняющейся рабочей нагрузки.

BCC

Порядок использования:

```
syscount [options] [-i interval] [-d duration]
```

Параметры `options`:

- **-T TOP**: выводить указанное число наиболее частых системных вызовов;
- **-L**: выводить общее время (задержку), проведенное в системных вызовах;
- **-P**: выполнять подсчет по процессам;
- **-p PID**: трассировать только этот процесс.

Пример применения параметра `-L` вы увидите в главе 13.

bpftrace

Есть версия `syscount(8)` для `bpftrace`, обладающая основными функциональными возможностями инструмента. Но можно использовать и следующий однострочный сценарий:

```
# bpftrace -e 't:syscalls:sys_enter_* { @[probe] = count(); }'
Attaching 316 probes...
^C
```

```
[...]
@[tracepoint:syscalls:sys_enter_ioctl]: 9465
@[tracepoint:syscalls:sys_enter_epoll_wait]: 9807
@[tracepoint:syscalls:sys_enter_gettid]: 10311
@[tracepoint:syscalls:sys_enter_futex]: 14062
@[tracepoint:syscalls:sys_enter_recvmmsg]: 22342
```

Здесь инструментятся точки трассировки всех 316 системных вызовов (в этой версии ядра), а подсчет частоты осуществляется по имени зонда. Сейчас во время запуска программы возникает некоторая задержка из-за необходимости инструментировать все 316 точек трассировки. Предпочтительнее было бы использовать одну точку трассировки `raw_syscalls:sys_enter`, как это делает ВСС, но в этом случае придется дополнительно преобразовывать идентификаторы системных вызовов в их имена. Эта задача включена в качестве упражнения в главу 14.

6.3.11. `argdist` и `trace`

Инструменты `argdist(8)` и `trace(8)` были представлены в главе 4. Они оба — инструменты для ВСС и применяются для исследования событий нестандартными способами. Эти инструменты можно использовать как дополнение к `syscount(8)` для дальнейшего исследования системных вызовов, обращения к которым происходят особенно часто.

Например, в предыдущем выводе `syscount(8)` мы видели системный вызов `read(2)` в числе наиболее частых. С помощью `argdist(8)` можно получить обобщенную информацию о его аргументах и возвращаемом значении, выполнив инструментацию точки трассировки системного вызова или функции ядра. Если вы решите использовать точку трассировки, найти имена аргументов поможет ВСС-инструмент `tplist(8)` с параметром `-v`:

```
# tplist -v syscalls:sys_enter_read
syscalls:sys_enter_read
    int __syscall_nr;
    unsigned int fd;
    char * buf;
    size_t count;
```

Аргумент `count` определяет размер блока для чтения. Мы можем использовать `argdist(8)`, чтобы обобщить значения этого аргумента в виде гистограммы (`-H`):

```
# argdist -H 't:syscalls:sys_enter_read():int:args->count'
[09:08:31]
  args->count      : count      distribution
    0 -> 1         : 169        *****
    2 -> 3         : 243        *****
    4 -> 7         : 1          |
    8 -> 15        : 0          |
   16 -> 31        : 384        *****
   32 -> 63        : 0          |
   64 -> 127       : 0          |
  128 -> 255       : 0          |
  256 -> 511       : 0          |
  512 -> 1023      : 0          |
 1024 -> 2047      : 267        *****
 2048 -> 4095      : 2          |
 4096 -> 8191      : 23         **          |
[...]
```

Как показывают эти результаты, во многих случаях данные читаются блоками размером в диапазоне от 16 до 31 байт, а также от 1024 до 2047 байт. Можно вызвать `argdist(8)` с параметром `-C` вместо `-H`, чтобы вместо гистограммы получить значения частот размеров.

Это пример показывает распределение величины запрашиваемого числа байтов для чтения, получаемого системным вызовом на входе. Сравним его с распределением величины возвращаемого значения, которое представляет фактически прочитанное число байтов:

```
# argdist -H 't:syscalls:sys_exit_read():int:args->ret'
[09:12:58]
  args->ret      : count      distribution
    0 -> 1       : 481        *****
    2 -> 3       : 116        *****
    4 -> 7       : 1          |
    8 -> 15      : 29         **          |
   16 -> 31      : 6          |
   32 -> 63      : 31         **          |
   64 -> 127     : 8          |
  128 -> 255     : 2          |
  256 -> 511     : 1          |
  512 -> 1023    : 2          |
 1024 -> 2047    : 13         *          |
 2048 -> 4095    : 2          |
[...]
```

Как видите, подавляющее большинство операций чтения вернуло ноль или 1 байт.

Благодаря тому что `argdist(8)` выполняет обобщение информации в пространстве ядра, этот инструмент с успехом можно использовать для исследования часто вызываемых системных вызовов. `tracex(8)` выводит данные о каждом событии и хорошо подходит для изучения менее частых системных вызовов, показывая отметки времени для каждого события и т. д.

bpfttrace

Этот уровень анализа системных вызовов доступен также однострочным сценариям для `bpfttrace`. Например, выведем гистограмму с распределением величины запрашиваемого числа байтов:

```
# bpftrace -e 't:syscalls:sys_enter_read { @ = hist(args->count); }'
```

Attaching 1 probe...

 $\wedge C$

@:

[1]	1102	@@
[2, 4)	902	@@
[4, 8)	20	
[8, 16)	17	
[16, 32)	538	@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32, 64)	56	@@
[64, 128)	0	
[128, 256)	0	
[256, 512)	0	
[512, 1K)	0	
[1K, 2K)	119	@@@@
[2K, 4K)	26	@
[4K, 8K)	334	@@@@@@@@@@@@@@@@@@@@

и возвращаемого значения:

```
# bpftrace -e 't:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Attaching 1 probe...

 $\wedge C$

@:

(..., 0)	105	@@@@
[0]	18	
[1]	1161	@@@
[2, 4)	196	@@@@@@@@
[4, 8)	8	
[8, 16)	384	@@@@@@@@@@@@@@@@@@@@@@
[16, 32)	87	@@@
[32, 64)	118	@@@@@
[64, 128)	37	@
[128, 256)	6	
[256, 512)	13	
[512, 1K)	3	
[1K, 2K)	3	
[2K, 4K)	15	

В `bpfttrace` есть отдельный диапазон для отрицательных значений («(..., 0)»), которые представляют коды ошибок, возвращаемые системным вызовом `read(2)`. Можно написать свой однострочник `bpfttrace`, подсчитывающий коды ошибок (как показано в главе 5) или возвращающий линейную гистограмму, позволяющую увидеть их распределение:

```
# bpftrace -e 't:syscalls:sys_exit_read /args->ret < 0/ {
    @ = lhist(- args->ret, 0, 100, 1); }'
Attaching 1 probe...
^C

@:
[11, 12)                123 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
```

Как показывают эти результаты, `read(2)` всегда возвращал код ошибки 11. Заглянув в заголовки Linux (`asm-generic/errno-base.h`), можно узнать, что это за ошибка:

```
#define EAGAIN 11 /* Try again */
```

Код 11 соответствует ошибке «try again», которая считается обычным явлением.

6.3.12. funccount

Инструмент `funccount(8)`, представленный в главе 4, основан на ВСС и может подсчитывать частоту вызовов функций и других событий. Его можно использовать для получения дополнительной информации об использовании процессором ПО, чтобы узнать, какие функции вызываются и как часто. `profile(8)` может показать, какие функции потребляют наибольшее процессорное время, но не может объяснить почему¹: выполняется ли функция слишком долго или просто вызывается миллионы раз в секунду.

Ниже показан пример подсчета частоты вызовов функций ядра, обрабатывающих протокол ТСР на высоконагруженном производственном экземпляре, путем сопоставления их имен с префиксом «tcp_»:

```
# funccount 'tcp_*'
Tracing 316 functions for "tcp_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
[...]
```

<code>tcp_stream_memory_free</code>	368048
<code>tcp_established_options</code>	381234
<code>tcp_v4_md5_lookup</code>	402945
<code>tcp_gro_receive</code>	484571
<code>tcp_md5_do_lookup</code>	510322

```
Detaching...
```

Согласно полученным результатам, при трассировке чаще других вызывалась функция `tcp_md5_do_lookup()` — 510 000 раз.

¹ Выяснить причину с помощью `profile(8)` непросто. Профилировщики, включая `profile(8)`, выбирают указатель инструкций процессора. Сравнивая значения указателя с дизассемблированным выводом функции, можно выяснить, зациклилась ли функция или просто вызывается очень много раз. Но сделать это на практике сложнее, чем кажется: см. раздел 2.12.2 в главе 2.

Добавив параметр `-i`, можно получить поинтервальный вывод. Так, в примере использования `profile(8)`, приводившемся выше, мы выяснили, что функция `get_page_from_freelist()` использовала значительный объем вычислительных ресурсов. Чем это обусловлено? Она выполняется слишком медленно или просто вызывается очень часто? Попробуем это выяснить, выполнив измерения с секундным интервалом:

```
# funcccount -i 1 get_page_from_freelist
Tracing 1 functions for "get_page_from_freelist"... Hit Ctrl-C to end.
FUNC                                COUNT
get_page_from_freelist              586452

FUNC                                COUNT
get_page_from_freelist              586241
[...]
```

Как показывают результаты, функция вызывается больше полумиллиона раз в секунду.

В своей работе `funcccount(8)` использует метод динамической трассировки функций: зонды `kprobes` для функций ядра и `uprobes` для функций в пространстве пользователя (`kprobes` и `uprobes` описаны в главе 2). Оверхед инструмента зависит от частоты вызовов функций. Некоторые функции, например `malloc()` и `get_page_from_freelist()`, вызываются очень часто, поэтому их трассировка может значительно замедлить целевое приложение, иногда более чем на 10%. Так что используйте его с осторожностью. См. раздел 18.1 в главе 18, где приводится дополнительная информация об оверхеде.

Порядок использования:

```
funcccount [options] [-i interval] [-d duration] pattern
```

Параметры `options`:

- **-r**: использовать регулярные выражения для сопоставления с шаблоном;
- **-p PID**: трассировать только этот процесс.

Шаблоны:

- **name** или **p:name**: инструментировать функцию ядра с именем `name()`;
- **lib:name**: инструментировать функцию в пространстве пользователя с именем `name()`, в библиотеке `lib`;
- **path:name**: инструментировать функцию в пространстве пользователя с именем `name()`, в файле `path`;
- **t:system:name**: инструментировать точку трассировки с именем `system:name`;
- *****: подстановочный символ, соответствующий любой строке.

Дополнительные примеры ищите в разделе 4.5 главы 4.

bpfttrace

Основные функциональные возможности `funccount(8)` можно реализовать в виде однострочного сценария для `bpfttrace`:

```
# bpfttrace -e 'k:tcp_* { @[probe] = count(); }'
Attaching 320 probes...
[...]
@[kprobe:tcp_release_cb]: 153001
@[kprobe:tcp_v4_md5_lookup]: 154896
@[kprobe:tcp_gro_receive]: 177187
```

В него можно добавить поддержку интервального вывода:

```
interval:s:1 { print(@); clear(@); }
```

Как и при использовании инструмента для ВСС, соблюдайте осторожность при трассировке часто вызываемых функций — это может привести к значительному оверхеду.

6.3.13. softirqs

`softirqs(8)` — это инструмент для ВСС, который показывает время, затраченное на обработку программных прерываний. Это время легко узнать с помощью разных инструментов. Например, `mpstat(1)` выводит его в столбце `%soft`. Есть и псевдофайл `/proc/softirqs`, откуда можно прочесть количество программных прерываний. Инструмент `softirqs(8)` отличается тем, что может сообщить время на обработку каждого типа программных прерываний. Вот пример 10-секундной трассировки производственного экземпляра с 48 процессорами:

```
# softirqs 10 1
Tracing soft irq event time... Hit Ctrl-C to end.
SOFTIRQ          TOTAL_usecs
net_tx           633
tasklet          30939
rcu              143859
sched            185873
timer            389144
net_rx           1358268
```

Согласно этим результатам, наибольшее время было потрачено на обработку прерывания `net_rx`, всего 1358 миллисекунд. Это довольно много — 3% процессорного времени в 48-процессорной системе.

В своей работе `softirqs(8)` использует точки трассировки `irq:softirq_enter` и `irq:softirq_exit`. Оверхед инструмента зависит от частоты событий, которая может быть довольно высокой в высоконагруженных системах и с высокой частотой следования сетевых пакетов. Будьте осторожны и обязательно оцените оверхед.

Порядок использования:

```
softirqs [options] [interval [count]]
```

Параметры `options`:

- **-d**: вывести распределение времени обработки прерываний в виде гистограмм;
- **-T**: добавить отметки времени в вывод.

Параметр `-d` можно использовать для изучения распределения и выявления задержек, возникающих при обработке прерываний.

bpfttrace

Версии `softirqs(8)` для `bpfttrace` нет, но ее легко реализовать. За отправную точку можно взять этот однострочный сценарий, подсчитывающий программные прерывания по номеру вектора:

```
# bpfttrace -e 'tracepoint:irq:softirq_entry { @[args->vec] = count(); }'
Attaching 1 probe...
^C

@[3]: 11
@[6]: 45
@[0]: 395
@[9]: 405
@[1]: 524
@[7]: 561
```

Номера векторов можно преобразовать в имена прерываний точно так же, как это делает инструмент для ВСС: с помощью таблицы поиска. Для определения времени, затраченного на обработку программных прерываний, используется точка трассировки `irq:softirq_exit`.

6.3.14. hardirqs

`hardirqs(8)`¹ — это инструмент для ВСС, который показывает время, затраченное на обработку аппаратных прерываний. Это время легко узнать с помощью разных инструментов. Например, `mpstat(1)` выводит его в столбце `%irq`. Есть и псевдофайл `/proc/interrupts`, откуда можно прочесть количество аппаратных прерываний. Инструмент `hardirqs(8)` отличается тем, что может сообщить время на обработку каждого типа аппаратных прерываний.

¹ Немного истории: первую версию — `inttimes.d` — для вывода суммарного времени, затраченного на обработку каждого прерывания, я создал 28 июня 2005 года. А 9 мая 2005 года, для вывода гистограмм, я создал инструмент `intoncpu.d`, основанный на `intr.d` из руководства «Dynamic Tracing Guide», изданного в январе 2005 года [Sun 05]. Я также разработал инструмент для `DTrace`, отображающий информацию об обработке прерываний на каждом процессоре, но не стал переносить его в BPF, потому что в Linux есть `/proc/interrupts` для этой задачи. Эту версию для ВСС, которая суммирует и выводит гистограммы, я разработал 20 октября 2015 года.

Вот результаты трассировки производственного экземпляра с 48 процессорами в течение 10 секунд:

```
# hardirqs 10 1
Tracing hard irq event time... Hit Ctrl-C to end.
HARDIRQ          TOTAL_usecs
ena-mgmt@pci:0000:00:05.0      43
nvme0q0              46
eth0-Tx-Rx-7           47424
eth0-Tx-Rx-6           48199
eth0-Tx-Rx-5           48524
eth0-Tx-Rx-2           49482
eth0-Tx-Rx-3           49750
eth0-Tx-Rx-0           51084
eth0-Tx-Rx-4           51106
eth0-Tx-Rx-1           52649
```

Согласно этому выводу, за 10 секунд трассировки на обработку всех аппаратных прерываний с именами `eth0-Tx-Rx*` ушло около 50 миллисекунд.

`hardirqs(8)` позволяет увидеть некоторые аспекты использования CPU, недоступные профилировщикам. Дополнительную информацию о профилировании облачных экземпляров, не имеющих аппаратных PMU, вы найдете в разделе 6.2.4, в подразделе «Внутреннее устройство».

В своей работе этот инструмент использует прием динамической трассировки функции ядра `handle_irq_event_percpu()`, но будущие версии будут применять точки трассировки `irq:irq_handler_entry` и `irq:irq_handler_exit`.

Порядок использования:

```
hardirqs [options] [interval [count]]
```

Параметры `options`:

- **-d**: вывести распределение времени обработки прерываний в виде гистограмм;
- **-T**: добавить отметки времени в вывод.

Параметр `-d` можно использовать для изучения распределения и выявления задержек, возникающих при обработке прерываний.

6.3.15. smpcalls

`smpcalls(8)`¹ — это инструмент `bpfttrace` для отслеживания и суммирования времени в функциях вызовов SMP (также известных как перекрестные вызовы). Поддержка

¹ Немного истории: инструмент `smpcalls.bt` я написал специально для этой книги 23 января 2019 года. Его название происходит от названия моего более раннего инструмента `xcallsbypid.d` (которое подразумевает перекрестные вызовы CPU), созданного мной 17 сентября 2005 года.

таких вызовов позволяет одному процессору запускать функции на любых других процессорах и может приводить к сильному оверхеду в больших многопроцессорных системах. Вот пример трассировки в системе с 36 процессорами:

[illegible]

Первый же запуск этого инструмента выявил проблему: перекрестный вызов `aperfperf_snapshot_khz` происходит довольно часто и выполняется медленно — до 128 микросекунд.

Вот исходный код `smprcalls(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing SMP calls. Hit Ctrl-C to stop.\n");
}

kprobe:smp call function single,
```

```

kprobe:smp_call_function_many
{
    @ts[tid] = nsecs;
    @func[tid] = arg1;
}

kretprobe:smp_call_function_single,
kretprobe:smp_call_function_many
/@@ts[tid]/
{
    @time_ns[ksym(@func[tid])] = hist(nsecs - @ts[tid]);
    delete(@ts[tid]);
    delete(@func[tid]);
}

kprobe:native_smp_send_reschedule
{
    @ts[tid] = nsecs;
    @func[tid] = reg("ip");
}

kretprobe:native_smp_send_reschedule
/@@ts[tid]/
{
    @time_ns[ksym(@func[tid])] = hist(nsecs - @ts[tid]);
    delete(@ts[tid]);
    delete(@func[tid]);
}

END
{
    clear(@ts);
    clear(@func);
}

```

Многие из SMP-вызовов можно трассировать с помощью зондов kprobes для функций ядра `smp_call_function_single()` и `smp_call_function_many()`. Во втором аргументе этим функциям передается указатель на функцию для запуска на удаленном CPU, который в `bpftrace` доступен как `arg1`. Этот указатель сохраняется в карте `@func` с ключом — идентификатором потока и потом используется в `kretprobe` для поиска, где также преобразуется в удобочитаемое имя с помощью встроенной в `bpftrace` функции `ksym()`.

Но есть особый SMP-вызов, не охватываемый этими функциями, `smp_send_reschedule()`, который трассируется с помощью `native_smp_send_reschedule()`. Надеюсь, что будущая версия ядра станет поддерживать точки трассировки SMP-вызовов, более простые в использовании.

Ключ гистограммы `@time_ns` можно изменить и включить в него трассировку стека ядра и имя процесса:

```
@time_ns[comm, kstack, ksym(@func[tid])] = hist(nsecs - @ts[tid]);
```

Это позволит получить более подробные сведения о медленном вызове:

```
@time_ns[snmp-pass,
    smp_call_function_single+1
    aperfmperf_snapshot_cpu+90
    arch_freq_prepare_all+61
    cpuinfo_open+14
    proc_reg_open+111
    do_dentry_open+484
    path_openat+692
    do_filp_open+153
    do_sys_open+294
    do_syscall_64+85
    entry_SYSCALL_64_after_hwframe+68
, aperfmperf_snapshot_khz]:
[2K, 4K)          2 |@@
[4K, 8K)          0 |
[8K, 16K)         1 |@
[16K, 32K)        1 |@
[32K, 64K)       51 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64K, 128K)      17 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

В этом выводе видно, что процесс snmp-pass, агент мониторинга, выполнил системный вызов open(), который заканчивается вызовом cpuinfo_open() и дорогостоящим перекрестным вызовом.

Запуск другого инструмента BPF — opensnoop(8) — подтвердил это поведение:

```
# opensnoop.py -Tn snmp-pass
TIME(s)      PID    COMM          FD ERR PATH
0.000000000  2440  snmp-pass     4   0 /proc/cpuinfo
0.000841000  2440  snmp-pass     4   0 /proc/stat
1.022128000  2440  snmp-pass     4   0 /proc/cpuinfo
1.024696000  2440  snmp-pass     4   0 /proc/stat
2.046133000  2440  snmp-pass     4   0 /proc/cpuinfo
2.049020000  2440  snmp-pass     4   0 /proc/stat
3.070135000  2440  snmp-pass     4   0 /proc/cpuinfo
3.072869000  2440  snmp-pass     4   0 /proc/stat
[...]
```

Этот вывод показывает, что snmp-pass читает файл /proc/cpuinfo каждую секунду! Большинство деталей в этом файле не меняется, кроме поля «CPU MHz».

Исследование исходного кода показало, что это ПО читает /proc/cpuinfo только для того, чтобы узнать количество процессоров. Поле «CPU MHz» им вообще не анализируется. Это пример выполнения ненужной работы, устранение которой должно обеспечить небольшой, но легкий выигрыш.

На процессорах Intel эти SMP-вызовы реализованы как вызовы x2APIC IPI (Inter-Processor Interrupt — межпроцессорные прерывания), включая x2apic_send_IPI(). Эти функции тоже можно инструментировать, как показано в разделе 6.4.2.

6.3.16. llcstat

llcstat(8)¹ — это инструмент для ВСС, использующий счетчики РМС для отображения частоты промахов и попаданий в кэш последнего уровня (Last-Level Cache, LLC) по процессам. Счетчики РМС были представлены в главе 2.

Вот пример использования llcstat(8) на производственном экземпляре с 48 процессорами:

```
# llcstat
Running for 10 seconds or hit Ctrl-C to end.
PID      NAME      CPU      REFERENCE      MISS      HIT%
0         swapper/15   15       1007300        1000     99.90%
4435      java        18       22000          200     99.09%
4116      java         7       11000          100     99.09%
4441      java        38       32200          300     99.07%
17387     java        17       10800          100     99.07%
4113      java        17       10500          100     99.05%
[...]
```

Этот вывод показывает, что процессы java (потoki) при выполнении имели очень высокую долю попаданий в кэш, более 99%.

В своей работе этот инструмент следит за переполнением счетчиков РМС и запускает программу BPF для чтения текущего процесса и записи статистики, когда происходит заданное число ссылок или промахов в каждый из множества кэшей. По умолчанию порог равен 100 и доступен для настройки с помощью параметра -с. Такая выборка «один на сотню» помогает снизить оверхед (и при необходимости может быть настроена на большее число). Тем не менее у такого подхода есть и проблемы. Например, процесс может случайно переполнить счетчик промахов раньше, чем счетчик ссылок, что не учитывается инструментом (потому что промахи — это подмножество ссылок).

Порядок использования:

```
llcstat [options] [duration]
```

Параметры options:

- **-c SAMPLE_PERIOD**: производить выборку один раз за указанное число событий.

llcstat(8) интересен тем, что это первый инструмент в ВСС, использующий счетчики РМС не для выборки по времени.

6.3.17. Другие инструменты

Другие инструменты BPF:

- **cpuwalk(8)** из числа примеров в проекте bpftrace, который определяет процессы, выполняющиеся на процессорах, и выводит результат в виде линейной

¹ Немного истории: этот инструмент создал Тен Цинь (Teng Qin) 19 октября 2016 года. Это первый инструмент в ВСС, использующий счетчики РМС.

гистограммы. Это позволяет получить представление о балансировке процессоров.

- **cpuunclaimed(8)** из проекта BCC — экспериментальный инструмент, который производит выборку длин очередей на выполнение и определяет, как часто возникает ситуация, когда процессоры простаивают, а потоки ожидают в очереди на выполнение другого процессора. Такое может происходить из-за привязки процессов к процессорам, но если это случается слишком часто, то это признак неправильной конфигурации планировщика или ошибки.
- **load(8)** из проекта bpftrace — пример получения средних значений загрузки из инструмента BPF. Как обсуждалось выше, эти цифры могут вводить в заблуждение.
- **vltrace** — инструмент от Intel. Это версия strace(1) на основе BPF, которую можно использовать для дальнейшего исследования особенностей системных вызовов, потребляющих процессорное время [79].

6.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечисляются однострочные сценарии для BCC и bpftrace. Там, где это возможно, один и тот же сценарий реализуется с использованием BCC и bpftrace.

6.4.1. BCC

Трассирует запуск новых процессов и их аргументы:

```
execsnoop
```

Сообщает, кто и что выполняет:

```
trace 't:syscalls:sys_enter_execve "-> %s", args->filename'
```

Выводит число системных вызовов, выполненных каждым процессом:

```
syscount -P
```

Выводит число обращений к каждому системному вызову:

```
syscount
```

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
profile -F 49 -U -p 189
```

Выбирает все трассировки стека и имена процессов:

```
profile
```

Подсчитывает число вызовов функций ядра с именами, начинающимися на «vfs_»:

```
funccount 'vfs_*
```

Трассирует запуск новых потоков вызовом pthread_create():

```
trace /lib/x86_64-linux-gnu/libpthread-2.27.so:pthread_create
```

6.4.2. bpftrace

Трассирует запуск новых процессов и их аргументы:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Сообщает, кто и что выполняет:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { printf("%s -> %s\n", comm, str(args->filename)); }'
```

Выводит число системных вызовов, выполненных каждой программой:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
```

Выводит число системных вызовов, выполненных каждым процессом:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Выводит число обращений к каждому системному вызову, используя его зонд:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Выводит число обращений к каждому системному вызову, используя имя функции:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
    @[sym(*(kaddr("sys_call_table") + args->id * 8))] = count(); }'
```

Выбирает имена работающих процессов с частотой 99 Гц:

```
bpftrace -e 'profile:hz:99 { @[comm] = count(); }'
```

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Выбирает все трассировки стека и имена процессов:

```
bpftrace -e 'profile:hz:49 { @[ustack, stack, comm] = count(); }'
```

Выбирает работающие процессоры с частотой 99 Гц и выводит собранную информацию в виде линейной гистограммы:

```
bpftrace -e 'profile:hz:99 { @cpu = lhist(cpu, 0, 256, 1); }'
```

Подсчитывает число вызовов функций ядра с именами, начинающимися на «vfs_»:

```
bpftrace -e 'kprobe:vfs_* { @[func] = count(); }'
```

Подсчитывает SMP-вызовы по именам и трассировкам стека ядра:

```
bpftrace -e 'kprobe:smp_call* { @[probe, kstack(5)] = count(); }'
```

Подсчитывает вызовы Intel x2APIC по именам и трассировкам стека ядра:

```
bpftrace -e 'kprobe:x2apic_send_IPI* { @[probe, kstack(5)] = count(); }'
```

Трассирует запуск новых потоков вызовом `pthread_create()`:

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpthread-2.27.so:pthread_create {  
    printf("%s by %s (%d)\n", probe, comm, pid); }'
```

6.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Упражнения можно выполнить с помощью `bpftrace` или ВСС, если явно не указано иное:

1. Используйте `hexcsnoop(8)`, чтобы показать новые процессы, запускаемые командой `man ls`.
2. Запустите `hexcsnoop(8)` с параметром `-t`, отправьте вывод в файл журнала и продолжайте трассировку в течение 10 минут в производственной или локальной системе. Какие вновь запущенные процессы вы увидели?
3. В тестовой системе сгенерируйте высокую нагрузку на процессор. Создайте для этого два потока, привязанных к процессору, например к CPU 0:

```
taskset -c 0 sh -c 'while ;; do ;; done' &  
taskset -c 0 sh -c 'while ;; do ;; done' &
```

Теперь используйте `uptime(1)` (чтобы определить средние значения загрузки), `mpstat(1)` (`-P ALL`), `runqlen(8)` и `runqlat(8)`, чтобы охарактеризовать рабочую нагрузку на CPU 0. (Не забудьте остановить потоки по окончании.)

4. Разработайте инструмент или однострочный сценарий для выборки трассировок стека ядра только на CPU 0.
5. Используйте `profile(8)` для захвата трассировок стека ядра, чтобы определить, сколько процессорного времени расходуется на следующую рабочую нагрузку:

```
dd if=/dev/nvme0n1p3 bs=8k iflag=direct | dd of=/dev/null bs=1
```

В параметре `if=` укажите локальный диск (выполните `df -h`, чтобы получить список кандидатов). Вы можете выполнить профилирование в масштабе всей системы или каждый процесс `dd(1)` в отдельности.

6. Сгенерируйте флейм-график использования процессора, используя вывод, полученный в упражнении 5.
7. Используйте `offcrutime(8)` для захвата трассировок стеков ядра, чтобы определить, какие функции блокируют выполнение рабочей нагрузки в упражнении 5.
8. Сгенерируйте флейм-график времени вне процессора, используя вывод, полученный в упражнении 7.
9. `exesnoop(8)` видит только новые процессы, которые запускаются вызовом `exec(2)` (`execve(2)`), тогда как некоторые процессы могут запускаться вызовом `fork(2)` или `clone(2)`, без `exec(2)` (например, так запускаются однопоточные рабочие процессы). Напишите новый инструмент — `procsnoop(8)`, который может обнаруживать запуск новых процессов и выводить максимально возможное количество деталей. Для этого можно организовать трассировку `fork()` и `clone()`, использовать точки трассировки `sched` или поступить как-то иначе.
10. Разработайте версию `softirqs(8)` для `bpfttrace`, которая будет выводить имена программных прерываний.
11. Разработайте версию `cpudist(8)` для `bpfttrace`.
12. Используя `cpudist(8)` (любую версию), выведите гистограммы отдельно для намеренных и принудительных переключений контекста.
13. (Продвинутый уровень, не решено.) Разработайте инструмент для вывода гистограммы времени, потраченного задачами в ожидании доступа к своим процессорам, пока другие процессоры простаивают, но не переносятся из-за «перегрева» кэша (см. `kernel.sched_migration_cost_ns`, `task_hot()` — которая может быть встроенной и недоступной для трассировки, и `can_migrate_task()`).

6.6. ИТОГИ

В этой главе я рассказал, как процессоры используются в системах и как анализировать их работу с помощью традиционных инструментов: статистик, профилировщиков и трассировщиков. Я также показал, как использовать инструменты BPF для выявления проблем с короткоживущими процессами, детального изучения задержек в очередях на выполнение, профилирования эффективности использования CPU, а также подсчета вызовов функций и времени, потраченного на обработку программных и аппаратных прерываний.

Глава 7

ПАМЯТЬ

Linux — это система, основанная на виртуальной памяти, где каждый процесс имеет свое виртуальное адресное пространство. Она выполняет отображение в физические адреса по мере необходимости. Ее дизайн допускает избыточную подписку на физическую память, которой Linux управляет с помощью демона выгрузки страниц и физических устройств подкачки, а также имеет компонент ядра Out-Of-Memory Killer, который останавливает процессы при критической нехватке памяти. Linux использует свободную память в качестве кэша файловой системы, об этом пойдет речь в главе 8.

В этой главе мы увидим, как BPF позволяет по-новому взглянуть на использование памяти приложениями и понять, как ядро реагирует на нехватку памяти. Поскольку производительность процессоров росла быстрее, чем производительность памяти, операции с памятью стали новым узким местом. Понимание того, как используется память, помогает получить дополнительный выигрыш в производительности.

Цели обучения:

- познакомиться с особенностями распределения памяти и механизма подкачки;
- исследовать стратегии успешного анализа использования памяти с помощью трассировщиков;
- применять традиционные инструменты для исследования использования памяти;
- применять инструменты BPF для выявления путей в коде, ведущих к увеличению потребления динамической памяти и размера резидентного набора (RSS);
- определять сбой страниц по именам файлов и трассировкам стека;
- познакомиться с особенностями поведения сканера виртуальной машины;
- определять, как влияет на производительность восстановление памяти;
- выявлять процессы, простаивающие в ожидании подкачки страниц;
- использовать однострочные сценарии bpftrace для исследования параметров использования памяти нестандартными способами.

В начале этой главы обсудим некоторые теоретические основы, необходимые для анализа памяти, уделив внимание использованию памяти в приложениях. Затем

поговорим о распределении виртуальной и физической памяти, а также о подкачке страниц. Попутно исследуем вопросы, на которые может ответить BPF, а также общую стратегию анализа. Затем перейдем к традиционным инструментам анализа памяти и рассмотрим инструменты BPF, включая однострочные сценарии для BPF. В конце главы вас ждут дополнительные упражнения. Дополнительные инструменты анализа потребления памяти будут представлены в главе 14.

7.1. ОСНОВЫ

В этом разделе рассмотрим основы управления памятью, познакомимся с возможностями BPF и оптимальной стратегией анализа потребления памяти.

7.1.1. Основы управления памятью

Механизмы распределения памяти

На рис. 7.1 показаны наиболее часто используемые механизмы управления памятью для ПО в пространстве пользователя и ядра. Процессы, управляющие памятью с помощью библиотеки `libc`, получают память из динамического сегмента виртуального адресного пространства процесса, который называется *кучей* (*heap*). Библиотека `libc` предоставляет функции для управления распределением памяти, включая `malloc()` и `free()`. Когда процесс освобождает блок памяти, `libc` запоминает его местоположение и использует эту информацию в последующих вызовах `malloc()`. `libc` увеличивает размер кучи только после исчерпания доступной памяти. Уменьшать размер кучи обычно не требуется, потому что это виртуальная, а не реальная физическая память.

За отображение виртуальных адресов в физические отвечает ядро и процессор. Для эффективности отображение адресов осуществляется группами, или *страницами*, где размер каждой страницы зависит от аппаратной архитектуры процессора. Обычно используются страницы размером 4 Кбайт, хотя многие процессоры поддерживают страницы большего размера — в Linux их называют *огромными страницами* (*huge pages*). Ядро может обслуживать запросы на получение страниц физической памяти из своих собственных списков свободной памяти, которые поддерживает для каждой группы микросхем ОЗУ (DRAM) и CPU. Само ядро тоже потребляет память из этих списков, обычно используя свой механизм управления памятью — распределитель блоков (*slab allocator*).

В числе других библиотек управления памятью в пространстве пользователя можно назвать `tcmalloc` и `jemalloc`. Среды времени выполнения, например JVM, тоже часто предоставляют свой механизм управления памятью со средствами сборки мусора. Другие механизмы управления могут также отображать частные сегменты для распределения памяти вне кучи.

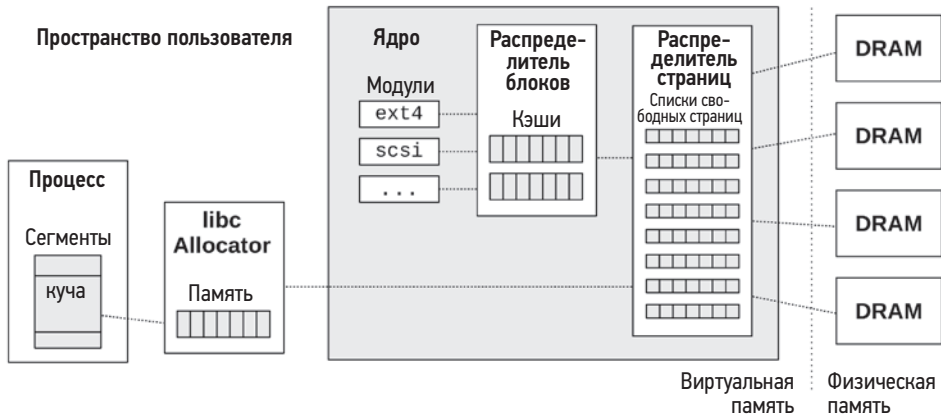


Рис. 7.1. Механизмы управления памятью

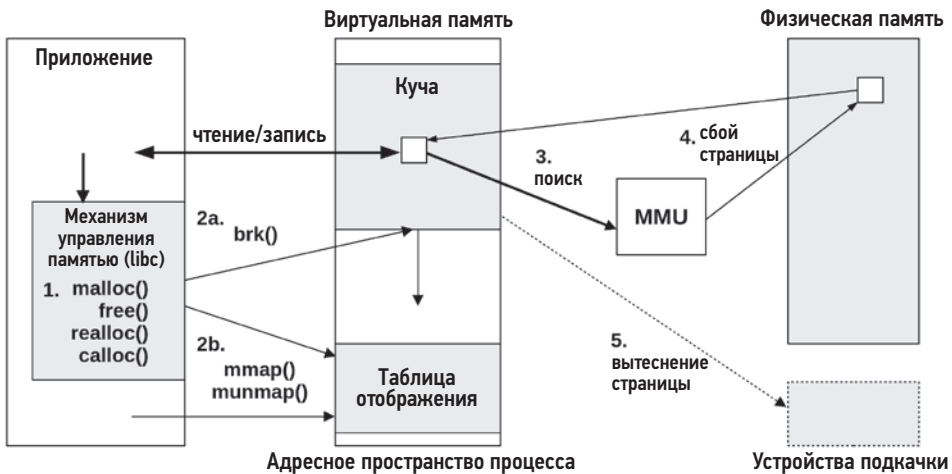


Рис. 7.2. Жизненный цикл страниц памяти

Страницы памяти и подкачка

На рис. 7.2 показан жизненный цикл типичной страницы памяти в пространстве пользователя. Его этапы:

1. Приложение запрашивает блок памяти (например, вызывает функцию malloc() из библиотеки libc).

2. Библиотека может сразу же выделить память из своего списка свободных страниц или сначала обратиться к системе для увеличения объема доступной ей виртуальной памяти. В зависимости от ситуации, библиотека:
 - а) увеличит размер кучи обращением к системному вызову `brk()` и использует дополнительную полученную память для удовлетворения запроса;
 - б) создаст новый сегмент обращением к системному вызову `mmap()`.
3. Спустя какое-то время приложение может попытаться использовать выделенный диапазон памяти с помощью инструкций записи и чтения, вовлекающих в работу блок управления памятью (Memory Management Unit, MMU) для преобразования виртуальных адресов в физические. И тогда вскрывается ложная природа виртуальной памяти: виртуальным адресам не соответствует никакая физическая память! Это приводит к ошибке MMU — сбою страницы (page fault).
4. Сбой страницы обрабатывается ядром, которое устанавливает соответствие между виртуальными адресами и адресами из своих списков свободных блоков физической памяти и затем информирует MMU об этом соответствии для использования в будущем. Затем процесс использует дополнительную страницу физической памяти. Такой объем физической памяти называется размером резидентного набора (Resident Set Size, RSS).
5. С увеличением спроса на память в работу включается демон подкачки (kswapd). Он отыскивает страницы физической памяти, которые можно освободить или вытеснить на устройство подкачки. Освободить можно страницы трех типов (на рис. 7.2 приведен только третий тип (с), так как рисунок иллюстрирует жизненный цикл страниц памяти в пространстве пользователя):
 - а) страницы файловой системы, которые были прочитаны с диска и не изменялись (их называют «зарезервированными на диске»): их можно удалить немедленно и при необходимости просто прочитать повторно. Эти страницы могут хранить выполняемый код приложений, данные и метаданные файловой системы;
 - б) страницы файловой системы, которые были изменены: это так называемые «грязные» страницы, которые следует записать на диск перед освобождением;
 - с) страницы памяти приложения: их называют анонимной памятью, потому что у них нет источника в файловой системе. Если в системе есть устройство подкачки, эти страницы можно освободить, предварительно сохранив на устройстве подкачки. Операция записи страниц на устройство подкачки называется *подкачкой* (в Linux).

Запросы на выделение памяти обычно следуют друг за другом очень часто: выделение памяти в пространстве пользователя может происходить миллионы раз в секунду. Операции чтения и записи, а также поиск соответствий в MMU происходят еще чаще — до нескольких миллиардов раз в секунду. На рис. 7.2 эти запросы показаны жирными стрелками. Другие операции — вызовы `brk()` и `mmap()`, сбой

страниц и вытеснение страниц — выполняются относительно редко (показаны тонкими стрелками).

Демон подкачки

Демон подкачки (kswapd) периодически ищет наиболее давно использовавшиеся (Least Recently Used, LRU) активные и неактивные страницы для освобождения. Он активируется, когда объем свободной памяти достигает нижнего предела, а затем переходит в фоновый режим, когда этот объем пересекает верхний предел, как показано на рис. 7.3.

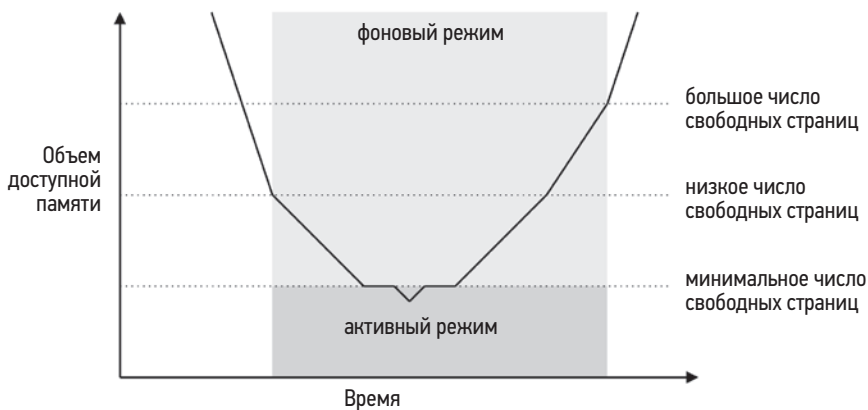


Рис. 7.3. Режимы работы демона kswapd

В фоновом режиме kswapd действует с минимальным приоритетом. За исключением конкуренции за процессор и выполнение операций дискового ввода/вывода, он не должен напрямую влиять на производительность приложений. Если kswapd не может освободить память достаточно быстро и объем доступной памяти достигает нижнего порога, то демон переходит в активный режим работы, стремясь освободить память для удовлетворения запросов на ее распределение. В этом режиме выделения памяти блокируются (зависают) и синхронно ждут освобождения страниц [Gorman 04] [81].

Прямое восстановление памяти может приводить к вызову функций модуля ядра shrinker: они освобождают память, которая могла храниться в кэшах, включая кэши ядра.

Устройства подкачки

Устройства подкачки обеспечивают ухудшенный режим работы системы в условиях нехватки памяти: процессы могут продолжать выделять память, но при этом реже используемые страницы начинают перемещаться между физической памятью

и устройствами подкачки туда и обратно, что приводит к сильному замедлению приложений. Некоторые производственные системы работают без устройств подкачки, так как ухудшенный режим работы для них неприемлем. Для таких критически важных систем проще и дешевле запустить множество избыточных (и исправных!) серверов, чем использовать устройство подкачки. (Это характерно, например, для облачных экземпляров Netflix.) Если системе без устройства подкачки не хватает памяти, механизм ядра OOM Killer жертвует процессом. Чтобы избежать этого, обычно приложения настраиваются так, чтобы никогда не превышать пределы доступной памяти системы.

OOM Killer

Механизм Linux OOM Killer — последнее средство освобождения памяти: он ищет процессы-жертвы, используя специальную эвристику, и жертвует ими, принудительно их завершая. Эвристика ищет самую большую и не самую важную жертву, которая освободит много страниц, такую как потоки ядра или процесс init (PID 1). Linux предоставляет возможность настройки поведения OOM Killer в масштабе всей системы и для каждого процесса в отдельности.

Сжатие страниц

Со временем растет фрагментация пространства свободных страниц, что затрудняет выделение большого непрерывного фрагмента, если это потребуется. Для предотвращения таких затруднений ядро использует процедуру сжатия (compaction), которая перемещает страницы для освобождения смежных областей [81].

Кэширование и буферизация в файловой системе

Linux активно заимствует свободную память для кэширования файловой системы и возвращает ее, когда возникает потребность в свободной памяти. Следствием такого заимствования является то, что объем свободной памяти, сообщаемый системой, стремится к нулю после загрузки Linux. Это может вызвать беспокойство у пользователя, хотя на самом деле система просто поддерживает кэш файловой системы в подогретом состоянии. Кроме того, файловая система использует память для буферизации в операциях записи.

Ядро Linux можно настроить так, чтобы оно предпочитало освобождать память, занятую кэшем файловой системы, или освобождало память путем вытеснения (vm.swappiness).

Дополнительно о кэшировании и буферизации мы поговорим в главе 8.

Для дальнейшего чтения

Это был лишь краткий обзор, чтобы снабдить вас минимальными знаниями перед использованием инструментов. Дополнительные темы, в том числе распределение

страниц ядра и NUMA, мы рассмотрим в главе 14. О распределении памяти и подкачке я подробнее рассказываю в главе 7 книги «Systems Performance»¹ [Gregg 13b].

7.1.2. Возможности BPF

Традиционные инструменты анализа производительности позволяют получить некоторое представление о распределении памяти. Например, они могут отображать распределение виртуальной и физической памяти и частоту операций со страницами. Эти традиционные инструменты кратко описываются в следующем разделе.

Инструменты трассировки BPF могут предоставить дополнительную информацию об операциях с памятью и ответить на вопросы:

- Почему продолжает увеличиваться объем физической памяти (RSS), потребляемой процессом?
- Какие пути в коде порождают сбой страниц? Для каких файлов?
- Какие процессы блокируются в ожидании завершения операций подкачки?
- Какие соответствия между виртуальной и физической памятью создаются во всей системе в целом?
- В каком состоянии находится система, когда активируется OOM Killer?
- Какие пути в коде приложений ведут к выделению дополнительной памяти?
- Какие типы объектов размещают приложения в памяти?
- Есть ли выделенные блоки памяти, которые не освобождаются через некоторое время? (Они могут указывать на потенциальные утечки.)

Чтобы получить ответы с помощью BPF, можно использовать программные события или точки трассировки сбоев и системных вызовов, kprobes для функций выделения памяти ядра, uprobes в библиотеках, средах выполнения и механизмах распределения памяти в приложениях, зонды USDT в механизмах распределения памяти libc, а также счетчики РМС для выборки обращений к памяти. Все эти источники событий можно смешивать в одной программе BPF для разделения контекста между различными системами.

BPF позволяет инструментировать события управления памятью, включая распределение, отображение, сбой страниц и подкачку, и извлекать трассировки стека, чтобы показать причины многих из этих событий.

Источники событий

В табл. 7.1 перечислены доступные для инструментации источники событий управления памятью.

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

Таблица 7.1. Источники событий управления памятью, доступные для инструментации

Типы событий	Источники событий
Выделение памяти в пространстве пользователя	Зонды <code>uprobes</code> для функций выделения памяти и зонды <code>USDT</code> в <code>libc</code>
Выделение памяти в пространстве ядра	Зонды <code>kprobes</code> для функций выделения памяти и точки трассировки <code>kmem</code>
Увеличение размера кучи	Точки трассировки системного вызова <code>brk</code>
Функции управления общей памятью	Точки трассировки системных вызовов
Сбои страниц	<code>kprobes</code> , программные события и точки трассировки исключений
Миграция страниц	Точки трассировки в механизме миграции
Сжатие страниц	Точки трассировки в механизме сжатия
Сканер VM	Точки трассировки <code>vmscan</code>
Циклы доступа к памяти	Счетчики <code>PMC</code>

Зонды `USDT`, доступные в `libc`:

```
# bpftrace -l usdt:/lib/x86_64-linux-gnu/libc-2.27.so
[...]
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_arena_max
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_arena_test
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_tunable_tcachе_max_bytes
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_tunable_tcachе_count
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_tunable_tcachе_unsorted_limit
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_trim_threshold
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_top_pad
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_mmap_threshold
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_mmap_max
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_perturb
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_heap_new
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_sbrk_less
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_arena_reuse
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_arena_reuse_wait
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_arena_new
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_arena_reuse_free_list
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_arena_retry
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_heap_free
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_heap_less
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_heap_more
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_sbrk_more
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_free_dyn_thresholds
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_malloc_retry
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_memalign_retry
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_realloc_retry
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_calloc_retry
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_mallocopt_mxfast
```


Используя их, можно получить представление о внутренней работе механизма распределения памяти в `libc`.

Оверхед

Как отмечалось выше, события управления памятью могут происходить миллионы раз в секунду. Конечно, программы BPF оптимизированы для быстрой работы, но их вызов миллионы раз в секунду может повлечь значительный оверхед и замедлить целевое ПО более чем на 10%, а в некоторых случаях даже в 10 раз, в зависимости от частоты трассируемых событий и особенностей программы BPF.

Чтобы было понятнее, как снизить оверхед, жирными стрелками на рис. 7.2 показаны наиболее частые события, а тонкими — более редкие. Ответы на многие вопросы об использовании памяти можно получить трассировкой более редких событий: сбоев страниц, вытеснения страниц, вызовов `brk()` и вызовов `mmap()`. Затраты на трассировку этих событий могут быть незначительными.

Часто трассировку вызовов `malloc()` производят, чтобы выяснить, какие пути в коде ведут к `malloc()`. Но эти пути можно выявить с использованием другого приема: производя выборку стеков по времени, как было показано в главе 6. Поиск «`malloc`» во флейм-графике использования процессора может служить грубым, но недорогим способом выявления наиболее часто встречающихся путей в коде, ведущих в эту функцию, без прямой ее трассировки.

В будущем производительность зондов `uprobes` может быть значительно улучшена (от 10 до 100 раз) с помощью динамических библиотек, использующих переходы внутри пространства пользователя вместо ловушек ядра (см. раздел 2.8.4 в главе 2).

7.1.3. Стратегия

Если вы новичок в анализе производительности памяти, вот рекомендуемая общая стратегия:

1. Исследуйте системные сообщения на наличие событий остановки приложений через OOM Killer (например, с помощью `dmesg(1)`).
2. Проверьте, есть ли в системе устройства подкачки и какой объем подкачки используется. Также проверьте, насколько активно выполняются операции ввода/вывода с этими устройствами (например, с помощью `swap(1)`, `iostat(1)` и `vmstat(1)`).
3. Проверьте объем свободной памяти в системе и то, какая часть памяти занята кэшами (например, `free(1)`).
4. Проверьте, какие объемы памяти используют отдельные процессы (например, с помощью `top(1)` и `ps(1)`).
5. Проверьте частоту сбоев страниц и изучите трассировки стека, ведущие к ним, — это поможет объяснить рост объема резидентной памяти.

6. Проверьте файлы, которым принадлежат страницы, «зарезервированные на диске», вызвавшие сбой.
7. Выполните трассировку системных вызовов `brk()` и `mmap()`, чтобы получить более широкий взгляд на использование памяти.
8. Примените другие инструменты BPF, перечисленные в разделе 7.3 «Инструменты BPF» далее в этой главе.
9. Оцените частоту промахов аппаратного кэша и операций доступа к памяти с помощью счетчиков РМС (особенно с включенной поддержкой PEBS), чтобы определить функции и инструкции, выполняющие операции ввода/вывода с памятью (например, с помощью `perf(1)`).

В следующих разделах рассмотрим эти инструменты более подробно.

7.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Традиционные инструменты анализа производительности предоставляют много статистики использования памяти на основе емкости, включая объем виртуальной и физической памяти, используемой каждым процессом и системой в целом, с некоторыми разбивками, например, по сегментам или слоям процесса. Анализ использования памяти, кроме получения простейших характеристик, таких как частота сбоев страниц, требует инструментации каждой операции выделения памяти библиотекой, средой выполнения или приложением. Для этого допустимо применять анализаторы, например Valgrind, однако это может замедлить целевое приложение в 10 раз и более, пока выполняется трассировка. Инструменты BPF более эффективны и имеют более низкий оверхед.

Даже там, где их возможностей недостаточно для решения проблем, традиционные инструменты могут дать подсказки и помочь выбрать правильные инструменты BPF. Они перечислены в табл. 7.2, где для каждого указан тип источника и получаемые характеристики.

Таблица 7.2. Традиционные инструменты

Инструмент	Тип	Описание
dmesg	Журнал ядра	Подробности о событиях OOM Killer
swapon	Статистики ядра	Использование устройства подкачки
free	Статистики ядра	Использование памяти в системе в целом
ps	Статистики ядра	Статистики процессов, включая характеристики использования памяти
pmap	Статистики ядра	Использование памяти конкретным процессом
vmstat	Статистики ядра	Разные статистики, включая статистики использования памяти

Инструмент	Тип	Описание
sar	Статистики ядра	Позволяет узнать частоту сбоев и сканирования страниц
perf	Программные события, аппаратные статистики, аппаратные выборки	Счетчики РМС, имеющие отношение к памяти, и выборки событий

В следующих разделах дается общее описание основных функциональных возможностей этих инструментов. Полную информацию ищите на справочных страницах и в других ресурсах, в том числе в моей книге «Systems Performance»¹ [Gregg 13b]. В главе 14 дополнительно обсуждается slabtop(1) — инструмент анализа размещения блоков памяти в пространстве ядра.

7.2.1. Журнал ядра

Когда возникает необходимость остановить процесс, OOM Killer записывает подробный отчет в системный журнал, который можно просмотреть с помощью dmesg(1). Например:

```
# dmesg
[2156747.865271] run invoked oom-killer: gfp_mask=0x24201ca, order=0, oom_score_adj=0
[...]
[2156747.865330] Mem-Info:
[2156747.865333] active_anon:3773117 inactive_anon:20590 isolated_anon:0
[2156747.865333] active_file:3 inactive_file:0 isolated_file:0
[2156747.865333] unevictable:0 dirty:0 writeback:0 unstable:0
[2156747.865333] slab_reclaimable:3980 slab_unreclaimable:5811
[2156747.865333] mapped:36 shmem:20596 pagetables:10620 bounce:0
[2156747.865333] free:18748 free_pcp:455 free_cma:0
[...]
[2156747.865385] [ pid ] uid tgid total_vm rss nr_ptes nr_pmds swapents
oom_score_adj name
[2156747.865390] [ 510] 0 510 4870 67 15 3 0
0 upstart-udev-br
[2156747.865392] [ 524] 0 524 12944 237 28 3 0
-1000 systemd-udev
[...]
[2156747.865574] Out of memory: Kill process 23409 (perl) score 329 or sacrifice child
[2156747.865583] Killed process 23409 (perl) total-vm:5370580kB, anon-rss:5224980kB,
file-rss:4kB
```

При этом он сообщает сводную информацию об использовании памяти в масштабе всей системы, выводит таблицу процессов и целевой процесс, принесенный в жертву.

Всегда проверяйте наличие таких сообщений с помощью dmesg(1), прежде чем углубляться в анализ памяти.

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

7.2.2. Статистики ядра

Инструменты, возвращающие статистики ядра, используют источники информации, часто доступные через интерфейс `/proc` (например, `/proc/meminfo`, `/proc/swaps`). Преимущество этих инструментов в том, что соответствующие метрики обычно вычисляются ядром, поэтому их использование влечет незначительный оверхед. Кроме того, многие из них доступны рядовым пользователям, не имеющим привилегий `root`.

swapon

`swapon(1)` позволяет узнать о наличии устройств подкачки в системе и то, какой их объем используется. Например:

```
$ swapon
NAME      TYPE      SIZE USED PRIO
/dev/dm-2 partition 980M   0B   -2
```

Мы видим, что система имеет один раздел подкачки объемом 980 Мбайт, который вообще не используется. Многие системы сейчас вообще не применяют устройства подкачки, и в этом случае `swapon(1)` ничего не выведет.

Если с устройством подкачки выполняются операции ввода/вывода, это можно увидеть в столбцах «si» и «so» в выводе инструмента `vmstat(1)`, а также в выводе `iostat(1)`.

free

Инструмент `free(1)` выводит сводную информацию об объеме использованной и свободной памяти в системе в целом. Вот пример вызова `free(1)` с параметром `-m` для вывода объемов в мегабайтах:

```
$ free -m
              total        used        free      shared  buff/cache   available
Mem:          189282       183022        1103           4         5156         4716
Swap:           0           0           0
```

В последних версиях вывод `free(1)` был улучшен и стал менее запутанным — теперь он включает столбец «available» (доступно), показывающий, сколько памяти доступно для использования, включая кэш файловой системы. Это вызывает меньше путаницы, чем столбец «free» (свободно), который показывает объем памяти, не используемый вообще ни для чего. Если вы считаете, что системе не хватает памяти из-за того, что в столбце «free» (свободно) выводится маленький объем, взгляните на значение в столбце «available» (доступно).

Объем кэшированных страниц файловой системы отображается в столбце «buff/cache» (буфер/кэш), в котором выводится сумма двух значений: объем буферов ввода/вывода и объем кэшированных страниц файловой системы. Эти значения

можно вывести в двух отдельных столбцах, выполнив команду с параметром `-w` (wide — широкий вывод).

Данный конкретный пример вывода получен в производственной системе с общим объемом памяти 184 Гбайт, из которых примерно 4 Гбайт доступно для использования. Подробную разбивку памяти можно получить с помощью команд `cat/proc/meminfo`.

ps

С помощью команды `ps(1)` — process status (состояние процесса) — можно получить объем памяти, используемый процессом:

```
$ ps aux
USER  PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
[...]
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	2499	0.0	0.0	30028	2720	?	Ss	Jan25	0:00	/usr/sbin/cron -f
root	2703	0.0	0.0	0	0	?	I	04:13	0:00	[kworker/41:0]
pcp	2951	0.0	0.0	116716	3572	?	S	Jan25	0:00	/usr/lib/pcp/bin/pmwe...
root	2992	0.0	0.0	0	0	?	I	Jan25	0:00	[kworker/17:2]
root	3741	0.0	0.0	0	0	?	I	Jan25	0:05	[kworker/0:3]
www	3785	1970	95.7	213734052	185542800	?	S1	Jan25	15123:15	/apps/java/bin/java...

```
[...]
```

Особый интерес представляют столбцы:

- **%MEM**: доля в процентах физической памяти системы, используемая процессом;
- **VSZ**: размер виртуальной памяти;
- **RSS**: Resident Set Size (размер резидентного набора): общий объем физической памяти, используемой процессом.

Как показывают эти результаты, процесс `java` потребляет 95.7% физической памяти в системе. Команда `ps(1)` позволяет указать, какие столбцы выводить, чтобы можно было сосредоточиться только на статистике памяти (например, `ps -eo pid, rmem, vsz, rss`). Эти статистики и многое другое можно найти в файлах `/proc/proc/PID/status`.

ptop

Команда `ptop(1)` позволяет получить список сегментов в адресном пространстве процесса. Например:

```
$ ptop -x 3785
3785: /apps/java/bin/java -Dnop -XX:+UseG1GC -...
XX:+ParallelRefProcEnabled -XX:+ExplicitGCIn
```

Address	Kbytes	RSS	Dirty	Mode	Mapping
0000000000400000	4	0	0	r-x--	java
0000000000400000	0	0	0	r-x--	java
0000000000600000	4	4	4	rw---	java
0000000000600000	0	0	0	rw---	java

```

00000000006c2000    5700    5572    5572 rw--- [ anon ]
00000000006c2000         0         0         0 rw--- [ anon ]
[...]
00007f2ce5e61000         0         0         0 ---- libjvm.so
00007f2ce6061000    832    832    832 rw--- libjvm.so
00007f2ce6061000         0         0         0 rw--- libjvm.so
[...]
fffffffffff600000         4         0         0 r-x-- [ anon ]
fffffffffff600000         0         0         0 r-x-- [ anon ]
-----
total kB          213928940 185743916 185732800

```

Эта информация позволяет выявить крупных потребителей памяти, например библиотеки или отображаемые файлы. Расширенный (-x) вывод включает столбец «dirty» с объемом памяти для хранения «грязных» страниц: страниц, которые изменились в памяти и еще не сохранены на диске.

vmstat

Команда `vmstat(1)` позволяет наблюдать за изменением различных системных статистик с течением времени, включая статистики использования памяти, процессоров и операций ввода/вывода. Например, вот как можно организовать вывод статистик с интервалом в 1 секунду:

```

$ vmstat 1
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
 r  b  swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
12  0      0 1075868 13232 5288396   0   0    14    26    16   19 38  2 59  0  0
14  0      0 1075000 13232 5288932   0   0     0     0 28751 77964 22  1 77  0  0
 9  0      0 1074452 13232 5289440   0   0     0     0 28511 76371 18  1 81  0  0
15  0      0 1073824 13232 5289828   0   0     0     0 32411 86088 26  1 73  0  0

```

В столбцах «free», «buff» и «cache» отображается объем памяти (в килобайтах) свободной, используемой для буферов ввода/вывода и используемой для кэшей файловой системы. В столбцах «si» и «so» отображается объем памяти прочитанной и вытесненной на устройство подкачки, если оно включено.

Первая строка в выводе — это «сводка с момента загрузки». В ней в большинстве столбцов отображаются средние значения, подсчитанные с момента загрузки системы. Но столбцы с информацией о памяти показывают текущее состояние. Вторая и последующие строки содержат текущие сводки, получаемые с интервалом в 1 секунду.

sar

Команда `sar(1)` — это универсальный инструмент, позволяющий получить самые разные метрики. При вызове с параметром `-B` команда отображает статистики страниц:

```

# sar -B 1
Linux 4.15.0-1031-aws (...)      01/26/2019      _x86_64_      (48 CPU)

```

```

06:10:38 PM pgpgin/s pgpgout/s   fault/s   majflt/s   pgfree/s pgscank/s pgscand/s
pgsteal/s   %vmeff
06:10:39 PM      0.00      0.00    286.00      0.00  16911.00      0.00      0.00
0.00      0.00
06:10:40 PM      0.00      0.00     90.00      0.00  19178.00      0.00      0.00
0.00      0.00
06:10:41 PM      0.00      0.00    187.00      0.00  18949.00      0.00      0.00
0.00      0.00
06:10:42 PM      0.00      0.00    110.00      0.00  24266.00      0.00      0.00
0.00      0.00
[...]
```

Этот вывод получен на высоконагруженном производственном сервере. Вывод очень широкий, поэтому из-за переноса строк его немного сложно читать. Частота сбоев страниц («faults/s») относительно низкая — менее 300 в секунду. Здесь также видно, что сканирование страниц не выполняется (столбцы «pgscan»), а это говорит о том, что, скорее всего, система не испытывает нехватки памяти.

А этот пример получен на сервере во время сборки ПО:

```

# sar -B 1
Linux 4.15.0-1031-aws (...)    01/26/2019      _x86_64_    (48 CPU)

06:16:08 PM pgpgin/s pgpgout/s   fault/s   majflt/s   pgfree/s pgscank/s pgscand/s
pgsteal/s   %vmeff
06:16:09 PM  1968.00    302.00 1454167.00      0.00 1372222.00      0.00      0.00
0.00      0.00
06:16:10 PM  1680.00    171.00 1374786.00      0.00 1203463.00      0.00      0.00
0.00      0.00
06:16:11 PM  1100.00    581.00 1453754.00      0.00 1457286.00      0.00      0.00
0.00      0.00
06:16:12 PM  1376.00    227.00 1527580.00      0.00 1364191.00      0.00      0.00
0.00      0.00
06:16:13 PM   880.00     68.00 1456732.00      0.00 1315536.00      0.00      0.00
0.00      0.00
[...]
```

Здесь наблюдается огромная частота сбоев страниц — более одного миллиона в секунду. Это связано с тем, что в сборку ПО вовлечено множество короткоживущих процессов, и каждый новый процесс вызывает сбой в своем адресном пространстве при первом выполнении.

7.2.3. Аппаратные статистики и выборки

Для событий ввода/вывода в память есть много счетчиков РМС. Уточню, что здесь имеются в виду операции ввода/вывода между блоками процессора и банками оперативной памяти (через кэш-память процессора). Счетчики РМС, представленные в главе 2, могут использоваться в двух режимах: подсчет и выборка. В режиме подсчета можно получать статистические сводки практически без всяких затрат. Но в режиме выборки некоторые события приходится записывать в файл для последующего анализа.

Вот пример использования `perf(1)` для подсчета обращений и промахов кэша последнего уровня (LLC) в масштабе всей системы (`-a`) с интервалом вывода в 1 секунду (`-I 1000`):

```
# perf stat -e LLC-loads,LLC-load-misses -a -I 1000
#
      time              counts unit events
1.000705801          8,402,738      LLC-loads
1.000705801          3,610,704      LLC-load-misses # 42.97% of all LL-cache hits
2.001219292          8,265,334      LLC-loads
2.001219292          3,526,956      LLC-load-misses # 42.32% of all LL-cache hits
3.001763602          9,586,619      LLC-loads
3.001763602          3,842,810      LLC-load-misses # 43.91% of all LL-cache hits
[...]
```

В данном случае `perf(1)` распознал связь между счетчиками РМС и вывел долю промахов в процентах. Промахи LLC — один из показателей ввода/вывода в основную память, потому что когда при выполнении операции чтения или записи обнаруживается промах, производится обращение к основной памяти.

А вот пример использования `perf(1)` в режиме выборки, и в данном случае в файл записывается подробная информация о каждом из 100 000 промахов кэша данных L1:

```
# perf record -e L1-dcache-load-misses -c 100000 -a
^C[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 3.075 MB perf.data (612 samples) ]
# perf report -n --stdio
# Overhead  Samples  Command  Shared Object      Symbol
# .....
.
#
30.56%      187  cksum    [kernel.kallsyms]  [k] copy_user_enhanced_fast_string
 8.33%       51  cksum    cksum              [.] 0x0000000000001cc9
 2.78%       17  cksum    cksum              [.] 0x0000000000001cb4
 2.45%       15  cksum    [kernel.kallsyms]  [k] generic_file_read_iter
 2.12%       13  cksum    cksum              [.] 0x0000000000001cbe
[...]
```

Такой большой порог выборки (`-c 100000`) использовался потому, что обращения к L1 происходят очень часто. Более низкий порог может привести к слишком частому срабатыванию и заметно ухудшить производительность работающего ПО. Если вы не знаете, как часто срабатывает счетчик РМС, сначала используйте режим подсчета (`perf stat`), чтобы определить эту величину, на основе которой можно будет рассчитать соответствующий порог.

Вывод `perf report` показывает символы, обращение к которым вызвало промахи кэша L1. Для анализа использования памяти с помощью РМС советую включать режим высокой точности PEBS, чтобы получать точные значения указателя команд. Для этого в команде `perf` добавьте `:p`, или `:pp` (лучше), или `:ppp` (еще лучше) в конец имени события. Чем больше символов `p`, тем выше точность. (См. раздел с описанием модификатора `p` на странице справочного руководства для `perf-list(1)`.)

7.3. ИНСТРУМЕНТЫ BPF

Этот раздел охватывает инструменты BPF, которые можно применять для анализа использования памяти и устранения проблем (рис. 7.4).

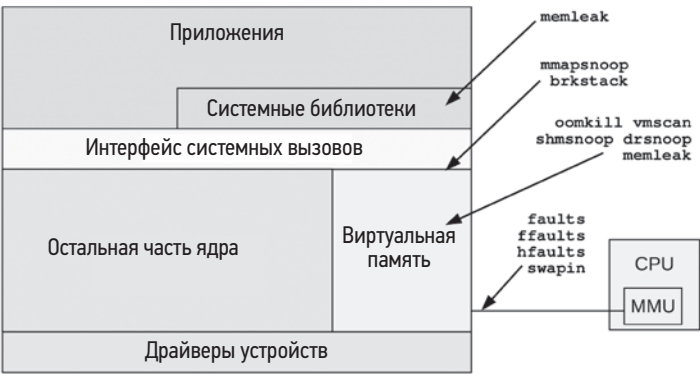


Рис. 7.4. Инструменты BPF для анализа использования памяти

Часть этих инструментов можно найти в репозиториях BCC и bpftrace, упоминавшихся в главах 4 и 5, а часть была создана специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях, BCC и bpftrace. Рассматриваемые здесь инструменты перечислены в табл. 7.3, где также указано их происхождение (BT — это сокращение от «bpftrace»).

Таблица 7.3. Инструменты для анализа использования памяти

Инструмент	Источник	Цель	Описание
oomkill	BCC/BT	OOM Killer	Выводит дополнительную информацию о событиях OOM Killer
memleak	BCC	Планировщик	Выводит пути в коде, возможно, вызывающие утечки памяти
mmapnoop	книга	Системные вызовы	Трассирует системный вызов mmap(2) для системы в целом
brkstack	книга	Системные вызовы	Выводит трассировки стека в пространстве пользователя, ведущие к вызову brk()
shmsnoop	BCC	Системные вызовы	Трассирует обращения к разделяемой памяти и выводит дополнительные детали
faults	книга	Сбои страниц	Выводит трассировки стека в пространстве пользователя, вызывающие сбои страниц
ffaulst	книга	Сбои страниц	Выводит события сбоев страниц по файлам

Таблица 7.3 (окончание)

Инструмент	Источник	Цель	Описание
vmscan	книга	Виртуальная машина	Измеряет время, потребовавшееся сканеру виртуальной машины на компактификацию и утилизацию памяти
drsnnoop	ВСС	Виртуальная машина	Трассирует события утилизации памяти, отображает задержки
swapi	книга	Виртуальная машина	Выводит события извлечения блоков из устройства подкачки по процессам
hfaults	книга	Сбои страниц	Выводит события сбоев огромных страниц (huge page) по процессам

Полные и актуальные списки параметров инструментов ВСС и bpftrace и описание их возможностей ищите в соответствующих репозиториях. Ниже я расскажу только о наиболее важных возможностях.

В главе 14 представлены дополнительные инструменты BPF для анализа памяти ядра: `kmem(8)`, `kpages(8)`, `slabratetop(8)` и `numamove(8)`.

7.3.1. oomkill

`oomkill(8)`¹ — это инструмент ВСС и bpftrace, трассирующий события OOM Killer и сообщающий дополнительные детали, такие как средние значения загрузки. Средние значения загрузки помогают получить более полное представление о состоянии системы на момент события нехватки памяти, показывая, насколько велика была нагрузка на систему.

Ниже показан пример использования `oomkill(8)` из ВСС на производственном экземпляре с 48 процессорами:

```
# oomkill
Tracing OOM kills... Ctrl-C to stop.
08:51:34 Triggered by PID 18601 ("perl"), OOM kill of PID 1165 ("java"), 18006224
pages, loadavg: 10.66 7.17 5.06 2/755 18643
[...]
```

В этом выводе сообщается, что процесс PID 18601 (perl) запросил дополнительную память, из-за чего возникла ситуация нехватки памяти и OOM Killer остановил процесс PID 1165 (java). К этому моменту процесс PID 1165 использовал 18 006 224 страницы памяти. Размер одной страницы зависит от аппаратной архитектуры процессора и настроек потребления памяти в процессе и обычно равен

¹ Немного истории: версию для ВСС я создал 9 февраля 2016 года, чтобы получить дополнительную отладочную информацию о событиях OOM Killer в производственной системе, возникавших время от времени. Версию для bpftrace я написал 7 сентября 2018 года.

4 Кбайт. Значения средней загрузки показывают, что в момент срабатывания механизма OOM Killer система работала под довольно высокой нагрузкой.

Этот инструмент трассирует функцию `oom_kill_process()`, используя `kprobes`, и выводит различные сведения. В данном случае средние значения загрузки извлекаются из `/proc/loadavg`. При желании этот инструмент можно расширить и добавить в него вывод других деталей. Кроме того, сейчас этот инструмент не использует точки трассировки `oom`, которые сообщают более подробную информацию о том, как выбираются задачи для остановки.

У текущей версии для ВСС нет параметров командной строки.

bpfttrace

Ниже приводится реализация `oomkill(8)` для `bpfttrace`:

```
#!/usr/local/bin/bpfttrace

#include <linux/oom.h>

BEGIN
{
    printf("Tracing oom_kill_process()... Hit Ctrl-C to end.\n");
}

kprobe:oom_kill_process
{
    $oc = (struct oom_control *)arg1;
    time("%H:%M:%S ");
    printf("Triggered by PID %d (\\"%s\\"), ", pid, comm);
    printf("OOM kill of PID %d (\\"%s\\"), %d pages, loadavg: ",
        $oc->chosen->pid, $oc->chosen->comm, $oc->totalpages);
    cat("/proc/loadavg");
}
```

Программа трассирует функцию `oom_kill_process()` и преобразует ее второй аргумент в структуру `struct oom_control`, которая содержит информацию об остававшемся процессе. Она выводит сведения о текущем процессе (`pid`, `comm`), вызвавшем событие ООМ, затем информацию о целевом процессе и, наконец, вызывает `system()`, чтобы вывести средние значения загрузки.

7.3.2. memleak

`memleak(8)`¹ — это инструмент ВСС, который трассирует события распределения и освобождения памяти и фиксирует трассировки стека. При продолжительной трассировке он помогает обнаружить утечки памяти, когда память выделялась, но

¹ Немного истории: этот инструмент создал Саша Гольдштейн и опубликовал его 7 февраля 2016 года.

не освобождалась. В примере ниже показано применение memleak(8) для анализа процесса командной оболочки bash¹:

```
# memleak -p 3126
Attaching to pid 3228, Ctrl+C to quit.

[09:14:15] Top 10 stacks with outstanding allocations:
[...]
    960 bytes in 1 allocations from stack
        xrealloc+0x2a [bash]
        strvec_resize+0x2b [bash]
        maybe_make_export_env+0xa8 [bash]
        execute_simple_command+0x269 [bash]
        execute_command_internal+0x862 [bash]
        execute_connection+0x109 [bash]
        execute_command_internal+0xc18 [bash]
        execute_command+0x6b [bash]
        reader_loop+0x286 [bash]
        main+0x969 [bash]
        __libc_start_main+0xe7 [libc-2.27.so]
        [unknown]

    1473 bytes in 51 allocations from stack
        xmalloc+0x18 [bash]
        make_env_array_from_var_list+0xc8 [bash]
        make_var_export_array+0x3d [bash]
        maybe_make_export_env+0x12b [bash]
        execute_simple_command+0x269 [bash]
        execute_command_internal+0x862 [bash]
        execute_connection+0x109 [bash]
        execute_command_internal+0xc18 [bash]
        execute_command+0x6b [bash]
        reader_loop+0x286 [bash]
        main+0x969 [bash]
        __libc_start_main+0xe7 [libc-2.27.so]
        [unknown]

[...]
```

По умолчанию вывод происходит раз в 5 секунд и включает трассировки стека, ведущие к выделению памяти, и общее количество байтов, которое еще должно быть освобождено. Последняя трассировка стека показывает, что с помощью `execute_command()` и `make_env_array_from_var_list()` было выделено 1473 байта.

Используя один только инструмент memleak(8), нельзя сказать, являются ли эти операции выделения настоящими утечками памяти (когда выделенная память никогда не освобождается) или просто память выделяется для каких-то долгосрочных нужд. Для окончательных выводов нужно изучить и понять пути выполнения кода.

¹ Чтобы гарантировать, что трассировка стека на основе указателя фрейма работает и используются обычные подпрограммы malloc, эта версия bash была скомпилирована с `CFLAGS=-fno-omit-frame-pointer ./configure - without-gnu-malloc`.

Без параметра `-p PID memleak(8)` трассирует операции выделения памяти ядром:

```
# memleak
Attaching to kernel allocators, Ctrl+C to quit.
[...]
[09:19:30] Top 10 stacks with outstanding allocations:
[...]
    15384576 bytes in 3756 allocations from stack
    __alloc_pages_nodemask+0x209 [kernel]
    alloc_pages_vma+0x88 [kernel]
    handle_pte_fault+0x3bf [kernel]
    __handle_mm_fault+0x478 [kernel]
    handle_mm_fault+0xb1 [kernel]
    __do_page_fault+0x250 [kernel]
    do_page_fault+0x2e [kernel]
    page_fault+0x45 [kernel]
[...]

```

При анализе конкретного процесса `memleak(8)` трассирует функции выделения в пространстве пользователя: `malloc()`, `calloc()`, `free()` и т. д. При анализе ядра используются точки трассировки `kmem: kmem:kmalloc`, `kmem:kfree` и т. д.

Порядок использования:

```
memleak [options] [-p PID] [-c COMMAND] [interval [count]]
```

Параметры `options`:

- **-s RATE**: требует производить одну выборку для каждых RATE операций распределения памяти, что можно использовать для уменьшения оверхеда;
- **-o OLDER**: отбрасывает операции распределения, имевшие место более OLDER микросекунд тому назад.

Операции распределения, особенно в пространстве пользователя, могут следовать друг за другом очень часто — до миллионов раз в секунду. Из-за этого трассировка приложения с помощью `memleak(8)` может замедлить его работу до 10 раз или более — в зависимости от величины нагрузки. Это означает, что `memleak(8)` — это больше инструмент для устранения неполадок или отладки, чем для повседневного анализа в производственных средах. Как я уже говорил, это будет актуально до тех пор, пока производительность `uprobes` значительно не улучшат.

7.3.3. mmapnoop

`mmapnoop(8)`¹ трассирует системный вызов `mmap(2)` в границах всей системы и выводит информацию о запрошенных отображениях. Может использоваться для

¹ Немного истории: первую версию — `mmap.d` — я написал для книги «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD» в 2010 году [Gregg 11], а BCC-версию для этой книги — 3 февраля 2019 года.

отладки использования отображений виртуальных адресов памяти в физические. Пример вывода:

```
# mmapnoop.py
PID    COMM      PROT MAP  OFFS(KB)  SIZE(KB)  FILE
6015   mmapnoop.py RW-  S---  0         260       [perf_event]
6015   mmapnoop.py RW-  S---  0         260       [perf_event]
[...]
6315   java       R-E  -P--  0         2222      libjava.so
6315   java       RW-  -PF-   168 8      libjava.so
6315   java       R--  -P--  0         43        ld.so.cache
6315   java       R-E  -P--  0        2081      libnss_compat-2.23.so
6315   java       RW-  -PF-   28 8        libnss_compat-2.23.so
6315   java       R-E  -P--  0        2146      libnsl-2.23.so
6315   java       RW-  -PF-   84 8        libnsl-2.23.so
6315   java       R--  -P--  0         43        ld.so.cache
6315   java       R-E  -P--  0        2093      libnss_nis-2.23.so
6315   java       RW-  -PF-   40 8        libnss_nis-2.23.so
6315   java       R-E  -P--  0        2117      libnss_files-2.23.so
6315   java       RW-  -PF-   40 8        libnss_files-2.23.so
6315   java       R--  S---  0         2         passwd
[...]
```

Вывод в этом примере начинается с отображений для кольцевых буферов `perf_event`, которые использует этот инструмент для извлечения событий. Затем следуют отображения, соответствующие запуску нового процесса `java`, с флагами защиты и сопоставления.

Флаги защиты (PROT):

- **R**: PROT_READ;
- **W**: PROT_WRITE;
- **E**: PROT_EXEC.

Флаги сопоставления (MAP):

- **S**: MAP_SHARED;
- **P**: MAP_PRIVATE;
- **F**: MAP_FIXED;
- **A**: MAP_ANON.

`mmapnoop(8)` поддерживает параметр командной строки `-T` для вывода столбца со значением времени.

Этот инструмент использует точку трассировки `syscalls:sys_enter_mmap`. Оверхед инструмента, как правило, незначительный, потому что частота появления новых отображений довольно низкая.

В главе 8 мы дополнительно рассмотрим вопросы анализа файлов, отображаемых в память, и инструменты `mmapfiles(8)` и `fmapfaults(8)`.

7.3.4. brkstack

Обычное хранилище памяти для данных приложения — это куча, которая увеличивается обращением к системному вызову `brk(2)`. Иногда полезно произвести трассировку `brk(2)`, чтобы получить последовательности вызовов в пространстве пользователя, приведшие к этому росту. Есть также вариант `sbrk(2)`, но в Linux `sbrk(2)` реализован как библиотечный вызов, вызывающий `brk(2)`.

Для анализа системного вызова `brk(2)` можно использовать точку трассировки `syscalls:syscall_enter_brk`. С помощью инструментов BCC, `trace(8)` и `stackcount(8)` в ней можно получить трассировку стека для каждого события и подсчитать их частоту. Также для анализа можно использовать однострочные сценарии для `bpfftrace` и `perf(1)`. Примеры использования инструментов BCC:

```
# trace -U t:syscalls:sys_enter_brk
# stackcount -PU t:syscalls:sys_enter_brk
```

Например:

```
# stackcount -PU t:syscalls:sys_enter_brk
Tracing 1 functions for "t:syscalls:sys_enter_brk"... Hit Ctrl-C to end.
^C
[...]
```

```
brk
__sbrk
__default_morecore
sysmalloc
_int_malloc
tcache_init
__libc_malloc
malloc_hook_ini
__libc_malloc
JLI_MemAlloc
JLI_List_new
main
__libc_start_main
_start
  java [8395]
    1

[unknown]
  cron [8385]
    2
```

В этом усеченном выводе показана трассировка стека процесса `java` от функции `JLI_List_new()`, через `JLI_MemAlloc()` и `sbrk(3)` до `brk(2)`. Все выглядит так, будто увеличение объема памяти в куче инициировано объектом списка. Вторая трассировка стека, принадлежащего процессу `cron`, оказалась нарушенной. Чтобы получить трассировку стека для `java`, пришлось использовать версию `libc` с указателями кадров. Эта проблема обсуждается в разделе 13.2.9 в главе 13.

Вызовы `brk(2)` происходят довольно редко, и его трассировка помогает выявить операции выделения необычно больших блоков памяти, не вмещающихся в доступный объем, и вполне обычных, размер которых превышает размер свободного места в куче всего на 1 байт. Чтобы выяснить истинную причину, нужно исследовать путь выполнения кода. Поскольку увеличение размера кучи происходит нечасто, переход на трассировку этого системного вызова незначителен, что делает этот прием недорогим методом поиска причин увеличения потребления памяти. Для сравнения, прямая трассировка гораздо более частых функций выделения памяти (например, `malloc()`) может оказаться слишком дорогим удовольствием из-за чрезмерного перехода. Другой инструмент анализа потребления памяти, обладающий минимальными издержками, это `faults(8)`, описанный в разделе 7.3.6, который отслеживает сбой страниц.

Иногда однострочники для `bpftrace` проще запомнить по имени файла, чем по содержанию. Их пример — `brkstack(8)`¹:

```
#!/usr/local/bin/bpftrace

tracepoint:syscalls:sys_enter_brk
{
    @[ustack, comm] = count();
}
```

7.3.5. shmsnoop

`shmsnoop(8)`² — это инструмент для ВСС, трассирующий системные вызовы для работы с разделяемой памятью и доставшийся в наследство от System V: `shmget(2)`, `shmat(2)`, `shmdt(2)` и `shmctl(2)`. Его можно использовать для анализа использования разделяемой памяти. Вот пример данных, полученных в момент запуска приложения на Java:

```
# shmsnoop
PID    COMM      SYS      RET ARGS
12520  java      SHMGET    58c000a key: 0x0, size: 65536, shmflg: 0x380 (IPC_
CREAT|0600)
12520  java      SHMAT 7fde9c033000 shmid: 0x58c000a, shmaddr: 0x0, shmflg: 0x0
12520  java      SHMCTL    0 shmid: 0x58c000a, cmd: 0, buf: 0x0
12520  java      SHMDT    0 shmaddr: 0x7fde9c033000
1863   Xorg      SHMAT 7f98cd3b9000 shmid: 0x58c000a, shmaddr: 0x0, shmflg: 0x1000
(SHM_RDONLY)
1863   Xorg      SHMCTL    0 shmid: 0x58c000a, cmd: 2, buf: 0x7fdddd9e240
1863   Xorg      SHMDT    0 shmaddr: 0x7f98cd3b9000
[...]
```

¹ Немного истории: я создал его для этой книги 26 января 2019 года. За прошедшие годы мне много раз приходилось заниматься трассировкой `brk()`, и в прошлом я публиковал соответствующие флейм-графики [82].

² Немного истории: его создал Иржи Оlsa (Jiri Olsa) 8 октября 2018 года.

Мы видим, как Java выделяет разделяемую память вызовом `shmget(2)`, за которым следуют различные операции с общей памятью и их аргументы. Возвращаемое значение `shmget(2)` — идентификатор `0x58c000a` — используется в последующих вызовах как самим процессом Java, так и сервером Xorg, то есть они совместно используют один и тот же блок памяти.

Этот инструмент трассирует системные вызовы для работы с разделяемой памятью, обращения к которым следуют достаточно редко, поэтому оверхед на их трассировку незначителен.

Порядок использования:

```
shmsnoop [options]
```

Параметры `options`:

- **-T**: включать в вывод отметки времени;
- **-p PID**: трассировать только этот процесс.

7.3.6. faults

Трассировка сбоев страниц и путей в коде, повлекших эти сбои, помогает получить более полное представление об использовании памяти: не пути кода, ведущие к разделению памяти, но пути, ведущие к первому использованию памяти и вызвавшие сбой страницы. Сбои страниц приводят к росту размера резидентного набора (RSS), поэтому их трассировка помогает понять, почему растет потребление памяти в процессе. Как и в случае с `brk()`, это событие можно отследить, используя однострочный сценарий в комплексе с другими инструментами, например BCC и `stackcount(8)`, для подсчета частоты сбоев страниц в пространстве пользователя и ядра и получения соответствующих им трассировок стека:

```
# stackcount -U t:exceptions:page_fault_user
# stackcount t:exceptions:page_fault_kernel
```

Пример вывода при вызове команды `stackcount(8)` с параметром `-P` для группировки информации по процессам:

```
# stackcount -PU t:exceptions:page_fault_user
Tracing 1 functions for "t:exceptions:page_fault_user"... Hit Ctrl-C to end.
^C
[...]

PhaseIdealLoop::Dominators()
PhaseIdealLoop::build_and_optimize(LoopOptsMode)
Compile::optimize_loops(PhaseIterGVN&, LoopOptsMode) [clone .part.344]
Compile::Optimize()
Compile::Compile(ciEnv*, C2Compiler*, ciMethod*, int, bool, bool, bool, Directiv...
C2Compiler::compile_method(ciEnv*, ciMethod*, int, DirectiveSet*)
CompileBroker::invoke_compiler_on_method(CompileTask*)
CompileBroker::compiler_thread_loop()
JavaThread::thread_main_inner()
```

```

Thread::call_run()
thread_native_entry(Thread*)
start_thread
__clone
   C2 CompilerThre [9124]
   1824

__memset_avx2_erms
PhaseCFG::global_code_motion()
PhaseCFG::do_global_code_motion()
Compile::Code_Gen()
Compile::Compile(ciEnv*, C2Compiler*, ciMethod*, int, bool, bool, bool, Directiv...
C2Compiler::compile_method(ciEnv*, ciMethod*, int, DirectiveSet*)
CompileBroker::invoke_compiler_on_method(CompileTask*)
CompileBroker::compiler_thread_loop()
JavaThread::thread_main_inner()
Thread::call_run()
thread_native_entry(Thread*)
start_thread
__clone
   C2 CompilerThre [9124]
   2934

```

Этот вывод соответствует запуску процесса Java, в котором поток компилятора C2 вызывает сбой страниц в процессе компиляции байт-кода в машинный код.

Флейм-графики сбоев страниц

Трассировки стеков, соответствующие событиям сбоя страниц, можно визуализировать в виде флейм-графика. (Флейм-графики были рассмотрены в главе 2.) Ниже приводятся команды создания флейм-графика сбоев страниц с использованием моей реализации [37]. На рис. 7.5 показан результат:

```

# stackcount -f -PU t:exceptions:page_fault_user > out.pagefaults01.txt
$ flamegraph.pl --hash --width=800 --title="Page Fault Flame Graph" \
  --colors=java --bgcolor=green < out.pagefaults01.txt > out.pagefaults01.svg

```

В увеличенной области показаны пути выполнения кода в компиляторе Java, ведущие к выделению дополнительной памяти и появлению сбоев страниц.

Netflix автоматически генерирует флейм-графики сбоев страниц с помощью Vector — инструмента анализа экземпляров, поэтому разработчики Netflix могут получать их одним кликом (см. главу 17).

bpfftrace

Для простоты использования был создан инструмент faults(8)¹ для bpfftrace, отбирающий трассировки стека по событиям отказов страниц:

¹ Немного истории: я создал его для этой книги 27 января 2019 года, а в прошлом для получения трассировок стеков по событиям сбоев страниц использовал другие трассировщики [82].

```
#!/usr/local/bin/bpftrace

software:page-faults:1
{
    @[ustack, comm] = count();
}
```

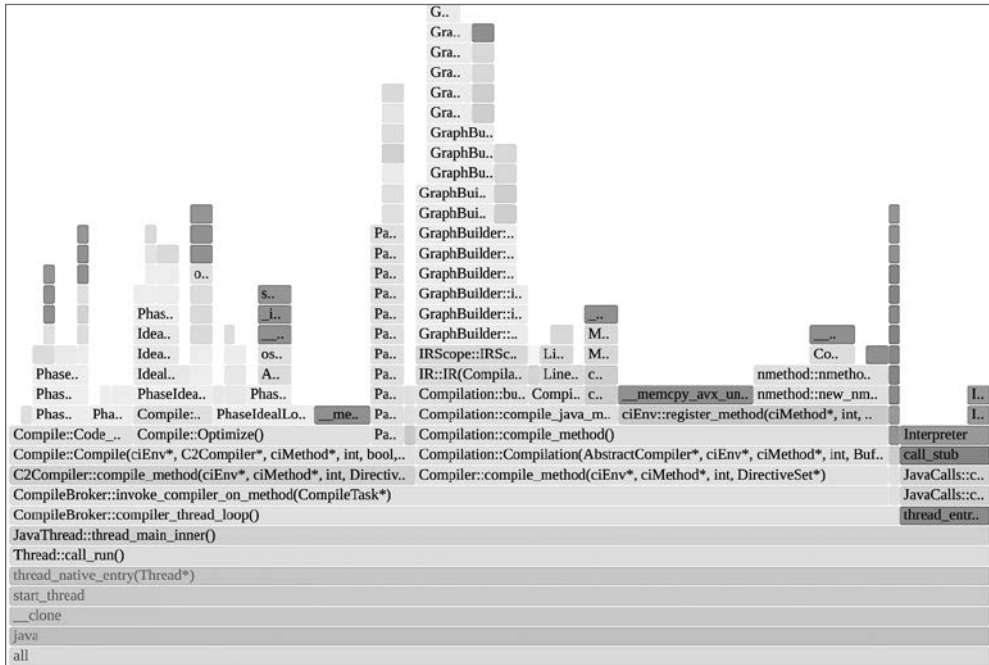


Рис. 7.5. Флейм-график сбоев страниц

Он инструментует события сбоев страниц и использует счетчик переполнений с порогом, равным 1: запускает программу BPF для каждого события сбоя страницы и подсчитывает частоту трассировок стека для каждого пользователя и имени процесса.

7.3.7. ffaults

`ffaults(8)`¹ трассирует сбои страниц по имени файла. Например, эти результаты получены во время сборки ПО:

```
# ffaults.bt
Attaching 1 probe...
```

¹ Немного истории: я создал его для этой книги 26 января 2019 года.

```
[...]
@[cat]: 4576
@[make]: 7054
@[libbfd-2.26.1-system.so]: 8325
@[libtinfo.so.5.9]: 8484
@[libdl-2.23.so]: 9137
@[locale-archive]: 21137
@[cc1]: 23083
@[ld-2.23.so]: 27558
@[bash]: 45236
@[libopcodes-2.26.1-system.so]: 46369
@[libc-2.23.so]: 84814
@[]: 537925
```

Согласно этим результатам, большинство сбоев страниц приходилось на области памяти, не связанные с именем файла, то есть области, находящиеся в кучах процессов. Всего за время трассировки было обнаружено 537 925 сбоев. Также при трассировке обнаружилось 84 814 сбоев, связанных с библиотекой libc. Это объясняется тем, что в ходе сборки ПО создается множество короткоживущих процессов, вызывающих сбои в их новых адресных пространствах.

Исходный код `ffaults(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/mm.h>

kprobe:handle_mm_fault
{
    $vma = (struct vm_area_struct *)arg0;
    $file = $vma->vm_file->f_path.dentry->d_name.name;
    @[str($file)] = count();
}
```

Этот инструмент трассирует функцию ядра `handle_mm_fault()` с помощью зондов `kprobes` и по ее аргументам определяет имя файла, ответственного за сбой. Частота сбоев меняется в зависимости от характера рабочей нагрузки. Убедиться в этом можно с помощью инструментов `perf(1)` и `sar(1)`. При очень высокой частоте сбоев страниц `ffaults` может иметь довольно заметный оверхед.

7.3.8. vmscan

`vmscan(8)`¹ инструментирует точки трассировки в функции `vmscan` в демоне подкачки (`kswapd`), который освобождает память для повторного использования, когда система испытывает нехватку памяти. Обратите внимание: хотя

¹ Немного истории: я создал этот инструмент для этой книги 26 января 2019 года. Более ранний инструмент, использующий эти же точки трассировки, которые вошли в состав ядра Linux в 2009 году, можно найти в файле `trace-vmscan-postprocess.pl`. Он был создан Мелом Горманом (Mel Gorman).

термин *сканер* (scanner) все еще используется для обозначения этой функции ядра, сейчас Linux управляет памятью с помощью связанных списков активной и неактивной памяти.

Пример трассировки системы с 36 процессорами с помощью `vmscan(8)` в период, когда система испытывала нехватку памяти:

```
# vmscan.bt
Attaching 10 probes...
TIME          S-SLABms  D-RECLAIMms  M-RECLAIMms  KSWAPD  WRITEPAGE
21:30:25             0           0           0         0         0
21:30:26             0           0           0         0         0
21:30:27          276          555           0         2         1
21:30:28         5459         7333           0        15        72
21:30:29           41           0           0        49        35
21:30:30            1          454           0         2         2
21:30:31            0           0           0         0         0
^C

@direct_reclaim_ns:
[256K, 512K)          5 |@|
[512K, 1M)           83 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1M, 2M)          174 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[2M, 4M)          136 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[4M, 8M)           66 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[8M, 16M)          68 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[16M, 32M)           8 |@@|
[32M, 64M)           3 |
[64M, 128M)          0 |
[128M, 256M)         0 |
[256M, 512M)        18 |@@@@@@|
@shrink_slab_ns:
[128, 256)         12228 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[256, 512)        19859 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[512, 1K)          1899 |@@@@|
[1K, 2K)           1052 |@@|
[2K, 4K)            546 |@|
[4K, 8K)            241 |
[8K, 16K)           122 |
[16K, 32K)           518 |@|
[32K, 64K)          600 |@|
[64K, 128K)         49 |
[128K, 256K)        19 |
[256K, 512K)         7 |
[512K, 1M)          6 |
[1M, 2M)            8 |
[2M, 4M)            4 |
[4M, 8M)            7 |
[8M, 16M)           29 |
[16M, 32M)           11 |
[32M, 64M)           3 |
[64M, 128M)          0 |
[128M, 256M)          0 |
[256M, 512M)        19 |
```

Столбцы в выводе показывают:

- **S-SLABms**: общее время в миллисекундах, потраченное на освобождение памяти, занимаемой разными кэшами в ядре.
- **D-RECLAIMms**: общее время в миллисекундах, потраченное непосредственно на операции утилизации памяти. Эти операции блокируют запросы на выделение памяти, пока происходит запись на диск.
- **M-RECLAIMms**: общее время в миллисекундах утилизации памяти в группах управления `sgroups`. Если для управления памятью используется механизм `sgroups`, в этом столбце отображается время, затраченное на утилизацию памяти, когда одна из групп `sgroup` превысила свой предел и ее собственная память была утилизирована.
- **KSWAPD**: число пробуждений `kswapd`.
- **WRITEPAGE**: число страниц, записанных `kswapd`.

Время подсчитывается по всем процессорам, из-за чего overhead оказывается выше, чем при использовании других инструментов, например `vmstat(1)`.

Обратите внимание на время, потраченное непосредственно на утилизацию памяти (**D-RECLAIMms**): это плохой способ утилизации, но необходимый, и он может вызвать проблемы с производительностью. Иногда расходы на непосредственную утилизацию можно сократить, настроив другие параметры `vm sysctl`, отвечающие за частоту сканирования в фоновом режиме.

В гистограммах для отдельных событий непосредственной утилизации и освобождения кэшей ядра время выводится в наносекундах.

Исходный код `vmscan(8)`:

```
#!/usr/local/bin/bpftrace
```

```
tracepoint:vmscan:mm_shrink_slab_start { @start_ss[tid] = nsecs; }
tracepoint:vmscan:mm_shrink_slab_end /@start_ss[tid]/
{
    $dur_ss = nsecs - @start_ss[tid];
    @sum_ss = @sum_ss + $dur_ss;
    @shrink_slab_ns = hist($dur_ss);
    delete(@start_ss[tid]);
}

tracepoint:vmscan:mm_vmscan_direct_reclaim_begin { @start_dr[tid] = nsecs; }
tracepoint:vmscan:mm_vmscan_direct_reclaim_end /@start_dr[tid]/
{
    $dur_dr = nsecs - @start_dr[tid];
    @sum_dr = @sum_dr + $dur_dr;
    @direct_reclaim_ns = hist($dur_dr);
    delete(@start_dr[tid]);
}

tracepoint:vmscan:mm_vmscan_memcg_reclaim_begin { @start_mr[tid] = nsecs; }
```

```

tracepoint:vmscan:mm_vmscan_memcg_reclaim_end /@start_mr[tid]/
{
    $dur_mr = nsecs - @start_mr[tid];
    @sum_mr = @sum_mr + $dur_mr;
    @memcg_reclaim_ns = hist($dur_mr);
    delete(@start_mr[tid]);
}

tracepoint:vmscan:mm_vmscan_wakeup_kswapd { @count_wk++; }

tracepoint:vmscan:mm_vmscan_writepage { @count_wp++; }

BEGIN
{
    printf("%-10s %-10s %-12s %-12s %6s %9s\n", "TIME",
        "S-SLABms", "D-RECLAIMms", "M-RECLAIMms", "KSWAPD", "WRITEPAGE");
}

interval:s:1
{
    time("%H:%M:%S");
    printf(" %10d %12d %12d %6d %9d\n",
        @sum_ss / 1000000, @sum_dr / 1000000, @sum_mr / 1000000,
        @count_wk, @count_wp);
    clear(@sum_ss);
    clear(@sum_dr);
    clear(@sum_mr);
    clear(@count_wk);
    clear(@count_wp);
}

```

Этот инструмент использует разные точки трассировки в `vmscan` для определения времени начала событий, чтобы подсчитать их продолжительности и сохранить в гистограмме.

7.3.9. drsnoop

`drsnoop(8)`¹ — это инструмент БСС для трассировки событий непосредственной утилизации памяти, показывающий затронутый процесс и задержку: время, затраченное на утилизацию. Его можно использовать для количественной оценки влияния приложения на производительность системы с ограниченным объемом памяти. Например:

```

# drsnoop -T
TIME(s)      COMM      PID      LAT(ms)  PAGES
0.000000000  java      11266    1.72     57
0.004007000  java      11266    3.21     57
0.011856000  java      11266    2.02     43
0.018315000  java      11266    3.09     55
0.024647000  acpid     1209     6.46     73
[...]

```

¹ Немного истории: был создан в Etherflow 10 февраля 2019 года.

Как показывают эти результаты, для процесса Java были события непосредственной утилизации памяти, длившиеся от 1 до 7 миллисекунд. Частота этих операций и их продолжительность могут учитываться при количественной оценке влияния на затронутое приложение.

В своей работе инструмент использует точки трассировки в функции `vmscan: mm_vm_scan_direct_reclaim_begin` и `mm_vm_scan_direct_reclaim_end`. Предполагается, что эти события будут возникать нечасто (как правило, пакетами), поэтому оверхед должен быть незначительным.

Порядок использования:

```
drsnnoop [options]
```

Параметры `options`:

- **-T**: включать в вывод отметки времени;
- **-p PID**: трассировать только этот процесс.

7.3.10. swapin

`swapin(8)`¹ показывает, какие процессы были загружены в память с устройств подкачки, если они есть и используются. Например, в этой системе часть памяти была вытеснена в устройство подкачки и затем 36 Кбайт были загружены обратно (столбец «si»), пока я использовал `vmstat(1)`:

```
# vmstat 1
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so    bi    bo   in   cs us sy id wa st
[...]
```

46	11	29696	1585680	4384	1828440	0	0	88047	2034	21809	37316	81	18	0	1	0
776	57	29696	2842156	7976	1865276	36	0	52832	2283	18678	37025	85	15	0	1	0
294	135	29696	448580	4620	1860144	0	0	36503	5393	16745	35235	81	19	0	0	0

```
[...]
```

`swapin(8)` выявляет процессы, загруженные в оперативную память с устройства подкачки. В то же время:

```
# swapin.bt
Attaching 2 probes...
[...]
```

06:57:43

06:57:44

¹ Немного истории: впервые похожий инструмент (`anonpgpid.d`) я создал 25 июля 2005 года с помощью Джеймса Диккенса (James Dickens), когда исследовал одну из застарелых проблем с производительностью: я видел, что система вытесняла память в устройство подкачки, но хотелось узнать, какие процессы были при этом затронуты. Эту версию для `bpftrace` я создал 26 января 2019 года, работая над этой книгой.


```
@[systemd-logind, 1354]: 9
```

```
06:57:45
```

```
[...]
```

Этот вывод показывает, что процесс `systemd-logind` (PID 1354) загружался в память с устройства подкачки 9 раз. При размере страницы 4 Кбайт общий объем загруженной памяти, видимый в `vmstat(1)`, составил до 36 Кбайт. Я вошел в систему через `ssh(1)`, и этот компонент ПО, реализующего вход, был вытеснен в устройство подкачки, из-за чего регистрация заняла больше времени, чем обычно.

Загрузка с устройства подкачки происходит, когда приложение пытается использовать память, которая была вытеснена в устройство подкачки. Это важная метрика, характеризующая потери производительности из-за вытеснения приложения в устройство подкачки. Другие метрики, такие как сканирование и выгрузка, не оказывают такого же непосредственного влияния на производительность приложения.

Исходный код `swarin(8)`:

```
#!/usr/local/bin/bpftrace

kprobe:swap_readpage
{
    @[comm, pid] = count();
}

interval:s:1
{
    time();
    print(@);
    clear(@);
}
```

Этот инструмент использует зонды `kprobes` для трассировки функции ядра `swap_readpage()`, которая выполняется в контексте потока подкачки, поэтому встроенные в `bpftrace` карты `comm` и `pid` отражают затронутый процесс.

7.3.11. hfaults

`hfaults(8)`¹ трассирует сбои огромных (`huge`) страниц по процессам и применяется для подтверждения использования таких страниц. Например:

```
# hfaults.bt
Attaching 2 probes...
Tracing Huge Page faults per process... Hit Ctrl-C to end.
^C
@[884, huge mmap]: 9
```

¹ Немного истории: этот инструмент был создан Амером Атером (Amer Ather) для этой книги 6 мая 2019 года.

В этом выводе содержится информация о тестовой программе `greatmmap` (PID 884), которая вызвала девять сбоев огромных страниц.

Исходный код `hfaults(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing Huge Page faults per process... Hit Ctrl-C to end.\n");
}

kprobe:hugetlb_fault
{
    @[pid, comm] = count();
}
```

При необходимости из аргументов функции можно получить дополнительные сведения, включая структуры `mm_struct` и `vm_area_struct`. Например, инструмент `hfaults(8)` (см. раздел 7.3.7) извлекает имя файла из `vm_area_struct`.

7.3.12. Другие инструменты

Вот еще два стоящих инструмента BPF:

- **llcstat(8)** из проекта ВСС рассматривался в главе 5. Сообщает частоту попаданий в кэш последнего уровня для каждого процесса.
- **profile(8)** из проекта ВСС рассматривался в главе 5. Выбирает трассировки стека и может использоваться как грубая и недорогая замена трассировке вызовов `malloc()`.

7.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечисляются однострочные сценарии для ВСС и `bpftrace`. Там, где это возможно, один и тот же сценарий реализуется с использованием ВСС и `bpftrace`.

7.4.1. ВСС

Подсчитывает события расширения кучи по процессам (`brk()`) путем трассировки стека в пространстве пользователя:

```
stackcount -U t:syscalls:sys_enter_brk
```

Подсчитывает сбои страниц путем трассировки стека в пространстве пользователя:

```
stackcount -U t:exceptions:page_fault_user
```

Подсчитывает операции в `vmscan` с использованием точек трассировки:

```
funccount 't:vmscan:*
```

Выводит события вызова `hugepage_madvise()` по процессам:

```
trace hugepage_madvise
```

Подсчитывает число миграций страниц:

```
funccount t:migrate:mm_migrate_pages
```

Трассирует события сжатия:

```
trace t:compaction:mm_compaction_begin
```

7.4.2. bpftrace

Подсчитывает события расширения кучи по процессам (`brk()`) по трассировкам стека:

```
bpftrace -e tracepoint:syscalls:sys_enter_brk { @[ustack, comm] = count(); }
```

Подсчитывает сбои страниц по процессам:

```
bpftrace -e 'software:page-fault:1 { @[comm, pid] = count(); }'
```

Подсчитывает сбои страниц путем трассировки стека в пространстве пользователя:

```
bpftrace -e 'tracepoint:exceptions:page_fault_user { @[ustack, comm] = count(); }'
```

Подсчитывает операции в `vmscan` с использованием точек трассировки:

```
bpftrace -e 'tracepoint:vmscan:* { @[probe] = count(); }'
```

Выводит события вызова `hugepage_madvise()` по процессам:

```
bpftrace -e 'kprobe:hugepage_madvise { printf("%s by PID %d\n", probe, pid); }'
```

Подсчитывает число миграций страниц:

```
bpftrace -e 'tracepoint:migrate:mm_migrate_pages { @ = count(); }'
```

Трассирует события сжатия:

```
bpftrace -e 't:compaction:mm_compaction_begin { time(); }'
```

7.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Упражнения можно выполнить с помощью `bpftrace` или ВСС, если явно не указано иное:

1. Запустите `vmscan(8)` на локальном или производственном сервере и продолжайте трассировку в течение 10 минут. Если обнаружатся события непосредственной

- утилизации памяти (D-RECLAIMs), запустите также `drsnop`(8), чтобы получить метрики для каждого события.
2. Измените реализацию `vmscan`(8) и организуйте вывод заголовка через каждые 20 строк, чтобы он был постоянно виден на экране.
 3. Используйте `fault`(8) во время запуска приложения (на производственном сервере или на ПК), чтобы подсчитать трассировки стека, ведущие к сбою страниц. Для выполнения этого упражнения вам может понадобиться приложение, поддерживающее трассировку стека и символы (см. главы 13 и 18).
 4. Создайте флейм-график сбоев страниц из результатов, полученных в упражнении 3.
 5. Разработайте инструмент, определяющий рост объема виртуальной памяти процесса путем трассировки `brk`(2) и `mmap`(2).
 6. Разработайте инструмент для вывода величины расширения кучи путем трассировки `brk`(2). При желании можете использовать точки трассировки системного вызова, зонды `kprobes` или `USDT` в библиотеке `libc`.
 7. Разработайте инструмент, отображающий время, затраченное на сжатие страниц. Можете использовать точки трассировки `compaction:mm_compaction_begin` и `compaction:mm_compaction_end`. Выведите время для каждого события и обобщите их в виде гистограммы.
 8. Разработайте инструмент, отображающий время, затраченное на сокращение блоков памяти в ядре, с разбивкой по их именам (или имени функции).
 9. С помощью `memleak`(8) найдите долгоживущие блоки памяти в некоторых программах в тестовой среде. Также подсчитайте оверхед `memleak`(8), оценив производительность в периоды, когда производится и не производится трассировка.
 10. (Усложненное, не решено.) Разработайте инструмент для анализа процесса подкачки: отобразите время, проведенное страницами в устройстве подкачки, в виде гистограммы. Скорее всего, потребуется измерить время от вытеснения до загрузки.

7.6. ИТОГИ

В этой главе кратко описано, как процессы используют виртуальную и физическую память, и показаны приемы анализа памяти с применением традиционных инструментов, отображающих объемы памяти по видам использования. Здесь также рассказано, как использовать инструменты BPF для измерения частоты и продолжительности работы OOM Killer, об операциях распределения памяти в пространстве пользователя, отображения памяти, о сбоях страниц, вызовах `vmscan`, непосредственной утилизации и загрузке из устройства подкачки.

Глава 8

ФАЙЛОВЫЕ СИСТЕМЫ

Анализ файловых систем исторически сосредоточен на дисковом вводе/выводе и его производительности, но файловые системы как таковые — это актуальные цели для начала анализа. Приложения часто взаимодействуют с файловыми системами напрямую, и файловые системы могут использовать кэширование, упреждающее чтение, буферизацию и асинхронный ввод/вывод, чтобы уменьшить задержки в операциях дискового ввода/вывода, выполняемых приложениями.

Поскольку для анализа файловой системы есть не так много традиционных инструментов, трассировка с использованием средств BPF может существенно помочь в этой области. Инструменты трассировки файловой системы способны измерять время ожидания приложений во время выполнения операций ввода/вывода, включая дисковый ввод/вывод, продолжительность блокировок и другие характеристики использования CPU. С их помощью можно определить процессы и файлы, с которыми они работают: это очень полезная информация, которую намного сложнее получить на дисковом уровне.

Цели обучения:

- познакомиться с компонентами файловых систем: VFS, кэшами и отложенной записью (write-back);
- понять цели анализа файловых систем с применением инструментов BPF;
- разобраться с успешной стратегией анализа производительности файловых систем;
- определять характеристики нагрузки на файловые системы по файлам, типам операций и процессам;
- оценивать распределение задержек в операциях с файловыми системами, выявлять бимодальные распределения и аномальные задержки;
- определять задержки, связанные с событиями отложенной записи;
- оценивать производительность кэша страниц и механизма опережающего чтения;
- познакомиться с приемами наблюдения за поведением кэшей каталогов и индексных узлов (inode);
- использовать однострочные сценарии `bpfttrace` для анализа работы файловых систем нестандартными способами.

В начале этой главы мы обсудим некоторые теоретические основы, необходимые для анализа файловой системы, уделив основное внимание стеку ввода/вывода и кэшированию. Попутно исследуем вопросы, на которые может ответить BPF, а также разберем общую стратегию анализа. Затем перейдем к традиционным инструментам анализа файловой системы и рассмотрим инструменты BPF, включая однострочные сценарии для BPF. В конце главы вас ждут дополнительные упражнения.

8.1. ОСНОВЫ

В этом разделе мы познакомимся с основными принципами работы файловых систем, возможностями BPF и оптимальной стратегией анализа производительности файловых систем.

8.1.1. Основы файловых систем

Стек ввода/вывода

На рис. 8.1 показана обобщенная структура стека ввода/вывода, где вы видите путь от приложения до дискового устройства.

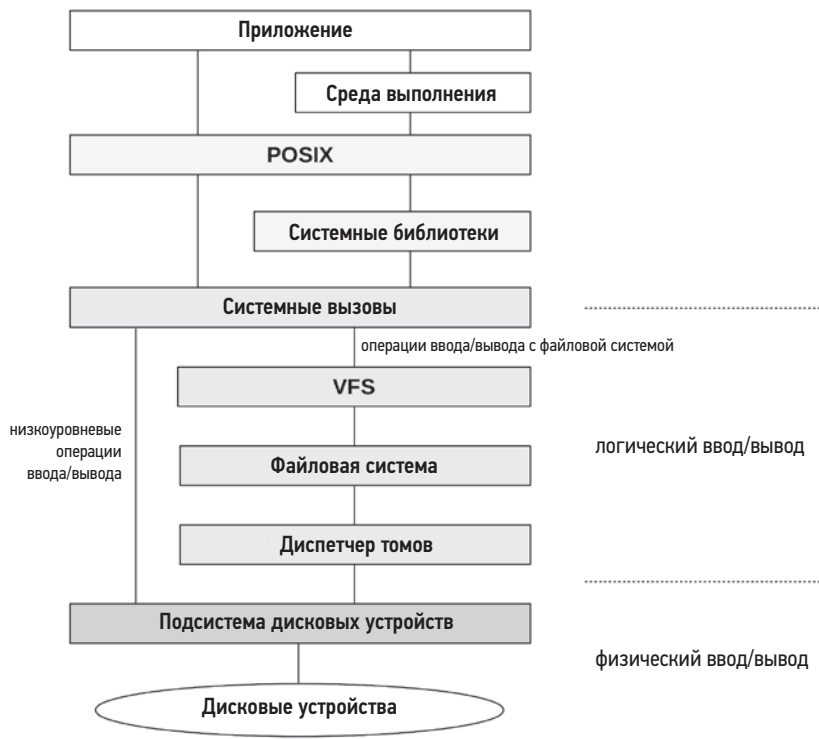


Рис. 8.1. Обобщенная структура стека ввода/вывода

Уточнения по использованной здесь терминологии: под *логическим вводом/выводом* подразумеваются запросы к файловой системе. Если в обработку этих запросов должны быть вовлечены устройства хранения, они преобразуются в *физический ввод/вывод*. Но не все логические запросы превращаются в операции физического ввода/вывода — многие логические запросы на чтение могут быть обслужены из кэша файловой системы и никогда не превратятся в физический ввод/вывод. В диаграмму включены также *низкоуровневые операции ввода/вывода*, хотя сейчас они используются очень редко: с их помощью приложения могут использовать дисковые устройства без файловой системы.

Доступ к файловым системам осуществляется через виртуальную файловую систему (Virtual File System, VFS) — обобщенный интерфейс ядра, который поддерживает множества разных файловых систем с использованием единого набора функций и позволяет с легкостью добавлять новые файловые системы. Этот интерфейс предоставляет операции для чтения, записи, открытия, закрытия и т. д., которые отображаются в функции фактических файловых систем.

Ниже файловой системы может находиться диспетчер томов, управляющий устройствами хранения. Есть также подсистема блочного ввода/вывода, управляющая вводом/выводом на устройства, включая обслуживание очередей, поддержку слияния операций и многое другое. Все это более подробно описано в главе 9.

Кэши файловых систем

Для увеличения производительности ввода/вывода файловые системы в Linux используют множество кэшей, как показано на рис. 8.2.

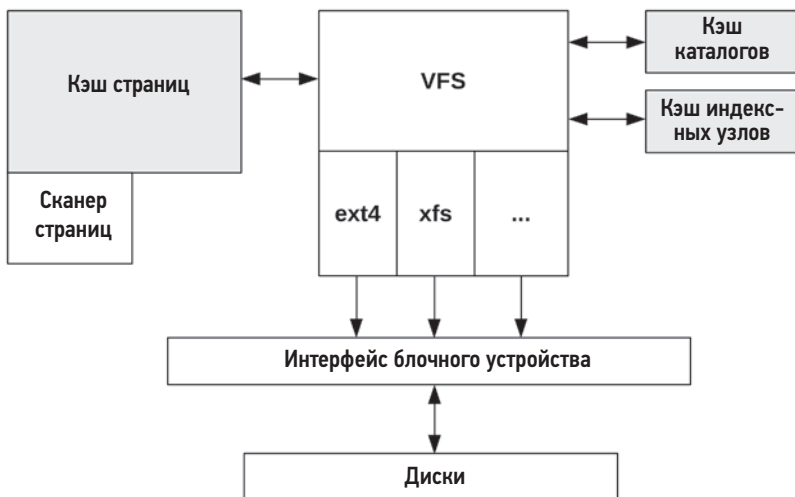


Рис. 8.2. Кэши файловых систем в Linux

Вот эти кэши:

- **Кэш страниц (page cache):** содержит страницы виртуальной памяти, включая содержимое файлов и буферы ввода/вывода (которые прежде хранились в отдельном «кэше буферов»), и увеличивает производительность операций ввода/вывода с файлами и каталогами.
- **Кэш индексных узлов (inode cache):** индексные узлы (inode) — это структуры данных, которые файловые системы используют для описания своих хранимых объектов. VFS имеет свою обобщенную версию индексного узла, и Linux хранит кэш этих узлов, потому что они часто используются для проверки разрешений и других метаданных.
- **Кэш каталогов (directory cache):** называется dcache и кэширует соответствия между именами каталогов и индексными узлами VFS, улучшая производительность поиска в каталогах.

Кэш страниц становится самым большим из всех, потому что кэширует не только содержимое файлов. Он также включает «грязные» страницы, которые были изменены, но еще не записаны на диск. Инициировать запись «грязных» страниц могут разные события, включая истечение установленного интервала времени (например, 30 секунд), явный вызов `sync()`, а также внутренние события демона подкачки `kswapd`, описанного в главе 7.

Опережающее чтение

Функция файловой системы под названием «опережающее чтение», или «предварительная выборка», определяет рабочие нагрузки, осуществляющие последовательное чтение, прогнозирует вероятность чтения следующих страниц и загружает их в кэш страниц. Такой предварительный «подогрев кэша» повышает производительность операций чтения в рабочих нагрузках, которые последовательно читают данные из файлов. В Linux есть и специальный системный вызов `readahead()`.

Отложенная запись

Linux поддерживает запись в файловую систему в отложенном режиме, когда буферы обновляются в памяти и через некоторое время сбрасываются на диск рабочими потоками ядра, что позволяет избежать блокировки приложений при работе с медленными устройствами ввода/вывода.

Для дальнейшего чтения

Это был лишь краткий обзор, чтобы снабдить вас минимальными знаниями перед использованием инструментов. Более подробное обсуждение файловых систем ищите в главе 8 моей книги «Systems Performance»¹ [Gregg 13b].

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

8.1.2. Возможности BPF

Традиционные инструменты позволяют проанализировать производительность дисковых операций ввода/вывода, но не файловых систем. Инструменты трассировки BPF способны восполнить этот недостаток и предоставить дополнительную информацию об операциях, задержках и характере работы каждой файловой системы.

BPF помогает найти ответы на следующие вопросы:

- Какие запросы отправляются файловым системам? Сколько запросов каждого типа посылается?
- Блоки каких размеров читаются из файловой системы?
- Сколько операций записи было выполнено в синхронном режиме?
- Как та или иная рабочая нагрузка обращается к файлам: произвольно или последовательно?
- Какие файлы используются? Какими процессами или какими путями в коде? Каков объем ввода/вывода в байтах, в операциях ввода/вывода?
- Какие ошибки возникают в файловых системах? Какого типа и в каких процессах?
- Где в файловой системе возникают задержки? Диски, пути в коде, блокировки?
- Как распределяются задержки в файловой системе?
- Как соотносится доля попаданий и промахов кэшей каталогов (Dcache) и индексных узлов (Icache)?
- Каков коэффициент попадания в кэш страниц в операциях чтения?
- Насколько эффективна предварительная выборка/опережающее чтение? Требуется ли настройка этой функции?

Ответить на многие из этих вопросов можно, выполнив трассировку ввода/вывода.

Источники событий

В табл. 8.1 перечислены типы ввода/вывода и источники событий, которые можно инструментировать для их трассировки.

Таблица 8.1. Типы ввода/вывода и источники событий

Типы событий	Источники событий
Операции ввода/вывода в приложениях и библиотеках	uprobes
Системные вызовы ввода/вывода	точки трассировки системных вызовов
Ввод/вывод в файловых системах	точки трассировки для ext4 (...), kprobes
Попадания в кэш (чтение), отложенная запись	kprobes

Таблица 8.1 (окончание)

Типы событий	Источники событий
Прوماхи кэша (чтение), немедленная запись	kprobes
Отложенная запись из кэша страниц	точки трассировки отложенной записи
Физический дисковый ввод/вывод	точки трассировки блочного ввода/вывода, kprobes
Низкоуровневый ввод/вывод	kprobes

Эти источники событий позволяют наблюдать, как протекают операции ввода/вывода от приложения до устройств. За вводом/выводом в файловых системах можно наблюдать с использованием точек трассировки в этих файловых системах, в зависимости от их типов. Например, файловая система ext4 предоставляет более ста точек трассировки.

Оверхед

Логический ввод/вывод, особенно чтение и запись в кэш файловой системы, может выполняться очень часто: более 100 000 событий в секунду. Будьте внимательны при их трассировке, потому что снижение производительности при такой частоте может стать очень заметным. Будьте внимательны и выполняя трассировку VFS: VFS также используется многими сетевыми операциями ввода/вывода, что увеличивает издержки на прием/отправку пакетов, которые могут производиться с большой частотой¹.

Физический дисковый ввод/вывод на большинстве серверов обычно производится настолько редко (менее 1000 операций в секунду), что его трассировка дает незначительный оверхед. Исключением могут быть некоторые серверы хранения и базы данных: предварительно проверьте частоту ввода/вывода с помощью iostat(1).

8.1.3. Стратегия

Если вы новичок в анализе производительности памяти, ниже представлена рекомендуемая общая стратегия. Упомянутые здесь инструменты более подробно рассматриваются в последующих разделах.

1. Определите смонтированные файловые системы: см. df(1) и mount(8).
2. Определите емкость смонтированных файловых систем: в прошлом проблемы с производительностью часто возникали из-за того, что некоторые файловые

¹ В Linux есть программные и аппаратные механизмы сегментирования для уменьшения общего числа пакетов и снижения нагрузки, поэтому частота событий может быть намного меньше количества пакетов, передаваемых по проводам; см. описание инструмента netsize(8) в главе 10.

системы были заполнены на 100% вследствие использования различных алгоритмов поиска свободных блоков (например, FFS, ZFS¹).

3. Для исследования неизвестной рабочей нагрузки вместо незнакомых инструментов BPF попробуйте сначала использовать те, что вы применяли для анализа известной рабочей нагрузки. В ничем не занятой системе создайте известную рабочую нагрузку для файловой системы, например, с помощью инструмента `fio(1)`.
4. Воспользуйтесь инструментом `opensnoop(8)`, чтобы определить, какие файлы открываются.
5. Воспользуйтесь инструментом `filelife(8)`, чтобы выявить возможные проблемы с короткоживущими файлами.
6. Найдите необычно медленные операции ввода/вывода и изучите особенности процесса и файла (например, с помощью `ext4slower(8)`, `btrfs slower(8)`, `zfs slower(8)` и т. д. или более универсального, но более дорогостоящего в смысле оверхеда инструмента `fileslower(8)`). Это поможет выявить рабочую нагрузку, которую можно устранить, или же количественно оценить проблему и понять, какие параметры файловой системы можно настроить.
7. Изучите распределение задержки в файловых системах (например, с помощью `ext4dist(8)`, `btrfssdist(8)`, `zfsdist(8)` и т. д.). Это поможет выявить бимодальные распределения и аномальные задержки, вызывающие проблемы с производительностью, которые можно изолировать и более тщательно исследовать с помощью других инструментов.
8. Посмотрите, как изменяется коэффициент попадания в кэш страниц со временем (например, с помощью `cachestat(8)`): влияет ли какая-либо другая рабочая нагрузка на коэффициент попаданий или, может быть, какая-то настройка улучшает его?
9. С помощью `vfsstat(8)` и `iostat(1)` сравните частоту операций логического и физического ввода/вывода: в идеале частота операций логического ввода/вывода должна быть намного выше, чем физического, что свидетельствует об эффективности кэширования.

Попробуйте использовать инструменты BPF, перечисленные в разделе «Инструменты BPF» далее в этой главе.

¹ Правило «заполнение `zpool` не должно превышать 80%», хотя мне при создании продуктов хранения доводилось увеличивать его заполнение до 99%. Также см. раздел «Pool performance can degrade when a pool is very full» в руководстве «ZFS Recommended Storage Pool Practices» [83].

8.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Поскольку раньше при анализе основное внимание уделялось дисковым операциям, было создано несколько инструментов для наблюдения за файловыми системами. В этом разделе кратко описаны приемы анализа файловых систем с использованием инструментов `df(1)`, `mount(1)`, `strace(1)`, `perf(1)` и `fatrace(1)`.

Обратите внимание, что анализ производительности файловых систем обычно считался сферой инструментов микробенчмаркинга, а не наблюдаемости. В числе таких инструментов можно назвать `fio(1)`.

8.2.1. df

`df(1)` сообщает параметры использования диска:

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            93G   0    93G   0% /dev
tmpfs           19G  4.0M   19G   1% /run
/dev/nvme0n1    9.7G  5.1G   4.6G  53% /
tmpfs           93G   0    93G   0% /dev/shm
tmpfs           5.0M   0   5.0M   0% /run/lock
tmpfs           93G   0    93G   0% /sys/fs/cgroup
/dev/nvme1n1    120G  18G   103G  15% /mnt
tmpfs           19G   0    19G   0% /run/user/60000
```

В этот вывод включены также виртуальные файловые системы, смонтированные с использованием устройства `tmpfs`, которые используются для хранения состояния системы.

Проверьте процент заполнения дисковых файловых систем (столбец «Use%»). Например, в результатах выше это файловые системы, смонтированные в каталоги «/» и «/mnt». Они заполнены на 53% и 15% соответственно. Как только файловая система заполнится примерно на 90%, производительность операций в ней может упасть из-за уменьшения количества доступных свободных блоков и их разрозненности, что превратит рабочие нагрузки последовательной записи в рабочие нагрузки произвольной записи. Этого может и не произойти — многое зависит от реализации файловой системы, но проверить эту особенность стоит.

8.2.2. mount

Команда `mount(1)` делает файловые системы доступными, а также может выводить их типы и флаги монтирования:

```
$ mount
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime,gid=60243,hidepid=2)
```


8.2.4. perf

perf(1) — это многоцелевой инструмент Linux, позволяющий использовать точки трассировки файловых систем, зонды kprobes в реализации VFS и внутренних компонентов файловых систем. Имеет подкоманду trace — более эффективную версию strace(1). Например:

```
# perf trace cksum /usr/bin/cksum
[...]
```

0.683 (0.013 ms):	cksum/20905 openat(dfd: CWD, filename: 0x4517a6cc)	= 3
0.698 (0.002 ms):	cksum/20905 fadvise64(fd: 3, advice: 2)	= 0
0.702 (0.002 ms):	cksum/20905 fstat(fd: 3, statbuf: 0x7fff45169610)	= 0
0.713 (0.059 ms):	cksum/20905 read(fd: 3, buf: 0x7fff45169790, count: 65536)	= 35000
0.774 (0.002 ms):	cksum/20905 read(fd: 3, buf: 0x7fff45172048, count: 28672)	= 0
0.875 (0.002 ms):	cksum/20905 lseek(fd: 3, whence: CUR)	= 35000
0.879 (0.002 ms):	cksum/20905 close(fd: 3)	= 0

```
[...]
```

Вывод команды perf trace улучшается в каждой версии Linux (в примере выше показан вывод команды в Linux 5.0). Для улучшения вывода Арнальдо Карвальо де Мело (Arnaldo Carvalho de Melo) использовал синтаксический анализ заголовочных файлов ядра и BPF [84]. В будущих версиях кроме указателя на имя файла должна выводиться строка с этим именем.

Более известные подкоманды perf(1) — stat и record — можно использовать с точками трассировки файловых систем, если они есть. Вот как можно подсчитать обращения к файловой системе ext4 в масштабе всей системы, используя ее точки трассировки:

```
# perf stat -e 'ext4:*' -a
^C
Performance counter stats for 'system wide':
```

0	ext4:ext4_other_inode_update_time
1	ext4:ext4_free_inode
1	ext4:ext4_request_inode
1	ext4:ext4_allocate_inode
1	ext4:ext4_evict_inode
1	ext4:ext4_drop_inode
163	ext4:ext4_mark_inode_dirty
1	ext4:ext4_begin_ordered_truncate
0	ext4:ext4_write_begin
260	ext4:ext4_da_write_begin
0	ext4:ext4_write_end
0	ext4:ext4_journalled_write_end
260	ext4:ext4_da_write_end
0	ext4:ext4_writepages
0	ext4:ext4_da_write_pages

```
[...]
```

Файловая система ext4 предлагает около ста точек трассировки для анализа запросов к ней и особенностей работы ее внутренних компонентов. Каждая из них

имеет строку формата для вывода соответствующей информации, например (**не запускайте эту команду**):

```
# perf record -e ext4:ext4_da_write_begin -a
^C[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 1376.293 MB perf.data (14394798 samples) ]
```

Это не самый удачный пример, но это важный урок для трассировки файловой системы. Дело в том, что `perf record` записывает события в файловую систему. Если при этом делать трассировку операций записи в файловую систему (или на диск), можно по неосторожности создать цикл, как в примере выше. Здесь было сделано 14 миллионов выборок, и сгенерировался файл `perf.data` с размером в 1.3 Гбайт!

Вот как выглядит строка формата для этого примера:

```
# perf script
[...]
```

pid	tid	event	target	dev	ino	pos	len	flags
26768	[005]	275068.339717:	ext4:ext4_da_write_begin:	253,1	1967479	pos	5260704	192
26768	[005]	275068.339723:	ext4:ext4_da_write_begin:	253,1	1967479	pos	5260896	8
26768	[005]	275068.339729:	ext4:ext4_da_write_begin:	253,1	1967479	pos	5260904	192
26768	[005]	275068.339735:	ext4:ext4_da_write_begin:	253,1	1967479	pos	5261096	8

```
[...]
```

Строка формата (выделена жирным шрифтом) включает устройство, индексный узел, позицию, длину и флаги записи.

Файловые системы могут поддерживать много точек трассировки, несколько или ни одной. Например, в XFS их около 500. Если в вашей файловой системе нет точек трассировки, попробуйте инструментировать внутренние компоненты зондами kprobes.

Для сравнения с более поздними инструментами BPF рассмотрим ту же точку трассировки, инструментированную с помощью bpftrace, чтобы обобщить аргумент длины в виде гистограммы:

```
# bpftrace -e 'tracepoint:ext4:ext4_da_write_begin { @ = hist(args->len); }'
```

[illegible]

Как видите, в большинстве случаев запись в файловую систему выполнялась блоками с размерами от 4 до 8 Кбайт. Эта информация обобщается в пространстве ядра, и для ее хранения не используется файл `perf.data`. Это не только уменьшает оверхед на запись данных в файловую систему, но и избавляет от дополнительного оверхеда на постобработку, а также от риска создать цикл.

8.2.5. `fatrace`

`fatrace(1)` — специализированный трассировщик, использующий прикладной интерфейс `fanotify` (file access notify — уведомления о попытках доступа к файлу) в Linux. Вот пример вывода этой команды:

```
# fatrace
cron(4794): CW /tmp/#9346 (deleted)
cron(4794): RO /etc/login.defs
cron(4794): RC /etc/login.defs
rsyslogd(872): W /var/log/auth.log
sshd(7553): O /etc/motd
sshd(7553): R /etc/motd
sshd(7553): C /etc/motd
[...]
```

В каждой строке выводятся: имя процесса, идентификатор процесса PID, тип события, полный путь и необязательный статус. Поддерживаются типы событий: (O) — открытие (open), (R) — чтение (read), (W) — запись (write) и (C) — закрытие (close). `fatrace(1)` можно использовать для определения характеристик рабочей нагрузки — имен файлов, к которым производится доступ, а также для поиска случаев, когда выполняется ненужная работа, которой можно избежать.

Для высоконагруженных файловых систем `fatrace(1)` может выводить десятки тысяч строк в секунду и расходовать значительные вычислительные ресурсы. Эту проблему можно немного смягчить фильтрацией по типу события, например выводить только события открытия:

```
# fatrace -f O
run(6383): O /bin/sleep
run(6383): RO /lib/x86_64-linux-gnu/ld-2.27.so
sleep(6383): O /etc/ld.so.cache
sleep(6383): RO /lib/x86_64-linux-gnu/libc-2.27.so
[...]
```

Ту же задачу можно решить с помощью специализированного инструмента `BPF` `opensnoop(8)`, который описан в следующем разделе: он предлагает больше параметров командной строки, а также действует намного эффективнее. Для примера сравним нагрузку на процессор, которую оказывают `fatrace -f O` и `opensnoop(8)` при анализе одной и той же значительной рабочей нагрузки на файловую систему:

```
# pidstat 10
[...]
```

09:38:54 PM	UID	PID	%usr	%system	%guest	%wait	%CPU	CPU	Command


```
09:39:04 PM    0  6075  11.19   56.44    0.00    0.20   67.63    1 fatrace
[...]  
09:50:32 PM    0  7079   0.90    0.20    0.00    0.00    1.10    2 opensnoop
[...]
```

Команда `opensnoop(8)` использовала 1.1% процессорного времени, тогда как `fatrace(1)` — 67%¹.

8.3. ИНСТРУМЕНТЫ BPF

В этом разделе описаны инструменты BPF, которые можно использовать для анализа производительности файловых систем и устранения проблем (рис. 8.3).

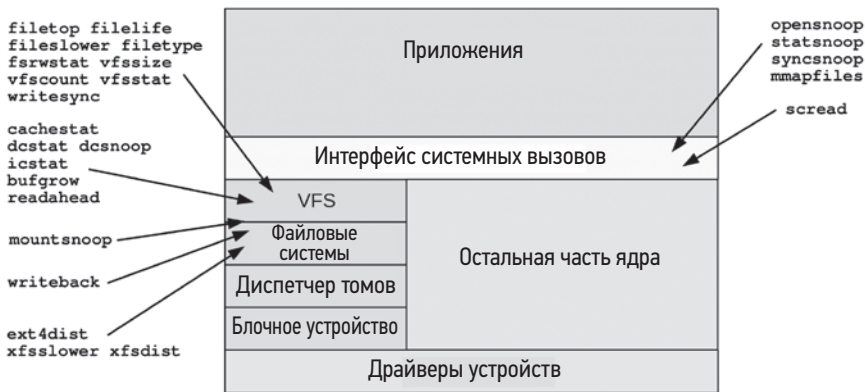


Рис. 8.3. Инструменты BPF для анализа производительности файловых

Часть этих инструментов можно найти в репозиториях ВСС и `bpfttrace`, упоминавшихся в главах 4 и 5, а часть была создана специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях, ВСС и `bpfttrace`. Рассматриваемые здесь инструменты перечислены в табл. 8.2, где также указано их происхождение (ВТ — это сокращение от «`bpfttrace`»).

Полные и актуальные списки параметров инструментов ВСС и `bpfttrace` и описание их возможностей ищите в соответствующих репозиториях. Ниже я расскажу только о наиболее важных особенностях.

В следующих подразделах вы также узнаете, как преобразовать дескрипторы в имена файлов (например, см. описание `scread(8)`).

¹ Здесь инструмент `opensnoop(8)` использовался без всяких настроек, как есть. Настроив цикл опроса (добавив задержку для увеличения буферизации), я смог уменьшить оверхед до 0.6%.

Таблица 8.2. Инструменты для анализа производительности файловых систем

Инструмент	Источник	Цель	Описание
opensnoop	БСС/ВТ	Системные вызовы	Трассирует события открытия файлов
statsnoop	БСС/ВТ	Системные вызовы	Трассирует вызовы вариантов функции stat(2)
syncsnoop	БСС/ВТ	Системные вызовы	Трассирует вызовы вариантов функции sync(2) с метками времени
mmapfiles	книга	Системные вызовы	Подсчитывает вызовы mmap(2) для файлов
scread	книга	Системные вызовы	Подсчитывает вызовы read(2) для файлов
fmapfault	книга	Кэш страниц	Посчитывает сбои отображения файлов
filelife	БСС/книга	VFS	Трассирует короткоживущие файлы и определяет продолжительность их жизни в секундах
vfsstat	БСС/ВТ	VFS	Определяет статистики, характеризующие работу VFS
vfscount	БСС/ВТ	VFS	Подсчитывает все операции с VFS
vfssize	книга	VFS	Определяет распределение размеров блоков в операциях чтения/записи
fsrwstat	книга	VFS	Подсчитывает операции чтения/записи по типам файловых систем
fileslower	БСС/книга	VFS	Определяет медленные операции чтения/записи
filetop	БСС	VFS	Определяет наиболее интенсивно используемые файлы
filetype	книга	VFS	Подсчитывает операции чтения/записи по типам файлов и процессам
writesync	книга	VFS	Подсчитывает операции записи с флагом sync в обычные файлы
cachestat	БСС	Кэш страниц	Выводит статистики, характеризующие использование кэша страниц
writeback	ВТ	Кэш страниц	Выводит события отложенной записи и задержки
dcstat	БСС/книга	Кэш каталогов	Выводит статистику попаданий в кэш каталогов
dcnoop	БСС/ВТ	Кэш каталогов	Трассирует операции поиска в кэше каталогов
mountsnoop	БСС	VFS	Трассирует события монтирования и размонтирования файловых систем в системе в целом
xfsslower	БСС	XFS	Определяет медленные операции с файловой системой XFS

Инструмент	Источник	Цель	Описание
xfsdist	ВСС	XFS	Выводит гистограммы типичных операций с файловой системой XFS
ext4dist	ВСС/книга	ext4	Выводит гистограммы типичных операций с файловой системой ext4
icstat	книга	Кэш индексных узлов	Выводит статистику попаданий в кэш индексных узлов
bufgrow	книга	Кэш буферов	Выводит события увеличения буферов по процессам
readahead	книга	VFS	Определяет попадания в кэш опережающего чтения и их эффективность

8.3.1. opensnoop

Инструмент `opensnoop(8)`¹ из глав 1 и 4 имеет версии для ВСС и `bpfttrace`. Он трассирует события открытия файлов и может использоваться для определения местоположения файлов данных, журналов и конфигураций. Он также обнаруживает проблемы с производительностью, вызванные частым открытием, и устраняет проблемы с отсутствующими файлами. Вот пример вывода, полученный в производственной системе, с параметром `-T` для включения в вывод отметок времени:

```
# opensnoop -T
TIME(s)    PID    COMM      FD ERR PATH
0.000000000 3862   java      5248   0 /proc/loadavg
0.000036000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs_quota_us
0.000051000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs_period_us
0.000059000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.shares
0.012956000 3862   java      5248   0 /proc/loadavg
0.012995000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs_quota_us
0.013012000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs_period_us
0.013020000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.shares
0.021259000 3862   java      5248   0 /proc/loadavg
0.021301000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs_quota_us
0.021317000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.cfs
0.021325000 3862   java      5248   0 /sys/fs/cgroup/cpu,cpuacct/.../cpu.shares
0.022079000 3862   java      5248   0 /proc/loadavg
[...]
```

¹ Немного истории: 9 мая 2004 года я создал первую версию под названием `opensnoop.d`. Это был простой и удобный инструмент, позволяющий увидеть все операции открытия файлов в системе в целом. До его создания для этой же цели я использовал утилиту `truss(1M)`, которая умеет трассировать только один процесс, или подсистему аудита BSM, требующую изменения состояния системы. Название «snoop» (разведывать, разносить, подсматривать. — *Примеч. пер.*) заимствовано из названия сетевого анализатора `snoop(1M)` в ОС Solaris и термина «snooping events». Со временем появились версии `opensnoop` для многих других трассировщиков, написанные мной и другими разработчиками. Я написал версию для ВСС 17 сентября 2015 года, а для `bpfttrace` — 8 сентября 2018 года.

В этом эксперименте строки выводились с высокой частотой, из них видно, что процесс Java обращается к группе из четырех файлов с частотой 100 раз в секунду (я только что обнаружил это¹). Имена файлов я частично урезал, чтобы уместить вывод по ширине книжной страницы. Эти файлы с системными метриками хранятся в памяти и операции их чтения должны выполняться быстро, но нужно ли процессу Java читать их 100 раз в секунду? Чтобы ответить на этот вопрос, я перешел к следующему шагу — получению соответствующих трассировок стека. Так как процесс Java открывал только эти файлы, я просто подсчитал трассировки стека в точке трассировки события открытия файлов для этого процесса:

```
stackcount -p 3862 't:syscalls:sys_enter_openat'
```

Эта команда вывела полную трассировку стека, включая соответствующие методы Java². Виновником оказалось новое ПО для балансировки нагрузки.

opensnoop(8) трассирует варианты системного вызова open(2): open(2) и openat(2). Оверхед на трассировку обычно незначителен, потому что open(2) вызывается не особенно часто.

BCC

Порядок использования:

```
opensnoop [options]
```

В числе параметров *options* можно назвать:

- **-x**: показать только неудачные вызовы open;
- **-p PID**: трассировать только этот PID;
- **-n NAME**: выводить только имена процессов, содержащие NAME.

bpfftrace

Ниже приводится реализация opensnoop(8) для bpfftrace, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpfftrace
```

```
BEGIN
{
    printf("Tracing open syscalls... Hit Ctrl-C to end.\n");
    printf("%-6s %-16s %4s %3s %s\n", "PID", "COMM", "FD", "ERR", "PATH");
}
```

¹ Я хотел попробовать запустить opensnoop на нескольких производственных серверах, чтобы найти интересный вывод для включения в книгу в качестве примера, и получил его с первой же попытки.

² Как получить трассировки стека и символы Java, рассказывается в главе 18.

```

tracepoint:syscalls:sys_enter_open,
tracepoint:syscalls:sys_enter_openat
{
    @filename[tid] = args->filename;
}

tracepoint:syscalls:sys_exit_open,
tracepoint:syscalls:sys_exit_openat
/@filename[tid]/
{
    $ret = args->ret;
    $fd = $ret > 0 ? $ret : -1;
    $errno = $ret > 0 ? 0 : - $ret;

    printf("%-6d %-16s %4d %3d %s\n", pid, comm, $fd, $errno,
        str(@filename[tid]));
    delete(@filename[tid]);
}

END
{
    clear(@filename);
}

```

Эта программа трассирует системные вызовы `open(2)` и `openat(2)` и извлекает дескриптор файла или код ошибки из возвращаемого значения. Имя файла кэшируется в точке входа, чтобы затем вывести в точке выхода из системного вызова вместе с возвращаемым значением.

8.3.2. statsnoop

`statsnoop(8)`¹ — это инструмент для BCC и `bpfttrace`, действующий как и `opensnoop(8)`, но производящий трассировку семейства системных вызовов `stat(2)`. `stat(2)` возвращает статистику файла. Этот инструмент можно использовать для решения тех же задач, что и `opensnoop(8)`: обнаружения местоположений файлов, поиска проблем с нагрузкой и устранения проблем с отсутствующими файлами. Вот пример вывода, полученный в производственной системе, с параметром `-t` для включения в вывод отметок времени:

```

# statsnoop -t
TIME(s)      PID    COMM                FD ERR PATH
0.000366347  9118   statsnoop           -1  2  /usr/lib/python2.7/encodings/ascii
0.238452415   744   systemd-resolve     0   0  /etc/resolv.conf
0.238462451   744   systemd-resolve     0   0  /run/systemd/resolve/resolv.conf
0.238470518   744   systemd-resolve     0   0  /run/systemd/resolve/stub-resolv.conf
0.238497017   744   systemd-resolve     0   0  /etc/resolv.conf
0.238506760   744   systemd-resolve     0   0  /run/systemd/resolve/resolv.conf

```

¹ Немного истории: первую версию инструмента на основе DTrace я создал 9 сентября 2007 года, как дополнение к `opensnoop`. Версию для BCC я написал 8 февраля 2016 года, а для `bpfttrace` — 8 сентября 2018 года.

```
0.238514099    744    systemd-resolve  0    0 /run/systemd/resolve/stub-resolv.conf
0.238645046    744    systemd-resolve  0    0 /etc/resolv.conf
0.238659277    744    systemd-resolve  0    0 /run/systemd/resolve/resolv.conf
0.238667182    744    systemd-resolve  0    0 /run/systemd/resolve/stub-resolv.conf
[...]
```

Здесь мы видим работу процесса `systemd-resolve` (на самом деле «`systemd-resolved`», имя было усечено при выводе), который в цикле вызывает `stat(2)` для тех же трех файлов.

Я обнаружил несколько случаев, когда без веской причины функция `stat(2)` вызывалась на производственных серверах десятки тысяч раз в секунду. К счастью, этот системный вызов выполняется очень быстро, поэтому такая частота обращений не вызывает серьезных проблем с производительностью. Но мне пришлось столкнуться с одним исключением, когда микросервис Netflix достиг 100% использования диска, вызванного постоянным обращением агента мониторинга к `stat(2)` в большой файловой системе, где кэшировались не все метаданные и поэтому обращения к `stat(2)` приводили к фактическим обращениям к диску.

Этот инструмент трассирует варианты системного вызова `stat(2)`: `statfs(2)`, `statx(2)`, `newstat(2)` и `newlstat(2)`. Оверхед на трассировку обычно незначителен, но может стать довольно заметным при очень высокой частоте обращений к `stat(2)`.

BCC

Порядок использования:

`statsnoop [options]`

Параметры `options`:

- **-x**: показать только неудачные вызовы `stat`;
- **-t**: добавить в вывод столбец с отметками времени (в секундах);
- **-p PID**: трассировать только этот PID.

bpfftrace

Ниже приводится реализация версии `statsnoop(8)` для `bpfftrace`, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpfftrace
```

```
BEGIN
```

```
{
    printf("Tracing stat syscalls... Hit Ctrl-C to end.\n");
    printf("%-6s %-16s %3s %s\n", "PID", "COMM", "ERR", "PATH");
}
```

```
tracepoint:syscalls:sys_enter_statfs
```

```
{
```

```

    @filename[tid] = args->pathname;
}

tracepoint:syscalls:sys_enter_statx,
tracepoint:syscalls:sys_enter_newstat,
tracepoint:syscalls:sys_enter_newlstat
{
    @filename[tid] = args->filename;
}

tracepoint:syscalls:sys_exit_statfs,
tracepoint:syscalls:sys_exit_statx,
tracepoint:syscalls:sys_exit_newstat,
tracepoint:syscalls:sys_exit_newlstat
/@filename[tid]/
{
    $ret = args->ret;
    $errno = $ret >= 0 ? 0 : - $ret;

    printf("%-6d %-16s %3d %s\n", pid, comm, $errno,
        str(@filename[tid]));
    delete(@filename[tid]);
}

END
{
    clear(@filename);
}

```

Программа сохраняет имя файла в точке входа в системный вызов и извлекает его в точке выхода, чтобы вывести вместе с другой информацией.

8.3.3. syncsnoop

`syncsnoop(8)`¹ — это инструмент для BCC и `bpfttrace`, отображающий информацию о вызовах `sync(2)` с отметками времени. Системный вызов `sync(2)` выталкивает измененные страницы на диск. Вот пример вывода версии для `bpfttrace`:

```

# syncsnoop.bt
Attaching 7 probes...
Tracing sync syscalls... Hit Ctrl-C to end.
TIME      PID      COMM      EVENT

```

¹ Немного истории: как-то я занимался устранением проблемы с операциями синхронизации. Они вызывали заметные задержки в работе приложения, когда операции чтения с диска ставились в очередь за множеством операций записи, инициированных вызовом `sync(8)`. Обычно обращения к `sync(8)` происходят нечасто, поэтому было достаточно получить фактическое время вызова, чтобы связать эти вызовы с провалами производительности, наблюдающимися на дашбордах мониторинга. Версию для BCC я создал 13 августа 2015 года, а для `bpfttrace` — 6 сентября 2018 года.

```

08:48:31  14172  TaskSchedulerFo  tracepoint:syscalls:sys_enter_fdatasync
08:48:31  14172  TaskSchedulerFo  tracepoint:syscalls:sys_enter_fdatasync
08:48:31  14172  TaskSchedulerFo  tracepoint:syscalls:sys_enter_fdatasync
08:48:31  14172  TaskSchedulerFo  tracepoint:syscalls:sys_enter_fdatasync
08:48:31  14172  TaskSchedulerFo  tracepoint:syscalls:sys_enter_fdatasync
08:48:40  17822  sync             tracepoint:syscalls:sys_enter_sync
[...]

```

Здесь мы видим работу процесса TaskSchedulerFo (это усеченное имя), вызывающего `fdatasync(2)` пять раз подряд. Обращения к `sync(2)` могут вызывать всплески операций дискового ввода/вывода и ухудшать производительность системы. Отметки времени выводятся в формате, позволяющем сопоставлять вызовы `sync(2)` с периодами ухудшения производительности, обнаруженными с помощью ПО для мониторинга.

Этот инструмент трассирует варианты системного вызова `sync(2)`: `sync(2)`, `syncfs(2)`, `fsync(2)`, `fdatasync(2)`, `sync_file_range(2)` и `msync(2)`. Оверхед на трассировку, как правило, незначителен, потому что обычно `sync(2)` вызывается нечасто.

BCC

Сейчас версия для BCC не поддерживает параметры командной строки и действует как и версия для `bpftrace`.

bpftrace

Ниже приведена реализация `syncsnoop(8)` для `bpftrace`:

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing sync syscalls... Hit Ctrl-C to end.\n");
    printf("%-9s %-6s %-16s %s\n", "TIME", "PID", "COMM", "EVENT");
}

tracepoint:syscalls:sys_enter_sync,
tracepoint:syscalls:sys_enter_syncfs,
tracepoint:syscalls:sys_enter_fsync,
tracepoint:syscalls:sys_enter_fdatasync,
tracepoint:syscalls:sys_enter_sync_file_range,
tracepoint:syscalls:sys_enter_msync
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %s\n", pid, comm, probe);
}

```

Если выяснится, что причина проблем — это вызовы `sync(2)`, их можно дополнительно изучить с помощью видоизмененной версии для `bpftrace`, отображающей

аргументы и возвращаемое значение. Также можно проанализировать фактически выполняющиеся операции дискового ввода/вывода.

8.3.4. mmapfiles

`mmapfiles(8)`¹ трассирует системный вызов `mmap(2)` и подсчитывает частоту обращений к файлу, отображенному в память. Например:

```
# mmapfiles.bt
Attaching 1 probe...
^C

@[usr, bin, x86_64-linux-gnu-ar]: 2
@[lib, x86_64-linux-gnu, libreadline.so.6.3]: 2
@[usr, bin, x86_64-linux-gnu-objcopy]: 2
[...]
@[usr, bin, make]: 226
@[lib, x86_64-linux-gnu, libz.so.1.2.8]: 296
@[x86_64-linux-gnu, gconv, gconv-modules.cache]: 365
@[/, bin, bash]: 670
@[lib, x86_64-linux-gnu, libtinfo.so.5.9]: 672
@[/, bin, cat]: 1152
@[lib, x86_64-linux-gnu, libdl-2.23.so]: 1240
@[lib, locale, locale-archive]: 1424
@[/, etc, ld.so.cache]: 1449
@[lib, x86_64-linux-gnu, ld-2.23.so]: 2879
@[lib, x86_64-linux-gnu, libc-2.23.so]: 2879
@[, , ]: 8384
```

В этом примере трассировка производилась во время сборки ПО. Для каждого файла выводятся его имя и имена двух родительских каталогов. В последней строке в этом примере нет имен файлов и каталогов: это анонимные отображения для приватных данных программы.

Исходный код `mmapfiles(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/mm.h>

kprobe:do_mmap
{
    $file = (struct file *)arg0;
    $name = $file->f_path.dentry;
    $dir1 = $name->d_parent;
    $dir2 = $dir1->d_parent;
    @[str($dir2->d_name.name), str($dir1->d_name.name),
     str($name->d_name.name)] = count();
}
```

¹ Немного истории: версию на основе DTgse я создал 18 октября 2005 года, а версию на основе bpftrace — 26 января 2019 года специально для этой книги.

В своей работе этот инструмент использует зонды kprobes для трассировки функции ядра `do_mmap()` и извлекает имя файла из аргумента `struct file*` через структуру `dentry` (запись в каталоге). В структуре `dentry` хранится только один компонент пути к файлу, поэтому, чтобы получить больше информации о местоположении файла, дополнительно извлекаются и выводятся имена родительского каталога и каталога, вмещающего родительский¹. Обычно обращения к `mmap()` случаются нечасто, поэтому оверхед этого инструмента будет незначительным.

Ключ агрегации (aggregation key) легко изменить и включить в него имя процесса, чтобы показать, какие процессы создают эти отображения ("`@ [comm, ...]`"), а также трассировку стека в пространстве пользователя, чтобы показать путь в коде ("`@ [comm, ustack, ...]`").

В главе 7 был представлен похожий инструмент для трассировки `mmap()` по типам событий: `mmapspnoor(8)`.

8.3.5. scread

`scread(8)`² трассирует системный вызов `read(2)` и выводит имена файлов, с которыми он работает. Например:

```
# scread.bt
Attaching 1 probe...
^C
@filename[org.chromium.BkPmzg]: 1
@filename[locale.alias]: 2
@filename[chrome_200_percent.pak]: 4
@filename[passwd]: 7
@filename[17]: 44
@filename[scriptCache-current.bin]: 48
[...]
```

Здесь видно, что при трассировке файл «`scriptCache-current.bin`» был прочитан с помощью `read(2)` 48 раз. Это представление файлового ввода/вывода на основе системных вызовов. Более поздний инструмент `filetop(8)` предназначен для представления на уровне VFS. Эти инструменты помогают получить характеристики использования файлов и обнаружить неэффективные операции.

Исходный код `scread(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/sched.h>
```

¹ Я предложил добавить в ядро BPF вспомогательную функцию, принимающую структуру `struct file` или `struct dentry` и возвращающую полный путь, как это делает функция `d_path()` в ядре Linux.

² Немного истории: я создал его для этой книги 26 января 2019 года.

```
#include <linux/fs.h>
#include <linux/fdtable.h>

tracepoint:syscalls:sys_enter_read
{
    $task = (struct task_struct *)curtask;
    $file = (struct file *)($task->files->fdt->fd + args->fd);
    @filename[str($file->f_path.dentry->d_name.name)] = count();
}
```

`scread(8)` извлекает имя файла из таблицы дескрипторов.

Поиск имени файла в таблице дескрипторов

Этот инструмент также был включен в книгу как пример получения имени файла по целочисленному дескриптору файла (FD). Сделать это можно как минимум двумя способами:

1. По ссылке в `task_struct` перейти к таблице дескрипторов файлов и найти структуру `file` по целочисленному дескриптору файла, в которой хранится и имя файла. Этот способ используется в `scread(2)`. Но это не самый надежный способ: поиск таблицы дескрипторов файлов (`task->files->fdt->fd`) основан на организации внутренних компонентов ядра, которая может измениться в будущем и сделать такой подход нерабочим¹.
2. Трассировать системный вызов `open(2)` и создать хеш для поиска с ключами PID и FD и значением, определяющим имя/путь файла. Этот хеш можно использовать при трассировке `read(2)` и других системных вызовов. Да, этот подход увеличивает число дополнительных зондов (и оверхед), зато он более надежный.

В этой книге описано много других инструментов (`fmapfault(8)`, `filelife(8)`, `vfssize(8)` и т. д.), которые извлекают имя файла при трассировке различных операций, но действуют через слой VFS, который предоставляет прямой доступ к структуре `file`. Это тоже не самый надежный интерфейс, но он позволяет получить строку с именем файла за меньшее количество шагов. Еще одно преимущество трассировки VFS — для каждой операции обычно есть только одна функция, тогда как в интерфейсе системных вызовов возможны варианты (например, `read(2)`, `readv(2)`, `preadv(2)`, `pread64()` и т. д.), все из них может потребоваться подвергнуть трассировке.

¹ Некоторые изменения уже рассматриваются. Дэйв Уотсон рассматривает возможность его перестановки для повышения производительности. Мэтью Уилокс также работает над реализацией схемы `task_struct->files_struct->maple_node->fd[i]`. [85], [86].

8.3.6. fmapfault

fmapfault(8)¹ трассирует сбои страниц, связанные с отображенными файлами, и производит подсчет по именам процессов и файлов. Например:

```
# fmapfault.bt
Attaching 1 probe...
^C

@[dirname, libc-2.23.so]: 1
@[date, libc-2.23.so]: 1
[...]
@[cat, libc-2.23.so]: 901
@[sh, libtinfo.so.5.9]: 962
@[sed, ld-2.23.so]: 984
@[sh, libc-2.23.so]: 997
@[cat, ld-2.23.so]: 1252
@[sh, ld-2.23.so]: 1427
@[as, libbfd-2.26.1-system.so]: 3984
@[as, libopcodes-2.26.1-system.so]: 68455
```

В этом примере трассировка производилась во время сборки ПО, и в выводе есть процессы и библиотеки, участвующие в сборке, в которых возникли сбои страниц.

Представленные далее инструменты: filetop(8), fileslower(8), xfsslower(8) и ext4dist(8) — показывают информацию о событиях файлового ввода/вывода, производя трассировку системных вызовов read(2) и write(2) (и их вариантов). Но это не единственный способ чтения файлов и записи в них: отображение файлов — еще один способ, позволяющий избежать явных обращений к системным вызовам. fmapfault(8) позволяет получить представление об их использовании, трассируя сбои страниц, соответствующих файлам, и создание новых отображений страниц. Обратите внимание: фактические операции чтения и записи в файлы могут следовать намного чаще, чем сбои.

Исходный код fmapfault(8):

```
#!/usr/local/bin/bpftrace

#include <linux/mm.h>

kprobe:filemap_fault
{
    $vf = (struct vm_fault *)arg0;
    $file = $vf->vma->vm_file->f_path.dentry->d_name.name;
    @[comm, str($file)] = count();
}
```

Этот инструмент использует зонды kprobes для трассировки функции ядра filemap_fault() и определяет имя отображаемого файла по ее аргументу struct vm_fault. Эти тонкости нужно учитывать, так как они могут быть затронуты изменениями в ядре.

¹ Немного истории: я создал этот инструмент специально для книги 26 января 2019 года.

Оверхед этого инструмента бывает весьма заметным в системах, где наблюдается высокая частота сбоев.

8.3.7. filelife

`filelife(8)`¹ — это инструмент для BCC и `bpfttrace`, отображающий продолжительность жизни короткоживущих файлов, созданных и удаленных в период, когда выполнялась трассировка.

Вот вывод версии `filelife(8)` для BCC, полученный во время, когда в системе выполнялась сборка ПО:

```
# filelife
TIME      PID      COMM          AGE(s)  FILE
17:04:51  3576     gcc           0.02    cc9JENsb.s
17:04:51  3632     rm            0.00    kernel.release.tmp
17:04:51  3656     rm            0.00    version.h.tmp
17:04:51  3678     rm            0.00    utsrelease.h.tmp
17:04:51  3698     gcc           0.01    ccTtEADr.s
17:04:51  3701     rm            0.00    .3697.tmp
17:04:51  736      systemd-udev  0.00    queue
17:04:51  3703     gcc           0.16    cc05cPSr.s
17:04:51  3708     rm            0.01    .purgatory.o.d
17:04:51  3711     gcc           0.01    ccgk4xfE.s
17:04:51  3715     rm            0.01    .stack.o.d
17:04:51  3718     gcc           0.01    ccPiK0gD.s
17:04:51  3722     rm            0.01    .setup-x86_64.o.d
[...]
```

Здесь мы видим множество короткоживущих файлов, созданных в процессе сборки, которые удалялись менее чем через секунду (столбец «AGE(s)» после создания).

Этот инструмент применялся для поиска путей оптимизации производительности: выявления случаев, когда приложения используют временные файлы, без которых можно обойтись.

Инструмент использует зонды `kprobes` для трассировки событий создания и удаления файлов через вызовы `vfs_create()` и `vfs_unlink()`. Оверхед инструмента должен быть незначительным, так как частота вызовов этих функций, как правило, относительно невысокая.

BCC

Порядок использования:

```
filelife [options]
```

¹ Немного истории: версию для BCC я создал 8 февраля 2015 года, когда занимался отладкой проблем, связанных с короткоживущими файлами. Версия для `bpfttrace` была написана 31 января 2019 года специально для этой книги. За основу был взят мой инструмент `vfslife.d` из книги о DTrace, вышедшей в 2011 году [Gregg 11].

Параметры options:

- **-p PID**: трассировать только этот PID.

bpfttrace

Ниже приведена реализация версии filelife(8) для bpfttrace:

```
#!/usr/local/bin/bpfttrace

#include <linux/fs.h>

BEGIN
{
    printf("%-6s %-16s %8s %s\n", "PID", "COMM", "AGE(ms)", "FILE");
}

kprobe:vfs_create,
kprobe:security_inode_create
{
    @birth[arg1] = nsecs;
}

kprobe:vfs_unlink
/@birth[arg1]/
{
    $dur = nsecs - @birth[arg1];
    delete(@birth[arg1]);
    $dentry = (struct dentry *)arg1;
    printf("%-6d %-16s %8d %s\n", pid, comm, $dur / 1000000,
        str($dentry->d_name.name));
}
```

Новые версии ядра могут не использовать `vfs_create()`, поэтому события создания файлов в них можно определять с помощью `security_inode_create()`, хука управления доступом (Linux Security Modules, LSM) для создания узла inode (если оба события происходят для одного и того же файла, то метка времени создания перезаписывается, но это не должно заметно влиять на измерение продолжительности жизни файла). Отметка времени создания хранится в ключе `arg1` этих функций, который является указателем `struct dentry`, и применяется в качестве уникального идентификатора. Имя файла также извлекается из структуры `dentry`.

8.3.8. vfststat

`vfstat(8)`¹ — это инструмент для BCC и bpfttrace, обобщающий статистику некоторых распространенных операций VFS: чтения и записи (ввод/вывод), создания, открытия и синхронизации. Эта информация помогает получить представление

¹ Немного истории: версию для BCC я создал 14 августа 2015 года, а версию для bpfttrace — 6 сентября 2018 года.

о характере рабочей нагрузки на уровне операций виртуальной файловой системы (VFS). Ниже показан пример использования `vfsstat(8)` для ВСС на рабочем сервере Hadoop с 36 процессорами:

```
# vfsstat
TIME      READ/s   WRITE/s  CREATE/s  OPEN/s    FSYNC/s
02:41:23: 1715013   38717    0          5379      0
02:41:24: 947879    30903    0         10547     0
02:41:25: 1064800   34387    0         57883     0
02:41:26: 1150847   36104    0          5105     0
02:41:27: 1281686   33610    0          2703     0
02:41:28: 1075975   31496    0          6204     0
02:41:29: 868243    34139    0          5090     0
02:41:30: 889394    31388    0          2730     0
02:41:31: 1124013   35483    0          8121     0
17:21:47: 11443     7876     0           507     0
[...]
```

Судя по этому выводу, каждую секунду в системе производилось огромное количество операций чтения, порой более миллиона. Но самое интересное — количество открываемых файлов в секунду: более пяти тысяч. Открытие файла — довольно медленная операция, требующая поиска файлов по именам и создания файловых дескрипторов и дополнительных структур с метаданными, если они еще не были кэшированы. Эту рабочую нагрузку можно дополнительно исследовать с помощью `orepsnoop(8)`, чтобы попытаться найти пути уменьшения количества операций открытия.

`vfsstat(8)` использует зонды `kprobes` для трассировки функций `vfs_read()`, `vfs_write()`, `vfs_fsync()`, `vfs_open()` и `vfs_create()` и выводит сводную информацию каждую секунду. Функции VFS могут вызываться очень часто, как показано в примере выше, где частота событий порой превышает один миллион раз в секунду, поэтому оверхед этого инструмента может быть заметным (например, 1–3% при такой частоте событий). Этот инструмент больше подходит для специальных исследований, чем для круглосуточного мониторинга, когда желательно, чтобы оверхед не превышал 0.1%.

Этот инструмент полезно использовать на начальных этапах исследований. Через VFS выполняются операции с файловыми системами и сетью. Для получения более детальной информации о них применяйте другие инструменты (например, `vfssize(8)`, о котором речь идет ниже).

ВСС

Порядок использования:

```
vfsstat [interval [count]]
```

Порядок использования этого инструмента отличается от других традиционных инструментов (`vmstat(1)`).

bpfttrace

Ниже приведена реализация версии `vfsstat(8)` для `bpfttrace`, которая выводит те же данные:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing key VFS calls... Hit Ctrl-C to end.\n");
}

kprobe:vfs_read*,
kprobe:vfs_write*,
kprobe:vfs_fsync,
kprobe:vfs_open,
kprobe:vfs_create
{
    @[func] = count();
}

interval:s:1
{
    time();
    print(@);
    clear(@);
}

END
{
    clear(@);
}
```

Каждая секунда здесь выводится в формате списка счетчиков. Для трассировки вариантов `vfs_read()` и `vfs_write()`, таких как `vfs_readv()`, применялись подстановочные символы. При желании эту версию можно дополнить использованием позиционных параметров, чтобы реализовать возможность задавать интервал вывода.

8.3.9. vfscount

Вместо подсчета вызовов этих пяти функций VFS, как это делает `vfsstat(8)`, можно подсчитать вызовы сразу всех функций VFS (их более 50), если использовать `vfscount(8)`¹ — инструмент для BCC и `bpfttrace`. Например:

```
# vfscount
Tracing... Ctrl-C to end.
^C
ADDR                FUNC                COUNT
ffffffffffb8473d01  vfs_fallocate                1
```

¹ Немного истории: версию для BCC я создал 14 августа 2015 года, а версию для `bpfttrace` — 6 сентября 2018 года.

ffffffffb849d301	vfs_kern_mount	1
ffffffffb84b0851	vfs_fsync_range	2
ffffffffb8487271	vfs_mknod	3
ffffffffb8487101	vfs_symlink	68
ffffffffb8488231	vfs_unlink	376
ffffffffb8478161	vfs_writev	525
ffffffffb8486d51	vfs_rmdir	638
ffffffffb8487971	vfs_rename	762
ffffffffb84874c1	vfs_mkdir	768
ffffffffb84a2d61	vfs_getxattr	894
ffffffffb84da761	vfs_lock_file	1601
ffffffffb848c861	vfs_readlink	3309
ffffffffb84b2451	vfs_statfs	18346
ffffffffb8475ea1	vfs_open	108173
ffffffffb847dbf1	vfs_statx_fd	193851
ffffffffb847dc71	vfs_statx	274022
ffffffffb847dbb1	vfs_getattr	330689
ffffffffb847db21	vfs_getattr_nosec	331766
ffffffffb84790a1	vfs_write	355960
ffffffffb8478df1	vfs_read	712610

В период, когда производилась трассировка, чаще всего вызывалась функция `vfs_read()` — 712 610 раз, а `vfs_fallocate()` была вызвана только один раз. Оверхед этого инструмента, как и `vfstat` (8), может стать заметным при высокой частоте вызовов VFS.

Ту же функциональность можно реализовать с использованием `funccount(8)` для BCC, а также в виде однострочного сценария для `bpfftrace(8)`:

```
# funccount 'vfs_*'
# bpfftrace -e 'kprobe:vfs_* { @[func] = count(); }'
```

Подобный подсчет вызовов VFS полезен только на этапе предварительных исследований для выяснения направления дальнейшего движения. Вызовы этих функций могут выполняться во всех подсистемах, использующих VFS, включая сокеты (сетевые операции), файлы `/dev` и `/proc`. Инструмент `fsrwstat(8)`, описанный далее, показывает один из вариантов разделения этих подсистем.

8.3.10. vfssize

`vfssize(8)`¹ — это инструмент для `bpfftrace`, который выводит распределение времени выполнения операций чтения и записи в VFS в виде гистограмм, группируя данные по именам процессов и именам или типам файлов. Вот пример вывода, полученного на сервере API с 48 процессорами:

```
# vfssize
Attaching 5 probes...

@[tomcat-exec-393, tomcat_access.log]:
```

¹ Немного истории: я написал его для этой книги 17 апреля 2019 года.


```

}

kretprobe:vfs_read,
kretprobe:vfs_readv,
kretprobe:vfs_write,
kretprobe:vfs_writev
/@file[tid]/
{
    if (retval >= 0) {
        $file = (struct file *)@file[tid];
        $name = $file->f_path.dentry->d_name.name;
        if ((($file->f_inode->i_mode >> 12) & 15) == DT_FIFO) {
            @[comm, "FIFO"] = hist(retval);
        } else {
            @[comm, str($name)] = hist(retval);
        }
    }
    delete(@file[tid]);
}

END
{
    clear(@file);
}

```

Он извлекает структуру `struct file` из первого аргумента `vfs_read()`, `vfs_readv()`, `vfs_write()` или `vfs_writev()`, а размер блока — из возвращаемого значения. К счастью, имена сетевых протоколов хранятся в именах файлов. (Они извлекаются из `struct proto`: подробнее об этом рассказывается в главе 10.) Когда выполняются операции с каналами FIFO, в имени файла (в настоящее время) ничего не хранится, поэтому текст FIFO зашит непосредственно в исходный код инструмента.

`vfssize(8)` можно расширить и включить разделение по типу вызова (чтение или запись), добавив ключ «probe», идентификатор процесса («pid») и другие детали, если потребуется.

8.3.11. fsrwstat

`fsrwstat(8)`¹ показывает, как можно усовершенствовать `vfsstat(8)` и включить в вывод тип файловой системы. Пример вывода:

```

# fsrwstat
Attaching 7 probes...
Tracing VFS reads and writes... Hit Ctrl-C to end.

18:29:27
@[sockfs, vfs_write]: 1

```

¹ Немного истории: я создал этот инструмент специально для книги 1 февраля 2019 года, взяв за основу собственный инструмент `fsrwcoun.d` из книги о DTrace, вышедшей в 2011 году [Gregg 11].

```
@[sysfs, vfs_read]: 4
@[sockfs, vfs_read]: 5
@[devtmpfs, vfs_read]: 57
@[pipefs, vfs_write]: 156
@[pipefs, vfs_read]: 160
@[anon_inodefs, vfs_read]: 164
@[sockfs, vfs_writev]: 223
@[anon_inodefs, vfs_write]: 292
@[devpts, vfs_write]: 2634
@[ext4, vfs_write]: 104268
@[ext4, vfs_read]: 10495
```

```
[...]
```

Тип файловой системы здесь выводится в первом столбце, что позволяет легко отличить ввод/вывод с использованием сокетов от ввода/вывода в файловой системе ext4. Этот конкретный пример показывает, что система испытывает большую нагрузку (более 100 000 операций ввода/вывода) на файловую систему ext4.

Исходный код `fsrwstat(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/fs.h>

BEGIN
{
    printf("Tracing VFS reads and writes... Hit Ctrl-C to end.\n");
}

kprobe:vfs_read,
kprobe:vfs_readv,
kprobe:vfs_write,
kprobe:vfs_writev
{
    @[str(((struct file *)arg0)->f_inode->i_sb->s_type->name), func] =
        count();
}

interval:s:1
{
    time(); print(@); clear(@);
}

END
{
    clear(@);
}
```

Программа трассирует четыре функции VFS и подсчитывает частоту вызовов для файловой системы каждого типа и каждой функции. Первый аргумент этих функций имеет тип `struct file*`, поэтому `arg0` можно привести к этому типу, а затем, следуя по членам-указателям, добраться до названия типа файловой системы: файл ->

индексный_узел -> суперблок -> тип_файловой_системы -> имя. Так как в реализации используются зонды kprobes, этот путь является ненадежным интерфейсом и его придется исправить в соответствии с изменениями в ядре.

В реализацию fsrwstat(8) можно включить трассировку вызовов других функций VFS при условии возможности получить тип файловой системы из их аргументов (из arg0, или arg1, или arg2 и т. д.).

8.3.12. fileslower

fileslower(8)¹ — это инструмент для BCC и bpftrace, который выявляет синхронные операции чтения и записи с файлами, выполняющиеся дольше заданного порога. Ниже показан пример использования fileslower(8) для BCC, где выявляются операции чтения/записи, выполняющиеся дольше 10 миллисекунд (порог по умолчанию), на производственном сервере Hadoop с 36 процессорами:

```
# fileslower
Tracing sync read/writes slower than 10 ms
TIME(s)  COMM      TID    D BYTES  LAT(ms)  FILENAME
0.142    java      111264 R 4096   25.53    part-00762-37d00f8d...
0.417    java      7122   R 65536  22.80    file.out.index
1.809    java      70560  R 8192   21.71    temp_local_3c9f655b...
2.592    java      47861  W 64512  10.43    blk_2191482458
2.605    java      47785  W 64512  34.45    blk_2191481297
4.454    java      47799  W 64512  24.84    blk_2191482039
4.987    java      111264 R 4096   10.36    part-00762-37d00f8d...
5.091    java      47895  W 64512  15.72    blk_2191483348
5.130    java      47906  W 64512  10.34    blk_2191484018
5.134    java      47799  W 504    13.73    blk_2191482039_1117768266.meta
5.303    java      47984  R 30     12.50    spark-core_2.11-2.3.2...
5.383    java      47899  W 64512  11.27    blk_2191483378
5.773    java      47998  W 64512  10.83    blk_2191487052
[...]
```

Как показывает этот вывод, инструмент обнаружил медленные операции чтения и записи, выполняющиеся в процессе Java до 34 миллисекунд, и вывел соответствующие имена файлов. Тип операции выводится в столбце «D» (direction — направление): «R» соответствует операциям чтения, а «W» — операциям записи. В столбце «TIME(s)» выводится время, когда была выявлена медленная операция, и судя по нему, такие операции выполняются довольно редко — всего несколько раз в секунду.

Синхронное чтение и запись играют важную роль, потому что на время их выполнения процессы блокируются, а их производительность падает из-за задержек. В начале этой главы я говорил, что анализ операций с файловой системой может быть более актуальным, чем анализ дискового ввода/вывода, и это как раз тот случай. В следующей главе мы измерим задержку дискового ввода/вывода, но на этом уровне

¹ Немного истории: версию для BCC я создал 6 февраля 2016 года, а версию для bpftrace — 31 января 2019 года специально для книги.

производительность приложений не зависит напрямую от задержек. Анализируя дисковый ввод/вывод, легко найти явления, которые выглядят как проблемы с задержкой, но на самом деле не являются таковыми. Но если `filelower(8)` показывает проблему с задержкой, она, скорее всего, действительно является проблемой.

Синхронное чтение и запись блокируют процесс на время их выполнения, что может вызвать проблемы на прикладном уровне. С другой стороны, чтобы вытолкнуть буферы на диск и «разогреть» кэш, приложение может выполнять синхронные операции ввода/вывода в фоновых потоках и тем самым избежать блокировки.

Этот инструмент можно использовать, чтобы доказать, что вину за задержки несет файловая система или, напротив, что ввод/вывод не является причиной низкой производительности.

`fileslower(8)` трассирует операции синхронного чтения и записи в VFS. Точнее говоря, текущая реализация трассирует все операции чтения и записи в VFS, а затем фильтрует синхронные, поэтому оверхед может оказаться больше ожидаемого.

BCC

Порядок использования:

```
fileslower [options] [min_ms]
```

Параметры `options`:

- **-p PID**: трассировать только этот PID.

Аргумент `min_ms` — это минимальное время в миллисекундах. Если передать число 0, инструмент выведет все синхронные операции чтения и записи. В этом случае каждую секунду на экране могут появляться тысячи новых строк, в зависимости от частоты операций, и если у вас нет веских причин, не советовал бы поступать так. По умолчанию, если аргумент `min_ms` не указан, используется порог, равный 10 миллисекундам.

bpfttrace

Ниже приведен код версии для `bpfttrace`:

```
#!/usr/local/bin/bpfttrace

#include <linux/fs.h>

BEGIN
{
    printf("%-8s %-16s %-6s T %-7s %7s %s\n", "TIMEms", "COMM", "PID",
        "BYTES", "LATms", "FILE");
}

kprobe:new_sync_read,
kprobe:new_sync_write
```

```

{
    $file = (struct file *)arg0;
    if ($file->f_path.dentry->d_name.len != 0) {
        @name[tid] = $file->f_path.dentry->d_name.name;
        @size[tid] = arg2;
        @start[tid] = nsecs;
    }
}

kretprobe:new_sync_read
/@start[tid]/
{
    $read_ms = (nsecs - @start[tid]) / 1000000;
    if ($read_ms >= 1) {
        printf("%-8d %-16s %-6d R %-7d %7d %s\n", nsecs / 1000000,
            comm, pid, @size[tid], $read_ms, str(@name[tid]));
    }
    delete(@start[tid]); delete(@size[tid]); delete(@name[tid]);
}

kretprobe:new_sync_write
/@start[tid]/
{
    $write_ms = (nsecs - @start[tid]) / 1000000;
    if ($write_ms >= 1) {
        printf("%-8d %-16s %-6d W %-7d %7d %s\n", nsecs / 1000000,
            comm, pid, @size[tid], $write_ms, str(@name[tid]));
    }
    delete(@start[tid]); delete(@size[tid]); delete(@name[tid]);
}

END
{
    clear(@start); clear(@size); clear(@name);
}

```

Здесь используются зонды kprobes для трассировки функций ядра `new_sync_read()` и `new_sync_write()`. Поскольку интерфейс kprobes считается нестабильным, нет гарантии, что эта реализация будет работать на разных версиях ядра, и я уже сталкивался с ядрами, где функции `new_sync_read()` и `new_sync_write()` недоступны для трассировки (компилятор встраивает их код в точки вызова). Версия для ВСС использует обходной путь: она трассирует внутренние функции более высокого уровня `__vfs_read()` и `__vfs_write()` и затем фильтрует синхронные операции.

8.3.13. filetop

`filetop(8)`¹ — это инструмент для ВСС, действующий как и утилита `top(1)`, только для файлов, — показывает имена файлов, используемых наиболее интенсивно. Вот пример использования `filetop(8)` на рабочем сервере Hadoop с 36 процессорами:

¹ Немного истории: версию для ВСС я создал 6 февраля 2016 года, вдохновившись утилитой

```
# filetop
```

```
Tracing... Output every 1 secs. Hit Ctrl-C to end
```

```
02:31:38 loadavg: 39.53 36.71 32.66 26/3427 30188
```

TID	COMM	READS	WRITES	R_Kb	W_Kb	T FILE
113962	java	15171	0	60684	0	R part-00903-37d00f8d-ecf9-4...
23110	java	7	0	7168	0	R temp_local_6ba99afa-351d-4...
25836	java	48	0	3072	0	R map_4141.out
26890	java	46	0	2944	0	R map_5827.out
26788	java	42	0	2688	0	R map_4363.out
26788	java	18	0	1152	0	R map_4756.out.merged
70560	java	130	0	1085	0	R temp_local_1bd4386b-b33c-4...
70560	java	130	0	1079	0	R temp_local_a3938a84-9f23-4...
70560	java	127	0	1053	0	R temp_local_3c9f655b-06e4-4...
26890	java	16	0	1024	0	R map_11374.out.merged
26890	java	15	0	960	0	R map_5262.out.merged
26788	java	15	0	960	0	R map_20423.out.merged
26788	java	14	0	896	0	R map_4371.out.merged
26890	java	14	0	896	0	R map_10138.out.merged
26890	java	13	0	832	0	R map_4991.out.merged
25836	java	13	0	832	0	R map_3994.out.merged
25836	java	13	0	832	0	R map_4651.out.merged
25836	java	13	0	832	0	R map_16267.out.merged
25836	java	13	0	832	0	R map_15255.out.merged
26788	java	12	0	768	0	R map_6917.out.merged
[...]						

По умолчанию выводятся двадцать наиболее интенсивно используемых файлов, отсортированных по столбцу прочитанных байтов. Список обновляется каждую секунду. Этот конкретный вывод показывает, что чаще других чтение производилось из файла «part-00903-37d00f8d» (имя усечено): за текущую секунду было выполнено примерно 15 000 операций чтения, и общий объем прочитанных данных составил около 60 Мбайт. Здесь не показан средний размер прочитанного блока, но его легко вычислить по имеющимся значениям, в данном случае он равен 4,0 Кбайт.

Этот инструмент можно использовать для определения характера рабочей нагрузки и наблюдения за файловой системой в целом. Как и `top(1)`, он помогает выявить процессы с неожиданно большим потреблением ЦП и обнаружить файлы, использующиеся необычно интенсивно.

По умолчанию `filetop(8)` выводит информацию только об обычных файлах¹. Но если добавить параметр `-a`, инструмент выведет информацию обо всех файлах, включая сокеты TCP:

`top(1)` Уильяма Ле Фебвра.

¹ Под «обычными» понимаются файлы с типом `DT_REG` в исходном коде ядра. В числе других типов файлов можно назвать: `DT_DIR` — каталоги, `DT_BLK` — специальные файлы блочных устройств и т. д.


```
# filetop -a
[...]
```

TID	COMM	READS	WRITES	R_Kb	W_Kb	T	FILE
32857	java	718	0	15756	0	S	TCP
120597	java	12	0	12288	0	R	temp_local_3807d4ca-b41e-3...
32770	java	502	0	10118	0	S	TCP
32507	java	199	0	4212	0	S	TCP
88371	java	186	0	1775	0	R	temp_local_215ae692-35a4-2...

```
[...]
```

filetop(8) выводит следующие столбцы:

- **TID:** Thread ID — идентификатор потока выполнения;
- **COMM:** имя процесса/потока;
- **READS:** количество операций чтения, выполненных в течение минувшего интервала;
- **WRITES:** количество операций записи, выполненных в течение минувшего интервала;
- **R_Kb:** общий объем данных в килобайтах, прочитанных в течение минувшего интервала;
- **W_Kb:** общий объем данных в килобайтах, записанных в течение минувшего интервала;
- **T:** Type — тип файла: R == Regular — обычный файл, S == Socket — сокет, O == Other — другой тип;
- **FILE:** имя файла.

filetop(8) использует зонды kprobes для трассировки функций `vfs_read()` и `vfs_write()` ядра. Тип файла извлекается из поля `mode` индексного узла с помощью макросов `S_ISREG()` и `S_ISSOCK()`.

Оверхед этого инструмента, как и описанных выше, может стать заметным при высокой частоте следования операций чтения/записи в VFS. Кроме того, сбор различных статистик, например имена файлов, делает оверхед этого инструмента немного выше, чем у других инструментов.

Порядок использования:

```
filetop [options] [interval [count]]
```

Параметры `options`:

- **-C:** не очищать экран — выводить данные с прокруткой;
- **-r ROWS:** выводить указанное число строк (по умолчанию 20);
- **-p PID:** трассировать только этот PID.

Параметр `-C` обеспечивает сохранность буфера вывода терминала, что позволяет выявлять изменение характера рабочей нагрузки с течением времени.

8.3.14. writesync

`writesync(8)`¹ — это инструмент для `bpfttrace`, который трассирует операции записи VFS в обычные файлы и показывает, в каких из них использовался флаг синхронной записи (`O_SYNC` или `O_DSYNC`). Например:

```
# writesync.bt
Attaching 2 probes...
Tracing VFS write sync flags... Hit Ctrl-C to end.
^C

@regular[cronolog, output_20190520_06.log]: 1
@regular[VM Thread, gc.log]: 2
@regular[cronolog, catalina_20190520_06.out]: 9
@regular[tomcat-exec-142, tomcat_access.log]: 15
[...]
@sync[dd, outfile]: 100
```

Как показывает этот вывод, в период трассировки было выполнено не так много обычных операций записи в файлы и сотня синхронных операций записи из процесса «dd» в файл с именем «outfile1». Здесь я специально использовал команду `dd(1)`, чтобы показать возможности инструмента:

```
dd if=/dev/zero of=outfile oflag=sync count=100
```

Синхронные операции записи ожидают завершения ввода/вывода, тогда как обычные операции ввода/вывода могут завершиться вводом/выводом в кэш (отложенная запись). Это замедляет синхронный ввод/вывод, и если синхронность не требуется, удаление флага синхронности может значительно повысить производительность.

Вот исходный код `writesync(8)`:

```
#!/usr/local/bin/bpfttrace

#include <linux/fs.h>
#include <asm-generic/fcntl.h>

BEGIN
{
    printf("Tracing VFS write sync flags... Hit Ctrl-C to end.\n");
}

kprobe:vfs_write,
kprobe:vfs_writev
{
    $file = (struct file *)arg0;
    $name = $file->f_path.dentry->d_name.name;
    if ((($file->f_inode->i_mode >> 12) & 15) == DT_REG) {
```

¹ Немного истории: этот инструмент я создал специально для книги 19 мая 2019 года.

```

        if ($file->f_flags & O_DSYNC) {
            @sync[comm, str($name)] = count();
        } else {
            @regular[comm, str($name)] = count();
        }
    }
}

```

Он убеждается, что операция выполняется с обычным файлом (DT_REG), а затем проверяет наличие флага O_DSYNC (который также устанавливается флагом O_SYNC).

8.3.15. filetype

`filetype(8)`¹ — это инструмент для `bpfttrace`, который трассирует операции чтения и записи в VFS и определяет тип файла и имя процесса. Например, вот результат трассировки в системе с 36 процессорами во время сборки ПО:

```

# filetype.bt
Attaching 4 probes...
^C

@[regular, vfs_read, expr]: 1
@[character, vfs_read, bash]: 10
[...]
@[socket, vfs_write, sshd]: 435
@[fifo, vfs_write, cat]: 464
@[regular, vfs_write, sh]: 697
@[regular, vfs_write, as]: 785
@[regular, vfs_read, objtool]: 932
@[fifo, vfs_read, make]: 1033
@[regular, vfs_read, as]: 1437
@[regular, vfs_read, gcc]: 1563
@[regular, vfs_read, cat]: 2196
@[regular, vfs_read, sh]: 8391
@[regular, vfs_read, fixdep]: 11299
@[fifo, vfs_read, sh]: 15422
@[regular, vfs_read, cc1]: 16851
@[regular, vfs_read, make]: 39600

```

Мы видим, что при трассировке большинство операций ввода/вывода выполнялись с «обычными» файлами и производились ПО для сборки (`make(1)`, `cc1(1)`, `gcc(1)` и т. д.). В выводе есть и операции записи в сокеты, выполнявшиеся процессом `sshd` — сервером SSH, который отправлял пакеты, и операции чтения символов в `bash`, когда командная оболочка читала входные данные из символического устройства `/dev/pts/1`.

¹ Немного истории: этот инструмент я создал специально для книги 2 февраля 2019 года.

Кроме того, в выводе есть операции чтения и записи в FIFO¹. Вот короткий пример, иллюстрирующий их роль:

```
window1$ tar cf - dir1 | gzip > dir1.tar.gz
window2# filetype.bt
Attaching 4 probes...
^C
[...]
@[regular, vfs_write, gzip]: 36
@[fifo, vfs_write, tar]: 191
@[fifo, vfs_read, gzip]: 191
@[regular, vfs_read, tar]: 425
```

Тип FIFO соответствует каналам в командной оболочке. В этом примере команда `tar(1)` читает содержимое обычных файлов и посылает его в канал FIFO. `gzip(1)` читает данные из канала FIFO, сжимает их и записывает результат в обычный файл. Это отчетливо видно в выводе инструмента `filetype(8)`.

Вот исходный код `filetype(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/fs.h>

BEGIN
{
    // из uapi/linux/stat.h:
    @type[0xc000] = "socket";
    @type[0xa000] = "link";
    @type[0x8000] = "regular";
    @type[0x6000] = "block";
    @type[0x4000] = "directory";
    @type[0x2000] = "character";
    @type[0x1000] = "fifo";
    @type[0] = "other";
}

kprobe:vfs_read,
kprobe:vfs_readv,
kprobe:vfs_write,
kprobe:vfs_writev
{
    $file = (struct file *)arg0;
    $mode = $file->f_inode->i_mode;
    @[@type[$mode & 0xf000], func, comm] = count();
}
```

¹ FIFO: (first-in, first-out — первым пришел, первым вышел) специальный файл (именованный канал). См. страницу справочного руководства `FIFO(7)`.

```
END
{
    clear(@type);
}
```

Обработчик BEGIN подготавливает хеш-таблицу (@type) с возможными значениями поля mode в индексных узлах и соответствующими им строками, которая затем используется в обработчике зондов kprobes в функциях VFS.

Через два месяца после создания этого инструмента я написал инструменты для анализа ввода/вывода через сокеты и заметил, что упустил из виду инструмент VFS, представляющий режимы файлов из include/linux/fs.h (DT_FIFO, DT_CHR и др.). Я написал этот инструмент (и отбросил префикс «DT_»):

```
#!/usr/local/bin/bpfttrace

#include <linux/fs.h>

BEGIN
{
    printf("Tracing VFS reads and writes... Hit Ctrl-C to end.\n");
    // из include/linux/fs.h:
    @type2str[0] = "UNKNOWN";
    @type2str[1] = "FIFO";
    @type2str[2] = "CHR";
    @type2str[4] = "DIR";
    @type2str[6] = "BLK";
    @type2str[8] = "REG";
    @type2str[10] = "LNK";
    @type2str[12] = "SOCK";
    @type2str[14] = "WHT";
}

kprobe:vfs_read,
kprobe:vfs_readv,
kprobe:vfs_write,
kprobe:vfs_writev
{
    $file = (struct file *)arg0;
    $type = ($file->f_inode->i_mode >> 12) & 15;
    @[@type2str[$type], func, comm] = count();
}

END
{
    clear(@type2str);
}
```

Когда я решил добавить его в эту главу, то обнаружил, что случайно написал вторую версию filetype(8), используя другой заголовочный файл для определения типов файлов. Я включил сюда исходный код, чтобы показать, что иногда один и тот же инструмент можно реализовать несколькими способами.

8.3.16. cachestat

`cachestat(8)`¹ — это инструмент для ВСС, который выводит статистику попаданий и промахов кэша страниц. Его можно использовать для проверки доли попаданий и эффективности кэша страниц, а также для исследования влияния разных настроек системы и приложений и получения информации об эффективности кэша. Вот пример, полученный на производственном экземпляре Hadoop с 36 процессорами:

```
# cachestat
  HITS    MISSES  DIRTIES  HITRATIO    BUFFERS_MB  CACHED_MB
53401    2755    20953    95.09%         14      90223
49599    4098    21460    92.37%         14      90230
16601    2689    61329    86.06%         14      90381
15197    2477    58028    85.99%         14      90522
18169    4402    51421    80.50%         14      90656
57604    3064    22117    94.95%         14      90693
76559    3777    3128     95.30%         14      90692
49044    3621    26570    93.12%         14      90743
[...]
```

Как показывает этот вывод, доля попаданий в кэш часто превышает 90%. Настройка системы и приложения, чтобы довести долю попаданий с 90% до 100%, может дать значительный выигрыш в производительности (намного больше, чем разница в 10%), потому что приложение чаще будет получать данные из памяти, не ожидая, пока выполнится дисковый ввод/вывод.

Масштабные облачные базы данных: Cassandra, Elasticsearch и PostgreSQL — часто очень активно используют кэш страниц, чтобы гарантировать размещение «горячего» набора данных в оперативной памяти. Это означает, что одним из наиболее важных вопросов при подготовке хранилищ данных является вопрос о соответствии размеров рабочего набора данных и выделенной памяти. Команды Netflix, управляющие stateful-сервисами, используют `cachestat(8)`, чтобы ответить на этот вопрос и выяснить, какие решения можно использовать. Например: поможет ли увеличить производительность применение алгоритма сжатия данных или добавление памяти в кластер.

Пара простых примеров поможет лучше понять вывод `cachestat(8)`. Пример ниже получен в бездействующей системе, где создается файл размером в 1 Гбайт. Параметр `-T` используется, чтобы вывести столбец с отметками времени:

¹ Немного истории: первую экспериментальную версию инструмента на основе Ftrace я создал для своей коллекции 28 декабря 2014 года, когда был в отпуске в австралийской Юларе [87]. Эта версия была тесно связана с внутренними компонентами ядра, поэтому я добавил блочный комментарий в заголовок, описывающий ее как «замок на песке»: эта версия легко может стать нерабочей при изменении кода ядра. На ее основе Аллан Макаливи (Allan McAleavy) реализовал версию для ВСС 6 ноября 2015 года.

```
# cachestat -T
TIME      HITS    MISSES  DIRTIES  HITRATIO  BUFFERS_MB  CACHED_MB
21:06:47      0        0        0    0.00%        9        191
21:06:48      0        0    120889    0.00%        9        663
21:06:49      0        0    141167    0.00%        9       1215
21:06:50     795        0        1   100.00%        9       1215
21:06:51      0        0        0    0.00%        9       1215
```

В столбце DIRTIES выводится число страниц, записанных в кэш страниц (это «грязные» страницы), а объем в столбце CACHED_MB увеличился на 1024 Мбайт: размер вновь созданного файла.

Теперь вытолкнем этот файл на диск из кэша страниц (эта операция удалит все страницы из кэша):

```
# sync
# echo 3 > /proc/sys/vm/drop_caches
```

Теперь прочитаем файл дважды после запуска cachestat(8) с интервалом в 10 секунд:

```
# cachestat -T 10
TIME      HITS    MISSES  DIRTIES  HITRATIO  BUFFERS_MB  CACHED_MB
21:08:58      771        0        1   100.00%        8        190
21:09:08    33036    53975     16   37.97%        9        400
21:09:18      15    68544        2    0.02%        9        668
21:09:28     798    65632        1    1.20%        9        924
21:09:38        5    67424        0    0.01%        9       1187
21:09:48     3757    11329        0   24.90%        9       1232
21:09:58     2082        0        1   100.00%        9       1232
21:10:08    268421     11     12   100.00%        9       1232
21:10:18        6        0        0   100.00%        9       1232
21:10:19     784        0        1   100.00%        9       1232
```

Чтение файла было произведено между 21:09:08 и 21:09:48, что видно по большому числу промахов MISSES, низкой доле попаданий HITRATIO и увеличению размера кэша страниц в CACHED_MB на 1024 Мбайт. В 21:10:08 было произведено повторное чтение файла, на этот раз доля попаданий в кэш составила 100%.

cachestat(8) использует kprobes и инструментирует следующие функции ядра:

- **mark_page_accessed():** для подсчета числа обращений к кэшу;
- **mark_buffer_dirty():** для подсчета числа операций записи в кэш;
- **add_to_page_cache_lru():** для подсчета числа добавляемых страниц;
- **account_page_dirtied():** для подсчета числа «грязных» страниц.

Этот инструмент позволяет получить ценную информацию о доле попаданий в кэш страниц, но сильно зависит от деталей реализации ядра и может потребовать

изменения исходного кода для работы с разными версиями ядра. Лучшее применение этого инструмента — показать, что такое вообще возможно¹.

Эти функции кэширования страниц могут вызываться очень часто: миллионы раз в секунду. По этой причине оверхед этого инструмента при экстремальных рабочих нагрузках может возрасти до 30% и более, хотя для обычных рабочих нагрузок они будут намного меньше. Обязательно протестируйте его в лабораторной среде и оцените возможность использования в производственной среде.

Порядок использования:

```
cachestat [options] [interval [count]]
```

Поддерживается единственный параметр -T, включающий вывод отметки времени.

Есть еще один инструмент для BCC, `cachetop(8)`², который выводит статистику `cachestat(8)` по процессам в стиле `top(1)`, используя библиотеку `curses`.

8.3.17. writeback

`writeback(8)`³ — это инструмент для `bpftrace`, который показывает операции отложенной записи кэша страниц. Эти операции выполняются при сканировании страниц, а также когда производится сброс на диск «грязных» страниц. Инструмент выводит время и тип события отложенной записи и продолжительность операции записи. Вот пример, полученный в системе с 36 процессорами:

```
# writeback.bt
Attaching 4 probes...
Tracing writeback... Hit Ctrl-C to end.
TIME      DEVICE  PAGES  REASON      ms
03:42:50  253:1      0       periodic    0.013
03:42:55  253:1     40       periodic    0.167
03:43:00  253:1      0       periodic    0.005
03:43:01  253:1    11268    background   6.112
03:43:01  253:1    11266    background   7.977
03:43:01  253:1    11314    background  22.209
```

¹ Когда я представил `cachestat(8)` в своем выступлении на саммите LSFMM, инженеры, занимающиеся подсистемой управления памятью (Memory Management, ММ), указали, что в будущем он перестанет работать, и объяснили некоторые проблемы, связанные с его правильной реализацией для будущих ядер (спасибо Мелу Горману). У некоторых, как у нас в Netflix, он прекрасно работает с нашими ядрами и рабочими нагрузками. Но для большей надежности, я думаю, что: (А) кто-то должен потратить несколько недель на изучение исходного кода ядра, пробуя разные рабочие нагрузки и взаимодействуя с инженерами ММ, чтобы найти настоящее решение; или, что может быть даже лучше, (В) добавить статистику в `/proc`, чтобы преобразовать этот инструмент в простой счетчик событий.

² Немного истории: `cachetop(8)` был создан Эммануэлем Бретелем (Emmanuel Bretelle) 13 июля 2016 года.

³ Немного истории: версию для `bpftrace` я создал 14 сентября 2018 года.


```

03:43:02 253:1 11266 background 20.698
03:43:02 253:1 11266 background 7.421
03:43:02 253:1 11266 background 11.382
03:43:02 253:1 11266 background 6.954
03:43:02 253:1 11266 background 8.749
03:43:02 253:1 11266 background 14.518
03:43:04 253:1 38836 sync 64.655
03:43:04 253:1 0 sync 0.004
03:43:04 253:1 0 sync 0.002
03:43:09 253:1 0 periodic 0.012
03:43:14 253:1 0 periodic 0.016
[...]
```

В начале вывода можно видеть периодические операции записи, которые выполняются каждые 5 секунд. Они вытолкнули на диск не так много страниц (0, 40, 0). Затем произошел всплеск операций записи в фоновом режиме, в ходе которых на диск были записаны десятки тысяч страниц, и на каждую операцию было потрачено от 6 до 22 миллисекунд. Такое асинхронное выталкивание страниц происходит, когда системе не хватает свободной памяти. Если временные отметки коррелируют с проблемами производительности приложений, обнаруженными другими инструментами мониторинга (например, средствами мониторинга производительности облачного хранилища), это может служить признаком, что проблемы вызваны отложенной записью страниц в файловую систему. Поведение механизма выталкивания страниц может настраиваться (например, с помощью `sysctl(8)` и `vm.dirty_writeback_centisecs`). В 3:43:04 была выполнена синхронная операция записи, в ходе которой было записано 38 836 страниц за 64 миллисекунды.

Исходный код `writeback(8)`:

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing writeback... Hit Ctrl-C to end.\n");
    printf("%-9s %-8s %-8s %-16s %s\n", "TIME", "DEVICE", "PAGES",
        "REASON", "ms");

    // см. /sys/kernel/debug/tracing/events/writeback/writeback_start/format
    @reason[0] = "background";
    @reason[1] = "vmscan";
    @reason[2] = "sync";
    @reason[3] = "periodic";
    @reason[4] = "laptop_timer";
    @reason[5] = "free_more_memory";
    @reason[6] = "fs_free_space";
    @reason[7] = "forker_thread";
}

tracepoint:writeback:writeback_start
{
    @start[args->sb_dev] = nsecs;
    @pages[args->sb_dev] = args->nr_pages;
}
```

```

tracepoint:writeback:writeback_written
/@start[args->sb_dev]/
{
    $sb_dev = args->sb_dev;
    $s = @start[$sb_dev];
    $lat = $s ? (nsecs - $s) / 1000 : 0;
    $pages = @pages[args->sb_dev] - args->nr_pages;

    time("%H:%M:%S ");
    printf("%-8s %-8d %-16s %d.%03d\n", args->name, $pages,
        @reason[args->reason], $lat / 1000, $lat % 1000);

    delete(@start[$sb_dev]);
    delete(@pages[$sb_dev]);
}
END
{
    clear(@reason);
    clear(@start);
}

```

В момент инициализации подготавливается массив @reason, отображающий числовые идентификаторы причин в человекочитаемые строки. В момент начала обратной записи для каждого устройства фиксируется время, а в точке трассировки writeback_written выводятся полученные результаты. Количество страниц определяется как величина уменьшения аргумента args->nr_pages, как это объясняется в исходном коде ядра (см. исходный код wb_writeback() в fs/fs-writeback.c).

8.3.18. dcstat

dcstat(8)¹ — это инструмент для BCC и bpftrace, отображающий статистику кэша каталогов (dcache). Ниже показан пример использования dcstat(8) для BCC на производственном экземпляре Hadoop с 36 процессорами:

```

# dcstat
TIME      REFS/s    SLOW/s    MISS/s    HIT%
22:48:20: 661815    27942     20814     96.86
22:48:21: 540677    87375     80708     85.07
22:48:22: 271719    4042      914       99.66
22:48:23: 434353    4765      37         99.99
22:48:24: 766316    5860      607       99.92
22:48:25: 567078    7866      2279      99.60
22:48:26: 556771    26845     20431     96.33
22:48:27: 558992    4095      747       99.87
22:48:28: 299356    3785      105       99.96
[...]
```

¹ Немного истории: первый похожий инструмент с именем dnlcstat я создал 10 марта 2004 года для инструментации поиска в кэше имен каталогов в Solaris и использовал в нем статистику ядра Kstat. Версию dcstat(8) для BCC я написал 9 февраля 2016 года, а версию для bpftrace — 26 марта 2019 года специально для этой книги.

Как показывает этот вывод, доля попаданий в кэш составляет более 99% при более 500 000 обращений к кэшу в секунду. Столбцы:

- **REFS/s**: частота обращений к кэшу каталогов (dcache);
- **SLOW/s**: начиная с версии Linux 2.5.11, в dcache предусмотрена оптимизация, позволяющая избежать срыва кэширования (cacheline bouncing) при поиске часто используемых каталогов («/», «/usr») [88]. Этот столбец показывает, когда данная оптимизация не использовалась и для поиска в dcache применялся «медленный» (slow) путь;
- **MISS/s**: количество промахов кэша dcache; запись о каталоге все еще может находиться в кэше страниц, но специализированный кэш dcache не вернул ее;
- **HIT%**: доля попаданий в кэш.

dcstat(8) использует kprobes для инструментации функции ядра `lookup_fast()` и `kretprobes` — для `d_lookup()`. Оверхед инструмента может стать заметным при большой рабочей нагрузке, когда увеличивается частота вызовов этих функций, как в примере выше. Обязательно проверяйте инструмент в лабораторных условиях, прежде чем использовать его в производственной среде.

BCC

Порядок использования:

```
dcstat [interval [count]]
```

Этот инструмент моделирует поведение других традиционных инструментов (например, `vmstat(1)`).

bpfftrace

Вот пример вывода версии для `bpfftrace`:

```
# dcstat.bt
Attaching 4 probes...
Tracing dcache lookups... Hit Ctrl-C to end.
      REFS      MISSES  HIT%
234096      16111   93%
495104      36714   92%
461846      36543   92%
460245      36154   92%
[...]
```

Исходный код:

```
#!/usr/local/bin/bpfftrace

BEGIN
{
    printf("Tracing dcache lookups... Hit Ctrl-C to end.\n");
```

```

        printf("%10s %10s %5s%\n", "REFS", "MISSES", "HIT%");
    }

kprobe:lookup_fast { @hits++; }
kretprobe:d_lookup /retval == 0/ { @misses++; }
interval:s:1
{
    $refs = @hits + @misses;
    $percent = $refs > 0 ? 100 * @hits / $refs : 0;
    printf("%10d %10d %4d%\n", $refs, @misses, $percent);
    clear(@hits);
    clear(@misses);
}

END
{
    clear(@hits);
    clear(@misses);
}

```

Здесь используется тернарный (трехместный) оператор, чтобы избежать деления на ноль в тех редких случаях, когда было получено 0 попаданий и промахов¹.

8.3.19. dcsnoop

`dcsnoop(8)`² — это инструмент для BCC и `bpfftrace`, производящий трассировку поиска в кэше каталогов (`dcache`) и отображающий подробности о каждом поиске. Вывод может получиться очень объемным, до нескольких тысяч строк в секунду, в зависимости от того, насколько часто выполняется поиск. Ниже показан пример использования `dcsnoop(8)` для BCC с параметром `-a`, чтобы показать все операции поиска:

```

# dcsnoop -a
TIME(s)      PID    COMM          T FILE
0.005463     2663   snmpd         R proc/sys/net/ipv6/conf/eth0/forwarding
0.005471     2663   snmpd         R sys/net/ipv6/conf/eth0/forwarding
0.005479     2663   snmpd         R net/ipv6/conf/eth0/forwarding
0.005487     2663   snmpd         R ipv6/conf/eth0/forwarding
0.005495     2663   snmpd         R conf/eth0/forwarding
0.005503     2663   snmpd         R eth0/forwarding
0.005511     2663   snmpd         R forwarding
[...]
```

¹ Обратите внимание, что в BPF есть защита от деления на ноль [89], но перед отправкой программы в BPF все равно желательно включить в нее такую защиту, чтобы она не была отклонена верификатором BPF.

² Немного истории: первый подобный инструмент под названием `dnlsnoop` и основанный на DTace, я написал 17 марта 2004 года, версия для BCC была написана 9 февраля 2016 года, а версия для `bpfftrace` — 8 сентября 2018 года.

Этот вывод иллюстрирует поиск каталога `/proc/sys/net/ipv6/conf/eth0/forwarding` в процессе `snmpd` и показывает, как происходил обход всех компонентов. Столбец «Т» отображает тип: R == reference — ссылка, M == miss — промах.

`dcсноор(8)` действует так же, как `dcstat(8)`, используя зонды `kprobes`. Предполагается, что инструмент будет иметь высокий оверхед для любой умеренной рабочей нагрузки, так как выводит отдельную строку для каждого события. Предназначен для использования в течение коротких периодов времени с целью изучения промахов, обнаруженных с помощью `dcstat(8)`.

BCC

Версия для BCC поддерживает только один параметр командной строки: `-a`. С этим параметром он отображает и события ссылки на каталог, и промахи. По умолчанию отображаются только события промаха.

bpfftrace

Ниже приведен исходный код версии для `bpfftrace`:

```
#!/usr/local/bin/bpfftrace

#include <linux/fs.h>
#include <linux/sched.h>

// из fs/namei.c:
struct nameidata {
    struct path path;
    struct qstr last;
    // [...]
};

BEGIN
{
    printf("Tracing dcache lookups... Hit Ctrl-C to end.\n");
    printf("%-8s %-6s %-16s %1s %s\n", "TIME", "PID", "COMM", "T", "FILE");
}

// прокомментируйте этот блок, чтобы предотвратить вывод информации
// о попаданиях в кэш:
kprobe:lookup_fast
{
    $nd = (struct nameidata *)arg0;
    printf("%-8d %-6d %-16s R %s\n", elapsed / 1000000, pid, comm,
        str($nd->last.name));
}

kprobe:d_lookup
{
    $name = (struct qstr *)arg1;
    @fname[tid] = $name->name;
}
```

```

kretprobe:d_lookup
/@fname[tid]/
{
    if (retval == 0) {
        printf("%-8d %-6d %-16s M %s\n", elapsed / 1000000, pid, comm,
            str(@fname[tid]));
    }
    delete(@fname[tid]);
}

```

Эта программа ссылается на член «last» в структуре nameidata, недоступной в заголовках ядра, поэтому достаточно было объявить только часть структуры.

8.3.20. mountsnoop

mountsnoop(8)¹ — это инструмент для ВСС, трассирующий события монтирования файловых систем. Его можно использовать для устранения неполадок, особенно в контейнерах, монтирующих файловые системы в момент запуска. Пример вывода:

```

# mountsnoop
COMM          PID    TID    MNT_NS      CALL
systemd-logind 1392   1392   4026531840  mount("tmpfs", "/run/user/116", "tmpfs",
MS_NOSUID|MS_NODEV, "mode=0700,uid=116,gid=65534,size=25778348032") = 0
systemd-logind 1392   1392   4026531840  umount("/run/user/116", MNT_DETACH) = 0
[...]

```

Как показывает этот вывод, процесс systemd-logind смонтировал (mount(2)) файловую систему tmpfs в каталог /run/user/116, а затем отмонтировал (umount(2)) ее.

mountsnoop(8) использует kprobes для трассировки системных вызовов mount(2) и unmount(2). Поскольку события монтирования файловых систем происходят нечасто, оверхед будет незначительным.

8.3.21. xfsslower

xfsslower(8)² — это инструмент для ВСС. Он трассирует наиболее часто используемые операции с файловой системой XFS и выводит подробные сведения об операциях, выполнявшихся дольше заданного порогового значения. К трассируемым относятся операции чтения, записи, открытия и синхронизации.

Ниже приводится пример использования версии xfsslower(8) для выявления операций, длящихся дольше 10 миллисекунд (значение порога по умолчанию) на производственном экземпляре с 36 процессорами:

¹ Немного истории: этот инструмент создал Омар Сандовал (Omar Sandoval) 14 октября 2016 года.

² Немного истории: этот инструмент я создал 11 февраля 2016 года, взяв за основу собственный инструмент zfsslower.d из книги о DTrace, вышедшей в 2011 году [Gregg 11].

```
# xfsslower
Tracing XFS operations slower than 10 ms
TIME      COMM      PID    T BYTES  OFF_KB  LAT(ms)  FILENAME
02:04:07  java          5565   R 63559   360237   17.16  shuffle_2_63762_0.data
02:04:07  java          5565   R 44203   151427   12.59  shuffle_0_12138_0.data
02:04:07  java          5565   R 39911   106647   34.96  shuffle_0_12138_0.data
02:04:07  java          5565   R 65536   340788   14.80  shuffle_2_101288_0.data
02:04:07  java          5565   R 65536   340744   14.73  shuffle_2_103383_0.data
02:04:07  java          5565   R 64182   361925   59.44  shuffle_2_64928_0.data
02:04:07  java          5565   R 44215   108517   12.14  shuffle_0_12138_0.data
02:04:07  java          5565   R 63370   338650   23.23  shuffle_2_104532_0.data
02:04:07  java          5565   R 63708   360777   22.61  shuffle_2_65806_0.data
[...]
```

Как показывает этот вывод, многие операции чтения в процессе Java выполнялись дольше 10 миллисекунд.

Как и `fileslower(8)`, этот инструмент действует близко к приложению, поэтому задержки, отображаемые им, вероятнее всего, обусловлены самим приложением.

Этот инструмент использует `kprobes` для трассировки функций ядра, ссылки на которые находятся в системной структуре `struct file_operations` — интерфейсе к VFS. Вот выдержка из `fs/xfs/xfs_file.c` в Linux:

```
const struct file_operations xfs_file_operations = {
    .llseek = xfs_file_llseek,
    .read_iter = xfs_file_read_iter,
    .write_iter = xfs_file_write_iter,
    .splice_read = generic_file_splice_read,
    .splice_write = iter_file_splice_write,
    .unlocked_ioctl = xfs_file_ioctl,
#ifdef CONFIG_COMPAT
    .compat_ioctl = xfs_file_compat_ioctl,
#endif
    .mmap = xfs_file_mmap,
    .mmap_supported_flags = MAP_SYNC,
    .open = xfs_file_open,
    .release = xfs_file_release,
    .fsync = xfs_file_fsync,
    .get_unmapped_area = thp_get_unmapped_area,
    .fallocate = xfs_file_fallocate,
    .remap_file_range = xfs_file_remap_range,
};
```

Для трассировки операций чтения производится инструментация функции `xfs_file_read_iter()`, для трассировки операций записи — функция `xfs_file_write_iter()` и т. д. В разных версиях ядра могут использоваться разные функции, поэтому этот инструмент требует постоянного сопровождения. Оверхед прямо зависит от частоты операций и частоты событий превышения порогового значения. В высоконагруженных системах операции могут следовать друг за другом очень часто, поэтому оверхед может быть существенным даже в отсутствие операций, выполняющихся дольше порога, когда результаты трассировки не выводятся.

Порядок использования:

```
xfsslower [options] [min_ms]
```

Параметры options:

- **-p PID**: трассировать только этот PID.

Аргумент `min_ms` — это минимальное пороговое время в миллисекундах. Если в этом аргументе передать 0, будут выводиться все трассируемые операции. В этом случае могут выводиться тысячи строк в секунду, в зависимости от частоты событий, поэтому поступать так нежелательно, если нет веских причин. Если аргумент не указан, по умолчанию используется порог 10 миллисекунд.

В описании следующего инструмента показана программа для `bpfttrace`, которая инструментирует те же функции, но не выводит информацию о каждом событии, а накапливает данные в гистограммах задержек.

8.3.22. xfsdist

`xfsdist(8)`¹ — это инструмент для BCC и `bpfttrace`, трассирующий и отображающий распределение задержек в наиболее часто используемых операциях с файловой системой XFS: чтение, запись, открытие и синхронизация. Ниже показан пример использования версии `xfsdist(8)` для BCC для трассировки производственного экземпляра Nadoor с 36 процессорами в течение 10 секунд:

```
# xfsdist 10 1
```

```
Tracing XFS operation latency... Hit Ctrl-C to end.
```

```
23:55:23:
```

```
operation = 'read'
```

usecs	: count	distribution
0 -> 1	: 5492	*****
2 -> 3	: 4384	*****
4 -> 7	: 3387	*****
8 -> 15	: 1675	*****
16 -> 31	: 7429	*****
32 -> 63	: 574	***
64 -> 127	: 407	**
128 -> 255	: 163	
256 -> 511	: 253	*
512 -> 1023	: 98	
1024 -> 2047	: 89	
2048 -> 4095	: 39	
4096 -> 8191	: 37	
8192 -> 16383	: 27	
16384 -> 32767	: 11	

¹ Немного истории: версию для BCC я создал 12 февраля 2016 года, а версию для `bpfttrace` — 8 сентября 2018 года. За основу я взял собственный инструмент `zfsdist.d` на базе DTrace, написанный в 2012 году.


```

32768 -> 65535      : 21      |
65536 -> 131071     : 10      |
operation = 'write'
  usecs              : count    distribution
    0 -> 1           : 414      |
    2 -> 3           : 1327     |
    4 -> 7           : 3367     | **
    8 -> 15          : 22415    | *****
   16 -> 31          : 65348    | *****
   32 -> 63          : 5955     | ***
   64 -> 127         : 1409     |
  128 -> 255         : 28       |
operation = 'open'
  usecs              : count    distribution
    0 -> 1           : 7557     | *****
    2 -> 3           : 263      | *
    4 -> 7           : 4         |
    8 -> 15          : 6         |
   16 -> 31          : 2         |

```

Судя по этому выводу, где видны отдельные гистограммы для операций чтения, записи и открытия со счетчиками, текущая рабочая нагрузка в исследуемой системе выполняет большое число операций записи. Гистограмма с задержками в операциях чтения показывает бимодальное распределение: большинство операций чтения выполняется менее чем за 7 микросекунд, но есть и значительное количество операций, продолжающихся от 16 до 31 микросекунды. Но судя по скорости, все они обслуживались из кэша страниц. Разница во времени может быть обусловлена объемом прочитанных данных или различными типами операций чтения, которые используют разные пути в коде. Самые медленные операции чтения длились от 65 до 131 миллисекунды: в этом случае чтение могло выполняться с устройств хранения, а сами операции могли ждать доступа к устройствам в очереди.

Гистограмма распределения задержек в операциях записи показывает, что большая часть из них длилась от 16 до 31 микросекунды, то есть они выполнялись очень быстро и, вероятно, с использованием буферизации и отложенной записи.

BCC

Порядок использования:

```
xfsdist [options] [interval [count]]
```

Параметры options:

- **-m**: выводить время в миллисекундах (по умолчанию время измеряется в микросекундах);
- **-p PID**: трассировать только этот PID.

Аргументы interval и count позволяют исследовать изменение гистограмм с течением времени.

bpfttrace

Ниже приводится реализация `xfsdist(8)` для `bpfttrace`, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing XFS operation latency... Hit Ctrl-C to end.\n");
}

kprobe:xfs_file_read_iter,
kprobe:xfs_file_write_iter,
kprobe:xfs_file_open,
kprobe:xfs_file_fsync
{
    @start[tid] = nsecs;
    @name[tid] = func;
}

kretprobe:xfs_file_read_iter,
kretprobe:xfs_file_write_iter,
kretprobe:xfs_file_open,
kretprobe:xfs_file_fsync
/@start[tid]/
{
    @us[@name[tid]] = hist((nsecs - @start[tid]) / 1000);
    delete(@start[tid]);
    delete(@name[tid]);
}

END
{
    clear(@start);
    clear(@name);
}
```

Эта программа трассирует функции XFS из структуры `file_operations`. Но как сказано в следующем разделе с описанием `ext4`, далеко не все файловые системы поддерживают такое простое отображение операций.

8.3.23. ext4dist

В репозитории ВСС есть инструмент `ext4dist(8)`¹, действующий как и `xfsdist(8)`, но предназначенный для файловой системы `ext4`. Описание вывода и порядка использования см. в разделе 8.3.22.

¹ Немного истории: этот инструмент я создал 12 февраля 2016 года, взяв за основу свой инструмент `zfsdist.d`, написанный в 2012 году на основе `DTrace`, а версию для `bpfttrace` я написал 2 февраля 2019 года специально для этой книги.

Этот инструмент имеет одну отличительную особенность, иллюстрирующую сложность использования зондов kprobes. Вот структура `struct ext4_file_operations` из исходных кодов ядра Linux 4.8:

```
const struct file_operations ext4_file_operations = {
    .llseek      = ext4_llseek,
    .read_iter   = generic_file_read_iter,
    .write_iter  = ext4_file_write_iter,
    .unlocked_ioctl = ext4_ioctl,
    [...]
}
```

Функция чтения, выделенная жирным шрифтом, — это обобщенная функция `generic_file_read_iter()`, которая не является характерной для `ext4`. Это проблема: при трассировке этой обобщенной функции инструмент будет фиксировать операции чтения в файловых системах других типов, в результате чего выходные данные окажутся загрязнены.

Чтобы решить эту проблему, при трассировке `generic_file_read_iter()` можно проверить аргументы и по ним определить, откуда пришел вызов — из файловой системы `ext4` или из какой-то другой. Код в BPF проверяет аргумент `struct kiocb *icb` и выходит из функции трассировки, если операция чтения выполняется не для файловой системы `ext4`:

```
// фильтр ext4 по условию file->f_op == ext4_file_operations
struct file *fp = iocb->ki_filp;
if ((u64)fp->f_op != EXT4_FILE_OPERATIONS)
    return 0;
```

Во время запуска программы значение `EXT4_FILE_OPERATIONS` заменяется фактическим адресом структуры `ext4_file_operations` из `/proc/kallsyms`. Конечно, это грубый прием, но работающий. Такой способ влечет за собой дополнительный оверхед, связанный с трассировкой всех вызовов `generic_file_read_iter()`, исходящих из файловых систем других типов, а также с дополнительной проверкой в программе BPF.

В Linux 4.10 используемые функции изменились. Теперь, вместо заявления о гипотетической возможности, можно исследовать фактическое изменение в ядре и его влияние на kprobes. Структура `file_operations` теперь приобрела вид:

```
const struct file_operations ext4_file_operations = {
    .llseek      = ext4_llseek,
    .read_iter   = ext4_file_read_iter,
    .write_iter  = ext4_file_write_iter,
    .unlocked_ioctl = ext4_ioctl,
    [...]
}
```

Сравните ее с предыдущей версией. В новой версии операция чтения ссылается на функцию `ext4_file_read_iter()`, которую можно трассировать непосредственно, соответственно, отпала необходимость отделять вызовы функций в `ext4` от вызова универсальной функции.

bpfttrace

Приветствуя это изменение в ядре, я написал версию ext4dist(8) для Linux 4.10 и выше (до версии, в которой произойдут обратные изменения). Вот пример вывода этого инструмента:

```
# ext4dist.bt
```

```
Attaching 9 probes...
```

```
Tracing ext4 operation latency... Hit Ctrl-C to end.
```

```
^C
```

```
@us[ext4_sync_file]:
```

```
[1K, 2K)      2 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[2K, 4K)      1 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4K, 8K)      0 | |
[8K, 16K)     1 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
```

```
@us[ext4_file_write_iter]:
```

```
[1]           14 | @@@@@@ |
[2, 4)        28 | @@@@@@@@@@@@@@ |
[4, 8)        72 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8, 16)       114 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[16, 32)      26 | @@@@@@@@@@@@@@ |
[32, 64)      61 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[64, 128)     5  | @@ |
[128, 256)    0  | |
[256, 512)    0  | |
[512, 1K)     1  | |
```

```
@us[ext4_file_read_iter]:
```

```
[0]           1 | |
[1]           1 | |
[2, 4)        768 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4, 8)        385 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8, 16)       112 | @@@@@@@@@@ |
[16, 32)      18 | @ |
[32, 64)       5 | |
[64, 128)     0  | |
[128, 256)    124 | @@@@@@@@@@ |
[256, 512)    70 | @@@@ |
[512, 1K)     3  | |
```

```
@us[ext4_file_open]:
```

```
[0]           1105 | @@@@@@@@@@ |
[1]           221 | @@ |
[2, 4)        5377 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4, 8)        359 | @@@ |
[8, 16)       42  | |
[16, 32)      5   | |
[32, 64)      1   | |
```

Время в гистограммах измеряется в микросекундах, соответственно, все задержки в этом выводе не превышают 1 миллисекунды.

Исходный код:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing ext4 operation latency... Hit Ctrl-C to end.\n");
}

kprobe:ext4_file_read_iter,
kprobe:ext4_file_write_iter,
kprobe:ext4_file_open,
kprobe:ext4_sync_file
{
    @start[tid] = nsecs;
    @name[tid] = func;
}

kretprobe:ext4_file_read_iter,
kretprobe:ext4_file_write_iter,
kretprobe:ext4_file_open,
kretprobe:ext4_sync_file
/@start[tid]/
{
    @us[@name[tid]] = hist((nsecs - @start[tid]) / 1000);
    delete(@start[tid]);
    delete(@name[tid]);
}

END
{
    clear(@start);
    clear(@name);
}
```

Для карты выбрано имя «@us», чтобы показать в выводе, что время измеряется в микросекундах.

8.3.24. icstat

icstat(8)¹ трассирует попадания и промахи в кэш индексных узлов и выводит статистику каждую секунду. Например:

¹ Немного истории: я создал этот инструмент специально для книги 2 февраля 2019 года. Свой первый инструмент для отображения статистики использования кэша индексных узлов — inodestat7 — я написал 11 марта 2004 года. До этого тоже были инструменты получения статистики использования индексных узлов (например, SE Toolkit).

```
# icstat.bt
Attaching 3 probes...
Tracing icache lookups... Hit Ctrl-C to end.
      REFS      MISSES  HIT%
        0         0    0%
    21647         0  100%
    38925     35250    8%
    33781     33780    0%
      815       806    1%
        0         0    0%
        0         0    0%

[...]
```

Как показывает этот вывод, в первую секунду наблюдались только попадания в кэш, а затем на протяжении нескольких секунд преобладали промахи. Роль рабочей нагрузки играла команда `/var -ls`, которая выполняет обход индексных узлов и выводит информацию о них.

Вот как выглядит исходный код `icstat(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing icache lookups... Hit Ctrl-C to end.\n");
    printf("%10s %10s %5s\n", "REFS", "MISSES", "HIT%");
}

kretprobe:find_inode_fast
{
    @refs++;
    if (retval == 0) {
        @misses++;
    }
}

interval:s:1
{
    $hits = @refs - @misses;
    $percent = @refs > 0 ? 100 * $hits / @refs : 0;
    printf("%10d %10d %4d%%\n", @refs, @misses, $percent);
    clear(@refs);
    clear(@misses);
}

END
{
    clear(@refs);
    clear(@misses);
}
```

Как и в `dcstat(8)`, для предотвращения деления на ноль при вычислении процентов выполняется проверка `@refs` на равенство нулю.

8.3.25. bufgrow

`bufgrow(8)`¹ — это инструмент для `bpftrace`, который дает некоторое представление о работе кэша буферов. Он показывает рост кэша страниц только для блочных страниц (кэш буферов, используемых для блочного ввода/вывода), сообщая, какие процессы увеличили кэш и насколько. Например:

```
# bufgrow.bt
Attaching 1 probe...
^C
```

```
@kb[dd]: 101856
```

В период трассировки процессы «`dd`» увеличили кэш буферов примерно на 100 Мбайт. Это был искусственный тест, в ходе которого с помощью команды `dd(1)` производилось копирование данных с блочного устройства и кэш буферов увеличился на 100 Мбайт:

```
# free -wm
```

	total	used	free	shared	buffers	cache	available
Mem:	70336	471	69328	26	2	534	68928
Swap:	0	0	0				
[...]							

```
# free -wm
```

	total	used	free	shared	buffers	cache	available
Mem:	70336	473	69153	26	102	607	68839
Swap:	0	0	0				

Вот как выглядит исходный код `bufgrow(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/fs.h>

kprobe:add_to_page_cache_lru
{
    $as = (struct address_space *)arg1;
    $mode = $as->host->i_mode;
    // отфильтровать операции блочного ввода/вывода, uapi/linux/stat.h:
    if ($mode & 0x6000) {
        @kb[comm] = sum(4); // размер страницы
    }
}
```

Этот инструмент использует `kprobes` для трассировки функции `add_to_page_cache_lru()` и производит фильтрацию по типу ввода/вывода. Поскольку для определения типа ввода/вывода требуется анализировать содержимое структуры, проверка осуществляется в операторе `if`, а не в фильтре зонда. Эта функция может вызываться очень часто, поэтому применение `bufgrow(8)` может привести к значительному оверхеду в высоконагруженных системах.

¹ Немного истории: этот инструмент я создал для книги 3 февраля 2019 года.

8.3.26. readahead

`readahead(8)`¹ трассирует автоматическое опережающее чтение файлов (не системный вызов `readahead(2)`) и сообщает, использовались ли в период трассировки страницы, созданные в ходе опережающего чтения, а также время между чтением страницы и ее использованием. Например:

```
# readahead.bt
Attaching 5 probes...
^C
Readahead unused pages: 128

Readahead used page age (ms):
@age_ms:
[1]                2455 |@@@@@@@@@@@@@@@@@@@@@|
[2, 4)            8424 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[4, 8)            4417 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[8, 16)           7680 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[16, 32)          4352 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[32, 64)           0   |
[64, 128)          0   |
[128, 256)         384 |@@|
```

Как показывает этот вывод, 128 страниц из прочитанных механизмом опережающего чтения во время трассировки остались неиспользованными (это не много). Гистограмма показывает, что тысячи страниц были прочитаны и использованы в основном за 32 миллисекунды. Когда это время измеряется десятками секунд, это может говорить о том, что механизм опережающего чтения действует слишком агрессивно и требует настройки.

Этот инструмент был создан для анализа поведения механизма опережающего чтения на производственных экземплярах Netflix, где используются твердотельные накопители, в комбинации с которыми опережающее чтение гораздо менее эффективно, чем в комбинации с обычными вращающимися жесткими дисками, и может отрицательно сказываться на производительности. Эта конкретная проблема описана и в разделе, посвященном инструменту `biosnoop(8)` в главе 9, который раньше использовался для такого анализа.

Исходный код `readahead(8)`:

```
#!/usr/local/bin/bpftrace

kprobe:__do_page_cache_readahead { @in_readahead[tid] = 1; }
kretprobe:__do_page_cache_readahead { @in_readahead[tid] = 0; }

kretprobe:__page_cache_alloc
/@in_readahead[tid]/
{
```

¹ Немного истории: этот инструмент я создал специально для книги 3 февраля 2019 года. Я много лет говорил, что нужно написать этот инструмент, и наконец сделал это.


```

    @birth[retval] = nsecs;
    @rapages++;
}

kprobe:mark_page_accessed
/@birth[arg0]/
{
    @age_ms = hist((nsecs - @birth[arg0]) / 1000000);
    delete(@birth[arg0]);
    @rapages--;
}

END
{
    printf("\nReadahead unused pages: %d\n", @rapages);
    printf("\nReadahead used page age (ms):\n");
    print(@age_ms); clear(@age_ms);
    clear(@birth); clear(@in_readahead); clear(@rapages);
}

```

readahead(8) использует kprobes для инструментации различных функций ядра. Он устанавливает флаг для текущего потока в `__do_page_cache_readahead()`, который проверяется во время размещения страницы в памяти, чтобы узнать, была ли она создана механизмом опережающего чтения. В этом случае для ключа с адресом структуры страницы в хеш-таблице сохраняется отметка времени. Позже, когда происходит обращение к странице, это время используется для добавления в гистограмму интервала от создания до использования. Количество неиспользованных страниц — это разность между количеством страниц, выделенных механизмом опережающего чтения, и использованных в течение работы программы.

Если реализация ядра изменится, в этот инструмент понадобится внести соответствующие корректировки. Кроме того, трассировка функций управления страницами и сохранение дополнительных метаданных почти наверняка приведут к значительному оверхеду, потому что эти функции вызываются часто. В высоконагруженных системах оверхед этого инструмента может достигать 30% и даже больше, поэтому желательно использовать его только для краткосрочного анализа.

В конце главы 9 показан однострочный сценарий `bpfttrace`, который вычисляет отношение числа прочитанных блоков к числу блоков, загруженных механизмом опережающего чтения.

8.3.27. Другие инструменты

Вот еще несколько стоящих инструментов BPF:

- **ext4slower(8), ext4dist(8)**: аналоги BCC-инструментов `xfsslower(8)` и `xfsdist(8)` для `ext4`;
- **btrfs slower(8), btrfssdist(8)**: аналоги BCC-инструментов `xfsslower(8)` и `xfsdist(8)` для `btrfs`;

- **zfsslower(8), zfsdist(8)**: аналоги BCC-инструментов **xfsslower(8)** и **xfsdist(8)** для **zfs**;
- **nfsslower(8), nfsdist(8)**: аналоги BCC-инструментов **xfsslower(8)** и **xfsdist(8)** для **NFSv3** и **NFSv4**.

8.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечисляются однострочные сценарии для BCC и **bpfftrace**. Там, где это возможно, один и тот же сценарий реализуется с использованием BCC и **bpfftrace**.

8.4.1. BCC

Трассирует события открытия файлов процессами с использованием **open(2)**:

```
opensnoop
```

Трассирует события создания файлов процессами с использованием **creat(2)**:

```
trace 't:syscalls:sys_enter_creat "%s", args->pathname'
```

Подсчитывает вызовы **newstat(2)** по именам файлов:

```
argdist -C 't:syscalls:sys_enter_newstat():char*:args->filename'
```

Подсчитывает операции чтения по типам системных вызовов:

```
funccount 't:syscalls:sys_enter_*read*'
```

Подсчитывает операции записи по типам системных вызовов:

```
funccount 't:syscalls:sys_enter_*write*'
```

Выводит гистограмму с распределением вызовов **read()** по запрошенным размерам блоков:

```
argdist -H 't:syscalls:sys_enter_read():int:args->count'
```

Выводит гистограмму с распределением вызовов **read()** по размерам прочитанных блоков (и кодам ошибок):

```
argdist -H 't:syscalls:sys_exit_read():int:args->ret'
```

Подсчитывает количество вызовов **read()**, завершившихся ошибкой, для каждого кода ошибки:

```
argdist -C 't:syscalls:sys_exit_read():int:args->ret:args->ret<0'
```

Подсчитывает количество вызовов **VFS**:

```
funccount 'vfs_*'
```

Подсчитывает количество прохождений через точки трассировки в ext4:

```
funccount 't:ext4:*
```

Подсчитывает количество прохождений через точки трассировки в xfs:

```
funccount 't:xfs:*
```

Посчитывает количество операций чтения файлов в файловой системе в ext4 по именам процессов и трассировкам стека:

```
stackcount ext4_file_read_iter
```

Посчитывает количество операций чтения файлов в файловой системе в ext4 по именам процессов и трассировкам стека в пространстве пользователя:

```
stackcount -U ext4_file_read_iter
```

Определяет время выполнения spa_sync() в ZFS:

```
trace -T 'spa_sync "ZFS spa_sync()"
```

Подсчитывает число операций чтения из устройств хранения с любой файловой системой через вызов read_pages() по именам процессов и трассировкам стека:

```
stackcount -P read_pages
```

Подсчитывает число операций чтения из устройств хранения с файловой системой ext4 по именам процессов и трассировкам стека:

```
stackcount -P ext4_readpages
```

8.4.2. bpftrace

Трассирует события открытия файлов процессами с использованием open(2):

```
bpftrace -e 't:syscalls:sys_enter_open { printf("%s %s\n", comm, str(args->filename)); }'
```

Трассирует события создания файлов процессами с использованием creat(2):

```
bpftrace -e 't:syscalls:sys_enter_creat { printf("%s %s\n", comm, str(args->pathname)); }'
```

Подсчитывает вызовы newstat(2) по именам файлов:

```
bpftrace -e 't:syscalls:sys_enter_newstat { @[str(args->filename)] = count(); }'
```

Подсчитывает операции чтения по типам системных вызовов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_*read* { @[probe] = count(); }'
```

Подсчитывает операции записи по типам системных вызовов:

```
bpfttrace -e 'tracepoint:syscalls:sys_enter_*write* { @[probe] = count(); }'
```

Выводит гистограмму с распределением вызовов read() по запрошенным размерам блоков:

```
bpfttrace -e 'tracepoint:syscalls:sys_enter_read { @ = hist(args->count); }'
```

Выводит гистограмму с распределением вызовов read() по размерам прочитанных блоков (и кодам ошибок):

```
bpfttrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Подсчитывает количество вызовов read(), завершившихся ошибкой, для каждого кода ошибки:

```
bpfttrace -e 't:syscalls:sys_exit_read /args->ret < 0/ { @[- args->ret] = count(); }'
```

Подсчитывает количество вызовов VFS:

```
bpfttrace -e 'kprobe:vfs_* { @[probe] = count(); }'
```

Подсчитывает количество прохождений через точки трассировки в ext4:

```
bpfttrace -e 'tracepoint:ext4:* { @[probe] = count(); }'
```

Подсчитывает количество прохождений через точки трассировки в xfs:

```
bpfttrace -e 'tracepoint:xfs:* { @[probe] = count(); }'
```

Подсчитывает количество операций чтения файлов в файловой системе в ext4 по именам процессов:

```
bpfttrace -e 'kprobe:ext4_file_read_iter { @[comm] = count(); }'
```

Подсчитывает количество операций чтения файлов в файловой системе в ext4 по именам процессов и трассировкам стека в пространстве пользователя:

```
bpfttrace -e 'kprobe:ext4_file_read_iter { @[ustack, comm] = count(); }'
```

Определяет время выполнения spa_sync() в ZFS:

```
bpfttrace -e 'kprobe:spa_sync { time("%H:%M:%S ZFS spa_sync()\n"); }'
```

Подсчитывает число обращений к кэшу каталогов по именам и идентификаторам (PID) процессов:

```
bpfttrace -e 'kprobe:lookup_fast { @[comm, pid] = count(); }'
```

Подсчитывает число операций чтения из устройств хранения с любой файловой системой через вызов read_pages() по именам процессов и трассировкам стека в пространстве ядра:

```
bpfttrace -e 'kprobe:read_pages { @[kstack] = count(); }'
```

Подсчитывает число операций чтения из устройств хранения с файловой системой ext4 по именам процессов и трассировкам стека в пространстве ядра:

```
bpftrace -e 'kprobe:ext4_readpages { @[kstack] = count(); }'
```

8.4.3. Примеры использования однострочных сценариев BPF

По аналогии с инструментами, описанными в этой главе, я включил сюда примеры вывода некоторых однострочников, чтобы показать их работу.

Подсчитывает операции чтения по типам системных вызовов

```
# funccount -d 10 't:syscalls:sys_enter_*read*'
Tracing 9 functions for "t:syscalls:sys_enter_*read*"... Hit Ctrl-C to end.
```

FUNC	COUNT
syscalls:sys_enter_pread64	3
syscalls:sys_enter_readlinkat	34
syscalls:sys_enter_readlink	294
syscalls:sys_enter_read	9863782

Detaching...

В этом примере используется параметр `-d 10`, ограничивающий время трассировки 10 секундами. Этот и другие подобные сценарии, использующие шаблоны «`*write*`» и «`*open*`», удобно применять для определения используемых вариантов системных вызовов, чтобы затем исследовать их более подробно. Этот вывод получен на производственном сервере с 36 процессорами, где почти всегда используется системный вызов `read(2)` — в данном случае за те 10 секунд, пока выполнялась трассировка, он был вызван почти 10 миллионов раз.

Вывод гистограммы с распределением вызовов read() по размерам прочитанных блоков (и кодам ошибок)

```
# bpftrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
Attaching 1 probe...
^C
```

@:		
(..., 0)	279	
[0]	2899	@@@@
[1]	15609	@@@
[2, 4)	73	
[4, 8)	179	
[8, 16)	374	
[16, 32)	2184	@@@@
[32, 64)	1421	@@@
[64, 128)	2758	@@@@

[128, 256)	3899	@@@@@@@@@@	
[256, 512)	8913	@@@@@@@@@@@@@@@@@@@@@@	
[512, 1K)	16498	@@	
[1K, 2K)	16170	@@	
[2K, 4K)	19885	@@	
[4K, 8K)	23926	@@	
[8K, 16K)	9974	@@@@@@@@@@@@@@@@@@@@@@@@@@	
[16K, 32K)	7569	@@@@@@@@@@@@@@@@@@	
[32K, 64K)	1909	@@@@	
[64K, 128K)	551	@	
[128K, 256K)	149		
[256K, 512K)	1		

Как показывает этот вывод, большинство операций чтения возвращает данные блоками от 512 байт до 8 Кбайт. Здесь также видно, что 15 609 операций чтения вернули только 1 байт, поэтому они, возможно, могут стать целью для оптимизации. Их можно исследовать подробнее, извлекая соответствующие им трассировки стеков, например, так:

```
bpfftrace -e 'tracepoint:syscalls:sys_exit_read /args->ret == 1/ { @[ustack] =
    count(); }'
```

Также в гистограмме видны 2899 операций чтения, вернувших 0 байт. Такие операции в целом считаются нормальным явлением в зависимости от источника данных, а также когда данные, доступные для чтения, закончились. 279 событий с отрицательным возвращаемым значением — это ошибки, которые также можно исследовать отдельно.

Подсчет прохождений через точки трассировки в xfs

```
# funccount -d 10 't:xfs:*'
Tracing 496 functions for "t:xfs:*"... Hit Ctrl-C to end.
FUNC                                COUNT
xfs:xfs_buf_delwri_queued           1
xfs:xfs_irele                        1
xfs:xfs_inactive_symlink            2
xfs:xfs_dir2_block_addname          4
xfs:xfs_buf_trylock_fail            5
[...]
xfs:xfs_trans_read_buf              9548
xfs:xfs_trans_log_buf               11800
xfs:xfs_buf_read                    13320
xfs:xfs_buf_find                    13322
xfs:xfs_buf_get                     13322
xfs:xfs_buf_trylock                 15740
xfs:xfs_buf_unlock                   15836
xfs:xfs_buf_rele                     20959
xfs:xfs_perag_get                    21048
xfs:xfs_perag_put                    26230
xfs:xfs_file_buffered_read           43283
xfs:xfs_getattr                      80541
xfs:xfs_write_extent                 121930
```

xfs:xfs_update_time	137315
xfs:xfs_log_reserve	140053
xfs:xfs_log_reserve_exit	140066
xfs:xfs_log_ungrant_sub	140094
xfs:xfs_log_ungrant_exit	140107
xfs:xfs_log_ungrant_enter	140195
xfs:xfs_log_done_nonperm	140264
xfs:xfs_iomap_found	188507
xfs:xfs_file_buffered_write	188759
xfs:xfs_writepage	476196
xfs:xfs_releasepage	479235
xfs:xfs_ilock	581785
xfs:xfs_iunlock	589775
Detaching...	

В XFS так много точек трассировки, что пришлось обрезать вывод в этом примере для экономии места. Они позволяют всесторонне исследовать внутреннюю работу XFS и найти причину проблем.

Подсчет числа операций чтения из устройств хранения по именам процессов и трассировкам стека

```
# stackcount -P ext4_readpages
```

```
Tracing 1 functions for "ext4_readpages"... Hit Ctrl-C to end.
```

```
^C
```

```
ext4_readpages
read_pages
__do_page_cache_readahead
filemap_fault
ext4_filemap_fault
__do_fault
__handle_mm_fault
handle_mm_fault
__do_page_fault
async_page_fault
__clear_user
load_elf_binary
search_binary_handler
__do_execve_file.isra.36
__x64_sys_execve
do_syscall_64
entry_SYSCALL_64_after_hwframe
[unknown]
head [28475]
1
```

```
ext4_readpages
read_pages
__do_page_cache_readahead
ondemand_readahead
generic_file_read_iter
__vfs_read
```

```

vfs_read
kernel_read
prepare_binprm
__do_execve_file.isra.36
__x64_sys_execve
do_syscall_64
entry_SYSCALL_64_after_hwframe
[unknown]
  bash [28475]
  1

```

Detaching...

В этом выводе есть только два события, но именно эти события я хотел здесь зафиксировать. Первое — это сбой страницы, приводящий к вызову `ext4_readpages()` и чтению с диска (на самом деле это был вызов `execve(2)`, загрузивший двоичный код программы). Второе — нормальный вызов `read(2)`, который достигает функции `ext4_readpages()` через механизм опережающего чтения. Это весьма наглядные примеры операций чтения из адресного пространства и файлов. Вывод также показывает, как анализ трассировки стека ядра позволяет получить больше информации о событиях. Эти стеки получены в Linux 4.18 и могут различаться в разных версиях ядра Linux.

8.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Все упражнения можно выполнить с помощью `bpfttrace` или BCC, если явно не указано иное.

1. Измените инструмент `filelife(8)` так, чтобы он использовал точки трассировки в системных вызовах `creat(2)` и `unlink(2)`.
2. Какие преимущества и недостатки дает такая переделка `filelife(8)`?
3. Напишите версию `vfsstat(8)`, которая выводит отдельные строки для локальной файловой системы и TCP (см. `vfssize(8)` и `fsrwstat(8)`). Вот возможный пример вывода этой версии:

```

# vfsstatx
TIME      FS      READ/s    WRITE/s   CREATE/s   OPEN/s    FSYNC/s
02:41:23: ext4    1715013   38717     0           5379      0
02:41:23: TCP      1431     1311      0            5        0
02:41:24: ext4    947879   30903     0          10547     0
02:41:24: TCP      1231      982      0            4        0
[...]
```

4. Напишите инструмент, отображающий отношение числа операций логического ввода/вывода (через VFS или интерфейс файловой системы) и физического ввода/вывода (через точки трассировки блочного ввода/вывода).
5. Напишите инструмент для выявления утечек файловых дескрипторов, то есть дескрипторов, которые были получены, но не были освобождены в течение

трассировки. Одно из возможных решений: трассировка функций ядра `__alloc_fd()` и `__close_fd()`.

6. (Усложненное.) Напишите инструмент, подсчитывающий операции ввода/вывода по точкам монтирования файловых систем.
7. (Усложненное, не решено.) Напишите инструмент, отображающий распределение времени между обращениями к кэшу страниц. В чем сложность создания такого инструмента?

8.6. ИТОГИ

В этой главе мы рассмотрели инструменты BPF для анализа файловых систем, которые в своей работе используют системные вызовы, вызовы VFS, а также функции и точки трассировки файловых систем, анализируют операции отложенной записи и опережающего чтения, трассируют деятельность кэшей страниц, каталогов, индексных узлов и буферов. Я включил инструменты, которые выводят гистограммы распределения задержек в работе файловых систем, помогающие выявлять мультимодальные распределения и аномальные выбросы, и решают проблемы производительности приложений.

Глава 9

ДИСКОВЫЙ ВВОД/ВЫВОД

Дисковый ввод/вывод — типичный источник проблем с производительностью, потому что задержка ввода/вывода при интенсивном использовании дисков может достигать десятков миллисекунд и даже больше, что на несколько порядков медленнее, чем наносекундные или микросекундные задержки в операциях CPU и с памятью. Анализ с применением инструментов BPF помогает найти оптимальные настройки дискового ввода/вывода или вообще избавиться от него и получить в результате значительное увеличение производительности приложений.

Под *дисковым вводом/выводом* понимаются любые операции ввода/вывода с устройствами хранения: вращающимися магнитными носителями, флеш-памятью и сетевыми хранилищами. Все они могут быть представлены в Linux как устройства хранения и проанализированы с использованием одних и тех же инструментов.

Между приложением и устройством хранения обычно находится файловая система. Файловые системы используют кэширование, опережающее чтение, буферизацию и асинхронный ввод/вывод, чтобы исключить возможность блокировки приложений при медленном вводе/выводе на диск. Поэтому советую начинать с анализа файловых систем, как было описано в главе 8.

Инструменты трассировки давно стали основой для исследования дискового ввода/вывода: первые популярные инструменты трассировки дискового ввода/вывода — `iosnoop(8)` и `iostat(8)` — я написал в 2004 и 2005 годах соответственно. Теперь они входят в состав различных ОС. Также я разработал BPF-версии: `biosnoop(8)` и `biostat(8)`, добавив наконец давно ожидавшуюся поддержку анализа устройств блочного ввода/вывода. В этой главе мы познакомимся с этими и другими инструментами исследования дискового ввода/вывода.

Цели обучения:

- познакомиться со стеком ввода/вывода и ролью планировщиков ввода/вывода в Linux;
- изучить стратегию успешного анализа производительности дискового ввода/вывода;

- определять аномальные задержки в операциях дискового ввода/вывода;
- анализировать мультимодальные распределения задержек в операциях дискового ввода/вывода;
- определять пути в коде, вызывающие проблемы и задержки в операциях дискового ввода/вывода;
- анализировать задержки в планировщиках ввода/вывода;
- использовать однострочные сценарии `bpfttrace` для исследования дискового ввода/вывода нестандартными способами.

В начале этой главы обсудим некоторые теоретические основы, необходимые для анализа дискового ввода/вывода, и познакомимся со стеком ввода/вывода. Попутно я расскажу, на какие вопросы сможет ответить BPF, и предложу общую стратегию анализа. Затем перейдем к знакомству с инструментами: от традиционных инструментов анализа дискового ввода/вывода до инструментов BPF, включая однострочники BPF. В заключение вас ждут дополнительные упражнения.

9.1. ОСНОВЫ

В этом разделе познакомимся с основными понятиями дискового ввода/вывода, возможностями BPF и оптимальной стратегией анализа производительности дисков.

9.1.1. Основы дисков

Стек блочного ввода/вывода

На рис. 9.1 показаны основные компоненты стека блочного ввода/вывода в Linux.

Под термином *блочный ввод/вывод* подразумевается доступ к устройству, производящему ввод и вывод данных блоками, размер которых традиционно измеряется количеством 512-байтных секторов. Интерфейс блочного устройства возник в Unix. В Linux реализован улучшенный блочный ввод/вывод с привлечением планировщиков, повышающих производительность ввода/вывода, диспетчеров томов для группировки нескольких устройств и механизма отображения для создания виртуальных устройств.

Внутреннее устройство

При обсуждении инструментов BPF я буду ссылаться на некоторые типы ядра, которые используются стеком ввода/вывода. Для начала отмечу, что ввод/вывод передается через стек в виде структуры `request` (определена в `include/linux/blkdev.h`), а на более низких уровнях в виде структуры `bio` (определена в `include/linux/blk_types.h`).



Рис. 9.1. Стек блочного ввода/вывода в Linux

rwbs

Для отслеживания наблюдаемости ядро предоставляет возможность описать тип каждой операции ввода/вывода с помощью строки символов с именем `rwbs`. Она определена в функции ядра `blk_fill_rwbs()` и использует следующие символы:

- **R**: Read — чтение;
- **W**: Write — запись;
- **M**: Metadata — метаданные;
- **S**: Synchronous — синхронная;
- **A**: Read-ahead — опережающее чтение;
- **F**: Flush — принудительное обращение к блоку;
- **D**: Discard — сброс;
- **E**: Erase — стирание;
- **N**: None — нет операции.

Символы могут комбинироваться. Например, "WM" означает «запись метаданных».

Планировщики ввода/вывода

Операция ввода/вывода ставится в очередь и планируется на блочном уровне классическими планировщиками (они есть только в версиях Linux до 5.0) или новыми планировщиками с несколькими очередями. К классическим планировщикам относятся:

- **Noop**: пустой планировщик (не выполняющий планирование);
- **Deadline**: гарантирующий выполнение операции не дольше определенного времени, может использоваться в системах реального времени;
- **CFQ**: полностью справедливый планировщик очередей (completely fair queueing scheduler); выделяет процессам кванты времени для ввода/вывода подобно планировщику процессора.

Проблемой классических планировщиков было использование одной очереди запросов, защищенной единственной блокировкой, которая становилась узким местом в производительности при высокой скорости ввода/вывода. Драйвер с несколькими очередями (blk-mq, добавленный в Linux 3.13) решает эту проблему, используя отдельные очереди передачи для каждого процессора и несколько очередей диспетчеризации для устройств. Это увеличивает производительность и уменьшает задержку ввода/вывода по сравнению с классическими планировщиками, потому что запросы могут обрабатываться параллельно и на том же процессоре, где была инициирована операция ввода/вывода. Это было необходимо для поддержки устройств на основе флеш-памяти, а также устройств других типов, способных обрабатывать миллионы операций ввода/вывода в секунду [90].

Доступные планировщики с несколькими очередями:

- **None**: планировщик без очередей;
- **BFQ**: планировщик очередей со справедливым распределением бюджета (budget fair queueing scheduler), аналогичен CFQ, но распределяет полосу пропускания и время ввода/вывода;
- **mq-deadline**: blk-mq-версия планировщика Deadline с несколькими очередями;
- **Kyber**: планировщик, регулирующий длину очереди чтения и записи в зависимости от производительности, чтобы достигнуть целевых значений задержек чтения и записи.

Классические планировщики и устаревший стек ввода/вывода были удалены в версии Linux 5.0. Все планировщики теперь управляют несколькими очередями.

Производительность дискового ввода/вывода

На рис. 9.2 показана организация дискового ввода/вывода в терминах ОС.

Время ожидания в ОС — это время, пока запрос находился в очередях планировщика блочного уровня и диспетчера устройств. Время обслуживания — это время

от передачи запроса в устройство до завершения. Сюда может входить время ожидания в очереди на устройство. Время обработки запроса — это общее время от момента, когда запрос на выполнение операции ввода/вывода был поставлен в очереди ОС, до завершения. Время обработки запроса имеет большое значение, так как приложения, производящие синхронный ввод/вывод, будут вынуждены приостановиться на это время.

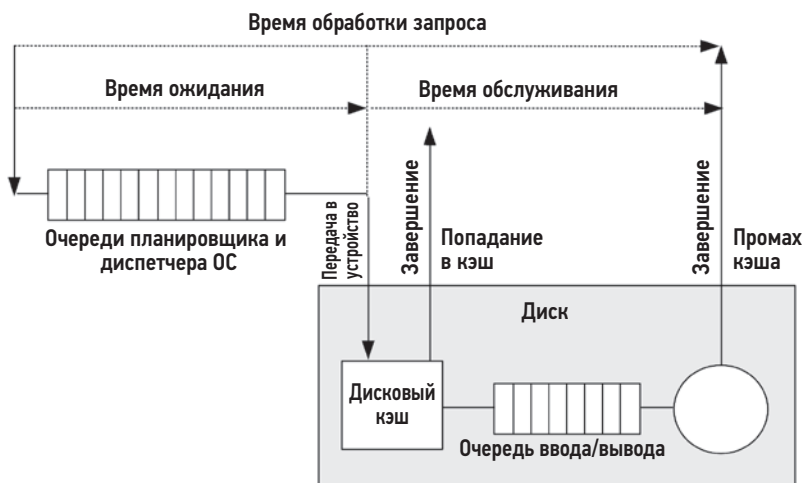


Рис. 9.2. Дискový ввод/вывод

На диаграмму не попала одна важная характеристика — использование диска. Она может показаться идеально подходящей для планирования емкости: когда диск загружен на 100%, можно предположить, что есть проблема с производительностью. Но степень загрузки рассчитывается ОС как время, в течение которого диск *что-то делал*, и не учитывает виртуальные диски, которые могут поддерживаться некоторыми устройствами, или очереди в дисковом устройстве. Это обстоятельство может исказить метрику использования диска, например, когда диск, загруженный на 90%, может принять дополнительную нагрузку гораздо больше оставшихся 10%. Показатель загруженности все еще можно использовать для диагностики, и он является легкодоступной метрикой. Но такие параметры, как время ожидания, позволяют точнее диагностировать проблемы с производительностью диска.

9.1.2. Возможности BPF

Традиционные инструменты дают некоторое представление о производительности дисковых операций ввода/вывода, включая количество операций ввода/вывода в секунду, среднюю задержку и длины очередей, а также информацию о вводе/выводе по процессам. Эти инструменты рассмотрены в следующем разделе.

Инструменты трассировки BPF помогут получить дополнительную информацию и найти ответы на вопросы:

- Какие запросы ввода/вывода отправляются дискам? Какого типа, как часто и каков объем ввода/вывода?
- Как долго обрабатываются запросы? Сколько времени они проводят в очередях?
- Есть ли аномальные задержки?
- Является ли распределение задержек мультимодальным?
- Были ли ошибки в работе дисков?
- Какие команды SCSI посылались?
- Были ли тайм-ауты?

Чтобы ответить на эти вопросы, трассировка должна выполняться по всему стеку блочного ввода/вывода.

Источники событий

В табл. 9.1 перечислены доступные для инструментации источники событий дискового ввода/вывода.

Таблица 9.1. Доступные для инструментации источники событий дискового ввода/вывода

Типы событий	Источники событий
Интерфейс блочных устройств и уровень блочного ввода/вывода	Точки трассировки блочного ввода/вывода, kprobes
События планировщика ввода/вывода	kprobes
Ввод/вывод SCSI	Точки трассировки SCSI, kprobes
Ввод/вывод в драйверах устройств	kprobes

Эти источники событий позволяют наблюдать, как протекают операции ввода/вывода от интерфейса блочного ввода/вывода до драйверов устройств.

Чтобы получить представление о событиях, взгляните на аргументы функции `block:block_rq_issue`, которая посылает запрос на блочный ввод/вывод в устройство:

```
# bpftrace -lv tracepoint:block:block_rq_issue
tracepoint:block:block_rq_issue
    dev_t dev;
    sector_t sector;
    unsigned int nr_sector;
    unsigned int bytes;
    char rwbs[8];
    char comm[16];
    __data_loc char[] cmd;
```

Такие сведения, как «запрашиваемый объем ввода/вывода», можно получить с помощью однострочного сценария, использующего следующую точку трассировки:

```
bpfftrace -e 'tracepoint:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Комбинирование точек трассировки позволяет измерить время между событиями.

9.1.3. Стратегия

Если вы новичок в анализе дискового ввода/вывода, вот рекомендуемая общая стратегия. Упомянутые здесь инструменты более подробно рассматриваются в следующих разделах.

1. При появлении проблем с производительностью в приложениях начните с анализа файловой системы, как описано в главе 8.
2. Проверьте основные показатели работы диска: время обработки запросов, количество операций ввода/вывода в секунду и загруженность (например, с помощью `iostat(1)`). Обратите внимание на высокую загруженность (это явная подсказка) и более высокие, чем обычно, время обработки запроса (задержку) и количество операций ввода/вывода в секунду.
 - а) Если вы не знаете, какие значения времени обработки запроса (задержка) и количества операций ввода/вывода в секунду нормальные, создайте в простаивающей системе некоторую нагрузку, например, используя инструмент `fio(1)`, и исследуйте ее с помощью `iostat(1)`.
3. Выполните трассировку задержек блочного ввода/вывода. Проверьте распределение на мультимодальность и наличие аномальных задержек (например, с помощью `BCC biolateness(8)`).
4. Выполните трассировку отдельных блочных операций ввода/вывода и попробуйте выявить закономерности, такие как попытка чтения после записи (с этой целью можно использовать `BCC biosnoop(8)`).
5. Используйте другие инструменты и однострочные сценарии, представленные в этой главе.

Остановимся на первом шаге немного подробнее: начав с инструментов анализа дискового ввода/вывода, можно быстро выявить случаи высокой задержки, но тогда возникает вопрос: насколько они критичны? Ввод/вывод может быть асинхронным по отношению к приложению, и в этом случае тоже бывает интересно проанализировать его, но по другим причинам: чтобы выявить источники конкуренции с синхронными операциями ввода/вывода и для планирования емкости устройства.

9.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

В этом разделе рассмотрим применение `iostat(1)` для получения сводных характеристик дисковой активности, `perf(1)` для трассировки блочного ввода/вывода, `blktrace(8)`, а также журнал SCSI.

9.2.1. iostat

`iostat(1)` выводит сводную статистику ввода/вывода для каждого диска, сообщая количество операций ввода/вывода в секунду, пропускную способность, время обработки запросов ввода/вывода и загрузенность. Эта команда не требует привилегий суперпользователя и обычно используется при исследовании проблем дискового ввода/вывода. Статистика, получаемая этим инструментом, по умолчанию поддерживается ядром, поэтому он не оказывает большой нагрузки на систему.

`iostat(1)` поддерживает множество параметров для настройки вывода. Одна из полезных комбинаций, `-dxz 1`, требует от команды вывести только загрузенность диска (`-d`), добавить в вывод дополнительные столбцы (`-x`), пропускать устройства с нулевыми показателями (`-z`) и выводить информацию раз в секунду (`1`). Поле вывода настолько широко, что я покажу левую и правую части отдельно. Вот пример проблемы в производственной системе, которую я помогал отлаживать:

```
# iostat -dxz 1
Linux 4.4.0-1072-aws (...)      12/18/2018      _x86_64_      (16 CPU)
Device:      rrqm/s    wrqm/s      r/s      w/s      rkB/s      wkB/s \ ...
xvda         0.00      0.29       0.21     0.17       6.29       3.09 / ...
xvdb         0.00      0.08     44.39     9.98     5507.39    1110.55 \ ...

/ ...

Device:      rrqm/s    wrqm/s      r/s      w/s      rkB/s      wkB/s \ ...
xvdb         0.00      0.00     745.00     0.00    91656.00       0.00 / ...
\ ...
Device:      rrqm/s    wrqm/s      r/s      w/s      rkB/s      wkB/s / ...
xvdb         0.00      0.00     739.00     0.00    92152.00       0.00 \ ...
```

Эти столбцы содержат обобщенную информацию о рабочей нагрузке и помогают понять ее характерные особенности. Первые два говорят о слиянии в очереди: здесь новая операция ввода/вывода производит чтение или запись в место на диске по соседству с другой операцией ввода/вывода, находящейся в очереди, поэтому они объединяются для повышения эффективности.

Столбцы:

- **rrqm/s**: количество запросов на чтение в секунду, добавленных в очередь и объединенных;
- **wrqm/s**: количество запросов на запись в секунду, добавленных в очередь и объединенных;
- **r/s**: количество выполненных запросов на чтение в секунду (после объединения);
- **w/s**: количество выполненных запросов на запись в секунду (после объединения);
- **rkB/s**: прочитано с диска килобайт в секунду;
- **wkB/s**: записано на диск килобайт в секунду.

Первый блок данных (с информацией о двух устройствах — `xvda` и `xvdb`) содержит сводную информацию с момента загрузки и может использоваться для сравнения

с последующими односекундными сводками. Судя по этим данным, в среднем с устройства xvdb читается 5507 Кбайт/с, но текущие односекундные сводки показывают сильное увеличение объемов чтения — более 9000 Кбайт/с. То есть система испытывает более высокую, чем обычно, рабочую нагрузку чтения.

На основе этих столбцов можно вычислить средний объем данных, вовлеченных в операции чтения и записи. Разделив значение в столбце rkB/s на значение в столбце r/s, получаем средний объем операции чтения — около 124 Кбайт. Более новые версии `iostat(1)` включают столбцы `rareq-sz` (read average request size — средний объем читаемых данных на запрос) и `wareq-sz`.

Столбцы в правой половине:

```
... \ avgrq-sz avgqu-sz   await r_await w_await  svctm  %util
... /   49.32    0.00   12.74   6.96   19.87   3.96   0.15
... \  243.43    2.28   41.96   41.75   42.88   1.52   8.25
... /
... \ avgrq-sz avgqu-sz   await r_await w_await  svctm  %util
... /  246.06   25.32   33.84   33.84    0.00   1.35 100.40
... \
... \ avgrq-sz avgqu-sz   await r_await w_await  svctm  %util
... \  249.40   24.75   33.49   33.49    0.00   1.35 100.00
```

Они отражают производительность устройства. Подробнее:

- **avgrq-sz**: средний объем данных, вовлеченных в запрос в секторах (512 байт);
- **avgqu-sz**: среднее количество запросов, в том числе ожидающие в очереди драйвера и активные на устройстве;
- **await**: среднее время обработки запроса ввода/вывода (также известное как время отклика), включая время ожидания в очереди драйвера и время отклика устройства ввода/вывода (миллисекунды);
- **r_await**: то же, что и `await`, но только для запросов чтения (миллисекунды);
- **w_await**: то же, что и `await`, но только для запросов записи (миллисекунды);
- **svctm**: среднее (предполагаемое) время отклика диска на запрос ввода/вывода (миллисекунды);
- **%util**: процент времени, в течение *которого* устройство было занято обработкой запросов ввода/вывода.

Самый важный показатель производительности — ожидание. Если приложение и файловая система используют приемы, уменьшающие задержки записи (например, сквозную запись), то столбец `w_await` может иметь низкие значения, и вместо него вы можете сосредоточить внимание на `r_await`.

Столбец `%util` важен для оценки загруженности и планирования емкости, но имейте в виду, что это всего лишь мера занятости (доля времени, когда устройство не простаивало) и может почти ничего не значить для виртуальных устройств, объединяющих несколько физических дисков. Более точно оценить нагрузку на такие устройства можно по приложенной нагрузке: количеству операций в секунду ($r/s + w/s$) и пропускной способности ($rkB/s + wkB/s$).

В этом примере видно, что диск загружен на 100%, а среднее время чтения составляет 33 миллисекунды. Для приложенной нагрузки и дискового устройства это оказалось ожидаемой производительностью. Реальная проблема заключалась в том, что читаемые файлы стали настолько большими, что не помещались в кэш страниц и читались непосредственно с диска.

9.2.2. perf

С `perf(1)` мы познакомились в главе 6, когда обсуждали анализ счетчиков производительности РМС и выборку стека по времени. Этот инструмент можно использовать и для анализа дискового ввода/вывода с помощью точек трассировки блочного ввода/вывода.

Вот пример трассировки постановки запросов в очередь (`block_rq_insert`), их передачи на устройство хранения (`block_rq_issue`) и завершения обработки (`block_rq_complete`):

```
# perf record -e block:block_rq_insert,block:block_rq_issue,block:block_rq_complete -a
^C[ perf record: Woken up 7 times to write data ]
[ perf record: Captured and wrote 6.415 MB perf.data (20434 samples) ]
# perf script
  kworker/u16:3 25003 [004] 543348.164811: block:block_rq_insert: 259,0 RM 4096 ( )
2564656 + 8 [kworker/u16:3]
  kworker/4:1H   533 [004] 543348.164815: block:block_rq_issue: 259,0 RM 4096 ( )
2564656 + 8 [kworker/4:1H]
  swapper       0 [004] 543348.164887: block:block_rq_complete: 259,0 RM ( )
2564656 + 8 [0]
  kworker/u17:0 23867 [005] 543348.164960: block:block_rq_complete: 259,0 R ( )
3190760 + 256 [0]
  dd 25337 [001] 543348.165046: block:block_rq_insert: 259,0 R 131072 ( )
3191272 + 256 [dd]
  dd 25337 [001] 543348.165050: block:block_rq_issue: 259,0 R 131072 ( )
3191272 + 256 [dd]
  dd 25337 [001] 543348.165111: block:block_rq_complete: 259,0 R ( )
3191272 + 256 [0]
[...]
```

Вывод содержит много подробностей, начиная с процесса, выполнявшегося на процессоре, когда произошло событие, причем этот процесс может быть или не быть ответственным за событие. Среди прочего — отметка времени, старший и младший номера диска, строка, представляющая тип ввода/вывода (строка `rwbs`, описанная выше), и другие детали.

Раньше я создавал инструменты для постобработки этих событий, конструирования гистограмм задержек и визуализации закономерностей доступа¹. Но для высоко-

¹ См. `iolatency(8)` в `perf-tools` [78]: здесь для доступа к данным о событиях в буфере трассировки используется `Ftrace`, что позволяет избежать оверхеда на создание и запись файла `perf.data`.

нагруженных систем это означает передачу всех событий блочного ввода/вывода в пространство пользователя для последующей обработки. BPF может выполнять эту обработку в ядре и передавать в пространство пользователя только готовый результат, что намного эффективнее. Пример этого — инструмент `biosnoop(8)`, который рассматривается далее в этой главе.

9.2.3. blktrace

`blktrace(8)` — специализированная утилита для трассировки событий блочного ввода/вывода. Вот как можно использовать ее интерфейс `btrace(8)` для трассировки всех событий:

```
# btrace /dev/nvme2n1
259,0    2      1      0.000000000  430  Q  WS  2163864 + 8 [jbd2/nvme2n1-8]
259,0    2      2      0.000009556  430  G  WS  2163864 + 8 [jbd2/nvme2n1-8]
259,0    2      3      0.000011109  430  P   N  [jbd2/nvme2n1-8]
259,0    2      4      0.000013256  430  Q  WS  2163872 + 8 [jbd2/nvme2n1-8]
259,0    2      5      0.000015740  430  M  WS  2163872 + 8 [jbd2/nvme2n1-8]
[...]
259,0    2     15      0.000026963  430  I  WS  2163864 + 48 [jbd2/nvme2n1-8]
259,0    2     16      0.000046155  430  D  WS  2163864 + 48 [jbd2/nvme2n1-8]
259,0    2     17      0.000699822  430  Q  WS  2163912 + 8 [jbd2/nvme2n1-8]
259,0    2     18      0.000701539  430  G  WS  2163912 + 8 [jbd2/nvme2n1-8]
259,0    2     19      0.000702820  430  I  WS  2163912 + 8 [jbd2/nvme2n1-8]
259,0    2     20      0.000704649  430  D  WS  2163912 + 8 [jbd2/nvme2n1-8]
259,0   11      1      0.000664811      0  C  WS  2163864 + 48 [0]
259,0   11      2      0.001098435      0  C  WS  2163912 + 8 [0]
[...]
```

Для каждой операции ввода/вывода выводится несколько строк с описанием событий. Значения столбцов:

1. Старший и младший номера устройства.
2. CPU ID.
3. Порядковый номер
4. Продолжительность действия в секундах.
5. ID процесса.
6. ID действия (см. `blkparse(1)`): Q == Queued (постановка в очередь), G == Get request (получение запроса), P == Plug (блокировка очереди для накопления нескольких запросов), M == Merge (объединение запросов), D == Dispatches (передача в устройство), C == Completed (запрос обработан) и т. д.
7. Строка в формате RWBS (см. раздел «rwbs» выше в этой главе): W == Write (запись), S == Synchronous (синхронно) и т. д.
8. Адрес + размер [устройства].

Вывод можно обработать и визуализировать с помощью seekwatcher Криса Мейсона [91].

Как и в случае с perf(1), в системах с большим объемом дискового ввода/вывода оверхед blktrace(8) на вывод информации о каждом событии может стать проблемой. Формирование результатов в ядре с помощью BPF может значительно снизить эти издержки.

9.2.4. Логирование SCSI

Linux также имеет встроенное средство логирования событий SCSI. Его можно включить через sysctl(8) или /proc. Например, обе следующие команды устанавливают максимальный уровень логирования — регистрацию событий всех типов (будьте осторожны: при большой нагрузке на диск это может вызвать переполнение системного журнала):

```
# sysctl -w dev.scsi.logging_level=0x1b6db6db
# echo 0x1b6db6db > /proc/sys/dev/scsi/logging_level
```

Число в команде — это битовая маска, устанавливающая уровень логирования от 1 до 7 для 10 разных типов событий. Формат числа определен в заголовке drivers/scsi/scsi_logging.h. Пакет sg3-utils включает инструмент scsi_logging_level(8) для настройки этих уровней. Например:

```
scsi_logging_level -s --all 3
```

Примеры событий:

```
# dmesg
[...]
[542136.259412] sd 0:0:0:0: tag#0 Send: scmd 0x00000000001fb89dc
[542136.259422] sd 0:0:0:0: tag#0 CDB: Test Unit Ready 00 00 00 00 00 00
[542136.261103] sd 0:0:0:0: tag#0 Done: SUCCESS Result: hostbyte=DID_OK
driverbyte=DRIVER_OK
[542136.261110] sd 0:0:0:0: tag#0 CDB: Test Unit Ready 00 00 00 00 00 00
[542136.261115] sd 0:0:0:0: tag#0 Sense Key : Not Ready [current]
[542136.261121] sd 0:0:0:0: tag#0 Add. Sense: Medium not present
[542136.261127] sd 0:0:0:0: tag#0 0 sectors total, 0 bytes done.
[...]
```

Эти записи в журнале можно использовать для отладки ошибок и тайм-аутов. Несмотря на наличие отметок времени (первый столбец), их сложно применять для расчета задержек ввода/вывода без наличия уникальных идентификационных данных.

Для создания нестандартных уровней логирования SCSI и регистрации других событий стека ввода/вывода, с более подробной информацией об операциях, включая задержку, вычисленную в ядре, можно использовать механизм трассировки BPF.

9.3. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа дискового ввода/вывода и устранения неполадок. Они показаны на рис. 9.3.

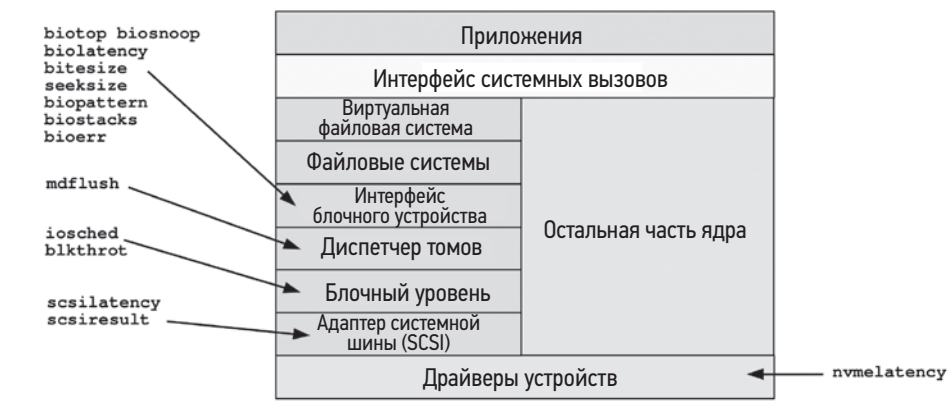


Рис. 9.3. Инструменты BPF для анализа дискового ввода/вывода

Часть этих инструментов можно найти в репозиториях ВСС и bpfftrace, упоминавшихся в главах 4 и 5, а часть была создана специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях, ВСС и bpfftrace. Рассматриваемые здесь инструменты перечислены в табл. 9.2, где также указано их происхождение (ВТ — это сокращение от «bpfftrace»).

Таблица 9.2. Инструменты, связанные с дисками

Инструмент	Источник	Цель	Описание
biolateness	ВСС/ВТ	Блочный ввод/вывод	Обобщает задержки блочного ввода/вывода в виде гистограммы
biosnoop	ВСС/ВТ	Блочный ввод/вывод	Трассирует блочный ввод/вывод и задержки по процессам
biotop	ВСС	Блочный ввод/вывод	Аналог команды top для дисков: обобщает характеристики блочного ввода/вывода по процессам
bitesize	ВСС/ВТ	Блочный ввод/вывод	Отображает гистограмму объемов блочного ввода/вывода по процессам
seeksize	книга	Блочный ввод/вывод	Отображает запрашиваемые расстояния для перемещения указателей
biopattern	книга	Блочный ввод/вывод	Идентифицирует закономерности случайных и последовательных обращений к диску
biostacks	книга	Блочный ввод/вывод	Отображает дисковый ввод/вывод со стеками инициализации

Инструмент	Источник	Цель	Описание
bioerr	книга	Блочный ввод/вывод	Трассирует дисковые ошибки
mdflush	BCC/БТ	Драйверы групп устройств	Трассировка запросов на выталкивание (flush request) в драйверах для групп устройств
iosched	книга	Планирование ввода/вывода	Обобщает информацию о задержках из планировщика ввода/вывода
scsilatency	книга	SCSI	Отображает распределение задержек при выполнении команд SCSI
scsiresult	книга	SCSI	Отображает коды завершения команд SCSI
nvmelateny	книга	NVME	Обобщает задержки выполнения команд в драйвере NVME (твердотельного накопителя)

Актуальные списки параметров инструментов BCC и bpftrace и описание их возможностей ищите в соответствующих репозиториях. Далее я расскажу только о наиболее важных особенностях. Читайте также главу 8, где описаны инструменты анализа файловых систем.

9.3.1. biolatency

biolatency(8)¹ — это инструмент для BCC и bpftrace, отображающий задержки в блочных устройствах ввода/вывода в виде гистограммы. Под «задержкой в блочных устройствах» здесь понимается время между передачей запроса устройству и завершением его выполнения, включая время, проведенное в очереди в ОС.

Вот пример трассировки блочного ввода/вывода в течение 10 секунд с помощью BCC-версии biolatency(8) на производственном экземпляре Hadoop:

```
# biolatency 10 1
```

```
Tracing block device I/O... Hit Ctrl-C to end.
```

```

usecs          : count  distribution
  0 -> 1       : 0      |
  2 -> 3       : 0      |
  4 -> 7       : 0      |
  8 -> 15      : 0      |
 16 -> 31      : 0      |
 32 -> 63      : 0      |

```

¹ Немного истории: первоначально я создал этот инструмент iolatency.d для книги о DTrace в 2011 году [Gregg 11], выбрав для него имя по аналогии с другими моими инструментами: iosnoop и iotop. Это привело к путанице из-за неоднозначной интерпретации приставки «io», поэтому в имя BPF-версии инструмента я добавил букву «b», обозначающую блочный (block) ввод/вывод. Версию biolatency для BCC я создал 20 сентября 2015 года, а версию для bpftrace — 13 сентября 2018 года.

64 -> 127	:	15			
128 -> 255	:	4475		*****	
256 -> 511	:	14222		*****	
512 -> 1023	:	12303		*****	
1024 -> 2047	:	5649		*****	
2048 -> 4095	:	995		**	
4096 -> 8191	:	1980		*****	
8192 -> 16383	:	3681		*****	
16384 -> 32767	:	1895		*****	
32768 -> 65535	:	721		**	
65536 -> 131071	:	394		*	
131072 -> 262143	:	65			
262144 -> 524287	:	17			

Здесь видно бимодальное распределение, охватывающее диапазоны между 128 и 2047 микросекундами и между 4 и 32 миллисекундами. Зная, что распределение задержек в устройстве имеет бимодальный характер, можно выяснить, в каких случаях операции ввода/вывода выполняются быстрее. Например, случайный ввод/вывод или ввод/вывод большого объема данных (который можно определить с помощью других инструментов BPF) может длиться дольше. Самые медленные операции ввода/вывода в этом примере выполнялись от 262 до 524 миллисекунд: похоже, что на устройстве образовалась очень длинная очередь.

Инструмент `biolatility(8)` и появившийся позднее `biosnoop(8)` применяются для решения многих проблем. Они особенно полезны для анализа операций с дисками в облачных средах, которые подвержены влиянию множества факторов и могут нарушать требования к уровню обслуживания. При работе с небольшими облачными экземплярами команда Netflix Cloud Database смогла, воспользовавшись `biolatility(8)` и `biosnoop(8)`, определить машины с недопустимо бимодальными или медленными дисками и убрать их из уровней распределенного кэширования и распределенных баз данных. Опираясь на результаты дальнейшего анализа, команда решила изменить стратегию развертывания и теперь развертывает кластеры на меньшем количестве узлов, достаточно больших, чтобы иметь выделенные диски. Это небольшое изменение помогло избавиться от аномальных задержек без дополнительных затрат на инфраструктуру.

Сейчас `biolatility(8)` производит трассировку различных функций блочного ввода/вывода в ядре с помощью `kprobes`. Он был написан до появления поддержки точек трассировки в ВСС, поэтому вместо них используются `kprobes`. Оверхед этого инструмента должен быть незначительным в большинстве систем, где частота операций ввода/вывода невысока (<1000).

Время в очереди

Версия `biolatility(8)` для ВСС имеет параметр `-Q`, предназначенный для анализа времени, проведенного запросами в очереди ОС:

```
# biolatility -Q 10 1
Tracing block device I/O... Hit Ctrl-C to end.
```


usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 0	
32 -> 63	: 0	
64 -> 127	: 1	
128 -> 255	: 2780	*****
256 -> 511	: 10386	*****
512 -> 1023	: 8399	*****
1024 -> 2047	: 4154	*****
2048 -> 4095	: 1074	****
4096 -> 8191	: 2078	*****
8192 -> 16383	: 7688	*****
16384 -> 32767	: 4111	*****
32768 -> 65535	: 818	***
65536 -> 131071	: 220	
131072 -> 262143	: 103	
262144 -> 524287	: 48	
524288 -> 1048575	: 6	

Эта гистограмма не сильно отличается от предыдущей, разве что в более медленную моду попало больше операций ввода/вывода. Вывод `iostat(1)` подтверждает, что очередь имеет малую длину (`avgqu-sz < 1`).

Диски

Системы могут иметь смешанные устройства хранения: диски для ОС, диски для хранилищ данных и съемные носители. Выполнив `biolatency(8)` с параметром `-D`, можно получить гистограммы для дисков по отдельности. Это поможет более точно проанализировать работу дисков каждого типа. Например:

```
# biolatency -D
```

```
Tracing block device I/O... Hit Ctrl-C to end.
```

```
^C
```

```
[...]
```

```
disk = 'sdb'
```

usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 0	
32 -> 63	: 0	
64 -> 127	: 0	
128 -> 255	: 1	
256 -> 511	: 25	**
512 -> 1023	: 43	****
1024 -> 2047	: 206	*****
2048 -> 4095	: 8	
4096 -> 8191	: 8	
8192 -> 16383	: 392	*****

```

disk = 'nvme0n1'
      usecs          : count    distribution
      0 -> 1         : 0        |
      2 -> 3         : 0        |
      4 -> 7         : 0        |
      8 -> 15        : 12       |
      16 -> 31       : 72       |
      32 -> 63       : 5980     | *****
      64 -> 127      : 1240     | *****
      128 -> 255     : 74       |
      256 -> 511     : 13       |
      512 -> 1023    : 4        |
      1024 -> 2047   : 23       |
      2048 -> 4095   : 10       |
      4096 -> 8191   : 63       |

```

Эти гистограммы соответствуют двум очень разным дисковым устройствам: `nvme0n1`, SSD-дysку на основе флеш-памяти с типичной задержкой ввода/вывода от 32 до 127 микросекунд, и `sdb`, внешнему USB-дysку с бимодальным распределением задержки ввода/вывода, исчисляемой миллисекундами.

Флаги

Версия `biolatency(8)` для ВСС имеет параметр `-F` для вывода каждого набора флагов ввода/вывода по отдельности. Вот пример вывода гистограммы с дополнительным параметром `-m` для отображения времени в миллисекундах:

```
# biolatency -Fm
```

```
Tracing block device I/O... Hit Ctrl-C to end.
```

```
^C
```

```
[...]
```

```

flags = Read
      msecs          : count    distribution
      0 -> 1         : 180      | *****
      2 -> 3         : 519      | *****
      4 -> 7         : 60       | ****
      8 -> 15        : 123      | *****
      16 -> 31       : 68       | *****
      32 -> 63       : 0        |
      64 -> 127      : 2        |
      128 -> 255     : 12       |
      256 -> 511     : 0        |
      512 -> 1023    : 1        |

```

```

flags = Sync-Write
      msecs          : count    distribution
      0 -> 1         : 8        | ***
      2 -> 3         : 26       | *****
      4 -> 7         : 37       | *****
      8 -> 15        : 65       | *****

```

```

16 -> 31      : 93      | ***** |
32 -> 63      : 20      | ***** |
64 -> 127     : 6       | **      |
128 -> 255    : 0       |         |
256 -> 511    : 4       | *       |
512 -> 1023   : 17      | ***** |

flags = Flush
msecs      : count  distribution
0 -> 1      : 2      | ***** |

flags = Metadata-Read
msecs      : count  distribution
0 -> 1      : 3      | ***** |
2 -> 3      : 2      | ***** |
4 -> 7      : 0      |         |
8 -> 15     : 1      | ***** |
16 -> 31    : 1      | ***** |

```

Эти флаги могут обрабатываться устройствами хранения по-разному: их разделение помогает исследовать их по отдельности. Гистограммы выше показывают, что распределение времени синхронной записи имеет бимодальный характер с более медленной модой в диапазоне от 512 до 1023 миллисекунд.

Эти флаги также можно наблюдать в точках трассировки блочного ввода/вывода, в поле `rwbs`, в виде однобуквенных кодов: описание этого поля ищите в разделе «`rwbs`» выше в этой главе.

BCC

Порядок использования:

```
biolatency [options] [interval [count]]
```

Параметры `options`:

- **-m**: выводить время в миллисекундах (по умолчанию выводится в микросекундах);
- **-Q**: включать время, проведенное в очередях ОС;
- **-D**: показывать каждый диск отдельно;
- **-F**: показывать каждый набор флагов ввода/вывода отдельно;
- **-T**: включать в вывод отметки времени.

При передаче в параметре `interval` значения 1 утилита будет выводить посекундные гистограммы. Эту информацию можно визуализировать в виде тепловой карты задержек, на которой ось X соответствует времени, ось Y — диапазонам задержек, а насыщенность цвета отражает количество операций ввода/вывода в этом диапазоне времени [Gregg 10]. Пример создания такой карты в Vector можно увидеть в главе 17.

bpfttrace

Ниже приведена реализация версии для bpfttrace, в которой обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing block device I/O... Hit Ctrl-C to end.\n");
}

kprobe:blk_account_io_start
{
    @start[arg0] = nsecs;
}

kprobe:blk_account_io_done
/@start[arg0]/
{
    @usecs = hist((nsecs - @start[arg0]) / 1000);
    delete(@start[arg0]);
}

END
{
    clear(@start);
}
```

Этот инструмент должен сохранять отметку времени в начале каждой операции ввода/вывода, чтобы вычислить ее продолжительность (задержку). Но иногда одновременно может запускаться несколько операций ввода/вывода. Поэтому единственной глобальной переменной для хранения отметки времени недостаточно: отметка времени должна быть связана с каждой операцией ввода/вывода. Во многих других инструментах BPF эта проблема решается сохранением отметок времени в хеше с идентификатором потока в качестве ключа. Но для дискового ввода/вывода это решение не подходит, так как операция может инициироваться в одном потоке, а завершаться в другом — в этом случае идентификатор потока изменится. Реализованное здесь решение основано на использовании аргумента `arg0` функций, который является адресом структуры `request`, в качестве хеш-ключа. Пока ядро не будет перемещать структуру `request` в памяти между передачей запроса и завершением его обработки, ее адрес можно использовать в качестве уникального идентификатора.

Точки трассировки

Версии `biolatency(8)` для BCC и bpfttrace должны по возможности использовать точки трассировки блочного ввода/вывода. Однако есть проблема: указатель на структуру `request` сейчас недоступен в аргументах точки трассировки, поэтому для

однозначной идентификации ввода/вывода нужно применять другой ключ. Одно из решений — использовать идентификатор устройства и номер сектора. Ядро программы можно изменить, как показано ниже (biolatency-tp.bt):

```
[...]
tracepoint:block:block_rq_issue
{
    @start[args->dev, args->sector] = nsecs;
}

tracepoint:block:block_rq_complete
/@start[args->dev, args->sector]/
{
    @usecs = hist((nsecs - @start[args->dev, args->sector]) / 1000);
    delete(@start[args->dev, args->sector]);
}
[...]
```

Это решение предполагает невозможность одновременного выполнения нескольких операций ввода/вывода с одним устройством и одним сектором. Приведенный код измеряет время выполнения запроса устройством, то есть без учета нахождения запросов в очереди ОС.

9.3.2. biosnoop

biosnoop(8)¹ — это инструмент для BCC и bpftrace, который выводит короткие однострочные сводки для каждой операции дискового ввода/вывода. Ниже показан пример использования версии biosnoop(8) для BCC на производственном экземпляре Nadoor:

¹ Немного истории: когда в 2000 году я работал системным администратором в Университете Ньюкасла в Австралии, файловый сервер страдал от низкой производительности диска, которая предположительно была обусловлена действиями исследователей, запускавших пакетные задания. Они согласились перенести свои рабочие нагрузки, только если я докажу, что те вызывают массивный поток операций дискового ввода/вывода, но ни один инструмент не мог помочь мне в этом. Тогда я и старший администратор Дуг Скотт придумали обходное решение: послать сигнал SIGSTOP процессам, выполняющим пакетные задания, в ходе просмотра результатов iostat(1), а затем спустя несколько секунд послать им сигнал SIGCONT. Резкое падение числа операций дискового ввода/вывода в этот период однозначно указывало на причину. Пытаясь найти более щадящий метод, я обнаружил утилиту трассировки Sun TNF/prxh в книге Адриана Кокрофта «Sun Performance and Tuning» [Cockcroft 98]. А 3 декабря 2003 года я создал psio(1M) — утилиту для вывода информации об операциях дискового ввода/вывода для процессов по отдельности [185], которая также имела режим трассировки отдельных событий дискового ввода/вывода. В том же месяце появилась бета-версия DTrace, и 12 марта 2004 года я переписал свой трассировщик дискового ввода/вывода, переименовав его в iosnoop(1M). Эта моя работа была процитирована в объявлении «DTrace The Register» [Vance 04]. Версию для BCC под названием biosnoop(8) я написал 16 сентября 2015 года, а версию для bpftrace — 15 ноября 2017 года.

```
# biosnoop
TIME(s)    COMM      PID    DISK    T SECTOR    BYTES    LAT(ms)
0.000000   java      5136   xvdq    R 980043184  45056    0.35
0.000060   java      5136   xvdq    R 980043272  45056    0.40
0.000083   java      5136   xvdq    R 980043360  4096     0.42
[...]
0.143724   java      5136   xvdy    R 5153784    45056    1.08
0.143755   java      5136   xvdy    R 5153872    40960    1.10
0.185374   java      5136   xvdm    R 2007186664 45056    0.34
0.189267   java      5136   xvdy    R 979232832  45056    14.00
0.190330   java      5136   xvdy    R 979232920  45056    15.05
0.190376   java      5136   xvdy    R 979233008  45056    15.09
0.190403   java      5136   xvdy    R 979233096  45056    15.12
0.190409   java      5136   xvdy    R 979233184  45056    15.12
0.190441   java      5136   xvdy    R 979233272  36864    15.15
0.190176   java      5136   xvdm    R 2007186752 45056    5.13
0.190231   java      5136   xvdm    R 2007186840 45056    5.18
[...]
```

Здесь видно, что процесс Java с PID 5136 выполняет чтение с разных дисков. Из них шесть операций чтения выполнено с задержкой около 15 миллисекунд. Внимательно рассмотрев столбец TIME(s), сообщающий время завершения операции, можно заметить, что все они завершились за доли миллисекунды и выполнялись на одном диске (xvdy). Отсюда можно сделать вывод, что эти шесть операций были поставлены в очередь почти одновременно: задержка, возрастающая с 14.00 до 15.15 миллисекунды, тоже подсказывает, что операции из очереди выполнялись по порядку. Кроме того, смещения секторов образуют непрерывную последовательность: 45 056 байт — это 88 секторов по 512 байт.

Реальный пример: команды в Netflix, сопровождающие stateful-сервисы, часто используют biosnoop(8) для выявления проблем с упреждающим чтением, которое влечет снижение производительности рабочих нагрузок с интенсивным вводом/выводом. Linux использует опережающее чтение для сохранения данных в кэше страниц ОС, но это может вызвать серьезные проблемы с производительностью хранилищ данных, действующих на быстрых твердотельных накопителях, особенно с настройками упреждающего чтения по умолчанию. После выявления фактов агрессивного упреждающего чтения эти команды целенаправленно анализируют гистограммы объемов ввода/вывода и задержек по потокам выполнения. Затем они устраняют проблемы производительности, используя соответствующие параметры, прямой ввод/вывод или уменьшая объем опережающего чтения по умолчанию, например, до 16 Кбайт. Гистограммы объемов ввода/вывода можно получить с помощью vfssize(8) и bitesize(8), как описано в главе 8. Также обратите внимание на инструмент readahead(8) из главы 8, который был создан совсем недавно для анализа таких проблем.

Столбцы в выводе biosnoop(8):

- **TIME(s)**: время выполнения операции в секундах с момента запуска biosnoop;
- **COMM**: имя процесса, если было кэшировано;

- **PID**: идентификатор процесса, если был кэширован;
- **DISK**: имя устройства хранения;
- **T**: тип операции: R == read (чтение), W == write (запись);
- **SECTOR**: адрес в дисковых единицах измерения — 512-байтных секторах;
- **BYTES**: объем ввода/вывода;
- **LAT(ms)**: продолжительность обработки запроса от передачи в устройство до завершения.

Этот инструмент действует как и `biolatency(8)`: трассирует функции блочного ввода/вывода в ядре. В будущей версии предполагается перейти на использование точек трассировки блочного ввода/вывода. Оверхед этого инструмента немного выше, чем `biolatency(8)` из-за того, что он выводит данные для каждого события.

Время в очереди ОС

Для определения времени между созданием запроса на выполнение операции ввода/вывода и его передачи в устройство можно использовать ВСС-версию `biosnoop(8)` с параметром `-Q`: большую часть этого времени запрос проводит в очереди ОС, но в него также могут включаться затраты времени на выделение памяти и получение блокировки. Например:

```
# biosnoop -Q
TIME(s)    COMM      PID    DISK    T SECTOR    BYTES    QUE(ms)    LAT(ms)
19.925329   cksum      20405   sdb     R 249631    16384    17.17      1.63
19.933890   cksum      20405   sdb     R 249663    122880   17.81      8.51
19.942442   cksum      20405   sdb     R 249903    122880   26.35      8.51
19.944161   cksum      20405   sdb     R 250143    16384    34.91      1.66
19.952853   cksum      20405   sdb     R 250175    122880   15.53      8.59
[...]
```

Время в очереди выводится в столбце `QUE(ms)`. Этот пример долгого ожидания запросов на чтение в очереди был получен для флеш-накопителя USB, который обслуживается планировщиком ввода/вывода CFQ. Запросы на запись проводят в очереди еще больше времени:

```
# biosnoop -Q
TIME(s)    COMM      PID    DISK    T SECTOR    BYTES    QUE(ms)    LAT(ms)
[...]
```

2.338149	?	0		W 0	8192	0.00	2.72
2.354710	?	0		W 0	122880	0.00	16.17
2.371236	kworker/u16:1	18754	sdb	W 486703	122880	2070.06	16.51
2.387687	cp	20631	nvme0n1	R 73365192	262144	0.01	3.23
2.389213	kworker/u16:1	18754	sdb	W 486943	122880	2086.60	17.92
2.404042	kworker/u16:1	18754	sdb	W 487183	122880	2104.53	14.81
2.421539	kworker/u16:1	18754	sdb	W 487423	122880	2119.40	17.43

```
[...]
```

Время ожидания для операций записи превышает 2 секунды. Обратите внимание, что в некоторых столбцах нет информации. Соответствующие запросы были поставлены в очередь до начала трассировки, поэтому biosnoop(8) не кэшировал эти сведения и показал только задержку на устройстве.

BCC

Порядок использования:

```
biosnoop [options]
```

В качестве параметра *options* можно передать *-Q*, и тогда инструмент будет выводить время, проведенное запросом в очереди.

bpfttrace

Ниже приведена реализация версии для bpfttrace, которая определяет полную продолжительность выполнения операции ввода/вывода, включая время, проведенное в очереди:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("%-12s %-16s %-6s %7s\n", "TIME(ms)", "COMM", "PID", "LAT(ms)");
}

kprobe:blk_account_io_start
{
    @start[arg0] = nsecs;
    @iopid[arg0] = pid;
    @iocomm[arg0] = comm;
}

kprobe:blk_account_io_done
/@start[arg0] != 0 && @iopid[arg0] != 0 && @iocomm[arg0] != ""/
{
    $now = nsecs;
    printf("%-12u %-16s %-6d %7d\n",
        elapsed / 1000000, @iocomm[arg0], @iopid[arg0],
        ($now - @start[arg0]) / 1000000);

    delete(@start[arg0]);
    delete(@iopid[arg0]);
    delete(@iocomm[arg0]);
}

END
{
    clear(@start);
    clear(@iopid);
    clear(@iocomm);
}
```


Функция `blk_account_io_start()` часто запускается в контексте процесса, когда запрос на выполнение операции ввода/вывода помещается в очередь. Более поздние события — передача запроса в устройство и завершение его обработки — могут происходить вне контекста процесса, поэтому нельзя полагаться на значения `pid` и `comm` в более поздние моменты времени. Эту проблему можно решить, сохраняя эти значения во время вызова `blk_account_io_start()` в картах BPF с идентификатором запроса в качестве ключа.

Как и `biolatency(8)`, этот инструмент можно переписать с применением точек трассировки блочного ввода/вывода (см. раздел 9.5).

9.3.3. `biotop`

`biotop(8)`¹ — это BCC-версия утилиты `top(1)` для дисков. Ниже показан пример выполнения этой утилиты на производственном экземпляре Nadoor с параметром `-C` для предотвращения очистки экрана между обновлениями:

```
# biotop -C
Tracing... Output every 1 secs. Hit Ctrl-C to end
06:09:47 loadavg: 28.40 29.00 28.96 44/3812 124008
PID  COMM      D MAJ MIN  DISK      I/O  Kbytes  AVGms
123693 kworker/u258:0 W 202 4096 xvdq      1979  86148  0.93
55024  kworker/u257:8 W 202 4608 xvds     1480  64068  0.73
123693 kworker/u258:0 W 202 5376 xvdv      143   5700  0.52
5381   java       R 202 176  xvd1       81   3456  3.01
43297  kworker/u257:0 W 202 80   xvdf       48   1996  0.56
5383   java       R 202 112  xvdh       27   1152 16.05
5383   java       R 202 5632 xvdw       27   1152  3.45
5383   java       R 202 224  xvdo       27   1152  6.79
5383   java       R 202 96   xvdg       24   1024  0.52
5383   java       R 202 192  xvdm       24   1024 39.45
5383   java       R 202 5888 xvdx       24   1024  0.64
5383   java       R 202 5376 xvdv       24   1024  4.74
5383   java       R 202 4096 xvdq       24   1024  3.07
5383   java       R 202 48   xvdd       24   1024  0.62
5383   java       R 202 5120 xvdu       24   1024  4.20
5383   java       R 202 208  xvdn       24   1024  2.54
5383   java       R 202 80   xvdf       24   1024  0.66
5383   java       R 202 64   xvde       24   1024  8.08
5383   java       R 202 32   xvdc       24   1024  0.63
5383   java       R 202 160  xvdk       24   1024  1.42
[...]
```

Мы видим, как процесс Java выполняет чтение с нескольких разных дисков. Вверху списка находятся потоки `kworker`, инициирующие запись: это фоновое выталкивание буферов на диск, а реальный процесс, который изменил страницы в кэше, сейчас

¹ Немного истории: первую версию `iotop` на основе DTrace я создал 15 июля 2005 года, а эту версию для BCC — 6 февраля 2016 года. Пробразом послужила утилита `top(1)` Уильяма Лефевра.

неизвестен (его можно определить с помощью инструментов анализа файловых систем из главы 8).

Этот инструмент использует те же события, что и `biolatency(8)`, и имеет аналогичный переход.

Порядок использования:

```
biotop [options] [interval [count]]
```

Параметры `options`:

- `-C`: не очищать экран;
- `-r ROWS`: число строк для вывода.

По умолчанию выводится не более 20 строк, но этот предел можно изменить с помощью параметра `-r`.

9.3.4. `bitesize`

`bitesize(8)`¹ — это инструмент для ВСС и `bpfttrace`, отображающий объемы данных, вовлеченные в операции дискового ввода/вывода. Вот пример использования версии на производственном экземпляре Hadoop:

```
# bitesize
```

```
Tracing... Hit Ctrl-C to end.
```

```
^C
```

```
[...]
```

```
Process Name = kworker/u257:10
```

Kbytes	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 17	
8 -> 15	: 12	
16 -> 31	: 79	*
32 -> 63	: 3140	*****

```
Process Name = java
```

Kbytes	: count	distribution
0 -> 1	: 0	
2 -> 3	: 3	
4 -> 7	: 60	
8 -> 15	: 68	
16 -> 31	: 220	**
32 -> 63	: 3996	*****

¹ Немного истории: первую версию этого инструмента под названием `bitesize.d` я написал на основе `DTtrace` 31 марта 2004 года, еще до появления провайдера `io`. Версию для ВСС создал Аллан Макаливи 5 февраля 2016 года, а версию для `bpfttrace` создал я 7 сентября 2018 года.

Мы видим, что поток `kworker` и процесс `java` выполняют ввод/вывод данных преимущественно блоками размером в диапазоне от 32 до 63 Кбайт. Проверка объемов данных ввода/вывода может привести к следующим оптимизациям:

- В последовательных рабочих нагрузках можно попробовать увеличить размер блоков данных для ввода/вывода, чтобы добиться максимальной производительности. Иногда с увеличением размера производительность ввода/вывода несколько снижается, поэтому необходимо найти золотую середину (например, 128 Кбайт) для этой комбинации механизмов распределения памяти и логики устройства.
- В случайных рабочих нагрузках можно попробовать поставить в соответствие размеры блоков ввода/вывода с объемом записи приложения. Большие блоки ввода/вывода будут забивать кэш страниц ненужными данными, маленькие приведут к увеличению оверхеда.

Принцип действия этой утилиты основан на инструментации точки трассировки `block:block_rq_issue`.

BCC

Сейчас версия `bitesize(8)` для BCC не имеет параметров.

bpfftrace

Ниже приведен исходный код версии для `bpfftrace`:

```
#!/usr/local/bin/bpfftrace

BEGIN
{
    printf("Tracing block device I/O... Hit Ctrl-C to end.\n");
}

tracepoint:block:block_rq_issue
{
    @[args->comm] = hist(args->bytes);
}

END
{
    printf("\nI/O size (bytes) histograms by process name:");
}
```

Эта точка трассировки предоставляет имя процесса как `args->comm` и объем данных как `args->bytes`. Она срабатывает, когда запрос добавляется в очередь ОС. Точки трассировки, срабатывающие позднее, такие как `block_rq_complete`, не предоставляют `args->comm` и не позволяют использовать встроенную команду `comm`, так как выполняются асинхронно с процессом, инициировавшим ввод/вывод (например, по прерываниям, генерируемым устройствами по завершении обработки).

упорядочена планировщиком ввода/вывода в ядре Linux и планировщиком в устройстве. Этот инструмент я создавал прежде всего для поиска приложений, вызывающих случайные рабочие нагрузки, поэтому использовал точку трассировки `block_rq_issue`.

Следующий инструмент — `biopattern(8)` — оценивает случайность по событиям завершения запросов.

9.3.6. biopattern

`biopattern(8)`¹ — это инструмент для `bpfttrace`, выявляющий закономерность в последовательности операций ввода/вывода: случайная или неслучайная. Например:

```
# biopattern.bt
Attaching 4 probes...
TIME      %RND  %SEQ  COUNT  KBYTES
00:05:54   83    16   2960    13312
00:05:55   82    17   3881    15524
00:05:56   78    21   3059    12232
00:05:57   73    26   2770    14204
00:05:58    0   100     1         0
00:05:59    0    0     0         0
00:06:00    0    99   1536   196360
00:06:01    0   100  13444  1720704
00:06:02    0    99  13864  1771876
00:06:03    0   100  13129  1680640
00:06:04    0    99  13532  1731484
[...]
```

В начале этого примера показано поведение рабочей нагрузки — резервного копирования файловой системы, для которой характерны случайные операции ввода/вывода. В 6:00 я запустил последовательное чтение с диска, которое, по оценке инструмента, на 99% или 100% выполнялось последовательно, что обеспечило гораздо более высокую пропускную способность (см. столбец `KBYTES`).

Исходный код `biopattern(8)`:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("%-8s %5s %5s %8s %10s\n", "TIME", "%RND", "%SEQ", "COUNT",
        "KBYTES");
}

tracepoint:block:block_rq_complete
{
```

¹ Немного истории: первую версию под названием `iopattern` я создал на основе `DTrace` 25 июля 2005 года, используя модель, предложенную Райаном Маттесоном (Ryan Matteson), где было больше столбцов. Версию для `bpfttrace` я создал специально для этой книги 19 марта 2019 года.

```

    if (@lastsector[args->dev] == args->sector) {
        @sequential++;
    } else {
        @random++;
    }
    @bytes = @bytes + args->nr_sector * 512;
    @lastsector[args->dev] = args->sector + args->nr_sector;
}

interval:s:1
{
    $count = @random + @sequential;
    $div = $count;
    if ($div == 0) {
        $div = 1;
    }
    time("%H:%M:%S ");
    printf("%5d %5d %8d %10d\n", @random * 100 / $div,
        @sequential * 100 / $div, $count, @bytes / 1024);
    clear(@random); clear(@sequential); clear(@bytes);
}

END
{
    clear(@lastsector);
    clear(@random); clear(@sequential); clear(@bytes);
}

```

Этот инструмент использует точку трассировки `block_rq_complete` и запоминает последний использовавшийся сектор (адрес на диске) для каждого устройства, который затем сравнивается с начальным сектором следующей операции ввода/вывода, чтобы оценить, является ли она продолжением предыдущей (последовательной) или нет (случайной)¹.

В этом инструменте можно заменить `tracepoint:block:block_rq_complete` на `tracepoint:block:block_rq_insert`, чтобы оценить случайность применяемой рабочей нагрузки (по аналогии с `seeksize(8)`).

9.3.7. biostacks

`biostacks(8)`² — это инструмент для `bpfftrace`, который определяет полную задержку ввода/вывода (от постановки в очередь ОС до завершения обработки устройством) по трассировке стека инициализации ввода/вывода. Например:

¹ До эры трассировки я бы идентифицировал случайные/последовательные рабочие нагрузки, интерпретируя вывод `iostat(1)` и отыскивая в нем большое время обслуживания с малым объемом ввода/вывода (случайные операции) или малое время обслуживания с большим объемом ввода/вывода (последовательные операции).

² Немного истории: я создал этот инструмент специально для этой книги 19 марта 2019 года. Аналогичный инструмент я написал прямо в переписке в Facebook в 2018 году и тогда же впервые увидел стеки инициализации, связанные с временем завершения ввода/вывода.

Этот инструмент сохраняет стек ядра и отметку времени, когда была запущена операция ввода/вывода, а затем извлекает этот сохраненный стек с отметкой времени по завершении ввода/вывода. Информация сохраняется в карте с ключом, роль которого играет указатель на структуру `request`, передаваемый функциям ядра в `arg0`. Трассировка стека ядра записывается встроенной функцией `kstack`. Ее можно заменить на `ustack`, чтобы сохранять трассировку стека в пространстве пользователя, или сохранять оба стека.

После перехода на использование планировщиков с несколькими очередями в Linux 5.0 функция `blk_start_request()` была удалена из ядра. Поэтому в этой и последующих версиях ядра этот инструмент выводит предупреждение:

```
Warning: could not attach probe kprobe:blk_start_request, skipping.1
```

Его можно игнорировать или вообще удалить `kprobe` из инструмента. Инструмент можно переписать и использовать в нем точки трассировки. См. подраздел «Точки трассировки» в разделе 9.3.1.

9.3.8. bioerr

`bioerr(8)`² трассирует ошибки блочного ввода/вывода и выводит подробную информацию о них. Вот пример выполнения `bioerr(8)` на моем ноутбуке:

```
# bioerr.bt
Attaching 2 probes...
Tracing block I/O errors. Hit Ctrl-C to end.
00:31:52 device: 0,0, sector: -1, bytes: 0, flags: N, error: -5
00:31:54 device: 0,0, sector: -1, bytes: 0, flags: N, error: -5
00:31:56 device: 0,0, sector: -1, bytes: 0, flags: N, error: -5
00:31:58 device: 0,0, sector: -1, bytes: 0, flags: N, error: -5
00:32:00 device: 0,0, sector: -1, bytes: 0, flags: N, error: -5
[...]
```

Полученный результат оказался намного интереснее, чем я ожидал. (Я не ожидал никаких ошибок и запустил `bioerr(8)` на всякий случай.) Каждые 2 секунды в устройство 0,0 поступает запрос на ввод/вывод 0 байт, который выглядит поддельным и завершается с ошибкой -5 (EIO).

Для исследования подобных проблем был создан предыдущий инструмент, `biostacks(8)`. Здесь мне не требовалось исследовать задержки, я хотел лишь увидеть трассировки стеков операций ввода/вывода с устройством 0,0. Для этого можно скорректировать исходный код `biostacks(8)` или написать однострочник

¹ Внимание: невозможно подключиться к зонду `kprobe:blk_start_request`, пропущено. — *Примеч. пер.*

² Немного истории: я создал его для этой книги 19 марта 2019 года.

для `bpfftrace`. В последнем случае нужно проверить, что трассировка стека все еще значима в момент срабатывания этой точки трассировки. Если это не так, придется вернуться к зонду ядра `blk_account_io_start()`, чтобы перехватить инициализацию этой операции ввода/вывода:

```
# bpfftrace -e 't:block:block_rq_issue /args->dev == 0/ { @[kstack]++ }'
Attaching 1 probe...
^C
```

```
@[
    blk_peek_request+590
    scsi_request_fn+51
    __blk_run_queue+67
    blk_execute_rq_nowait+168
    blk_execute_rq+80
    scsi_execute+227
    scsi_test_unit_ready+96
    sd_check_events+248
    disk_check_events+101
    disk_events_workfn+22
    process_one_work+478
    worker_thread+50
    kthread+289
    ret_from_fork+53
]: 3
```

Мы видим, что операция ввода/вывода с устройством 0 была запущена из `scsi_test_unit_ready()`. Немного покопавшись в родительских функциях, можно заметить, что эта операция связана с проверкой наличия съемного USB-носителя. Для эксперимента я выполнил трассировку `scsi_test_unit_ready()` после вставки USB-накопителя, что изменило возвращаемое значение. Как оказалось, именно таким способом мой ноутбук обнаруживает USB-накопители.

Исходный код `bioerr(8)`:

```
#!/usr/local/bin/bpfftrace

BEGIN
{
    printf("Tracing block I/O errors. Hit Ctrl-C to end.\n");
}

tracepoint:block:block_rq_complete
/args->error != 0/
{
    time("%H:%M:%S ");
    printf("device: %d,%d, sector: %d, bytes: %d, flags: %s, error: %d\n",
        args->dev >> 20, args->dev & ((1 << 20) - 1), args->sector,
        args->nr_sector * 512, args->rwbs, args->error);
}
```

Логика отображения идентификатора устройства (`args->dev`) в старший и младший номера взята из файла `format` для этой точки трассировки:

```
# cat /sys/kernel/debug/tracing/events/block/block_rq_complete/format
name: block_rq_complete
[...]

print fmt: "%d,%d %s (%s) %llu + %u [%d]", ((unsigned int) ((REC->dev) >> 20)),
((unsigned int) ((REC->dev) & ((1U << 20) - 1))), REC->rwbs, __get_str(cmd), (unsigned
long long)REC->sector, REC->nr_sector, REC->error
```

Конечно, `bioerr(8)` — удобный инструмент, но тот же результат можно получить и с помощью `perf(1)`, реализовав фильтрацию ошибок. Для оформления вывода используется строка формата из файла `format` в `/sys`. Например:

```
# perf record -e block:block_rq_complete --filter 'error != 0'
# perf script
ksoftirqd/2      22 [002] 2289450.691041: block:block_rq_complete: 0,0 N ()
18446744073709551615 + 0 [-5]
[...]
```

Версию для BPF можно настроить так, чтобы она хранила больше информации, чем позволяет `perf(1)`.

Например, возвращаемая ошибка, в данном случае `-5` (EIO), была получена отображением ошибки блочного ввода/вывода. Возможно, будет интересно увидеть исходный код ошибки, который можно отследить по функциям, которые его обрабатывают, например:

```
# bpftrace -e 'kprobe:blk_status_to_errno /arg0/ { @[arg0]++ }'
Attaching 1 probe...
^C
```

```
@[10]: 2
```

В действительности блочная операция ввода/вывода завершилась с кодом 10, то есть `BLK_STS_IOERR`. Этот символ определяется в `linux/blk_types.h`:

```
#define BLK_STS_OK                0
#define BLK_STS_NOTSUPP           ((__force blk_status_t)1)
#define BLK_STS_TIMEOUT           ((__force blk_status_t)2)
#define BLK_STS_NOSPC            ((__force blk_status_t)3)
#define BLK_STS_TRANSPORT        ((__force blk_status_t)4)
#define BLK_STS_TARGET           ((__force blk_status_t)5)
#define BLK_STS_NEXUS            ((__force blk_status_t)6)
#define BLK_STS_MEDIUM           ((__force blk_status_t)7)
#define BLK_STS_PROTECTION       ((__force blk_status_t)8)
#define BLK_STS_RESOURCE         ((__force blk_status_t)9)
#define BLK_STS_IOERR            ((__force blk_status_t)10)
```

В `bioerr(8)` можно добавить вывод этих символов `BLK_STS` вместо кодов ошибок. Они фактически вычисляются на основе кодов SCSI, которые отслеживаются по событиям `scsi`. Примеры трассировки SCSI я приведу в разделах 9.3.11 и 9.3.12.

9.3.9. mdflush

`mdflush(8)`¹ — это инструмент для BCC и `bpfftrace`, трассирующий события выталкивания в драйверах групп устройств (`md`, `multiple devices`), которые используются в некоторых системах для программной организации дисковых массивов RAID. Вот пример запуска BCC-версии инструмента на производственном сервере, использующем групповое устройство `md`:

```
# mdflush
Tracing md flush requests... Hit Ctrl-C to end.
TIME      PID      COMM          DEVICE
23:43:37  333      kworker/0:1H  md0
23:43:37  4038     xfsaild/md0   md0
23:43:38  8751     filebeat      md0
23:43:43  5575     filebeat      md0
23:43:48  5824     filebeat      md0
23:43:53  5575     filebeat      md0
23:43:58  5824     filebeat      md0
[...]
```

События выталкивания в групповых устройствах — нечастое явление, но обычно они вызывают всплески операций записи на диск, снижая производительность системы. Точное знание, когда такие события происходят, пригодится для проверки, совпадают ли они с пиками задержки или другими нежелательными явлениями.

В этом выводе виден процесс `filebeat`, генерирующий события выталкивания для драйвера группового устройства каждые 5 секунд (я сам только что обнаружил это). `filebeat` — это сервис, отправляющий файлы журналов в Logstash или сразу в Elasticsearch.

Этот инструмент трассирует вызов функции `md_flush_request()` с помощью `kprobe`. Поскольку частота следования событий невысока, оверхед должен быть незначительным.

BCC

Сейчас у `mdflush(8)` нет параметров.

bpfftrace

Вот исходный код версии для `bpfftrace`:

```
#!/usr/local/bin/bpfftrace

#include <linux/genhd.h>
```

¹ Немного истории: версию для BCC я создал 13 февраля 2015 года, а версию для `bpfftrace` 8 сентября 2018 года.

```
#include <linux/bio.h>

BEGIN
{
    printf("Tracing md flush events... Hit Ctrl-C to end.\n");
    printf("%-8s %-6s %-16s %s", "TIME", "PID", "COMM", "DEVICE");
}

kprobe:md_flush_request
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %s\n", pid, comm,
           ((struct bio *)arg1)->bi_disk->disk_name);
}
```

Как показывает этот вывод, в рассматриваемой системе используется планировщик CFQ и время ожидания в очереди обычно составляет от 8 до 64 миллисекунд.

Исходный код `iosched(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/blkdev.h>

BEGIN
{
    printf("Tracing block I/O schedulers. Hit Ctrl-C to end.\n");
}

kprobe:__elv_add_request
{
    @start[arg1] = nsecs;
}

kprobe:blk_start_request,
kprobe:blk_mq_start_request
/@start[arg0]/
{
    $r = (struct request *)arg0;
    @usecs[$r->q->elevator->type->elevator_name] =
        hist((nsecs - @start[arg0]) / 1000);
    delete(@start[arg0]);
}

END
{
    clear(@start);
}
```

Этот инструмент запоминает отметку времени, когда запрос передается планировщику ввода/вывода с помощью функции `__elv_add_request()`, а затем вычисляет время в очереди после передачи запроса в устройство. Инструмент трассирует только операции, которые проходят через планировщик ввода/вывода, и только время ожидания в очереди. Имя планировщика извлекается из структуры `request`.

После перехода на использование планировщиков с несколькими очередями в Linux 5.0 функция `blk_start_request()` была удалена из ядра. Поэтому в этой и последующих версиях ядра этот инструмент выводит предупреждение о пропуске зонда в `blk_start_request()`, которое можно игнорировать или вообще удалить `kprobe` из инструмента.

9.3.11. scsilatency

scsilatency(8)¹ — это инструмент для трассировки команд SCSI и получения распределения их задержек. Например:

```
# scsilatency.bt
Attaching 4 probes...
Tracing scsi latency. Hit Ctrl-C to end.
^C

@usecs[0, TEST_UNIT_READY]:
[128K, 256K)      2 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[256K, 512K)      2 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[512K, 1M)        0 | |
[1M, 2M)          1 | @@@@@@@@@@@@@@@@@@@@@@ |
[2M, 4M)          2 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4M, 8M)          3 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8M, 16M)         1 | @@@@@@@@@@@@@@@@@@@@@@ |

@usecs[42, WRITE_10]:
[2K, 4K)          2 | @ |
[4K, 8K)          0 | |
[8K, 16K)         2 | @ |
[16K, 32K)       50 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[32K, 64K)      57 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |

@usecs[40, READ_10]:
[4K, 8K)          15 | @ |
[8K, 16K)        676 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[16K, 32K)      447 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[32K, 64K)       2 | |
[...]
```

Для каждого типа команд SCSI выводится своя гистограмма задержек, сообщающая код операции и имя команды (если доступно).

Исходный код scsilatency(8):

```
#!/usr/local/bin/bpftrace

#include <scsi/scsi_cmnd.h>

BEGIN
{
    printf("Tracing scsi latency. Hit Ctrl-C to end.\n");
```

¹ Немного истории: я создал его специально для этой книги 21 марта 2019 года, вдохновившись аналогичными инструментами, созданными мной для книги о DTrace в 2011 году [Gregg 11].


```

// коды операций SCSI из scsi/scsi_proto.h;
// добавьте преобразование других кодов, если потребуется:
@opcode[0x00] = "TEST_UNIT_READY";
@opcode[0x03] = "REQUEST_SENSE";
@opcode[0x08] = "READ_6";
@opcode[0x0a] = "WRITE_6";
@opcode[0x0b] = "SEEK_6";
@opcode[0x12] = "INQUIRY";
@opcode[0x18] = "ERASE";
@opcode[0x28] = "READ_10";
@opcode[0x2a] = "WRITE_10";
@opcode[0x2b] = "SEEK_10";
@opcode[0x35] = "SYNCHRONIZE_CACHE";
}

kprobe:scsi_init_io
{
    @start[arg0] = nsecs;
}

kprobe:scsi_done,
kprobe:scsi_mq_done
/@start[arg0]/
{
    $cmdnd = (struct scsi_cmnd *)arg0;
    $opcode = *$cmdnd->req.cmd & 0xff;
    @usecs[$opcode, @opcode[$opcode]] = hist((nsecs - @start[arg0]) / 1000);
}

END
{
    clear(@start); clear(@opcode);
}

```

Набор команд SCSI довольно обширен. Этот инструмент преобразует лишь часть кодов операций в имена. Поскольку вместе с именем выводится код операции, то когда нет соответствующего преобразования, все равно можно определить имя, обратившись к `scsi/scsi_proto.h`. Кроме того, сам инструмент можно расширить, добавив необходимые преобразования.

Есть точки трассировки `scsi`, и одна из них используется в следующем инструменте. Однако они не предоставляют уникального идентификатора, который можно было бы взять в качестве ключа карты BPF для хранения отметки времени.

После перехода на применение планировщиков с несколькими очередями в Linux 5.0 функция `scsi_done()` была удалена из ядра и, соответственно, был удален зонд ядра `kprobe:scsi_done`. Поэтому в этой и последующих версиях ядра этот инструмент выводит предупреждение о пропуске инструментации зонда в `scsi_done()`, которое можно игнорировать или вообще удалить `kprobe` из инструмента.

9.3.12. scsiresult

scsiresult(8)¹ обобщает результаты команд SCSI: коды хоста и состояния. Например:

```
# scsiresult.bt
Attaching 3 probes...
Tracing scsi command results. Hit Ctrl-C to end.
^C
```

```
@[DID_BAD_TARGET, SAM_STAT_GOOD]: 1
@[DID_OK, SAM_STAT_CHECK_CONDITION]: 10
@[DID_OK, SAM_STAT_GOOD]: 2202
```

Согласно этому выводу, за время трассировки было получено 2202 результата с кодами DID_OK и SAM_STAT_GOOD и один с DID_BAD_TARGET и SAM_STAT_GOOD. Они определены в исходном коде ядра. Например, вот выдержка из заголовка include/scsi/scsi.h:

```
#define DID_OK                0x00 /* НЕТ ошибки */
#define DID_NO_CONNECT        0x01 /* Не удалось подключиться до истечения тайм-аута */
#define DID_BUS_BUSY          0x02 /* ШИНА оставалась занятой до истечения тайм-аута */
#define DID_TIME_OUT           0x03 /* ТАЙМ-АУТ по какой-то другой причине */
#define DID_BAD_TARGET         0x04 /* НЕВЕРНАЯ цель. */
[...]
```

Этот инструмент можно применять для выявления аномальных результатов на устройствах SCSI.

Исходный код scsiresult(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing scsi command results. Hit Ctrl-C to end.\n");
    // однобайтные коды хостов из include/scsi/scsi.h:
    @host[0x00] = "DID_OK";
    @host[0x01] = "DID_NO_CONNECT";
    @host[0x02] = "DID_BUS_BUSY";
    @host[0x03] = "DID_TIME_OUT";
    @host[0x04] = "DID_BAD_TARGET";
    @host[0x05] = "DID_ABORT";
    @host[0x06] = "DID_PARITY";
    @host[0x07] = "DID_ERROR";
    @host[0x08] = "DID_RESET";
    @host[0x09] = "DID_BAD_INTR";
    @host[0x0a] = "DID_PASSTHROUGH";
    @host[0x0b] = "DID_SOFT_ERROR";
```

¹ Немного истории: я создал его специально для этой книги 21 марта 2019 года, вдохновившись аналогичными инструментами, созданными мной для книги о DTrace в 2011 году [Gregg 11].

```

@host[0x0c] = "DID_IMM_RETRY";
@host[0x0d] = "DID_REQUEUE";
@host[0x0e] = "DID_TRANSPORT_DISRUPTED";
@host[0x0f] = "DID_TRANSPORT_FAILFAST";
@host[0x10] = "DID_TARGET_FAILURE";
@host[0x11] = "DID_NEXUS_FAILURE";
@host[0x12] = "DID_ALLOC_FAILURE";
@host[0x13] = "DID_MEDIUM_ERROR";

// однобайтные коды состояния из include/scsi/scsi_proto.h:
@status[0x00] = "SAM_STAT_GOOD";
@status[0x02] = "SAM_STAT_CHECK_CONDITION";
@status[0x04] = "SAM_STAT_CONDITION_MET";
@status[0x08] = "SAM_STAT_BUSY";
@status[0x10] = "SAM_STAT_INTERMEDIATE";
@status[0x14] = "SAM_STAT_INTERMEDIATE_CONDITION_MET";
@status[0x18] = "SAM_STAT_RESERVATION_CONFLICT";
@status[0x22] = "SAM_STAT_COMMAND_TERMINATED";
@status[0x28] = "SAM_STAT_TASK_SET_FULL";
@status[0x30] = "SAM_STAT_ACA_ACTIVE";
@status[0x40] = "SAM_STAT_TASK_ABORTED";
}

tracepoint:scsi:scsi_dispatch_cmd_done
{
    @[@host[(args->result >> 16) & 0xff], @status[args->result & 0xff]] =
        count();
}

END
{
    clear(@status);
    clear(@host);
}

```

Инструмент использует точку трассировки `scsi:scsi_dispatch_cmd_done` и извлекает однобайтные коды хоста и состояния, а затем отображает их в имена. Аналогичные таблицы поиска есть в исходном коде ядра, в заголовке `include/trace/events/scsi.h`.

В результате, извлекаемом в точке трассировки, есть байты с кодами драйвера и сообщения, не отображаемые этим инструментом. В целом результат имеет такой формат:

```
driver_byte << 24 | host_byte << 16 | msg_byte << 8 | status_byte
```

Этот инструмент можно улучшить, добавив сохранение этих байтов и другой информации в карте в виде дополнительных ключей. Вот какие дополнительные сведения доступны в этой точке трассировки:

```

# bpftrace -lv t:scsi:scsi_dispatch_cmd_done
tracepoint:scsi:scsi_dispatch_cmd_done
    unsigned int host_no;

```



```

[1K, 2K)          0 |
[2K, 4K)          0 |
[4K, 8K)          7 |@@@@@@@@
@usecs[nvme0n1, nvme_cmd_read]:
[32, 64)          7653 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64, 128)         568 |@@@
[128, 256)        45 |
[256, 512)        4 |
[512, 1K)         0 |
[1K, 2K)          0 |
[2K, 4K)          0 |
[4K, 8K)          1 |

```

Этот вывод показывает, что в период трассировки использовался только один диск — `nvme0n1`, и сообщает задержки для трех типов команд `nvme`.

Точки трассировки для `nvme` были добавлены в Linux недавно, но я написал этот инструмент еще раньше, чтобы показать, что можно сделать с помощью `kprobes` и драйверов хранилищ. Я начал с подсчета частоты и определения функций `nvme`, использовавшихся при различных рабочих нагрузках ввода/вывода:

```

# bpftrace -e 'kprobe:nvme* { @[func] = count(); }'
Attaching 184 probes...
^C

```

```

@[nvme_pci_complete_rq]: 5998
@[nvme_free_iod]: 6047
@[nvme_setup_cmd]: 6048
@[nvme_queue_rq]: 6071
@[nvme_complete_rq]: 6171
@[nvme_irq]: 6304
@[nvme_process_cq]: 12327

```

Обзор исходного кода этих функций показал, что задержку можно определить как промежуток между вызовами `nvme_setup_cmd()` и `nvme_complete_rq()`.

Наличие точек трассировки помогает в создании инструмента. Исследование точек трассировки `nvme` [187] помогло мне ускорить разработку инструмента за счет того, что я увидел в исходном коде точек трассировки, как правильно интерпретировать коды операций `nvme`.

Исходный код `nvmelateny(8)`:

```

#!/usr/local/bin/bpftrace

#include <linux/blkdev.h>
#include <linux/nvme.h>

BEGIN
{
    printf("Tracing nvme command latency. Hit Ctrl-C to end.\n");
    // из linux/nvme.h:

```

```

        @ioopcode[0x00] = "nvme_cmd_flush";
        @ioopcode[0x01] = "nvme_cmd_write";
        @ioopcode[0x02] = "nvme_cmd_read";
        @ioopcode[0x04] = "nvme_cmd_write_uncor";
        @ioopcode[0x05] = "nvme_cmd_compare";
        @ioopcode[0x08] = "nvme_cmd_write_zeroes";
        @ioopcode[0x09] = "nvme_cmd_dsm";
        @ioopcode[0x0d] = "nvme_cmd_resv_register";
        @ioopcode[0x0e] = "nvme_cmd_resv_report";
        @ioopcode[0x11] = "nvme_cmd_resv_acquire";
        @ioopcode[0x15] = "nvme_cmd_resv_release";
    }

kprobe:nvme_setup_cmd
{
    $req = (struct request *)arg1;
    if ($req->rq_disk) {
        @start[arg1] = nsecs;
        @cmd[arg1] = arg2;
    } else {
        @admin_commands = count();
    }
}

kprobe:nvme_complete_rq
/@start[arg0]/
{
    $req = (struct request *)arg0;
    $cmd = (struct nvme_command *)@cmd[arg0];
    $disk = $req->rq_disk;
    $opcode = $cmd->common.opcode & 0xff;
    @usecs[$disk->disk_name, @ioopcode[$opcode]] =
        hist((nsecs - @start[arg0]) / 1000);
    delete(@start[tid]); delete(@cmd[tid]);
}

END
{
    clear(@ioopcode); clear(@start); clear(@cmd);
}

```

Если запрос создан без номера диска, то это команда администратора. В сценарий можно добавить декодирование и сохранение времени выполнения команд администратора (см. `nvme_admin_opcode` в `include/linux/nvme.h`). Но для компактности инструмента я просто подсчитываю команды администратора — в случае их появления они будут отмечены в выводе.

9.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ VPF

В этом разделе перечислены однострочные сценарии для ВСС и `bpfttrace`. Там, где это возможно, один и тот же сценарий реализуется с помощью ВСС и `bpfttrace`.

9.4.1. BCC

Подсчитывает операции блочного ввода/вывода по точкам трассировки:

```
funccount t:block:*
```

Выводит гистограмму с распределением объемов данных, вовлеченных в операции блочного ввода/вывода:

```
argdist -H 't:block:block_rq_issue():u32:args->bytes'
```

Подсчитывает количество запросов на выполнение блочного ввода/вывода по трассировкам стека в пространстве пользователя:

```
stackcount -U t:block:block_rq_issue
```

Подсчитывает операции блочного ввода/вывода по флагам типа:

```
argdist -C 't:block:block_rq_issue():char*:args->rwbs'
```

Трассирует ошибки блочного ввода/вывода, группируя их по устройствам и типам операций:

```
trace 't:block:block_rq_complete (args->error) "dev %d type %s error %d", args->dev, args->rwbs, args->error'
```

Подсчитывает операции SCSI по их кодам:

```
argdist -C 't:scsi:scsi_dispatch_cmd_start():u32:args->opcode'
```

Подсчитывает операции SCSI по кодам результатов:

```
argdist -C 't:scsi:scsi_dispatch_cmd_done():u32:args->result'
```

Подсчитывает вызовы функций драйвера nvme:

```
funccount 'nvme*'
```

9.4.2. bpftrace

Подсчитывает операции блочного ввода/вывода по точкам трассировки:

```
bpftrace -e 'tracepoint:block:* { @[probe] = count(); }'
```

Выводит гистограмму с распределением объемов данных, вовлеченных в операции блочного ввода/вывода:

```
bpftrace -e 't:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Подсчитывает количество запросов на выполнение блочного ввода/вывода по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:block:block_rq_issue { @[ustack] = count(); }'
```

Подсчитывает операции блочного ввода/вывода по флагам типа:

```
bpftrace -e 't:block:block_rq_issue { @[args->rwbs] = count(); }'
```

Выводит общее число байтов данных, вовлеченных в операции, по типам ввода/вывода:

```
bpftrace -e 't:block:block_rq_issue { @[args->rwbs] = sum(args->bytes); }'
```

Трассирует ошибки блочного ввода/вывода, группируя их по устройствам и типам операций:

```
bpftrace -e 't:block:block_rq_complete /args->error/ {  
    printf("dev %d type %s error %d\n", args->dev, args->rwbs, args->error); }'
```

Выводит гистограмму с распределением времени блокировки очереди блочного ввода/вывода:

```
bpftrace -e 'k:blk_start_plug { @ts[arg0] = nsecs; }  
    k:blk_flush_plug_list /@ts[arg0]/ { @plug_ns = hist(nsecs - @ts[arg0]);  
    delete(@ts[arg0]); }'
```

Подсчитывает операции SCSI по их кодам:

```
bpftrace -e 't:scsi:scsi_dispatch_cmd_start { @opcode[args->opcode] = count(); }'
```

Подсчитывает операции SCSI по кодам результатов (учитывает все 4 байта):

```
bpftrace -e 't:scsi:scsi_dispatch_cmd_done { @result[args->result] = count(); }'
```

Выводит распределение процессорного времени на обработку запросов blk_mq:

```
bpftrace -e 'k:blk_mq_start_request { @swqueues = lhist(cpu, 0, 100, 1); }'
```

Подсчитывает вызовы функций драйвера scsi:

```
bpftrace -e 'kprobe:scsi* { @[func] = count(); }'
```

Подсчитывает вызовы функций драйвера nvme:

```
bpftrace -e 'kprobe:nvme* { @[func] = count(); }'
```

9.4.3. Примеры использования однострочных сценариев BPF

Чтобы показать работу однострочных сценариев, я включил сюда примеры их вывода.

Подсчет операций блочного ввода/вывода по флагам типа:

```
# bpftrace -e 't:block:block_rq_issue { @[args->rwbs] = count(); }'  
Attaching 1 probe...  
^C
```



```
@[N]: 2
@[WFS]: 9
@[FF]: 12
@[N]: 13
@[WSM]: 23
@[WM]: 64
@[WS]: 86
@[R]: 201
@[R]: 285
@[W]: 459
@[RM]: 1112
@[RA]: 2128
@[R]: 3635
@[W]: 4578
```

Этот сценарий подсчитывает частоту разных видов операций, анализируя значение в поле `rwbs`. При трассировке было выполнено 3635 операций чтения («R») и 2128 операций ввода/вывода с упреждающим чтением («RA»). Поле `rwbs` описывается в разделе «`rwbs`» в начале этой главы.

Этот однострочный сценарий помогает ответить на вопросы о характеристиках рабочей нагрузки:

- Каково соотношение блочных операций чтения и ввода/вывода с упреждающим чтением?
- Каково соотношение блочных операций записи и синхронной записи?

Заменив `count()` на `sum(args->bytes)` в этом сценарии, можно получить суммарные объемы операций ввода/вывода в байтах по типам.

9.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Упражнения можно выполнить с помощью `bpfttrace` или ВСС, если явно не указано иное:

1. Измените инструмент `biolatency(8)`, чтобы он выводил линейную гистограмму для диапазона от 0 до 100 миллисекунд с шагом в 1 миллисекунду.
2. Измените инструмент `biolatency(8)`, чтобы он выводил сводную линейную гистограмму раз в секунду.
3. Разработайте инструмент, показывающий завершение операций дискового ввода/вывода процессором, чтобы с его помощью можно было проверить сбалансированность этих прерываний. Результаты могут отображаться в виде линейной гистограммы.
4. Разработайте инструмент, напоминающий `biosnoop(8)`, для вывода сведений о каждом событии блочного ввода/вывода в формате CSV со следующими полями: время завершения, направление, задержка в миллисекундах. В поле направления должен отражаться тип операции: чтение или запись.

5. С помощью инструмента из п. 4 сохраните данные за 2 минуты. С помощью ПО для построения графиков визуализируйте результаты в виде диаграммы рассеяния, окрасив операции чтения красным цветом, а записи — синим.
6. С помощью инструмента из п. 2 сохраните данные за 2 минуты, а с помощью ПО для построения графиков визуализируйте результаты в виде тепловой карты задержек. (При желании можете разработать свое ПО для построения графиков: например, используя `awk(1)` для превращения столбца `count` в строки HTML-таблицы с цветом фона, соответствующим значению.)
7. Перепишите инструмент `biosnoop(8)` с использованием точек трассировки блочного ввода/вывода.
8. Измените инструмент `seeksize(8)`, чтобы он выводил фактические расстояния перехода, которые выполняют устройства хранения, измеряя их по завершении.
9. Напишите инструмент, отображающий тайм-ауты дискового ввода/вывода. Для этой цели можно использовать точки трассировки блочного ввода/вывода и `BLK_STS_TIMEOUT` (см. `bioerr(8)`).
10. (Усложненное, не решено.) Напишите инструмент, отображающий объемы объединенных блочных операций ввода/вывода в виде гистограммы.

9.6. ИТОГИ

Эта глава показала, как BPF позволяет производить трассировку на всех уровнях стека ввода/вывода хранилища. Я привел примеры трассировки уровня блочного ввода/вывода, планировщиков ввода/вывода, а также драйверов SCSI и nvme.

Глава 10

СЕТИ

С появлением моделей распределенных облачных вычислений, увеличивающих сетевой трафик в центрах обработки данных или облачных средах, а также сетевых приложений, увеличивающих внешний сетевой трафик, сеть играет все большую роль в анализе производительности систем. Потребность в эффективных инструментах анализа сетей тоже растет, потому что серверы способны масштабироваться и обрабатывать миллионы пакетов в секунду. Расширенный BPF начинался как технология обработки пакетов, поэтому изначально ориентировался на работу с такими скоростями. Проект Cilium по созданию контейнерных сетей и политик безопасности, а также масштабируемый сетевой балансировщик нагрузки Katran от Facebook — это дополнительные примеры способности BPF обрабатывать высокую скорость передачи пакетов в производственных средах, в том числе для смягчения последствий распределенных атак типа «отказ в обслуживании» (Distributed Denial of Service, DDoS)¹.

Сетевой ввод/вывод обрабатывается множеством протоколов и на самых разных уровнях, включая приложение, библиотеки протоколов, системные вызовы, TCP, UDP и IP, а также драйверы сетевых интерфейсов. Все эти компоненты можно трассировать с помощью инструментов BPF, которые будут показаны в этой главе, чтобы получить представление о рабочих нагрузках и задержках.

Цели обучения:

- получить общее представление о сетевом стеке и подходах к масштабированию, включая масштабирование приема и передачи, буферах TCP и дисциплинах организации очередей;
- изучить стратегию успешного анализа производительности сети;
- определить характеристики сокетов, TCP и UDP для выявления проблем;
- измерять различные задержки: задержки подключения, задержку передачи первого байта, продолжительность соединения;
- изучить эффективный способ анализа и трассировки повторных передач TCP;
- исследовать задержки, возникающие внутри сетевого стека;

¹ Оба этих проекта с открытым исходным кодом [93], [94].

- измерять время, проводимое запросами в очередях операционной системы и сетевых устройств;
- использовать однострочные сценарии `bpfttrace` для исследования сети нестандартными способами.

В начале этой главы обсудим некоторые теоретические основы, необходимые для анализа сети, и познакомимся с подходами к организации и масштабированию сетевого стека. Попутно я расскажу, на какие вопросы отвечает BPF, и предложу общую стратегию анализа. Затем мы познакомимся с инструментами: от традиционных инструментов до инструментов BPF, включая однострочные сценарии BPF. В заключение я представлю набор дополнительных упражнений.

10.1. ОСНОВЫ

В этом разделе познакомимся с основами организации сетей, возможностями BPF, оптимальной стратегией анализа сети и типичными ошибками при трассировке.

10.1.1. Основы организации сетей

Эта глава предполагает наличие у читателя базовых знаний о протоколах IP и TCP, включая знакомство с трехэтапным процессом установки соединения в TCP, понимание, что такое пакеты подтверждения и как устанавливаются активные/пассивные соединения.

Сетевой стек

На рис. 10.1 изображен сетевой стек Linux. Здесь видно, как перемещаются данные от приложения к сетевой карте (Network Interface Card, NIC).

Основные компоненты:

- **Сокеты:** конечные точки для отправки или получения данных. К ним также относятся буферы приема и отправки, используемые в TCP.
- **TCP (Transmission Control Protocol — протокол управления передачей):** широко используемый транспортный протокол для упорядоченной и надежной передачи данных с проверкой ошибок.
- **UDP (User Datagram Protocol — протокол дейтаграмм пользователя):** простой транспортный протокол для отправки сообщений без оверхеда или гарантий TCP.
- **IP (Internet Protocol — интернет-протокол):** сетевой протокол для доставки пакетов между узлами в сети. Основные версии: IPv4 и IPv6.
- **ICMP (Internet Control Message Protocol — протокол межсетевых управляющих сообщений):** протокол уровня IP для поддержки IP и ретрансляции сообщений о маршрутах и ошибках.

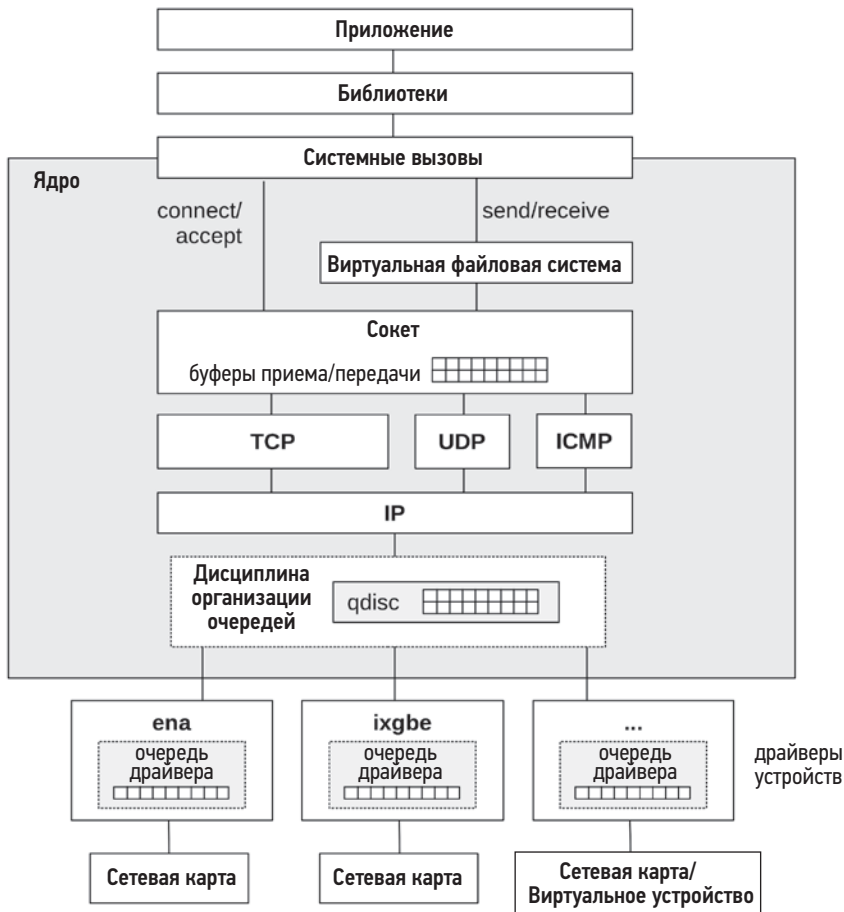


Рис. 10.1. Сетевой стек Linux

- **Дисциплина организации очередей:** дополнительный уровень для классификации, планирования, управления, фильтрации и формирования трафика [95]¹.
- **Драйверы устройств:** драйверы могут иметь свои очереди (например, очереди передачи и очереди приема).
- **Сетевая карта (Network Interface Card, NIC):** устройство, содержащее физические сетевые порты. Это также могут быть виртуальные устройства, такие как туннели, виртуальные устройства Ethernet (veth) и петлевые устройства (loopback).

¹ Замечательное описание всего, что связано с этими очередями, можно найти в статье «Queueing in the Linux Network Stack» Дэна Симона, опубликованной в Linux Journal в 2013 году. Так совпало, что примерно через полтора часа после написания этого раздела я оказался на онлайн-конференции iovisor с Дэном и смог поблагодарить его лично.

На рис. 10.1 показан типичный путь следования данных, но для повышения производительности определенных рабочих нагрузок могут использоваться и другие пути, в том числе путь в обход ядра и новый высокопроизводительный путь XDP на основе BPF.

В обход ядра

Приложения могут работать в обход сетевого стека ядра на таких технологиях, как Data Plane Development Kit (DPDK), чтобы достичь более высокой скорости передачи пакетов и производительности. Сюда входят приложения, реализующие свои сетевые протоколы в пользовательском пространстве и выполняющие запись в сетевой драйвер через библиотеку DPDK и драйвер ввода/вывода пользовательского пространства (UIO) или виртуальные функции ввода/вывода (VFIO). Расходов на копирование пакетов можно избежать за счет прямого доступа к памяти сетевой карты.

В этих случаях сетевой стек ядра не используется, что затрудняет анализ производительности с использованием традиционных инструментов и метрик.

XDP

Технология eXpress Data Path (XDP) предлагает другой путь передачи сетевых пакетов: быстрый и программируемый путь, который использует расширенный BPF и интегрируется в существующий стек ядра [Høiland-Jørgensen 18]. Благодаря доступу к исходному сетевому фрейму Ethernet, в обработчиках BPF внутри драйвера сетевой карты уже на самых ранних этапах можно принимать решения о пересылке или фильтрации без дополнительных затрат на обработку стека TCP/IP. При необходимости можно также вернуться к обычному пути обработки в сетевом стеке. Эта технология часто используется для предотвращения DDoS-атак и программной маршрутизации.

Внутреннее устройство

Понимание внутреннего устройства сетевого стека в ядре поможет разобраться в работе инструментов BPF, описанных далее в главе. Самое главное: пакеты проходят через ядро с использованием структуры `sk_buff` (socket buffer — буфер сокета). Сокеты определяются структурой `sock`, входящей в состав вариантов для разных протоколов, таких как `tcp_sock`. Сетевые протоколы присоединяются к сокетам с помощью структуры `proto`, например `tcp_prot`, `udp_prot` и т. д. Эта структура определяет функции обратного вызова для работы с протоколом, в том числе `connect`, `sendmsg` и `recvmsg`.

Масштабирование приема и отправки

Без стратегии балансировки нагрузки на процессоры сетевая карта способна прерывать только один процессор, что может загрузить его на 100% обработкой прерываний и сетевого стека и превратить в узкое место. Есть разные политики,

помогающие ослабить эту проблему и распределить обработку прерываний от сетевой карты и пакетов между несколькими процессорами, что улучшает масштабируемость и производительность. К ним относятся новые программные интерфейсы Receive Side Scaling (RSS)¹, Receive Packet Steering (RPS), Receive Flow Steering (RFS), Accelerated RFS и Transmit Packet Steering (XPS). Все они описаны в исходном коде Linux [96].

Масштабирование приема в сокетах

Обычно для обработки пассивных TCP-соединений, следующих друг за другом с большой частотой, используется отдельный поток выполнения, обрабатывающий вызовы `accept(2)` и затем передающий установленные соединения пулу рабочих потоков. Для дальнейшего масштабирования в Linux 3.9 в `setsockopt(3)` был добавлен параметр `SO_REUSEPORT`, позволяющий пулу процессов или потоков использовать один и тот же адрес сокета в вызовах `accept(2)`. Ядро должно само балансировать новые соединения в пуле связанных потоков. Для управления этой балансировкой можно передать программу BPF через параметр `SO_ATTACH_REUSEPORT_EBPF`, добавленный для UDP в версии Linux 4.5 и для TCP — в Linux 4.6.

Очереди соединений TCP

Пассивные TCP-соединения иницируются ядром после получения TCP-пакета SYN. Ядро должно хранить состояние потенциальных соединений до получения подтверждения. В прошлом это использовалось злоумышленниками, посылавшими потоки пакетов SYN (атаки SYN-flood), чтобы вызвать исчерпание памяти ядра. Для предотвращения таких атак в Linux применяются две очереди: резервная, с минимальными метаданными, которая способна принять большой поток пакетов SYN, и очередь для установленных соединений, готовых для передачи приложениям. Они показаны на рис. 10.2.

Пакеты могут выбрасываться из резервной очереди в случае поступления слишком большого их количества и из очереди установленных соединений, если приложение не может принимать соединения достаточно быстро. В таких случаях легитимный удаленный узел повторит попытку по таймеру.

Кроме модели с двумя очередями был реализован и путь приема TCP-соединений без блокировки, чтобы улучшить масштабируемость на случай атак SYN-flood [98]².

¹ RSS обрабатывается исключительно аппаратным обеспечением сетевой карты. Некоторые сетевые карты поддерживают выгрузку сетевых программ BPF (например, Netronome), что позволяет программировать RSS средствами BPF [97].

² Разработчик Эрик Дюмазе смог достичь частоты в 6 миллионов пакетов SYN в секунду в своей системе после исправления последней проблемы с ложным разделением данных [99].

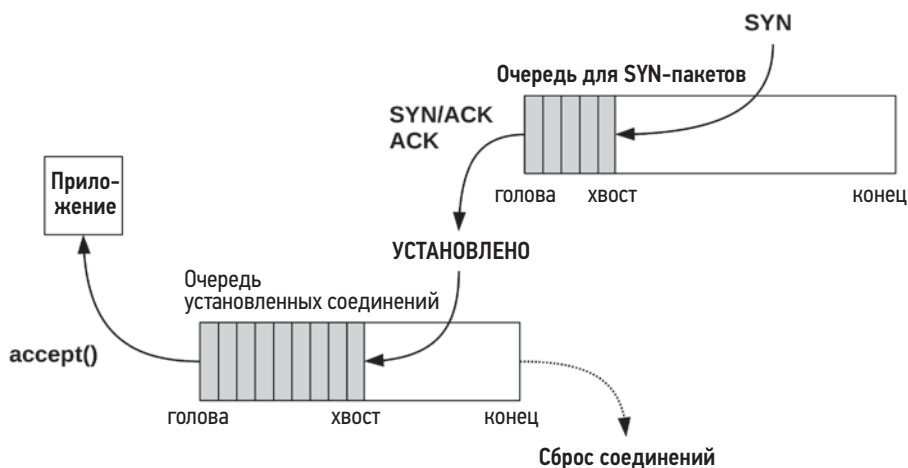


Рис. 10.2. Очереди TCP SYN

Повторные передачи TCP

TCP обнаруживает потерю пакетов и повторно передает их, используя один из двух методов:

- **Повторная передача по таймеру:** выполняется, если в течение определенного времени не был получен пакет подтверждения. Это время — тайм-аут повторной передачи TCP — рассчитывается динамически на основе времени от передачи запроса до получения ответа (Round Trip Time, RTT). В Linux это время составляет не менее 200 мс (TCP_RTO_MIN) для первой повторной передачи, а последующие повторные передачи выполняются намного позже, согласно алгоритму экспоненциальной задержки, который удваивает величину тайм-аута.
- **Быстрая повторная передача:** при получении повторных пакетов ACK. В таких случаях TCP может предположить, что пакет был сброшен, и немедленно выполнит повторную передачу.

Повторные передачи по таймеру могут вызывать проблемы с производительностью, добавляя задержки до 200 мс и выше. Алгоритмы управления загруженностью тоже могут ограничивать пропускную способность при наличии повторных передач.

Для повторной передачи может потребоваться заново отправить целую последовательность пакетов, начиная с потерянного, даже если следующие были получены правильно. Выборочные подтверждения (Selective ACKnowledgments, SACK) — это один из механизмов TCP, помогающий избежать этого: он позволяет подтвердить более поздние пакеты, чтобы не отправлять их повторно, что способствует увеличению производительности.

ТСР-буферы отправки и приема

Пропускная способность ТСР повышается за счет управления буферами отправки и приема в сокетах. Ядро Linux динамически изменяет размеры буферов в зависимости от активности соединения и позволяет настраивать их минимальный, типичный и максимальный размеры. Большие размеры буферов улучшают производительность за счет выделения большего объема памяти для каждого соединения. Они показаны на рис. 10.3.

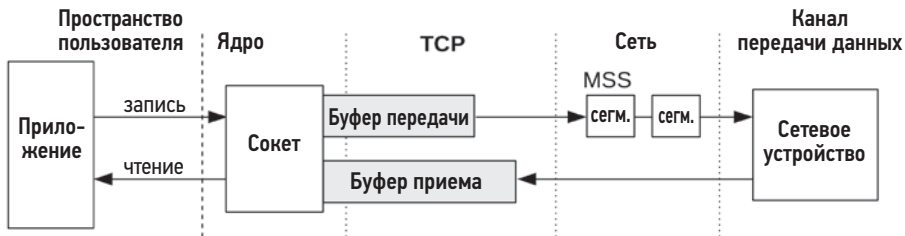


Рис. 10.3. ТСР-буферы отправки и приема

Сетевые устройства и сети ограничивают размеры пакетов максимальным размером сегмента (Maximum Segment Size, MSS), который может составлять всего 1500 байт. Чтобы избежать перегрузки на отправку большого количества маленьких пакетов, для отправки пакетов размером до 64 Кбайт («суперпакетов») ТСР использует универсальный механизм уменьшения затрат на сегментацию (Generic Segmentation Offload, GSO), который разбивает пакеты на сегменты с размером MSS непосредственно перед передачей в сетевое устройство. Если сетевая карта и драйвер поддерживают уменьшение затрат на сегментацию ТСР (TCP Segmentation Offload, TSO), то само сегментирование делегируется устройству, что дополнительно увеличивает пропускную способность сетевого стека. Есть и дополнение к GSO [100] — механизм уменьшения затрат на прием (Generic Receive Offload, GRO). Оба механизма — GRO и GSO — реализуются в ядре, а TSO реализуется аппаратным обеспечением сетевых карт.

Управление загруженностью в ТСР

Реализация ТСР в Linux поддерживает несколько алгоритмов управления загруженностью, включая Cubic (по умолчанию), Reno, Tahoe, DCTCP и BBR. Эти алгоритмы изменяют размер окон отправки и приема в зависимости от обнаруженной загруженности линии, что помогает поддерживать оптимальную работу сетевых подключений.

Дисциплина очереди

Этот необязательный уровень управляет классификацией трафика (traffic classification, tc), планированием, обработкой, фильтрацией и формированием

сетевых пакетов. В Linux реализовано несколько алгоритмов, определяющих дисциплины очередей, которые можно настраивать с помощью команды `tc(8)`. Для каждого алгоритма есть своя страница в справочном руководстве, которые можно вызвать командой `man(1)`:

```
# man -k tc-
actions (8)      - действия классификации трафика, определяемые независимо
tc-basic (8)     - простой фильтр управления трафиком
tc-bfifo (8)     - очередь пакетов "первым пришел, первым вышел"
tc-bpf (8)       - BPF-программируемые классификаторы и действия для добавления
                  в очередь и извлечения из нее
tc-cbq (8)       - организация очередей на основе классов
tc-cbq-details (8) - организация очередей на основе классов
tc-cbs (8)       - дисциплина формирования очередей на основе разрешений
tc-cgroup (8)    - фильтр трафика на основе групп управления
tc-choke (8)     - планировщик выбора и сохранения
tc-codel (8)     - алгоритм активного управления очередью с контролируемой
                  задержкой
tc-connmark (8)  - действие для получения маркера соединения (CONNMARK),
                  установленного брандмауэром netfilter
tc-csum (8)      - действие по обновлению контрольной суммы
tc-drr (8)       - планировщик на основе циклического алгоритма с дефицитом
                  времени
tc-ematch (8)    - дополнительные средства сопоставления для использования
                  с фильтрами "basic" или "flow"
tc-flow (8)      - фильтр управления трафиком на основе потоков
tc-flower (8)    - фильтр управления трафиком на основе потоков
tc-fq (8)        - политика организации справедливых очередей
tc-fq_codel (8)  - комбинированная политика организации справедливых очередей
                  с контролируемой задержкой

[...]
```

BPF позволяет расширить возможности этого уровня с помощью программ типа `BPF_PROG_TYPE_SCHED_CLS` и `BPF_PROG_TYPE_SCHED_ACT`.

Другие оптимизации производительности

В сетевом стеке есть и другие алгоритмы увеличения производительности:

- **Нейгла (Nagle):** снижает число отправляемых маленьких сетевых пакетов, задерживая их передачу, что позволяет дождаться дополнительных пакетов и объединить их.
- **Byte Queue Limits (BQL):** автоматически задает достаточно большие размеры очередей в драйверах, чтобы избежать исчерпания, но достаточно малые, чтобы уменьшить максимальную задержку пакетов в очереди. Принцип действия основан на приостановке добавления пакетов в очередь драйвера, когда это необходимо. Был добавлен в Linux 3.3 [95].
- **Pacing:** управляет частотой отправки пакетов, распределяя (задавая ритм — *pacing*) передачи так, чтобы избежать всплесков, которые могут негативно сказаться на производительности.

- **TCP Small Queues (TSQ):** управляет (уменьшает) количеством очередей в сетевом стеке, чтобы избежать проблем, в том числе с избыточным разбуханием сетевых буферов (bufferbloat) [101].
- **Early Departure Time (EDT):** для упорядочивания пакетов, отправляемых в сетевую карту, вместо очереди используется временное колесо. Каждому пакету, в зависимости от заданной политики и скорости, присваивается временная отметка. Был добавлен в Linux 4.20 и своими возможностями напоминает алгоритмы BQL и TSQ [Jacobson 18].

Для повышения производительности часто используются комбинации этих алгоритмов. Отправляемый TCP-пакет может обрабатываться каким-либо алгоритмом управления загруженностью, TSO, TSQ, Pacing и дисциплинами организации очередей, прежде чем попасть в сетевую карту [Cheng 16].

Измерение задержек

Чтобы получить представление о производительности сети и определить наличие узких мест в отправляющих или принимающих приложениях либо в самой сети, можно выполнить множество разных измерений задержек. К ним относятся [Gregg 13b]:

- **Задержка разрешения имени:** время, за которое имя узла разрешается в его IP-адрес, обычно с помощью службы разрешения имен DNS. Такая задержка — частый источник проблем с производительностью.
- **Задержка зондирования:** время от отправки эхо-запроса ICMP до получения ответа. Измеряет суммарное время обработки пакетов сетевым стеком и ядром на каждом узле.
- **Задержка создания TCP-соединения:** время от момента отправки SYN-пакета до получения пакета SYN/ACK. Поскольку в этом случае никакие приложения не задействованы, эта характеристика соответствует суммарной задержке в сетевом стеке и ядре на каждом узле, аналогично задержке зондирования, плюс некоторая дополнительная обработка в ядре, связанная с созданием сеанса TCP. Технология TCP Fast Open (TFO) позволяет устранить задержки при создании последующих соединений путем предоставления криптографического cookie с SYN-пакетом для немедленной аутентификации клиента, что позволяет серверу отправлять данные в ответ, не дожидаясь завершения трехэтапного рукопожатия.
- **Задержка до первого байта TCP:** также известная как задержка времени до первого байта (Time-To-First-Byte, TTFB), измеряет время от момента установления соединения до момента, когда клиент получает первый байт данных. Включает время на планирование процессора и время принятия решения в приложении, что делает ее более важным показателем производительности приложения и текущей нагрузки, чем задержка соединения TCP.
- **Время от передачи запроса до получения ответа (Round Trip Time, RTT):** время прохождения запроса и ответа между конечными точками. Ядро

может использовать эту характеристику в алгоритмах управления загруженностью.

- **Продолжительность соединения:** продолжительность жизни сетевого соединения от инициализации до закрытия. Некоторые протоколы (например, HTTP) могут использовать стратегию поддержания активности, чтобы сохранить соединение открытым для будущих запросов и тем самым избежать расходов и задержек на повторную установку соединения.

Применение этих метрик в комбинации поможет определить источник задержек. Их также следует использовать в сочетании с другими метриками для оценки состояния сети, включая частоту событий и пропускную способность.

Для дополнительного чтения

Цель этого краткого обзора — познакомить вас с основами, которые нужно знать для анализа работы сети. Реализация сетевого стека Linux описана в исходном коде ядра, в каталоге Documentation/networking [102], а работа сети более подробно рассмотрена в главе 10 книги «Systems Performance»¹ [Gregg 13b].

10.1.2. Возможности BPF

Традиционные инструменты оценки производительности сети используют статистики из ядра и могут производить захват сетевых пакетов. Инструменты трассировки BPF позволяют ответить на вопросы:

- Какие операции ввода/вывода выполняются с сокетом и почему? Что происходит в стеках пользовательского уровня?
- Какие новые сеансы TCP создаются и какими процессами?
- Какие ошибки возникают на уровне сокета, TCP или IP?
- Какой размер имеет окно TCP? Есть ли передачи нулевого размера?
- Какой объем ввода/вывода на разных уровнях стека? А в устройствах?
- Какие пакеты сбрасывает сетевой стек и почему?
- Какие значения имеют задержки на создание TCP-соединений, задержки до первого байта и продолжительность жизни соединений?
- Насколько велика задержка в сетевом стеке ядра?
- Как долго пакеты находятся в очередях, регулируемых дисциплинами? А в очередях сетевых драйверов?
- Какие протоколы высокого уровня используются?

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

Ответить на эти вопросы можно с помощью BPF, инструментируя точки трассировки, если они доступны, и зонды kprobes и uprobes, когда нужны детали, недоступные в точках трассировки.

Источники событий

В табл. 10.1 перечислены доступные для инструментации источники сетевых событий.

Таблица 10.1. Сетевые события и их источники

Типы событий	Источники событий
Прикладные протоколы	uprobes
Сокеты	Точки трассировки системных вызовов
TCP	Точки трассировки протокола TCP, kprobes
UDP	kprobes
IP и ICMP	kprobes
Пакеты	Точки трассировки skb, kprobes
Очереди сетевого стека и драйверов	Точки трассировки qdisc и net, kprobes
XDP	Точки трассировки XDP
Драйверы сетевых устройств	kprobes

Во многих случаях нужно использовать kprobes из-за отсутствия точек трассировки. Одна из причин нехватки точек трассировки — исторически сложившееся (до появления BPF) отсутствие спроса. С ростом спроса на BPF в ядра 4.15 и 4.16 были добавлены первые точки трассировки TCP. В Linux 5.2 добавлены следующие точки трассировки TCP:

```
# bpftrace -l 'tracepoint:tcp:*'  
tracepoint:tcp:tcp_retransmit_skb  
tracepoint:tcp:tcp_send_reset  
tracepoint:tcp:tcp_receive_reset  
tracepoint:tcp:tcp_destroy_sock  
tracepoint:tcp:tcp_rcv_space_adjust  
tracepoint:tcp:tcp_retransmit_synack  
tracepoint:tcp:tcp_probe
```

В будущем в ядро добавятся дополнительные точки трассировки сетевых протоколов. Наиболее очевидными претендентами на добавление выглядят точки трассировки отправки и получения для разных протоколов, но для этого нужно изменить чувствительные к задержкам пути в коде. Для этого следует проявить особую осторожность и сначала выяснить, какой недопустимый оверхед может при этом возникнуть.

Оверхед

Сетевые события могут следовать друг за другом очень часто и достигать нескольких миллионов в секунду на некоторых серверах и при определенных рабочих нагрузках. К счастью, BPF изначально создавался как эффективный фильтр, поэтому он добавляет к каждому событию минимальный объем оверхеда. Но даже если незначительный оверхед умножить на миллионы или десятки миллионов событий в секунду, то суммарно он может стать заметным или весьма ощутимым.

К счастью, многие потребности в наблюдаемости можно удовлетворить, трассируя события, возникающие намного реже, чем следуют отдельные пакеты, что, соответственно, влечет меньший оверхед. Например, трассировку повторных передач TCP можно произвести с использованием функции ядра `tcp_retransmit_skb()`, без анализа каждого пакета. Я недавно использовал этот прием для решения проблемы, когда частота передачи превышала 100 000 пакетов в секунду, а частота повторной передачи — 1000 пакетов в секунду. Каким бы ни был оверхед на трассировку пакетов, мой выбор события для трассировки сократил его в 100 раз.

В случаях, когда нужно трассировать каждый пакет, эффективнее использовать точки трассировки без обработки аргументов (о них рассказано в главе 2).

Часто для анализа производительности сети используются методы, основанные на захвате всех пакетов (`tcpdump(8)`, `libpcap` и т. д.), что не только увеличивает оверхед на анализ каждого пакета, но и создает дополнительную нагрузку на процессор, память и файловую систему, а кроме того, требует дополнительного оверхеда на повторное чтение данных с целью последующей обработки. Для сравнения, трассировка отдельных пакетов в BPF — это уже значительное улучшение эффективности, потому что формирует сводки в памяти ядра без использования файлов.

10.1.3. Стратегия

Если вы новичок в анализе производительности сети, вот предлагаемая общая стратегия, которой можно следовать. В следующих разделах эти инструменты описаны подробнее.

Начните анализ с определения характеристик рабочей нагрузки для выявления неэффективности (шаги 1 и 2), затем проверьте ограничения интерфейса (шаг 3) и различные источники задержек (шаги 4, 5 и 6). На этом этапе, возможно, стоит поэкспериментировать (шаг 7). Но помните, что эксперименты могут помешать работе промышленных рабочих нагрузок. Затем переходите к более продвинутым видам анализа (шаги 8, 9 и 10).

1. С помощью инструментов на основе счетчиков получите основные характеристики сетевой нагрузки: частоту следования пакетов и пропускную способность, а если используется TCP, то частоту создания новых соединений TCP и частоту повторной передачи пакетов (например, `ss(8)`, `nstat(8)`, `netstat(1)` и `sar(1)`).

2. Определите, с какой целью создаются новые TCP-соединения и продолжительность их жизни, чтобы получить дополнительные характеристики рабочей нагрузки, которые помогут в поиске источников неэффективности (например, с использованием ВСС-инструмента `tcplife(8)`). Скажем, можно определить, как часто создаются новые соединения для чтения данных из удаленной службы, которые можно кэшировать локально.
3. Проверьте достижение предела пропускной способности сетевого интерфейса (например, с помощью `sar(1)` или `nicstat(1)` можно выяснить процент загрузки сетевого интерфейса).
4. Выполните трассировку повторных передач TCP и других необычных событий (например, с применением ВСС-инструментов `tcpretrans(8)`, `tcpdrop(8)` и точки трассировки `skb:kfree_skb`).
5. Измерьте задержку разрешения имен узлов (DNS), так как это частый источник проблем с производительностью (например, с помощью ВСС-инструмента `gethostlatency(8)`).
6. Измерьте задержку в разных точках: задержку соединения, задержку до получения первого байта, задержку в стеке и т. д.
 - а) Обратите внимание, что результаты измерений задержек могут сильно меняться из-за разбухания сетевых буферов (вызывает рост задержки при постановке в очередь). При возможности измерьте задержки в периоды высокой нагрузки и простоя системы для сравнения.
7. Примените инструменты, генерирующие нагрузку, чтобы исследовать ограничения пропускной способности сети между узлами и сравнить сетевые события с известной рабочей нагрузкой (например, с помощью `iperf(1)` и `netperf(1)`).
8. Воспользуйтесь другими инструментами BPF, перечисленными в разделе 10.3 «Инструменты BPF» этой главы.
9. Примените профилирование процессора или трассировку стека ядра для количественной оценки процессорного времени, затрачиваемого на обработку протокола и драйвера.
10. Используйте точки трассировки и зонды `kprobes` для исследования работы внутренних механизмов сетевого стека.

10.1.4. Типичные ошибки трассировки

Вот несколько типичных ошибок, которые допускают при разработке инструментов BPF для анализа сетей:

- События могут происходить вне контекста приложения. Пакеты могут поступать, когда процессор выполняет поток бездействия (`idle`), а сеансы TCP могут быть инициализированы и изменить состояние к этому времени. Идентификатор и имя процесса, выполняемого процессором в момент появления этих событий, могут не соответствовать приложению, которое является конечной точкой

соединения. Выбирайте события, имеющие отношение к контексту приложения, или кэшируйте контекст приложения по идентификатору (например, по адресу структуры `sock`), который можно получить позже.

- Пути обработки пакетов могут быть быстрыми и медленными. Вы можете написать программу, которая как будто работает, но трассирует только один из этих путей. Используйте известные рабочие нагрузки и убедитесь, что подсчет пакетов и байтов выполняется верно.
- В TCP есть полные и неполные сокеты: последние являются сокетами запроса, которые действуют до завершения процедуры установки соединения или когда сокет находится в состоянии `TIME_WAIT`. Некоторые поля структуры сокета могут быть недействительными для неполных сокетов.

10.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Традиционные инструменты оценки производительности могут отображать статистику ядра, отражающую частоту передачи пакетов, различные события и пропускную способность, а также состояние открытых сокетов. Эти статистические данные обычно отображаются в виде графиков с помощью инструментов мониторинга. Инструменты другого типа захватывают пакеты и позволяют изучать заголовки и содержимое каждого пакета.

Традиционные инструменты могут не только решать проблемы, но и давать подсказки для дальнейшего использования инструментов BPF. В табл. 10.2 перечислены некоторые традиционные инструменты, сгруппированные по категориям, в зависимости от используемого источника информации: статистики ядра или захватываемых пакетов.

Таблица 10.2. Традиционные инструменты

Инструмент	Тип	Описание
<code>ss</code>	Статистики ядра	Статистики сокетов
<code>ip</code>	Статистики ядра	Статистики IP
<code>nstat</code>	Статистики ядра	Статистики сетевого стека
<code>netstat</code>	Статистики ядра	Многоцелевой инструмент для отображения состояния и статистик сетевого стека
<code>sar</code>	Статистики ядра	Многоцелевой инструмент для отображения статистик сетевой и других подсистем
<code>nicstat</code>	Статистики ядра	Статистики сетевого интерфейса
<code>ethtool</code>	Статистики драйвера	Статистики драйвера сетевого интерфейса
<code>tcpdump</code>	Захват пакетов	Производит захват пакетов для анализа

В разделах ниже кратко описаны основные возможности этих инструментов. Дополнительную информацию об инструментах и приемах их использования ищите на страницах справочного руководства и на других ресурсах, включая книгу «Systems Performance»¹ [Gregg 13a].

Обратите внимание, что есть и инструменты, помогающие проводить эксперименты с целью анализа работы сети, такие как `iperf(1)` и `netperf(1)`, инструменты ICMP, включая `ping(1)`, и инструменты выявления сетевых маршрутов, включая `tracert(1)` и `pathchar`. Есть графический интерфейс Flent для автоматизации сетевых тестов [103]. Есть инструменты для статического анализа: проверка конфигурации системы и оборудования без обязательного применения какой-либо рабочей нагрузки [Elling 00]. Эти экспериментальные и статические инструменты описаны в других источниках (например, [Gregg 13a]).

Сначала рассмотрим инструменты `ss(8)`, `ip(8)` и `nstat(8)`, так как все они входят в состав пакета `iproute2`, мейнтейнерами которого являются инженеры сетевого стека в ядре; а значит, эти инструменты почти наверняка будут поддерживать новейшие функции в ядре Linux.

10.2.1. ss

`ss(8)` — это инструмент доступа к статистике открытых сокетов. По умолчанию он выводит обобщенную информацию о сокетях, например:

```
# ss
Netid State      Recv-Q  Send-Q   Local Address:Port    Peer Address:Port
[...]
tcp    ESTAB        0        0      100.85.142.69:65264    100.82.166.11:6001
tcp    ESTAB        0        0      100.85.142.69:6028    100.82.16.200:6101
[...]
```

Этот вывод отражает текущее состояние. В первом столбце выводится протокол, используемый сокетами: в данном случае TCP. Поскольку этот инструмент перечисляет все установленные соединения с информацией об IP-адресах, его можно использовать для исследования текущей рабочей нагрузки и выяснения информации: число открытых клиентских соединений, число одновременных соединений с зависимой службой и т. д.

Еще больше информации можно получить, используя дополнительные параметры. Например, можно вывести информацию только о сокетах TCP (`-t`) с внутренней информацией TCP (`-i`), расширенной информацией о сокете (`-e`), информацией о процессе (`-p`) и информацией о потреблении памяти (`-m`):

```
# ss -tiepm
State      Recv-Q  Send-Q   Local Address:Port    Peer Address:Port
ESTAB      0        0      100.85.142.69:65264    100.82.166.11:6001
```

¹ Грегг Б. «Производительность систем». Санкт-Петербург, издательство «Питер».

```

users(("java",pid=4195,fd=10865)) uid:33 ino:2009918 sk:78 <->
      skmem:(r0,rb12582912,t0,tb12582912,f266240,w0,o0,b10,d0) ts sack bbr ws
cale:9,9 rto:204 rtt:0.159/0.009 ato:40 mss:1448 pmtu:1500 rcvmss:1448 advmss:14
48 cwnd:152 bytes_acked:347681 bytes_received:1798733 segs_out:582 segs_in:1397
data_segs_out:294 data_segs_in:1318 bbr:(bw:328.6Mbps,mrtt:0.149,pacing_gain:2.8
8672,cwnd_gain:2.88672) send 11074.0Mbps lastsnd:1696 lastrcv:1660 lastack:1660
pacing_rate 2422.4Mbps delivery_rate 328.6Mbps app_limited busy:16ms rcv_rtt:39.
822 rcv_space:84867 rcv_ssthresh:3609062 minrtt:0.139
[...]
```

В этом выводе множество деталей. Жирным выделены адреса конечных точек и следующие сведения:

- **"java",pid=4195**: имя процесса «java», идентификатор процесса PID 4195;
- **fd=10865**: дескриптор файла 10865 (для процесса с PID 4195);
- **rto:204**: тайм-аут повторной передачи TCP: 204 миллисекунды;
- **rtt:0.159/0.009**: среднее время от отправки запроса до получения ответа 0.159 миллисекунды, со средним отклонением 0.009 миллисекунды;
- **mss:1448**: максимальный размер сегмента: 1448 байт;
- **cwnd:152**: размер окна насыщения: $152 \times \text{MSS}$;
- **bytes_acked:347681**: 340 Кбайт успешно отправлено;
- **bytes_received:1798733**: 1.72 Мбайт принято;
- **bbr:...**: статистики BBR (Bottleneck Bandwidth and RTT — минимальная пропускная способность и время от запроса до ответа) механизма управления загруженностью;
- **pacing_rate 2422.4Mbps**: частота отправки 2422.4 Мбит/с.

Этот инструмент использует интерфейс netlink, который получает информацию из ядра с помощью сокетов семейства AF_NETLINK.

10.2.2. ip

ip(8) — это инструмент для управления маршрутизацией, сетевыми устройствами, интерфейсами и туннелями. Его можно использовать для вывода статистики по различным объектам: интерфейсам, адресам, маршрутам и т. д. Вот пример вывода дополнительной статистики (-s) по интерфейсам (link):

```

# ip -s link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    RX: bytes    packets  errors  dropped  overrun  mcast
    26550075    273178      0        0         0         0
    TX: bytes    packets  errors  dropped  carrier  collsns
    26550075    273178      0        0         0         0
```

```

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT
group default qlen 1000
    link/ether 12:c0:0a:b0:21:b8 brd ff:ff:ff:ff:ff:ff
    RX: bytes    packets  errors  dropped overrun mcast
       512473039143 568704184 0        0        0        0
    TX: bytes    packets  errors  dropped carrier collsns
       573510263433 668110321 0        0        0        0

```

По этим данным можно проверить наличие различных ошибок, например, для приема (RX): число ошибок приема (errors), число сброшенных пакетов (dropped) и переполнений (overrun). Для передачи (TX): ошибок передачи (errors), сброшенных пакетов (dropped), ошибок в канале передачи (carrier) и коллизий (collisions). Такие ошибки могут стать источником проблем с производительностью и, в зависимости от конкретной ошибки, могут говорить о неисправности сетевого оборудования.

При выводе объекта route отображается таблица маршрутизации:

```

# ip route
default via 100.85.128.1 dev eth0
default via 100.85.128.1 dev eth0 proto dhcp src 100.85.142.69 metric 100
100.85.128.0/18 dev eth0 proto kernel scope link src 100.85.142.69
100.85.128.1 dev eth0 proto dhcp scope link src 100.85.142.69 metric 100

```

Неправильно настроенные маршруты тоже могут вызывать проблемы с производительностью.

10.2.3. nstat

nstat(8) выводит разные характеристики сети, поддерживаемые ядром, с их именами SNMP:

```

# nstat -s
#kernel
IpInReceives          462657733          0.0
IpInDelivers          462657733          0.0
IpOutRequests         497050986          0.0
[...]
TcpActiveOpens        362997             0.0
TcpPassiveOpens       9663983            0.0
TcpAttemptFails       12718              0.0
TcpEstabResets        14591              0.0
TcpInSegs             462181482          0.0
TcpOutSegs            938958577          0.0
TcpRetransSegs        129212             0.0
TcpOutRsts            52362              0.0
[...]

```

Здесь использовался дополнительный параметр -s, чтобы избежать сброса этих счетчиков, что делает nstat(8) по умолчанию. Иногда бывает полезно сбросить

счетчики, так как потом можно повторно запустить `nstat(8)` и увидеть значения счетчиков, соответствующих этому интервалу, а не промежутку с момента загрузки. Если возникла сетевая проблема, которую можно воспроизвести с помощью команды, то запустите `nstat(8)` до и после команды, чтобы увидеть, какие счетчики изменились.

`nstat(8)` может выполняться в режиме демона (`-d`) и собирать статистику за интервал времени; она отображается в последнем столбце.

10.2.4. netstat

`netstat(8)` — это традиционный инструмент для получения различных сетевых статистик в зависимости от используемых параметров, например:

- **без параметров (по умолчанию):** список открытых сокетов;
- **-a:** информация обо всех сокетах;
- **-s:** статистики сетевого стека;
- **-i:** статистики сетевого интерфейса;
- **-r:** таблица маршрутизации.

Вот вывод, полученный с параметрами `-a` (для отображения всех сокетов), `-n` (чтобы запретить разрешение IP-адресов, иначе это может создать тяжелую рабочую нагрузку) и `-p` (чтобы вывести информацию о процессе):

```
# netstat -anp
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address   Foreign Address State      PID/Program name
tcp    0      0 192.168.122.1:53 0.0.0.0:*      LISTEN    8086/dnsmasq
tcp    0      0 127.0.0.53:53   0.0.0.0:*      LISTEN    1112/systemd-resolv
tcp    0      0 0.0.0.0:22      0.0.0.0:*      LISTEN    1440/ssh
[...]
tcp    0      0 10.1.64.90:36426 10.2.25.52:22  ESTABLISHED 24152/ssh
[...]
```

Параметр `-i` выводит статистику интерфейса. Вот пример вывода, полученный на производственном облачном экземпляре:

```
# netstat -i
Kernel Interface table
Iface  MTU    RX-OK RX-ERR RX-DRP RX-OVR    TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0   1500  743442015    0    0 0      882573158    0    0    0 BMRU
lo     65536   427560    0    0 0      427560    0    0    0 LRU
```

Интерфейс `eth0` — это основной сетевой интерфейс. Поля делятся на две группы и отображают статистику приема (RX-) и передачи (TX-):

- **OK:** успешно отправленные пакеты;
- **ERR:** ошибочные пакеты;

- **DRP:** сброшенные пакеты;
- **OVR:** переполнения.

Дополнительный параметр `-c` (continuous — непрерывно) выводит информацию каждую секунду.

Параметр `-s` выводит статистику сетевого стека. Вот пример вывода (усеченный), полученный в промышленной системе:

```
# netstat -s
```

```
Ip:
```

```
Forwarding: 2
454143446 total packets received
0 forwarded
0 incoming packets discarded
454143446 incoming packets delivered
487760885 requests sent out
42 outgoing packets dropped
2260 fragments received ok
13560 fragments created
```

```
Icmp:
```

```
[...]
```

```
Tcp:
```

```
359286 active connection openings
9463980 passive connection openings
12527 failed connection attempts
14323 connection resets received
13545 connections established
453673963 segments received
922299281 segments sent out
127247 segments retransmitted
0 bad segments received
51660 resets sent
```

```
Udp:
```

```
[...]
```

```
TcpExt:
```

```
21 resets received for embryonic SYN_RECV sockets
12252 packets pruned from receive queue because of socket buffer overrun
201219 TCP sockets finished time wait in fast timer
11727438 delayed acks sent
1445 delayed acks further delayed because of locked socket
Quick ack mode was activated 17624 times
169257582 packet headers predicted
76058392 acknowledgments not containing data payload received
111925821 predicted acknowledgments
TCPSackRecovery: 1703
Detected reordering 876 times using SACK
Detected reordering 19 times using time stamp
2 congestion windows fully recovered without slow start
```

```
[...]
```

Здесь показаны суммарные величины, накопленные с момента загрузки. Этот вывод дает много информации: можно вычислить частоту следования пакетов для разных

протоколов, частоту создания новых соединений (TCP, активных и пассивных), частоту ошибок, пропускную способность и т. д. Некоторые показатели, которые я исследую в первую очередь, я выделил жирным.

Этот вывод содержит удобочитаемые описания. Он не предназначен для анализа другим ПО, например агентами мониторинга, которые должны читать метрики непосредственно из `/proc/net/snmp` и `/proc/net/netstat` (или даже из вывода `nstat(8)`).

10.2.5. sar

Генератор отчетов о работе системы (system activity reporter) `sar(1)` может выводить различные отчеты со статистикой о работе сети. `sar(1)` можно использовать в режиме реального времени или настроить для периодической записи данных в роли инструмента мониторинга. Параметры анализа сети для `sar(1)`:

- **-n DEV**: статистики сетевого интерфейса;
- **-n EDEV**: ошибки сетевого интерфейса;
- **-n IP,IP6**: статистики дейтаграмм IPv4 и IPv6;
- **-n EIP,EIP6**: статистики ошибок IPv4 и IPv6;
- **-n ICMP,ICMP6**: статистики ICMP IPv4 и IPv6;
- **-n EICMP,EICMP6**: статистики ошибок ICMP IPv4 и IPv6;
- **-n TCP**: статистики TCP;
- **-n ETCP**: статистики ошибок TCP;
- **-n SOCK,SOCK6**: статистики использования IPv4 и IPv6.

Для примера ниже показано применение четырех из этих параметров на производственном экземпляре Nadoor с выводом раз в 1 секунду:

```
# sar -n SOCK,TCP,ETCP,DEV 1
Linux 4.15.0-34-generic (...)      03/06/2019   _x86_64_      (36 CPU)

08:06:48 PM      IFACE      rxpck/s    txpck/s    rxkB/s    txkB/s    rxcmp/s    txcmp/s
rxmcst/s    %ifutil
08:06:49 PM      eth0    121615.00  108725.00  168906.73  149731.09      0.00      0.00
0.00      13.84
08:06:49 PM      lo      600.00      600.00    11879.12    11879.12      0.00      0.00
0.00      0.00

08:06:48 PM      totscs      tcpsck      udpsck      rawsck      ip-frag      tcp-tw
08:06:49 PM      2133          108          5          0          0          7134

08:06:48 PM active/s    passive/s      iseg/s      oseg/s
08:06:49 PM      16.00      134.00    15230.00    109267.00

08:06:48 PM atmptf/s      estres/s    retrans/s    isegerr/s      orsts/s
08:06:49 PM      0.00          8.00          1.00          0.00      14.00
[...]
```

Этот многострочный вывод повторяется по истечении каждого интервала. Эту команду можно использовать, чтобы определить:

- количество открытых сокетов TCP (tcpsock);
- текущую частоту создания новых соединений TCP (active/s + passive/s);
- частоту повторных передач TCP (retrans/s / oseg/s);
- частоту следования пакетов через интерфейс и пропускную способность (txrpk/s + txkB/s, rxkB/s + txkB/s).

Это облачный экземпляр, и я не ожидаю увидеть на нем ошибки сетевого интерфейса: на физических серверах: включите группу EDEV для проверки таких ошибок.

10.2.6. nicstat

Этот инструмент выводит статистику сетевого интерфейса и основан на модели iostat (1)¹. Например:

```
# nicstat 1
Time      Int      rKB/s    wKB/s    rPk/s    wPk/s    rAvs     wAvs    %Util    Sat
20:07:43  eth0    122190  81009.7  89435.8  61576.8  1399.0   1347.2   10.0     0.00
20:07:43   lo    13000.0  13000.0   646.7    646.7   20583.5  20583.5   0.00     0.00

Time      Int      rKB/s    wKB/s    rPk/s    wPk/s    rAvs     wAvs    %Util    Sat
20:07:44  eth0    268115  42283.6  185199  40329.2  1482.5   1073.6   22.0     0.00
20:07:44   lo    1869.3   1869.3    400.3    400.3   4782.1   4782.1   0.00     0.00

Time      Int      rKB/s    wKB/s    rPk/s    wPk/s    rAvs     wAvs    %Util    Sat
20:07:45  eth0    146194  40685.3  102412  33270.4  1461.8   1252.2   12.0     0.00
20:07:45   lo    1721.1   1721.1    109.1    109.1  16149.1  16149.1   0.00     0.00
[...]
```

Сюда входит статистика насыщения, объединяющая различные ошибки, которые указывают на уровень насыщения интерфейса. Добавив параметр -U, можно вывести отдельно процент использования пропускной способности для чтения и записи, чтобы определить, достигает ли то или иное направление своего предела.

10.2.7. ethtool

ethtool(8) можно использовать для проверки статистики в сетевых интерфейсах, если добавить параметры -i и -k, а также статистики драйвера, если добавить параметр -S. Например:

```
# ethtool -S eth0
NIC statistics:
```

¹ Немного истории: версию для Solaris я написал 18 июля 2004 года. Тим Кук написал версию для Linux.

```

tx_timeout: 0
suspend: 0
resume: 0
wd_expired: 0
interface_up: 1
interface_down: 0
admin_q_pause: 0
queue_0_tx_cnt: 100219217
queue_0_tx_bytes: 84830086234
queue_0_tx_queue_stop: 0
queue_0_tx_queue_wakeup: 0
queue_0_tx_dma_mapping_err: 0
queue_0_tx_linearize: 0
queue_0_tx_linearize_failed: 0
queue_0_tx_napi_comp: 112514572
queue_0_tx_tx_poll: 112514649
queue_0_tx_doorbells: 52759561
[...]
```

Эта команда извлекает статистику из инфраструктуры ethtool в ядре, которую поддерживают множество разных драйверов сетевых устройств. Драйверы устройств могут определять свои метрики ethtool.

При запуске с параметром `-i` команда выводит сведения о драйвере, а с параметром `-k` — сведения об интерфейсе. Например:

```

# ethtool -i eth0
driver: ena
version: 2.0.3K
[...]
# ethtool -k eth0
Features for eth0:
rx-checksumming: on
[...]
tcp-segmentation-offload: off
    tx-tcp-segmentation: off [fixed]
    tx-tcp-ecn-segmentation: off [fixed]
    tx-tcp-mangleid-segmentation: off [fixed]
    tx-tcp6-segmentation: off [fixed]
udp-fragmentation-offload: off
generic-segmentation-offload: on
generic-receive-offload: on
large-receive-offload: off [fixed]
rx-vlan-offload: off [fixed]
tx-vlan-offload: off [fixed]
ntuple-filters: off [fixed]
receive-hashing: on
highdma: on
[...]
```

Этот пример был получен на облачном экземпляре с драйвером `ena` и отключенной настройкой `tcp-segmentation-offload`. Для изменения настроек можно использовать параметр `-K`.

10.2.8. tcpdump

Наконец, tcpdump(8) может захватывать пакеты для дальнейшего изучения. Это называется «прослушиванием сети». Вот пример прослушивания интерфейса en0 (-i) с записью (-w) в файл дампа и последующим чтением этого файла (-r) без разрешения имен (-n)¹:

```
# tcpdump -i en0 -w /tmp/out.tcpdump01
tcpdump: listening on en0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C451 packets captured
477 packets received by filter
0 packets dropped by kernel
# tcpdump -nr /tmp/out.tcpdump01
reading from file /tmp/out.tcpdump01, link-type EN10MB (Ethernet)
13:39:48.917870 IP 10.0.0.65.54154 > 69.53.1.1.4433: UDP, length 1357
13:39:48.921398 IP 108.177.1.2.443 > 10.0.0.65.59496: Flags [P.], seq
3108664869:3108664929, ack 2844371493, win 537, options [nop,nop,TS val 2521261
368 ecr 4065740083], length 60
13:39:48.921442 IP 10.0.0.65.59496 > 108.177.1.2.443: Flags [.), ack 60, win 505,
options [nop,nop,TS val 4065741487 ecr 2521261368], length 0
13:39:48.921463 IP 108.177.1.2.443 > 10.0.0.65.59496: Flags [P.], seq 0:60, ack 1,
win 537, options [nop,nop,TS val 2521261793 ecr 4065740083], length 60
[...]
```

Файлы, созданные tcpdump(8), можно анализировать другими инструментами, включая графический интерфейс Wireshark [104]. Он позволяет исследовать заголовки пакетов и «проследивать» сеансы TCP, повторно выбирая передававшиеся и принимавшиеся байты, чтобы посмотреть, как протекало взаимодействие между клиентом и данным узлом.

Несмотря на оптимизацию захвата пакетов в ядре и в библиотеке libpcap, у прослушивания могут быть значительные затраты процессорного времени на сбор пакетов при высокой частоте их следования, а также большие затраты вычислительной мощности, памяти и дискового пространства для хранения, а затем и для последующей обработки. Этот оверхед можно несколько уменьшить, используя фильтр, чтобы сохранять только пакеты с определенным содержимым в заголовке. Но и в этом случае на пакеты, которые не сохраняются, тоже расходуется процессорное время². Поскольку выражение фильтра должно применяться ко всем пакетам, оно должно обрабатываться максимально эффективно. Так появился фильтр пакетов Беркли (Berkeley Packet Filter), созданный как фильтр для захвата пакетов, а затем развившийся до технологии трассировки, которую я использую в этой книге. См. раздел 2.2, где приводится пример программы фильтрации для tcpdump (8).

¹ Разрешение имен может породить дополнительный сетевой трафик при чтении файла, что иногда нежелательно.

² Каждый skb нужно клонировать перед передачей одному из обработчиков пакетов и только потом его можно отфильтровать (см. dev_queue_xmit_nit()). Решения на основе BPF помогают избежать копирования skb.

Может возникнуть ощущение, что инструменты захвата пакетов показывают исчерпывающие детали о сетевых взаимодействиях, но на самом деле они показывают только детали, отправленные по сети. Они не видят состояния ядра и процессов, отвечающие за пакеты, трассировку стека и состояние сокетов и TCP. Такие детали можно увидеть с помощью инструментов трассировки BPF.

10.2.9. /proc

Многие инструменты, описанные выше, извлекают информацию из каталога /proc, в частности из /proc/net. Этот каталог можно изучить в командной строке:

```
$ ls /proc/net/
anycast6      if_inet6      ip_tables_names      ptype         sockstat6
arp           igmp          ip_tables_targets    raw           softnet_stat
bnep          igmp6         ipv6_route           raw6          stat/
connector     ip6_flowlabel l2cap                rfcomm        tcp
dev           ip6_mr_cache  mcfilter             route         tcp6
dev_mcast     ip6_mr_vif    mcfilter6            rt6_stats     udp
dev_snmp6/    ip6_tables_matches netfilter/           rt_acct       udp6
fib_trie      ip6_tables_names netlink              rt_cache      udplite
fib_triestat  ip6_tables_targets netstat              sco           udplite6
hci           ip_mr_cache   packet               snmp          unix
icmp          ip_mr_vif     protocols            snmp6         wireless
icmp6         ip_tables_matches psched              sockstat      xfrm_stat
```

```
$ cat /proc/net/snmp
```

```
Ip: Forwarding DefaultTTL InReceives InHdrErrors InAddrErrors ForwDatagrams
InUnknownProtos InDiscards InDelivers OutRequests OutDiscards OutNoRoutes
ReasmTimeout ReasmReqds ReasmOKs ReasmFails FragOKs FragFails FragCreates
```

```
Ip: 2 64 45794729 0 28 0 0 0 45777774 40659467 4 6429 0 0 0 0 0 0
[...]
```

Инструменты netstat(1) и sar(1) отображают многие из этих значений. Как было показано выше, они используют общесистемную статистику для вычисления частоты передачи пакетов, создания новых активных и пассивных соединений TCP, повторных передач TCP, ошибок ICMP и многого другого.

Интерес представляют и файлы /proc/interrupts и /proc/softirqs, в которых можно найти информацию о распределении прерываний от сетевых устройств по процессорам. Вот пример содержимого этих файлов в двухпроцессорной системе:

```
$ cat /proc/interrupts
```

```
          CPU0          CPU1
[...]
```

28:	1775400	80	PCI-MSI 81920-edge	ena-mgmt@pci:0000:00:05.0
29:	533	5501189	PCI-MSI 81921-edge	eth0-Tx-Rx-0
30:	4526113	278	PCI-MSI 81922-edge	eth0-Tx-Rx-1

```
$ cat /proc/softirqs
```

```
          CPU0          CPU1
```

```
[...]
    NET_TX:      332966      34
    NET_RX:      10915058    11500522
[...]
```

В этой системе есть интерфейс `eth0`, обслуживаемый драйвером `eпа`. Как показывает вывод выше, `eth0` использует отдельную очередь для каждого процессора, а ошибки приема распределяются по обоим процессорам. (Передачи выглядят несбалансированными, но сетевой стек часто пропускает это программное прерывание (`softirq`) и передает данные непосредственно в устройство.) Инструмент `mpstat(8)` имеет параметр `-I` для вывода статистики прерываний.

Инструменты BPF из этой главы создавались для расширения, а не для дублирования возможностей наблюдения за работой сети, предоставляемых файловой системой `/proc` и традиционными инструментами. В BPF есть инструмент `sockstat(8)`, отображающий общесистемные показатели, отражающие использование сокетов, так как эти конкретные метрики недоступны в `/proc`. Однако нет аналогичных инструментов `tcpstat(8)`, `udpstat(8)` или `ipstat(8)` для вывода общесистемных показателей: их можно было бы написать в BPF, но такие инструменты должны использовать только метрики, уже поддерживаемые в `/proc`, а всю эту информацию легко получить с помощью `netstat(1)` и `sar(1)`.

Представленные далее инструменты BPF расширяют возможности наблюдения, разбивая статистики по идентификаторам и именам процессов, IP-адресам и портам, помогают выявлять трассировки стека, приведшие к событиям, раскрывают состояние ядра и показывают величины задержек. Может показаться, что эти инструменты охватывают все аспекты работы сети, но это не так. Они разрабатывались для расширения наблюдаемости и совместного использования с `/proc/net` и более ранними традиционными инструментами.

10.3. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрим инструменты BPF, которые можно использовать для анализа производительности сети и устранения неполадок. Они показаны на рис. 10.4.

Инструмент `bpfttrace` показан на рис. 10.4 как средство для наблюдения за драйверами устройств. Примеры его использования в таком качестве будут приведены в разделе 10.4.3. Часть остальных инструментов, приведенных на рисунке, можно найти в репозиториях BCC и `bpfttrace`, упоминавшихся в главах 4 и 5, а часть была создана специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях — BCC и `bpfttrace`. Все рассматриваемые инструменты перечислены в табл. 10.3, где указано их происхождение (BT — это сокращение от «`bpfttrace`»).

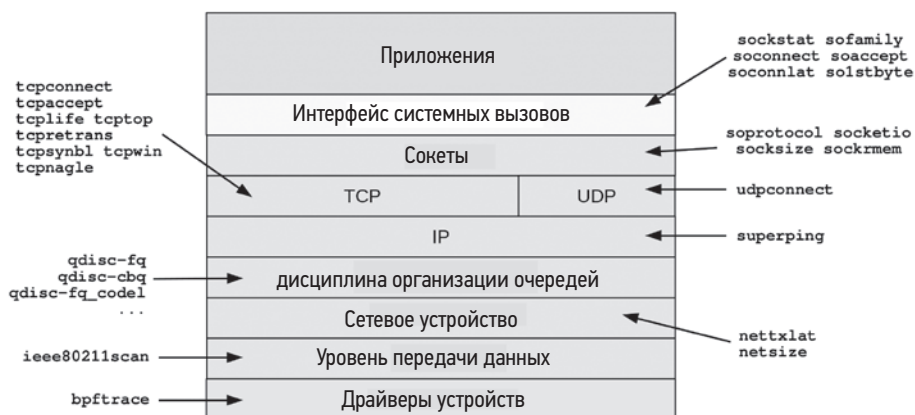


Рис. 10.4. Инструменты BPF для анализа работы сети

Таблица 10.3. Инструменты для анализа работы сети

Инструмент	Источник	Цель	Описание
sockstat	книга	Сокеты	Выводит обобщенную статистику по сокетам
sofamily	книга	Сокеты	Подсчет новых сокетов по семействам адресов и процессам
soprotocol	книга	Сокеты	Подсчет новых сокетов по транспортным протоколам и процессам
soconnect	книга	Сокеты	Трассирует запросы на соединение по протоколу IP
soaccept	книга	Сокеты	Трассирует прием соединений по протоколу IP
socketio	книга	Сокеты	Отображает сводную информацию о сокетах со счетчиками ввода/вывода
socksize	книга	Сокеты	Отображает объемы ввода/вывода через сокеты по процессам
sormem	книга	Сокеты	Отображает статистику использования и переполнения буферов чтения в сокетах
soconnlat	книга	Сокеты	Обобщает информацию о задержках соединения с IP-сокетами
solstbyte	книга	Сокеты	Обобщает информацию о задержках до получения первого байта
tcpconnect	ВСС/ВТ/книга	TCP	Трассирует создание активных TCP-соединений (connect())
tcpaccept	ВСС/ВТ/книга	TCP	Трассирует создание пассивных TCP-соединений (accept())
tcplife	ВСС/книга	TCP	Трассирует продолжительность сеансов TCP и отображает информацию о соединениях

Инструмент	Источник	Цель	Описание
tcptop	ВСС	TCP	Отображает пропускную способность узла по протоколу TCP
tcpretrans	ВСС/ВТ	TCP	Трассирует повторные передачи TCP и выводит информацию с адресами и состоянием TCP
tcpsynbl	книга	TCP	Выводит размеры очередей TCP SYN в виде гистограммы
tcpwin	книга	TCP	Трассирует изменение параметра, определяющего размер окна насыщения
tcpnagle	книга	TCP	Трассирует работу алгоритма Нейгла и задержки передачи
udpconnect	книга	UDP	Трассирует новые локальные соединения UDP
gethostlatency	книга/ВТ	DNS	Трассирует задержки поиска в DNS через библиотечные вызовы
ipecn	книга	IP	Трассирует уведомления о перегрузке входящего IP-трафика
superping	книга	ICMP	Измеряет время выполнения запросов ICMP ECHO
qdisc-fq (...)	книга	очереди	Отображает задержку в очереди с дисциплиной FQ
netsize	книга	сеть	Отображает объемы ввода/вывода через сетевые устройства
nettxlat	книга	сеть	Отображает задержку передачи через сетевые устройства
skbdrop	книга	skb	Трассирует события сброса буферов sk_buff и отображает соответствующие им трассировки стека ядра
skblife	книга	skb	Измеряет продолжительность жизни буферов sk_buff (skb) как задержки между вызовами функций сетевого стека
ieee80211scan	книга	Wi-Fi	Трассирует сканирование Wi-Fi согласно IEEE 802.11

Полные и актуальные списки параметров инструментов ВСС и bpftrace и описание их возможностей ищите в соответствующих репозиториях. Ниже я расскажу только о наиболее важных особенностях.

10.3.1. sockstat

sockstat(8)¹ выводит обобщенную статистику по сокетам и каждую секунду подсчитывает обращения к системным вызовам, имеющим отношение к сокетам. Вот пример вывода, полученного на производственном пограничном сервере:

¹ Немного истории: я написал его для этой книги 14 апреля 2019 года.

```
# sockstat.bt
Attaching 10 probes...
Tracing sock statistics. Output every 1 second.
01:11:41
@[tracepoint:syscalls:sys_enter_bind]: 1
@[tracepoint:syscalls:sys_enter_socket]: 67
@[tracepoint:syscalls:sys_enter_connect]: 67
@[tracepoint:syscalls:sys_enter_accept4]: 89
@[kprobe:sock_sendmsg]: 5280
@[kprobe:sock_recvmsg]: 10547

01:11:42
[...]
```

Каждую секунду выводится время (например, «21:22:56»), за которым следуют счетчики различных событий в сокетах. В этом примере видно, что за секунду возникло 10 547 событий `sock_recvmsg()` и 5280 событий `sock_sendmsg()`, а также немногим менее сотни событий `accept4(2)` и `connect(2)`.

Цель этого инструмента — представить обобщенную статистику, характеризующую рабочую нагрузку, и дать отправную точку для дальнейшего анализа. В вывод включены также имена зондов, чтобы вы могли продолжить исследования. Например, отметив повышенную частоту событий `kprobe:sock_sendmsg`, вы сможете узнать имя процесса с помощью следующего однострочника для `bpftrace`¹:

```
# bpftrace -e 'kprobe:sock_sendmsg { @[comm] = count(); }'
Attaching 1 probe...
^C

@[sshd]: 1
@[redis-server]: 3
@[snmpd]: 6
@[systemd-resolve]: 28
@[java]: 17377
```

Можно исследовать и трассировку стека в пространстве пользователя, если добавить ключ `ustack` в карту.

Инструмент `sockstat(8)` трассирует системные вызовы, связанные с сокетами, используя точки трассировки, а функции ядра `sock_recvmsg()` и `sock_sendmsg()` — с помощью зондов `kprobes`. Оверхед на зонды `kprobes`, вероятно, будет наиболее заметным и может стать измеримым в системах с высокой сетевой нагрузкой.

Исходный код `sockstat(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing sock statistics. Output every 1 second.\n");
```

¹ Примечание для этого и последующих инструментов: приложения могут переопределить свои строки `comm`, записав новое значение в `/proc/self/comm`.

```

}

tracepoint:syscalls:sys_enter_accept*,
tracepoint:syscalls:sys_enter_connect*,
tracepoint:syscalls:sys_enter_bind*,
tracepoint:syscalls:sys_enter_socket*,
kprobe:sock_recvmsg,
kprobe:sock_sendmsg
{
    @[probe] = count();
}

interval:s:1
{
    time();
    print(@);
    clear(@);
}

```

Использование этих зондов kprobes — упрощенное решение. Точно так же можно было бы использовать точки трассировки системных вызовов. Добавив в код дополнительные точки трассировки, можно отследить системные вызовы `recvfrom(2)`, `recvmsg(2)`, `sendto(2)`, `sendmsg(2)` и их варианты. Ситуация с семейством системных вызовов `read(2)` и `write(2)` несколько сложнее, потому что требуется предусмотреть обработку дескриптора файла, чтобы определить его тип и отфильтровать операции чтения и записи, не связанные с сокетами.

10.3.2. sofamily

`sofamily(8)`¹ трассирует создание новых соединений с использованием системных вызовов `accept(2)` и `connect(2)` и обобщает информацию по именам процессов и семействам адресов. Это может пригодиться для определения характеристик рабочей нагрузки: количественной оценки приложенной нагрузки и поиска любых неожиданных случаев использования сокетов, требующих дальнейшего изучения. Вот пример, полученный на производственном пограничном сервере:

```

# sofamily.bt
Attaching 7 probes...
Tracing socket connect/accepts. Ctrl-C to end.
^C

@accept[sshd, 2, AF_INET]: 2
@accept[java, 2, AF_INET]: 420

@connect[sshd, 2, AF_INET]: 2
@connect[sshd, 10, AF_INET6]: 2
@connect[(systemd), 1, AF_UNIX]: 12
@connect[sshd, 1, AF_UNIX]: 34
@connect[java, 2, AF_INET]: 215

```

¹ Немного истории: я написал его для этой книги 10 апреля 2019 года.

Как показывает этот вывод, в период трассировки процесс Java принял 420 и сам инициировал 215 соединений AF_INET (IPv4), что ожидаемо для этого сервера. Вывод формируется на основе карт @accept (принятые соединения) и @connect (инициированные соединения), роль ключа в которых играют имя процесса, номер семейства адресов, а также его имя, если известно.

Нумерация семейств адресов (например, AF_INET == 2) специфична для Linux и определяется в заголовке include/linux/socket.h. (Таблица соответствий приводится на следующей странице.) Другие ОС могут использовать свою нумерацию.

Поскольку трассируемые вызовы выполняются относительно редко (если сравнивать их с пакетными событиями), ожидается, что у этого инструмента незначительный оверхед.

Исходный код sofamily(8):

```
#!/usr/local/bin/bpftrace

#include <linux/socket.h>

BEGIN
{
    printf("Tracing socket connect/accepts. Ctrl-C to end.\n");
    // из linux/socket.h:
    @fam2str[AF_UNSPEC] = "AF_UNSPEC";
    @fam2str[AF_UNIX] = "AF_UNIX";
    @fam2str[AF_INET] = "AF_INET";
    @fam2str[AF_INET6] = "AF_INET6";
}

tracepoint:syscalls:sys_enter_connect
{
    @connect[comm, args->uservaddr->sa_family,
        @fam2str[args->uservaddr->sa_family]] = count();
}

tracepoint:syscalls:sys_enter_accept,
tracepoint:syscalls:sys_enter_accept4
{
    @sockaddr[tid] = args->upeer_sockaddr;
}

tracepoint:syscalls:sys_exit_accept,
tracepoint:syscalls:sys_exit_accept4
/@sockaddr[tid]/
{
    if (args->ret > 0) {
        $sa = (struct sockaddr *)@sockaddr[tid];
        @accept[comm, $sa->sa_family, @fam2str[$sa->sa_family]] =
            count();
    }
    delete(@sockaddr[tid]);
}
```



```

END
{
    clear(@sockaddr); clear(@fam2str);
}

```

Семейство адресов извлекается из поля `sa_family` структуры `sockaddr` — число типа `sa_family_t`, которое преобразуется в `unsigned short`. Инструмент включает этот номер в вывод и для некоторых пытается определить имя семейства для удобства, используя следующую таблицу из `linux/socket.h`¹:

```

/* Поддерживаемые семейства адресов. */
#define AF_UNSPEC      0
#define AF_UNIX       1      /* Сокеты домена Unix */
#define AF_LOCAL      1      /* Синоним AF_UNIX в POSIX */
#define AF_INET       2      /* Интернет протокол IP */
#define AF_AX25       3      /* Радилюбительская связь AX.25 */
#define AF_IPX        4      /* Novell IPX */
#define AF_APPLETALK  5      /* AppleTalk DDP */
#define AF_NETROM     6      /* Радилюбительская связь NET/ROM */
#define AF_BRIDGE     7      /* Многопротокольный мост */
#define AF_ATMPVC     8      /* Пластиковые карты (ATM PVC) */
#define AF_X25        9      /* Зарезервировано для проекта X.25 */
#define AF_INET6     10     /* IP версии 6 */
[...]
```

Этот заголовок подключается при запуске программы `bpfftrace`, соответственно, строка:

```
@fam2str[AF_INET] = "AF_INET";
```

превращается в:

```
@fam2str[2] = "AF_INET";
```

отображающую число «два» в строку `"AF_INET"`.

Для системного вызова `connect(2)` вся информация извлекается в точке вызова. Системные вызовы `accept(2)` трассируются иначе: указатель на структуру `sockaddr` сохраняется в хеше, а затем извлекается при выходе из системного вызова, чтобы получить семейство адресов. Это связано с тем, что структура `sockaddr` заполняется во время выполнения системного вызова, поэтому ее нужно читать в конце. Также в конце проверяется возвращаемое значение `accept(2)` (успех или неудача), так как в случае неудачи содержимое структуры `sockaddr` может быть недействительным. Этот сценарий можно усовершенствовать и добавить аналогичную проверку для `connect(2)`, чтобы подсчитывались только успешно инициированные новые соединения. Инструмент `soconnect(8)` просто подсчитывает вызовы `connect(2)` без учета результата.

¹ В файле — таблица на английском языке, здесь для удобства приведен перевод. — *Примеч. пер.*

10.3.3. soprocol

soprocol(8)¹ трассирует новые соединения с сокетами и обобщает полученные результаты по именам процессов и транспортным протоколам. Это еще один инструмент для идентификации рабочих нагрузок по транспортному протоколу. Вот пример, полученный на производственном пограничном сервере:

```
# soprocol.bt
Attaching 4 probes...
Tracing socket connect/accepts. Ctrl-C to end.
^C

@accept[java, 6, IPPROTO_TCP, TCP]: 1171

@connect[setuidgid, 0, IPPROTO, UNIX]: 2
@connect[ldconfig, 0, IPPROTO, UNIX]: 2
@connect[systemd-resolve, 17, IPPROTO_UDP, UDP]: 79
@connect[java, 17, IPPROTO_UDP, UDP]: 80
@connect[java, 6, IPPROTO_TCP, TCP]: 559
```

Мы видим, что в период трассировки процесс Java принял 559 и сам инициировал 1171 TCP-соединение. Вывод формируется на основе карт @ассепт (принятые соединения) и @connect (инициированные соединения), роль ключа в которых играют имя процесса, номер протокола с его именем, если известно, а также имя модуля протокола.

Поскольку эти вызовы выполняются относительно редко (если сравнивать их с пакетными событиями), ожидается, что у этого инструмента незначительный оверхед.

Исходный код soprocol(8):

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

BEGIN
{
    printf("Tracing socket connect/accepts. Ctrl-C to end.\n");
    // из include/uapi/linux/in.h:
    @prot2str[IPPROTO_IP] = "IPPROTO_IP";
    @prot2str[IPPROTO_ICMP] = "IPPROTO_ICMP";
    @prot2str[IPPROTO_TCP] = "IPPROTO_TCP";
    @prot2str[IPPROTO_UDP] = "IPPROTO_UDP";
}

kprobe:security_socket_accept,
kprobe:security_socket_connect
{
    $sock = (struct socket *)arg0;
    $protocol = $sock->sk->sk_protocol & 0xff;
    @connect[comm, $protocol, @prot2str[$protocol],
```

¹ Немного истории: я написал его для этой книги 13 апреля 2019 года.

```

        $sock->sk->__sk_common.skc_prot->name] = count();
    }

END
{
    clear(@prot2str);
}

```

Здесь используется короткая таблица преобразования номера протокола в строковое название для четырех наиболее известных протоколов. Она основана на заголовке `in.h`:

```

#if __UAPI_DEF_IN_IPPROTO
/* Стандартные и хорошо известные протоколы IP. */
enum {
    IPPROTO_IP = 0,          /* Фиктивный протокол для TCP */
#define IPPROTO_IP IPPROTO_IP
    IPPROTO_ICMP = 1,        /* Протокол управляющих сообщений интернета, ICMP */
#define IPPROTO_ICMP IPPROTO_ICMP
    IPPROTO_IGMP = 2,        /* протокол управления группами интернета, IGMP */
#define IPPROTO_IGMP IPPROTO_IGMP
    IPPROTO_IPIP = 4,        /* Туннели IPIP (старые туннели KA9Q используют 94) */
#define IPPROTO_IPIP IPPROTO_IPIP
    IPPROTO_TCP = 6,         /* Протокол управления передачей, TCP */
#define IPPROTO_TCP IPPROTO_TCP
    [...]

```

При необходимости таблицу `@prot2str` можно расширить.

Имена модулей протоколов, которые можно видеть в предыдущем выводе, — «TCP», «UDP» и т. д. — доступны в виде строк в структуре `sock`: `__sk_common.skc_prot->name`. Я использовал эту удобную возможность в других инструментах для вывода транспортного протокола. Вот пример из `net/ipv4/tcp_ipv4.c`:

```

struct proto tcp_prot = {
    .name           = "TCP",
    .owner          = THIS_MODULE,
    .close          = tcp_close,
    .pre_connect    = tcp_v4_pre_connect,
    [...]

```

Наличие поля с именем (`.name = "TCP"`) — это деталь реализации ядра Linux. Она дает определенные удобства, но в будущем поле `.name` может измениться или исчезнуть. При этом номер транспортного протокола должен быть всегда, поэтому я также включил его в вывод в этом инструменте.

Точки трассировки для системных вызовов `accept(2)` и `connect(2)` не позволяют легко определить протокол, и сейчас для этих событий нет других точек трассировки. По этой причине я переключился на использование зондов `kprobes` и выбрал функции монитора сокетов в Linux (Linux Socket Monitor, LSM) `security_socket_*`, которые принимают структуру `sock` в первом аргументе и являются относительно стабильным интерфейсом.

10.3.4. soconnect

soconnect(8)¹ отображает запросы на соединение по протоколу IP. Например:

```
# soconnect.bt
Attaching 4 probes...
PID    PROCESS    FAM ADDRESS          PORT    LAT(us) RESULT
11448  ssh         2   127.0.0.1         22      43 Success
11449  ssh         2   10.168.188.1      22     45134 Success
11451  curl        2   100.66.96.2       53        6 Success
11451  curl        10  2406:da00:ff00::36d0:a866 80      3 Network unreachable
11451  curl        2   52.43.200.64      80        7 Success
11451  curl        2   52.39.122.191     80        3 Success
11451  curl        2   52.24.119.28      80       19 In progress
[...]
```

Здесь показаны два подключения процесса ssh(1) к порту 22, за которыми следуют подключения, выполненные процессом curl(1): первое — к порту 53 (DNS), затем попытка подключения по протоколу IPv6 к порту 80, завершившаяся неудачей с сообщением «Network unreachable» («Сеть недоступна»), и далее — успешное подключение по протоколу IPv4. Значения столбцов:

- **PID:** идентификатор процесса, вызвавшего connect(2);
- **PROCESS:** имя процесса, вызвавшего connect(2);
- **FAM:** номер семейства адресов (см. значения номеров в разделе с описанием sofamily(8) выше);
- **ADDRESS:** IP-адрес;
- **PORT:** удаленный порт;
- **LAT(us):** задержка (продолжительность) выполнения системного вызова connect(2), подробнее об этом — ниже;
- **RESULT:** статус выполнения системного вызова.

Обратите внимание, что адреса IPv6 могут быть настолько длинными, что будут выходить за границы столбцов² (как показано в этом примере).

Этот инструмент использует точки трассировки системного вызова connect(2). Одно из преимуществ этого решения — они срабатывают в контексте процесса, поэтому можно достоверно узнать, какой процесс обратился к системному вызову.

¹ Немного истории: я создал этот инструмент для книги о DTrace в 2011 году [Gregg 11], а версию для bpftrace написал 9 апреля 2019 года.

² Вы спросите: почему я просто не сделал столбцы шире? Это привело бы к переносу всех строк в этом примере. Я всегда стараюсь отформатировать вывод во всех моих инструментах так, чтобы с настройками по умолчанию он не превышал 80 символов в ширину и без проблем умещался по ширине страниц в книгах, в слайдах, письмах, тикетах и чатах. Некоторые инструменты в ВСС поддерживают режим широкого вывода, чтобы обеспечить аккуратный вывод адресов IPv6.

Для сравнения, инструмент `tcpconnect(8)`, который появился позднее и трассирует соединения TCP, может неверно идентифицировать процесс, инициировавший соединение. Обращения к системному вызову `connect(2)` происходят относительно редко (если сравнивать их с пакетными событиями), поэтому ожидается, что у этого инструмента будет незначительный оверхед.

Задержка, сообщаемая инструментом, относится только к системному вызову `connect()`. Для некоторых приложений, например для `ssh(1)`, как показано в выводе выше, это время может включать время задержки в сети. Другие приложения могут использовать неблокирующие сокеты (`SOCK_NONBLOCK`), и тогда системный вызов `connect()` будет завершаться раньше, до того как соединение будет фактически установлено. Это можно увидеть в последней строке в выводе выше, где процесс `curl(1)` в ответ на попытку установить соединение получил результат «In progress» («В процессе»). Чтобы измерить полное время установки соединения при использовании таких неблокирующих сокетов, необходимо инструментировать большее количество событий. Пример этого — более поздний инструмент `soconnlat(8)`.

Исходный код `soconnect(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/in.h>
#include <linux/in6.h>

BEGIN
{
    printf("%-6s %-16s FAM %-16s %-5s %8s %s\n", "PID", "PROCESS",
        "ADDRESS", "PORT", "LAT(us)", "RESULT");
    // connect(2) может сообщать больше подробностей:
    @err2str[0] = "Success";
    @err2str[EPERM] = "Permission denied";
    @err2str[EINTR] = "Interrupted";
    @err2str[EBADF] = "Invalid sockfd";
    @err2str[EAGAIN] = "Routing cache insuff.";
    @err2str[EACCES] = "Perm. denied (EACCES)";
    @err2str[EFAULT] = "Sock struct addr invalid";
    @err2str[ENOTSOCK] = "FD not a socket";
    @err2str[EPROTOTYPE] = "Socket protocol error";
    @err2str[EAFNOSUPPORT] = "Address family invalid";
    @err2str[EADDRINUSE] = "Local addr in use";
    @err2str[EADDRNOTAVAIL] = "No port available";
    @err2str[ENETUNREACH] = "Network unreachable";
    @err2str[EISCONN] = "Already connected";
    @err2str[ETIMEDOUT] = "Timeout";
    @err2str[ECONNREFUSED] = "Connect refused";
    @err2str[EALREADY] = "Not yet completed";
    @err2str[EINPROGRESS] = "In progress";
}

tracepoint:syscalls:sys_enter_connect
/args->uservaddr->sa_family == AF_INET ||
  args->uservaddr->sa_family == AF_INET6/
```

```

{
    @sockaddr[tid] = args->servaddr;
    @start[tid] = nsecs;
}

tracepoint:syscalls:sys_exit_connect
/@start[tid]/
{
    $dur_us = (nsecs - @start[tid]) / 1000;
    printf("%-6d %-16s %-3d ", pid, comm, @sockaddr[tid]->sa_family);

    if (@sockaddr[tid]->sa_family == AF_INET) {
        $s = (struct sockaddr_in *)@sockaddr[tid];
        $port = ($s->sin_port >> 8) | (($s->sin_port << 8) & 0xff00);
        printf("%-16s %-5d %8d %s\n",
            ntop(AF_INET, $s->sin_addr.s_addr),
            $port, $dur_us, @err2str[- args->ret]);
    } else {
        $s6 = (struct sockaddr_in6 *)@sockaddr[tid];
        $port = ($s6->sin6_port >> 8) | (($s6->sin6_port << 8) & 0xff00);
        printf("%-16s %-5d %8d %s\n",
            ntop(AF_INET6, $s6->sin6_addr.in6_u.u6_addr8),
            $port, $dur_us, @err2str[- args->ret]);
    }

    delete(@sockaddr[tid]);
    delete(@start[tid]);
}

END
{
    clear(@start); clear(@err2str); clear(@sockaddr);
}

```

Этот код сохраняет указатель на структуру `sockaddr` в начале системного вызова, получая его из `args->servaddr`, и отметку времени, чтобы потом получить их на выходе из системного вызова. Структура `sockaddr` содержит сведения о соединении, но ее предварительно нужно преобразовать в `sockaddr_in` или в `sockaddr_in6`, в зависимости от версии IPv4 или IPv6, которая определяется полем `sin_family`. Для преобразования кодов ошибок используется таблица из описания `connect(2)` на странице справочного руководства.

Номер порта переключается с сети на хост с помощью побитовых операций.

10.3.5. `soaccept`

`soaccept(8)`¹ отображает попытки принять запросы на соединение по протоколу IP. Например:

¹ Немного истории: я создал этот инструмент для книги о DTrace в 2011 году [Gregg 11], а версию для `bpftrace` написал 13 апреля 2019 года.

```
# soaccept.bt
Attaching 6 probes...
PID    PROCESS      FAM ADDRESS          PORT  RESULT
4225   java          2  100.85.215.60    65062 Success
4225   java          2  100.85.54.16     11742 Success
4225   java          2  100.82.213.228   18500 Success
4225   java          2  100.85.209.40    20150 Success
4225   java          2  100.82.21.89     27278 Success
4225   java          2  100.85.192.93    32490 Success
[...]
```

Как показывает этот вывод, процесс Java принимает множество подключений с разных адресов. Номер порта здесь — это удаленный эфемерный порт. См. более позднюю версию инструмента `tcraaccept(8)` для отображения обоих портов конечных точек. Значения столбцов:

- **PID:** идентификатор процесса, вызвавшего `accept(2)`;
- **COMM:** имя процесса, вызвавшего `accept(2)`;
- **FAM:** номер семейства адресов (см. описание в разделе 10.3.2);
- **ADDRESS:** IP-адрес;
- **PORT:** удаленный порт;
- **RESULT:** статус выполнения системного вызова.

Этот инструмент использует точки трассировки системного вызова `accept(2)`. Как и точки трассировки системного вызова `connect(2)`, которые применяются в `sosconnect(8)`, они срабатывают в контексте процесса, поэтому можно достоверно узнать, какой процесс обратился к системному вызову `accept(2)`. Обращения к системному вызову `accept(2)` происходят относительно редко (если сравнивать их с пакетными событиями), поэтому ожидается, что у этого инструмента незначительный оверхед.

Исходный код `soaccept(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/in.h>
#include <linux/in6.h>

BEGIN
{
    printf("%-6s %-16s FAM %-16s %-5s %s\n", "PID", "PROCESS",
        "ADDRESS", "PORT", "RESULT");
    // accept(2) может сообщать больше подробностей:
    @err2str[0] = "Success";
    @err2str[EPERM] = "Permission denied";
    @err2str[EINTR] = "Interrupted";
    @err2str[EBADF] = "Invalid sockfd";
    @err2str[EAGAIN] = "None to accept";
    @err2str[ENOMEM] = "Out of memory";
    @err2str[EFAULT] = "Sock struct addr invalid";
```

```

    @err2str[EINVAL] = "Args invalid";
    @err2str[ENFILE] = "System FD limit";
    @err2str[EMFILE] = "Process FD limit";
    @err2str[EPROTO] = "Protocol error";
    @err2str[ENOTSOCK] = "FD not a socket";
    @err2str[EOPNOTSUPP] = "Not SOCK_STREAM";
    @err2str[ECONNABORTED] = "Aborted";
    @err2str[ENOBUFS] = "Memory (ENOBUFS)";
}

tracepoint:syscalls:sys_enter_accept,
tracepoint:syscalls:sys_enter_accept4
{
    @sockaddr[tid] = args->upeer_sockaddr;
}

tracepoint:syscalls:sys_exit_accept,
tracepoint:syscalls:sys_exit_accept4
/@sockaddr[tid]/
{
    $sa = (struct sockaddr *)@sockaddr[tid];
    if ($sa->sa_family == AF_INET || $sa->sa_family == AF_INET6) {
        printf("%-6d %-16s %-3d ", pid, comm, $sa->sa_family);
        $error = args->ret > 0 ? 0 : - args->ret;

        if ($sa->sa_family == AF_INET) {
            $s = (struct sockaddr_in *)@sockaddr[tid];
            $port = ($s->sin_port >> 8) |
                (($s->sin_port << 8) & 0xff00);
            printf("%-16s %-5d %s\n",
                ntop(AF_INET, $s->sin_addr.s_addr),
                $port, @err2str[$error]);
        } else {
            $s6 = (struct sockaddr_in6 *)@sockaddr[tid];
            $port = ($s6->sin6_port >> 8) |
                (($s6->sin6_port << 8) & 0xff00);
            printf("%-16s %-5d %s\n",
                ntop(AF_INET6, $s6->sin6_addr.in6_u.u6_addr8),
                $port, @err2str[$error]);
        }
    }

    delete(@sockaddr[tid]);
}

END
{
    clear(@err2str); clear(@sockaddr);
}

```

Этот код очень похож на код `soconnect(8)`: он точно так же преобразует и обрабатывает структуру `sockaddr` на выходе из системного вызова. Описания кодов ошибок были изменены согласно описанию на странице справочного руководства для `accept(2)`.

10.3.6. socketio

`socketio(8)`¹ отображает количество операций ввода/вывода с сокетами по процессам, направлениям, протоколам и портам. Например:

```
# socketio.bt
Attaching 4 probes...
^C
@io[sshd, 13348, write, TCP, 49076]: 1
@io[redis-server, 2583, write, TCP, 41154]: 5
@io[redis-server, 2583, read, TCP, 41154]: 5
@io[snmpd, 1242, read, NETLINK, 0]: 6
@io[snmpd, 1242, write, NETLINK, 0]: 6
@io[systemd-resolve, 1016, read, UDP, 53]: 52
@io[systemd-resolve, 1016, read, UDP, 0]: 52
@io[java, 3929, read, TCP, 6001]: 1367
@io[java, 3929, write, TCP, 8980]: 24979
@io[java, 3929, read, TCP, 8980]: 44462
```

Как показывает последняя строка в этом выводе, за время трассировки процесс Java с идентификатором 3929 выполнил 44 462 операции чтения из сокета TCP с номером порта 8980. Ключ карты состоит из пяти полей: имя процесса, идентификатор процесса, направление (чтение/запись), протокол и порт.

Для получения необходимой информации этот инструмент трассирует функции ядра `sock_recvmmsg()` и `sock_sendmmsg()`. Чтобы понять, почему я выбрал именно эти функции, рассмотрим структуру `socket_file_ops`, которая определена в `net/socket.c`:

```
/*
 * Файлы сокетов поддерживают набор "специальных" операций вдобавок к общим
 * для всех файлов. Они отсутствуют в структурах операций и выполняются
 * напрямую через мультиплексор socketcall().
 */

static const struct file_operations socket_file_ops = {
    .owner = THIS_MODULE,
    .llseek = no_llseek,
    .read_iter = sock_read_iter,
    .write_iter = sock_write_iter,
    [...]
}
```

Этот код сохраняет в структуре указатели на функции чтения и записи для сокета, `sock_read_iter()` и `sock_write_iter()`, и я сначала попытался трассировать их. Но тестирование с различными рабочими нагрузками показало, что при трассировке этих конкретных функций оставались незамеченными некоторые события. Комментарий в коде объясняет причину: есть дополнительные специальные операции, которые отсутствуют в структуре операций и тоже могут выполнять ввод/вывод

¹ Немного истории: первую версию этого инструмента с именем `socketio.d` я создал для книги о DTrace в 2011 году [Gregg 11], а версию для `bpfttrace` написал специально для этой книги 11 апреля 2019 года.

для сокетов. К ним относятся `sock_recvmsg()` и `sock_sendmsg()`, которые вызываются напрямую через системные вызовы или другими способами, в том числе через `sock_read_iter()` и `sock_write_iter()`. Это делает их общей точкой для трассировки ввода/вывода сокетов.

В системах, обрабатывающих объемный сетевой трафик, эти функции могут вызываться очень часто, что делает оверхед довольно заметным.

Исходный код `socketio(8)`:

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

kprobe:sock_recvmsg
{
    $sock = (struct socket *)arg0;
    $dport = $sock->sk->__sk_common.skc_dport;
    $dport = ($dport >> 8) | (($dport << 8) & 0xff00);
    @io[comm, pid, "read", $sock->sk->__sk_common.skc_prot->name, $dport] =
        count();
}

kprobe:sock_sendmsg
{
    $sock = (struct socket *)arg0;
    $dport = $sock->sk->__sk_common.skc_dport;
    $dport = ($dport >> 8) | (($dport << 8) & 0xff00);
    @io[comm, pid, "write", $sock->sk->__sk_common.skc_prot->name, $dport] =
        count();
}
```

Порт назначения хранится в формате с прямым порядком следования байтов (big endian) и преобразуется в формат с обратным порядком (little endian, для этого процессора x86) перед включением в карту @io¹. Этот сценарий можно изменить, чтобы он выводил переданные байты вместо счетчиков операций ввода/вывода. Примером может служить инструмент `socksize(8)`, описываемый далее.

`socketio(8)` основан на использовании зондов `kprobes`, которые являются деталями реализации ядра и могут измениться, что приведет к нарушению работы инструмента. Приложив больше усилий, можно переписать этот инструмент, используя точки трассировки системных вызовов. В этом случае придется трассировать системные вызовы `sendto(2)`, `sendmsg(2)`, `sendmmsg(2)`, `recvfrom(2)`, `recvmsg(2)` и `recvmmsg(2)`. Для некоторых типов сокетов, таких как сокет домена UNIX, также нужно предусмотреть трассировку семейства системных вызовов `read(2)` и `write(2)`. Было бы проще инструментировать точки трассировки в операциях ввода/вывода сокетов, но их пока нет.

¹ Чтобы этот инструмент работал на процессорах с прямым порядком следования байтов, он должен проверить, какой порядок следования байтов использует процессор, и выполнять преобразование, только если нужно. Например, с помощью `#ifdef LITTLE_ENDIAN`.

10.3.7. socksize

socksize(8)¹ отображает количество операций ввода/вывода и принятых/переданных байтов по процессам и направлениям. Вот пример вывода, полученный на производственном пограничном сервере с 48 процессорами:

```
# socksize.bt
Attaching 2 probes...
^C
```

```
@read_bytes[sshd]:  
[32, 64)           1 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
  
@read_bytes[java]:  
[0]                431 | @@@@  
[1]                 4   |  
[2, 4)             10  |  
[4, 8)            542  | @@@@@@  
[8, 16)          3445  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[16, 32)        2635  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[32, 64)       3497  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[64, 128)      776   | @@@@@@@@@@  
[128, 256)     916   | @@@@@@@@@@@@@@  
[256, 512)    3123  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[512, 1K)     4199  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[1K, 2K)      2972  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[2K, 4K)      1863  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[4K, 8K)      2501  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[8K, 16K)     1422  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[16K, 32K)     148  | @  
[32K, 64K)      29  |  
[64K, 128K)      6   |  
  
@write_bytes[sshd]:  
[32, 64)         1 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
  
@write_bytes[java]:  
[8, 16)          36   |  
[16, 32)          6   |  
[32, 64)       6131  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[64, 128)     1382  | @@@@@@@@@@@@@@@@@@  
[128, 256)      30   |  
[256, 512)      87   |  
[512, 1K)     169   | @  
[1K, 2K)       522   | @@@@  
[2K, 4K)     3607  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[4K, 8K)     2673  | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[8K, 16K)      394  | @@@@  
[16K, 32K)     815  | @@@@@@@@@@  
[32K, 64K)     175  | @  
[64K, 128K)      1   |  
[128K, 256K)      1   |
```

¹ Немного истории: я написал его для этой книги 12 апреля 2019 года, взяв за основу свой же инструмент bitesize, отображающий объем дискового ввода/вывода.

Основной трафик связан с приложением Java, и обе операции — чтение и запись — показывают бимодальное распределение размеров блоков ввода/вывода. Это может быть вызвано различными причинами: разными путями в коде или разным содержанием сообщений. Инструмент можно изменить и добавить в вывод трассировку стека и контекст приложения, чтобы ответить на этот вопрос.

`socksize(8)` трассирует функции ядра `sock_recvmmsg()` и `sock_sendmsg()`, как и `socketio(8)`. Исходный код `socksize(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/fs.h>
#include <net/socket.h>

kprobe:sock_recvmmsg,
kprobe:sock_sendmsg
{
    @socket[tid] = arg0;
}

kretprobe:sock_recvmmsg
{
    if (retval < 0xffffffff) {
        @read_bytes[comm] = hist(retval);
    }
    delete(@socket[tid]);
}

kretprobe:sock_sendmsg
{
    if (retval < 0xffffffff) {
        @write_bytes[comm] = hist(retval);
    }
    delete(@socket[tid]);
}

END
{
    clear(@socket);
}
```

Эти функции возвращают либо число переданных байтов, либо отрицательный код ошибки. Казалось бы, для фильтрации кодов ошибок можно использовать проверку `if (retval >= 0)`, но это невозможно, так как `retval` — это 64-битное целое число без знака, тогда как функции `sock_recvmmsg()` и `sock_sendmsg()` возвращают 32-битные целые числа со знаком. Проблему можно было бы решить приведением `retval` к правильному типу с помощью выражения `(int)retval`, но приведение к типу `int` пока недоступно в `bpftrace`, поэтому используется обходное решение, основанное на сравнении со значением `0xffffffff`¹.

¹ Приведение к типу `int` в `bpftrace` было реализовано Басом Смитом (Bas Smit) и вскоре должно быть выпущено. См. `bpftrace` PR #772.

При желании можно добавить дополнительные ключи, например PID, номер порта и трассировку стека в пространстве пользователя. Также карты `hist()` можно заменить картами `stats()`, чтобы получить сводную информацию другого типа:

```
# socksize.bt
Attaching 2 probes...
^C

@read_bytes[sshd]: count 1, average 36, total 36
@read_bytes[java]: count 19874, average 1584, total 31486578

@write_bytes[sshd]: count 1, average 36, total 36
@write_bytes[java]: count 11061, average 3741, total 41379939
```

Этот пример показывает количество операций ввода/вывода («count»), средний размер блока в байтах («average») и общий объем в байтах («total»). В период, когда выполнялась трассировка, процесс Java записал 41 Мбайт.

10.3.8. sormem

`sormem(8)`¹ трассирует размеры очередей приема сокетов и отображает их заполненность относительно настраиваемого предела в виде гистограммы. Когда размер очереди достигает предела, вновь поступающие пакеты просто отбрасываются, что вызывает проблемы с производительностью. Вот пример вывода, полученный на производственном пограничном сервере:

```
# sormem.bt
Attaching 4 probes...
Tracing socket receive buffer size. Hit Ctrl-C to end.
^C

@rmem_alloc:
[0]          72870 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1]           0 |
[2, 4)       0 |
[4, 8)       0 |
[8, 16)      0 |
[16, 32)     0 |
[32, 64)     0 |
[64, 128)    0 |
[128, 256)   0 |
[256, 512)   0 |
[512, 1K)    113831 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)     113 |
[2K, 4K)     105 |
[4K, 8K)     99221 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[8K, 16K)    26726 | @@@@@@@@@@@@@@@@@@
[16K, 32K)   58028 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32K, 64K)   31336 | @@@@@@@@@@@@@@@@@@
```

¹ Немного истории: я написал его для этой книги 14 апреля 2019 года.

```

[64K, 128K)      15039 |@@@@@
[128K, 256K)     6692 |@@@
[256K, 512K)      697 |
[512K, 1M)        91 |
[1M, 2M)          45 |
[2M, 4M)          80 |

@rmem_limit:
[64K, 128K)      14447 |@
[128K, 256K)     262 |
[256K, 512K)      0 |
[512K, 1M)        0 |
[1M, 2M)          0 |
[2M, 4M)          0 |
[4M, 8M)          0 |
[8M, 16M)       410158 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16M, 32M)        7 |

```

@rmem_alloc сообщает, сколько памяти было выделено для приемного буфера. @rmem_limit — это предельный размер приемного буфера, настраиваемый с помощью sysctl(8). Как показывает этот пример, предел часто находится в диапазоне от 8 до 16 Мбайт, тогда как объем фактически выделяемой памяти намного меньше, обычно от 512 байт до 256 Кбайт.

Вот искусственный пример, объясняющий это: здесь с помощью iperf(1) выполняется тестирование пропускной способности с указанным параметром tcp_rmem для sysctl(1) (будьте осторожны с настройками, так как большие размеры могут вызвать задержку из-за коллапса и объединения буферов сокетов [105]):

```

# sysctl -w net.ipv4.tcp_rmem='4096 32768 10485760'
# sormem.bt
Attaching 4 probes...
Tracing socket receive buffer size. Hit Ctrl-C to end.
[...]

@rmem_limit:
[64K, 128K)      17 |
[128K, 256K)    26319 |@@@@
[256K, 512K)     31 |
[512K, 1M)       0 |
[1M, 2M)         26 |
[2M, 4M)         0 |
[4M, 8M)         8 |
[8M, 16M)     320047 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

```

И повторные тесты с уменьшенным максимальным значением rmem:

```

# sysctl -w net.ipv4.tcp_rmem='4096 32768 100000'
# sormem.bt
Attaching 4 probes...
Tracing socket receive buffer size. Hit Ctrl-C to end.
[...]

@rmem_limit:

```


встретился только один раз за время трассировки, и в этом единственном случае задержка соединения составила от 32 до 64 микросекунд. Второй стек соответствует более чем 200 случаям создания соединений из Java с задержкой от 128 микросекунд до 2 миллисекунд. Но эта вторая трассировка стека содержит только один фрейм «__connect+71». Причина в том, что это приложение на Java использует библиотеку `libc`, скомпилированную без указателей фреймов. Как исправить эту проблему, пойдет речь в разделе 13.2.9.

Задержка соединения показывает, сколько времени потребовалось, чтобы установить соединение по сети, включая процедуру согласования TCP. В это время также входит задержка обработки пакета SYN на удаленном узле и отправки ответа: обычно это происходит очень быстро в контексте обработки прерывания, поэтому в задержке соединения, как правило, преобладает время приема/передачи по сети.

Этот инструмент трассирует семейство системных вызовов `connect(2)`, `select(2)` и `poll(2)` по их точкам трассировки. Оверхед может стать ощутимым в высоконагруженных системах, которые часто обращаются к системным вызовам `select(2)` и `poll(2)`.

Исходный код `soconnlat(8)`:

```
#!/usr/local/bin/bpftrace

#include <asm-generic/errno.h>
#include <linux/in.h>

BEGIN
{
    printf("Tracing IP connect() latency with ustacks. Ctrl-C to end.\n");
}

tracepoint:syscalls:sys_enter_connect
/args->useraddr->sa_family == AF_INET ||
args->useraddr->sa_family == AF_INET6/
{
    @conn_start[tid] = nsecs;
    @conn_stack[tid] = ustack();
}

tracepoint:syscalls:sys_exit_connect
/@conn_start[tid] && args->ret != - EINPROGRESS/
{
    $dur_us = (nsecs - @conn_start[tid]) / 1000;
    @us[@conn_stack[tid], comm] = hist($dur_us);
    delete(@conn_start[tid]);
    delete(@conn_stack[tid]);
}

tracepoint:syscalls:sys_exit_poll*,
tracepoint:syscalls:sys_exit_epoll*,
tracepoint:syscalls:sys_exit_select*,
tracepoint:syscalls:sys_exit_pselect*
```

```

/@conn_start[tid] && args->ret > 0/
{
    $dur_us = (nsecs - @conn_start[tid]) / 1000;
    @us[@conn_stack[tid], comm] = hist($dur_us);
    delete(@conn_start[tid]);
    delete(@conn_stack[tid]);
}

END
{
    clear(@conn_start); clear(@conn_stack);
}

```

Этот код решает проблему, упоминавшуюся выше в описании инструмента `sosconnect(8)`. Задержка соединения измеряется как время выполнения системного вызова `connect(2)`, если он возвращает код результата, отличный от `EINPROGRESS`, в противном случае истинное время установки соединения определяется, когда системный вызов `poll(2)` или `select(2)` успешно находит событие для этого файлового дескриптора. С этой целью инструмент должен был бы запомнить аргументы на входе в системный вызов `poll(2)` или `select(2)`, а затем проверить их на выходе, чтобы убедиться, что дескриптор файла сокета соединения соответствует событию. Но вместо этого инструмент использует огромное упрощение, предполагая связь между первым успешным вызовом `poll(2)` или `select(2)` после вызова `connect(2)`, вернувшего `EINPROGRESS` в том же потоке. Часто это действительно так и есть, но имейте в виду, что инструмент может ошибаться, если приложение вызвало `connect(2)`, а затем — в том же потоке — получило событие для другого файлового дескриптора, также ожидавшееся. Вы можете усовершенствовать инструмент или изучить использование этих системных вызовов вашим приложением, чтобы увидеть, насколько правдоподобным может быть такой сценарий.

Например, вот результаты подсчета количества файловых дескрипторов, события для которых ожидают приложения через `poll(2)` на производственном пограничном сервере:

```

# bpftrace -e 't:syscalls:sys_enter_poll { @[comm, args->nfds] = count(); }'
Attaching 1 probe...
^C

@[python3, 96]: 181
@[java, 1]: 10300

```

Во время трассировки процесс Java вызывал `poll(2)` только для одного файлового дескриптора, поэтому только что описанный сценарий кажется маловероятным, если только `poll(2)` не вызывается отдельно для разных дескрипторов файлов. Аналогичные тесты можно выполнить для других системных вызовов `poll(2)` и `select(2)`.

В вывод также попал процесс `python3`, вызвавший `poll(2)` для... 96 дескрипторов файлов! Добавив `pid` в ключ карты, чтобы идентифицировать конкретный процесс `python3`, и изучив файловые дескрипторы в `lsof(8)`, я обнаружил, что он действительно открывает 96 файловых дескрипторов, но делает это по ошибке и часто

опрашивает их на производственных серверах. Я смог исправить эту ошибку и сэкономить несколько тактов процессора¹.

10.3.10. so1stbyte

`so1stbyte(8)`² определяет время от вызова `connect(2)` до получения первого байта данных. Если `soconnlat(8)` измеряет задержку создания соединения, то `so1stbyte(8)` добавляет еще и время, необходимое удаленному приложению, чтобы подготовить и отправить данные. Это помогает понять, насколько загружен удаленный узел, а если измерять достаточно долго, можно выявить периоды, когда удаленные узлы загружены сильнее и имеют большую задержку. Например:

```
# so1stbyte.bt
Attaching 21 probes...
Tracing IP socket first-read-byte latency. Ctrl-C to end.
^C

@us[java]:
[256, 512)      4 |
[512, 1K)       5 |@
[1K, 2K)        34 |@@@@@
[2K, 4K)       212 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)       260 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[8K, 16K)       35 |@@@@@@@
[16K, 32K)      6 |@
[32K, 64K)      1 |
[64K, 128K)     0 |
[128K, 256K)    4 |
[256K, 512K)    3 |
[512K, 1M)      1 |
```

Мы видим, что этот процесс Java получает первый байт данных от начала установки нового соединения в течение от 1 до 16 миллисекунд.

Этот инструмент использует точки трассировки семейства системных вызовов `connect(2)`, `read(2)` и `recv(2)`. Оверхед может стать ощутимым в высоконагруженных системах, которые часто обращаются к этим системным вызовам.

Исходный код `so1stbyte(8)`:

```
#!/usr/local/bin/bpftrace

#include <asm-generic/errno.h>
#include <linux/in.h>

BEGIN
```

¹ Но прежде я проверил время безотказной работы сервера, количество CPU и потребление CPU процессом с помощью `ps(1)` (процесс, как предполагалось, должен большую часть времени простаивать), чтобы найти, какая доля вычислительных ресурсов тратится впустую: получилось всего 0.02 %.

² Немного истории: первую версию с именем `so1stbyte.d` я создал для книги о DTrace в 2011 году [Gregg 11], а эту версию я написал 16 апреля 2019 года.

```

{
    printf("Tracing IP socket first-read-byte latency. Ctrl-C to end.\n");
}

tracepoint:syscalls:sys_enter_connect
/args->uservaddr->sa_family == AF_INET ||
args->uservaddr->sa_family == AF_INET6/
{
    @connfd[tid] = args->fd;
    @connstart[pid, args->fd] = nsecs;
}

tracepoint:syscalls:sys_exit_connect
{
    if (args->ret != 0 && args->ret != - EINPROGRESS) {
        // connect() потерпел неудачу, удалить флаг, если есть
        delete(@connstart[pid, @connfd[tid]]);
    }
    delete(@connfd[tid]);
}

tracepoint:syscalls:sys_enter_close
/@connstart[pid, args->fd]/
{
    // read так и не был вызван
    delete(@connstart[pid, @connfd[tid]]);
}

tracepoint:syscalls:sys_enter_read,
tracepoint:syscalls:sys_enter_readv,
tracepoint:syscalls:sys_enter_pread*,
tracepoint:syscalls:sys_enter_recvfrom,
tracepoint:syscalls:sys_enter_recvmsg,
tracepoint:syscalls:sys_enter_recvmsg
/@connstart[pid, args->fd]/
{
    @readfd[tid] = args->fd;
}

tracepoint:syscalls:sys_exit_read,
tracepoint:syscalls:sys_exit_readv,
tracepoint:syscalls:sys_exit_pread*,
tracepoint:syscalls:sys_exit_recvfrom,
tracepoint:syscalls:sys_exit_recvmsg,
tracepoint:syscalls:sys_exit_recvmsg
/@readfd[tid]/
{
    $fd = @readfd[tid];
    @us[comm, pid] = hist((nsecs - @connstart[pid, $fd]) / 1000);
    delete(@connstart[pid, $fd]);
    delete(@readfd[tid]);
}

END
{
    clear(@connstart); clear(@connfd); clear(@readfd);
}

```

Этот инструмент записывает начальную отметку времени на входе в `connect(2)` в карту `@connstart` с ключом, состоящим из идентификатора процесса и дескриптора файла. Если `connect(2)` потерпел неудачу (за исключением случаев, когда он является неблокирующим и вернул `EINPROGRESS`) или было обращение к системному вызову `close(2)`, то отметка времени удаляется и трассировка соединения прекращается. При первом входе в системный вызов `read` или `recv` с записанным ранее файловым дескриптором сокета в `@readfd` записывается этот дескриптор, чтобы его можно было получить на выходе из системного вызова, а в `@connstart` — время начала чтения.

Этот интервал практически идентичен времени TCP до получения первого байта, описанному выше, с той лишь разницей, что он включает еще и продолжительность выполнения `connect(2)`.

Чтобы перехватить первую операцию чтения из сокета, нужно инструментировать множество точек трассировки, что добавляет оверхед ко всем этим путям чтения. Этот оверхед и количество трассируемых событий можно уменьшить, переключившись на использование зондов `kprobes`, таких как `sock_recvmsg()`, и используя указатель на структуру `sock` как уникальный идентификатор вместо пары PID и FD. Проблема в том, что зонды `kprobes` нестабильны и могут изменяться с выходом новых версий ядра.

10.3.11. tcpconnect

`tcpconnect(8)`¹ — это инструмент BCC и `bpftool` для трассировки новых активных TCP-соединений. В отличие от инструментов, представленных выше, `tcpconnect(8)` и инструменты, описанные ниже, глубже погружаются в сетевой стек, не ограничиваясь трассировкой системных вызовов. Имя `tcpconnect(8)` было дано в честь системного вызова `connect(2)`. Такие соединения часто называют *исходящими*, даже при том, что могут устанавливаться соединения с `localhost`.

`tcpconnect(8)` помогает оценить характеристики рабочей нагрузки: определить, кто с кем соединяется и как часто. Вот вывод BCC-версии `tcpconnect(8)`:

```
# tcpconnect.py -t
TIME(s)  PID  COMM  IP  SADDR          DADDR          DPORT
0.000    4218 java   4   100.1.101.18   100.2.51.232   6001
0.011    4218 java   4   100.1.101.18   100.2.135.216  6001
0.072    4218 java   4   100.1.101.18   100.2.135.94   6001
0.073    4218 java   4   100.1.101.18   100.2.160.87   8980
0.124    4218 java   4   100.1.101.18   100.2.177.63   6001
0.212    4218 java   4   100.1.101.18   100.2.58.22    6001
0.214    4218 java   4   100.1.101.18   100.2.43.148   6001
[...]
```

¹ Немного истории: аналогичный инструмент под названием `tcpconnect.d` я создал для книги о DTrace в 2011 году [Gregg 11], версию для BCC я написал 25 сентября 2015 года и версию для `bpftool` — `tcpconnect-tp(8)` — 7 апреля 2019 года.

При трассировке было обнаружено несколько подключений разных удаленных узлов к одному и тому же порту 6001. Значения столбцов:

- **TIME(s)**: время, когда было принято соединение, отсчитывается от первого увиденного события;
- **PID**: идентификатор процесса, установившего соединение. За неимением лучшего варианта в этом столбце выводится идентификатор текущего процесса — на уровне TCP события могут происходить вне контекста процесса. Для надежной идентификации процессов используйте трассировку сокетов;
- **COMM**: имя процесса, установившего соединение. Как и в случае с PID, для надежной идентификации процессов используйте трассировку сокетов;
- **IP**: версия протокола IP;
- **SADDR**: адрес отправителя;
- **DADDR**: адрес получателя;
- **DPORT**: порт получателя.

Инструмент поддерживает обе версии протокола, IPv4 и IPv6, но адреса IPv6 бывают настолько длинными, что вывод станет неаккуратным.

В своей работе инструмент трассирует события, связанные с созданием новых сеансов TCP, а не с отправкой отдельных пакетов. На производственном сервере, где был получен этот пример, в секунду отправляется около 50 000 пакетов, тогда как новых сеансов TCP создается около 350 в секунду. За счет трассировки событий уровня сеанса вместо пакетов оверхед получается ниже примерно в сто раз и становится незначительным.

Сейчас версия для ВСС трассирует функции ядра `tcp_v4_connect()` и `tcp_v6_connect()`. В будущем предполагается переключиться на использование точки трассировки `sock:inet_sock_set_state`, если она будет доступна.

BCC

Порядок использования:

```
tcpconnect [options]
```

Параметры `options`:

- **-t**: включить вывод столбца с отметками времени;
- **-p PID**: трассировать только этот процесс;
- **-P PORT[,PORT,...]**: трассировать только эти порты.

bpfttrace

Ниже приведен исходный код `tcpconnect-tp(8)` версии `tcpconnect(8)` для `bpfttrace`, которая использует точку трассировки `sock:inet_sock_set_state` `tracepoint`:

```
#!/usr/local/bin/bpftrace

#include <net/tcp_states.h>
#include <linux/socket.h>

BEGIN
{
    printf("%-8s %-6s %-16s %-3s ", "TIME", "PID", "COMM", "IP");
    printf("%-15s %-15s %-5s\n", "SADDR", "DADDR", "DPORT");
}

tracepoint:sock:inet_sock_set_state
/args->oldstate == TCP_CLOSE && args->newstate == TCP_SYN_SENT/
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %-3d ", pid, comm, args->family == AF_INET ? 4 : 6);
    printf("%-15s %-15s %-5d\n", ntop(args->family, args->saddr),
        ntop(args->family, args->daddr), args->dport)
}
```

Здесь проверяется переход из состояния TCP_CLOSE в TCP_SYN_SENT.

В репозитории bpftrace есть версия tcpconnect(8)¹ для старых ядер Linux, где нет точки трассировки sock:inet_sock_set_state. Она трассирует функцию ядра tcp_connect().

10.3.12. tcрасцепт

tcрасцепт(8)² — это инструмент ВСС и bpftrace для трассировки новых пассивных TCP-соединений; аналог инструмента tcpconnect(8). Имя tcрасцепт(8) было дано в честь системного вызова accept(2). Такие соединения часто называют *входящими*, даже при том, что они могут исходить от localhost. Как и tcpconnect(8), этот инструмент помогает оценить характеристики рабочей нагрузки: определить, кто подключается к локальной системе и как часто.

Ниже показан пример вывода ВСС-версии tcрасцепт(8) с параметром `-t` для включения столбца с отметками времени, полученный на производственном экземпляре с 48 процессорами:

¹ Немного истории: она была создана Дейлом Хамелем 23 ноября 2018 года, специально для этой версии он также добавил в bpftrace встроенную функцию ntop().

² Немного истории: аналогичный инструмент под названием tcрасцепт.d я создал для книги о DTrace в 2011 году [Gregg 11]. Еще раньше, в 2006 году, работая над провайдером DTrace TCP [106], я написал инструменты tcрасцепт1.d и tcрасцепт2.d, которые подсчитывали соединения. Помню, как я допоздна засиделся над ними, стараясь успеть подготовить их для показа на конференции на SEC2006 в Сан-Франциско [107], из-за чего проспал и едва успел добраться до места, где должен был выступать. Версию для ВСС я написал 13 октября 2015 года, а tcpconnect-tp(8), версию для bpftrace, 7 апреля 2019 года.

```
# tcpaccept -t
TIME(s)  PID    COMM   IP  RADDR          RPORT  LADDR          LPORT
0.000    4218   java    4   100.2.231.20    53422  100.1.101.18   6001
0.004    4218   java    4   100.2.236.45    36400  100.1.101.18   6001
0.013    4218   java    4   100.2.221.222   29836  100.1.101.18   6001
0.014    4218   java    4   100.2.194.78    40416  100.1.101.18   6001
0.016    4218   java    4   100.2.239.62    53422  100.1.101.18   6001
0.016    4218   java    4   100.2.199.236   28790  100.1.101.18   6001
0.021    4218   java    4   100.2.192.209   35840  100.1.101.18   6001
0.022    4218   java    4   100.2.215.219   21450  100.1.101.18   6001
0.026    4218   java    4   100.2.231.176   47024  100.1.101.18   6001
[...]
```

В этом выводе видно множество новых подключений к локальному порту 6001 с разных удаленных адресов, принятых процессом Java с PID 4218. Значения столбцов аналогичны столбцам в `tcpconnect(8)` с некоторыми отличиями:

- **RADDR**: удаленный адрес;
- **RPORT**: удаленный порт;
- **LADDR**: локальный адрес;
- **LPORT**: локальный порт.

Этот инструмент трассирует функцию ядра `inet_csk_accept()`. Имя кажется несколько необычным по сравнению с именами других высокоуровневых функций TCP, поэтому может возникнуть вопрос, почему я выбрал его. Причина проста: это функция `accept` из структуры `tcp_prot` (`net/ipv4 / tcp_ipv4.c`):

```
struct proto tcp_prot = {
    .name      = "TCP",
    .owner     = THIS_MODULE,
    .close     = tcp_close,
    .pre_connect = tcp_v4_pre_connect,
    .connect   = tcp_v4_connect,
    .disconnect = tcp_disconnect,
    .accept    = inet_csk_accept,
    .ioctl     = tcp_ioctl,
    [...]
}
```

Адреса IPv6 тоже поддерживаются инструментом, но если эти адреса окажутся слишком длинными, вывод станет неаккуратным. Вот пример, полученный на другом производственном сервере:

```
# tcpaccept -t
TIME(s)  PID    COMM   IP  RADDR          LADDR          LPORT
0.000    7013   java    6   ::ffff:100.1.54.4 ::ffff:100.1.58.46 13562
0.103    7013   java    6   ::ffff:100.1.7.19 ::ffff:100.1.58.46 13562
0.202    7013   java    6   ::ffff:100.1.58.59 ::ffff:100.1.58.46 13562
[...]
```

Здесь адреса IPv4 отображаются в адреса IPv6.

BCC

Порядок использования:

```
tcpaccept [options]
```

tcpaccept(8) поддерживает те же параметры *options*, что и tcpconnect(8), в том числе:

- **-t**: включить вывод столбца с отметками времени;
- **-p PID**: трассировать только этот процесс;
- **-P PORT[,PORT,...]**: трассировать только эти локальные порты.

bpfftrace

Ниже приведен исходный код tcpaccept-tp(8) версии tcpaccept(8) для bpfftrace, разработанной специально для этой книги и использующей точку трассировки sock:inet_sock_set_state:

```
#!/usr/local/bin/bpfftrace

#include <net/tcp_states.h>
#include <linux/socket.h>

BEGIN
{
    printf("%-8s %-3s %-14s %-5s %-14s %-5s\n", "TIME", "IP",
        "RADDR", "RPORT", "LADDR", "LPORT");
}

tracepoint:sock:inet_sock_set_state
/args->oldstate == TCP_SYN_RECV && args->newstate == TCP_ESTABLISHED/
{
    time("%H:%M:%S ");
    printf("%-3d %-14s %-5d %-14s %-5d\n", args->family == AF_INET ? 4 : 6,
        ntop(args->family, args->daddr), args->dport,
        ntop(args->family, args->saddr), args->sport);
}
```

Поскольку ожидается, что во время изменения состояния TCP процесс не будет выполняться процессором, встроенные значения *pid* и *comm* исключены из этой версии. Пример вывода:

```
# tcpaccept-tp.bt
Attaching 2 probes...
TIME IP RADDR RPORT LADDR LPORT
07:06:46 4 127.0.0.1 63998 127.0.0.1 28527
07:06:47 4 127.0.0.1 64002 127.0.0.1 28527
07:06:48 4 127.0.0.1 64004 127.0.0.1 28527
[...]
```

В репозитории `bpftrace` есть версия `tcrascept(8)`¹, делающая динамическую трассировку функции `inet_csk_accept()`, используемой в версии для ВСС. Предполагается, что эта функция будет вызываться прикладными процессами синхронно, поэтому идентификатор PID и имя процесса выводятся с использованием встроенных значений `pid` и `comm`. Вот соответствующий фрагмент исходного кода:

```
[...]
kretprobe:inet_csk_accept
{
    $sk = (struct sock *)retval;
    $inet_family = $sk->__sk_common.skc_family;

    if ($inet_family == AF_INET || $inet_family == AF_INET6) {
        $daddr = ntop(0);
        $saddr = ntop(0);
        if ($inet_family == AF_INET) {
            $daddr = ntop($sk->__sk_common.skc_daddr);
            $saddr = ntop($sk->__sk_common.skc_rcv_saddr);
        } else {
            $daddr = ntop(
                $sk->__sk_common.skc_v6_daddr.in6_u.u6_addr8);
            $saddr = ntop(
                $sk->__sk_common.skc_v6_rcv_saddr.in6_u.u6_addr8);
        }
        $lport = $sk->__sk_common.skc_num;
        $dport = $sk->__sk_common.skc_dport;
        $qlen = $sk->sk_ack_backlog;
        $qmax = $sk->sk_max_ack_backlog;
    }
[...]
```

Программа извлекает информацию о протоколе из структуры `sock`, а также сведения об очередях запросов на соединение. Так можно расширить инструменты для получения и отображения дополнительных сведений. Вывод информации об очередях запросов на соединение был добавлен для диагностики проблемы в компании *Shopify*, проявлявшейся в деградации производительности *Redis* при пиковой нагрузке: было обнаружено, что это связано с отбрасыванием запросов². Добавление столбца в `tcrascept.bt` позволило увидеть текущую длину очереди запросов и помогло определить необходимые характеристики.

Будущее изменение в правилах определения области видимости переменных в `bpftrace` может привести к тому, что переменные, инициализированные в операторах `if`, будут видимы только в блоке этого оператора. Это нарушит работоспособность этой программы, потому что она использует переменные `$daddr` и `$saddr` за пределами оператора `if`. Чтобы избежать такого, программа заранее инициализирует эти переменные как `ntop(0)` (`ntop(0)` возвращает тип `inet`, который выводится в виде строки). Эта инициализация не требуется в текущей версии `bpftrace` (0.9.1), но была добавлена, чтобы сохранить работоспособность программы в будущем.

¹ Немного истории: она была создана Дейлом Хамелем 23 ноября 2018 года.

² Пример был представлен Дейлом Хамелем.

10.3.13. tcplife

`tcplife(8)`¹ — это инструмент BCC и `bpfttrace` для трассировки продолжительности сеансов TCP. Он отображает продолжительность каждого сеанса, адреса, объем трафика и, если возможно, идентификатор и имя процесса.

Ниже показан пример вывода BCC-версии `tcplife(8)`, полученный на производственном экземпляре с 48 процессорами:

```
# tcplife
PID  COMM      LADDR      LPORT  RADDR      RPORT  TX_KB  RX_KB  MS
4169  java      100.1.111.231 32648  100.2.0.48  6001    0      0    3.99
4169  java      100.1.111.231 32650  100.2.0.48  6001    0      0    4.10
4169  java      100.1.111.231 32644  100.2.0.48  6001    0      0    8.41
4169  java      100.1.111.231 40158  100.2.116.192 6001    7     33  3590.91
4169  java      100.1.111.231 56940  100.5.177.31 6101    0      0    2.48
4169  java      100.1.111.231 6001   100.2.176.45 49482   0      0   17.94
4169  java      100.1.111.231 18926  100.5.102.250 6101    0      0    0.90
4169  java      100.1.111.231 44530  100.2.31.140 6001    0      0    2.64
4169  java      100.1.111.231 44406  100.2.8.109  6001   11     28  3982.11
34781 sshd      100.1.111.231 22     100.2.17.121 41566   5      7   2317.30
4169  java      100.1.111.231 49726  100.2.9.217  6001   11     28  3938.47
4169  java      100.1.111.231 58858  100.2.173.248 6001    9     30  2820.51
[...]
```

В этом выводе показана последовательность соединений, часть из которых короткоживущие (жившие менее 20 миллисекунд), а часть — долгоживущие (действовавшие дольше 3 секунд), согласно столбцу «MS», отображающему продолжительность в миллисекундах. Это пул серверов приложений, прослушивающий порт 6001. Большинство сеансов в этом примере соответствуют соединениям с портом 6001 на удаленных серверах приложений, и только одно соединение установлено с локальным портом 6001. Также в процессе трассировки был сеанс `ssh` с локальным портом 22, который прослушивает демон `sshd`, — это входящий сеанс.

Инструмент трассирует события изменения состояния сокета TCP и выводит обобщенные данные, когда состояние изменяется на `TCP_CLOSE`. События изменения состояния случаются с намного меньшей частотой, чем частота следования пакетов, что делает этот подход гораздо менее затратным в плане оверхеда и позволяет использовать `tcplife(8)` для непрерывного анализа потока TCP на производственных серверах Netflix.

¹ Немного истории: все началось с твита Джулии Эванс: «Мне бы очень хотелось иметь инструмент командной строки, который выдавал бы мне статистику по длине TCP-соединений на заданном порте» [108]. В ответ на это я создал `tcplife(8)` как инструмент BCC 18 октября 2016 года, а версию `bpfttrace` создал 17 апреля 2019 года после того, как утром смерджил `bpfttrace` от Матеуса Марчини. Это один из самых популярных инструментов, созданных мной. Он составляет основу нескольких высокоуровневых графических интерфейсов, так как позволяет получить статистику сетевого потока, которую можно визуализировать в виде ориентированных графов.

Первоначальная версия `tcplife(8)` трассировала функцию ядра `tcp_set_state()` с помощью зондов ядра `kprobes`. В Linux 4.16 для этой цели была добавлена точка трассировки `sock:inet_sock_set_state`. Инструмент `tcplife(8)` использует эту точку трассировки, если она доступна, в противном случае возвращается к использованию `kprobes`. Между этими решениями есть небольшая разница, которую можно увидеть, запустив такой однострочный сценарий. Он подсчитывает события изменения состояния TCP:

```
# bpfftrace -e 'k:tcp_set_state { @kprobe[arg1] = count(); }
      t:sock:inet_sock_set_state { @tracepoint[args->newstate] = count(); }'
Attaching 2 probes...
^C

@kprobe[4]: 12
@kprobe[5]: 12
@kprobe[9]: 13
@kprobe[2]: 13
@kprobe[8]: 13
@kprobe[1]: 25
@kprobe[7]: 25

@tracepoint[3]: 12
@tracepoint[4]: 12
@tracepoint[5]: 12
@tracepoint[2]: 13
@tracepoint[9]: 13
@tracepoint[8]: 13
@tracepoint[7]: 25
@tracepoint[1]: 25
```

Заметили? Зонд `tcp_set_state()` не видит состояния с кодом 3, соответствующим состоянию `TCP_SYN_RECV`. Причина в том, что зонд `kprobe` — это деталь реализации ядра, а ядро никогда не вызывает `tcp_set_state()` с аргументом `TCP_SYN_RECV`, потому что в этом нет необходимости. Эта деталь реализации принадлежит к числу деталей, обычно скрытых от конечных пользователей. Но после добавления точки трассировки, специально предназначенной для выявления изменений состояния, этот недостаток был обнаружен и исправлен, чтобы ее имя соответствовало ожиданиям.

BCC

Порядок использования:

```
tcplife [options]
```

Параметры `options`:

- **-t**: включать в вывод отметки времени (в формате ЧЧ:ММ:СС);
- **-w**: широкие столбцы (лучше подходят для вывода адресов IPv6);
- **-p PID**: трассировать только этот процесс;

- **-L PORT[,PORT[,...]]**: трассировать только сеансы с этими локальными портами;
- **-D PORT[,PORT[,...]]**: трассировать только сеансы с этими удаленными портами.

bpftrace

Ниже приведен исходный код версии для bpftrace, разработанной специально для этой книги, в которой обобщены основные возможности инструмента. Эта версия использует зонд ядра kprobe tcp_set_state() и, соответственно, может использоваться со старыми версиями ядер. Она не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpftrace

#include <net/tcp_states.h>
#include <net/sock.h>
#include <linux/socket.h>
#include <linux/tcp.h>

BEGIN
{
    printf("%-5s %-10s %-15s %-5s %-15s %-5s ", "PID", "COMM",
        "LADDR", "LPORT", "RADDR", "RPORT");
    printf("%5s %5s %s\n", "TX_KB", "RX_KB", "MS");
}

kprobe:tcp_set_state
{
    $sk = (struct sock *)arg0;
    $newstate = arg1;

    /*
     * Этот инструмент включает в вывод значения PID и comm. Однако иногда
     * они могут содержать недостоверные значения. Вот как он действует:
     * - запоминает время любого состояния < TCP_FIN_WAIT1
     *   обратите внимание, что некоторые переходы между состояниями могут
     *   не попадать в этот зонд
     * - кэширует контекст задачи по:
     *   TCP_SYN_SENT: трассировка со стороны клиента
     *   TCP_LAST_ACK: завершение соединения с клиентом со стороны сервера
     * - выводит накопленную информацию по TCP_CLOSE:
     *   извлекает контекст задачи, если кэшировался, или использует
     *   текущую задачу
     */

    // запомнить первую отметку времени для этого сокета
    if ($newstate < TCP_FIN_WAIT1 && @birth[$sk] == 0) {
        @birth[$sk] = nsecs;
    }

    // запомнить PID и comm по SYN_SENT
    if ($newstate == TCP_SYN_SENT || $newstate == TCP_LAST_ACK) {
        @skpid[$sk] = pid;
        @skcomm[$sk] = comm;
    }
}
```

```

// окончание сеанса: вычислить продолжительность и вывести
if ($newstate == TCP_CLOSE && @birth[$sk]) {
    $delta_ms = (nsecs - @birth[$sk]) / 1000000;
    $lport = $sk->__sk_common.skc_num;
    $dport = $sk->__sk_common.skc_dport;
    $dport = ($dport >> 8) | (($dport << 8) & 0xff00);
    $tp = (struct tcp_sock *)$sk;
    $pid = @skpid[$sk];
    $comm = @skcomm[$sk];
    if ($comm == "") {
        // не было кэшировано, использовать текущую задачу
        $pid = pid;
        $comm = comm;
    }

    $family = $sk->__sk_common.skc_family;
    $saddr = ntop(0);
    $daddr = ntop(0);
    if ($family == AF_INET) {
        $saddr = ntop(AF_INET, $sk->__sk_common.skc_rcv_saddr);
        $daddr = ntop(AF_INET, $sk->__sk_common.skc_daddr);
    } else {
        // AF_INET6
        $saddr = ntop(AF_INET6,
            $sk->__sk_common.skc_v6_rcv_saddr.in6_u.u6_addr8);
        $daddr = ntop(AF_INET6,
            $sk->__sk_common.skc_v6_daddr.in6_u.u6_addr8);
    }
    printf("%-5d %-10.10s %-15s %-5d %-15s %-6d ", $pid,
        $comm, $saddr, $lport, $daddr, $dport);
    printf("%5d %5d %d\n", $tp->bytes_acked / 1024,
        $tp->bytes_received / 1024, $delta_ms);

    delete(@birth[$sk]);
    delete(@skpid[$sk]);
    delete(@skcomm[$sk]);
}
}

END
{
    clear(@birth); clear(@skpid); clear(@skcomm);
}

```

Логика работы этого инструмента довольно сложна, поэтому я добавил в код комментарии, объясняющие работу обеих версий, для ВСС и bpftrace. Ниже эта логика описывается подробнее:

- Измеряет время от первого изменения состояния сокета до появления состояния TCP_CLOSE. Это время выводится как продолжительность.
- Извлекает статистику относительно трафика из структуры tcp_sock. Это позволяет избежать трассировки каждого пакета, чтобы получить суммарный объем трафика. Счетчики трафика появились относительно недавно, в 2015 году [109].

- Кэширует контекст процесса по переходу в состояние `TCP_SYN_SENT` или `TCP_LAST_ACK` и выводит кэшированные данные (а если они не были кэшированы, то данные о текущем процессе) по переходу в состояние `TCP_CLOSE`. Эта логика работает достаточно хорошо, но полагается на события, происходящие в контексте процесса, которые являются деталью реализации ядра. В будущих версиях ядра логика может измениться, что сделает этот подход гораздо менее надежным, и тогда придется обновить этот инструмент, чтобы он кэшировал контекст задачи по событиям в сокете (см. предыдущие инструменты).

Версия этого инструмента для BCC была расширена командой сетевых инженеров из Netflix, они добавили вывод других полезных полей структуры `sock` и `tcp_sock`.

Версию для `bpfttrace` можно дополнить и использовать в ней точку трассировки `sock:inet_sock_set_state` с дополнительной проверкой `args->protocol == IPPROTO_TCP`, потому что эта точка трассировки срабатывает не только для TCP. Ее использование улучшит стабильность, но в инструменте все еще будут оставаться нестабильные части: количество переданных байтов по-прежнему нужно извлекать из структуры `tcp_sock`.

10.3.14. tcptop

`tcptop(8)`¹ — это инструмент BCC, отображающий процессы, наиболее активно использующие TCP. Вот пример, полученный на производственном экземпляре Hadoop с 36 процессорами:

```
# tcptop
09:01:13 loadavg: 33.32 36.11 38.63 26/4021 123015
PID    COMM      LADDR          RADDR          RX_KB  TX_KB
118119  java      100.1.58.46:36246 100.2.52.79:50010 16840   0
122833  java      100.1.58.46:52426 100.2.6.98:50010   0 3112
122833  java      100.1.58.46:50010 100.2.50.176:55396 3112   0
120711  java      100.1.58.46:50010 100.2.7.75:23358 2922   0
121635  java      100.1.58.46:50010 100.2.5.101:56426 2922   0
121219  java      100.1.58.46:50010 100.2.62.83:40570 2858   0
121219  java      100.1.58.46:42324 100.2.4.58:50010   0 2858
122927  java      100.1.58.46:50010 100.2.2.191:29338 2351   0
[...]
```

Как показывает этот вывод, за время трассировки через соединение в начале списка было получено более 16 Мбайт. По умолчанию информация на экране обновляется каждую секунду.

Инструмент трассирует отправку и прием данных по протоколу TCP и суммирует результаты в высокопроизводительной карте BPF. Но события могут следовать

¹ Немного истории: версию `tcptop` на основе `DTrace` я создал 5 июля 2005 года, по образу и подобию инструмента `top(1)` Уильяма Лефевра. Версию для BCC я написал 2 сентября 2016 года.

очень часто, и в системах с высокой сетевой нагрузкой затраты на трассировку могут стать ощутимыми.

В действительности инструмент трассирует функции `tcp_sendmsg()` и `tcp_cleanup_rbuf()`. Последнюю я выбрал просто потому, что она получает в аргументах структуру `sock` и размер. Чтобы получить те же сведения при трассировке `tcp_recvmsg()`, потребуется использовать два зонда `kprobes`, что увеличит оверхед: `kprobe` на входе, чтобы получить структуру `sock`, и `kretprobe`, чтобы получить возвращаемый счетчик байтов.

Обратите внимание, что сейчас `tcptop(8)` не может наблюдать TCP-трафик, отправленный через системный вызов `sendfile(2)`, потому что тот может не вызывать `tcp_sendmsg()`. Если ваша рабочая нагрузка использует `sendfile(2)`, проверьте наличие обновленной версии `tcptop(8)` или сами усовершенствуйте ее.

Порядок использования:

```
tcptop [options] [interval [count]]
```

Параметры `options`:

- **-C**: не очищать экран;
- **-p PID**: выполнять измерения только для этого процесса.

В будущем предполагается добавить возможность сокращать количество отображаемых строк.

10.3.15. tcpsnoop

`tcpsnoop(8)` — мой самый популярный инструмент для DTrace в Solaris, который я бы представил здесь, если бы он существовал для Linux BPF. Но я решил не переносить его, поэтому версия, показанная ниже, — это версия для Solaris. Я привожу ее здесь, потому что она преподала мне некоторые важные уроки.

`tcpsnoop(8)` для каждого пакета выводит строку с адресами, размером, идентификатором процесса и идентификатором пользователя. Например:

```
solaris# tcpsnoop.d
  UID   PID  LADDR          LPORT DR  RADDR          RPORT  SIZE CMD
   0    242 192.168.1.5         23 <- 192.168.1.1   54224   54  inetd
   0    242 192.168.1.5         23 -> 192.168.1.1   54224   54  inetd
   0    242 192.168.1.5         23 <- 192.168.1.1   54224   54  inetd
   0    242 192.168.1.5         23 <- 192.168.1.1   54224   78  inetd
   0    242 192.168.1.5         23 -> 192.168.1.1   54224   54  inetd
   0   20893 192.168.1.5         23 -> 192.168.1.1   54224   57  in.telnetd
   0   20893 192.168.1.5         23 <- 192.168.1.1   54224   54  in.telnetd
[...]
```

Когда я писал его в 2004 году, анализ сетевых событий основывался на анализе пакетов: `snoop(1M)` для Solaris и `tcpdump(8)` для Linux. Один из недостатков этих инструментов — они не выводят идентификатор процесса. Мне нужен был

инструмент, помогающий увидеть, какой процесс порождает сетевой трафик, и очевидным решением было создать версию `snoop(1M)` со столбцом PID. Для проверки своего решения я запустил его вместе со `snoop(1M)`, чтобы убедиться, что оба инструмента видят одни и те же события, касающиеся пакетов.

Это оказалось сложной задачей: нужно было кэшировать PID по событиям на уровне сокета и получать размер пакета с другого конца стека после фрагментирования MTU. Я должен был трассировать код передачи данных, код создания TCP-соединения и другой код, обрабатывающий пакеты, закрывающие порты, и еще кучу других событий. Мне это удалось, но пришлось трассировать 11 разных точек в ядре и исследовать разные структуры, что сделало инструмент очень хрупким, потому что он полагался на многие нестабильные детали ядра. Исходный код инструмента насчитывал более 500 строк.

За шесть лет ядро Solaris обновлялось более десяти раз, и семь из них привели к нарушению работоспособности `tcpsnoop(8)`. Исправление превратилось в кошмар: я исправлял проблему, наблюдавшуюся в одной версии ядра, а затем тестировал инструмент со всеми предыдущими версиями, чтобы убедиться в отсутствии регресса. Наконец я признал непрактичность создания универсальной версии и стал выпускать версии `tcpsnoop(8)` для конкретных ядер.

Из этой истории я вынес два урока.

Во-первых, код ядра может измениться, и чем больше зондов `kprobes` и структур вы будете использовать, тем выше вероятность, что инструмент сломается. Инструменты в этой книге намеренно используют минимальное количество зондов `kprobes`, что упрощает их исправление в случае поломки. По возможности старайтесь использовать точки трассировки.

Во-вторых, сама идея создать такой инструмент была ошибочной. Чтобы определить, какие процессы порождают сетевой трафик, не нужно анализировать каждый пакет. Я мог бы написать инструмент, просто суммирующий переданные данные, то есть пропускающий другие пакеты, включая пакеты, используемые для создания TCP-соединения, и его было бы вполне достаточно для решения большинства задач. Примерами послужат `socketio(8)` и `tcptop(8)`, рассмотренные выше, каждый из которых использует только два зонда `kprobes`, или `tcplife(8)`, использующий одну точку трассировки и анализирующий несколько структур.

10.3.16. `tcpretrans`

`tcpretrans(8)`¹ — это инструмент BCC и `bpfftrace` для трассировки повторных передач TCP, который выводит IP-адреса и номера портов, а также состояние

¹ Немного истории: в 2011 году я написал несколько похожих инструментов для трассировки повторных передач TCP на основе DTrace [110]. Версию `tcpretrans(8)` на основе Ftrace я написал 28 июля 2014 года [111], а версию для BCC — 14 февраля 2016 года. Маттиас Тафельмайер (Matthias Tafelmeier) добавил в нее режим подсчета. Версию для `bpfftrace` написал Дейл Хамел 23 ноября 2018 года.

TCP. Вот пример вывода версии `tcpretrans(8)` для ВСС на производственном экземпляре:

`tcpretrans`

Tracing retransmits ... Hit Ctrl-C to end

TIME	PID	IP	LADDR:LPORT	T>	RADDR:RPORT	STATE
00:20:11	72475	4	100.1.58.46:35908	R>	100.2.0.167:50010	ESTABLISHED
00:20:11	72475	4	100.1.58.46:35908	R>	100.2.0.167:50010	ESTABLISHED
00:20:11	72475	4	100.1.58.46:35908	R>	100.2.0.167:50010	ESTABLISHED
00:20:12	60695	4	100.1.58.46:52346	R>	100.2.6.189:50010	ESTABLISHED
00:20:12	60695	4	100.1.58.46:52346	R>	100.2.6.189:50010	ESTABLISHED
00:20:12	60695	4	100.1.58.46:52346	R>	100.2.6.189:50010	ESTABLISHED
00:20:12	60695	4	100.1.58.46:52346	R>	100.2.6.189:50010	ESTABLISHED
00:20:13	60695	6	::ffff:100.1.58.46:13562	R>	::ffff:100.2.51.209:47356	FIN_WAIT1
00:20:13	60695	6	::ffff:100.1.58.46:13562	R>	::ffff:100.2.51.209:47356	FIN_WAIT1

[...]

Как показывает этот вывод, в период трассировки повторные передачи выполнялись нечасто, всего несколько раз в секунду (об этом можно судить по столбцу `TIME`), и в основном в сеансах с состоянием `ESTABLISHED`. Высокая частота повторных передач в состоянии `ESTABLISHED` может указывать на проблемы с внешней сетью. Высокая частота в состоянии `SYN_SENT` может указывать на большую нагрузку на серверное приложение, которое недостаточно быстро обслуживает свою очередь запросов на соединение (`SYN`).

Этот инструмент трассирует события повторной передачи TCP в ядре. Так как обычно они происходят нечасто, оверхед должен быть незначительным. Для сравнения, в традиционном подходе с использованием анализатора пакетов для захвата всех пакетов и последующей их обработкой в поисках повторных передач оба шага могут стоить больших затрат вычислительных ресурсов. Кроме того, захват пакетов может наблюдать только детали, имеющие отношение к сети, тогда как `tcpretrans(8)` выводит состояние TCP-соединений непосредственно из ядра и его можно расширить для вывода большего количества информации, если потребуется.

В Netflix этот инструмент использовался для диагностики производственной проблемы, вызванной превышением возможностей внешней сети, из-за чего происходили потери пакетов и повторные передачи. Возможность наблюдать за повторными передачами на разных производственных экземплярах и видеть сведения об отправителе, получателе и состоянии TCP-соединений без дополнительных затрат на обработку дампов пакетов оказалась очень полезной.

Компания Shopify также использовала этот инструмент для диагностики проблем в своей производственной сети, когда рабочая нагрузка вынуждала `tcpdump(8)` отбрасывать так много пакетов, что результаты оказывались ненадежными, а оверхед слишком большим. По этой причине для диагностики использовались `tcpretrans(8)` и `tcpdrop(8)` (описывается ниже). Данные, собранные с помощью этих инструментов, указали на внешнюю проблему: это была конфигурация межсетевого экрана, который не справлялся с нагрузкой и отбрасывал пакеты.

BCC

Порядок использования:

`tcpretrans [options]`

Параметры `options`:

- **-l**: включать попытки повторной отправки «хвостового» сегмента (Tail Loss Probe, добавляет трассировку зонда ядра для `tcp_send_loss_probe()`);
- **-c**: подсчитывать повторные передачи по потокам данных.

Параметр `-c` меняет поведение `tcpretrans(8)`, заставляя выводить суммарные счетчики вместо подробностей о каждом событии.

bpfftrace

Ниже приведен исходный код версии для `bpfftrace`, где обобщены основные возможности инструмента. Эта версия не поддерживает параметры командной строки.

```
#!/usr/local/bin/bpfftrace

#include <linux/socket.h>
#include <net/sock.h>

BEGIN
{
    printf("Tracing TCP retransmits. Hit Ctrl-C to end.\n");
    printf("%-8s %-8s %20s %21s %6s\n", "TIME", "PID", "LADDR:LPORT",
        "RADDR:RPORT", "STATE");

    // См. include/net/tcp_states.h:
    @tcp_states[1] = "ESTABLISHED";
    @tcp_states[2] = "SYN_SENT";
    @tcp_states[3] = "SYN_RECV";
    @tcp_states[4] = "FIN_WAIT1";
    @tcp_states[5] = "FIN_WAIT2";
    @tcp_states[6] = "TIME_WAIT";
    @tcp_states[7] = "CLOSE";
    @tcp_states[8] = "CLOSE_WAIT";
    @tcp_states[9] = "LAST_ACK";
    @tcp_states[10] = "LISTEN";
    @tcp_states[11] = "CLOSING";
    @tcp_states[12] = "NEW_SYN_RECV";
}

kprobe:tcp_retransmit_skb
{
    $sk = (struct sock *)arg0;
    $inet_family = $sk->__sk_common.skc_family;

    if ($inet_family == AF_INET || $inet_family == AF_INET6) {
        $daddr = ntop(0);
```

```

$saddr = ntop(0);
if ($inet_family == AF_INET) {
    $daddr = ntop($sk->__sk_common.skc_daddr);
    $saddr = ntop($sk->__sk_common.skc_rcv_saddr);
} else {
    $daddr = ntop(
        $sk->__sk_common.skc_v6_daddr.in6_u.u6_addr8);
    $saddr = ntop(
        $sk->__sk_common.skc_v6_rcv_saddr.in6_u.u6_addr8);
}
$lport = $sk->__sk_common.skc_num;
$dport = $sk->__sk_common.skc_dport;

// Порт получателя в формате с прямым порядком следования байтов,
// их следует поменять местами
$dport = ($dport >> 8) | (($dport << 8) & 0x00FF00);

$state = $sk->__sk_common.skc_state;
$statestr = @tcp_states[$state];

time("%H:%M:%S ");
printf("%-8d %14s:%-6d %14s:%-6d %6s\n", pid, $saddr, $lport,
    $daddr, $dport, $statestr);
}
}

END
{
    clear(@tcp_states);
}

```

Эта версия трассирует функцию ядра `tcp_retransmit_skb()`. В Linux 4.15 были добавлены точки трассировки `tcp:tcp_retransmit_skb` и `tcp:tcp_retransmit_synack`, и этот инструмент можно было бы обновить, чтобы использовать их.

10.3.17. tcpsynbl

`tcpsynbl(8)`¹ измеряет размеры очередей TCP SYN и выводит результаты в виде гистограммы. Вот пример, полученный на производственном пограничном сервере с 48 процессорами:

```

# tcpsynbl.bt
Attaching 4 probes...
Tracing SYN backlog size. Ctrl-C to end.
^C
@backlog[backlog limit]: histogram of backlog size

@backlog[128]:

```

¹ Немного истории: в 2012 году я написал несколько аналогичных инструментов для оценки очереди TCP SYN на основе DTrace [110]. Эту версию для `bpftape` я написал 19 апреля 2019 года.

```
[0]                2 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@backlog[500]:    2783 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[0]              9   | 
[1]             4   | 
[2, 4)          1   |
```

Первая гистограмма показывает, что на предел размера очереди в 128 запросов приходится два соединения, при этом длина очереди была равна нулю. Вторая гистограмма показывает, что на предел размера очереди в 500 запросов приходится более 2000 соединений, и длина обычно была равна нулю, но иногда достигала диапазона от 4 до 8. Если длина очереди превышает предел, то инструмент выводит строку, сообщающую о сбросе пакета SYN, что вызывает задержку на узле клиента, который должен повторить передачу.

Размер очереди настраивается и передается как аргумент системному вызову `listen(2)`:

```
int listen(int sockfd, int backlog);
```

При этом он может усекаться системным ограничением в `/proc/sys/net/core/somaxconn`.

Этот инструмент трассирует события создания новых соединений и проверяет величину предела и размер очереди. Оверхед должен быть незначительным, поскольку по сравнению с другими событиями такие события следуют относительно редко.

Исходный код tcpsynbl(8)¹:

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

BEGIN
{
    printf("Tracing SYN backlog size. Ctrl-C to end.\n");
}

kprobe:tcp_v4_syn_rcv_sock,
kprobe:tcp_v6_syn_rcv_sock
{
    $sock = (struct sock *)arg0;
    @backlog[$sock->sk_max_ack_backlog & 0xffffffff] =
        hist($sock->sk_ack_backlog);
    if ($sock->sk_ack_backlog > $sock->sk_max_ack_backlog) {
        time("%H:%M:%S dropping a SYN.\n");
    }
}
```

¹ Этот инструмент включает обходное решение проблемы приведения к типу `int: & 0xffffffff`. В будущих версиях `bpfftrace` этот прием станет ненужным.

```
END
{
    printf("\n@backlog[backlog limit]: histogram of backlog size\n");
}
```

Когда размер очереди превышает предел, используется встроенная функция `time()`, чтобы вывести строку, содержащую время и сообщение о сбросе пакета SYN. В предыдущем примере такого сообщения нет, так как во время трассировки предел не был превышен.

10.3.18. tcpwin

`tcpwin(8)`¹ трассирует размер окна насыщения TCP и другие параметры ядра, позволяющие исследовать работу механизма управления загруженностью сети. Этот инструмент выводит значения, разделенные запятыми, которые можно импортировать в ПО построения графиков. Вот пример запуска `tcpwin.bt` с сохранением вывода в текстовый файл:

```
# tcpwin.bt > out.tcpwin01.txt

^C
# more out.tcpwin01.txt
Attaching 2 probes...
event,sock,time_us,snd_cwnd,snd_ssthresh,sk_sndbuf,sk_wmem_queued
rcv,0xfffff9212377a9800,409985,2,2,87040,2304
rcv,0xfffff9216fe306e80,534689,10,2147483647,87040,0
rcv,0xfffff92180f84c000,632704,7,7,87040,2304
rcv,0xfffff92180b04f800,674795,10,2147483647,87040,2304
[...]
```

Вторая строка в выводе — это строка заголовка, за которой следуют строки с фактическими сведениями о событии. Второе поле — это адрес структуры `sock` в памяти, который можно использовать для однозначной идентификации соединений. Для подсчета частоты адресов `sock` можно использовать утилиту `awk(1)`:

```
# awk -F, '$1 == "rcv" { a[$2]++ } END { for (s in a) { print s, a[s] } }'
out.tcpwin01.txt
[...]
0xfffff92166fede000 1
0xfffff92150a03c800 4564
0xfffff9213db2d6600 2
[...]
```

Здесь видно, что сокет с наибольшим количеством событий приема во время трассировки имел структуру `sock`, расположенную по адресу `0xfffff92150a03c800`. Вот

¹ Немного истории: этот инструмент я создал 20 апреля 2019 года, вдохновившись модулем `tcp_probe` и опытом его использования для построения графика изменения размера окна насыщения с течением времени.

как с помощью `awk` можно извлечь строку заголовка и события только для этого адреса в новый файл `out.csv`:

```
# awk -F, '$2 == "0xffff92150a03c800" || NR == 2' out.tcpwin01.txt > out.csv
```

Этот файл можно импортировать в ПО на R для статистического анализа и построить график (рис. 10.5).

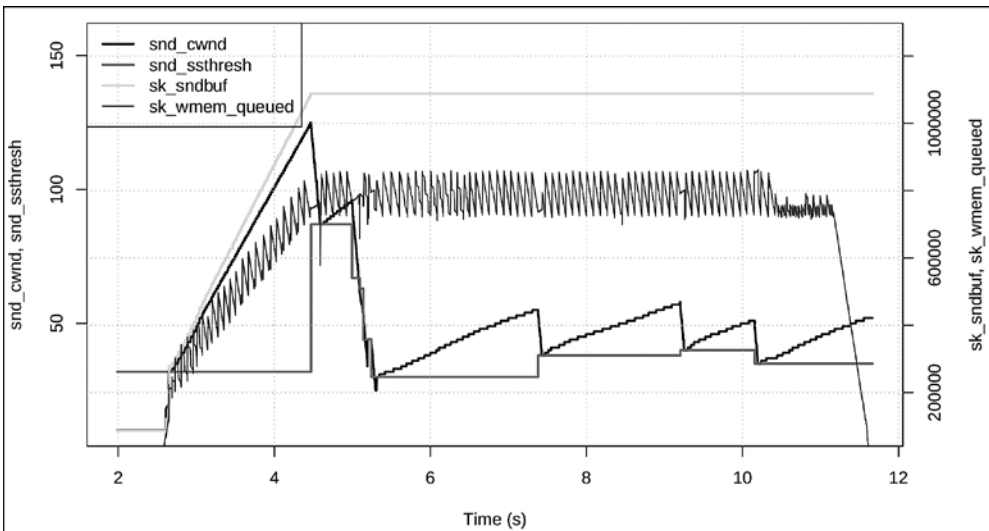


Рис. 10.5. Изменение окна насыщения TCP и размера буфера отправки с течением времени

Эта система использует кубический алгоритм управления загруженностью сети. На графике видно, как размер окна насыщения сначала постепенно увеличивается, а затем резко уменьшается при обнаружении перегрузки (потери пакетов). Это происходит несколько раз, из-за чего график приобретает пилообразный вид, пока наконец не будет найден оптимальный размер окна.

Исходный код `tcpwin(8)`:

```
#!/usr/local/bin/bpftrace
#include <net/sock.h>
#include <linux/tcp.h>

BEGIN
{
    printf("event,sock,time_us,snd_cwnd,snd_ssthresh,sk_sndbuf,");
    printf("sk_wmem_queued\n");
}

kprobe:tcp_rcv_established
{
    $sock = (struct sock *)arg0;
```

```

$tcps = (struct tcp_sock *)arg0; // см. tcp_sk()
printf("rcv,0x%llx,%lld,%d,%d,%d,%d\n", arg0, elapsed / 1000,
      $tcps->snd_cwnd, $tcps->snd_ssthresh, $sock->sk_sndbuf,
      $sock->sk_wmem_queued);
}

```

Этот код можно дополнить. Первое поле — это тип события, но этот инструмент использует только «rcv». Вы можете добавить дополнительные зонды kprobes или точки трассировки и для каждого(-ой) выводить свое название события. Например, можно добавить тип события «new», отражающий создание соединения, с полями, содержащими IP-адреса и номера портов.

Для анализа работы механизма управления загруженностью использовался модуль ядра tcp_probe, который в Linux 4.16 стал точкой трассировки tcp:tcp_probe. Инструмент tcpwin(8) можно переписать, использовав эту точку трассировки, хотя в ее аргументах доступна не вся информация о сокете.

10.3.19. tcpnagle

tcpnagle(8)¹ трассирует использование алгоритма Нейгла, измеряет задержки передачи TCP и выводит результаты в виде гистограммы: эти задержки вызваны работой алгоритма Нейгла и другими событиями. Вот пример, полученный на производственном пограничном сервере:

```

# tcpnagle.bt
Attaching 4 probes...
Tracing TCP nagle and xmit delays. Hit Ctrl-C to end.
^C

@blocked_us:
[2, 4)          3 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[4, 8)          2 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|

@nagle[CORK]: 2
@nagle[OFF|PUSH]: 5
@nagle[ON]: 32
@nagle[PUSH]: 11418
@nagle[OFF]: 226697

```

Как показывает этот вывод, во время трассировки алгоритм Нейгла часто отключался (возможно, потому, что приложение вызывало setsockopt(2) с TCP_NODELAY или выполняло отправку в обход алгоритма Нейгла (например, используя сокет с TCP_CORK)). Отправка пакетов задерживалась только 5 раз, самое большее на время от 4 до 8 микросекунд.

Инструмент трассирует вход и выход из функции передачи в реализации протокола TCP. Она может вызываться очень часто, поэтому в системах с высокой сетевой нагрузкой оверхед может стать ощутимым.

¹ Немного истории: я написал его специально для этой книги 23 апреля 2019 года.

Исходный код tcpnagle(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing TCP nagle and xmit delays. Hit Ctrl-C to end.\n");
    // из include/net/tcp.h; добавьте другие комбинации, если необходимо:
    @flags[0x0] = "ON";
    @flags[0x1] = "OFF";
    @flags[0x2] = "CORK";
    @flags[0x3] = "OFF|CORK";
    @flags[0x4] = "PUSH";
    @flags[0x5] = "OFF|PUSH";
}

kprobe:tcp_write_xmit
{
    @nagle[@flags[arg2]] = count();
    @sk[tid] = arg0;
}

kretprobe:tcp_write_xmit
/@sk[tid]/
{
    $inflight = retval & 0xff;
    $sk = @sk[tid];

    if ($inflight && !@start[$sk]) {
        @start[$sk] = nsecs;
    }
    if (!$inflight && @start[$sk]) {
        @blocked_us = hist((nsecs - @start[$sk]) / 1000);
        delete(@start[$sk]);
    }
    delete(@sk[tid]);
}

END
{
    clear(@flags); clear(@start); clear(@sk);
}
```

На входе в `tcp_write_xmit()` флаги отключения алгоритма Нейгла (`arg2`) преобразуются в читаемую строку с помощью карты `@flags`. Указатель на структуру `sock` тоже запоминается, так как используется в `kretprobe` для сохранения отметки времени в информации о соединении, чтобы потом можно было вычислить длительность задержки передачи. Длительность измеряется от первого случая, когда `tcp_write_xmit()` возвращает ненулевое значение (признак того, что по какой-то причине пакеты не были отправлены, одна из возможных причин — действие алгоритма Нейгла), до момента, когда `tcp_write_xmit()` успешно отправит пакеты для этого сокета.

10.3.20. udpconnect

udpconnect(8)¹ трассирует создание новых UDP-соединений, инициированных локальным хостом посредством вызова connect(2) (он не трассирует входящие соединения UDP). Например:

```
# udpconnect.bt
Attaching 3 probes...
TIME     PID      COMM                IP RADDR              RPORT
20:58:38 6039     DNS Res~er #540    4  10.45.128.25        53
20:58:38 2621     TaskSchedulerFo    4  127.0.0.53          53
20:58:39 3876     Chrome_IOThread    6  2001:4860:4860::8888 53
[...]
```

Здесь видны два соединения, оба с удаленным портом 53, одно инициировано локальной службой разрешения имен, а другое — Chrome_IOThread.

Инструмент трассирует функции ядра, устанавливающие UDP-соединения. Частота их вызова обычно невелика, поэтому оверхед будет незначительным.

Исходный код udpconnect(8):

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

BEGIN
{
    printf("%-8s %-6s %-16s %-2s %-16s %-5s\n", "TIME", "PID", "COMM",
        "IP", "RADDR", "RPORT");
}

kprobe:ip4_datagram_connect,
kprobe:ip6_datagram_connect
{
    $sa = (struct sockaddr *)arg1;
    if ($sa->sa_family == AF_INET || $sa->sa_family == AF_INET6) {
        time("%H:%M:%S ");
        if ($sa->sa_family == AF_INET) {
            $s = (struct sockaddr_in *)arg1;
            $port = ($s->sin_port >> 8) |
                (($s->sin_port << 8) & 0xff00);
            printf("%-6d %-16s 4 %-16s %-5d\n", pid, comm,
                ntop(AF_INET, $s->sin_addr.s_addr), $port);
        } else {
            $s6 = (struct sockaddr_in6 *)arg1;
            $port = ($s6->sin6_port >> 8) |
                (($s6->sin6_port << 8) & 0xff00);
            printf("%-6d %-16s 6 %-16s %-5d\n", pid, comm,
                ntop(AF_INET6, $s6->sin6_addr.in6_u.u6_addr8),
                $port);
        }
    }
}
```

¹ Немного истории: я написал его для этой книги 20 апреля 2019 года.

Указатели на функции `ip4_datagram_connect()` и `ip6_datagram_connect()` передаются в поле `connect` структур `udp_prot` и `udp6_prot`, которые определяют функции поддержки протокола UDP. Информация выводится точно так же, как в предыдущих инструментах.

Также смотрите описание инструмента `socketio(8)`, который отображает отправку и прием дейтаграмм UDP по процессам. Инструмент, ориентированный исключительно на UDP, можно реализовать, организовав в нем трассировку функций `udp_sendmsg()` и `udp_recvmsg()`, и исключить оверхед на анализ других протоколов, кроме UDP.

10.3.21. gethostlatency

`gethostlatency(8)`¹ — это инструмент BCC и `bpfttrace` для трассировки обращений к службе разрешения имен (DNS) через вызовы библиотечных функций `getaddrinfo(3)`, `gethostbyname(3)` и т. д. Например:

```
# gethostlatency
TIME      PID    COMM                LATms HOST
13:52:39  25511  ping                9.65  www.netflix.com
13:52:42  25519  ping                2.64  www.netflix.com
13:52:49  24989  DNS Res~er #712    43.09  docs.google.com
13:52:52  25527  ping                99.26  www.cilium.io
13:52:53  19025  DNS Res~er #709    2.58  drive.google.com
13:53:05  21903  ping                279.09 www.kubernetes.io
13:53:06  25459  TaskSchedulerFo    23.87  www.informit.com
[...]
```

Этот вывод показывает задержки разрешения разных имен в системе. Первой к службе разрешения имен обратилась команда `ping(1)`, чтобы получить адрес для имени `www.netflix.com`. Это заняло 9.65 миллисекунды. Следующая попытка получить адрес для того же имени заняла 2.64 миллисекунды (вероятно, благодаря кэшированию). В выводе можно также видеть другие поисковые запросы, самый медленный из которых — определение адреса для `www.kubernetes.io` — выполнялся 279 миллисекунд².

Инструмент использует прием динамической инструментации библиотечных функций в пространстве пользователя. На входе в функцию записывается имя хоста и отметка времени, а на выходе вычисляется и выводится продолжительность ее выполнения вместе с сохраненным именем узла. Так как эти события возникают относительно редко, оверхед на их трассировку должен быть незначительным.

¹ Немного истории: похожий инструмент под названием `getaddrinfo.d` я написал для книги о DTrace в 2011 году [Gregg 11]. Версию для BCC я написал 28 января 2016 года, а версию для `bpfttrace` — 8 сентября 2018 года.

² Медленное время DNS для домена `.io` из США — известная проблема, которая, как полагают, связана с расположением серверов имен `.io` [112].

DNS — типичный источник задержек в производственной среде. Компания Shopify использовала версию этого инструмента для bpftrace в кластере Kubernetes, чтобы диагностировать проблему задержки разрешения имен. Результаты показали, что проблема связана не с определенным сервером или целью поиска, а с большим числом запросов, поступающих одновременно. После дополнительного анализа также выяснилось, что фактически проблема связана с ограничением облака на количество сеансов UDP, которые могут быть открыты на каждом узле. Увеличение этого предела решило проблему.

BCC

Порядок использования:

```
gethostlatency [options]
```

Сейчас инструмент поддерживает единственный параметр командной строки: `-p PID`, чтобы трассировать только указанный процесс.

bpftrace

Ниже приведен исходный код версии для bpftrace. Эта версия не поддерживает параметры командной строки:

```
#!/usr/local/bin/bpftrace
```

```
BEGIN
```

```
{
    printf("Tracing getaddr/gethost calls... Hit Ctrl-C to end.\n");
    printf("%-9s %-6s %-16s %6s %s\n", "TIME", "PID", "COMM", "LATms",
        "HOST");
}
```

```
uprobe:/lib/x86_64-linux-gnu/libc.so.6:getaddrinfo,
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname,
uprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname2
```

```
{
    @start[tid] = nsecs;
    @name[tid] = arg0;
}
```

```
uretprobe:/lib/x86_64-linux-gnu/libc.so.6:getaddrinfo,
uretprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname,
uretprobe:/lib/x86_64-linux-gnu/libc.so.6:gethostbyname2
/@start[tid]/
{
    $latms = (nsecs - @start[tid]) / 1000000;
    time("%H:%M:%S ");
    printf("%-6d %-16s %6d %s\n", pid, comm, $latms, str(@name[tid]));
    delete(@start[tid]);
    delete(@name[tid]);
}
```

Инструмент трассирует несколько функций механизма разрешения имен из библиотеки `libc`, доступной в файловой системе как `/lib/x86_64-linux-gnu/libc.so.6`. Если для разрешения имен используется другая библиотека или если функции реализуются приложением или подключены статически (статическая сборка), то инструмент нужно изменить для трассировки этих других местоположений.

10.3.22. `ipecn`

Инструмент `ipecn(8)`¹ трассирует входящие уведомления о перегрузке IPv4 (Explicit Congestion Notification, ECN) и создавался для проверки идеи. Вот пример вывода:

```
# ipecn.bt
Attaching 3 probes...
Tracing inbound IPv4 ECN Congestion Encountered. Hit Ctrl-C to end.
10:11:02 ECN CE from: 100.65.76.247
10:11:02 ECN CE from: 100.65.76.247
10:11:03 ECN CE from: 100.65.76.247
10:11:21 ECN CE from: 100.65.76.247
[...]
```

В этом выводе — события получения уведомлений о перегрузке (CE) на пути к узлу с адресом `100.65.76.247`. Уведомления могут посылаются коммутаторами и маршрутизаторами, чтобы уведомить конечные точки о перегрузке. Также уведомления могут посылаются ядрами, использующими политики организации очередей, хотя обычно это делается только для тестирования и моделирования (с помощью модуля `netem`). Уведомления о перегрузке также используются алгоритмом управления загруженностью DataCenter TCP (DCTCP) [Alizadeh 10] [113].

`ipecn(8)` трассирует функцию ядра `ip_rcv()` и читает состояние перегрузки из IP-заголовка. Это далеко не идеальный метод, так как основан на анализе каждого полученного пакета, что увеличивает оверхед. Скорее это реализация для проверки идеи. Гораздо эффективнее было бы трассировать функции ядра, обрабатывающие только события CE, потому что они будут вызываться намного реже. Однако эти функции реализованы как встраиваемые и недоступны для прямой трассировки (в моих ядрах). Лучшим решением этой проблемы было бы создание точки трассировки для событий ECN, связанных с перегрузкой.

Исходный код `ipecn(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/skbuff.h>
#include <linux/ip.h>

BEGIN
{
```

¹ Немного истории: я написал его для этой книги 28 мая 2019 года по предложению Саргуна Диллона (Sargun Dhillon).

```

    printf("Tracing inbound IPv4 ECN Congestion Encountered. ");
    printf("Hit Ctrl-C to end.\n");
}

kprobe:ip_rcv
{
    $skb = (struct sk_buff *)arg0;
    // получить заголовок IPv4; см. реализацию skb_network_header():
    $iph = (struct iphdr *)($skb->head + $skb->network_header);
    // см. определение INET_ECN_MASK:
    if (($iph->tos & 3) == 3) {
        time("%H:%M:%S ");
        printf("ECN CE from: %s\n", ntop($iph->saddr));
    }
}

```

Этот код — пример анализа заголовка IPv4 из структуры `sk_buff`. Он использует ту же логику, что и функция ядра `skb_network_header()`, и его придется изменить, если в эту функцию будут внесены какие-либо изменения (еще один пример, почему предпочтительнее использовать более стабильные точки трассировки). Этот инструмент можно дополнить трассировкой исходящего пути и IPv6 (см. раздел 10.5).

10.3.23. superping

`superping(8)`¹ измеряет время выполнения запросов ICMP ECHO сетевым стеком ядра и может использоваться для проверки времени приема/передачи, сообщаемого утилитой `ping(8)`. Старые версии `ping(8)` измеряют время прохождения туда/обратно из пользовательского пространства, которое может включать задержку планирования процесса, довольно большую в нагруженных системах, и завышать измеренное время. Этот старый метод также используется утилитой `ping(8)` для ядер, не поддерживающих отметки времени в сокетах (`SIOCGSTAMP` или `SO_TIMESTAMP`).

Поскольку я пользуюсь новой версией `ping(8)` и новым ядром, то для демонстрации старого поведения я запустил ее с параметром `-U`, который измеряет задержку в пользовательском пространстве. Например, в одном терминале я запустил команду `ping(8)`:

```

terminal1# ping -U 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=6.44 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=6.60 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=5.93 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=7.40 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=5.87 ms
[...]
```

¹ Немного истории: первую версию этого инструмента я написал для книги о DTrace в 2011 году [Gregg 11], а версия для этой книги была написана 20 апреля 2019 года.

А в другом `superping(8)`:

```
terminal2# superping.bt
Attaching 6 probes...
Tracing ICMP echo request latency. Hit Ctrl-C to end.
IPv4 ping, ID 28121 seq 1: 6392 us
IPv4 ping, ID 28121 seq 2: 6474 us
IPv4 ping, ID 28121 seq 3: 5811 us
IPv4 ping, ID 28121 seq 4: 7270 us
IPv4 ping, ID 28121 seq 5: 5741 us
[...]
```

Если сравнить результаты, то можно заметить, что `ping(8)` сообщает время, которое примерно на 0.10 мс больше для этой системы и текущей рабочей нагрузки. При запуске без параметра `-U ping(8)` использует временные отметки из сокета, и разница во времени обычно не превышает 0.01 мс.

Этот инструмент трассирует отправку и получение пакетов ICMP, сохраняя отметки времени в карте BPF для каждого запроса ICMP ЕЧО и сравнивая заголовки ICMP при получении эхо-пакетов. Оверхед должен быть незначительным, потому что обрабатываются пакеты IP, а не TCP.

Исходный код `superping(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/skbuff.h>
#include <linux/icmp.h>
#include <linux/ip.h>
#include <linux/ipv6.h>
#include <linux/in.h>

BEGIN
{
    printf("Tracing ICMP ping latency. Hit Ctrl-C to end.\n");
}

/*
 * IPv4
 */
kprobe:ip_send_skb
{
    $skb = (struct sk_buff *)arg1;
    // извлечь заголовок IPv4; см. реализацию skb_network_header():
    $iph = (struct iphdr *)($skb->head + $skb->network_header);
    if ($iph->protocol == IPPROTO_ICMP) {
        // извлечь заголовок ICMP; см. реализацию skb_transport_header():
        $icmph = (struct icmphdr *)($skb->head +
            $skb->transport_header);
        if ($icmph->type == ICMP_ECHO) {
            $id = $icmph->un.echo.id;
            $seq = $icmph->un.echo.sequence;
            @start[$id, $seq] = nsecs;
        }
    }
}
```

```

    }
}

kprobe:icmp_rcv
{
    $skb = (struct sk_buff *)arg0;
    // извлечь заголовок ICMP; см. реализацию skb_transport_header():
    $icmph = (struct icmphdr *)($skb->head + $skb->transport_header);

    if ($icmph->type == ICMP_ECHOREPLY) {
        $id = $icmph->un.echo.id;
        $seq = $icmph->un.echo.sequence;
        $start = @start[$id, $seq];
        if ($start > 0) {
            $idhost = ($id >> 8) | (($id << 8) & 0xff00);
            $seqhost = ($seq >> 8) | (($seq << 8) & 0xff00);
            printf("IPv4 ping, ID %d seq %d: %d us\n",
                $idhost, $seqhost, (nsecs - $start) / 1000);
            delete(@start[$id, $seq]);
        }
    }
}

/*
 * IPv6
 */
kprobe:ip6_send_skb
{
    $skb = (struct sk_buff *)arg0;
    // извлечь заголовок IPv6; см. реализацию skb_network_header():
    $ip6h = (struct ipv6hdr *)($skb->head + $skb->network_header);
    if ($ip6h->nexthdr == IPPROTO_ICMPV6) {
        // извлечь заголовок ICMP; см. реализацию skb_transport_header():
        $icmp6h = (struct icmp6hdr *)($skb->head +
            $skb->transport_header);
        if ($icmp6h->icmp6_type == ICMPV6_ECHO_REQUEST) {
            $id = $icmp6h->icmp6_dataun.u_echo.identifier;
            $seq = $icmp6h->icmp6_dataun.u_echo.sequence;
            @start[$id, $seq] = nsecs;
        }
    }
}

kprobe:icmpv6_rcv
{
    $skb = (struct sk_buff *)arg0;
    // извлечь заголовок ICMPV6; см. реализацию skb_transport_header():
    $icmp6h = (struct icmp6hdr *)($skb->head + $skb->transport_header);
    if ($icmp6h->icmp6_type == ICMPV6_ECHO_REPLY) {
        $id = $icmp6h->icmp6_dataun.u_echo.identifier;
        $seq = $icmp6h->icmp6_dataun.u_echo.sequence;
        $start = @start[$id, $seq];
        if ($start > 0) {
            $idhost = ($id >> 8) | (($id << 8) & 0xff00);
            $seqhost = ($seq >> 8) | (($seq << 8) & 0xff00);

```



```

        printf("IPv6 ping, ID %d seq %d: %d us\n",
               $idhost, $seqhost, (nsecs - $start) / 1000);
        delete(@start[$id, $seq]);
    }
}

END { clear(@start); }

```

Версии IPv4 и IPv6 обрабатываются разными функциями ядра и трассируются отдельно. Этот код может послужить примером анализа заголовков пакетов: заголовки пакетов IPv4, IPv6, ICMP и ICMPv6 читаются инфраструктурой BPF. Метод извлечения этих заголовков из структуры `sk_buff` зависит от исходного кода ядра и его функций `skb_network_header()` и `skb_transport_header()`. Как и `kprobes`, это нестабильный интерфейс, и изменения в сетевом стеке, касающиеся обработки заголовков, потребуют изменить этот инструмент, чтобы привести его в соответствие.

Небольшое замечание к исходному коду: идентификатор ICMP и порядковый номер выводятся после преобразования значений из сетевого порядка следования байтов в аппаратный (см. `$idhost =` и `$seqhost =`). В карте `@start`, где сохраняются отметки времени, я использовал сетевой порядок байтов. Это позволило сэкономить несколько инструкций на обработке зондов, трассирующих отправку.

10.3.24. qdisc-fq

qdisc-fq(8)¹ показывает задержку в очереди со справедливой дисциплиной (Fair Queue, FQ). Вот пример, полученный на производственном пограничном сервере:

```
# qdisc-fq.bt
Attaching 4 probes...
Tracing qdisc fq latency. Hit Ctrl-C to end.
^C
```

```
@us:
[0] 6803 | @@@@@@@@@@@@@@
[1] 20084 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2, 4) 29230 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4, 8) 755 | @
[8, 16) 210 |
[16, 32) 86 |
[32, 64) 39 |
[64, 128) 90 |
[128, 256) 65 |
[256, 512) 61 |
[512, 1K) 26 |
[1K, 2K) 9 |
[2K, 4K) 2 |
```

¹ Немного истории: я написал его для этой книги 21 апреля 2019 года.

Как показывает этот вывод, обычно пакеты остаются в очереди менее 4 микросекунд, и очень немногие оставались в ней от 2 до 4 миллисекунд. Если возникнет проблема с задержкой пакетов в очереди, ее нетрудно будет заметить на этой гистограмме.

Инструмент трассирует функции постановки пакетов в очередь и извлечения их из очереди. Для систем с высокой сетевой нагрузкой оверхед может стать ощутимым, поскольку события могут следовать очень часто.

Исходный код `qdisc-fq(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing qdisc fq latency. Hit Ctrl-C to end.\n");
}

kprobe:fq_enqueue
{
    @start[arg0] = nsecs;
}

kretprobe:fq_dequeue
/@start[retval]/
{
    @us = hist((nsecs - @start[retval]) / 1000);
    delete(@start[retval]);
}

END
{
    clear(@start);
}
```

Аргумент функции `fq_enqueue()` и возвращаемое значение `fq_dequeue()` — адрес структуры `sk_buff`, который используется как уникальный ключ для сохранения отметок времени.

Обратите внимание, что этот инструмент работает, только когда загружен планировщик дисциплины FQ, в противном случае инструмент выведет сообщение об ошибке:

```
# qdisc-fq.bt
Attaching 4 probes...
cannot attach kprobe, Invalid argument
Error attaching probe: 'kretprobe:fq_dequeue'
```

Эту проблему можно исправить, принудительно загрузив модуль ядра планировщика FQ:

```
# modprobe sch_fq
# qdisc-fq.bt
Attaching 4 probes...
Tracing qdisc fq latency. Hit Ctrl-C to end.
^C
#
```

Но если эта дисциплина не используется, то и ожидаемых событий возникать не будет. Для добавления и администрирования планировщиков очередей используйте `tc(1)`.

10.3.25. `qdisc-cbq`, `qdisc-cbs`, `qdisc-codel`, `qdisc-fq_codel`, `qdisc-red` и `qdisc-tbf`

Кроме FQ есть множество других дисциплин очередей, и предыдущий инструмент `qdisc-fq(8)` можно адаптировать для трассировки каждой из них. Например, вот версия для трассировки очереди на основе классов (Class Based Queueing, CBQ):

```
# qdisc-cbq.bt
Attaching 4 probes...
Tracing qdisc cbq latency. Hit Ctrl-C to end.
^C

@us:
[0]          152 |@@
[1]          766 |@@@@@@@@@@@@@@@@
[2, 4)      2033 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4, 8)      2279 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[8, 16)     2663 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16, 32)    427  |@@@@@@@@
[32, 64)    15   |
[64, 128)   1    |
```

Функции постановки в очередь и извлечения из очереди для трассировки определяются по структуре `Qdisc_ops`, в которой задаются их аргументы и возвращаемое значение (`include/net/sch_generic.h`):

```
struct Qdisc_ops {
    struct Qdisc_ops      *next;
    const struct Qdisc_class_ops *cl_ops;
    char                  id[IFNAMSIZ];
    int                   priv_size;
    unsigned int          static_flags;
    int                   (*enqueue)(struct sk_buff *skb,
                                     struct Qdisc *sch,
                                     struct sk_buff **to_free);
    struct sk_buff *      (*dequeue)(struct Qdisc *);
[...]
```

Вот почему адрес `skb_buff` передается в первом аргументе функции постановки в очередь и возвращается функцией извлечения из очереди.

Эта структура `Qdisc_ops` объявлена и для других планировщиков. Для планировщика CBQ (`net/sched/sch_cbq.c`):

```
static struct Qdisc_ops cbq_qdisc_ops __read_mostly = {
    .next      = NULL,
    .cl_ops    = &cbq_class_ops,
```

```

        .id           =        "cbq",
        .priv_size    =        sizeof(struct cbq_sched_data),
        .enqueue      =        cbq_enqueue,
        .dequeue      =        cbq_dequeue,
[...]
```

То есть инструмент `qdisc-cbq.bt` можно написать, заменив `fq_enqueue` в исходном коде в `qdisc-fq(8)` на `cbq_enqueue`, а `fq_dequeue` на `cbq_dequeue`. Вот таблица замен для некоторых дисциплин:

Инструмент BPF	Дисциплина	Функция добавления в очередь	Функция извлечения из очереди
<code>qdisc-cbq.bt</code>	Очереди на основе классов (Class Based Queueing, CBQ)	<code>cbq_enqueue()</code>	<code>cbq_dequeue()</code>
<code>qdisc-cbs.bt</code>	Очереди на основе разрешений (Credit Based Shaper, CBS)	<code>cbs_enqueue()</code>	<code>cbs_dequeue()</code>
<code>qdisc-codel.bt</code>	Активное управление очередью с контролируемой задержкой (Controlled-Delay Active Queue Management, CoDel)	<code>codel_qdisc_enqueue()</code>	<code>codel_qdisc_dequeue()</code>
<code>qdisc-fq_codel.bt</code>	Справедливая очередь с контролируемой задержкой (Fair Queueing with Controlled Delay, FQ_CoDel)	<code>fq_codel_enqueue()</code>	<code>fq_codel_dequeue()</code>
<code>qdisc-red</code>	Произвольное раннее обнаружение (Random Early Detection, RED)	<code>red_enqueue()</code>	<code>red_dequeue()</code>
<code>qdisc-tbf</code>	Фильтр корзины токенов (Token Bucket Filter, TBF)	<code>tbf_enqueue()</code>	<code>tbf_dequeue()</code>

Было бы несложно создать сценарий-оболочку для `bpfttrace`, например, с именем `qdisclat`, который принимает имя дисциплины и затем создает и запускает программу `bpfttrace`, отображающую соответствующую задержку.

10.3.26. netsize

`netsize(8)`¹ отображает объем ввода/вывода на уровне сетевых устройств до и после сегментации (GSO и GRO). По результатам этого инструмента можно судить о том, как сегментируются пакеты перед отправкой. Вот пример с нагруженного производственного сервера:

¹ Немного истории: я написал его для этой книги 21 апреля 2019 года.

```
# netsize.bt
```

Attaching 5 probes...

Tracing net device send/receive. Hit Ctrl-C to end.

 $\wedge C$

```
@nic_recv_bytes:
```

[32, 64)	16291	@@
[64, 128)	668	@@
[128, 256)	19	
[256, 512)	18	
[512, 1K)	24	
[1K, 2K)	157	

```
@nic_send_bytes:
```

[32, 64)	107
[64, 128)	356
[128, 256)	139
[256, 512)	31
[512, 1K)	15
[1K, 2K)	45850

```
@recv_bytes:
```

[32, 64)	16417	@@
[64, 128)	688	@@
[128, 256)	20	
[256, 512)	33	
[512, 1K)	35	
[1K, 2K)	145	
[2K, 4K)	1	
[4K, 8K)	5	
[8K, 16K)	3	
[16K, 32K)	2	

```
@send_bytes:
```

[illegible]

Здесь показаны размеры пакетов на уровне сетевой карты (@nic_recv_bytes, @nic_send_bytes) и на уровне сетевого стека ядра (@recv_bytes, @send_bytes). Как видите, сервер получал небольшие пакеты, часто меньше 64 байт, а отправлял в основном довольно большие пакеты, размером от 8 до 64 Кбайт (которые после сегментации перед передачей в сетевую карту превращались в пакеты размером от 1 до 2 Кбайт, совпадающим с размером MTU; здесь он, вероятнее всего, равен 1500 байт).

Сетевой интерфейс в этой системе не поддерживает уменьшение затрат на сегментацию TCP (TCP Segmentation Offload, TSO), поэтому использовался механизм GSO, выполнивший сегментацию непосредственно перед передачей пакетов в сетевую карту. Если бы сетевая карта поддерживала TSO и эта возможность была включена, то гистограмма @nic_send_bytes показала бы пакеты большего размера, потому что фактическая сегментация выполнялась бы позже, внутри сетевой карты.

Переключение на jumbo-фреймы увеличит размер пакета и пропускную способность системы, но могут возникнуть проблемы с поддержкой jumbo-фреймов в центре обработки данных, в том числе увеличенное потребление памяти в коммутаторах и усугубление проблемы TCP Incast, выражающейся в потере пакетов TCP, не уместившихся в память коммутатора.

Этот пример вывода можно сравнить с примером вывода `socksize(8)`.

Инструмент использует точки трассировки сетевого устройства и суммирует значения аргументов длины в картах BPF. Оверхед может стать ощутимым в системах с большой сетевой нагрузкой.

В Linux есть и инструмент `iptraf-ng(8)`, который тоже выводит гистограммы распределения сетевых пакетов по размерам. Однако `iptraf-ng(8)` захватывает и анализирует пакеты в пространстве пользователя, из-за чего имеет более высокий оверхед, чем `netsize(8)`, который суммирует данные в пространстве ядра. Например, вот результаты исследования потребления вычислительных ресурсов каждым инструментом во время тестирования с помощью `iperf(1)`:

```
# pidstat -p $(pgrep iptraf-ng) 1
Linux 4.15.0-47-generic (lgud-bgregg)    04/22/2019    _x86_64_    (8 CPU)

11:32:15 AM  UID      PID      %usr %system %guest  %wait   %CPU   CPU  Command
11:32:16 AM    0    30825    18.00   74.00    0.00    0.00   92.00    2  iptraf-ng
11:32:17 AM    0    30825    21.00   70.00    0.00    0.00   91.00    1  iptraf-ng
11:32:18 AM    0    30825    21.00   71.00    0.00    1.00   92.00    6  iptraf-ng
[...]
# pidstat -p $(pgrep netsize) 1
Linux 4.15.0-47-generic (lgud-bgregg)    04/22/2019    _x86_64_    (8 CPU)

11:33:39 AM  UID      PID      %usr %system %guest  %wait   %CPU   CPU  Command
11:33:40 AM    0    30776    0.00    0.00    0.00    0.00    0.00    5  netsize.bt
11:33:41 AM    0    30776    0.00    0.00    0.00    0.00    0.00    7  netsize.bt
11:33:42 AM    0    30776    0.00    0.00    0.00    0.00    0.00    1  netsize.bt
[...]
```

`iptraf-ng(8)` потребляет более 90% времени одного процессора для анализа пакетов и формирования гистограммы, тогда как `netsize(8)` потребляет 0%. Это подчеркивает главное отличие между подходами, несмотря на то что здесь не показан дополнительный оверхед на обработку в пространстве ядра.

Исходный код `netsize(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing net device send/receive. Hit Ctrl-C to end.\n");
}

tracepoint:net:netif_receive_skb
{
    @recv_bytes = hist(args->len);
}

tracepoint:net:net_dev_queue
{
    @send_bytes = hist(args->len);
}

tracepoint:net:napi_gro_receive_entry
{
    @nic_recv_bytes = hist(args->len);
}

tracepoint:net:net_dev_xmit
{
    @nic_send_bytes = hist(args->len);
}
```

Здесь для подсчета объемов ввода/вывода используются точки трассировки из пространства имен `net`.

10.3.27. nettxlat

`nettxlat(8)`¹ отображает задержку передачи через сетевые устройства: время от передачи пакета на уровень драйвера для включения в очередь отправки до момента, когда оборудование сообщит ядру, что передача пакета завершена (обычно через NAPI). Вот пример вывода, полученный на нагруженном производственном пограничном сервере:

```
# nettxlat.bt
Attaching 4 probes...
Tracing net device xmit queue latency. Hit Ctrl-C to end.
^C

@us:
[4, 8)          2230 |
[8, 16)        150679 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

¹ Немного истории: я написал его для этой книги 21 апреля 2019 года.

Инструмент можно доработать, добавив разбивку по имени устройства. Вот пример изменения, в результате которого `nettxlat(8)` превратился в `nettxlat-dev(8)`:

```
[...]
#include <linux/skbuff.h>
#include <linux/netdevice.h>
[...]
tracepoint:skb:consume_skb
/@start[args->skbaddr]/
{
    $skb = (struct sk_buff *)args->skbaddr;
    @us[$skb->dev->name] = hist((nsecs - @start[args->skbaddr]) / 1000);
[...]
```

и его вывод:

```
# nettxlat-dev.bt
Attaching 4 probes...
Tracing net device xmit queue latency. Hit Ctrl-C to end.
^C
```

```
@us[eth0]:
[4, 8)          65 |
[8, 16)         6438 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16, 32)        10899 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32, 64)         2265 | @@@@@@@@@@@@@@
[64, 128)         977 | @@@@
[...]
```

На этом сервере есть только один сетевой интерфейс, `eth0`, но если бы использовались другие интерфейсы, то для каждого была бы создана отдельная гистограмма.

Обратите внимание, что это изменение ухудшает стабильность инструмента, потому что полагается на нестабильные внутренние особенности, а не только на точки трассировки и их аргументы.

10.3.28. skbdrop

skbdrop(8)¹ трассирует необычные события сброса skb и отображает соответствующие им трассировки стека ядра вместе с сетевыми счетчиками, накопленными за время трассировки. Вот пример вывода, полученного на производственном сервере:

```
# bpftrace --unsafe skbdrop.bt
Attaching 3 probes...
Tracing unusual skb drop stacks. Hit Ctrl-C to end.
^C#kernel
IpInReceives          28717          0.0
IpInDelivers           28717          0.0
IpOutRequests          32033          0.0
```

¹ Немного истории: я написал его для этой книги 21 апреля 2019 года.

```

TcpActiveOpens          173          0.0
TcpPassiveOpens         278          0.0
[...]
TcpExtTCPSackMerged      1          0.0
TcpExtTCPSackShiftFallback 5          0.0
TcpExtTCPDeferAcceptDrop 278         0.0
TcpExtTCPRcvCoalesce     3276        0.0
TcpExtTCPAutoCorking     774         0.0
[...]

```

```
[...]
```

```
@[
```

```

    kfree_skb+118
    skb_release_data+171
    skb_release_all+36
    __kfree_skb+18
    tcp_recvmmsg+1946
    inet_recvmmsg+81
    sock_recvmmsg+67
    SYSC_recvfrom+228

```

```
] : 50
```

```
@[
```

```

    kfree_skb+118
    sk_stream_kill_queues+77
    inet_csk_destroy_sock+89
    tcp_done+150
    tcp_time_wait+446
    tcp_fin+216
    tcp_data_queue+1401
    tcp_rcv_state_process+1501

```

```
] : 142
```

```
@[
```

```

    kfree_skb+118
    tcp_v4_rcv+361
    ip_local_deliver_finish+98
    ip_local_deliver+111
    ip_rcv_finish+297
    ip_rcv+655
    __netif_receive_skb_core+1074
    __netif_receive_skb+24

```

```
] : 276
```

Сначала выводятся сетевые счетчики, накопленные за время работы инструмента, а затем следуют трассировки стека, соответствующие событиям сброса `skb`, с количеством таких событий. В примере выше видно, что чаще всего сброс происходил в `tcp_v4_rcv()` — всего было зафиксировано 276 таких событий. Близкое значение 278 имеют сетевые счетчики `TcpPassiveOpens` и `TcpExtTCPDeferAcceptDrop`. (Немного большее значение объясняется тем, что для получения этих счетчиков требуется чуть больше времени.) Такое сходство может говорить о связи этих событий.

Инструмент использует точку трассировки `skb:kfree_skb` и запускает инструмент `nstat(8)` для подсчета статистики во время трассировки. Поэтому перед использованием этого инструмента нужно установить `nstat(8)`: он находится в пакете `iproute2`.

Точка трассировки `skb:kfree_skb` — это аналог `skb:consumer_skb`. Точка трассировки `consumer_skb` срабатывает, когда обработка буфера `skb` идет по обычному пути, а `kfree_skb` — в случае появления необычных событий, которые стоит исследовать.

Исходный код `skbdrop(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing unusual skb drop stacks. Hit Ctrl-C to end.\n");
    system("nstat > /dev/null");
}

tracepoint:skb:kfree_skb
{
    @[kstack(8)] = count();
}

END
{
    system("nstat; nstat -rs > /dev/null");
}
```

На запуске, в обработчике события BEGIN, производится сброс счетчиков `nstat(8)` в ноль. В конце, в обработчике END, инструмент `nstat(8)` используется повторно для вывода счетчиков и возврата `nstat(8)` в исходное состояние (`-rs`). Это будет мешать другим пользователям `nstat(8)` во время трассировки. Обратите внимание, что из-за использования `system()` инструмент следует запускать с параметром `bpftool --unsafe`.

10.3.29. skblife

skblife(8)¹ измеряет продолжительность жизни буферов sk_buff(skb) — объектов, используемых для передачи пакетов через ядро. Измерение продолжительности жизни помогает выявить задержки в сетевом стеке, включая задержки, вызванные блокировкой пакетов. Вот пример вывода, полученный на нагруженном производственном сервере:

```
# skblife.bt
Attaching 6 probes...
^C
```

[illegible]

¹ Немного истории: я написал его для этой книги 4 апреля 2019 года.

[32K, 64K)	11025	@@	
[64K, 128K)	3277	@@@@@@@@@@@@@@@@@	
[128K, 256K)	2954	@@@@@@@@@@@@@@@	
[256K, 512K)	1608	@@@@@@@	
[512K, 1M)	1594	@@@@@@	
[1M, 2M)	583	@@	
[2M, 4M)	435	@	
[4M, 8M)	317	@	
[8M, 16M)	104		
[16M, 32M)	10		
[32M, 64M)	12		
[64M, 128M)	1		
[128M, 256M)	1		

Мы видим, что типичная продолжительность жизни буферов `sk_buff` составила от 16 до 64 микросекунд, но есть единичные выбросы, достигающие сегмента от 128 до 256 миллисекунд. Их можно дополнительно исследовать с помощью других инструментов, включая предыдущие инструменты трассировки задержек. Так можно выяснить, не связана ли задержка с очередями.

Инструмент трассирует операции распределения сегментов кэша ядра и определяет, когда создаются и освобождаются буферы `sk_buff`. Такое распределение может происходить очень часто, поэтому в нагруженных системах у этого инструмента бывает заметный или значительный оверхед. Его можно использовать для краткосрочного анализа, но не для продолжительного мониторинга.

Исходный код `skblife(8)`:

```
#!/usr/local/bin/bpftrace
```

```
kprobe:kmem_cache_alloc,
kprobe:kmem_cache_alloc_node
{
    $cache = arg0;
    if ($cache == *kaddr("skbuff_fclone_cache") ||
        $cache == *kaddr("skbuff_head_cache")) {
        @is_skb_alloc[tid] = 1;
    }
}

kretprobe:kmem_cache_alloc,
kretprobe:kmem_cache_alloc_node
/@is_skb_alloc[tid]/
{
    delete(@is_skb_alloc[tid]);
    @skb_birth[retval] = nsecs;
}

kprobe:kmem_cache_free
/@skb_birth[arg1]/
{
    @skb_residency_nsecs = hist(nsecs - @skb_birth[arg1]);
    delete(@skb_birth[arg1]);
}
```

```
END
{
    clear(@is_skb_alloc);
    clear(@skb_birth);
}
```

На входе в функции `kmem_cache_alloc()` инструмент проверяет аргумент, чтобы узнать, является ли он кэшем `sk_buff`. Если это так, то на выходе адрес `sk_buff` связывается с отметкой времени, которая затем извлекается на входе в `kmem_cache_free()`.

У такого подхода есть недостатки: `sk_buff` может сегментироваться механизмом GSO на несколько других буферов `sk_buff`, аналогично несколько буферов `sk_buff` может объединяться механизмом GRO в один буфер. Также буферы `sk_buff` могут объединяться реализацией протокола TCP (`tcp_try_coalesce()`). То есть оценка продолжительности жизни полного пакета может оказаться заниженной. Этот инструмент можно усовершенствовать: предусмотреть копирование исходной отметки времени создания `sk_buff` во вновь создаваемые буферы.

Поскольку этот инструмент трассирует все операции выделения и освобождения памяти в кэше ядра (не только для `sk_buff`), оверхед может быть значительным. В будущем появится возможность уменьшить их. В ядре уже есть точки трассировки `skb:consumer_skb` и `skb:free_skb`. Если появится точка трассировки выделения памяти для `skb`, ее можно будет использовать вместо зондов ядра `kprobe` и уменьшить оверхед, ограничившись исследованием только операций выделения памяти для `sk_buff`.

10.3.30. `ieee80211scan`

`ieee80211scan(8)`¹ трассирует операции сканирования Wi-Fi согласно IEEE 802.11. Например:

```
# ieee80211scan.bt
Attaching 5 probes...
Tracing ieee80211 SSID scans. Hit Ctrl-C to end.
13:55:07 scan started (on-CPU PID 1146, wpa_supplicant)
13:42:11 scanning channel 2GHZ freq 2412: beacon_found 0
13:42:11 scanning channel 2GHZ freq 2412: beacon_found 0
13:42:11 scanning channel 2GHZ freq 2412: beacon_found 0
[...]
13:42:13 scanning channel 5GHZ freq 5660: beacon_found 0
13:42:14 scanning channel 5GHZ freq 5785: beacon_found 1
13:42:14 scanning channel 5GHZ freq 5785: beacon_found 1
13:42:14 scanning channel 5GHZ freq 5785: beacon_found 1
13:42:14 scanning channel 5GHZ freq 5785: beacon_found 1
13:42:14 scanning channel 5GHZ freq 5785: beacon_found 1
13:42:14 scan completed: 3205 ms
```

¹ Я написал его для этой книги 23 апреля 2019 года. Свой первый трассировщик сканирования Wi-Fi я написал в номере отеля в 2004 году, когда у меня не получилось подключить ноутбук к Wi-Fi. Система не выводила никаких сообщений об ошибках, и я хотел узнать причину. Тогда я придумал похожий сканер на основе DTrace, но не помню, чтобы публиковал его код.

Судя по этому выводу, процесс сканирования был инициирован процессом `wpa_supplicant`, который перебирает разные каналы и частоты. Сканирование заняло 3205 мс. Полученный результат дает понимание для устранения проблем с Wi-Fi.

Инструмент трассирует процедуры сканирования `ieee80211`. Оверхед должен быть незначительным, поскольку процедуры выполняются нечасто.

Исходный код `ieee80211scan(8)`:

```
#!/usr/local/bin/bpftrace

#include <net/mac80211.h>

BEGIN
{
    printf("Tracing ieee80211 SSID scans. Hit Ctrl-C to end.\n");
    // из include/uapi/linux/nl80211.h:
    @band[0] = "2GHZ";
    @band[1] = "5GHZ";
    @band[2] = "60GHZ";
}

kprobe:ieee80211_request_scan
{
    time("%H:%M:%S ");
    printf("scan started (on-CPU PID %d, %s)\n", pid, comm);
    @start = nsecs;
}

kretprobe:ieee80211_get_channel
/retval/
{
    $ch = (struct ieee80211_channel *)retval;
    $band = 0xff & *retval; // $ch->band; обходное решение ошибки #776
    time("%H:%M:%S ");
    printf("scanning channel %s freq %d: beacon_found %d\n",
        @band[$band], $ch->center_freq, $ch->beacon_found);
}

kprobe:ieee80211_scan_completed
/@start/
{
    time("%H:%M:%S ");
    printf("scan completed: %d ms\n", (nsecs - @start) / 1000000);
    delete(@start);
}

END
{
    clear(@start); clear(@band);
}
```

Можно добавить дополнительную информацию, чтобы показать различные флаги и настройки, используемые при сканировании. Обратите внимание: сейчас этот

инструмент предполагает, что только одно сканирование будет активно одновременно и оно имеет глобальную временную метку @start. Если сканирование может выполняться параллельно, потребуется ключ для привязки временной метки к каждому сканированию.

10.3.31. Другие инструменты

Вот еще несколько инструментов BPF:

- **solisten(8)**: инструмент BCC для вывода информации о прослушивающих сокетах¹;
- **tcpstates(8)**: инструмент BCC, выводит сообщение для каждого изменения состояния сеанса TCP, содержащее IP-адреса и номера портов, а также продолжительность пребывания в каждом состоянии;
- **tcpdrop(8)**: инструмент BCC и bpftrace, выводит IP-адрес и информацию о состоянии соединения TCP, а также трассировку стека ядра для пакетов, отброшенных функцией ядра `tcp_drop()`;
- **sofidsnoop(8)**: инструмент BCC для трассировки файловых дескрипторов, которые передаются через сокеты Unix;
- **profile(8)**: описан в главе 6, выбирает трассировки стека ядра и может использоваться для оценки времени выполнения сетевого кода;
- **hardirqs(8) и softirqs(8)**: описаны в главе 6, могут использоваться для измерения времени, затрачиваемого на обработку аппаратных и программных прерываний, сгенерированных сетевым стеком;
- **filetype(8)**: описан в главе 8, трассирует вызовы функций `vfs_read()` и `vfs_write()`, может использоваться для выявления операций чтения и записи с сокетами через индексные узлы.

Пример вывода инструмента `tcpstates(8)`:

```
# tcpstates
SKADDR          C-PID C-COMM LADDR      LPORT  RADDR      RPORT OLDSTATE -> NEWSTATE  MS
ffff88864fd55a00 3294 record 127.0.0.1 0      127.0.0.1 28527 CLOSE  -> SYN_SENT 0.00
ffff88864fd55a00 3294 record 127.0.0.1 0      127.0.0.1 28527 SYN_SENT -> ESTABLISHED 0.08
ffff88864fd56300 3294 record 127.0.0.1 0      0.0.0.0 0      LISTEN  -> SYN_RECV 0.00
[...]
```

Он использует точку трассировки `sock:inet_sock_set_state`.

10.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечислены однострочные сценарии для BCC и bpftrace. Там, где это возможно, один и тот же сценарий реализуется с использованием BCC и bpftrace.

¹ `solisten(8)` был добавлен Жан-Тиаром Ле Биго (Jean-Tiare Le Bigot) 4 марта 2016 года.

10.4.1. ВСС

Подсчитывает вызовы connect(2), завершившиеся ошибкой, группируя их по кодам ошибок:

```
argdist -C 't:syscalls:sys_exit_connect():int:args->ret:args->ret<0'
```

Подсчитывает вызовы connect(2) по трассировкам стека в пространстве пользователя:

```
stackcount -U t:syscalls:sys_enter_connect
```

Выводит гистограмму размеров отправленных сообщений TCP в байтах:

```
argdist -H 'p::tcp_sendmsg(void *sk, void *msg, int size):int:size'
```

Выводит гистограмму размеров полученных сообщений TCP в байтах:

```
argdist -H 'r::tcp_recvmsg():int:$retval:$retval>0'
```

Подсчитывает вызовы всех функций TCP (добавляет значительный оверхед в работу TCP):

```
funccount 'tcp_*
```

Выводит гистограмму с размерами отправленных пакетов UDP:

```
argdist -H 'p::udp_sendmsg(void *sk, void *msg, int size):int:size'
```

Выводит гистограмму с размерами полученных пакетов UDP:

```
argdist -H 'r::udp_recvmsg():int:$retval:$retval>0'
```

Подсчитывает вызовы всех функций UDP (добавляет значительный оверхед в работу UDP):

```
funccount 'udp_*
```

Подсчитывает трассировки стека операций передачи:

```
stackcount t:net:net_dev_xmit
```

Подсчитывает вызовы функций ieee80211 (добавляет значительный оверхед в операции с пакетами):

```
funccount 'ieee80211_*
```

Подсчитывает вызовы всех функций драйвера устройства ixgbevf (добавляет значительный оверхед в работу ixgbevf):

```
funccount 'ixgbevf_*
```


10.4.2. bpftrace

Подсчитывает вызовы `accept(2)` по идентификаторам (PID) и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_accept* { @[pid, comm] = count(); }'
```

Подсчитывает вызовы `connect(2)` по идентификаторам (PID) и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_connect { @[pid, comm] = count(); }'
```

Подсчитывает вызовы `connect(2)`, завершившиеся ошибкой, группируя их по именам процессов и кодам ошибок:

```
bpftrace -e 't:syscalls:sys_exit_connect /args->ret < 0/ { @[comm, - args->ret] = count(); }'
```

Подсчитывает вызовы `connect(2)` по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:syscalls:sys_enter_connect { @[ustack] = count(); }'
```

Подсчитывает операции приема/передачи с сокетами по направлениям, идентификаторам (PID) и именам процессов, выполняющихся на процессоре¹:

```
bpftrace -e 'k:sock_sendmsg,k:sock_recvmsg { @[func, pid, comm] = count(); }'
```

Подсчитывает операции приема/передачи с сокетами по идентификаторам (PID) и именам процессов, выполняющихся на процессоре:

```
bpftrace -e 'kr:sock_sendmsg,kr:sock_recvmsg /(int32)retval > 0/ { @[pid, comm] = sum((int32)retval); }'
```

Подсчитывает активные TCP-соединения по идентификаторам (PID) и именам процессов:

```
bpftrace -e 'k:tcp_v*_connect { @[pid, comm] = count(); }'
```

Подсчитывает пассивные TCP-соединения по идентификаторам (PID) и именам процессов:

```
bpftrace -e 'k:inet_csk_accept { @[pid, comm] = count(); }'
```

Подсчитывает операции приема и передачи по протоколу TCP:

```
bpftrace -e 'k:tcp_sendmsg,k:tcp*recvmsg { @[func] = count(); }'
```

¹ Описанные выше системные вызовы выполняются в контексте процесса, где можно доверять значениям `pid` и `comm`. Зонды `kprobes` находятся глубже в ядре, и к моменту их вызова процесс, использующий соединение, может оставить процессор. Это означает, что значения `pid` и `comm` в `bpftrace` могут быть недостоверными. Такое случается редко, но все же случается.

Подсчитывает операции приема и передачи по протоколу TCP с группировкой по идентификаторам (PID) и именам процессов:

```
bpfttrace -e 'k:tcp_sendmsg,k:tcp_recvmmsg { @[func, pid, comm] = count(); }'
```

Выводит гистограмму размеров отправленных сообщений TCP в байтах:

```
bpfttrace -e 'k:tcp_sendmsg { @send_bytes = hist(arg2); }'
```

Выводит гистограмму размеров полученных сообщений TCP в байтах:

```
bpfttrace -e 'kr:tcp_recvmmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчитывает повторные передачи TCP по типам и именам удаленных узлов (предполагается использование версии протокола IPv4):

```
bpfttrace -e 't:tcp:tcp_retransmit_* { @[probe, ntop(2, args->saddr)] = count(); }'
```

Подсчитывает вызовы всех функций TCP (добавляет значительный оверхед в работу TCP):

```
bpfttrace -e 'k:tcp_* { @[func] = count(); }'
```

Подсчитывает операции приема и передачи по протоколу UDP с группировкой по идентификаторам (PID) и именам процессов:

```
bpfttrace -e 'k:udp*_sendmsg,k:udp*_recvmmsg { @[func, pid, comm] = count(); }'
```

Выводит гистограмму с размерами отправленных пакетов UDP:

```
bpfttrace -e 'k:udp_sendmsg { @send_bytes = hist(arg2); }'
```

Выводит гистограмму с размерами полученных пакетов UDP:

```
bpfttrace -e 'kr:udp_recvmmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчитывает вызовы всех функций UDP (добавляет значительный оверхед в работу UDP):

```
bpfttrace -e 'k:udp_* { @[func] = count(); }'
```

Подсчитывает трассировки стека в ядре, связанные с отправкой пакетов:

```
bpfttrace -e 't:net:net_dev_xmit { @[kstack] = count(); }'
```

Выводит гистограмму затрат процессора на прием для каждого устройства:

```
bpfttrace -e 't:net:netif_receive_skb { @[str(args->name)] = lhist(cpu, 0, 128, 1); }'
```

Подсчитывает вызовы функций ieee80211 (добавляет значительный оверхед в операции с пакетами):

```
bpfttrace -e 'k:ieee80211_* { @[func] = count()}'
```

Подсчитывает вызовы всех функций драйвера устройства `ixgbev` (добавляет значительный оверхед в работу `ixgbev`):

```
bpfftrace -e 'k:ixgbev_* { @[func] = count(); }'
```

Подсчитывает срабатывания всех точек трассировки драйвера устройства `iwl` (добавляет значительный оверхед в работу `iwl`):

```
bpfftrace -e 't:iwlfwifio:* { @[probe] = count(); }'
```

10.4.3. Примеры использования однострочных сценариев BPF

Чтобы показать работу однострочных сценариев, я включил сюда примеры их вывода.

Подсчет трассировок стека в ядре, связанных с отправкой пакетов

```
# bpfftrace -e 't:net:net_dev_xmit { @[kstack] = count(); }'
```

```
Attaching 1 probe...
```

```
^C
```

```
[...]
```

```
@[
```

```
dev_hard_start_xmit+945
sch_direct_xmit+882
__qdisc_run+1271
__dev_queue_xmit+3351
dev_queue_xmit+16
ip_finish_output2+3035
ip_finish_output+1724
ip_output+444
ip_local_out+117
__ip_queue_xmit+2004
ip_queue_xmit+69
__tcp_transmit_skb+6570
tcp_write_xmit+2123
__tcp_push_pending_frames+145
tcp_rcv_established+2573
tcp_v4_do_rcv+671
tcp_v4_rcv+10624
ip_protocol_deliver_rcu+185
ip_local_deliver_finish+386
ip_local_deliver+435
ip_rcv_finish+342
ip_rcv+212
__netif_receive_skb_one_core+308
__netif_receive_skb+36
netif_receive_skb_internal+168
napi_gro_receive+953
ena_io_poll+8375
```

```

net_rx_action+1750
__do_softirq+558
irq_exit+348
do_IRQ+232
ret_from_intr+0
native_safe_halt+6
default_idle+146
arch_cpu_idle+21
default_idle_call+59
do_idle+809
cpu_startup_entry+29
start_secondary+1228
secondary_startup_64+164
]: 902
@[
dev_hard_start_xmit+945
sch_direct_xmit+882
__qdisc_run+1271
__dev_queue_xmit+3351
dev_queue_xmit+16
ip_finish_output2+3035
ip_finish_output+1724
ip_output+444
ip_local_out+117
__ip_queue_xmit+2004
ip_queue_xmit+69
__tcp_transmit_skb+6570
tcp_write_xmit+2123
__tcp_push_pending_frames+145
tcp_push+1209
tcp_sendmsg_locked+9315
tcp_sendmsg+44
inet_sendmsg+278
sock_sendmsg+188
sock_write_iter+740
__vfs_write+1694
vfs_write+341
ksys_write+247
__x64_sys_write+115
do_syscall_64+339
entry_SYSCALL_64_after_hwframe+68
]: 10933

```

Этот однострочный сценарий производит многостраничный вывод. Я включил в пример только две последние трассировки стека. Стек отражает путь выполнения системного вызова `write(2)`, пролегающий через VFS, сокеты, TCP, IP, сетевое устройство и, наконец, начало передачи в драйвер. Это стек вызовов от приложения до драйвера устройства.

Первая трассировка более интересная. Стек начинается с того, что поток, выполняющийся во время бездействия системы, получает прерывание, выполняет программное прерывание `net_rx_action()`, функцию `ena_io_poll()` драйвера `ena`, процедуру приема в сетевом интерфейсе NAPI (new API), затем `IP`, `tcp_rcv_established()`, а затем ... `__tcp_push_pending_frames()`. Настоящий путь выполнения кода:

`tcp_rcv_established()` -> `tcp_data_snd_check()` -> `tcp_push_pending_frames()` -> `__tcp_push_pending_frames()`. Но две функции в середине очень короткие и были встроены компилятором, из-за чего не попали в трассировку стека. Так TCP проверяет ожидающие передачи во время приема.

Подсчет вызовов всех функций драйвера устройства ixgbevf (добавляет значительный оверхед в работу ixgbevf)

```
# bpftrace -e 'k:ixgbevf_* { @[func] = count(); }'
Attaching 116 probes...
^C
```

```
@[ixgbevf_get_link_ksettings]: 2
@[ixgbevf_get_stats]: 2
@[ixgbevf_obtain_mbx_lock_vf]: 2
@[ixgbevf_read_mbx_vf]: 2
@[ixgbevf_service_event_schedule]: 3
@[ixgbevf_service_task]: 3
@[ixgbevf_service_timer]: 3
@[ixgbevf_check_for_bit_vf]: 5
@[ixgbevf_check_for_rst_vf]: 5
@[ixgbevf_check_mac_link_vf]: 5
@[ixgbevf_update_stats]: 5
@[ixgbevf_read_reg]: 21
@[ixgbevf_alloc_rx_buffers]: 36843
@[ixgbevf_features_check]: 37842
@[ixgbevf_xmit_frame]: 37842
@[ixgbevf_msix_clean_rings]: 66417
@[ixgbevf_poll]: 67013
@[ixgbevf_maybe_stop_tx]: 75684
@[ixgbevf_update_itr.isra.39]: 132834
```

С помощью этих зондов kprobes можно подробно изучить внутреннее устройство драйверов сетевых устройств. Не забудьте также проверить поддержку драйвером точек трассировки, как показано в следующем примере.

Подсчет срабатываний всех точек трассировки драйвера устройства iwl (добавляет значительный оверхед в работу iwl)

```
# bpftrace -e 't:iwlfwifio:* { @[probe] = count(); }'
Attaching 15 probes...
^C
```

```
@[tracepoint:iwlfwifio:iwlfwifio_dev_hcmd]: 39
@[tracepoint:iwlfwifio:iwlfwifio_dev_irq]: 3474
@[tracepoint:iwlfwifio:iwlfwifio_dev_tx]: 5125
@[tracepoint:iwlfwifio:iwlfwifio_dev_iowrite8]: 6654
@[tracepoint:iwlfwifio:iwlfwifio_dev_ict_read]: 7095
@[tracepoint:iwlfwifio:iwlfwifio_dev_rx]: 7493
@[tracepoint:iwlfwifio:iwlfwifio_dev_iowrite32]: 19525
```

Этот пример показал только две группы точек трассировки iwl из нескольких.

10.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Упражнения можно выполнить с помощью `bpfttrace` или ВСС, если явно не указано иное:

1. Напишите инструмент `solife(8)` для вывода продолжительности сеанса от вызова `connect(2)` или `accept(2)` (и их вариантов) до вызова `close(2)` с дескриптором данного сокета. Он может быть похож на `tcplife(8)`, но не обязательно должен выводить все те же поля (часть из них получить будет очень сложно).
2. Напишите инструмент `tcpbind(8)` для трассировки событий связывания сокетов с локальными IP-адресом и портом.
3. Добавьте в `tcpwin.bt` вывод событий типа «retrans» с полями, содержащими адрес структуры сокета в памяти и время.
4. Добавьте в `tcpwin.bt` вывод событий типа «new» с полями, содержащими адрес структуры сокета в памяти, время, IP-адрес и номер порта ТСП. Эта строка должна выводиться в момент, когда сеанс ТСП достигнет состояния «established».
5. Измените `tcplife(8)` так, чтобы данные о соединении выводились в формате DOT, а затем постройте график с помощью ПО для построения графиков (например, `GraphViz`).
6. Напишите инструмент `udplife(8)`, отображающий продолжительность UDP-соединений, по аналогии с `tcplife(8)`.
7. Добавьте в `ipecn.bt` обработку получения событий уведомления о перегрузке (СЕ), а также поддержку протокола IPv6. События СЕ можно генерировать на уровне дисциплины очереди, например `netem`. Команда в следующем примере заменяет текущую дисциплину в `eth0` на ту, которая вызывает 1% событий ECN СЕ:

```
tc qdisc replace dev eth0 root netem loss 1% ecn
```

Если вы решите использовать эту дисциплину во время разработки, имейте в виду, что она добавляет события СЕ на более низком уровне, чем IP. То есть если вы попытаете трассировать, скажем, `ip_output()`, то можете не увидеть события СЕ, потому что они добавляются позже.

8. (Усложненное.) Напишите инструмент, отображающий время от отправки запроса до получения ответа ТСП по узлам. Он может отображать среднее значение RTT по узлам или гистограммы RTT для каждого узла. Инструмент может запоминать время отправки пакетов по порядковому номеру и привязывать отметку времени в момент отправки АСК, использовать структуру `tcp_sock->rtt_min` или какой-то другой подход. При использовании первого подхода заголовок ТСП можно прочитать через указатель `sk_buff *` в `$skb` (с использованием средств `bpfttrace`), как показано ниже:

```
$tcp_h = (struct tcphdr *)($skb->head + $skb->transport_header);
```

9. (Усложненное, не решено.) Напишите инструмент, отображающий задержку ARP или обнаружение соседей IPv6 либо для каждого события, либо в виде гистограммы.
10. (Усложненное, не решено.) Напишите инструмент, отображающий продолжительность жизни `sk_buff`, обрабатывая (когда и если необходимо) `GRO`, `GSO`, `tcp_try_coalesce()`, `skb_split()`, `skb_append()`, `skb_insert()` и другие события, изменяющие `sk_buff` в течение его жизни. Этот инструмент получится намного сложнее, чем `skblife(8)`.
11. (Усложненное, не решено.) Напишите инструмент, разбивающий период жизни `sk_buff` (получаемый инструментом в задании 10) на компоненты или состояния ожидания.
12. (Усложненное, не решено.) Напишите инструмент, отображающий задержку, вызванную действием алгоритма `Pacing`.
13. (Усложненное, не решено.) Напишите инструмент, отображающий задержку, вызванную действием алгоритма `Byte Queue Limits (BQL)`.

10.6. ИТОГИ

В этой главе кратко описаны характеристики сетевого стека Linux и их анализ с помощью традиционных инструментов: `netstat(8)`, `sar(1)`, `ss(8)` и `tcpdump(8)`. Затем показано, как инструменты `BPF` используются для обеспечения расширенной наблюдаемости уровня сокетов, `TCP`, `UDP`, `ICMP`, `qdiscs`, очередей и драйверов сетевого устройства. Такая наблюдаемость включает показ новых соединений и их продолжительности жизни, задержки при создании соединений и до получения первого байта, размер очередей пакетов `SYN`, повторные передачи `TCP` и другие события.

Глава 11

БЕЗОПАСНОСТЬ

В этой главе кратко описана защищенность BPF и приемы использования BPF для анализа безопасности, а также различные инструменты, которые пригодятся и для целей безопасности, и для наблюдения за производительностью. Эти инструменты можно использовать для обнаружения вторжений, создания белых списков для исполняемых файлов, обычных и требующих повышенных привилегий, а также для применения политик.

Цели обучения:

- познакомиться со сценариями использования BPF для анализа безопасности;
- отображать порядок выполнения процессов для обнаружения возможных вредоносных программ;
- отображать TCP-соединения и сбросы для выявления подозрительной активности;
- изучить возможности создания белых списков в Linux;
- познакомиться с другими источниками информации для анализа, такими как логирование активности в командной оболочке и в консоли.

Эта глава начинается с введения в безопасность, а затем описывает возможности BPF, настройки безопасности BPF, стратегии и инструменты BPF.

11.1. ОСНОВЫ

Термин *безопасность* охватывает широкий спектр задач, в том числе:

- Анализ безопасности:
 - выявление подозрительной активности в масштабе реального времени;
 - отладка привилегий;
 - белые списки исполняемых файлов;
 - реверс-инжиниринг вредоносного ПО.

- Мониторинг:
 - собственная проверка;
 - системы обнаружения вторжений на узлах сети;
 - системы обнаружения вторжений в контейнерах.
- Применение политик:
 - межсетевые брандмауэры;
 - обнаружение вредоносных программ, динамическая блокировка пакетов и прочие превентивные меры.

Анализ безопасности имеет много общего с анализом производительности, потому что может включать проверку широкого спектра ПО.

11.1.1. Возможности BPF

BPF поможет в решении всех упомянутых выше задач, включая анализ, мониторинг и применение политик. Для оценки безопасности инструменты BPF помогут ответить на вопросы:

- Какие процессы выполняются?
- Какие сетевые соединения установлены? Какими процессами?
- Какие системные привилегии и какими процессами запрашиваются?
- Какие ошибки отказа в разрешении есть в системе?
- Вызывается ли эта функция (в пространстве ядра или пользователя) с этими конкретными аргументами (для проверки активных эксплойтов)?

Еще один способ представить возможности BPF в отношении анализа и мониторинга — показать цели, доступные для трассировки (рис. 11.1)¹.

На рис. 11.1 показано множество конкретных целей, однако с помощью зондов `uprobes` и `kprobes` можно инструментировать любую функцию в пространстве пользователя или ядра, что пригодится для обнаружения уязвимости нулевого дня.

Обнаружение уязвимости нулевого дня

Иногда необходимо срочно определить, не используют ли злоумышленники только что обнаруженную уязвимость ПО, и в идеале это надо выяснить сразу же после ее обнаружения (нулевой день). Трассировщик `bpfttrace` особенно хорошо подходит для решения таких задач, поскольку его простой язык программирования позволяет создавать нестандартные инструменты за считанные минуты, а сам он имеет

¹ Мы с Алексом Маэстретти (Alex Maestretti) представили эту схему в докладе «Linux Monitoring at Scale with eBPF» на конференции BSidesSF в 2017 году [114].

доступ не только к точкам трассировки и событиям USDT, но и к зондам kprobes и uprobes и их аргументам.

Linux Security Monitoring Probes



Netflix 2017

Рис. 11.1. Цели мониторинга безопасности, доступные для BPF

В качестве примера рассмотрим уязвимость Docker, обнаруженную, когда я работал над этой книгой. Она позволяет проводить атаки *symlink-race* [115], предполагает обращение к системному вызову *renameat2(2)* с флагом *RENAME_EXCHANGE* в цикле, а также использование команды *docker cp*.

Такие атаки можно обнаружить несколькими способами. Поскольку в моих производственных системах вызов *renameat2(2)* с флагом *RENAME_EXCHANGE* — это необычное действие (естественных случаев его применения я не обнаружил), то один из способов обнаружить попытки использовать эту уязвимость — трассировать комбинации этого системного вызова и флага. Например, для трассировки всех контейнеров на хосте можно запустить такой сценарий:

```
# bpftrace -e 't:syscalls:sys_enter_renameat2 /args->flags == 2/ { time();
printf("%s RENAME_EXCHANGE %s <-> %s\n", comm, str(args->oldname),
str(args->newname)); }'
Attaching 1 probe...
22:03:47
symlink_swap RENAME_EXCHANGE totally_safe_path <-> totally_safe_path-stashed
22:03:47
```

```
symlink_swap RENAME_EXCHANGE totally_safe_path <-> totally_safe_path-stashed
22:03:47
symlink_swap RENAME_EXCHANGE totally_safe_path <-> totally_safe_path-stashed
[...]
```

В обычных условиях этот короткий сценарий ничего не выводит, но здесь он вывел поток данных, так как код проверки концепции уязвимости выполнялся в качестве теста. Вывод включает отметки времени, имена процессов и аргументы системного вызова `renameat2(2)`. Можно было бы организовать и трассировку процесса `docker` `sr`, работающего с символическими ссылками — либо через системные вызовы, либо через вызовы функций ядра.

Могу представить будущее, в котором сообщение об уязвимости станет сопровождаться однострочником `bpfttrace` или инструментом ее обнаружения. На основе таких инструментов можно создать целую систему обнаружения вторжений для использования в инфраструктуре компании. Она не будет отличаться от некоторых других систем обнаружения сетевых вторжений вроде `Snort` [116], которая использует общие правила для обнаружения новых червей.

Мониторинг безопасности

Программы трассировки для BPF можно использовать для мониторинга безопасности и обнаружения вторжений. Текущие решения мониторинга часто используют загружаемые модули ядра, чтобы обеспечить видимость пакетов и событий ядра. Однако такие модули увеличивают риск появления ошибок и уязвимостей в ядре. BPF-программы проходят через верификатор и используют существующие технологии ядра, что делает их более безопасными и надежными.

Кроме того, BPF оптимизирован для эффективной трассировки. Во внутреннем исследовании 2016 года я сравнил `overhead auditd` и аналогичной программы для BPF. Последняя имела `overhead` в 6 раз меньше [117].

Важная особенность мониторинга в BPF — поведение при экстремальной нагрузке. Выходные буферы и карты BPF имеют ограниченные размеры и могут исчерпаться, в результате чего события перестанут фиксироваться. Эта особенность может быть использована злоумышленником — он может попытаться воспрепятствовать логированию или применению политик, завалив систему событиями. BPF знает, когда происходит исчерпание, и может сообщить об этом в пространство пользователя, чтобы позволить принять соответствующие меры. Любое решение безопасности, созданное с помощью средств трассировки BPF, должно регистрировать такие события, чтобы удовлетворить требованиям к безотказности.

Другой подход — добавить карту для каждого процессора со счетчиками важных событий. В отличие от выходного буфера или карт с ключами, после добавления в BPF карт фиксированных счетчиков для каждого процессора исчез риск потери событий. Это можно использовать в дополнение к выводу событий с более подробной информацией: детали могут быть потеряны, но счетчики событий — нет.

Применение политик

Многие технологии применения политик уже используют BPF. Хотя эта тема выходит за рамки книги, тем не менее она важна и заслуживает краткого описания. Вот эти технологии:

- **seccomp**: механизм безопасных вычислений (secure computing, seccomp) позволяет выполнять программы BPF (сейчас классические), определяющие доступность системных вызовов [118]. seccomp поддерживает программируемые действия, такие как остановка вызывающего процесса (SECCOMP_RET_KILL_PROCESS) и возврат ошибки (SECCOMP_RET_ERRNO). Также программы BPF могут выгружать сложные решения в программы пространства пользователя (SECCOMP_RET_USER_NOTIF), блокирующие процесс, пока вспомогательная программа в пространстве пользователя не получит уведомление через файловый дескриптор. Такая программа может читать и обрабатывать события, а также отвечать на них, записывая структуру `seccomp_notif_resp` в тот же файловый дескриптор [119].
- **Cilium**: обеспечивает прозрачную защиту сетевых соединений и балансировку нагрузки для контейнеров или процессов. Использует комбинацию программ BPF на разных уровнях: XDP, cgroup и tc (traffic control — управление трафиком). Например, основной путь передачи сетевых данных на уровне tc использует `sch_clsact qdisc` в сочетании с программой BPF через `cls_bpf`, которая может изменять, пересылать или отбрасывать пакеты [24] [120] [121].
- **bpfilter**: это проверка концепции (proof of concept, PoC) полной замены брандмауэра iptables на BPF. Для облегчения миграции с iptables на BPF можно переслать набор правил iptables вспомогательному модулю в пространстве пользователя, который преобразует его в программу BPF [122] [123].
- **Landlock**: модуль безопасности на основе BPF для управления доступом к ресурсам ядра с помощью BPF [124]. Один из примеров его использования — ограничение доступа к отдельным областям файловой системы на основе карты индексных узлов BPF, которая может обновляться из пользовательского пространства.
- **KRSI**: Kernel Runtime Security Instrumentation — новый модуль безопасности для LSM от Google для поддержки расширяемого контроля и управления. Использует новый тип программы BPF, `BPF_PROG_TYPE_KRSI` [186].

В релиз Linux 5.3 должна быть включена новая вспомогательная функция BPF `bpf_send_signal()` [125]. Это позволит создавать программы нового типа для принудительного применения политик, способные посылать SIGKILL и другие сигналы процессам напрямую из программ BPF, без использования seccomp. Продолжая предыдущий пример обнаружения уязвимости, представьте себе программу `bpfttrace`, которая не только обнаруживает уязвимость, но и немедленно убивает процесс, использующий ее. Например:

```
bpfttrace --unsafe -e 't:syscalls:sys_enter_renameat2 /args->flags == 2/ {
    time(); printf("killing PID %d %s\n", pid, comm); signal(9); }'
```

Такие инструменты можно использовать в качестве временного решения, пока ПО не будет исправлено¹. Использование функции `signal()` требует осторожности: этот пример убьет *любой* процесс, злонамеренный или добропорядочный, если тот обратится к системному вызову `renameat2(2)` и передаст в аргументе флаг `RENAME_EXCHANGE`.

Также можно использовать другие сигналы, например `SIGABRT`, чтобы получить дамп памяти процесса для дальнейшего изучения вредоносного ПО.

Пока функция `bpf_send_signal()` не станет доступной, можно завершать процессы с помощью трассировщика пользовательского пространства по событиям из буфера `perf`. Например, вызовом `system()` из сценария для `bpftrace`:

```
bpftrace --unsafe -e 't:syscalls:sys_enter_renameat2 /args->flags == 2/ {
    time(); printf("killing PID %d %s\n", pid, comm);
    system("kill -9 %d", pid); }'
```

Функция `system()` действует асинхронно (см. главу 5). Ее вызов передается через выходной буфер `perf`, а через какое-то время выполняется механизмом `bpftrace`. Из-за этого возникает задержка между обнаружением и выполнением, что в некоторых средах может быть неприемлемо. Функция `bpf_send_signal()` решает эту проблему, посылая сигнал немедленно — в контексте ядра во время выполнения программы BPF.

11.1.2. Непривилегированные пользователи BPF

Рядовые пользователи, в частности те, кто не обладает привилегией `CAP_SYS_ADMIN`, начиная с Linux 5.2, могут использовать BPF только для фильтрации сокетов. Соответствующая проверка находится в системном вызове `bpf(2)`, в файле `kernel/bpf/syscall.c`:

```
if (type != BPF_PROG_TYPE_SOCKET_FILTER &&
    type != BPF_PROG_TYPE_CGROUP_SKB &&
    !capable(CAP_SYS_ADMIN))
    return -EPERM;
```

Этот код также позволяет программам `cgroup skb` проверять и отбрасывать пакеты `cgroup`. Но этим программам требуется привилегия `CAP_NET_ADMIN` для подключения к `BPF_CGROUP_INET_INGRESS` и `BPF_CGROUP_INET_EGRESS`.

Вызов `bpf(2)` без привилегии `CAP_SYS_ADMIN` завершится ошибкой `EPERM`, а инструменты BCC выведут сообщение: «Need super-user privileges to run» («Для запуска необходимы привилегии суперпользователя»). Сейчас программы `bpftrace` проверяют идентификатор пользователя `UID 0`. Если идентификатор текущего пользователя отличен от 0, то сообщают: «bpftrace currently only supports running as

¹ В прошлом Red Hat публиковала аналогичные инструменты трассировки SystemTap для устранения уязвимостей, например Bugzilla [126].

the root user» («bpfttrace поддерживает работу только от имени пользователя root»). Поэтому руководства по инструментам BPF из этой книги находятся в разделе 8 map: это все инструменты суперпользователя.

Когда-нибудь BPF должен будет поддерживать непривилегированный доступ не только для фильтрации сокетов¹. Один из конкретных примеров — это контейнерная среда, где доступ к вмещающему узлу ограничен и желательно иметь возможность запускать инструменты BPF из самих контейнеров. (Этот вариант упоминается в главе 15.)

11.1.3. Настройка безопасности BPF

Для настройки безопасности BPF есть несколько системных параметров. Их можно настроить с помощью команды `sysctl(8)` или файлов в `/proc/sys`. Вот эти параметры:

```
# sysctl -a | grep bpf
kernel.unprivileged_bpf_disabled = 1
net.core.bpf_jit_enable = 1
net.core.bpf_jit_harden = 0
net.core.bpf_jit_kallsyms = 0
net.core.bpf_jit_limit = 264241152
```

С помощью `kernel.unprivileged_bpf_disabled` можно запретить непривилегированный доступ, выполнив любую из следующих команд:

```
# sysctl -w kernel.unprivileged_bpf_disabled=1
# echo 1 > /proc/sys/kernel/unprivileged_bpf_disabled
```

Это действие нельзя отменить: попытка записать ноль в этот параметр будет отклонена. Остальные параметры `sysctls` также можно установить с помощью аналогичных команд.

`net.core.bpf_jit_enable` включает динамический (just-in-time, JIT) компилятор BPF. Это улучшает производительность и безопасность. Для устранения уязвимости Spectre v2 в ядро был добавлен параметр `CONFIG_BPF_JIT_ALWAYS_ON`, позволяющий включить JIT-компилятор и отключить интерпретатор BPF на постоянной основе. Возможные настройки (в Linux 5.2) [127]:

- **0:** JIT-компиляция отключена (по умолчанию);
- **1:** JIT-компиляция включена;
- **2:** JIT-компиляция включена и в журнал ядра выводится отладочная информация компилятора (эту настройку следует использовать только для отладки, не в производственной среде).

¹ На конференции LSFMM 2019 в Пуэрто-Рико уже обсуждались некоторые предложения [128]. Одно из них связано с использованием устройства `/dev/bpf`, которое при открытии устанавливает флаг `task_struct` для разрешения доступа, а также флаг `close-on-exec`.

Этот параметр включается по умолчанию, например, в Netflix и Facebook. Обратите внимание, что JIT-компилятор зависит от аппаратной архитектуры. Ядро Linux включает JIT-компиляторы BPF для подавляющего большинства поддерживаемых архитектур, включая x86_64, arm64, riscv64, s390x, sparc64 и даже mips64 и riscv. Компиляторы x86_64 и arm64 полнофункциональные и протестированы в производственных условиях, однако другие могут находиться на стадии разработки.

`net.core.bpf_jit_harden` включает дополнительную защиту, в том числе против атак вида JIT Spray, за счет снижения производительности [129]. Возможные настройки (в Linux 5.2) [127]:

- **0:** защита JIT отключена (по умолчанию);
- **1:** защита JIT включена только для непривилегированных пользователей;
- **2:** защита JIT включена для всех пользователей.

`net.core.bpf_jit_kallsyms` открывает привилегированным пользователям доступ к символам в JIT-скомпилированных образах через `/proc/kallsyms` для упрощения отладки [130]. Этот параметр отключается при включении `bpf_jit_harden`.

`net.core.bpf_jit_limit` устанавливает ограничение в байтах на объем памяти, которую модуль может использовать. После достижения предела запросы непривилегированных пользователей блокируются и перенаправляются интерпретатору, если он включен.

Дополнительную информацию о защите BPF ищите в справочном руководстве по Cilium BPF, написанном одним из разработчиков BPF Дэниелом Боркманом [131].

11.1.4. Стратегия

Ниже описана предлагаемая стратегия анализа активности системы с точки зрения безопасности, еще не покрытой другими инструментами BPF:

1. Проверьте доступность точек трассировки или зондов USDT для оценки активности.
2. Проверьте возможность трассировки обработчиков LSM: их имена начинаются с «security_».
3. При необходимости используйте зонды kprobes/uprobes для инструментации исходного кода.

11.2. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа безопасности. Они показаны на рис. 11.2.

Часть этих инструментов можно найти в репозиториях BCC и bpftrace, упоминавшихся в главах 4 и 5, а часть была создана специально для этой книги. Инструменты

перечислены в табл. 11.1, где указано их происхождение (BT — это сокращение от «bpftrace»).

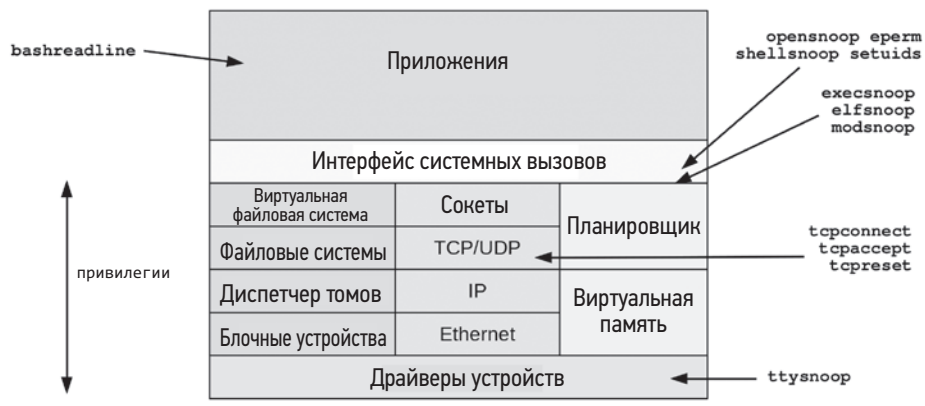


Рис. 11.2. Инструменты BPF для анализа безопасности

Таблица 11.1. Инструменты, связанные с анализом безопасности

Инструмент	Источник	Цель	Описание
execsnoop	ВСС/BT	Системные вызовы	Трассирует запуск новых процессов
elfsnoop	Книга	Ядро	Трассирует операции загрузки файлов ELF
modsnoop	Книга	Ядро	Трассирует операции загрузки модулей ядра
bashreadline	ВСС/BT	bash	Трассирует ввод команд в оболочке bash
shellsnoop	Книга	Командные оболочки	Отражает вывод командной оболочки
ttysnoop	ВСС/книга	TTY	Отражает вывод в устройства tty
opensnoop	ВСС/BT	Системные вызовы	Трассирует события открытия файлов
eperm	Книга	Системные вызовы	Подсчитывает случаи завершения системных вызовов с ошибками EPERM и EACCES
tcpconnect	ВСС/BT	TCP	Трассирует создание TCP-соединений (активных)
tcpaccept	ВСС/BT	TCP	Трассирует создание TCP-соединений (пассивных)
tcpreset	Книга	TCP	Трассирует события сброса TCP (пакеты RST): сканирование портов
capable	ВСС/BT	Безопасность	Трассировка использования привилегий безопасности
setuids	Книга	Системные вызовы	Трассировка обращений к системному вызову setuid: повышение привилегий

Актуальные списки параметров инструментов ВСС и bpftrace и описание их возможностей ищите в соответствующих репозиториях. Некоторые из инструментов были представлены в предыдущих главах и здесь описаны повторно.

Дополнительную информацию о любой из упоминаемых здесь подсистем ищите в других главах: о сетевых соединениях — в главе 10, об использовании файлов — в главе 8, о выполнении ПО — в главе 6.

11.2.1. execsnoop

execsnoop(8) был представлен в главе 6. Это инструмент для ВСС и bpftrace, трассирующий запуск новых процессов. Он может использоваться для выявления подозрительных случаев запуска процессов. Пример вывода:

```
# execsnoop
PCOMM      PID   PPID  RET  ARGS
ls          7777  21086  0    /bin/ls -F
a.out       7778  21086  0    /tmp/a.out
[...]
```

Мы видим запуск процесса из файла a.out в каталоге /tmp.

execsnoop(8) трассирует системный вызов execve(2). Это типичный шаг при создании новых процессов, который начинается с вызова fork(2) или clone(2), создающего новый процесс, за которым следует вызов execve(2) для запуска в этом процессе другой программы. Обратите внимание, что это не единственный способ запуска новых программ: атака вида «переполнение буфера» может добавлять новые инструкции в текущий процесс и выполнять вредоносное ПО без вызова execve(2).

Более подробно execsnoop(8) описывается в главе 6.

11.2.2. elfsnoop

elfsnoop(8)¹ — это инструмент bpftrace для трассировки попыток запуска двоичных файлов в формате исполняемых и компонуемых модулей (Executable and Linking Format, ELF), широко используемом в Linux. Этот инструмент трассирует функцию в глубине ядра, которая вызывается при попытке запустить любой файл в формате ELF. Например:

```
# elfsnoop.bt
Attaching 3 probes...
Tracing ELF loads. Ctrl-C to end
TIME      PID   INTERPRETER      FILE              MOUNT      INODE      RET
11:18:43  9022  /bin/ls          /bin/ls           /           29098068   0
11:18:45  9023  /tmp/ls          /tmp/ls           /           23462045   0
11:18:49  9029  /usr/bin/python  ./opensnoop.py    /           20190728   0
[...]
```

¹ Немного истории: я создал его для этой книги 25 февраля 2019 года.

Этот вывод показывает некоторые детали, связанные с запуском файла. Значения столбцов:

- **TIME:** отметка времени (в формате ЧЧ:ММ:СС);
- **PID:** идентификатор процесса;
- **INTERPRETER:** для сценариев, интерпретатор которых был выполнен;
- **FILE:** исполняемый файл;
- **MOUNT:** точка монтирования файловой системы, где находится исполняемый файл;
- **INODE:** номер индексного узла исполняемого файла, в паре с точкой монтирования образует уникальный идентификатор;
- **RET:** результат попытки запустить файл, ноль означает успех.

Точка монтирования и номер индексного узла (inode) выводятся, чтобы позволить дополнительно проверить запускавшийся двоичный файл. Злоумышленник может создать свою версию системного двоичного файла с тем же именем (и даже использовать управляющие символы в имени, чтобы при выводе оно выглядело так, будто имеет тот же путь), но не сможет подменить комбинацию из точки монтирования и номера индексного узла.

Этот инструмент трассирует функцию ядра `load_elf_binary()`, которая отвечает за загрузку программ ELF для выполнения. Оверхед на работу этого инструмента должен быть незначительным, так как эта функция вызывается не очень часто.

Исходный код `elfsnoop(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/binfmts.h>
#include <linux/fs.h>
#include <linux/mount.h>

BEGIN
{
    printf("Tracing ELF loads. Ctrl-C to end\n");
    printf("%-8s %-6s %-18s %-18s %-10s %-10s RET\n",
           "TIME", "PID", "INTERPRETER", "FILE", "MOUNT", "INODE");
}

kprobe:load_elf_binary
{
    @arg0[tid] = arg0;
}

kretprobe:load_elf_binary
/@arg0[tid]/
{
    $bin = (struct linux_binprm *)@arg0[tid];
    time("%H:%M:%S ");
    printf("%-6d %-18s %-18s %-10s %-10d %3d\n", pid,
           str($bin->interp), str($bin->filename),
```

```

        str($bin->file->f_path.mnt->mnt_root->d_name.name),
        $bin->file->f_inode->i_ino, retval);
    delete(@arg0[tid]);
}

```

В инструмент можно добавить вывод дополнительной информации об исполняемом файле, включая полный путь. Обратите внимание: сейчас функция `printf()` в `bpfttrace` может принимать не более семи аргументов, поэтому для вывода дополнительной информации используйте несколько инструкций `printf()`.

11.2.3. modsnoop

`modsnoop(8)`¹ — это инструмент `bpfttrace` для трассировки попыток загрузить модуль ядра. Например:

```

# modsnoop.bt
Attaching 2 probes...
Tracing kernel module loads. Hit Ctrl-C to end.
12:51:38 module init: msr, by modprobe (PID 32574, user root, UID 0)
[...]

```

Как показывает этот вывод, в 10:50:26 произошла попытка загрузить модуль «msr» с помощью инструмента `modprobe(8)` и с UID 0. Загрузка модулей — это другой способ выполнить код, часто используемый различными руткитами (rootkit), что делает его еще одной целью для трассировки безопасности.

Исходный код `modsnoop(8)`:

```

#!/usr/local/bin/bpfttrace

#include <linux/module.h>

BEGIN
{
    printf("Tracing kernel module loads. Hit Ctrl-C to end.\n");
}

kprobe:do_init_module
{
    $mod = (struct module *)arg0;
    time("%H:%M:%S ");
    printf("module init: %s, by %s (PID %d, user %s, UID %d)\n",
        $mod->name, comm, pid, username, uid);
}

```

Этот инструмент трассирует функцию ядра `do_init_module()`, которая имеет доступ к структуре `module`.

Есть и точка трассировки `module:module_load`, которую мы используем в однострочных сценариях, см. далее в этой главе.

¹ Немного истории: я создал его для этой книги 14 марта 2019 года.

11.2.4. **bashreadline**

bashreadline(8)¹ — это инструмент BCC и bpftrace для трассировки интерактивного выполнения команд в оболочке **bash** во всей системе. Вот пример трассировки с помощью версии для BCC:

```
# bashreadline
bashreadline
TIME      PID      COMMAND
11:43:51  21086    ls
11:44:07  21086    echo hello book readers
11:44:22  21086    eccho hi
11:44:33  21086    /tmp/ls
[...]
```

Этот вывод показывает, какие команды выполнялись в период трассировки, включая встроенные команды оболочки (**echo**) и команды, попытки выполнить которые завершились ошибками (**eccho**). Этот инструмент трассирует функцию **readline()** в командной оболочке **bash**, поэтому будет отображать любые введенные команды. Обратите внимание, что **bashreadline** не может трассировать команды других оболочек, и злоумышленник может установить свою оболочку (например, **nanoshell**), которая не трассируется.

bpftrace

Ниже приведен исходный код версии для **bpftrace**.

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing bash commands... Hit Ctrl-C to end.\n");
    printf("%-9s %-6s %s\n", "TIME", "PID", "COMMAND");
}

uretprobe:/bin/bash:readline
{
    time("%H:%M:%S ");
    printf("%-6d %s\n", pid, str(retval));
}
```

Этот код трассирует функцию **readline()** в **/bin/bash** с помощью зонда **uretprobe**. В некоторых дистрибутивах Linux командная оболочка **bash** собирается иначе,

¹ Немного истории: 28 января 2016 года я написал версию для BCC, а 6 сентября 2018 года — версию для bpftrace. Они создавались как простые примеры программ, использующие зонды **uprobe** в BPF. Но позже этот инструмент привлек внимание специалистов по безопасности, в частности, для логирования активности в заблокированных средах, где можно использовать только одну оболочку (**bash**).

и в них следует трассировать функцию `readline()` из библиотеки `libreadline`. Чтобы больше узнать о трассировке `readline()`, см. раздел 12.2.3 в главе 12.

11.2.5. shellsnoop

`shellsnoop(8)`¹ — это инструмент BCC и `bpfttrace`, отражающий вывод командной оболочки в другом сеансе. Например:

```
# shellsnoop 7866
bgregg:~/Build/bpfttrace/tools> date
Fri May 31 18:11:02 PDT 2019
bgregg:~/Build/bpfttrace/tools> echo Hello BPF
Hello BPF
bgregg:~/Build/bpfttrace/tools> typo
```

Command 'typo' not found, did you mean:

command 'typop' from deb terminology

Try: apt install <deb name>

В этом примере показано, как `shellsnoop(8)` вывел команды и их результаты, выполнявшиеся в сеансе оболочки с PID 7866. Этот инструмент захватывает вывод процесса в `STDOUT` и `STDERR`, включая его дочерние процессы. Трассировка дочерних процессов нужна для перехвата вывода команды `date(1)`.

`shellsnoop(8)` также может сгенерировать сценарий оболочки для воспроизведения вывода. Например:

```
# shellsnoop -r 7866
echo -e 'd\c'
sleep 0.10
echo -e 'a\c'
sleep 0.06
echo -e 't\c'
sleep 0.07
echo -e 'e\c'
sleep 0.25
echo -e '
\c'
sleep 0.00
echo -e 'Fri May 31 18:50:35 PDT 2019
\c'
```

Его можно сохранить в файл и выполнить с помощью оболочки `bash(1)`: он воспроизведет вывод сеанса оболочки с сохранением скорости и задержек оригинала. Это выглядит необычно.

¹ Немного истории: версию для BCC я написал 15 октября 2016 года, а версию для `bpfttrace` — 31 мая 2019 года. Они были основаны на моем инструменте `shellsnoop`, написанном 24 марта 2004 года по образу и подобию `ttywatcher`. В 2005 году Борис Лоза (Boris Loza) упомянул мою раннюю версию `shellsnoop` в журнале *Phrack* как инструмент безопасности [132].

BCC

Порядок использования:

```
shellsnoop [options] PID
```

Параметры options:

- **-s**: захватывать вывод только самой оболочки (не захватывать вывод подоболочек);
- **-r**: создать сценарий для воспроизведения вывода.

bpfttrace

Вот версия для bpfttrace, обладающая основными возможностями версии для BCC¹:

```
#!/usr/local/bin/bpfttrace
```

```
BEGIN
/$1 == 0/
{
    printf("USAGE: shellsnoop.bt PID\n");
    exit();
}

tracepoint:sched:sched_process_fork
/args->parent_pid == $1 || @descendent[args->parent_pid]/
{
    @descendent[args->child_pid] = 1;
}

tracepoint:syscalls:sys_enter_write
/(pid == $1 || @descendent[pid]) && (args->fd == 1 || args->fd == 2)/
{
    printf("%s", str(args->buf, args->count));
}
```

11.2.6. ttysnoop

ttysnoop(8)² — это инструмент BCC и bpfttrace, отражающий вывод в устройства tty или pts. Его можно использовать для трассировки подозрительных сеансов входа в систему в реальном времени.

¹ Сейчас она усекает вывод до BPFTRACE_STRLEN (64) байт. В будущем мы значительно увеличим это ограничение, используя для хранения строк карты BPF вместо стека.

² Немного истории: версию для BCC я написал 15 октября 2016 года, взяв за основу более старый инструмент Unix ttywatcher и мой более ранний инструмент cuckoo.d, написанный в 2011 году. Я был системным администратором и использовал ttywatcher для наблюдения за действиями реального злоумышленника, не обладающего привилегиями root, в производственной системе. Он загружал различные эксплойты для повышения привилегий,

Вот пример наблюдения за `/dev/pts/16`:

```
# ttysnoop 16
$ uname -a
Linux lgud-bgregg 4.15.0-43-generic #46-Ubuntu SMP Thu Dec 6 14:45:28 UTC 2018
x86_64
x86_64 x86_64 GNU/Linux
$ gcc -o a.out crack.c
$ ./a.out
Segmentation fault
[...]
```

Здесь инструмент выводит все, что видит пользователь `/dev/pts/16`. Он трассирует функцию ядра `tty_write()` и выводит все, что передается ей для записи.

БСС

Порядок использования:

```
ttysnoop [options] device
```

Параметры `options`:

- **-C**: не очищать экран.

Для идентификации устройства псевдотерминала можно указать полный путь к нему, например `/dev/pts/2`, или просто номер 2. Для идентификации устройств ТТУ требуется указывать полный путь, например `/dev/tty0`. Если команде `ttysnoop(8)` передать `/dev/console`, она будет выводить все, что печатается в системной консоли.

bpfftrace

Вот исходный код версии для `bpfftrace`:

```
#!/usr/local/bin/bpfftrace

#include <linux/fs.h>

BEGIN
{
    if ($1 == 0) {
        printf("USAGE: ttysnoop.bt pts_device # eg, pts14\n");
        exit();
    }
}
```

компилировал и безуспешно пытался запустить их. Больше всего в его действиях меня раздражало то, что он использовал текстовый редактор `risco` вместо моего любимого `vi`. Более увлекательную историю наблюдения за ТТУ, которая подтолкнула меня к созданию `cusko0.d`, ищите в [Stoll 89]. Версию для `bpfftrace` я написал специально для этой книги 26 февраля 2019 года.

```

        printf("Tracing tty writes. Ctrl-C to end.\n");
    }

kprobe:tty_write
{
    $file = (struct file *)arg0;
    // +3 -- чтобы перешагнуть через символы "pts":
    if (str($file->f_path.dentry->d_name.name) == str($1 + 3)) {
        printf("%s", str(arg1, arg2));
    }
}

```

Это пример программы для bpftrace, которая принимает обязательный аргумент. Если имя устройства не указано, она выводит сообщение с инструкцией о порядке использования и завершает работу. Выходить нужно, так как при трассировке всех устройств вывод смешивается и образуется цикл обратной связи с самим инструментом.

11.2.7. opensnoop

opensnoop(8) рассматривался в главе 8 и не раз упоминался в более ранних главах. Это инструмент ВСС и bpftrace для трассировки событий открытия файлов, который можно использовать для решения задач, связанных с безопасностью, например для исследования поведения вредоносных программ и мониторинга использования файлов. Пример вывода версии для ВСС:

```

# opensnoop
PID    COMM          FD ERR PATH
12748  opensnoop     -1  2 /usr/lib/python2.7/encodings/ascii.x86_64-linux-gnu.so
12748  opensnoop     -1  2 /usr/lib/python2.7/encodings/ascii.so
12748  opensnoop     -1  2 /usr/lib/python2.7/encodings/asciimodule.so
12748  opensnoop     18  0 /usr/lib/python2.7/encodings/ascii.py
12748  opensnoop     19  0 /usr/lib/python2.7/encodings/ascii.pyc
1222   polkitd       11  0 /etc/passwd
1222   polkitd       11  0 /proc/11881/status
1222   polkitd       11  0 /proc/11881/stat
1222   polkitd       11  0 /etc/passwd
1222   polkitd       11  0 /proc/11881/status
1222   polkitd       11  0 /proc/11881/stat
1222   polkitd       11  0 /proc/11881/cgroup
1222   polkitd       11  0 /proc/1/cgroup
1222   polkitd       11  0 /run/systemd/sessions/2
[...]
```

Как можно судить по этому выводу, opensnoop(8) выполняет поиск и загрузку модуля `ascii` для Python: первые три попытки открыть файл были неудачными. Далее был пойман `polkitd`(8) — демон PolicyKit — на открытии файла `passwd` и проверке статуса процессов. Принцип действия opensnoop(8) основан на трассировке разных вариантов системного вызова `open`(2).

Более подробное описание opensnoop(8) ищите в главе 8.

11.2.8. eperm

`eperm(8)`¹ — это инструмент `bpfttrace` для подсчета обращений к системным вызовам, завершившихся ошибкой `EPERM` «Операция запрещена» или `EACCESS` «Доступ запрещен», что может быть интересно для анализа безопасности. Например:

```
# eperm.bt
Attaching 3 probes...
Tracing EACCESS and EPERM syscall errors. Ctrl-C to end.
^C
```

```
@EACCESS[systemd-logind, sys_setsockopt]: 1
```

```
@EPERM[cat, sys_openat]: 1
@EPERM[gmain, sys_inotify_add_watch]: 6
```

Как показано в этом выводе, инструмент отображает имена процессов и системные вызовы, завершившиеся ошибкой, группируя их по видам ошибок. Так, пример выше показывает, что во время трассировки в `cat(1)` произошла одна ошибка `EPERM` при попытке вызвать `openat(2)`. Эти ошибки можно дополнительно исследовать с помощью других инструментов, таких как `opensnoop(8)`.

В своей работе инструмент использует точку трассировки `raw_syscalls:sys_exit`, которая срабатывает при обращении к любому системному вызову. Оверхед может стать заметным в системах с высокой частотой операций ввода/вывода, поэтому сначала следует поэкспериментировать в тестовой среде.

Исходный код `eperm(8)`:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing EACCESS and EPERM syscall errors. Ctrl-C to end.\n");
}

tracepoint:raw_syscalls:sys_exit
/args->ret == -1/
{
    @EACCESS[comm, ksym(*(kaddr("sys_call_table") + args->id * 8))] =
        count();
}

tracepoint:raw_syscalls:sys_exit
/args->ret == -13/
{
    @EPERM[comm, ksym(*(kaddr("sys_call_table") + args->id * 8))] =
        count();
}
```

¹ Немного истории: я создал его для этой книги 25 февраля 2019 года.

Точка трассировки `raw_syscalls:sys_exit` предоставляет только идентификационный номер системного вызова. Чтобы преобразовать его в имя, используйте таблицу из ВСС-инструмента `syscount(8)`. Но в `erpm(8)` используется другой прием: он читает таблицу системных вызовов ядра (`sys_call_table`), отыскивает функцию, которая обрабатывает системный вызов, и преобразует адрес этой функции в имя символа ядра.

11.2.9. `tcpconnect` и `tcpraccept`

`tcpconnect(8)` и `tcpraccept(8)` были представлены в главе 10 — это инструменты ВСС и `bpfttrace` для трассировки создания ТСП-соединений и могут использоваться для выявления подозрительной сетевой активности. Многие типы атак предполагают создание хотя бы одного соединения с системой. Вот пример вывода ВСС-версии `tcpconnect(8)`:

```
# tcpconnect
PID      COMM      IP  SADDR      DADDR      DPORT
22411    a.out      4   10.43.1.178 10.0.0.1    8080
[...]
```

В этом примере вывода `tcpconnect(8)` видно, что процесс `a.out` устанавливает соединение с адресом `10.0.0.1` и портом `8080`, что выглядит подозрительно. (`a.out` — это имя файла по умолчанию для некоторых компиляторов и обычно не используется никаким устанавливаемым ПО.)

Вот пример вывода ВСС-версии `tcpraccept(8)` с параметром `-t` для отображения отметок времени:

```
# tcpraccept -t
TIME(s)  PID      COMM      IP  RADDR      LADDR      LPORT
0.000    1440    sshd      4   10.10.1.201 10.43.1.178 22
0.201    1440    sshd      4   10.10.1.201 10.43.1.178 22
0.408    1440    sshd      4   10.10.1.201 10.43.1.178 22
0.612    1440    sshd      4   10.10.1.201 10.43.1.178 22
[...]
```

Мы видим несколько соединений, установленных с адреса `10.10.1.201` с локальным портом `22`, который обслуживается демоном `sshd(8)`. Соединения устанавливаются с частотой раз в 200 миллисекунд (судя по столбцу «TIME(s)»), что может быть попыткой атаки методом простого перебора.

Ключевая особенность этих инструментов в том, что для большей эффективности они инструментируют только события в сеансах ТСП. Другие инструменты трассируют каждый сетевой пакет, что может повлечь большой оверхед в нагруженных системах.

Более подробное описание `tcpconnect(8)` и `tcpraccept(8)` ищите в главе 10.

11.2.10. tcpreset

`tcpreset(8)`¹ — это инструмент `bpftool` для трассировки TCP-пакетов RST (сброса). Его можно использовать для обнаружения попыток сканирования TCP-портов, когда ряду портов, включая закрытые, отправляются пакеты, что вызывает отправку в ответ пакетов RST. Например:

```
# tcpreset.bt
Attaching 2 probes...
Tracing TCP resets. Hit Ctrl-C to end.
TIME      LADDR          LPORT  RADDR          RPORT
20:50:24  100.66.115.238  80     100.65.2.196   45195
20:50:24  100.66.115.238  443    100.65.2.196   45195
20:50:24  100.66.115.238  995    100.65.2.196   45451
20:50:24  100.66.115.238  5900   100.65.2.196   45451
20:50:24  100.66.115.238  443    100.65.2.196   45451
20:50:24  100.66.115.238  110    100.65.2.196   45451
20:50:24  100.66.115.238  135    100.65.2.196   45451
20:50:24  100.66.115.238  256    100.65.2.196   45451
20:50:24  100.66.115.238  21     100.65.2.196   45451
20:50:24  100.66.115.238  993    100.65.2.196   45451
20:50:24  100.66.115.238  3306   100.65.2.196   45451
20:50:24  100.66.115.238  25     100.65.2.196   45451
20:50:24  100.66.115.238  113    100.65.2.196   45451
20:50:24  100.66.115.238  1025   100.65.2.196   45451
20:50:24  100.66.115.238  18581  100.65.2.196   45451
20:50:24  100.66.115.238  199    100.65.2.196   45451
20:50:24  100.66.115.238  56666  100.65.2.196   45451
20:50:24  100.66.115.238  8080   100.65.2.196   45451
20:50:24  100.66.115.238  53     100.65.2.196   45451
20:50:24  100.66.115.238  587    100.65.2.196   45451
[...]
```

Как показывает этот вывод, в течение 1 секунды было отправлено множество пакетов TCP RST с разных локальных портов: это похоже на сканирование портов. Инструмент трассирует функцию ядра, которая отправляет пакеты RST, поэтому оверхед должен быть незначительным, так как при нормальной работе это происходит нечасто.

Обратите внимание, что есть разные типы сканирования портов TCP, и разные реализации стека TCP/IP могут по-разному реагировать на них. Я тестировал ядро Linux 4.15 с помощью сканера портов `npmp(1)`, и это ядро отправляло пакеты RST в ответ на сканирование SYN, FIN, NULL и Xmas. Все эти попытки были замечены инструментом `tcpreset(8)`.

Значения столбцов:

- **TIME:** время в формате ЧЧ:ММ:СС;
- **LADDR:** локальный адрес;

¹ Немного истории: я написал его для этой книги 26 февраля 2019 года.

- **LPORT:** локальный порт TCP;
- **RADDR:** удаленный IP-адрес;
- **RPORT:** удаленный порт TCP.

Исходный код `tcpreset(8)`:

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>
#include <uapi/linux/tcp.h>
#include <uapi/linux/ip.h>

BEGIN
{
    printf("Tracing TCP resets. Hit Ctrl-C to end.\n");
    printf("%-8s %-14s %-6s %-14s %-6s\n", "TIME",
        "LADDR", "LPORT", "RADDR", "RPORT");
}

kprobe:tcp_v4_send_reset
{
    $skb = (struct sk_buff *)arg1;
    $tcp = (struct tcphdr *)($skb->head + $skb->transport_header);
    $ip = (struct iphdr *)($skb->head + $skb->network_header);
    $dport = ($tcp->dest >> 8) | (($tcp->dest << 8) & 0xff00);
    $sport = ($tcp->source >> 8) | (($tcp->source << 8) & 0xff00);

    time("%H:%M:%S ");
    printf("%-14s %-6d %-14s %-6d\n", ntop(AF_INET, $ip->daddr), $dport,
        ntop(AF_INET, $ip->saddr), $sport);
}
```

Этот код трассирует функцию ядра `tcp_v4_send_reset()`, которая поддерживает только IPv4. При желании в инструмент можно добавить поддержку IPv6.

Этот инструмент может служить примером чтения заголовков IP и TCP из буфера сокета: строки, где присваиваются значения `$tcp` и `$ip`. Такая логика основана на функциях ядра `ip_hdr()` и `tcp_hdr()` и ее придется изменить в случае изменения логики в ядре.

11.2.11. `capable`

`capable(8)`¹ — это инструмент BCC и `bpftrace` для трассировки использования привилегий безопасности. Он пригодится для создания белых списков привилегий, необходимых приложениям, чтобы блокировать другие приложения для большей безопасности.

¹ Немного истории: первую версию для BCC я написал 13 сентября 2016 года и 8 сентября 2018 года перенес ее на `bpftrace`. Я создал ее после обсуждения с Майклом Уордропом из команды безопасности платформы Netflix, который хотел получить подобную видимость.

```
# capable
TIME      UID      PID      COMM      CAP NAME      AUDIT
22:52:11  0         20007    capable    21 CAP_SYS_ADMIN  1
22:52:11  0         20007    capable    21 CAP_SYS_ADMIN  1
22:52:11  0         20007    capable    21 CAP_SYS_ADMIN  1
22:52:11  0         20007    capable    21 CAP_SYS_ADMIN  1
22:52:11  0         20007    capable    21 CAP_SYS_ADMIN  1
22:52:12  1000      20108    ssh        7 CAP_SETUID     1
22:52:12  0         20109    sshd       6 CAP_SETGID     1
22:52:12  0         20109    sshd       6 CAP_SETGID     1
22:52:12  0         20110    sshd       18 CAP_SYS_CHROOT  1
22:52:12  0         20110    sshd       6 CAP_SETGID     1
22:52:12  0         20110    sshd       6 CAP_SETGID     1
22:52:12  0         20110    sshd       7 CAP_SETUID     1
22:52:12  122      20110    sshd       6 CAP_SETGID     1
22:52:12  122      20110    sshd       6 CAP_SETGID     1
22:52:12  122      20110    sshd       7 CAP_SETUID     1
[...]
```

Как показывает этот вывод, инструмент `capable(8)` проверяет наличие привилегии `CAP_SYS_ADMIN` (суперпользователь), `ssh(1)` — привилегии `CAP_SETUID`, а `sshd(8)` — нескольких разных привилегий. Документацию с описанием этих привилегий ищите на странице справочного руководства `capabilities(7)`.

Значения столбцов:

- **CAP:** номер привилегии;
- **NAME:** кодовое имя привилегии (см. `capabilities(7)`);
- **AUDIT:** записывается ли попытка проверить наличие этой привилегии в журнал аудита.

Инструмент трассирует функцию ядра `cap_capable()`, которая определяет, обладает ли текущая задача заданной привилегией. Обычно эта функция вызывается нечасто, поэтому оверхед должен быть незначительным.

Поддерживаются параметры для отображения трассировки стека в пространствах пользователя и ядра. Вот пример использования обоих:

```
# capable -KU
[...]
TIME      UID      PID      COMM      CAP NAME      AUDIT
12:00:37  0         26069    bash       2  CAP_DAC_READ_SEARCH  1
    cap_capable+0x1 [kernel]
    ns_capable_common+0x68 [kernel]
    capable_wrt_inode_uidgid+0x33 [kernel]
    generic_permission+0xfe [kernel]
    __inode_permission+0x36 [kernel]
    inode_permission+0x14 [kernel]
    may_open+0x5a [kernel]
    path_openat+0x4b5 [kernel]
    do_filp_open+0x9b [kernel]
    do_sys_open+0x1bb [kernel]
    sys_openat+0x14 [kernel]
```

```

do_syscall_64+0x73 [kernel]
entry_SYSCALL_64_after_hwframe+0x3d [kernel]
open+0x4e [libc-2.27.so]
read_history+0x22 [bash]
load_history+0x8c [bash]
main+0x955 [bash]
__libc_start_main+0xe7 [libc-2.27.so]
[unknown]
[...]
```

Здесь показан стек ядра, ведущий к системному вызову `openat(2)`, и стек процесса `bash` в пространстве пользователя, ведущий к функции `read_history()`.

BCC

Порядок использования:

```
capable [options]
```

Параметры `options`:

- **-v**: включать все проверки (`verbose` — подробный режим);
- **-p PID**: трассировать только этот процесс;
- **-K**: добавить в вывод трассировки стека ядра;
- **-U**: добавить в вывод трассировки стека в пространстве пользователя.

Некоторые проверки считаются «не относящимися к аудиту» и не фиксируются в журнале аудита. По умолчанию они исключены, если не используется параметр `-v`.

bpfttrace

Ниже приведен исходный код версии для `bpfttrace`, где обобщены основные функциональные возможности инструмента. Эта версия не поддерживает параметры командной строки и трассирует проверки всех привилегий без исключения.

```
#!/usr/local/bin/bpfttrace
```

```

BEGIN
{
    printf("Tracing cap_capable syscalls... Hit Ctrl-C to end.\n");
    printf("%-9s %-6s %-6s %-16s %-4s %-20s AUDIT\n", "TIME", "UID", "PID",
        "COMM", "CAP", "NAME");
    @cap[0] = "CAP_CHOWN";
    @cap[1] = "CAP_DAC_OVERRIDE";
    @cap[2] = "CAP_DAC_READ_SEARCH";
    @cap[3] = "CAP_FOWNER";
    @cap[4] = "CAP_FSETID";
    @cap[5] = "CAP_KILL";
    @cap[6] = "CAP_SETGID";
    @cap[7] = "CAP_SETUID";
    @cap[8] = "CAP_SETPCAP";
    @cap[9] = "CAP_LINUX_IMMUTABLE";
    @cap[10] = "CAP_NET_BIND_SERVICE";
```

```

@cap[11] = "CAP_NET_BROADCAST";
@cap[12] = "CAP_NET_ADMIN";
@cap[13] = "CAP_NET_RAW";
@cap[14] = "CAP_IPC_LOCK";
@cap[15] = "CAP_IPC_OWNER";
@cap[16] = "CAP_SYS_MODULE";
@cap[17] = "CAP_SYS_RAWIO";
@cap[18] = "CAP_SYS_CHROOT";
@cap[19] = "CAP_SYS_PTRACE";
@cap[20] = "CAP_SYS_PACCT";
@cap[21] = "CAP_SYS_ADMIN";
@cap[22] = "CAP_SYS_BOOT";
@cap[23] = "CAP_SYS_NICE";
@cap[24] = "CAP_SYS_RESOURCE";
@cap[25] = "CAP_SYS_TIME";
@cap[26] = "CAP_SYS_TTY_CONFIG";
@cap[27] = "CAP_MKNOD";
@cap[28] = "CAP_LEASE";
@cap[29] = "CAP_AUDIT_WRITE";
@cap[30] = "CAP_AUDIT_CONTROL";
@cap[31] = "CAP_SETFCAP";
@cap[32] = "CAP_MAC_OVERRIDE";
@cap[33] = "CAP_MAC_ADMIN";
@cap[34] = "CAP_SYSLOG";
@cap[35] = "CAP_WAKE_ALARM";
@cap[36] = "CAP_BLOCK_SUSPEND";
@cap[37] = "CAP_AUDIT_READ";
}

kprobe:cap_capable
{
    $cap = arg2;
    $audit = arg3;
    time("%N:%M:%S ");
    printf("%-6d %-6d %-16s %-4d %-20s %d\n", uid, pid, comm, $cap,
        @cap[$cap], $audit);
}

END
{
    clear(@cap);
}

```

Программа объявляет хеш для преобразования номеров привилегий в имена. Его нужно будет обновлять в соответствии с дополнениями к ядру.

11.2.12. setuids

setuids(8)¹ — это инструмент bpftrace для трассировки системных вызовов повышения привилегий: `setuid(2)`, `setresuid(2)` и `setfsuid(2)`. Например:

¹ Немного истории: я создал первую версию под названием `setuids.d` 9 мая 2004 года, найдя ее полезной для отслеживания входов в систему, поскольку она перехватывала их установку `uid`: `login`, `su` и `sshd`. Версию `bpftrace` я написал 26 февраля 2019 года специально для этой книги.

```
# setuids.bt
Attaching 7 probes...
Tracing setuid(2) family syscalls. Hit Ctrl-C to end.
TIME      PID      COMM      UID      SYSCALL  ARGS (RET)
23:39:18  23436  sudo      1000      setresuid ruid=-1 euid=1000 suid=-1 (0)
23:39:18  23436  sudo      1000      setresuid ruid=-1 euid=0 suid=-1 (0)
23:39:18  23436  sudo      1000      setresuid ruid=-1 euid=0 suid=-1 (0)
23:39:18  23436  sudo      1000      setresuid ruid=0 euid=-1 suid=-1 (0)
23:39:18  23436  sudo      0         setresuid ruid=1000 euid=-1 suid=-1 (0)
23:39:18  23436  sudo      1000      setresuid ruid=-1 euid=-1 suid=-1 (0)
23:39:18  23436  sudo      1000      setuid    uid=0 (0)
23:39:18  23437  sudo      0         setresuid ruid=0 euid=0 suid=0 (0)
[...]
```

Здесь показаны результаты применения команды `sudo(8)` для смены UID с 1000 на 0, а также то, какие системные вызовы она использовала. `setuids(8)` также видит попытки входа через `sshd(8)`, потому что они тоже изменяют UID.

Значения столбцов:

- **UID:** идентификатор пользователя до вызова `setuid`;
- **SYSCALL:** имя системного вызова;
- **ARGS:** аргументы системного вызова;
- **(RET):** возвращаемое значение. Для `setuid(2)` и `setresuid(2)` показывает успех вызова, для `setfsuid(2)` содержит предыдущий UID.

Инструмент использует точки трассировки системных вызовов. Так как частота обращений к этим системным вызовам невысока, оверхед должен быть незначительным.

Исходный код `setuids(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing setuid(2) family syscalls. Hit Ctrl-C to end.\n");
    printf("%-8s %-6s %-16s %-6s %-9s %s\n", "TIME",
        "PID", "COMM", "UID", "SYSCALL", "ARGS (RET)");
}

tracepoint:syscalls:sys_enter_setuid,
tracepoint:syscalls:sys_enter_setfsuid
{
    @uid[tid] = uid;
    @setuid[tid] = args->uid;
    @seen[tid] = 1;
}

tracepoint:syscalls:sys_enter_setresuid
{
    @uid[tid] = uid;
    @ruid[tid] = args->ruid;
    @euid[tid] = args->euid;
```



```

@suid[tid] = args->suid;
@seen[tid] = 1;
}

tracepoint:syscalls:sys_exit_setuid
/@seen[tid]/
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %-6d setuid uid=%d (%d)\n", pid, comm,
        @uid[tid], @setuid[tid], args->ret);
    delete(@seen[tid]); delete(@uid[tid]); delete(@setuid[tid]);
}

tracepoint:syscalls:sys_exit_setfsuid
/@seen[tid]/
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %-6d setfsuid uid=%d (prevuid=%d)\n", pid, comm,
        @uid[tid], @setuid[tid], args->ret);
    delete(@seen[tid]); delete(@uid[tid]); delete(@setuid[tid]);
}

tracepoint:syscalls:sys_exit_setresuid
/@seen[tid]/
{
    time("%H:%M:%S ");
    printf("%-6d %-16s %-6d setresuid ", pid, comm, @uid[tid]);
    printf("ruid=%d euid=%d suid=%d (%d)\n", @ruid[tid], @euid[tid],
        @suid[tid], args->ret);
    delete(@seen[tid]); delete(@uid[tid]); delete(@ruid[tid]);
    delete(@euid[tid]); delete(@suid[tid]);
}

```

Этот код использует три точки трассировки на входе и на выходе системных вызовов, сохраняя информацию в картах на входе и извлекая ее на выходе.

11.3. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечислены однострочные сценарии для BCC и bpftrace. Там, где это возможно, один и тот же сценарий реализуется с помощью BCC и bpftrace.

11.3.1. BCC

Подсчитывает события аудита безопасности для PID 1234:

```
funccount -p 1234 'security_*
```

Трассирует запуск сеанса аутентификации с использованием механизма подключаемых модулей аутентификации (Pluggable Authentication Module, PAM):

```
trace 'pam:pam_start "%s: %s", arg1, arg2'
```

Трассирует загрузку модулей ядра:

```
trace 't:module:module_load "load: %s", args->name'
```

11.3.2. bpftrace

Подсчитывает события аудита безопасности для PID 1234:

```
bpftrace -e 'k:security_* /pid == 1234 { @[func] = count(); }'
```

Трассирует запуск сеанса аутентификации с использованием механизма подключаемых модулей аутентификации (PAM):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpam.so.0:pam_start { printf("%s: %s\n", str(arg0), str(arg1)); }'
```

Трассирует загрузку модулей ядра:

```
bpftrace -e 't:module:module_load { printf("load: %s\n", str(args->name)); }'
```

11.3.3. Примеры использования однострочных сценариев BPF

По аналогии с инструментами, описанными в этой главе, для иллюстрации работы однострочных сценариев я привел примеры их вывода. Ниже показаны некоторые однострочные сценарии с примерами вывода.

Подсчет событий аудита безопасности

```
# funccount -p 21086 'security_*'
Tracing 263 functions for "security_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
security_task_setpgid               1
security_task_alloc                  1
security_inode_alloc                 1
security_d_instantiate                1
security_prepare_creds                1
security_file_alloc                   2
security_file_permission              13
security_vm_enough_memory_mm         27
security_file_ioctl                   34
Detaching...
```

Этот сценарий подсчитывает количество вхождений в обработчики Linux Security Module (LSM) для обработки и проверки событий безопасности. Чтобы получить дополнительную информацию, можно организовать трассировку каждой отдельной функции-обработчика.

Трассировка запуска сеансов аутентификации с использованием механизма PAM

```
# trace 'pam:pam_start "%s: %s", arg1, arg2'
PID      TID      COMM      FUNC      -
25568    25568    sshd      pam_start  sshd: bgregg
25641    25641    sudo      pam_start  sudo: bgregg
25646    25646    sudo      pam_start  sudo: bgregg
[...]
```

Этот вывод показывает, что `sshd(8)` и `sudo(8)` запускают для пользователя `bgregg` сеанс аутентификации с использованием подключаемых модулей аутентификации (PAM). Аналогично можно трассировать обращения к другим функциям PAM, чтобы увидеть окончательный результат аутентификации.

11.4. ИТОГИ

BRF можно использовать для решения самых разных задач в сфере безопасности, включая анализ данных в реальном времени, отладку привилегий, создание белых списков для исполняемых файлов и т. д. В главе приведено описание этих возможностей и показано их применение на примере реализации некоторых инструментов BRF.

Глава 12

ЯЗЫКИ

Существует множество языков программирования, компиляторов и сред выполнения для них. У каждого языка есть свои особенности, по которым можно отследить запуск программ, написанных на нем. Эта глава объясняет такие различия и помогает реализовать трассировку программ, написанных на любом языке.

Цели обучения:

- понять инструментацию компилированного языка (например, C);
- понять инструментацию JIT-компилированного языка (например, Java, Node.js);
- понять инструментацию интерпретируемого языка (например, bash shell);
- научиться трассировать вызовы функций, аргументы, возвращаемое значение и задержку, когда это возможно;
- отслеживать трассировку стека на уровне пользователя в языке.

Начнем с описания способов реализации разных языков программирования, а затем исследуем возможности трассировки, используя в качестве примеров несколько языков: язык C будет представлять компилируемые языки, Java — языки с JIT-компиляцией, а язык командной оболочки bash — интерпретируемые языки. На примере каждого из них я расскажу, как искать имена функций (символы), аргументы функций, а также как получать и исследовать трассировки стека. В конце главы я также затрону вопросы трассировки программ на других языках: JavaScript (Node.js), C++ и Golang.

Неважно, с каким языком вы работаете. Прочтя эту главу, вы научитесь его инструментировать, а также узнаете проблемы и их решения, которые встречались в других языках.

12.1. ОСНОВЫ

Чтобы понять, как инструментировать программы на каком-либо языке, важно знать, как код на этом языке преобразуется в исполняемый машинный код. Обычно это особенность не самого языка, а его реализации. Например, Java не

является JIT-компилируемым — это просто язык. Для выполнения программ на Java обычно используется среда выполнения JVM из OracleJDK или OpenJDK, которая выполняет методы Java, переходя от интерпретации к JIT-компиляции, но это особенность JVM. Сама JVM состоит из скомпилированного кода на C++, который выполняет такие функции, как загрузка классов и сборка мусора. Таким образом, в приложении на Java можно встретить скомпилированный код (функции на C++ в JVM), интерпретируемый код (методы на Java) и динамически (JIT) скомпилированный код (методы на Java), и все они должны инструментироваться для трассировки по-разному. Другие языки имеют отдельные реализации компиляторов и интерпретаторов, и для трассировки программ на этих языках важно знать, какие из них используются.

Если вам нужно трассировать программу на языке X, то прежде всего вы должны выяснить, какой механизм используется для запуска программ на X и как он работает. Что это — компилятор, JIT-компилятор, интерпретатор, животное, растение или минерал?

Этот раздел содержит общие рекомендации по трассировке программ на любом языке с помощью BPF, классифицируя реализации языка по способу генерации машинного кода: компилированный, JIT-компилированный или интерпретированный. Некоторые реализации (например, JVM) поддерживают несколько методов.

12.1.1. Компилируемые языки

Типичные примеры компилируемых языков: C, C++, Golang, Rust, Pascal, Fortran и COBOL.

Функции на таких языках компилируются в машинный код и сохраняются в исполняемом двоичном файле, обычно в формате ELF, со следующими атрибутами:

- В ПО, выполняющемся в пространстве пользователя, в двоичный файл ELF включаются таблицы символов для сопоставления адресов с именами функций и объектов. Эти адреса не изменяются во время выполнения, поэтому в любой момент можно прочитать таблицу символов и узнать местоположение любого объекта или функции. ПО, выполняющееся в пространстве ядра, отличается тем, что его таблица символов динамическая и размещается в `/proc/kallsyms`. Таблица может увеличиваться по мере загрузки дополнительных модулей.
- Аргументы функций и их возвращаемые значения хранятся в регистрах и в стеке. Их местоположение обычно определяется стандартным соглашением о вызовах для каждого типа процессора. Но в некоторых компилируемых языках (например, Golang) используются другие соглашения, а в других (например, встраиваемый V8) вообще нет никаких соглашений.

- Получить трассировку стека можно с помощью регистра указателя фрейма (RBP в архитектуре x86_64), если компилятор инициализирует его в прологах функций. Но очень часто компиляторы используют его как регистр общего назначения (для оптимизации производительности процессоров с ограниченным числом регистров). В результате этого становится невозможно организовать обход стека с использованием указателя фрейма.

ПО на компилируемых языках обычно легко поддается трассировке зондами `uiprobes` и `kprobes`. В этой книге будет множество подобных примеров.

Приступая к анализу скомпилированного ПО, проверьте наличие таблицы символов (например, с помощью `nm(1)`, `objdump(1)` или `readelf(1)`). Если такой таблицы нет, проверьте доступность пакета с отладочной информацией для этого ПО, который может описывать недостающие символы. В случае неудачи проверьте компилятор и ПО для сборки, чтобы понять, почему нет символов: они могли быть удалены с помощью `strip(1)`. Как один из вариантов исправления проблемы — повторно скомпилировать программу без вызова `strip(1)`.

Проверьте и возможность обхода стека с использованием указателя фрейма. Сейчас это стандартный способ обхода стека в пространстве пользователя, используемый в BPF. Если такой возможности нет, может потребоваться повторная компиляция ПО с флагом компилятора, включающим поддержку указателя фрейма (например, `gcc -fno-omit-frame-pointer`). Если и это невозможно, исследуйте другие методы обхода стека, например запись последней ветви (Last Branch Record, LBR)¹, DWARF, ORC в пространстве пользователя и BTF. Для этого понадобятся инструменты BPF, как рассказывалось в главе 2.

12.1.2. Языки с динамической компиляцией

Примеры языков с динамической (JIT) компиляцией: Java, JavaScript, Julia, .NET и Smalltalk.

Программы на языках с JIT-компиляцией преобразуются в байт-код, который затем компилируется в машинный код во время выполнения, часто с обратной связью со средой выполнения для прямой оптимизации компилятора. Они обладают следующими атрибутами (речь только о пространстве пользователя):

- Поскольку функции компилируются «на лету», таблица символов не создается заранее. Карты символов обычно хранятся в памяти среды выполнения и используются для вывода трассировки стека в случае исключения. Кроме того, эти карты могут меняться, потому что среда выполнения может повторно компилировать и перемещать функции.

¹ Сейчас в BPF нет поддержки LBR, но мы намерены добавить ее. В `perf(1)` поддержка этого метода включается параметром `--call-graph lbr`.

- Аргументы и возвращаемые значения функций могут размещаться не так, как определено стандартными соглашениями о вызове.
- Среда выполнения с поддержкой JIT-компиляции может не использовать регистр указателя фрейма, из-за чего прием обхода стека с применением указателя фрейма может не работать (в этом случае можно увидеть, что трассировка стека внезапно заканчивается фиктивным адресом). Среда выполнения обычно поддерживает возможность обхода своего стека, чтобы обработчик исключений мог вывести трассировку стека в случае ошибки.

Трассировка динамически скомпилированного кода — сложная задача. В двоичном файле нет таблицы символов, так как она создается динамически и хранится в памяти. Некоторые приложения предоставляют дополнительные файлы символов (/tmp/perf-PID.map), но их нельзя использовать для инструментации по двум причинам:

1. Компилятор может перемещать функции в памяти, не уведомляя об этом ядро. Когда инструментация не нужна, ядро возвращает машинные инструкции в исходное состояние, но если функция будет перемещена, то запись будет выполнена в неправильное место и память пользовательского пространства окажется повреждена¹.
2. Зонды `uprobes` основаны на индексных узлах, и для их работы нужно знать местоположение инструментируемого кода в файле, тогда как динамически скомпилированные функции могут храниться в анонимных приватных областях памяти².

Трассировка скомпилированных функций возможна, если среда выполнения предоставляет зонды USDT для каждой функции, хотя этот метод обычно влечет большой оверхед, независимо от того, включена трассировка или нет. Более эффективный подход — динамическая инструментация отдельных точек. (USDT и динамическая инструментация были представлены в главе 2.) Зонды USDT также позволяют исследовать аргументы и возвращаемые значения функций.

Если трассировка стека из BPF уже поддерживается, то можно использовать дополнительные файлы символов для преобразования их в имена функций. В средах выполнения, не поддерживающих USDT, это один из путей организации наблюдения за выполнением динамически скомпилированных функций: трассировки стека могут группироваться по системным вызовам, событиям ядра и использоваться для выявления выполняемых функций с помощью средств временного профилирования. Это, пожалуй, самый простой способ наблюдения

¹ Я обратился к команде разработчиков JVM с предложением реализовать возможность приостановки компилятора `c2` на время трассировки с помощью `uprobes`, чтобы предотвратить перемещение функций.

² Параллельно с другими разработчиками я искал возможность снять это ограничение с ядра.

за работой динамически скомпилированных функций, он может решить многие проблемы.

Если трассировка стека не поддерживается, то, возможно, среда выполнения поддерживает указатели фреймов через дополнительный параметр или позволяет использовать LBR. Если нет, то существуют другие способы получить трассировку стека, хотя они могут потребовать значительных инженерных усилий. Один из способов — изменить компилятор, используемый средой выполнения, чтобы он сохранял указатель фрейма. Другой — добавить точки трассировки USDT, которые используют встроенные средства языка для получения стека вызовов и предоставляют его в виде строкового аргумента. Еще один способ — послать сигнал процессу из BPF, чтобы заставить среду выполнения записать трассировку стека в память, которую BPF сможет прочитать. Примерно так была реализована виртуальная машина hhvm в Facebook [133].

Ниже мы обсудим работу перечисленных методов на примере Java.

12.1.3. Интерпретируемые языки

Примеры интерпретируемых языков: bash, Perl, Python и Ruby. Есть также языки, которые используют интерпретацию как предварительный этап перед JIT-компиляцией, например Java и JavaScript. Анализ программ на таких языках на этапе их интерпретации аналогичен анализу программ на исключительно интерпретируемых языках.

Среда выполнения интерпретируемого языка не компилирует программные функции в машинный код, а просто анализирует и выполняет программу, используя собственные встроенные процедуры. Интерпретируемые языки имеют следующие атрибуты:

- Двоичная таблица символов отражает внутреннее устройство интерпретатора, но в ней нет функций из прикладной программы. Функции почти всегда хранятся в таблице в памяти, структура которой зависит от реализации интерпретатора и отображается в объекты интерпретатора.
- Аргументы и возвращаемые значения функции обрабатываются интерпретатором. Обычно они передаются между вызовами функций интерпретатора и могут быть реализованы как объекты интерпретатора, а не как простые целые числа и строки.
- Если сам интерпретатор скомпилирован с поддержкой указателя фрейма, обход стека с его использованием будет работать, но покажет только внутреннее устройство интерпретатора, без учета имени функции из пользовательской программы. Программный стек почти наверняка поддерживается интерпретатором и выводится в случае исключений, но хранится в нестандартной структуре данных.

Среда выполнения может поддерживать зонды USDT, соответствующие началу и концу вызовов функций, предоставляя возможность получить имя функции и ее аргументы. Например, среда выполнения Ruby имеет встроенные зонды USDT в интерпретаторе. Это позволяет трассировать вызовы функций, но может повлечь значительный оверхед: инструментация такого зонда обычно означает инструментацию всех вызовов функций с фильтрацией интересующих функций по именам. Если есть динамическая библиотека USDT для среды выполнения, то ее можно использовать для вставки пользовательских зондов USDT только в интересующие функции. (Введение в динамическую трассировку ищите в главе 2.) Такую возможность, например, предлагает пакет `ruby-static-tracing` для Ruby.

Если среда выполнения не имеет встроенных зондов USDT или пакетов поддержки динамической трассировки (например, `libstapsdt/libusdt`), то можно попробовать организовать трассировку функций самого интерпретатора с помощью зондов `uprobes` и исследовать имена прикладных функций и их аргументы. Они могут храниться в виде объектов интерпретатора и требуют знания их структуры.

Трассировки стека может быть очень сложно извлечь из памяти интерпретатора. Один из подходов, хотя и имеющий значительный оверхед, заключается в трассировке вызовов и возвратов всех функций в BPF и создании в памяти BPF искусственного стека для каждого потока выполнения, который затем можно прочесть, когда потребуется. По аналогии с языками, поддерживающими динамическую компиляцию, бывают и другие способы трассировки стека, в том числе с помощью пользовательских зондов USDT и собственного метода среды выполнения, возвращающего трассировку стека (как это реализовано в Ruby), или отправкой сигнала из BPF в среду выполнения в пространстве пользователя.

12.1.4. Возможности BPF

Целевые средства трассировки языка с помощью BPF должны отвечать на следующие вопросы:

- имя вызванной функции;
- аргументы функции;
- возвращаемое значение функции или код ошибки;
- путь в коде (трассировка стека), приведший к данному событию;
- продолжительность выполнения функции, возможно, в виде гистограммы.

Что из этого можно получить с помощью BPF, зависит от реализации языка. Многие реализации включают свои средства отладки, позволяющие с легкостью ответить на первые четыре вопроса. Тогда возникает вопрос: зачем вообще использовать BPF? Основная причина в том, чтобы иметь возможность трассировать в одном инструменте сразу несколько уровней программного стека. Вместо исследования ошибок

ввода/вывода или отказов страниц только в контексте ядра можно выполнить трассировку также в пространстве пользователя, в контексте приложения, и выяснить, какие запросы пользователя привели к ошибкам ввода/вывода или отказам страниц. Во многих случаях события в ядре позволяют выявить и количественно оценить проблему, но именно пользовательский код показывает, как ее исправить.

В некоторых языках (например, Java) проще показать трассировку стека, приведшую к событию, чем проследить последовательность вызовов функций и методов. В сочетании с множеством других событий ядра, которые доступны механизму BPF для инструментации, трассировки стека могут многое. Благодаря им вы поймете, как приложение пришло к операции ввода/вывода или к использованию других ресурсов. Вы увидите, как код пришел к блокировке потока и оставлению процессора. И наконец, вы сможете использовать синхронизированную выборку для профилирования процессора и построения флейм-графиков.

12.1.5. Стратегия

Ниже описана предлагаемая стратегия анализа программного кода на разных языках:

1. Определите, как выполняется код, написанный на конкретном языке. Компилируется ли он в двоичные файлы, используется ли JI-компиляция или интерпретация либо их сочетание? Это поможет выбрать соответствующий подход, как описано в этой главе.
2. Исследуйте возможности инструментов и однострочных сценариев, оцените это для языка каждого типа.
3. Погуглите фразы «[e]BPF *название языка*», «BCC *название языка*» и «bpftrace *название языка*», чтобы узнать, есть ли готовые инструменты и методики анализа работы кода на этом языке с помощью BPF.
4. Проверьте наличие в ПО поддержки языка зондов USDT, а также то, включены ли они в распространяемые двоичные файлы (возможно, придется повторно скомпилировать ПО, чтобы включить необходимые зонды). Это стабильный интерфейс, и лучше использовать его. Если в ПО поддержки языка нет зондов USDT, подумайте об их добавлении. В большинстве своем такое ПО распространяется с открытым исходным кодом.
5. Напишите пример программы для инструментации. Вызовите известную функцию определенное количество раз и с определенной задержкой (явно приостанавливая выполнение между вызовами). Так можно проверить правильную работу инструментов анализа.
6. Для ПО, выполняющегося в пространстве пользователя, используйте `uprobes`, чтобы проверить работу кода на уровне языка. Для ПО, выполняющегося в пространстве ядра, используйте `kprobes`.

Ниже мы подробно рассмотрим три примера языков: C, как представителя компилируемых языков; Java, как представителя языков с JIT-компиляцией; и bash, как представителя интерпретируемых языков.

12.1.6. Инструменты BPF

Инструменты BPF, описанные в этой главе, представлены на рис. 12.1.



Рис. 12.1. Инструменты BPF для анализа работы кода на разных языках

Эти инструменты можно использовать для анализа работы кода на C, Java и bash.

12.2. C

C — самый простой язык с точки зрения трассировки.

Программное обеспечение на C, выполняющееся в пространстве ядра, имеет общую таблицу символов, и в большинстве дистрибутивов их ядра собираются с включенной поддержкой указателя фрейма (`CONFIG_FRAME_POINTER = y`). Это упрощает трассировку функций ядра с помощью `kprobes`: функции доступны для наблюдения, передача аргументов производится в соответствии с ABI-соглашениями аппаратной архитектуры, и можно извлекать трассировки стека. По крайней мере, для наблюдения и трассировки доступно *большинство* функций: исключение составляют встраиваемые функции и функции, которые отмечены как небезопасные для инструментации и трассировки.

ПО на C, выполняющееся в пространстве пользователя, может трассироваться непосредственно с помощью `uprobes`, если из скомпилированного двоичного файла не была удалена таблица символов и компиляция выполнялась с поддержкой указателя фрейма: функции доступны для наблюдения и трассировки, передача аргументов осуществляется в соответствии с ABI-соглашениями аппаратной архитектуры, и можно извлекать трассировки стека. К сожалению, нередко случаи, когда таблицы символов удаляются из двоичных файлов, и компиляция производится без поддержки указателя фрейма, это означает, что вам придется повторно скомпилировать такое ПО или найти другие способы чтения символов и стеков.

Для статической инструментации программ на С можно использовать зонды USDT. Некоторые библиотеки на С, включая libc, по умолчанию предоставляют такие зонды.

В этом разделе описаны символы функций на С, трассировки стека на С, трассировка функций на С, трассировка смещений в функциях на С, зонды USDT и однострочные сценарии для анализа программного кода на С. В табл. 12.1 перечислены средства инструментации пользовательского кода на С, которые уже рассматривались в других главах.

Трассировка программного кода на С++ похожа на трассировку кода на С и рассматривается в разделе 12.5.

Таблица 12.1. Инструменты для анализа работы программного кода на С

Инструмент	Источник	Цель	Описание	Глава
funccount	BCC	Функции	Подсчитывает вызовы функций	4
stackcount	BCC	Стек	Подсчитывает трассировки стека, приведшие к событию	4
trace	BCC	Функции	Выводит информацию о вызовах функций	4
argdist	BCC	Функции	Выводит информацию об аргументах или возвращаемых значениях функций	4
bpftrace	BT	Все	Гибкая инструментация функций и стеков	5

12.2.1. Символы функций на С

Символы функций могут быть прочитаны из таблиц символов ELF. Для проверки их наличия можно использовать `readelf(1)`. Например, вот символы в программе микробенчмарка:

```
$ readelf -s bench1
Symbol table '.dynsym' contains 10 entries:
  Num:      Value              Size Type      Bind    Vis      Ndx Name
   0: 0000000000000000      0 NOTYPE  LOCAL   DEFAULT UND
   1: 0000000000000000      0 NOTYPE  WEAK    DEFAULT UND _ITM_deregisterTMCloneTab
   2: 0000000000000000      0 FUNC    GLOBAL  DEFAULT UND puts@GLIBC_2.2.5 (2)
   3: 0000000000000000      0 FUNC    GLOBAL  DEFAULT UND __libc_start_main@GLIBC...
   4: 0000000000000000      0 NOTYPE  WEAK    DEFAULT UND __gmon_start__
   5: 0000000000000000      0 FUNC    GLOBAL  DEFAULT UND malloc@GLIBC_2.2.5 (2)
   6: 0000000000000000      0 FUNC    GLOBAL  DEFAULT UND atoi@GLIBC_2.2.5 (2)
   7: 0000000000000000      0 FUNC    GLOBAL  DEFAULT UND exit@GLIBC_2.2.5 (2)
   8: 0000000000000000      0 NOTYPE  WEAK    DEFAULT UND _ITM_registerTMCloneTable
   9: 0000000000000000      0 FUNC    WEAK    DEFAULT UND __cxa_finalize@GLIBC_2.2.5 (2)

Symbol table '.symtab' contains 66 entries:
  Num:      Value              Size Type      Bind    Vis      Ndx Name
   0: 0000000000000000      0 NOTYPE  LOCAL   DEFAULT UND
```

```

1: 0000000000000238 0 SECTION LOCAL DEFAULT 1
2: 0000000000000254 0 SECTION LOCAL DEFAULT 2
3: 0000000000000274 0 SECTION LOCAL DEFAULT 3
4: 0000000000000298 0 SECTION LOCAL DEFAULT 4
[...]
61: 0000000000000000 0 FUNC GLOBAL DEFAULT UND exit@@GLIBC_2.2.5
62: 0000000000201010 0 OBJECT GLOBAL HIDDEN 23 __TMC_END__
63: 0000000000000000 0 NOTYPE WEAK DEFAULT UND __ITM_registerTMCloneTable
64: 0000000000000000 0 FUNC WEAK DEFAULT UND __cxa_finalize@@GLIBC_2.
65: 0000000000000590 0 FUNC GLOBAL DEFAULT 11 _init

```

Таблица символов «.symtab» содержит десятки записей (здесь показаны не все). Для динамического связывания используется дополнительная таблица символов «.dynsym», которая включает всего шесть символов функций.

Теперь посмотрим, как выглядят эти таблицы после применения утилиты strip(1) к двоичному файлу, что часто делается на практике для уменьшения размеров двоичных файлов:

```
$ readelf -s bench1
```

```
Symbol table '.dynsym' contains 10 entries:
```

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__ITM_deregisterTMCloneTab
2:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	puts@GLIBC_2.2.5 (2)
3:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	__libc_start_main@GLIBC...
4:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__gmon_start__
5:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	malloc@GLIBC_2.2.5 (2)
6:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	atoi@GLIBC_2.2.5 (2)
7:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	exit@GLIBC_2.2.5 (2)
8:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__ITM_registerTMCloneTable
9:	0000000000000000	0	FUNC	WEAK	DEFAULT	UND	__cxa_finalize@GLIBC_2...

strip(1) удалила таблицу символов .symtab, но оставила таблицу .dynsym. Таблица .dynsym содержит глобальные символы, вызываемые извне, а таблица .symtab — те же глобальные символы плюс локальные, используемые только самим приложением. Даже после удаления .symtab в двоичном файле все еще остается информация о глобальных библиотечных символах, но интересующих вас символов может не быть.

Статически скомпилированные приложения могут потерять все символы после применения strip(1), потому что в этом случае все символы помещаются в удаляемую таблицу .symtab.

Исправить ситуацию можно как минимум двумя способами:

- убрать вызов strip(1) из процесса сборки и повторно скомпилировать ПО;
- использовать другой источник символов: отладочную информацию в формате DWARF или BTF.

Иногда с ПО распространяется дополнительный пакет с расширением -dbg, -dbgsym или -debuginfo, содержащий отладочную информацию. Такие пакеты поддерживаются командой perf(1), BCC и bpfttrace.

Отладочная информация

Файл с отладочной информацией может иметь то же имя, что и двоичный файл, но с расширением `.debuginfo` или с уникальной контрольной суммой, идентифицирующей сборку. Он может находиться в `/usr/lib/debug/.build-id` или с его пользовательской версией. В последнем случае идентификатор сборки хранится в разделе примечаний ELF и его можно получить с помощью `readelf -n`.

Возьмем в качестве примера систему, где установлены пакеты `openjdk-11-jre` и `openjdk-11-dbg` с файлами `libjvm.so` и `libjvm.debuginfo`. Вот как можно подсчитать символы в каждом из них:

```
$ readelf -s /usr/lib/jvm/.../libjvm.so | wc -l
456
$ readelf -s /usr/lib/jvm/.../libjvm.debuginfo | wc -l
52299
```

Версия без отладочной информации включает 456 символов, а версия с отладочной информацией — 52 299.

Удаление ненужной отладочной информации

Может показаться, что всегда целесообразно устанавливать файл с отладочной информацией, однако он занимает дополнительное место на диске. Например, файл с отладочной информацией для `libjvm.so` имеет размер 222 Мбайт, тогда как сама библиотека — всего 17 Мбайт. Значительную часть этого объема составляют не символы, а другие разделы с отладочной информацией. Объем, занимаемый информацией о символах, можно узнать с помощью `readelf(1)`:

```
$ readelf -S libjvm.debuginfo
There are 39 section headers, starting at offset 0xdd40468:
Section Headers:
  [Nr] Name                Type              Address            Offset
       Size              EntSize          Flags  Link  Info  Align
[...]
  [36] .symtab               SYMTAB            0000000000000000 0da07530
       00000000001326c0 0000000000000018             37 51845  8
[...]
```

Как видите, размер `.symtab` составляет всего 1.2 Мбайт. Для сравнения: пакет `openjdk`, куда входит библиотека `libjvm.so`, имеет размер 175 Мбайт.

Если хранить полный объем отладочной информации нежелательно, то можно удалить из файла ненужную отладочную информацию. В примере ниже — применение команды `objcopy(1)` для удаления ненужных разделов с отладочной информацией (имена которых начинаются с `«.debug_»`), чтобы получить облегченный файл. Его можно использовать как замену полновесному файлу с отладочной информацией или даже присоединить к двоичному файлу с помощью `eu-unstrip(1)`. Например:

```
$ objcopy -R.debug_* libjvm.debuginfo libjvm.symtab
$ eu-unstrip -o libjvm.new.so libjvm.so libjvm.symtab
$ ls -lh libjvm.orig.so libjvm.debuginfo libjvm.symtab libjvm.new.so
-rwxr-xr-x 1 root root 222M Nov 13 04:53 libjvm.debuginfo*
-rwxr-xr-x 1 root root 20M Feb 16 19:02 libjvm.new.so*
-rw-r--r-- 1 root root 17M Nov 13 04:53 libjvm.so
-rwxr-xr-x 1 root root 3.3M Feb 16 19:00 libjvm.symtab*
$ readelf -s libjvm.new.so | wc -l
52748
```

Новый файл `libjvm.new.so` занимает всего 20 Мбайт и содержит все символы. Хочу обратить ваше внимание на то, что упомянутый прием я разработал специально для этой книги только для того, чтобы показать концептуальную возможность. Я не испытывал его в производственных условиях.

BTF

В будущем формат BPF Type Format (BTF) сможет предложить еще один облегченный источник отладочной информации. Этот формат был разработан для использования в BPF. Сейчас реализована только поддержка BTF для ядра: работа над версией для пространства пользователя еще не началась. Описание BTF ищите в главе 2.

bpfftrace

Извлечь список символов из двоичного файла можно не только с помощью `readelf(1)`, но также с `bpfftrace`. Для этого нужно лишь запросить список `uprobes`, доступных для инструментации¹:

```
# bpfftrace -l 'uprobe:/bin/bash'
uprobe:/bin/bash:rl_old_menu_complete
uprobe:/bin/bash:maybe_make_export_env
uprobe:/bin/bash:initialize_shell_builtins
uprobe:/bin/bash:extglob_pattern_p
uprobe:/bin/bash:dispose_cond_node
[...]
```

Также можно использовать шаблонные символы:

```
# bpfftrace -l 'uprobe:/bin/bash:read*'
uprobe:/bin/bash:reader_loop
uprobe:/bin/bash:read_octal
uprobe:/bin/bash:readline_internal_char
uprobe:/bin/bash:readonly_builtin
uprobe:/bin/bash:read_tty_modified
[...]
```

В разделе 12.2.3 приведен пример инструментации одного из этих зондов.

¹ Эту функцию разработал Матеус Марчини. Он прочитал рукопись этой главы и понял, что такая функция нужна.

12.2.2. Трассировка стека программного кода на C

BPF поддерживает обход стека с использованием указателя фрейма. Для этого ПО должно быть скомпилировано с поддержкой использования регистра указателя фрейма. В gcc за эту поддержку отвечает параметр `-fno-omit-frame-pointer`. Планируется, что BPF будет поддерживать также другие способы обхода стека.

Поскольку BPF — программируемый инструмент, я смог реализовать обход стека по указателю фрейма на чистом BPF еще до того, как была добавлена фактическая поддержка такой возможности [134]. Алексей Старовойтов добавил официальную поддержку, включающую карту нового типа `BPF_MAP_TYPE_STACK_TRACE` и вспомогательную функцию `bpf_get_stackid()`. Функция возвращает уникальный идентификатор стека, а карта хранит его содержимое. Это минимизирует объем памяти, необходимой для хранения трассировок стека, потому что результаты, извлекаемые повторно, хранятся с одним и тем же идентификатором.

Из `bpftool` доступ к стекам в пространствах пользователя и ядра можно получить с помощью встроенных модулей `ustack` и `kstack` соответственно. Вот пример трассировки оболочки `bash`, которая фактически является большой программой, написанной на C, с выводом трассировки стека, ведущего к чтению дескриптора файла 0 (STDIN):

```
# bpftool -e 't:syscalls:sys_enter_read /comm == "bash" &&
    args->fd == 0/ { @[ustack] = count(); }'
Attaching 1 probe...
^C

@[
    read+16
    0x6c63004344006d
]: 7
```

Этот стек на самом деле неполный: за функцией `read()` следует шестнадцатеричное число, не похожее на адрес (определить принадлежность некоторого адреса адресному пространству процесса с заданным PID можно с помощью `rmap(1)`, в данном случае это число не является адресом).

А вот результат после компиляции `bash` с флагом `-fno-omit-frame-pointer`:

```
# bpftool -e 't:syscalls:sys_enter_read /comm == "bash" &&
    args->fd == 0/ { @[ustack] = count(); }'
Attaching 1 probe...
^C

@[
    read+16
    r1_read_key+307
    readline_internal_char+155
    readline_internal_charloop+22
    readline_internal+23
    readline+91
```



```

yy_readline_get+142
yy_readline_get+412
yy_getc+13
shell_getc+464
read_token+250
yylex+184
yyparse+776
parse_command+122
read_command+203
reader_loop+377
main+2355
__libc_start_main+240
0xa9de258d4c544155

```

]: 30

Теперь трассировка стека видна полностью. Она выводится сверху вниз, от листа к корню. Иначе говоря, «сверху вниз» означает от дочерних вызовов к родительским.

Как показывает этот пример, командная оболочка `bash` читает из устройства `STDIN`, вызывая функции `getline()` и `read_command()`.

В конце стека, после `__libc_start_main`, показан еще один фиктивный адрес. Дело в том, что стек ушел в системную библиотеку `libc`, которая была скомпилирована без поддержки указателя фрейма.

Подробности о том, как BPF выполняет обход стека, и о будущих возможностях ищите в разделе 2.4.

12.2.3. Трассировка функций на C

Функции ядра можно трассировать с помощью `kprobes` и `kretprobes`, а функции в пространстве пользователя — с помощью `uprobes` и `uretprobes`. Эти технологии были представлены в главе 2, а в главе 5 рассказывалось, как их применять из `bpftool`. В этой книге есть много примеров их использования.

Рассмотрим еще один пример. Следующий сценарий трассирует функцию `getline()`, которая обычно включается в командную оболочку `bash`. Поскольку оболочка `bash` выполняется в пространстве пользователя, трассировку можно организовать с помощью `uprobes`. Вот сигнатура трассируемой функции:

```
char * readline(char *prompt)
```

Она принимает строковый аргумент — текст приглашения к вводу — и возвращает строку. В `uprobe` аргумент с приглашением доступен как `arg0`:

```

# bpftool -e 'uprobe:/bin/bash:readline { printf("readline: %s\n", str(arg0)); }'
Attaching 1 probe...
readline: bgregg:~/Build/bpftool/tools>
readline: bgregg:~/Build/bpftool/tools>

```

Здесь сценарий сформировал приглашение (`$PS1`), которое оболочка вывела в другом окне терминала.

Теперь посмотрим, что возвращает эта функция, используя `uretprobe`:

```
# bpfttrace -e 'uretprobe:/bin/bash:readline { printf("readline: %s\n",
    str(retval)); }'
Attaching 1 probe...
readline: date
readline: echo hello reader
```

Это текст, который ввел пользователь в другом окне.

Аналогично можно трассировать разделяемые библиотеки, если заменить путь `/bin/bash` в сценарии на путь к библиотеке. Некоторые дистрибутивы Linux¹ собирают оболочку `bash` так, что она вызывает функцию `readline` из библиотеки `libreadline`, из-за чего сценарий выше не работает, потому что символ `readline()` находится не в `/bin/bash`. Отследить вызов `readline` в этом случае можно, подставив путь к `libreadline`, например:

```
# bpfttrace -e 'uretprobe:/usr/lib/libreadline.so.8:readline {
    printf("readline: %s\n", str(retval)); }'
```

12.2.4. Трассировка смещений в функциях на C

Иногда нужно выполнить трассировку произвольного смещения внутри функции, а не только точек входа и выхода. Кроме получения более полного представления о потоке выполнения функции, исследование регистров также позволяет проверить содержимое локальных переменных.

Трассировку произвольных смещений можно выполнить с помощью `uprobes` и `kprobes`, а также функций `attach_uprobe()` и `attach_kprobe()` из ВСС. Но этой возможности пока нет в таких инструментах ВСС, как `trace(8)` и `funccount(8)`, и в `bpfttrace`. Добавить ее в эти инструменты не составит труда. Сложность будет заключаться в безопасном добавлении. `uprobes` не проверяет выравнивание инструкций, поэтому трассировка неправильного адреса (например, в середине многобайтной инструкции) повредит инструкции в целевой программе, что приведет к непредсказуемому сбою. Другие трассировщики, такие как `perf(1)`, используют отладочную информацию для проверки выравнивания инструкций.

12.2.5. Трассировка программного кода на C с помощью зондов USDT

Программы на C могут добавлять зонды USDT для статической инструментации. Это надежный API, предназначенный для использования в инструментах трассировки. Некоторые программы и библиотеки уже предоставляют зонды USDT. Например, вот список зондов USDT в `libc`, полученный с помощью `bpfttrace`:

¹ Например, Arch Linux.

```
# bpftrace -l 'usdt:/lib/x86_64-linux-gnu/libc-2.27.so'
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:setjmp
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:longjmp
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:longjmp_target
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_malloc_arena_max
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_malloc_arena_test
usdt:/lib/x86_64-linux-gnu/libc-2.27.so:libc:memory_tunable_tcache_max_bytes
[...]
```

Возможность инструментации USDT предоставляют многие библиотеки, включая `systemtap-sdt-dev` и Facebook Folly. Пример добавления зондов USDT в программу на языке C ищите в главе 2.

12.2.6. Трассировка программного кода на C с помощью однострочных сценариев

В этом разделе перечисляются однострочные сценарии для BCC и `bpftrace`. Там, где это возможно, один и тот же сценарий реализуется с использованием BCC и `bpftrace`.

BCC

Подсчитывает вызовы функций ядра с именами, начинающимися с «attach»:

```
funccount 'attach*'
```

Подсчитывает вызовы функций в двоичном файле (например, `/bin/bash`) с именами, начинающимися с «a»:

```
funccount '/bin/bash:a*'
```

Подсчитывает вызовы функций в библиотеке (например, `libc.so.6`) с именами, начинающимися с «a»:

```
funccount '/lib/x86_64-linux-gnu/libc.so.6:a*'
```

Трассирует функцию и ее аргумент (например, `readline()` в `bash`):

```
trace '/bin/bash:readline "%s", arg1'
```

Трассирует функцию и ее возвращаемое значение (например, `readline()` в `bash`):

```
trace 'r:/bin/bash:readline "%s", retval'
```

Трассирует библиотечную функцию и ее аргумент (например, `fopen()` в `libc`):

```
trace '/lib/x86_64-linux-gnu/libc.so.6:fopen "%s", arg1'
```

Оценивает распределение значений, возвращаемых функцией (например, `fopen()` в `libc`):

```
argdist -C 'r:/lib/x86_64-linux-gnu/libc.so.6:fopen():int:$retval'
```

Подсчитывает трассировки стека в пространстве пользователя, ведущие к функции (например, `readline()` в `bash`):

```
stackcount -U '/bin/bash:readline'
```

Производит выборку стека в пространстве пользователя с частотой 49 Гц:

```
profile -U -F 49
```

bpfttrace

Подсчитывает вызовы функций ядра с именами, начинающимися с «`attach`»:

```
bpfttrace -e 'kprobe:attach* { @[probe] = count(); }'
```

Подсчитывает вызовы функций в двоичном файле (например, `/bin/bash`) с именами, начинающимися с «`a`»:

```
bpfttrace -e 'uprobe:/bin/bash:a* { @[probe] = count(); }'
```

Подсчитывает вызовы функций в библиотеке (например, `libc.so.6`) с именами, начинающимися с «`a`»:

```
bpfttrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:a* { @[probe] = count(); }'
```

Трассирует функцию и ее аргумент (например, `readline()` в `bash`):

```
bpfttrace -e 'u:/bin/bash:readline { printf("prompt: %s\n", str(arg0)); }'
```

Трассирует функцию и ее возвращаемое значение (например, `readline()` в `bash`):

```
bpfttrace -e 'ur:/bin/bash:readline { printf("read: %s\n", str(retval)); }'
```

Трассирует библиотечную функцию и ее аргумент (например, `fopen()` в `libc`):

```
bpfttrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:fopen { printf("opening: %s\n", str(arg0)); }'
```

Оценивает распределение значений, возвращаемых функцией (например, `fopen()` в `libc`):

```
bpfttrace -e 'ur:/lib/x86_64-linux-gnu/libc.so.6:fopen { @[retval] = count(); }'
```

Подсчитывает трассировки стека в пространстве пользователя, ведущие к функции (например, `readline()` в `bash`):

```
bpfttrace -e 'u:/bin/bash:readline { @[ustack] = count(); }'
```

Производит выборку стека в пространстве пользователя с частотой 49 Гц:

```
bpfttrace -e 'profile:hz:49 { @[ustack] = count(); }'
```

12.3. JAVA

Код на Java — сложная цель для трассировки. JVM компилирует методы Java в байт-код, а затем выполняет их в интерпретаторе. Когда количество вызовов метода превышает некоторый порог (`-XX:CompileThreshold`), они компилируются JIT-компилятором в машинные инструкции. JVM также профилирует выполнение методов и повторно компилирует их для увеличения производительности, изменяя их местоположение в памяти на лету. Для компиляции, управления потоками выполнения и сборки мусора JVM использует библиотеки на C++. Наиболее широкое распространение получила JVM с названием HotSpot, которая первоначально была разработана в Sun Microsystems.

Компоненты JVM на C++ (`libjvm`) можно инструментировать точно так же, как скомпилированный код, написанный на других языках, о чем шла речь в предыдущем разделе. JVM предоставляет множество зондов USDT, упрощая трассировку своих внутренних компонентов. Эти зонды USDT также позволяют инструментировать методы Java, но при этом могут возникать проблемы, о которых мы поговорим в этом разделе.

В начале этого раздела дается краткий обзор трассировки `libjvm` на C++, а затем обсуждаются имена потоков выполнения в Java, символы методов Java, приемы трассировки стека Java, зонды USDT в Java и однострочные сценарии для трассировки кода на Java. Здесь же мы познакомимся с инструментами для анализа работы программного кода на Java. Они перечислены в табл. 12.2.

Таблица 12.2. Инструменты для анализа работы программного кода на Java

Инструмент	Источник	Цель	Описание
<code>jnistacks</code>	ВСС	<code>libjvm</code>	Выводит список потребителей JNI по трассировке стека объектов
<code>profile</code>	ВСС	Процессоры	Выбирает трассировки стека, включая методы Java
<code>offcputime</code>	ВСС	Планировщик	Трассировки стека и времена ожидания доступности процессора, включая методы Java
<code>stackcount</code>	ВСС	События	Подсчитывает трассировки стека, приводящие к заданному событию
<code>javastat</code>	ВСС	USDT	Выводит статистику работы языка высокого уровня
<code>javathreads</code>	Книга	USDT	Трассирует события запуска и остановки потоков выполнения
<code>javacalls</code>	ВСС/книга	USDT	Подсчитывает вызовы методов Java
<code>javaflow</code>	ВСС	USDT	Отображает поток выполнения кода метода Java
<code>javagc</code>	ВСС	USDT	Трассирует события сборки мусора
<code>javaobjnew</code>	ВСС	USDT	Подсчитывает события выделения памяти для новых объектов Java

Некоторые из этих инструментов отображают методы Java, но чтобы показать их вывод на производственных серверах Netflix, потребовалось бы изменить внутренний код, что сделает примеры труднодоступными. Поэтому я покажу приемы их использования на примере Java-игры с открытым исходным кодом: freecol. У игры сложный и чувствительный к производительности программный код, что делает его похожим на производственный код Netflix¹. Сайт freecol: <http://www.freecol.org>.

12.3.1. Трассировка libjvm

Основная библиотека JVM, libjvm, содержит тысячи функций для запуска потоков выполнения Java, загрузки классов, компиляции методов, выделения памяти, сборки мусора и т. д. В основном они написаны на C++, и их можно трассировать, чтобы получить представление о работе программы на Java.

В качестве примера я покажу подсчет вызовов всех функций стандартного интерфейса Java (Java Native Interface, JNI) с помощью ВСС-функции `funccount(8)` (аналогичный подсчет можно выполнить с помощью `bpfttrace`):

```
# funccount '/usr/lib/jvm/java-11-openjdk-amd64/lib/server/libjvm.so:jni_*'
Tracing 235 functions
for "/usr/lib/jvm/java-11-openjdk-amd64/lib/server/libjvm.
so:jni_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
jni_GetObjectClass                  1
jni_SetLongArrayRegion              2
jni_GetEnv                          15
jni_SetLongField                    42
jni_NewWeakGlobalRef                84
jni_FindClass                       168
jni_GetMethodID                     168
jni_NewObject                       168
jni_GetObjectField                  168
jni_ExceptionOccurred               719
jni_CallStaticVoidMethod            1144
jni_ExceptionCheck                  1186
jni_ReleasePrimitiveArrayCritical   3787
jni_GetPrimitiveArrayCritical       3787
Detaching...
```

Этот сценарий подсчитывает вызовы функций из библиотеки libjvm.so, имена которых начинаются с «jni_*». Судя по результатам, чаще других вызывались функции `jni_GetPrimitiveArrayCritical()` и `jni_ReleasePrimitiveArrayCritical`, за время трассировки они были вызваны по 3787 раз. Путь к libjvm.so в выводе был усечен, чтобы предотвратить перенос строк.

¹ На конференции SCaLE 2019 я провел анализ с использованием BPF другой сложной Java-игры — Minecraft. Анализировать эту игру было так же сложно, как производственные приложения Netflix и freecol, но она не подходит для примера в книге, поскольку не опенсорсная.

Символы *libjvm*

Библиотека `libjvm.so`, входящая в состав дистрибутива JDK, обработана утилитой `strip(1)`, то есть в ней нет таблицы локальных символов и вызовы этих функций JNI нельзя подсчитать без дополнительных усилий. Проверить это можно с помощью `file(1)`:

```
$ file /usr/lib/jvm/java-11-openjdk-amd64/lib/server/libjvm.orig.so
/usr/lib/jvm/java-11-openjdk-amd64/lib/server/libjvm.orig.so: ELF 64-bit LSB shared
object, x86-64, version 1 (GNU/Linux), dynamically linked,
BuildID[sha1]=f304ff36e44ce8a68a377cb07ed045f97aee4c2f, stripped
```

Возможные решения:

- Собрать свою версию `libjvm` из исходного кода без использования (1).
- Установить (если доступен) пакет JDK с отладочной информацией, анализ которой поддерживают BCC и `bpfttrace`.
- Установить пакет JDK с отладочной информацией и использовать `unstrip(1)` из `elfutils`, чтобы добавить таблицу символов в `libjvm.so` (как это сделать, рассказано в подразделе «Отладочная информация», в разделе 12.2.1).
- Использовать BTF, когда появится его поддержка (описывается в главе 2).

Для этого примера я взял второе решение.

12.3.2. `jnistacks`

В качестве примера инструментации `libjvm` ниже показан подсчет трассировок стека с использованием `jnistacks(8)`¹, которые привели к вызову функции `jni_NewObject()` из предыдущего вывода, и других, имена которых начинаются с «`jni_NewObject`». Этот пример показывает, какие пути в коде, включая методы Java, привели к созданию новых объектов JNI. Пример вывода:

```
# bpfttrace --unsafe jnistacks.bt
Tracing jni_NewObject* calls... Ctrl-C to end.
^C
Running /usr/local/bin/jmaps to create Java symbol files in /tmp...
Fetching maps for all java processes...
Mapping PID 25522 (user bgregg):
wc(1): 8350 26012 518729 /tmp/perf-25522.map

[...]
@[
  jni_NewObject+0
  Lsun/awt/X11GraphicsConfig::pGetBounds+171
  Ljava/awt/MouseInfo::getPointerInfo+2048
  Lnet/sf/freecol/client/gui/plaf/FreeColButtonUI::paint+1648
  Ljavax/swing/plaf/metal/MetalButtonUI::update+232
```

¹ Немного истории: я написал его для этой книги 8 февраля 2019 года.

```

Ljavax/swing/JComponent;::paintComponent+672
Ljavax/swing/JComponent;::paint+2208
Ljavax/swing/JComponent;::paintChildren+1196
Ljavax/swing/JComponent;::paint+2256
Ljavax/swing/JComponent;::paintChildren+1196
Ljavax/swing/JComponent;::paint+2256
Ljavax/swing/JLayeredPane;::paint+2356
Ljavax/swing/JComponent;::paintChildren+1196
Ljavax/swing/JComponent;::paint+2256
Ljavax/swing/JComponent;::paintToOffscreen+836
Ljavax/swing/BufferStrategyPaintManager;::paint+3244
Ljavax/swing/RepaintManager;::paint+1260
Interpreter+5955
Ljavax/swing/JComponent;::paintImmediately+3564
Ljavax/swing/RepaintManager$4;::run+1684
Ljavax/swing/RepaintManager$4;::run+132
call_stub+138
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
JVM_DoPrivileged+1600
Ljava/security/AccessController;::doPrivileged+216
Ljavax/swing/RepaintManager;::paintDirtyRegions+4572
Ljavax/swing/RepaintManager;::paintDirtyRegions+660
Ljavax/swing/RepaintManager;::prePaintDirtyRegions+1556
Ljavax/swing/RepaintManager$ProcessingRunnable;::run+572
Ljava/awt/EventQueue$4;::run+1100
call_stub+138
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
]: 232

```

Для краткости я включил только последний стек. Когда мы просматриваем его снизу вверх, то видим путь к этому вызову, сверху вниз — истоки. Судя по всему, путь к этой функции начинается с извлечения события из очереди (EventQueue), затем выполняются методы рисования и, наконец, вызывается метод `sun.awt.X11GraphicsConfig::pGetBounds()`, который выполняет вызов JNI — я бы предположил, что он делает это потому, что ему потребовалось вызвать графическую библиотеку X11.

В трассировке видно несколько фреймов `Interpreter()`: это виртуальная машина выполняет методы Java с использованием своего интерпретатора, количество вызовов которых еще не превысило порог `CompileThreshold` и они не были скомпилированы в машинный код.

Читать этот стек немного сложно, потому что символы Java — это сигнатуры классов. `bpfftrace` пока не поддерживает форматирование сигнатур. Инструмент `c++filt(1)` на момент написания книги тоже не поддерживает эту версию сигнатур классов Java¹. Для примера, следующий символ:

```
Ljavax/swing/RepaintManager;::prePaintDirtyRegions+1556
```

¹ Вы можете сами попробовать исправить `bpfftrace` и `c++filter(1)`.

следовало бы преобразовать в:

```
javax.swing.RepaintManager::prePaintDirtyRegions()+1556
```

Исходный код jnistacks(8):

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing jni_NewObject* calls... Ctrl-C to end.\n");
}

uprobe:/usr/lib/jvm/java-11-openjdk-amd64/lib/server/libjvm.so:jni_NewObject*
{
    @[ustack] = count();
}

END
{
    $jmaps = "/usr/local/bin/jmaps";
    printf("\nRunning %s to create Java symbol files in /tmp...\n", $jmaps);
    system("%s", $jmaps);
}
```

Зонд uprobe трассирует вызовы всех функций в libjvm.so, имена которых начинаются с «jni_NewObject*», и подсчитывает их частоту в пространстве пользователя.

Блок END запускает внешнюю программу jmaps, которая записывает дополнительный файл с символами методов Java в /tmp. Затем вызывается функция system(), для чего нужен аргумент командной строки --unsafe, потому что команды, выполняемые с помощью system(), не могут быть проверены верификатором BPF.

Вывод jmaps включается в вывод bpfttrace. Его содержимое описывается в разделе 12.3.4. jmaps можно запускать извне и не обязательно в этой программе bpfttrace (при желании вы можете удалить блок END). Но чем больше пройдет времени между запуском jmaps и использованием дампа символов, тем выше вероятность появления устаревших или неправильно переведенных символов. Включение вызова этой программы в END гарантирует, что она выполнится непосредственно перед выводом стека, что снизит время между его извлечением и использованием.

12.3.3. Имена потоков выполнения в Java

JVM позволяет настраивать имена потоков выполнения. Если вы попытаетесь использовать «java» в качестве имени процесса, то можете не найти никаких событий, поскольку потоки могут иметь другое имя. Вот пример с использованием bpfttrace:

```
# bpfttrace -e 'profile:hz:99 /comm == "java"/ { @ = count(); }'
Attaching 1 probe...
^C
#
```

Здесь сопоставление выполняется по идентификатору процесса Java и отображаются идентификаторы потоков и содержимое встроенного значения `comm`:

```
# bpftrace -e 'profile:hz:99 /pid == 16914/ { @[tid, comm] = count(); }'
Attaching 1 probe...
^C

@[16936, VM Periodic Tas]: 1
[...]
@[16931, Sweeper thread]: 4
@[16989, FreeColClient:b]: 4
@[21751, FreeColServer:A]: 7
@[21779, FreeColClient:b]: 18
@[21780, C2 CompilerThre]: 20
@[16944, AWT-XAWT]: 22
@[16930, C1 CompilerThre]: 24
@[16946, AWT-EventQueue-]: 51
@[16929, C2 CompilerThre]: 241
```

Встроенное значение `comm` содержит имя потока (задачи), а не имя родительского процесса. Это позволяет получить более конкретную информацию о потоках: в примере выше видно, что в период трассировки `C2 CompilerThread` (имя в примере вывода усечено) потребил больше всего вычислительных ресурсов. Но это может сбивать с толку, потому что другие инструменты, включая `top(1)`, показывают имя родительского процесса: «java»¹.

Эти имена потоков можно увидеть в `/proc/PID/task/TID/comm`. Пример ниже использует `grep(1)`, чтобы вывести их с именами файлов:

```
# grep . /proc/16914/task/*/comm
/proc/16914/task/16914/comm:java
[...]
/proc/16914/task/16959/comm:GC Thread#7
/proc/16914/task/16963/comm:G1 Conc#1
/proc/16914/task/16964/comm:FreeColClient:W
/proc/16914/task/16981/comm:FreeColClient:S
/proc/16914/task/16982/comm:TimerQueue
/proc/16914/task/16983/comm:Java Sound Even
/proc/16914/task/16985/comm:FreeColServer:S
/proc/16914/task/16989/comm:FreeColClient:b
/proc/16914/task/16990/comm:FreeColServer:-
```

В дальнейших примерах будет выполняться сопоставление с идентификатором процесса Java вместо имени «java», и теперь вы знаете почему. Есть еще одна причина: зонды USDT, использующие семафор, передают PID в `bpftrace`, чтобы установить семафор для этого PID. Подробности о семафорах ищите в разделе 2.10.1.

¹ В будущем мы планируем добавить в ядро функцию `bpf_get_current_pcomm()`, возвращающую имя процесса, которое можно будет использовать в дополнение к имени потока. В `bpftrace` это расширение может появиться в виде значения «`pcomm`».

12.3.4. Символы методов Java

Для создания вспомогательных файлов символов, содержащих адреса соответствующих методов Java, можно использовать открытое ПО perf-map-agent [135]. Это нужно делать каждый раз, когда вы выводите трассировку стека или адреса методов Java, иначе адреса будут неизвестны. perf-map-agent использует соглашение, принятое в Linux perf(1), о записи текстового файла в /tmp/perf-PID.map в следующем формате [136]:

```
START SIZE symbolname
```

Вот несколько примеров символов из производственного Java-приложения, содержащих «sun» (как пример):

```
$ grep sun /tmp/perf-3752.map
[...]
7f9ce1a04f60 80 Lsun/misc/FormattedFloatingDecimal::getMantissa
7f9ce1a06d60 7e0 Lsun/reflect/GeneratedMethodAccessor579::invoke
7f9ce1a08de0 80 Lsun/misc/FloatingDecimal$BinaryToASCIIBuffer::isExceptional
7f9ce1a23fc0 140 Lsun/security/util/Cache::newSoftMemoryCache
7f9ce1a243c0 120 Lsun/security/util/Cache::<init>
7f9ce1a2a040 1e80 Lsun/security/util/DerInputBuffer::getBigInteger
7f9ce1a2ccc0 980 Lsun/security/util/DisabledAlgorithmConstraints::permits
7f9ce1a36c20 200 Lcom/sun/jersey/core/reflection/ReflectionHelper::findMethodOnCl...
7f9ce1a3a360 6e0 Lsun/security/util/MemoryCache::<init>
7f9ce1a523c0 760 Lcom/sun/jersey/core/reflection/AnnotatedMethod::hasMethodAnnota...
7f9ce1a60b60 860 Lsun/reflect/GeneratedMethodAccessor682::invoke
7f9ce1a68f20 320 Lsun/nio/ch/EPollSelectorImpl::wakeup
[...]
```

perf-map-agent можно запускать по мере необходимости и подключать к действующему процессу Java, чтобы выгрузить таблицу символов. Обратите внимание, что эта процедура может ухудшать производительность на время выгрузки дампа символов, а для больших приложений Java оверхед может достигать 1 секунды процессорного времени.

Имейте в виду, что это моментальный снимок таблицы символов, и он быстро устареваает, потому что компилятор Java может повторно скомпилировать методы даже после того, как рабочая нагрузка, казалось бы, достигла устойчивого состояния. Чем больше времени между моментальным снимком с дампом символов и запуском инструмента BPF для перевода символов методов, тем выше вероятность, что символы окажутся устаревшими или будут неправильно переведены. При анализе производственных приложений, выполняющихся под большой нагрузкой, когда частота повторной компиляции особенно высока, я не доверяю дампам символов Java, срок жизни которых превышает 60 секунд.

В разделе 12.3.5 приведен пример трассировки стека без таблицы символов, полученной с помощью perf-map-agent, а затем с ней — после запуска jmaps.

Автоматизация

Чтобы снизить время между созданием и использованием дампов символов в инструментах ВРФ, можно воспользоваться средствами автоматизации. Такие средства есть в проекте perf-map-agent, а кроме того, я опубликовал свою программу под названием jmaps [137]. Она отыскивает все java-процессы (по имени) и выгружает их таблицы символов. Вот пример запуска jmaps на производственном сервере с 48 процессорами:

```
# time ./jmaps
Fetching maps for all java processes...
Mapping PID 3495 (user www):
wc(1): 116736 351865 9829226 /tmp/perf-3495.map

real    0m10.495s
user    0m0.397s
sys     0m0.134s
```

Мы видим несколько разных статистик: jmaps запускает утилиту wc(1) для анализа полученного дампа символов, которая сообщает, что этот дамп содержит более 116 000 строк (символов) и имеет общий объем 9.4 Мбайт (9 829 226 байт). Я также использовал утилиту time(1), чтобы показать, сколько времени потребовалось для создания дампа: в роли предмета анализа использовалось нагруженное Java-приложение, занимающее 174 Гбайт оперативной памяти, и для его анализа потребовалось 10.5 секунды. (Большая часть использованного процессорного времени не видна в статистике user и sys, поскольку оно было использовано JVM.)

При использовании в паре с ВСС программу jmaps можно запускать непосредственно перед запуском инструмента. Например:

```
./jmaps; trace -U '...'
```

Здесь сразу после завершения jmaps вызывается команда trace(8), что снижает вероятность устаревания символов. В случае с инструментами, обобщающими трассировки стека (например, stackcount(8)), можно изменить сам инструмент и вызывать jmaps в нем, непосредственно перед выводом результатов.

В инструментах bpftrace, использующих printf(), программу jmaps можно запускать в блоке BEGIN, а в инструментах, которые выводят информацию, накопленную в картах, — в блоке END. Пример последнего варианта — инструмент jnistacks(8), обсуждавшийся выше.

Другие приемы и новые возможности в будущем

Благодаря перечисленным выше методам, уменьшающим вероятность устаревания символов, решение на основе perf-map-agent хорошо подходит для многих сред. Но есть и другие приемы, лучше справляющиеся с проблемой устаревания

таблицы символов, поддержку которых добавят в BCC в будущем. Вот их краткое описание:

- **Регистрация символов с отметкой времени:** `perf(1)` поддерживает такую возможность, и нужное ПО уже есть в исходном коде ядра Linux¹. Сейчас эта поддержка делает непрерывное логирование, что влечет за собой некоторый оверхед, сказывающийся на производительности. В идеале желательно, чтобы логирование производилось по запросу и включалось в начале трассировки, а затем, после отключения, создавался полный снимок таблицы символов. Это позволило бы восстанавливать состояние символа во времени на основе данных хронометража и снапшотов без потери производительности, связанной с непрерывным логированием².
- **Обеспечение видимости устаревших символов:** должна быть возможность выгрузить таблицу символов до и после, чтобы отыскать изменившиеся места, а затем построить новую таблицу символов, в которой эти места отмечены как ненадежные.
- **async-profile:** этот метод объединяет трассировки стека `perf_events` с трассировками, полученными с помощью Java-интерфейса `AsyncGetCallTrace`. Этот подход не требует включения указателей фреймов.
- **Поддержка в ядре:** этот метод обсуждался в сообществе BPF. В будущем мы, возможно, добавим в ядро поддержку дополнительных методов трассировки стека, реализовав встроенную процедуру трансляции символов. Об этом упоминалось в главе 2.
- **Встроенная в JVM поддержка создания дампов символов:** `perf-map-agent` — это однопоточный модуль, который ограничен интерфейсом `JVMTI`. Если бы JVM поддерживала прямой вывод файлов символов в `/tmp/perf-PID`, например, при получении сигнала или при обращении к `JVMTI`, то вполне вероятно, что такая поддержка могла бы быть намного более эффективной.

Все это — направления для развития в будущем.

12.3.5. Приемы трассировки стека Java

По умолчанию Java не использует регистр указателя фрейма, и метод обхода стека на его основе не работает. Вот пример использования `bpfftrace` для выборки трассировок стека Java-процесса по времени:

```
# bpfftrace -e 'profile:hz:99 /pid == 3671/ { @[ustack] = count(); }'
Attaching 1 probe...
^C
```

¹ См. `tools/perf/jvmti` в исходном коде Linux.

² Я говорил об этом со Стефаном Эранианом (Stephane Eranian), который добавил поддержку `jvmti` в `perf(1)`, но не думаю, что у него или у меня найдется время реализовать это.

```

@[
    0x7efcfff88a7bd
    0x12f023020020fd4
]: 1
@[
    0x7efcfff88a736
    0x12f023020020fd4
]: 1
@[
    IndexSet::alloc_block_containing(unsigned int)+75
    PhaseChaitin::interfere_with_live(unsigned int, IndexSet*)+628
    PhaseChaitin::build_ifg_physical(ResourceArea*)+1812
    PhaseChaitin::Register_Allocate()+1834
    Compile::Code_Gen()+628
    Compile::Compile(ciEnv*, C2Compiler*, ciMethod*, int, bool, bool, bool, Direct...
    C2Compiler::compile_method(ciEnv*, ciMethod*, int, DirectiveSet*)+177
    CompileBroker::invoke_compiler_on_method(CompileTask*)+931
    CompileBroker::compiler_thread_loop()+1224
    JavaThread::thread_main_inner()+259
    thread_native_entry(Thread*)+240
    start_thread+219
]: 1
@[
    0x7efcfff72fc9e
    0x620000cc4
]: 1
@[
    0x7efcfff969ba8
]: 1
[...]
```

Этот вывод включает неполные стеки, имеющие вид одного или двух шестнадцатеричных адресов. Компилятор Java использует регистр указателя фрейма для хранения локальных переменных с целью оптимизации. Это немного ускоряет работу Java-кода (на процессорах с ограниченным числом регистров) ценой нарушения работоспособности метода обхода стека, который используется отладчиками и трассировщиками. Попытка получить трассировку стека обычно проваливается после первого адреса. Вывод, приведенный выше, включает результаты таких неудачных попыток, а также полный стек, созданный кодом на C++: поскольку код не вызывал никаких методов на Java, указатель фрейма не был поврежден.

PreserveFramePointer

Начиная с обновления 60 для Java 8, JVM предоставляет параметр `-XX: +Preserve-FramePointer` для включения указателя фрейма¹, который исправляет проблему

¹ Я разработал эту возможность и отправил патч в список рассылки `hotspot-compiler-devs`, приложив флейм-график процессора, чтобы объяснить ценность. Золтан Майо (Zoltán Majó) из Oracle переписал его, реализовав в виде параметра (`PreserveFramePointer`), и интегрировал в официальный JDK.

с трассировкой стека на основе указателя фрейма. После добавления этого параметра (для этого примера я добавил параметр `-XX: +PreserveFramePointer` в сценарий запуска `/usr/games/freecol`, в строку `run_java`) тот же однострочный сценарий для `bpfttrace` вывел:

```
# bpfttrace -e 'profile:hz:99 /pid == 3671/ { @[ustack] = count(); }'
Attaching 1 probe...
^C
[...]
```

@[

0x7fdbdf74ba04
0x7fdbd8be8814
0x7fdbd8bed0a4
0x7fdbd8beb874
0x7fdbd8ca336c
0x7fdbdf96306c
0x7fdbdf962504
0x7fdbdf62fef8
0x7fdbd8cd85b4
0x7fdbd8c8e7c4
0x7fdbdf9e9688
0x7fdbd8c83114
0x7fdbd88817184
0x7fdbdf9e96b8
0x7fdbd8ce57a4
0x7fdbd8cbecac
0x7fdbd8cb232c
0x7fdbd8cc715c
0x7fdbd8c846ec
0x7fdbd8cbb154
0x7fdbd8c7fdc4
0x7fdbd7b25849
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
JVM_DoPrivileged+1600
0x7fdbdf77fe18
0x7fdbd8ccd37c
0x7fdbd8cd1674
0x7fdbd8cd0c74
0x7fdbd8c8783c
0x7fdbd8bd8fac
0x7fdbd8b8a7b4
0x7fdbd8b8c514

```
]: 1
[...]
```

Теперь трассировка стека получилась полной, за исключением трансляции адресов в символы.

Стеки и символы

Как рассказывалось в разделе 12.3.4, с помощью `perf-map-agent` и `jmaps` можно создать вспомогательный файл символов. Вот результат после выполнения этого шага в блоке `END`:

```
# bpftrace --unsafe -e 'profile:hz:99 /pid == 4663/ { @[ustack] = count(); }
  END { system("jmaps"); }'
Attaching 2 probes...
^CFetching maps for all java processes...
Mapping PID 4663 (user bgregg):
wc(1): 6555 20559 388964 /tmp/perf-4663.map
@[
  Lsun/awt/X11/XlibWrapper;::RootWindow+31
  Lsun/awt/X11/XDecoratedPeer;::getLocationOnScreen+3764
  Ljava/awt/Component;::getLocationOnScreen_NoTreeLock+2260
  Ljavax/swing/SwingUtilities;::convertPointFromScreen+1820
  Lnet/sf/freecol/client/gui/plaf/FreeColButtonUI;::paint+1068
  Ljavax/swing/plaf/ComponentUI;::update+1804
  Ljavax/swing/plaf/metal/MetalButtonUI;::update+4276
  Ljavax/swing/JComponent;::paintComponent+612
  Ljavax/swing/JComponent;::paint+2120
  Ljavax/swing/JComponent;::paintChildren+13924
  Ljavax/swing/JComponent;::paint+2168
  Ljavax/swing/JLayeredPane;::paint+2356
  Ljavax/swing/JComponent;::paintChildren+13924
  Ljavax/swing/JComponent;::paint+2168
  Ljavax/swing/JComponent;::paintToOffscreen+836
  Ljavax/swing/BufferStrategyPaintManager;::paint+3244
  Ljavax/swing/RepaintManager;::paint+1260
  Ljavax/swing/JComponent;::_paintImmediately+12636
  Ljavax/swing/JComponent;::paintImmediately+3564
  Ljavax/swing/RepaintManager$4;::run+1684
  Ljavax/swing/RepaintManager$4;::run+132
  call_stub+138
  JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
  JVM_DoPrivileged+1600
  Ljava/security/AccessController;::doPrivileged+216
  Ljavax/swing/RepaintManager;::paintDirtyRegions+4572
  Ljavax/swing/RepaintManager;::paintDirtyRegions+660
  Ljavax/swing/RepaintManager;::prePaintDirtyRegions+1556
  Ljavax/swing/RepaintManager$ProcessingRunnable;::run+572
  Ljava/awt/event/InvocationEvent;::dispatch+524
  Ljava/awt/EventQueue;::dispatchEventImpl+6260
  Ljava/awt/EventQueue$4;::run+372
]: 1
```

Теперь мы имеем полную трассировку стека с адресами, преобразованными в символы. По всей видимости, этот стек принадлежит функции рисования кнопки в пользовательском интерфейсе (`FreeColButtonUI::paint()`).

Стеки библиотек

И последний пример: на этот раз трассировка стека системного вызова `read(2)`:

```
# bpftrace -e 't:syscalls:sys_enter_read /pid == 4663/ { @[ustack] = count(); }'
Attaching 1 probe...
^C

@[
```



```

    read+68
    0xc528280f383da96d
]: 11
@[
    read+68
]: 25

```

Эти стеки получились неполными, даже при условии, что Java-приложение запущено с параметром `-XX: +PreserveFramePointer`. Причина в том, что это обращение к системному вызову берет начало в функции `read()` библиотеки `libc`, которая скомпилирована с отключенной поддержкой указателя фрейма. Чтобы исправить проблему, можно пересобрать библиотеку или использовать другое средство обхода стека, который поддерживается инструментами BPF (например, DWARF или LBR).

Внесение исправлений для поддержки трассировки стека может потребовать много работы. Но эти затраты окупятся, поскольку такая возможность позволяет профилировать приложения, получать флейм-графики процессора и контекст выполнения для любого события.

12.3.6. Зонды USDT в Java

Зонды USDT, представленные в главе 2, обладают существенным преимуществом, предоставляя стабильный интерфейс для инструментации событий. В JVM есть зонды USDT для разных событий, в том числе для событий:

- жизненного цикла виртуальной машины;
- жизненного цикла потоков выполнения;
- загрузки классов;
- сборки мусора;
- компиляции методов;
- мониторинга;
- трассировки приложений;
- вызова методов;
- размещения объектов в памяти;
- мониторинга событий.

Трассировка этих событий доступна, только если JDK был скомпилирован с параметром `--enable-dtrace`, который, к сожалению, отключается при сборке дистрибутивов JDK для Linux. Чтобы использовать эти зонды USDT, скомпилируйте JDK из исходного кода с параметром `--enable-dtrace` или попросите мейнтейнеров пакета добавить этот параметр.

Зонды описаны в разделе «DTrace Probes in HotSpot VM» в руководстве «Java Virtual Machine Guide» [138]. Там подробно объясняется назначение каждого зонда и его аргументы. В табл. 12.3 перечислены некоторые из этих зондов.

Таблица 12.3. Зонды USDT

Группа USDT	Зонд USDT	Аргументы
hotspot	thread__start, thread__stop	char *thread_name, u64 thread_name_len, u64 thread_id, u64 os_thread_id, bool is_daemon
hotspot	class__loaded	char *class_name, u64 class_name_len, u64 loader_id, bool is_shared
hotspot	gc__begin	bool is_full_gc
hotspot	gc__end	—
hotspot	object__alloc	int thread_id, char *class_name, u64 class_name_len, u64 size
hotspot	method__entry, method__return	int thread_id, char *class_name, int class_name_len, char *method_name, int method_name_len, char *signature, int signature_len
hotspot_jni	AllocObject__entry	void *env, void *clazz

Полный список ищите в руководстве «Java Virtual Machine Guide».

Реализация USDT в Java

В качестве примера реализации зондов USDT в JDK ниже показан код зонда hotspot:gc__begin. Большинству людей не нужны эти детали. Здесь они даны лишь для некоторого представления о том, как работают зонды.

Зонд объявлен в файле определений зондов USDT: src/hotspot/os/posix/dtrace/hotspot.d:

```
provider hotspot {
[...]  
    probe gc__begin(uintptr_t);
```

Как следует из этого определения, зонд имеет имя hotspot:gc__begin. Во время сборки этот файл компилируется в заголовочный файл hotspot.h, содержащий макрос HOTSPOT_GC_BEGIN:

```
#define HOTSPOT_GC_BEGIN(arg1) \  
DTRACE_PROBE1 (hotspot, gc__begin, arg1)
```

Этот макрос вставляется в код JVM в нужное место. Здесь он вставлен в функцию notify_gc_begin(), соответственно, для выполнения зонда нужно вызвать эту функцию. Вот выдержка из src/hotspot/share/gc/shared/gcVMOperations.cpp:

```
void VM_GC_Operation::notify_gc_begin(bool full) {  
    HOTSPOT_GC_BEGIN(  
        full);  
    HS_DTRACE_WORKAROUND_TAIL_CALL_BUG();  
}
```

Эта функция включает также макрос для обхода ошибки в DTrace, который объявлен в заголовочном файле `dtrace.hpp` с комментарием «`// Work around dtrace tail call bug 6672627 until it is fixed in solaris 10`» («`// Для обхода ошибки (6672627) хвостового вызова в dtrace, пока она не будет исправлена в solaris 10`»).

Если сборка пакета JDK производится без параметра `--enable-dtrace`, то вместо него используется заголовочный файл `dtrace_disabled.hpp`, в котором эти макросы объявлены как пустые.

Есть макрос `HOTSPOT_GC_BEGIN_ENABLED`, применяемый для поддержки этого зонда: он возвращает истину, если зонд инструментирован, и используется реализацией, чтобы узнать, следует ли вычислять аргументы зонда или же эту операцию можно пропустить, если сейчас зонд не используется.

Получение списка зондов USDT в Java

Извлечь список зондов USDT из файла или выполняющегося процесса можно с помощью инструмента `tplist(8)` из BCC. Список, извлеченный из JVM, содержит более 500 зондов. Ниже приведен усеченный пример вывода, содержащий только некоторые интересные зонды, в котором также полный путь к `libjvm.so` заменен многоточием («...»):

```
# tplist -p 6820
/.../libjvm.so hotspot:class__loaded
/.../libjvm.so hotspot:class__unloaded
/.../libjvm.so hs_private:cms__initmark__begin
/.../libjvm.so hs_private:cms__initmark__end
/.../libjvm.so hs_private:cms__remark__begin
/.../libjvm.so hs_private:cms__remark__end
/.../libjvm.so hotspot:method__compile__begin
/.../libjvm.so hotspot:method__compile__end
/.../libjvm.so hotspot:gc__begin
/.../libjvm.so hotspot:gc__end
[...]
/.../libjvm.so hotspot_jni:NewObjectArray__entry
/.../libjvm.so hotspot_jni:NewObjectArray__return
/.../libjvm.so hotspot_jni:NewDirectByteBuffer__entry
/.../libjvm.so hotspot_jni:NewDirectByteBuffer__return
[...]
/.../libjvm.so hs_private:safepoint__begin
/.../libjvm.so hs_private:safepoint__end
/.../libjvm.so hotspot:object__alloc
/.../libjvm.so hotspot:method__entry
/.../libjvm.so hotspot:method__return
/.../libjvm.so hotspot:monitor__waited
/.../libjvm.so hotspot:monitor__wait
/.../libjvm.so hotspot:thread__stop
/.../libjvm.so hotspot:thread__start
/.../libjvm.so hotspot:vm__init__begin
/.../libjvm.so hotspot:vm__init__end
[...]
```

Зонды сосредоточены в библиотеках `hotspot` и `hotspot_jni`. В примере выше можно видеть зонды для трассировки загрузки классов, сборки мусора, точек безопасности, создания объектов, вызовов методов, потоков выполнения и т. д. Двойные подчеркивания использовались в именах зондов, чтобы была возможность ссылаться на них в `DTrace` с помощью одного тире и избежать проблемы с добавлением знаков «минус» в код.

В этом примере `tplist(8)` используется для анализа процесса, однако с его помощью можно было бы исследовать файл `libjvm.so`. Аналогично можно использовать `readelf(1)`, чтобы увидеть зонды `USDT` в разделе примечаний `ELF (-n)` в двоичном файле:

```
# readelf -n ../jdk/lib/server/libjvm.so

Displaying notes found in: .note.gnu.build-id
Owner      Data size  Description
GNU        0x00000014 NT_GNU_BUILD_ID (unique build ID bitstring)
Build ID: 264bc78da04c17524718c76066c6b535dcc380f2

Displaying notes found in: .note.stapsdt
Owner      Data size  Description
stapsdt    0x00000050 NT_STAPSDT (SystemTap probe descriptors)
Provider: hotspot
Name: class__loaded
Location: 0x00000000005d18a1, Base: 0x00000000010bdf68, Semaphore:
0x0000000000000000
Arguments: 8@%rdx -4@%eax 8@152(%rdi) 1@%sil
stapsdt    0x00000050 NT_STAPSDT (SystemTap probe descriptors)
Provider: hotspot
Name: class__unloaded
Location: 0x00000000005d1cba, Base: 0x00000000010bdf68, Semaphore:
0x0000000000000000
Arguments: 8@%rdx -4@%eax 8@152(%r12) 1@$0
[...]
```

Использование зондов *USDT* в *Java*

Эти зонды доступны в `BCC` и в `bpfttrace`. Их назначение и аргументы описаны в руководстве «*Java Virtual Machine Guide*» [138]. Вот пример использования `BCC`-инструмента `trace(8)` для трассировки зонда `gc-begin` и проверки его первого аргумента, логического флага, обозначающего тип сборки мусора — полная (1) или частичная (0):

```
# trace -T -p $(pidof java) 'u:../libjvm.so:gc__begin "%d", arg1'
TIME     PID      TID      COMM      FUNC      -
09:30:34 11889    11900    VM Thread  gc__begin  0
09:30:34 11889    11900    VM Thread  gc__begin  0
09:30:34 11889    11900    VM Thread  gc__begin  0
09:30:38 11889    11900    VM Thread  gc__begin  1
```

Здесь можно видеть три частичные сборки мусора в 9:30:34 и одну полную в 9:30:38. Обратите внимание, что в руководстве «*Java Virtual Machine Guide*» этот аргумент

обозначен как `args[0]`, однако `trace(8)` начинает нумерацию с 1, поэтому в команде этот аргумент обозначен как `arg1`.

Ниже приведен пример извлечения строкового аргумента: зонд `method__compile__begin` получает имя компилятора, имя класса и имя метода в первом, третьем и пятом аргументах. Здесь с помощью `trace(8)` выводится имя метода:

```
# trace -p $(pidof java) 'u:/.../libjvm.so:method__compile__begin "%s", arg5'
PID      TID      COMM      FUNC      -
12600    12617    C1 CompilerThre method__compile__begin getLocationOnScreen
12600    12617    C1 CompilerThre method__compile__begin getAbsoluteX
12600    12617    C1 CompilerThre method__compile__begin getAbsoluteY
12600    12617    C1 CompilerThre method__compile__begin currentSegmentD
12600    12617    C1 CompilerThre method__compile__begin next
12600    12617    C1 CompilerThre method__compile__begin drawJoin
12600    12616    C2 CompilerThre method__compile__begin needsSyncData
12600    12617    C1 CompilerThre method__compile__begin getMouseInfoPeer
12600    12617    C1 CompilerThre method__compile__begin fillPointWithCoords
12600    12616    C2 CompilerThre method__compile__begin isHeldExclusively
12600    12617    C1 CompilerThre method__compile__begin updateChildGraphicsData
Traceback (most recent call last):
  File "_ctypes/callbacks.c", line 315, in 'calling callback function'
  File "/usr/local/lib/python2.7/dist-packages/bcc/table", line 572, in raw_cb_
    callback(cpu, data, size)
  File "/home/bgregg/Build/bcc/tools/trace", line 567, in print_event
    self._display_function(), msg))
UnicodeDecodeError: 'ascii' codec can't decode byte 0xff in position 10: ordinal not
in range(128)
12600    12616    C2 CompilerThre method__compile__begin getShowingSubPanel%
[...]
```

В первых 11 строках имя метода выводится в последнем столбце. Далее следует сообщение об ошибке в интерпретаторе Python, возникшей при попытке декодировать байт как символ ASCII. Эта проблема объясняется в руководстве «Java Virtual Machine Guide»: строки не завершаются символом NULL, и их длины передаются в дополнительных аргументах. Чтобы избежать подобных ошибок, программа BPF должна использовать длину строки, получая ее из аргумента зонда.

Вот какой результат получается, если переключиться на `bpfftrace`, который может использовать аргумент длины в своей встроенной функции `str()`:

```
# bpfftrace -p $(pgrep -n java) -e 'U:/.../libjvm.so:method__compile__begin
{ printf("compiling: %s\n", str(arg4, arg5)); }'
Attaching 1 probe...
compiling: getDisplayedMnemonicIndex
compiling: getMinimumSize
compiling: getBaseline
compiling: fillParallelogram
compiling: preConcatenate
compiling: last
compiling: nextTile
compiling: next
[...]
```

Ошибки исчезли, и теперь строки выводятся с правильной длиной. Любая программа для ВСС или bpftrace, применяющая эти зонды, должна использовать аргумент длины, как показано в примере.

Предваряя следующий раздел, рассмотрим еще один пример: он подсчитывает количество вызовов всех зондов USDT, имена которых начинаются с «method»:

```
# funccount -p $(pidof java) 'u:/.../libjvm.so:method*'
Tracing 4 functions for "u:/.../libjvm.so:method*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
method__compile__begin              2056
method__compile__end                 2056
Detaching...
```

За время, пока выполнялась трассировка, по 2056 раз были вызваны зонды `method__compile__begin` и `method__compile__end`. Обратите внимание, что зонды `method__entry` и `method__return` не попали в результаты. Причина в том, что они — часть расширенного набора зондов USDT, о котором мы поговорим ниже.

Расширенный набор зондов USDT в Java

Некоторые зонды USDT в JVM выключены по умолчанию. К ним относятся зонды входа и выхода из методов, создания объектов и мониторинга Java. Это связано с тем, что они соответствуют очень частым событиям и их выполнение может приводить к большому оверхеду и значительному снижению производительности, иногда более 10%. Это слишком высокая плата всего лишь за их доступность, даже когда они не используются! Включение и применение этих зондов может замедлить работу Java в 10 раз и более (10x). Чтобы пользователи Java не платили такую цену за возможности, которые не используют, эти зонды по умолчанию выключены. Однако их можно включить, запустив Java с параметром: `-XX: +ExtendedDTraceProbes`.

В примере ниже мы видим запуск Java-игры `freecol` с включенным параметром `ExtendedDTraceProbes`, он подсчитывает вызовы зондов USDT, имена которых начинаются с «method», как и в предыдущем примере:

```
# funccount -p $(pidof java) 'u:/.../libjvm.so:method*'
Tracing 4 functions for "u:/.../libjvm.so:method*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
method__compile__begin              357
method__compile__end                 357
method__return                       26762077
method__entry                        26762245
Detaching...
```

За время трассировки зонды `method__entry` и `method__return` были вызваны по 26 миллионов раз. Кроме того, сильно пострадала производительность игры: обработка любого ввода занимала около 3 секунд. Чтобы вы понимали, насколько упала производительность игры `freecol`, приведу несколько цифр: время до появления

заставки по умолчанию составляло 2 секунды, а после инструментации этих зондов — 22 секунды, производительность упала более чем в 10 раз.

Эти зонды, соответствующие частым событиям, могут пригодиться для устранения неполадок в ПО в тестовой среде, но никак не для мониторинга производственных рабочих нагрузок.

Теперь, познакомившись с необходимыми основами (libjvm, символы Java, трассировка стека Java и зонды USDT в Java), рассмотрим различные инструменты BPF для наблюдаемости Java.

12.3.7. profile

С инструментом profile(8) для BCC мы познакомились в главе 6. Есть множество разнообразных профилировщиков для Java. Преимущество profile(8) заключается в его эффективности: он подсчитывает трассировки стека в пространстве ядра и отображает всех потребителей процессора в пространствах пользователя и ядра. profile(8) позволяет также увидеть время, потраченное на выполнение кода в низкоуровневых библиотеках (например, libc), libjvm, методах Java и в ядре.

Предварительные требования Java

Чтобы profile(8) видел полный стек, виртуальная машина Java должна быть запущена с параметром -XX: +PreserveFramePointer. Также с помощью perf-map-agent нужно создать вспомогательный файл символов, он будет использоваться инструментом profile(8) (см. раздел 12.3.4). Таблица символов нужна для трансляции фреймов стека в libjvm.so. Эти требования обсуждались в предыдущих разделах.

Флейм-график потребления процессора

Рассмотрим пример использования profile(8) с Java для создания флейм-графика потребления процессора в смешанном режиме.

Java-программа, freecol, запускалась с параметром -XX: +PreserveFramePointer и таблицей символов из libjvm. Утилита jmaps, представленная выше в этой главе, запускалась непосредственно перед profile(8), чтобы снизить вероятность устаревания символов. Профилирование продолжалось 10 секунд с частотой по умолчанию (99 Гц), с включенными аннотациями ядра (-a), для PID 16914 (-p), а результаты выводились в свернутом формате, пригодном для построения флейм-графиков (-f):

```
# jmaps; profile -afp 16914 10 > out.profile01.txt
Fetching maps for all java processes...
Mapping PID 16914 (user bgregg):
wc(1): 9078 28222 572219 /tmp/perf-16914.map
# wc out.profile01.txt
  215   3347 153742 out.profile01.txt
# cat out.profile01.txt
AWT-EventQueue-;start_thread;thread_native_entry(Thread*);Thread::call... 1
[...]
```


времени было потрачено компилятором C2, которому соответствует широкая башня (вертикальный столбец прямоугольников) в середине фреймов, соответствующих коду на C++, и только 29% времени потрачено игрой freecol, о чем говорят башни, соответствующие коду на Java.

Щелкнув на башне Java слева, можно увеличить фреймы Java, как показано на рис. 12.3.

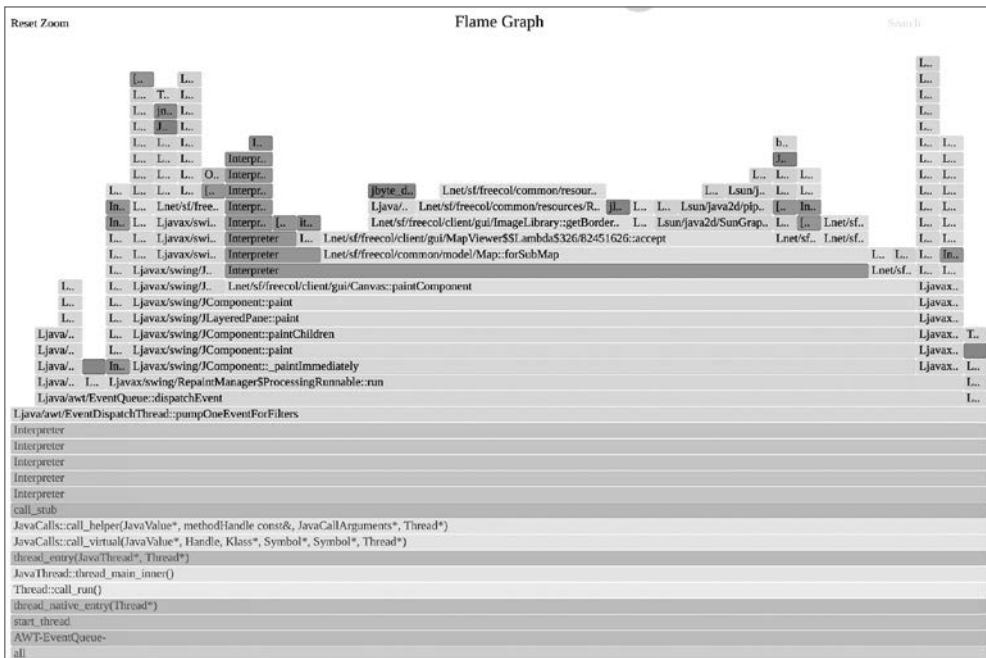


Рис. 12.3. Флейм-график потребления CPU в увеличенном масштабе

Теперь можно детальнее исследовать особенности выполнения игры freecol и ее методов. Большую часть процессорного времени потрачено методами рисования. Какими именно — можно увидеть на верхнем крае флейм-графика.

Если бы вы были заинтересованы в улучшении производительности freecol, то этот флейм-график потребления процессора сразу же подсказал бы вам две цели: просмотреть настройки JVM, которые заставят компилятор C2 потреблять меньше процессорного времени¹, и детально изучить методы рисования в исходном коде freecol, чтобы попробовать найти более эффективные алгоритмы.

¹ В число параметров настройки компилятора входят: -XX:CompileThreshold, -XX:MaxInlineSize, -XX:InlineSmallCode и -XX:FreqInlineSize. Интересные результаты также можно получить с помощью параметра -Xcomp, управляющего предварительной компиляцией.

Для длинных профилей (например, продолжительностью более 2 минут) интервал между получением таблицы символов и окончанием сбора трассировок стека может оказаться настолько большим, что компилятор C2 к тому времени переместит некоторые методы, и таблица символов окажется неточной. Это можно заметить по путям в коде, не имеющим смысла, когда некоторые фреймы будут транслированы неправильно. Гораздо более распространенная проблема с неожиданными путями в коде — это встраивание (inlining).

Встраивание

Поскольку это визуализация трассировки стека кода, выполняемого процессором, она отображает методы Java уже после встраивания. Встраивание в JVM может быть весьма активным и покрывать до двух фреймов из каждого трех. Это может усложнить анализ флейм-графика, так как будет создаваться ощущение, что методы напрямую вызывают другие методы, которых нет в исходном коде.

Для проблемы встраивания есть решение: ПО perf-map-agent поддерживает выгрузку таблицы символов, которая включает все встроенные символы. В jmaps за поддержку этой возможности отвечает параметр `-u`:

```
# jmaps -u; profile -afp 16914 10 > out.profile03.txt
Fetching maps for all java processes...
Mapping PID 16914 (user bgregg):
wc(1):      75467    227393 11443144 /tmp/perf-16914.map
```

Это приводит к существенному увеличению количества символов, с 9078, как было показано выше, до более чем 75 000. (Я снова запустил jmaps без параметра `-u` и получил прежнее количество символов, что-то около 9000.)

На рис. 12.4 показан флейм-график, полученный с информацией о встроенных символах.

Башня в стеке freecol теперь намного выше, потому что включает фреймы, соответствующие встроенным методам.

Включение фреймов встраиваемых методов замедляет этап выполнения утилиты jmaps, потому что теперь она должна вывести в файл больше символов, а также этап создания флейм-графика из-за необходимости анализа и включения дополнительных фреймов. На практике эта необходимость возникает довольно редко. Чаще для решения проблем достаточно флейм-графика без фреймов встроенных методов, потому что он и так наглядно показывает общий поток выполнения, просто следует помнить, что некоторые методы не видны.

bpfttrace

Возможности profile(8) также можно реализовать средствами bpfttrace, что дает свое преимущество: инструмент jmaps можно запустить в блоке END с помощью функции system(). Например, в предыдущем разделе был показан следующий однострочный сценарий:

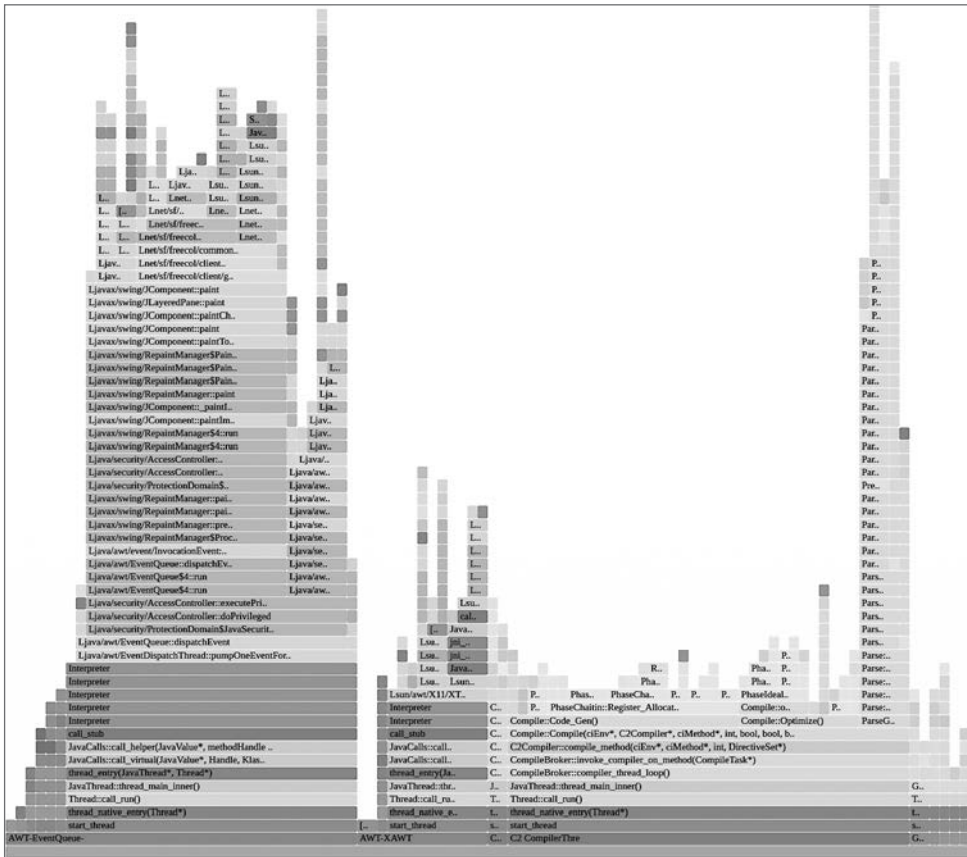


Рис. 12.4. Флейм-график потребления CPU с информацией о встроенных символах

```
bpfttrace --unsafe -e 'profile:hz:99 /pid == 4663/ { @[ustack] = count(); } END
{ system("jmaps"); }'
```

Он отбирает трассировки стека в пространстве пользователя для PID 4663 с частотой 99 Гц на всех процессорах, на которых выполняется этот процесс. В него можно добавить отбор трассировок стека в пространстве ядра по именам процессов, создав карту `@[kstack, ustack, comm]`.

12.3.8. offcputime

С инструментом `offcputime(8)` для ВСС мы познакомились в главе 6. Он отбирает трассировки стеков по событиям блокировки CPU (переключения контекста планировщика) и суммирует время, в течение которого процесс находился в состоянии ожидания. Необходимые условия для анализа работы кода на Java с помощью `offcputime(8)` приводятся в разделе 12.3.7.

Пример использования `offcputime(8)` для анализа Java-игры `freecol`:

```
# jmaps; offcputime -p 16914 10
```

```
Fetching maps for all java processes...
```

```
Mapping PID 16914 (user bgregg):
```

```
wc(1):  9863  30589 623898 /tmp/perf-16914.map
```

```
Tracing off-CPU time (us) of PID 16914 by user + kernel stack for 10 secs.
```

```
^C
```

```
[...]
```

```
finish_task_switch
schedule
futex_wait_queue_me
futex_wait
do_futex
Sys_futex
do_syscall_64
entry_SYSCALL_64_after_hwframe
__lll_lock_wait
SafepointSynchronize::block(JavaThread*, bool)
SafepointMechanism::block_if_requested_slow(JavaThread*)
JavaThread::check_safepoint_and_suspend_for_native_trans(JavaThread*)
JavaThread::check_special_condition_for_native_trans(JavaThread*)
Lsun/awt/X11/XlibWrapper::XEventsQueued
Lsun/awt/X11/XToolkit::run
Interpreter
Interpreter
call_stub
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
JavaCalls::call_virtual(JavaValue*, Handle, Klass*, Symbol*, Symbol*, Thread*)
thread_entry(JavaThread*, Thread*)
JavaThread::thread_main_inner()
Thread::call_run()
thread_native_entry(Thread*)
start_thread
-                               AWT-XAWT (16944)
5171
```

```
[...]
```

```
finish_task_switch
schedule
io_schedule
bit_wait_io
__wait_on_bit
out_of_line_wait_on_bit
__wait_on_buffer
ext4_find_entry
ext4_unlink
vfs_unlink
do_unlinkat
sys_unlink
```

```

do_syscall_64
entry_SYSCALL_64_after_hwframe
__GI_unlink
Ljava/io/UnixFileSystem::~delete0
Ljava/io/File::~delete
Interpreter
Interpreter
Interpreter
Lnet/sf/freecol/client/control/InGameInputHandler::~handle
Interpreter
Lnet/sf/freecol/client/control/InGameInputHandler::~handle
Lnet/sf/freecol/common/networking/Connection::~handle
Interpreter
call_stub
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Th...
JavaCalls::call_virtual(JavaValue*, Handle, Klass*, Symbol*, Symbol*, Thread*)
thread_entry(JavaThread*, Thread*)
JavaThread::thread_main_inner()
Thread::call_run()
thread_native_entry(Thread*)
start_thread
-                               FreeColClient:b (8168)
7679

```

[...]

```

finish_task_switch
schedule
futex_wait_queue_me
futex_wait
do_futex
Sys_futex
do_syscall_64
entry_SYSCALL_64_after_hwframe
pthread_cond_timedwait@@GLIBC_2.3.2
__pthread_cond_timedwait
os::PlatformEvent::park(long) [clone .part.12]
Monitor::IWait(Thread*, long)
Monitor::wait(bool, long, bool)
WatcherThread::sleep() const
WatcherThread::run()
thread_native_entry(Thread*)
start_thread
__clone
-                               VM Periodic Tas (22029)
9970501

```

Вывод был усечен, так как занимает много места. Сюда я включил только наиболее интересные трассировки стеков.

В первой трассировке стека видно, что в общей сложности процесс Java приостанавливался на 5.1 миллисекунды (5171 микросекунду) в безопасном состоянии, которое обслуживается с помощью фьютекса в ядре. Это общее время, то есть 5.1 миллисекунды могут состоять из нескольких событий блокировки.

Последняя трассировка стека показывает, что приложение на Java простаивало в `pthread_cond_timedwait()` в течение почти всех 10 секунд, пока выполнялась трассировка: это `WatcherThread` с именем потока «VM Periodic Tas» (имя усечено и поэтому в результатах отображается без «k»), ожидающий, когда появится работа для него. Для приложений некоторых типов, использующих большое число потоков, ожидающих работы, в выводе `offcputime(8)` могут преобладать подобные стеки с большим временем ожидания. Пропускайте их и ищите стеки, имеющие значение: события ожидания во время обработки запросов приложения.

Но меня особенно удивила вторая трассировка стека: она показывает, что приложение Java было заблокировано в системном вызове `unlink(2)`, отвечающем за удаление файла, что в итоге привело к блокировке дискового ввода/вывода (`io_schedule()` и т. д.). Мне стало интересно, какие файлы удаляет `freecol` во время игры. Я запустил однострочник `bpfftrace`, фиксирующий обращения к `unlink(2)` и отображающий пути к удаляемым файлам:

```
# bpfftrace -e 't:syscalls:sys_enter_unlink /pid == 16914/ { printf("%s\n",
    str(args->pathname)); }'
Attaching 1 probe...
/home/bggregg/.local/share/freecol/save/autosave/Autosave-before
/home/bggregg/.local/share/freecol/save/autosave/Autosave-before
[...]
```

`freecol` удаляет файлы, в которые автоматически сохраняет состояние игры.

Стеку *libpthread*

Поскольку эта проблема может встречаться часто, приведу окончательный стек с библиотекой `libpthread`, устанавливаемой по умолчанию:

```
finish_task_switch
schedule
futex_wait_queue_me
futex_wait
do_futex
Sys_futex
do_syscall_64
entry_SYSCALL_64_after_hwframe
pthread_cond_timedwait
- VM Periodic Tas (16936)
9934452
```

Стек заканчивается вызовом `pthread_cond_timedwait()`. На момент написания книги библиотека `libpthread`, включаемая во многие дистрибутивы Linux, компилируется с флагом оптимизации компилятора `-fomit-frame-pointer`, препятствующим обходу стека по указателям фрейма. В предыдущем примере использовалась моя собственная версия `libpthread`, скомпилированная с флагом `-fno-omit-frame-pointer`. Подробнее об этом идет речь в разделе 2.4.

Флейм-графики простоев

Вывод `offcpuetime(8)` в предыдущем разделе получился длиной в несколько сотен страниц. Чтобы упростить задачу, можно попробовать создать флейм-график простоев. Вот пример использования программы `FlameGraph` [37]:

```
# jmaps; offcpuetime -fp 16914 10 > out.offcpu01.txt
Fetching maps for all java processes...
Mapping PID 16914 (user bgregg):
wc(1): 12015 37080 768710 /tmp/perf-16914.map
# flamegraph.pl --color=java --bgcolor=blue --hash --countname=us --width=800 \
  --title="Off-CPU Time Flame Graph" < out.offcpu01.txt > out.offcpu01.svg
```

Получившийся график изображен на рис. 12.5.

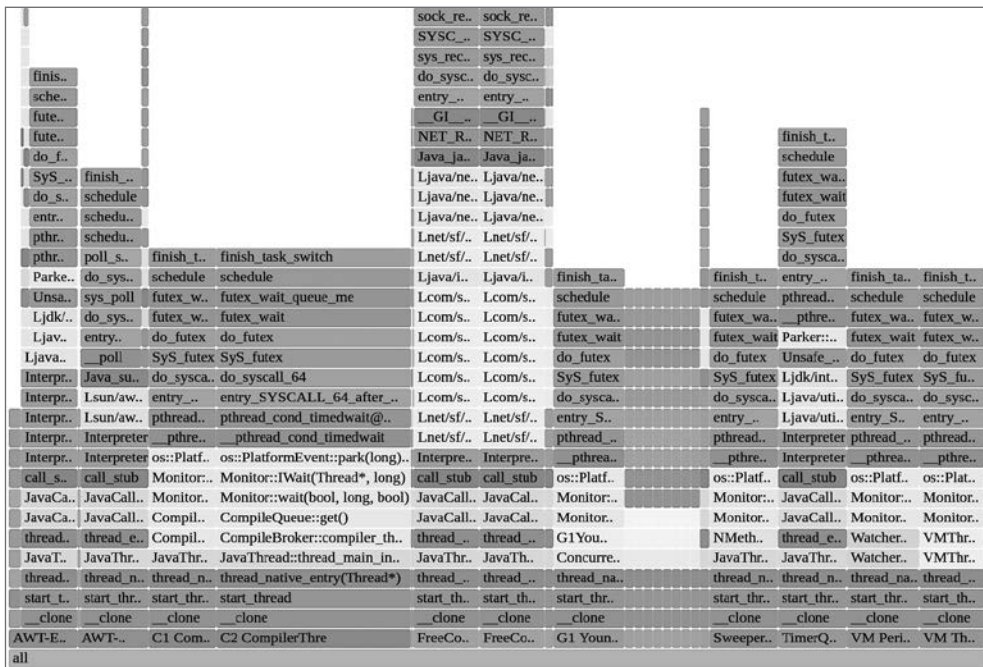


Рис. 12.5. Флейм-график простоев

Верхняя часть этого флейм-графика усечена. Ширина каждого фрейма отражает время ожидания вне процессора. Поскольку `offcpuetime(8)` показывает трассировки стека с суммарным временем ожидания в микросекундах, программе `flamegraph.pl` был передан параметр `--countname = us`, который влияет на отображение информации при наведении курсора. Кроме того, я изменил цвет фона на синий, чтобы он визуально напоминал, что график отражает трассировки стеков с простоями. (На флейм-графиках потребления процессора используется фон желтого цвета.)

На этом флейм-графике преобладают потоки, ожидающие событий. Поскольку в первый фрейм стека включается имя потока, все потоки группируются по именам, образуя башни. Каждая башня на этом флейм-графике соответствует ожидающему потоку.

Но меня интересуют не потоки, ожидающие событий, а потоки, простаивающие во время обработки запросов приложения. Здесь приложение называется `freecol`, поэтому с помощью функции поиска в графике по слову «freecol» эти фреймы были выделены пурпурным цветом (рис. 12.6).

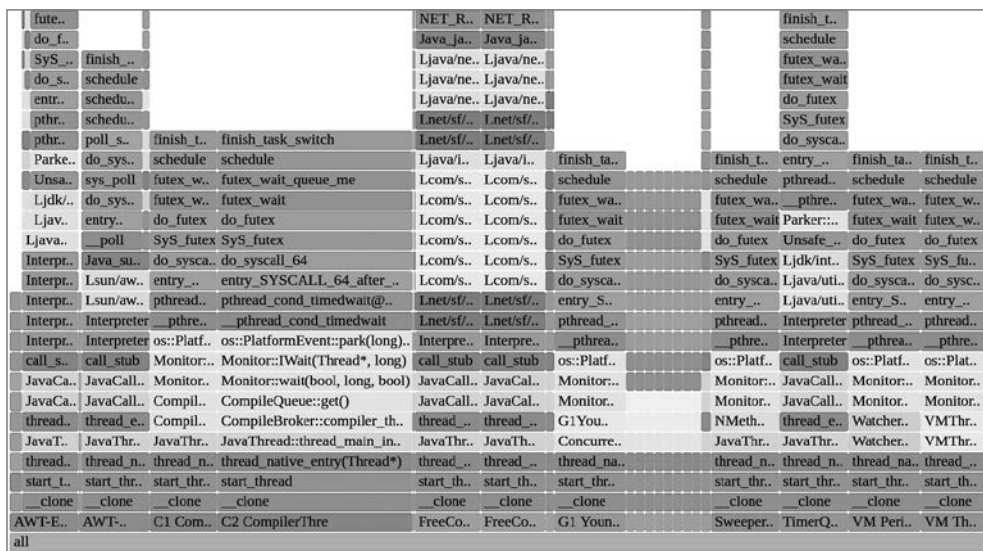


Рис. 12.6. Флейм-график простоев с результатами поиска по имени приложения

На рис. 12.7 показан тот же график после щелчка мышью на узкой третьей башне для увеличения масштаба.

График на рис. 12.7 показывает пути в коде freecol, приведшие к ожиданию, которые могут служить целями для оптимизации. Многие из этих фреймов подписаны как «Interpreter», потому что JVM еще не успела выполнить соответствующие методы достаточное количество раз, чтобы достичь порогового значения `CompileThreshold`.

Иногда пути в коде приложения могут быть настолько узкими из-за других ожидающих потоков, что исключаются из флейм-графика. Одно из возможных решений этой проблемы — использовать `grep(1)` в командной строке, чтобы включить только интересующие стеки, например, содержащие имя приложения «freecol»:

```
# grep freecol out.offcpu01.txt | flamegraph.pl ... > out.offcpu01.svg
```

Это одно из достоинств формата представления трассировок стека в свернутом виде: им легко манипулировать перед генерацией флейм-графика.

pthread_...	entry_SYSCALL_64_after_hwframe	entry_SYSCALL_64_after_hwframe
pthread_...	pthread_cond_wait	pthread_cond_wait
os::Platf...	pthread_cond_wait@@GLIBC_2.3.2	pthread_cond_wait@@GLIBC_2.3.2
ObjectM...	os::PlatformEvent::park()	os::PlatformEvent::park()
ObjectSy...	ObjectMonitor::wait(long, bool, Thread*)	ObjectMonitor::wait(long, bool, Thread*)
JVM_Mo...	ObjectSynchronizer::wait(Handle, long, Thread*)	ObjectSynchronizer::wait(Handle, long, T...
Ljava/lan...	JVM_MonitorWait	JVM_MonitorWait
[unknown]	Ljava/lang/Object::wait	Ljava/lang/Object::wait
Interpreter	[unknown]	[unknown]
Interpreter	Interpreter	Interpreter
Interpreter	Interpreter	Interpreter
Lnet/sf/fr...	Interpreter	Interpreter
Lnet/sf/fr...	Lnet/sf/freecol/server/model/ServerPlayer::askElement	Lnet/sf/freecol/server/model/ServerPlayer...
Interpreter	Lnet/sf/freecol/server/model/ServerGame::sendToList	Lnet/sf/freecol/server/model/ServerPlayer...
Interpreter		
Interpreter		
Lnet/sf/freecol/common/networking/CurrentPlayerNetworkRequestHandler::handle		
Lnet/sf/freecol/server/control/InputHandler::handle		
Lnet/sf/freecol/server/networking/DummyConnection::ask		
Lnet/sf/freecol/server/ai/AIMessage::ask		
Lnet/sf/freecol/server/ai/AIMessage::askHandling		
Lnet/sf/freecol/server/ai/AIMessage::sendMessage		
Interpreter		
Interpreter		
Interpreter		
call_stub		
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Thread*)		
JavaCalls::call_virtual(JavaValue*, Handle, Klass*, Symbol*, Symbol*, Thread*)		
hread_entry(JavaThread*, Thread*)		
JavaThread::thread main inner()		
hread_native_entry(Thread*)		
start_thread		
__clone		
FreeColServer::A		
all		

Рис. 12.7. Флейм-график простоев в увеличенном масштабе

12.3.9. stackcount

С инструментом `stackcount(8)` для ВСС мы познакомились в главе 4. Он может отбирать стеки для любых событий, а значит, может отображать пути кода в `libjvm` и Java-методах, которые привели к событию. Необходимые условия для анализа работы кода на Java с помощью `stackcount(8)` приведены в разделе 12.3.7.

Вот пример использования `stackcount(8)` для анализа отказов страниц в пространстве пользователя, которые могут служить признаком увеличения потребления оперативной памяти:

```
# stackcount -p 16914 t:exceptions:page_fault_user
Tracing 1 functions for "t:exceptions:page_fault_user"... Hit Ctrl-C to end.
^C
```

[...]

```
do_page_fault
page_fault
Interpreter
Lnet/sf/freecol/server/control/ChangeSet$MoveChange::consequences
[unknown]
```

```

[unknown]
Lnet/sf/freecol/server/control/InGameController::~move
Lnet/sf/freecol/common/networking/MoveMessage::~handle
Lnet/sf/freecol/server/control/InGameInputHandler$37::~handle
Lnet/sf/freecol/common/networking/CurrentPlayerNetworkRequestHandler::~handle
[unknown]
Lnet/sf/freecol/server/ai/AIMessage::~ask
Lnet/sf/freecol/server/ai/AIMessage::~askHandling
Lnet/sf/freecol/server/ai/AIUnit::~move
Lnet/sf/freecol/server/ai/mission/Mission::~moveRandomly
Lnet/sf/freecol/server/ai/mission/UnitWanderHostileMission::~doMission
Ljava/awt/Container;::isParentOf
[unknown]
Lcom/sun/org/apache/xerces/internal/impl/XMLEntityScanner::~reset
call_stub
JavaCalls::call_helper(JavaValue*, methodHandle const&, JavaCallArguments*, Thre...
JavaCalls::call_virtual(JavaValue*, Handle, Klass*, Symbol*, Symbol*, Thread*)
thread_entry(JavaThread*, Thread*)
JavaThread::thread_main_inner()
Thread::call_run()
thread_native_entry(Thread*)
start_thread
4

```

[...]

```

do_page_fault
page_fault
__memset_avx2_erms
PhaseChaitin::Register_Allocate()
Compile::Code_Gen()
Compile::Compile(ciEnv*, C2Compiler*, ciMethod*, int, bool, bool, bool, Directiv...
C2Compiler::compile_method(ciEnv*, ciMethod*, int, DirectiveSet*)
CompileBroker::invoke_compiler_on_method(CompileTask*)
CompileBroker::compiler_thread_loop()
JavaThread::thread_main_inner()
Thread::call_run()
thread_native_entry(Thread*)
start_thread
414

```

Здесь я привел только две трассировки стека, хотя в действительности их было намного больше. Первая показывает путь в коде искусственного интеллекта в freecol, вторая — в коде компилятора JVM C2.

Флейм-график отказов страниц

Чтобы упростить анализ, построим на основе этих результатов флейм-график. Вот пример использования программы FlameGraph [37]:

```

# jmaps; stackcount -p 16914 t:exceptions:page_fault_user > out.faults01.txt
Fetching maps for all java processes...
Mapping PID 16914 (user bgregg):

```

```

wc(1): 12015 37080 768710 /tmp/perf-16914.map
# stackcollapse.pl < out.faults01.txt | flamegraph.pl --width=800 \
  --color=java --bgcolor=green --title="Page Fault Flame Graph" \
  --countname=pages > out.faults01.svg

```

В результате получился усеченный график, изображенный на рис. 12.8.

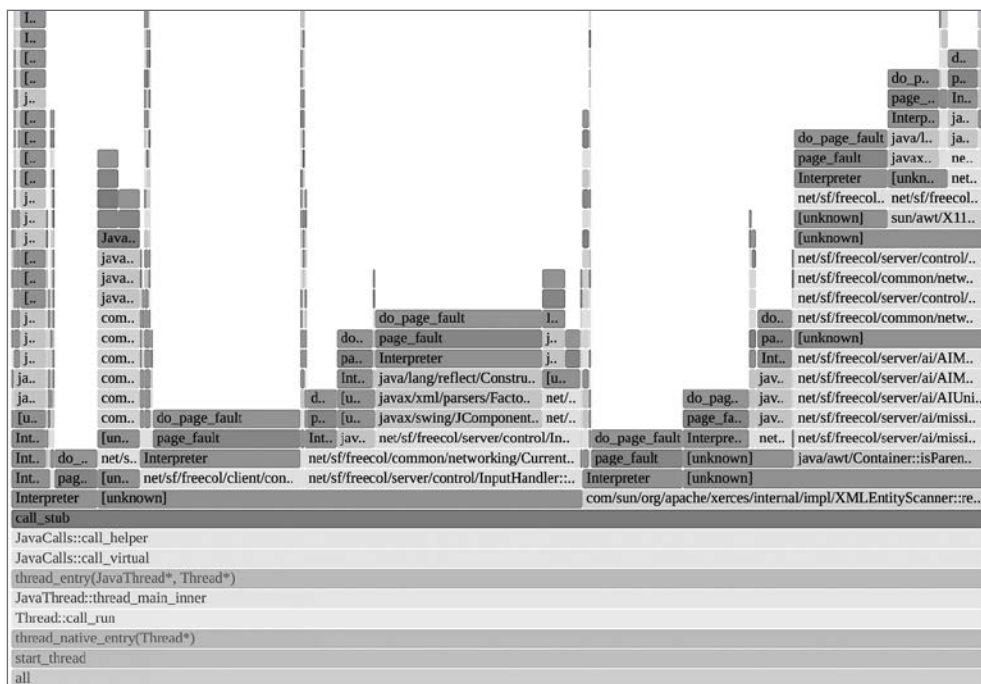


Рис. 12.8. Флейм-график отказов страниц

Для визуального напоминания, что график связан с памятью, использовался зеленый цвет фона. На этом скриншоте я увеличил масштаб, чтобы исследовать пути кода в приложении freesol. Это позволяет определить, какие пути кода в приложении приводят к увеличению объема потребляемой памяти, и количественно оценить этот объем (по ширине).

bpftrace

Возможности stackcount(8) также можно реализовать средствами bpftrace в виде однострочного сценария, например:

```

# bpftrace --unsafe -e 't:exceptions:page_fault_user /pid == 16914/ {
  @[kstack, ustack, comm] = count(); } END { system("jmaps"); }'
Attaching 1 probe...

```

```

^C
[...]

@[
  do_page_fault+204
  page_fault+69
,
  0x7fa369bbef2d
  PhaseChaitin::Register_Allocate()+930
  Compile::Code_Gen()+650
  Compile::Compile(ciEnv*, C2Compiler*, ciMethod*, int, bool, bool, bool, Direct...
  C2Compiler::compile_method(ciEnv*, ciMethod*, int, DirectiveSet*)+188
  CompileBroker::invoke_compiler_on_method(CompileTask*)+1016
  CompileBroker::compiler_thread_loop()+1352
  JavaThread::thread_main_inner()+446
  Thread::call_run()+376
  thread_native_entry(Thread*)+238
  start_thread+219
, C2 CompilerThre]: 3
[...]

```

Запуск утилиты `jmaps` для получения таблицы символов Java-методов перемещен в блок END, поэтому она запускается непосредственно перед выводом стеков.

12.3.10. javastat

`javastat(8)`¹ — это инструмент для ВСС, предоставляющий высокоуровневую статистику о выполнении кода на Java и JVM. При запуске без параметра `-C` он обновляет экран подобно `top(1)`. Пример трассировки игры `freecol` с помощью `javastat(8)`:

```

# javastat -C
Tracing... Output every 1 secs. Hit Ctrl-C to end

14:16:56 loadavg: 0.57 3.66 3.93 2/3152 32738

PID    CMDLINE                METHOD/s    GC/s    OBJNEW/s    CLOAD/s    EXC/s    THR/s
32447   /home/bggregg/Build/o  0          0       0           0          169     0

14:16:58 loadavg: 0.57 3.66 3.93 8/3157 32744

PID    CMDLINE                METHOD/s    GC/s    OBJNEW/s    CLOAD/s    EXC/s    THR/s
32447   /home/bggregg/Build/o  0          1       0           730        522     6

14:16:59 loadavg: 0.69 3.64 3.92 2/3155 32747

```

¹ Немного истории: этот инструмент создан Сашей Гольдштейном 26 октября 2016 года как обертка для его инструмента `ustat(8)`. Я создал аналогичный инструмент для DTrace под названием `j_stat.d` 9 сентября 2007 года, чтобы показать работу новых зондов в DTraceToolkit.

PID	CMDLINE	METHOD/s	GC/s	OBJNEW/s	CLOAD/s	EXC/s	THR/s
32447	/home/bgregg/Build/o	0	2	0	8	484	1
[...]							

Значения столбцов:

- **PID:** идентификатор процесса;
- **CMDLINE:** команда, запустившая процесс. В этом примере путь к моей собственной сборке JDK усечен;
- **METHOD/s:** количество вызовов методов в секунду;
- **GC/s:** количество событий сборки мусора в секунду;
- **OBJNEW/s:** количество событий создания новых объектов в секунду;
- **CLOAD/s:** количество событий загрузки классов в секунду;
- **EXC/s:** количество исключений в секунду;
- **THR/s:** количество событий создания новых потоков выполнения в секунду.

Этот инструмент использует зонды USDT для Java. Столбцы METHOD/s и OBJNEW/s будут содержать нулевые значения, если не использовать параметр `-XX: +ExtendedDTraceProbes`, который активирует зонды из дополнительного набора, несущие высокий оверхед. Как отмечалось выше, если включить и инструментировать эти зонды, скорость работы приложения упадет в 10 раз и более.

Порядок использования:

```
javastat [options] [interval [count]]
```

Параметры options:

- **-C:** не очищать экран.

`javastat(8)` — это обертка для инструмента `ustat(8)` в каталоге `tools/lib` с исходным кодом BCC, который поддерживает несколько языков.

12.3.11. javathreads

`javathreads(8)`¹ — это инструмент для `bpftrace`, регистрирующий события запуска и остановки потоков выполнения. Пример вывода этого инструмента, соответствующий моменту запуска игры `freecol`:

```
# javathreads.bt
Attaching 3 probes...
```

¹ Немного истории: я написал его для этой книги 19 февраля 2019 года.

```

TIME                PID/TID  -- THREAD
14:15:00    3892/3904 => Reference Handler
14:15:00    3892/3905 => Finalizer
14:15:00    3892/3906 => Signal Dispatcher
14:15:00    3892/3907 => C2 CompilerThread0
14:15:00    3892/3908 => C1 CompilerThread0
14:15:00    3892/3909 => Sweeper thread
14:15:00    3892/3910 => Common-Cleaner
14:15:01    3892/3911 => C2 CompilerThread1
14:15:01    3892/3912 => Service Thread
14:15:01    3892/3911 <= C2 CompilerThread1
14:15:01    3892/3917 => Java2D Disposer
14:15:01    3892/3918 => AWT-XAWT
14:15:02    3892/3925 => AWT-Shutdown
14:15:02    3892/3926 => AWT-EventQueue-0
14:15:02    3892/3934 => C2 CompilerThread1
14:15:02    3892/3935 => FreeColClient:-Resource loader
14:15:02    3892/3937 => FreeColClient:Worker
14:15:02    3892/3935 <= FreeColClient:-Resource loader
14:15:02    3892/3938 => FreeColClient:-Resource loader
14:15:02    3892/3939 => Image Fetcher 0
14:15:03    3892/3952 => FreeColClient:-Resource loader
[...]
```

Мы видим, как в период трассировки создавались и запускались, а также останавливались («<=>») некоторые недолговечные потоки выполнения.

Этот инструмент использует зонды USDT для Java. Поскольку частота создания новых потоков невысока, переход инструмента должен быть незначительным. Вот его исходный код:

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("%-20s %6s/%-5s -- %s\n", "TIME", "PID", "TID", "THREAD");
}

usdt:/.../libjvm.so:hotspot:thread__start
{
    time("%H:%M:%S ");
    printf("%6d/%-5d => %s\n", pid, tid, str(arg0, arg1));
}

usdt:/.../libjvm.so:hotspot:thread__stop
{
    time("%H:%M:%S ");
    printf("%6d/%-5d <= %s\n", pid, tid, str(arg0, arg1));
}
```

Путь к библиотеке в этом исходном коде усечен («...»), вам следует заменить его путем к вашей библиотеке libjvm.so. В будущем предполагается добавить в bpftrace возможность указывать имя библиотеки без пути, например «libjvm.so».

12.3.12. javacalls

`javacalls(8)`¹ — это инструмент для BCC и bpftrace, подсчитывающий вызовы Java-методов. Например:

```
# javacalls 16914
Tracing calls in process 16914 (language: java)... Ctrl-C to quit.
If you do not see any results, make sure you ran java with option -XX:
+ExtendedDTraceProbes
^C
METHOD                                # CALLS
net/sf/freecol/client/control/InGameInputHandler$$Lambda$4443.get$Lambda      1
sun/awt/X11/XWindowPeer.getLocalHostname                                     1
net/sf/freecol/common/model/UnitType.getSpace                               1
[...]
java/awt/image/Raster.getHeight                                           129668
java/lang/Math.min                                                         177085
jdk/internal/misc/Unsafe.getByte                                           201047
java/lang/AbstractStringBuilder.putStringAt                               252367
java/lang/AbstractStringBuilder.getCoder                                  252367
java/lang/String.getBytes                                                  253184
java/lang/AbstractStringBuilder.append                                     258491
java/lang/Object.<init>                                                     258601
java/lang/AbstractStringBuilder.ensureCapacityInternal                    258611
java/lang/String.isLatin1                                                  265540
java/lang/StringBuilder.append                                             286637
jdk/internal/misc/Unsafe.putInt                                           361628
java/lang/System.arraycopy                                                399118
java/lang/String.length                                                    427242
jdk/internal/misc/Unsafe.getInt                                           700137
java/lang/String.coder                                                     1268791
```

При трассировке чаще других вызывался метод `java/lang/String.code()`. Всего был зафиксирован 1 268 791 вызов этого метода.

Инструмент использует зонды USDT для Java из расширенного набора, которые включаются параметром `-XX: +ExtendedDTraceProbes`, что влечет значительный оверхед. Как отмечалось выше, если включить и инструментировать эти зонды, скорость работы приложения может упасть в 10 раз и более.

BCC

Порядок использования:

```
javacalls [options] pid [interval]
```

¹ Немного истории: он был создан 19 октября 2016 года Сашей Гольдштейном как обертка для его инструмента `ucalls(8)`, версию для `bpftrace` я написал специально для этой книги 11 марта 2019 года. 9 сентября 2007 года я написал похожий инструмент для `DTrace`, который называется `j_calls.d`.

Параметры `options`:

- **-L**: вместо количества вызовов методов подсчитывать сумму задержек в них;
- **-m**: выводить задержки в миллисекундах.

`javacalls(8)` — это обертка для инструмента `ucalls(8)` в каталоге `tools/lib` с исходным кодом ВСС, который поддерживает несколько языков.

bpfttrace

Вот исходный код версии для `bpfttrace`:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing Java method calls. Ctrl-C to end.\n");
}

usdt:/.../libjvm.so:hotspot:method__entry
{
    @[str(arg1, arg2), str(arg3, arg4)] = count();
}
```

Ключ карты состоит из двух строк: имени класса и имени метода. Как и версия для ВСС, этот сценарий работает, только если приложение запущено с параметром `-XX: +ExtendedDTraceProbes`, что сильно снижает производительность. Обратите внимание, что в исходном коде полный путь к `libjvm.so` был усечен, и вам нужно его заменить на путь к `libjvm.so`.

12.3.13. **javaflow**

`javaflow(8)`¹ — это инструмент для ВСС, регистрирующий последовательности вызовов Java-методов. Например:

```
# javaflow 16914
Tracing method calls in java process 16914... Ctrl-C to quit.
CPU PID  TID  TIME(us) METHOD
5   622   652   0.135   -> sun/awt/SunToolkit-.awtUnlock
5   622   652   0.135   -> java/util/concurrent/locks/ReentrantLock.unlock
5   622   652   0.135   -> java/util/concurrent/locks/AbstractQueuedSynchroniz...
5   622   652   0.135   -> java/util/concurrent/locks/ReentrantLock$Sync.tryR...
5   622   652   0.135   -> java/util/concurrent/locks/AbstractQueuedSynchron...
5   622   652   0.135   <- java/util/concurrent/locks/AbstractQueuedSynchron...
5   622   652   0.135   -> java/lang/Thread.currentThread
5   622   652   0.135   <- java/lang/Thread.currentThread
```

¹ Немного истории: он был создан 27 октября 2016 года Сашей Гольдштейном как обертка для его инструмента `uflow(8)`. 9 сентября 2007 года я написал похожий инструмент для DTrace, который называется `j_flowtime.d`.


```

5  622  652  0.135      -> java/util/concurrent/locks/AbstractOwnableSynchr...
5  622  652  0.135      <- java/util/concurrent/locks/AbstractOwnableSynchr...
5  622  652  0.135      -> java/util/concurrent/locks/AbstractQueuedSynchr...
5  622  652  0.135      <- java/util/concurrent/locks/AbstractQueuedSynchr...
5  622  652  0.135      <- java/util/concurrent/locks/ReentrantLock$Sync.tryR...
5  622  652  0.135      <- java/util/concurrent/locks/AbstractQueuedSynchronize...
5  622  652  0.135      <- java/util/concurrent/locks/ReentrantLock.unlock
5  622  652  0.135      <- sun/awt/SunToolkit-.awtUnlock
5  622  652  0.135      <- sun/awt/X11/XToolkit.getNextTaskTime
5  622  652  0.135      -> sun/awt/X11/XToolkit.waitForEvents
5  622  652  0.135      -> sun/awt/SunToolkit-.awtUnlock
[...]
```

```

1  622  654  4.159      <- sun/java2d/SunGraphics2D.drawI...
Possibly lost 9 samples
1  622  654  4.159      <- net/sf/freecol/common/model/Ti...
Possibly lost 9 samples
1  622  654  4.159      <- java/util/AbstractList.<init>
[...]
```

Мы видим последовательность вызова одних методов из других. Отступы в столбце METHOD показывают глубину вложенности вызовов методов.

Инструмент использует зонды USDT для Java из расширенного набора, которые включаются параметром `-XX: +ExtendedDTraceProbes`, что влечет значительный оверхед. Как отмечалось выше, если включить и инструментировать эти зонды, скорость работы приложения может упасть в 10 раз и более. В этом примере также можно видеть сообщения «Possibly lost 9 samples» («Возможно, потеряно 9 образцов»): BPF может не успевать за событиями и пропускать их, чтобы не блокировать приложение. В таких случаях он информирует пользователя о возможных пропусках.

Порядок использования:

```
javaflow [options] pid
```

Параметры options:

- **-M METHOD**: трассировать только методы, имена которых начинаются с этого префикса.

`javaflow(8)` — это обертка для инструмента `uflow(8)` в каталоге `tools/lib` с исходным кодом BCC, который поддерживает несколько языков.

12.3.14. javagc

`javagc(8)`¹ — это инструмент для BCC, регистрирующий события сборки мусора в JVM. Например:

¹ Немного истории: этот инструмент был создан 19 октября 2016 года Сашей Гольдштейном как обертка для его инструмента `ugc(8)`.

```
# javagc 16914
Tracing garbage collections in java process 16914... Ctrl-C to quit.
START    TIME(us)  DESCRIPTION
5.586    1330.00   None
5.586    1339.00   None
5.586    1340.00   None
5.586    1342.00   None
5.586    1344.00   None
[...]
```

Для каждого события сборки мусора выводится его смещение в секундах относительно момента запуска `javagc(8)` (столбец `START`) и продолжительность в микросекундах (столбец `TIME`).

Инструмент применяет стандартные зонды `USDT` для Java.

Порядок использования:

```
javagc [options] pid
```

Параметры `options`:

- **-m**: выводить время в миллисекундах.

`javagc(8)` — это обертка для инструмента `ugc(8)` в каталоге `tools/lib` с исходным кодом `BCC`, который поддерживает несколько языков.

12.3.15. javaobjnew

`javaobjnew(8)`¹ — это инструмент для `BCC`, подсчитывающий события создания Java-объектов. Вот пример его запуска с параметром `-c 10` для получения 10 наиболее часто создаваемых объектов:

```
# javaobjnew 25102
Tracing allocations in process 25102 (language: java)... Ctrl-C to quit.
^C
NAME/TYPE                                # ALLOCS      # BYTES
java/util/ArrayList                      429837         0
[Ljava/lang/Object;                      434980         0
java/util/ArrayList$Itr                  458430         0
java/util/HashMap$KeySet                  545194         0
[B                                         550624         0
java/util/HashMap$Node                    572089         0
net/sf/freecol/common/model/Map$Position 663721         0
java/util/HashSet                         696829         0
java/util/HashMap                        714633         0
java/util/HashMap$KeyIterator             904244         0
```

¹ Немного истории: он был создан 25 октября 2016 года Сашей Гольдштейном как обертка для его инструмента `uobjnew(8)`. 9 сентября 2007 года я написал похожий инструмент для `DTrace`, который называется `j_objnew.d`.

В период трассировки чаще других создавался объект `java/util/HashMap$KeyIterator`. Всего он создавался 904 244 раза. Столбец `BYTES` содержит нули, поскольку определение размеров объектов не поддерживается для этого типа языка.

Инструмент использует зонды `USDТ` для Java из расширенного набора, которые включаются параметром `-XX: +ExtendedDTraceProbes`, что влечет значительный оверхед. Как отмечалось выше, если включить и инструментировать эти зонды, скорость работы приложения может упасть в 10 раз и более.

Порядок использования:

```
javaobjnew [options] pid [interval]
```

В числе параметров `options` можно назвать:

- **-C TOP_COUNT**: показывать не более `TOP_COUNT` объектов;
- **-S TOP_SIZE**: показывать только объекты, занимающие в памяти не менее `TOP_SIZE` байт.

`javaobjnew(8)` — это обертка для инструмента `uobjnew(8)` в каталоге `tools/lib` с исходным кодом `BCC`, который поддерживает несколько языков (для некоторых из них поддерживается вывод действительных значений в столбце `BYTES`).

12.3.16. Однострочные сценарии для трассировки кода на Java

В этом разделе перечислены однострочные сценарии для `BCC` и `bpfftrace`. Там, где это возможно, один и тот же сценарий реализуется с использованием `BCC` и `bpfftrace`.

BCC

Подсчитывает события `JNI`, имена которых начинаются с «`jni_Call`»:

```
funccount '/.../libjvm.so:jni_Call*
```

Подсчитывает события вызова `Java`-методов:

```
funccount -p $(pidof java) 'u:/.../libjvm.so:method*'
```

Профилирует трассировки стека `Java` и имена потоков выполнения с частотой 49 Гц:

```
profile -p $(pidof java) -UF 49
```

bpfftrace

Подсчитывает события `JNI`, имена которых начинаются с «`jni_Call`»:

```
bpfftrace -e 'u:/.../libjvm.so:jni_Call* { @[probe] = count(); }'
```

Подсчитывает события вызова Java-методов:

```
bpfftrace -e 'usdt:/.../libjvm.so:method* { @[probe] = count(); }'
```

Профилирует трассировки стека Java и имена потоков выполнения с частотой 49 Гц:

```
bpfftrace -e 'profile:hz:49 /execname == "java"/ { @[ustack, comm] = count(); }'
```

Трассирует события компиляции методов:

```
bpfftrace -p $(pgrep -n java) -e 'U:/.../libjvm.so:method__compile__begin {  
    printf("compiling: %s\n", str(arg4, arg5)); }'
```

Трассирует события загрузки классов:

```
bpfftrace -p $(pgrep -n java) -e 'U:/.../libjvm.so:class__loaded {  
    printf("loaded: %s\n", str(arg0, arg1)); }'
```

Подсчитывает события создания объектов (требуется включить параметр ExtendedDTraceProbes):

```
bpfftrace -p $(pgrep -n java) -e 'U:/.../libjvm.so:object__alloc {  
    @[str(arg1, arg2)] = count(); }'
```

12.4. КОМАНДНАЯ ОБОЛОЧКА BASH

В качестве последнего примера рассмотрим интерпретируемый язык командной оболочки `bash`. Интерпретируемые языки обычно намного медленнее компилируемых, поскольку интерпретаторы вызывают множество своих внутренних функций, чтобы выполнить каждый шаг целевой программы. Это делает их нетипичной целью для анализа производительности, так как для рабочих нагрузок, чувствительных к производительности, обычно выбираются другие языки. Трассировка интерпретируемого кода с помощью BPF делается для устранения неполадок в программе, а не для поиска узких мест производительности.

Порядок трассировки интерпретируемого кода различается для разных языков и зависит от внутренней структуры ПО, которое выполняет этот код. В этом разделе я покажу, как сам подступаю к коду на неизвестном интерпретируемом языке и определяю, как его трассировать. Этот подход также можно использовать для трассировки других языков.

Выше в этой главе был пример трассировки функции `readline()` в `bash`, но я не останавливался подробно на этом языке. В этом разделе я покажу, как трассировать функции `bash` и встроенные вызовы, и представлю некоторые инструменты для автоматизации этого процесса. См. табл. 12.4.

Как упоминалось выше, способ сборки `bash` влияет на местоположение символов. Вот список динамических библиотек, которые использует `bash` в Ubuntu, полученный с помощью инструмента `ldd(1)`:

```
$ ldd /bin/bash
linux-vdso.so.1 (0x00007ffe7197b000)
libtinfo.so.5 => /lib/x86_64-linux-gnu/libtinfo.so.5 (0x00007f08aeb86000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f08ae982000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f08ae591000)
/lib64/ld-linux-x86-64.so.2 (0x00007f08af0ca000)
```

Таблица 12.4. Инструменты для анализа работы программного кода на bash

Инструмент	Источник	Цель	Описание
bashfunc	Книга	bash	Трассирует вызовы функций bash
bashfunclat	Книга	bash	Трассирует задержки функций bash

Цели для трассировки — исполняемый файл /bin/bash и перечисленные выше динамические библиотеки. Пример того, как это может вызвать различия: во многих дистрибутивах bash использует функцию readline() из /bin/bash, но некоторые дистрибутивы ссылаются на libreadline и вызывают ее оттуда.

Подготовка

Для подготовки я собрал bash, выполнив следующие шаги:

```
CFLAGS=-fno-omit-frame-pointer
./configure
make
```

Я добавил поддержку указателя фрейма, чтобы получить возможность обхода стека с его использованием во время анализа. В результате я получил двоичный файл bash с таблицами локальных символов, которые удалены из стандартного файла /bin/bash.

Пример программы

Ниже приведен пример программы welcome.sh на bash, которую я написал для анализа:

```
#!/home/bggregg/Build/bash-4.4.18/bash

function welcome {
    echo "Hello, World!"
    echo "Hello, World!"
    echo "Hello, World!"
}

welcome
welcome
welcome
welcome
```

```
welcome
welcome
welcome
sleep 60
```

Она начинается с определения пути к собранной мной версии `bash`. Программа 7 раз вызывает функцию `welcome`, а та, в свою очередь, трижды вызывает функцию `echo(1)` (которая, как я ожидаю, является встроенной функцией в `bash`), то есть всего программа вызывает `echo(1)` 21 раз. Я выбрал такие числа в надежде выделить эти вызовы на фоне других действий при трассировке¹.

12.4.1. Подсчет вызовов функций

Используя `funccount(8)` из ВСС, я предположил, что вызов функции в коде на `bash` выполняется внутренней функцией с именем, содержащим «func»:

```
# funccount 'p:/home/bggregg/Build/bash-4.4.18/bash:*func*'
Tracing 55 functions for "p:/home/bggregg/Build/bash-4.4.18/bash:*func*"... Hit
Ctrl-C
to end.
^C
FUNC                                COUNT
copy_function_def                   1
sv_funcnest                         1
dispose_function_def                1
bind_function                       1
make_function_def                   1
execute_intern_function              1
init_funcname_var                   1
bind_function_def                   2
dispose_function_def_contents        2
map_over_funcs                      2
copy_function_def_contents           2
make_func_export_array              2
restore_funcarray_state              7
execute_function                     7
find_function_def                    9
make_funcname_visible               14
execute_builtin_or_function          28
get_funcname                         29
find_function                        31
Detaching...
```

Начав трассировку, я запустил программу `welcome.sh`, которая 7 раз вызывает функцию `welcome`. Похоже, что мое предположение было верным: обнаружилось семь вызовов `restore_funcarray_state()` и `execute_function()`. Последнее имя выглядит наиболее многообещающим.

¹ В подобных случаях я часто использую простое число 23, но его применение сделало бы этот пример слишком длинным.

Имя `execute_function()` натолкнуло меня на мысль: вызываются ли еще какие-то функции, имена которых начинаются с «`execute_`»? Вот результат проверки с помощью `funccount(8)`:

```
# funccount 'p:/home/bggregg/Build/bash-4.4.18/bash:execute_*'
Tracing 29 functions for "p:/home/bggregg/Build/bash-4.4.18/bash:execute_*"... Hit
Ctrl-C to end.
^C
FUNC                                COUNT
execute_env_file                    1
execute_intern_function              1
execute_disk_command                1
execute_function                     7
execute_connection                  14
execute_builtin                     21
execute_command                     23
execute_builtin_or_function          28
execute_simple_command               29
execute_command_internal             51
Detaching...
```

Здесь особенно выделяется несколько цифр: `execute_builtin()` вызывалась 21 раз, что равно количеству вызовов `echo(1)`. То есть если мне потребуется выполнить трассировку `echo(1)` и других встроенных функций, я могу начать с трассировки `execute_builtin()`. Также 23 раза была вызвана функция `execute_command()`. Это число соответствует числу вызовов `echo(1)` плюс объявление функции плюс вызов `sleep(1)`. Похоже, это еще одна многообещающая функция, которую нужно проверить при исследовании `bash`.

12.4.2. Трассировка аргументов функции (`bashfunc.bt`)

Теперь займемся трассировкой вызовов `execute_function()`. Я хочу узнать, какую функцию она выполняет: может, она покажет мне имя «`welcome`». Надеюсь, имя функции передается в одном из аргументов. Судя по исходному коду `bash`, такой аргумент есть (`execute_cmd.c`):

```
static int
execute_function (var, words, flags, fds_to_close, async, subshell)
    SHELL_VAR *var;
    WORD_LIST *words;
    int flags;
    struct fd_bitmap *fds_to_close;
    int async, subshell;
{
    int return_val, result;
[...]
```

```
    if (subshell == 0)
    {
        begin_unwind_frame ("function_calling");
        push_context (var->name, subshell, temporary_env);
[...]
```

По всей вероятности, первый аргумент — `var`, представляет исполняемую функцию. Он имеет тип структуры `SHELL_VAR`, которая определяется в `variables.h`:

```
typedef struct variable {
    char *name;           /* Символ, который ввел пользователь. */
    char *value;          /* Возвращаемое значение. */
    char *exportstr;      /* Строка для среды. */
    sh_var_value_func_t *dynamic_value; /* Какую функцию вызвать, чтобы вернуть
                                         'динамическое' значение для переменной,
                                         такой как $SECONDS или $RANDOM. */
    sh_var_assign_func_t *assign_func; /* Какую функцию вызвать при попытке
                                         присвоить значение этой 'особой переменной'
                                         в bind_variable. */
    int attributes;       /* экспортируемая, только для чтения, массив,
                           невидимая... */
    int context;          /* Контекст, которому принадлежит переменная. */
} SHELL_VAR;
```

Значения типа `char *` трассируются достаточно просто. Посмотрим с помощью `bpfttrace`, что передается в члене `name`. Я могу подключить этот заголовок или объявить структуру непосредственно в сценарии для `bpfttrace`. Покажу оба подхода, начав с первого. Вот сценарий `bashfunc.bt`¹:

```
#!/usr/local/bin/bpfttrace

#include "/home/bgregg/Build/bash-4.4.18/variables.h"

uprobe:/home/bgregg/Build/bash-4.4.18/bash:execute_function
{
    $var = (struct variable *)arg0;
    printf("function: %s\n", str($var->name));
}
```

Результат выполнения:

```
# ./bashfunc.bt
/home/bgregg/Build/bash-4.4.18/variables.h:24:10: fatal error: 'stdc.h' file not
found
Attaching 1 probe...
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
^C
```

Работает! Теперь я могу трассировать вызовы функций в коде на `bash`.

Обратите внимание, что он также вывел предупреждение об отсутствии еще одного файла заголовка.

¹ Немного истории: я написал его для этой книги 9 февраля 2019 года.

Теперь рассмотрим второй подход, когда структура объявляется непосредственно в сценарии. На самом деле мне нужен только первый член, поэтому я объявляю только этот член и назову структуру «partial».

```
#!/usr/local/bin/bpfttrace

struct variable_partial {
    char *name;
};

uprobe:/home/bggregg/Build/bash-4.4.18/bash:execute_function
{
    $var = (struct variable_partial *)arg0;
    printf("function: %s\n", str($var->name));
}
```

Вот результат использования этой версии bashfunc.bt:

```
# ./bashfunc.bt
Attaching 1 probe...
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
^C
```

Тоже работает, причем без ошибки или требования исходного кода bash.

Обратите внимание, что uprobes — нестабильный интерфейс, поэтому этот сценарий может перестать работать, если bash изменит имена функций и аргументы.

12.4.3. Задержки в функциях (bashfunclat.bt)

Теперь, научившись трассировать вызовы функций, посмотрим их задержки, то есть то, как долго выполняются функции. Для начала я изменил определение функции в welcome.sh:

```
function welcome {
    echo "Hello, World!"
    sleep 0.3
}
```

Это обеспечивает известную продолжительность выполнения функции: 0.3 секунды.

Теперь проверим, ожидает ли execute_function() завершения функции на языке оболочки, измерив ее задержку с помощью funclatency(8) из BCC:

```
# funclatency -m /home/bggregg/Build/bash-4.4.18/bash:execute_function
Tracing 1 functions for "/home/bggregg/Build/bash-4.4.18/bash:execute_function"... Hit
Ctrl-C to end.
^C
```

```
Function = execute_function [7083]
      msec      : count      distribution
      0 -> 1      : 0
      2 -> 3      : 0
      4 -> 7      : 0
      8 -> 15     : 0
      16 -> 31    : 0
      32 -> 63    : 0
      64 -> 127   : 0
      128 -> 255  : 0
      256 -> 511  : 7
      *****
Detaching...
```

Задержка ее выполнения составила от 256 до 511 миллисекунд, что соответствует нашей известной задержке. Похоже, я могу просто измерять задержку выполнения этой функции, чтобы узнать время задержки функции на языке оболочки.

Превратим этот однострочный сценарий в инструмент `bashfunclat.bt`¹, который будет принимать имя функции на языке оболочки и выводить гистограмму ее задержек:

```
#!/usr/local/bin/bpftrace

struct variable_partial {
    char *name;
};

BEGIN
{
    printf("Tracing bash function latency, Ctrl-C to end.\n");
}
uprobe:/home/bgregg/Build/bash-4.4.18/bash:execute_function
{
    $var = (struct variable_partial *)arg0;
    @name[tid] = $var->name;
    @start[tid] = nsecs;
}

uretprobe:/home/bgregg/Build/bash-4.4.18/bash:execute_function
/@start[tid]/
{
    @ms[str(@name[tid])] = hist((nsecs - @start[tid]) / 1000000);
    delete(@name[tid]);
    delete(@start[tid]);
}
```

Он запоминает указатель на имя функции и отметку времени по достижении зонда `uprobe`. Когда срабатывает `uretprobe`, он получает имя и отметку времени начала, чтобы создать гистограмму.

Результат:

```
# ./bashfunclat.bt
Attaching 3 probes...
Tracing bash function latency, Ctrl-C to end.
```

¹ Немного истории: я написал его для этой книги 9 февраля 2019 года.

`execute_function()` — это один из локальных символов, таблица с которыми была удалена из исполняемого файла `/bin/bash` для уменьшения его размера.

К счастью, у меня все-таки кое-что есть: в выводе `funcstack(8)` 7 раз встретился вызов `restore_funcarray_state()`, что соответствует нашей известной рабочей нагрузке. Чтобы проверить, связано ли это с вызовами функций в интерпретируемом коде, я воспользуюсь инструментом `stackcount(8)` из ВСС и исследую трассировку стека этой функции:

```
# stackcount -P /bin/bash:restore_funcarray_state
Tracing 1 functions for "/bin/bash:restore_funcarray_state"... Hit Ctrl-C to end.
^C
  [unknown]
  [unknown]
    welcome0.sh [8514]
  7
```

Detaching...

Стек явно неполный: я включил этот вывод в книгу, чтобы показать, как по умолчанию выглядят стеки `/bin/bash`. Это одна из причин, почему я скомпилировал свою версию `bash` с поддержкой указателей фреймов. Переключимся на нее и исследуем:

```
# stackcount -P /home/bgregg/Build/bash-4.4.18/bash:restore_funcarray_state
Tracing 1 functions for
"/home/bgregg/Build/bash-4.4.18/bash:restore_funcarray_state"... Hit Ctrl-C to end.
^C
  restore_funcarray_state
  without_interrupts
  run_unwind_frame
  execute_function
  execute_builtin_or_function
  execute_simple_command
  execute_command_internal
  execute_command
  reader_loop
  main
  __libc_start_main
  [unknown]
    welcome.sh [8542]
  7
```

Detaching...

Как видите, `restore_funcarray_state()` вызывается из `execute_function()`, то есть ее вызов действительно связан с вызовами функций в коде на языке оболочки.

Эта функция находится в файле `execute_cmd.c`:

```
void
restore_funcarray_state (fa)
{
    struct func_array_state *fa;
```

Структура `func_array_state` определена в заголовке `execute_cmd.h`:

```
struct func_array_state
{
    ARRAY    *funcname_a;
    SHELL_VAR *funcname_v;
    ARRAY    *source_a;
    SHELL_VAR *source_v;
    ARRAY    *lineno_a;
    SHELL_VAR *lineno_v;
};
```

Похоже, что она используется для создания локальных контекстов перед вызовом функций в интерпретируемом коде. Я подумал, что `funcname_a` и `funcname_v` могут содержать то, что мне нужно: имя вызываемой функции. Поэтому я объявил структуры и вывел строки, как это уже делал в предыдущем инструменте `bashfunc.bt`. Но в результатах искомого имени функции я не увидел.

Дальше можно пойти разными путями, а учитывая, что я использую нестабильный интерфейс (`uprobes`), они могут оказаться неправильными (правильный путь — `USDT`). Вот пример следующих шагов:

- `funccount(8)` показал несколько других интересных функций: `find_function()`, `make_funcname_visible()` и `find_function_def()`, которые вызываются чаще, чем наша известная функция. Возможно, имя функции находится в их аргументах или в возвращаемом значении и я смогу извлечь его для дальнейшего поиска в `restore_funcarray_state()`.
- `stackcount(8)` показал функции более высокого уровня. Возможно, какие-то из этих символов есть в `/bin/bash` и могут дать другие пути к трассировке функции.

Вот результаты второй попытки отыскать функции `execute` — видимые в `/bin/bash`:

```
# funccount '/bin/bash:execute_*'
Tracing 4 functions for "/bin/bash:execute_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
execute_command                     24
execute_command_internal             52
Detaching...
```

Заглянув в исходный код, можно заметить, что `execute_command()` запускает много чего, в том числе и функции, и их можно идентифицировать по номеру типа в первом аргументе. Вот одно из возможных направлений дальнейших исследований: отфильтровать вызовы функций и изучить другие аргументы. Возможно, в них обнаружится имя искомой функции.

Первая же попытка дала желаемый результат: `find_function()` получает имя в аргументе, которое я мог сохранить для последующего поиска. Вот обновленная версия `bashfunc.bt`:

```
#!/usr/local/bin/bpftrace

uprobe:/bin/bash:find_function_def
{
    @currfunc[tid] = arg0;
}

uprobe:/bin/bash:restore_funcarray_state
{
    printf("function: %s\n", str(@currfunc[tid]));
    delete(@currfunc[tid]);
}
```

и результат ее использования:

```
# bashfunc.bt
Attaching 2 probes...
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
function: welcome
```

Эта версия работает, но во многом зависит от конкретной версии `bash` и особенностей ее реализации.

12.4.5. Зонды USDT в `/bin/bash`

Чтобы не столкнуться с проблемами при трассировке после изменения внутренних компонентов `bash`, в код командной оболочки можно было бы добавить зонды USDT. Например, представьте такие зонды USDT:

```
bash:execute__function__entry(char *name, char **args, char *file, int lineno)
bash:execute__function__return(char *name, int retval, char *file, int lineno)
```

Они позволят с легкостью узнать имя функции, ее аргументы и возвращаемое значение, задержку, имя файла с исходным кодом и номер строки.

Для примера в командную оболочку Bourne для Solaris [139] были добавлены зонды USDT:

```
provider sh {
    probe function-entry(file, function, lineno);
    probe function-return(file, function, rval);
    probe builtin-entry(file, function, lineno);
    probe builtin-return(file, function, rval);
    probe command-entry(file, function, lineno);
    probe command-return(file, function, rval);
    probe script-start(file);
```

```

    probe script-done(file, rval);
    probe subshell-entry(file, childpid);
    probe subshell-return(file, rval);
    probe line(file, lineno);
    probe variable-set(file, variable, value);
    probe variable-unset(file, variable);
};

```

Этот пример должен также подсказать идеи для будущих зондов USDT в оболочке `bash`.

12.4.6. Однострочные сценарии для трассировки `bash`

В этом разделе перечислены однострочные сценарии для `BCC` и `bpfttrace`, позволяющие анализировать командную оболочку `bash`.

BCC

Подсчитывает вызовы функций, имена которых начинаются с «`execute_`» (требуется наличия таблицы символов):

```
funccount '/bin/bash:execute_*
```

Трассирует интерактивный ввод команд:

```
trace 'r:/bin/bash:readline "%s", retval'
```

bpfttrace

Подсчитывает вызовы функций, имена которых начинаются с «`execute_`» (требуется наличия таблицы символов):

```
bpfttrace -e 'uprobe:/bin/bash:execute_* { @[probe] = count(); }'
```

Трассирует интерактивный ввод команд:

```
bpfttrace -e 'ur:/bin/bash:readline { printf("read: %s\n", str(retval)); }'
```

12.5. ДРУГИЕ ЯЗЫКИ

Есть множество языков программирования и сред выполнения, а в будущем их появится еще больше. Для трассировки кода на этих языках сначала нужно определить, как они реализованы: выполняется ли компиляция кода в двоичные файлы, используется ли JIT-компиляция, интерпретируется ли код или используется комбинация из интерпретации и динамической компиляции? Изучение предыдущих разделов о `C` (для компилируемых языков), `Java` (для языков с поддержкой

JIT-компиляции) и оболочке `bash` (для интерпретируемых языков) даст вам представление о подходах и связанных с ними проблемах.

На сайте этой книги [140] я буду размещать ссылки на статьи об использовании BPF для трассировки других языков по мере их появления. Ниже приведено несколько советов для других языков, которые мне приходилось трассировать с помощью BPF: JavaScript (Node.js), C++ и Golang.

12.5.1. JavaScript (Node.js)

Трассировка кода на JavaScript (Node.js) средствами BPF похожа на трассировку кода на Java. Сейчас в Node.js используется среда выполнения V8, разработанная Google для браузера Chrome. V8 может интерпретировать функции на JavaScript или динамически компилировать их в машинный код. Среда выполнения также управляет памятью и имеет процедуру сборки мусора.

Ниже кратко описаны зонды USDT в Node.js, а также приемы обхода стека, получения символов и трассировки функций.

Зонды USDT

Для целей трассировки есть встроенные зонды USDT и библиотека `node-usdt`, позволяющая добавлять динамические зонды USDT в код на JavaScript [141]. На момент написания книги дистрибутивы Linux включают пакеты Node.js с отключенными зондами USDT: чтобы использовать их, нужно собрать Node.js из исходных кодов с параметром `--with-dtrace`, как показано ниже:

```
$ wget https://nodejs.org/dist/v12.4.0/node-v12.4.0.tar.gz
$ tar xf node-v12.4.0.tar.gz
$ cd node-v12.4.0
$ ./configure --with-dtrace
$ make
```

Вот список зондов USDT, доступных в `bpftool`:

```
# bpftool -l 'usdt:/usr/local/bin/node'
usdt:/usr/local/bin/node:node:gc__start
usdt:/usr/local/bin/node:node:gc__done
usdt:/usr/local/bin/node:node:http__server__response
usdt:/usr/local/bin/node:node:net__stream__end
usdt:/usr/local/bin/node:node:net__server__connection
usdt:/usr/local/bin/node:node:http__client__response
usdt:/usr/local/bin/node:node:http__client__request
usdt:/usr/local/bin/node:node:http__server__request
[...]
```

В списке есть зонды USDT для событий сборки мусора, обработки HTTP-запросов и сетевых взаимодействий. Подробнее о зондах USDT в Node.js я пишу в своем блоге «Linux bcc/BPF Node.js USDT Tracing» [142].

Обход стека

Обход стека (на основе указателей фрейма) не должен вызывать проблем, хотя для преобразования функций JavaScript, скомпилированных JIT-компилятором, в символы нужен еще один шаг (о нем речь ниже).

Символы

По аналогии с Java для преобразования адресов динамически скомпилированных функций в их имена нужно поместить в /tmp дополнительные файлы символов. Создать такие файлы для версии Node.js 10.x или выше можно двумя способами:

1. Использовать флаги `--perf_basic_prof` и `--perf_basic_prof_only_functions`, поддерживаемые средой выполнения V8. В этом случае будут создаваться непрерывно обновляемые журналы символов, в отличие от среды выполнения Java, которая выгружает моментальные снимки таблицы символов. Поскольку эти журналы нельзя отключить, пока процесс не завершится, в результате могут получиться гигантские файлы с таблицами символов (до нескольких гигабайт), содержащих в основном устаревшие символы.
2. С помощью модуля `linux-perf` [143], который представляет собой комбинацию работы флагов и работы `perf-map-agent` в Java: он захватывает все функции в куче, записывает таблицу символов в файл, а затем продолжает обновлять файл по мере компиляции новых функций. Запустить модуль можно в любой момент. Я рекомендую использовать на практике именно этот подход.

При использовании любого из этих подходов требуется дополнительная пост-обработка файлов символов с целью удаления устаревших записей¹.

Еще один рекомендуемый флаг: `--interpreted-frames-native-stack` (также доступен в Node.js 10.x и выше). С этим флагом инструменты `perf` и `BPF` в Linux могут транслировать интерпретируемые функции JavaScript в их фактические имена (вместо вывода фреймов «Interpreter» в стеке).

Внешние символы Node.js обычно используются для профилирования потребления процессора и создания флейм-графиков [144]. Последние можно сгенерировать с помощью инструментов `perf(1)` или `BPF`.

Трассировка функций

Сейчас нет зондов USDT для трассировки функций JavaScript, и из-за особенностей архитектуры V8 было бы сложно их добавить. Даже если кто-то сделает это, то, как я уже говорил в разделе о трассировке кода на Java, оверхед может оказаться

¹ Можно было бы подумать, что такие инструменты, как `perf(1)`, будут читать файл символов в обратном порядке и использовать самые последние их версии. Но я обнаружил, что в действительности используются более старые версии символов. Поэтому и потребовалась постобработка журналов: чтобы сохранить только самые новые версии символов.

существенным и приводить к замедлению приложений до 10 раз и более во время использования этих зондов.

Функции JavaScript видны в трассировках стека в пространстве пользователя и могут отбираться по событиям ядра, например по времени, по событиям дискового ввода/вывода и TSP или переключения контекста. Это дает массу возможностей для исследования работы Node.js, в том числе в контексте функций, без ущерба для непосредственной трассировки.

12.5.2. C++

Код на C++ можно трассировать почти так же, как код на C, используя зонды `uprobes` на входах в функции и на выходах из них и приемы обхода стека на основе указателя фрейма, если установлен соответствующий флаг компилятора. Но есть несколько отличий:

- Имена символов являются сигнатурами C++. Вместо `ClassLoader::initialize()` этот символ может быть виден как `_ZN11ClassLoader10initializeEv`. Инструменты BCC и `bpfttrace` учитывают эту особенность при выводе символов.
- Аргументы функций могут не соответствовать ABI процессора с точки зрения поддержки объектов и объекта-владельца (`this`).

Подсчет вызовов функций, измерение задержек и отображение трассировки стека не должны вызывать сложностей. Для сопоставления имен функций с их сигнатурами можно использовать шаблонные символы (например, `uprobe:/path:*ClassLoader*initialize*`).

Для проверки аргументов потребуется дополнительная работа. Иногда их позиции в списке аргументов просто смещаются на единицу, потому что в первом аргументе передается ссылка на объект-владелец (`this`). Строки нередко являются не обычными строками C, а объектами C++, и их нельзя просто разыменовывать. Объекты требуют объявления структур в программах BPF, чтобы BPF мог получить доступ к членам.

Все это может стать намного проще с BTF, представленным в главе 2, который позволяет указать местоположение аргументов и членов объекта.

12.5.3. Golang

Программы на Golang компилируются в двоичные файлы и трассируются подобно двоичным файлам C, но есть некоторые важные отличия в соглашениях о вызове функций, горутинах (`goroutines`) и динамическом управлении стеком. Из-за последнего зонды `uretprobes` небезопасно использовать с кодом на Golang, так как их инструментация может привести к сбою целевой программы. Есть и различия между компиляторами: Go-компилятор по умолчанию `gc` генерирует статически связанные двоичные файлы, тогда как `gccgo` создает динамически связанные двоичные файлы. Эти темы рассмотрены ниже.

Обратите внимание, что уже разработаны другие способы отладки и трассировки программ на Go, о которых следует знать, включая поддержку времени выполнения Go в gdb, трассировщик выполнения go [145] и GODEBUG с gctrace и schedtrace.

Обход стека и символы

Оба компилятора Go, gc и gccgo, по умолчанию поддерживают указатель фрейма (начиная с версии Go 1.7) и добавляют таблицы символов в двоичные файлы. Это означает, что трассировки стека, включающие функции на Go, легко можно собирать по событиям в пространстве пользователя или ядра и проводить профилирование путем выборки стеков по времени.

Трассировка точек входа в функции

Точки входа в функции можно трассировать с помощью uprobes. Вот пример применения bpftrace для подсчета количества вызовов функций, имена которых начинаются с «fmt», в программе на Golang с названием “hello”, скомпилированной Go-компилятором gc:

```
# bpftrace -e 'uprobe:/home/bggregg/hello:fmt* { @[probe] = count(); }'
Attaching 42 probes...
^C

@[uprobe:/home/bggregg/hello:fmt.(*fmt).fmt_s]: 1
@[uprobe:/home/bggregg/hello:fmt.newPrinter]: 1
@[uprobe:/home/bggregg/hello:fmt.Fprintln]: 1
@[uprobe:/home/bggregg/hello:fmt.(*pp).fmtString]: 1
@[uprobe:/home/bggregg/hello:fmt.glob..func1]: 1
@[uprobe:/home/bggregg/hello:fmt.(*pp).printArg]: 1
@[uprobe:/home/bggregg/hello:fmt.(*pp).free]: 1
@[uprobe:/home/bggregg/hello:fmt.Println]: 1
@[uprobe:/home/bggregg/hello:fmt.init]: 1
@[uprobe:/home/bggregg/hello:fmt.(*pp).doPrintln]: 1
@[uprobe:/home/bggregg/hello:fmt.(*fmt).padString]: 1
@[uprobe:/home/bggregg/hello:fmt.(*fmt).truncate]: 1
```

Начав трассировку, я один раз запустил программу hello. Судя по полученному выводу, было вызвано несколько функций fmt, включая fmt.Println(), которая, как я подозреваю, выводит сообщение «Hello, World!».

Теперь выполним точно такой же подсчет, используя двоичный файл, сгенерированный компилятором gccgo. В этом случае искомые функции находятся в библиотеке libgo, поэтому следует трассировать эту библиотеку:

```
# bpftrace -e 'uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt* { @[probe] = count(); }'
Attaching 143 probes...
^C

@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.fmt.clearflags]: 1
```

```

@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.fmt.truncate]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.Println]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.newPrinter]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.buffer.WriteByte]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.pp.printArg]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.pp.fmtString]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.fmt.fmt_s]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.pp.free]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.fmt.init]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.buffer.WriteString]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.pp.doPrintln]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.fmt.padString]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt.Fprintln]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt..import]: 1
@[uprobe:/usr/lib/x86_64-linux-gnu/libgo.so.13:fmt..go..func1]: 1

```

Здесь используется немного другое соглашение об именах функций. В этом выводе мы видим `fmt.Println()`, как и прежде.

Количество вызовов функций также можно подсчитать с помощью `funccount(8)` из ВСС. Первая команда в примере ниже предназначена для версии, скомпилированной с помощью `gc`, а вторая — с помощью `gccgo`:

```

funccount '/home/bggregg/hello:fmt.*'
funccount 'go:fmt.*'

```

Аргументы функции в точке входа

Компиляторы `gc` и `gccgo` следуют разным соглашениям о вызове функций: `gccgo` использует стандартный AMD64 ABI, а `gc` — прием передачи стека Plan 9. Это означает, что доступ к аргументам функции должен осуществляться по-разному: в `gccgo` можно использовать обычный подход (например, через `arg0 ... argN` в `bpftrace`), а в `gc` нужно использовать специальный код (см. [146], [147]).

Например, рассмотрим функцию `add(x int, y int)` из руководства по `Golang` [148], которая вызывается с аргументами 42 и 13. Вот как можно получить ее аргументы в процессе трассировки двоичного файла, созданного компилятором `gccgo`:

```

# bpftrace -e 'uprobe:/home/bggregg/func:main*add { printf("%d %d\n", arg0, arg1); }'
Attaching 1 probe...
42 13

```

Как видите, можно использовать встроенные значения `arg0` и `arg1`. Обратите внимание: чтобы компилятор не сделал функцию `add()` встраиваемой, пришлось скомпилировать программу командой `gccgo -O0`.

Теперь получим аргументы в двоичном файле, созданном компилятором `gc`:

```

# bpftrace -e 'uprobe:/home/bggregg/Lang/go/func:main*add { printf("%d %d\n",
    *(reg("sp") + 8), *(reg("sp") + 16)); }'
Attaching 1 probe...
42 13

```

На этот раз к аргументам пришлось обращаться по их смещениям в стеке, через `reg("sp")`. В будущем, возможно, `bpfttrace` станет поддерживать псевдонимы — `sarg0`, `sarg1` [149], сокращенно от «stack argument» (аргумент в стеке). Обратите внимание: чтобы компилятор не сделал функцию `add()` встраиваемой, пришлось скомпилировать программу командой `go build -gcflags '-N -l' ....`

Возвращаемые значения функций

К сожалению, трассировка зондов `uretprobes` небезопасна в их текущей реализации. Скомпилированный код на Go может изменить стек в любое время, не подозревая, что ядро добавило в стек обработчик `uretprobe`¹. Это может привести к повреждению памяти: после отключения зонда `uretprobe` ядро вернет байты в коде в нормальное состояние, но там, где раньше находились эти байты, могут находиться уже совсем другие данные, и они будут повреждены ядром. Это может вызвать сбой программы (если повезет) или она продолжит выполняться с поврежденными данными (если не повезет).

Джанлука Борелло экспериментировал с решением, основанным на использовании зондов `uprobes` в точках возврата из функций вместо `uretprobes`. Для этого он дизассемблировал функции, чтобы найти точки возврата, а затем помещал в них зонды `uretprobe` (см. [150]).

Еще одна проблема — горутины: они могут планироваться для выполнения в разных потоках ОС, поэтому обычный метод определения времени выполнения функции, когда отметка времени привязывается к идентификатору потока (например, в `bpfttrace: @start[tid] = nsecs`), оказался ненадежным.

USDT

Библиотека `Salp` предлагает динамические зонды `USDT`, доступные через `libstapsdt` [151]. Она позволяет добавлять статические зонды в код на Go.

12.6. ИТОГИ

Независимо от типа языка — компилируемый, с поддержкой JIT-компиляции или интерпретируемый — почти наверняка найдется способ проанализировать его с помощью BPF. В этой главе я рассмотрел эти три типа языков, а затем показал, как трассировать код на каждом из них: C, Java и `bash`. В процессе трассировки можно проверить вызовы функций или методов, исследовать их аргументы и возвращаемые значения, определить время выполнения функции или метода, а также отобразить трассировку стека по различным событиям. Также я дал некоторые советы по трассировке кода на JavaScript, C++ и Golang.

¹ Спасибо Сурешу Кумару, что помог объяснить эту проблему; см. его комментарий в [146].

Глава 13

ПРИЛОЖЕНИЯ

Работающие в системе приложения можно исследовать путем статической и динамической инструментации, что даст важный контекст для понимания других событий. В предыдущих главах приложения исследовались через используемые ими ресурсы: процессоры, память, диски и сеть. Подход, основанный на анализе ресурсов, позволяет решать многие проблемы, но может упускать важные прикладные детали — информацию о запросах, обрабатываемых в настоящее время. Чтобы выяснить все аспекты работы приложения, нужен как анализ ресурсов, так и анализ на уровне приложения. Механизм трассировки BPF позволяет изучить поток приложения, его код и контекст через библиотечные и системные вызовы, службы ядра и драйверы устройств.

В этой главе в качестве примера я использую систему управления базами данных MySQL. Запросы к MySQL послужат примером прикладного контекста. Представьте, что к инструментации различных событий дискового ввода/вывода, как было показано в главе 9, можно добавить еще одно измерение — строку запроса. Теперь можно увидеть, какие запросы требуют наибольшего количества операций дискового ввода/вывода, их задержки, закономерности и т. д.

Цели обучения:

- обнаруживать проблемы, связанные с созданием чрезмерного числа процессов и потоков;
- решать проблемы с нагрузкой на процессор с помощью профилирования;
- решать проблемы с блокировкой в ожидании доступности процессора путем трассировки планировщика;
- решать проблемы с чрезмерным количеством операций ввода/вывода путем трассировки стека ввода/вывода;
- исследовать контекст приложения с помощью зондов USDT и uprobes;
- исследовать пути в коде, ответственные за конфликт блокировок;
- выявлять операции явной приостановки приложения.

Эта глава — дополнение к предыдущим главам, освещающим подходы к трассировке ресурсов. Для получения полной информации о программном стеке читайте:

- главу 6 «Процессоры»;
- главу 7 «Память»;
- главу 8 «Файловые системы»;
- главу 9 «Дисковый ввод/вывод»;
- главу 10 «Сети».

Эта глава охватывает аспекты поведения приложений, не рассматривавшиеся в других главах: получение контекста приложения, управление потоками, сигналы, блокировки и приостановки.

13.1. ОСНОВЫ

Приложение может быть сервисом, отвечающим на сетевые запросы, программой, откликающейся на действия пользователя, программой, работающей с базой данных или файловой системой, либо чем-то еще. Приложения обычно реализуются как ПО, действующее в пространстве пользователя и видимое как процессы, обращающиеся к ресурсам через интерфейс системных вызовов (или отображаемую память).

13.1.1. Основы приложений

Управление потоками

В многопроцессорных системах конструкция ОС, называемая потоками (thread), позволяет приложениям эффективно выполнять работу на нескольких процессорах параллельно, разделяя при этом одно и то же адресное пространство процесса. Приложения могут использовать потоки различными способами, включая:

- **Пул сервисных потоков:** пул потоков, обслуживающих сетевые запросы, где каждый поток соответствует одному соединению с клиентом и последовательно обрабатывает поступающие запросы. Если обработку запроса необходимо заблокировать до освобождения некоторого ресурса, включая блокировки синхронизации с другими потоками в пуле, то поток приостанавливается. Приложение может иметь фиксированное количество потоков в пуле или увеличивать и уменьшать его в зависимости от потребностей. Пример: сервер БД MySQL.
- **Пул потоков по числу процессоров:** для выполнения работы приложение создает по одному потоку для каждого процессора. Обычно такая организация используется в приложениях пакетной обработки, которые обрабатывают один или несколько запросов из очереди, непрерывно и без дополнительного ввода, независимо от того, занимает это минуты, часы или дни. Пример: кодирование видео.

- **Поток обработки событий:** один или несколько потоков, обрабатывающих очередь заданий, пока она не опустеет, после чего поток приостанавливается. Каждый такой поток может обслуживать сразу несколько клиентов, выполняя задания по частям, обрабатывая клиентский запрос до появления более позднего события и переключаясь на следующее задание в очереди. Приложения с одним потоком обработки событий могут избежать необходимости использовать синхронизирующие блокировки, но рискуют превратиться в однопоточные при высокой нагрузке. Пример: Node.js, использующий один поток обработки событий.
- **Поэтапная событийная архитектура (Staged Event-Driven Architecture, SEDA):** приложения с архитектурой SEDA разбивают обработку запросов на этапы, которые могут выполняться пулами из одного или нескольких потоков [Welsh 01].

Блокировки

Блокировки — это примитивы синхронизации для многопоточных приложений. Они управляют доступом к памяти из потоков, выполняющихся параллельно, как светофоры регулируют движение через перекресток. Как и светофоры, они могут останавливать движение потока, заставляя его ждать. В Linux приложения обычно используют блокировки посредством библиотеки `libpthread`, которая предлагает разные типы блокировок, включая взаимоисключающие (мьютексы), блокировки чтения-записи и циклические блокировки.

Блокировки защищают доступ к памяти и могут стать источником проблем с производительностью. Когда сразу несколько потоков состязаются за захват одной блокировки, они приостанавливаются в ожидании своей очереди.

Приостановка

Приложения могут преднамеренно приостанавливаться на какое-то время. Иногда в этом есть смысл (в зависимости от причины приостановки), а иногда нет, поэтому такое их поведение может быть целью для оптимизации. Если вы разрабатывали приложения, то, возможно, вас посещала мысль: «Я просто добавлю сюда приостановку на одну секунду, чтобы дождаться появления ожидаемого события, а позже уберу приостановку и реализую реакцию на события». Но часто про это «позже» забывают, и тогда конечные пользователи задаются вопросом, почему обработка некоторых запросов занимает не менее одной секунды.

13.1.2. Пример приложения: сервер MySQL

В качестве примера приложения для анализа в этой главе я выбрал сервер MySQL. Он обрабатывает сетевые запросы, используя пул сервисных потоков. В зависимости от объема данных, обрабатываемых в большинстве запросов, ожидается, что MySQL будет выполнять преимущественно дисковые операции (когда обрабатываются

большие объемы данных) или вычислительные (когда объем обрабатываемых данных невелик и есть возможность извлекать результаты из кэша).

Сервер MySQL написан на C++ и имеет встроенные зонды USDT для трассировки запросов, команд, сортировки файлов, вставки и изменения данных, сетевого ввода/вывода и других событий. Несколько примеров приводится в табл. 13.1.

Таблица 13.1. Примеры зондов в MySQL

Зонд USDT	Аргументы
connection__start	unsigned long connection_id, char *user, char *host
connection__done	int status, unsigned long connection_id
command__start	unsigned long connection_id, int command, char *user, char *host
command__done	int status
query__start	char *query, unsigned long connection_id, char *db_name, char *user, char *host
query__done	int status
filesort__start	char *db_name, char *table
filesort__done	int status, unsigned long rows
net__write__start	unsigned long bytes
net__write__done	int status

Полный список зондов ищите в разделе «MySQL DTrace Probe Reference» в справочном руководстве MySQL [152]. Зонды USDT в MySQL доступны, только если MySQL собран с параметром `-DENABLE_DTRACE = 1` smake(1). Текущий пакет `mysql-server` для Linux собран без этого параметра, поэтому вам придется собрать свою версию сервера MySQL, чтобы использовать зонды USDT, или попросить мейнтейнеров включить этот параметр.

Во многих случаях нет возможности собрать свой сервер MySQL с поддержкой зондов USDT, поэтому в этой главе представлены приемы инструментации MySQL, использующие зонды `uprobes`.

13.1.3. Возможности BPF

Инструменты трассировки в BPF позволяют получить из ядра дополнительные сведения, кроме показателей, предоставляемых приложением: рабочую нагрузку и задержки, гистограммы задержек, а также потребление ресурсов. Можно получить ответы на вопросы:

- Какие запросы были получены от приложений? Как долго они обрабатывались?
- Где больше всего тратится времени при обработке запросов от приложений?
- Почему приложение долго занимает процессор?

- Почему приложение приостанавливается и освобождает процессор?
- Какие операции ввода/вывода выполняет приложение и почему (путь в коде)?
- По каким причинам приложение приостанавливается и на какой срок?
- Какие еще ресурсы ядра использует приложение и почему?

Ответы получают, инструментируя зонды USDT и `uprobes` (срабатывающие при изменении контекста запроса или обращении к ресурсам ядра), точки трассировки событий (включая системные вызовы) и `kprobes`, а также посредством выборки трассировки стека по времени.

Оверхед

Оверхед на трассировку приложений зависит от частоты трассируемых событий. Обычно трассировка самих запросов не влечет значительного оверхеда, но трассировка конфликтов блокировок, событий освобождения процессора и системных вызовов может привести к сильному оверхеду при больших рабочих нагрузках.

13.1.4. Стратегия

Ниже приведены общие рекомендации, которые пригодятся при анализе приложений. Указанные инструменты подробно рассмотрены в следующих разделах.

1. Выясните, что делает приложение: в каких единицах измеряется его работа. Для этого можно исследовать показатели работы приложения и его журналы. Также определите, что означает «улучшение производительности» для этого приложения: более высокая пропускная способность, меньшая задержка или меньшее потребление ресурсов (или их комбинация).
2. Посмотрите, есть ли какая-либо документация с описанием внутреннего устройства приложения: перечень основных компонентов — библиотеки и кэши, описание API и то, как производится обслуживание запросов: пулом потоков — потоками обработки событий или как-то иначе.
3. Помимо меры единицы работы приложения, выясните, выполняет ли оно какие-либо фоновые периодические задания, которые могут повлиять на производительность (например, чистка диска каждые 30 секунд).
4. Проверьте, доступны ли зонды USDT для приложения или для языка программирования, на котором оно написано.
5. Проанализируйте потребление процессора, чтобы понять его особенности и выявить неэффективность (например, используя `profile(8)` из BCC).
6. Выполните анализ простоев, чтобы понять, почему приложение приостанавливается, и найдите потенциальные цели для оптимизации (например, с помощью `offcputime(8)`, `wakeuptime(8)` и `offwaketime(8)` из BCC). Сосредоточьтесь на фактах приостановки во время обработки запросов.

7. Выполните профилирование системных вызовов, чтобы понять, как приложение использует ресурсы (например, с помощью `syscount(8)` из BCC).
8. Используйте другие инструменты BPF, перечисленные в главах 6–10.
9. Используйте зонды `uprobes` для исследования внутреннего устройства приложения: предыдущие результаты анализа использования процессора и простоев помогут выявить функции для последующего изучения.
10. В случае распределенных вычислений рассмотрите возможность трассировки как на стороне сервера, так и на стороне клиента. Например, анализируя работу MySQL, можно выполнить трассировку сервера и клиентов, выполняющих запросы, путем трассировки клиентской библиотеки MySQL.

Возможно, уже известно, какие операции составляют подавляющее большинство в приложении: вычислительные либо ввод/вывод (дисковый или сетевой), в зависимости от ресурса, на ожидание которого оно тратит большую часть времени. Убедившись, что это предположение верно, можно ограничиться исследованием конкретного ресурса, как говорилось в предыдущих главах этой книги.

Если вы решите написать программы для BPF с целью трассировки запросов, то нужно учесть, как обрабатываются эти запросы. Пулы сервисных потоков обрабатывают запросы полностью в одном потоке, поэтому для связывания асинхронных событий из разных источников можно использовать идентификатор потока (идентификатор задачи). Например, когда база данных начинает обработку запроса, строку запроса можно сохранить в карте BPF с идентификатором потока в роли ключа. Эту строку запроса можно позже прочитать при первой инициализации дискового ввода/вывода, чтобы связать операцию ввода/вывода с запросом, который его вызвал. Для приложений с другими архитектурами, например, основанных на использовании потока обработки событий, требуются другие подходы, потому что один поток обрабатывает разные запросы одновременно, а идентификатор потока не уникален для одного и того же запроса.

13.2. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа производительности приложений и устранения неполадок. Они показаны на рис. 13.1.

Все эти инструменты либо находятся в репозиториях BCC и `bpfttrace`, описанных в главах 4 и 5, либо написаны специально для этой книги. Некоторые инструменты можно найти в обоих репозиториях, BCC и `bpfttrace`. Происхождение инструментов также указывается в табл. 13.2 (BT — это сокращение от «`bpfttrace`»).

Актуальные списки параметров инструментов BCC и `bpfttrace` и описание их возможностей ищите в соответствующих репозиториях. Ниже я расскажу только о важных особенностях.

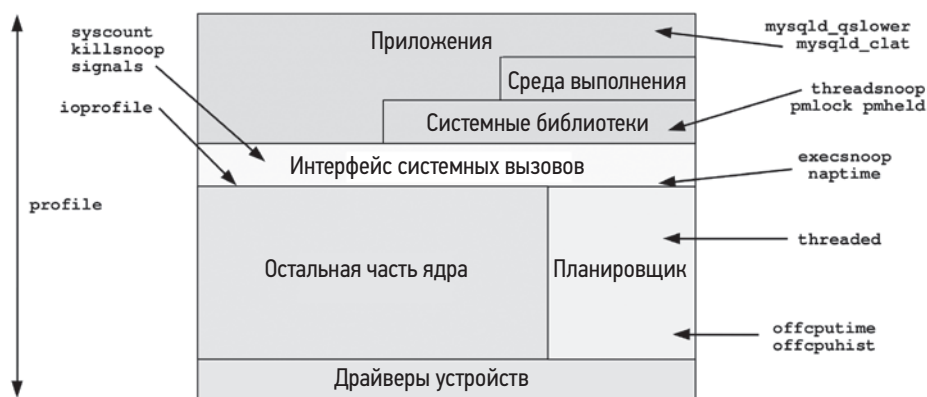


Рис. 13.1. Инструменты BPF для анализа приложений

Таблица 13.2. Инструменты для анализа приложений

Инструмент	Источник	Цель	Описание
execsnoop	ВСС/ВТ	Планировщик	Выводит список вновь запущенных процессов
threadsnoop	Книга	Библиотека pthread	Выводит список вновь запущенных потоков
profile	ВСС	Процессоры	Выбирает трассировки стека по процессорам
threaded	Книга	Процессоры	Выбирает трассировки стека по потокам
offcputime	ВСС	Планировщик	Трассировки стека и времена ожидания доступности процессора
offcpuhist	Книга	Планировщик	Трассировки стека и времена ожидания доступности процессора в виде гистограммы
syscount	ВСС	Системные вызовы	Подсчет системных вызовов по типам и процессам
ioprofile	Книга	Ввод/вывод	Подсчет трассировок стека по операциям ввода/вывода
mysqlq_qlower	ВСС/книга	Сервер MySQL	Отображает запросы, выполняющиеся дольше заданного времени
mysqlq_clat	Книга	Сервер MySQL	Отображает задержки выполнения команд в виде гистограммы
signals	Книга	Сигналы	Выводит сводную информацию по сигналам, посылаемым целевому процессу
killsnoop	ВСС/ВТ	Системные вызовы	Отображает обращения к системному вызову kill(2) с информацией об отправителях
pmlock	Книга	Блокировки	Отображает время ожидания на мьютексе в pthread с соответствующими трассировками стека в пространстве пользователя

Инструмент	Источник	Цель	Описание
pmheld	Книга	Блокировки	Отображает время удержания мьютекса в pthread с соответствующими трассировками стека в пространстве пользователя
naptime	Книга	Системные вызовы	Отображает события добровольной приостановки

Эти инструменты можно объединить в группы:

- **анализ потребления процессора:** profile(8), threaded(8) и syscount(8);
- **анализ периодов ожидания доступности процессора:** offcputime(8), offcpuhist(8) и ioprofile(8);
- **анализ прикладного контекста:** mysql_d_slower(8) и mysql_d_clat(8);
- **выполнение потоков:** execsnoop(8), threadsnoop(8) и threaded(8);
- **анализ блокировок:** rmlck(8) и pmheld(8);
- **сигналы:** signals(8) и killsnoop(8);
- **анализ приостановок:** naptime(8).

В конце этой главы также приведены однострочные сценарии. Среди разделов с описаниями инструментов есть и раздел, посвященный поддержке указателей фреймов в libc, — он идет за разделом с описанием ioprofile(8).

13.2.1. execsnoop

execsnoop(8) был представлен в главе 6. Это инструмент для BCC и bpftrace, трассирующий запуск новых процессов, он может использоваться для выявления случаев запуска короткоживущих процессов. Вот пример вывода, полученный на сервере, простаивающем без нагрузки:

```
# execsnoop
PCOMM      PID   PPID  RET  ARGS
sh          17788 17787  0    /bin/sh -c /usr/lib/sysstat/sa1 1 1 -S ALL
sa1         17789 17788  0    /usr/lib/sysstat/sa1 1 1 -S ALL
sadc        17789 17788  0    /usr/lib/sysstat/sadc -F -L -S DISK 1 1 -S ALL /var/
log/sysstat
[...]
```

Как можно заметить, сервер особо не простаивает: здесь мне удалось поймать запуск регистратора активности системы. execsnoop(8) пригодится для обнаружения неожиданного запуска процессов приложениями. Иногда приложения вызывают сценарии оболочки для выполнения некоторых действий, возможно, как временное решение, пока эти действия не будут реализованы в самом приложении, что часто становится причиной неэффективности.

Более подробно execsnoop(8) описывается в главе 6.

13.2.2. threadsnoop

threadsnoop(8)¹ трассирует события запуска новых потоков вызовом библиотечной функции `pthread_create()`. Вот пример вывода, полученный в момент запуска сервера MySQL:

```
# threadsnoop.bt
Attaching 3 probes...
TIME(ms)  PID  COMM  FUNC
2049      14456 mysql  timer_notify_thread_func
2234      14460 mysql  pfs_spawn_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  io_handler_thread
2243      14460 mysql  buf_flush_page_cleaner_coordinator
2274      14460 mysql  trx_rollback_or_clean_all_recovered
2296      14460 mysql  lock_wait_timeout_thread
2296      14460 mysql  srv_error_monitor_thread
2296      14460 mysql  srv_monitor_thread
2296      14460 mysql  srv_master_thread
2296      14460 mysql  srv_purge_coordinator_thread
2297      14460 mysql  srv_worker_thread
2297      14460 mysql  srv_worker_thread
2297      14460 mysql  srv_worker_thread
2298      14460 mysql  buf_dump_thread
2298      14460 mysql  dict_stats_thread
2298      14460 mysql  _Z19fts_optimize_threadPv
2298      14460 mysql  buf_resize_thread
2381      14460 mysql  pfs_spawn_thread
2381      14460 mysql  pfs_spawn_thread
```

Исследуя столбец `TIME(ms)`, можно определить, как часто и какие процессы и функции (столбцы `PID`, `COMM` и `FUNC`) запускают новые потоки. Здесь видно, что в процессе запуска MySQL создает пулы рабочих потоков сервера (`srv_worker_thread()`), потоков обработки ввода/вывода (`io_handler_thread()`) и другие потоки.

threadsnoop(8) трассирует вызовы библиотечной функции `pthread_create()`, которые, как ожидается, будут происходить относительно нечасто, поэтому оверхед инструмента должен быть незначительным.

¹ Немного истории: я написал его для этой книги 15 февраля 2019 года, взяв за основу свой же инструмент `hexesnoop`.

Исходный код threadsnoop(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("%-10s %-6s %-16s %s\n", "TIME(ms)", "PID", "COMM", "FUNC");
}

uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_create
{
    printf("%-10u %-6d %-16s %s\n", elapsed / 1000000, pid, comm,
        usym(arg2));
}
```

Возможно, вам потребуется скорректировать путь к библиотеке libpthread в своей системе.

Инструкцию вывода тоже можно расширить. Например, вот как добавить вывод трассировок стека в пространстве пользователя:

```
printf("%-10u %-6d %-16s %s\n", elapsed / 1000000, pid, comm,
    usym(arg2), ustack);
```

В этом случае вывод будет таким:

```
# ./threadsnoop-ustack.bt
Attaching 3 probes...
TIME(ms)  PID    COMM      FUNC
1555      14976  mysqld    timer_notify_thread_func
          0x7fb5ced4b9b0
          0x55f6255756b7
          0x55f625577145
          0x7fb5ce035b97
          0x2246258d4c544155

1729      14981  mysqld    pfs_spawn_thread
          __pthread_create_2_1+0
          my_timer_initialize+156
          init_server_components()+87
          mysqld_main(int, char**)+1941
          __libc_start_main+231
          0x2246258d4c544155

1739      14981  mysqld    io_handler_thread
          __pthread_create_2_1+0
          innobase_start_or_create_for_mysql()+6648
          innobase_init(void*)+3044
          ha_initialize handlerton(st_plugin_int*)+79
          plugin_initialize(st_plugin_int*)+101
          plugin_register_builtin_and_init_core_se(int*, char**)+485
          init_server_components()+960
          mysqld_main(int, char**)+1941
          __libc_start_main+231
          0x2246258d4c544155
```

[...]

Здесь дополнительно выводится путь в коде, закончившийся созданием потока. О назначении потоков в MySQL можно достаточно уверенно судить по именам функций, инициировавших создание потоков. Однако так бывает не всегда, поэтому при анализе некоторых приложений может понадобиться трассировка стека. Она сможет точно определить, с какой целью создаются новые потоки.

13.2.3. profile

profile(8) был представлен в главе 6. Это инструмент для ВСС, который отбирает трассировки стека активных процессов через заданный интервал. Это дешевое и грубое средство, помогающее увидеть, какие пути в коде занимают процессы. Вот пример использования profile(8) для профилирования сервера MySQL:

```
# profile -d -p $(pgrep mysqld)
```

```
Sampling at 49 Hertz of PID 9908 by user + kernel stack... Hit Ctrl-C to end.
```

```
[...]
```

```
my_hash_sort_simple
hp_rec_hashnr
hp_write_key
heap_write
ha_heap::write_row(unsigned char*)
handler::ha_write_row(unsigned char*)
end_write(JOIN*, QEP_TAB*, bool)
evaluate_join_record(JOIN*, QEP_TAB*)
sub_select(JOIN*, QEP_TAB*, bool)
JOIN::exec()
handle_query(THD*, LEX*, Query_result*, unsigned long long, unsigned long long)
execute_sqlcom_select(THD*, TABLE_LIST*)
mysql_execute_command(THD*, bool)
Prepared_statement::execute(String*, bool)
Prepared_statement::execute_loop(String*, bool, unsigned char*, unsigned char*)
mysqld_stmt_execute(THD*, unsigned long, unsigned long, unsigned char*, unsigned...)
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)
handle_connection
pfs_spawn_thread
start_thread
-                               mysqld (9908)
14
```

```
[...]
```

```
ut_delay(unsigned long)
srv_worker_thread
start_thread
-                               mysqld (9908)
```



```

16
_raw_spin_unlock_irqrestore
_raw_spin_unlock_irqrestore
__wake_up_common_lock
__wake_up_sync_key
sock_def_readable
unix_stream_sendmsg
sock_sendmsg
SYS_sendto
Sys_sendto
do_syscall_64
entry_SYSCALL_64_after_hwframe
--
__send
vio_write
net_write_packet
net_flush
net_send_ok(THD*, unsigned int, unsigned int, unsigned long long, unsigned lon...
Protocol_classic::send_ok(unsigned int, unsigned int, unsigned long long, unsi...
THD::send_statement_status()
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)
handle_connection
pfs_spawn_thread
start_thread
__clone
-                               mysqld (9908)
17

```

Я получил сотни трассировок стека, но здесь показал только три. Первая трассировка стека соответствует выполнению инструкции SQL с оператором соединения (JOIN), которая завершается вызовом `my_hash_sort_simple()`. Последняя трассировка соответствует отправке данных через сокет в ядре: здесь есть разделитель между стеками в пространствах ядра и пользователя («--»), который был добавлен благодаря использованию `profile(8)` с параметром `-d`.

Так как вывод включает сотни трассировок стека, для удобства анализа результаты можно представить в виде флейм-графика. `profile(8)` умеет генерировать выходные данные в свернутом формате (`-f`) для ввода в программу создания флейм-графиков. Вот пример 30-секундного профилирования:

```

# profile -p $(pgrep mysqld) -f 30 > out.profile01.txt
# flamegraph.pl --width=800 --title="CPU Flame Graph" < out.profile01.txt \
  > out.profile01.svg

```

На рис. 13.2 показан соответствующий флейм-график.

Флейм-график показывает, какие пути в коде расходуют больше всего процессорного времени (самые широкие фреймы): функция `dispatch_command()` есть в 69%

выборок, а функция `JOIN::exec()` — в 19%. Эти числа отображаются при наведении курсора на фреймы. По фреймам можно щелкать, чтобы увеличить масштаб и отобразить детали.

Флейм-графики помогают не только понять, на какие действия уходит больше всего процессорного времени, но и выяснить возможные цели для дальнейшей трассировки средствами BPF. На этом графике можно отметить функции `do_command()`, `mysql_stmt_execute()`, `JOIN::exec()` и `JOIN::optimize()`. Их все можно инструментировать напрямую с помощью `uprobes`, чтобы изучить их аргументы и задержки.

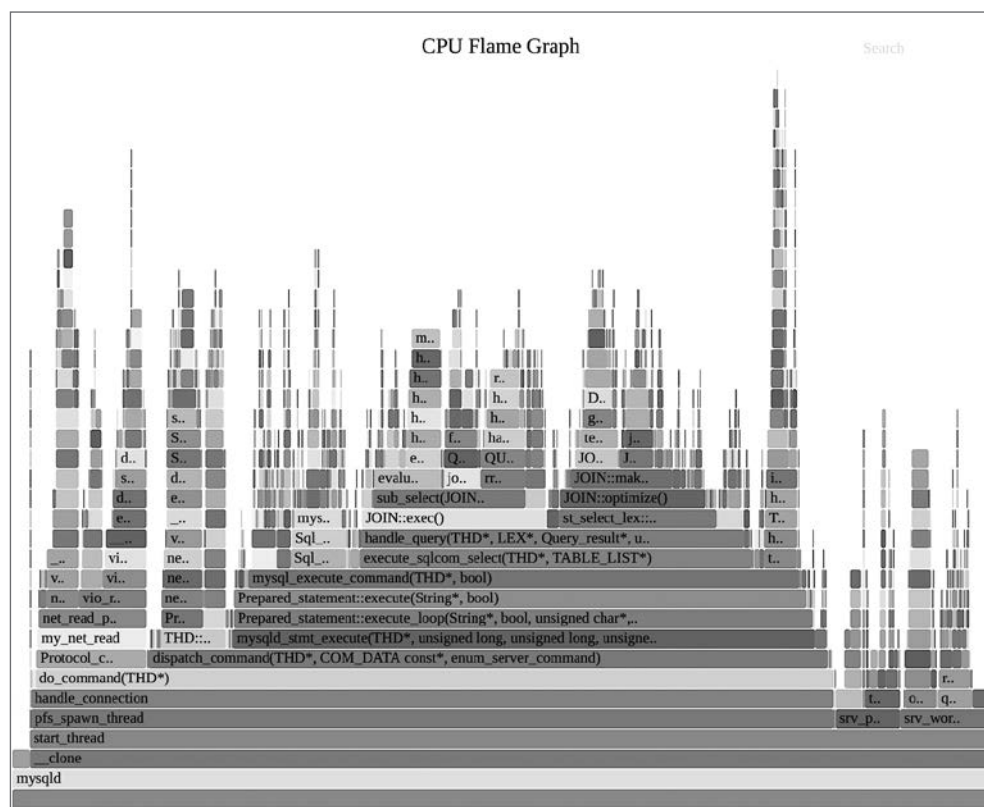


Рис. 13.2. Флейм-график потребления процессора сервером MySQL

Это работает только потому, что я профилирую сервер MySQL, который был скомпилирован с указателями фреймов, с версиями `libc` и `libpthread`, которые также имеют указатели фреймов. Без этого BPF не смог бы произвести обход стека. Подробнее об этом идет речь в разделе 13.2.9.

Более подробно `profile(8)` и флейм-графики потребления процессора описаны в главе 6.

13.2.4. threaded

threaded(8)¹ отбирает трассировки стека для заданного процесса и показывает, как часто он загружал процессор. Это позволяет оценить, насколько оптимально он использует многопоточность. Пример трассировки сервера MySQL:

```
# threaded.bt $(pgrep mysqld)
Attaching 3 probes...
Sampling PID 2274 threads at 99 Hertz. Ctrl-C to end.
23:47:13
@[mysqld, 2317]: 1
@[mysqld, 2319]: 2
@[mysqld, 2318]: 3
@[mysqld, 2316]: 4
@[mysqld, 2534]: 55

23:47:14
@[mysqld, 2319]: 2
@[mysqld, 2316]: 4
@[mysqld, 2317]: 5
@[mysqld, 2534]: 51

[...]
```

threaded(8) выводит данные раз в секунду, и здесь он показывает, что основная нагрузка в сервере MySQL легла только на один поток (с идентификатором 2534).

По полученным результатам можно судить, насколько равномерно многопоточное приложение распределяет работу между своими потоками. Но так как выборка стеков выполняется по времени, инструмент может пропускать потоки, которые возобновляют работу и вновь приостанавливаются между выборками.

Некоторые приложения могут менять имена потоков. Вот пример использования threaded(8) для трассировки Java-приложения freecol из предыдущей главы:

```
# threaded.bt $(pgrep java)
Attaching 3 probes...
Sampling PID 32584 threads at 99 Hertz. Ctrl-C to end.
23:52:12
@[GC Thread#0, 32591]: 1
@[VM Thread, 32611]: 1
@[FreeColClient:b, 32657]: 6
@[AWT-EventQueue-, 32629]: 6
@[FreeColServer:~, 974]: 8
@[FreeColServer:A, 977]: 11
@[FreeColServer:A, 975]: 26
```

¹ Немного истории: я создал первую версию с именем threaded.d 25 июля 2005 года и использовал ее во время занятий по производительности. Там я написал два примера приложений с пулами рабочих потоков, один из которых включал проблему борьбы за блокировку, и на примере threaded.d показывал, как другие потоки не могли работать с этой проблемой. Представленную здесь версию я написал специально для этой книги.

```
@[C1 CompilerThre, 32618]: 29
@[C2 CompilerThre, 32617]: 44
@[C2 CompilerThre, 32616]: 44
@[C2 CompilerThre, 32615]: 48
```

[...]

Мы видим, что основное процессорное время в этом приложении расходуется на работу потоков компилятора.

threaded(8) выбирает стеки по времени. При выборке с такой низкой частотой оверхед должен быть незначительным.

Исходный код threadaded(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    if ($1 == 0) {
        printf("USAGE: threaded.bt PID\n");
        exit();
    }
    printf("Sampling PID %d threads at 99 Hertz. Ctrl-C to end.\n", $1);
}

profile:hz:99
/pid == $1/
{
    @[comm, tid] = count();
}

interval:s:1
{
    time();
    print(@);
    clear(@);
}
```

Этот инструмент ожидает получить аргумент с идентификатором процесса и просто завершается, если ничего не получил (по умолчанию \$1 получает нулевое значение).

13.2.5. offcputime

offcputime(8) был представлен в главе 6. Это инструмент для ВСС, который трассирует события приостановки потоков и запоминает продолжительность бездействия вместе с трассировками стека. Вот пример вывода для сервера MySQL:

```
# offcputime -d -p $(pgrep mysqld)
Tracing off-CPU time (us) of PID 9908 by user + kernel stack... Hit Ctrl-C to end.
```

[...]

```

finish_task_switch
schedule
jbd2_log_wait_commit
jbd2_complete_transaction
ext4_sync_file
vfs_fsync_range
do_fsync
sys_fsync
do_syscall_64
entry_SYSCALL_64_after_hwframe
--
fsync
fil_flush(unsigned long)
log_write_up_to(unsigned long, bool) [clone .part.56]
trx_commit_complete_for_mysql(trx_t*)
innobase_commit(handlerton*, THD*, bool)
ha_commit_low(THD*, bool, bool)
TC_LOG_DUMMY::commit(THD*, bool)
ha_commit_trans(THD*, bool, bool)
trans_commit(THD*)
mysql_execute_command(THD*, bool)
Prepared_statement::execute(String*, bool)
Prepared_statement::execute_loop(String*, bool, unsigned char*, unsigned char*)
mysqld_stmt_execute(THD*, unsigned long, unsigned long, unsigned char*, unsign...
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)
handle_connection
pfs_spawn_thread
start_thread
-                               mysqld (9962)
2458362

```

[...]

```

finish_task_switch
schedule
futex_wait_queue_me
futex_wait
do_futex
Sys_futex
do_syscall_64
entry_SYSCALL_64_after_hwframe
--
pthread_cond_timedwait@@GLIBC_2.3.2
__pthread_cond_timedwait
os_event::timed_wait(timespec const*)
os_event_wait_time_low(os_event*, unsigned long, long)
lock_wait_timeout_thread
start_thread
__clone
-                               mysqld (2311)
10000904

finish_task_switch
schedule

```

```

do_nanosleep
hrtimer_nanosleep
sys_nanosleep
do_syscall_64
entry_SYSCALL_64_after_hwframe
--
__nanosleep
os_thread_sleep(unsigned long)
srv_master_thread
start_thread
__clone
-                               mysqld (2315)
10001003

```

Я получил сотни трассировок стека, но здесь показал лишь некоторые из них. Первая трассировка соответствует выполнению инструкции `commit`, которая выполняет запись в журнал, а затем вызывает `fsync()`. После этого управление передается ядру (“--”), где файловая система `ext4` выполняет `fsync` и поток блокируется в функции `jbd2_log_wait_commit()`. Продолжительность блокировки `mysqld` здесь составила 2 458 362 микросекунды (2.45 секунды): это суммарное время для всех потоков.

Последние две трассировки показывают, что `lock_wait_timeout_thread()` ожидает событий, вызывая `pthread_cond_timewait()`, а `srv_master_thread()` приостанавливается преднамеренно. В выводе `offcputime(8)` часто могут преобладать такие ожидающие и преднамеренно приостанавливающиеся потоки, и обычно это нормальное поведение. Ваша задача — найти стеки, которые блокируются во время обработки запросов, что может быть признаком проблем.

Флейм-графики простоев

Создание флейм-графика простоев позволяет быстро выявить пути в коде, ведущие к блокировке, которые могут стать целью для оптимизации. Эти команды захватывают стеки простаивающего кода и используют мою программу для создания флейм-графика:

```

# offcputime -f -p $(pgrep mysqld) 10 > out.offcputime01.txt
# flamegraph.pl --width=800 --color=io --title="Off-CPU Time Flame Graph" \
  --countname=us < out.offcputime01.txt > out.offcputime01.svg

```

Получившийся график изображен на рис. 13.3. Здесь я использовал функцию поиска, чтобы выделить пурпурным цветом фреймы с именем функции «`do_command`»: это пути в коде, обрабатывающие запросы и блокирующие клиентов.

Большую часть графика на рис. 13.3 занимают пулы потоков, ожидающих работы. Время, когда сервер оставался заблокированным при обработке запросов, показано узкой пирамидой, которая включает фрейм `do_command()`, выделенный пурпурным цветом. Флейм-графики интерактивны, и можно щелкнуть на этой пирамиде, чтобы увеличить масштаб, — результат показан на рис. 13.4.

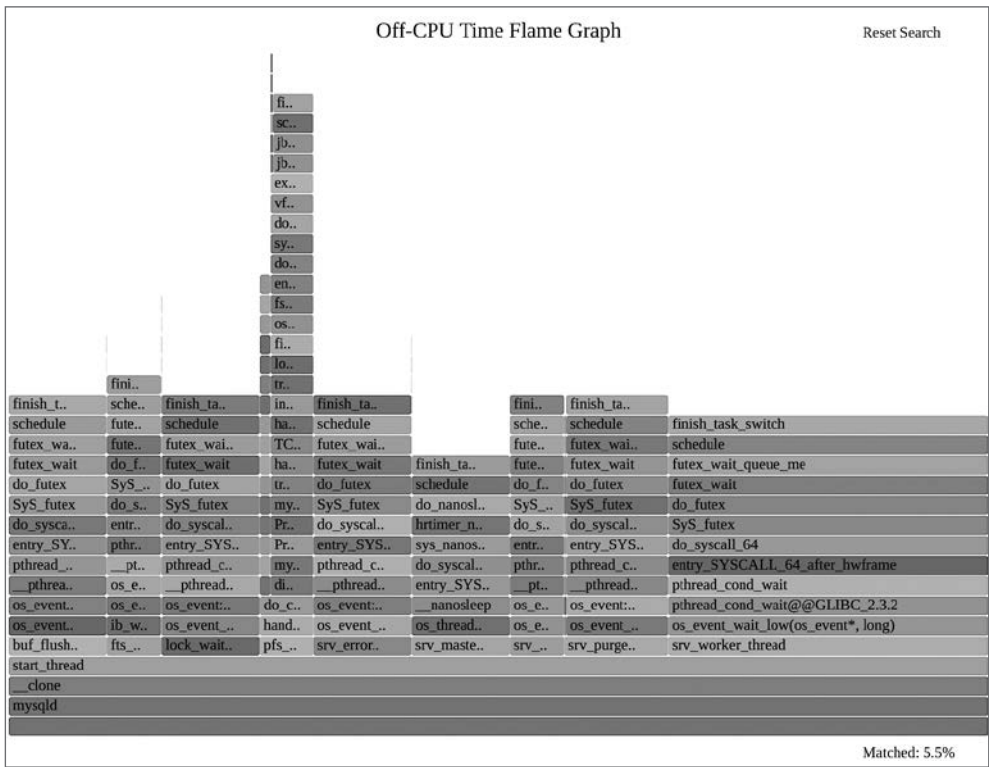


Рис. 13.3. Флейм-график простоев сервера MySQL, пурпурным выделена функция `do_command`

Я навел курсор на фрейм функции `ext4_sync_file()`, чтобы показать время, проведенное в этом пути кода (см. внизу на рис. 13.4): всего 3.95 секунды. Она ответственна за большую часть времени, когда `do_command()` оставалась заблокированной, и может стать целью для оптимизации, чтобы увеличить производительность сервера.

bpfftrace

Я написал версию `offcputime(8)` для `bpfftrace`. Ее исходный код приведен в следующем разделе, где идет речь о `offcputime(8)`.

Заключительные примечания

Эта возможность анализа простоев дополняет анализ потребления процессора с помощью `profile(8)`, и вместе эти инструменты помогут выявить широкий спектр проблем производительности.

Оверхед `offcputime(8)` может быть значительным и превышать 5%, в зависимости от частоты переключения контекста. Но эту проблему можно смягчить, запуская инструмент на короткое время. До BPF анализ простоев предполагал сброс всех трассировок

стека в пространстве пользователя и последующую их обработку, из-за чего оверхед часто оказывался недопустимым для использования в промышленной среде.

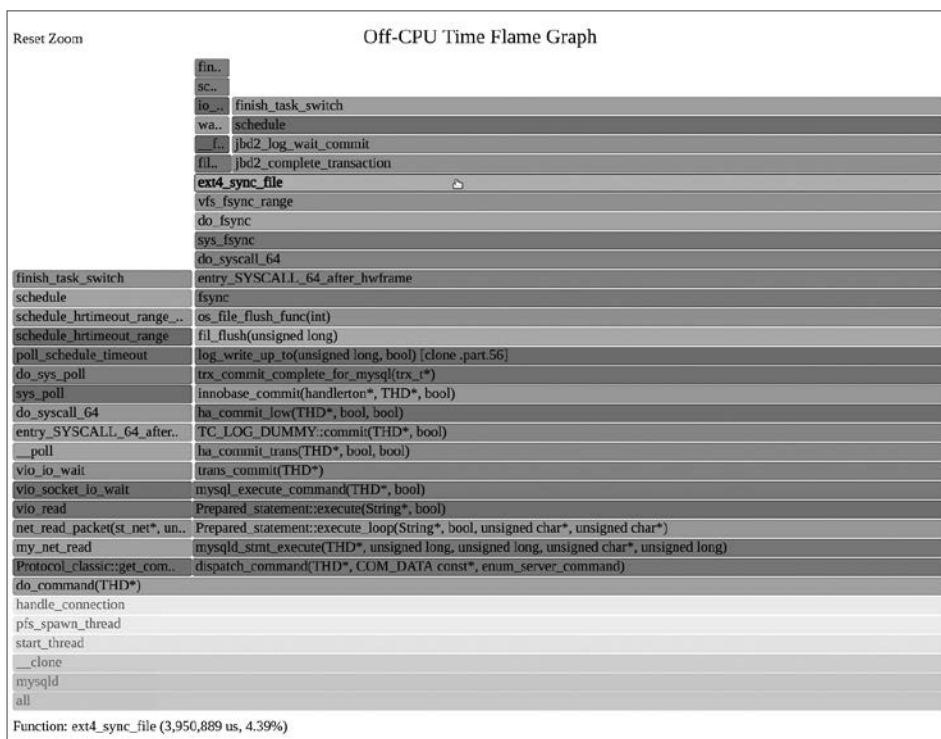


Рис. 13.4. Флейм-график простоев в увеличенном масштабе: пирамида соответствует обработке запросов

Как и в случае с `gprofile(8)`, в примерах выше создаются полные стеки для всего кода, потому что я скомпилировал сервер MySQL и системные библиотеки с поддержкой указателей фреймов. Подробнее об этом идет речь в разделе 13.2.9.

Более подробно `offcputime(8)` описан в главе 6. В главе 14 представлены дополнительные инструменты для анализа простоев: `wakeuptime(8)` и `offwaketime(8)`.

13.2.6. offcputhist

`offcputhist(8)`¹ действует как и `offcputime(8)`. Этот инструмент трассирует события планирования и фиксирует время простоя с трассировками стека, но отображает

¹ Немного истории: я написал его для этой книги 16 февраля 2019 года, взяв за основу свой инструмент `uoffcpu.d` из книги о DTrace 2011 года [Gregg 11], который отображал трассировки стеков, ведущих к простоям, с гистограммами. Это первый инструмент анализа простоев процессора, написанный для `bpfttrace`.

это время в виде гистограмм. Вот пример вывода, полученный в результате трассировки моего сервера MySQL:

```
# offcputhist.bt $(pgrep mysqld)
```

```
Attaching 3 probes...
```

```
Tracing nanosecond time in off-CPU stacks. Ctrl-C to end.
```

```
[...]
```

```
@[
```

```
    finish_task_switch+1
    schedule+44
    futex_wait_queue_me+196
    futex_wait+266
    do_futex+805
    Sys_futex+315
    do_syscall_64+115
    entry_SYSCALL_64_after_hwframe+61
```

```
,
```

```
    __pthread_cond_wait+432
    pthread_cond_wait@@GLIBC_2.3.2+36
    os_event_wait_low(os_event*, long)+64
    srv_worker_thread+503
    start_thread+208
    __clone+63
```

```
, mysqld]:
```

[2K, 4K)	134	@@@@@@	
[4K, 8K)	293	@@@@@@@@@@@@@@@@@@	
[8K, 16K)	886	@@	
[16K, 32K)	493	@@	
[32K, 64K)	447	@@	
[64K, 128K)	263	@@@@@@@@@@@@@@@@@@	
[128K, 256K)	85	@@@@	
[256K, 512K)	7		
[512K, 1M)	0		
[1M, 2M)	0		
[2M, 4M)	0		
[4M, 8M)	306	@@@@@@@@@@@@@@@@@@	
[8M, 16M)	747	@@	

```
@[
```

```
    finish_task_switch+1
    schedule+44
    schedule_hrtimeout_range_clock+185
    schedule_hrtimeout_range+19
    poll_schedule_timeout+69
    do_sys_poll+960
    sys_poll+155
    do_syscall_64+115
    entry_SYSCALL_64_after_hwframe+61
```

```
,
```

```
    __GI___poll+110
    vio_io_wait+141
```

```

vio_socket_io_wait+24
vio_read+226
net_read_packet(st_net*, unsigned long*)+141
my_net_read+412
Protocol_classic::get_command(COM_DATA*, enum_server_command*)+60
do_command(THD*)+192
handle_connection+680
pfs_spawn_thread+337
start_thread+208
__clone+63
, mysqld]:
[2K, 4K)          753 |@@@@@|
[4K, 8K)          2081 |@@@@@@@@@@@@@@@@@@@@|
[8K, 16K)         5759 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[16K, 32K)        3595 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[32K, 64K)        4045 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[64K, 128K)       3830 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[128K, 256K)      751 |@@@@@|
[256K, 512K)      48 |
[512K, 1M)        16 |
[1M, 2M)          0 |
[2M, 4M)          7 |

```

Здесь показаны только две последние трассировки стека. В первом случае видно бимодальное распределение простоев в потоках `srv_worker_thread()`, ожидающих работы: значения диапазонов указаны в наносекундах, соответственно, одна мода приходится на диапазон от 8 до 16 мс, а другая — на диапазон 8 до 16 мс («[8M, 16M]»). Во втором случае наблюдается множество коротких простоев в пути кода `net_read_packet()`, длящихся обычно менее 128 мс.

`offcpuhist(8)` трассирует события планировщика с помощью `kprobes`. Оверхед, как и в случае с `offcputime(8)`, бывает значительным, поэтому трассировка может выполняться только в течение коротких интервалов времени.

Исходный код `offcpuhist(8)`:

```

#!/usr/local/bin/bpftrace

#include <linux/sched.h>

BEGIN
{
    printf("Tracing nanosecond time in off-CPU stacks. Ctrl-C to end.\n");
}

kprobe:finish_task_switch
{
    // запомнить время приостановки потока
    $prev = (struct task_struct *)arg0;
    if ($1 == 0 || $prev->tgid == $1) {
        @start[$prev->pid] = nsecs;
    }
}

```

```

// получить время возобновления потока
$last = @start[tid];
if ($last != 0) {
    @[kstack, ustack, comm, tid] = hist(nsecs - $last);
    delete(@start[tid]);
}
}

END
{
    clear(@start);
}

```

Этот код записывает время для потока, покидающего процессор, а также гистограмму для потока, возобновившего выполнение, используя единственный зонд `kprobe finish_task_switch()`.

13.2.7. syscount

`syscount(8)`¹ — это инструмент для ВСС, подсчитывающий системные вызовы, что позволяет получить представление об использовании ресурсов приложением. Его можно запустить для трассировки системы в целом или отдельного процесса. Вот пример трассировки моего сервера MySQL с посекундным выводом результатов (`-i 1`):

```

# syscount -i 1 -p $(pgrep mysqld)
Tracing syscalls, printing top 10... Ctrl+C to quit.
[11:49:25]
SYSCALL                COUNT
sched_yield            10848
recvfrom               6576
futex                  3977
sendto                 2193
poll                   2187
pwrite                 128
fsync                  115
nanosleep              1

[11:49:26]
SYSCALL                COUNT
sched_yield            10918
recvfrom               6957
futex                  4165

```

¹ Немного истории: этот инструмент написал Саша Гольдштейн 15 февраля 2017 года. Свой первый инструмент `syscount` я написал 7 июля 2014 года и использовал в нем `perf(1)`. Первоначально я задумывал его как более дешевую версию `strace -s` с режимом для подсчета по процессам. За основу я частично взял свой инструмент `procsystime`, созданный 22 сентября 2005 года.

sendto	2314
poll	2309
pwrite	131
fsync	118
setsockopt	2
close	2
accept	1
[...]	

Мы видим, что чаще всего происходил системный вызов `sched_yield()` — более 10 000 раз в секунду. Наиболее частые системные вызовы можно исследовать с помощью соответствующих точек трассировки, а также с помощью этого и других инструментов. Например, воспользовавшись `stackcount(8)` из ВСС, мы получим трассировки стека, ведущие к системному вызову, а с помощью `argdist(8)` из ВСС исследуем его аргументы. Для каждого системного вызова должна быть страница в справочном руководстве, объясняющая его назначение, аргументы и возвращаемое значение.

`syscount(8)` также может отображать общее время, проведенное в системных вызовах, если запустить его с параметром `-l`. Вот пример трассировки в течение 10 секунд (`-d 10`) и суммирование в миллисекундах (`-m`):

```
# syscount -mL -d 10 -p $(pgrep mysqld)
Tracing syscalls, printing top 10... Ctrl+C to quit.
[11:51:40]
SYSCALL          COUNT      TIME (ms)
futex            42158     108139.607626
nanosleep         9         9000.992135
fsync            1176      4393.483111
poll             22700     1237.244061
sendto           22795      276.383209
recvfrom         68311     275.933806
sched_yield      79759     141.347616
pwrite           1352       53.346773
shutdown         1          0.015088
openat           1          0.013794
Detaching...
```

Суммарное время, проведенное в `futex(2)`, составило более 108 с, что может показаться странным для 10-секундной трассировки: это можно объяснить тем, что несколько потоков вызывали его параллельно. Для понимания функции `futex(2)` нужно исследовать аргументы и пути кода: вероятно, он вызывается так часто, потому что используется как механизм ожидания работы. Это предположение было подтверждено с помощью предыдущего инструмента `offcputime(8)`.

Еще одним интересным системным вызовом в этих результатах является `fsync(2)`, общая длительность работы которого составила 4393 мс. Это предполагает одну из возможных целей для оптимизации: файловую систему и устройства хранения.

Более подробно `syscount(8)` описан в главе 6.

13.2.8. ioprofile

ioprofile(8)¹ трассирует системные вызовы, выполняющие операции ввода/вывода — чтение, запись, отправку и прием, — и выводит их количество и соответствующие трассировки стека в пространстве пользователя. Пример трассировки сервера MySQL:

```
# ioprofile.bt $(pgrep mysqld)
Attaching 24 probes...
Tracing I/O syscall user stacks. Ctrl-C to end.
^C

[...]

@[tracepoint:syscalls:sys_enter_pwrite64,
  pwrite64+114
  os_file_io(IORequest const&, int, void*, unsigned long, unsigned long, dberr_t...
  os_file_write_page(IORequest&, char const*, int, unsigned char const*, unsigne...
  fil_io(IORequest const&, bool, page_id_t const&, page_size_t const&, unsigned ...
  log_write_up_to(unsigned long, bool) [clone .part.56]+2426
  trx_commit_complete_for_mysql(trx_t*)+108
  innobase_commit(handlerton*, THD*, bool)+727
  ha_commit_low(THD*, bool, bool)+372
  TC_LOG_DUMMY::commit(THD*, bool)+20
  ha_commit_trans(THD*, bool, bool)+703
  trans_commit(THD*)+57
  mysql_execute_command(THD*, bool)+6651
  Prepared_statement::execute(String*, bool)+1410
  Prepared_statement::execute_loop(String*, bool, unsigned char*, unsigned char*...
  mysqld_stmt_execute(THD*, unsigned long, unsigned long, unsigned char*, unsign...
  dispatch_command(THD*, COM_DATA const*, enum_server_command)+5582
  do_command(THD*)+544
  handle_connection+680
  pfs_spawn_thread+337
  start_thread+208
  __clone+63
, mysqld]: 636

[...]

@[tracepoint:syscalls:sys_enter_recvfrom,
  __GI___recv+152
  vio_read+167
  net_read_packet(st_net*, unsigned long*)+141
  my_net_read+412
  Protocol_classic::get_command(COM_DATA*, enum_server_command*)+60
  do_command(THD*)+192
```

¹ Немного истории: я написал его для этой книги 15 февраля 2019 года, намереваясь использовать для создания флейм-графиков с помощью новой программы Vector, которую использовал мой работодатель. Этот инструмент как никакой другой показал, насколько болезненным может быть отсутствие поддержки указателей фреймов в libc и libpthread, и это может вдохновить на изменение библиотек Netflix BaseAMI.

```

        handle_connection+680
        pfs_spawn_thread+337
        start_thread+208
        __clone+63
, mysql[d]: 24255

```

Я получил сотни трассировок стека, но здесь показал лишь две. Первая трассировка показывает, как `mysqld` вызывает `rwwrite64(2)`, чтобы зафиксировать транзакцию и выполнить запись в файл. Вторая показывает, как `mysqld` читает пакет из сети вызовом `recvfrom(2)`.

Выполнение слишком большого количества ненужных операций ввода/вывода — часто встречающаяся проблема, вызывающая снижение производительности приложений. Это могут быть операции записи в журнал, которые можно отключить, ввод/вывод небольшими блоками, которые можно увеличить, и т. д. Этот инструмент поможет выявить подобные проблемы.

`ioprofile(8)` использует точки трассировки системных вызовов. Оверхед этого инструмента может быть большим, потому что системные вызовы могут происходить часто.

Исходный код `ioprofile(8)`:

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing I/O syscall user stacks. Ctrl-C to end.\n");
}
tracepoint:syscalls:sys_enter_*read*,
tracepoint:syscalls:sys_enter_*write*,
tracepoint:syscalls:sys_enter_*send*,
tracepoint:syscalls:sys_enter_*recv*
/$1 == 0 || pid == $1/
{
    @[probe, ustack, comm] = count();
}

```

При желании можно передать необязательный аргумент PID. Без него будет трассироваться вся система.

13.2.9. Указатели фреймов в `libc`

Важно отметить, что вывод инструмента `ioprofile(8)` содержит только полные стеки, потому что испытуемый сервер MySQL работает с библиотекой `libc`, скомпилированной с поддержкой указателей фреймов. Приложения часто выполняют ввод/вывод через библиотеку `libc`, а та, в свою очередь, во многих дистрибутивах собирается без поддержки указателей фреймов. Из-за этого обход стека из ядра в приложение часто останавливается на `libc`. Эта проблема есть и в других инструментах, но в `ioprofile(8)` и в `brkstack(8)` из главы 7 она особенно заметна.

Вот как она проявляется: в этом примере сервер MySQL скомпилирован с поддержкой указателей фреймов, но использует библиотеку `libc`, поставляемую в составе дистрибутива:

```
# ioprofile.bt $(pgrep mysqld)
[...]
@[tracepoint:syscalls:sys_enter_pwrite64,
  __pwrite+79
  0x2fffffdcd020000
, mysqld]: 5
[...]
@[tracepoint:syscalls:sys_enter_recvfrom,
  __libc_recv+94
, mysqld]: 22526
```

Трассировки стека получились неполными и обрываются после одного или двух фреймов. Исправить проблему можно так:

- скомпилировать `libc` с параметром `-fno-omit-frame-pointer`;
- трассировать интерфейсные функции `libc` до повторного использования регистра указателя фрейма;
- трассировать функции сервера MySQL, например `os_file_io()`. Это решение ориентировано на конкретное приложение;
- использовать другое средство обхода стека (см. раздел 2.4, где перечислены другие возможности).

Библиотека `libc` находится в пакете `glibc` [153], который также включает `libpthread` и другие библиотеки. Уже вносилось предложение, чтобы Debian предоставлял альтернативный пакет `libc` с поддержкой указателей фреймов [154].

Более подробный обзор проблемы неполных стеков ищите в разделах 2.4 и 18.8.

13.2.10. `mysqld_qlower`

`mysqld_qlower(8)`¹ — это инструмент для BCC и `bpftool`, трассирующий запросы на сервере MySQL, которые обрабатываются дольше заданного порога. Он может служить примером инструмента, отображающего контекст приложения: строку запроса. Пример вывода версии для BCC:

```
# mysqld_qlower $(pgrep mysqld)
Tracing MySQL server queries for PID 9908 slower than 1 ms...
TIME(s)   PID      MS QUERY
0.000000  9962    169.032 SELECT * FROM words WHERE word REGEXP '^bre.*n$'
1.962227  9962    205.787 SELECT * FROM words WHERE word REGEXP '^bpf.tools$'
9.043242  9962     95.276 SELECT COUNT(*) FROM words
23.723025 9962    186.680 SELECT count(*) AS count FROM words WHERE word REGEXP
```

¹ Немного истории: я написал его для этой книги 15 февраля 2019 года, взяв за основу свой более ранний инструмент `mysqld_qlower.d` из книги DTrace 2011 года [Gregg 11].

```
'^bre.*n$'
30.343233  9962  181.494 SELECT * FROM words WHERE word REGEXP '^bre.*n$' ORDER BY
word
[...]
```

Инструмент выводит смещение во времени от начала трассировки, когда началась обработка запроса, идентификатор процесса (PID) сервера MySQL, продолжительность обработки запроса в миллисекундах и строку запроса. Такая информация доступна и в журнале MySQL, где регистрируются медленные запросы; однако BPF позволяет настроить этот инструмент для вывода сведений, которых нет в журнале, например, об операциях дискового ввода/вывода либо о других ресурсах, задействованных в процессе обработки запроса.

`mysqld_qlower(8)` использует зонды USDT в MySQL: `mysql:query__start` и `mysql:query__done`. Оверхед этого инструмента ожидается небольшим или незначительным из-за относительно низкой частоты следования запросов к серверу.

BCC

Порядок использования:

```
mysqld_qlower PID [min_ms]
```

В необязательном аргументе `min_ms` можно передать желаемое значение порога в миллисекундах. Если его нет, то берется порог по умолчанию, равный 1 миллисекунде. Если передать ноль, инструмент будет фиксировать все запросы.

bpfttrace

Ниже приведен исходный код версии для `bpfttrace`, разработанной специально для этой книги:

```
#!/usr/local/bin/bpfttrace
```

```
BEGIN
```

```
{
    printf("Tracing mysqld queries slower than %d ms. Ctrl-C to end.\n",
           $1);
    printf("%-10s %-6s %6s %s\n", "TIME(ms)", "PID", "MS", "QUERY");
}
```

```
usdt:/usr/sbin/mysqld:mysql:query__start
```

```
{
    @query[tid] = str(arg0);
    @start[tid] = nsecs;
}
```

```
usdt:/usr/sbin/mysqld:mysql:query__done
/@start[tid]/
```



```

{
    $dur = (nsecs - @start[tid]) / 1000000;
    if ($dur > $1) {
        printf("%-10u %-6d %6d %s\n", elapsed / 1000000,
            pid, $dur, @query[tid]);
    }
    delete(@query[tid]);
    delete(@start[tid]);
}

```

Эта программа принимает пороговое значение задержки в миллисекундах в позиционном параметре \$1. Если он не указан, по умолчанию используется нулевое значение и инструмент выводит информацию обо всех запросах.

Сервер MySQL использует пул сервисных потоков, и запрос от начала до конца обрабатывается одним потоком, поэтому для уникальной идентификации запросов я использовал идентификатор потока. Он играет роль ключа в картах @query и @start, где сохраняется указатель на строку запроса и отметка времени начала обработки, которые затем извлекаются, когда обработка запроса завершается.

Пример вывода:

```

# mysqld_qslower.bt -p $(pgrep mysqld)
Attaching 4 probes...
Tracing mysqld queries slower than 0 ms. Ctrl-C to end.
TIME(ms)  PID      MS QUERY
984       9908      87 select * from words where word like 'perf%'
[...]
```

Параметр -p нужен, чтобы включить зонды USDT. Как и в версии для BCC, требуется передать идентификатор процесса, поэтому порядок использования выглядит так:

```
mysqld_qslower.bt -p PID [min_ms]
```

bpfftrace: зонды uprobes

Если mysqld скомпилирован без зондов USDT, можно реализовать аналогичный инструмент, используя зонды uprobes во внутренних функциях. В трассировках стека, которые мы видели при обсуждении предыдущих команд, можно заметить несколько возможных функций для инструментации. Вот фрагмент вывода profile(8), показанного выше в этой главе:

```

handle_query(THD*, LEX*, Query_result*, unsigned long long, unsigned long long)
execute_sqlcom_select(THD*, TABLE_LIST*)
mysql_execute_command(THD*, bool)
Prepared_statement::execute(String*, bool)
Prepared_statement::execute_loop(String*, bool, unsigned char*, unsigned char*)
mysqld_stmt_execute(THD*, unsigned long, unsigned long, unsigned char*, unsigned...)
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)

```

Следующий инструмент — `mysqld_qslower-uprobes.bt` — трассирует функцию `dispatch_command()`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing mysqld queries slower than %d ms. Ctrl-C to end.\n",
        $1);
    printf("%-10s %-6s %6s %s\n", "TIME(ms)", "PID", "MS", "QUERY");
}

uprobe:/usr/sbin/mysqld:*dispatch_command*
{
    $COM_QUERY = 3;          // см. include/my_command.h
    if (arg2 == $COM_QUERY) {
        @query[tid] = str(*arg1);
        @start[tid] = nsecs;
    }
}

uretprobe:/usr/sbin/mysqld:*dispatch_command*
/@start[tid]/
{
    $dur = (nsecs - @start[tid]) / 1000000;
    if ($dur > $1) {
        printf("%-10u %-6d %6d %s\n", elapsed / 1000000,
            pid, $dur, @query[tid]);
    }
    delete(@query[tid]);
    delete(@start[tid]);
}
```

Функция `dispatch_command()` обрабатывает не только запросы, поэтому этот инструмент проверяет тип команды — `COM_QUERY`. Строка запроса извлекается из аргумента `COM_DATA`, где строка является первым членом структуры.

Как характерно для `uprobes`, имена трассируемых функций, аргументы и логика зависят от версии MySQL (здесь я трассировал версию MySQL 5.7), и этот инструмент может не работать с другими версиями, если какая-либо из указанных деталей изменится. Поэтому предпочтительнее использовать зонды `USDT`.

13.2.11. `mysqld_clat`

`mysqld_clat(8)`¹ — это инструмент для `bpftrace`, разработанный специально для этой книги. Он трассирует задержки обработки команд MySQL и выводит гистограмму для команд каждого типа. Например:

¹ Немного истории: я написал его для этой книги 15 февраля 2019 года. Он похож на мой инструмент `mysqld_command.d`, который я написал 25 июня 2013 года, но с некоторыми улучшениями: он использует общесистемные сводные данные и выводит удобочитаемые имена команд.

[illegible]

Как видите, обработка запросов занимала от 8 до 256 микросекунд, а у задержки выполнения операторов бимодальное распределение.

mysql_d_clat(8) измеряет время (задержки) между зондами `USDT mysql:command__start` и `mysql:command__done` и читает тип команды из зонда `mysql:command__start`. Оверхед инструмента должен быть незначительным, потому что скорость выполнения команд обычно невысока (менее тысячи в секунду).

Исходный код `mysqld` `clat(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing mysqld command latencies. Ctrl-C to end.\n");

    // из include/my_command.h:
    @com[0] = "COM SLEEP";
```

```

@com[1] = "COM_QUIT";
@com[2] = "COM_INIT_DB";
@com[3] = "COM_QUERY";
@com[4] = "COM_FIELD_LIST";
@com[5] = "COM_CREATE_DB";
@com[6] = "COM_DROP_DB";
@com[7] = "COM_REFRESH";
@com[8] = "COM_SHUTDOWN";
@com[9] = "COM_STATISTICS";
@com[10] = "COM_PROCESS_INFO";
@com[11] = "COM_CONNECT";
@com[12] = "COM_PROCESS_KILL";
@com[13] = "COM_DEBUG";
@com[14] = "COM_PING";
@com[15] = "COM_TIME";
@com[16] = "COM_DELAYED_INSERT";
@com[17] = "COM_CHANGE_USER";
@com[18] = "COM_BINLOG_DUMP";
@com[19] = "COM_TABLE_DUMP";
@com[20] = "COM_CONNECT_OUT";
@com[21] = "COM_REGISTER_SLAVE";
@com[22] = "COM_STMT_PREPARE";
@com[23] = "COM_STMT_EXECUTE";
@com[24] = "COM_STMT_SEND_LONG_DATA";
@com[25] = "COM_STMT_CLOSE";
@com[26] = "COM_STMT_RESET";
@com[27] = "COM_SET_OPTION";
@com[28] = "COM_STMT_FETCH";
@com[29] = "COM_DAEMON";
@com[30] = "COM_BINLOG_DUMP_GTID";
@com[31] = "COM_RESET_CONNECTION";
}

usdt:/usr/sbin/mysqld:mysql:command__start
{
    @command[tid] = arg1;
    @start[tid] = nsecs;
}

usdt:/usr/sbin/mysqld:mysql:command__done
/@start[tid]/
{
    $dur = (nsecs - @start[tid]) / 1000;
    @us[@com[@command[tid]]] = hist($dur);
    delete(@command[tid]);
    delete(@start[tid]);
}

END
{
    clear(@com);
}

```

Здесь определяется таблица для преобразования целочисленного идентификатора команды в удобочитаемую строку с именем команды. Имена взяты из заголовочного

файла `include/my_command.h` в исходном коде сервера MySQL. Они также задокументированы в справочнике по зондам USDT [155].

Если зонды USDT недоступны, этот инструмент можно переписать и использовать зонды `uprobes` в функции `dispatch_command()`. Я не буду воспроизводить весь код этой версии инструмента, а покажу только результат, который дает утилита `diff`:

```
$ diff mysqld_clat.bt mysqld_clat_uprobes.bt
42c42
< usdt:/usr/sbin/mysqld:mysql:command__start
---
> uprobe:/usr/sbin/mysqld:*dispatch_command*
44c44
< @command[tid] = arg1;
---
> @command[tid] = arg2;
48c48
< usdt:/usr/sbin/mysqld:mysql:command__done
---
> uretprobe:/usr/sbin/mysqld:*dispatch_command*
```

Команда извлекается из другого аргумента, и вместо зондов USDT используются зонды `uprobes`, но остальная часть инструмента осталась прежней.

13.2.12. signals

`signals(8)`¹ трассирует сигналы, посылаемые процессам, и обобщает распределение сигналов по типам и процессам-получателям. Это полезный инструмент поиска и устранения неисправностей для выяснения причин неожиданного завершения работы приложений, если это связано с отправкой сигнала. Пример вывода:

```
# signals.bt
Attaching 3 probes...
Counting signals. Hit Ctrl-C to end.
^C
@[SIGNAL, PID, COMM] = COUNT

@[SIGKILL, 3022, sleep]: 1
@[SIGINT, 2997, signals.bt]: 1
@[SIGCHLD, 21086, bash]: 1
@[SIGSYS, 3014, ServiceWorker t]: 4
@[SIGALRM, 2903, mpstat]: 6
@[SIGALRM, 1882, Xorg]: 87
```

Мы видим, что за время трассировки приостановленному процессу с идентификатором 3022 был послан единственный сигнал `SIGKILL` (его достаточно послать только один раз), в то время как процессу `Xorg` с идентификатором 1882 сигнал `SIGALRM` был послан в общей сложности 87 раз.

¹ Немного истории: я написал его для этой книги 16 февраля 2019 года. Есть и более ранние версии этого инструмента для других трассировщиков. Основой послужил инструмент `sig.d` из руководства «Dynamic Tracing Guide», изданного в январе 2005-го [Sun 05].

signals(8) использует точку трассировки signal:signal_generate. Поскольку она срабатывает нечасто, то ожидаемый оверхед будет незначительным.

Исходный код signals(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Counting signals. Hit Ctrl-C to end.\n");

    // из /usr/include/asm-generic/signal.h:
    @sig[0] = "0";
    @sig[1] = "SIGHUP";
    @sig[2] = "SIGINT";
    @sig[3] = "SIGQUIT";
    @sig[4] = "SIGILL";
    @sig[5] = "SIGTRAP";
    @sig[6] = "SIGABRT";
    @sig[7] = "SIGBUS";
    @sig[8] = "SIGFPE";
    @sig[9] = "SIGKILL";
    @sig[10] = "SIGUSR1";
    @sig[11] = "SIGSEGV";
    @sig[12] = "SIGUSR2";
    @sig[13] = "SIGPIPE";
    @sig[14] = "SIGALRM";
    @sig[15] = "SIGTERM";
    @sig[16] = "SIGSTKFLT";
    @sig[17] = "SIGCHLD";
    @sig[18] = "SIGCONT";
    @sig[19] = "SIGSTOP";
    @sig[20] = "SIGTSTP";
    @sig[21] = "SIGTTIN";
    @sig[22] = "SIGTTOU";
    @sig[23] = "SIGURG";
    @sig[24] = "SIGXCPU";
    @sig[25] = "SIGXFSZ";
    @sig[26] = "SIGVTALRM";
    @sig[27] = "SIGPROF";
    @sig[28] = "SIGWINCH";
    @sig[29] = "SIGIO";
    @sig[30] = "SIGPWR";
    @sig[31] = "SIGSYS";
}

tracepoint:signal:signal_generate
{
    @[@sig[args->sig], args->pid, args->comm] = count();
}

END
{
    printf("\n@[SIGNAL, PID, COMM] = COUNT");
    clear(@sig);
}
```

Здесь определяется таблица для преобразования номеров сигналов в удобочитаемые имена. В ядре не определено имя для сигнала с номером 0. Однако он широко используется для проверки работоспособности процессов.

13.2.13. killsnoop

`killsnoop(8)`¹ — это инструмент для BCC и `bpfttrace`. Он трассирует сигналы, посылаемые через системный вызов `kill(2)`, и может показать отправителя сигнала. Однако в отличие от `signals(8)`, он трассирует не все сигналы, а только те, что посылаются через `kill(2)`. Пример вывода:

```
# killsnoop
TIME      PID      COMM          SIG  TPID  RESULT
00:28:00  21086    bash          9    3593   0
[...]
```

Здесь видно, что командная оболочка `bash` послала сигнал 9 (KILL) процессу с идентификатором 3593.

`killsnoop(8)` использует точки трассировки `syscalls:sys_enter_kill` и `syscalls:sys_exit_kill`. Оверхед должен быть незначительным.

BCC

Порядок использования:

```
killsnoop [options]
```

Параметры `options`:

- **-x**: трассировать только неудачные попытки обратиться к системному вызову `kill`;
- **-p PID**: трассировать только этот процесс.

bpfttrace

Ниже приведена реализация `killsnoop(8)` для `bpfttrace`, где обобщены основные возможности инструмента. Эта версия не поддерживает параметры.

```
#!/usr/local/bin/bpfttrace
```

```
BEGIN
{
    printf("Tracing kill() signals... Hit Ctrl-C to end.\n");
    printf("%-9s %-6s %-16s %-4s %-6s %s\n", "TIME", "PID", "COMM", "SIG",
        "TPID", "RESULT");
```

¹ Немного истории: первую версию — `kill.d` — я написал 9 мая 2004 года для отладки загадочного завершения одного приложения. Версию для BCC я написал 20 сентября 2015 года, а версию для `bpfttrace` — 7 сентября 2018 года.

[illegible]

Последние две трассировки стека соответствуют задержке на блокировке с адресом 0x7f37280019f0. В обоих случаях путь выполнения кода протекает через функцию `THD::set_query()`, а время ожидания на блокировке обычно составляет от 4 до 16 микросекунд.

Теперь запустим `rmheld(8)`:

```
# pmheld.bt $(pgrep mysqld)
Attaching 5 probes...
Tracing libpthread mutex held times, Ctrl-C to end.
^C
[...]
```

```
@held_time_ns[0x7f37280019c0,  
__pthread_mutex_unlock+0  
close_thread_table(THD*, TABLE**)+169  
close_thread_tables(THD*)+923  
mysql_execute_command(THD*, bool)+887  
Prepared_statement::execute(String*, bool)+1410  
, mysqlid]:  
[2K, 4K)          3311 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
[4K, 8K)          4523 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

```
@held_time_ns[0x7f37280019f0,
__pthread_mutex_unlock+0
THD::set_query(st_mysql_const_lex_string const&)+147
dispatch_command(THD*, COM_DATA const*, enum_server_command)+1045
do_command(THD*)+544
```

```

    handle_connection+680
, mysql]:
[2K, 4K)          3848 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4K, 8K)          5038 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8K, 16K)         0 | |
[16K, 32K)        0 | |
[32K, 64K)        1 | |

@held_time_ns[0x7f37280019c0,
__pthread_mutex_unlock+0
Prepared_statement::execute(String*, bool)+321
Prepared_statement::execute_loop(String*, bool, unsigned char*, unsigned char*...
mysql_stmt_execute(THD*, unsigned long, unsigned long, unsigned char*, unsign...
dispatch_command(THD*, COM_DATA const*, enum_server_command)+5582
, mysql]:
[1K, 2K)          2204 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[2K, 4K)          4803 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4K, 8K)          2845 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8K, 16K)         0 | |
[16K, 32K)        11 | |

```

Мы видим пути в коде, удерживавшие одну и ту же блокировку, и продолжительность ее удержания в виде гистограммы.

Учитывая эти данные, можно предпринять различные действия: размер пулов потоков может быть настроен для уменьшения числа блокировок, а разработчик может просмотреть пути кода, удерживающие блокировку, и оптимизировать их, чтобы они удерживали блокировку более короткое время.

Результаты, возвращаемые этими инструментами, лучше выводить в файлы для последующего анализа. Например:

```

# pmlock.bt PID > out.pmlock01.txt
# pmheld.bt PID > out.pmhld01.txt

```

Оба инструмента принимают необязательный параметр PID, позволяющий ограничиться трассировкой определенного процесса, что также уменьшает оверхед. Без этого параметра инструменты фиксируют все события блокировки в libpthread в масштабе всей системы.

Используя uprobes и uretprobes, pmlock(8) и pmheld(8) инструментируют функции в библиотеке libpthread: pthread_mutex_lock() и pthread_mutex_unlock(). Оверхед бывает значительным, потому что события блокировки могут происходить очень часто. Например, подсчитаем, сколько таких событий возникает в течение 1 секунды с помощью funccount:

```

# funccount -d 1 '/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_*lock'
Tracing 4 functions for
"/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_*lock"... Hit Ctrl-C to end.

FUNC                                COUNT
pthread_mutex_trylock               4525
pthread_mutex_lock                   44726
pthread_mutex_unlock                 49132

```

При такой частоте добавление к каждому вызову даже малого оверхеда приведет к заметному снижению производительности.

pmlock

Исходный код `pmlock(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing libpthread mutex lock latency, Ctrl-C to end.\n");
}

uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock
/$1 == 0 || pid == $1/
{
    @lock_start[tid] = nsecs;
    @lock_addr[tid] = arg0;
}

uretprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock
/($1 == 0 || pid == $1) && @lock_start[tid]/
{
    @lock_latency_ns[usym(@lock_addr[tid]), ustack(5), comm] =
        hist(nsecs - @lock_start[tid]);
    delete(@lock_start[tid]);
    delete(@lock_addr[tid]);
}

END
{
    clear(@lock_start);
    clear(@lock_addr);
}
```

Эта программа запоминает отметку времени и адрес блокировки на входе в `pthread_mutex_lock()`, затем извлекает их, когда ожидание заканчивается, чтобы вычислить задержку и сохранить ее с адресом блокировки и трассировкой стека. При желании вызов `ustack(5)` можно изменить, чтобы сохранить другое количество фреймов.

Может потребоваться изменить путь `/lib/x86_64-linux-gnu/libpthread.so.0` к библиотеке в вашей системе. Трассировки стека могут не сохраняться, если трассируемое ПО и библиотека `libpthread` скомпилированы без поддержки указателей фреймов. (Если без указателей фреймов скомпилирована только `libpthread`, то трассировки стека могут сохраняться как точки входа в библиотеку и дальше, пока регистр указателя фрейма не будет использован для других целей.)

Инструмент не измеряет задержку в `pthread_mutex_trylock()`, потому что предполагается, что эта функция выполняется очень быстро, в чем и заключается ее основная задача. (В этом можно убедиться с помощью `funclatency(8)`.)

pmheld

Исходный код `pmheld(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing libpthread mutex held times, Ctrl-C to end.\n");
}

uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock,
uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_trylock
/$1 == 0 || pid == $1/
{
    @lock_addr[tid] = arg0;
}

uretprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock
/($1 == 0 || pid == $1) && @lock_addr[tid]/
{
    @held_start[pid, @lock_addr[tid]] = nsecs;
    delete(@lock_addr[tid]);
}

uretprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_trylock
/retval == 0 && ($1 == 0 || pid == $1) && @lock_addr[tid]/
{
    @held_start[pid, @lock_addr[tid]] = nsecs;
    delete(@lock_addr[tid]);
}

uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_unlock
/($1 == 0 || pid == $1) && @held_start[pid, arg0]/
{
    @held_time_ns[usym(arg0), ustack(5), comm] =
        hist(nsecs - @held_start[pid, arg0]);
    delete(@held_start[pid, arg0]);
}

END
{
    clear(@lock_addr);
    clear(@held_start);
}
```

Этот инструмент отсчитывает время от момента возврата из функции `pthread_mutex_lock()` или `pthread_mutex_trylock()`, то есть когда вызывающая сторона приобрела блокировку, до момента вызова `unlock()`.

Оба инструмента используют зонды `uprobes`, но библиотека `libpthread` имеет также зонды `USDT`, а значит, `pmlock(8)` и `pmheld(8)` можно переписать так, чтобы они инструментировали эти зонды.

13.2.15. nptime

`nptime(8)`¹ трассирует системный вызов `nanosleep(2)` и показывает, какие процессы к нему обращаются и на какой интервал они приостанавливаются. Я написал этот инструмент, чтобы решить проблему с медленной работой внутреннего процесса сборки, который мог работать минутами без видимого результата. Я подозревал, что он по каким-то причинам приостанавливает сам себя. Вывод:

```
# nptime.bt
Attaching 2 probes...
Tracing sleeps. Hit Ctrl-C to end.
TIME      PPID    PCOMM          PID    COMM          SECONDS
19:09:19  1        systemd        1975   iscsid         1.000
19:09:20  1        systemd        2274   mysqld         1.000
19:09:20  1        systemd        1975   iscsid         1.000
19:09:21  2998     build-init      25137  sleep         30.000
19:09:21  1        systemd        2274   mysqld         1.000
19:09:21  1        systemd        1975   iscsid         1.000
19:09:22  1        systemd        2421   irqbalance     9.999
[...]
```

В ходе трассировки выяснилось, что процесс `build-init` приостанавливался на 30 секунд. Благодаря этому инструменту я смог найти виновника, настроить приостановку и ускорить сборку в 10 раз. В выводе также видно, что потоки `mysqld` и `iscsid` приостанавливаются на 1 секунду через каждую секунду. (Такие приостановки `mysqld` мы наблюдали в выводе инструментов, представленных выше.) Иногда приложения могут использовать приостановки как обходное решение других проблем, и такие обходные решения могут оставаться в коде годами, вызывая проблемы с производительностью. `nptime(8)` помогает обнаружить эти проблемы.

`nptime(8)` использует точку трассировки `syscalls:sys_enter_nanosleep` и, как предполагается, имеет незначительный оверхед.

Исходный код `nptime(8)`:

```
#!/usr/local/bin/bpftrace

#include <linux/time.h>
#include <linux/sched.h>

BEGIN
{
    printf("Tracing sleeps. Hit Ctrl-C to end.\n");
    printf("%-8s %-6s %-16s %-6s %-16s %s\n", "TIME", "PPID", "PCOMM",
```

¹ Немного истории: я написал его для этой книги 16 февраля 2019 года, взяв за основу пример `Sys_nanosleep()` Саши Гольдштейна из `trace(8)`, и использовал для выяснения причин описанной здесь медленной сборки — сборки внутреннего пакета `nflx-bpftrace`, разработкой которого я занимался.

```

        "PID", "COMM", "SECONDS");
    }

    tracepoint:syscalls:sys_enter_nanosleep
    /args->rqltp->tv_sec + args->rqltp->tv_nsec/
    {
        $task = (struct task_struct *)curtask;
        time("%H:%M:%S ");
        printf("%-6d %-16s %-6d %-16s %d.%03d\n", $task->real_parent->pid,
            $task->real_parent->comm, pid, comm,
            args->rqltp->tv_sec, args->rqltp->tv_nsec / 1000000);
    }

```

Информация о вызывающем процессе извлекается из `task_struct`, но этот способ нестабилен. Поэтому может потребоваться изменить исходный код, если `task_struct` изменится.

Инструмент можно расширить: добавить трассировку стека в пространстве пользователя, чтобы показать путь кода, который привел к приостановке (при условии, что код скомпилирован с поддержкой указателей фреймов).

13.2.16. Другие инструменты

`deadlock(8)`¹ — еще один инструмент для ВСС, способный обнаруживать потенциальные взаимоблокировки с использованием мьютексов. Он строит ориентированный граф, отражающий использование мьютексов, и с помощью этого графа обнаруживает возможные взаимоблокировки. Инструмент может иметь высокий оверхед, но он помогает решить сложную проблему.

13.3. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечислены однострочные сценарии для ВСС и `bpftrace`. Там, где это возможно, один и тот же сценарий реализуется с использованием ВСС и `bpftrace`.

13.3.1. ВСС

Трассирует запуск новых процессов и их аргументы:

```
execsnoop
```

Выводит число системных вызовов, выполненных каждым процессом:

```
syscount -P
```

Выводит число обращений к каждому системному вызову:

```
syscount
```

¹ `deadlock(8)` был разработан Кенни Ю (Kenny Yu) 1 февраля 2017 года.

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
profile -U -F 49 -p 189
```

Подсчитывает трассировки стека, ведущие к приостановке процесса:

```
stackcount -U t:sched:sched_switch
```

Выбирает все трассировки стека и имена процессов:

```
profile
```

Подсчитывает количество вызовов в секунду функций из библиотеки libpthread для блокировки мьютексов:

```
funccount -d 1 '/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock'
```

Подсчитывает количество вызовов в секунду функций из библиотеки libpthread для доступа к условным переменным:

```
funccount -d 1 '/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_*
```

13.3.2. bpftrace

Трассирует запуск новых процессов и их аргументы:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Выводит число системных вызовов, выполненных каждым процессом:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Выводит число обращений к каждому системному вызову:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для процесса с именем «mysqld»:

```
bpftrace -e 'profile:hz:49 /comm == "mysqld"/ { @[ustack] = count(); }'
```

Подсчитывает трассировки стека, ведущие к приостановке процесса:

```
bpftrace -e 'tracepoint:sched:sched_switch { @[ustack] = count(); }'
```

Выбирает все трассировки стека и имена процессов:

```
bpftrace -e 'profile:hz:49 { @[ustack, stack, comm] = count(); }'
```

Суммирует объем памяти в байтах, выделяемой вызовом `malloc()`, с группировкой по трассировкам стека в пространстве пользователя (имеет высокий оверхед):

```
bpfftrace -e 'u:/lib/x86_64-linux-gnu/libc-2.27.so:malloc { @[ustack(5)] =
    sum(arg0); }'
```

Трассирует вызовы `kill()` для отправки сигнала и отображает имя процесса-отправителя, идентификатор (PID) получателя и номер сигнала:

```
bpfftrace -e 't:syscalls:sys_enter_kill { printf("%s -> PID %d SIG %d\n", comm,
    args->pid, args->sig); }'
```

Подсчитывает количество вызовов в секунду функций из библиотеки `libpthread` для блокировки мьютексов:

```
bpfftrace -e 'u:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock {
    @[probe] = count(); } interval:s:1 { exit(); }'
```

Подсчитывает количество вызовов в секунду функций из библиотеки `libpthread` для доступа к условным переменным:

```
bpfftrace -e 'u:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_* {
    @[probe] = count(); } interval:s:1 { exit(); }'
```

Подсчитывает количество промахов кэша последнего уровня по именам процессов:

```
bpfftrace -e 'hardware:cache-misses: { @[comm] = count(); }'
```

13.4. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ ОДНОСТРОЧНЫХ СЦЕНАРИЕВ BPF

Для иллюстрации работы однострочных сценариев полезно увидеть примеры их вывода. Ниже приведены некоторые однострочные сценарии с примерами вывода.

13.4.1. Подсчет количества вызовов в секунду функций из библиотеки `libpthread` для доступа к условным переменным

```
# bpfftrace -e 'u:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_* {
    @[probe] = count(); } interval:s:1 { exit(); }'
Attaching 19 probes...
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_wait@@GLIBC_2.3.2]: 70
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_wait]: 70
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_init@@GLIBC_2.3.2]: 573
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_
    timedwait@@GLIBC_2.3.2]: 673
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_destroy@@GLIBC_2.3.2]:
    939
```



```
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_
broadcast@@GLIBC_2.3.2]: 1796
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_broadcast]: 1796
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_signal]: 4600
@[uprobe:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_signal@@GLIBC_2.3.2]:
4602
```

Эти функции из библиотеки `libpthread` могут вызываться очень часто, и чтобы уменьшить оверхед, трассировка выполняется в течение только 1 секунды. Эти результаты показывают, как используются условные переменные (Conditional Variables, CV). Здесь мы видим две категории потоков: ожидающие на условных переменных и посылающие сигналы или широковещательные рассылки, чтобы возобновить выполнение ожидающих потоков.

Этот однострочник можно расширить: например, добавить вывод имени процесса, трассировки стека, продолжительности ожидания и другие детали.

13.5. ИТОГИ

В этой главе я показал дополнительные инструменты BPF, не вписывающиеся в рамки предыдущих глав, ориентированных на трассировку ресурсов. Они применяются для анализа приложений, включая их контекст, использование потоков, сигналов, блокировок и функций приостановки выполнения. В качестве примера приложения я взял сервер MySQL и показал, как из BPF получить контекст запроса, используя зонды USDT и `uprobes`. По причине особой важности я снова показал приемы того, как анализировать потребление CPU приложением с помощью инструментов BPF.

Глава 14

ЯДРО

Ядро — это сердце системы и одновременно сложный программный комплекс. Ядро Linux использует множество различных стратегий планирования процессорного времени, распределения памяти, управления дисковым вводом/выводом и сетевым стеком. Как и в любом ПО, в ядре иногда возникают проблемы. В предыдущих главах я показал приемы инструментации ядра, которые помогают понять особенности поведения приложений. В этой главе те же приемы мы используем для анализа ПО ядра. Они будут полезны и для устранения неполадок ядра, и для помощи в разработке.

Цели обучения:

- продолжая анализ простоев CPU, научиться отслеживать пробуждения;
- выявлять потребителей памяти в ядре;
- выявлять конфликты мьютексов в ядре;
- отображать операции в очереди заданий.

Если вас интересует какая-то определенная подсистема, то сначала обратите внимание на инструменты из предыдущих глав. Вот перечень подсистем в Linux и соответствующих им глав:

- планирование процессов: глава 6;
- память: глава 7;
- файловые системы: глава 8;
- блочные устройства ввода/вывода: глава 9;
- сети: глава 10.

В главе 2 рассмотрены технологии трассировки, включая BPF, точки трассировки и kprobes. В этой главе основное внимание уделено исследованию ядра, а не ресурсов и рассмотрены дополнительные темы о ядре, не вошедшие в предыдущие главы. Сначала я расскажу об основах анализа ядра, затем о возможностях BPF. Я покажу примерную стратегию анализа и традиционные инструменты, включая Ftrace, а затем познакомлю вас с дополнительными инструментами BPF для анализа событий

пробуждения, выделения памяти и использования блокировок ядра, а также операций с тасклетами и очередями заданий.

14.1. ОСНОВЫ

Ядро управляет доступом к ресурсам и планирует выполнение процессов на процессорах. В предыдущих главах мы рассмотрели множество тем, касающихся ядра. В частности:

- в разделе 6.1.1 есть подразделы «Режимы работы процессоров» и «Планировщик»;
- в разделе 7.1.1 есть подразделы «Механизмы распределения памяти», «Страницы памяти и подкачка», «Демон подкачки» и «Кэширование и буферизация в файловой системе»;
- в разделе 8.1.1 есть подразделы «Стек ввода/вывода» и «Кэши файловых систем»;
- в разделе 9.1.1 есть подразделы «Стек блочного ввода/вывода» и «Планировщики ввода/вывода»;
- в разделе 10.1.1 есть подраздел «Сетевой стек», а также подразделы, посвященные проблемам масштабирования сетевых взаимодействий и работе протокола TCP.

В этой главе рассмотрим дополнительные возможности анализа ядра.

14.1.1. Основы ядра

Пробуждения

Когда потоки приостанавливаются, переходя в режим ожидания некоторого события, они обычно возобновляют работу с его появлением. Яркий пример — дисковый ввод/вывод: поток может заблокироваться в операции чтения из файловой системы на то время, пока выполняется дисковый ввод/вывод, после чего его выполнение будет возобновлено рабочим потоком, обрабатывающим прерывание завершения операции.

В некоторых случаях пробуждение потоков происходит по цепочке: один поток возобновляет другой, а тот, в свою очередь, — третий, и так далее, пока не возобновит работу само заблокированное приложение.

На рис. 14.1 показано, как при обращении к системному вызову поток приложения может блокироваться и затем пробуждаться по событию, генерируемому потоком управления ресурсом, возможно, через цепочку потоков-зависимостей.

Трассировка пробуждений может дать больше информации о времени простоя в ожидании события.

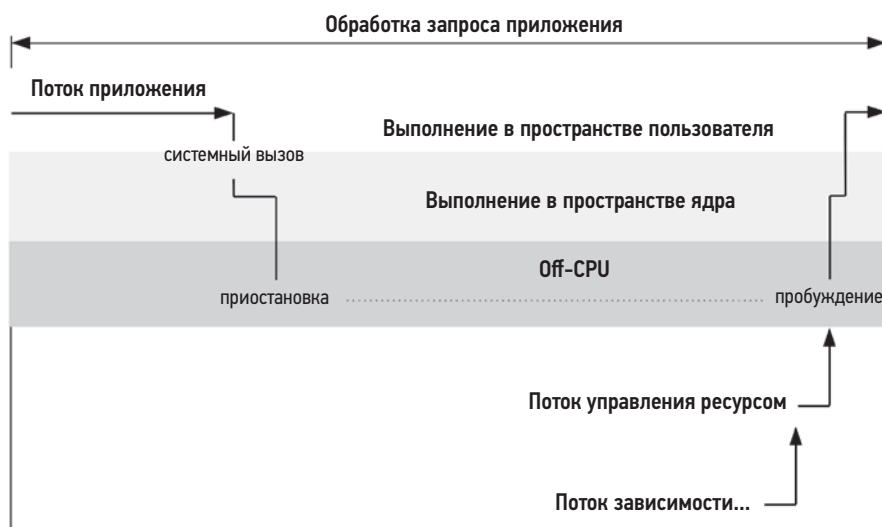


Рис. 14.1. Приостановка и пробуждение

Распределение памяти ядра

В ядре есть два основных механизма распределения памяти:

- **Распределитель блоков (slab allocator):** универсальный механизм распределения памяти для размещения объектов фиксированного размера. Поддерживает кэширование выделенных блоков памяти и их повторное использование для большей эффективности. В Linux вместо него используется механизм с названием «slub allocator»: принцип его действия заимствован из статьи с описанием slab allocator [Bonwick 94], но имеет меньшую сложность.
- **Распределитель страниц (page allocator):** предназначен для распределения памяти страницами. Использует алгоритм близнецов (buddy algorithm), суть которого заключается в поиске непрерывной последовательности страниц свободной памяти, чтобы их можно было выделить вместе. Этот метод распределения памяти также известен как NUMA (Non-Uniform Memory Access — неоднородный доступ к памяти).

Эти механизмы распределения упоминались в главе 7, где шла речь о необходимых для анализа использования памяти приложением основах. Эта глава посвящена анализу использования памяти ядром.

Программный интерфейс распределения памяти в ядре включает: `kmalloc()`, `kzalloc()` и `kmem_cache_alloc()` (распределение блоками) — для выделения небольших фрагментов памяти, `vmalloc()` и `vzalloc()` — для выделения больших фрагментов и `alloc_pages()` — для выделения памяти страницами [156].

Блокировки в ядре

Блокировки, действующие в пространстве пользователя, рассматривались в главе 13. Ядро поддерживает несколько типов блокировок: циклические, или спин-блокировки, мьютексы и блокировки чтения/записи. Поскольку блокировки приостанавливают работу потоков, они отрицательно сказываются на производительности.

Мьютексы в ядре Linux — это гибридные блокировки с тремя путями получения, которые проверяются в следующем порядке [157]:

1. **Быстрый путь:** с использованием инструкции «сравнить и обменять» (compare-and-swap, cmpxchg).
2. **Промежуточный путь:** выполняется цикл ожидания блокировки в оптимистичной надежде, что она вот-вот освободится.
3. **Долгий путь:** приостановка до момента, когда блокировка освободится.

Есть и механизм синхронизации чтения-копирования-обновления (Read-Copy-Update, RCU), который позволяет выполнять несколько операций чтения одновременно с изменением, увеличивая производительность и масштабируемость при работе с данными, которые редко изменяются и часто читаются.

Тасклеты и очереди заданий

Драйверы устройств в Linux следуют модели «двух половин», согласно которой верхняя половина быстро обрабатывает прерывания и создает задания для нижней половины, которые будут обработаны позже [Corbet 05]. Быстрая обработка прерываний чрезвычайно важна, потому что верхняя половина работает в режиме с отключенными прерываниями, чтобы отложить доставку новых прерываний и обеспечить их последовательную обработку. Это может вызвать проблемы с задержкой, если прерывания будут обрабатываться слишком долго. Нижняя половина может быть реализована как тасклет или как очередь заданий, последняя — это поток, который может запускаться и приостанавливаться ядром при необходимости (рис. 14.2).

14.1.2. Возможности BPF

Инструменты трассировки BPF помогают получить дополнительную информацию о работе ядра и ответить на вопросы:

- Почему потоки оставляют процессор и как долго они простаивают?
- Каких событий ждут простаивающие потоки?
- Кто сейчас использует распределитель блоков?
- Перемещает ли ядро страницы памяти для балансировки NUMA?

- Какие события обрабатываются в очереди заданий? С какими задержками?
- Для разработчиков ядра: какие из моих функций вызываются? С какими аргументами? Какие значения возвращаются? С какой задержкой?

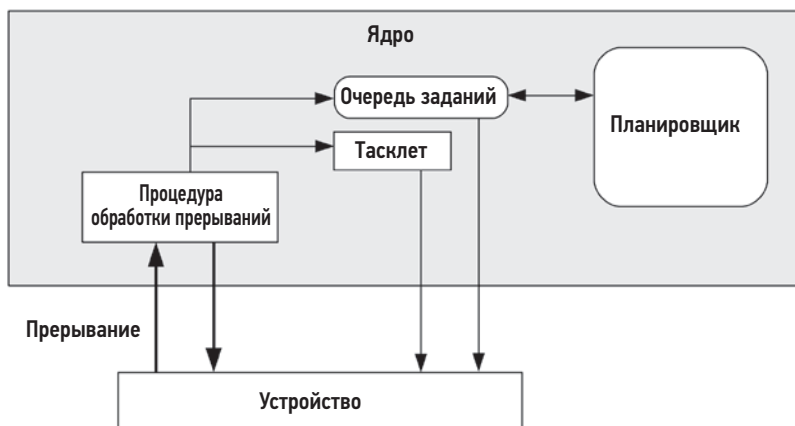


Рис. 14.2. Тасклеты и очереди заданий

Получить эти ответы можно, инструментируя точки трассировки и функции ядра для измерения их задержек, аргументов и получения трассировок стека. Также можно использовать временную выборку трассировки стека для просмотра путей выполнения кода на процессоре. Обычно это работает, поскольку ядро компилируется с поддержкой стека (либо указатели фреймов, либо ORC).

Источники событий

В табл. 14.1 перечислены типы событий в ядре и их источники для инструментации.

Таблица 14.1. Типы событий в ядре и их источники для инструментации

Типы событий	Источники событий
Функции ядра	kprobes
События планировщика	Точки трассировки планировщика (sched)
Системные вызовы	Точки трассировки системных вызовов (syscalls и raw_syscalls)
Выделение памяти в пространстве ядра	Точки трассировки операций с памятью в ядре (kmem)
Выгрузка страниц демоном сканирования	Точки трассировки vmscan
Прерывания	Точки трассировки прерываний (irq и irq_vectors)
Работа очереди заданий	Точки трассировки очереди заданий (workqueue)

Типы событий	Источники событий
Таймеры	Точки трассировки таймеров (timer)
События включения/отключения прерываний и вытеснения	Точки трассировки preemptirq ¹

Проверьте, какие еще точки трассировки поддерживает ваше ядро. Сделать это можно с помощью bpftrace:

```
bpftrace -l 'tracepoint:*
```

```
или perf(1):
```

```
perf list tracepoint
```

События, касающиеся ресурсов, включая блочный и сетевой ввод/вывод, рассматривались в предыдущих главах.

14.2. СТРАТЕГИЯ

Если вы новичок в анализе производительности ядра, вам поможет описанная здесь общая стратегия. В разделах ниже более подробно описаны соответствующие инструменты.

1. Если возможно, создайте рабочую нагрузку, которая запускает интересующие вас события; желательно, чтобы запуск происходил известное количество раз. Для этого может потребоваться написать короткую программу на C.
2. Проверьте наличие точек трассировки для инструментации интересующих событий, а также инструментов (включая те, что описаны в этой главе).
3. Если событие может возникать слишком часто и потреблять значительные вычислительные ресурсы (> 5%), для быстрого определения используемых функций ядра можно применить прием профилирования процессора. В противном случае используйте более продолжительное профилирование, чтобы собрать достаточное количество образцов для исследования (например, с помощью perf(1) или profile(8) из BCC (8) и с флейм-графиками потребления процессора). Профилирование процессора также позволит выявить оптимистичные циклы ожидания спин-блокировок и мьютексов.
4. Еще один способ найти интересующие функции в ядре — подсчитать вызовы функций и обратить внимание на те, количество вызовов которых соответствует количеству появлений события. Например, при анализе событий в файловой системе ext4 можно попробовать подсчитать вызовы функций с именами «ext4_*» (используя funcckcount(8) из BCC).

¹ Ядро должно быть скомпилировано с параметром CONFIG_PREEMPTIRQ_EVENTS.

5. Подсчитайте трассировки стека от функций ядра, чтобы исследовать пути, которыми код приходит к ним (с помощью `stackcount(8)` из ВСС). Эти пути уже должны быть известны, если выполнялось профилирование.
6. Выполните трассировку вызовов функции через дочерние события (с использованием `funcgraph(8)` из набора инструментов `perf-tools` на основе `Ftrace`).
7. Исследуйте аргументы функции (с помощью `trace(8)` и `argdist(8)` из ВСС или `bpfttrace`).
8. Измерьте задержку функции (с помощью `funclatency(8)` из ВСС или `bpfttrace`).
9. Напишите свой инструмент для инструментации событий, который выводит или обобщает информацию о них.

Далее показано, как выполнить некоторые из этих шагов с помощью традиционных инструментов, которые можно попробовать, прежде чем обращаться к инструментам BPF.

14.3. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Многие традиционные инструменты уже рассматривались в предыдущих главах. В этом разделе мы познакомимся с дополнительными инструментами для анализа ядра. Они перечислены в табл. 14.2.

Таблица 14.2. Традиционные инструменты

Инструмент	Тип	Описание
<code>Ftrace</code>	Трассировка	Трассировщик, встроенный в ядро Linux
<code>perf sched</code>	Трассировка	Официальный профилировщик Linux: анализ команд планировщика
<code>slabtop</code>	Статистика ядра	Статистика использования кэша блоков памяти в ядре

14.3.1. Ftrace

`Ftrace`¹ был создан Стивеном Ростедтом и добавлен в Linux 2.6.27 в 2008 году. Как и `perf(1)`, `Ftrace` — это многофункциональный инструмент, обладающий широкими возможностями. Есть как минимум четыре способа использования `Ftrace`:

1. Посредством файлов `/sys/kernel/debug/tracing` с использованием команд `cat(1)` и `echo(1)` или программ на языках высокого уровня. Этот способ описан в исходных кодах ядра, в файле `Documentation/trace/ftrace.rst` [158].
2. Посредством программы `trace-cmd`, написанной Стивеном Ростедтом [159], [160].

¹ Его название часто пишут как «ftrace», но Стивен рекомендует использовать стандартное название «Ftrace». (Я специально наводил справки для этой книги.)

3. Посредством программы с графическим интерфейсом KernelShark, написанной Стивеном Ростедтом с соавторами [161].
4. Посредством инструментов из коллекции perf-tools, написанной мной [78]. Это сценарии на языке командной оболочки, использующие файлы /sys/kernel/debug/tracing.

Я продемонстрирую возможности Ftrace с помощью perf-tools, но можно использовать любой из этих методов.

Подсчет вызовов функций

Допустим, я решил проанализировать работу механизма упреждающего чтения в файловой системе в ядре. Для начала можно подсчитать вызовы всех функций, содержащих «readahead» в своем имени, например, с помощью funccount(8) (из коллекции perf-tools), создав рабочую нагрузку, которая, как ожидается, вызовет нужные мне события:

```
# funccount '*readahead*'
Tracing '*readahead*'... Ctrl-C to end.
^C
FUNC                                COUNT
page_cache_async_readahead         12
__breadahead                        33
page_cache_sync_readahead           69
ondemand_readahead                  81
__do_page_cache_readahead           83
Ending tracing...
```

В результате получился список из пяти функций с количеством их вызовов.

Подсчет трассировок стека

Следующий шаг — узнать больше об этих функциях. Ftrace может собирать трассировки стека для событий, чтобы потом по ним выяснить, почему была вызвана та или иная функция, то есть определить их родительские функции. Проанализируем первую функцию из предыдущего вывода с помощью kprobe(8):

```
# kprobe -hs 'p:page_cache_async_readahead'
Tracing kprobe page_cache_async_readahead. Ctrl-C to end.
# tracer: nop
#
#          _-----> irqsoft
#          / _-----> need-resched
#          | / _-----> hardirq/softirq
#          || / _-----> preempt-depth
#          ||| / _-----> delay
#
# TASK-PID   CPU#  | |||   TIMESTAMP  FUNCTION
#   | |       |   |   |   |
cksum-32372 [006] .... 1952191.125801: page_cache_async_readahead:
```

```
(page_cache_async_readahead+0x0/0x80)
      cksum-32372 [006] .... 1952191.125822: <stack trace>
=> page_cache_async_readahead
=> ext4_file_read_iter
=> new_sync_read
=> __vfs_read
=> vfs_read
=> SyS_read
=> do_syscall_64
=> entry_SYSCALL_64_after_hwframe
      cksum-32372 [006] .... 1952191.126704: page_cache_async_readahead:
(page_cache_async_readahead+0x0/0x80)
      cksum-32372 [006] .... 1952191.126722: <stack trace>
=> page_cache_async_readahead
=> ext4_file_read_iter
[...]
```

Эта команда также выводит трассировку стека для события, откуда видно, что оно было инициировано системным вызовом `read()`. `kprobe(8)` позволяет исследовать аргументы и возвращаемое значение функции.

Для большей эффективности трассировки стека желательно подсчитывать в контексте ядра, а не выводить одну за другой. Для этого используйте новую возможность `Ftrace` — триггеры `hist` (сокращенно от `histogram` — гистограмма). Например:

```
# cd /sys/kernel/debug/tracing/
# echo 'p:kprobes/myprobe page_cache_async_readahead' > kprobe_events
# echo 'hist:key=stacktrace' > events/kprobes/myprobe/trigger
# cat events/kprobes/myprobe/hist
# event histogram
#
# trigger info: hist:keys=stacktrace:vals=hitcount:sort=hitcount:size=2048 [active]
#

{ stacktrace:
    ftrace_ops_assist_func+0x61/0xf0
    0xfffffffffc0e1b0d5
    page_cache_async_readahead+0x5/0x80
    generic_file_read_iter+0x784/0xbf0
    ext4_file_read_iter+0x56/0x100
    new_sync_read+0xe4/0x130
    __vfs_read+0x29/0x40
    vfs_read+0x8e/0x130
    SyS_read+0x55/0xc0
    do_syscall_64+0x73/0x130
    entry_SYSCALL_64_after_hwframe+0x3d/0xa2
} hitcount: 235

Totals:
  Hits: 235
  Entries: 1
  Dropped: 0
[...шаги для отмены состояния трассировки...]
```

Как показывает этот вывод, за время трассировки данный путь в коде использовался 235 раз.

Диаграммы вызовов функций

Наконец, с помощью `funcgraph(8)` можно узнать, какие дочерние функции вызывались:

```
# funcgraph page_cache_async_readahead
Tracing "page_cache_async_readahead"... Ctrl-C to end.
3) | page_cache_async_readahead() {
3) | | inode_congested() {
3) | | | dm_any_congested() {
3) 0.582 us | | | dm_request_based();
3) | | | dm_table_any_congested() {
3) | | | | dm_any_congested() {
3) | | | | | dm_request_based();
3) 0.267 us | | | | | dm_table_any_congested();
3) 1.824 us | | | | }
3) 4.604 us | | | }
3) 7.589 us | | }
3) + 11.634 us | }
3) + 13.127 us | }
3) | ondemand_readahead() {
3) | | __do_page_cache_readahead() {
3) | | | __page_cache_alloc() {
3) | | | | alloc_pages_current() {
3) | | | | | get_task_policy.part.30();
3) 0.234 us | | | | | policy_nodemask();
3) 0.124 us | | | | }
[...]
```

Этот инструмент показывает не только выбранный путь кода, но и возможных кандидатов для дальнейшей трассировки, а также исследования их аргументов и возвращаемых значений с целью получения дополнительной информации.

14.3.2. perf sched

`perf(1)` — еще один многофункциональный инструмент. Его применение для чтения счетчиков производительности РМС, профилирования и трассировки уже описывалось в главе 6. У него есть подкоманда `sched` для анализа работы планировщика. Например:

```
# perf sched record
# perf sched timehist
Samples do not have callchains.
```

time	cpu	task name [tid/pid]	wait time (msec)	sch delay (msec)	run time (msec)
991962.879971	[0005]	perf[16984]	0.000	0.000	0.000
991962.880070	[0007]	:17008[17008]	0.000	0.000	0.000
991962.880070	[0002]	cc1[16880]	0.000	0.000	0.000

```

991962.880078 [0000] cc1[16881]          0.000      0.000      0.000
991962.880081 [0003] cc1[16945]          0.000      0.000      0.000
991962.880093 [0003] ksoftirqd/3[28]      0.000      0.007      0.012
991962.880108 [0000] ksoftirqd/0[6]       0.000      0.007      0.030
[...]
```

Инструмент выводит временные характеристики событий планирования: время, потраченное в ожидании снятия блокировки («wait time»), задержка планировщика (известная как задержка в очереди ожидания на выполнение, «sch delay») и время выполнения на процессоре («run time»).

14.3.3. slabtop

slabtop(1) показывает текущие размеры кэшей распределителя блоков памяти в ядре. Вот пример, полученный в большой производственной системе с сортировкой по размеру кэша (-s c):

```

# slabtop -s c
Active / Total Objects (% used) : 1232426 / 1290213 (95.5%)
Active / Total Slabs (% used)    : 29225 / 29225 (100.0%)
Active / Total Caches (% used)   : 85 / 135 (63.0%)
Active / Total Size (% used)     : 288336.64K / 306847.48K (94.0%)
Minimum / Average / Maximum Object : 0.01K / 0.24K / 16.00K
```

	OBJS	ACTIVE	USE	OBJ	SIZE	SLABS	OBJ/SLAB	CACHE	SIZE	NAME
	76412	69196	0%	0.57K	2729	28		43664K		radix_tree_node
	313599	313599	100%	0.10K	8041	39		32164K		buffer_head
	3732	3717	0%	7.44K	933	4		29856K		task_struct
	11776	8795	0%	2.00K	736	16		23552K		TCP
	33168	32277	0%	0.66K	691	48		22112K		proc_inode_cache
	86100	79990	0%	0.19K	2050	42		16400K		dentry
	25864	24679	0%	0.59K	488	53		15616K		inode_cache

[...]

Как показывает этот вывод, около 43 Мбайт занимает кэш radix_tree_node и около 23 Мбайт — кэш TCP. Для системы с общим объемом памяти 180 Гбайт это относительно небольшие значения.

Этот инструмент пригодится для анализа проблем, связанных с нехваткой памяти. С его помощью можно проверить, не потребляет ли какой-то компонент ядра неожиданно большой объем памяти.

14.3.4. Другие инструменты

Файл /proc/lock_stat содержит различные статистики о блокировках в ядре, но он доступен, только если ядро собрано с параметром CONFIG_LOCK_STAT.

Файл /proc/sched_debug содержит множество метрик, которые помогут разработчикам планировщиков.

Таблица 14.3 (окончание)

Инструмент	Источник	Цель	Описание
memleak	ВСС	Память	Выводит пути в коде, способные вызвать утечки памяти
slabratetop	ВСС/книга	Блочная память (Slab)	Отображает частоту выделения блоков памяти из разных кэшей
numamove	Книга	NUMA	Отображает статистику миграции страниц NUMA
workq	Книга	Очереди заданий	Отображает время обработки заданий из очереди

Актуальные списки параметров инструментов ВСС и bpftrace и описание их возможностей ищите в соответствующих репозиториях.

В предыдущих главах вы найдете описание других инструментов для анализа работы компонентов ядра, включая системные вызовы, сетевую подсистему и блочный ввод/вывод.

Далее мы рассмотрим приемы инструментации спин-блокировок и тасклетов.

14.4.1. loads

loads(8)¹ — инструмент для bpftrace, который раз в секунду выводит значения средней нагрузки на систему:

```
# loads.bt
Attaching 2 probes...
Reading load averages... Hit Ctrl-C to end.
18:49:16 load averages: 1.983 1.151 0.931
18:49:17 load averages: 1.824 1.132 0.926
18:49:18 load averages: 1.824 1.132 0.926
[...]
```

Как говорилось в главе 6, эти средние значения нагрузки не очень информативны, и после знакомства с ними желательно тут же перейти к анализу других, более глубоких метрик. Инструмент load(8) пригодится как пример извлечения и вывода значений переменных ядра, в данном случае avenrun:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Reading load averages... Hit Ctrl-C to end.\n");
}
```

¹ Немного истории: первую версию под названием load.d я написал для DTrace 10 июня 2005 года, а версию для bpftrace — 10 сентября 2018 года.

```

interval:s:1
{
    /*
     * Комментарии к вычислениям ниже вы найдете в fs/proc/loadavg.c и
     * include/linux/sched/loadavg.h.
     */
    $avenrun = kaddr("avenrun");
    $load1 = *$avenrun;
    $load5 = *($avenrun + 8);
    $load15 = *($avenrun + 16);
    time("%H:%M:%S ");
    printf("load averages: %d.%03d %d.%03d %d.%03d\n",
        ($load1 >> 11), (($load1 & ((1 << 11) - 1)) * 1000) >> 11,
        ($load5 >> 11), (($load5 & ((1 << 11) - 1)) * 1000) >> 11,
        ($load15 >> 11), (($load15 & ((1 << 11) - 1)) * 1000) >> 11
    );
}

```

Здесь встроенная функция `kaddr()` извлекает адрес символа `avenrun` в ядре, который затем разыменовывается для получения значений нагрузки. Аналогично можно получать значения других переменных ядра.

14.4.2. `offcputime`

`offcputime(8)` был представлен в главе 6. В этом разделе я покажу, как с его помощью исследовать состояние задач, а также расскажу о проблемах, которые привели к созданию дополнительных инструментов, включенных в эту главу.

Непрерываемый ввод/вывод

Сопоставление с состоянием `TASK_UNINTERRUPTIBLE` потока помогает выяснить, сколько времени приложение было заблокировано в ожидании ресурса, и исключить из профиля `offcputime(8)` время простоя между заданиями, которое иначе может замаскировать реальные проблемы с производительностью. В Linux время `TASK_UNINTERRUPTIBLE` также включается в средние значения нагрузки, что сбивает с толку тех, кто ожидает, что значения будут отражать только время использования процессора.

Вот пример анализа потоков в этом состоянии (2) для процессов в пространстве пользователя и стеков ядра:

```

# offcputime -uK --state 2
Tracing off-CPU time (us) of user threads by kernel stack... Hit Ctrl-C to end.
[...]

finish_task_switch
__schedule
schedule
io_schedule
generic_file_read_iter
xfs_file_buffered_aio_read

```

```

xfs_file_read_iter
__vfs_read
vfs_read
ksys_read
do_syscall_64
entry_SYSCALL_64_after_hwframe
-               tar (7034)
1088682

```

Из полученных трассировок стека я включил только последнюю, которая соответствует процессу tar(1), ожидающему завершения операции ввода/вывода в файловой системе XFS. Эта команда отфильтровала потоки в других состояниях, в том числе:

- **TASK_RUNNING (0):** потоки в этом состоянии могут приостанавливаться из-за принудительного переключения контекста, когда процессоры работают в режиме насыщения. Трассировки стека таких прерванных потоков не особенно интересны, так как не сообщают причин, почему поток покинул процессор.
- **TASK_INTERRUPTIBLE (1):** потоки в этом состоянии обычно захламляют вывод множеством трассировок стека потоков. Они простаивают в ожидании возобновления работы.

Исключение из вывода потоков в этих состояниях помогает сосредоточиться на потоках, заблокированных во время обработки запросов от приложений, которые сильно влияют на производительность.

Неинформативные стеки

Многие трассировки стека, которые выводит `offcputime(8)`, неинформативны. Они показывают путь до блокировки, но не ее причину. Вот пример трассировки в течение 5 секунд простоев процесса `gzip(1)` в пространстве ядра:

```
# offcputime -Kp $(pgrep -n gzip) 5
```

```
Tracing off-CPU time (us) of PID 5028 by kernel stack for 5 secs.
```

```

finish_task_switch
__schedule
schedule
exit_to_usermode_loop
prepare_exit_to_usermode
swapgs_restore_regs_and_return_to_usermode
-               gzip (5028)
21

finish_task_switch
__schedule
schedule
pipe_wait
pipe_read
__vfs_read

```



```

vfs_read
ksys_read
do_syscall_64
entry_SYSCALL_64_after_hwframe
-                               gzip (5028)
4404219

```

Судя по этому выводу, 4.4 секунды из 5 секунд процесс провел в функции `pipe_read()`, но по этой трассировке нельзя сказать, какие действия, завершения которых ждал процесс `gzip`, выполнялись на другом конце пайплайна и почему они заняли так много времени. Трассировка просто сообщает, что процесс чего-то ждал.

Такие неинформативные трассировки стека, ведущие к простоям, — обычное явление не только для пайплайнов, но и для операций ввода/вывода и конфликтов блокировок. Вы видите, что потоки простаивают в ожидании освобождения блокировки, но не можете узнать причину недоступности блокировки (например, кто владел ею и что делал все это время).

Изучение стеков пробуждения с помощью `wakeuptime(8)` часто помогает выяснить, кто скрывается по другую сторону ожидания.

Более подробно `offcputime(8)` описан в главе 6.

14.4.3. wakeuptime

`wakeuptime(8)`¹ — инструмент для BCC, отображающий трассировки стека потоков, которые освобождают блокировки, что приводит к возобновлению выполнения других потоков, ожидавших на этих блокировках, а также время удержания блокировок. Может использоваться для дальнейшего анализа простоев. Продолжим предыдущий пример:

```

# wakeuptime -p $(pgrep -n gzip) 5
Tracing blocked time (us) by kernel stack for 5 secs.
target:      gzip
ffffffff94000088 entry_SYSCALL_64_after_hwframe
ffffffff93604175 do_syscall_64
ffffffff93874d72 ksys_write
ffffffff93874af3 vfs_write
ffffffff938748c2 __vfs_write
ffffffff9387d50e pipe_write
ffffffff936cb11c __wake_up_common_lock
ffffffff936caffc __wake_up_common

```

¹ Немного истории: первую версию `wakeuptime` для DTrace я написал и визуализировал ее результаты в виде флейм-графиков 7 ноября 2013 года. Все началось с 45-минутного доклада о флейм-графиках, которые я подготовил в последнюю минуту перед конференцией USENIX LISA [Gregg 13a], когда изначально планировавшееся выступление внезапно оказалось невозможным. Меня попросили заменить другого спикера, который заболел, и в итоге я представил этот инструмент как вторую часть 90-минутного доклада по флейм-графикам. Версию для BCC я написал 14 января 2016 года.

```

ffffff936cb65e autoremove_wake_function
waker:         tar
4551336

```

Detaching...

Как показывает этот вывод, процесс `gzip(1)` был заблокирован процессом `tar(1)`, вызвавшим `vfs_write()`. Теперь я покажу команду, которая вызвала эту рабочую нагрузку:

```
tar cf - /mnt/data | gzip -> /mnt/backup.tar.gz
```

Из этого однострочника очевидно, что большую часть своего времени `gzip(1)` тратит на ожидание данных от `tar(1)`. `tar(1)`, в свою очередь, большую часть своего времени тратит на ожидание чтения данных с диска, что можно показать с помощью `offcputime(8)`:

```

# offcputime -Kp $(pgrep -n tar) 5
Tracing off-CPU time (us) of PID 5570 by kernel stack for 5 secs.
[...]

  finish_task_switch
  __schedule
  schedule
  io_schedule
  generic_file_read_iter
  xfs_file_buffered_aio_read
  xfs_file_read_iter
  __vfs_read
  vfs_read
  ksys_read
  do_syscall_64
  entry_SYSCALL_64_after_hwframe
  -               tar (5570)
4204994

```

Этот стек показывает, что `tar(1)` был заблокирован в `io_schedule()`: функции, выполняющей операции с устройством блочного ввода/вывода. Сопоставляя результаты, полученные с помощью `offcputime(8)` и `wakeuptime(8)`, можно увидеть причину приостановки приложения (вывод `offcputime(8)`) и причину пробуждения (вывод `wakeuptime(8)`). Иногда причина пробуждения точнее определяет источник проблем, чем причина приостановки.

Для краткости я использовал в примерах параметр `-p`, чтобы трассировка выполнялась только для процесса с указанным идентификатором PID. Но при желании можно выполнить трассировку в масштабе всей системы, опустив `-p`.

Этот инструмент трассирует функции планировщика `schedule()` и `try_to_wake_up()`. В нагруженных системах они могут вызываться очень часто, поэтому оверхед бывает существенным.

Порядок использования:

```
wakeuptime [options] [duration]
```

Параметры `options`:

- **-f**: вывести результаты в свернутом формате, чтобы их можно было использовать для создания флейм-графика;
- **-p PID**: трассировать только этот процесс.

По аналогии с `offcputime(8)`, если `wakeuptime(8)` запустить без параметра `-p`, он будет трассировать всю систему целиком и, вероятно, произведет сотни страниц вывода. Флейм-график поможет быстро сориентироваться в этом выводе.

14.4.4. offwaketime

`offwaketime(8)`¹ — это инструмент для BCC, сочетающий возможности `offcputime(8)` и `wakeuptime(8)`. Продолжим предыдущий пример:

```
# offwaketime -Kp $(pgrep -n gzip) 5
```

Tracing blocked time (us) by kernel off-CPU and waker stack for 5 secs.

[...]

```
waker:          tar 5852
entry_SYSCALL_64_after_hwframe
do_syscall_64
ksys_write
vfs_write
__vfs_write
pipe_write
__wake_up_common_lock
__wake_up_common
autoremove_wake_function
--
--
finish_task_switch
__schedule
schedule
pipe_wait
pipe_read
__vfs_read
vfs_read
ksys_read
do_syscall_64
```

¹ Немного истории: первую версию этого инструмента я разработал в форме цепного графа для конференции USENIX LISA 2013 7 ноября 2013 года [Gregg 13a]. С его помощью я выполнил трассировку нескольких событий пробуждения и показал результат в виде флейм-графика. Эта версия была основана на DTrace, а поскольку DTrace не может сохранять и извлекать стеки, мне пришлось сбрасывать все события в пространство пользователя и делать постобработку, что было слишком накладно для реального применения в продакшене. BPF позволяет сохранять и извлекать трассировки стека (чем я и воспользовался, написав версию этого инструмента для BCC 13 января 2016 года), а также ограничивать одним уровнем пробуждения. Алексей Старовойтов добавил эту версию в исходный код ядра в `samples/bpf/offwaketime_*.c`.

```
entry_SYSCALL_64_after_hwframe
target:          gzip 5851
4490207
```

Как показывает этот вывод, `tar(1)` возобновляет выполнение процесса `gzip(1)`, ожидавшего 4.49 секунды. Здесь есть обе трассировки стека, разделенные знаками «--», причем верхняя трассировка показана в обратном порядке. То есть трассировки встречаются посередине, где процесс `tar(1)` (вверху) возобновляет выполнение процесса `gzip(1)` (внизу).

Этот инструмент трассирует функции планировщика `schedule()` и `try_to_wake_up()` и сохраняет в карте BPF трассировку стека процесса, владевшего блокировкой, для последующего сопоставления с трассировкой стека заблокированного потока в контексте ядра. В нагруженных системах это может происходить очень часто, поэтому оверхед бывает значительным.

Порядок использования:

```
offwaketime [options] [duration]
```

Параметры `options`:

- **-f**: вывести результаты в свернутом формате, пригодном для создания флейм-графика времени ожидания/удержания блокировок;
- **-p PID**: трассировать только этот процесс;
- **-K**: выбирать трассировки стека только в пространстве ядра;
- **-U**: выбирать трассировки стека только в пространстве пользователя.

Без параметра `-p` инструмент может трассировать всю систему и, вероятно, создаст сотни страниц вывода. Использование параметров `-p`, `-K` и `-U` поможет уменьшить оверхед.

Флейм-графики времени ожидания/удержания блокировок

Свернутый вывод (полученный при использовании параметра `-f`) можно визуализировать в виде флейм-графика, при этом используется та же ориентация стеков, что и в текстовом выводе: перевернутый стек процесса, освободившего блокировку, отображается вверху, а стек процесса, ожидавшего на блокировке, — внизу. Пример такого графика показан на рис. 14.4.

14.4.5. `mlock` и `mheld`

Инструменты `mlock(8)`¹ и `mheld(8)` трассируют продолжительность ожидания и удержания мьютексов ядра и выводят результаты в виде гистограмм со стеками

¹ Немного истории: я написал `mlock(8)` и `mheld(8)` специально для этой книги 14 марта 2019 года. За основу я взял инструмент `lockstat(1M)` для Solaris, разработанный Джеффом Бонвиком (Jeff Bonwick), который тоже выводит частичные стеки с гистограммами времени ожидания и удержания блокировок.

в пространстве ядра. `mlock(8)` можно использовать для выявления проблем, связанных с конфликтом блокировок, а `mheld(8)` — для выяснения причин: пути в коде, который привел к захвату блокировки. Начнем с `mlock(8)`:

```
# mlock.bt
Attaching 6 probes...
Tracing mutex_lock() latency, Ctrl-C to end.
^C
[...]

@lock_latency_ns[0xffff9d015738c6e0,
    kretprobe_trampoline+0
    unix_stream_recvmsg+81
    sock_recvmsg+67
    __sys_recvmsg+245
    __sys_recvmsg+81
, chrome]:
[512, 1K)      5859 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)       8303 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)       1689 | @@@@@@@@@@@
[4K, 8K)        476 | @@
[8K, 16K)      101 | |
```

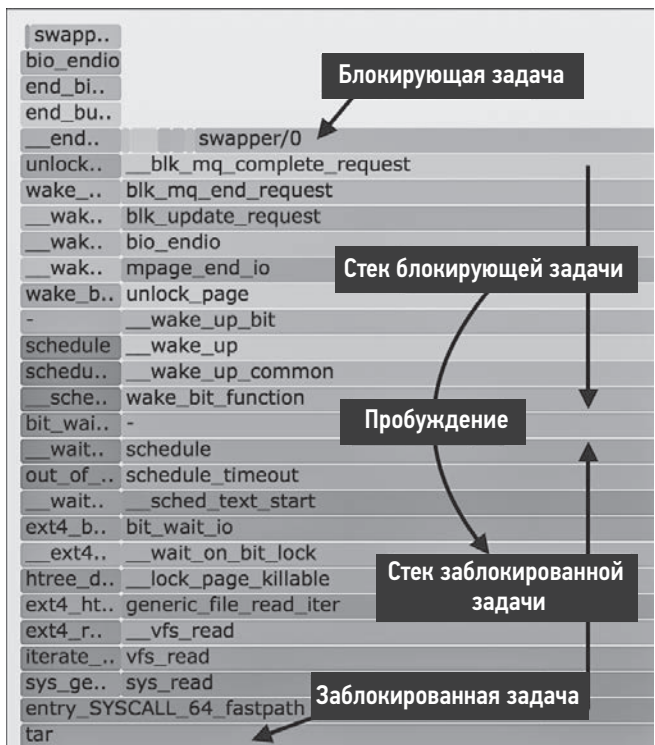


Рис. 14.4. Флейм-график времени ожидания/удержания блокировок

Создавая этот пример, я получил множество трассировок стека и блокировок, но сюда я включил только одну. Здесь мы видим адрес блокировки (0xffff9d015738c6e0), трассировку стека до `mutex_lock()`, имя процесса («chrome») и задержку выполнения `mutex_lock()`. За время трассировки эта блокировка приобреталась тысячи раз, и в большинстве случаев время ожидания ее освобождения было невелико: например, как показывает гистограмма, в 8303 случаях продолжительность ожидания составила от 1024 до 2048 наносекунд (примерно от 1 до 2 микросекунд).

Теперь перейдем к `mheld(8)`:

```
# mheld.bt
Attaching 9 probes...
Tracing mutex_lock() held times, Ctrl-C to end.
^C
[...]

@held_time_ns[0xffff9d015738c6e0,
  mutex_unlock+1
  unix_stream_recvmsg+81
  sock_recvmsg+67
  __sys_recvmsg+245
  __sys_recvmsg+81
, chrome]:
[512, 1K)      16459 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[1K, 2K)      7427  |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
```

Здесь показан тот же процесс и стек, соответствующий удержанию той же блокировки.

Эти инструменты трассируют функции ядра `mutex_lock()`, `mutex_lock_interruptible()` и `mutex_trylock()`, потому что точек трассировки для мьютексов пока нет. Поскольку эти функции могут вызываться очень часто, то в нагруженных системах оверхед на трассировку бывает значительным.

mlock

Исходный код `mlock(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing mutex_lock() latency, Ctrl-C to end.\n");
}

kprobe:mutex_lock,
kprobe:mutex_lock_interruptible
/$1 == 0 || pid == $1/
{
    @lock_start[tid] = nsecs;
    @lock_addr[tid] = arg0;
}
```

```

kretprobe:mutex_lock
/($1 == 0 || pid == $1) && @lock_start[tid]/
{
    @lock_latency_ns[ksym(@lock_addr[tid]), kstack(5), comm] =
        hist(nsecs - @lock_start[tid]);
    delete(@lock_start[tid]);
    delete(@lock_addr[tid]);
}

kretprobe:mutex_lock_interruptible
/retval == 0 && ($1 == 0 || pid == $1) && @lock_start[tid]/
{
    @lock_latency_ns[ksym(@lock_addr[tid]), kstack(5), comm] =
        hist(nsecs - @lock_start[tid]);
    delete(@lock_start[tid]);
    delete(@lock_addr[tid]);
}

END
{
    clear(@lock_start);
    clear(@lock_addr);
}

```

Этот код фиксирует продолжительность выполнения `mutex_lock()` и `mutex_lock_interruptible()`, только если возвращаемое значение сообщает об успехе. `mutex_trylock()` не трассируется, потому что, как предполагается, она выполняется без задержки. `mlock(8)` принимает необязательный аргумент с идентификатором процесса для трассировки, без него трассируется вся система.

mheld

Исходный код `mheld(8)`:

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing mutex_lock() held times, Ctrl-C to end.\n");
}

kprobe:mutex_lock,
kprobe:mutex_trylock,
kprobe:mutex_lock_interruptible
/$1 == 0 || pid == $1/
{
    @lock_addr[tid] = arg0;
}

kretprobe:mutex_lock
/($1 == 0 || pid == $1) && @lock_addr[tid]/
{
    @held_start[@lock_addr[tid]] = nsecs;
}

```

```

        delete(@lock_addr[tid]);
    }

kretprobe:mutex_trylock,
kretprobe:mutex_lock_interruptible
/retval == 0 && ($1 == 0 || pid == $1) && @lock_addr[tid]/
{
    @held_start[@lock_addr[tid]] = nsecs;
    delete(@lock_addr[tid]);
}

kprobe:mutex_unlock
/($1 == 0 || pid == $1) && @held_start[arg0]/
{
    @held_time_ns[ksym(arg0), kstack(5), comm] =
        hist(nsecs - @held_start[arg0]);
    delete(@held_start[arg0]);
}

END
{
    clear(@lock_addr);
    clear(@held_start);
}

```

Этот код фиксирует продолжительность удержания мьютекса, трассируя различные функции. Как и `mlock(8)`, он принимает необязательный аргумент с идентификатором процесса.

14.4.6. Спин-блокировки

Спин-блокировки (или циклические блокировки), как и мьютексы, не имеют своих точек трассировки. Обратите внимание, что есть несколько видов спин-блокировок, включая `spin_lock_bh()`, `spin_lock()`, `spin_lock_irq()` и `spin_lock_irqsave()` [162]. Они определены в заголовке `include/linux/spinlock.h`, как показано ниже:

```

#define spin_lock_irqsave(lock, flags)          \
do {                                           \
    raw_spin_lock_irqsave(spinlock_check(lock), flags); \
} while (0)
[...]
#define raw_spin_lock_irqsave(lock, flags)      \
do {                                           \
    typecheck(unsigned long, flags);          \
    flags = _raw_spin_lock_irqsave(lock);      \
} while (0)

```

С помощью `funccount(8)` можно узнать, насколько часто они вызываются:

```

# funccount '*spin_lock*'
Tracing 16 functions for "*spin_lock*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT

```



```

_raw_spin_lock_bh          7092
native_queued_spin_lock_slowpath 7227
_raw_spin_lock_irq        261538
_raw_spin_lock            1215218
_raw_spin_lock_irqsave    1582755
Detaching...

```

funccount(8) инструментирует точки входа в эти функции с помощью kprobes. Трассировка выхода из этих функций с помощью kretprobes невозможна¹, поэтому нельзя непосредственно вычислить продолжительность их выполнения. Поищите выше в стеке другие функции для трассировки, например, с помощью stackcount(8) по kprobe.

Проблемы с производительностью спин-блокировок я обычно анализирую методом профилирования процессора и построения флейм-графиков, так как эти функции проявляют себя как потребители процессора.

14.4.7. kmem

kmem(8)² — это инструмент для bpftrace, трассирующий распределение памяти ядра и отображающий количество операций выделения памяти, средний размер выделенного блока и общее количество выделенных байтов. Например:

```

# kmem.bt
Attaching 3 probes...
Tracing kmem allocation stacks (kmalloc, kmem_cache_alloc). Hit Ctrl-C to end.
^C
[...]
@bytes[
    kmem_cache_alloc+288
    getname_flags+79
    getname+18
    do_sys_open+285
    Sys_openat+20
, Xorg]: count 44, average 4096, total 180224
@bytes[
    __kmalloc_track_caller+368
    kmemdup+27
    intel_crtc_duplicate_state+37
    drm_atomic_get_crtc_state+119
    page_flip_common+51
, Xorg]: count 120, average 2048, total 245760

```

¹ Как обнаружилось, инструментация точек выхода из этих функций с помощью kretprobes может заблокировать систему [163], поэтому в БСС добавили раздел, запрещающий kretprobes. В ядре есть функции, которые запрещены для инструментации с помощью NOKPROBE_SYMBOL. Я надеюсь, что упомянутые выше функции не попадут в их число, поскольку это сделает невозможным их трассировку с помощью kprobes, а kprobes имеет много применений даже без kretprobes.

² Немного истории: я написал его для этой книги 15 марта 2019 года.

Здесь я привел только два последних стека. Первый соответствует обращению процесса Xorg к системному вызову `open(2)`, который выделил блок памяти (`kmem_cache_alloc()`) во время выполнения `getname_flags()`. За время, пока выполнялась трассировка, это произошло 44 раза, и в среднем выделялось по 4096 байт, а всего было выделено 180 224 байт.

`kmem(8)` использует точки трассировки `kmem`. Так как операция выделения памяти может выполняться очень часто, то в нагруженных системах оверхед на трассировку бывает значительным.

Исходный код `kmem(8)`:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing kmem allocation stacks (kmalloc, kmem_cache_alloc). ");
    printf("Hit Ctrl-C to end.\n");
}

tracepoint:kmem:kmalloc,
tracepoint:kmem:kmem_cache_alloc
{
    @bytes[kstack(5), comm] = stats(args->bytes_alloc);
}
```

Этот код использует встроенную функцию `stats()` для вывода количества операций выделения памяти, среднего и общего количества байтов. При желании вместо `stats()` можно использовать `hist()` для вывода гистограмм.

14.4.8. `kpages`

`kpages(8)`¹ — это инструмент для `bpftrace`, трассирующий операции выделения памяти ядра другим способом, с помощью `alloc_pages()`, и использующий точку трассировки `kmem:mm_page_alloc`. Например:

```
# kpages.bt
Attaching 2 probes...
Tracing page allocation stacks. Hit Ctrl-C to end.
^C
[...]
@pages[
    __alloc_pages_nodemask+521
    alloc_pages_vma+136
    handle_pte_fault+959
    __handle_mm_fault+1144
    handle_mm_fault+177
, chrome]: 11733
```

¹ Немного истории: я написал его для этой книги 15 марта 2019 года.

Я сократил вывод и привел здесь только один стек: пока выполнялась трассировка, процесс Chrome выделил 11 733 страницы. Этот инструмент использует точки трассировки kmem. Так как операция выделения памяти может выполняться очень часто, оверхед на трассировку может оказаться значительным в нагруженных системах.

Исходный код kpages(8):

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing page allocation stacks. Hit Ctrl-C to end.\n");
}

tracepoint:kmem:mm_page_alloc
{
    @pages[kstack(5), comm] = count();
}
```

Все это можно реализовать в форме однострочного сценария, но я хотел подчеркнуть важность такого анализа и поэтому написал отдельный инструмент kpages(8).

14.4.9. memleak

memleak(8) был представлен в главе 7. Это инструмент для ВСС, сообщающий о событиях выделения памяти, для которых за время трассировки не возникло парных событий освобождения памяти, что может служить признаком утечки памяти. По умолчанию трассируются распределители памяти ядра, например:

```
# memleak
Attaching to kernel allocators, Ctrl+C to quit.

[13:46:02] Top 10 stacks with outstanding allocations:
[...]
```

```
6922240 bytes in 1690 allocations from stack
__alloc_pages_nodemask+0x209 [kernel]
alloc_pages_current+0x6a [kernel]
__page_cache_alloc+0x81 [kernel]
pagecache_get_page+0x9b [kernel]
grab_cache_page_write_begin+0x26 [kernel]
ext4_da_write_begin+0xcb [kernel]
generic_perform_write+0xb3 [kernel]
__generic_file_write_iter+0x1aa [kernel]
ext4_file_write_iter+0x203 [kernel]
new_sync_write+0xe7 [kernel]
__vfs_write+0x29 [kernel]
vfs_write+0xb1 [kernel]
sys_pwrite64+0x95 [kernel]
do_syscall_64+0x73 [kernel]
entry_SYSCALL_64_after_hwframe+0x3d [kernel]
```

Я включил сюда только один стек, показывающий выделение памяти в функции записи в ext4. Более подробно memleak(8) описан в главе 7.

14.4.10. slabratetop

slabratetop(8)¹ — это инструмент для ВСС и bpftrace, сообщающий частоту выделения блоков памяти из разных кэшей блочной памяти ядра и напрямую трассирующий вызовы kmem_cache_alloc(). Этот инструмент дополняет утилиту slabtop(1), которая показывает объем кэшей (получая их из /proc/slabinfo). Вот пример, полученный на производственном сервере с 48 процессорами:

```
# slabratetop
09:48:29 loadavg: 6.30 5.45 5.46 4/3377 29884
CACHE                ALLOCS          BYTES
kmalloc-4096          654        2678784
kmalloc-256           2637       674816
filp                   392       100352
sock_inode_cache       94        66176
TCP                    31        63488
kmalloc-1024           58        59392
proc_inode_cache       69       46920
eventpoll_epi         354       45312
sigqueue               227       36320
dentry                 165       31680
[...]
```

Как показывает этот вывод, при трассировке наибольшее количество байтов было выделено из кэша kmalloc-4096. Как и slabtop(1), этот инструмент можно использовать для устранения непредвиденных проблем с памятью.

Он трассирует функцию ядра kmem_cache_alloc() с помощью kprobes. Поскольку эта функция может вызываться довольно часто, оверхед инструмента может быть заметным в нагруженных системах.

ВСС

Порядок использования:

```
slabratetop [options] [interval [count]]
```

Параметры options:

- **-C**: не очищать экран.

bpftrace

Эта версия подсчитывает события выделения памяти по имени кэша и выводит результаты раз в секунду:

¹ Немного истории: версию для ВСС я написал 15 октября 2016 года, а версию для bpftrace — 26 января 2019 года специально для этой книги.

```
#!/usr/local/bin/bpftrace

#include <linux/mm.h>
#include <linux/slab.h>
#ifdef CONFIG_SLUB
#include <linux/slub_def.h>
#else
#include <linux/slab_def.h>
#endif

kprobe:kmem_cache_alloc
{
    $cachep = (struct kmem_cache *)arg0;
    @[str($cachep->name)] = count();
}

interval:s:1
{
    time();
    print(@);
    clear(@);
}
```

Проверка параметра компиляции ядра CONFIG_SLUB нужна, чтобы подключить правильные заголовочные файлы с определениями распределителя блоков.

14.4.11. numamove

`numamove(8)`¹ трассирует события миграции страниц типа «NUMA misplaced». Эти страницы перемещаются на разные узлы NUMA, чтобы улучшить локальность памяти и общую производительность системы. В свое время мне пришлось столкнуться с проблемами, когда до 40% процессорного времени тратилось на подобные миграции страниц NUMA. Потеря производительности перевесила преимущества балансировки страниц NUMA. Этот инструмент поможет проследить миграции страниц NUMA, если проблема появится вновь. Пример вывода:

```
# numamove.bt
Attaching 4 probes...
TIME          NUMA_migrations NUMA_migrations_ms
22:48:45              0              0
22:48:46              0              0
22:48:47             308             29
22:48:48              2              0
22:48:49              0              0
22:48:50              1              0
22:48:51              1              0
[...]
```

¹ Немного истории: я написал его для этой книги 26 января 2019 года, для того чтобы использовать его, когда проблема появится вновь.

Эта попытка трассировки зафиксировала всплеск миграции страниц NUMA в 22:48:47. В этот момент было выполнено 208 миграций, на что в общей сложности потребовалось 29 миллисекунд. Столбцы показывают количество миграций в секунду и время, затраченное на миграции, в миллисекундах. Обратите внимание: чтобы эти события возникали, должна быть включена балансировка NUMA (`sysctl kernel.numa_balancing = 1`).

Исходный код `numamove(8)`:

```
#!/usr/local/bin/bpftrace

kprobe:migrate_misplaced_page { @start[tid] = nsecs; }

kretprobe:migrate_misplaced_page /@start[tid]/
{
    $dur = nsecs - @start[tid];
    @ns += $dur;
    @num++;
    delete(@start[tid]);
}

BEGIN
{
    printf("%-10s %18s %18s\n", "TIME",
        "NUMA_migrations", "NUMA_migrations_ms");
}

interval:s:1
{
    time("%H:%M:%S");
    printf(" %18d %18d\n", @num, @ns / 1000000);
    delete(@num);
    delete(@ns);
}
```

Этот инструмент трассирует вход и выход из функции ядра `migrate_misplaced_page()` с помощью `kprobe` и `kretprobe`, а также зонд `interval` для вывода статистики.

14.4.12. workq

`workq(8)`¹ трассирует выполнение заданий из очереди и время их задержки. Например:

```
# workq.bt
Attaching 4 probes...
Tracing workqueue request latencies. Ctrl-C to end.
^C
[...]

@us[intel_atomic_commit_work]:
```

¹ Немного истории: я написал его для этой книги 14 марта 2019 года.

[illegible]

Мы видим, что функция `kcryptd_crypt()` обслуживания очереди заданий вызывалась часто. На ее выполнение в большинстве случаев уходило от 4 до 32 микросекунд.

Инструмент использует точки трассировки `workqueue:workqueue_execute_start` и `workqueue:workqueue_execute_end`.

Исходный код `workq(8)`:

```
#!/usr/local/bin/bpftrace
```

```
BEGIN
{
    printf("Tracing workqueue request latencies. Ctrl-C to end.\n");
}

tracepoint:workqueue:workqueue_execute_start
{
    @start[tid] = nsecs;
    @wqfunc[tid] = args->function;
}

tracepoint:workqueue:workqueue_execute_end
/@start[tid]/
{
    $dur = (nsecs - @start[tid]) / 1000;
    @us[ksym(@wqfunc[tid])] = hist($dur);
    delete(@start[tid]);
    delete(@wqfunc[tid]);
}

END
{
    clear(@start);
    clear(@wqfunc);
}
```

Этот код измеряет время между срабатываниями этих точек трассировки и сохраняет его в гистограмме с именем функции в качестве ключа.

14.4.13. Тасклеты

В 2009 году Антон Бланшар (Anton Blanchard) предложил патч, добавляющий точки трассировки для тасклетов, но они так и не появились в ядре [164]. Функции тасклетов, инициализированные с помощью `tasklet_init()`, можно трассировать с помощью `kprobes`. Вот выдержка из `net/ipv4/tcp_output.c`:

```
[...]
    tasklet_init(&tsq->tasklet,
                tcp_tasklet_func,
                (unsigned long)tsq);
[...]
```

Этот код создает тасклет, реализованный в функции `tcp_tasklet_func()`. Вот как можно выполнить трассировку продолжительности его выполнения с помощью `funclatency(8)` из BCC:

```
# funclatency -u tcp_tasklet_func
Tracing 1 functions for "tcp_tasklet_func"... Hit Ctrl-C to end.
^C
      usecs                : count      distribution
      0 -> 1                :      0      |
      2 -> 3                :      0      |
      4 -> 7                :      3      | *
      8 -> 15               :     10      | ****
     16 -> 31               :     22      | *****
     32 -> 63               :    100      | *****
     64 -> 127              :     61      | *****
Detaching...
```

При желании на основе `bpfttrace` и `kprobes` можно создать свои инструменты для трассировки функций тасклетов.

14.4.14. Другие инструменты

Другие инструменты для анализа работы ядра:

- **runqlat(8)**: определяет, как долго потоки ждут своей очереди на процессоре (глава 6);
- **syscount(8)**: подсчитывает количество системных вызовов по типам и процессам (глава 6);
- **hardirq(8)**: показывает время, затраченное на обработку аппаратных прерываний (глава 6);
- **softirq(8)**: показывает время, затраченное на обработку программных прерываний (глава 6);

- **xcalls(8)**: трассирует и суммирует время выполнения функций на нескольких процессорах (глава 6);
- **vmscan(8)**: измеряет время сжатия и восстановления сканера виртуальных машин (глава 7);
- **vfsstat(8)**: обобщает статистику некоторых распространенных операций VFS (глава 8);
- **cachestat(8)**: выводит статистику попаданий и промахов кэша страниц (глава 8);
- **biostacks(8)**: выводит полную задержку блочного ввода/вывода от постановки запроса в очередь ОС до завершения обработки устройством;
- **skblife(8)**: измеряет продолжительность жизни буферов `sk_buff` (глава 10);
- **inject(8)**: использует `bpf_override_return()` для изменения функций ядра так, чтобы они возвращали ошибки. Используется для проверки обработки ошибок (это инструмент BCC);
- **criticalstat(8)**¹: измеряет продолжительность выполнения атомарных критических секций в ядре и выводит длительность и трассировку стека. По умолчанию показывает пути в коде, выполняющиеся с отключенными прерываниями и дольше 100 микросекунд. Этот инструмент BCC помогает найти источник задержек в ядре. Для его работы нужно, чтобы ядро было скомпилировано с параметрами `CONFIG_DEBUG_PREEMPT` и `CONFIG_PREEMPTIRQ_EVENTS`.

Нередко в процессе анализа работы ядра требуется инструментировать события, не поддерживаемые готовыми инструментами, и в этом помогут однострочные сценарии, с которых обычно начинается создание собственных инструментов.

14.5. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

В этом разделе перечислены однострочные сценарии для BCC и `bpfttrace`. Там, где это возможно, один и тот же сценарий реализуется с использованием BCC и `bpfttrace`.

14.5.1. BCC

Подсчитывает обращения к системным вызовам по процессам:

```
syscount -P
```

Подсчитывает обращения к каждому системному вызову:

```
syscount
```

Подсчитывает вызовы функций ядра с именами, начинающимися с «attach»:

```
funccount 'attach*'
```

¹ Немного истории: был написан Джоэлем Фернандесом (Joel Fernandes) 18 июня 2018 года.

Измеряет время выполнения функции ядра `vfs_read()` и выводит результаты в виде гистограммы:

```
funclatency vfs_read
```

Подсчитывает разные значения в первом целочисленном аргументе в вызове функции ядра «func1»:

```
argdist -C 'p::func1(int a):int:a'
```

Подсчитывает разные значения, возвращаемые функцией ядра «func1»:

```
argdist -C 'r::func1():int:$retval'
```

Интерпретирует первый аргумент в вызове функции ядра «func1» как указатель на структуру `sk_buff` и подсчитывает разные значения в члене `len` этой структуры:

```
argdist -C 'p::func1(struct sk_buff *skb):unsigned int:skb->len'
```

Выбирает трассировки стека в пространстве ядра с частотой 99 Гц:

```
profile -K -F99
```

Подсчитывает переключения контекста по трассировкам стека:

```
stackcount -p 123 t:sched:sched_switch
```

14.5.2. bpftrace

Подсчитывает обращения к системным вызовам по процессам:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Подсчитывает обращения к системным вызовам по именам зондов системных вызовов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Подсчитывает обращения к системным вызовам по именам функций системных вызовов:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
    @[ksym(*(kaddr("sys_call_table") + args->id * 8))] = count(); }'
```

Подсчитывает вызовы функций ядра с именами, начинающимися с «attach»:

```
bpftrace -e 'kprobe:attach* { @[probe] = count(); }'
```

Измеряет время выполнения функции ядра `vfs_read()` и выводит результаты в виде гистограммы:

```
bpftrace -e 'k:vfs_read { @ts[tid] = nsecs; } kr:vfs_read /@ts[tid]/ {
    @ = hist(nsecs - @ts[tid]); delete(@ts[tid]); }'
```

Подсчитывает разные значения в первом целочисленном аргументе в вызове функции ядра «func1»:

```
bpfftrace -e 'kprobe:func1 { @[arg0] = count(); }'
```

Подсчитывает разные значения, возвращаемые функцией ядра «func1»:

```
bpfftrace -e 'kretprobe:func1 { @[retval] = count(); }'
```

Выбирает трассировки стека в пространстве ядра с частотой 99 Гц, исключая поток, обслуживающий простой системы:

```
bpfftrace -e 'profile:hz:99 /pid/ { @[kstack] = count(); }'
```

Выбирает трассировки стека функций ядра, выполняющихся на процессоре, с частотой 99 Гц:

```
bpfftrace -e 'profile:hz:99 { @[kstack(1)] = count(); }'
```

Подсчитывает количество переключений контекста по трассировкам стека:

```
bpfftrace -e 't:sched:sched_switch { @[kstack, ustack, comm] = count(); }'
```

Подсчитывает запросы к очереди заданий по функциям ядра:

```
bpfftrace -e 't:workqueue:workqueue_execute_start { @[ksym(args->function)] = count(); }'
```

Подсчитывает запуски hrtimer функциями ядра:

```
bpfftrace -e 't:timer:hrtimer_start { @[ksym(args->function)] = count(); }'
```

14.6. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ ОДНОСТРОЧНЫХ СЦЕНАРИЕВ BPF

Для иллюстрации однострочников я включил сюда некоторые примеры их вывода, как делал это для каждого инструмента.

14.6.1. Подсчет обращений к системным вызовам по именам функций системных вызовов

```
# bpfftrace -e 'tracepoint:raw_syscalls:sys_enter {
  @[ksym(*(kaddr("sys_call_table") + args->id * 8))] = count(); }'
Attaching 1 probe...
^C
[...]
@[sys_writev]: 5214
@[sys_sendto]: 5515
@[Sys_read]: 6047
```

```
@[sys_epoll_wait]: 13232
@[sys_poll]: 15275
@[Sys_ioctl]: 19010
@[sys_futex]: 20383
@[Sys_write]: 26907
@[sys_gettid]: 27254
@[sys_recvmsg]: 51683
```

Мы видим, что при трассировке чаще всего вызывалась функция `sys_recvmsg()`, которая, вероятно, соответствует системному вызову `recvmsg(2)`: 51 683 раза.

Этот однострочный сценарий использует не все точки трассировки `syscalls:sys_enter_*`, а только одну из них — `raw_syscalls:sys_enter`, что значительно ускоряет этапы инициализации и завершения. Но точка трассировки `raw_syscall` предоставляет только идентификационный номер системного вызова, поэтому сценарий преобразует его в имя функции, просматривая соответствующую запись в таблице `sys_call_table` в ядре.

14.6.2. Подсчет запусков `hrtimer` функциями ядра

```
# bpftrace -e 't:timer:hrtimer_start { @[ksym(args->function)] = count(); }'
Attaching 1 probe...
^C
```

```
@[timerfd_tmrproc]: 2
@[sched_rt_period_timer]: 2
@[watchdog_timer_fn]: 8
@[intel_uncore_fw_release_timer]: 63
@[it_real_fn]: 78
@[perf_swevent_hrtimer]: 3521
@[hrtimer_wakeup]: 6156
@[tick_sched_timer]: 13514
```

Здесь показаны функции таймера, использовавшиеся при трассировке. Сценарий зафиксировал вызов `perf_swevent_hrtimer()`, потому что в это же время `perf(1)` выполнял программное профилирование процессора. Я написал этот сценарий, чтобы проверить, какой режим профилирования использовался (событие `cpu-clock` или `cycles`), поскольку программная версия использует таймеры.

14.7. СЛОЖНОСТИ

Вот некоторые сложности, возникающие при трассировке функций ядра:

- Некоторые функции компилятор делает встроенными. Из-за этого они могут стать невидимыми для трассировки с помощью BPF. Одно из решений — трассировать родительскую или дочернюю функцию, которая компилируется не как встроенная (понадобится использовать фильтр). Другой вариант — выполнить трассировку по смещению инструкций с помощью `kprobe`.

- Некоторые функции небезопасно трассировать, поскольку они выполняются в специальных режимах, например, с отключенными прерываниями, или являются частью самой инфраструктуры трассировки. Они заносятся ядром в черный список, чтобы исключить возможность их трассировки.
- Любой инструмент на основе kprobe требует сопровождения, чтобы своевременно изменять его в соответствии с изменениями в ядре. Некоторые инструменты для ВСС уже стали неработоспособными и требуют исправлений для поддержки новых версий ядра. Долгосрочное решение — по возможности использовать точки трассировки.

14.8. ИТОГИ

В этой главе основное внимание было уделено возможностям анализа ядра, дополняющим те, что рассматривались в предыдущих главах, ориентированных на ресурсы. Сначала я представил традиционные инструменты, включая Ftrace, а затем более подробно рассмотрел вопросы анализа простоев с помощью BPF, распределения памяти в ядре, пробуждения потоков и работы очереди заданий.

Глава 15

КОНТЕЙНЕРЫ

Контейнеры стали широко используемым методом развертывания сервисов в Linux, предлагая безопасную изоляцию, быстрый запуск приложений, управление ресурсами и простоту развертывания. В этой главе я расскажу, как использовать инструменты BPF в контейнерных средах, а также покажу, чем отличаются инструменты и методы анализа для контейнеров.

Цели обучения:

- познакомиться с устройством контейнеров и целями для трассировки в них;
- понять сложности, связанные с привилегиями, идентификаторами контейнеров и FaaS;
- количественно оценивать совместное использование процессора контейнерами;
- измерять ограничение ввода/вывода в группе cgroup blk;
- измерять производительность файловой системы OverlayFS.

Эта глава начинается с описания основ, необходимых для анализа контейнеров, потом знакомит с возможностями BPF и, наконец, переходит к представлению различных инструментов BPF и однострочных сценариев.

Базовые сведения и инструменты для анализа производительности приложений в контейнерах были приведены в предыдущих главах: процессоры в контейнерах так и остаются процессорами, файловые системы — файловыми системами, а диски — дисками. Основное внимание в этой главе уделяется характерным особенностям контейнеров: пространствам имен и контрольным группам (cgroups).

15.1. ОСНОВЫ

Контейнеры позволяют запустить сразу несколько экземпляров ОС на одном компьютере. Есть два основных способа реализации контейнеров:

- **Виртуализация ОС:** предполагает разделение системы с использованием пространств имен в Linux и обычно комбинируется с контрольными группами для

управления ресурсами. Все контейнеры используют одно общее ядро. Этот подход применяется в Docker, Kubernetes и других контейнерных средах.

- **Виртуализация оборудования:** предполагает запуск облегченных виртуальных машин, каждая из которых имеет свое ядро. Этот подход применяется в Intel Clear Containers (сейчас Kata Containers [165]) и AWS Firecracker [166].

Некоторые идеи по анализу контейнеров при использовании виртуализации оборудования представлены в главе 16. А в этой главе рассмотрим контейнеры, реализованные с помощью виртуализации ОС.

На рис. 15.1 показана типичная реализация контейнера в Linux.

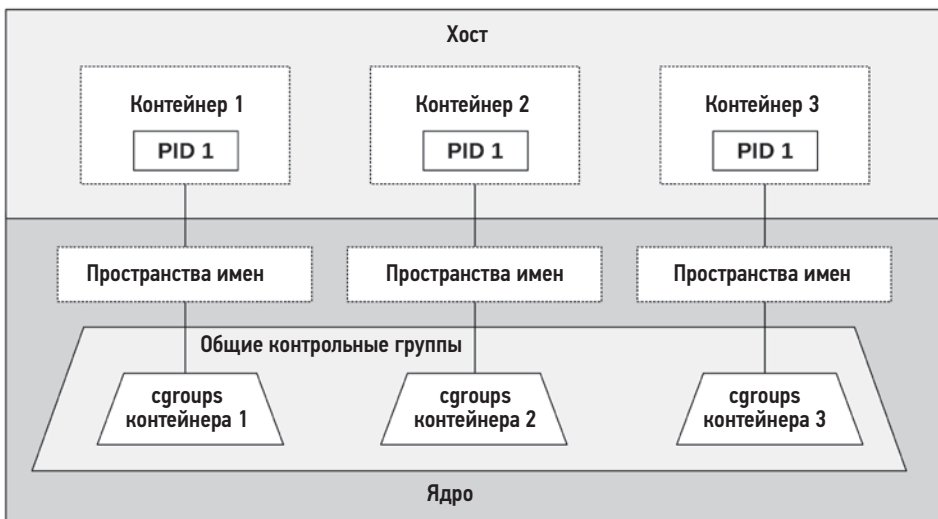


Рис. 15.1. Контейнеры в Linux, реализованные с помощью виртуализации ОС

Пространство имен ограничивает видимую часть системы. Пространства имен: `cgroup`, `ipc`, `mnt`, `net`, `pid`, `user` и `uts`. Пространство имен `pid` ограничивает видимость процессов в `/proc` только собственными процессами контейнера. Пространство имен `mnt` ограничивает видимость смонтированных файловых систем. Пространство имен `uts`¹ ограничивает детали, возвращаемые системным вызовом `uname(2)`, и т. д.

Контрольная группа ограничивает доступность ресурсов. В ядре Linux есть две версии контрольных групп: `v1` и `v2`. Многие проекты, например Kubernetes, продолжают использовать `v1`. В контрольные группы версии `v1` входят: `blkio`,

¹ Названо так в честь структуры `utsname`, используемой системным вызовом `uname(2)`, который сам унаследовал свое имя от «UNIX Time-sharing System» (система разделения времени UNIX) [167].

`cru`, `cruacct`, `cpuset`, `devices`, `hugetlb`, `memory`, `net_cls`, `net_prio`, `pids` и `rmда`. Они поддерживают возможность настройки для ограничения конфликтов между контейнерами, например, при использовании ресурсов путем установки аппаратных ограничений на использование процессора и памяти или программных ограничений на совместное использование процессора и диска. Есть и иерархия контрольных групп, включая системные, которые совместно используются контейнерами, как показано на рис. 15.1¹.

Во второй версии (v2) контрольных групп устранены различные недостатки, имеющиеся в v1. Ожидается, что в ближайшие годы контейнерные технологии перейдут на версию v2, а версия v1 в конечном итоге устареет и будет удалена из ядра.

Типичная сложность анализа производительности контейнеров — это потенциальные «шумные соседи»: арендаторы контейнеров, которые активно потребляют ресурсы и вызывают конкуренцию за доступ к ним. Поскольку все контейнерные процессы выполняются под управлением одного ядра и могут анализироваться на уровне хоста, их анализ ничем не отличается от традиционного анализа производительности нескольких приложений, выполняющихся в одной системе разделения времени. Основное отличие заключается в использовании контрольных групп, накладывающих дополнительные программные ограничения на ресурсы, которые обычно меньше аппаратных ограничений. Инструменты мониторинга, которые не поддерживают контейнеры, могут не замечать этих программных ограничений и вызываемых ими проблем с производительностью.

15.1.1. Возможности BPF

Инструменты анализа контейнеров обычно основаны на метриках, показывающих, какие существуют контейнеры, контрольные группы и пространства имен, их настройки и размеры. Инструменты трассировки BPF могут дать гораздо больше деталей и ответить на вопросы:

- Как долго процессы в каждом контейнере находятся в очереди на выполнение?
- Использует ли планировщик один и тот же процессор, переключаясь между контейнерами?
- Достигнуто ли программное ограничение на использование процессора или диска?

Ответы на них можно получить с помощью BPF, инструментируя точки трассировки для событий планировщика и зонды `kprobes` для функций ядра. В предыдущих главах я говорил, что некоторые из этих событий (например, планирование) могут происходить очень часто и их трассировка — это удел специального анализа, а не непрерывного мониторинга.

¹ Для обозначения контрольных групп я использовал трапеции, чтобы подчеркнуть их отличие от пространств имен, изображенных прямоугольниками, и диапазон применения для наложения ограничений от аппаратных до программных.

Есть точки трассировки для событий контрольных групп, включая `cgroup:cgroup_setup_root`, `cgroup:cgroup_attach_task` и др. Это высокоуровневые события, которые помогают в отладке запуска контейнера.

Есть и возможность писать программы BPF с типом `BPF_PROG_TYPE_CGROUP_SKB` (не показаны в этой главе) для анализа сетевых пакетов и подключать их к контрольным группам на входе и на выходе.

15.1.2. Сложности

В подразделах ниже описаны некоторые сложности, свойственные трассировке контейнеров с использованием BPF.

Привилегии для BPF

На момент написания книги для выполнения трассировки с помощью BPF требуются привилегии `root`, и для большинства контейнерных сред это означает, что трассировка средствами BPF может выполняться только с хоста, но не из контейнеров. В будущем это требование должно быть ослаблено: обсуждается возможность непривилегированного доступа к BPF специально для решения проблемы с контейнерами¹. Об этом также шла речь в разделе 11.1.2.

Идентификаторы контейнеров

Управление идентификаторами контейнеров, которые используются в Kubernetes и Docker, осуществляется ПО, действующим в пространстве пользователя. Например (выделено жирным шрифтом):

```
# kubectl get pod
NAME                                READY  STATUS             RESTARTS  AGE
kubernetes-b94cb9bff-kqvm1  0/1    ContainerCreating  0          3m
[...]
```

```
# docker ps
CONTAINER ID  IMAGE    COMMAND    CREATED      STATUS      PORTS    NAMES
6280172ea7b9  ubuntu  "bash"     4 weeks ago  Up 4 weeks  eager_bhaskara
[...]
```

Контейнер представлен в ядре набором контрольных групп и пространств имен, но в пространстве ядра нет идентификатора, связывающего их воедино. В свое время предлагалось добавить идентификатор контейнера в ядро [168], но пока это не сделано.

Это может вызвать проблемы при запуске инструментов трассировки BPF на уровне хоста (как они обычно и запускаются: см. подраздел «Привилегии для BPF»

¹ Я пишу эти строки с трека BPF на саммите LSFMM Summit 2019, где дискуссии проходят в режиме реального времени.

в разделе 15.1.2). Выполняясь на уровне хоста, инструменты трассировки BPF перехватывают события из всех контейнеров, поэтому лучше иметь возможность оставлять только события, принадлежащие определенному контейнеру, или группировать их по контейнерам. Но в ядре нет идентификатора контейнера, который можно использовать для этого.

К счастью, есть ряд обходных решений, каждое из которых зависит от конкретной конфигурации исследуемых контейнеров. Контейнеры используют некоторую комбинацию пространств имен. Детали этой комбинации можно прочитать из структуры `nsproxy` в ядре. Вот определение этой структуры в `linux/nsproxy.h`:

```
struct nsproxy {
    atomic_t count;
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns_for_children;
    struct net            *net_ns;
    struct cgroup_namespace *cgroup_ns;
};
```

Контейнеры почти всегда используют пространство имен PID, которое позволяет различать их. Вот пример получения пространства имен для текущей задачи из `bpfttrace`:

```
#include <linux/sched.h>
[...]
$task = (struct task_struct *)curtask;
$pidns = $task->nsproxy->pid_ns_for_children->ns.inum;
```

Этот код записывает в `$pidns` идентификатор (целое число) пространства имен PID, который можно вывести или использовать для фильтрации. Он будет соответствовать идентификатору пространства имен PID, указанному в символической ссылке `/proc/PID/ns/pid_for_children`.

Если среда выполнения контейнера использует пространство имен UTS и в качестве имени узла берет имя контейнера (как это часто бывает в Kubernetes и Docker), то это имя тоже можно использовать в программах BPF для идентификации контейнеров. Вот пример с применением синтаксиса `bpfttrace`:

```
#include <linux/sched.h>
[...]
$task = (struct task_struct *)curtask;
$nodename = $task->nsproxy->uts_ns->name.nodename;
```

Именно так реализован инструмент `pidnss(8)` (см. раздел 15.3.2).

При анализе подов Kubernetes идентификатором будет сетевое пространство имен, поскольку контейнеры в поде почти всегда используют одно и то же сетевое пространство имен.

Эти идентификаторы, включая идентификатор пространства имен PID или строку с именем узла UTS вместе с PID, можно добавить в инструменты из предыдущих глав и тем самым сделать их совместимыми с контейнерами. Обратите внимание, что это возможно, только если точка инструментации находится в контексте процесса, когда есть действительная структура `curtask`.

Оркестрация

Запуск инструментов BPF на нескольких хостах с контейнерами — это проблема, похожая на развертывание облака на нескольких виртуальных машинах. В вашей компании уже может быть нужное ПО оркестрации, которое умеет запускать заданную команду на нескольких хостах и собирать выходные данные. Есть и специализированные решения, в том числе `kubectl-trace`.

`kubectl-trace` — это планировщик Kubernetes для запуска программ `bpfttrace` в кластере Kubernetes. Он предоставляет переменную `$container_pid` для использования в программах `bpfttrace`, содержащую идентификатор `pid` корневого процесса. Например, следующая команда:

```
kubectl trace run -e 'k:vfs* /pid == $container_pid/ { @[probe] = count() }' mypod -a
```

считает вызовы функции ядра `vfs*()` из приложения в контейнере `mypod`, пока вы не нажмете `Ctrl-C`. Программы могут быть однострочными сценариями, как в этом примере, или извлекаться из файлов с помощью параметра `-f [169]`. Более подробно `kubectl-trace` рассмотрен в главе 17.

Function as a Service (FaaS)

Новая модель вычислений предполагает определение прикладных функций, которые выполняются провайдером, возможно, в контейнерах. Конечный пользователь определяет только функции и может не иметь SSH-доступа к системе, в которой эти функции выполняются. Ожидается, что такая среда не будет поддерживать конечных пользователей, использующих инструменты трассировки BPF. (Она также не сможет запускать другие инструменты.) Когда ядро будет поддерживать непривилегированную трассировку BPF, прикладная функция сможет напрямую выполнять вызовы BPF, но это создаст множество проблем. Анализ FaaS с использованием BPF, вероятно, будет возможен только на уровне хоста и только пользователями или интерфейсами, имеющими доступ к хосту.

15.1.3. Стратегия

Если вы еще только осваиваете анализ контейнеров, то, вероятно, не знаете, с чего начать — с какой цели и с какого инструмента. Я приведу общую стратегию для старта. Упомянутые здесь инструменты подробно рассмотрены в последующих разделах.

1. Исследуйте систему и выясните нехватку ресурсов и другие проблемы, рассмотренные в предыдущих главах (6, 7 и т. д.). В частности, создайте флейм-графики потребления процессора для работающих приложений.
2. Проверьте, не было ли достигнуто ограничение, накладываемое контрольной группой.
3. Подберите и запустите инструменты BPF, перечисленные в главах с 6-й по 14-ю.

Большинство встречавшихся мне сложностей с контейнерами были обусловлены проблемами в приложениях или в оборудовании, но не конфигурацией контейнера. Флейм-графики потребления процессора часто помогают выявить проблемы в приложениях, которые никак не связаны с выполнением внутри контейнеров. Обязательно проверьте такие проблемы и исследуйте ограничения контейнера.

15.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Работу контейнеров можно анализировать с помощью многих инструментов оценки производительности из предыдущих глав. В этом разделе обобщаются особенности анализа контейнеров традиционными инструментами как на уровне хоста, так и внутри контейнеров¹.

15.2.1. Анализ на уровне хоста

Для анализа поведения контейнера, особенно использования контрольных групп, можно брать инструменты и метрики, доступные на уровне хоста и перечисленные в табл. 15.1.

Таблица 15.1. Традиционные инструменты для анализа поведения контейнеров на уровне хоста

Инструмент	Тип	Описание
systemd-cgtop	Статистика ядра	Показывает топ потребителей вычислительных ресурсов среди контрольных групп
kubectl top	Статистика ядра	Показывает топ потребителей ресурсов в Kubernetes
docker stats	Статистика ядра	Показывает потребление ресурсов контейнером Docker
/sys/fs/cgroups	Статистика ядра	Исходные характеристики контрольных групп
perf	Статистика и трассировка	Многоцелевой инструмент, поддерживающий фильтрацию по контрольным группам

В разделах ниже дано общее описание основных возможностей этих инструментов.

¹ Дополнительную информацию по этой теме ищите в видео «Linux Container Performance Analysis» и презентации на USENIX LISA 2017 [Gregg 17].

15.2.2. Анализ на уровне контейнера

Традиционные инструменты также можно применять на уровне отдельных контейнеров, но при этом важно помнить, что значения некоторых характеристик будут относиться ко всему хосту, а не только к контейнеру. В табл. 15.2 перечислены некоторые часто используемые инструменты и их описание, действительное для ядра Linux 4.8.

Таблица 15.2. Традиционные инструменты для анализа на уровне контейнера

Инструмент	Описание
top(1)	В таблице процессов отображаются только процессы, выполняющиеся в текущем контейнере, но суммарные показатели в заголовочной части соответствуют хосту в целом
ps(1)	Выводит список процессов в контейнере
uptime(1)	Выводит статистики хоста, включая средние значения нагрузки
mpstat(1)	Выводит статистику по процессорам хоста и потребление процессоров хоста
vmstat(8)	Выводит статистику потребления процессоров хоста, памяти и других ресурсов
iostat(1)	Выводит статистику потребления дисков хоста
free(1)	Выводит статистику потребления памяти хоста

Обратите внимание, что под термином *совместимые с контейнерами* подразумеваются инструменты, которые при использовании внутри контейнера отображают только процессы и ресурсы контейнера. Ни один из инструментов этой таблицы не является полностью совместимым с контейнерами. По мере развития ядра и инструментов ситуация может измениться, но пока сложность анализа производительности в контейнерах остается.

15.2.3. systemd-cgtop

Команда `systemd-cgtop(1)` выводит самые ресурсоемкие контрольные группы. Вот пример, полученный на промышленном хосте с контейнерами:

```
# systemd-cgtop
Control Group           Tasks   %CPU    Memory  Input/s  Output/s
/                        -       798.2   45.9G   -         -
/docker                 1082    790.1   42.1G   -         -
/docker/dcf3a...9d28fc4a1c72bbaff4a24834  200    610.5   24.0G   -         -
/docker/370a3...e64ca01198f1e843ade7ce21  170    174.0   3.0G    -         -
/system.slice           748     5.3     4.1G    -         -
/system.slice/daemontools.service          422    4.0     2.8G    -         -
/docker/dc277...42ab0603bbda2ac8af67996b  160     2.5     2.3G    -         -
/user.slice              5       2.0     34.5M   -         -
/user.slice/user-0.slice          5       2.0     15.7M   -         -
/user.slice/u....slice/session-c26.scope    3       2.0     13.3M   -         -
```

```

/docker/ab452...c946f8447f2a4184f3ccff2a    174    1.0    6.3G    -    -
/docker/e18bd...26ffdd7368b870aa3d1deb7a    156    0.8    2.9G    -    -
[...]
```

Мы видим, что в течение интервала обновления контрольная группа «/docker/dfc3a...» потребила 610.5% процессорного времени (на нескольких процессорах) и 24 Гбайт оперативной памяти на выполнение 200 задач. В выводе видны и контрольные группы, созданные демоном `systemd` для сервисов (`/system.slice`) и пользовательских сеансов (`/user.slice`).

15.2.4. kubectl top

Kubernetes предлагает команду `kubectl top` для проверки использования основных ресурсов. Вот пример проверки хостов (узлов, «nodes»):

```

# kubectl top nodes
NAME                                CPU(cores)   CPU%   MEMORY(bytes)  MEMORY%
bgregg-i-03cb3a7e46298b38e        1781m        10%    2880Mi         9%
```

В столбце «CPU(cores)» отображается накопленное процессорное время в миллисекундах, а в столбце «CPU%» — текущее потребление процессора узлом. А вот пример проверки контейнеров (подов, «pods»):

```

# kubectl top pods
NAME                                CPU(cores)   MEMORY(bytes)
kubernetes-b94cb9bff-p7jsp         73m          9Mi
```

Здесь отображается накопленное процессорное время и текущий объем занятой памяти.

Для правильной работы этих команд должен быть запущен сервер метрик, что может происходить по умолчанию, в зависимости от того, как был инициализирован Kubernetes [170]. Есть и другие инструменты мониторинга, отображающие эти метрики в графическом интерфейсе: `cAdvisor`, `Sysdig` и `Google Cloud Monitoring` [171].

15.2.5. docker stats

Docker предлагает несколько подкоманд команды `docker(1)` для анализа, включая `stats`. Вот пример с промышленного хоста:

```

# docker stats
CONTAINER    CPU %    MEM USAGE / LIMIT    MEM %    NET I/O    BLOCK I/O    PIDS
353426a09db1  526.81%  4.061 GiB / 8.5 GiB  47.78%    0 B / 0 B    2.818 MB / 0 B    247
6bf166a66e08  303.82%  3.448 GiB / 8.5 GiB  40.57%    0 B / 0 B    2.032 MB / 0 B    267
58dcf8aed0a7  41.01%   1.322 GiB / 2.5 GiB  52.89%    0 B / 0 B    0 B / 0 B         229
61061566ffe5  85.92%   220.9 MiB / 3.023 GiB  7.14%     0 B / 0 B    43.4 MB / 0 B     61
bdc721460293  2.69%    1.204 GiB / 3.906 GiB  30.82%    0 B / 0 B    4.35 MB / 0 B     66
[...]
```

Здесь видно, что за интервал обновления контейнер с UUID «353426a09db1» потребил в общей сложности 527% процессорного времени и использовал 4 Гбайт оперативной памяти из доступных 8.5 Гбайт. В течение этого интервала сетевой ввод/вывод отсутствовал и был небольшой объем (несколько мегабайт) дискового ввода/вывода.

15.2.6. /sys/fs/cgroups

Этот каталог содержит виртуальные файлы со статистиками контрольных групп. Они используются и отображаются в виде графиков различными продуктами для мониторинга контейнеров. Например:

```
# cd /sys/fs/cgroup/cpu,cpuacct/docker/02a7cf65f82e3f3e75283944caa4462e82f...
# cat cpuacct.usage
1615816262506
# cat cpu.stat
nr_periods 507
nr_throttled 74
throttled_time 3816445175
```

Файл `cpuacct.usage` содержит общее процессорное время в наносекундах, использованное этой контрольной группой. Файл `cpu.stat` сообщает, сколько раз потребление процессора этой контрольной группой подвергалось ограничению (`nr_throttled`), а также общее время ограничения в наносекундах. Здесь видно, что потребление процессора этой контрольной группой ограничивалось 74 раза в течение 507 периодов, и в общей сложности из-за ограничений группа недополучила 3.8 секунды.

Есть и файл `cpuacct.usage_percpu`, показывающий время, затраченное контрольной группой Kubernetes на каждом процессоре:

```
# cd /sys/fs/cgroup/cpu,cpuacct/kubepods/burstable/pod82e745...
# cat cpuacct.usage_percpu
37944772821 35729154566 35996200949 36443793055 36517861942 36156377488 36176348313
35874604278 37378190414 35464528409 35291309575 35829280628 36105557113 36538524246
36077297144 35976388595
```

Для этой системы с 16 процессорами вывод содержит 16 полей с общим временем, потраченным этой контрольной группой на каждом процессоре.

Метрики контрольных групп версии v1 задокументированы в исходном коде ядра, в файле `Documentation/cgroup-v1/cpuacct.txt` [172].

15.2.7. perf

Инструмент `perf(1)` из главы 6 можно использовать на уровне хоста и фильтровать данные по контрольным группам с применением параметра `--cgroup (-G)`. С помощью подкоманды `perf record` можно профилировать потребление процессора:

```
perf record -F 99 -e cpu-clock --cgroup=docker/1d567... -a -- sleep 30
```

Событием может быть все, что происходит в контексте процесса, включая системные вызовы.

Этот параметр доступен и в подкоманде `perf stat`. Он позволяет подсчитывать количество событий вместо их записи в файл `perf.data`. Вот пример подсчета обращений к системным вызовам из семейства `read` и использования другого формата определения контрольных групп (с опущенными идентификаторами):

```
perf stat -e syscalls:sys_enter_read* --cgroup /containers.slice/5aad.../...
```

Можно указать несколько контрольных групп.

`perf(1)` может трассировать те же события, что и BPF, но не предлагает возможности программирования, которые поддерживают BCC и `bpftrace`. В действительности `perf(1)` имеет свой интерфейс BPF: пример вы найдете в приложении D. На странице с примерами в [73] вы найдете другие варианты применения `perf` для анализа контейнеров.

15.3. ИНСТРУМЕНТЫ BPF

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа работы контейнеров и устранения неполадок. Эти инструменты находятся в репозитории BCC или были созданы специально для этой книги. Они перечислены в табл. 15.3.

Таблица 15.3. Инструменты для анализа контейнеров

Инструмент	Источник	Цель	Описание
<code>runqlat</code>	BCC	Планировщик	Обобщает время ожидания в очереди на выполнения, группируя результаты по пространству имен PID
<code>pidnss</code>	Книга	Планировщик	Подсчитывает переключения контекста: контейнеров, совместно использующих процессор
<code>blkthrot</code>	Книга	Блочный ввод/вывод	Подсчитывает количество ограничений блочного ввода/вывода в контрольной группе <code>blk</code>
<code>overlayfs</code>	Книга	Файловая система OverlayFS	Отображает задержки чтения и записи в файловой системе OverlayFS

Для анализа контейнеров используйте их вместе с другими инструментами из предыдущих глав.

15.3.1. `runqlat`

`runqlat(8)` был представлен в главе 6: он показывает задержки в очереди на выполнение в виде гистограмм, помогая выявлять проблемы с насыщением CPU.

Поддерживает параметр `--pidnss`, при использовании которого показывает пространство имен PID. Вот пример из промышленной системы с контейнерами:

```
host# runqlat --pidnss -m
Tracing run queue latency... Hit Ctrl-C to end.
^C
pidns = 4026532382
  msecs          : count      distribution
    0 -> 1       : 646        |*****|
    2 -> 3       : 18         |*      |
    4 -> 7       : 48         |**     |
    8 -> 15      : 17         |*      |
   16 -> 31      : 150        |*****|
   32 -> 63      : 134        |*****|
[...]
```

msecs	count	distribution
0 -> 1	646	*****
2 -> 3	18	*
4 -> 7	48	**
8 -> 15	17	*
16 -> 31	150	*****
32 -> 63	134	*****

```
[...]
pidns = 4026532870
  msecs : count distribution
    0 -> 1 : 264        |*****|
    2 -> 3 : 0          |        |
[...]
```

Мы видим, что одно из пространств имен PID (4026532382) страдает от значительно большей задержки в очереди на выполнение, чем другое.

Этот инструмент не сообщает имена контейнеров, потому что соответствие пространства имен контейнеру зависит от используемой контейнерной технологии. Для определения пространства имен, которому принадлежит конкретный процесс, можно использовать команду `ls(1)`, запуская ее с привилегиями `root`. Например:

```
# ls -lh /proc/181/ns/pid
lrwxrwxrwx 1 root root 0 May 6 13:50 /proc/181/ns/pid -> 'pid:[4026531836]'
```

Здесь видно, что процесс с идентификатором PID 181 принадлежит пространству имен PID 4026531836.

15.3.2. pidnss

`pidnss(8)`¹ подсчитывает переключения контекста между контейнерами по смене пространства имен PID. Этот инструмент можно использовать, чтобы убедиться, нет ли проблемы конкуренции нескольких контейнеров за один процессор. Например:

```
# pidnss.bt
Attaching 3 probes...
Tracing PID namespace switches. Ctrl-C to end
^C
Victim PID namespace switch counts [PIDNS, nodename]:
```

¹ Немного истории: я написал его для этой книги 6 мая 2019 года по предложению моего коллеги Саргуна Диллона.

```
@[0, ]: 2
@[4026532981, 6280172ea7b9]: 27
@[4026531836, bgregg-i-03cb3a7e46298b38e]: 28
```

В каждой строке инструмент выводит два поля (идентификатор пространства имен PID и имя узла, если есть) и количество переключений. Как показывает этот вывод, за время трассировки пространство имен PID с именем узла «bgregg-i-03cb3a7e46298b38e» (хост) переключалось на другое пространство имен 28 раз, а другое пространство имен PID с именем узла «6280172ea7b9» (контейнер Docker) переключалось 27 раз. Эти данные можно подтвердить на уровне хоста:

```
# uname -n
bgregg-i-03cb3a7e46298b38e
# docker ps
CONTAINER ID  IMAGE    COMMAND    CREATED      STATUS      PORTS      NAMES
6280172ea7b9  ubuntu  "bash"     4 weeks ago  Up 4 weeks                eager_bhaskara
[...]
```

pidnss(8) трассирует переключение контекста с помощью kprobes. Ожидается, что в системах с большим объемом ввода/вывода он будет иметь значительный оверхед.

Вот еще один пример, полученный во время настройки кластера Kubernetes:

```
# pidnss.bt
Attaching 3 probes...
Tracing PID namespace switches. Ctrl-C to end
^C
Victim PID namespace switch counts [PIDNS, nodename]:

@[-268434577, cilium-operator-95ddb5fc-gkspv]: 33
@[-268434291, cilium-etcd-g9wgxqsnjv]: 35
@[-268434650, coredns-fb8b8dccf-w7khw]: 35
@[-268434505, default-mem-demo]: 36
@[-268434723, coredns-fb8b8dccf-crrn9]: 36
@[-268434509, etcd-operator-797978964-7c2mc]: 38
@[-268434513, kubernetes-b94cb9bff-p7jsp]: 39
@[-268434810, bgregg-i-03cb3a7e46298b38e]: 203
[...]
@[-268434222, cilium-etcd-g9wgxqsnjv]: 597
@[-268434295, etcd-operator-797978964-7c2mc]: 1301
@[-268434808, bgregg-i-03cb3a7e46298b38e]: 1582
@[-268434297, cilium-operator-95ddb5fc-gkspv]: 3961
@[0, ]: 8130
@[-268434836, bgregg-i-03cb3a7e46298b38e]: 8897
@[-268434846, bgregg-i-03cb3a7e46298b38e]: 15813
@[-268434581, coredns-fb8b8dccf-w7khw]: 39656
@[-268434654, coredns-fb8b8dccf-crrn9]: 40312
[...]
```

Исходный код pidnss(8):

```
#!/usr/local/bin/bpfttrace

#include <linux/sched.h>
```

```

#include <linux/nsproxy.h>
#include <linux/utsname.h>
#include <linux/pid_namespace.h>

BEGIN
{
    printf("Tracing PID namespace switches. Ctrl-C to end\n");
}

kprobe:finish_task_switch
{
    $prev = (struct task_struct *)arg0;
    $curr = (struct task_struct *)curtask;
    $prev_pidns = $prev->nsproxy->pid_ns_for_children->ns.inum;
    $curr_pidns = $curr->nsproxy->pid_ns_for_children->ns.inum;
    if ($prev_pidns != $curr_pidns) {
        @[$prev_pidns, $prev->nsproxy->uts_ns->name.nodename] = count();
    }
}

END
{
    printf("\nVictim PID namespace switch counts [PIDNS, nodename]:\n");
}

```

Этот код — пример извлечения идентификаторов пространств имен. Идентификаторы других пространств имен извлекаются похожим способом.

Если вам нужно получить подробности о конкретном контейнере, помимо пространства имен и контрольной группы, то этот инструмент можно перенести в BCC и реализовать в нем извлечение данных непосредственно из Kubernetes, Docker и т. д.

15.3.3. blkthrot

`blkthrot(8)`¹ подсчитывает, сколько раз контрольная группа `blk` блочного ввода/вывода ограничивалась по достижении аппаратного предела. Например:

```

# blkthrot.bt
Attaching 3 probes...
Tracing block I/O throttles by cgroup. Ctrl-C to end
^C

```

```
@notthrottled[1]: 506
```

```
@throttled[1]: 31
```

В ходе трассировки я увидел, что контрольная группа `blk` с ID 1 ограничивалась 31 раз и не ограничивалась 506 раз.

¹ Немного истории: я написал его для этой книги 6 мая 2019 года.

blkthrot(8) трассирует функцию ядра blk_throtl_bio(). Оверхед должен быть небольшим, потому что обычно события блочного ввода/вывода следуют с относительно невысокой частотой.

Исходный код blkthrot(8):

```
#!/usr/local/bin/bpftrace

#include <linux/cgroup-defs.h>
#include <linux/blk-cgroup.h>

BEGIN
{
    printf("Tracing block I/O throttles by cgroup. Ctrl-C to end\n");
}

kprobe:blk_throtl_bio
{
    @blkg[tid] = arg1;
}

kretprobe:blk_throtl_bio
/@blkg[tid]/
{
    $blkg = (struct blkcg_gq *)@blkg[tid];
    if (retval) {
        @throttled[$blkg->blkcg->css.id] = count();
    } else {
        @notthrottled[$blkg->blkcg->css.id] = count();
    }
    delete(@blkg[tid]);
}
```

Этот код — пример извлечения идентификаторов контрольных групп, он находится в структуре cgroup_subsys_state, в данном случае как css в blkcg.

То же самое можно реализовать иначе: проверять наличие флага BIO_THROTTLED в структуре bio после завершения блока.

15.3.4. overlayfs

overlayfs(8)¹ трассирует задержки чтения и записи в OverlayFS. Подобные файловые системы широко используются в контейнерах, поэтому инструмент помогает получить представление о производительности файловой системы контейнера. Например:

```
# overlayfs.bt 4026532311
Attaching 7 probes...
21:21:06 -----
```

¹ Немного истории: этот инструмент написал мой коллега Джейсон Кох 18 марта 2019 года, когда занимался проблемой производительности контейнеров.

```
@write_latency_us:
[128, 256)          1 |
[256, 512)        238 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

@read_latency_us:
[1]                 3 | @
[2, 4)              1 | 
[4, 8)              3 | @
[8, 16)             0 | 
[16, 32)           115 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32, 64)           123 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64, 128)           0 | 
[128, 256)          1 | 

21:21:07 -----
[...]

```

Инструмент отображает распределение задержек чтения и записи. Как показывает вывод, в интервале от 21:21:06 до 21:21:06 задержки чтения обычно составляли от 16 до 64 микросекунд.

overlaysfs(8) трассирует функции ядра из структуры `file_operations_t` в `overlaysfs`. Оверхед зависит от частоты вызова этих функций и для большинства рабочих нагрузок должен быть незначительным.

Исходный код overlays (8):

```
#!/usr/local/bin/bpftrace

#include <linux/nsproxy.h>
#include <linux/pid_namespace.h>

kprobe:ovl_read_iter
/((struct task_struct *)curtask)->nsproxy->pid_ns_for_children->ns.inum == $1/
{
    @read_start[tid] = nsecs;
}

kretprobe:ovl_read_iter
/((struct task_struct *)curtask)->nsproxy->pid_ns_for_children->ns.inum == $1/
{
    $duration_us = (nsecs - @read_start[tid]) / 1000;
    @read_latency_us = hist($duration_us);
    delete(@read_start[tid]);
}

kprobe:ovl_write_iter
/((struct task_struct *)curtask)->nsproxy->pid_ns_for_children->ns.inum == $1/
{
    @write_start[tid] = nsecs;
}

kretprobe:ovl_write_iter
/((struct task_struct *)curtask)->nsproxy->pid_ns_for_children->ns.inum == $1/
{
}
```

```

    $duration_us = (nsecs - @write_start[tid]) / 1000;
    @write_latency_us = hist($duration_us);
    delete(@write_start[tid]);
}

interval:ms:1000
{
    time("\n%H:%M:%S ----- \n");
    print(@write_latency_us);
    print(@read_latency_us);
    clear(@write_latency_us);
    clear(@read_latency_us);
}

END
{
    clear(@write_start);
    clear(@read_start);
}

```

Функции `ovl_read_iter()` и `ovl_write_iter()` были добавлены в Linux 4.19. Этот инструмент принимает аргумент с идентификатором пространства имен PID: он создавался для работы с Docker и запускался из следующего сценария командной оболочки (`overlayfs.sh`), который принимает аргумент с идентификатором контейнера Docker.

```

#!/bin/bash

PID=$(docker inspect -f='{{.State.Pid}}' $1)
NSID=$(stat /proc/$PID/ns/pid -c "%N" | cut -d[ -f2 | cut -d] -f1)

bpftrace ./overlayfs.bt $NSID

```

Вы можете изменить его, чтобы привести в соответствие с используемой контейнерной технологией. Необходимость этого шага обсуждается в разделе 15.1.2: в ядре нет ID контейнера, он есть только в пространстве пользователя. Этот сценарий преобразует идентификатор контейнера в пространство имен PID, с которым может работать ядро.

15.4. ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPF

Здесь перечислены однострочные сценарии для `bpftrace`.

Подсчитывает идентификаторы контрольных групп с частотой 99 Гц:

```
bpftrace -e 'profile:hz:99 { @[cgroup] = count(); }'
```

Трассирует имена открытых файлов для `cgroup v2` с именем «`container1`»:

```
bpftrace -e 't:syscalls:sys_enter_openat
/cgroup == cgroupid("/sys/fs/cgroup/unified/container1")/ {
printf("%s\n", str(args->filename)); }'
```

15.5. ДОПОЛНИТЕЛЬНЫЕ УПРАЖНЕНИЯ

Упражнения можно выполнить с помощью `bpftool` или `BCC`, если явно не указано иное.

1. Измените `runqlat(8)` из главы 6 и добавьте вывод имени узла из пространства имен UTS (см. `pidnss(8)`).
2. Измените `opensnoop(8)` из главы 8 и добавьте вывод имени узла из пространства имен UTS.
3. Разработайте инструмент, показывающий, какие контейнеры вытесняются из оперативной памяти из-за ограничений в контрольной группе `mem` (см. функцию ядра `mem_cgroup_swapout()`).

15.6. ИТОГИ

В этой главе я рассмотрел контейнеры Linux и показал, как механизм трассировки BPF может выявлять конкуренцию между контейнерами за обладание процессором, регулирование доступности ресурсов контрольными группами, а также задержки в OverlayFS.

Глава 16

ГИПЕРВИЗОРЫ

В этой главе рассмотрим использование инструментов BPF с гипервизорами виртуальных машин для виртуализации оборудования. Популярные примеры — Xen и KVM. Инструменты BPF с виртуализацией ОС — контейнеры — обсуждались в предыдущей главе.

Цели обучения:

- познакомиться с устройством гипервизоров и возможностями трассировки с BPF;
- трассировать гостевые гипервызовы и выходы, где это возможно;
- научиться измерять недополученное процессорное время.

Эта глава начинается с описания основ, необходимых для анализа виртуализации оборудования, потом знакомит с возможностями BPF и стратегиями анализа разных ситуаций с гипервизорами. В заключение я дам несколько примеров инструментов BPF.

16.1. ОСНОВЫ

Виртуализация оборудования заключается в создании виртуальной машины, в которой можно запустить полноценную ОС, включая ее ядро. На рис. 16.1 показаны две наиболее распространенные конфигурации гипервизоров.

Согласно распространенной классификации, гипервизоры делятся на два типа: тип 1 и тип 2 [Goldberg 73]. Но с развитием технологий различия между типами практически исчезли [173]: тип 2 стал похож на тип 1 из-за использования модулей ядра. Мы будем придерживаться другой распространенной классификации, основанной на различиях конфигураций, как показано на рис. 16.1:

- **Конфигурация А:** гипервизоры, выполняющиеся непосредственно на аппаратной платформе, без операционной системы. ПО гипервизора выполняется непосредственно на процессорах, создает домены для запуска гостевых виртуальных машин и распределяет виртуальные гостевые процессоры между физическими процессорами. Привилегированный домен (с номером 0 на рис. 16.1) может

использоваться для администрирования остальных доменов. Популярный пример — гипервизор Xen.

- **Конфигурация В:** ПО гипервизора выполняется ядром ОС хоста и может состоять из модулей ядра и процессов, выполняющихся в пространстве пользователя. ОС хоста имеет привилегии, необходимые для администрирования гипервизора, а ее ядро распределяет процессорное время между виртуальными машинами и другими процессами в ОС хоста. Благодаря использованию модулей ядра, эта конфигурация обеспечивает прямой доступ к оборудованию. Популярный пример — гипервизор KVM.

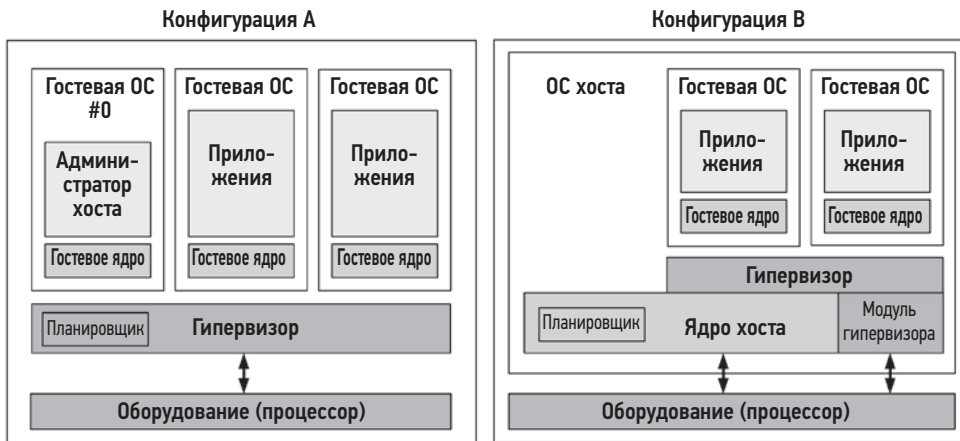


Рис. 16.1. Распространенные конфигурации гипервизоров

Обе конфигурации могут предусматривать запуск прокси-сервера ввода/вывода (например, программного обеспечения QEMU) в домене 0 (Xen) или в ОС хоста (KVM) для обслуживания гостевого ввода/вывода. Это решение увеличивает overhead на ввод/вывод, но с годами оно было оптимизировано за счет добавления транспортов с разделяемой памятью и других технологий.

Первоначальный гипервизор оборудования, впервые предложенный компанией VMware в 1998 году, использовал двоичную трансляцию для выполнения полной виртуализации оборудования [VMware 07]. С тех пор были добавлены улучшения:

- **Поддержка виртуализации процессора:** в 2005–2006 годах в реализацию добавили использование расширений AMD-V и Intel VT-x для ускорения операций виртуальной машины на процессоре.
- **Паравиртуализация (Paravirtualization, PV):** вместо выполнения ОС в исходном виде паравиртуализация позволяет уведомить гостевую ОС, что она выполняется в виртуальной машине и может производить специальные вызовы (гипервызовы) гипервизора для более эффективной обработки некоторых операций. Для эффективности Xen группирует эти гипервызовы в мультивызовы.

- **Аппаратная поддержка устройств:** для еще большей оптимизации производительности виртуальных машин в аппаратные устройства кроме процессоров добавили поддержку виртуальных машин. Сюда входит SR-IOV для сетевых устройств и устройств хранения, а также специальные драйверы для их использования: `ixgbe`, `e1000` и `nvme`.

С годами гипервизор Xen развился и улучшил производительность. Современные виртуальные машины Xen часто загружаются в режиме аппаратной виртуализации (Hardware VM, HVM), а затем используют драйверы PV с поддержкой HVM, чтобы получить лучшее из двух миров: эта конфигурация получила название PVHVM. Ее можно еще больше усовершенствовать, если применить полную аппаратную виртуализацию для некоторых драйверов, например SR-IOV, сетевых устройств и устройств хранения.

В 2017 году AWS выпустила гипервизор Nitro с компонентами на основе KVM и аппаратной поддержкой всех основных ресурсов: процессоров, сетевых карт, хранилищ, прерываний и таймеров [174]. Прокси-сервер QEMU не используется.

16.1.1. Возможности BPF

Поскольку в каждой виртуальной машине выполняется собственное ядро, для анализа работы виртуальных машин можно применять инструменты BPF гостевой системы. При использовании на уровне гостевой системы BPF может ответить на вопросы:

- Какова производительность виртуализированных аппаратных ресурсов? Здесь можно применить инструменты, описанные в предыдущих главах.
- Какова задержка гипервызова при использовании паравиртуализации, которая служит мерой производительности гипервизора?
- Как часто и в каком объеме виртуальные машины недополучают процессорное время?
- Мешают ли работе приложений обработчики прерываний гипервизора?

При использовании на уровне хоста BPF помогает ответить на дополнительные вопросы (физические хосты доступны облачным провайдерам, но недоступны конечным пользователям):

- Если используется QEMU, то какая рабочая нагрузка производится гостевой системой? Какая в результате получается производительность?
- По каким причинам гостевые системы выходят в гипервизор при использовании конфигурации B?

Анализ гипервизора оборудования с помощью BPF — еще одна область, где в будущем появятся новые возможности. Потенциальные направления развития упоминаются в разделах, посвященных инструментам.

Гостевые системы в AWS EC2

Как отмечалось выше, гипервизоры оптимизируют производительность, поэтому при переходе от эмуляции к паравиртуализации в поддержке оборудования остается меньше целей, доступных для трассировки на уровне гостевой системы, из-за перемещения событий на оборудование. Это стало очевидным с развитием экземпляров AWS EC2 и перечисленных ниже типов целевых гипервизоров, которые можно трассировать:

- **PV:** гипервызовы (мультивызовы), обратные вызовы гипервизора, вызовы драйверов, недополученное время;
- **PVHVM:** обратные вызовы гипервизора, вызовы драйверов, недополученное время;
- **PVHVM + драйверы SR-IOV:** обратные вызовы гипервизора, недополученное время;
- **KVM (Nitro):** недополученное время.

Самый последний гипервизор, Nitro, имеет небольшой компонент, выполняющийся в гостевой системе, что нетипично для гипервизоров. Это сделано для увеличения производительности за счет переноса функций гипервизора на оборудование.

16.1.2. Возможные стратегии

Для начала определите, какая конфигурация гипервизора оборудования используется. Есть ли гипервызовы или специальные драйверы устройств?

На уровне гостевой системы:

1. Инструментируйте гипервызовы (если они есть) для проверки выполнения избыточных операций.
2. Проверьте недополученное процессорное время.
3. Используйте инструменты из предыдущих глав для анализа потребления ресурсов, но не забывайте, что это виртуальные ресурсы. Их производительность может ограничиваться гипервизором или внешним оборудованием и страдать от конфликтов с другими гостевыми системами.

На уровне хоста:

1. Инструментируйте выходы из виртуальной машины для проверки выполнения избыточных операций.
2. Если используется прокси-сервер ввода/вывода (QEMU), измерьте его рабочую нагрузку и задержку.
3. Примените инструменты из предыдущих глав для анализа потребления ресурсов.

По мере того как гипервизоры будут переносить функциональность на аппаратное обеспечение, как в случае с Nitro, будет возникать необходимость проводить анализ, используя инструменты из предыдущих глав, а не специализированные инструменты для гипервизоров.

16.2. ТРАДИЦИОННЫЕ ИНСТРУМЕНТЫ

Инструментов для анализа производительности гипервизора и устранения неполадок не так много. Иногда в гостевых системах можно использовать точки трассировки для гипервызовов, как показано в разделе 16.3.1.

На уровне хоста гипервизор Xen предлагает свои инструменты, включая `xl top` и `xentrace`, для исследования потребления гостевых ресурсов. Для KVM утилита `perf(1)` в Linux предоставляет подкоманду `kvm`. Пример вывода:

```
# perf kvm stat live
11:12:07.687968
```

Analyze events for all VMs, all VCPUs:

	VM-EXIT	Samples	Samples%	Time%	Min Time	Max Time	Avg time
	MSR_WRITE	1668	68.90%	0.28%	0.67us	31.74us	3.25us (+- 2.20%)
	HLT	466	19.25%	99.63%	2.61us	100512.98us	4160.68us (+- 14.77%)
	PREEMPTION_TIMER	112	4.63%	0.03%	2.53us	10.42us	4.71us (+- 2.68%)
	PENDING_INTERRUPT	82	3.39%	0.01%	0.92us	18.95us	3.44us (+- 6.23%)
	EXTERNAL_INTERRUPT	53	2.19%	0.01%	0.82us	7.46us	3.22us (+- 6.57%)
	IO_INSTRUCTION	37	1.53%	0.04%	5.36us	84.88us	19.97us (+- 11.87%)
	MSR_READ	2	0.08%	0.00%	3.33us	4.80us	4.07us (+- 18.05%)
	EPT_MISCONFIG	1	0.04%	0.00%	19.94us	19.94us	19.94us (+- 0.00%)

Total Samples:2421, Total events handled time:1946040.48us.

Этот вывод показывает причины выхода виртуальной машины в гипервизор и статистику по каждой причине. Наиболее продолжительные выходы в этом примере были обусловлены операцией HLT (остановка), выполняя которую виртуальные процессоры переходят в состояние ожидания.

Для событий KVM тоже есть точки трассировки, включая выходы в гипервизор, которые можно использовать для создания более подробных инструментов BPF.

16.3. ИНСТРУМЕНТЫ BPF ДЛЯ АНАЛИЗА НА УРОВНЕ ГОСТЕВОЙ ОС

В этом разделе рассмотрены инструменты BPF, которые можно использовать для анализа производительности гостевых систем. Все эти инструменты либо находятся в репозиториях BCC и `bpftool`, описанных в главах 4 и 5, либо были написаны специально для этой книги.

16.3.1. Гипервызовы Xen

Если гостевая система использует паравиртуализацию и выполняет гипервызовы, их можно трассировать с помощью `funccount(8)`, `trace(8)`, `argdist(8)` и `stackcount(8)`. Более того, для Xen есть даже точки трассировки. Для измерения задержки гипервызова понадобятся специальные инструменты.

Xen PV

Например, следующая система загружается в режиме паравиртуализации:

```
# dmesg | grep Hypervisor
[ 0.000000] Hypervisor detected: Xen PV
```

Получить частоту срабатывания точек трассировки Xen можно с помощью `funccount(8)` из BCC:

```
# funccount 't:xen:*'
Tracing 30 functions for "t:xen:*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
xen:xen_mmu_flush_tlb_one_user      70
xen:xen_mmu_set_pte                 84
xen:xen_mmu_set_pte_at              95
xen:xen_mc_callback                  97
xen:xen_mc_extend_args              194
xen:xen_mmu_write_cr3               194
xen:xen_mc_entry_alloc              904
xen:xen_mc_entry                    924
xen:xen_mc_flush                    1175
xen:xen_mc_issue                    1378
xen:xen_mc_batch                    1392
Detaching...
```

Точки трассировки `xen_mc` предназначены для анализа мультивызовов: пакетных гипервызовов. Каждый гипервызов начинается с вызова `xen:xen_mc_batch` и `xen:xen_mc_entry` и заканчивается вызовом `xen:xen_mc_issue`. Фактический гипервызов происходит только тогда, когда выполняется сброс (`flush`), которому соответствует точка трассировки `xen:xen_mc_flush`. Для оптимизации производительности есть два «отложенных» режима паравиртуализации, в которых вызовы гипервизора (`issue`) будут игнорироваться, накапливаться в буфере и выполняться позже: один для обновлений MMU и один для переключения контекста.

Различные пути в коде ядра заключены в операторные скобки `xen_mc_batch` и `xen_mc_issue`, чтобы сгруппировать возможные `xen_mc_calls`. Но если `xen_mc_calls` не выполняется, вызов (`issue`) и сброс (`flush`) производятся немедленно.

Пример использования одной из этих точек трассировки — инструмент `xenhyper(8)`, представленный далее. При таком количестве доступных точек трассировки можно было бы написать больше подобных инструментов, но, к сожалению, гостевые

системы Xen PV используются все реже, уступая место HVM (PVHVM). Для примера я включил в книгу только один инструмент и однострочные сценарии, показанные далее.

Xen PV: подсчет гипервызовов

Количество выполненных гипервызовов можно подсчитать с помощью точки трассировки `xen:xen_mc_flush` и ее аргумента `mcidx`, который сообщает, сколько гипервызовов было сделано. Вот пример с использованием `argdist(8)` из BCC:

```
# argdist -C 't:xen:xen_mc_flush():int:args->mcidx'
[17:41:34]
t:xen:xen_mc_flush():int:args->mcidx
      COUNT      EVENT
      44         args->mcidx = 0
      136        args->mcidx = 1
[17:41:35]
t:xen:xen_mc_flush():int:args->mcidx
      COUNT      EVENT
      37         args->mcidx = 0
      133        args->mcidx = 1
[...]
```

Здесь счетчик показывает, сколько гипервызовов было отправлено в каждой операции сброса (`flush`). Если счетчик равен нулю, то гипервызов не производился. Судя по результатам в этом примере, при трассировке каждую секунду производилось около 130 гипервызовов и не было зафиксировано ни одного случая пакетной обработки, то есть все гипервызовы выполнялись пакетами по одному.

Xen PV: стеки гипервызовов

Любую точку трассировки Xen можно инструментировать с помощью `stackcount(8)`, чтобы выяснить, каким путем код пришел к ней. Вот пример трассировки мультивызова:

```
# stackcount 't:xen:xen_mc_issue'
Tracing 1 functions for "t:xen:xen_mc_issue"... Hit Ctrl-C to end.
^C
[...]

xen_load_sp0
__switch_to
__schedule
schedule
schedule_preempt_disabled
cpu_startup_entry
cpu_bringup_and_idle
6629

xen_load_tls
16448
```

```

xen_flush_tlb_single
flush_tlb_page
ptep_clear_flush
wp_page_copy
do_wp_page
handle_mm_fault
__do_page_fault
do_page_fault
page_fault
    46604

xen_set_pte_at
copy_page_range
copy_process.part.33
_do_fork
sys_clone
do_syscall_64
return_from_SYSCALL_64
    565901

```

Detaching...

Избыточные мультивызовы (гипервызовы) могут быть причиной проблем с производительностью, и таким способом можно выяснить их причину. Оверхед на трассировку гипервызовов зависит от их частоты. На нагруженных системах она может быть очень высокой и повлечь сильный оверхед.

Хен PV: задержка гипервызова

Фактический гипервызов происходит только при выполнении операции сброса (flush), и к сожалению, нет точек трассировки, помогающих узнать, когда она начинается и заканчивается. Но вы можете переключиться на kprobes и трассировать функцию ядра `xen_mc_flush()`, время выполнения которой включает время выполнения фактического гипервызова. Вот как можно сделать это с помощью `funclatency(8)` из BCC:

```
# funclatency xen_mc_flush
```

```
Tracing 1 functions for "xen_mc_flush"... Hit Ctrl-C to end.
```

```
^C
```

nsecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 0	
32 -> 63	: 0	
64 -> 127	: 0	
128 -> 255	: 0	
256 -> 511	: 32508	*****
512 -> 1023	: 80586	*****
1024 -> 2047	: 21022	*****
2048 -> 4095	: 3519	*
4096 -> 8191	: 12825	*****

8192 -> 16383	:	7141	***	
16384 -> 32767	:	158		
32768 -> 65535	:	51		
65536 -> 131071	:	845		
131072 -> 262143	:	2		

Это может быть важным показателем производительности гипервизора с точки зрения гостевой системы. Можно даже написать инструмент ВСС, который запоминает, какие гипервызовы объединялись в пакеты, чтобы эту задержку разбить по типам операций.

Другой способ выявить проблемы с задержками гипервызовов — профилирование процессора, как описывалось в главе 6, и обзор затрат процессорного времени на гипервызовы либо в функции `hypercall_page()` (которая на самом деле является таблицей функций гипервызовов), либо в `xen_hyercall*()`. Пример показан на рис. 16.2.

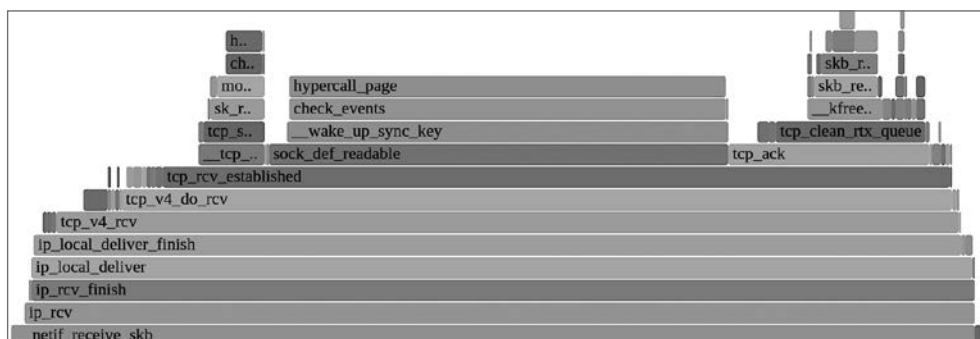


Рис. 16.2. Фрагмент флейм-графика потребления процессора, показывающий затраты на гипервызовы Xen PV

Здесь показан путь кода, соответствующий приему пакетов TCP, который заканчивается в `hypercall_page()`. Обратите внимание, что такой подход к профилированию может запутывать из-за невозможности отобразить некоторые пути кода в гостевой системе, приводящие к гипервызову. Это связано с тем, что гостевые системы с поддержкой PV обычно не поддерживают профилирование на основе счетчиков производительности РМС и вместо них будет использоваться программное профилирование по умолчанию, которое не может трассировать все пути в коде с отключенными прерываниями, приводящими к гипервызовам. Эта проблема описана в разделе 6.2.4.

Xen HVM

Для HVM точки трассировки хен обычно не срабатывают:

```
# dmesg | grep Hypervisor
[ 0.000000] Hypervisor detected: Xen HVM
# funccount 't:xen:xen*'
```



```
Tracing 27 functions for "t:xen:xen*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
Detaching...
```

Это объясняется тем, что вместо гипервызова виртуальные машины выполняют обычные вызовы, которые перехватываются и обрабатываются гипервизором HVM. Это усложняет проверку производительности гипервизора: для ее анализа придется использовать обычные инструменты, ориентированные на ресурсы и описанные в предыдущих главах, учитывая тот факт, что доступ к этим ресурсам осуществляется через гипервизор и наблюдаемые задержки связаны с задержкой в работе гипервизора.

16.3.2. xenhyper

`xenhyper(8)`¹ — это инструмент для `bpfttrace`, подсчитывающий гипервызовы с использованием точки трассировки `xen:xen_mc_entry`. Он выводит количество обращений к каждому гипервызову. Этот инструмент пригодится только для анализа гостевых систем Xen, загружающихся в режиме паравиртуализации и использующих гипервызовы. Пример вывода:

```
# xenhyper.bt
Attaching 1 probe...
^C

@[mmu_update]: 44
@[update_va_mapping]: 78
@[mmuext_op]: 6473
@[stack_switch]: 23445
```

Исходный код `xenhyper(8)`:

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Counting Xen hypercalls (xen_mc_entry). Ctrl-C to end.\n");

    // следующие определения необходимо обновить в соответствии
    // с версией вашего ядра: xen-hypercalls.h
    @name[0] = "set_trap_table";
    @name[1] = "mmu_update";
    @name[2] = "set_gdt";
    @name[3] = "stack_switch";
    @name[4] = "set_callbacks";
    @name[5] = "fpu_taskswitch";
    @name[6] = "sched_op_compat";
    @name[7] = "dom0_op";
    @name[8] = "set_debugreg";
```

¹ Немного истории: я написал его для этой книги 22 февраля 2019 года.

```

@name[9] = "get_debugreg";
@name[10] = "update_descriptor";
@name[11] = "memory_op";
@name[12] = "multicall";
@name[13] = "update_va_mapping";
@name[14] = "set_timer_op";
@name[15] = "event_channel_op_compat";
@name[16] = "xen_version";
@name[17] = "console_io";
@name[18] = "physdev_op_compat";
@name[19] = "grant_table_op";
@name[20] = "vm_assist";
@name[21] = "update_va_mapping_otherdomain";
@name[22] = "iret";
@name[23] = "vcpu_op";
@name[24] = "set_segment_base";
@name[25] = "mmuext_op";
@name[26] = "acm_op";
@name[27] = "nmi_op";
@name[28] = "sched_op";
@name[29] = "callback_op";
@name[30] = "xenoprof_op";
@name[31] = "event_channel_op";
@name[32] = "physdev_op";
@name[33] = "hvm_op";
}

tracepoint:xen:xen_mc_entry
{
    @[@name[args->op]] = count();
}

END
{
    clear(@name);
}

```

Для перевода номера операции гипервызова в соответствующее имя здесь используется таблица преобразования, взятая из исходного кода ядра. Ее нужно обновить согласно версии вашего ядра, поскольку эти соответствия меняются со временем.

xenhyper(8) можно расширить и добавить вывод имени процесса или трассировку стека в пространстве пользователя, меняя ключи карты @.

16.3.3. Обратные вызовы Xen

Обратные вызовы происходят не тогда, когда гостевая система обращается к гипервизору, а когда гипервизор Xen вызывает гостевую систему, например, для уведомления о прерываниях. В /proc/interrupts есть счетчики для каждого процессора:

```

# grep HYP /proc/interrupts
HYP: 12156816 9976239 10156992 9041115 7936087 9903434 9713902
8778612 Hypervisor callback interrupts

```

Каждое число — это счетчик для одного процессора (здесь у системы восемь процессоров). Обратные вызовы также можно трассировать с помощью BPF, используя зонд `kprobe` для функции ядра `xen_evtchn_do_upcall()`. Вот пример подсчета прерывавшихся процессов с помощью `bpfftrace`:

```
# bpfftrace -e 'kprobe:xen_evtchn_do_upcall { @[comm] = count(); }'
Attaching 1 probe...
^C

@[ps]: 9
@[bash]: 15
@[java]: 71
@[swapper/7]: 100
@[swapper/3]: 110
@[swapper/2]: 130
@[swapper/4]: 131
@[swapper/0]: 164
@[swapper/1]: 192
@[swapper/6]: 207
@[swapper/5]: 248
```

Как показывает вывод, чаще всего обратными вызовами Xen прерывались потоки, обслуживающие периоды бездействия системы («swapper/*»).

Задержки обратных вызовов можно измерить, например, с помощью `funclatency(8)` из BCC:

```
# funclatency xen_evtchn_do_upcall
Tracing 1 functions for "xen_evtchn_do_upcall"... Hit Ctrl-C to end.
^C
```

nsecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 0	
32 -> 63	: 0	
64 -> 127	: 0	
128 -> 255	: 0	
256 -> 511	: 1	
512 -> 1023	: 6	
1024 -> 2047	: 131	*****
2048 -> 4095	: 351	*****
4096 -> 8191	: 365	*****
8192 -> 16383	: 602	*****
16384 -> 32767	: 89	*****
32768 -> 65535	: 13	
65536 -> 131071	: 1	

Как показывает этот вывод, в большинстве случаев обработка обратного вызова занимала от 1 до 32 микросекунд.

Более подробную информацию о типе прерывания можно получить трассировкой дочерних функций `xen_evtchn_do_upcall()`.

16.3.4. cpustolen

crustolen(8)¹ — это инструмент для bpftrace, показывающий распределение процессорного времени, недополученного виртуальной машиной. Это процессорное время, не доставшееся гостевой системе, поскольку оно было использовано другими гостевыми системами (в некоторых конфигурациях гипервизора сюда может входить время, потребляемое прокси-сервером ввода/вывода в другом домене, выполняющем операции от имени самой гостевой системы, поэтому термин «недополученное время» не совсем верен²). Пример вывода:

```
# cpustolen.bt
```

Attaching 4 probes...

Tracing stolen CPU time. Ctrl-C to end.

 $\wedge C$

@stolen us:

[illegible]

Как показывает этот вывод, большая часть процессорного времени не была недополучена (интервал «[0]»), хотя в четырех случаях недополученное время составило от 8 до 16 микросекунд. Интервал «[0]» включен в результаты, чтобы позволить вычислить отношение недополученного времени к общему времени: в данном случае отношение составило 0.1% (32/30416).

crustolen(8) трассирует операцию паравиртуализации `stolen_clock`, используя версии `kprobes` для Xen и KVM: `xen_stolen_clock()` и `kvm_stolen_clock()`. Эти функции вызываются для многих частых событий, например переключения контекста и прерывания, поэтому оверхед инструмента может быть заметным в зависимости от рабочей нагрузки.

Исходный код `crustolen(8)`:

```
#!/usr/local/bin/bpftrace
```

BEGIN

```
{
    printf("Tracing stolen CPU time. Ctrl-C to end.\n");
}
```

```
kretprobe:xen_steal_clock,
kretprobe:kvm_steal_clock
{
    if (@last[cpu] > 0) {
```

¹ Немного истории: я написал его для этой книги 22 февраля 2019 года.

² Из-за особенностей измерения в недополученное время также может входить время, потраченное механизмом управления виртуальной памятью от имени гостевой системы [Yamamoto 16].

```

        @stolen_us = hist((retval - @last[cpu]) / 1000);
    }
    @last[cpu] = retval;
}

END
{
    clear(@last);
}

```

При работе с гипервизорами, отличными от Xen и KVM, код нужно изменить. Другие гипервизоры, вероятно, будут иметь аналогичную функцию `steal_clock`, как того требует таблица операций паравиртуализации (`pv_ops`). Обратите внимание, что есть функция более высокого уровня `paravirt_steal_clock()`, которая выглядит более подходящей для трассировки, так как не привязана к конкретному типу гипервизора. Но она недоступна для трассировки (скорее всего, она встраиваемая).

16.3.5. Трассировка выходов в HVM

С переходом от паравиртуализации к аппаратной виртуализации мы теряем возможность инструментации явных гипервызовов. При этом гостевые системы все еще делают выход в гипервизор для доступа к ресурсам, и мы хотели бы их трассировать. Текущий подход опирается на анализ задержек с использованием инструментов, описанных в предыдущих главах, с учетом того, что некоторая доля этих задержек может быть связана с гипервизором и мы не сможем измерить ее напрямую. Вероятно, определить эту долю можно, сравнив измеренные задержки на «голом железе».

Интересный исследовательский прототип, который поможет в трассировке выходов гостевых систем в гипервизор, — это новая технология «восходящий гипервызов» (`hyperupcalls`) [Amit 18]. Она позволяет гостевой системе безопасно обратиться к гипервизору для запуска мини-программы, например для трассировки гипервизора на уровне гостевой системы. Такие восходящие гипервызовы реализуются с использованием расширенной виртуальной машины BPF в гипервизоре, которой гостевая система передает свой скомпилированный байт-код BPF для выполнения. На момент написания книги никто из облачных провайдеров не предлагает такой возможности (вероятно, ее никогда и не будет), но это еще один интересный проект, где используется BPF.

16.4. ИНСТРУМЕНТЫ BPF ДЛЯ АНАЛИЗА НА УРОВНЕ ХОСТА

В этом разделе описаны инструменты BPF для анализа производительности виртуальных машин на уровне хоста. Эти инструменты либо находятся в репозиториях BCC и `bpfttrace`, описанных в главах 4 и 5, либо были написаны специально для этой книги.

16.4.1. kvmexits

kvmexits(8)¹ — это инструмент для bpftrace, показывающий распределение времени выхода гостевой системы в гипервизор по причинам. Он выявляет проблемы производительности, связанные с гипервизором, и задает направление для дальнейшего анализа. Пример вывода:

```
# kvmexits.bt
Attaching 4 probes...
Tracing KVM exits. Ctrl-C to end
^C
[...]
```

```
@exit_ns[30, IO_INSTRUCTION]:
[1K, 2K)           1 |
[2K, 4K)           12 | @@@
[4K, 8K)           71 | @@@@@@@@@@@@@@@@@@
[8K, 16K)          198 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16K, 32K)         129 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32K, 64K)          94 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64K, 128K)         37 | @@@@@@@@@@
[128K, 256K)        12 | @@@
[256K, 512K)        23 | @@@@@@
[512K, 1M)          2 |
[1M, 2M)            0 |
[2M, 4M)            1 |
[4M, 8M)            2 |
```

```
@exit_ns[1, EXTERNAL_INTERRUPT]:
[256, 512)          28 | @@@
[512, 1K)           460 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)             463 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)             150 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)             116 | @@@@@@@@@@@@@@@@@@
[8K, 16K)            31 | @@@
[16K, 32K)           12 | @
[32K, 64K)            7 |
[64K, 128K)          2 |
[128K, 256K)         1 |
```

```
@exit_ns[32, MSR_WRITE]:
[512, 1K)           5690 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)            2978 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)            2080 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)             854 | @@@@@@@@@@
[8K, 16K)            826 | @@@@@@@@@@
[16K, 32K)          110 | @
[32K, 64K)           3 |
```

¹ Немного истории: первую версию этого инструмента на основе Dtrace, с названием kvmexitlatency.d, я опубликовал в книге «Systems Performance» в 2013 году [Gregg 13b]. Версию на основе bpftrace для этой книги я написал 25 февраля 2019 года.

```

@exit_ns[12, HLT]:
[512, 1K)      13 |
[1K, 2K)      23 |
[2K, 4K)      10 |
[4K, 8K)      76 |
[8K, 16K)     234 |@@
[16K, 32K)    4167 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32K, 64K)    3920 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64K, 128K)   4467 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[128K, 256K)  3483 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[256K, 512K)  1764 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[512K, 1M)    922 |@@@@@@@@@@@@@@@@
[1M, 2M)     113 |@
[2M, 4M)     128 |@
[4M, 8M)      35 |
[8M, 16M)     40 |
[16M, 32M)    42 |
[32M, 64M)    97 |@
[64M, 128M)   95 |@
[128M, 256M)  58 |
[256M, 512M)  24 |
[512M, 1G)     1 |

@exit_ns[48, EPT_VIOLATION]:
[512, 1K)      6160 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)      6885 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)      7686 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)      2220 |@@@@@@@@@@@@@@@@@@@@
[8K, 16K)      582 |@@@
[16K, 32K)     244 |@
[32K, 64K)      47 |
[64K, 128K)      3 |

```

Этот вывод показывает распределение выходов в гипервизор по типам (причинам) и включает код выхода и строку с описанием причины, если она известна. Самые долгие выходы, до 1 секунды, обусловлены выполнением операции HLT (остановка), и это нормальное поведение: это поток, обслуживающий бездействие системы. Вывод также показал, что на выполнение операции IO_INSTRUCTIONS уходило до 8 миллисекунд.

kvmexits(8) использует точки трассировки kvm:kvm_exit и kvm:kvm_entry, которые доступны только при использовании модуля KVM ядра для повышения производительности.

Исходный код kvmexit(8):

```

#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Tracing KVM exits. Ctrl-C to end\n");

    // из arch/x86/include/uapi/asm/vmx.h:

```

```

@exitreason[0] = "EXCEPTION_NMI";
@exitreason[1] = "EXTERNAL_INTERRUPT";
@exitreason[2] = "TRIPLE_FAULT";
@exitreason[7] = "PENDING_INTERRUPT";
@exitreason[8] = "NMI_WINDOW";
@exitreason[9] = "TASK_SWITCH";
@exitreason[10] = "CPUID";
@exitreason[12] = "HLT";
@exitreason[13] = "INVD";
@exitreason[14] = "INVLPG";
@exitreason[15] = "RDPMC";
@exitreason[16] = "RDTSC";
@exitreason[18] = "VMCALL";
@exitreason[19] = "VMCLEAR";
@exitreason[20] = "VMLAUNCH";
@exitreason[21] = "VMPTRLD";
@exitreason[22] = "VMPTRST";
@exitreason[23] = "VMREAD";
@exitreason[24] = "VMRESUME";
@exitreason[25] = "VMWRITE";
@exitreason[26] = "VMOFF";
@exitreason[27] = "VMON";
@exitreason[28] = "CR_ACCESS";
@exitreason[29] = "DR_ACCESS";
@exitreason[30] = "IO_INSTRUCTION";
@exitreason[31] = "MSR_READ";
@exitreason[32] = "MSR_WRITE";
@exitreason[33] = "INVALID_STATE";
@exitreason[34] = "MSR_LOAD_FAIL";
@exitreason[36] = "MWAIT_INSTRUCTION";
@exitreason[37] = "MONITOR_TRAP_FLAG";
@exitreason[39] = "MONITOR_INSTRUCTION";
@exitreason[40] = "PAUSE_INSTRUCTION";
@exitreason[41] = "MCE_DURING_VMENTRY";
@exitreason[43] = "TPR_BELOW_THRESHOLD";
@exitreason[44] = "APIC_ACCESS";
@exitreason[45] = "EOI_INDUCED";
@exitreason[46] = "GDTR_IDTR";
@exitreason[47] = "LDTR_TR";
@exitreason[48] = "EPT_VIOLATION";
@exitreason[49] = "EPT_MISCONFIG";
@exitreason[50] = "INVEPT";
@exitreason[51] = "RDTSCP";
@exitreason[52] = "PREEMPTION_TIMER";
@exitreason[53] = "INVVPID";
@exitreason[54] = "WBINVD";
@exitreason[55] = "XSETBV";
@exitreason[56] = "APIC_WRITE";
@exitreason[57] = "RDRAND";
@exitreason[58] = "INVPCID";
}

```

```

tracepoint:kvm:kvm_exit
{

```



```

    @start[tid] = nsecs;
    @reason[tid] = args->exit_reason;
}

tracepoint:kvm:kvm_entry
/@start[tid]/
{
    $num = @reason[tid];
    @exit_ns[$num, @exitreason[$num]] = hist(nsecs - @start[tid]);
    delete(@start[tid]);
    delete(@reason[tid]);
}

END
{
    clear(@exitreason);
    clear(@start);
    clear(@reason);
}

```

В некоторых конфигурациях KVM не используется модуль ядра KVM, поэтому нужные точки трассировки не срабатывают и инструмент не может измерить продолжительность выхода гостевой системы в гипервизор. В таких случаях можно напрямую инструментировать процесс qemu с помощью uprobes и читать причины выхода. (Зонды USDT предпочтительнее.)

16.4.2. Возможные направления развития в будущем

В KVM и других аналогичных гипервизорах гостевые процессоры представлены обычными процессами, которые доступны для наблюдения многим инструментам, включая `top(1)`. Это заставляет меня задуматься, можно ли ответить на такие вопросы:

- Какие процессы выполняются на гостевом процессоре? Можно ли узнать, какие функции вызываются, и можно ли читать трассировку стека?
- Какие операции ввода/вывода выполняет гостевая система?

Хосты могут выбирать указатель инструкций процессора, а также читать его при выполнении ввода/вывода в момент выхода в гипервизор. Вот пример использования `bpftrace` для чтения указателя инструкций в операциях ввода/вывода:

```

# bpftrace -e 't:kvm:kvm_exit /args->exit_reason == 30/ {
    printf("guest exit instruction pointer: %llx\n", args->guest_rip); }'
Attaching 1 probe...
guest exit instruction pointer: ffffffff81c9edc9
guest exit instruction pointer: ffffffff81c9ee8b
guest exit instruction pointer: ffffffff81c9edc9
guest exit instruction pointer: ffffffff81c9edc9
guest exit instruction pointer: ffffffff81c9ee8b
guest exit instruction pointer: ffffffff81c9ee8b
[...]
```

Но на хосте нет ни таблицы символов для преобразования указателя инструкций в имена функций, ни контекста процесса, чтобы узнать, какое адресное пространство использовать или хотя бы какой процесс выполняется. Возможные решения обсуждались годами, в том числе в моей последней книге [Gregg 13b]. В их числе: чтение регистра CR3, хранящего корень текущей таблицы страниц, чтобы попытаться выяснить, какой процесс выполняется, и использование таблиц символов, предоставляемых гостевыми системами.

На момент написания книги на эти вопросы можно ответить только на уровне гостевой системы, но не хоста.

16.5. ИТОГИ

В этой главе я представил гипервизоры для виртуализации оборудования и показал, как использование средств BPF помогает выяснять детали на уровне гостевой системы и хоста, включая гипервызовы, недополученное процессорное время и выходы в гипервизор.

Глава 17

ДРУГИЕ ИНСТРУМЕНТЫ BPF ДЛЯ АНАЛИЗА ПРОИЗВОДИТЕЛЬНОСТИ

В этой главе я расскажу о других инструментах анализа производительности, основанных на BPF. Это программы с открытым исходным кодом, и они доступны бесплатно в интернете. (Спасибо моему коллеге Джейсону Коху из команды обеспечения эффективности в Netflix за создание большей части этой главы.)

В этой книге описаны десятки инструментов командной строки для BPF, но как ожидается, большинство людей в итоге будут использовать BPF для трассировки через графический интерфейс. Это особенно актуально для облачных сред, насчитывающих тысячи или даже сотни тысяч экземпляров. Управление такими средами обычно выполняется через графический интерфейс. Изучение инструментов BPF, описанных в предыдущих главах, поможет понять, как работают графические интерфейсы на основе BPF, которые используют все те же инструменты, и применять их на практике.

В этой главе поговорим о таких графических интерфейсах и инструментах:

- **Vector и Performance Co-Pilot (PCP):** для удаленного мониторинга BPF;
- **Grafana с PCP:** для удаленного мониторинга BPF;
- **eBPF Exporter:** для интеграции BPF с Prometheus и Grafana;
- **kubecttl-trace:** для трассировки модулей и узлов Kubernetes.

Цель главы — показать на примерах некоторые возможности графических интерфейсов и средств автоматизации на основе BPF. Каждому инструменту посвящен свой раздел, в котором кратко описаны функции инструмента, внутреннее устройство и порядок использования, а также приведены ссылки для дополнительного чтения. Обратите внимание, что на момент написания главы эти инструменты продолжали активно развиваться, и, вероятно, когда вы будете читать эти строки, инструменты получат новые, более широкие возможности.

17.1. VECTOR И PERFORMANCE CO-PILOT (PCP)

Netflix Vector — это инструмент с открытым исходным кодом для мониторинга производительности на уровне хоста, который визуализирует метрики системы и приложений с высоким разрешением практически в реальном времени. Он реализован в виде веб-приложения, использует проверенный опенсорс-фреймворк для мониторинга Performance Co-Pilot (PCP) и предлагает гибкий и удобный пользовательский интерфейс. Пользовательский интерфейс собирает метрики раз в секунду или чуть реже и отображает данные в полностью настраиваемых дашбордах, которые упрощают сопоставление и анализ этих метрик.

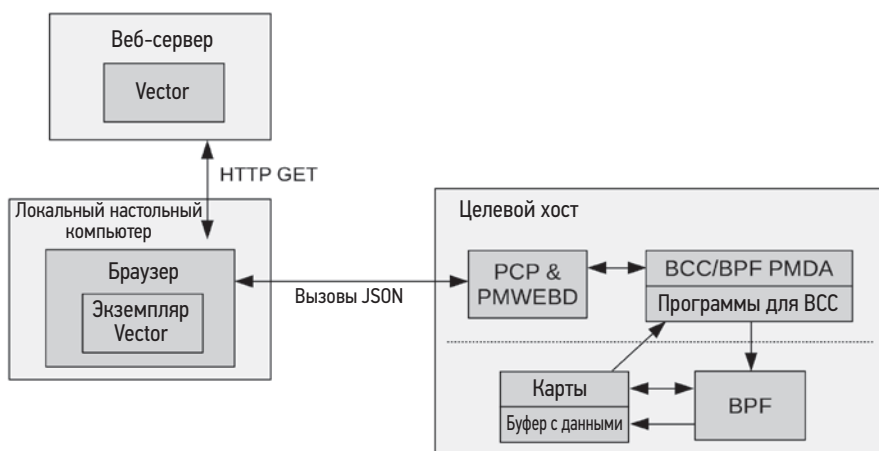


Рис. 17.1. Vector получает вывод удаленных программ BCC с помощью PCP

На рис. 17.1 показано, как Vector, загружаемый с веб-сервера в локальный браузер, подключается непосредственно к целевому хосту и использует PCP для запуска программ BPF. Обратите внимание, что внутренние компоненты PCP могут измениться в будущих версиях.

Вот некоторые возможности Vector:

- Высокоуровневые дашборды для отображения потребления ресурсов (процессор, диск, сеть, память) на работающем экземпляре.
- Более 2000 метрик для глубокого анализа. Можно добавлять или удалять метрики, изменяя конфигурацию агентов домена метрик производительности (Performance Metrics Domain Agent, PMDA).
- Поддержка визуализации данных во времени с точностью до 1 секунды.
- Сравнение значений разных метрик и на разных хостах, включая метрики, полученные из контейнеров и хоста. Например, есть возможность сравнить потребление ресурсов на уровне контейнера и хоста, чтобы увидеть, как они соотносятся.

В дополнение к другим источникам информации Vector поддерживает также метрики, получаемые с помощью BPF. Это стало возможным благодаря добавлению агента PCP для доступа к BPF через интерфейс BCC, который рассматривается в главе 4.

17.1.1. Визуализация

Vector умеет отображать данные в нескольких разных форматах. Данные временных рядов можно визуализировать с помощью линейных диаграмм, как показано на рис. 17.2.

Vector поддерживает и другие типы графиков, которые лучше подходят для визуализации данных, создаваемых посекундными гистограммами BPF и журналами событий: тепловые карты и таблицы с данными.

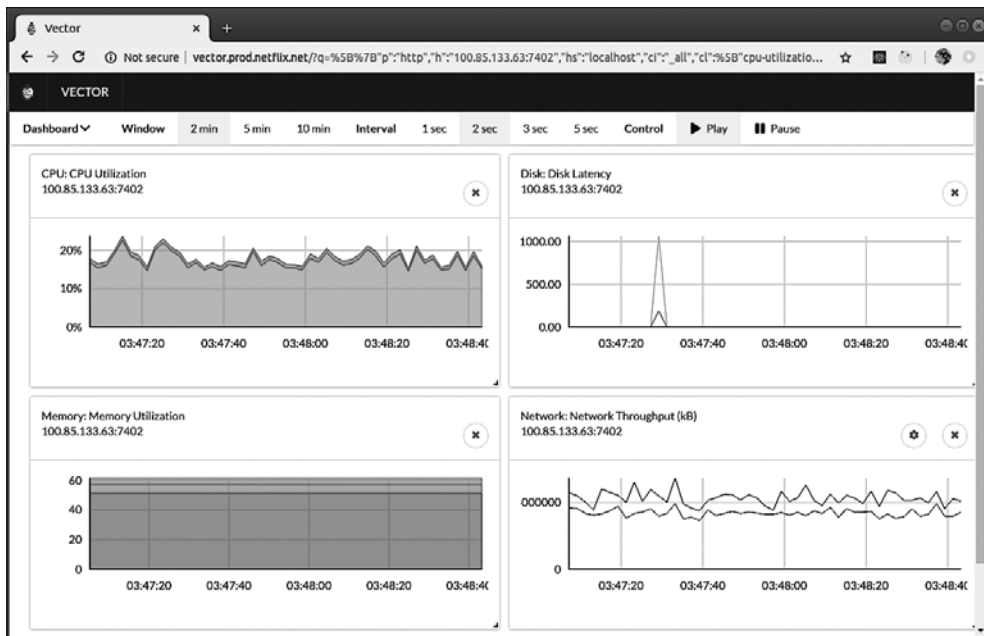


Рис. 17.2. Примеры отображения в Vector линейных диаграмм с системными метриками

17.1.2. Визуализация: тепловые карты

Тепловые карты можно использовать для отображения гистограмм с течением времени, а также для визуализации посекундных сводок с информацией о задержках. Тепловая карта задержек отмеряет время по обеим осям и состоит из сегментов со счетчиками, соответствующими определенным моментам времени и диапазонам задержки [Gregg 10].

Оси на графике:

- **ось X:** отражает течение времени. Каждый столбец на этой оси соответствует 1 секунде (или одному интервалу измерений);
- **ось Y:** задержка;
- **ось Z (насыщенность цвета):** показывает количество операций ввода/вывода, попавших в этот диапазон времени и задержек.

Для визуализации времени и задержек можно использовать диаграмму рассеяния. Но при отображении на графике тысяч и миллионов операций ввода/вывода точки начинают сливаться и детали теряются. Тепловая карта решает эту проблему, масштабируя цветовой диапазон по мере необходимости.

В Vector тепловые карты доступны для соответствующих инструментов BCC. На момент написания книги в их число входили: `biolateness(8)` — для анализа задержек блочного ввода/вывода, `gunqlat(8)` — для задержки в очереди готовности к выполнению на процессоре, а также инструменты для мониторинга задержек в файловых системах — `ext4-`, `xfs-` и `zfsdist`. Настроив BCC PMDA (как объясняется в разделе 17.1.5) и запустив соответствующую диаграмму BCC в Vector, вы увидите визуальное представление результатов применения этих инструментов. На рис. 17.3 показана тепловая карта задержек блочного ввода/вывода, полученная выборкой данных с двухсекундным интервалом на хосте, выполняющем несколько простых заданий `fiio(1)`.

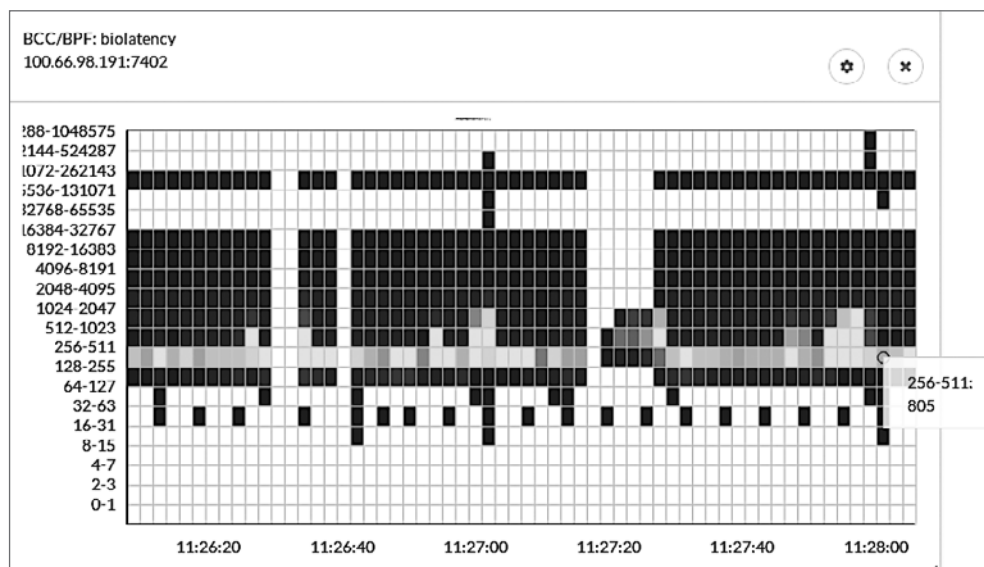


Рис. 17.3. Тепловая карта задержек в Vector, отражающая результаты трассировки с помощью BCC/BPF `biolateness(8)`

Наиболее распространенные задержки блочного ввода/вывода находятся в диапазоне от 256 до 511 микросекунд, а всплывающая подсказка у курсора показывает, что в этом интервале есть 805 выборов.

Для сравнения, ниже приводится результат выполнения команды `biolateness(8)` за такой же период времени:

```
# biolateness
Tracing block device I/O... Hit Ctrl-C to end.
^C
usecs      : count      distribution
 0 -> 1      : 0          |
 2 -> 3      : 0          |
 4 -> 7      : 0          |
 8 -> 15     : 0          |
16 -> 31     : 5          |
32 -> 63     : 19         |
64 -> 127    : 1          |
128 -> 255   : 2758       | *****
256 -> 511   : 12989      | *****
512 -> 1023  : 11425      | *****
1024 -> 2047 : 2406         | *****
2048 -> 4095 : 1034         | ***
4096 -> 8191 : 374          | *
8192 -> 16383: 189           |
16384 -> 32767: 343          | *
32768 -> 65535: 0            |
65536 -> 131071: 0           |
131072 -> 262143: 42          |
```

Все те же задержки в совокупности видны и здесь, но тепловая карта наглядно показывает, как они меняются со временем. Также на тепловой карте отчетливо видно, что задержки ввода/вывода в диапазоне от 128 до 256 миллисекунд равномерно распределены во времени, а не являются результатом кратковременной рабочей нагрузки.

Есть множество инструментов BPF, создающих такие гистограммы не только для задержки, но и для размеров буферов в байтах, длины очереди выполнения и других метрик. Все эти результаты можно визуализировать с помощью тепловых карт в Vector.

17.1.3. Визуализация: табличное представление данных

Помимо визуализации в виде графиков иногда удобно просматривать исходные данные в табличном виде. Это может быть особенно полезно при работе с некоторыми инструментами ВСС, потому что таблицы могут дать дополнительный контекст или помочь разобраться со смыслом списка значений.

Например, можно организовать мониторинг результатов `exesnoop(8)` и показать список недавно запущенных процессов. На рис. 17.4 показан процесс Tomcat (catalina) на контролируемом хосте. Таблица прекрасно подходит для визуализации деталей подобных событий.

BCC/BPF: ehexcsnoop
100.66.98.191:7402

COMM	PID	PPID	RET	ARGS
dirname	19709	19682	0	/usr/bin/diiname /apps/tomcat/bin/catalina.sh
catalina.sh	19682	4680	0	/apps/tomcat/bin/catalina.sh start
setuidgid	19682	4680	0	/usr/local/bin/setuidgid www-data/apps/tomcat/bin/catalina.sh start
ldconfig.real	19708	19707	0	/sbin/ldconfig.real -p

Рис. 17.4. Отображение в Vector событий, наблюдаемых с помощью ehexcsnoop(8) из BCC/BPF

Или можно наблюдать за сокетами TCP с помощью tcplife(8) и показывать адреса хостов и номера портов, количество переданных байтов и продолжительность сеанса, как показано на рис. 17.5. (Инструмент tcplife(8) был представлен в главе 10.)

BCC/BPF: tcplife
100.66.98.191:7402

PID	COMM	LADDR	LPORT	DADDR	DPORT	TX_KB	RX_KB	MS
3745	amazon-ssm-agent	100.66.98.191	29104	52.94.209.3	443	4	6	20037
17336	wget	100.66.98.191	30424	128.112.18.21	80	0	2050272	41595
3745	amazon-ssm-agent	100.66.98.191	25798	52.94.210.188	443	4	6	20025
3745	amazon-ssm-agent	100.66.98.191	45840	52.119.164.173	443	2	6	75016

Рис. 17.5. Отображение в Vector списка сеансов, наблюдаемых с помощью tcplife(8) из BCC/BPF

Здесь мы видим amazon-ssm-agent, который судя по всему выполняет долгий опрос в течение 20 секунд, а также команду wget(1), которая получила 2 Гбайта данных за 41.595 секунды.

17.1.4. Метрики BCC

Большинство инструментов из пакета bcc-tools сейчас доступно для PCP PMDA. Vector имеет предварительно настроенные диаграммы для следующих инструментов BCC:

- biolateny(8) и biotop(8);
- ext4dist(8), xfsdist(8) и zfsdist(8);
- tcplife(8), tcptop(8) и tcptrans(8);

- `runqlat(8)`;
- `execsnoop(8)`.

Многие из этих инструментов поддерживают параметры конфигурации, которые можно передать на хост. Настраиваемые диаграммы, таблицы и тепловые карты в Vector позволяют добавлять дополнительные инструменты ВСС и отображать их результаты.

Vector также позволяет добавлять пользовательские метрики событий для точек трассировки, зондов `uprobes` и событий `USDT`.

17.1.5. Внутреннее устройство

Как уже отмечалось, Vector — это веб-приложение, работающее в браузере пользователя. Оно построено на основе библиотеки React и использует D3.js для построения графиков. Метрики собираются и предоставляются фреймворком Performance Co-Pilot [175], предназначенным для сбора, архивирования и обработки метрик производительности из нескольких ОС. Типичная конфигурация РСР в Linux по умолчанию предлагает более 1000 метрик и может расширяться с помощью плагинов или агентов `PMDA`.

Чтобы понять, как Vector визуализирует метрики `BPF`, нужно знать, как РСР собирает их (рис. 17.6):

- `PMCD` (Performance Metrics Collector Daemon — демон сборщика метрик производительности) — центральный компонент РСР. Обычно выполняется на целевом хосте и координирует сбор метрик от множества агентов.
- `PMDA` (Performance Metrics Domain Agent — агент домена метрик производительности) — этим термином обозначаются агенты для РСР. Сейчас доступно множество агентов `PMDA`, поставляющих разные метрики. Например, есть агенты для сбора метрик, характеризующих работу ядра, файловых систем, графических процессоров `NVIDIA` и многих других. Чтобы получать в РСР метрики ВСС, нужно установить агент `PMDA` для ВСС.
- Vector — одностраничное веб-приложение, которое можно развернуть на сервере или выполнять локально. Оно позволяет подключаться к целевому экземпляру `pmwebd`.
- `pmwebd` (Performance Metrics Web Daemon — веб-демон метрик производительности) действует как шлюз REST для экземпляра `pmcd` на целевом хосте. Vector подключается к открытому порту REST и использует его для взаимодействия с `pmcd`.

Модель фреймворка РСР не предусматривает сохранения состояния, что делает его простым и надежным. Нагрузка, оказываемая на хосты, незначительна, потому что ответственность за хранение состояния, частоту сбора данных и вычисления несут клиенты. Кроме того, метрики не агрегируются по хостам и не сохраняются за пределами сеанса браузера пользователя, что обеспечивает фреймворку легковесность.

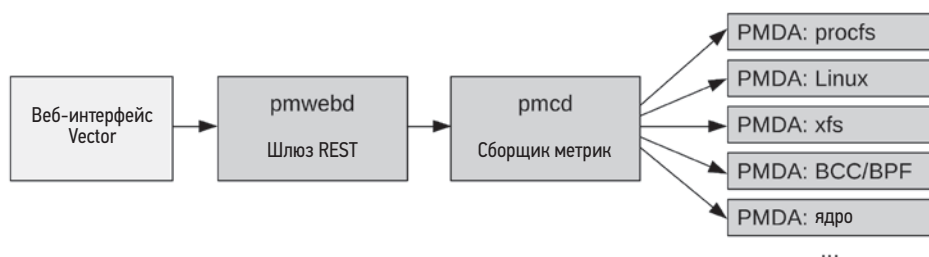


Рис. 17.6. Сбор метрик для визуализации в Vector

17.1.6. Установка PCP и Vector

PCP и Vector можно запустить на одном хосте и выполнить локальный мониторинг. В промышленных средах Vector и агенты PCP и PMDA обычно выполняются на разных хостах. Дополнительные подробности ищите в документации проекта.

Шаги по установке Vector задокументированы и обновляются онлайн [176], [177]. Сейчас они включают установку пакетов `pcp` и `pcp-webapi` и запуск пользовательского интерфейса Vector из контейнера Docker. Следуйте этим инструкциям, чтобы убедиться, что PMDA BCC включен:

```

$ cd /var/lib/pcp/pmdas/bcc/
$ ./Install
[Wed Apr 3 20:54:06] pmdabcc(18942) Info: Initializing, currently in 'notready'
state.
[Wed Apr 3 20:54:06] pmdabcc(18942) Info: Enabled modules:
[Wed Apr 3 20:54:06] pmdabcc(18942) Info: ['biolatency', 'sysfork', 'tcppeerpid',
'runqlat']

```

В системе с настроенным агентом BCC PMDA к нему можно подключиться с помощью Vector и PCP и просматривать системные метрики.

17.1.7. Подключение и просмотр данных

Перейдите в браузере по адресу `http://localhost/` (если тестирование производится на локальной машине) или по адресу, где установлен Vector. Введите имя хоста целевой системы в диалоге, показанном на рис. 17.7.

В области со списком соединений отобразится новое подключение. Как показано на рис. 17.8, вскоре значок (1) должен стать зеленым, и тогда станут доступными большие кнопки. В этом примере вместо готового дашборда будет использоваться специальная диаграмма, поэтому перейдите на вкладку (2) Custom (Специальные) и выберите `runqlat` (3). Все модули, недоступные на сервере, отображаются серым цветом и недоступны для выбора. Щелкните на названии выбранного модуля и затем на кнопке (4) Dashboard ^ (Дашборд ^) в панели инструментов, чтобы закрыть диалог.

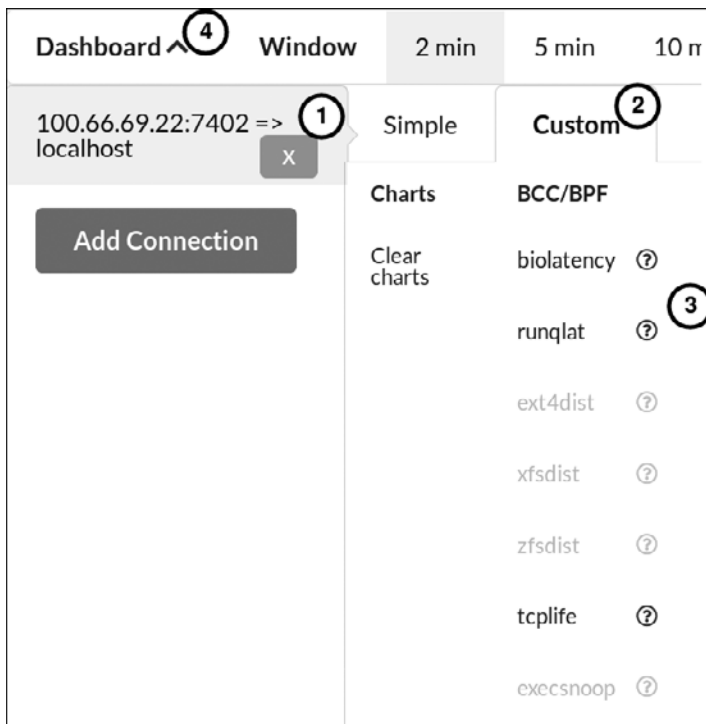
Add connection

Hostname

Port

Hostspec

Container

Рис. 17.7. Выбор целевой системы в Vector**Рис. 17.8.** Выбор инструмента BCC/BPF в Vector

В диалоге настройки подключения на вкладке Custom (Специальные), в разделе BCC/BPF, можно увидеть доступные для просмотра метрики BCC/BPF. В данном случае многие из программ BPF отображаются серым цветом, потому что не включены в агенте PMDA. После выбора `runqlat` и щелчка на кнопке Dashboard ^ (Дашборд ^) отобразится тепловая карта задержек в очереди на выполнение, которая обновляется каждую секунду, как показано на рис. 17.9. Источником информации для нее служит BCC-инструмент `runqlat(8)`.

Обязательно исследуйте виджет с настройками, чтобы узнать о других доступных метриках BCC.

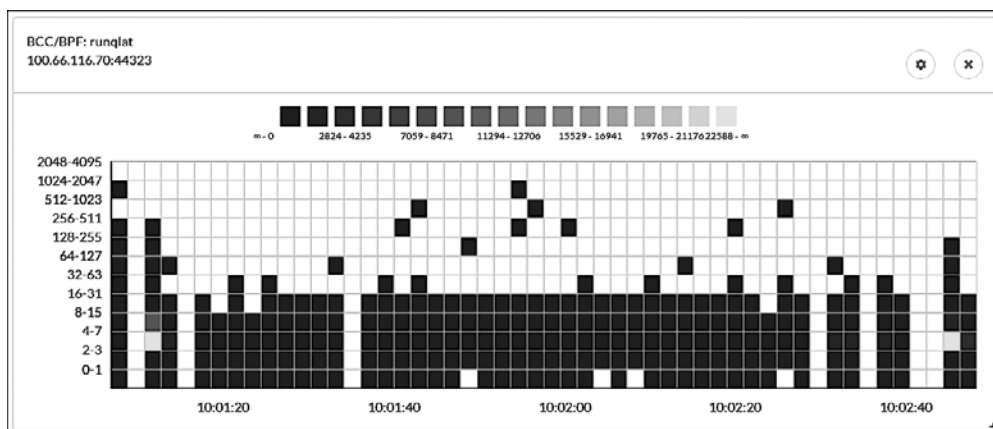


Рис. 17.9. Тепловая карта задержек в очереди на выполнение в Vector

17.1.8. Настройка BCC PMDA

Как отмечалось выше, большая часть возможностей BCC PMDA недоступна без специальной настройки. Подробное описание формата файла конфигурации ищите на странице справочного руководства для BCC PMDA (`pmdabcc(1)`). Ниже показано, как настроить BCC-модуль `tcpretrans`, чтобы с его помощью просматривать в Vector статистику сеансов TCP.

```
$ cd /var/lib/pcp/pmdas/bcc
$ sudo vi bcc.conf
[pmda]
# Список включенных модулей
modules = biolateness,sysfork,tcppeerpid,runqlat,tcp life
```

В этом файле вы увидите дополнительные параметры конфигурации для модуля `tcp life` и многих других. Этот файл очень важен в настройке BCC PMDA.

```
# Этот модуль обобщает информацию о сеансах TCP
#
# Параметры настройки:
```

```

# Имя          - тип      - значение по умолчанию
#
# process      - строка   - нет : список имен/идентификаторов или регулярное
#              -          -       выражение для выбора контролируемых процессов
# dport        - целое    - нет : список удаленных портов для мониторинга
# lport        - целое    - нет : список локальных портов для мониторинга
# session_count - целое    - 20  : число закрытых сеансов TCP для хранения в кэше
# buffer_page_count - целое - 64  : количество страниц, выделяемых для кольцевого
#              -          -       буфера, кратное степени двойки
[tcplife]
module = tcplife
cluster = 3
#process = java
#lport = 8443
#dport = 80,443

```

При любом изменении конфигурации нужно повторно скомпилировать и запустить PMDA:

```

$ cd /var/lib/pcp/pmdas/bcc
$ sudo ./Install
...

```

Теперь обновите страницу в браузере и выберите диаграмму `tcpretrans`.

17.1.9. Возможные направления развития в будущем

Для улучшения интеграции с полным набором ВСС-инструментов Vector и PCP все еще требуют дополнительной работы. Vector уже много лет применяется в Netflix и хорошо зарекомендовал себя как решение для мониторинга метрик хостов. В Netflix изучают возможность использовать для решения тех же задач систему Grafana, которая позволила бы сосредоточить внимание разработчиков на хостах и метриках. Подробнее о Grafana рассказывается в разделе 17.2.

17.1.10. Для дополнительного чтения

Дополнительную информацию о Vector и PCP ищите по ссылкам:

- <https://getvector.io/>;
- <https://pcp.io/>.

17.2. GRAFANA И PERFORMANCE CO-PILOT (PCP)

Grafana — популярный инструмент с открытым исходным кодом для построения диаграмм и визуализации, поддерживающий подключение и отображение данных

из множества источников. Используя в качестве источника данных Performance Co-Pilot (PCP), можно визуализировать любые метрики, доступные в PCP. Более подробно о PCP рассказывается в разделе 17.1.

Есть два подхода к настройке PCP для представления метрик в Grafana: в виде исторических и оперативных данных, меняющихся в реальном времени. Каждый из них имеет немного отличающиеся варианты использования и настройки.

17.2.1. Установка и настройка

Есть два варианта представления данных PCP в Grafana:

- **Оперативные данные, меняющиеся в реальном времени:** в этом случае используется плагин `grafana-pcp-live`. Он запрашивает у экземпляра PCP последние значения метрик и сохраняет в браузере краткую историю результатов (протяженностью в несколько минут). Данные хранятся очень недолго. Преимущество этого варианта — нет нагрузки на контролируемую систему, пока вы не решите посмотреть, как меняются значения ее метрик в реальном времени.
- **Архивные данные:** в этом случае используется плагин `grafana-pcp-redis`. Этот плагин извлекает данные из источника с помощью хранилища PCP `pmseries` и собирает их в экземпляре Redis. Частота получения данных и продолжительность их хранения определяются настройками экземпляра `pmseries`. Это делает такой вариант более подходящим для сбора больших временных рядов, которые будут просматриваться на нескольких хостах.

Далее предполагается, что вы выполнили шаги по настройке PCP, описанные в разделе 17.1.

Оба проекта продолжают активно развиваться, поэтому лучший подход к установке — заглянуть в инструкции по установке, ссылки на которые приводятся в разделе 17.2.4.

17.2.2. Подключение и просмотр данных

Плагин `grafana-pcp-live` продолжает активно разрабатываться. На момент написания книги настройка подключения основывалась на переменных для клиента PCP. Отсутствие отдельного файла конфигурации позволяет динамически перенастраивать подключение дашборда к хостам. К этим переменным относятся `_proto`, `_host` и `_port`.

Создайте новый дашборд, настройте его, создайте переменные и присвойте им желаемые значения. Результат можно видеть на рис. 17.10 (здесь в поле `$_host` следует ввести имя хоста для мониторинга):

После настройки подключения дашборда в него можно добавить новую диаграмму. Выберите метрику PCP; хорошим кандидатом для старта может быть метрика `bcc.runq.latency` (рис. 17.11).

Variable	Definition
<code>\$_proto</code>	http
<code>\$_host</code>	
<code>\$_port</code>	7402

Рис. 17.10. Настройка переменных дашборда в grafana-pcp-live

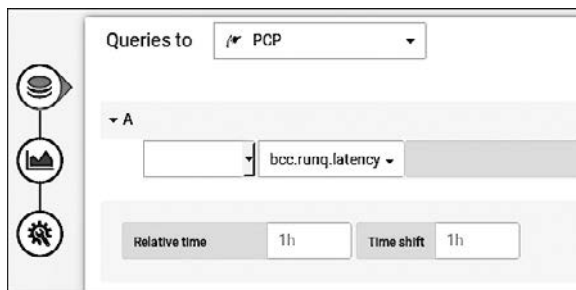


Рис. 17.11. Выбор метрики для запроса в Grafana

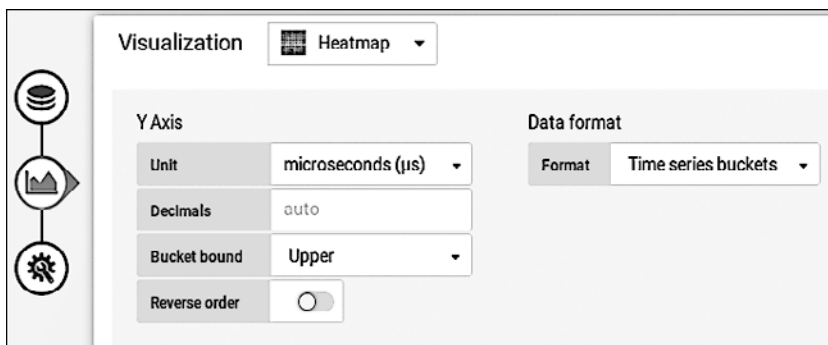


Рис. 17.12. Отображение в Grafana стандартных метрик PCP (переключения контекста, счетчик выполняющихся процессов), а также задержек в очереди на выполнение (`runqlat`)

Настройте визуализацию (рис. 17.12). В данном случае выберите визуализацию Heatmap (Тепловая карта) и установите формат Time series buckets (Интервалы временных рядов) с единицей измерения microseconds (μ s) (микросекунды (мкс)). В поле Bucket bound (Граница интервала) выберите Upper (Верхняя), как показано на рис. 17.13.

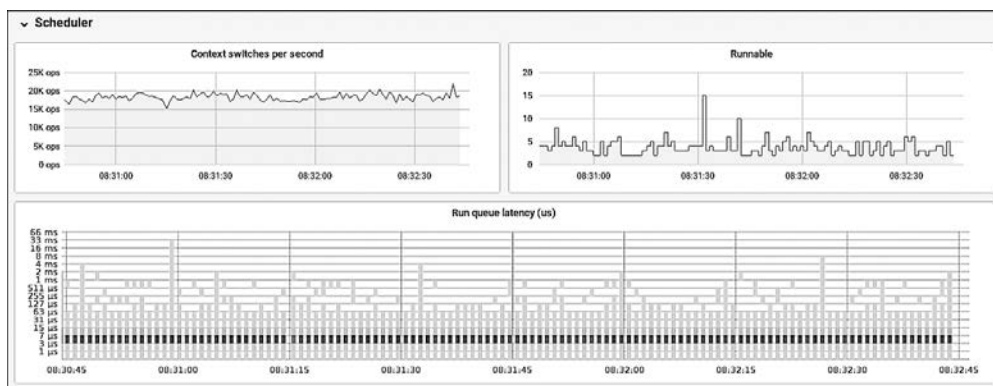


Рис. 17.13. Настройка визуализации в Grafana

17.2.3. Возможные направления развития в будущем

Для улучшения интеграции с полным набором ВСС-инструментов Grafana и PCP все еще требуют дополнительной работы. Надеюсь, что поддержка визуализации пользовательских программ bpftrace будет доступна в одном из будущих обновлений. Плагин grafana-pcp-live также требует значительной доработки, прежде чем его можно будет считать готовым для использования в продакшене.

17.2.4. Для дополнительного чтения

Эти ссылки могут измениться по мере развития проекта:

- источник данных grafana-pcp-live: <https://github.com/Netflix-Skunkworks/grafana-pcp-live/>
- источник данных grafana-pcp-redis: <https://github.com/performancecopilot/grafana-pcp-redis/>

17.3. ЭКСПОРТЕР CLOUDFLARE EBPF PROMETHEUS (С GRAFANA)

Экспортер Cloudflare eBPF — это инструмент с открытым исходным кодом, использующий четко определенный формат мониторинга Prometheus. Prometheus

пользуется особенной популярностью для сбора и хранения метрик, предлагая простой и широко известный протокол. Это упрощает интеграцию с любыми языками программирования, для которых доступно множество простых библиотек поддержки. Prometheus поддерживает возможность оповещения и хорошо интегрируется с динамическими средами, например Kubernetes. Несмотря на простоту пользовательского интерфейса Prometheus, некоторые инструменты визуализации, включая Grafana, основываются именно на нем для обеспечения согласованного взаимодействия с дашбордами.

Prometheus также интегрируется с инструментами управления приложениями. В Prometheus есть инструмент, производящий сбор и предоставление метрик, известный как *экспортер*. Для сбора статистики с хостов Linux есть множество официальных и сторонних экспортеров, например экспортеры JMX для приложений на Java, которые могут извлекать метрики веб-серверов, хранилищ, оборудования и баз данных. В Cloudflare есть опенсорс-экспортер для экспорта метрик BPF, который позволяет получать и визуализировать эти метрики через Prometheus и Grafana.

17.3.1. Сборка и запуск экспортера ebpf

Обратите внимание, что для сборки нужен Docker:

```
$ git clone https://github.com/cloudflare/ebpf_exporter.git
$ cd ebpf_exporter
$ make
...
$ sudo ./release/ebpf_exporter-*/ebpf_exporter --config.file=./examples/runqlat.yaml
2019/04/10 17:42:19 Starting with 1 programs found in the config
2019/04/10 17:42:19 Listening on :9435
```

17.3.2. Настройка Prometheus для мониторинга экземпляра ebpf_exporter

Настройка Prometheus зависит от используемого подхода к мониторингу целей. Если предположить, что на экземпляре запущен ebpf_exporter, прослушивающий порт 9435, то вы сможете найти пример целевой конфигурации, как показано ниже:

```
$ kubectl edit configmap -n monitoring prometheus-core
- job_name: 'kubernetes-nodes-ebpf-exporter'
  scheme: http
  kubernetes_sd_configs:
  - role: node
  relabel_configs:
  - source_labels: [__address__]
    regex: '(.):10250'
    replacement: '${1}:9435'
    target_label: __address__
```

17.3.3. Создание запроса в Grafana

Сразу после запуска `ebpf_exporter` начинает производить метрики. Их можно отобразить в виде графика, используя следующий запрос и дополнительные настройки форматирования (рис. 17.14):

```
query : rate(ebpf_exporter_run_queue_latency_seconds_bucket[20s])
legend format : {{le}}
axis unit : seconds
```

(Дополнительную информацию о запросах и настройках графиков ищите в документации Grafana и Prometheus.)

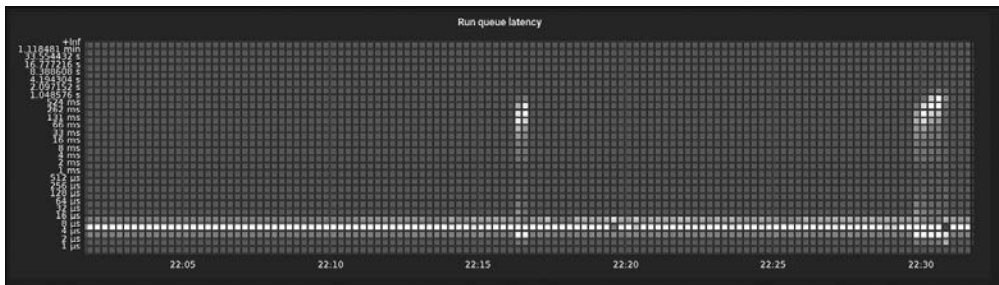


Рис. 17.14. Тепловая карта задержек в очереди на выполнение в Grafana, на которой видны всплески задержек, когда число потоков, запускаемых `schbench`, превышает число ядер процессора

17.3.4. Для дополнительного чтения

Дополнительную информацию о Grafana и Prometheus ищите по ссылкам:

- <https://grafana.com/>
- <https://github.com/prometheus/prometheus>

Дополнительную информацию об экспортере Cloudflare eBPF ищите по ссылкам:

- https://github.com/cloudflare/ebpf_exporter
- https://blog.cloudflare.com/introducing-ebpf_exporter/

17.4. KUBECTL-TRACE

`kubectl-trace` — это интерфейс командной строки Kubernetes для запуска программ `bpfftrace` на узлах в кластере Kubernetes. Его создал Лоренцо Фонтана в рамках проекта IO Visor (см. <https://github.com/iovisor/kubectl-trace>).

Чтобы опробовать примеры, приведенные ниже, нужно загрузить и установить `kubectl-trace`. Понадобится установить и Kubernetes (что выходит за рамки этой книги):

```
$ git clone https://github.com/iovisor/kubectl-trace.git
$ cd kubectl-trace
$ make
$ sudo cp ./_output/bin/kubectl-trace /usr/local/bin
```

17.4.1. Трассировка узлов

kubectl — это интерфейс командной строки Kubernetes. kubectl-trace поддерживает возможность выполнения команд bpftrace на узлах кластера. Трассировка целых узлов — самый простой из доступных вариантов, но имейте в виду, что оверхед, связанный с BPF, довольно высок: вызов bpftrace, потребляющий значительные ресурсы, повлияет на весь узел кластера.

Пример использования vfsstat.bt для захвата вывода bpftrace на узле Kubernetes в кластере:

```
$ kubectl trace run node/ip-1-2-3-4 -f /usr/share/bpftrace/tools/vfsstat.bt
trace 8fc22ddb-5c84-11e9-9ad2-02d0df09784a created
$ kubectl trace get
NAMESPACE NODE NAME STATUS AGE
default ip-1-2-34 kubectl-trace-8fc22ddb-5c84-11e9-9ad2-02d0df09784a Running 3s
$ kubectl trace logs -f kubectl-trace-8fc22ddb-5c84-11e9-9ad2-02d0df09784a
00:02:54
@[vfs_open]: 940
@[vfs_write]: 7015
@[vfs_read]: 7797

00:02:55
@[vfs_write]: 252
@[vfs_open]: 289
@[vfs_read]: 924

^C
$ kubectl trace delete kubectl-trace-8fc22ddb-5c84-11e9-9ad2-02d0df09784a
trace job kubectl-trace-8fc22ddb-5c84-11e9-9ad2-02d0df09784a deleted
trace configuration kubectl-trace-8fc22ddb-5c84-11e9-9ad2-02d0df09784a deleted
```

Этот вывод показывает всю статистику vfs для всего узла, а не только для пода. Поскольку bpftrace выполняется на уровне хоста, kubectl-trace запускается и в контексте хоста. Соответственно, трассируются все приложения, запущенные на этом узле. Иногда это может пригодиться системным администраторам, но во многих случаях важнее сосредоточиться на процессах внутри контейнера.

17.4.2. Трассировка подов и контейнеров

bpftrace — и, следовательно, kubectl-trace — косвенно поддерживает контейнеры, осуществляя трассировку через структуры данных ядра. kubectl-trace предлагает два вида помощи в трассировке подов. Во-первых, если указать имя пода, kubectl-trace автоматически найдет и развернет программу bpftrace на соответствующем узле. Во-вторых, kubectl-trace добавляет в сценарии дополнительную переменную

\$container_pid, в которой передается идентификатор (PID) корневого процесса контейнера с использованием пространства имен PID хоста. Это позволяет фильтровать и выполнять другие действия, нацеленные на нужный под.

В этом примере мы гарантируем, что PID будет единственным выполняющимся внутри рассматриваемого контейнера. Для более сложных сценариев, например когда запускается процесс инициализации или происходит ветвление сервера, нужно организовать сопоставление PID с родительскими PID.

Создайте новое развертывание, используя следующую спецификацию. Обратите внимание, что команда определяет точку входа Docker, чтобы гарантировать, что процесс узла — это единственный процесс внутри контейнера, а vfsstat-pod.bt включает дополнительный фильтр по PID:

```
$ cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-hello
spec:
  selector:
    matchLabels:
      app: node-hello
  replicas: 1
  template:
    metadata:
      labels:
        app: node-hello
    spec:
      containers:
      - name: node-hello
        image: duluca/minimal-node-web-server
        command: ['node', 'index']
        ports:
        - containerPort: 3000
EOF
deployment.apps/node-hello created
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
node-hello-56b8dbc757-th2k2        1/1     Running   0           4s
```

Создайте копию vfsstat.bt с именем vfsstat-pod.bt, как показано ниже, а затем запустите реализацию трассировщика (эти шаги показывают, как запустить трассировку и просмотреть вывод):

```
$ cat vfsstat-pod.bt
...
kprobe:vfs_read,
kprobe:vfs_write,
kprobe:vfs_fsync,
kprobe:vfs_open,
```

```

kprobe:vfs_create
/pid == $container_pid/
{
...
$ kubectl trace run pod/node-hello-56b8dbc757-th2k2 -f vfsstat-pod.bt
trace 552a2492-5c83-11e9-a598-02d0df09784a created
$ kubectl trace logs -f 552a2492-5c83-11e9-a598-02d0df09784a
if your program has maps to print, send a SIGINT using Ctrl-C, if you want to
interrupt the execution send SIGINT two times
Attaching 8 probes...
Tracing key VFS calls... Hit Ctrl-C to end.
[...]
```

17:58:34

```

@[vfs_open]: 1
@[vfs_read]: 3
@[vfs_write]: 4
```

17:58:35

```

17:58:36
@[vfs_read]: 3
@[vfs_write]: 4
[...]
```

Вы заметите, что операций `vfs` на уровне пода значительно меньше, чем на уровне узла, чего и следовало ожидать от по большей части простаивающего веб-сервера.

17.4.3. Для дополнительного чтения

- <https://github.com/iovisor/kubectl-trace>.

17.5. ДРУГИЕ ИНСТРУМЕНТЫ

Некоторые другие инструменты на основе BPF:

- **Cilium**: использует BPF для применения политик безопасности сети и приложений в контейнерных средах;
- **Sysdig**: использует BPF для расширения наблюдаемости контейнеров;
- **Android eBPF**: трассирует и управляет сетевыми устройствами в Android;
- **osquery eBPF**: предоставляет информацию об операционной системе для анализа и мониторинга; поддерживает мониторинг `kprobes` с помощью BPF;
- **ply**: трассировщик командной строки на основе BPF, похожий на `bpftool`, но с минимальными зависимостями. Из-за этого он хорошо подходит, например, для встраиваемых устройств [5]. `ply` был создан Тобиасом Вальдекранцем.

Поскольку использование BPF растет, то, скорее всего, на основе BPF будет создано гораздо больше инструментов с графическим интерфейсом.

17.6. ИТОГИ

Набор инструментов BPF быстро растет, и в будущем появится еще больше инструментов и возможностей. В этой главе были представлены четыре инструмента, основанные на BPF. Vector/PCP, Grafana и экспортер Cloudflare eBPF — это инструменты с графическим интерфейсом, позволяющие визуально представлять большие объемы сложных данных, включая временные ряды BPF. Последний инструмент, о котором рассказано в этой главе, `kubectl-trace`, позволяет запускать сценарии `bpfttrace` в кластере Kubernetes. В конце главы я привел краткий список других инструментов BPF.

Глава 18

СОВЕТЫ, РЕКОМЕНДАЦИИ И ТИПИЧНЫЕ ПРОБЛЕМЫ

В этой главе приведены советы и рекомендации по трассировке с использованием VRF, а также описаны типичные проблемы и способы их решения.

Советы и рекомендации:

- 18.1. Типичная частота событий и оверхед.
- 18.2. Выборка с частотой 49 или 99 Гц.
- 18.3. Желтые свиньи и серые крысы.
- 18.4. Пишите целевое ПО.
- 18.5. Изучайте системные вызовы.
- 18.6. Не усложняйте.

Типичные проблемы:

- 18.7. Отсутствие событий.
- 18.8. Отсутствие трассировок стека.
- 18.9. Отсутствие символов (имен функций) в выводе.
- 18.10. Отсутствие функций в трассировке.
- 18.11. Циклы обратной связи.
- 18.12. Сброс событий.

18.1. ТИПИЧНАЯ ЧАСТОТА СОБЫТИЙ И ОВЕРХЕД

Оверхед на трассировку программ определяется тремя основными факторами:

- частота трассируемого события;
- действия при трассировке;
- количество процессоров в системе.

Величина оверхеда на трассировку одного процессора определяется по формуле:

$$\text{Оверхед} = (\text{Частота} \times \text{Выполненное действие}) / \text{Количество процессоров}.$$

Трассировка событий, следующих с частотой один миллион в секунду, в системе с одним процессором может привести к сильному снижению производительности приложения, в то время как трассировка тех же событий в системе со 128 процессорами может практически не сказываться на производительности. Всегда учитывайте количество процессоров.

Количество процессоров и оверхед могут варьироваться в пределах порядка. При этом частота событий может варьироваться в пределах нескольких порядков, что делает ее самым важным членом уравнения оценки оверхеда.

18.1.1. Частота

Бывает полезно иметь примерное представление о типичной частоте событий, поэтому я составил таблицу (табл. 18.1)¹. В ней есть столбец, масштабирующий максимальную частоту в более привычные человеку величины, так «один раз в секунду» превращается в «один раз в год». Представьте, что вы подписаны на новостную рассылку и вам присылают письма с этой масштабированной частотой.

Таблица 18.1. Типичная частота событий

Событие	Типичная частота ¹	Максимальная частота	Максимальный предполагаемый оверхед на трассировку ²
Приостановка потоков	1 раз в секунду	Раз в год	Незначительный
Запуск процессов	10 раз в секунду	Раз в месяц	Незначительный
Открытие файлов	10–50 раз в секунду	Раз в неделю	Незначительный

¹ Идея этой таблицы заимствована из главы 2 книги «Systems Performance» [Gregg 13b]. Она очень популярна и неоднократно использовалась другими авторами. Но изначальная идея масштабирования частот не моя: я впервые познакомился с ней, когда был студентом университета.

² Трудно выбрать что-то «типичное», потому что рабочие нагрузки различаются. Базы данных часто имеют более высокую частоту операций дискового ввода/вывода, а веб-серверы и прокси-серверы часто имеют более высокую частоту передачи пакетов.

³ Это предполагаемый оверхед процессорного времени при трассировке события с максимальной частотой (см. ниже). Нельзя трассировать отдельные инструкции и такты процессора непосредственно, хотя теоретически это можно реализовать, если они эмулируются программными симуляторами процессора.

Событие	Типичная частота ¹	Максимальная частота	Максимальный предполагаемый оверхед на трассировку ²
Профилирование с частотой 100 Гц	100 раз в секунду	Два раза в неделю	Незначительный
Создание новых сеансов TCP	10–500 раз в секунду	Раз в день	Незначительный
Дисковый ввод/вывод	10–1000 раз в секунду	Раз в 8 часов	Незначительный
Вызовы VFS	1000–10 000 раз в секунду	Раз в час	Измеримый
Системные вызовы	1000–50 000 раз в секунду	Раз в 10 минут	Значительный
Сетевые пакеты	1000–100 000 раз в секунду	Раз в 5 минут	Значительный
Выделение памяти	10 000–1 000 000 раз в секунду	Раз в 30 секунд	Высокий
Приобретение блокировки	50 000–5 000 000 раз в секунду	Раз в 5 секунд	Высокий
Вызовы функций	До 100 000 000 раз в секунду	Три раза в секунду	Очень высокий
Инструкции процессора	До 1 000 000 000+ раз в секунду	Тридцать раз в секунду. Для справки: это частота звука ноты «до» контрпоясы фортепиано, предел восприятия человеческого уха	Очень высокий (в симуляторах процессора)
Такты процессора	До 3 000 000 000+ раз в секунду	Девяносто раз в секунду: нота «соль» большой октавы фортепиано	Очень высокий (в симуляторах процессора)

В этой книге я описывал оверхед, связанный с работой инструментов BPF, иногда приводя конкретные цифры, иногда употребляя слова «незначительный», «измеримый», «значительный» и «высокий». Я намеренно выбрал эти термины, хоть и расплывчатые, но достаточно описательные. Точные цифры могут вводить в заблуждение, поскольку конкретные значения зависят от рабочей нагрузки и системы. А вот как можно представить эти термины в численном выражении:

- незначительный: <0.1%;
- измеримый: ~1%;
- значительный: >5%;

- высокий: >30%;
- очень высокий: >300%.

В табл. 18.1 я экстраполировал частоту событий в описания, предполагая минимальный объем операций, выполняемых во время трассировки: подсчет в пространстве ядра и типичный размер современной системы. В следующем разделе показано, как разные действия могут увеличить оверхед.

18.1.2. Выполняемые действия

Следующие измерения описывают оверхед BPF как абсолютную стоимость каждого события и показывают, как различные действия могут его увеличивать. В ходе эксперимента я инструментировал операции чтения с рабочей нагрузкой `dd(1)`, выполняющей более одного миллиона чтений в секунду: после прочтения предыдущего раздела, описывающего типичную частоту разных событий, нетрудно догадаться, что оверхед на трассировку с BPF при такой частоте будет высоким.

Рабочая нагрузка:

```
dd if=/dev/zero of=/dev/null bs=1 count=10000k
```

Трассировка производилась с разными однострочниками `bpftrace`:

```
bpftrace -e 'kprobe:vfs_read { @ = count(); }'
```

Зная разницу во времени выполнения и количество событий, можно рассчитать затраты процессорного времени на каждое событие (без учета затрат на запуск и завершение процесса `dd(1)`). Они показаны в табл. 18.2.

Таблица 18.2. Затраты `bpftrace` на событие

bpftrace	Цель тестирования	Время выполнения dd (с)	Затраты BPF на событие (нс)
нет	Контрольный запуск без трассировки	5,97243	–
k:vfs_read { 1 }	Kprobe	6,75364	76
kr:vfs_read { 1 }	Kretprobe	8,13894	212
t:syscalls:sys_enter_read { 1 }	Точка трассировки	6,95894	96
t:syscalls:sys_exit_read { 1 }	Точка трассировки	6,9244	93
u:libc:__read { 1 }	Uprobe	19,1466	1287
ur:libc:__read { 1 }	Uretprobe	25,7436	1931

bpfttrace	Цель тестирования	Время выполнения dd (с)	Затраты BPF на событие (нс)
k:vfs_read /arg2 > 0/ { 1 }	Фильтр	7,24849	124
k:vfs_read { @ = count() }	Карта	7,91737	190
k:vfs_read { @[pid] = count() }	Один ключ	8,09561	207
k:vfs_read { @[comm] = count() }	Строковый ключ	8,27808	225
k:vfs_read { @[pid, comm] = count() }	Два ключа	8,3167	229
k:vfs_read { @[kstack] = count() }	Стек в пространстве ядра	9,41422	336
k:vfs_read { @[ustack] = count() }	Стек в пространстве пользо- вателя	12,648	652
k:vfs_read { @ = hist(arg2) }	Гистограмма	8,35566	233
k:vfs_read { @[tid] = nsecs } kr:vfs_read /@s[tid]/ { @ = hist(nsecs - @s[tid]); delete(@s[tid]); }	Измерение времени	12,4816	636 / 2 ¹
k:vfs_read { @[kstack, ustack] = hist(arg2) }	Несколько гистограмм	14,5306	836
k:vfs_read { printf("%d bytes\n", arg2) } > out.txt	Вывод каждого события	14,6719	850

Как видите, kprobes (в этой системе) работают довольно быстро, добавляя всего 76 наносекунд на вызов, и увеличивают это время примерно до 200 наносекунд при использовании карты с ключом. Зонды kretprobes работают намного медленнее, как и следовало ожидать, что обусловлено инструментированием точки входа в функцию и вставкой обработчика возврата (подробности см. в главе 2). Зонды uprobes и uretprobes увеличивают оверхед до более чем 1 микросекунды на событие: это известная проблема, которую хотелось бы устранить в будущих версиях Linux.

Это были короткие программы BPF. Длинные программы стоят намного дороже, и время их выполнения может исчисляться микросекундами.

Измерения проводились в Linux 4.15 со включенной поддержкой JIT-компилятора BPF, процессорами Intel (R) Core (TM) i7-8650U @ 1,90 ГГц и с использованием

¹ В этом случае к каждой операции чтения добавлялись 636 наносекунд, но использовались два зонда — kprobe и kretprobe, поэтому в действительности эти 636 наносекунд расходовались на два события BPF.

набора `taskset(1)` для привязки только к одному процессору для согласованности. Из 10 запусков выбирался самый быстрый (принцип наименьшего возмущения) при проверке стандартного отклонения на непротиворечивость. Имейте в виду, что все эти числа могут меняться в зависимости от быстродействия и архитектуры системы, текущей рабочей нагрузки и изменений в BPF.

18.1.3. Проверь себя

Если вы можете точно измерить производительность приложения, сделайте это с работающим инструментом трассировки BPF и без него и измерьте разницу. Если система работает с насыщением процессора (100%), то BPF будет отнимать такты процессора у приложения и разницу можно будет заметить по падению частоты запросов. Если система в основном простаивает, то разницу можно определить по снижению доступности процессора в режиме ожидания.

18.2. ВЫБОРКА С ЧАСТОТОЙ 49 ИЛИ 99 ГЦ

Цель выборки с такой, на первый взгляд странной, частотой — избежать попадания в одну и ту же точку.

Образцы отбираются по времени, чтобы получить приблизительную картину работы целевого ПО. Частоты 100 выборок в секунду (100 Гц) или 50 обычно достаточно, чтобы получить детали для решения больших и малых проблем производительности.

Возьмем частоту 100 Гц. При такой частоте данные выбираются каждые 10 миллисекунд. Теперь рассмотрим поток приложения, который возобновляет выполнение каждые 10 миллисекунд и за 2 миллисекунды завершает всю свою работу. Оно потребляет 20% времени одного процессора. Если выбирать данные с частотой 100 Гц и по совпадению инструмент профилирования будет запущен в нужный момент, то каждая выборка будет совпадать с двухмиллисекундным рабочим окном, поэтому профиль будет отображать приложение как потребляющее 100% процессорного времени. Или, если нажать Enter в другой момент, то ни в одном образце приложение не будет замечено, и профиль покажет, что оно потребляет 0% процессорного времени. Оба результата вводят в заблуждение, это примеры ошибки смещения эффектов.

При выборке данных с частотой 99 Гц вместо 100 моменты отбора образцов не всегда будут совпадать с работой приложения. При трассировке в течение достаточного количества секунд профиль покажет, что приложение выполняется на процессоре 20% времени. Эта частота достаточно близка к 100 Гц, чтобы можно было рассуждать о ней, как о частоте 100 Гц. Трассировка в системе с восемью процессорами выполняется в течение 1 секунды? Будет отобрано примерно 800 образцов. Я часто делаю такие расчеты при проверке своих результатов.

Если бы вместо этого мы выбрали, скажем, 73, это также позволило бы избежать блокировки выборки, но такие быстрые подсчеты не сделать в уме. 73 Гц на четырех процессорах в течение 8 секунд? Нужен калькулятор!

Стратегия 99 Гц работает только потому, что программисты обычно выбирают круглые числа для синхронизации действий: раз в секунду, 10 раз в секунду, каждые 20 миллисекунд и т. д. Если бы разработчики приложений выбирали частоту синхронизации 99 раз в секунду, то снова возникла бы проблема с попаданием в одну точку.

Назовем 99 «числом профилировщика». Не используйте его ни для чего, кроме профилирования!

18.3. ЖЕЛТЫЕ СВИНЬИ И СЕРЫЕ КРЫСЫ

Число 17 в математике — особенное. Его называют числом «желтой свиньи», есть даже день желтой свиньи, 17 июля [178]. Это число также удобно использовать для трассировки, хотя я предпочитаю 23.

Вы часто будете сталкиваться с неизвестными системами, не зная, с каких событий начать трассировку. Если вы сможете создать рабочую нагрузку с известными характеристиками, то подсчет частоты событий поможет выяснить, какие из них связаны с вашей рабочей нагрузкой.

Допустим, что вы хотите понять, как файловая система ext4 выполняет запись, но не знаете, какие события трассировать. Можно создать известную рабочую нагрузку с помощью dd(1) и выполнить 23 операции записи или, еще лучше, 230 000 операций, чтобы они выделялись на общем фоне:

```
# dd if=/dev/zero of=test bs=1 count=230000
230000+0 records in
230000+0 records out
230000 bytes (230 kB, 225 KiB) copied, 0.732254 s, 314 kB/s
```

Перед запуском этой команды в течение 10 секунд была запущена трассировка всех функций с именами, начинающимися с «ext4_»:

```
# funccount -d 10 'ext4_*'
Tracing 509 functions for "ext4_*"... Hit Ctrl-C to end.
^C
FUNC                                COUNT
ext4_rename2                        1
ext4_get_group_number               1
[...]
ext4_bio_write_page                 89
ext4_es_lookup_extent               142
ext4_es_can_be_merged               217
ext4_getattr                        5125
ext4_file_getattr                   6143
ext4_write_checks                   230117
ext4_file_write_iter                230117
ext4_da_write_end                   230185
ext4_nonda_switch                   230191
ext4_block_write_begin              230200
```

ext4_da_write_begin	230216
ext4_dirty_inode	230299
ext4_mark_inode_dirty	230329
ext4_get_group_desc	230355
ext4_inode_csum.isra.56	230356
ext4_inode_csum_set	230356
ext4_reserve_inode_write	230357
ext4_mark_iloc_dirty	230357
ext4_do_update_inode	230360
ext4_inode_table	230446
ext4_journal_check_start	460551
Detaching...	

Обратите внимание, что 15 из этих функций вызывались чуть больше 230 000 раз. Отсюда можно сделать вывод, что они, скорее всего, связаны с известной нам рабочей нагрузкой. Из 509 трассируемых функций ext4 мы выделили 15 кандидатов, используя простой трюк. Мне нравится число 23 (или 230, 2300 и т. д.), так как оно едва ли совпадет с другими счетчиками событий, не имеющими отношения к нашей рабочей нагрузке. Что еще может произойти 230 000 раз за 10 секунд трассировки?

23 и 17 — простые числа, которые обычно встречаются в вычислениях намного реже других, например степени 2 или 10. Я предпочитаю 23, потому что оно удалено от других чисел, равных степени двойки и 10, в отличие от 17. Назовем его числом «серой крысы»¹.

См. также раздел 12.4, где использовался этот же прием для выявления функций.

18.4. ПИШИТЕ ЦЕЛЕВОЕ ПО

Если сначала написать ПО, генерирующее нагрузку, а затем инструмент трассировки для ее измерения, то это может сэкономить вам время и избавить от головной боли.

Допустим, вы решили выполнить трассировку запросов DNS и отобразить задержку с информацией о запросе. С чего начать и как узнать, работает ли ваша программа? Если сначала вы напишете простой генератор запросов DNS, то узнаете, какие функции трассировать, в каких структурах хранится информация о запросах в структурах и что возвращают функции. Вы наверняка быстро научитесь этому, потому что в интернете всегда можно найти массу документации с примерами кода.

В данном случае страница справочного руководства для функции `getaddrinfo(3)` содержит целые программы, которые можно использовать:

```
$ man getaddrinfo
[...]  
    memset(&hints, 0, sizeof(struct addrinfo));  
    hints.ai_family = AF_UNSPEC;    /* Допускается IPv4 или IPv6 */  
    hints.ai_socktype = SOCK_DGRAM; /* Сокет для дейтаграмм */
```

¹ Количество усов у серой крысы. У меня есть куча игрушечных серых крыс из Ikea, возможно, их как раз 23.

```

hints.ai_flags = 0;
hints.ai_protocol = 0;          /* Любой протокол */

s = getaddrinfo(argv[1], argv[2], &hints, &result);
if (s != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(s));
    exit(EXIT_FAILURE);
}
[...]
```

Начав с этого, вы получите инструмент, генерирующий известные запросы. Вы даже можете изменить его так, чтобы он выполнял 23 запроса (или 2300), что упростит поиск среди других похожих функций в стеке (см. раздел 18.3).

18.5. ИЗУЧАЙТЕ СИСТЕМНЫЕ ВЫЗОВЫ

Системные вызовы — превосходные цели для трассировки.

Они подробно описаны на страницах справочного руководства, имеют точки трассировки и предоставляют полезную информацию об использовании ресурсов приложениями. Представьте, что вы использовали ВСС-инструмент `syscount(8)` и обнаружили высокую частоту вызова `setitimer(2)`. Что это?

```
$ man setitimer
GETITIMER(2)                  Справочник программиста Linux          GETITIMER(2)
```

НАЗВАНИЕ
`getitimer`, `setitimer` - читает или устанавливает значение таймера интервалов

СИНТАКСИС

```
#include <sys/time.h>

int getitimer(int which, struct itimerval *curr_value);
int setitimer(int which, const struct itimerval *new_value,
               struct itimerval *old_value);
```

ОПИСАНИЕ
Эти системные вызовы предоставляют доступ к интервальным таймерам, время срабатывания которых истекает где-то в будущем, и (при необходимости) через равные интервалы после этого момента. Когда таймер истекает, вызывающему процессу посылается сигнал и таймер переустанавливается для срабатывания через указанный интервал (если он не равен нулю).

[...]

`setitimer()`
Функция `setitimer()` взводит или сбрасывает таймер `which`, устанавливая его в значение, указанное в `new_value`. Если `old_value` не равно `NULL`, буфер, на который указывает этот аргумент, используется для возврата предыдущего значения таймера (то есть той же информации, которую возвращает `getitimer()`).

[...]

ВОЗВРАЩАЕМОЕ ЗНАЧЕНИЕ

В случае успеха возвращается ноль. В случае ошибки возвращается -1, а код ошибки записывается в errno.

Здесь объясняется, что делает `setitimer(2)`, какие аргументы получает и что возвращает¹. Эти данные доступны в точках трассировки: `syscalls:sys_enter_setitimer` и `syscalls:sys_exit_setitimer`.

18.6. НЕ УСЛОЖНЯЙТЕ

Старайтесь не писать длинных и сложных программ трассировки.

BPF — необычайно мощный механизм трассировки, способный трассировать всё и вся, поэтому так легко увлечься добавлением в программу все большего количества событий и упустить из виду первоначальную задачу. Это влечет проблемы:

- **Нежелательный оверхед:** первоначальную задачу можно было решить трассировкой лишь нескольких событий, но теперь инструмент трассирует намного больше событий, добавляя мало информации в сравнении с первоначальным вариантом, но требуя дополнительных затрат вычислительных ресурсов от всех, кто его использует.
- **Бремя обслуживания:** это особенно касается зондов `kprobes` и `uprobes`, поскольку они представляют нестабильный интерфейс, который может меняться между версиями ПО. В ветке Linux 4.x уже были прецеденты, когда в ядро вносились изменения, нарушавшие работу инструментов ВСС. Для исправления требовалось включать код для каждой версии ядра (часто выбираемый путем проверки наличия функций, потому что номера версий ядра — ненадежные индикаторы из-за обратного переноса новых особенностей в предыдущие версии) или просто дублировать инструменты, сохраняя копии для более старых ядер в каталоге `tools/old`. В лучшем случае добавлялись точки трассировки, чтобы прекратить такие поломки (например, `sock:inet_sock_set_state` для инструментов `tcp`).

Исправлять инструменты ВСС было нетрудно, потому что каждый из них обычно трассирует всего несколько событий или типов событий (именно так я их спроектировал). Если бы они трассировали десятки событий, поломки были бы более частыми, а устранять их было бы сложнее. Кроме того, пришлось бы увеличить количество тестов, чтобы проверить все типы событий во всех версиях ядра, для которых у инструмента есть уникальный код.

Я усвоил этот урок на собственном горьком опыте, когда 15 лет назад разработал инструмент `tcpsnoop(1m)`. Я хотел показать, какие процессы производят ввод/

¹ Справка выводится на английском языке, здесь приведен перевод. — *Примеч. пер.*

вывод по протоколу TCP, но написал инструмент для трассировки пакетов всех типов с помощью PID (пакеты процедуры инициализации соединения TCP, отклонения соединения с портом, UDP, ICMP и т. д.), чтобы его вывод напоминал вывод сетевого sniffера. Для этого требовалось трассировать множество нестабильных интерфейсов, и инструмент несколько раз ломался из-за обновлений ядра. Я упустил исходную задачу и создал то, что стало очень сложно поддерживать. (Подробнее об этом рассказывается в разделе с описанием tcpsnoop в главе 10.)

Инструменты bpftrace, которые я разработал и включил в эту книгу, — это результат моего 15-летнего опыта: я намеренно ограничиваю их минимальным количеством трассируемых событий, используя только те, что действительно нужны для решения конкретной задачи, и не более того. Рекомендую и вам поступать так же.

18.7. ОТСУТСТВИЕ СОБЫТИЙ

Это распространенная проблема: событие можно успешно инструментировать, но оно не возникает или инструмент не выводит никаких результатов. (Если событие вообще невозможно инструментировать, то см. раздел 18.10.) Инструментация событий с помощью утилиты perf(1) поможет выяснить, связана проблема с механизмом трассировки BPF или с самим событием.

Ниже показано, как с помощью perf(1) проверить наличие событий в точках трассировки block:block_rq_insert и block:block_rq_requeue:

```
# perf stat -e block:block_rq_insert,block:block_rq_requeue -a
^C
```

```
Performance counter stats for 'system wide':
```

```
41      block:block_rq_insert
0        block:block_rq_requeue
```

```
2.545953756 seconds time elapsed
```

В этом примере точка трассировки block:block_rq_insert сработала 41 раз, а точка трассировки block:block_rq_requeue не сработала ни разу. Если инструмент BPF, трассирующий точку block:block_rq_insert в то же самое время, не отметил ни одного события, это может указывать на проблему с инструментом BPF. Если инструмент BPF и perf(1) не отметили ни одного события, это означает, что проблема с событием: оно не возникает.

Вот как с помощью kprobes можно проверить, вызывается ли функция ядра vfs_read():

```
# perf probe vfs_read
```

```
Added new event:
```

```
probe: vfs_read      (on vfs_read)
```

You can now use it in all perf tools, such as:

```

perf record -e probe:vfs_read -aR sleep 1

# perf stat -e probe:vfs_read -a
^C
Performance counter stats for 'system wide':

      3,029 probe:vfs_read

      1.950980658 seconds time elapsed
# perf probe --del probe:vfs_read
Removed event: probe:vfs_read

```

Интерфейс perf(1) требует выполнять отдельные команды для создания и удаления зонда kprobe. С зондами uprobes он работает аналогично. Этот пример показывает, что за время трассировки функция vfs_read() вызывалась 3029 раз.

Иногда события пропадают после изменений в ПО, из-за которых ранее инструментировавшиеся события не возникают.

Бывает, что библиотечная функция трассируется по ее местоположению в разделяемой библиотеке, а целевое приложение скомпилировано статически, и функция вызывается из двоичного файла приложения.

18.8. ОТСУТСТВИЕ ТРАССИРОВОК СТЕКА

Под отсутствием здесь подразумеваются неполные или вообще отсутствующие трассировки стека. Еще одна схожая проблема — отсутствие символов (имен функций, см. раздел 18.9), из-за этого фреймы отображаются в трассировке как «[unknown]» (неизвестно).

Вот пример попытки использовать ВСС-инструмент trace(8), чтобы вывести трассировки стека в пространстве пользователя для точки трассировки execve() (запуск нового процесса):

```

# trace -U t:syscalls:sys_enter_execve
PID      TID      COMM      FUNC
26853    26853    bash      sys_enter_execve
          [unknown]
          [unknown]
26854    26854    bash      sys_enter_execve
          [unknown]
          [unknown]
[...]
```

Это еще один повод использовать perf(1) для перекрестной проверки, прежде чем углубляться в отладку ВСС/BPF. Вот результаты попытки решить ту же задачу с помощью perf(1):

```

# perf record -e syscalls:sys_enter_execve -a -g
^C[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 3.246 MB perf.data (2 samples) ]

```

```
# perf script
bash 26967 [007] 2209173.697359: syscalls:sys_enter_execve: filename:
                                0x56172df05030,
argv: 0x56172df3b680, envp: 0x56172df2df00
                                e4e37 __GI__execve (/lib/x86_64-linux-gnu/libc-2.27.so)
                                56172df05010 [unknown] ([unknown])

bash 26968 [001] 2209174.059399: syscalls:sys_enter_execve: filename:
0x56172df05090,
argv: 0x56172df04440, envp: 0x56172df2df00
                                e4e37 __GI__execve (/lib/x86_64-linux-gnu/libc-2.27.so)
                                56172df05070 [unknown] ([unknown])
```

Здесь можно видеть похожие неполные стеки, и тут есть три проблемы:

- **Неполные стеки.** Здесь видно, как оболочка `bash(1)` запускает новую программу: по своему опыту я знаю, что стек для этого действия имеет несколько фреймов в глубину, но в выводе выше видно только два фрейма (строки). Если ваши трассировки стека состоят из одной или двух строк и не заканчиваются начальным фреймом (например, «`main`» или «`start_thread`»), можно предположить, что они тоже неполные.
- **Последняя строка `[unknown]`.** Даже `perf(1)` не может разрешить символ. Проблема может быть связана с отсутствием таблицы символов в `bash(1)`, а может быть, что `__GI__execve()` в библиотеке `libc` подавила указатель фрейма, исключив возможность дальнейшего обхода стека.
- **Вызов `__GI__execve()` из `libc` был замечен утилитой `perf(1)`, но его нет в выводе `BCC`.** Это указывает на еще одну проблему с `trace(8)`, которую нужно исправить¹.

18.8.1. Как исправить проблему отсутствия трассировок стека

К сожалению, неполные трассировки стека — распространенная проблема. Обычно она обусловлена слиянием двух факторов: (1) инструмент наблюдения использует прием обхода стека на основе указателя фрейма, чтобы прочесть трассировку стека, и (2) целевой двоичный файл не резервирует регистр (RBP в архитектуре `x86_64`) для хранения указателя фрейма и повторно использует его как регистр общего назначения для оптимизации производительности. Инструмент наблюдения читает этот регистр, предполагая, что он содержит указатель фрейма, но храниться там может что угодно: число, адрес объекта, указатель на строку. Инструмент наблюдения пытается разрешить это число с использованием таблицы символов и, если повезет, не находит его и выводит «`[unknown]`». Если не повезет, то это случайное число преобразуется в символ, не имеющий никакого отношения к фактическому

¹ Могу предположить, что `perf(1)` использует отладочную информацию для получения этого фрейма. См. аналогичное исследование в `bpftrace #646 [179]`.

пути выполнения кода, и тогда в трассировке стека появляется неправильное имя функции, что сбивает с толку конечного пользователя.

Самое простое решение — исправить проблему с регистром указателя фрейма:

- **Для ПО на C/C++ и других языках, скомпилированного с помощью gcc или LLVM:** повторно скомпилируйте ПО с флагом `-fno-omit-frame-pointer`.
- **Для Java:** запустите `java(1)` с флагом `-XX:+PreserveFramePointer`.

Это может отрицательно сказаться на производительности, но часто такое снижение составляет менее 1%; преимущества возможности использовать трассировку стека для поиска проблем с производительностью и их устранения обычно оправдывают эти затраты. Этот вопрос я затрагивал в главе 12.

Другое решение — использовать метод обхода стека, не основанный на указателе фрейма. `perf(1)` поддерживает обход стека на основе DWARF, ORC и записи последней ветви (LBR). На момент написания этих строк возможность обхода стека на основе DWARF и LBR не поддерживалась в BPF, а технология ORC еще оставалась недоступной для ПО, выполняющегося в пространстве пользователя. Дополнительную информацию по этой теме ищите в разделе 2.4.

18.9. ОТСУТСТВИЕ СИМВОЛОВ (ИМЕН ФУНКЦИЙ) В ВЫВОДЕ

Под отсутствием символов подразумевается полное отсутствие имен функций в трассировке стека или вывод шестнадцатеричных чисел или строки «[unknown]» вместо имен функций. Одна из причин — неполные стеки, о чем говорилось в предыдущем разделе. Другая причина — скоротечность процессов, которые завершаются до того, как инструмент BPF успеет прочитать адресное пространство процесса и найти таблицы символов. Третья причина — отсутствие таблицы символов. Как это исправить, зависит от среды выполнения: в средах с JIT и в двоичных файлах ELF эти проблемы исправляются по-разному.

18.9.1. Как исправить проблему отсутствия символов: среда выполнения с JIT (Java, Node.js, ...)

Пропадание символов часто наблюдается в средах с поддержкой JIT-компиляции — Java и Node.js. В них JIT-компилятор имеет свою таблицу символов, которая изменяется во время выполнения и не является частью предварительно скомпилированных таблиц символов в двоичном файле. Типичное решение — использовать дополнительные таблицы символов, сгенерированные средой выполнения, которые сохраняются в файлы `/tmp/perf-<PID>.map` и используются утилитой `perf(1)` и инструментами BCC. Этот подход, некоторые пояснения и направления развития затрагиваются в разделе 12.3.

18.9.2. Как исправить проблему отсутствия символов: двоичные файлы ELF (C, C++, ...)

Таблицы символов могут отсутствовать в скомпилированных двоичных файлах, особенно в упакованных файлах, которые при подготовке к распространению обрабатывают утилитой `strip(1)`, чтобы уменьшить их размеры.

Один из способов исправить проблему — исключить из процесса сборки этап удаления символов. Другой способ — использовать иной источник информации о символах, например файлы с отладочной информацией или BTF. BCC и `bpfttrace` поддерживают извлечение символов из файлов с отладочной информацией. Этот подход, некоторые пояснения и направления развития затрагиваются в разделе 12.2.

18.10. ОТСУТСТВИЕ ФУНКЦИЙ В ТРАССИРОВКЕ

Имеется в виду ситуация, когда известная функция не поддается трассировке с помощью зондов `uprobes`, `uretprobes`, `kprobes` или `kretprobes`: кажется, что она вообще отсутствует или не вызывается. Проблема может быть связана с отсутствием таблицы символов (о чем говорилось в предыдущем разделе), оптимизациями компилятора или другими причинами:

- **Встраивание:** при встраивании инструкции, составляющие тело функции, включаются в тело вызывающей функции. Это может случиться с короткими функциями — так компилятор экономит на инструкциях вызова, возврата и создания пролога функции. Имя функции может вообще отсутствовать в таблице символов или присутствовать, но не поддаваться трассировке.
- **Оптимизация хвостового вызова:** для потока кода `A() -> B() -> C()`, где вызов `C()` — это последняя инструкция в `B()`, компилятор может заставить `C()` вернуться непосредственно в `A()`, оптимизируя скорость выполнения. Для такой функции зонды `uretprobe` или `kretprobe` не сработают.
- **Статическое и динамическое связывание:** в этом случае `uprobe` определяет функцию в библиотеке, но целевое ПО переключилось с динамического на статическое связывание, и местоположение функции изменилось: теперь она находится в двоичном файле. То же самое возможно и в обратном случае, когда `uprobe` определяет функцию в двоичном файле, но затем она переместилась в разделяемую библиотеку.

В таких случаях можно порекомендовать произвести трассировку другого события: вызов родительской, дочерней или соседней функции. `kprobes` и `uprobes` также поддерживают трассировку по смещениям инструкций (эта поддержка должна появиться и в `bpfttrace`), поэтому местоположение встроенной функции можно инструментировать, если известно ее смещение.

18.11. ЦИКЛЫ ОБРАТНОЙ СВЯЗИ

Иногда, выполняя трассировку, можно создать цикл обратной связи.

Вот пример того, чего следует избегать:

```
# bpfttrace -e 't:syscalls:sys_write_enter { printf(...) }'  
remote_host# bpfttrace -e 'k:tcp_sendmsg { printf(...) }'  
# bpfttrace -e 'k:ext4_file_write_iter{ printf(...) }' > /ext4fs/out.file
```

Первые две команды непреднамеренно запускают трассировку события вызова функции `printf()` из `bpfttrace` и создают другое событие вызова `printf()`, которое перехватывается трассировщиком и создает другое событие вызова `printf()`. Частота событий резко возрастет и производительность катастрофически уменьшается, пока вы не прервете `bpfttrace`.

Третья команда страдает той же проблемой: `bpfttrace` выполняет операцию записи в `ext4`, чтобы сохранить вывод, что приводит к созданию и сохранению большего объема вывода, и т. д.

Избежать образования таких циклов можно, применив фильтры и исключив трассировку вашего инструмента BPF или просто трассируя конкретный целевой процесс.

18.12. СБРОС СОБЫТИЙ

Помните, что пропущенные события делают вывод инструмента неполным.

Инструменты BPF могут выводить результаты очень быстро, вызывать переполнение выходного буфера `perf` или пытаться сохранить слишком много идентификаторов, переполнять карту стека BPF и т. д.

Например:

```
# profile  
[...]  
WARNING: 5 stack traces could not be displayed.
```

Инструменты должны сообщать, когда какие-то события были отброшены, как в примере выше. Часто ситуацию можно исправить настройкой. `profile(8)`, например, имеет параметр `-stack-storage-size`, с помощью которого можно увеличить размер карты стека, которая по умолчанию хранит 16 384 уникальные трассировки стека. Если к подобным настройкам приходится прибегать слишком часто, то следует обновить настройки инструмента по умолчанию, чтобы пользователям не приходилось их менять.

Приложение А

ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPFTRACE

Здесь приведены избранные однострочные сценарии, представленные в книге.

Глава 6. Процессоры

Трассирует запуск новых процессов и их аргументов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Выводит число системных вызовов, выполненных каждым процессом:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Выбирает имена работающих процессов с частотой 99 Гц:

```
bpftrace -e 'profile:hz:99 { @[comm] = count(); }'
```

Выбирает трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Трассирует запуск новых потоков вызовом pthread_create():

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpthread-2.27.so:pthread_create {  
    printf("%s by %s (%d)\n", probe, comm, pid); }'
```

Глава 7. Память

Подсчитывает события расширения кучи по процессам (brk()) по трассировкам стека:

```
bpftrace -e tracepoint:syscalls:sys_enter_brk { @[ustack, comm] = count(); }
```

Подсчитывает сбои страниц по процессам:

```
bpftrace -e 'software:page-fault:1 { @[comm, pid] = count(); }'
```

Подсчитывает сбои страниц путем трассировки стека в пространстве пользователя:

```
bpftrace -e 'tracepoint:exceptions:page_fault_user { @[ustack, comm] = count(); }'
```

Подсчитывает операции в vmscan с использованием точек трассировки:

```
bpftrace -e 'tracepoint:vmscan:* { @[probe]++; }'
```

Глава 8. Файловые системы

Трассирует события открытия файлов процессами с использованием open(2):

```
bpftrace -e 't:syscalls:sys_enter_open { printf("%s %s\n", comm, str(args->filename)); }'
```

Выводит гистограмму с распределением вызовов read() по запрошенным размерам блоков:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_read { @ = hist(args->count); }'
```

Выводит гистограмму с распределением вызовов read() по размерам прочитанных блоков (и кодам ошибок):

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Подсчитывает количество вызовов VFS:

```
bpftrace -e 'kprobe:vfs_* { @[probe] = count(); }'
```

Подсчитывает количество прохождений через точки трассировки в ext4:

```
bpftrace -e 'tracepoint:ext4:* { @[probe] = count(); }'
```

Глава 9. Дисковый ввод/вывод

Подсчитывает операции блочного ввода/вывода по точкам трассировки:

```
bpftrace -e 'tracepoint:block:* { @[probe] = count(); }'
```

Суммирует размер блочного ввода-вывода в виде гистограммы:

```
bpftrace -e 't:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Подсчитывает запросы на блочный ввод/вывод по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:block:block_rq_issue { @[ustack] = count(); }'
```

Подсчитывает операции блочного ввода/вывода по флагам типа:

```
bpftrace -e 't:block:block_rq_issue { @[args->rwbs] = count(); }'
```


Трассирует ошибки блочного ввода/вывода, группируя их по устройствам и типам операций:

```
bpftrace -e 't:block:block_rq_complete /args->error/ {
    printf("dev %d type %s error %d\n", args->dev, args->rwbs, args->error); }'
```

Подсчитывает операции SCSI по их кодам:

```
bpftrace -e 't:scsi:scsi_dispatch_cmd_start { @opcode[args->opcode] =
    count(); }'
```

Подсчитывает операции SCSI по кодам результатов (учитывает все 4 байта):

```
bpftrace -e 't:scsi:scsi_dispatch_cmd_done { @result[args->result] = count(); }'
```

Подсчитывает вызовы функций драйвера scsi:

```
bpftrace -e 'kprobe:scsi* { @[func] = count(); }'
```

Глава 10. Сети

Подсчитывает вызовы accept(2) по идентификаторам (PID) и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_accept* { @[pid, comm] = count(); }'
```

Подсчитывает вызовы connect(2) по идентификаторам (PID) и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_connect { @[pid, comm] = count(); }'
```

Подсчитывает байты отправки/получения сокета по PID на процессоре и имени процесса:

```
bpftrace -e 'kr:sock_sendmsg,kr:sock_recvmsg /retval > 0/ {
    @[pid, comm, retval] = sum(retval); }'
```

Подсчитывает операции приема и передачи по протоколу TCP:

```
bpftrace -e 'k:tcp_sendmsg,k:tcp*recvmsg { @[func] = count(); }'
```

Выводит гистограмму размеров отправленных сообщений TCP в байтах:

```
bpftrace -e 'k:tcp_sendmsg { @send_bytes = hist(arg2); }'
```

Выводит гистограмму размеров полученных сообщений TCP в байтах:

```
bpftrace -e 'kr:tcp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчитывает повторные передачи TCP по типам и именам удаленных узлов (предполагается использование IPv4):

```
bpftrace -e 't:tcp:tcp_retransmit_* { @[probe, ntop(2, args->saddr)] =
    count(); }'
```

Выводит гистограмму с размерами отправленных пакетов UDP:

```
bpftrace -e 'k:udp_sendmsg { @send_bytes = hist(arg2); }'
```

Подсчитывает трассировки стека в ядре, связанные с отправкой пакетов:

```
bpftrace -e 't:net:net_dev_xmit { @[kstack] = count(); }'
```

Глава 11. Безопасность

Подсчитывает события аудита безопасности для PID 1234:

```
bpftrace -e 'k:security_* /pid == 1234 { @[func] = count(); }'
```

Трассирует запуск сеанса аутентификации с использованием механизма подключаемых модулей аутентификации (PAM):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpam.so.0:pam_start {  
    printf("%s: %s\n", str(arg0), str(arg1)); }'
```

Трассирует загрузку модулей ядра:

```
bpftrace -e 't:module:module_load { printf("load: %s\n", str(args->name)); }'
```

Глава 13. Приложения

Суммирует объем памяти в байтах, выделяемой вызовом malloc(), с группировкой по трассировкам стека в пространстве пользователя (имеет высокий оверхед):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libc-2.27.so:malloc { @[ustack(5)] =  
    sum(arg0); }'
```

Трассирует вызовы kill() для отправки сигнала и отображает имя процесса-отправителя, идентификатор (PID) получателя и номер сигнала:

```
bpftrace -e 't:syscalls:sys_enter_kill { printf("%s -> PID %d SIG %d\n",  
    comm, args->pid, args->sig); }'
```

Подсчитывает количество вызовов в секунду функций из библиотеки libpthread для блокировки мьютексов:

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_mutex_lock {  
    @[probe] = count(); } interval:s:1 { exit(); }'
```

Подсчитывает количество вызовов в секунду функций из библиотеки libpthread для доступа к условным переменным:

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpthread.so.0:pthread_cond_* {  
    @[probe] = count(); } interval:s:1 { exit(); }'
```

Глава 14. Ядро

Подсчитывает обращения к системным вызовам по именам функций системных вызовов:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
    @[ksym(*(kaddr("sys_call_table") + args->id * 8))] = count(); }'
```

Подсчитывает вызовы функций ядра с именами, начинающимися с «attach»:

```
bpftrace -e 'kprobe:attach* { @[probe] = count(); }'
```

Измеряет время выполнения функции ядра `vfs_read()` и выводит результаты в виде гистограммы:

```
bpftrace -e 'k:vfs_read { @ts[tid] = nsecs; } kr:vfs_read /@ts[tid]/ {
    @ = hist(nsecs - @ts[tid]); delete(@ts[tid]); }'
```

Подсчитывает разные значения в первом целочисленном аргументе в вызове функции ядра «func1»:

```
bpftrace -e 'kprobe:func1 { @[arg0] = count(); }'
```

Подсчитывает разные значения, возвращаемые функцией ядра «func1»:

```
bpftrace -e 'kretprobe:func1 { @[retval] = count(); }'
```

Выбирает трассировки стека в пространстве ядра с частотой 99 Гц, исключая поток, обслуживающий простой системы:

```
bpftrace -e 'profile:hz:99 /pid/ { @[kstack] = count(); }'
```

Подсчитывает количество переключений контекста по трассировкам стека:

```
bpftrace -e 't:sched:sched_switch { @[kstack, ustack, comm] = count(); }'
```

Подсчитывает запросы к очереди заданий по функциям ядра:

```
bpftrace -e 't:workqueue:workqueue_execute_start { @[ksym(args->function)] =
    count() }'
```

Приложение В

ШПАРГАЛКА ПО BPFTRACE

Синтаксис

```
bpftrace -e 'probe /filter/ { action; }'
```

Зонды

BEGIN, END	Запуск и завершение программы
tracepoint:syscalls:sys_enter_execve	Системный вызов execve(2)
tracepoint:syscalls:sys_enter_open	Системный вызов open(2) (также трассирует openat(2))
tracepoint:syscalls:sys_exit_read	Выход из системного вызова read(2) (один вариант)
tracepoint:raw_syscalls:sys_enter	Все системные вызовы
block:block_rq_insert	Постановка в очередь запроса на блочный ввод/вывод
block:block_rq_issue	Передача запроса на блочный ввод/вывод из очереди в устройство хранения
block:block_rq_complete	Завершение блочного ввода/вывода
sock:inet_sock_set_state	Изменение состояния сокета
sched:sched_process_exec	Запуск процесса
sched:sched_switch	Переключение контекста
sched:sched_wakeup	Событие пробуждения потока
software:faults:1	Отказы страниц
hardware:cache-misses:1000000	Один раз на 1 000 000 промахов кэша LLC
kprobe:vfs_read	Вход в функцию ядра vfs_read()
kretprobe:vfs_read	Выход из функции ядра vfs_read()
uprobe:/bin/bash:readline	Трассировка функции readline() в /bin/bash
uretprobe:/bin/bash:readline	Трассировка выхода из функции readline() в /bin/bash
usdt:path:probe	Трассировка зонда USDT probe в пути path
profile:hz:99	Выборка стеков всех процессоров с частотой 99 Гц
interval:s:1	Выполнение на одном процессоре раз в секунду

Псевдонимы зондов

t	tracepoint	U	usdt	k	kprobe	kr	kretprobe	p	profile
s	software	h	hardware	u	uprobe	ur	uretprobe	i	interval

Переменные

comm	Имя процесса, выполняющегося на процессоре	username	Строка с именем пользователя
pid, tid	Идентификатор PID выполняющегося процесса, идентификатор TID выполняющегося потока	uid	Идентификатор пользователя
cpu	Идентификатор процессора	kstack	Трассировка стека в пространстве ядра
nsecs	Время, в наносекундах	ustack	Трассировка стека в пространстве пользователя
elapsed	Время, прошедшее с начала программы, в наносекундах	probe	Полное имя текущего зонда
arg0..N	Аргументы [uk]probe	func	Имя текущей функции
args->	Аргументы точки трассировки	\$1..\$N	Аргументы командной строки, целое число
retval	Возвращаемое значение [uk]retprobe	str(\$1)...	Аргументы командной строки, строка
cgroup	Идентификатор текущей контрольной группы cgroup	curtask	Указатель на структуру task текущей задачи

Действия

@map[key1, ...] = count()	Подсчет частоты
@map[key1, ...] = sum(var)	Сумма значений переменной var
@map[key1, ...] = hist(var)	Гистограмма с шагом по степеням двойки
@map[key1, ...] = lhist(var, min, max, step)	Линейная гистограмма
@map[key1, ...] = stats(var)	Статистики: количество, среднее и сумма
min(var), max(var), avg(var)	Минимальное, максимальное и среднее значения
printf("format", var0..varN)	Вывод переменных; для вывода агрегатов используйте print()
kstack(num), ustack(num)	Вывод заданного количества кадров (num) из стека в пространстве ядра и пользователя
ksym(ip), usym(ip)	Строка с символом в пространстве ядра/пользователя из указателя инструкций
kaddr("name"), uaddr("name")	Адрес в пространстве ядра/пользователя, соответствующий символу name
str(str[, Len])	Строка из адреса
ntop([af], addr)	IP-адрес в виде строки

Асинхронные действия

<code>printf("format", var0..varN)</code>	Вывод переменных, для вывода агрегатов используйте <code>print()</code>
<code>system("format", var0..varN)</code>	Запуск команды в командной строке
<code>time("format")</code>	Вывод времени в форматированном виде
<code>clear(@map)</code>	Очистка карты <code>map</code> : удаляет все ключи
<code>print(@map)</code>	Вывод карты <code>map</code>
<code>exit()</code>	Выход

Ключи

<code>-e</code>	Трассировать зонд, определенный в описании
<code>-l</code>	Вывести список зондов вместо трассировки
<code>-p PID</code>	Включить зонды USDT для PID
<code>-c 'command'</code>	Вызвать команду <code>command</code>
<code>-v, -d</code>	Подробный вывод для отладки

Приложение С

РАЗРАБОТКА ИНСТРУМЕНТОВ ВСС

Здесь приведены приемы разработки инструментов ВСС. Это продолжение главы 4, и оно опционально. В главе 5 рассказано, как разрабатывать инструменты для bpftrace на языке более высокого уровня, которого, как ожидается, во многих случаях будет достаточно. Также см. главу 18, где затронуты вопросы минимизации оверхеда, общие для разработки инструментов для ВСС и bpftrace.

РЕСУРСЫ

Я написал три подробных руководства по разработке инструментов ВСС и выложил их в общий доступ в репозитории ВСС, где они поддерживаются другими участниками проекта и доступны всем желающим:

- **BCC Python Developer Tutorial:** содержит более 15 уроков по разработке инструментов ВСС с использованием интерфейса Python. В каждом из этих уроков рассмотрен свой набор деталей, которые нужно знать [180].
- **BCC Reference Guide:** полный справочник по BPF C API и BCC Python API. Охватывает все возможности ВСС и на каждую возможность приводит короткие примеры. Главная его цель — быть справочником [181].
- **Contributing BCC/eBPF scripts:** чек-лист для разработчиков инструментов, которые хотят внести свой вклад в репозиторий ВСС. В нем обобщен опыт многих лет разработки и поддержки инструментов трассировки [63].

Это приложение — дополнительный ресурс для изучения разработки инструментов ВСС: экспресс-курс обучения на примерах. Здесь представлены четыре программы на Python: `hello_world.py` (базовый пример), `sleepsnoop.py` (вывод по событиям), `bitehist.py` (представление гистограмм, сигнатур функций и структур) и `biolatency.py` (пример действующего инструмента).

ПЯТЬ СОВЕТОВ

Вот что нужно знать перед написанием инструментов ВСС:

1. BPF C — ограниченный язык: в нем нет циклов и возможности вызывать функции ядра. Допустимо использовать только вспомогательные функции ядра `bp*_*` и некоторые встроенные функции компилятора.

2. Чтение данных из памяти производится только с помощью функции `bpf_probe_read()`, которая выполняет все проверки. Если вы захотите разыменовать `a->b->c->d`, попробуйте сначала сделать это напрямую, поскольку компилятор BCC способен превращать подобные инструкции в вызовы `bpf_probe_read()`. Если не получится, тогда используйте явные вызовы `bpf_probe_reads()`.
 - Содержимое из памяти можно читать только в стек или в карты BPF. Стек имеет ограниченный размер, поэтому для хранения больших объектов используйте карты BPF.
3. Вывести данные из пространства ядра в пространство пользователя можно тремя способами:
 - **BPF_PERF_OUTPUT():** этот способ используется для передачи сведений о каждом событии через определяемую вами структуру;
 - **BPF_HISTOGRAM()** или **через другие карты BPF:** карта — это хеш (словарь), хранящий пары ключ — значение, на основе которого можно строить довольно сложные структуры данных. Карты можно использовать для хранения статистик или гистограмм и периодически читать (весьма эффективно) из пространства пользователя;
 - **bpf_trace_printk():** только для отладки. Эта функция выводит данные в `trace_pipe` и может конфликтовать с другими программами и трассировщиками.
4. По возможности используйте статические инструменты (точки трассировки, USDT) вместо динамических (`kprobes`, `uprobes`). Динамическая инструментация — это нестабильный API, поэтому ваши инструменты будут терять работоспособность, если инструментируемый ими код изменится.
5. Следите за развитием проекта BCC и появлением в нем новых функций и возможностей, а также за развитием `bpftool` на тот случай, если он окажется достаточным для ваших нужд.

ПРИМЕРЫ ИНСТРУМЕНТОВ

Для обучения основам программирования BCC я выбрал следующие примеры инструментов: программы `hello_world.py` и `sleepsnoop.py`, показывающие вывод каждого события, а также `bitehist.py` и `biolatency.py`, показывающие вывод гистограмм.

Инструмент 1: `hello_world.py`

Простой инструмент начального уровня. Рассмотрим сначала его вывод:

```
# hello_world.py
ModuleProcessTh-30136 [005] .... 2257559.959119: 0x00000001: Hello, World!
SendControllerT-30135 [002] .... 2257559.971135: 0x00000001: Hello, World!
SendControllerT-30142 [007] .... 2257559.974129: 0x00000001: Hello, World!
ModuleProcessTh-30153 [000] .... 2257559.977401: 0x00000001: Hello, World!
SendControllerT-30135 [003] .... 2257559.996311: 0x00000001: Hello, World!
[...]
```


Он выводит строку в ответ на некоторое событие, заканчивающуюся текстом «Hello, World!».

Вот исходный код программы `hello_world.py`:

```
1  #!/usr/bin/python
2  from bcc import BPF
3  b = BPF(text="""
4  int kprobe__do_nanosleep()
5  {
6      bpf_trace_printk("Hello, World!\\n");
7      return 0;
8  }""");
9  b.trace_print()
```

Строка 1 определяет местоположение интерпретатора Python. Некоторые предпочитают использовать «#!/usr/bin/env python», чтобы командная оболочка сама отыскивала первый попавшийся исполняемый файл python.

Строка 2 импортирует библиотеку BPF из BCC.

Строки с 4-й по 8-ю, выделенные жирным, содержат исходный код программы BPF для выполнения в пространстве ядра. Она написана на C, включена в родительскую программу на Python как текстовая строка в тройных кавычках и передается в аргумент `text` новому объекту BPF().

В строке 4 используется сокращенный способ инструментации зонда `kprobe`. Он заключается в объявлении функции, имя которой начинается с «`kprobe__`». Остальная часть имени функции интерпретируется как имя инструментируемой функции, в данном случае `do_nanosleep()`. Этот способ пока редко используется другими инструментами, потому что многие из них были написаны до появления такой возможности. Инструменты часто используют вызов Python-функции `BPF.attach_kprobe()`.

Строка 6 вызывает `bpf_trace_printk()` с сообщением «Hello World!», за которым следует символ перевода строки (экранирован дополнительным символом слеша «\», чтобы сохранить «\n» до последнего этапа компиляции). `bpf_trace_printk()` выводит строку в разделяемый буфер трассировки.

Строка 9 вызывает Python-функцию `trace_print()` объекта BPF. Она извлекает сообщения из буфера трассировки в пространстве ядра и выводит их.

Здесь я использовал интерфейс `bpf_trace_printk()`, только чтобы сделать этот пример максимально коротким. На самом деле он применяется только для отладки, так как использует буфер, доступный другим инструментам (и может читаться из пространства пользователя через `/sys/kernel/debug/tracing/trace_pipe`). Запуск этого инструмента одновременно с другими может привести к смешиванию их вывода. Рекомендуемый интерфейс показывает инструмент `sleepsnoop.py`.

Инструмент 2: `sleepsnoop.py`

Трассирует вызовы `do_nanosleep()`, выводит отметки времени и идентификаторы процессов, а также иллюстрирует использование выходного буфера. Пример вывода:

```
# sleepsnoop.py
TIME(s)      PID    CALL
489488.676744000  5008  Hello, World!
489488.676740000  4942  Hello, World!
489488.676744000  32469  Hello, World!
489488.677674000  5006  Hello, World!
[...]
```

Исходный код:

```
1  #!/usr/bin/python
2
3  from bcc import BPF
4
5  # Программа BPF
6  b = BPF(text="""
7  struct data_t {
8      u64 ts;
9      u32 pid;
10 };
11
12 BPF_PERF_OUTPUT(events);
13
14 int kprobe__do_nanosleep(void *ctx) {
15     struct data_t data = {};
16     data.pid = bpf_get_current_pid_tgid();
17     data.ts = bpf_ktime_get_ns() / 1000;
18     events.perf_submit(ctx, &data, sizeof(data));
19     return 0;
20 };
21 """)
22
23 # заголовок
24 print("%-18s %-6s %s" % ("TIME(s)", "PID", "CALL"))
25
26 # обработка события
27 def print_event(cpu, data, size):
28     event = b["events"].event(data)
29     print("%-18.9f %-6d Hello, World!" % ((float(event.ts) / 1000000),
30     event.pid))
31
32 # цикл с обратным вызовом print_event
33 b["events"].open_perf_buffer(print_event)
34 while 1:
35     try:
36         b.perf_buffer_poll()
37     except KeyboardInterrupt:
38         exit()
```

Строки с 7-й по 10-ю определяют выходную структуру `data_t`. Она содержит два члена: `u64` (64-битное целое без знака) для отметки времени и `u32` для идентификатора процесса (`pid`).

Строка 12 объявляет буфер вывода с именем «events» для событий.

В строке 14 инструментируется функция `do_nanosleep()`, как и в предыдущем примере `hello_world.py`.

Строка 15 объявляет переменную `data` с типом структуры `data_t` и инициализирует ее нулями (это нужно, потому что верификатор BPF отклонит попытку доступа к неинициализированной памяти).

Строки 16 и 17 заполняют члены структуры `data` с помощью вспомогательных функций BPF.

Строка 18 передает структуру `data` в буфер событий `events`.

Строки с 27-й по 30-ю объявляют функцию обратного вызова с именем `print_event()`, обрабатывающую события из буфера. Она читает событие в строке 28 в объект с именем `event` и обращается к его членам в строках 29 и 30. (В старых версиях ВСС требовалось писать больше кода и объявлять в Python макет структуры данных, теперь это происходит автоматически.)

Строка 33 регистрирует функцию `perf_event()` как обработчик событий в буфере с именем `events`.

Строки с 34-й по 38-ю опрашивают открытые выходные буферы. Если обнаруживаются новые события, они передаются зарегистрированным обработчикам. Завершить программу можно нажатием `Ctrl-C`.

Если события следуют часто, то программа на Python, выполняющаяся в пространстве пользователя, будет часто возобновлять выполнение для их обработки. Для оптимизации некоторые инструменты вводят непродолжительную приостановку в последнем цикле `while`. Она позволяет буферизовать несколько событий, сократить процессорное время, расходуемое интерпретатором Python, и снизить общий оверхед.

Если события следуют очень часто, то желательно подумать о возможности их обобщения в пространстве ядра, поскольку это должно снизить оверхед. Инструмент `bitehist.py` — пример реализации такого решения.

Инструмент 3: `bitehist.py`

Выводит объемы данных, вовлеченные в операции дискового ввода/вывода, в виде гистограммы с шагом по степеням двойки. Похожий инструмент можно найти в каталоге ВСС `examples/tracing`. Сначала рассмотрим вывод этого инструмента, чтобы понять, что он делает, а затем перейдем к коду:

```
# bitehist.py
Tracing block I/O... Hit Ctrl-C to end.
^C
```

kbytes	: count	distribution
0 -> 1	: 3	**
2 -> 3	: 0	
4 -> 7	: 55	*****
8 -> 15	: 26	*****
16 -> 31	: 9	*****
32 -> 63	: 4	**
64 -> 127	: 0	

128 -> 255	: 1		
256 -> 511	: 0		
512 -> 1023	: 1		

Полный код программы BCC с пронумерованными строками:

```

1  #!/usr/bin/python
2  #[...]
3  from __future__ import print_function
4  from bcc import BPF
5  from time import sleep
6
7  # загрузка программы BPF
8  b = BPF(text="""
9  #include <uapi/linux/ptrace.h>
10
11  BPF_HISTOGRAM(dist);
12
13  int kprobe__blk_account_io_completion(struct pt_regs *ctx,
14    void *req, unsigned int bytes)
15  {
16      dist.increment(bpf_log2l(bytes / 1024));
17      return 0;
18  }
19  """)
20
21  # заголовок
22  print("Tracing block I/O... Hit Ctrl-C to end.")
23
24  # трассировать до нажатия Ctrl-C
25  try:
26      sleep(99999999)
27  except KeyboardInterrupt:
28      print()
29
30  # вывод
31  b["dist"].print_log2_hist("kbytes")

```

Строки 1–8 содержат тот же код, что уже был описан в примере `hello_world.py`.

Строка 9 подключает заголовок, информацию из которого использует программа BPF (`struct pt_regs`).

Строка 11 объявляет карту BPF с именем «`dist`» для хранения и вывода гистограммы.

Строки 13 и 14 объявляют сигнатуру функции `blk_account_io_completion()`. Первый аргумент, «`struct pt_regs *ctx`», хранит состояние регистров в точке инструментации и не имеет отношения к аргументам целевой функции. Остальные аргументы соответствуют аргументам функции, исходный код которой можно найти в `block/blk-core.c`:

```
void blk_account_io_completion(struct request *req, unsigned int bytes)
```

Меня интересует аргумент `bytes`. Но чтобы позиции аргументов совпадали, нужно также объявить аргумент «`struct request *req`», даже при том, что структура `request`

не используется в программе BPF. Эта структура по умолчанию не известна BPF, поэтому простое включение ее в сигнатуру функции приведет к сбою компиляции инструмента в BPF. Есть два решения этой проблемы: (1) добавить инструкцию `#include <linux/blkdev.h>`, чтобы получить определение структуры `request` из заголовочного файла, или (2) заменить «`struct request *req`» на «`void *req`», потому что тип `void` уже известен BPF, а фактическая потеря действительной информации о типе не так важна, поскольку программа не обращается к этой структуре. В этом примере я использовал второе решение.

Строка 16 делит аргумент `bytes` на 1024 и передает результат в килобайтах функции `bpf_log2l()`, которая генерирует показатель степени двойки. Значение показателя затем сохраняется в гистограмме `dist` вызовом функции `dist.increment()`, которая увеличивает на единицу значение в интервале, соответствующем этому показателю. Поясню на примере:

1. Представьте, что с первым событием в переменной `bytes` получено значение 4096.
2. $4096 / 1024 = 4$
3. `bpf_log2l(4) = 3`
4. `dist.increment(3)` прибавит 1 к значению в интервале с индексом 3, после чего гистограмма `dist` будет содержать:
 - интервал 1: значение 0 (для объемов от 0 до 1 Кбайт)
 - интервал 2: значение 0 (для объемов от 2 до 3 Кбайт)
 - интервал 3: значение 1 (для объемов от 4 до 7 Кбайт)
 - интервал 4: значение 0 (для объемов от 8 до 15 Кбайт)
 - ...

Эти значения интервалов читаются программой из пространства пользователя и выводятся в виде гистограммы.

Строка 22 выводит заголовок. При использовании этого инструмента полезно заметить, когда выводится заголовок: его появление на экране сообщает, что этапы компиляции ВСС и инструментации событий завершены и вот-вот начнется трассировка. Содержимое начального сообщения соответствует соглашению и объясняет, что делает инструмент и когда завершится:

- **Tracing (трассировка):** это слово сообщает пользователю, что инструмент выполняет трассировку событий. Если бы инструмент производил выборку (профилирование), он сообщил бы в первом слове `Sampling` (выборка) или `Profiling` (профилирование).
- **block I/O (блочный ввод/вывод):** сообщает пользователю, какие события инструментятся.
- **Hit Ctrl-C to end. (Нажмите Ctrl-C для завершения):** сообщает пользователю, как завершить программу. Инструменты, генерирующие вывод через равные интервалы, могут включать, например, такой текст: «Output every 1

second, Ctrl-C to end» (Данные выводятся раз в секунду, нажмите Ctrl-C для завершения).

Строки с 25-й по 28-ю приостанавливают программу и проверяют факт нажатия Ctrl-C. Если нажатие было определено, то программа выводит символ перевода строки для подготовки экрана к выводу.

Строка 31 выводит гистограмму `dist` как гистограмму с шагом по степеням с подписями столбцов, отражающих диапазон в килобайтах. Для этого она извлекает значения интервалов из пространства ядра. Как этот вызов `BPF.print_log2_hist()` понимает, к какому диапазону относится каждое значение? Диапазоны не передаются из ядра, передаются только значения. Однако диапазоны известны, потому что алгоритмы `log2`, действующие в пространствах пользователя и ядра, совпадают.

Вот еще один способ написать код BPF, который может служить примером разыменования структуры:

```
#include <uapi/linux/ptrace.h>
#include <linux/blkdev.h>

BPF_HISTOGRAM(dist);

int kprobe__blk_account_io_completion(struct pt_regs *ctx, struct request *req)
{
    dist.increment(bpf_log2l(req->__data_len / 1024));
    return 0;
}
```

Здесь значение `bytes` извлекается из структуры `request`, из ее члена `__data_len`. Поскольку здесь обрабатывается структура `request`, в коде потребовалось подключить заголовок `linux/blkdev.h`, где находится определение этой структуры. Кроме того, в этой функции не используется второй аргумент `bytes`, поэтому я опустил его объявление в сигнатуре функции: завершающие неиспользуемые аргументы можно опустить, потому что они не влияют на позиции предыдущих аргументов.

На самом деле аргументы (следующие после `struct pt_regs *ctx`), определяемые в программе BPF, отображаются в регистры, соответствующие соглашению о вызове функций. В архитектуре `x86_64` это `%rdi`, `%rsi`, `%rdx` и т. д. Если ошибиться в определении сигнатуры функции, инструмент BPF успешно скомпилирует и применит эту сигнатуру к регистрам, но вы получите неверные данные.

Разве ядро не должно знать имена и позиции аргументов функции? Почему я повторно объявляю их в программе BPF? Ответ прост: да, ядро знает, если в системе установлено ядро с отладочной информацией. Но на практике это случается редко, потому что файлы с отладочной информацией могут быть большими.

Сейчас проводится работа над облегченными форматами представления метаданных, которые должны решить эту проблему: формат `BPF Type`, например, в будущем может включаться в двоичный файл ядра `vmlinux` и когда-нибудь, возможно,

станет доступен и для двоичных файлов пространства пользователя. Мы надеемся, что это избавит нас от необходимости подключать файлы заголовков и повторно объявлять сигнатуры функций (см. раздел 2.3.9).

Инструмент 4: biolateness

Ниже приведен код моего оригинального инструмента `biolateness.py` с комментариями:

```
1  #!/usr/bin/python
2  # @lint-avoid-python-3-compatibility-imports
```

Строка 1: путь к интерпретатору Python.

Строка 2 подавляет вывод предупреждений статического анализатора (используется в среде сборки в Facebook).

```
3  #
4  # biolateness Выводит задержки в устройстве блочного ввода/вывода в виде гистограммы.
5  #           Только для Linux, использует BCC, eBPF.
6  #
7  # ПОРЯДОК ИСПОЛЬЗОВАНИЯ: biolateness [-h] [-T] [-Q] [-m] [-D] [интервал] [счетчик]
8  #
9  # Copyright (c) 2015 Brendan Gregg.
10 # Распространяется на условиях лицензии Apache License, Version 2.0 (the "License")
11 #
12 # Создан 20-сен-2015, автор Брендан Грегг (Brendan Gregg).
```

У меня есть свой стиль оформления комментариев в заголовках. Строка 4 определяет название инструмента и содержит краткое описание из одного предложения. Строка 5 добавляет дополнительные предостережения: «только для Linux, использует BCC/eBPF»¹. Далее следует описание использования, авторские права и история основных изменений.

```
13
14 from __future__ import print_function
15 from bcc import BPF
16 from time import sleep, strftime
17 import argparse
```

Обратите внимание, что я импортирую класс BPF, он нужен для взаимодействия с инфраструктурой BPF в ядре.

```
18
19 # аргументы
20 examples = """примеры :
21     ./biolateness      # выводит задержки блочного ввода/вывода в виде
                        # гистограммы
```

¹ Здесь упоминается eBPF, потому что инструмент написан еще во времена, когда инфраструктура называлась eBPF. Сейчас мы называем ее просто BPF.

```

22     ./biolatency 1 10 # выводит сводки раз в 1 секунду, 10 раз
23     ./biolatency -mT 1 # посекундные сводки, задержки в миллисекундах
                        # и отметки времени
24     ./biolatency -Q    # включает в задержки ввода/вывода время в очереди ОС
25     ./biolatency -D    # показывает каждое дисковое устройство отдельно
26     """
27     parser = argparse.ArgumentParser(
28         description="Выводит задержки блочного ввода/вывода в виде гистограммы",
29         formatter_class=argparse.RawDescriptionHelpFormatter,
30         epilog=examples)
31     parser.add_argument("-T", "--timestamp", action="store_true",
32         help="включить в вывод отметки времени")
33     parser.add_argument("-Q", "--queued", action="store_true",
34         help="включить в задержки ввода/вывода время в очереди ОС")
35     parser.add_argument("-m", "--milliseconds", action="store_true",
36         help="выводить время в гистограмме в миллисекундах")
37     parser.add_argument("-D", "--disks", action="store_true",
38         help="выводить для каждого дискового устройства отдельную гистограмму")
39     parser.add_argument("interval", nargs="?", default=999999999,
40         help="выводить через равные интервалы, в секундах")
41     parser.add_argument("count", nargs="?", default=999999999,
42         help="количество циклов вывода")
43     args = parser.parse_args()
44     countdown = int(args.count)
45     debug = 0
46

```

Строки с 19-й по 44-ю обрабатывают аргументы командной строки. Здесь я использую модуль `argparse` из Python.

Я собирался сделать этот инструмент Unix-подобным, похожим на `vmstat`(8) или `iostat`(1), чтобы другие могли изучать его, поэтому выбрал такой стиль параметров и аргументов. Также я хотел, чтобы инструмент делал что-то одно, но хорошо (в данном случае он сообщает распределение задержек дискового ввода/вывода в виде гистограммы). Я мог бы добавить режим вывода каждого события, но написал для этого отдельный инструмент `biosnoop.py`.

У вас могут быть другие причины писать программы BCC/eBPF, например, создавая агенты для мониторинга другого ПО, и в таких случаях можно не беспокоиться о пользовательском интерфейсе.

```

47 # определение программы BPF
48 bpf_text = """
49 #include <uapi/linux/ptrace.h>
50 #include <linux/blkdev.h>
51
52 typedef struct disk_key {
53     char disk[DISK_NAME_LEN];
54     u64 slot;
55 } disk_key_t;
56 BPF_HASH(start, struct request *);
57 STORAGE
58
59 // время начала операции блочного ввода/вывода

```



```

60 int trace_req_start(struct pt_regs *ctx, struct request *req)
61 {
62     u64 ts = bpf_ktime_get_ns();
63     start.update(&req, &ts);
64     return 0;
65 }
66
67 // вывод
68 int trace_req_completion(struct pt_regs *ctx, struct request *req)
69 {
70     u64 *tsp, delta;
71
72     // получить текущее время и вычислить разность
73     tsp = start.lookup(&req);
74     if (tsp == 0) {
75         return 0; // пропущенный запуск операции
76     }
77     delta = bpf_ktime_get_ns() - *tsp;
78     FACTOR
79
80     // сохранить в гистограмме
81     STORE
82
83     start.delete(&req);
84     return 0;
85 }
86 """

```

Объявление программы BPF на C встраивается в программу на Python в виде строки и присваивается переменной `bpf_text`.

Строка 56 объявляет хеш-массив с именем «start», который использует указатель на структуру `request` в качестве ключа. Функция `trace_req_start()` извлекает текущее время вызовом `bpf_ktime_get_ns()` и сохраняет его в хеше с ключом `*req`. (Я просто использую адрес указателя как UUID.) Функция `trace_req_completion()` выполняет поиск `*req` в хеше, чтобы получить время начала операции, которое затем используется в строке 77 для вычисления разности. Строка 83 удаляет отметку времени из хеша.

Первым аргументом в прототипах этих функций объявляется структура `pt_regs*`, хранящая состояние регистров, затем следуют все остальные аргументы трассируемой функции, которые вам нужны. Я включил во все прототипы первые аргументы `request*` трассируемых функций.

Эта программа также объявляет хранилище для выходных данных, но есть проблема: инструмент `biolatency` поддерживает параметр `-D` для вывода отдельных гистограмм для каждого диска, и это меняет код хранилища. Итак, эта программа BPF содержит инструкции `STORAGE` и `STORE` (и еще `FACTOR`). Это самые обычные текстовые строки, которые я буду искать и заменять кодом в зависимости от параметров. По возможности лучше избегать кода, который вставляет другой код в программу, поскольку это затрудняет отладку.

```

87
88 # подстановка кода
89 if args.milliseconds:
90     bpf_text = bpf_text.replace('FACTOR', 'delta /= 1000000;')
91     label = "msecs"
92 else:
93     bpf_text = bpf_text.replace('FACTOR', 'delta /= 1000;')
94     label = "usecs"
95 if args.disks:
96     bpf_text = bpf_text.replace('STORAGE',
97     'BPF_HISTOGRAM(dist, disk_key_t);')
98     bpf_text = bpf_text.replace('STORE',
99     'disk_key_t key = {.slot = bpf_log2l(delta)}; ' +
100     'bpf_probe_read(&key.disk, sizeof(key.disk), ' +
101     'req->rq_disk->disk_name); dist.increment(key);')
102 else:
103     bpf_text = bpf_text.replace('STORAGE', 'BPF_HISTOGRAM(dist);')
104     bpf_text = bpf_text.replace('STORE',
105     'dist.increment(bpf_log2l(delta));')

```

Код, соответствующий инструкции FACTOR, просто изменяет единицы времени в зависимости от параметра -m.

Строка 95 проверяет, был ли запрошен вывод отдельных гистограмм для каждого диска (-D). Если да, то инструкции STORAGE и STORE заменяются кодом создания гистограмм для каждого диска. Он использует структуру `disk_key`, объявленную в строке 52 и содержащую имя диска и слот (интервал) в гистограмме. Строка 99 преобразует разницу во времени в индекс слота с помощью вспомогательной функции `bpf_log2l()`. Строки 100 и 101 определяют имя диска с помощью `bpf_probe_read()`, именно так все данные копируются в стек BPF для дальнейшей обработки. Строка 101 выполняет несколько разыменований: `req->rq_disk`, `rq_disk->disk_name`: компилятор BCC прозрачно превратит их в вызовы `bpf_probe_read()`.

Строки 103–105 обрабатывают случай с одной общей гистограммой (для всех дисков). Здесь с помощью макроса `BPF_HISTOGRAM` объявляется гистограмма с именем «dist». С помощью вспомогательной функции `bpf_log2l()` определяется слот (интервал), после чего значение в нем увеличивается на единицу.

Этот пример сложноват, он одновременно хорош (реалистичен) и плох (пугает). Более простые примеры ищите в руководстве, упоминавшемся выше.

```

106 if debug:
107     print(bpf_text)

```

Поскольку в программе есть код, генерирующий другой код, нужен какой-то способ отладки программы. Если установлен флаг `debug`, выводится текст получившейся программы.

```

108
109 # загрузка программы BPF
110 b = BPF(text=bpf_text)
111 if args.queued:

```

```

112     b.attach_kprobe(event="blk_account_io_start", fn_name="trace_req_start")
113 else:
114     b.attach_kprobe(event="blk_start_request", fn_name="trace_req_start")
115     b.attach_kprobe(event="blk_mq_start_request", fn_name="trace_req_start")
116 b.attach_kprobe(event="blk_account_io_completion",
117                 fn_name="trace_req_completion")
118

```

Строка 110 загружает программу BPF.

Поскольку эта программа была написана до того, как в BPF появилась поддержка точек трассировки, я использовал в ней kprobes (зонды динамической трассировки ядра). Ее следовало бы переписать и использовать точки трассировки, потому что они являются стабильным API. Правда, для обновленной программы потребуется более поздняя версия ядра (Linux 4.7+).

biolatency.py поддерживает параметр `-Q` для включения в задержки ввода/вывода времени, когда запрос находился в очереди ОС. В этом фрагменте можно видеть, как реализована поддержка этого параметра. Если программа запущена с параметром `-Q`, то строка 112 подключит BPF-функцию `trace_req_start()` к функции ядра `blk_account_io_start()`, которая помещает запрос в очередь в ядре. Если программа запущена без параметра `-Q`, то строки 114 и 115 подключают BPF-функцию к двум другим функциям ядра, обрабатывающим дисковый ввод/вывод (это может быть любая из этих функций). Это возможно только потому, что обе эти функции имеют одинаковый первый аргумент: `struct request*`. Если бы их аргументы отличались, пришлось бы писать отдельные функции BPF для каждой.

```

119 print("Tracing block device I/O... Hit Ctrl-C to end.")
120
121 # вывод
122 exiting = 0 if args.interval else 1
123 dist = b.get_table("dist")

```

Строка 123 извлекает гистограмму «dist», объявленную и заполненную кодом `STORAGE/STORE`.

```

124 while (1):
125     try:
126         sleep(int(args.interval))
127     except KeyboardInterrupt:
128         exiting = 1
129
130     print()
131     if args.timestamp:
132         print("%-8s\n" % strftime("%H:%M:%S"), end="")
133
134     dist.print_log2_hist(label, "disk")
135     dist.clear()
136
137     countdown -= 1
138     if exiting or countdown == 0:
139         exit()

```

Здесь находится логика вывода через равные интервалы заданное число раз (переменная `countdown`). Строки 131 и 132 выводят отметку времени, если программа запущена с параметром `-T`.

Строка 134 выводит гистограмму или гистограммы для каждого диска. Первый аргумент — это переменная `label` с текстом «usecs» или «msecs», который выводится над столбцом значений. Второй аргумент определяет вторичный ключ, если в `dist` хранятся гистограммы для каждого диска. Попробуйте понять сами, как `print_log2_hist()` определяет наличие вторичного ключа — это будет ваше домашнее задание по изучению кода внутренних компонентов ВСС и ВРЕ.

Строка 135 очищает гистограмму, готовясь к следующему интервалу.

Вот пример вывода программы, запущенной с параметром `-D` для вывода отдельных гистограмм для каждого диска:

```
# biolateness -D
Tracing block device I/O... Hit Ctrl-C to end.
^C
disk = 'xvdb'
    usecs          : count    distribution
      0 -> 1        : 0        |
      2 -> 3        : 0        |
      4 -> 7        : 0        |
      8 -> 15       : 0        |
     16 -> 31       : 0        |
     32 -> 63       : 0        |
     64 -> 127      : 18       |
    128 -> 255      : 167      |
    256 -> 511      : 90       |
disk = 'xvdc'
    usecs : count distribution
      0 -> 1        : 0        |
      2 -> 3        : 0        |
      4 -> 7        : 0        |
      8 -> 15       : 0        |
     16 -> 31       : 0        |
     32 -> 63       : 0        |
     64 -> 127      : 22       |
    128 -> 255      : 179      |
    256 -> 511      : 88       |
[...]
```

ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

В разделе «Ресурсы» в начале этого приложения вы найдете больше информации о разработке инструментов для ВСС. В главе 4 описан ВСС в целом.

Приложение D

С BPF

Здесь приведены примеры инструментов BPF, реализованных на С, представляющие собой скомпилированные программы на С или программы, выполняемые с помощью утилиты `perf(1)`. Это приложение — дополнительный материал для тех читателей, кто хочет более глубоко изучить особенности работы механизма BPF и его интерфейсов, которые поддерживаются ядром Linux.

В главе 5 рассказывается, как писать инструменты на `bpftool`, языке высокого уровня, которого, как ожидается, будет достаточно во многих случаях. В приложении С рассматривается интерфейс BCC — еще один предпочтительный вариант. Это приложение — продолжение разделов о BPF в главе 2.

Оно начинается с обсуждения вопросов программирования на С и пяти советов. Первая программа из представленных здесь — `hello_world.c` — демонстрирует приемы программирования на уровне инструкций BPF, за ней следуют еще два инструмента на С — `bigreads` и `btehist`, показывающие вывод каждого события и гистограммы соответственно. Последний инструмент — версия `bigreads` для `perf(1)`, служащая примером программирования на С через `perf(1)`.

ПОЧЕМУ НА С?

В 2014 году у нас был только язык С. Затем появился проект BCC, предоставивший улучшенный язык C¹ для создания BPF-программ, которые выполняются в пространстве ядра, и другие языки для создания интерфейсов с пользователем. А совсем недавно появился проект `bpftool`, позволяющий писать программы целиком на языке высокого уровня.

Вот несколько аргументов и контраргументов, почему стоит продолжать писать инструменты трассировки полностью на С:

- **Низкий оверхед на запуск:** в моей системе на запуск `bpftool` уходит около 40 мс процессорного времени, а на запуск BCC — около 160 мс. Оверхед можно

¹ BCC включает механизм на основе Clang для замены операций разыменования памяти, например, он автоматически заменяет инструкции вида `a->b->c` вызовами `bpf_probe_read()`. В программах на С это придется делать явно.

устранить, если программа будет иметь вид автономного файла с двоичным кодом на С. Его также можно уменьшить, скомпилировав объектный файл BPF один раз и повторно передавая его в ядро, если требуется: Cilium и Cloudflare имеют системы оркестрации, которые действуют именно так, используя шаблонные объектные файлы BPF, в которых при необходимости можно изменить некоторые данные (IP-адрес и т. д.). Подумайте и решите, насколько это важно для вас: как часто вы будете запускать программы BPF? Если часто, то, возможно, есть смысл оставить их включенными («закрепленными»)? Я также полагаю, что нам удастся настроить ВСС и уменьшить оверхед на запуск bpftrace¹. Кроме того, следующий пункт, возможно, поможет еще больше сократить время запуска.

- **Нет громоздких зависимостей компилятора:** для компиляции своих программ ВСС и bpftrace я использую LLVM и Clang, которые могут занимать более 80 Мбайт в файловой системе. В некоторых средах, в том числе во встраиваемых системах, это бывает недопустимо. Двоичному файлу на С, содержащему скомпилированную BPF-программу, эти зависимости не нужны. Еще одна проблема с LLVM и Clang заключается в том, что часто выходят новые версии с изменениями в API (в процессе разработки bpftrace мы использовали LLVM версий 5.0, 6.0, 7 и 8), что создает дополнительные сложности в поддержке. Но уже сейчас есть проекты, находящиеся на разных стадиях разработки и предназначенные для изменения компиляции. Цель некоторых — создание легковесного и самодостаточного компилятора BPF, способного заменить LLVM и Clang, пусть и за счет потери возможностей оптимизации LLVM. Трассировщик SystemTap с его компонентом BPF и трассировщик ptv(1) [5] уже используют его. Другие просто выполняют предварительную компиляцию BPF-программ из кода для ВСС/bpftrace и посылают двоичные файлы в целевые системы. Эти проекты также должны уменьшить оверхед на запуск.
- **Меньший оверхед во время выполнения:** на первый взгляд это утверждение кажется бессмысленным, потому что любой интерфейсный модуль в итоге будет запускать один и тот же байт-код BPF в ядре и платить ту же цену за использование kprobe и uprobe. Есть множество инструментов для ВСС и bpftrace, которые получают из ядра уже обобщенные данные и во время работы не потребляют процессорное время в пространстве пользователя. Переписывание таких инструментов на С ничего не даст. Один из случаев, когда быстроедействие внешнего интерфейса важно, — если ему приходится читать и обрабатывать тысячи событий в секунду (то есть настолько много, что потребление процессора внешним интерфейсом в пространстве пользователя можно заметить в таких инструментах, как top(1)). В этом случае реализация интерфейса на С обеспечит более высокую эффективность. Увеличить эффективность также можно настройкой кода опроса кольцевого буфера в ВСС², после чего разница между кодом на С и Python будет незначительная. Еще одна возможная оптимизация, которая пока не используется в ВСС и bpftrace, заключается в создании потоков-потребителей,

¹ См. <https://github.com/iovisor/bcc/issues/2367>.

² См. <https://github.com/iovisor/bcc/issues/1033>.

каждый из которых привязан к своему процессору и читает кольцевой буфер этого процессора.

- **Более полное использование возможностей BPF:** если нужно реализовать нечто, что выходит за рамки возможностей BCC и bpftrace, то на C можно написать любой код, который принимает верификатор BPF. Обратите внимание, что BCC уже принимает произвольный код на C, поэтому сложно представить, когда такая возможность понадобится.
- **Для использования с perf(1):** perf(1) поддерживает BPF-программы для расширения своих подкоманд `record` и `trace`. perf(1) имеет множество других применений в дополнение к другим инструментам BPF: например, если понадобится инструмент для эффективной записи потока событий в двоичный файл, то perf(1) уже готов предложить оптимизированное решение для этого случая. См. раздел «perf C» в этом приложении.

Обратите внимание, что многие проекты сетевых инструментов для BPF используют язык C, включая Cilium [182]. Ожидается, что для трассировки почти всегда будет достаточно возможностей bpftrace и BCC.

ПЯТЬ СОВЕТОВ

Вот что нужно знать, прежде чем начинать разработку инструментов на C:

1. BPF C — ограниченный язык: он не допускает потенциально бесконечных циклов и не позволяет вызывать функции ядра. Можно использовать только вспомогательные функции ядра `bpf_*`, хвостовые вызовы BPF, вызовы функций BPF из BPF и некоторые встроенные функции компилятора.
2. Чтение памяти производится только с помощью функции `bpf_probe_read()`, которая выполняет необходимые проверки. Обычно место назначения для прочитанных данных — это память стека, но для больших объектов можно использовать хранилище карт BPF.
3. Есть три способа вывести данные из ядра в пространство пользователя:
 - **bpf_perf_event_output() (BPF_FUNC_perf_event_output):** это предпочтительный способ отправки информации о каждом событии в пространство пользователя через определяемую вами структуру.
 - **BPF_MAP_TYPE.* и вспомогательные карты (например, bpf_map_update_elem()):** карта — это хеш, хранящий пары ключ — значение, на основе которого можно строить более сложные структуры данных. Карты можно использовать для хранения обобщенных статистик или гистограмм и периодически читать (весьма эффективно) из пространства пользователя.
 - **bpf_trace_printk():** только для отладки. Эта функция записывает данные в `trace_pipe` и может конфликтовать с другими программами и трассировщиками.

4. По возможности используйте статические инструменты (точки трассировки, USDT) вместо динамических (kprobes, uprobes), так как средства статической инструментации предлагают более стабильный интерфейс.
5. Если застряли, перепишите инструмент с использованием BCC или bpftrace. Исследовав отладочные или выходные данные, вы сможете заметить шаги, которые пропустили. Например, в режиме `DEBUG_PREPROCESSOR BCC` выводит код C после препроцессора.

Некоторые инструменты используют макрос, обертывающий вызов `bpf_probe_read()`:

```
#define _(P) ({typeof(P) val; bpf_probe_read(&val, sizeof(val), &P); val;})
```

Этот макрос развернет инструкцию «`_(skb->dev)`» в соответствующий вызов `bpf_probe_read()`, возвращающий значение указанного члена.

ПРОГРАММЫ НА C

Когда в BPF добавляется новая возможность, в патч часто добавляется пример программы на C и/или тест для самопроверки ядра, показывающий использование этой возможности. Программы на C хранятся в исходном коде Linux в папке `samples/bpf`, а тесты — в `tools/testing/selftests/bpf`¹. Эти примеры и тесты показывают два способа определения BPF-программ на C [Zannoni 16]:

- **В виде инструкций BPF:** массив, встроенный в программу на C, который передается системному вызову `bpf(2)`.
- **В виде программы на C:** которая может быть скомпилирована в BPF и передана системному вызову `bpf(2)`. Этот метод предпочтительнее.

Компиляторы обычно поддерживают кросс-компиляцию для разных архитектур. Компилятор LLVM имеет цель BPF², благодаря которой программы на C могут компилироваться в BPF в файлы ELF или x86/ELF. Инструкции BPF могут храниться в разделе ELF с именем, совпадающим с названием типа программы BPF («`socket`», «`kprobe/...`» и т. д.). Некоторые загрузчики объектного кода могут анализировать этот тип для использования с системным вызовом `bpf(2)`³. Другие загрузчики (включая представленные в этом приложении) используют тип как метку.

¹ Эти примеры и тесты написаны многими членами сообщества BPF. Разработчики, отправившие более двадцати коммитов: Алексей Старовойтов, Даниэль Боркманн, Йонгхонг Сонг, Станислав Фомичев, Мартин Ка Фай Лау, Джон Фастабенд, Джеспер Дангаард Броуэр, Якуб Кичински и Андрей Игнатов. Сейчас ведется большая работа над тестами для самопроверки, и для того, чтобы все в BPF работало по мере его развития, новым разработчикам предлагается добавлять тесты для самопроверки вместо примеров программ.

² Для gcc тоже была разработана цель BPF, но пока ее не добавили в главную ветвь.

³ Включая `samples/bpf/bpf_load.*`, хотя эта библиотека уже считается устаревшей.

Обратите внимание, что возможны другие способы сборки BPF-программ: например, запись программы BPF в формате промежуточного представления LLVM, который компилятор LLVM сможет преобразовать в байт-код BPF.

Ниже описаны изменения в API, порядок компиляции и примеры инструментов для каждого типа, описанного ранее: пример с набором инструкций: `hello_world.c` и примеры программ на C: `bigread_kern.c` и `bitehist_kern.c`.

ВНИМАНИЕ: изменения в API

С декабря 2018 по август 2019 года это приложение дважды переписывалось из-за изменений в API библиотеки BPF для C. Старайтесь регулярно следить за обновлениями библиотек: `libbpf`, которая находится в дереве исходного кода Linux (`tools/lib/bpf`), и `libbss` из проекта `iowisior BCC` [183], так как возможны новые изменения.

Старый API, реализованный в ветке Linux 4.x, — это самая обычная библиотека с обычными функциями, исходный код которых находится в файлах `bpf_load.c` и `bpf_load.h` в `samples/bpf`. Он объявлен устаревшим, и вместо него используется библиотека `libbpf` в ядре. В какой-то момент старый API `bpf_load` будет удален. Большинство примеров трассировки сетевой подсистемы уже преобразовано для использования библиотеки `libbpf`, которая разрабатывается синхронно с разработкой функций ядра и используется внешними проектами (`BCC`, `bpftrace`). Рекомендую использовать `libbpf` и `libbss` вместо `bpf_load` или создания своей библиотеки, так как они неизбежно будут отставать от развития ядра, в отличие от `libbpf` и `libbss`, и препятствовать внедрению BPF.

Инструменты трассировки, представленные в этом приложении, используют `libbpf` и `libbss`. Спасибо Андрию Накрийко за то, что переписал их с использованием последней версии API, которая должна быть в Linux 5.4, и за его работу над `libbpf`. Более ранние версии этих инструментов были написаны для Linux 4.15 и доступны в репозитории для этой книги (ссылку на него ищите на <http://www.brendangregg.com/bpf-performance-tools-book.html>).

Компиляция

Ниже приведены шаги, которые нужно выполнить на сервере Ubuntu 18.04 (Bionic) для поиска, компиляции и установки нового ядра, а также компиляции образцов `bpf`. (ВНИМАНИЕ: сначала попробуйте на тестовой системе, поскольку из-за отсутствия необходимых конфигурационных параметров для поддержки виртуальных сред система может перестать загружаться):

```
# apt-get update
# apt-get install bc libssl-dev llvm-9 clang libelf-dev
# ln -s llc-9 /usr/bin/llc
# cd /usr/src
# wget https://git.kernel.org/torvalds/t/linux-5.4.tar.gz
# cd linux-5.4
# make olddefconfig
```

```
# make -j $(getconf _NPROCESSORS_ONLN)
# make modules_install && make install && make headers_install
# reboot
[...]
# make samples/bpf/
```

Для поддержки BTF требуется версия llvm-9 или выше. Эти служат лишь примером: в зависимости от дистрибутива, версии ядра, LLVM, Clang и примеров BPF эти шаги, возможно, придется скорректировать.

Иногда у нас возникали проблемы с устаревшими пакетами LLVM, и приходилось собирать последнюю версию LLVM и Clang из исходного кода. Вот пример, как это сделать:

```
# apt-get install -y cmake gcc g++
# git clone --depth 1 http://llvm.org/git/llvm.git
# cd llvm/tools
# git clone --depth 1 http://llvm.org/git/clang.git
# cd ..; mkdir build; cd build
# cmake -DLLVM_TARGETS_TO_BUILD="X86;BPF" -DLLVM_BUILD_LLVM_DYLIB=ON \
  -DLLVM_ENABLE_RTTI=ON -DCMAKE_BUILD_TYPE=Release ..
# make -j $(getconf _NPROCESSORS_ONLN)
# make install
```

Обратите внимание, что на этих этапах список целей сборки включает только X86 и BPF.

Инструмент 1: Hello, World!

Для примера создания BPF-программ в виде последовательностей инструкций я переписал программу hello_world.py из приложения C на языке C (hello_world.c). Ее можно найти в каталоге samples/bpf/ и скомпилировать, как было описано выше, после добавления ее в файл samples/bpf/Makefile. Вот пример вывода:

```
# ./hello_world
svscan-1991 [007] .... 2582253.708941: 0: Hello, World!
cron-983 [008] .... 2582254.363956: 0: Hello, World!
svscan-1991 [007] .... 2582258.709153: 0: Hello, World!
[...]
```

Она выводит текст «Hello, World!» вместе с другими полями по умолчанию из буфера трассировки (имя и идентификатор процесса, идентификатор процессора, флаги и отметка времени).

Исходный код в файле hello_world.c:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <errno.h>
5 #include <unistd.h>
6 #include <linux/version.h>
7 #include <bpf/bpf.h>
```

```

8 #include <bcc/libbpf.h>
9
10 #define DEBUGFS "/sys/kernel/debug/tracing/"
11
12 char bpf_log_buf[BPF_LOG_BUF_SIZE];
13
14 int main(int argc, char *argv[])
15 {
16     int prog_fd, probe_fd;
17
18     struct bpf_insn prog[] = {
19         BPF_MOV64_IMM(BPF_REG_1, 0xa21), /* '!\\n' */
20         BPF_STX_MEM(BPF_H, BPF_REG_10, BPF_REG_1, -4),
21         BPF_MOV64_IMM(BPF_REG_1, 0x646c726f), /* 'orld' */
22         BPF_STX_MEM(BPF_W, BPF_REG_10, BPF_REG_1, -8),
23         BPF_MOV64_IMM(BPF_REG_1, 0x57202c6f), /* 'o, w' */
24         BPF_STX_MEM(BPF_W, BPF_REG_10, BPF_REG_1, -12),
25         BPF_MOV64_IMM(BPF_REG_1, 0x6c6c6548), /* 'Hell' */
26         BPF_STX_MEM(BPF_W, BPF_REG_10, BPF_REG_1, -16),
27         BPF_MOV64_IMM(BPF_REG_1, 0),
28         BPF_STX_MEM(BPF_B, BPF_REG_10, BPF_REG_1, -2),
29         BPF_MOV64_REG(BPF_REG_1, BPF_REG_10),
30         BPF_ALU64_IMM(BPF_ADD, BPF_REG_1, -16),
31         BPF_MOV64_IMM(BPF_REG_2, 15),
32         BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0,
33                     BPF_FUNC_trace_printk),
34         BPF_MOV64_IMM(BPF_REG_0, 0),
35         BPF_EXIT_INSN(),
36     };
37     size_t insns_cnt = sizeof(prog) / sizeof(struct bpf_insn);
38
39     prog_fd = bpf_load_program(BPF_PROG_TYPE_KPROBE, prog, insns_cnt,
40                               "GPL", LINUX_VERSION_CODE,
41                               bpf_log_buf, BPF_LOG_BUF_SIZE);
42     if (prog_fd < 0) {
43         printf("ERROR: failed to load prog '%s'\\n", strerror(errno));
44         return 1;
45     }
46
47     probe_fd = bpf_attach_kprobe(prog_fd, BPF_PROBE_ENTRY, "hello_world",
48                                "do_nanosleep", 0, 0);
49     if (probe_fd < 0)
50         return 2;
51
52     system("cat " DEBUGFS "/trace_pipe");
53
54     close(probe_fd);
55     bpf_detach_kprobe("hello_world");
56     close(prog_fd);
57     return 0;
58 }

```

Сами инструкции BPF программы «Hello, World!» находятся в строках с 19-й по 35-ю. Остальная часть программы, ради уменьшения размера примера, использует более старый API на основе дескрипторов файлов и получает выходные данные из

конвейера трассировки. Новые API и методы вывода будут показаны в последующих примерах bigreads и bitehist в этом приложении, и как вы увидите, они делают программу намного длиннее.

BPF-программа объявляется как массив prog, включающий последовательность макросов инструкций BPF. Обзор этих макросов и инструкций BPF ищите в приложении E. Эта программа также использует функции из libbpf и libbss для загрузки программы и подключения к kprobe.

Строки с 19-й по 26-ю сохраняют текст «Hello, World!\n» в стеке BPF. Для большей эффективности эта строка сохраняется группами по четыре символа в виде 32-битных целых чисел (BPF_W — word, или слово). Последние два байта сохраняются как 16-битное целое число (BPF_H — half-word, или полуслово).

Строки 27–33 подготавливают и вызывают функцию BPF_FUNC_trace_printk, которая записывает строку в разделяемый буфер трассировки.

Строки с 39-й по 41-ю вызывают функцию bpf_load_program() из libbpf (библиотека в исходном коде Linux в tools/lib/bpf). Она загружает программу BPF, устанавливает ее тип как kprobe и возвращает дескриптор файла для программы.

Строки с 47-й по 48-ю вызывают функцию bpf_attach_kprobe() из libbss (библиотека из репозитория iovisor BCC; она определена в src/cc/libbpf.h), которая подключает программу к kprobe на входе в функцию ядра do_nanosleep(). Используется имя события «hello_world», которое пригодится для отладки (оно находится в /sys/kernel/debug/tracing/kprobe_events). bpf_attach_kprobe() возвращает дескриптор файла для зонда. Эта библиотечная функция также выводит сообщение об ошибке, поэтому в случае ошибки я ничего не вывожу в строке 49.

Строка 52 использует system() для запуска команды cat(1), которая выведет сообщение из разделяемого конвейера трассировки¹.

Строки с 54-й по 56-ю закрывают дескриптор файла зонда, отключаются от kprobe и закрывают дескриптор файла программы. Если забыть сделать эти вызовы, более ранние версии ядра Linux продолжат выполняться с настроенными и включенными зондами, что увеличит оверхед. Проверить правильное завершение программы можно с помощью cat /sys/kernel/debug/tracing/kprobe_events или bpftool(8) prog show, а отключить зонды — с помощью reset-trace(8) из BCC (отключает все трассировщики). В Linux 5.2 был произведен переход на зонды, основанные на файловых дескрипторах, которые автоматически закрываются с завершением процесса.

Чтобы сделать этот пример как можно короче, я применил BPF_FUNC_trace_printk и system(). Они используют разделяемый буфер трассировки (/sys/kernel/debug/tracing/trace_pipe) и могут конфликтовать с другими программами трассировки или отладки, потому что ядро не обеспечивает никакой защиты. Рекомендваемый интерфейс — через BPF_FUNC_perf_event_output. Почему это так, рассказано в разделе «Инструмент 2: bigreads» ниже.

¹ Конвейер трассировки также можно прочитать программой tracelog из репозитория bpftool.

Чтобы скомпилировать эту программу, в Makefile была добавлена ссылка на файл `hello_world`. Ниже показан вывод утилиты `diff`, где видны добавленные строки в Linux 5.3, они выделены жирным:

```
# diff -u Makefile.orig Makefile
--- ../orig/Makefile 2019-08-03 19:50:23.671498701 +0000
+++ Makefile 2019-08-03 21:23:04.440589362 +0000
@@ -10,6 +10,7 @@
 hostprogs-y += sockex1
 hostprogs-y += sockex2
 hostprogs-y += sockex3
+hostprogs-y += hello_world
 hostprogs-y += tracex1
 hostprogs-y += tracex2
 hostprogs-y += tracex3
@@ -64,6 +65,7 @@
 sockex1-objs := sockex1_user.o
 sockex2-objs := sockex2_user.o
 sockex3-objs := bpf_load.o sockex3_user.o
+hello_world-objs := hello_world.o
 tracex1-objs := bpf_load.o tracex1_user.o
 tracex2-objs := bpf_load.o tracex2_user.o
 tracex3-objs := bpf_load.o tracex3_user.o
@@ -180,6 +182,7 @@
 HOSTCFLAGS_bpf_load.o += -I$(objtree)/usr/include -Wno-unused-variable
 KBUILD_HOSTLDLIBS += $(LIBBPF) -lelf
+HOSTLDLIBS_hello_world += -lbcc
 HOSTLDLIBS_tracex4 += -lrt
 HOSTLDLIBS_trace_output += -lrt
 HOSTLDLIBS_map_perf_test += -lrt
```

Эту программу можно скомпилировать и выполнить, как описано в разделе «Компиляция» далее в этом приложении.

Как показывает этот инструмент, BPF-программы можно писать на уровне инструкций, но я не рекомендую так поступать. Поэтому следующие два инструмента написаны на С и используют функции BPF.

Инструмент 2: bigreads

`bigreads` трассирует выход из `vfs_read()` и выводит сообщения о попытках прочитать блоки размером больше 1 Мбайт. На этот раз BPF-программа написана полностью на С¹. Программа `bigreads` эквивалентна этому однострочнику для `bpfftrace`:

```
# bpfftrace -e 'kr:vfs_read /retval > 1024 * 1024/ {
    printf("READ: %d bytes\n", retval); }'
```

¹ Первую версию я написал 6 июня 2014 года, когда С был самым высокоуровневым языком из доступных. 1 августа 2019 года Андрей Накрийко переписал эти инструменты BPF на С и использовал новейшие интерфейсы BPF.

Пример вывода программы bigreads на C:

```
# ./bigreads
dd-5145 [003] d... 2588681.534759: 0: READ: 2097152 bytes
dd-5145 [003] d... 2588681.534942: 0: READ: 2097152 bytes
dd-5145 [003] d... 2588681.535085: 0: READ: 2097152 bytes
[...]
```

Здесь видно, что для эксперимента использовалась команда `dd(1)` и с ее помощью выполнили три операции чтения блоками размером 2 Мбайт. Как и в программе `hello_world.c`, здесь в выходные данные из разделяемого буфера трассировки добавлены дополнительные поля.

Программа `bigreads` разбита на два компонента в отдельных файлах на C — для пространства ядра и пространства пользователя. Это позволяет компилировать компонент для ядра отдельно с использованием BPF в качестве целевой архитектуры, а пользовательский компонент будет читать этот файл и передавать инструкции BPF ядру.

Компонент ядра в `bigreads_kern.c`:

```
1 #include <uapi/linux/bpf.h>
2 #include <uapi/linux/ptrace.h>
3 #include <linux/version.h>
4 #include "bpf_helpers.h"
5
6 #define MIN_BYTES (1024 * 1024)
7
8 SEC("kretprobe/vfs_read")
9 int bpf_myprog(struct pt_regs *ctx)
10 {
11     char fmt[] = "READ: %d bytes\n";
12     int bytes = PT_REGS_RC(ctx);
13     if (bytes >= MIN_BYTES) {
14         bpf_trace_printk(fmt, sizeof(fmt), bytes, 0, 0);
15     }
16
17     return 0;
18 }
19
20 char _license[] SEC("license") = "GPL";
21 u32 _version SEC("version") = LINUX_VERSION_CODE;
```

Строка 6 определяет пороговый размер операций в байтах.

В строке 8 объявляется раздел ELF с именем «`kretprobe/vfs_read`», за которым следует BPF-программа. Это имя будет доступно в конечном двоичном ELF-файле. Некоторые загрузчики пользовательского уровня используют эти заголовки секций для определения места вложения программ. Загрузчик `bitehist_user.c` (о котором мы поговорим чуть ниже) этого не делает, хотя этот заголовок раздела может быть полезен для отладки.

В строке 9 начинается функция, которая вызывается по событию kretprobe. Аргумент `pt_regs` содержит состояние регистров и контекст BPF. Из регистров можно прочитать аргументы функции и возвращаемые значения. Этот указатель на структуру — обязательный аргумент для вспомогательных функций BPF (см. `include/uapi/linux/bpf.h`).

Строка 11 объявляет строку формата для использования с `printf()`.

Строка 12 извлекает возвращаемое значение из регистра в структуре `pt_regs`, используя макрос (в архитектуре x86 он отображает `long bytes = PT_REGS_RC(ctx)` в `ctx->rax`).

Строка 13 выполняет проверку.

Строка 14 выводит сообщение с использованием функции `bpf_trace_printk()`. Она записывает сообщение в разделяемый буфер трассировки и используется здесь, только чтобы сократить этот пример. В отношении нее действуют те же предостережения из приложения C: она может конфликтовать с другими трассировщиками, выполняющимися одновременно.

В строках 20 и 21 объявляются другие нужные разделы и значения.

Компонент для пространства пользователя `bigreads_user.c`:

```

1 // SPDX-License-Identifier: GPL-2.0
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <string.h>
6 #include <errno.h>
7 #include <sys/resource.h>
8 #include "bpf/libbpf.h"
9
10 #define DEBUGFS "/sys/kernel/debug/tracing/"
11
12 int main(int ac, char *argv[])
13 {
14     struct bpf_object *obj;
15     struct bpf_program *prog;
16     struct bpf_link *link;
17     struct rlimit lim = {
18         .rlim_cur = RLIM_INFINITY,
19         .rlim_max = RLIM_INFINITY,
20     };
21     char filename[256];
22
23     snprintf(filename, sizeof(filename), "%s_kern.o", argv[0]);
24
25     setrlimit(RLIMIT_MEMLOCK, &lim);
26
27     obj = bpf_object__open(filename);
28     if (libbpf_get_error(obj)) {

```

```
29     printf("ERROR: failed to open prog: '%s'\n", strerror(errno));
30     return 1;
31 }
32
33 prog = bpf_object__find_program_by_title(obj, "kretprobe/vfs_read");
34 bpf_program__set_type(prog, BPF_PROG_TYPE_KPROBE);
35
36 if (bpf_object__load(obj)) {
37     printf("ERROR: failed to load prog: '%s'\n", strerror(errno));
38     return 1;
39 }
40
41 link = bpf_program__attach_kprobe(prog, true /*retprobe*/, "vfs_read");
42 if (libbpf_get_error(link))
43     return 2;
44
45 system("cat " DEBUGFS "/trace_pipe");
46
47 bpf_link__destroy(link);
48 bpf_object__close(obj);
49
50 return 0;
51 }
```

Строки с 17-й по 19-ю и строка 25 устанавливают ограничение `RLIMIT_MEMLOCK` равным бесконечности, чтобы не возникало проблем с распределением памяти в BPF.

Строка 27 создает экземпляр структуры `bpf_object` для ссылки на компоненты BPF в файле `_kern.o`. Объект `bpf_object` может содержать несколько программ и карт BPF.

Строка 28 проверяет успешность инициализации `bpf_object`.

Строка 33 создает экземпляр структуры `bpf_program`, используя BPF-программу, соответствующую заголовку раздела «`kretprobe/vfs_read`», установленному функцией `SEC()` в исходном коде ядра.

Строка 36 инициализирует и загружает объекты BPF из компонента ядра в ядро, включая все карты и программы.

Строка 41 подключает ранее выбранную программу к `kprobe` для `vfs_read()` и возвращает объект `bpf_link`. Он используется в строке 47 для отключения программы.

Строка 45 выводит разделяемый буфер трассировки с помощью `system()`. Это решение использовано для сокращения примера.

Строка 48 выгружает BPF-программы в объекте `bpf_object` из ядра и освобождает все зарезервированные ресурсы.

Эти файлы можно добавить в `samples/bpf` и скомпилировать, добавив цель `bigreads` в файл `samples/bpf/Makefile`. Для этого допишите следующие строки (поместите каждую из них в соответствующий раздел в `Makefile` среди похожих строк):


```
# grep bigreads Makefile
hostprogs-y += bigreads
bigreads-objs := bigreads_user.o
always += bigreads_kern.o
```

Компиляция и запуск выполняются так же, как в предыдущем примере `hello_world`. На этот раз будет создан отдельный файл `bigreads_kern.o` с BPF-программой в разделе, который ищет `bigreads_user.o`. Убедиться в его наличии можно с помощью `readelf(1)` или `objdump(1)`:

```
# objdump -h bigreads_kern.o
```

```
bigreads_kern.o:  file format elf64-little
```

```
Sections:
```

Idx	Name	Size	VMA	LMA	File off	Align
0	.text	00000000	0000000000000000	0000000000000000	00000040	2**2
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
1	kretprobe/vfs_read	000000a0	0000000000000000	0000000000000000	00000040	2**3
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
2	.rodata.str1.1	0000000f	0000000000000000	0000000000000000	000000e0	2**0
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
3	license	00000004	0000000000000000	0000000000000000	000000ef	2**0
			CONTENTS, ALLOC, LOAD, DATA			
4	version	00000004	0000000000000000	0000000000000000	000000f4	2**2
			CONTENTS, ALLOC, LOAD, DATA			
5	.llvm_addrsig	00000003	0000000000000000	0000000000000000	00000170	2**0
			CONTENTS, READONLY, EXCLUDE			

Раздел «`kretprobe/vfs_read`» выделен жирным.

Чтобы превратить эту программу в надежный инструмент, замените вызов `bpf_trace_printk()` вызовом функции `print_bpf_output()`, отправляющей сообщения в пространство пользователя через карту BPF, которая доступна в кольцевых буферах `perf` каждого процессора. Затем программа ядра подключит следующий код (здесь приведен новый прием на основе BTF)¹:

```
struct {
    __uint(type, BPF_MAP_TYPE_PERF_EVENT_ARRAY)
    __uint(key_size, sizeof(int));
    __uint(value_size, sizeof(u32));
} my_map SEC(".maps");
[...]
bpf_perf_event_output(ctx, &my_map, 0, &bytes, sizeof(bytes));
```

В программу, выполняющуюся в пространстве пользователя, нужно внести больше изменений: удалить вызов `system()` и добавить функцию для обработки событий

¹ В более ранних ядрах это делалось иначе и также включало установку `max_entries` в `__NR_CPUS`, чтобы на каждый процессор был буфер. Сейчас параметру `max_entries` присваивается значение по умолчанию `BPF_MAP_TYPE_PERF_EVENT_ARRAY`.

вывода карты. Затем функцию следует зарегистрировать с помощью `perf_event_poller()`. Соответствующий пример ищите в каталоге с исходным кодом `Linux samples/bpf`, в файле `trace_output_user.c`.

Инструмент 3: bitehist

`bitehist` — переделанная версия инструмента `bitehist.py` для ВСС из приложения С. Он показывает вывод через карты BPF, которые здесь используются для хранения гистограммы объемов данных, вовлеченных в операции блочного ввода/вывода. Вот пример вывода:

```
# ./bitehist
Tracing block I/O... Hit Ctrl-C to end.
^C
    kbytes          : count  distribution
      4 -> 7        : 11      |*****|
      8 -> 15       : 24      |*****|
     16 -> 31      : 12      |*****|
     32 -> 63      : 10      |*****|
     64 -> 127     : 5       |*****|
    128 -> 255     : 4       |*****|
Exiting and clearing kprobes...
```

Как и `bigreads`, инструмент `bitehist` состоит из двух файлов с кодом на С: `bitehist_kern.c` и `bitehist_user.c`. Полный исходный код можно найти на сайте этой книги <http://www.brendangregg.com/bpf-performance-tools-book.html>. Ниже приведены фрагменты кода.

Фрагмент из `bitehist_kern.c`:

```
[...]
struct hist_key {
    u32 index;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, struct hist_key);
    __type(value, long);
} hist_map SEC(".maps");
[...]
SEC("kprobe/blk_account_io_completion")
int bpf_prog1(struct pt_regs *ctx)
{
    long init_val = 1;
    long *value;
    struct hist_key key = {};

    key.index = log2l(PT_REGS_PARM2(ctx) / 1024);
    value = bpf_map_lookup_elem(&hist_map, &key);
```

```

    if (value)
        __sync_fetch_and_add(value, 1);
    else
        bpf_map_update_elem(&hist_map, &key, &init_val, BPF_ANY);
    return 0;
}
[...]
```

Этот код объявляет карту типа `BPF_MAP_TYPE_HASH` с именем `hist_map`: такой стиль объявления будет распространяться с расширением использования BTF. Ключ `key` — это структура `hist_key`, которая хранит только индекс интервала, а значение `value` — это счетчик для интервала с типом `long`.

Программа BPF читает размер блока из второго аргумента `blk_account_io_completion` с помощью макроса `PT_REGS_PARM2(ctx)` и преобразует его в индекс интервала в гистограмме с помощью `log2()` (не включена в вывод).

Указатель на значение для полученного индекса выбирается с помощью `bpf_map_lookup_elem()`. Если значение найдено, оно увеличивается вызовом `__sync_fetch_and_add()`. Если не найдено, то инициализируется с помощью `bpf_map_update_elem()`.

Фрагмент из `bitehist_user.c`:

```

struct bpf_object *obj;
struct bpf_link *kprobe_link;
struct bpf_map *map;

static void print_log2_hist(int fd, const char *type)
{
    [...]
    while (bpf_map_get_next_key(fd, &key, &next_key) == 0) {
        bpf_map_lookup_elem(fd, &next_key, &value);
        ind = next_key.index;

// логика вывода гистограммы
    [...]
}

static void int_exit(int sig)
{
    printf("\n");
    print_log2_hist(bpf_map__fd(map), "kbytes");
    bpf_link__destroy(kprobe_link);
    bpf_object__close(obj);
    exit(0);
}

int main(int argc, char *argv[])
{
    struct rlimit lim = {
        .rlim_cur = RLIM_INFINITY,
        .rlim_max = RLIM_INFINITY,
    };
    rlimit_set(&lim);
    obj = bpf_object__open(argv[1]);
    if (!obj) {
        perror("bpf_object__open");
        return 1;
    }
    map = bpf_map__lookup(obj, "hist_map");
    if (!map) {
        perror("bpf_map__lookup");
        return 1;
    }
    fd = bpf_map__fd(map);
    if (fd < 0) {
        perror("bpf_map__fd");
        return 1;
    }
    key = hist_key__new(0);
    if (!key) {
        perror("hist_key__new");
        return 1;
    }
    kprobe_link = bpf_link__new(obj, "bpf_prog", fd, key);
    if (!kprobe_link) {
        perror("bpf_link__new");
        return 1;
    }
    print_log2_hist(fd, "kbytes");
    int_exit(0);
}
```

```
};
struct bpf_program *prog;
char filename[256];

snprintf(filename, sizeof(filename), "%s_kern.o", argv[0]);

setrlimit(RLIMIT_MEMLOCK, &lim);

obj = bpf_object__open(filename);
if (libbpf_get_error(obj))
    return 1;

prog = bpf_object__find_program_by_title(obj,
    "kprobe/blk_account_io_completion");
if (prog == NULL)
    return 2;
bpf_program__set_type(prog, BPF_PROG_TYPE_KPROBE);

if (bpf_object__load(obj)) {
    printf("ERROR: failed to load prog: '%s'\n", strerror(errno));
    return 3;
}

kprobe_link = bpf_program__attach_kprobe(prog, false /*retprobe*/,
    "blk_account_io_completion");
if (libbpf_get_error(kprobe_link))
    return 4;

if ((map = bpf_object__find_map_by_name(obj, "hist_map")) == NULL)
    return 5;

signal(SIGINT, int_exit);

printf("Tracing block I/O... Hit Ctrl-C to end.\n");
sleep(-1);

return 0;
}
```

Функция `main()` загружает BPF-программу, выполняя ту же последовательность действий, что и `bigreads`.

Объект карты BPF извлекается с помощью `bpf_object__find_map_by_name()` и сохраняется в глобальной переменной, которая позже выводится в вызове `int_exit()`.

`int_exit()` — это обработчик сигнала `SIGINT` (Ctrl-C). После инициализации обработчика сигнала функция `main()` приостанавливает программу. При нажатии Ctrl-C вызывается `int_exit()`, который, в свою очередь, вызывает `print_log2_hist()`.

`print_log2_hist()` выполняет обход содержимого карты, используя цикл `bpf_get_next_key()`, и вызывает `bpf_lookup_elem()` для чтения каждого значения. Остальная часть функции, которой здесь нет, превращает ключи и значения в гистограмму.

Этот инструмент можно скомпилировать и запустить из каталога `samples/bpf`, добавив нужные ссылки на имена в `Makefile`, как и в случае с `bigreads`.

PERF C

Утилита `perf(1)` в Linux позволяет запускать BPF-программы по событиям¹, используя один из двух интерфейсов:

- **perf record:** предназначен для запуска по событиям программ, которые могут использовать свои фильтры и создавать дополнительные записи в файле `perf.data`;
- **perf trace:** предназначен для «улучшения» вывода и фильтрации событий `perf trace` с помощью BPF-программ (например, для вывода строкового имени файла при трассировке системных вызовов вместо простого указателя на имя [84]).

Поддержка BPF в `perf(1)` быстро расширяется, но сейчас нет документации по их совместному использованию. Лучший источник информации — поиск в архивах списков рассылки ядра Linux по ключевым словам «perf» и «BPF».

В следующем разделе показана совместная работа `perf` и BPF.

Инструмент 1: bigreads

`bigreads` основан на инструменте из раздела «Программы на C», который трассирует выход из `vfs_read()` и показывает операции чтения, где размер блока превышает 1 Мбайт. Следующий пример вывода поможет понять, как он работает:

```
# perf record -e bpf-output/no-inherit,name=evt/ \
  -e ./bigreads.c/map:channel.event=evt/ -a
^C[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.255 MB perf.data (3 samples) ]
# perf script
      dd 31049 [009] 2652091.826549: 0 evt:
ffffffffffb5945e20 kretprobe_trampoline+0x0
(/lib/modules/5.0.0-rc1-virtual/build/vmlinux)
      BPF output: 0000: 00 00 20 00 00 00 00 00 .. .....
                  0008: 00 00 00 00 ....

      dd 31049 [009] 2652091.826718: 0 evt:
ffffffffffb5945e20 kretprobe_trampoline+0x0
(/lib/modules/5.0.0-rc1-virtual/build/vmlinux)
      BPF output: 0000: 00 00 20 00 00 00 00 00 .. .....
                  0008: 00 00 00 00 ....

      dd 31049 [009] 2652091.826838: 0 evt:
```

¹ Впервые поддержку BPF в `perf(1)` добавил Ван Нань.

```
fffffffffb5945e20 kretprobe_trampoline+0x0
(/lib/modules/5.0.0-rc1-virtual/build/vmlinux)
BPF output: 0000: 00 00 20 00 00 00 00 00 .. .....
             0008: 00 00 00 00 ....
```

Файл perf.data содержит только записи, соответствующие операциям чтения блоками с размером более 1 Мбайт, за которыми следуют события вывода BPF, содержащие размеры блоков. Начав трассировку, я выполнил три операции чтения блоками по 2 Мбайт с помощью dd(1), которые можно увидеть в выводе BPF: «00 00 20» — это два мегабайта, 0x200000, в формате с обратным порядком следования байтов (x86).

Исходный код bigreads.c:

```
#include <uapi/linux/bpf.h>
#include <uapi/linux/ptrace.h>
#include <linux/types.h>

#define SEC(NAME) __attribute__((section(NAME), used))

struct bpf_map_def {
    unsigned int type;
    unsigned int key_size;
    unsigned int value_size;
    unsigned int max_entries;
};

static int (*perf_event_output)(void *, struct bpf_map_def *, int, void *,
    unsigned long) = (void *)BPF_FUNC_perf_event_output;

struct bpf_map_def SEC("maps") channel = {
    .type = BPF_MAP_TYPE_PERF_EVENT_ARRAY,
    .key_size = sizeof(int),
    .value_size = sizeof(__u32),
    .max_entries = __NR_CPUS,
};

#define MIN_BYTES (1024 * 1024)

SEC("func=vfs_read")
int bpf_myprog(struct pt_regs *ctx)
{
    long bytes = ctx->rdx;
    if (bytes >= MIN_BYTES) {
        perf_event_output(ctx, &channel, BPF_F_CURRENT_CPU,
            &bytes, sizeof(bytes));
    }
    return 0;
}

char _license[] SEC("license") = "GPL";
int _version SEC("version") = LINUX_VERSION_CODE;
```

Этот код выводит карту «channel» вызовом `perf_event_output()`, если размер прочитанного блока превысит `MIN_BYTES`: она становится событием вывода BPF в файле `perf.data`.

Возможности интерфейса `perf(1)` продолжают расширяться, и сейчас можно запускать BPF-программы с помощью простой команды «`perf record -e program.c`». Следите за новыми возможностями и примерами.

ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Больше о программировании для BPF на языке C ищите:

- в документации `Documentation/networking/filter.txt` в исходном коде Linux [17];
- в руководстве Cilium «BPF and XDP Reference Guide» [19].

Приложение Е

ИНСТРУКЦИИ BPF

Это приложение кратко описывает отдельные инструкции BPF, оно предназначено для помощи в чтении выводов инструкций из инструментов трассировки и исходного кода в программе `hello_world.c` в приложении D. Разработка BPF-программ с нуля с использованием инструкций не рекомендуется и здесь не рассматривается.

Здесь приведены не все инструкции BPF. Полный перечень инструкций ищите в следующих заголовочных файлах в исходном коде Linux, а также в справочнике в конце этого приложения:

- **инструкции классического BPF:** `include/uapi/linux/filter.h` и `include/uapi/linux/bpf_common.h`;
- **инструкции расширенного BPF:** `include/uapi/linux/bpf.h` и `include/uapi/linux/bpf_common.h`.

Файл `bpf_common.h` указан дважды, потому что содержит инструкции из обоих наборов.

ВСПОМОГАТЕЛЬНЫЕ МАКРОСЫ

В программе `hello_world.c` из приложения D использовались инструкции:

```
BPF_MOV64_IMM(BPF_REG_1, 0xa21), /* '!\\n' */
BPF_STX_MEM(BPF_H, BPF_REG_10, BPF_REG_1, -4),
BPF_MOV64_IMM(BPF_REG_1, 0x646c726f), /* 'orld' */
BPF_STX_MEM(BPF_W, BPF_REG_10, BPF_REG_1, -8),
[...]
BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0,
             BPF_FUNC_trace_printk),
BPF_MOV64_IMM(BPF_REG_0, 0),
BPF_EXIT_INSN(),
```

На самом деле это высокоуровневые вспомогательные макросы. Они перечислены в табл. Е.1.

Таблица Е.1. Некоторые вспомогательные макросы для определения инструкций BPF¹

Макросы инструкций BPF	Описание
BPF_ALU64_REG(OP, DST, SRC)	Арифметико-логическая операция OP с 64-битными регистрами DST и SRC
BPF_ALU32_REG(OP, DST, SRC)	Арифметико-логическая операция OP с 32-битными регистрами DST и SRC
BPF_ALU64_IMM(OP, DST, IMM)	Арифметико-логическая операция OP с регистром DST и 64-битным непосредственным значением IMM
BPF_ALU32_IMM(OP, DST, IMM)	Арифметико-логическая операция OP с регистром DST и 32-битным непосредственным значением IMM
BPF_MOV64_REG(DST, SRC)	Копирование 64-битного значения из регистра SRC в регистр DST
BPF_MOV32_REG(DST, SRC)	Копирование 32-битного значения из регистра SRC в регистр DST
BPF_MOV64_IMM(DST, IMM)	Запись 64-битного непосредственного значения IMM в регистр DST
BPF_MOV32_IMM(DST, IMM)	Запись 32-битного непосредственного значения IMM в регистр
BPF_LD_IMM64(DST, IMM)	Загрузка 64-битного непосредственного значения IMM в регистр DST
BPF_LD_MAP_FD(DST, MAP_FD)	Загрузка файлового дескриптора карты MAP_FD в регистр DST
BPF_LDX_MEM(SIZE, DST, SRC, OFF)	Загрузка в регистр DST значения из памяти, адресуемой регистром SRC
BPF_STX_MEM(SIZE, DST, SRC, OFF)	Сохранение значения из регистра SRC в памяти, адресуемой регистром DST
BPF_STX_XADD(SIZE, DST, SRC, OFF)	Атомарное сложение значения в памяти, адресуемой регистром DST, со значением в регистре SRC. Результат сохраняется в памяти, адресуемой регистром DST
BPF_ST_MEM(SIZE, DST, OFF, IMM)	Сохранение непосредственного значения IMM в памяти, адресуемой регистром DST
BPF_JMP_REG(OP, DST, SRC, OFF)	Условный переход по результатам сравнения регистров DST и SRC
BPF_JMP_IMM(OP, DST, IMM, OFF)	Условный переход по результатам сравнения значения в регистре DST и непосредственного значения IMM
BPF_JMP32_REG(OP, DST, SRC, OFF)	Условный переход по результатам 32-битного сравнения регистров DST и SRC

¹ BPF_LD_ABS() и BPF_LD_IND() были исключены, поскольку устарели. Они остались в исходном коде в основном по историческим причинам.

Таблица Е.1 (окончание)

Макросы инструкций BPF	Описание
BPF_JMP32_IMM(OP, DST, IMM, OFF)	Условный переход по результатам 32-битного сравнения значения в регистре DST и непосредственного значения IMM
BPF_JMP_A(OFF)	Безусловный переход
BPF_LD_MAP_VALUE(DST, MAP_FD, OFF)	Загрузка значения из карты MAP_FD в регистр DST
BPF_CALL_REL(IMM)	Вызов подпрограммы с относительным смещением IMM
BPF_EMIT_CALL(FUNC)	Вызов вспомогательной функции FUNC
BPF_RAW_INSN(CODE, DST, SRC, OFF, IMM)	Низкоуровневая инструкция BPF с кодом CODE
BPF_EXIT_INSN()	Выход

В этих макросах и аргументах используются сокращения, которые могут быть неочевидными. Вот их расшифровка в алфавитном порядке:

- **32:** 32 бита;
- **64:** 64 бита;
- **ALU:** арифметико-логическое устройство (Arithmetic Logic Unit);
- **DST:** место назначения (destination);
- **FUNC:** функция (function);
- **IMM:** непосредственное (immediate) значение, то есть константа;
- **INSN:** инструкция (instruction);
- **JMP:** переход (jump);
- **LD:** загрузка (load);
- **LDX:** загрузка в регистр из памяти, адресуемой другим регистром;
- **MAP_FD:** файловый дескриптор карты (map file descriptor);
- **MEM:** память (memory);
- **MOV:** копирование/перенос (move);
- **OFF:** смещение (offset);
- **OP:** операция (operation);
- **REG:** регистр (register);
- **REL:** относительный (relative);
- **ST:** сохранение (store; запись в память);
- **SRC:** источник (source);
- **STX:** сохранение значения из регистра в памяти, адресуемой другим регистром.

Эти макросы разворачиваются в инструкции BPF, в некоторых случаях с учетом указанной операции ОР.

ИНСТРУКЦИИ

В табл. Е.2 приведен список некоторых инструкций BPF. (Полный перечень ищите в заголовочных файлах, перечисленных в начале приложения.)

Таблица Е.2. Некоторые инструкции BPF, поля и регистры

Инструкция	Тип	Набор	Числовое представление	Описание
BPF_LD	Класс инструкций	Классический	0x00	Загрузка
BPF_LDX	Класс инструкций	Классический	0x01	Загрузка в X
BPF_ST	Класс инструкций	Классический	0x02	Сохранение
BPF_STX	Класс инструкций	Классический	0x03	Сохранение из X
BPF_ALU	Класс инструкций	Классический	0x04	Арифметико-логическое устройство
BPF_JMP	Класс инструкций	Классический	0x05	Переход
BPF_RET	Класс инструкций	Классический	0x06	Возврат из подпрограммы
BPF_ALU64	Класс инструкций	Расширенный	0x07	64-битное ALU
BPF_W	Размер	Классический	0x00	Слово, 32 бита
BPF_H	Размер	Классический	0x08	Половина слова, 16 бит
BPF_B	Размер	Классический	0x10	Байт, 8 бит
BPF_DW	Размер	Расширенный	0x18	Двойное слово, 64 бита
BPF_XADD	Модификатор сохранения	Расширенный	0xc0	Атомарное сложение
BPF_ADD	Арифметико-логические операции и переходы	Классический	0x00	Сложение
BPF_SUB	Арифметико-логические операции и переходы	Классический	0x10	Вычитание
BPF_K	Операнды арифметико-логических операций и переходов	Классический	0x00	Операнд — непосредственное значение
BPF_X	Операнды арифметико-логических операций и переходов	Классический	0x08	Операнд — регистр

Таблица Е.2 (окончание)

Инструкция	Тип	Набор	Числовое представление	Описание
BPF_MOV	Арифметико-логические операции и переходы	Расширенный	0xb0	Копирование из регистра в регистр
BPF_JLT	Переходы	Расширенный	0xa0	Условный переход «если меньше, чем» по результатам сравнения операндов как целых без знака
BPF_REG_0	Номера регистров	Расширенный	0x00	Регистр 0
BPF_REG_1	Номера регистров	Расширенный	0x01	Регистр 1
BPF_REG_10	Номера регистров	Расширенный	0x0a	Регистр 10

Инструкции обычно формируются как объединение класса инструкций и полей поразрядным OR.

КОДИРОВАНИЕ

В табл. Е.3 описан формат инструкций BPF из расширенного набора (структура `bpf_insn`).

Таблица Е.3. Формат инструкций BPF из расширенного набора

Код операции	Регистр-приемник	Регистр-источник	Смещение со знаком	Константа со знаком
8 бит	8 бит	8 бит	16 бит	32 бита

Первый макрос в программе `hello_world.c`:

```
BPF_MOV64_IMM(BPF_REG_1, 0xa21)
```

будет развернут в комбинацию, представляющую код операции:

```
BPF_ALU64 | BPF_MOV | BPF_K
```

Согласно табл. Е.3 и Е.2, этой комбинации соответствует код 0xb7. Аргументы инструкции: регистр-приемник `BPF_REG_1` (0x01) и константа (операнд) 0xa21. Получившиеся байты инструкции можно проверить с помощью `bpftool`(8):

```
# bpftool prog
[...]
907: kprobe tag 9abf0e9561523153 gpl
```

```

loaded_at 2019-01-08T23:22:00+0000 uid 0
xlated 128B jited 117B memlock 4096B
# bpftool prog dump xlated id 907 opcodes
0: (b7) r1 = 2593
    b7 01 00 00 21 0a 00 00
1: (6b) *(u16 *) (r10 -4) = r1
    6b 1a fc ff 00 00 00 00
2: (b7) r1 = 1684828783
    b7 01 00 00 6f 72 6c 64
3: (63) *(u32 *) (r10 -8) = r1
    63 1a f8 ff 00 00 00 00
[...]
```

В инструментах трассировки большая часть инструкций BPF будет выполнять загрузку данных из структур и вызывать вспомогательные функции BPF для сохранения значений в картах или вывода записей perf. См. подраздел «Вспомогательные функции BPF» в разделе 2.3.6.

ССЫЛКИ

Дополнительную информацию о программировании на уровне инструкций BPF ищите в заголовочных файлах в исходном коде Linux, перечисленных в начале этого приложения, а также в:

- Documentation/networking/filter.txt [17];
- include/uapi/linux/bpf.h [184];
- руководстве Cilium «BPF and XDP Reference Guide» [19].

ГЛОССАРИЙ

ALU	Arithmetic logic unit — арифметико-логическое устройство. Компонент процессора, обрабатывающий арифметические инструкции.
API	Application Programming Interface — прикладной программный интерфейс.
BCC	BPF Compiler Collection — коллекция компиляторов BPF. Фреймворк и набор инструментов с открытым исходным кодом для использования BPF, см. главу 4.
bpfttrace	Трассировщик с открытым исходным кодом на основе BPF и высокоуровневый язык программирования, см. главу 5.
BPF	Berkeley Packet Filter — фильтр пакетов Berkeley. Легковесная технология для ядра, созданная в 1992 году для ускорения фильтрации пакетов и расширенная в 2014 году до уровня универсальной среды выполнения (см. eBPF).
BTF	Формат представления метаданных BPF (BPF Type Format), см. главу 2.
C	Язык программирования C.
CPU	Central Processor Unit — центральный процессор. В этой книге аббревиатурой CPU обозначается виртуальный процессор, управляемый ОС, который может быть ядром или гиперпотокком.
CSV	Comma-Separated Values — значения, разделенные запятыми. Формат файлов.
DNS	Domain Name System — система доменных имен.
DTrace	Механизм динамической трассировки, разработанный в Sun Microsystems для Solaris 10 в 2005 году.
DTraceToolkit	Коллекция из 230 инструментов для DTrace, в основном написанных мной. Впервые была выпущена 20 апреля 2005 года как ПО с открытым исходным кодом. В DTraceToolkit впервые появились многие инструменты трассировки — exectnoop, iostnoop, iotop и т. д., которые потом были перенесены на разные языки и операционные системы.

eBPF	Extended BPF (см. <i>BPF</i>). Начиная с 2014 года аббревиатура eBPF описывала расширенный фильтр пакетов BPF, в котором изменились размеры регистров и набор инструкций и были добавлены хранилище карт и ограниченные возможности вызова функций ядра. К 2015 году от буквы «е» в названии отказались, и расширенный фильтр пакетов стал называться просто BPF.
ELF	Executable and Linkable Format — формат исполняемых и компонуемых файлов. Широко используемый формат исполняемых файлов программ.
Ftrace	Технология, встроенная в ядро Linux и обеспечивающая различные возможности трассировки. Сейчас отделена от eBPF. См. главу 14.
GUI	Graphical user interface — графический интерфейс пользователя.
HTTP	Hypertext Transfer Protocol — протокол передачи гипертекста.
ICMP	Internet Control Message Protocol — протокол управляющих сообщений интернета. Протокол, используемый командой ping(1) (ICMP echo запрос/ответ).
IOPS	I/O per second — операций ввода/вывода в секунду.
IO Visor	Проект Linux Foundation, который размещает репозитории BCC и bpftrace на Github и способствует сотрудничеству между разработчиками BPF в различных компаниях.
iovisor	См. <i>IO Visor</i> .
IP	Internet Protocol — протокол интернета. Протокол, имеющий две основные версии — IPv4 и IPv6. См. главу 10.
IPC	Instructions per cycle — инструкций за такт.
Java	Язык программирования Java.
JavaScript	Язык программирования JavaScript.
kprobes	Технология динамической инструментации ядра Linux.
kretprobe	Технология динамической инструментации точек выхода из функций ядра.
LBR	Last branch record — запись последней ветви. Особенность процессоров Intel, позволяющая собирать ограниченные трассировки стека. См. главу 2.
LRU	Least recently used — наименее давно использовавшиеся.
malloc	Memory allocate — выделение памяти. Под этим обычно подразумевается функция, распределяющая память.

MMU	Memory management unit — блок управления памятью. Аппаратный компонент, отвечающий за представление памяти центральному процессору и преобразование виртуальных адресов в физические.
MySQL	Открытое ПО системы управления реляционными базами данных.
ORC	Oops Rewind Capability — технология раскручивания трассировки стека, поддерживаемая ядром Linux.
PEBS	Precise event-based sampling — точная выборка на основе событий. Аппаратная технология для использования со счетчиками контроля производительности РМС, которая обеспечивает более точную запись состояния процессора на момент события.
perf(1)	Стандартный профилировщик и трассировщик для Linux, включенный в дерево исходного кода Linux. Первоначально perf(1) создавался как инструмент для анализа счетчиков РМС и позднее был расширен за счет добавления возможности трассировки.
perf_events	Инфраструктура в ядре Linux для поддержки команды perf(1), инструментации событий и записи информации о событиях в кольцевые буферы. Другие трассировщики, включая BPF, используют эту инфраструктуру для инструментации событий и буферизации информации о них.
PID	Process identifier — идентификатор процесса. Уникальный в пределах ОС числовой идентификатор процессов.
PMC	Performance monitoring counters — счетчики контроля производительности. Специальные аппаратные регистры процессора, которые можно запрограммировать для обработки низкоуровневых событий CPU: такты, циклы простоя, инструкции, операции загрузки из памяти и сохранения в память и т. д.
POSIX	Portable Operating System Interface for Unix — интерфейс переносимой ОС для Unix. Семейство стандартов, разработанных организацией IEEE и определяющих Unix API.
Python	Язык программирования Python.
RCU	Read-copy-update — чтение-копирование-обновление. Механизм синхронизации в Linux.
RFC	Request for Comments — запрос на комментарии. Общедоступный документ рабочей группы инженеров интернета (Internet Engineering Task Force, IETF). Документы RFC используются для определения сетевых протоколов, например, RFC 793 определяет протокол TCP.
RSS	Resident set size — размер резидентного набора. Мера оперативной памяти.

SCSI	Small Computer System Interface — интерфейс малых вычислительных систем. Стандартный интерфейс для устройств хранения.
SLA	Service level agreement — соглашение о гарантированном уровне обслуживания.
SLO	Service level objective — цель гарантированного уровня обслуживания. Конкретная и измеримая цель.
SNMP	Simple Network Management Protocol — простой протокол управления сетью.
Solaris	Операционная система Unix, первоначально разработанная в Sun Microsystems, в составе которой с 2005 года распространялся механизм динамической трассировки DTrace. Корпорация Oracle приобрела Sun, и теперь Solaris называется Oracle Solaris.
SSH	Secure Shell — безопасная оболочка. Протокол защищенного доступа к удаленной командной оболочке.
SVG	Scalable Vector Graphics — масштабируемая векторная графика. Формат файлов.
sysctl	Инструмент для просмотра и изменения параметров ядра, часто используется и для описания параметра.
TCP	Transmission Control Protocol — протокол управления передачей. Сетевой протокол, определяемый документом RFC 793. См. главу 10.
TLB	Translation Lookaside Buffer — буфер ассоциативной трансляции. Кэш, используемый механизмом MMU (см. <i>MMU</i>), который преобразует виртуальные адреса в физические.
UDP	User Datagram Protocol — протокол дейтаграмм пользователя: сетевой протокол, определяемый документом RFC 768. См. главу 10.
uprobes	Технология динамической инструментации, реализованная в ядре Linux и предназначенная для инструментации ПО пространства пользователя.
uretprobe	Разновидность uprobe для инструментации точек выхода из функций в пространстве пользователя.
USDT	User-level Statically Defined Tracing — статически определяемые точки трассировки на уровне пользователя. Разновидность точек трассировки, помещаемых в код приложения программистом для получения полезной информации.
VFS	Virtual File System — виртуальная файловая система. Абстракция, используемая ядром для поддержки файловых систем различных типов.
ZFS	Комбинированная файловая система и диспетчер томов, созданные в Sun Microsystems.

Ассоциативный массив (associative array)	Коллекция значений, каждое из которых сохраняется и извлекается с использованием уникального ключа.
Байт (byte)	Единица представления данных в цифровом виде. Эта книга следует индустриальному стандарту, согласно которому один байт хранит восемь бит. Один бит может представлять только два значения: ноль и единицу.
Блокировка чтения/записи (reader/writer lock)	Примитив исключительного доступа. Используется в многопоточных приложениях для защиты общих данных.
Буфер (buffer)	Область памяти, используемая для хранения данных, нередко данных ввода/вывода.
Вне CPU (off-CPU)	Относится к потоку, который в настоящий момент не выполняется на CPU и поэтому находится «вне CPU», либо добровольно оставляя процессор, либо ожидая завершения операции ввода/вывода, освобождения блокировки или другого события.
Встраивание (inline)	Оптимизация компилятора, суть которой заключается во встраивании инструкций функции в точку ее вызова.
Выборка (sampling)	Методика исследования цели путем извлечения подмножества (или выборки) измерений. В трассировке под выборкой обычно понимается выборка по времени, когда указатель инструкций или трассировки стека выбираются с заданным интервалом (например, 99 Гц для всех CPU).
Герц (Гц) (Hertz)	Количество циклов в секунду.
Гиперпоточность (hyperthreading)	Технология Intel для масштабирования CPU, которая позволяет ОС создавать несколько виртуальных CPU для одного ядра и планировать задания для них, которые процессор пытается выполнять параллельно.
Демон (daemon)	Системная программа, выполняющаяся постоянно для предоставления услуги.
Динамическая инструментация (dynamic instrumentation)	Также известна как «динамическая трассировка» по названиям инструментов трассировки, использующих динамическую инструментацию. Это технология инструментации произвольного программного события, включая вызовы и завершение функций, посредством динамической подмены инструкций временными инструкциями трассировки. Целевое ПО обычно не должно обладать особыми возможностями для поддержки динамической инструментации. Поскольку этот подход может инструментировать любую программную функцию, он не считается стабильным API.

Динамическая трассировка (dynamic tracing)	Программное обеспечение, реализующее динамическую инструментацию.
Загрузка/вытеснение страниц (pagein/pageout)	Операции, выполняемые операционной системой (ядром) для перемещения фрагментов памяти (страниц) из внешнего устройства хранения в оперативную память и обратно.
Задача (task)	Термин, используемый в Linux для обозначения потока выполнения.
Задержка (latency)	Время до возникновения события, например время до завершения операции ввода/вывода. Оценка величины задержек — важная часть анализа производительности, потому что задержки часто вносят наибольший вклад в проблему производительности. Как измеряется задержка, не всегда очевидно без дополнительных пояснений. Например, «задержка диска» может означать время, проведенное запросом только в очереди драйвера диска, или, с точки зрения приложения, она может означать все время от отправки запроса ввода/вывода до его выполнения, включая время ожидания в очереди и время обслуживания.
Зонд (probe)	Точка инструментации, программная или аппаратная.
Карта (map)	См. <i>карта BPF</i> .
Карта BPF (BPF map)	Объект-хранилище BPF в ядре, используется для хранения показателей, трассировок стека и других данных.
Кбайт (Kbytes)	Килобайт.
Кольцевой буфер (ring buffer)	Основной способ организации буфера, согласно которой при переполнении новые данные начинают затирать старые, благодаря чему гарантируется сохранность только недавних событий.
Команда (command)	Программа, выполняемая командной оболочкой.
Командная оболочка (shell)	Интерпретатор команд и язык сценариев.
Массив (array)	Тип переменной, содержащий несколько значений и обеспечивающий доступ к ним по целочисленному индексу.
Мбайт (Mbytes)	Мегабайт.
мкс (μs)	Микросекунды.
мс (ms)	Миллисекунды.

Мьютекс (mutex)	Взаимоисключающая блокировка: программная блокировка, которая может стать причиной снижения производительности и поэтому часто требует пристального внимания. См. главы 13 и 14.
На CPU (on-CPU)	Относится к потоку, который в настоящий момент выполняется на CPU.
Наблюдаемость (observability)	Приемы и инструменты, используемые для наблюдения и анализа состояния вычислительных систем. Инструменты, представленные в этой книге, являются инструментами наблюдаемости.
Непривилегированный режим	Обычный режим работы процессора, в котором выполняются программы из пространства пользователя. Это более низкий уровень привилегий, чем у ядра. В этом режиме запрещается прямой доступ к ресурсам, что вынуждает ПО из пространства пользователя запрашивать доступ к этим ресурсам через ядро.
Нестабильный (unstable)	Уровень обязательств программного интерфейса, при котором не декларируется никаких обязательств. Нестабильный интерфейс может меняться со временем и отличаться в разных версиях программного обеспечения. Поскольку kprobes и uprobes инструментируют внутренний код ПО, их API считается нестабильным интерфейсом.
Низкоуровневый (нативный) (native)	В информатике обозначает программный код и данные, которые могут обрабатываться процессорами непосредственно, без дополнительной интерпретации или компиляции.
Обратная трассировка стека (stack back trace)	См. <i>трассировка стека</i> .
ОС (OS)	Операционная система: набор ПО, включая ядро, для управления ресурсами и процессами в пространстве пользователя.
Очередь на выполнение (run queue)	Очередь планировщика задач, в которой задачи ожидают, когда им будет предоставлена возможность выполниться на процессоре. Очередь может быть реализована как древовидная структура, но до сих пор используется термин «очередь на выполнение».
Память (memory)	Системная оперативная память, обычно реализованная на микросхемах DRAM.
Переменная (variable)	Именованный объект, используемый в языках программирования для хранения данных.

Подстановка имен файлов (globbing)	Набор шаблонных знаков, обычно используемых для подстановки имен файлов (*, ?, []).
Попадание в ту же точку (lockstep)	Относится к выборке с частотой, совпадающей с частотой другого синхронизированного события, из-за чего это событие может быть чрезмерно представлено в полученной выборке.
Поток (thread)	Программная абстракция, представляющая программу, которую можно запланировать и выполнить.
Привилегированный режим (kernel level)	Уровень привилегий процессора, с которыми выполняется ядро.
Провайдер (provider)	Термин из DTase, используется для обозначения библиотеки связанных зондов и аргументов. В терминологии Linux термин «провайдер» может иметь разные значения в зависимости от инструмента: это может быть система, категория или тип зонда.
Пропуски (drops)	События трассировки, которые пропускаются (не фиксируются), потому что следуют с более высокой частотой, чем может обеспечить выходной буфер.
Пространство пользователя (user space)	Адресное пространство процессов, выполняющихся в непривилегированном режиме.
Пространство ядра (kernel space)	Адресное пространство ядра.
Профилирование (profiling)	Методика сбора данных, характеризующих эффективность цели. В практике широко распространен метод профилирования, основанный на выборке данных по времени (см. <i>выборка</i>).
Процесс (process)	Абстракция выполняемой программы пространства пользователя в ОС. Каждый процесс имеет уникальный идентификатор PID (см. <i>PID</i>) и может состоять из одного или нескольких потоков (см. <i>поток</i>).
Пуск (fire)	В контексте трассировки «пуск» относится к моменту, когда точка инструментации запускает программу трассировки.
Рабочая нагрузка (workload)	Запрос на потребление некоторого ресурса.
Сбой (fault)	Возможный сбой оборудования и ПО. Обычно сбои являются ожидаемыми и для их надлежащего устранения используется обработчик сбоев.

Сбой страницы (pagefault)	Системное событие, возникающее, когда программа ссылается на область памяти, страница поддержки которой в данный момент не отображена в виртуальную память. Сбой страниц — это следствие модели распределения памяти по требованию, используемой в Linux.
Сервер (server)	Физический компьютер, обычно смонтированный в стойке в центре обработки данных. На сервере обычно работает ядро, операционная система и приложения.
Системный вызов (system call)	Интерфейс для процессов, запрашивающих привилегированные операции у ядра.
Сокет (socket)	Программная абстракция конечной точки сетевого соединения.
Спин (spin)	Программный механизм, включающий выполнение короткого цикла ожидания при попытке получить ресурс. Обычно это спин-блокировка или адаптивная взаимноисключающая блокировка (мьютекс).
Стабильный (stable)	Уровень обязательств программного интерфейса, при котором ожидается неизменность интерфейса.
Статическая instrumentation/трассировка (static instrumentation/tracing)	Предполагает явное включение в код точек instrumentation. ПО поддерживает статическую instrumentation, только если программист предусмотрел это. Статическая instrumentation часто дает дополнительное преимущество в виде стабильного интерфейса.
Стек (stack)	Краткое название трассировки стека.
Страница (page)	Часть памяти, управляемая ядром и процессором. Вся память, используемая системой, разбита на страницы. Типичные размеры страниц — 4 Кбайт и 2 Мбайт (в зависимости от процессора).
Структура (struct)	Структурированный объект, обычно в языке C.
Сценарий (скрипт) (script)	В информатике — выполняемая программа, обычно короткая и написанная на языке высокого уровня. bpftrace можно рассматривать как язык сценариев.
Трассировка (tracing)	Запись данных по событию. События могут быть статическими или динамическими, а также могут быть основаны на таймере. Инструменты, представленные в этой книге, — это инструменты трассировки, они регистрируют события и запускают BPF-программы для записи данных.

Трассировка стека (stack trace)	Стек вызовов, состоящий из нескольких фреймов стека, демонстрирующий порядок вызовов функций. Читая трассировку стека снизу вверх, можно определить, какие функции вызвали другие функции и, соответственно, путь выполнения кода. Такой порядок чтения также называют обратной трассировкой стека, поскольку при чтении стека сверху вниз первой следует самая последняя вызванная функции, а далее функции следуют в порядке, обратном порядку их вызова.
Трассировщик (tracer)	Инструмент трассировки (см. <i>трассировка</i>).
Точки трассировки (tracepoints)	Технология поддержки статической инструментации, реализованная в ядре Linux.
Уровень пользователя (user land)	Режим привилегий процессора, используемый при выполнении на уровне пользователя. Это более низкий уровень привилегий, чем у ядра. Он запрещает прямой доступ к ресурсам, вынуждая ПО пользовательского уровня запрашивать доступ к ним через ядро.
Уровень ядра (kernel level)	Уровень привилегий процессора, устанавливаемый при выполнении кода ядра.
Флейм-график (flame graph)	Прием визуализации трассировок стека. См. главу 2.
Фрейм стека (stack frame)	Структура данных, содержащая информацию о состоянии функции, включая указатель на функцию, адрес возврата и аргументы.
Экземпляр (instance)	Виртуальный сервер. Облачные среды предоставляют экземпляры серверов.
Ядро (процессора) (core)	Конвейер выполнения в процессоре. Ядра могут быть доступны в ОС как единый процессор или, в случае поддержки гиперпоток, как несколько процессоров.
Ядро (операционной системы) (kernel)	Основная программа в системе, которая выполняется в привилегированном режиме и управляет ресурсами и процессами в пространстве пользователя.

БИБЛИОГРАФИЯ

- [Aho 78] Aho, A. V., Kernighan, B. W., and Weinberger, P. J., «Awk: A Pattern Scanning and Processing Language (Second Edition)», *Unix 7th Edition man pages*, 1978, <http://plan9.bell-labs.com/7thEdMan/index.html>
- [Alizadeh 10] Alizadeh, M., Greenberg, A., Maltz, D., Padhye, J., Patel, P., Prabhakar, B., Sengupta, S., and Sridharan, M., «DCTCP: Efficient Packet Transport for the Commoditized Data Center», MSR-TR-2010-68, January 2010, <https://www.microsoft.com/en-us/research/publication/dctcp-efficient-packet-transport-for-the-commoditized-data-center/>
- [AMD 10] AMD, *BIOS and Kernel Developer's Guide (BKDG) for AMD Family 10h Processors*, April 2010, <https://developer.amd.com/wordpress/media/2012/10/31116.pdf>
- [Amit 18] Amit, N., and Wei, M., «The Design and Implementation of Hyperupcalls», *USENIX Annual Technical Conference*, 2018.
- [Bezemer 15] Bezemer, D.-P., Pouwelse, J., and Gregg, B., «Understanding Software Performance Regressions Using Differential Flame Graphs», *IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2015.
- [Bonwick 94] Bonwick, J., «The Slab Allocator: An Object-Caching Kernel Memory Allocator», *USENIX Summer Conference*, 1994.
- [Bostock 10] Heer, J., Bostock, M., and Ogievetsky, V., «A Tour Through the Visualization Zoo», *acmqueue*, Volume 8, Issue 5, May 2010, <http://queue.acm.org/detail.cfm?id=1805128>
- [Cheng 16] Cheng, Y., and Cardwell, N., «Making Linux TCP Fast», *netdev 1.2*, Tokyo, 2016, <https://netdevconf.org/1.2/papers/bbr-netdev-1.2.new.new.pdf>
- [Cockcroft 98] Cockcroft, A., and Pettit, R., *Sun Performance and Tuning: Java and the Internet*. Prentice Hall, 1998.
- [Corbet 05] Corbet, J., Rubini, A., and Kroah-Hartman, G., *Linux Device Drivers*, 3rd edition, O'Reilly, 2005.
- [Desnoyers 09a] Desnoyers, M., *Low-Impact Operating System Tracing*, University of Montreal, December 2009, <https://ltnng.org/files/thesis/desnoyers-dissertation-2009-12-v27.pdf>
- [Desnoyers 09b] Desnoyers, M., and Dagenais, M., *Adaptive Fault Probing*, École Polytechnique de Montréal, December 2009, <http://dmct.dorsal.polymtl.ca/sites/dmct.dorsal.polymtl.ca/files/SOTA2009-Desnoyers.pdf>
- [Elling 00] Elling, R., *Static Performance Tuning*, Sun Blueprints, 2000.
- [Goldberg 73] Goldberg, R. P., *Architectural Principles for Virtual Computer Systems*, Harvard University, 1973.
- [Gorman 04] Gorman, M., *Understanding the Linux Virtual Memory Manager*. Prentice Hall, 2004.

- [Graham 82] Graham, S., Kessler, P., and McKusick, M., «gprof: A Call Graph Execution Profiler», *Proceedings of the SIGPLAN '82 Symposium on Compiler Construction, SIGPLAN Notices*, Volume 6, Issue 17, pp. 120–126, June 1982.
- [Gregg 10] Gregg, B. «Visualizing System Latency», *Communications of the ACM*, July 2010.
- [Gregg 11] Gregg, B., and Mauro, J., *DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD*, Prentice Hall, 2011.
- [Gregg 13a] Gregg, B., «Blazing Performance with Flame Graphs», *USENIX LISA '13 Conference*, November 2013, <https://www.usenix.org/conference/lisa13/technicalsessions/plenary/gregg>
- [Gregg 13b] Gregg, B., *Systems Performance: Enterprise and the Cloud*, Prentice Hall, 2013.
- [Gregg 13c] Gregg, B., «Thinking Methodically About Performance», *Communications of the ACM*, Volume 56 Issue 2, February 2013.
- [Gregg 16] Gregg, B., «The Flame Graph», *Communications of the ACM*, Volume 59, Issue 6, pp. 48–57, June 2016.
- [Gregg 17] Gregg, B., «Linux Container Performance Analysis», *USENIX LISA '17 Conference*, November 2017, <https://www.usenix.org/conference/lisa17/conference-program/presentation/gregg>
- [Hiramatsu 14] Hiramatsu, M., «Scalability Efforts for Kprobes or: How I Learned to Stop Worrying and Love a Massive Number of Kprobes», *LinuxCon Japan*, 2014, https://events.static.linuxfound.org/sites/events/files/slides/Handling%20the%20Massive%20Multiple%20Kprobes%20v2_1.pdf
- [Hollingsworth 94] Hollingsworth, J., Miller, B., and Cargille, J., «Dynamic Program Instrumentation for Scalable Performance Tools», *Scalable High-Performance Computing Conference (SHPCC)*, May 1994.
- [Høiland-Jørgensen 18] Høiland-Jørgensen, T., et al., «The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel», *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies*, 2018.
- [Hubicka 13] Hubicka, J., Jaeger, A., Matz, M., and Mitchell, M., *System V Application Binary Interface AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models)*, July 2013, <https://software.intel.com/sites/default/files/article/402129/mpx-linux64-abi.pdf>
- [Intel 16] Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3B: System Programming Guide, Part 2*, September 2016, <https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-3b-part-2-manual.html>
- [Jacobson 18] Jacobson, V., «Evolving from AFAP: Teaching NICs About Time», *netdev 0x12*, July 2018, <https://www.files.netdevconf.org/d/4ee0a09788fe49709855/files/?p=/Evolving%20from%20AFAP%20%E2%80%93%20Teaching%20NICs%20about%20time.pdf>
- [McCanne 92] McCanne, S., and Jacobson, V., «The BSD Packet Filter: A New Architecture for User-Level Packet Capture», *USENIX Winter Conference*, 1993.
- [Stoll 89] Stoll, C., *The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage*, The Bodley Head Ltd., 1989.

- [Sun 05] Sun, *Sun Microsystems Dynamic Tracing Guide* (Part No: 817–6223–11), January 2005.
- [Tamches 99] Tamches, A., and Miller, B., «Fine-Grained Dynamic Instrumentation of Commodity Operating System Kernels», *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation*, February 1999.
- [Tikhonovsky 13] Tikhonovsky, I., «Web Inspector: Implement Flame Chart for CPU Profiler», *Webkit Bugzilla*, 2013, https://bugs.webkit.org/show_bug.cgi?id=111162
- [Vance 04] Vance, A., «Sun Delivers Unix Shocker with DTrace: It Slices, It Dices, It Spins, It Whirls», *The Register*, July 2004, https://www.theregister.co.uk/2004/07/08/dtrace_user_take_anc
- [VMware 07] VMware, *Understanding Full Virtualization, Paravirtualization, and Hardware Assist*, 2007, https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/techpaper/VMware_paravirtualization.pdf
- [Welsh 01] Welsh, M., Culler, D., and Brewer, E., «seda: An Architecture for Well-Conditioned, Scalable Internet Services», *ACM SIGOPS*, Volume 35, Issue 5, December 2001.
- [Yamamoto 16] Yamamoto, M., and Nakashima, K., «Execution Time Compensation for Cloud Applications by Subtracting Steal Time Based on Host-Level Sampling», *ICPE*, 2016, https://research.spec.org/icpe_proceedings/2016/companion/p69.pdf
- [Zannoni 16] Zannoni, E., «BPF and Linux Tracing Infrastructure», *LinuxCon Europe*, 2016, <https://events.static.linuxfound.org/sites/events/files/slides/tracing-linux-ezannoni-berlin-2016-final.pdf>
- [1] https://events.static.linuxfound.org/sites/events/files/slides/bpf_collabsummit_2015feb20.pdf
- [2] <https://lkml.org/lkml/2013/9/30/627>
- [3] <https://lore.kernel.org/netdev/1395404418-25376-1-git-send-email-dborkman@redhat.com/T/>
- [4] <https://lore.kernel.org/lkml/1435328155-87115-1-git-send-email-wangnan0@huawei.com/T/>
- [5] <https://github.com/iovisor/ply>
- [6] <http://halobates.de/on-submitting-patches.pdf>
- [7] <https://www.usenix.org/legacy/publications/library/proceedings/sd93/>
- [8] <https://www.slideshare.net/vh21/meet-cutebetweenebpfandtracing>
- [9] <https://lwn.net/Articles/437981/>
- [10] <https://lwn.net/Articles/475043/>
- [11] <https://lwn.net/Articles/575444/>
- [12] <https://patchwork.ozlabs.org/patch/334837/>
- [13] https://kernelnewbies.org/Linux_3.18
- [14] <http://vger.kernel.org/vger-lists.html#netdev>
- [15] <http://www.brendangregg.com/blog/2015-05-15/ebpf-one-small-step.html>
- [16] <http://www.brendangregg.com/blog/2014-07-10/perf-hacktogram.html>
- [17] <https://www.kernel.org/doc/Documentation/networking/filter.txt>

- [18] <https://lkml.org/lorenz/2018/02/28/1477>
- [19] <https://cilium.readthedocs.io/en/latest/bpf/>
- [20] <https://graphviz.org/>
- [21] <https://lore.kernel.org/lkml/CAHk-=wib9VSbwbS+N82ZPNtvt4croYyHyQduhFimX8nyjCyZyA@mail.gmail.com/>
- [22] <http://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>
- [23] <https://patchwork.ozlabs.org/patch/1030266/>
- [24] <https://github.com/cilium/cilium>
- [25] https://source.android.com/devices/architecture/kernel/bpf#files_available_in_sysfs
- [26] <https://www.kernel.org/doc/Documentation/bpf/btf.rst>
- [27] <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=c04c0d2b968ac45d6ef020316808ef6c82325a82>
- [28] https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/tree/Documentation/bpf/bpf_design_QA.rst#n90
- [29] https://www.kernel.org/doc/Documentation/bpf/bpf_design_QA.txt
- [30] <http://www.man7.org/linux/man-pages/man2/bpf.2.html>
- [31] <http://man7.org/linux/man-pages/man7/bpf-helpers.7.html>
- [32] <https://lwn.net/Articles/599755/>
- [33] <https://www.iovisor.org/blog/2015/10/15/bpf-internals-ii>
- [34] <https://gcc.gnu.org/ml/gcc-patches/2004-08/msg01033.html>
- [35] <https://blogs.oracle.com/eschrock/debugging-on-amd64-part-one>
- [36] <https://github.com/sysstat/sysstat/pull/105>
- [37] <http://www.brendangregg.com/flamegraphs.html>
- [38] <https://github.com/spiermar/d3-flame-graph>
- [39] <https://medium.com/netflix-techblog/netflix-flamescope-a57ca19d47bb>
- [40] <https://lwn.net/Articles/132196/>
- [41] <http://phrack.org/issues/67/6.html#article>
- [42] <https://www.kernel.org/doc/Documentation/kprobes.txt>
- [43] <https://www.ibm.com/developerworks/library/l-kprobes/index.html>
- [44] <https://lwn.net/Articles/499190/>
- [45] https://events.static.linuxfound.org/images/stories/pdf/eeus2012_desnoyers.pdf
- [46] <https://www.kernel.org/doc/Documentation/trace/uprobetracer.txt>
- [47] <https://www.kernel.org/doc/Documentation/trace/tracepoints.rst>
- [48] <https://lkml.org/lkml/2018/2/28/1477>

- [49] <http://www.brendangregg.com/blog/2015-07-03/hacking-linux-usdt-ftrace.html>
- [50] <http://blogs.microsoft.co.il/sasha/2016/03/30/usdt-probe-support-in-bpfbcc/>
- [51] <http://blog.srvthe.net/usdt-report-doc/>
- [52] <https://github.com/sthima/libstapsdt>
- [53] <https://medium.com/sthima-insights/we-just-got-a-new-super-power-runtime-usdt-comes-to-linux-814dc47e909f>
- [54] <https://github.com/dalehamel/ruby-static-tracing>
- [55] <https://xenbits.xen.org/docs/4.11-testing/misc/xen-command-line.html>
- [56] <https://medium.com/netflix-techblog/linux-performance-analysis-in-60-000-milliseconds-acc10403c55>
- [57] <http://www.brendangregg.com/blog/2016-10-27/dtrace-for-linux-2016.html>
- [58] <https://github.com/iovisor/bcc/blob/master/INSTALL.md>
- [59] <https://github.com/iovisor/bcc/blob/master/CONTRIBUTING-SCRIPTS.md>
- [60] <https://github.com/iovisor/bcc#tools>
- [61] <https://snapcraft.io/bpftrace>
- [62] <https://packages.debian.org/sid/bpftrace>
- [63] <https://github.com/iovisor/bpftrace/blob/master/INSTALL.md>
- [64] <https://github.com/iovisor/bcc/blob/master/docs/kernel-versions.md>
- [65] https://github.com/iovisor/bpftrace/blob/master/docs/tutorial_one_liners.md
- [66] https://github.com/iovisor/bpftrace/blob/master/docs/reference_guide.md
- [67] <https://github.com/iovisor/bpftrace/issues/26>
- [68] <https://github.com/iovisor/bpftrace/issues/305>
- [69] <https://github.com/iovisor/bpftrace/issues/614>
- [70] <https://lore.kernel.org/lkml/CAHk-=wib9VSbwS+N82ZPNvt4vrYyHyQduhFimX8nyjCyZyA@mail.gmail.com/>
- [71] <https://github.com/iovisor/bpftrace/pull/790>
- [72] <http://www.brendangregg.com/blog/2017-08-08/linux-load-averages.html>
- [73] <http://www.brendangregg.com/perf.html>
- [74] <https://github.com/brendangregg/pmc-cloud-tools>
- [75] <http://www.brendangregg.com/blog/2018-02-09/kpti-kaiser-meltdownperformance.html>
- [76] <https://github.com/Netflix/flamescope>
- [77] <https://medium.com/netflix-techblog/netflix-flamescope-a57ca19d47bb>
- [78] <https://github.com/brendangregg/perf-tools>
- [79] <https://github.com/pmem/vltrace>

- [80] <http://agentzh.org/misc/slides/off-cpu-flame-graphs.pdf>
- [81] <https://www.kernel.org/doc/Documentation/admin-guide/mm/concepts.rst>
- [82] <http://www.brendangregg.com/FlameGraphs/memoryflamegraphs.html#brk>
- [83] https://docs.oracle.com/cd/E23824_01/html/821-1448/zfspools-4.html#gentextid-11970
- [84] <http://vger.kernel.org/~acme/perf/linuxdev-br-2018-perf-trace-eBPF>
- [85] <https://www.spinics.net/lists/linux-fsdevel/msg139937.html>
- [86] <https://lwn.net/Articles/787473/>
- [87] <http://www.brendangregg.com/blog/2014-12-31/linux-page-cache-hit-ratio.html>
- [88] <https://www.kernel.org/doc/ols/2002/ols2002-pages-289-300.pdf>
- [89] <https://github.com/torvalds/linux/blob/16d72dd4891fecc1e1bf7ca193bb7d5b9804c038/kernel/bpf/verifier.c#L7851-L7855>
- [90] <https://lwn.net/Articles/552904/>
- [91] <https://oss.oracle.com/~mason/seekwatcher/>
- [92] <https://github.com/facebook/folly/tree/master/folly/tracing>
- [93] <https://cilium.io/>
- [94] <https://code.fb.com/open-source/open-sourcing-katran-a-scalable-network-load-balancer/>
- [95] <https://www.coverfire.com/articles/queueing-in-the-linux-network-stack/>
- [96] <https://www.kernel.org/doc/Documentation/networking/scaling.rst>
- [97] <https://patchwork.ozlabs.org/cover/910614/>
- [98] <https://lwn.net/Articles/659199/>
- [99] <https://patchwork.ozlabs.org/patch/610370/>
- [100] <https://www.kernel.org/doc/Documentation/networking/segmentationoffloads.rst>
- [101] <https://www.bufferbloat.net/>
- [102] <https://www.kernel.org/doc/Documentation/networking>
- [103] <https://flent.org/>
- [104] <https://www.wireshark.org/>
- [105] <https://blog.cloudflare.com/the-story-of-one-latency-spike/>
- [106] http://www.brendangregg.com/DTrace/DTrace_Network_Providers.html
- [107] http://www.brendangregg.com/DTrace/CEC2006_demo.html
- [108] <https://twitter.com/b0rk/status/765666624968003584>
- [109] <http://www.brendangregg.com/blog/2016-10-15/linux-bcc-tcptop.html>
- [110] <https://github.com/brendangregg/dtrace-cloud-tools/tree/master/net>
- [111] <http://www.brendangregg.com/blog/2014-09-06/linux-ftrace-tcp-retransmittracing.html>

- [112] https://www.reddit.com/r/networking/comments/47jc98/dns_resolution_time_for_io_in_us/
- [113] <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=e3118e8359bb7c59555aca60c725106e6d78c5ce>
- [114] <https://www.slideshare.net/AlexMaestretti/security-monitoring-with-ebpf>
- [115] <https://seclists.org/oss-sec/2019/q2/131>
- [116] <https://www.snort.org/>
- [117] <https://www.slideshare.net/AlexMaestretti/security-monitoring-with-ebpf/17>
- [118] https://www.kernel.org/doc/Documentation/userspace-api/seccomp_filter.rst
- [119] <https://lwn.net/Articles/756233/>
- [120] <https://lore.kernel.org/netdev/cover.1425208501.git.daniel@iogearbox.net/T/>
- [121] <https://lore.kernel.org/netdev/61198814638d88ce3555dbecf8ef875523b95743.1452197856.git.daniel@iogearbox.net>
- [122] <https://lwn.net/Articles/747551/>
- [123] <https://lore.kernel.org/netdev/20180216134023.15536-1-daniel@iogearbox.net/>
- [124] <https://landlock.io/>
- [125] <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-ext.git/commit/?id=8b401f9ed2441ad9e219953927a842d24ed051fc>
- [126] https://bugzilla.redhat.com/show_bug.cgi?id=1384344
- [127] <https://www.kernel.org/doc/Documentation/sysctl/net.txt>
- [128] <http://vger.kernel.org/bpfconf.html>
- [129] <https://lore.kernel.org/netdev/2f24a9bbf761accb982715c761c0840a14c0b5cd.1463158442.git.daniel@iogearbox.net/>
- [130] <https://lore.kernel.org/netdev/36bb0882151c63dcf7c624f52bf92db8adbf80a.1487279499.git.daniel@iogearbox.net/>
- [131] <https://cilium.readthedocs.io/en/stable/bpf/#hardening>
- [132] <http://phrack.org/issues/63/3.html>
- [133] <https://lore.kernel.org/netdev/8f751452-271f-6253-2f34-9e4cecb347b8@iogearbox.net/T/>
- [134] <http://www.brendangregg.com/blog/2016-01-18/ebpf-stack-trace-hack.html>
- [135] <https://github.com/jvm-profiling-tools/perf-map-agent>
- [136] <https://github.com/torvalds/linux/blob/master/tools/perf/Documentation/jit-interface.txt>
- [137] <https://github.com/brendangregg/FlameGraph/blob/master/jmaps>
- [138] <https://docs.oracle.com/en/java/javase/11/vm/dtrace-probes-hotspot-vm.html>
- [139] <http://www.brendangregg.com/blog/2007-08-10/dtrace-bourne-shell-shprovider1.html>
- [140] <http://www.brendangregg.com/bpfperfertools.html>
- [141] <https://github.com/sthima/node-usdt>

- [142] <http://www.brendangregg.com/blog/2016-10-12/linux-bcc-nodejs-usdt.html>
- [143] <https://github.com/mmarchini/node-linux-perf>
- [144] <http://www.brendangregg.com/blog/2014-09-17/node-flame-graphs-on-linux.html>
- [145] <https://golang.org/pkg/runtime/trace/>
- [146] <http://www.brendangregg.com/blog/2017-01-31/golang-bcc-bpf-functiontracing.html>
- [147] <https://github.com/iovisor/bcc/issues/934>
- [148] <https://tour.golang.org/basics/4>
- [149] <https://github.com/iovisor/bpfttrace/issues/740>
- [150] <https://github.com/iovisor/bcc/issues/1320#issuecomment-407927542>
- [151] <https://github.com/mmcshane/salp>
- [152] <https://wiki.tcl-lang.org/page/DTrace>
- [153] <https://www.gnu.org/software/libc/>
- [154] <https://bugs.debian.org/cgi-bin/bugreport.cgi?bug=767756>
- [155] <https://dev.mysql.com/doc/refman/5.7/en/dba-dtrace-ref-command.html>
- [156] <https://www.kernel.org/doc/html/latest/core-api/memory-allocation.html>
- [157] <https://www.kernel.org/doc/Documentation/locking/mutex-design.txt>
- [158] <https://www.kernel.org/doc/Documentation/trace/ftrace.rst>
- [159] <https://github.com/rostedt/trace-cmd>
- [160] <https://git.kernel.org/pub/scm/linux/kernel/git/rostedt/trace-cmd.git>
- [161] <http://kernelshark.org/>
- [162] <https://www.kernel.org/doc/Documentation/kernel-hacking/locking.rst>
- [163] <https://github.com/iovisor/bpfttrace/pull/534>
- [164] <https://lore.kernel.org/patchwork/patch/157488/>
- [165] <https://clearlinux.org/news-blogs/kata-containers-next-evolution-clearcontainers>
- [166] <https://github.com/firecracker-microvm/firecracker/blob/master/docs/design.md>
- [167] <https://lwn.net/Articles/531114/>
- [168] <https://lwn.net/Articles/750313/>
- [169] <https://github.com/iovisor/kubectrl-trace>
- [170] <https://github.com/kubernetes-incubator/metrics-server>
- [171] <https://kubernetes.io/docs/tasks/debug-application-cluster/resource-usagemonitoring/>
- [172] <https://www.kernel.org/doc/Documentation/cgroup-v1/cpuacct.txt>
- [173] <http://blog.codemonkey.ws/2007/10/myth-of-type-i-and-type-ii-hypervisors.html>
- [174] <http://www.brendangregg.com/blog/2017-11-29/aws-ec2-virtualization-2017.html>

- [175] <http://www.pcp.io/>
- [176] <http://getvector.io/>
- [177] <https://github.com/Netflix/vector>
- [178] <https://www.timeanddate.com/holidays/fun/yellow-pig-day>
- [179] <https://github.com/iovisor/bpftrace/issues/646>
- [180] https://github.com/iovisor/bcc/blob/master/docs/tutorial_bcc_python_developer.md
- [181] https://github.com/iovisor/bcc/blob/master/docs/tutorial_bcc_python_developer.md
- [182] <https://github.com/cilium/cilium/tree/master/bpf>
- [183] <https://github.com/iovisor/bcc>
- [184] <https://github.com/torvalds/linux/blob/master/include/uapi/linux/bpf.h>
- [185] <http://www.brendangregg.com/psio.html>
- [186] https://static.sched.com/hosted_files/lssna19/8b/Kernel%20Runtime%20Security%20Instrumentation.pdf
- [187] <https://github.com/torvalds/linux/commits/master/drivers/nvme/host/trace.h>

Брендан Грегг

BRF: профессиональная оценка производительности

Перевел с английского А. Киселев

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>К. Тульцева</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>С. Беляева, Н. Викторова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:
194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 12.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 31.10.23. Формат 70×100/16. Бумага офсетная. Усл. п. л. 70,950. Тираж 700. Заказ 0000.

Брендан Грегг

ПРОИЗВОДИТЕЛЬНОСТЬ СИСТЕМ



Книга посвящена концепциям, стратегиям, инструментам и настройке операционных систем и приложений на примере систем на базе Linux. Понимание этих инструментов и методов критически важно при разработке современного ПО. Применение стратегий, изложенных в обновленном и переработанном издании, позволит перформанс-инженерам улучшить взаимодействие с конечными пользователями и снизить затраты, особенно для облачных сред.

Брендан Грегг — эксперт в области производительности систем и автор нескольких бестселлеров — лаконично, но емко излагает наиболее важные сведения о работе операционных систем, оборудования и приложений, которые позволяют специалистам быстро добиться результатов, даже если раньше они никогда не занимались анализом производительности. Далее автор дает детальные объяснения по применению современных инструментов и методов, включая расширенный BPF, и показывает, как добиться максимальной эффективности ваших систем в облачных, веб- и крупных корпоративных средах.

КУПИТЬ

Майкл Керриск

LINUX API. ИСЧЕРПЫВАЮЩЕЕ РУКОВОДСТВО



Linux Programming Interface — исчерпывающее руководство по программированию приложений для Linux и UNIX. Описанный здесь интерфейс применяется практически с любыми приложениями, работающими в операционных системах Linux или UNIX.

В этой авторитетной книге эксперт по Linux Майкл Керриск, долгое время отвечавший за наполнение справочного ресурса man pages, подробно описывает библиотечные вызовы и библиотечные функции, которые понадобятся вам при системном программировании в Linux. Книга содержит множество продуманных полнофункциональных программ, доступно иллюстрирующих все теоретические концепции.

КУПИТЬ

Кристофер Негус

БИБЛИЯ LINUX

10-е издание



Полностью обновленное 10-е издание «Библии Linux» поможет как начинающим, так и опытным пользователям приобрести знания и навыки, которые выведут на новый уровень владения Linux. Известный эксперт и автор бестселлеров Кристофер Негус делает акцент на инструментах командной строки и новейших версиях Red Hat Enterprise Linux, Fedora и Ubuntu. Шаг за шагом на подробных примерах и упражнениях вы досконально поймете операционную систему Linux и пустите знания в дело. Кроме того, в 10-м издании содержатся материалы для подготовки к экзаменам на различные сертификаты по Linux.

КУПИТЬ